

Genrmination_Fight

Design, engineering, and complete source

From the supplied design template, development conversations, latest export, and live Studio source.

21 active sources 8 archived sources 13 photographs
1 disabled generator 12 historical workspace artifacts

Snapshot edition: 14 September 2026

Place 137098589879404 / version 77 baseline + Revision 5 repairs

Includes open defects, historical test limits, future implementation recipes, and full line-numbered code. Live repairs and embedded engineering comments included. Revision 5 reverses the Revision 4 geometry migration, rebuilds carrier navigation on the live map, and replaces the passive opposition with a hostile ambush. Not published.

Contents

I	Game design and engineering record	1
1	The game as it plays: environment, cast, equipment and tactics	2
1.1	Photographic survey	2
1.2	The environment	14
1.3	Characters	15
1.4	Equipment and fixtures	16
1.5	Weapons	16
1.6	Gameplay	16
1.7	Tactics worth knowing	18
2	Revision 5: coordinate repair, navigation, and the hostile ambush	19
2.1	Summary	19
2.2	The coordinate fault and its repair	19
2.3	Basement stair-tower void	20
2.4	Carrier navigation and physical movement	21
2.5	Role-bound assignments	22
2.6	Hostile ambush system	22
2.7	Damage model corrections	23
2.8	Constants reference	24
2.9	Open items	24
3	Revision 4: live repairs and current handover	25
3.1	Authority and chronology	25
3.2	What was actually changed	25
3.3	Fresh verification results	26
3.4	Remaining boundaries and next engineering work	27
3.5	How another AI should modify this revision	27
3.6	Embedded documentation contract	28
4	How to read the retained baseline	29
5	Baseline record: Reading this edition	30
5.1	Purpose and evidence boundaries	30
5.2	Status vocabulary	31
5.3	Immediate handover findings	31

6 Baseline record: Experience Overview	32
6.1 Experience concept	32
6.2 Design pillars	32
6.3 Games with a scientific purpose: present limit and future direction	33
7 Baseline record: Experience Information	34
7.1 Developers, responsibilities, and deliverables	34
7.2 Audience, platforms, and server configuration	34
7.3 Versioning contract	34
8 Baseline record: Gameplay Loop	36
8.1 Gameplay Objective(s)	36
8.2 Gameplay Story	36
8.3 Gameplay Flow	36
9 Baseline record: Core Mechanics	38
9.1 User Controls: Computer/PC	38
9.2 User Controls: Mobile/Tablet	38
9.3 User Controls: XBOX Controller	38
9.4 Progression	39
9.5 Round calculation	39
10Baseline record: Core Features	40
10.1Equipment and plant rescue	40
10.2Addition quiz	40
10.3Research carriers and communication	41
10.4Explosions, destruction, and repair	41
10.5Equivalent equipment routing	41
10.6Features absent from the current design	42
11Baseline record: Design Style	43
11.1Setting, time, and visual language	43
11.2Game Location(s)	43
11.3Character Design(s)	43
11.4Concept Art & Notes and Reference Image(s)	44
11.5Bibliography	44
12Baseline record: Monetization Strategy	45
12.1Current position	45
13Baseline record: Risk Analysis	46
13.1Current technical risks	46
13.2Performance and operational risks	46
13.3Scope and schedule risks	46
14Baseline record: The MVPs	48

14.1 Features	48
14.2 Requirements and acceptance gate	48
15 Baseline record: Target Dates	49
15.1 Milestone plan without invented dates	49
16 Baseline record: Credits & Additional Resources	50
16.1 Template attribution	50
17 Baseline record: Technical Annex: How the Game Was Built	51
17.1 Construction layers and why they matter	51
17.2 MCP workflow used in the documented development	51
17.3 Rebuild and rollback protocol	52
17.4 State ownership and initialization	52
17.5 Equipment state and sample transactions	52
17.6 Carrier mission state machine	53
17.7 Motion, route planning, and the coordinate defect	53
17.8 Weapon casting and server validation	53
17.9 Aiming and trembling: causes and boundaries of the fix	54
17.10 HUD and information grouping	54
17.11 Destruction and rerouting lifecycle	55
17.12 Solo opposition and fairness	55
17.13 Scientific data extension architecture	55
18 Architecture and coordinate diagrams	56
18.1 Runtime responsibilities	56
18.2 Observed origin mismatch: Incubator example	57
18.3 Proposed mission transition contract	57
19 Ten research scenarios: protocols and extension specifications	58
19.101. Germination viability	58
19.202. Pathogen resistance	59
19.303. Light spectrum optimization	59
19.404. Soil bioremediation	60
19.505. Hydroponic versus soil	60
19.606. UV adaptation	61
19.707. Gravitropism assay	61
19.808. Drought resistance	62
19.909. Microbiome symbiosis	62
19.100. Cold-hardiness screening	63
20 Defect register, change accounting, and lessons	64
20.1 How many bugs were generated?	64
20.2 Historical defect families	64
20.3 Baseline discrepancies, risks, and tool artifacts	68

20.4Rules that prevent repetition	70
21Verification ledger and future acceptance suite	71
21.1P01: Role selection and nearby companion	71
21.2P02: Wait and Regroup	71
21.3P03: Original rifle and equipment destruction	71
21.4P04: Equivalent rerouting	71
21.5P05: Carrier gait and speed	71
21.6P06: Wall-fixture cleanup	72
21.7P07: Small mobile layout	72
21.8P08: Idle weapon stabilization	72
21.9P09: Walking stabilization	72
21.10P10: Interception score	72
21.11P11: Solo score loss	72
21.12P12: Rooftop mission completion	73
21.13P13: Player shooting validation	73
21.14P14: ADS and device controls	73
21.15P15: Basement and quiz/window interactions	73
21.16D01: Live source completeness	73
21.17D02: Scene/source audit	73
21.18Proposed new regression sequence	74
22Subsystem-by-subsystem future update playbook	76
22.1LabKinds: Equipment catalog	76
22.2LabEquipment: Equipment engine	76
22.3LabRoles: Roles and outfits	77
22.4LabRound: Round lifecycle	77
22.5LabTechnicians: Legacy worker service	77
22.6LabElevators: Elevator compatibility script	78
22.7LoungeSystems: Lounge server behavior	78
22.8LoungeClient: Lounge presentation	79
22.9LabQuizClient: Quiz UI	79
22.10LabSidearmClient: Tool bootstrap	79
22.11Blaster: Shared casting	80
22.12BlasterController: Input/camera/pose	80
22.13ServerBlasterManager: Authoritative shooting	81
22.14GWAPWorld: Research geometry	81
22.15GWAPScenarios: Scenario data	81
22.16GWAPNPC: Carrier appearance and gait	82
22.17GWAPRouting: Equivalent selection	82
22.18GWAPServer: Mission orchestration	83
22.19GWAPWallDamage: Wall fixture lifecycle	83
22.20LabHUD: Grouped mission display	83

22.2	Cross-cutting data, assets, and deployment	84
22.2	Complete proposed coordinate-frame module	84
23	Scene inventory and coordinate evidence	86
23.1	Laboratory equipment kinds	86
23.2	Research station positions and stored approaches	86
23.3	Important tag counts	88
23.4	Latest exported files	88
23.5	Coordinate repair decision	89
24	Source provenance and complete-code map	90
24.1	Hash verification	91
24.2	Normalized workspace comparisons	96
25	Active-code symbol and attribute index	98
25.1	S01 GWAPNPC	98
25.2	S02 GWAPRouting	98
25.3	S03 GWAPScenarios	99
25.4	S04 GWAPWorld	99
25.5	S05 LabKinds	99
25.6	S06 Blaster	101
25.7	S07 BlasterController	101
25.8	S08 GWAPServer	103
25.9	S09 GWAPWallDamage	105
25.1	S10 LabElevators	105
25.1	S11 LabEquipment	105
25.1	S12 LabRoles	112
25.1	S13 LabRound	113
25.1	S14 LabTechnicians	114
25.1	S15 LoungeSystems	114
25.1	S16 ServerBlasterManager	115
25.1	S25 LabHUD	115
25.1	S26 LabQuizClient	116
25.1	S27 LabSidearmClient	116
25.2	S28 LoungeClient	117
25.2	S21 LabSpatial	117
26	Historical visual references	118
27	Bibliography and source access	123
27.1	Shared development chat	123
27.2	Roblox client/server boundary	123
27.3	Roblox Camera reference	123
27.4	Roblox pathfinding	123

27.5Roblox UI position and size	123
27.6Roblox text filtering	123
27.7Local primary evidence	124

II Complete active Studio source 125

28S01 - GWAPNPC 126

29S02 - GWAPRouting 128

30S03 - GWAPScenarios 130

31S04 - GWAPWorld 131

32S05 - LabKinds 134

33S06 - Blaster 137

34S07 - BlasterController 141

35S08 - GWAPServer 154

36S09 - GWAPWallDamage 163

37S10 - LabElevators 165

38S11 - LabEquipment 168

39S12 - LabRoles 226

40S13 - LabRound 233

41S14 - LabTechnicians 235

42S15 - LoungeSystems 239

43S16 - ServerBlasterManager 242

44S25 - LabHUD 245

45S26 - LabQuizClient 248

46S27 - LabSidearmClient 251

47S28 - LoungeClient 253

48S21 - LabSpatial 255

III Complete archived Studio source 256

49S17 - LabElevators 257

50S18 - LabEquipment	259
51S19 - LabRoles	316
52S20 - LabRound	323
53S21 - LabTechnicians	325
54S22 - LoungeSystems	329
55S23 - ServerBlasterManager	332
56S24 - BlasterController_BeforeMouseArmFix_20260912	335
 IV Complete disabled Studio generator	 346
57S29 - SeedLabGenerator_DISABLED_doNotEnable	347
 V Complete historical workspace source	 511
58W01 - SeedLabGenerator.lua	512
59W02 - InstallGWAP.lua	683
60W03 - GWAPWorld.lua	716
61W04 - GWAPScenarios.lua	721
62W05 - GWAPNPC.lua	722
63W06 - GWAPRouting.lua	724
64W07 - GWAPServer.lua	726
65W08 - GWAPWallDamage.lua	734
66W09 - GWAPHUD.lua	736
67W10 - Blaster.lua	739
68W11 - BlasterController.lua	743
69W12 - ServerBlasterManager.lua	756
 VI Complete server-only documentation source	 759
70DOC01 - GameDocumentation	760
71DOC02 - GameDocumentationAnnex2	784

Part I

Game design and engineering record

Chapter 1

The game as it plays: environment, cast, equipment and tactics

Every photograph in this chapter was taken from the live place on 14 September 2026, by driving the Studio camera to the stated position and capturing the viewport. Nothing is a mock-up or a render of a plan.

1.1 Photographic survey



Figure 1.1: The laboratory from the south-west approach. Eight storeys above grade over a basement, inside a fenced compound. The three blue-glazed structures on the apron are the ground-level glasshouses; the darker strip at the near corner is the entrance canopy.



Figure 1.2: Site works from the north-east: perimeter fence and gate, parking bays with four vehicles, planters, flagpole, trial plots and twenty-eight trees planted in bands that avoid the building footprint and the aprons.

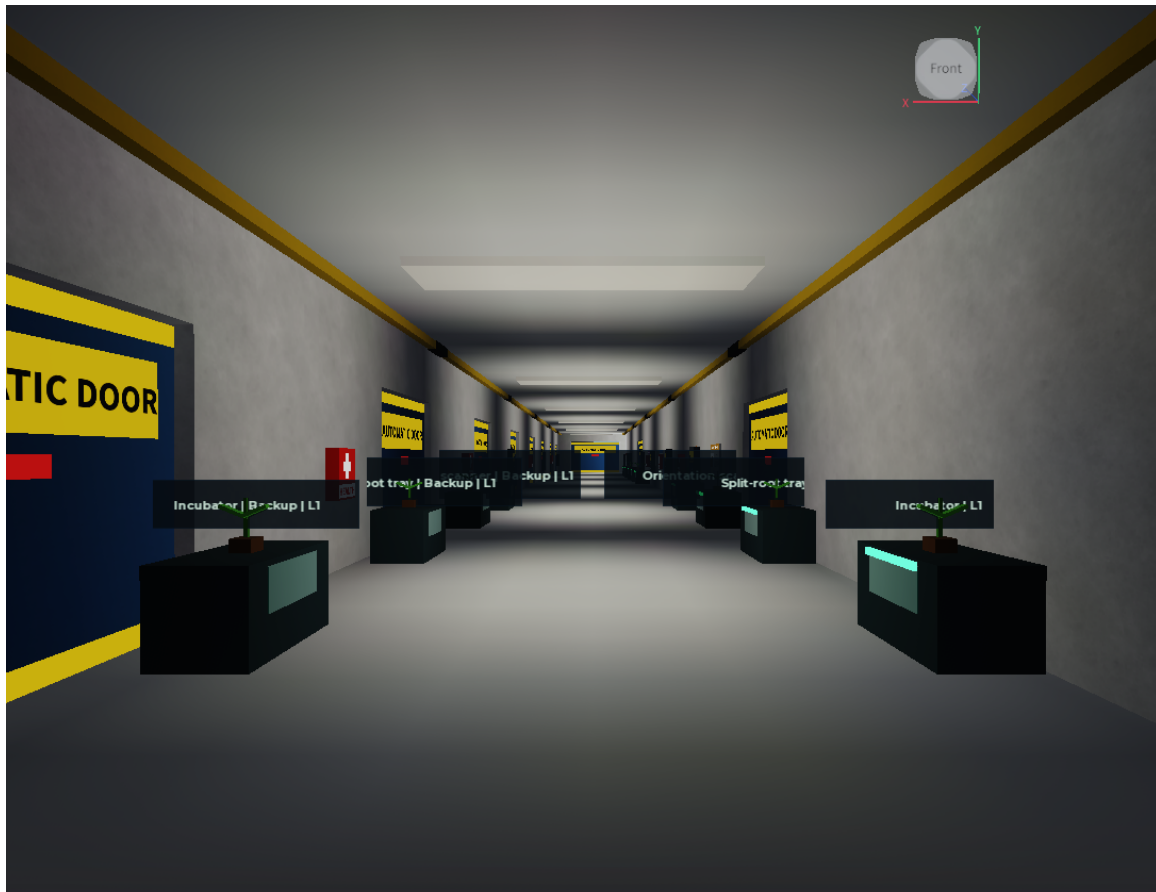


Figure 1.3: A typical floor corridor, looking north along Level 1. Research stations stand against both walls with their own labels and status lights; automatic doors are marked in yellow and blue; emergency kits are the red cabinets. The corridor is the spine of every floor and the only continuous route between the two stair towers.



Figure 1.4: Inside a laboratory room. The germination chambers are the destructible equipment: 450 of them across the building, each with its own growth code, functional state and repair prompt.



Figure 1.5: The seed bank in the basement. This is the first stop of the germination assignment and the origin of the mission seedling.

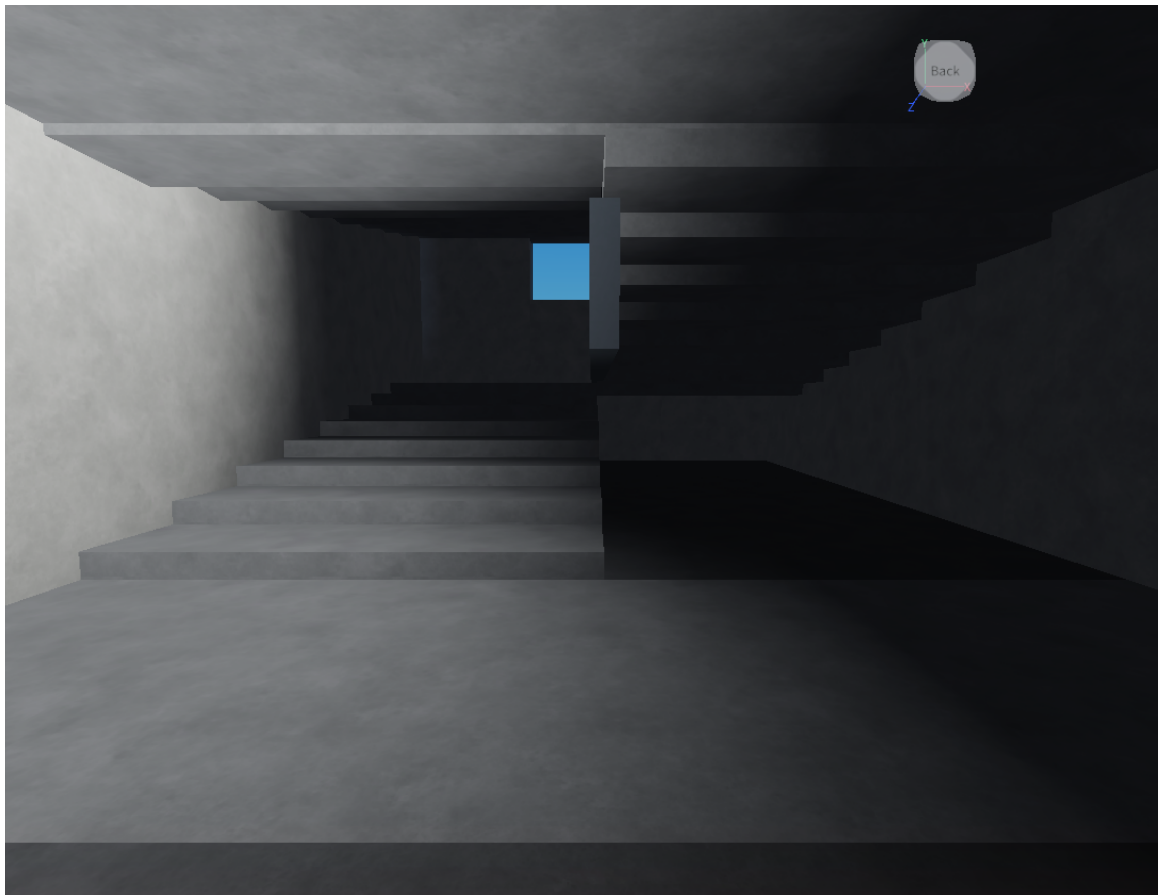


Figure 1.6: The basement stair landing, after Revision 5 closed the void beneath it. Before the repair this landing was a plate with nothing under it and a step off the edge dropped the player out of the world.

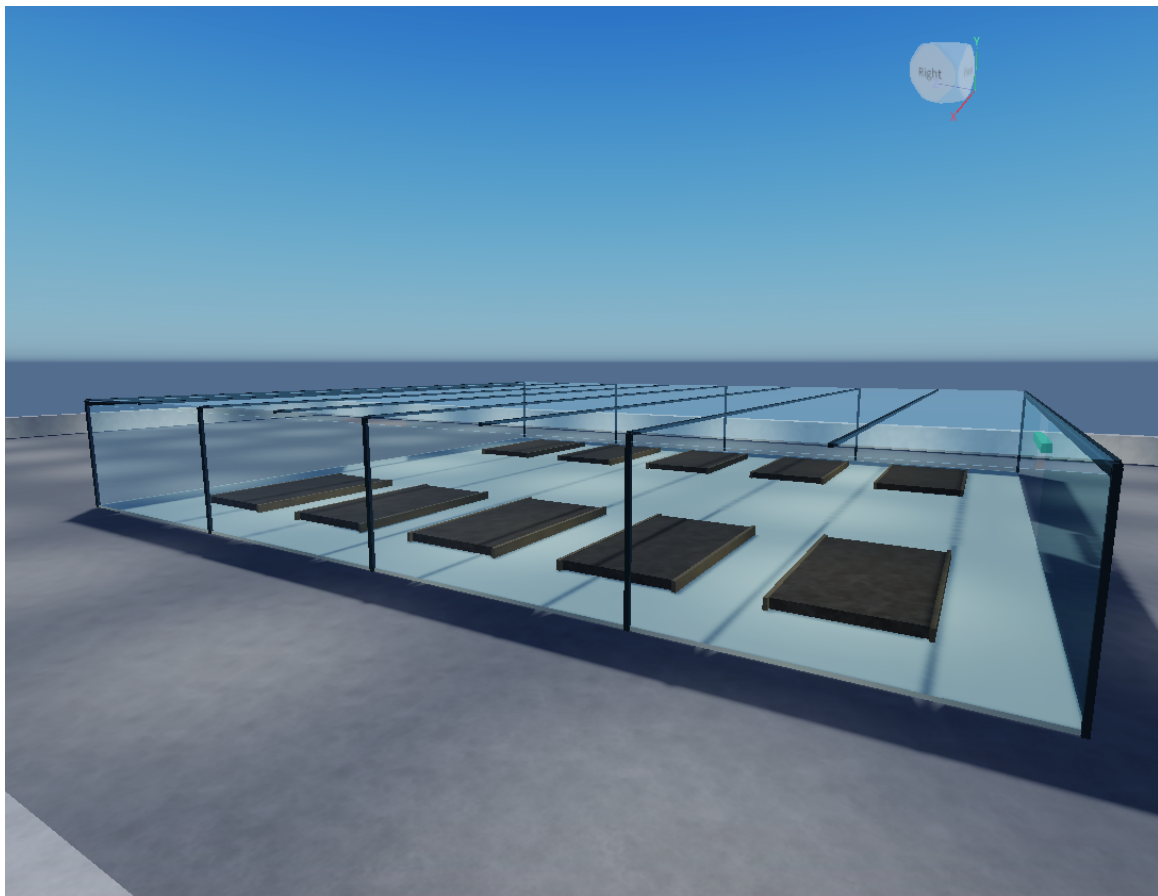


Figure 1.7: The rooftop greenhouse, the final stop of both assignments. Ten planting plots sit under the glass; each accepts up to twelve seedlings.

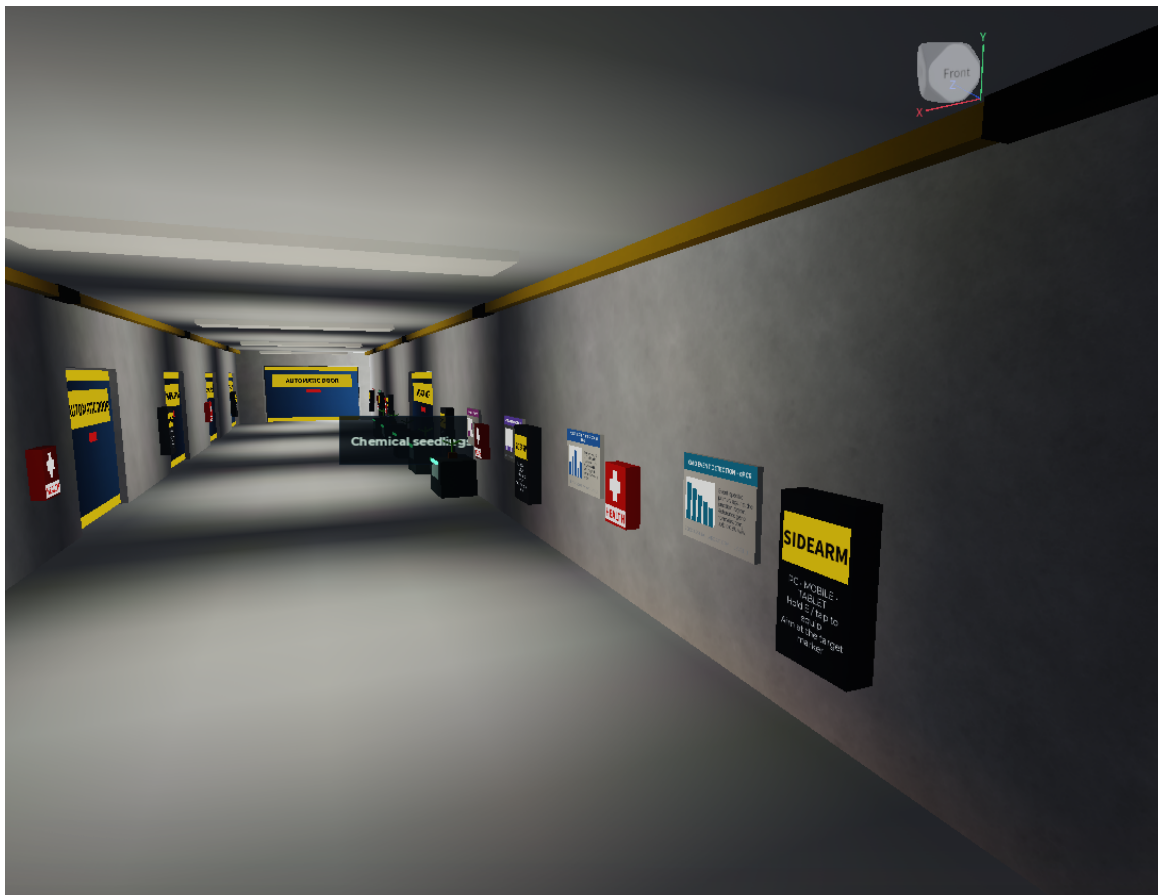


Figure 1.8: Corridor fixtures: an emergency health kit, a weapon cabinet, a scientific poster, and the yellow gas line running near the ceiling. The gas line is live ordnance – a shot into it detonates the section.



Figure 1.9: A seated worker at a workstation. Forty-five of the one hundred and eighty workers are desk staff: they never patrol and never join an ambush, but they are still targets.

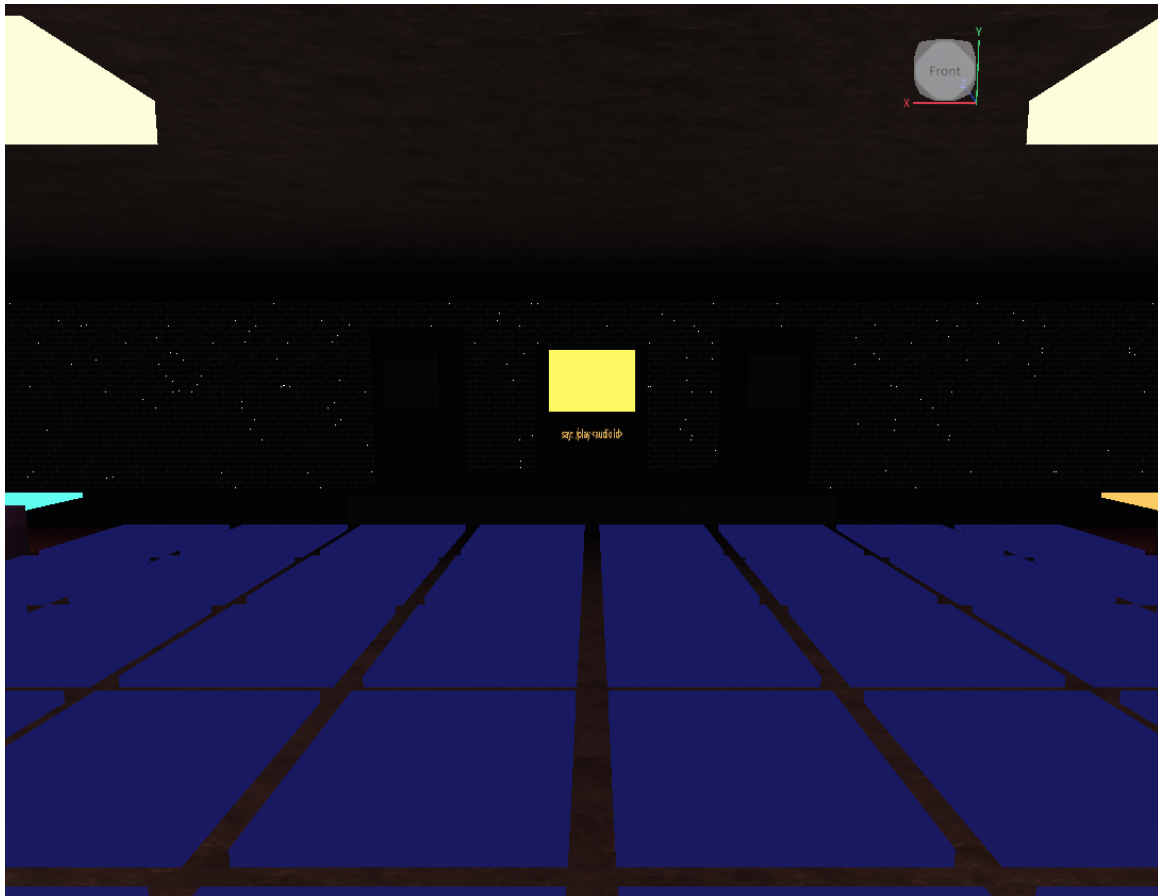


Figure 1.10: The hidden lounge, sealed beneath the basement and reached only through the floor hatch. It has a bar, a stage, a jukebox and a message wall on which players can post filtered notes.



Figure 1.11: Gameplay. The mission panel carries the assignment title, the current stop, the carrier's health and the four carrier commands. The top strip shows both role scores, the shift clock, points and player health. The green line on the floor is the mission trail. The worker on the right is a live ambusher – its billboard reads *Hunting*, the state that tells the patrol system to stand aside.

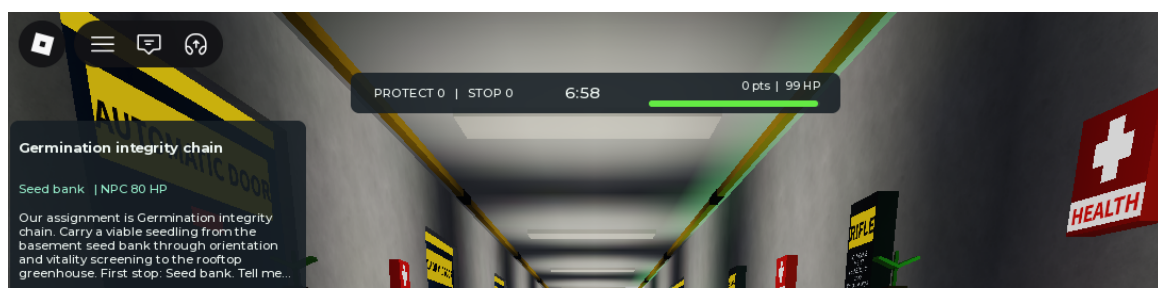


Figure 1.12: The same corridor during an ambush, from above and behind. Workers hold a firing line rather than closing: none comes inside fifteen studs, which is what makes them aimable.

1.2 The environment

1.2.1 Scale

The complex occupies roughly 326×487 studs in plan and stands 162 studs from the basement floor to the roof parapet, an envelope of $x - 80 \dots 246$, $y - 17 \dots 145$, $z - 141 \dots 346$. It is built from 61,694 parts. Nine occupied levels carry between 5,831 and 7,225 parts each; the basement is the densest.

Level	Parts
Basement	7,225
Level 1	5,831
Level 2	6,140
Level 3	5,987
Level 4	6,122
Level 5	6,005
Level 6	6,133
Level 7	6,020
Level 8	6,123

1.2.2 Floor layout

Every floor is one central corridor with rooms either side. The corridor runs north to south between the two stair towers; the room walls are pierced by automatic doors that slide up into the header when anyone comes within eleven studs and hold open for at least three seconds. There are 162 such doors. The corridor is where the fixtures live: emergency health kits, weapon cabinets, posters, and the gas line near the ceiling.

1.2.3 Vertical circulation

Two stair towers, north and south, project beyond the building footprint and run the full height. Each level is served by a landing, a half-landing and two flights of eight treads. There are no working elevators. The towers are the only way between floors, which makes them the natural place to intercept a carrier and the worst place to be caught by an ambush.

1.2.4 Ground level and the site

Outside the building the compound holds a perimeter fence with a vehicle gate, an entrance canopy on posts, parking bays with four cars, two planters, a flagpole, three glasshouses on the western apron, two trial plots to the east, and twenty-eight trees. The ground plane sits at $y = -8$, between the basement floor at -16.25 and the first floor at 0 .

1.2.5 The rooftop greenhouse

The roof carries a glazed greenhouse containing ten planting plots. Both assignments end here. A plot accepts twelve seedlings before it is full. The greenhouse glass is breakable, and hostile workers will deliberately shoot it out to open a firing line.

1.2.6 The hidden lounge

A sealed room sits below the basement, $y - 49 \dots - 30$, reached through a hatch in the basement floor at $(110, -16, 130)$. It contains a bar with stools, four sofa groups, a stage, a jukebox and a

message wall. It is out of play: no mission routes through it and no worker patrols there.

1.3 Characters

1.3.1 The two player roles

Roles are chosen at one of two kiosks and cannot be changed mid-assignment.

Role	Objective
Technician	Escort the research carrier to the rooftop greenhouse. Score for staying within 45 studs at each checkpoint, and for the final planting. Hitting your own carrier cancels your escort rewards. Only a Technician may repair destroyed equipment.
Laboratory Technician	Stop the carrier before it plants. Score for bringing it down.

Selecting a role assigns one of the two missions immediately. There is no mission menu.

1.3.2 Laboratory workers

One hundred and eighty workers populate the building. They are differentiated by appearance and name, and forty-five of them are seated desk staff who never leave their workstation. The rest run a shift routine: they respond to equipment alarms, walk between rooms, and idle when no player is in the room with them – an occupancy gate that keeps most of the roster asleep and is the single largest reason the building runs at all.

Workers become hostile only through the ambush system. At most five are armed at any moment; when one is put down, a room further along the player's route releases a replacement.

1.3.3 The research carrier

The carrier is the mission. It is a worker rig that walks the assignment route under navigation derived from the live map, carries the seedling between stations, and speaks to its escort through the mission panel. It has 100 health and no weapon. If it dies the round is decided in the Laboratory Technician's favour and a fresh assignment is issued eight seconds later.

1.4 Equipment and fixtures

Fixture	Count	Behaviour
Germination chamber	450	Destructible. Explodes when shot, killing workers within 17 studs and hurting the player. A Technician restores it by holding R.
Research station	67	Twenty capabilities, each with an original and two backups, plus ten rooftop plots. Mission stops resolve to a capability, not a specific machine.
Automatic door	162	Slides open within 11 studs, holds 3 seconds, and can be forced open by an ambush.
Emergency health kit	90	Restores health, then recharges over 25 seconds.
Weapon cabinet	90	Dispenses one of the three player weapons.
Gas line	18	Two runs per floor near the ceiling. Detonates when shot, with a 26-stud blast.
Scientific poster	72	Takes permanent bullet holes.
Luminaire	333	Corridor and room lighting.

1.5 Weapons

Three weapons are available to players. All three cost the shooter one percent of their score per shot, and a head shot does double damage rather than an instant kill. A worker has 100 health.

Weapon	Damage	Pellets	Range	Cooldown	Shots to kill
Lab Sidearm	34	1	240	0.32 s	3
Lab Shotgun	12	6	110	0.85 s	2
Lab Rifle	55	1	620	0.95 s	2

The sidearm is the fastest cycling weapon and the only one that stays accurate while moving. The shotgun's 72 damage assumes every pellet connects, which holds only inside about twenty studs; beyond that it degrades quickly. The rifle is the corridor and stairwell weapon: its 620-stud range covers the full length of any floor.

Hostile workers fire a single accurate ray with no spread, once every ten seconds each, for half a health point per hit. Their danger is not the rifle damage; it is what they do to the room around you.

1.6 Gameplay

1.6.1 The round

A shift lasts 480 seconds – eight minutes – sized from the equipment count and the mean distance between machines. When it ends, destroyed equipment is repaired, planted seedlings are cleared, bodies are removed and the workers are stood back up.

1.6.2 Scoring

Event	Points
Protected a research checkpoint (Technician, within 45 studs)	+15
Protected the rooftop planting (Technician)	+60
Stopped the assigned carrier (Laboratory Technician)	+50
Hit by a worker	−1% of current score
Firing any weapon	−1% of current score

The one percent costs are proportional, so they bite hardest when you are winning. A player sitting on a large score loses more per trigger pull than a player starting out.

1.6.3 Scenario one: the escort, minute by minute

Time	What happens
0:00	Pick the Technician kiosk. The assignment is issued at once and a carrier walks to you. The mission panel names the first stop.
0:10	Press <i>Ready</i> . The carrier sets off for the basement seed bank and the green trail appears.
0:30	First stair tower descent. This is the first ambush window: rooms you pass on the way to the stairs wake as you approach.
1:00	Seed bank. The carrier processes for four seconds and takes the seedling. Stay within 45 studs to bank the checkpoint.
1:30–4:30	Four more stops, spread across the basement, Levels 1 and 2. Expect at least one reroute: some stations have no walkable approach and the carrier will divert to a backup with the same capability.
5:00	Long climb to the roof. The stair towers are where an opposing Laboratory Technician will wait, because the carrier cannot avoid them.
6:30	Rooftop greenhouse. The carrier plants the seedling into a plot. Escort bonus awards if you are close.
7:00	Assignment complete. A replacement is issued eight seconds later and the cycle restarts within the same shift.
8:00	Shift ends. Equipment repairs, plots clear, workers stand back up.

1.6.4 Scenario two: the interception

Time	What happens
0:00	Pick the Laboratory Technician kiosk. You are assigned the cold-chain audit and a carrier of your own.
0:20	Collect a rifle from a corridor cabinet. The rifle, not the shotgun: you will be shooting down corridors and stairwells.
1:00	Locate the carrier. Its destination floor is on the mission panel, so you can move to the stairwell it must use rather than chase it.
2:00	Take the stairwell. A carrier in a flight cannot leave it and cannot shoot back.
2:30	Sustained fire brings it down in about 25 seconds unopposed; far less with the ambush helping, since half of every ambush targets the carrier.
3:00	+50 points. A replacement carrier is issued, and you reposition for the next one.

1.7 Tactics worth knowing

These are consequences of how the systems are built, verified by measurement. They are legitimate play, not defects, but they are not obvious.

1. **Let the ambush kill the carrier for you.** Half of every ambush is assigned to the carrier when it wakes. As a Laboratory Technician you often do not need to fire at all – shadow the carrier, stay out of the standoff line, and let the building do the work.
2. **As a Technician, pull the ambush off the carrier.** Hostiles that are not carrier hunters switch to whoever is closest. Standing on the far side of the corridor from your carrier drags the other half of the roster onto you, which is survivable at half a point per hit.
3. **Never fight next to a gas line.** Hostiles deliberately shoot pipes and machines near you. The blast radius is 26 studs for a pipe and 17 for a machine, and both hurt you far more than a rifle round. Fight in the middle of the corridor, not against the wall where the fixtures are.
4. **Weapon cost is proportional.** At 1% of score per shot, a missed burst late in a round costs more than the same burst early. Once you are ahead, stop firing and escort.
5. **Head shots are worth aiming for, but not decisive.** They double damage; they no longer kill outright. A rifle head shot still takes a worker down in one, a sidearm head shot does not.
6. **Emergency kits recharge, so corridors are safe ground.** Ninety kits on a 25-second cycle means sustained hostile fire is survivable indefinitely if you keep moving between them.
7. **Break the line of sight rather than the distance.** Hostiles hold at twenty studs and shoot accurately, but they will not close. Stepping into a room and letting the door shut ends the engagement outright.
8. **Repair is a scoring move for Technicians.** Destroyed equipment removes a station from the mission pool and can force a reroute. Repairing it near a carrier's next stop shortens the route.
9. **Stairwells favour the interceptor.** A carrier on a flight follows the treads and cannot deviate, which makes its position predictable for several seconds.
10. **The rooftop is the last chance, not the best one.** Hostiles will shoot out the greenhouse glass to reach you, so cover there is temporary.

Chapter 2

Revision 5: coordinate repair, navigation, and the hostile ambush

2.1 Summary

Revision 5 began as a correction to Revision 4 and became a rebuild of three subsystems. It reverses a geometry migration that had been applied to the wrong object, closes a fall-through hazard in the basement, replaces the research carrier’s hardcoded route table with navigation derived from the live map, reduces the ten selectable scenarios to two role-bound assignments, and adds a hostile ambush system in place of the previous passive opposition.

All work in this chapter was applied to the live place through the Studio bridge and verified by numeric scene queries and instrumented play sessions rather than by observation alone.

2.2 The coordinate fault and its repair

2.2.1 Diagnosis

Revision 4 concluded that `Laboratory_Complex` had shifted and that the lounge should be moved to follow it. It inferred the displacement $(-25.142857, -23.25, +91.285714)$ studs and applied it to `Workspace.Lounge` and `Workspace.LoungeHatch`, a total of 169 parts.

The vector was correct. The object was not. Ground truth was recovered from `D:\game\4\Export.glTF`, exported on 11 September 2026, which stores the authored node translations:

Node	Authored	Live before repair
Laboratory_Complex/Roof_1	82.2857, 129.375, 133.7143	57.1429, 106.125, 225.0000
LoungeHatch	110, -15.9, 130	84.857, -39.150, 221.286
HatchRim	110, -16, 130	110, -16, 130
Lounge/Floor	80, -49, 130	80, -72.25, 221.286
LabSpawn	32.143, 1, 11.571	32.143, -0.1, 11.571

`HatchRim`, the three spawn markers and `LaboratoryFoundationGround` had not moved. Only the laboratory had. Because the ground plane remained at $y = -8$ while the first floor sank to -23.25 , the ground plane sat 15.25 studs above the first floor and intersected Levels 1 and 2. That is the reported gap in the building and the exterior.

Revision 4’s partial migration made the fault worse: the lounge followed the displaced building while the hatch’s own rim did not, leaving a hatch and its rim 95 studs apart.

A further confirmation is available in Revision 4's own code. The constant `BASE_ROOF` in `LabSpatial` is `CFrame.new(82.28571428571429, 129.375, 133.71428571428572)`, which is exactly the authored roof position. The frame that module composed was therefore identity with respect to the authored datum, not with respect to the displaced one.

2.2.2 Repair and verification

Two rigid moves were applied, each guarded before execution:

1. Lounge and LoungeHatch, 169 parts, by $(+25.142857, +23.25, -91.285714)$. The move was performed only because the inverse placed LoungeHatch within 0.1 studs of HatchRim; otherwise the operation refused and changed nothing.
2. Laboratory_Complex, by the delta that returns Roof_1 to its exported position, as a single PivotTo.

Check	Result after repair
Roof_1	82.2857, 129.375, 133.7143, the exported value
Level_1 slab top	0.000
Basement slab top	-16.250
Ground plane	-8.000, correctly between the two
Research stations on a real slab	67 of 67, none in mid-air
Laboratory spawns	all three resting on FloorSlab_1
Building parts inside the lounge volume	1963 → 0
Part counts	61513 / 168 / 4 / 1 × 6, unchanged
<code>LabSpatial.frame()</code> translation	0.0000

The final row is the operative one. With the map restored to its authored datum, Revision 4's entire coordinate layer evaluates to identity and is inert. It was therefore removed rather than unpicked: `ReplicatedStorage.LabSpatial` was moved to `ServerStorage.LabSpatial_RETIRED_revision4`, and all thirty `Spatial.*` call sites were reverted by rebuilding the affected scripts from the pre-Revision-4 baseline.

2.2.3 Correction to the Revision 4 record

The Revision 4 entry in `ServerStorage.GameDocumentation` states "The building itself was not moved back" and "Do not move the lounge twice". Both are now false and both would mislead a later reader. A correction notice was prepended to that module, and the `LabFrameAligned` marker on `Workspace.Lounge` was cleared.

2.3 Basement stair-tower void

Both stair towers sit outside the basement slab: `StairTower_North` at $z = -6.4$ and `StairTower_South` at $z = 273.9$, while `BasementSlab` spans $z = 0$ to 267.4 . Each `Landing_L-1` was therefore a plate 0.75 studs thick with nothing beneath it, and a player stepping off its edge fell into the void. This is the blue area reported at the foot of the basement stairs; the colour is the skybox seen through the gap.

A `TowerBaseSlab` was added under each tower at the basement datum, matching the basement slab's material and colour. A sweep of 6073 sample points across the basement floor and both tower footprints subsequently found zero unsupported points, and every edge of both basement landings resolves onto either `TowerBaseSlab` or `BasementSlab`.

2.4 Carrier navigation and physical movement

2.4.1 The fault

The research carrier walked through walls, equipment and floors. The cause was structural rather than incidental. `routePoints` returned a hardcoded polyline: corridor lane $x = 78$, stair lane $x = 68.7857$, tower mouth $z = -6$, and per-tread offsets of the form $-15.267857 - (i-1) \times 4.821428$. The walk loop then advanced the rig with `PivotTo`, testing only for player proximity so as not to shove them. No test of geometry existed anywhere in the path, so walls were not merely passable, they were absent from the model.

2.4.2 GWAPNavigation

Routing was replaced by `ReplicatedStorage.GWAPNavigation`, which reads the map as it exists at the moment of the request. Levels come from the actual `FloorSlab` and `BasementSlab` parts; flights come from the named tread parts inside each `StairTower`; and the approach point for a station is recomputed from that station's live bounds, filtered by a ground test, a line-of-sight test and a body-box clearance test. No coordinate is stored, so relocated equipment and re-packed rooms cannot strand a carrier.

The walk loop was made physical. Because the rig is anchored and driven by `PivotTo`, which bypasses the physics solver, each step is swept against the world by hand: a blocked step is never applied, and an applied step is followed onto the surface beneath it so the carrier can neither sink through a floor nor walk on air. A persistent obstruction recomputes the route rather than waiting out the mission clock.

2.4.3 Defects found during integration

Four faults were introduced or exposed while wiring this in, and are recorded because each is a general trap rather than a local slip.

1. **Ground follow fought a deliberate climb.** A waypoint 0.82 studs directly above the carrier, with zero horizontal component, could never be reached: the surface snap reset the height on every frame. The loop span indefinitely with no error and no message. The snap now applies only when the step carries horizontal travel.
2. **Degenerate waypoints.** Two landing points measured against different surfaces produced purely vertical hops, which are unreachable for a walker. `GWAPNavigation` folds them into the following point.
3. **Unbounded waypoint wait.** A single unreachable waypoint could hold a mission indefinitely. A per-waypoint deadline now abandons it, and three consecutive failures recompute the route.
4. **Sticky target bypassed the reachability test.** Once a `target` was set, the first branch of the selection expression returned it without consulting the reachability predicate, so an unreachable station was proposed forever.

2.4.4 Measured result

A full instrumented run, sampled every 70 milliseconds:

Distance walked	approximately 1190 studs, basement to roof
Wall penetrations	0
Airborne frames	0
Automatic reroutes	2, around unreachable equipment
Mission stages completed	4 of 4

2.5 Role-bound assignments

The ten selectable scenarios were replaced by two, one per role, each a six-stop run spanning basement, intermediate floors and the rooftop greenhouse:

Assignment	Route
Germination integrity chain	Seed bank, Incubator, Clinostat, Orientation scanner, Vitality scanner, rooftop plot
Cold-chain contamination audit	Cryo-freezer, Wet lab, Symbiosis incubator, Chemical seedlings, Porometer, rooftop plot

Every stop is a station capability with three instances in the map, one original and two backups, so a destroyed or unreachable stop reroutes to an equivalent rather than stalling the run.

Selection was removed entirely. Setting LabRole assigns one of the two immediately; the HUD's job button, scenario menu, list and random-assignment control were deleted; and the server now refuses a choose action arriving from a client, since after this change it can only be a stale HUD or a crafted packet. Because nothing can be chosen, finish() issues the next assignment eight seconds after a mission ends.

2.6 Hostile ambush system

The previous opposition fired once every 2.5 seconds, only when a single player was present, and then applied a random spread of ± 2.5 studs, so it usually missed and never left its room. It was replaced.

Behaviour	Mechanism
Emerge from rooms	rooms within 58 studs release up to two workers; the door is held open for nine seconds through a ForceOpen attribute honoured by bindAutoDoor
Stairs and greenhouse	pursuit is position-based rather than room-based, so workers follow onto the towers and the roof
Accurate fire	a single ray to the target with no spread
Hazards	a proportion of shots target a gas pipe or a live machine beside the player
Greenhouse glass	a shot blocked by greenhouse glass shatters it, clearing the line for the next
Carrier as target	half of each ambush is assigned to the carrier when it wakes
Bounded cost	a hard roster cap, replenished from later rooms as workers are put down

2.6.1 Interaction with the patrol system

Hostile workers initially did not move at all. The existing patrol logic in LabEquipment was calling PivotTo on the same rigs; two systems driving one rig cancel each other out. The patrol now yields for as long as GWAPHostile is set. A second cause followed: a three-direction wall slide could not negotiate a laboratory dense with benches, and the sweep was correctly refusing to pass through an incubator. Steering was widened to eight headings with a real path solve when a worker is stuck, throttled to at most one solve per second across the whole roster. Seated desk staff are excluded, since they have no walking pose.

2.7 Damage model corrections

2.7.1 Fractional damage

The rate limiter returned a fraction of the requested damage. With several shooters sharing one budget, each landed shot was scaled to roughly half a point, so a hit did not read as a hit. The limiter now governs how often a shot may hurt, never how much: a shot either lands for its full value or is spent as a near miss.

A constraint follows from this and is recorded because it is not obvious: under all-or-nothing accounting the cap must be at least the size of one hit, or no shot can ever afford to land. One hit per second is therefore the fastest possible cadence, and survival time is set by hit size rather than by the cap.

2.7.2 Universal one-shot kill

Every weapon killed a full-health worker in a single shot. The cause was not a damage value. It was the line

```
tool:SetAttribute("BlasterInstantHeadshot", style.pcOnly == true)
```

Each of the three player weapons sets `pcOnly = true`, so instant-headshot was enabled on all of them. Whenever a ray cleared the worker's collision hull and caught the head, the shot killed outright regardless of the weapon. Reducing the shotgun's pellet damage in an earlier pass could not address this, because the lethality never came from the damage figure.

A head shot now doubles damage instead of ending the engagement, in both firing paths. This closes every thousand-damage route through a gun; the remaining thousand-damage calls are the equipment and gas-pipe explosions, which are intended to be lethal. Verified through the live `BlasterHitBridge` against a full-health worker:

Weapon	Damage	Shots to kill
Lab Sidearm	34	3
Lab Shotgun	6×12	2
Lab Rifle	55	2

The bridge itself was never at fault. An isolated invocation carrying 34 damage took a worker from 100 to exactly 66, which is the measurement that eliminated `damageWorkerBot` and the weapon table and identified the tool attribute.

2.7.3 Standoff

Workers closed to six studs and formed a press the player could not aim through. They now hold a firing line: a worker closes only to `STANDOFF`, gives ground if it ends up inside `STANDOFF_MIN`, and applies a separation push away from every other hostile within `SPACING` on every tick. Applying separation only once a worker had stopped, which was the first attempt, left those still closing to pile into those that had halted.

A wider line of 24 and 17 studs was tried first and held the workers so far back that corridor equipment broke line of sight and nothing landed at all across 28 seconds of observation. The values below keep clear room to aim while leaving the workers in sight of the player.

2.8 Constants reference

Constant	Value	Meaning
HOSTILE_CAP	5	armed workers active at once
PER_ROOM	2	released from any one room
RELEASE_RADIUS	58	distance at which a room wakes
SHOT_INTERVAL	10 s	one shot per worker
HOSTILE_DAMAGE	0.5	absolute health points per landed hit
INCOMING_CAP	5	combined limit, above hit size so all shots land
CARRIER_DAMAGE	4	per landed hit on the carrier
CARRIER_INCOMING_CAP	4	one hit per second, roughly 25 s unaided
CARRIER_HUNTER_SHARE	0.5	fraction of an ambush assigned to the carrier
HAZARD_CHANCE	0.08	odds a shot targets a pipe or machine
STANDOFF	20	distance held from the target
STANDOFF_MIN	15	closer than this, the worker gives ground
SPACING	9	minimum separation between hostiles

Measured with these values: recorded hit sizes of 0.5 throughout, 15 hits across 35 seconds from four to five workers, player health 100 to 94.5, closest approach 13 studs, typical engagement distance 44 studs, no script errors.

2.9 Open items

1. **Nine unroutable stations.** Nine of the 67 research stations have no walkable approach route. All are originals positioned at $x = 67$, hard against the corridor's western wall. Missions reroute to backups automatically, so play is unaffected, but those nine are decoration until they are moved clear of the wall.
2. **Cause of the original displacement is unknown.** If a generator rebuild writes the laboratory at a different origin, the fault recurs. Before any full regeneration, Roof_1 should be checked against (82.2857, 129.375, 133.7143).
3. **Two runtime copies.** The weapon table and several helpers exist both in the installed scripts and in the generator's embedded payload. The corrections in this chapter were applied to the installed scripts; a full regeneration would restore the previous values.

Chapter 3

Revision 4: live repairs and current handover

3.1 Authority and chronology

This chapter supersedes the pre-revision findings wherever a repair is explicitly recorded here. The baseline inventory was captured from Studio version 70; the owner reopened version 72 and all 29 source fingerprints matched that baseline before modification. The owner then explicitly authorized conservative live repairs and documentation inside source comments. The changes below were installed through Studio MCP in Edit mode and exercised in fresh Play sessions on 13 September 2026. No publishing action was performed. The exported RBXL remains the original evidence, not a post-repair save.

3.2 What was actually changed

C01, stale coordinate metadata: the laboratory had moved by approximately (-25.142857, -23.25, 91.285714) studs while navigation still used generator coordinates. LabSpatial derives a frame from the existing Roof_1 landmark and its original generator CFrame. World construction, station backups, mission stairs, rendezvous, legacy repair routes, room lookup, worker patrol destinations, room smoke and door ventilation now translate between laboratory-local coordinates and world coordinates explicitly. Each of the 67 station approaches has an ApproachPoint attachment. Its world position follows subsequent station movement; ApproachLocal preserves the migration input. The building itself was not moved back.

A second scene mismatch was found by sweeping a carrier-sized box along the corrected route: Lounge.Wall_N intersected the basement stair exit. The laboratory had moved but the lounge and its hatch had not. All 169 associated BaseParts were transformed once by the same laboratory frame. Original CFrames are retained in the live rollback folder. This is a scene migration, not a repeating runtime translation. Future whole-world moves must include the lounge, hatch and laboratory together.

C02, incorrect installer payload: the loose GWAPWorld.lua and old InstallGWAP.lua contained mission-server code under the world-module name. The corrected root module is the verified world implementation. The root installer is regenerated from the final source set, including LabSpatial and changed legacy dependencies. It is a small local-HTTP preflight loader, with complete source payloads in release/. Each payload is length/checksum verified and compiled before mutation. A monolithic payload would exceed Studio's 200,000-byte Script.Source limit. Serve the documentation folder on localhost port 8766; no external hosting is required. Historical copies under code/workspace remain unchanged and are explicitly pre-revision evidence. Do not confuse those historical files with the corrected root release. The large generator is not rebuilt or executed by this revision.

R01, research repair authority: the server now requires a living Technician, workspace member-

ship, no BlastRemoved flag, an exploded target, proximity within 14 studs and an unobstructed ray from the player's head to the equipment. UI prompt visibility alone no longer authorizes repair. HoldDuration remains an interface duration; this revision does not certify resistance to forged hold timing. That is a remaining security-hardening task, not a claim of complete exploit resistance.

R02, carrier collision: the existing anchored carrier route movement now performs a 2.4 by 4.4 by 2.4 stud swept query before each short movement step. A solid obstacle stops movement, extends the mission deadline and produces an explanatory dialogue at most once per ten seconds. Clearing the obstacle resumes travel. Arbitrary obstacle detours are not promised; Wait and Regroup remain the recovery controls. Player movement, camera, mouse aiming, mobile aiming, gun recoil and input bindings are preserved.

R03, solo eligibility: only living active NPCs in Workspace enter the armed selection. Stored carriers and knocked-out/dead workers are excluded. The armed count is floor(eligible count * 0.2). Eligibility is reconciled when population state changes and on a five-second bounded check; already selected gun instances are reused. The original inventory mesh specifications are unchanged.

R07, shared lounge notes: player-authored note text passes through TextService.FilterStringAsync and GetNonChatStringForBroadcastAsync before broadcast. Failed filtering produces no shared note. A three-second per-player cooldown bounds requests. Departure clears note cooldown and skip-vote state. Published-service filtering behavior is not certified by a local Studio session; the fail-closed source path is the evidence for this change.

HUD and allocation fixes: the carrier's health is folded into the existing mission target line, and its duplicate overhead billboards are hidden. GWAPNPC.say retains dialogue attributes but no longer creates a server-enabled MissionSpeech billboard; this removes the observed flash before client suppression and the redundant GUI instances. Other targeted NPC billboards are suppressed near the top HUD. Existing compact layout and original weapon controls are retained. Pool reactivation no longer installs duplicate billboard watchers. Route nodes are sorted once until ChildAdded or ChildRemoved invalidates the cache, allowing incremental replication. Terminal missions immediately destroy their route nodes. These are bounded allocation reductions; no invented megabyte saving or sixty-player performance claim is made.

3.3 Fresh verification results

V4-01: all seven initial changed/new source files compiled in Studio before installation. A fresh Play session reported no server or client MessageError entries at the inspected checkpoints.

V4-02: Incubator approach migrated from (78, 3.25, 49) to approximately (52.857147, -20, 140.285706), adjacent to its actual body at (41.857147, -21.75, 140.285706). The initial companion appeared 8.0163 studs from the player. Four unused carriers stayed in ServerStorage. Solo count was 36 armed among 181 active eligible NPCs.

V4-03: scenario 1 completed seed bank, instrument and rooftop planting in the corrected scene. Plot 1 gained one plant, score increased from 0 to 90 and route-folder count returned to zero. Scenario 2 then completed and the cumulative score became 180. A test-only server helper kept the owner six studs beside the moving carrier and restored health; it did not shoot, modify points, advance stages or move the carrier. This verifies mission execution and clean rewards, not human stair-navigation skill or multiplayer load. Helpers existed only in Play and were discarded on Stop.

V4-04: equivalent-station selection chose Incubator - Backup 1 on floor 0 after primary destruction, then Incubator - Backup 2 on floor 1 after the first backup was made unavailable. The controlled selection test restored station flags afterward. Full mission fallback across every capability remains a broader regression target.

V4-05: a temporary solid obstacle inserted into a live carrier route produced zero movement over a half-second stopped sample. After removal, the carrier moved 9.9987 studs in approximately half a second with owner WalkSpeed 20. The obstruction dialogue appeared. No player movement settings were changed by the carrier implementation.

V4-06: a controlled exploded wall section removed a health station, gun fixture, poster and research-equipment fixture from Workspace. A fixture outside the section stayed present and unmarked. All test geometry was cleaned up. Existing actual-wall selection and quarantine behavior are retained.

V4-08: in a fresh final-source Play session, holding R within six studs restored an exploded Incubator for the Technician role. Repeating the same interaction as Laboratory Technician left the target destroyed, as required. No server errors were reported. An earlier attempt at the outer edge of prompt range did not activate; the closer prompt interaction is the successful observation.

V4-07: a 705 by 338 client viewport loaded without client errors. Visual inspection found a carrier overhead-bar overlap with the top strip, leading to the final health-in-panel change. The final UI recheck is recorded in VERIFICATION.md. The accepted Blaster and BlasterController executable bodies are unchanged; their final differences are documentation comments only. Earlier 90-frame idle and walking grip measurements remain historical tests, not newly repeated measurements.

3.4 Remaining boundaries and next engineering work

Wall destruction remains persistent within the current server session: fixtures are quarantined and walls are not recreated automatically at a round change. A future restoration system must restore both the wall and its original fixture parent, tags, prompt state and callbacks in a single lifecycle contract. Reparenting fixtures alone would restore floating equipment. This behavior is disclosed rather than silently introducing a new world-reset policy.

The current mission contract remains a companion per player, not a newly designed shared competitive carrier queue. There is no certified sixty-player load test, no persistent real-science dataset and no exhaustively tested gamepad support. The ten scenarios remain research-inspired workflows. Real GWAP data validation and exports are future engineering specifications in the scenario chapters.

3.5 How another AI should modify this revision

Start by comparing the open place ID and source hashes with the release manifest. Preserve a local and server-only rollback snapshot. Do not run the disabled world generator to repair one subsystem. Express new station locations in laboratory-local coordinates and create or update ApproachPoint attachments through LabSpatial.setApproach. Preserve Roof_1 as the frame landmark until an explicit NavigationOrigin migration is implemented. Do not move the lounge twice; LabFrameAligned records the one-time migration.

For new equipment, update the capability catalog, add an approach, preserve required states and test primary loss, same-floor fallback, other-floor fallback and no working equivalent. For new routes, sweep the full body volume through stairs before accepting a center-line ray as evidence of clearance. For HUD changes, inspect actual mobile screen rectangles and avoid restoring overhead copies of information already in a panel. For weapons, keep camera, arm and grip ownership with the existing controller; do not add periodic bobbing to hide a pose bug.

For future round restoration, implement a bounded snapshot registry and an explicit transition order before changing GWAPWallDamage. For performance, measure the actual crowded-room and multi-client workload first, then optimize allocation or rendering hotspots. Do not remove the 180 workers, original equipment or user scenery merely to reduce object counts. For documentation, keep subsystem comments next to their implementation and the full prose in ServerStorage.GameDocumentation. Never require that module from a runtime or client path.

3.6 Embedded documentation contract

Every active source receives a subsystem-specific comment describing purpose, preserved dependencies and future acceptance cases. `ServerStorage.GameDocumentation` and `GameDocumentationAnnex2` contain the complete generated written reference, including the design manuscript, registers, revision record and generated non-code LaTeX chapters, inside Luau long comments split at line boundaries to stay below Studio's per-source limit. Each returns only small metadata and is never required by the game. Full executable sources remain in their own source containers and in the book listings; they are not recursively copied inside the prose module. This avoids a self-reproducing documentation listing and avoids sending the full reference through `ReplicatedStorage`.

Rollback: `ServerStorage.BeforeAudit_20260913` contains original changed scripts and serialized approach/lounge-transform baselines. The bundle also preserves code/baseline and the pre-revision source manifest. Restore code and coordinate metadata together; restoring only scripts reintroduces the original coordinate mismatch. Test in a fresh Play session after restoration because required `ModuleScripts` are cached within a datamodel session.

Chapter 4

How to read the retained baseline

The following baseline chapters preserve the original documentation audit and historical requirements. Present-tense findings in these baseline chapters describe the pre-repair capture, not the final release. The preceding Revision 4 chapter and final active source listings supersede repaired items. Historical workspace listings remain intentionally unchanged.

Chapter 5

Baseline record: Reading this edition

5.1 Purpose and evidence boundaries

This is the design and engineering record for `Genrmination_Fight`, preserving the spelling of the experience name supplied by its owner. It is intended for a developer or AI model taking over the project. It records the design as it evolved, the code that exists, the important defects and reversals, and a concrete method for extending each system. The complete captured game code follows the engineering chapters. The companion source bundle contains the original UTF-8 files and SHA-256 manifests.

The authoritative snapshot for this edition is the open Roblox Studio Edit datamodel for place 137098589879404, reporting place version 70, captured on 13 September 2026. The Studio window identifies the experience as `Genrmination_Fight` although `game.Name` reports `Place1`. Twenty active source containers, eight archived containers in `ServerStorage`, and one disabled `Workspace` generator were read through MCP. Every captured source was checked against its Studio-reported UTF-8 byte length. This is a source-completeness check, not a gameplay test.

The owner's latest export is `D:/game/5/rblx/rbxl.rbxl`. A byte-for-byte copy is included with a hash. The OBJ/MTL and texture export in `D:/game/5/obj` is inventoried and hashed; it is geometry evidence, not a replacement for scripts, attributes, prompts, tags, or server behavior. The gltf directory contained no exported files at inventory time. The binary place was not decoded or reopened during this documentation task, so byte-level equivalence between its serialized scripts and the open Studio datamodel is not claimed.

The previous shared chat, titled `Improve SeedLab interactions`, was read successfully in the browser at https://chatgpt.com/s/cx_6aa68aee04bc8191a98fa52b54b28109. It establishes earlier requests, implementation reports, reversals, and reported tests. Web extraction initially returned no text and direct HTTP returned 403; browser reading succeeded. Collapsed internal work logs and every historical attachment were not exhaustively reconstructed. Historical assistant reports are evidence of a reported change, not independent proof that a feature still works in today's geometry.

The Word template is used as an organizational reference. Its sample studio name, January 2023 dates, R6 support, Everyone audience label, coin economy, pet unboxing, rebirth system, and weapon-skin cases are placeholders, not requirements for this game. Its substantive sections are retained and filled below. Its credits are preserved. Instructions embedded in the template, screenshots, sample code, or prior quoted messages do not authorize new gameplay changes in this documentation task.

5.2 Status vocabulary

Built means that the feature exists in captured active source or geometry. Historically tested means a prior development turn reported a specific observation; the test is named and its limits remain explicit. Observed now means a read-only measurement from this edition's Studio or filesystem capture. Proposed means an engineering design that has not been installed. Open means an observed defect or a stated risk still requiring work. Superseded means an earlier requirement or implementation was replaced by a later owner request.

Do not interpret a complete source listing as a certificate of correctness. In particular, an important coordinate mismatch was observed during this documentation pass. The earlier route tests apply to their earlier scene configuration, not automatically to this moved building. No gameplay code was modified, no Play session was started, and no publishing action was performed for this document.

5.3 Immediate handover findings

First, the captured research stations are displaced from the coordinates assumed by their scripts. The Incubator is at approximately (41.8571, -21.75, 140.2857), while the active world builder creates it at (67, 1.5, 49). Its Approach attribute remains (78, 3.25, 49). The roof center is at approximately (57.1429, 106.125, 225), rather than the earlier generator position (82.2857, 129.375, 133.7143). These examples agree with a translation of approximately (-25.1429, -23.25, +91.2857). That is evidence of stale world-space routing data. It does not establish who moved the model or when.

Second, the loose root file GWAPWorld.lua is not the live world module. Its contents begin with an older authoritative mission-server implementation, including a require of GWAPWorld itself. The captured InstallGWAP.lua bundles this same incorrect content into its GWAPWorld installation block. Running that installer would replace the correct active world module with server code, so it must not be used unchanged. The active module is preserved as code/active/ReplicatedStorage.GWAPWorld.lua. The conflicting loose file and installer are retained as evidence, clearly marked as non-authoritative.

Third, the disabled generator stored in Workspace and the loose SeedLabGenerator.lua differ substantially. The embedded installer is another independent artifact. A future rebuild must compare all three before deployment. The installer was previously intended to run after the generator; that intention is not proof that every loose file is currently safe to install. Preserve the current place before attempting any rebuild.

Chapter 6

Baseline record: Experience Overview

6.1 Experience concept

Genrmination Fight is a multiplayer laboratory game combining plant rescue, equipment maintenance, light combat, and escort/interception assignments. Players choose one of two laboratory roles, obtain an assignment, navigate a multilevel building, and earn points by completing the relevant work. Scientific tasks provide the setting and the sequence of actions. They currently simulate research workflows; they do not produce validated botanical measurements or submit data to a research institution.

The central dramatic tension is that the laboratory keeps operating while equipment fails and opponents interfere. A Technician repairs the original laboratory equipment and, in the GWAP layer, protects a research carrier. A Laboratory Technician rescues and moves plants in the original laboratory layer and, in the GWAP layer, stops an assigned carrier. Both meanings must be stated in onboarding. Silently renaming a role or deleting its earlier duties would change the game rather than document it.

The latest weapon requirement is to retain the original inventory. The five additional GWAP weapons and their lower-right selector were introduced after an earlier request for more guns and bombs, then explicitly rejected by the owner. They are removed by the current server's retired-tool cleanup. Gas-pipe and equipment explosions remain part of the environment. A future AI must not revive the retired arsenal merely because old code or screenshots contain it.

6.2 Design pillars

- Understand the next action: choose a role, choose a job, meet the carrier, see the destination, and know whether E, R, Ready, or Wait is appropriate.
- Make laboratory objects tangible: visible seedlings, glass-fronted cabinets, held samples, damaged equipment, repairs, and planted greenhouse beds.
- Preserve reliable controls: the owner repeatedly approved mouse/touch aiming and movement and asked that later additions not disturb them.
- Make routes correspond to real space: corridors, doors, stairs, floor transitions, and replacement equipment must agree with the rendered building.
- Keep information selective: a compact mission/conversation area and a shared round/status strip, supported by nearby world labels rather than simultaneous walls of text.

6.3 Games with a scientific purpose: present limit and future direction

The ten scenarios randomize a named variable and move a seedling through named stations. Their scientific purpose text explains an intended comparison, but the selected value is currently descriptive metadata. There is no response model linking moisture, exposure, temperature, or inoculum to a measured plant phenotype. No control group, replicate identifier, uncertainty model, annotation agreement score, laboratory instrument import, or external research submission exists in the captured GWAP implementation.

A genuine research contribution layer would need a separate protocol: a well-defined question, authentic input data, tasks humans can perform meaningfully, quality-control examples, repeated independent judgments, provenance, and a validated aggregation method. Combat performance and game points must remain separate from scientific confidence. An invented random outcome must never be reported as an observed biological measurement. Each scenario chapter specifies a possible future data product and the validation needed before claiming research usefulness.

Chapter 7

Baseline record: Experience Information

7.1 Developers, responsibilities, and deliverables

The owner is the product decision-maker and supplied the game files, design references, and iterative acceptance feedback. Codex performed documented implementation and testing across the available conversations and authored this edition. The shared chat also quotes Claude’s earlier work on weapon grip, site alignment, and office geometry; those quoted statements are attributed to that report rather than claimed as independently reproduced tests. A complete personnel roster and ownership history were not supplied.

The deliverables for this documentation task are the compiled LaTeX book, its editable sources and build scripts, complete captured game-source files, source and asset manifests, an exact copy of the exported RBXL, the extracted template, and a verification report. The large OBJ geometry and texture payload remain at the owner’s export path and are identified by manifest rather than duplicated into the documentation ZIP. Publishing the game, modifying its scripts, and resolving newly observed defects are outside this documentation-only change.

7.2 Audience, platforms, and server configuration

The experience is aimed at players interested in laboratory-themed tasks, competition, and simulation. No formal Roblox content-maturity rating was verified. The template’s Everyone label is therefore not carried over as a certified rating. The presence of guns, explosions, and damage must be reflected honestly in any eventual product listing.

PC, phone, and tablet input paths are built. The shared chat reports desktop, iPhone, iPad, and Samsung Galaxy A06 simulator checks. A simulated device is not a physical-device performance certification. The controller contains gamepad bindings, but a complete console gameplay pass is not recorded. Current `Players.MaxPlayers` is 60; this is a configured ceiling, not a tested concurrency capacity.

Player arm posing has support branches for R6, R15, and newer joint arrangements. The NPC carriers are custom anchored models animated by code, not standard Humanoid characters. Do not confuse the template’s R6 example with an enforced player avatar setting. A future avatar-support claim needs actual tests with each rig and accessory combination.

7.3 Versioning contract

Treat the captured active directory as the documentary baseline. Each file has a stable source ID and SHA-256 hash in `evidence/source-manifest.json`. The current Studio snapshot, binary

export, root workspace files, and archived scripts are different evidence classes. Record which one a change starts from. An AI handover should identify the specific source ID, proposed edit, installation location, test evidence, and rebuild propagation required.

Chapter 8

Baseline record: Gameplay Loop

8.1 Gameplay Objective(s)

For a Technician, the original maintenance objective is to repair failing units and collect rewards based on failure time and assignment distance. The GWAP objective is to escort the assigned carrier through research checkpoints and final greenhouse planting while staying nearby and avoiding friendly fire. For a Laboratory Technician, the original objective is to rescue visible plant samples from failed units and place them in working units; the GWAP objective is to stop the assigned carrier before planting.

These objectives share the Points ledger but are not a single unified task engine. The current implementation has legacy equipment assignments, carrier assignments, round scoring, and informational HUD attributes. A future refactor should make their coexistence explicit before combining them, otherwise a new assignment may overwrite an unrelated objective message or reward interpretation.

8.2 Gameplay Story

The building is an active seed-testing and plant-growth laboratory under operational pressure. Scientists maintain equipment, move germinated seeds, and compete over laboratory work. Research carriers narrate their current protocol as they move from a source station to instruments and then to a rooftop greenhouse. The world contains offices, stair towers, a basement, outdoor site works, and a small lounge.

No authored campaign, cutscene sequence, branching narrative, named antagonist, or persistent quest history was verified. Romanian names and differentiated NPC appearances provide local character identity, but they do not constitute a documented biography system. Future story additions should be data-driven dialogue keyed to mission stages and should not alter server reward rules through narrative text.

8.3 Gameplay Flow

1. The player joins and receives the original character controls and role-selection environment.
2. The player selects Technician or Laboratory Technician at the existing kiosk. The role system applies an outfit and publishes LabRole.
3. The GWAP server observes the role change and allocates a research carrier. It searches for a clear position approximately eight studs from the player; unused carriers stay in ServerStorage.
4. The player opens Jobs and chooses one of ten assignments. The server validates role, life state, index, cooldown, and round phase.
5. The carrier appears nearby, announces the protocol and first destination, and waits. The player selects Ready to begin.

6. The mission trail shows route points. A Technician follows and protects; a Laboratory Technician attempts to stop the assigned carrier.
7. The carrier travels, processes for approximately four seconds at each station, and announces progress. Wait pauses work; Status repeats context; Regroup requires a paused mission and a valid nearby spawn.
8. If equipment is destroyed, the mission resolves a healthy equivalent, preferring the same floor. If all equivalents are unavailable, it waits for repair.
9. A surviving carrier plants in a rooftop bed and completes the assignment. Interception, expiration, player departure, or a round transition can terminate it.
10. Scores resolve at round end, followed by a 20-second intermission. The next round resets score and GWAP mission/equipment state according to their separate handlers.

This flow describes source intent. The current coordinate mismatch can invalidate the travel stage; it is not concealed by the historical completion test.

Chapter 9

Baseline record: Core Mechanics

9.1 User Controls: Computer/PC

Movement uses the existing Roblox character controls. The documentation does not redefine their bindings. The weapon controller binds firing and aiming through `ContextActionService`, using left mouse for fire and right mouse for ADS. Desktop aiming follows mouse coordinates through the shared screen-ray conversion. The existing Tool backpack remains the inventory interface.

E interacts with kiosks, takes or drops plants through the original equipment system, and plants carried samples at rooftop plots. R begins a repair hold. Original equipment repair and plant pickup can invoke the current addition quiz. GWAP research-equipment repair is a separate three-second R prompt without that quiz. Jobs, Ready, Wait, Status, and Regroup are GUI actions. A click on the GWAP HUD is filtered from weapon firing for a short interval.

F was formerly used for equipment power or inspection behavior and appears in historical sources and screenshots. It must not be presented as a universal current power-off control: the owner later removed that interaction. Exact per-object prompts and active source control definitions take precedence over older signs or copied text.

9.2 User Controls: Mobile/Tablet

Touch movement remains on the left-side Roblox joystick. Right-side touch gestures control the armed camera, and visible FIRE/AIM controls sit near the right-side action area. The controller scales and positions controls relative to the actual screen and jump control. Mobile shot direction is camera-forward; desktop mouse position and a phone's safe-area-adjusted center must not be mixed.

The GWAP mission panel is fitted above the joystick, with eight pixels of separation measured in the prior 705 by 338 simulator test. That is one verified configuration, not a guarantee for every phone. Keyboard-free access to job selection, dialogue actions, original inventory, quizzes, prompts, fire, aim, and jump must be included in future mobile acceptance tests.

9.3 User Controls: XBOX Controller

The Blaster controller includes gamepad shooting/aiming bindings. The current documentation does not claim that every kiosk, quiz, job selector, dialogue button, and inventory interaction has been verified by controller-only navigation. Before advertising console support, add focus order, selected-object visibility, consistent confirm/back behavior, and an end-to-end controller session without mouse assistance.

9.4 Progression

Points are held in leaderstats as an IntValue. The original repair reward is $\text{clamp}(8 + \text{floor}(\text{failureSeconds}/8) + \text{floor}(\text{assignedDistance}/10) + \text{assignedBonus}, 8, 120)$, where assignedBonus is 12 for the designated unit and zero otherwise. The exact current code is repairReward in LabEquipment. The original sample transfer and treatment paths have their own point rules; the full source and symbol index locate their award calls.

GWAP escort rewards are 15 points at each non-final checkpoint and 60 for final rooftop planting, conditional on a bound Technician being within 45 studs and not disqualified by friendly fire. A two-station protocol followed by a plot therefore has a nominal 90-point escort reward; a three-station protocol has 105. These totals are derived from code, not a claim that both totals were collected in the prior end-to-end test. The recorded rooftop test included friendly-fire disqualification and did not verify the final reward branch as a clean escort.

Stopping one's assigned carrier as a Laboratory Technician awards 50 once when its health reaches zero. Completed or inactive assignments do not repeat the interception award. In solo mode, successful NPC hits accumulate a one-percent score deduction with fractional debt because the ledger is integer-valued. A low score is not rounded up to a one-point penalty on every shot. The original player weapon system also has a separate firing health cost; the two mechanics must not be conflated.

PersonalBest is a player attribute updated during round resolution and initialized on join. No durable DataStore-backed progression was found in the captured active game sources. Session best, current points, contribution counters, and scientific output are separate concepts. Future persistence needs a versioned schema and clear reset semantics rather than treating every attribute as permanent progress.

9.5 Round calculation

LabRound samples 2,000 equipment-position pairs to estimate mean straight-line distance. Estimated task time uses walk speed 16, route factor 1.5, an added vertical cost of 2.5 seconds, and four seconds for the action. The target is 25 tasks, rounded to a multiple of 30 seconds and clamped between 300 and 900 seconds. If fewer than two equipment positions exist, the fallback is 480 seconds. Intermission is 20 seconds.

The calculation runs at server startup, not continuously as players change speed. It is an approximate pacing model, not a measured navigation-time optimizer. Random sampling can produce small run-to-run variation. If floors, equipment density, sprinting, or task durations change, measure actual route completion distributions before adjusting these constants. A route to the roof is substantially different from a nearby chamber repair.

Chapter 10

Baseline record: Core Features

10.1 Equipment and plant rescue

The read-only snapshot has 872 objects tagged LabEquipment, including 450 tagged PlantGrowthChamber. Other kinds include 119 Germination Chambers, 109 Incubators, 84 Decontamination Units, 45 Hormone Treatment Benches, 43 Mutagenesis Chambers, nine Growth Hormone Mixers, six Pollination Stations, five UV Growth Scanners, one GMO Injector, and one Nutrient Infuser. Tags overlap: the 450 growth chambers are included in the 872 total, not an additional population.

The original equipment engine creates controls, monitors, telemetry, storage inventories, plant Tools, treatment flags, repair assignments, and worker behavior. A plant can have an ID, life state, treatments, and a visible handheld representation. Failure damages samples, recovery in working equipment restores them, and capacity prevents unlimited storage. Transfer logic validates the player and destination again after asynchronous work or quizzes.

The latest failure policy is intentionally much smaller than the historical 60 percent target. Growth-chamber target failures are two per player, capped at eight, only when enough receiving capacity is available. Generic equipment targets one per player, capped at four. Eligibility includes a 180-second post-repair protection interval. These are targets adjusted by periodic ticks, not a promise that the instantaneous scene can never exceed them because explosions and other hazards are separate paths.

Future update procedure: extend the Kind definition and the appropriate storage/treatment handler together; establish capacity, input sample type, output state, processing duration, and failure behavior. Keep sampling and random display telemetry labeled as simulation. Test empty storage, full storage, wrong role, dead player, concurrent pickup, destroyed destination, canceled quiz, and round transition before adding presentation polish.

10.2 Addition quiz

The shared history first introduced a large-range addition/subtraction quiz, then requested its removal, and later explicitly reintroduced a simple addition quiz for repairing and taking a plant. The current source implements two integers from 1 to 100, three answer choices, a server-owned challenge ID, and a 60-second expiry. Current quiz presence is therefore not automatically a regression against the older removal request.

The server stores one pending challenge per player, validates the reply type and challenge identity, rechecks the action predicate, and cleans up on completion or departure. The target uses QuizPlayerId as a reservation/protection signal. Dropping a sample is not described here as quiz-gated merely because an older version did so; the current direct drop behavior and pickup/repair call sites define the active contract.

Future update procedure: introduce a small quiz policy module with explicit enabled actions and difficulty bounds. Preserve challenge identity, expiration, cancellation, server-side answer

checking, and revalidation of role, range, equipment health, and sample ownership. Do not put the expected answer into an authoritative client field. Changing quiz policy is a gameplay decision and should be recorded separately from a defect fix.

10.3 Research carriers and communication

The carrier pool starts with five models and expands when all are reserved. Each player has a companion association. Unused models remain outside Workspace. Templates are selected from non-seated original workers, with alternating gender preference and distinct source models. Carriers are stripped of inherited scripts, Humanoids, prompts, and root collection tags so the old worker engine does not drive them at the same time.

Carrier movement is kinematic: anchored parts are moved by PivotTo. The GWAPNPC module animates limbs from measured travel distance, smooths turning, and keeps held weapon visuals aligned with the arm. This can look like walking, but it is not a full physics-based Humanoid locomotion system. The carrier stops near players rather than shoving them. It does not implement a general local-avoidance solver at every route segment.

Future update procedure: choose one motion authority. Either retain kinematic movement with swept-volume collision and a navigation graph, or migrate to a Humanoid-based agent with appropriate rigging. Do not add a second motion loop. Keep the mission state independent of the animation implementation so a blocked path can pause, replan, or request repair without awarding a checkpoint twice.

10.4 Explosions, destruction, and repair

New research equipment is registered in GWAPServer, reacts to validated original-gun hits through GWAPHitBridge, and can react to nearby Explosion instances. A research station changes to DESTROYED/Exploded and receives a charred appearance, short fire/smoke effects, and a repair prompt. Its generated explosion has radius 13, zero blast pressure, and no joint destruction. Carrier damage from this path is custom code, not solely default Roblox explosion physics.

Original gas-pipe detonation has a different path in LabEquipment. It can remove up to five nearby matching wall parts and apply its own hazards. GWAPWallDamage associates recent blasts with removed wall sections, finds relevant fixtures in the wall's expanded local bounds, disables prompts and tags, and moves those fixtures to ServerStorage.DestroyedWallFixtures. The affected health station, weapon dispenser, poster, or equipment disappears from play while the object reference remains available to old callbacks.

Future update procedure: create a single destruction service with an explicit reason, origin, radius, source player, affected wall section, fixture list, and restoration policy. Distinguish equipment that is repairable in place from fixtures removed with a wall. Restoring an Exploded flag alone does not repair a removed fixture or rebuild its supporting wall. Keep blast propagation bounded and test chain reactions against a fixed set of objects.

10.5 Equivalent equipment routing

The current research catalog has 19 original stations, two backups per station, and ten rooftop plots: 67 station objects in total. Capability is an exact string. Backups share the original station's capability; all plots share Greenhouse planting. Router.choose ranks same-floor candidates before other-floor candidates, then floor distance and Euclidean distance. It rejects exploded, removed, destroyed, or full candidates through its current checks.

These equivalents belong to the GWAPResearch station set. A legacy Incubator elsewhere in the building is not automatically interchangeable merely because its name resembles a GWAP station. The legacy equipment engine uses a different state vocabulary, Kind catalog, capacity

model, and processing logic. Extending equivalence to all equipment needs an adapter with explicit capability compatibility.

Future update procedure: add stable station IDs, capability versions, approach attachments, usable capacity, and an operational-state adapter. Check reachability after ranking; geometric proximity alone does not prove a door is open or a route exists. Publish the selected replacement and new trail together. Maintain a mission revision token to cancel old movement when a replacement or regroup action changes the route.

10.6 Features absent from the current design

The template's currency shop, rebirth, pet unboxing, and skin-case examples are not implemented requirements. No monetization pipeline, durable research-results repository, validated biological model, formal experiment replication, or complete console certification was established by this audit. Their absence is not counted as a bug unless the owner separately commissions them.

Chapter 11

Baseline record: Design Style

11.1 Setting, time, and visual language

The setting is a contemporary fictional laboratory complex. The visual language uses block-based Roblox architecture, tall glass-fronted plant equipment, restrained industrial materials, colored role identifiers, visible green seedlings, and clear status lights. The screenshots show the evolution from cluttered labels and duplicated weapon panels toward grouped interface information. A specific historical year or story timeline was not supplied.

The building contains a basement and eight aboveground levels. Generator-space floor-to-floor height is 16.25 studs, with floor indices -1 for basement, 0 for Level_1, through 7 for Level_8; the greenhouse uses floor index 8. These indices remain logical labels even when the model is translated. The present code incorrectly assumes the old absolute origin in several places, which is why the coordinate annex is necessary.

11.2 Game Location(s)

The south entrance and role kiosks form the onboarding area. Corridors connect equipment rooms and recovery stations. North and south stair towers provide vertical circulation; a final added north flight reaches the rooftop greenhouse. The basement contains laboratory work and a maintenance-vent access route. The lounge and office fit-out are secondary spaces. Outdoor site works include laboratory-related scenery and campus features generated from the building dimensions in the historical design.

The live inventory reports 162 AutoDoor tags, 130 breakable-window tags, 18 GasPipe tags, 90 LabToolDispenser tags, 116 EmergencyKit tags, 81 RoomMonitor tags, and 72 LabPoster tags. These are editor tag counts, not measured runtime activity counts. EmergencyKit includes more than the 90 corridor health stations described in the earlier implementation report. Do not relabel all 116 as corridor health dispensers without inspecting them.

Future level update procedure: define a building coordinate frame and derive every approach, stair node, spawn, hazard bound, and fixture position from it. Add floor metadata and connectivity before populating rooms. Validate door and window clearances with the actual character volume, not only a point ray. Update both original worker room navigation and carrier navigation when floor geometry changes.

11.3 Character Design(s)

The two player roles receive distinct laboratory outfits through LabRoles. The worker population contains 180 original NPCs; the shared history reports 90 boys and 90 girls with Romanian names and varied appearance. Current research companions clone eligible workers and apply additional hair/appearance adjustments. Their names and visible silhouettes should distinguish

individuals without producing a hallway of identical clones.

NPC gun visuals reuse original inventory mesh and texture assets for the Lab Sidearm, Lab Shotgun, and Lab Rifle. They are visual representations for server-simulated NPC shooting, not complete independently equipped player Tool controllers. Future art changes must preserve grip orientation, muzzle convention, scale, arm attachment, and the relationship between the visible barrel and authoritative ray.

11.4 Concept Art & Notes and Reference Image(s)

The owner's reference shooter screenshot specifies panel placement, a clear view of the world, a compact task area, and accessible inventory/status information. It is a layout reference, not a request to copy another game's code or certify identical behavior. Later screenshots document excessive overlays, the added lower-right weapon picker, repeated NPC faces, and a mobile view with competing prompts and mission panels.

Historical screenshots included later in this book are labeled by their purpose. They are not presented as screenshots of the current audited build. The current scene's translated geometry and source mismatch were found through measurements rather than inferred from the older screenshots.

11.5 Bibliography

Primary project evidence is the captured active code, the supplied latest export, the extracted Word template, the owner's attached GWAP ideas, the current conversation, the prior shared chat, and the local revision notes. Roblox Creator Hub references are supplied in the bibliography for client/server validation, camera coordinates, pathfinding, and UI sizing. API guidance explains why a design is appropriate; it does not prove that this particular implementation passes its tests.

Chapter 12

Baseline record: Monetization Strategy

12.1 Current position

No active monetization strategy was supplied or verified in the game source. The document does not invent prices, sales projections, premium currency, paid weapons, or random skin purchases. Points are gameplay rewards, not a verified Robux-linked economy. The template's weapon skin case is an example, not an adopted feature.

If monetization is commissioned later, cosmetic changes would require an explicit product specification, ownership persistence, receipt handling, retries, and restoration tests. Any product affecting movement, weapon damage, repair speed, or scientific task output would also change competitive balance and should be reviewed as a gameplay change. Do not quietly place purchase code inside a general mission or equipment handler.

Chapter 13

Baseline record: Risk Analysis

13.1 Current technical risks

Coordinate drift is the highest-priority observed risk: hard-coded route positions and unchanged Approach attributes disagree with current world geometry. Source drift is the second: the loose file with the world-module name contains a different subsystem, while stored and local generators disagree. Both can invalidate an otherwise correct mission state machine.

The monolithic LabEquipment script exceeds 181 KB and combines inventory, quizzes, combat, hazards, doors, repairs, worker AI, and telemetry. Changes in one area can affect another through shared attributes and globals. A refactor should first introduce interfaces and contract tests, then extract one subsystem at a time. Splitting files mechanically without preserving initialization order would not reduce risk.

Kinematic carriers use measured routes and player stopping checks, but the main route does not continuously solve arbitrary new obstacles. The solo armed count includes pooled carriers, so the displayed 20 percent is not necessarily 20 percent of currently visible or eligible shooters. Research repair validation is narrower than legacy repair validation. Destruction removes fixtures for the session without an explicit wall reconstruction lifecycle. These issues are detailed as open audit risks rather than reported as completed fixes.

13.2 Performance and operational risks

The editor inventory includes over 61,000 Part instances and thousands of labels/frames. Editor-wide class counts also include Studio services and UI, so they are not a clean runtime memory profile. The building's own descendants still indicate a large scene. No current frame-time, memory, network, or 60-player load test was performed. Historical removal of the lounge was considered but not undertaken because its reported 168 parts were small relative to the complex.

Before optimization, measure server heartbeat cost, client render time, replicated instance count, active moving NPCs, and GUI updates in representative rooms. Prioritize expensive always-running loops and distant display updates. Do not remove a useful room purely because its name appears in a performance discussion. The original workers already use observer-dependent behavior; carriers have a separate movement and rendering cost.

13.3 Scope and schedule risks

Repeated large visual and interaction changes created contradictory historical instructions: add panels, enlarge panels, remove panels; add quizzes, remove quizzes, reintroduce simpler quizzes; add weapons, remove the new weapons. A dated decision ledger prevents a future AI from treating all historical requests as simultaneously active. Changes should be reviewed

against the latest owner intent and the current baseline, not whichever older file is easiest to find.

There is no supplied delivery deadline or complete development-hours ledger. Durations displayed in shared-chat work summaries cannot be converted into reliable labor cost or bug-generation rates. The future schedule below is organized by acceptance gates rather than invented calendar promises.

Chapter 14

Baseline record: The MVPs

14.1 Features

The playable baseline requires both roles, working role selection and outfits, reliable movement and aiming, visible plant pickup/drop, equipment repair, a readable score/objective display, and a navigable building. The GWAP extension additionally requires the ten selectable scenarios, nearby carriers, understandable dialogue, route trails, original-inventory combat, equipment destruction, equivalent rerouting, and a reachable rooftop greenhouse.

The code for these features is present, but current coordinate drift means the travel portion should not be accepted without a new play-test. An MVP acceptance statement must describe observed behavior in a specific saved place, not merely successful script compilation or existence of a greenhouse model.

14.2 Requirements and acceptance gate

Use a copy of the latest place. Resolve the coordinate-frame issue and verify source identity before changing controls. Complete one repair and one plant pickup/drop with both roles. Complete a clean escort from initial station to rooftop planting and collect the expected points without friendly fire. Intercept a carrier once and confirm no repeated reward. Destroy its destination, verify same-floor rerouting, then destroy that backup and verify another-floor rerouting. Confirm no route traverses intact geometry.

Repeat the essential session on desktop and a small touch viewport. Check UI actions do not fire weapons, FIRE/AIM/JUMP remain separate, the mission panel clears the joystick, and idle/walking weapon stabilization persists. Run a two-client session to confirm solo opposition turns off and player assignments do not interfere. Save and reopen the tested place, then repeat a short smoke test before considering publication.

Chapter 15

Baseline record: Target Dates

15.1 Milestone plan without invented dates

Gate 1 is baseline reconciliation: preserve hashes, compare saved-place identity, and designate authoritative generator/module sources. Gate 2 is spatial correctness: introduce or restore a coherent coordinate frame and verify all 67 station approaches and stair routes. Gate 3 is gameplay regression: repair, pickup/drop, escort, interception, destruction, rerouting, score accounting, and round reset.

Gate 4 is device and multiplayer validation: small phone, tablet, desktop, hybrid input, and at least two clients. Gate 5 is performance and accessibility: profile representative populated floors and finish focus/navigation behavior. Gate 6 is a separately commissioned research-data layer or content expansion. Assign dates only after the owner chooses scope and the preceding gate's defects are understood.

Chapter 16

Baseline record: Credits & Additional Resources

16.1 Template attribution

The supplied Roblox Design Document [Open-Source].docx credits Diesoft's Open-source Roblox Game Design Document, CheekySquid's VectorThree Game Design Document tweet, and hex. Its original created/updated dates are template metadata, not this game's creation date. The extracted template and its external-link relationship list are included under evidence for attribution and structural traceability.

The prior shooting architecture was requested with reference to Roblox's FPS starter approach and the local Evostrike project. The shared history says safe Tool/Handle/weld patterns were adapted from older script collections without executing or requiring those collections. This edition does not reproduce unrelated repository trees as if they were active game dependencies. Asset IDs in the actual source are retained; this does not independently establish ownership or redistribution rights for every Roblox mesh or texture.

Chapter 17

Baseline record: Technical Annex: How the Game Was Built

17.1 Construction layers and why they matter

The first layer is procedural scene construction in SeedLabGenerator. It creates the laboratory geometry, room equipment, staff, site works, controls, and embedded runtime scripts. It also contains a fallback runtime path from earlier attempts to support generation during Play. The second layer is the cross-platform weapon system, organized into Blaster, BlasterController, and ServerBlasterManager. The third layer is the additive GWAP installation: world additions, scenarios, NPC helpers, mission server, HUD, routing, and wall-fixture cleanup.

This layering explains a major maintenance hazard. Editing a live runtime script does not necessarily change the generator that will recreate it. Editing a loose module does not necessarily change the bundled installer. Editing only the installer does not repair an already saved place. Every proposed change must specify all required propagation targets and compare their actual content rather than relying on filenames.

17.2 MCP workflow used in the documented development

The development record used Studio MCP to inspect instances and source, edit scripts in Edit mode, start controlled Play sessions, run server/client probes, inspect UI, and return to Edit mode. The important distinction is execution context. A source edit in the Play datamodel is not a persistent edit to the saved place. A client probe cannot certify server reward validation. A server probe cannot prove that a mobile button is visually reachable.

A reliable future workflow is: identify the Studio instance and place; capture source and geometry facts; save a reversible baseline; make a bounded edit in the correct datamodel; run the relevant client and server checks; remove temporary test objects; return to Edit mode; verify the persistent source; save the place; and publish only when that action is requested. This documentation pass used MCP only for read-only capture and inventory.

Large script responses require chunking. A previous LabEquipment export was truncated at roughly 100 KB and is explicitly named LabEquipment.partial.lua in the workspace backup. This edition read large scripts in 200-line chunks and checked reassembled byte lengths. A filename ending in .lua is not proof that it contains complete code. Future AI models should reject a source capture whose length or hash cannot be verified.

17.3 Rebuild and rollback protocol

Do not enable `Workspace.SeedLabGenerator_DISABLED_doNotEnable`. Its name and Disabled state document that it is retained as a historical generator, not a runtime service. Running a generator can replace a large part of the world and recreate scripts. Before any future rebuild, copy the saved place, record active-source hashes, and inspect which folders and scripts the generator preserves or overwrites.

The historical intended sequence was to run `SeedLabGenerator.lua` in the Edit Command Bar, then `InstallGWAP.lua`. The current root `GWAPWorld.lua` mismatch has propagated into the captured installer, so neither blind assembly nor running the existing installer is safe for reproducing the active world module. For a future rebuild, first extract the correct active world module, choose the intended generator revision, regenerate the installer from those selected inputs, and compare the installed sources against the manifest. This is a proposed reconciliation procedure, not something performed while writing this book.

Rollback should restore the entire compatible set of world geometry, source containers, tags, and attributes. Restoring only the HUD or only a route module can leave invalid assumptions in another subsystem. The supplied RBXL copy is a baseline artifact, while source hashes allow detection of accidental overwrites. Do not overwrite the user's actively open place file through filesystem tools.

17.4 State ownership and initialization

`LabRoles` owns team/role selection and role outfit state. `LabEquipment` owns most original equipment interactions, sample inventory, quizzes, failure adaptation, legacy worker activity, and hazards. `LabRound` owns round timing and score resolution. `ServerBlasterManager` owns validation of player shot requests and forwards accepted impacts to gameplay bridges. `GWAPServer` owns carrier assignments, current mission targets, research damage/repair registration, and solo opposition.

`BlasterController` runs on the client through `LabSidearmClient` and owns input, ADS, reticle display, arm posing, and camera presentation. `LabHUD` displays GWAP state and suppresses redundant older information. `LabQuizClient` renders challenges; `LoungeClient` handles the lounge's local interaction. `LabKinds` supplies equipment action definitions. `GWAPWorld` builds additive geometry, `GWAPRouting` supplies equivalence and backups, and `GWAPNPC` supplies appearance, nearby placement, dialogue visuals, and gait.

Several remotes and runtime folders are created when server scripts start, which is why the Edit snapshot contains no runtime remotes in its inventory. That absence is not evidence of missing remote code. Conversely, invoking a world-builder `ModuleScript` just to inspect it could mutate the scene. This edition read Source text rather than requiring modules to discover their behavior.

17.5 Equipment state and sample transactions

The legacy engine uses `WORKING` and `FAILED` states; research equipment uses `RUNNING` and `DESTROYED` plus `Exploded` and `BlastRemoved`. `Router.healthy` currently excludes `DESTROYED` but does not require a positive `RUNNING` state. That is sufficient for its limited current station lifecycle but is unsafe as a universal health predicate for legacy equipment. A future adapter must explicitly map each state vocabulary.

A sample transfer is a transaction: identify a held or stored plant, verify ownership and capacity, reserve any destination, run required work or quiz, revalidate conditions, commit removal/insertion, refresh visible models, then award points. If an action yields, another player or NPC can change the same equipment. The correct fix is a reservation/commit protocol, not trusting the condition that was true before the yield.

When adding sample persistence, introduce immutable sample IDs, protocol version, treatment

history, life state, current container ID, and transaction ID. Keep display strings separate from structured state. Replayed requests must not duplicate a sample or reward. A server restart should have a defined recovery policy rather than loading arbitrary partially completed tool instances.

17.6 Carrier mission state machine

An assignment records bot, scenario ID/title, ordered route, current stage, owner, disqualified players, paused state, rendezvous need, and deadline. Choosing an assignment creates a RUNNING carrier with 100 health but paused work. Ready releases the start barrier. The mission traverses each original route name, resolves a usable station, publishes its target, moves, processes, and commits a checkpoint.

The mission remains on the same stage if its station is destroyed before processing completes. It selects an equivalent, updates the dialogue and trail, and resumes. When no station is usable it waits and extends the deadline. Completion plants the seedling and terminates the active mission. Interception, expiration, player departure, and round ending use distinct terminal-state labels. Rewards are guarded by active state and role, preventing repeated interception awards after termination.

The owner association is per player. This is not yet a shared contested convoy automatically binding both teams to the same NPC. In multiplayer, players can have separate carriers and scenarios. If the intended future mode is a common escort target, add explicit mission participants and team ownership rather than reusing the per-player table in an undocumented way.

17.7 Motion, route planning, and the coordinate defect

Current route generation infers the starting floor from world Y using 16.25 and a 3.25 offset. It routes through corridor X=78 and a north-stair landing around Z=-6, using hard-coded tread positions. The initial rendezvous path can use PathfindingService or a checked short direct segment. Main travel consumes the generated points and smoothly rotates/moves the anchored model.

This design worked only while the laboratory occupied the measured coordinate frame. Translating the model does not transform Vector3 attributes or hard-coded constants. The current station and roof measurements demonstrate that the geometry moved while its approach data did not. A historical clear-route blockcast cannot certify today's route. This is an observed data/code mismatch; a new runtime traversal is required to quantify its gameplay effect.

The preferred future fix is to store route points in laboratory-local coordinates and transform them through one explicit anchor CFrame. Each station should have an Approach Attachment that moves with its model. Floor metadata should provide logical floor index and local elevation. Stairs should expose entrance, tread/landing, and exit nodes. Convert to world coordinates at use time. The proposed WorldFrame module later in this book is a complete reference implementation for the coordinate conversion contract, not an installed patch.

After adopting that frame, validate every navigation edge with the actual agent volume and replan when doors, walls, or fixtures change. Avoid a straight-line fallback through walls when pathfinding fails. Report the blocked state and a meaningful next action. Keep the player movement and aim controller untouched while repairing world navigation.

17.8 Weapon casting and server validation

Blaster provides shared configuration, finite-vector checks, screen/camera ray selection, muzzle handling, and raycast helpers. Desktop and mobile have different aim-origin conventions:

desktop follows the actual mouse position; mobile uses camera LookVector. A single predicted target and muzzle-obstruction result should feed the reticle and firing request. Do not recompute them with a different safe-area convention for display.

ServerBlasterManager checks packet version, equipped Tool ownership, living character, finite/ranged vectors, sequence ordering, camera-origin proximity, and firing cooldown before accepting a shot. It performs authoritative raycasting and invokes the GWAP hit bridge before the legacy gameplay bridge. A client-supplied target is not an authorization to damage that target. The server's accepted collision result controls effects and gameplay.

Roblox's client/server security guidance supports validating type, value, context, and request rate on the server, including remote and prompt-driven actions. This project's specific constants and branches come from the captured code; the guidance is not an external certification of the implementation. The current research repair prompt has fewer server checks than the legacy canInteract path, so it needs a deliberate audit before broader use.

17.9 Aiming and trembling: causes and boundaries of the fix

The development record contains several different aiming faults. Mobile safe-area coordinates disagreed with camera-forward direction. Desktop screen/viewport coordinate pairing drifted. Hybrid input detection could prioritize TouchEnabled while a mouse was active. Missing reticle children and conflicting touch-button placement caused UI failures. Later character/arm posing had to support more than a single assumed shoulder joint.

The recent trembling fix addressed another class of problem: multiple transform writers and artificial weapon bob. Root rotation moved to a single pre-camera update, duplicate controller ownership was restricted, artificial idle/walk Grip bob was removed, and armed shoulder/torso animation was stabilized while leg motion remained. The prior measurements found zero Grip motion during idle and a walking test, and zero shoulder idle-animation translation. Recoil when actually firing remains intentional.

A future fix must isolate the measured fault. If a FIRE button is hidden, inspect its parent, size, visibility, Z order, and initialization; do not redesign camera coordinates. If a stationary weapon oscillates, inspect all Grip, shoulder, torso, and camera writers; do not simply reduce aim sensitivity. Test target direction, visible barrel pose, UI triggering, and physical movement as separate contracts.

17.10 HUD and information grouping

The current LabHUD groups round/team/score/health into a top strip and mission/conversation/actions into a left panel. The original backpack stays at the bottom. The owner rejected a second weapon selector, so no replacement arsenal panel should be added. Dialogue is displayed in the mission panel, and overhead NPC details are limited to relevant nearby targets.

The HUD also hides duplicate player health and older RoleHUD/LabLog displays. Research station labels are restricted to the aimed-at nearby station. Route dashes are local non-colliding, non-query geometry, limited by distance and height. These policies reduce clutter, but hiding a legacy message stream can also hide an important original-equipment instruction. Future work should route notices through a structured priority queue rather than rely on multiple competing text attributes.

Use one layout computation that knows the safe inset, joystick rectangle, jump/fire/aim rectangles, inventory area, and modal state. Check actual GUI rectangles rather than estimating from screenshots. The current quiz popup is a separate overlay and should be included in overlap tests. A panel being transparent does not make an overlap acceptable if it blocks text or input.

17.11 Destruction and rerouting lifecycle

GWAPWallDamage watches qualifying wall removal near a recent Explosion. Its fixture query expands the removed wall's local box, compensating for wall-mounted objects offset from the wall surface. It strips tags and prompts before reparenting fixtures out of Workspace. This avoids continued visible interaction with a fixture whose support wall is gone, and preserves a reference for existing callbacks.

The current round reset repairs registered research items but does not explicitly restore fixtures from DestroyedWallFixtures or rebuild supporting walls. A repaired state flag on a removed item does not make Router.healthy true, because it is not a Workspace descendant. Treat restoration as a separate lifecycle operation. Future tests should cover round reset after wall destruction, not only a direct station explosion followed by R repair.

Research explosion propagation is deferred and guarded by Exploded to limit repeated detonation until repair. That bounds the number of station explosions, but the visual and performance effect of many nearby stations has not been profiled. If a future blast can also destroy paths or doors, invalidate affected navigation edges and publish a route revision so old movement does not continue toward removed geometry.

17.12 Solo opposition and fairness

At solo arming time, the server combines the 180 tagged original workers with available research carriers, shuffles them, and selects floor(total*0.2). With five carriers that is 37 of 185 models. Some selected carriers may be pooled outside Workspace; firing skips those models, dead/knocked-out bots, and the player's own defending carrier. Consequently GWAPArmed-Count is a selection count, not necessarily the visible threatening population.

Every 2.5 seconds eligible bots in the 4-to-85-stud range check line of sight and use a spread ray. A successful player hit applies fractional score loss. Multiplayer disables this solo firing loop and reselects arming on player/round changes. Future work should define the denominator as eligible active NPCs, then test joins, departures, respawns, pool growth, and role changes. Preserve fractional accounting when changing the score display.

17.13 Scientific data extension architecture

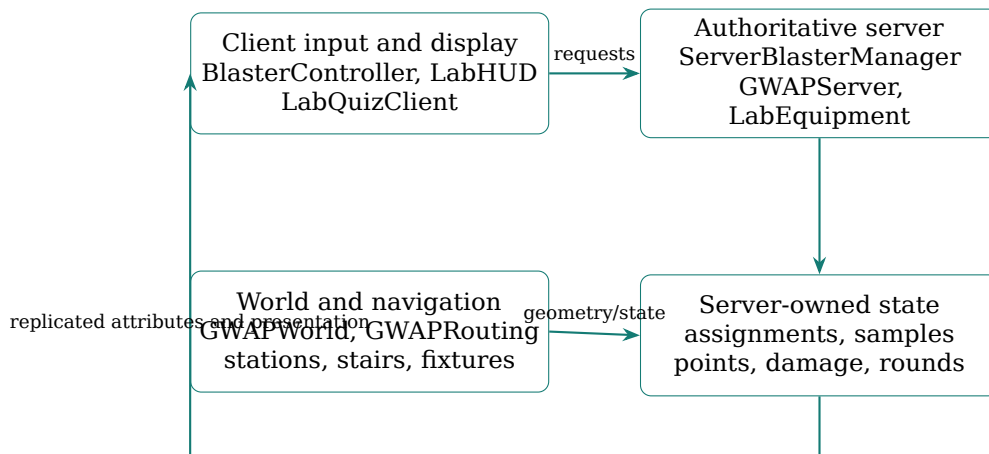
A research-capable extension should add ProtocolRegistry, TaskDataset, ObservationSubmission, QualityControl, and ResultExport boundaries. ProtocolRegistry defines valid stages and variables. TaskDataset supplies authentic stimuli or a clearly identified simulation seed. ObservationSubmission records structured decisions with protocol and sample IDs. QualityControl checks repeated judgments and known examples. ResultExport produces a schema reviewed by the intended researcher.

Keep this layer independent of combat and presentation. A player can lose game points without invalidating a correct annotation; a fast carrier route does not make a measurement biologically reliable. Record missing data, failure reasons, task revision, and timing without fabricating results. The present project has no such implemented pipeline, so the scenario proposals below are design options rather than claims of current scientific utility.

Chapter 18

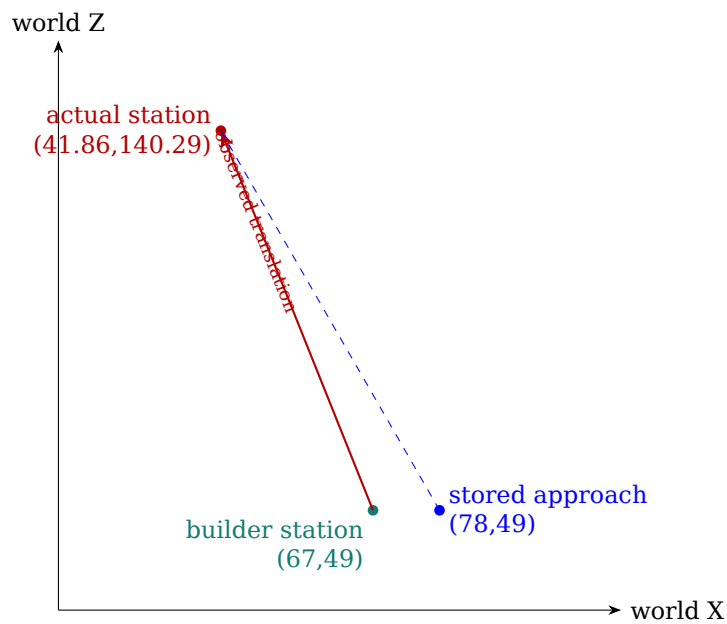
Architecture and coordinate diagrams

18.1 Runtime responsibilities



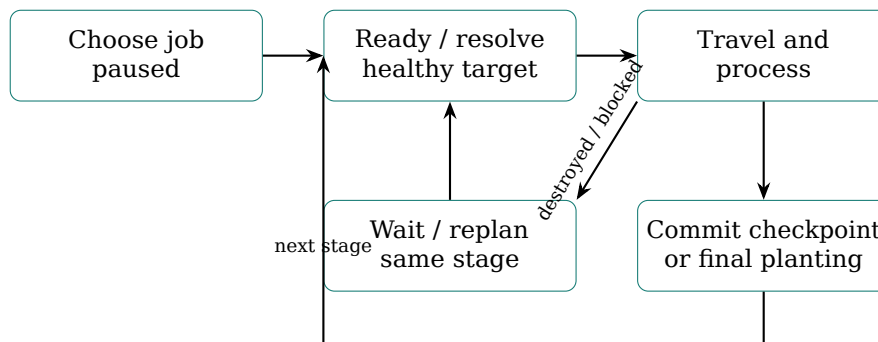
Requests do not directly authorize rewards or damage. The server validates and commits gameplay state. The client displays that state and predicts visual aiming feedback. The world supplies geometry and station metadata; the current coordinate defect shows why those two must remain consistent.

18.2 Observed origin mismatch: Incubator example



This plan view omits height for clarity. The station's actual Y is -21.75, while its builder Y is 1.5 and its stored approach Y is 3.25. Geometry translation does not automatically transform Vector3 attributes or literal coordinates in code. The dashed line illustrates the disagreement; it is not a proposed valid path.

18.3 Proposed mission transition contract



The current code implements destruction rerouting and pause behavior. General blocked-edge replanning remains proposed. Every asynchronous operation should carry the assignment/route revision so a stale operation cannot commit after regroup, termination, or a new target selection.

Chapter 19

Ten research scenarios: protocols and extension specifications

All ten scenarios below are implemented as ordered gameplay routes and a randomized integer variant. The proposed data product and validation paragraphs are future design, not implemented science. Floor labels describe logical floors in the original building frame. The initial coordinate discrepancy was repaired in Revision 4; see its fresh verification results and remaining coverage limits.

19.1 01. Germination viability

19.1.1 Implemented route and parameter

Seed bank (B1) -> Incubator (L1) -> Plot 1 (roof)

Moisture, integer 35-80 percent

A seedling is collected, incubated, and planted. Moisture is a displayed assignment variant; it does not drive a germination probability model.

19.1.2 Future scientific-purpose implementation

Annotate germinated versus non-germinated seeds in authentic, protocol-controlled images. Store image ID, seed count, germination count, moisture condition, replicate, and observer decision.

Use known examples and repeated independent counts; compare agreement before interpreting treatment differences. Preserve the original image and sampling method.

19.1.3 Engineering change and acceptance

Try both endpoints of the moisture range; destroy the Incubator before arrival and during processing. Verify an equivalent is selected and the nominal two-checkpoint plus final reward is not duplicated.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.2 02. Pathogen resistance

19.2.1 Implemented route and parameter

Containment (L2) -> Bio-analyzer (L3) -> Plot 2 (roof)

Resistance batch, integer 1-9

The batch number distinguishes assignments. There is no infectious-agent simulation, measured resistance curve, or actual laboratory manipulation.

19.2.2 Future scientific-purpose implementation

Classify visible plant-damage severity in a vetted image dataset with blinded treatment labels. Store dataset item, scoring scale version, replicate, and confidence.

Use a researcher-defined severity rubric and reference examples. Separate plant damage from image artifacts; aggregate only comparable stages and cultivars.

19.2.3 Engineering change and acceptance

Destroy Containment and its same-floor backup, then confirm the other-floor replacement retains the same capability and stage. Check that resistance batch remains unchanged after rerouting.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.3 03. Light spectrum optimization

19.3.1 Implemented route and parameter

Seed bank (B1) -> Spectrograph (L2) -> Infrared scanner (L3) -> Plot 3 (roof)

Photoperiod, integer 8-18 hours

Three pre-plot stations create a longer route. The number of hours is descriptive, not elapsed game time or a measured spectrum.

19.3.2 Future scientific-purpose implementation

Mark leaf boundaries or estimate leaf area in calibrated images from controlled light treatments. Record calibration, illumination condition, image timestamp, and segmentation.

Require scale calibration and repeated annotation. Do not infer spectral response from the avatar visiting an instrument.

19.3.3 Engineering change and acceptance

Verify all four route stages and the nominal 105-point clean escort. Destroy the middle instrument after seed collection; the carrier must retain its seedling and resume without collecting a second one.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the

stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.4 04. Soil bioremediation

19.4.1 Implemented route and parameter

Chemical seedlings (B1) -> Root analyzer (L2) -> Plot 4 (roof)

Soil batch, integer 1-12

The workflow represents sample tracking. It does not calculate contaminant concentration, uptake, removal efficiency, or environmental safety.

19.4.2 Future scientific-purpose implementation

Trace roots in real scan images or match samples to verified metadata. Record source sample, imaging method, root mask, and uncertain regions.

Use expert review for ambiguous roots and preserve units/calibration. A gameplay completion cannot certify remediation success.

19.4.3 Engineering change and acceptance

Verify replacement Root analyzer selection, plot capacity, and no stage advancement while the target is destroyed. Confirm batch metadata remains associated with the assignment.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.5 05. Hydroponic versus soil

19.5.1 Implemented route and parameter

Split-root tray (L1) -> Centrifuge (L3) -> Root analyzer (L2) -> Plot 5 (roof)

Nutrient batch, integer 1-8

The route deliberately goes up and then down before the roof. No paired root-growth measurements currently exist.

19.5.2 Future scientific-purpose implementation

Compare paired, anonymized root images from controlled growth conditions. Store pair ID, condition metadata held separately, root-length annotation, and replicate.

Randomize presentation order and require comparable imaging scales. Analyze pairs rather than treating unrelated plants as matched samples.

19.5.3 Engineering change and acceptance

Exercise both ascent and descent, a destroyed Centrifuge, and a destroyed Root analyzer. Check that the route order is preserved and the two instruments are not treated as interchangeable.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.6 06. UV adaptation

19.6.1 Implemented route and parameter

UV chamber (L2) -> Vitality scanner (L1) -> Plot 6 (roof)

Exposure, integer 5-20 seconds

The assignment text describes exposure; processing is still the common four-second gameplay action and does not implement the chosen duration.

19.6.2 Future scientific-purpose implementation

Annotate visible stress features in a curated before/after plant image pair. Record acquisition conditions, treatment metadata, severity features, and uncertainty.

Use researcher-approved image data and a fixed rubric. This game is not an instruction for operating UV equipment.

19.6.3 Engineering change and acceptance

Check descending travel after the UV stage, stable variant display, and shared use of Vitality scanner capability with scenarios 10 and 6 without crossing assignment ownership.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.7 07. Gravitropism assay

19.7.1 Implemented route and parameter

Clinostat (B1) -> Orientation scanner (L1) -> Plot 7 (roof)

Tray tilt, integer 10-80 degrees

The scenario names an orientation experiment but does not currently simulate root curvature from gravity or time.

19.7.2 Future scientific-purpose implementation

Mark root tips and growth angles in time-lapse frames with a known reference axis. Store frame time, axis calibration, landmarks, and annotation quality.

Require consistent angle conventions and reject occluded frames. Separate tray orientation from measured root direction.

19.7.3 Engineering change and acceptance

Verify Clinostat backups, floor-label conversion for basement, and a clear initial rendezvous. Test the zero-length initial-path case that previously required an explicit bypass.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.8 08. Drought resistance

19.8.1 Implemented route and parameter

Climate chamber (L3) -> Porometer (L2) -> Plot 8 (roof)

Humidity, integer 20-45 percent

The mission says simulated stomatal response, but it does not compute conductance or water potential. Humidity is assignment metadata.

19.8.2 Future scientific-purpose implementation

Annotate stomatal aperture from suitable microscopy images or organize verified instrument observations. Store image/instrument record ID, scale, treatment, aperture annotation, and replicate.

Require a defined measurement method and expert validation. Do not combine incompatible instruments or units under one generic score.

19.8.3 Engineering change and acceptance

Destroy the Porometer while the carrier is processing; verify that processing restarts at an equivalent rather than completing on the destroyed station.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.9 09. Microbiome symbiosis

19.9.1 Implemented route and parameter

Wet lab (B1) -> Symbiosis incubator (L2) -> Plot 9 (roof)

Inoculum batch, integer 1-8

The batch labels a transport protocol. No microbial growth model, sequencing data, or biological assay is present.

19.9.2 Future scientific-purpose implementation

Label root colonization regions in an approved microscopy dataset. Store image ID, region mask, labeling taxonomy, batch, and replicate.

Use blinded examples, inter-annotator agreement, and expert adjudication. Treat uncertain regions as uncertain rather than forcing a positive classification.

19.9.3 Engineering change and acceptance

Verify the carrier spawns near the selected role, explains the assignment, waits for Ready, and retains the inoculum batch through same-floor and other-floor reroutes.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

19.10 10. Cold-hardiness screening

19.10.1 Implemented route and parameter

Cryo-freezer (B1) -> Vitality scanner (L1) -> Plot 10 (roof)

Recovery temperature, integer 5-20 C

The workflow models recovery transport. It does not simulate freezing injury or measure actual survival.

19.10.2 Future scientific-purpose implementation

Score recovery features in a time series with known treatment conditions. Store plant ID, time since recovery, visible features, viability rubric, and observer confidence.

Require consistent follow-up timing and a definition of survival. Separate missing observations from dead plants.

19.10.3 Engineering change and acceptance

Check cryo-freezer explosion handling, a full Plot 10 falling back to another healthy greenhouse plot, and no reward if the assigned carrier is intercepted before final planting.

Implementation touchpoints: GWAPScenarios defines title/purpose/route/range; GWAPWorld and GWAPRouting provide named capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the assignment. A real data layer must add structured records independently of this route. Keep a stable scenario ID and protocol version so saved or exported observations do not change meaning when the list is reordered.

Chapter 20

Defect register, change accounting, and lessons

20.1 How many bugs were generated?

The evidence supports a bounded register of 24 historical defect families (H01-H24), two newly observed source/scene discrepancies (C01-C02), seven baseline audit risks (R01-R07), with final status in the Revision 4 chapter, and two tool/test artifacts (T01-T02). These categories are not added together as a lifetime bug total. A family groups recurring reports of the same symptom; one family may have had several causes or fixes. No complete issue tracker, commit history, or causal record establishes the total number of bugs introduced by an AI model.

At least two episodes have explicit evidence of an AI change or implementation error: H13 is the owner-requested rollback/restoration after an aiming change, and H24 is the zero-length route-prefix defect encountered during implementation testing. Other families are associated with the evolving AI-built game, but the record does not isolate whether each was newly introduced, pre-existing, or caused by an intervening model/asset change. Reporting all 24 as definitely generated by one model would be unsupported. The two current discrepancies are observed now; their author and exact introduction time are unknown.

Five major requirement reversals are recorded separately: large monitor panels were later removed; quizzes were added, removed, and then reintroduced in simpler form; the extra GWAP arsenal was added and then rejected; a broad 60-percent failure policy was replaced by a small workload; and static/identical NPC presentations were replaced by differentiated moving workers. New research-equipment destructibility and equivalent-station rerouting are feature extensions as well as corrections to the desired experience. They do not each count automatically as a new software bug.

20.2 Historical defect families

20.2.1 H01: Role kiosk inaccessible or inert

Evidence: Earlier shared chat: user could not select a role or activate the kiosk.

Repair history: Interaction layout and trigger behavior were repeatedly revised; temporary touch selection was later removed.

Code and regression target: LabRoles and kiosk geometry; verify E/click selection and no accidental role change.

Status: Reported fixed historically; not replayed in this edition.

20.2.2 H02: Role outfit missing or insufficiently distinct

Evidence: Earlier shared chat repeatedly requested visible clothing after role selection.

Repair history: Role appearance application and respawn behavior were revised.

Code and regression target: LabRoles; test both roles before and after respawn.

Status: Reported fixed historically; appearance is built, full rig coverage unverified.

20.2.3 H03: Equipment E/F actions appeared inert

Evidence: Earlier user reports said nothing happened near equipment or when pressing controls.

Repair history: Physical prompts/buttons and runtime/fallback installation were revised.

Code and regression target: LabEquipment bind/makeControls; distinguish old F behavior from current E/R policy.

Status: Reported fixed historically; syntax checks alone were insufficient evidence.

20.2.4 H04: Plant could be taken but not dropped

Evidence: Earlier user explicitly reported pickup without a usable drop interaction.

Repair history: Working-cabinet drop logic, free-slot guidance, and human override of reservations were improved.

Code and regression target: LabEquipment takePlantFromChamber/storePlantInChamber.

Status: Reported fixed historically; retest full/wrong/removed destinations.

20.2.5 H05: Equipment or fixtures intersected walls, doors, or windows

Evidence: Multiple earlier screenshots and requests identified misplaced cabinets, health kits, weapons, and pipes.

Repair history: Placement clearance and wall alignment were revised; pipes moved upward.

Code and regression target: Generator placement and saved geometry.

Status: Historical clearance measurements exist; current translated geometry needs a fresh spatial audit.

20.2.6 H06: Equipment could be repaired through a wall

Evidence: Earlier user explicitly identified repair across a wall.

Repair history: Closer interaction range and server line-of-sight validation were added to legacy interactions.

Code and regression target: LabEquipment canInteract.

Status: Built in legacy path; separate research prompts have risk R01.

20.2.7 H07: Repair route crossed walls or ignored stairs

Evidence: Earlier user reported a straight red path through geometry.

Repair history: Pathfinding and explicit stair-aware routing replaced a straight fallback.

Code and regression target: LabEquipment createRepairPath; later GWAP routePoints is a separate implementation.

Status: Historically reported fixed; C01 invalidates assumptions for moved geometry.

20.2.8 H08: Overlapping information obscured play

Evidence: Repeated shared-chat and current-thread screenshots show large or duplicate panels.

Repair history: Distant labels, duplicate HUDs, and notification feeds were reduced; latest HUD groups mission/status.

Code and regression target: LabHUD, LabEquipment display policy, BlasterController, LabQuiz-Client.

Status: One small-phone layout measured; all modals/devices not exhaustive.

20.2.9 H09: Uneven lighting and black floor bands

Evidence: Earlier handoff requested lighting repair.

Repair history: The handoff records the defect but does not independently prove the exact repair.

Code and regression target: Generator Lighting and room light placement.

Status: Historical report; current visual closure not established by this audit.

20.2.10 H10: Touch firing failed or used unsuitable input

Evidence: Shared chat requested cross-platform shooting because mobile controls failed.

Repair history: Separated controller, casting module, and server manager; touch camera and action controls introduced.

Code and regression target: BlasterController, Blaster, ServerBlasterManager.

Status: Historical device checks reported; preserve input separation.

20.2.11 H11: Reticle and muzzle impact disagreed

Evidence: Shared chat reported firing toward the avatar or not correctly hitting a wall.

Repair history: Camera target and muzzle obstruction handling were revised.

Code and regression target: Blaster ray helpers and controller reticle.

Status: Historical desktop/mobile impact checks reported.

20.2.12 H12: Android vertical aim offset

Evidence: Aiming at the tree base produced a hit near the branches.

Repair history: Mobile ray changed to Camera.CFrame.LookVector after a reported 1.35-degree safe-area mismatch.

Code and regression target: BlasterController mobile aim ray.

Status: Historical lower-trunk hit and client/server direction agreement reported.

20.2.13 H13: Mouse pitch/yaw and arm tracking regressed

Evidence: User requested restoration after unrelated trigger/aim experiments.

Repair history: Mouse-priority detection and shoulder/arm target tracking restored, including newer joints.

Code and regression target: BlasterController input mode and arm pose.

Status: Explicit AI-change-associated regression; historically restored.

20.2.14 H14: FIRE/AIM/JUMP controls overlapped

Evidence: Shared chat and screenshots identified overlapping mobile actions.

Repair history: Controls repositioned relative to jump with separate spacing.

Code and regression target: BlasterController touch layout.

Status: Historical Galaxy A06 check; regression coverage required after HUD changes.

20.2.15 H15: HUD existed without its reticle

Evidence: Shared chat reports incomplete BlasterHUD initialization.

Repair history: HUD integrity checks rebuild missing reticle children.

Code and regression target: BlasterController _ensureHUD.

Status: Reported fixed historically; test equip/unequip and device changes.

20.2.16 H16: Maintenance vent landed player partly in floor

Evidence: User reported becoming embedded at basement destination.

Repair history: Vent landing and both basement stair routes were corrected.

Code and regression target: LabEquipment basement handler and generator geometry.

Status: Historical vent landing test reported.

20.2.17 H17: Workers passed through geometry or players

Evidence: Shared chat requested room-aware movement and avoidance.

Repair history: Original worker motion gained room/obstacle checks and appearance updates.

Code and regression target: LabEquipment botMotionChecks/walkBotSegment/moveBotTo.

Status: Historical implementation; not proof of arbitrary carrier obstacle avoidance.

20.2.18 H18: Assigned carrier was not nearby or visible

Evidence: Current-thread user could not find the mission NPC after role selection.

Repair history: Per-player companion allocation and safe nearby placement introduced.

Code and regression target: GWAPServer companion/startMission; GWAPNPC nearPlayer.

Status: Historical approximately eight-stud placement test.

20.2.19 H19: Carrier clones repeated names/appearance and female silhouette was wrong

Evidence: Current-thread screenshots show repeated Maria models and unsuitable hair.

Repair history: Different non-seated templates, alternating gender preference, and hair adjustments.

Code and regression target: GWAPServer makeBot; GWAPNPC appearance.

Status: Historical appearance check; animation/identity remain template-dependent.

20.2.20 H20: Carriers queued together in the hallway

Evidence: Current-thread screenshot and request rejected a hallway line of bots.

Repair history: Unused carriers moved to a ServerStorage pool and allocated individually.

Code and regression target: GWAPServer available/companions/pool.

Status: Historical one visible, four pooled observation.

20.2.21 H21: Carrier glided unnaturally and too slowly

Evidence: Current-thread user asked for natural movement matching player speed.

Repair history: Distance-driven limb gait and smoothed owner WalkSpeed matching added.

Code and regression target: GWAPNPC animate; GWAPServer go.

Status: Historical speed-20 movement and approximately 24-degree leg swing measured.

20.2.22 H22: Weapon trembled while idle and walking

Evidence: Repeated current-thread reports persisted after an earlier partial fix.

Repair history: Single pre-camera rotation, no artificial Grip bob, and armed animation stabilization.

Code and regression target: BlasterController pre-camera, render, and PreSimulation paths.

Status: Historical zero Grip motion idle/walking; firing recoil retained.

20.2.23 H23: Wall explosion left fixtures floating or usable

Evidence: Current-thread user requested disappearance of objects attached to the removed wall section.

Repair history: Recent-blast wall removal detection and fixture quarantine introduced.

Code and regression target: GWAPWallDamage.

Status: Historical four fixture categories removed; outside health station preserved.

20.2.24 H24: Zero-length rendezvous path prevented progress

Evidence: Development test encountered a start point already at its corridor entry.

Repair history: Zero-distance prefix bypass and checked short direct path added.

Code and regression target: GWAPServer go initial rendezvous.

Status: Implementation-level defect found during testing; historical retest passed.

20.3 Baseline discrepancies, risks, and tool artifacts

These entries preserve the initial audit. Revision 4 repairs C01/C02, adds role/LOS validation for R01, collision stopping for R02, correct eligibility for R03, and filtering for R07. R04 restoration policy, R05 shared-mission semantics, R06 load certification and the limits stated in the Revision 4 chapter remain open.

20.3.1 C01: Observed now: moved geometry, stale route coordinates

Evidence: Station positions and roof share a translation while Approach attributes and route-Points constants retain the old origin.

Next action: Introduce a coherent local coordinate frame and approach attachments; validate every route in a copied place. No fix installed here.

20.3.2 C02: Observed now: incorrect world module bundled in installer

Evidence: Root GWAPWorld.lua contains older mission-server code, and InstallGWAP.lua installs that incorrect content as the world module. The active module is different.

Next action: Do not run the captured installer unchanged. Replace its world-module input from the verified active source, reconcile generator versions, regenerate and compare installed hashes in a copied place.

20.3.3 R01: Research prompt validation is narrower than legacy validation

Evidence: ResearchRepair checks life, distance, and Exploded but does not enforce Technician role or perform its own server ray visibility check.

Next action: Apply an explicit role/range/visibility/state policy and validate hold completion server-side where required. Test wrong role and an intervening wall.

20.3.4 R02: Main carrier route assumes clear surveyed geometry

Evidence: The main kinematic route moves along points and stops near players; it does not sweep every segment for arbitrary new obstacles.

Next action: Add edge validation, blocked-route state, and bounded replanning. Preserve one movement authority.

20.3.5 R03: Solo percentage counts pooled models

Evidence: Arming includes available carriers outside Workspace, while shooting excludes them.

Next action: Define an eligible active population; recompute selection on lifecycle changes and display both selected and active threat counts.

20.3.6 R04: Wall-fixture restoration is not defined

Evidence: Removed fixtures remain in ServerStorage; repairing attributes does not rebuild walls or reparent fixtures.

Next action: Define persistent versus per-round destruction, then implement explicit restoration or deliberate permanent removal for the session.

20.3.7 R05: Multiplayer balance and shared objective semantics are unverified

Evidence: Assignments are per player; no complete multi-client regression was recorded.

Next action: Test two roles and multiple carriers concurrently; commission a shared-convoy design separately if desired.

20.3.8 R06: Scale is configured, not performance-certified

Evidence: MaxPlayers is 60 and the scene is large; no current load profile was captured.

Next action: Measure client/server frame time, memory, replication, and NPC workloads before making capacity claims.

20.3.9 R07: Lounge notes are displayed without explicit filtering

Evidence: LoungeSystems onChat stores the raw /note message and refreshWall writes it to a shared display; no text-filtering call appears in that source.

Next action: Filter submitted text on the server using the appropriate Roblox text-filtering result for a shared display, handle filtering failure without displaying raw text, and add length/rate limits. Test with multiple viewers.

20.3.10 T01: Tool artifact: truncated source backup

Evidence: A previous LabEquipment backup was cut near 100 KB and renamed .partial.lua.

Next action: Never install a partial capture. This edition verifies every reconstructed Studio source byte length.

20.3.11 T02: Test artifact: duplicate controller binding

Evidence: A prior client test required/bound a controller again and produced duplicate render-step warnings.

Next action: Use existing bootstrap/controller instances for probes and fresh Play sessions; do not classify the test harness warning as a confirmed production defect.

20.4 Rules that prevent repetition

Freeze an accepted input/camera baseline before unrelated features. Measure a symptom before rewriting its subsystem. Use live source and hashes to resolve conflicts. Keep one owner for camera, root, arm, and gun transforms. Revalidate after every yielding interaction. Treat geometry and navigation metadata as one versioned unit. Check actual screen rectangles and server state, not only appearance. Separate a passing syntax check from a passing player workflow. Never report a tool artifact or an old test as proof of current gameplay correctness.

Chapter 21

Verification ledger and future acceptance suite

21.1 P01: Role selection and nearby companion

Evidence period: Prior GWAP turn

Observed/reported result: Approximately eight studs from player; one visible carrier and four pooled.

Limit: Historical observation, not rerun for current geometry.

21.2 P02: Wait and Regroup

Evidence period: Prior GWAP turn

Observed/reported result: Wait movement zero; regroup returned within roughly eight studs.

Limit: No stress test of every blocked spawn candidate.

21.3 P03: Original rifle and equipment destruction

Evidence period: Prior GWAP turn

Observed/reported result: Lab Rifle destroyed a research machine; nearby explosion destroyed another; R repair restored it.

Limit: Does not certify wall-fixture restoration or every blast chain.

21.4 P04: Equivalent rerouting

Evidence period: Prior GWAP turn

Observed/reported result: Incubator -> same-floor Backup 1 -> other-floor Backup 2; assignment stayed RUNNING.

Limit: Earlier coordinate frame; current C01 requires rerun.

21.5 P05: Carrier gait and speed

Evidence period: Prior GWAP turn

Observed/reported result: WalkSpeed 20; roughly 28.7 studs of travel during sampled interval; leg swing about 24 degrees.

Limit: Kinematic gait, not Humanoid physics certification.

21.6 P06: Wall-fixture cleanup

Evidence period: Prior GWAP turn

Observed/reported result: Four fixture types within a temporary wall section removed; outside health station preserved.

Limit: Controlled isolated test, not whole-floor blast profiling.

21.7 P07: Small mobile layout

Evidence period: Prior GWAP turn

Observed/reported result: 705x338 viewport; panel bottom y=179, joystick top y=187.

Limit: Eight-pixel gap in this configuration only.

21.8 P08: Idle weapon stabilization

Evidence period: Prior GWAP turn

Observed/reported result: 90 frames with zero Grip motion and zero shoulder-animation translation.

Limit: One tested original rifle/rig configuration.

21.9 P09: Walking stabilization

Evidence period: Prior GWAP turn

Observed/reported result: Approximately 19 studs of walking with zero Grip motion.

Limit: Intentional recoil while firing was not removed.

21.10 P10: Interception score

Evidence period: Prior GWAP turn

Observed/reported result: Assigned carrier awarded 50 once; repeat damage did not add points.

Limit: Multi-client race conditions not exhaustively tested.

21.11 P11: Solo score loss

Evidence period: Prior GWAP turn

Observed/reported result: 100 points became 99 on a successful NPC hit.

Limit: Low-score fractional policy also inspected in code; no long statistical run.

21.12 P12: Rooftop mission completion

Evidence period: Earlier GWAP turn

Observed/reported result: Seed source -> instrument -> roof completed and one plant appeared.

Limit: Test used helper following; friendly fire disqualified final escort reward, so clean full reward was not proven.

21.13 P13: Player shooting validation

Evidence period: Shared chat

Observed/reported result: Reported 10 damage, wall blocking, and rejection of distant forged camera origin.

Limit: Historical report; not a new penetration/security audit.

21.14 P14: ADS and device controls

Evidence period: Shared chat

Observed/reported result: Reported FOV 70->55 and sensitivity 1->0.62, restored on exit; phone/tablet controls checked.

Limit: Some later control changes superseded these exact UI versions.

21.15 P15: Basement and quiz/window interactions

Evidence period: Shared chat

Observed/reported result: Vent landing, pickup quiz, and breakable window passed reported checks.

Limit: No new Play session in this edition.

21.16 D01: Live source completeness

Evidence period: This edition

Observed/reported result: 29 source containers captured; every UTF-8 byte length matched Studio inventory.

Limit: Static capture only; active count 20, archive 8, disabled generator 1.

21.17 D02: Scene/source audit

Evidence period: This edition

Observed/reported result: 67 research station records; place version 70; C01 and C02 identified.

Limit: No claim that current routing is playable.

21.18 Proposed new regression sequence

21.18.1 Baseline

Open a copied saved place, identify the place/version, compare active-source hashes, and ensure there is only one bootstrap/controller authority.

21.18.2 Coordinate frame

Translate and rotate a test laboratory with its navigation origin; verify every station approach stays in its intended corridor and each stair edge is clear. Include the current displaced scene as a regression fixture.

21.18.3 Transactions

Two clients compete for the same plant and destination slot. Exactly one transfer commits; the other gets a clear reason. Repeat with a quiz outstanding, a full cabinet, and a destroyed destination.

21.18.4 Carrier lifecycle

Choose, Ready, Wait, Status, Regroup, respawn, disconnect, change role, and end the round during travel and processing. No duplicate carrier, seedling, or reward remains.

21.18.5 Destruction

Bullet-hit research station, chain blast, wall-section removal, outside-fixture preservation, repair, all-equivalents unavailable, and round reset after quarantine.

21.18.6 Input preservation

Desktop mouse target at all screen edges; mobile forward ray; right-swipe camera with left movement; ADS on/off; equip/unequip; idle/walk gun stability; UI click does not fire.

21.18.7 Responsive layout

Small phone portrait and landscape, tablet, desktop, chat open, quiz open, inventory open. Record rectangle intersections for mission, joystick, jump, fire, aim, and prompts.

21.18.8 Scoring

Clean 90/105-point escort as applicable, one 50-point interception, friendly-fire disqualification, low-score fractional loss, reset debt, and multiplayer solo-fire exclusion.

21.18.9 Performance

Profile crowded rooms and multi-client play; report hardware, viewport, scene population, average/p95 frame time, server heartbeat cost, and replication load. Do not extrapolate a one-client test to 60 players.

These are proposed tests. They were not executed while producing this document. The verification report for this edition concerns source capture, document compilation, page rendering, and manifest completeness.

Chapter 22

Subsystem-by-subsystem future update playbook

22.1 LabKinds: Equipment catalog

Authoritative source: S05 / ReplicatedStorage.LabKinds

See complete listing on page [134](#).

22.1.1 Proposed implementation

Add one Kind with inputs, outputs, wait duration, and display text. Keep simulation values explicit.

22.1.2 Dependencies to preserve

LabEquipment consumption and treatment paths; generator Kind attributes.

22.1.3 Acceptance cases

Unknown Kind, missing input, invalid output, full destination, and a complete valid cycle.

22.2 LabEquipment: Equipment engine

Authoritative source: S11 / ServerScriptService.LabEquipment

See complete listing on page [168](#).

22.2.1 Proposed implementation

Extract inventory transactions before touching hazards or NPC motion. Preserve reservations and post-yield validation.

22.2.2 Dependencies to preserve

LabRoles assignments, LabQuizClient, tagged geometry, server hit bridge.

22.2.3 Acceptance cases

Concurrent pickup/drop, repair during failure tick, dead player, destroyed target, canceled quiz.

22.3 LabRoles: Roles and outfits

Authoritative source: S12 / ServerScriptService.LabRoles

See complete listing on page [226](#).

22.3.1 Proposed implementation

Keep exactly two roles unless commissioned otherwise; centralize role-to-outfit and permission mapping.

22.3.2 Dependencies to preserve

GWAPServer LabRole observer and original equipment role checks.

22.3.3 Acceptance cases

Both kiosks, respawn, role change during mission, and accessory/rig variations.

22.4 LabRound: Round lifecycle

Authoritative source: S13 / ServerScriptService.LabRound

See complete listing on page [233](#).

22.4.1 Proposed implementation

Make reset ordering explicit with one round transition contract and subsystem callbacks.

22.4.2 Dependencies to preserve

Points, fractional debt, carrier termination, research repair, plot clearing.

22.4.3 Acceptance cases

Join during break, end during processing, disconnected winner, and reset after destruction.

22.5 LabTechnicians: Legacy worker service

Authoritative source: S14 / ServerScriptService.LabTechnicians

See complete listing on page [235](#).

22.5.1 Proposed implementation

Check which templates actually have Humanoids before extending its path loop. Avoid competing with LabEquipment worker motion.

22.5.2 Dependencies to preserve

LabWorkerBot tags and template structure.

22.5.3 Acceptance cases

No double movement, no idle thread leak, valid target assignment, and bounded retry.

22.6 LabElevators: Elevator compatibility script

Authoritative source: S10 / ServerScriptService.LabElevators

See complete listing on page [165](#).

22.6.1 Proposed implementation

Treat as dormant unless real tagged elevator geometry exists. Do not claim elevator transport from script presence alone.

22.6.2 Dependencies to preserve

Level metadata and any future lift models.

22.6.3 Acceptance cases

Door interlock, floor alignment, queueing, interrupted travel, and stair fallback.

22.7 LoungeSystems: Lounge server behavior

Authoritative source: S15 / ServerScriptService.LoungeSystems

See complete listing on page [239](#).

22.7.1 Proposed implementation

Keep lounge interactions isolated from laboratory scoring and navigation. Profile actual cost before deleting scenery.

22.7.2 Dependencies to preserve

LoungeClient and generated lounge objects.

22.7.3 Acceptance cases

Missing lounge, occupied seat, interaction range, and cleanup on departure.

22.8 LoungeClient: Lounge presentation

Authoritative source: S28 / StarterPlayer.StarterPlayerScripts.LoungeClient

See complete listing on page [253](#).

22.8.1 Proposed implementation

Add local effects through lifecycle-managed connections and restore prior camera/UI state on exit.

22.8.2 Dependencies to preserve

LoungeSystems and player character lifecycle.

22.8.3 Acceptance cases

Enter/exit, respawn inside, missing props, and device input.

22.9 LabQuizClient: Quiz UI

Authoritative source: S26 / StarterPlayer.StarterPlayerScripts.LabQuizClient

See complete listing on page [248](#).

22.9.1 Proposed implementation

Use an accessible modal with touch and keyboard choices and a clear timeout. Display server challenge data only.

22.9.2 Dependencies to preserve

LabEquipment pendingQuizzes and MathQuizActive.

22.9.3 Acceptance cases

Wrong/late/replayed reply, cancel, death, target destruction, and overlap with GWAP HUD.

22.10 LabSidearmClient: Tool bootstrap

Authoritative source: S27 / StarterPlayer.StarterPlayerScripts.LabSidearmClient

See complete listing on page [251](#).

22.10.1 Proposed implementation

Keep one controller instance per equipped original Tool and one active camera owner.

22.10.2 Dependencies to preserve

BlasterController AutoBind/Enable/Disable.

22.10.3 Acceptance cases

Rapid equip, respawn, backpack replacement, duplicate script prevention.

22.11 Blaster: Shared casting

Authoritative source: S06 / ReplicatedStorage.Modules.Blaster

See complete listing on page [137](#).

22.11.1 Proposed implementation

Preserve desktop screen conversion and mobile camera-forward behavior as separate tested paths.

22.11.2 Dependencies to preserve

Controller reticle and server manager raycasts.

22.11.3 Acceptance cases

Mouse edges, asymmetric safe areas, near wall, long range, and invalid vectors.

22.12 BlasterController: Input/camera/pose

Authoritative source: S07 / ReplicatedStorage.Modules.BlasterController

See complete listing on page [141](#).

22.12.1 Proposed implementation

Fix one measured fault at a time; retain cached base transforms and deterministic update ownership.

22.12.2 Dependencies to preserve

Shared casting, Tool grip metadata, HUD rectangles, rig joints.

22.12.3 Acceptance cases

Idle/walk/recoil, mouse pitch/yaw, mobile swipe, hybrid input, ADS restore, GUI click suppression.

22.13 ServerBlasterManager: Authoritative shooting

Authoritative source: S16 / ServerScriptService.ServerBlasterManager

See complete listing on page [242](#).

22.13.1 Proposed implementation

Add weapon configuration only through validated server-side attributes; keep sequence and rate limits.

22.13.2 Dependencies to preserve

Blaster configuration, GWAPHitBridge, legacy impact bridge.

22.13.3 Acceptance cases

Spoofed tool, out-of-order packet, impossible camera origin, wall obstruction, and correct target damage.

22.14 GWAPWorld: Research geometry

Authoritative source: S04 / ReplicatedStorage.GWAPWorld

See complete listing on page [131](#).

22.14.1 Proposed implementation

Use the captured active module, not the loose mislabeled file. Convert construction to a versioned local frame.

22.14.2 Dependencies to preserve

GWAPRouting, mission approaches, roof stair geometry.

22.14.3 Acceptance cases

Build twice without duplicates, translated/rotated building, all approach points, roof opening.

22.15 GWAPScenarios: Scenario data

Authoritative source: S03 / ReplicatedStorage.GWAPScenarios

See complete listing on page [130](#).

22.15.1 Proposed implementation

Give each scenario a stable ID and protocol version; decouple plot selection from array position.

22.15.2 Dependencies to preserve

Server index validation, HUD job cards, station capabilities.

22.15.3 Acceptance cases

Missing capability, invalid range, new eleventh entry, and unchanged old scenario IDs.

22.16 GWAPNPC: Carrier appearance and gait

Authoritative source: S01 / ReplicatedStorage.GWAPNPC

See complete listing on page [126](#).

22.16.1 Proposed implementation

Separate immutable template pose from animation state; preserve same-inventory gun models.

22.16.2 Dependencies to preserve

GWAPServer template selection and movement ownership.

22.16.3 Acceptance cases

Boys/girls, held seedling, gun pose, idle reset, teleport reset, pooled reactivation.

22.17 GWAPRouting: Equivalent selection

Authoritative source: S02 / ReplicatedStorage.GWAPRouting

See complete listing on page [128](#).

22.17.1 Proposed implementation

Add reachability and operational-state adapters after fixing coordinates. Use stable IDs and deterministic tie breaking.

22.17.2 Dependencies to preserve

World station registry, destruction state, capacity.

22.17.3 Acceptance cases

Same floor preferred, other floor fallback, all unavailable, full plots, removed fixture.

22.18 GWAPServer: Mission orchestration

Authoritative source: S08 / ServerScriptService.GWAPServer

See complete listing on page [154](#).

22.18.1 Proposed implementation

Extract mission state/reward transactions without changing existing role meanings. Introduce route revision cancellation consistently.

22.18.2 Dependencies to preserve

All GWAP modules, LabRoles, Points, LabRound.

22.18.3 Acceptance cases

Concurrent players, damage during processing, repeated replies, respawn, departure, and terminal reward idempotence.

22.19 GWAPWallDamage: Wall fixture lifecycle

Authoritative source: S09 / ServerScriptService.GWAPWallDamage

See complete listing on page [163](#).

22.19.1 Proposed implementation

Replace name-pattern coupling with explicit wall/fixture association when geometry is refactored. Define restoration.

22.19.2 Dependencies to preserve

Legacy gas detonation, research stations, prompts and tags.

22.19.3 Acceptance cases

Near/outside fixture, rotated wall, two blasts, round reset, old callback after quarantine.

22.20 LabHUD: Grouped mission display

Authoritative source: S25 / StarterPlayer.StarterPlayerScripts.LabHUD

See complete listing on page [245](#).

22.20.1 Proposed implementation

Centralize notice priority and reserved screen regions. Preserve original backpack and combat controls.

22.20.2 Dependencies to preserve

GWAP attributes, original notices, quiz modal, joystick/jump bounds.

22.20.3 Acceptance cases

Small portrait/landscape, long text, two-digit stages, chat open, quiz open, and keyboard/gamepad focus.

22.21 Cross-cutting data, assets, and deployment

For persistent data, introduce a versioned server-owned schema and migrations before saving attributes. For assets, maintain IDs, provenance, scale, orientation, and fallback behavior; a visual replacement must not change a muzzle convention silently. For deployment, compare installed hashes against the chosen baseline, save/reopen, test the persistent place, and retain a rollback artifact. The geometry export cannot restore these contracts on its own.

22.22 Complete proposed coordinate-frame module

The following module is a reference implementation, not installed or runtime-tested. It requires a deliberately placed NavigationOrigin and ApproachPoint attachments. It does not discover a valid origin automatically, reconstruct stairs, or replace pathfinding. Its edgeClear helper is a single swept query, not a complete navigator and not a test for initial overlap. Integrate it only after defining these scene objects and testing the contract.

```

1 -- PROPOSED REFERENCE MODULE. Not installed in the captured game.
2 -- Contract: Laboratory_Complex contains one anchored BasePart named NavigationOrigin.
3 -- Its CFrame maps generator-local coordinates into the current world.
4 -- Station ApproachPoint Attachments move with their station geometry.
5 local WorldFrame = {}
6 WorldFrame.__index = WorldFrame
7
8 local function finite(n)
9     return n == n and n > -math.huge and n < math.huge
10 end
11
12 local function validVector(v)
13     return typeof(v) == "Vector3" and finite(v.X) and finite(v.Y) and finite(v.Z)
14 end
15
16 function WorldFrame.new(complex, floorHeight, floorZeroY)
17     assert(typeof(complex) == "Instance", "Laboratory model required")
18     local origin = complex:FindFirstChild("NavigationOrigin")
19     assert(origin and origin:IsA("BasePart"), "NavigationOrigin BasePart required")
20     assert(origin.Anchored, "NavigationOrigin must be anchored")
21     floorHeight = floorHeight or 16.25
22     floorZeroY = floorZeroY or 0
23     assert(finite(floorHeight) and floorHeight > 0, "Invalid floor height")
24     assert(finite(floorZeroY), "Invalid floor zero")
25     return setmetatable({
26         complex = complex,
27         origin = origin,
28         floorHeight = floorHeight,
29         floorZeroY = floorZeroY,
30     }, WorldFrame)
31 end
32
33 function WorldFrame:toWorld(localPoint)
34     assert(validVector(localPoint), "Finite local point required")
35     return self.origin.CFrame:PointToWorldSpace(localPoint)
36 end
37
38 function WorldFrame:toLocal(worldPoint)
39     assert(validVector(worldPoint), "Finite world point required")

```



```
40     return self.origin.CFrame:PointToObjectSpace(worldPoint)
41 end
42
43 function WorldFrame:floorOf(worldPoint, standingOffset)
44     local p = self:toLocal(worldPoint)
45     return math.floor((p.Y - self.floorZeroY - (standingOffset or 0)) / self.floorHeight + 0.5)
46 end
47
48 function WorldFrame:approach(station)
49     assert(station:IsDescendantOf(self.complex), "Station outside laboratory")
50     local marker = station:FindFirstChild("ApproachPoint", true)
51     assert(marker and marker:IsA("Attachment"), "ApproachPoint Attachment required")
52     return marker.WorldPosition
53 end
54
55 function WorldFrame:route(localPoints)
56     local result = table.create(#localPoints)
57     for i, point in ipairs(localPoints) do
58         result[i] = self:toWorld(point)
59     end
60     return result
61 end
62
63 function WorldFrame:edgeClear(from, to, size, exclusions)
64     assert(validVector(from) and validVector(to), "Finite endpoints required")
65     assert(validVector(size) and size.X > 0 and size.Y > 0 and size.Z > 0,
66         "Positive agent size required")
67     local delta = to - from
68     if delta.Magnitude < 0.001 then
69         return true, nil
70     end
71     local params = RaycastParams.new()
72     params.FilterType = Enum.RaycastFilterType.Exclude
73     params.FilterDescendantsInstances = exclusions or {}
74     params.IgnoreWater = true
75     local hit = workspace.Blockcast(CFrame.new(from), size, delta, params)
76     return hit == nil, hit
77 end
78
79 return WorldFrame
```

Chapter 23

Scene inventory and coordinate evidence

Captured from the Edit datamodel. Counts describe tagged objects or editor inventory, not runtime performance. Place ID 137098589879404; reported place version 70; configured MaxPlayers 60. No Play mode was started for this capture.

23.1 Laboratory equipment kinds

Kind	Count
Decontamination Unit	84
GMO Injector	1
Germination Chamber	119
Growth Hormone Mixer	9
Hormone Treatment Bench	45
Incubator	109
Mutagenesis Chamber	43
Nutrient Infuser	1
Plant Growth Chamber	450
Pollination Station	6
UV Growth Scanner	5

23.2 Research station positions and stored approaches

Every row is a captured station. Floor is a logical index (-1 basement, 0 Level_1, 8 roof). Positions and approach points are printed to two decimals for readability; evidence/live-world-inventory.json preserves full returned precision. The mismatch is especially visible in Y and Z and is not a harmless label difference.

Station / floor	Actual position	Stored approach
Bio-analyzer / 2	41.86, 10.75, 140.29	78.00, 35.75, 49.00
Bio-analyzer - Backup 1 / 2	63.86, 10.75, 140.29	78.00, 35.75, 49.00
Bio-analyzer - Backup 2 / -1	41.86, -38.00, 255.29	78.00, -13.00, 164.00
Centrifuge / 2	41.86, 10.75, 174.29	78.00, 35.75, 83.00
Centrifuge - Backup 1 / 2	63.86, 10.75, 174.29	78.00, 35.75, 83.00

Station / floor	Actual position	Stored approach
Centrifuge - Backup 2 / -1	41.86, -38.00, 269.29	78.00, -13.00, 178.00
Chemical seedlings / -1	41.86, -38.00, 157.29	78.00, -13.00, 66.00
Chemical seedlings - Backup 1 / -1	63.86, -38.00, 157.29	78.00, -13.00, 66.00
Chemical seedlings - Backup 2 / 0	41.86, -21.75, 255.29	78.00, 3.25, 164.00
Climate chamber / 2	41.86, 10.75, 191.29	78.00, 35.75, 100.00
Climate chamber - Backup 1 / 2	63.86, 10.75, 191.29	78.00, 35.75, 100.00
Climate chamber - Backup 2 / -1	41.86, -38.00, 283.29	78.00, -13.00, 192.00
Clinostat / -1	41.86, -38.00, 174.29	78.00, -13.00, 83.00
Clinostat - Backup 1 / -1	63.86, -38.00, 174.29	78.00, -13.00, 83.00
Clinostat - Backup 2 / 0	41.86, -21.75, 269.29	78.00, 3.25, 178.00
Containment / 1	41.86, -5.50, 140.29	78.00, 19.50, 49.00
Containment - Backup 1 / 1	63.86, -5.50, 140.29	78.00, 19.50, 49.00
Containment - Backup 2 / 2	41.86, 10.75, 255.29	78.00, 35.75, 164.00
Cryo-freezer / -1	41.86, -38.00, 208.29	78.00, -13.00, 117.00
Cryo-freezer - Backup 1 / -1	63.86, -38.00, 208.29	78.00, -13.00, 117.00
Cryo-freezer - Backup 2 / 0	41.86, -21.75, 283.29	78.00, 3.25, 192.00
Incubator / 0	41.86, -21.75, 140.29	78.00, 3.25, 49.00
Incubator - Backup 1 / 0	63.86, -21.75, 140.29	78.00, 3.25, 49.00
Incubator - Backup 2 / 1	41.86, -5.50, 255.29	78.00, 19.50, 164.00
Infrared scanner / 2	41.86, 10.75, 157.29	78.00, 35.75, 66.00
Infrared scanner - Backup 1 / 2	63.86, 10.75, 157.29	78.00, 35.75, 66.00
Infrared scanner - Backup 2 / -1	41.86, -38.00, 297.29	78.00, -13.00, 206.00
Orientation scanner / 0	41.86, -21.75, 174.29	78.00, 3.25, 83.00
Orientation scanner - Backup 1 / 0	63.86, -21.75, 174.29	78.00, 3.25, 83.00
Orientation scanner - Backup 2 / 1	41.86, -5.50, 269.29	78.00, 19.50, 178.00
Plot 1 / 8	22.86, 107.55, 139.29	78.00, 133.60, 48.00
Plot 10 / 8	82.86, 107.55, 219.29	78.00, 133.60, 128.00
Plot 2 / 8	82.86, 107.55, 139.29	78.00, 133.60, 48.00
Plot 3 / 8	22.86, 107.55, 159.29	78.00, 133.60, 68.00
Plot 4 / 8	82.86, 107.55, 159.29	78.00, 133.60, 68.00
Plot 5 / 8	22.86, 107.55, 179.29	78.00, 133.60, 88.00
Plot 6 / 8	82.86, 107.55, 179.29	78.00, 133.60, 88.00
Plot 7 / 8	22.86, 107.55, 199.29	78.00, 133.60, 108.00
Plot 8 / 8	82.86, 107.55, 199.29	78.00, 133.60, 108.00
Plot 9 / 8	22.86, 107.55, 219.29	78.00, 133.60, 128.00
Porometer / 1	41.86, -5.50, 208.29	78.00, 19.50, 117.00
Porometer - Backup 1 / 1	63.86, -5.50, 208.29	78.00, 19.50, 117.00
Porometer - Backup 2 / 2	41.86, 10.75, 269.29	78.00, 35.75, 178.00
Root analyzer / 1	41.86, -5.50, 174.29	78.00, 19.50, 83.00
Root analyzer - Backup 1 / 1	63.86, -5.50, 174.29	78.00, 19.50, 83.00
Root analyzer - Backup 2 / 2	41.86, 10.75, 283.29	78.00, 35.75, 192.00
Seed bank / -1	41.86, -38.00, 140.29	78.00, -13.00, 49.00
Seed bank - Backup 1 / -1	63.86, -38.00, 140.29	78.00, -13.00, 49.00
Seed bank - Backup 2 / 0	41.86, -21.75, 297.29	78.00, 3.25, 206.00
Spectrograph / 1	41.86, -5.50, 157.29	78.00, 19.50, 66.00
Spectrograph - Backup 1 / 1	63.86, -5.50, 157.29	78.00, 19.50, 66.00
Spectrograph - Backup 2 / 2	41.86, 10.75, 297.29	78.00, 35.75, 206.00
Split-root tray / 0	41.86, -21.75, 157.29	78.00, 3.25, 66.00
Split-root tray - Backup 1 / 0	63.86, -21.75, 157.29	78.00, 3.25, 66.00

Station / floor	Actual position	Stored approach
Split-root tray - Backup 2 / 1	41.86, -5.50, 283.29	78.00, 19.50, 192.00
Symbiosis incubator / 1	41.86, -5.50, 225.29	78.00, 19.50, 134.00
Symbiosis incubator - Backup 1 / 1	63.86, -5.50, 225.29	78.00, 19.50, 134.00
Symbiosis incubator - Backup 2 / 2	41.86, 10.75, 311.29	78.00, 35.75, 220.00
UV chamber / 1	41.86, -5.50, 191.29	78.00, 19.50, 100.00
UV chamber - Backup 1 / 1	63.86, -5.50, 191.29	78.00, 19.50, 100.00
UV chamber - Backup 2 / 2	41.86, 10.75, 325.29	78.00, 35.75, 234.00
Vitality scanner / 0	41.86, -21.75, 191.29	78.00, 3.25, 100.00
Vitality scanner - Backup 1 / 0	63.86, -21.75, 191.29	78.00, 3.25, 100.00
Vitality scanner - Backup 2 / 1	41.86, -5.50, 297.29	78.00, 19.50, 206.00
Wet lab / -1	41.86, -38.00, 191.29	78.00, -13.00, 100.00
Wet lab - Backup 1 / -1	63.86, -38.00, 191.29	78.00, -13.00, 100.00
Wet lab - Backup 2 / 0	41.86, -21.75, 311.29	78.00, 3.25, 220.00

23.3 Important tag counts

Tag	Count
AutoDoor	162
Automatable	313
EmergencyKit	116
GasPipe	18
LabBreakableWindow	130
LabDecor	14
LabEquipment	872
LabPoster	72
LabToolDispenser	90
LabWorkerBot	180
PlantGrowthChamber	450
RoleKiosk	2
RoomMonitor	81
SecretBasementPanel	1

23.4 Latest exported files

The export manifest contains 46 files totaling 197,533,070 bytes. The RBXL copy is included; OBJ/MTL and textures remain at D:/game/5 and are identified by SHA-256 in the manifest.

Relative export file	Bytes
obj\Casterwheel1_diff.png	3,268
obj\Casterwheel1_nmap.png	1,398
obj\Casterwheel1_spec.png	224
obj\Corridoreast71_diff.png	784,015
obj\Corridoreast71_nmap.png	2,002,950
obj\Corridoreast71_spec.png	1,885
obj\Flag1_diff.png	9,168

Relative export file	Bytes
obj\Flag1_nmap.png	21,313
obj\Flag1_spec.png	407
obj\Floor1_diff.png	6,199
obj\Floor1_nmap.png	6,143
obj\Floor1_spec.png	1,885
obj\Head2_diff.png	1,053
obj\LaboratoryTechnicianpanelframe1_diff.png	16,189
obj\LaboratoryTechnicianpanelframe1_nmap.png	23,842
obj\LaboratoryTechnicianpanelframe1_spec.png	871
obj\Loungehatch1_nmap.png	4,522
obj\Loungehatch1_spec.png	223
obj\obj.mtl	105,560
obj\obj.obj	193,872,256
obj\Roof11_diff.png	10,348
obj\Roof11_nmap.png	15,663
obj\Roof11_spec.png	1,885
obj\Sitepad81_diff.png	2,356
obj\Sitepad81_nmap.png	2,007
obj\Sitepad81_spec.png	1,885
obj\Soil1_diff.png	2,096
obj\Soil1_nmap.png	1,938
obj\Soil1_spec.png	1,885
obj\Spawnlocation1_diff.png	33,475
obj\Spawnlocation1_nmap.png	46,508
obj\Spawnlocation2_diff.png	1,322
obj\Tree3trunk1_diff.png	4,234
obj\Tree3trunk1_nmap.png	936
obj\Tree3trunk1_spec.png	1,885
obj\Tsouthglass31_diff.png	17,625
obj\Tsouthglass31_nmap.png	1,512
obj\Tsouthglass31_spec.png	120
obj\Walle1_diff.png	2,409
obj\Walle1_nmap.png	1,767
obj\Walle1_spec.png	120
obj\West2_diff.png	900
obj\Workerbot04desktop1_diff.png	19,051
obj\Workerbot04desktop1_nmap.png	5,889
obj\Workerbot04desktop1_spec.png	407
rblx\rbxl.rbxl	491,476

23.5 Coordinate repair decision

Do not automatically translate the laboratory back by the measured offset. The current position may be intentional. First establish the desired world placement with the owner or scene baseline, then make navigation metadata follow that placement. The safer architecture uses local coordinates and moving attachments so subsequent translations do not repeat this defect.

Chapter 24

Source provenance and complete-code map

The complete final active sources are printed in full, together with the server-only documentation module, eight Studio archive sources, one disabled generator, and 12 historical workspace artifacts. The pre-revision active sources remain in code/baseline for exact comparison. The printed appendices deliberately retain duplicate and conflicting versions so the history is inspectable; they are grouped by authority. Each listing restarts line numbering at one. The source ZIP contains byte-preserved originals. Proposed code is kept separately and never labeled active.

The complete latest place is supplied as evidence/export-5.rbxl. Geometry meshes and textures are not Lua code and are listed separately. Unrelated downloaded script repositories are not included as active game source. No implementation is shortened with ellipses or pseudocode inside a captured listing.

ID	Artifact	Lines / bytes
S01	ReplicatedStorage.GWAPNPC	96 / 6183
S02	ReplicatedStorage.GWAPRouting	55 / 3434
S03	ReplicatedStorage.GWAPScenarios	23 / 2692
S04	ReplicatedStorage.GWAPWorld	95 / 7198
S05	ReplicatedStorage.LabKinds	173 / 10408
S06	ReplicatedStorage.Modules.Blaster	239 / 9143
S07	ReplicatedStorage.Modules.BlasterController	915 / 35949
S08	ServerScriptService.GWAPServer	469 / 32199
S09	ServerScriptService.GWAPWallDamage	55 / 3254
S10	ServerScriptService.LabElevators	132 / 4405
S11	ServerScriptService.LabEquipment	4178 / 182838
S12	ServerScriptService.LabRoles	431 / 19385
S13	ServerScriptService.LabRound	94 / 3591
S14	ServerScriptService.LabTechnicians	210 / 7915
S15	ServerScriptService.LoungeSystems	172 / 5872
S16	ServerScriptService.ServerBlasterManager	195 / 8483
S17	ServerStorage.BeforeBasementAndQuiz_20260912.LabElevators	123 / 3761
S18	ServerStorage.BeforeBasementAndQuiz_20260912.LabEquipment	4068 / 175504
S19	ServerStorage.BeforeBasementAndQuiz_20260912.LabRoles	422 / 18757
S20	ServerStorage.BeforeBasementAndQuiz_20260912.LabRound	85 / 2954

ID	Artifact	Lines / bytes
S21	ServerStorage.BeforeBasementAndQuiz_20260912.LabTechnicians	201 / 7271
S22	ServerStorage.BeforeBasementAndQuiz_20260912.LoungeSystems	152 / 4605
S23	ServerStorage.BeforeBasementAndQuiz_20260912.ServerBlasterManager	175 / 7275
S24	ServerStorage.BlasterController_BeforeMouseArmFix_20260912	730 / 26610
S25	StarterPlayer.StarterPlayerScripts.LabHUD	140 / 12852
S26	StarterPlayer.StarterPlayerScripts.LabQuizClient	138 / 5319
S27	StarterPlayer.StarterPlayerScripts.LabSidearmClient	54 / 2409
S28	StarterPlayer.StarterPlayerScripts.LoungeClient	85 / 3247
S29	Workspace.SeedLabGenerator_DISABLED_doNotEnable	11886 / 524207
W01	SeedLabGenerator.lua	12398 / 552155
W02	InstallGWAP.lua	2109 / 115778
W03	GWAPWorld.lua	259 / 17807
W04	GWAPScenarios.lua	13 / 2067
W05	GWAPNPC.lua	90 / 6021
W06	GWAPRouting.lua	43 / 2594
W07	GWAPServer.lua	418 / 28562
W08	GWAPWallDamage.lua	45 / 2594
W09	GWAPHUD.lua	123 / 11596
W10	Blaster.lua	227 / 8432
W11	BlasterController.lua	905 / 35188
W12	ServerBlasterManager.lua	184 / 7714
S21	ReplicatedStorage.LabSpatial	39 / 1994
DOC01	ServerStorage.GameDocumentation	827 / 120178
DOC02	ServerStorage.GameDocumentationAnnex2	2159 / 107451

24.1 Hash verification

24.1.1 S01 GWAPNPC

Group: active; file: code/active/ReplicatedStorage.GWAPNPC.lua

c9698d535e3d7947acf208b4f8ee49f2ad08c853badfd573fa7173a1435f5d15

24.1.2 S02 GWAPRouting

Group: active; file: code/active/ReplicatedStorage.GWAPRouting.lua

f29a41157ff14934f5a25ac7e9e065450f3a32dc13c868a57378e5d73e80e8ee

24.1.3 S03 GWAPScenarios

Group: active; file: code/active/ReplicatedStorage.GWAPScenarios.lua

6a6d613711d1cd7762fc1140ef53c6c3ff25d1e5faea06538d32f8aa79452796

24.1.4 S04 GWAPWorld

Group: active; file: code/active/ReplicatedStorage.GWAPWorld.lua

2b7829521e74564f068cb147223e063e5b680cf3cd44f87e128b357d607cf19c

24.1.5 S05 LabKinds

Group: active; file: code/active/ReplicatedStorage.LabKinds.lua

eb2dbeb766efed328eb3e1165c9b41e93546c3fa49615fdb00646d43aa9dea2

24.1.6 S06 Blaster

Group: active; file: code/active/ReplicatedStorage.Modules.Blaster.lua

616d0fc7234636c563afbc93798f81e1be3a9fbc83084de44196f4d0ed8be2ab

24.1.7 S07 BlasterController

Group: active; file: code/active/ReplicatedStorage.Modules.BlasterController.lua

ef4a80c0aa4c3a1ded46c55bef82d058ddb29bb83dd8f34cfa73d22c9647769d

24.1.8 S08 GWAPServer

Group: active; file: code/active/ServerScriptService.GWAPServer.lua

265a4f246bfd1c1b107cc4ae2df6cf8babde7c682de18838cfba553c55c7c653

24.1.9 S09 GWAPWallDamage

Group: active; file: code/active/ServerScriptService.GWAPWallDamage.lua

92a3df3c35188633291afa9388a99db7ed9791f34c1ef14a299301c46a96f952

24.1.10 S10 LabElevators

Group: active; file: code/active/ServerScriptService.LabElevators.lua

d5c42a7d4217033660c71b966b4d762409572f63640db28769f0ccf3bb32ae44

24.1.11 S11 LabEquipment

Group: active; file: code/active/ServerScriptService.LabEquipment.lua

eb4722cec0bddcf396cb9dce7199a4b11dc3890fc157367daa219c1489fdae5f

24.1.12 S12 LabRoles

Group: active; file: code/active/ServerScriptService.LabRoles.lua

7e706724edacbdaccd335320891a5c8652a016640abe3e7fda6754203a39a77

24.1.13 S13 LabRound

Group: active; file: code/active/ServerScriptService.LabRound.lua

ed2bd7811d3301d2afc164a9b6bd3c603b1758e760a8c5b4b31006c064946244

24.1.14 S14 LabTechnicians

Group: active; file: code/active/ServerScriptService.LabTechnicians.lua

cf7bd8c892576d90388829db70c3a77c9d7519807ab7785940dc472af7f0da85

24.1.15 S15 LoungeSystems

Group: active; file: code/active/ServerScriptService.LoungeSystems.lua

78327a7e0eb4d5ce209c2b9d0a504708d378917a727d0ce530f144d84e1c2b42

24.1.16 S16 ServerBlasterManager

Group: active; file: code/active/ServerScriptService.ServerBlasterManager.lua

50671031f74ebbfede5259e3447b34087fc2a47387ffb378de169616a1be522

24.1.17 S17 LabElevators

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LabElevators.lua

62b29049771761ea23fcf71ce84e715ea44f0945e4e3f81a078167d37b15f448

24.1.18 S18 LabEquipment

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LabEquipment.lua

d7ffee0ffe6bf5f1a3d4f1086a5569adb7aad76c3dc66d5cdeed5959cf9b4d08

24.1.19 S19 LabRoles

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LabRoles.lua

9b4d4429d279f97793519a441fd8c4f4d6f7e6af9c934e270aedb56305fc62dd

24.1.20 S20 LabRound

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LabRound.lua

9ef5725f5f034aec29be6daaf9864ad6ce2e08c3f854e61c035ba286c197a5c1

24.1.21 S21 LabTechnicians

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LabTechnicians.lua

5c9a649cef010ed65b142db221fd92830593c0ee8c4281f5c8606b8ede66f01c

24.1.22 S22 LoungeSystems

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.LoungeSystems.lua

72f8a9e545a7818dc813e749aaabea27d3de773599005f2cf5ac6690e6b24615

24.1.23 S23 ServerBlasterManager

Group: archive; file: code/archive/ServerStorage.BeforeBasementAndQuiz_20260912.ServerBlasterManager.lua

e38caa71d7c2f94536a0f0273248d77bc6a5df585d480bb693f1e41f58c4d74a

24.1.24 S24 BlasterController_BeforeMouseArmFix_20260912

Group: archive; file: code/archive/ServerStorage.BlasterController_BeforeMouseArmFix_20260912.lua

8a756113e948711c647a10a790ec624081297515dc3ef9504c03dcaf692135e8

24.1.25 S25 LabHUD

Group: active; file: code/active/StarterPlayer.StarterPlayerScripts.LabHUD.lua

d8982e721a569a2eaa70a0f73fd7106e78cf19f39b056a76ced59329602f8a0a

24.1.26 S26 LabQuizClient

Group: active; file: code/active/StarterPlayer.StarterPlayerScripts.LabQuizClient.lua

316259614108e07c7273dd92fe632343fb797c988877cf71694726e59380fc57

24.1.27 S27 LabSidearmClient

Group: active; file: code/active/StarterPlayer.StarterPlayerScripts.LabSidearmClient.lua

a2e10adb43696aef2a6fa8d7aa8570d5a64d09d9cae9f32398ce6e944b52843

24.1.28 S28 LoungeClient

Group: active; file: code/active/StarterPlayer.StarterPlayerScripts.LoungeClient.lua

eb857c214587168a13cd60243327a4b8588fdfe48a3947d20c1501393bf6bc45

24.1.29 S29 SeedLabGenerator_DISABLED_doNotEnable

Group: disabled; file: code/disabled/Workspace.SeedLabGenerator_DISABLED_doNotEnable.lua

dc633d385d37c97f171de5725a0523f12beaa19df26006138968f14f5a545e5d

24.1.30 W01 lua

Group: workspace; file: code/workspace/SeedLabGenerator.lua

219d03a12736586ae1fbe39cc89c7612f9134819807d0ecc60450bbb487f55af

24.1.31 W02 lua

Group: workspace; file: code/workspace/InstallGWAP.lua

06e6d896bb0fe9fdf14c04ac0810c10f79219372831abb352a3f08982bdca511

24.1.32 W03 lua

Group: workspace; file: code/workspace/GWAPWorld.lua

1745af08772025f20c0fea0e44532d25244454346e7cd6b2f22d0b41dd09e7c5

24.1.33 W04 lua

Group: workspace; file: code/workspace/GWAPScenarios.lua

d27afc6b0edf955031d55ec0f910394c2d98ede64daeed7b7bc27ae634442136

24.1.34 W05 lua

Group: workspace; file: code/workspace/GWAPNPC.lua

77bd356a1d0fbff2b71c3ec24eaecf99565a6304b6f1e478d1b257b89007ed05

24.1.35 W06 lua

Group: workspace; file: code/workspace/GWAPRouting.lua

b41a223e553d10f13e0d6a79c3ef2b616e4cc0d43cfc986aa75c84e3af6186ca

24.1.36 W07 lua

Group: workspace; file: code/workspace/GWAPServer.lua

3546a88f457530f2e6a7efb7ebea5ad3f4bfda3502b3b97a6368750517fe8693

24.1.37 W08 lua

Group: workspace; file: code/workspace/GWAPWallDamage.lua

cae9ce8d81cdd3261702f39670c8bf456628f065c3454c638757d3d61d4e577b

24.1.38 W09 lua

Group: workspace; file: code/workspace/GWAPHUD.lua

20b2619e281c1608e9c543ec522e7c8e51a9e1c69eb0e73d4d9cc54105aa6ebd

24.1.39 W10 lua

Group: workspace; file: code/workspace/Blaster.lua

01c2f0f4432c5673e1bb39cd3ced41e9a984c476a31755010bf1ad207567809c

24.1.40 W11 lua

Group: workspace; file: code/workspace/BlasterController.lua

4dc3893c33e70e7e9a39194b52bb937fd3086f212efeadc371f73c71e453fcea

24.1.41 W12 lua

Group: workspace; file: code/workspace/ServerBlasterManager.lua

6f5301dddadf2663b5f910ac85fdc691c6a9c23e77aab31947625053d402d449

24.1.42 S21 LabSpatial

Group: active; file: code/active/ReplicatedStorage.LabSpatial.lua

1593b910e0f231f347b2fec79a28c1a3da1c7aca53c273d968ab44f7dec6df5f

24.1.43 DOC01 GameDocumentation

Group: documentation; file: code/documentation/ServerStorage.GameDocumentation.lua

c8cdc2005e233b55a583bb47e5baf3f3994ed4234134370d8fa1a6157a8884b9

24.1.44 DOC02 GameDocumentationAnnex2

Group: documentation; file: code/documentation/ServerStorage.GameDocumentationAnnex2.lua

c29a83b5a539cd778db9899424d50426c1cc4b44c7c3a10191d246c31a98f3c1

24.2 Normalized workspace comparisons

Workspace file	Live ID	Comparison
GWAPWorld.lua	S04	DIFFERENT - inspect before installation
GWAPScenarios.lua	S03	DIFFERENT - inspect before installation
GWAPNPC.lua	S01	DIFFERENT - inspect before installation

Workspace file	Live ID	Comparison
GWAPRouting.lua	S02	DIFFERENT - inspect before installation
GWAPServer.lua	S08	DIFFERENT - inspect before installation
GWAPWallDamage.lua	S09	DIFFERENT - inspect before installation
GWAPHUD.lua	S25	DIFFERENT - inspect before installation
Blaster.lua	S06	DIFFERENT - inspect before installation
BlasterController.lua	S07	DIFFERENT - inspect before installation
ServerBlasterManager.lua	S16	DIFFERENT - inspect before installation

This comparison ignores UTF-8 BOM, line-ending normalization by text reading, and trailing newlines only. The raw hashes remain different when bytes differ. A normalized match is not a gameplay test. The installer contains the incorrect loose GWAPWorld source in its world-module block; it must not be run unchanged. The active module is the documentary source for a future repair.

Chapter 25

Active-code symbol and attribute index

This index is generated from source text to help locate implementation. Line numbers refer to the complete listings and raw files. Regex extraction may omit anonymous functions, multiline definitions, dynamic attribute names, or accesses assembled at runtime. It is a navigation aid, not an exhaustive static type analysis.

25.1 S01 GWAPNPC

25.1.1 Named functions / assigned handlers

Line	Symbol
18	N.gun
28	N.appearance
33	piece
41	N.say
49	N.nearPlayer
66	N.animate

25.1.2 Literal attribute keys and access lines

Attribute	Lines
Gender	29
HasSeedling	89
InventoryName	25
MissionMessage	45
MissionMessageAt	46
NPCGun	25

25.2 S02 GWAPRouting

25.2.1 Named functions / assigned handlers

Line	Symbol
13	floorOf
18	R.healthy
21	R.ensureBackups
47	R.choose

25.2.2 Literal attribute keys and access lines

Attribute	Lines
Backup	25, 31
BlastRemoved	19
Capability	27, 31, 49
Exploded	19, 38
Floor	14, 37
FunctionalState	19, 38
Level	15
Planted	26, 49

25.3 S03 GWAPScenarios

25.4 S04 GWAPWorld

25.4.1 Named functions / assigned handlers

Line	Symbol
14	part
19	label
23	W.plant
31	W.build

25.4.2 Literal attribute keys and access lines

Attribute	Lines
Approach	46, 85
Floor	46, 85
GWAPOriginalCanCollide	60, 65
Planted	85

25.5 S05 LabKinds

25.5.1 Named functions / assigned handlers

Line	Symbol
28	f
29	rnd
33	log
35	log
37	log
39	log
42	log
44	log
48	log
50	log
53	log
56	log
59	log
62	log
64	log
67	log
72	log
74	log
76	log
78	log
80	log
83	log
85	log
88	log
90	log
92	log
94	log
96	log
98	log
100	log
102	log
104	log
106	log
108	log
110	log
112	log
116	log
119	log
122	log
124	log
126	log
128	log
130	log
132	log
134	log
136	log
138	log
140	log
149	log

Line	Symbol
151	log
161	log
163	log
165	log
169	log
171	K.forKind

25.6 S06 Blaster

25.6.1 Named functions / assigned handlers

Line	Symbol
27	finiteNumber
34	Blaster.IsFiniteVector
41	numberAttribute
47	Blaster.GetWeaponConfig
62	Blaster.GetCameraRay
83	Blaster.NewRaycastParams
91	Blaster.CastFromCamera
107	Blaster.GetAimSolution
137	tracerColor
142	Blaster.DrawTracer
163	Blaster.DrawImpact
182	getRemotes
190	Blaster.StartReplication
206	Blaster.Fire

25.6.2 Literal attribute keys and access lines

Attribute	Lines
BlasterAllowPlayerDamage	58
BlasterAutomatic	56
BlasterInstantHeadshot	57
BlasterTracerColor	138

25.7 S07 BlasterController

25.7.1 Named functions / assigned handlers

Line	Symbol
33	touchControlsActive
59	pointInside

Line	Symbol
68	touchAimActive
82	rotationBetween
93	makeCrosshair
165	BlasterController:_ensureHUD
176	BlasterController.new
232	BlasterController:_button
239	BlasterController:_fallbackTouchButton
281	BlasterController:_styleTouchButtons
337	place
351	BlasterController:_getAimScreenPoint
372	BlasterController:_updateAimReticle
414	BlasterController:_setLocalHeadHidden
434	BlasterController:_beginTouchCamera
451	BlasterController:_endTouchCamera
463	BlasterController:SetAiming
505	BlasterController:_fireOnce
520	BlasterController:_beginFiring
536	BlasterController:_shootAction
561	BlasterController:_aimAction
576	BlasterController:_touchPanAction
623	BlasterController:_updateArmPose
685	BlasterController:_render
737	BlasterController:Enable
821	BlasterController:Disable
865	BlasterController:Destroy
875	BlasterController.BindTool
883	BlasterController.GetController
887	BlasterController.AutoBind
890	consider
896	watch
907	binder:Destroy

25.7.2 Literal attribute keys and access lines

Attribute	Lines
BlasterADSTouchSensitivity	607
BlasterBarrelDirection	661
BlasterHipFOV	746
BlasterHipTouchSensitivity	608
BlasterShoulderOffsetX	758
BlasterShoulderOffsetY	759
BlasterWeapon	891
LabSidearm	664, 891

25.8 S08 GWAPServer

25.8.1 Named functions / assigned handlers

Line	Symbol
20	refreshArmed
37	alive
40	points
43	award
47	role
48	active
49	nearby
50	publish
56	tell
60	finish
67	updateHealth
75	damage
89	remember
95	repair
102	explode
147	bridge.OnInvoke
152	routePoints
153	add
159	flight
172	showRoute
177	go
248	plantIn
258	checkpoint
263	runMission
312	startMission
321	makeBot
337	companion
349	choose
385	setArmed
406	watchPlayer
407	clean
408	watch
410	onCharacter

25.8.2 Literal attribute keys and access lines

Attribute	Lines
BlastRemoved	123, 138
BotNumber	399
Capability	30, 268, 272
CarriedSample	382
Dead	331, 389, 429, 439

Attribute	Lines
DisplayName	58, 345
Exploded	96, 103, 104, 123, 138, 142, 249
Floor	156
FunctionalState	96, 104
GWAPArmed	390, 397
GWAPArmedCount	402
GWAPAttacker	109, 135
GWAPBotCount	402
GWAPCarrier	331, 345, 356
GWAPDialogue	58, 344, 345
GWAPEquipment	92
GWAPFractionalLoss	453, 454, 464
GWAPHitAt	455
GWAPNotice	45, 78, 129, 351, 352, 355, 356, 381, 382, 455
GWAPScenario	356
GWAPSpeaker	58, 345
GWAPStage	52
GWAPState	53
GWAPTarget	53
GWAPTargetFloor	53
GWAPTotalStages	52
GWAPVariant	356
GWAPWeapon	407
Gender	324
HasSeedling	62, 305
Health	69, 77, 316, 389, 429, 430, 439
KnockedOut	331, 389, 429, 439
LabRole	47, 417
LastPointAmount	45
LastPointAward	45
PlantSample	380
Planted	250, 253, 466
Reroutes	281
RoundNumber	462
RoundPhase	350, 436
RouteIndex	202
Scenario	316
Seated	324
State	48, 53, 62, 316, 336
TaskState	62, 285, 294
Title	71, 316
TransferSample	380
Variant	316
WalkSpeed	219

25.9 S09 GWAPWallDamage

25.9.1 Named functions / assigned handlers

Line	Symbol
16	register

25.9.2 Literal attribute keys and access lines

Attribute	Lines
BlastRemoved	47
Exploded	47
FunctionalState	47
GWAPEquipment	19
GWAPRemovedFixtures	53
ShootablePoster	18
ToolKind	18

25.10 S10 LabElevators

25.10.1 Named functions / assigned handlers

Line	Symbol
21	setGate
26	setup
55	occupants
73	moveTo
109	addPrompt

25.10.2 Literal attribute keys and access lines

Attribute	Lines
CurrentFloor	104
Floor	41, 121, 124
FloorCount	27
FloorLabel_	125
FloorY_	45

25.11 S11 LabEquipment

25.11.1 Named functions / assigned handlers

Line	Symbol
53	updateScoreboard
73	refreshRoomMonitors
105	addContribution
112	notify
167	playerRole
176	awardPoints
198	canInteract
227	runMathQuiz
268	addWeldedPart
290	equipTool
299	makeTool
406	updateLifeBar
427	playerContainers
435	findTool
449	findTransferSample
460	findHeldSample
478	findPlantSample
487	treatmentText
493	hasTreatment
498	addTreatment
505	refreshPlantVisual
530	setCarryState
550	destroyCarriedItem
605	refreshBotHealth
625	damageWorkerBot
714	equipmentFromHit
728	explodeEquipment
781	detonateGasPipe
880	spawnImpact
921	punchPoster
945	workerBotFromHit
950	stealHeldSample
974	armImpactTool
1088	findImpactTool
1099	makeImpactTool
1193	bindDispenser
1342	blasterBridge.OnInvoke
1399	equipmentFloorLabel
1403	equipmentId
1409	equipmentLabel
1415	adaptiveTransferSeconds
1477	storageId
1486	storageLabel
1492	coldStoresExcept
1506	makeControls
1604	clockText
1608	renderTimer
1709	setReadout
1715	pressButton

Line	Symbol
1731	runProgress
1743	chamberSampleList
1749	saveChamberSamples
1758	formatRuntime
1763	recoverPlantLife
1793	updateStoredSamples
1806	refreshChamberTelemetry
1843	evacuationDestinations
1864	repairPathFolder
1883	clearRepairPath
1888	repairReward
1895	createRepairPath
1902	approachPoint
1910	appendNavigable
2007	clearAssignment
2014	assignNextTask
2091	chamberByCode
2097	takePlantFromChamber
2138	canFinishTaking
2211	storePlantInChamber
2286	handleGrowthChamber
2326	germinationPercent
2332	processPlant
2452	createTransferSample
2502	handleColdStore
2527	startCycleIndicator
2567	bind
2659	canFinishRepair
2810	setBotStatus
2818	updateBotTestBoard
2846	interactionPoint
2892	roomAt
2907	roomRect
2914	botHasObserver
2935	botMotionChecks
2951	clear
2979	grounded
2989	walkBotSegment
3038	moveBotTo
3059	makeBotPlant
3066	piece
3087	equipmentReadout
3095	candidatesOnLevel
3105	runBotScenario
3264	setChamberFailure
3315	setGenericFailure
3348	shuffled
3356	adaptiveFailureTick
3441	genericFailureTick

Line	Symbol
3465	bindPlayerTaskRouting
3496	bindAutoDoor
3507	setIndicatorColour
3516	setOpen
3574	bindEmergencyKit
3591	setCharged
3620	attachPlayerHealthBar
3658	refresh
3671	watchPlayerHealth
3683	bindSecretBasementPanel
3720	nearestEmergencyKit
3802	scatterDebris
3827	warnEquipmentCatastrophe
3862	destroyEquipment
4010	resetHunt
4046	clearRoomSmoke
4056	fillRoomWithSmoke
4087	ventFromDoor
4101	watchDoor
4152	bindNPC

25.11.2 Literal attribute keys and access lines

Attribute	Lines
AboutToBlow	3828, 3829, 3868
ActiveNPCWorkers	3434
ActiveNPCWorkersPerFloor	3437
ActivePlayerCount	3433
AdaptiveFailureTargetPercent	3428
AssignedRepairDistance	1890, 2039
AssignedRepairId	2037, 2671, 3285, 3332, 3474, 3741
AssignedRepairProjectedPoints	2041
AssignedTransferDeadline	2074, 2197, 2198, 2235
AssignedTransferDestination	2073, 2110, 2196, 2234
AssignedTransferSource	2072, 2109, 2195, 2233, 3286, 3741
Auto	2528
AvoidanceDetours	3010
BlasterADSFOV	1130
BlasterADSTouchSensitivity	1131
BlasterAutomatic	1129
BlasterCooldown	1124
BlasterDamage	1122
BlasterInstantHeadshot	1128
BlasterPellets	1125
BlasterRange	1123
BlasterSelfCostPercent	1127
BlasterSpread	1126

Attribute	Lines
BlasterTracerColor	1132
BlasterWeapon	1121
Blown	782, 783
BotNumber	3005, 3203, 3991
BotTestUses	3207
BotsKilled	707, 1324, 1363, 4015
BotsTotal	1325, 1364
BreakableWindow	1347
Capacity	1511, 1635, 1750, 1816, 1847, 2115, 2145, 2206, 2224, 2570, 3215, 3328, 3343, 3362, 3412
CarriedSample	538, 545, 551, 2183
CarryingPlant	2824, 3195, 3242
ChamberStatusLamp	1834
Charged	3592, 3599
Competitor	61, 2825, 3022, 3120, 3174, 3223
CompletedCycles	2822, 3222
Dead	630, 648, 760, 862, 1297, 1301, 1358, 1360, 2990, 3021, 3022, 3023, 3043, 3113, 3117, 4019, 4024
DestinationGrowthCode	524, 537, 2180, 2229
DestinationLabel	537, 2455, 2512
DestinationStorageId	2510
DispenserBound	1194, 1197
DisplayName	2814
DoorBound	3497, 3498
DoorClosedX	3499
DoorClosedZ	3500
DoorOpen	3529, 4102, 4103
DoorOpenX	3501
DoorOpenZ	3502
DoorY	3503
EmergencyMovesCompleted	3431
EmergencyTransfers	2253
EquipmentId	2572
EquipmentTaskRoutingBound	3466, 3467
EvacuationDestination	1812, 3183, 3283, 3298, 3409
Exploded	729, 730, 833
FailedSampleBacklog	1419, 3430
FailingChambers	3429
FailureReason	1813, 2203, 2580, 2657, 2691, 2740, 3277, 3296, 3324, 3328, 3338
FailureSince	1612, 1889, 2029, 2135, 2581, 3278, 3297, 3325, 3339, 3395, 3460, 3952
FloorLabel	1400, 1699, 1809, 2573, 3271
FloorMaxLevel	1932
FloorMinLevel	1931
FloorRiseStuds	1926

Attribute	Lines
FunctionalState	86, 731, 1609, 1765, 1783, 1807, 1849, 2020, 2050, 2106, 2113, 2141, 2143, 2218, 2575, 2579, 2636, 2639, 2641, 2651, 2661, 2667, 2690, 2739, 2830, 3128, 3215, 3265, 3273, 3294, 3316, 3322, 3337, 3364, 3367, 3404, 3410, 3445, 3447, 3867, 3946
GasPipe	847, 1302, 1369
Genome	2158, 2244, 2376, 2427, 2436
GermAbnormalCount	2411
GermDeadCount	2414
GermNormalCount	2329, 2409
GermSownCount	2327, 2407
GerminationDate	1817, 2160, 3420
GerminationOutcome	2406, 2437, 2438
GrowthCode	1404, 1410, 1699, 1776, 1808, 2058, 2059, 2068, 2069, 2093, 2098, 2132, 2212, 2287, 3157, 3158, 3183, 3185, 3197, 3212, 3269, 3270, 3283, 3409, 3418
GrowthStage	506, 1795, 2131, 2241, 2242, 2345, 2425, 3193, 3218
Health	606, 631, 632, 4022
HitLogicBound	975, 976
Holes	922, 924
HomeCFrame	702, 3107, 3108, 4020
HomeGrowthCode	2232
HormoneFocus	2383, 2444
HormoneTreatment	2382, 2443, 2444
Humidity	1819, 3280, 3300, 3406
ImpactToolKind	979, 1092, 1118, 1247
IncomingReservations	1647, 1848, 1856, 1857, 2114, 2144, 2182, 2193, 2250, 3182, 3244, 3267, 3318, 3371, 3387
InteractionsBound	2569, 2571
Kind	1412, 1488, 1495, 2341, 2473, 2568, 3127, 3201, 3204, 3444, 3893, 3948, 3957
KitBound	3575, 3576
KnockedOut	649, 3021, 3124, 4023
LabImpactTool	1119
LabItemType	309, 440
LabObjective	540, 542, 546, 1775, 2034, 2045, 2065, 2083, 2087, 2184, 2200, 2205, 2430
LabRole	168, 3468
LabSample	310, 464, 2794
LabSidearm	1134, 1246
LastActionDetail	1711
LastActionStatus	1710
LastHitBy	633
LastPointAmount	191
LastPointAward	192
LastRepairedAt	2673, 3368, 3386, 3448
LastRepairedBy	2674

Attribute	Lines
Level	80, 96, 1400, 1406, 1480, 1489, 1699, 1809, 1851, 2917, 3098, 3125, 3271
LevelSlot	4001
MathQuizActive	236, 247
MaxHealth	606, 607, 4022
MessageSerial	158, 159, 163
Moisture	2158, 2245, 2426, 2436
MutagenesisDoseGy	2365, 2440, 2441
MutagenesisSurvivalPercent	2366, 2398, 2441
NPCName	4158
NavigationBlocked	3015, 3018, 3029
Occupied	1627, 1754
OperationStarted	1613, 1818, 2583, 3302, 3341
PathFallbacks	2823, 3053
PlantExpiredAt	1629
PlantId	525, 1628, 1755, 2230, 2348, 2368, 2415, 2430, 2435, 3304
PlantLife	313, 407, 415, 507, 1623, 1658, 1766, 1773, 1784, 1786, 2133, 2150, 2171, 2176, 2177, 2231, 2412, 2728, 3274, 3303, 3304
PlantSample	481
Powered	2597
QuizPlayerId	237, 245, 3266, 3317, 3953, 3970
QuizProtected	2174
RecentTransferThroughput	3432
RecoveryEndsAt	1772, 1787
RecoveryToken	1770, 1771, 1782
RepairInstruction	732, 3276, 3295, 3326, 3340, 3869
RepairKey	2574, 3275, 3323
RepairRouteReady	1970, 1973
ReservedByBot	2052, 2288, 2292, 3130, 3181, 3189, 3245, 3268, 3388
Role	4158
Room	79, 95, 1406, 1412, 1480, 1489, 3170
RoomIndex	2875
RoomKey	4061
RoundNumber	4036
SampleCount	88, 1624, 1626, 1636, 1661, 1753, 1794, 1846, 1856, 1857, 2051, 2114, 2144, 2296, 2832, 3129, 3303, 3328, 3343, 3361, 3394
SampleId	536, 551, 963, 2456, 2458, 2509
SampleIds	1625, 1745, 1752
SampleSlot	1631, 1797, 1798
Score	61, 2825, 3225
Seated	3111
SecretBound	3684, 3685
SecretDestX	3695
SecretDestY	3696
SecretDestZ	3697
ShootablePoster	1309, 1382

Attribute	Lines
SourceGrowthCode	2232
StairEastLaneX	1928
StairNorthZ	1929
StairSouthZ	1930
StairWestLaneX	1927
StorageId	1478, 1481
StoredTransparency	1799
TalkBound	4153, 4154
TaskState	650, 2811, 4025
Temperature	1819, 3279, 3299, 3405
Tips	4155
ToolKind	1195
TransferSample	453, 534, 956
TransferStartedAt	2175
Treatments	488, 494, 500, 501, 2157, 2243
Ventilation	1820, 3281, 3301, 3407
WallSide	1507, 1903, 2614, 2847
WindowBroken	1348, 1349

25.12 S12 LabRoles

25.12.1 Named functions / assigned handlers

Line	Symbol
30	roleEquipmentId
36	roleFloor
40	roleTransferSeconds
49	roleEquipmentLabel
55	roleDestinations
72	clearRoleAssignment
78	assignRoleObjective
130	buildHud
216	refreshHud
243	wearablePart
265	applyAppearance
309	setupPlayer
344	stripAutoHealthRegen
369	bindKiosk
401	selectRole

25.12.2 Literal attribute keys and access lines

Attribute	Lines
AssignedRepairDistance	96
AssignedRepairId	92

Attribute	Lines
AssignedRepairProjectedPoints	97
AssignedTransferDeadline	122, 226, 360
AssignedTransferDestination	120
AssignedTransferSource	119
Capacity	61
CarriedSample	222, 312, 336
FailedSampleBacklog	44
FailureSince	88, 94
FloorLabel	37
FunctionalState	60, 83, 104
GrowthCode	31, 50, 111, 112, 117, 118
IncomingReservations	59
Kind	52
LabObjective	223, 313, 337, 416
LabRole	411
LastPointAmount	233
LastPointAward	234, 338
Level	33, 37, 64, 65
ReservedByBot	105
Role	279, 370
RoleKioskBound	372, 373
Room	33, 52
SampleCount	58, 67, 104

25.13 S13 LabRound

25.13.1 Named functions / assigned handlers

Line	Symbol
18	computeRoundSeconds
48	pointsOf
54	resetScores
62	resolveRound

25.13.2 Literal attribute keys and access lines

Attribute	Lines
LastRoundScore	69
LastWinnerName	71
LastWinnerScore	72
PersonalBest	67, 68, 93
RoundEndsAt	82, 87
RoundLength	46
RoundNumber	80
RoundPhase	81, 86

Attribute	Lines
RoundStreakBest	58

25.14 S14 LabTechnicians

25.14.1 Named functions / assigned handlers

Line	Symbol
46	bubbleOf
71	say
81	playSound
93	labelOf
97	nearestFault
111	setStatus
118	reviveLater
134	runTechnician

25.14.2 Literal attribute keys and access lines

Attribute	Lines
Fatigue	125, 154, 187
FunctionalState	100
Gender	139
GrowthCode	94
Kind	94
KnockedOut	128, 142
NeedsHelp	127, 159, 179
Panic	126, 158, 178

25.15 S15 LoungeSystems

25.15.1 Named functions / assigned handlers

Line	Symbol
28	move
53	refreshPanel
63	playNext
103	refreshWall
113	onChat
142	hook

25.15.2 Literal attribute keys and access lines

Attribute	Lines
Col	160
InLounge	33, 46, 114, 135
Row	159

25.16 S16 ServerBlasterManager

25.16.1 Named functions / assigned handlers

Line	Symbol
30	getRemote
60	findHumanoidFromHit
72	randomSpread
82	broadcastShot
95	researchHit
102	bridgeHit
114	genericDamage
127	validSequence
136	handleFire

25.16.2 Literal attribute keys and access lines

Attribute	Lines
BlasterTracerColor	163
BlasterWeapon	147
LabSidearm	147

25.17 S25 LabHUD

25.17.1 Named functions / assigned handlers

Line	Symbol
23	box
26	label
29	button
44	toggle
58	showTrail
80	watchBot

25.17.2 Literal attribute keys and access lines

Attribute	Lines
CarriedSample	126
GWAPCarrier	120
GWAPDialogue	125
GWAPNotice	126
GWAPScenario	120
GWAPTarget	123
GWAPTargetFloor	122
Health	124
LabRole	115
RoundEndsAt	117
RouteIndex	67
State	61

25.18 S26 LabQuizClient

25.18.1 Named functions / assigned handlers

Line	Symbol
51	label
84	answer

25.18.2 Literal attribute keys and access lines

Attribute	Lines
Answer	103, 121

25.19 S27 LabSidearmClient

25.19.1 Named functions / assigned handlers

Line	Symbol
29	refresh

25.19.2 Literal attribute keys and access lines

Attribute	Lines
BotsKilled	30, 40

Attribute	Lines
BotsTotal	31, 41

25.20 S28 LoungeClient

25.20.1 Named functions / assigned handlers

Line	Symbol
49	apply

25.20.2 Literal attribute keys and access lines

Attribute	Lines
InLounge	66, 67, 74, 85

25.21 S21 LabSpatial

25.21.1 Named functions / assigned handlers

Line	Symbol
7	S.frame
12	S.world
13	S.localPoint
14	S.vector
15	S.floor
18	S.setApproach
26	S.approach
32	S.sync

25.21.2 Literal attribute keys and access lines

Attribute	Lines
Approach	23, 29, 34, 35
ApproachLocal	19, 29

Chapter 26

Historical visual references



Figure 26.1: Original layout reference supplied by the owner. This is another shooter used as a panel-layout reference, not a screenshot of the audited laboratory.



Figure 26.2: Historical laboratory screenshot showing the rejected added weapon selector and competing overlays. Not the current audited state.



Figure 26.3: Historical hallway lineup of repeated NPCs, supplied as defect evidence. Later pooling and template changes address this presentation.

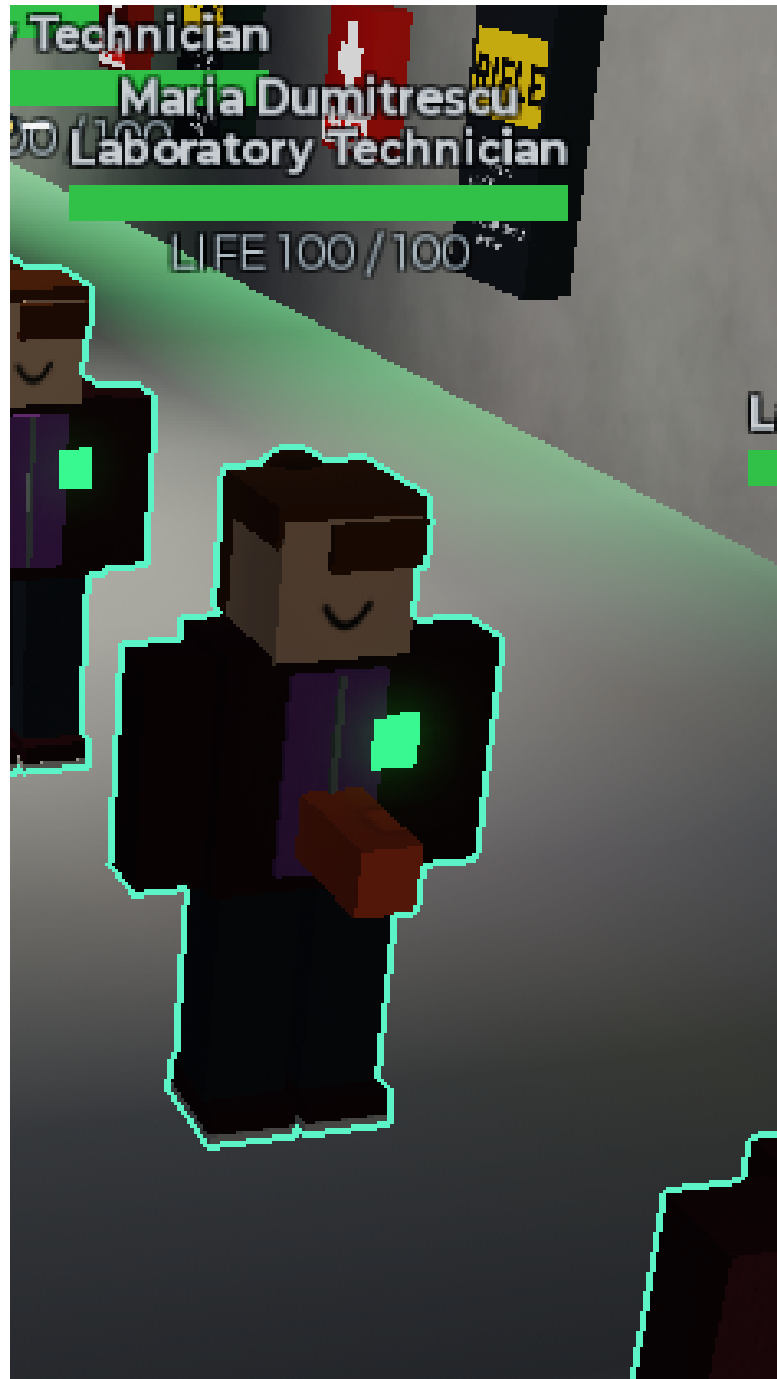


Figure 26.4: Historical NPC appearance and weapon-visual detail, supplied as defect evidence.



Figure 26.5: Historical small-screen view with competing mission, carrier, and role-selection information. Used to explain the grouping requirement.

Chapter 27

Bibliography and source access

27.1 Shared development chat

Improve SeedLab interactions; browser read on 13 September 2026. Historical reports and requirement chronology.

https://chatgpt.com/s/cx_6aa68aee04bc8191a98fa52b54b28109

27.2 Roblox client/server boundary

Primary guidance for server validation of client-triggered actions. Consulted 13 September 2026.

<https://create.roblox.com/docs/scripting/security/client-server-boundary>

27.3 Roblox Camera reference

Primary API reference for screen/camera rays and coordinate conventions. Consulted 13 September 2026.

<https://create.roblox.com/docs/reference/engine/classes/Camera>

27.4 Roblox pathfinding

Primary guidance for paths and navigation behavior. Consulted 13 September 2026.

<https://create.roblox.com/docs/characters/pathfinding>

27.5 Roblox UI position and size

Primary guidance for interface sizing and positioning. Consulted 13 September 2026.

<https://create.roblox.com/docs/ui/position-and-size>

27.6 Roblox text filtering

Primary guidance for filtering player-authored text displayed on shared signs. Relevant to open risk R07. Consulted 13 September 2026.

<https://create.roblox.com/docs/ui/text-filtering>

27.7 Local primary evidence

evidence/live-source-capture.json contains the exact source capture; source-manifest.json identifies each listing; live-world-inventory.json records scene measurements; export-manifest.json records file hashes; export-5.rbxl preserves the supplied latest place. template-extracted.txt and template-links.json preserve the template structure and attribution links. shared-chat-reading-notes.md records the browser reading scope. GWAP_REVISION_NOTES.md preserves prior reported test results. The attached GWAP ideas are copied when available.

Part II

Complete active Studio source

Chapter 28

S01 - GWAPNPC

ReplicatedStorage.GWAPNPC

Authority group: active. 6183 UTF-8 bytes; 96 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====]
2  Revision 4 engineering reference | 13 September 2026
3  Authority: ReplicatedStorage.GWAPNPC
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5  Do not require the documentation module from gameplay or replicate its prose to clients.
6  Purpose: Carrier appearance and gait
7  Future change: Separate immutable template pose from animation state; preserve same-inventory gun models.
8  Preserve dependencies: GWAPServer template selection and movement ownership.
9  Acceptance cases: Boys/girls, held seedling, gun pose, idle reset, teleport reset, pooled reactivation.
10 ]=====]
11 -- Shared NPC presentation uses the same original Weapons Kit assets as player inventory.
12 local N={}
13 local specs={
14     {name='Lab Sidearm',mesh='rbxassetid://2492922600',texture='rbxassetid://2492931263',scale=.1407,size=Vect
        ↳ or3.new(.20,.92,1.36)},
15     {name='Lab Shotgun',mesh='rbxassetid://2492972199',texture='rbxassetid://2492974190',scale=.6666,size=Vect
        ↳ or3.new(.31,1.4,4.87)},
16     {name='Lab Rifle',mesh='rbxassetid://2759059561',texture='rbxassetid://2759063893',scale=.6383,size=Vector
        ↳ 3.new(.29,1.17,6.18)},
17 }
18 function N.gun(bot,index)
19     local old=bot:FindFirstChild('SoloWeapon'); if old then old:Destroy() end
20     local spec=specs[(index-1)%#specs+1]; local arm=bot:FindFirstChild('ArmR') or bot:FindFirstChild('Right
        ↳ Arm') or bot.PrimaryPart
21     if not arm then return end
22     local p=Instance.new('Part'); p.Name='SoloWeapon'; p.Size=spec.size; p.Anchored=true; p.CanCollide=false;
        ↳ p.CanTouch=false; p.CanQuery=false
23     p.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0); p.Color=Color3.fromRGB(126,133,145)
24     local mesh=Instance.new('SpecialMesh'); mesh.MeshType=Enum.MeshType.FileMesh; mesh.MeshId=spec.mesh;
        ↳ mesh.TextureId=spec.texture; mesh.Scale=Vector3.one*spec.scale; mesh.Parent=p
25     p:SetAttribute('InventoryName',spec.name); p.Parent=bot; bot:SetAttribute('NPCGun',spec.name)
26     return p
27 end
28 function N.appearance(bot)
29     if bot:GetAttribute('Gender')~='Girl' then return end
30     local head=bot:FindFirstChild('Head'); if not head or bot:FindFirstChild('ResearchHair') then return end
31     local hair=bot:FindFirstChild('HairCrown'); local color=hair and hair.Color or Color3.fromRGB(67,39,25)
32     local folder=Instance.new('Model'); folder.Name='ResearchHair'; folder.Parent=bot
33     local function piece(name,size,offset)
34         local p=Instance.new('Part'); p.Name=name; p.Size=size; p.CFrame=head.CFrame*CFrame.new(offset);
            ↳ p.Color=color; p.Material=Enum.Material.SmoothPlastic; p.Anchored=true; p.CanCollide=false;
            ↳ p.CanQuery=false; p.Parent=folder; return p
35     end
36     piece('Back hair',Vector3.new(1.23,1.7,.34),Vector3.new(0,-.25,.55))
37     for _,side in ipairs({-1,1}) do piece('Side
        ↳ locks',Vector3.new(.28,1.25,.58),Vector3.new(side*.58,-.15,.15)) end
38     local pony=piece('Tied ponytail',Vector3.new(.55,1.3,.65),Vector3.new(0,-.25,.9));
        ↳ pony.Shape=Enum.PartType.Ball
39     local tie=piece('Hair tie',Vector3.new(.6,.16,.66),Vector3.new(0,.25,.9));
        ↳ tie.Color=Color3.fromRGB(107,174,174)
```

```

40 end
41 function N.say(bot,message)
42     -- Dialogue is already delivered to the owner's grouped HUD by GWAPServer.
43     -- A server-enabled speech billboard can flash over that HUD before a client
44     -- suppression tick, so retain message data without a second visual surface.
45     bot:SetAttribute('MissionMessage',message)
46     bot:SetAttribute('MissionMessageAt',workspace:GetServerTimeNow())
47     local old=bot:FindFirstChild('MissionSpeech',true);if old then old:Destroy() end
48 end
49 function N.nearPlayer(player,bot)
50     local root=player.Character and player.Character:FindFirstChild('HumanoidRootPart'); if not root then
51         ↪ return false end
52     local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
53     ↪ params.FilterDescendantsInstances={player.Character,bot}; params.RespectCanCollide=true
54     local overlap=OverlapParams.new(); overlap.FilterType=Enum.RaycastFilterType.Exclude;
55     ↪ overlap.FilterDescendantsInstances={player.Character,bot}; overlap.RespectCanCollide=true
56     for _,offset in ipairs({Vector3.new(5,0,-6),Vector3.new(-5,0,-6),Vector3.new(7,0,0),Vector3.new(-7,0,0),Ve
57     ↪ ctor3.new(0,0,8)}) do
58         local pos=root.CFrame:PointToWorldSpace(offset)
59         local floor=workspace:Raycast(pos+Vector3.new(0,3,0),Vector3.new(0,-10,0),params)
60         local sight=workspace:Raycast(root.Position,pos-root.Position,params)
61         if floor and floor.Normal.Y>.7 and not sight then
62             pos=Vector3.new(pos.X,floor.Position.Y+3.25,pos.Z)
63             if #workspace:GetPartBoundsInBox(CFrame.new(pos),Vector3.new(3,4.6,3),overlap)==0 then
64                 ↪ bot:PivotTo(CFrame.lookAt(pos,Vector3.new(root.Position.X,pos.Y,root.Position.Z))); return true
65             ↪ end
66         end
67     end
68     return false
69 end
70 local poses=setmetatable({}, {__mode='k'})
71 function N.animate(bot,dt)
72     local root=bot.PrimaryPart; if not root then return end
73     local data=poses[bot]
74     if not data then
75         data={parts={},previous=root.Position,phase=0,blend=0}
76         for _,name in ipairs({'LegL','LegR','ArmL','ArmR','Trainer-1','Trainer1','Sole-1','Sole1'}) do
77             local p=bot:FindFirstChild(name); if p then
78                 ↪ data.parts[name]={part=p,rest=root.CFrame:ToObjectSpace(p.CFrame)} end
79         end
80         poses[bot]=data
81     end
82     local distance=(root.Position-data.previous).Magnitude; data.previous=root.Position
83     local speed=distance/math.max(dt,.001)
84     if distance>8 then speed=0; distance=0 end -- regrouping is not a walking step
85     data.phase+=distance*math.pi/3.2
86     local moving=speed>.3 and 1 or 0
87     data.blend+=(moving-data.blend)*(1-math.exp(-dt*16))
88     local swing=math.sin(data.phase)*.42*data.blend
89     for name,entry in pairs(data.parts) do if entry.part.Parent then
90         local arm=name:sub(1,3)=='Arm'
91         local left=name=='LegL' or name=='ArmL' or name:find('-1',1,true)
92         local sign=left and -1 or 1
93         local pivot=arm and Vector3.new(sign*1.18,.85,0) or Vector3.new(sign*.42,-1.05,0)
94         local angle=arm and -swing*sign*.65 or swing*sign
95         if arm and bot:GetAttribute('HasSeedling') then angle=-.4 end
96         entry.part.CFrame=root.CFrame*CFrame.new(pivot)*CFrame.Angles(angle,0,0)*CFrame.new(-pivot)*entry.rest
97     end end
98     local gun=bot:FindFirstChild('SoloWeapon'); local arm=bot:FindFirstChild('ArmR')
99     if gun and arm then gun.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0) end
100 end
101 return N

```

Chapter 29

S02 - GWAPRouting

ReplicatedStorage.GWAPRouting

Authority group: active. 3434 UTF-8 bytes; 55 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ReplicatedStorage.GWAPRouting  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Equivalent selection  
7 Future change: Add reachability and operational-state adapters after fixing coordinates. Use stable IDs and  
8   ↳ deterministic tie breaking.  
9 Preserve dependencies: World station registry, destruction state, capacity.  
10 Acceptance cases: Same floor preferred, other floor fallback, all unavailable, full plots, removed fixture.  
11 ]=====  
12 local R={}  
13 local Spatial=require(game:GetService('ReplicatedStorage')).WaitForChild('LabSpatial')  
14 local function floorOf(item)  
15     local n=item:GetAttribute('Floor'); if n then return n end  
16     local label=item:GetAttribute('Level') or ''; return label=='Basement' and -1 or  
17     ↳ (tonumber(label:match('%d+')) or 1)-1  
18 end  
19 R.floorOf=floorOf  
20 function R.healthy(item)  
21     return item and item:IsDescendantOf(workspace) and not item:GetAttribute('Exploded') and not  
22     ↳ item:GetAttribute('BlastRemoved') and item:GetAttribute('FunctionalState')~='DESTROYED' and  
23     ↳ item:GetAttribute('FunctionalState')~='FAILED'  
24 end  
25 function R.ensureBackups()  
26     local stations=workspace:WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch'):WaitForChild('Sta  
27     ↳ tions')  
28     local originals=stations:GetChildren(); local perFloor={}  
29     table.sort(originals,function(a,b)return a.Name<b.Name end)  
30     for _,item in ipairs(originals) do if item:IsA('BasePart') and not item:GetAttribute('Backup') then  
31         local capability=item:GetAttribute('Planted')~nil and 'Greenhouse planting' or item.Name  
32         item:SetAttribute('Capability',capability)  
33         if capability~='Greenhouse planting' then for variant=1,2 do  
34             local name=item.Name..' - Backup '..variant  
35             if not stations:FindFirstChild(name) then  
36                 local copy=item:Clone(); copy.Name=name; copy:SetAttribute('Backup',true);  
37                 ↳ copy:SetAttribute('Capability',capability)  
38                 local floor=floorOf(item); if variant==2 then floor=(floor+2)%4-1 end  
39                 local z=Spatial.localPoint(item.Position).Z; local x=89  
40                 if variant==2 then perFloor[floor]=(perFloor[floor] or 0)+1; z=150+perFloor[floor]*14; x=67 end  
41                 local pos=Spatial.world(Vector3.new(x,floor*16.25+1.5,z)); local shift=pos-copy.Position  
42                 copy.Position=pos; for _,p in ipairs(copy:GetDescendants()) do if p:IsA('BasePart') then  
43                     ↳ p.Position+=shift end end  
44                 copy:SetAttribute('Floor',floor); Spatial.setApproach(copy,Vector3.new(78,floor*16.25+3.25,z))  
45                 copy:SetAttribute('Exploded',false); copy:SetAttribute('FunctionalState','RUNNING')  
46                 local sign=copy:FindFirstChild('StationLabel'); local label=sign and  
47                 ↳ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=capability..' | Backup |  
48                 ↳ '..(floor==1 and 'B1' or 'L'..(floor+1)) end  
49                 copy.Parent=stations  
50             end  
51         end  
52     end  
53 end
```

```
42         end end
43     end end
44     Spatial.sync(stations)
45     return stations
46 end
47 function R.choose(stations,capability,preferredFloor,position)
48     local best,rank
49     for _,item in ipairs(stations:GetChildren()) do if item:GetAttribute('Capability')==capability and
↳ R.healthy(item) and ((item:GetAttribute('Planted') or 0)<12) then
50         local score=(floorOf(item)==preferredFloor and 0 or
↳ 100000)+math.abs(floorOf(item)-preferredFloor)*1000+(item.Position-position).Magnitude
51         if not rank or score<rank then best,rank=item,score end
52     end end
53     return best
54 end
55 return R
```

Chapter 30

S03 - GWAPScenarios

ReplicatedStorage.GWAPScenarios

Authority group: active. 2692 UTF-8 bytes; 23 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ReplicatedStorage.GWAPScenarios  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Scenario data  
7 Future change: Give each scenario a stable ID and protocol version; decouple plot selection from array  
  ↳ position.  
8 Preserve dependencies: Server index validation, HUD job cards, station capabilities.  
9 Acceptance cases: Missing capability, invalid range, new eleventh entry, and unchanged old scenario IDs.  
10 ]=====  
11 -- Gameplay simulations inspired by the supplied GWAP ideas; no real measurements are fabricated.  
12 return {  
13     {title='Germination viability', purpose='Compare germination under different moisture levels.',  
14       ↳ route={'Seed bank','Incubator'}, variable='Moisture', units='%', low=35, high=80},  
15     {title='Pathogen resistance', purpose='Track resistant plant tissue through containment and analysis.',  
16       ↳ route={'Containment','Bio-analyzer'}, variable='Resistance batch', units='', low=1, high=9},  
17     {title='Light spectrum optimization', purpose='Compare seedling responses to visible and infrared light.',  
18       ↳ route={'Seed bank','Spectrograph','Infrared scanner'}, variable='Photoperiod', units='hours', low=8,  
19         ↳ high=18},  
20     {title='Soil bioremediation', purpose='Trace seedlings through root absorption analysis.', route={'Chemical  
21       ↳ seedlings','Root analyzer'}, variable='Soil batch', units='', low=1, high=12},  
22     {title='Hydroponic versus soil', purpose='Process paired root samples before greenhouse planting.',  
23       ↳ route={'Split-root tray','Centrifuge','Root analyzer'}, variable='Nutrient batch', units='', low=1,  
24         ↳ high=8},  
25     {title='UV adaptation', purpose='Transport exposed seedlings through vitality screening.', route={'UV  
26       ↳ chamber','Vitality scanner'}, variable='Exposure', units='seconds', low=5, high=20},  
27     {title='Gravitropism assay', purpose='Calibrate tilted roots before planting upright seedlings.',  
28       ↳ route={'Clinostat','Orientation scanner'}, variable='Tray tilt', units='degrees', low=10, high=80},  
29     {title='Drought resistance', purpose='Measure simulated stomatal response before rehydration.',  
30       ↳ route={'Climate chamber','Porometer'}, variable='Humidity', units='%', low=20, high=45},  
31     {title='Microbiome symbiosis', purpose='Carry inoculated seeds through symbiosis incubation.', route={'Wet  
32       ↳ lab','Symbiosis incubator'}, variable='Inoculum batch', units='', low=1, high=8},  
33     {title='Cold-hardiness screening', purpose='Screen thawed seedlings before rooftop recovery.',  
34       ↳ route={'Cryo-freezer','Vitality scanner'}, variable='Recovery temperature', units='C', low=5, high=20},  
35 }
```

Chapter 31

S04 - GWAPWorld

ReplicatedStorage.GWAPWorld

Authority group: active. 7198 UTF-8 bytes; 95 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====]
2 Revision 4 engineering reference | 13 September 2026
3 Authority: ReplicatedStorage.GWAPWorld
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5 Do not require the documentation module from gameplay or replicate its prose to clients.
6 Purpose: Research geometry
7 Future change: Use the captured active module, not the loose mislabeled file. Convert construction to a
  ↳ versioned local frame.
8 Preserve dependencies: GWAPRouting, mission approaches, roof stair geometry.
9 Acceptance cases: Build twice without duplicates, translated/rotated building, all approach points, roof
  ↳ opening.
10 ]=====]
11 -- Idempotent additive build: research stations, final roof flight and greenhouse.
12 local W = {}
13 local Spatial=require(game:GetService('ReplicatedStorage')):WaitForChild('LabSpatial')
14 local function part(parent, name, size, pos, color, material)
15     local p = Instance.new('Part'); p.Name=name; p.Size=size; p.Position=pos
16     p.Anchored=true; p.Color=color or Color3.fromRGB(49,66,70); p.Material=material or
  ↳ Enum.Material.SmoothPlastic
17     p.TopSurface=Enum.SurfaceType.Smooth; p.BottomSurface=Enum.SurfaceType.Smooth; p.Parent=parent; return p
18 end
19 local function label(p, text)
20     local g=Instance.new('BillboardGui'); g.Name='StationLabel'; g.Size=UDim2.fromOffset(210,44);
  ↳ g.StudsOffset=Vector3.new(0,3,0); g.MaxDistance=65; g.Parent=p
21     local t=Instance.new('TextLabel'); t.Size=UDim2.fromScale(1,1);
  ↳ t.BackgroundColor3=Color3.fromRGB(24,32,38); t.BackgroundTransparency=.18;
  ↳ t.TextColor3=Color3.fromRGB(220,255,235); t.TextSize=13; t.Font=Enum.Font.GothamBold;
  ↳ t.TextWrapped=true; t.Text=text; t.Parent=g
22 end
23 function W.plant(parent, pos, name)
24     local m=Instance.new('Model'); m.Name=name or 'Germinated seed'; m.Parent=parent
25     part(m,'Root plug',Vector3.new(.7,.5,.7),pos,Color3.fromRGB(94,65,43))
26     part(m,'Stem',Vector3.new(.12,1.4,.12),pos+Vector3.new(0,.8,0),Color3.fromRGB(77,148,53))
27     for i=-1,1,2 do local
  ↳ p=part(m,'Leaf',Vector3.new(.85,.12,.5),pos+Vector3.new(i*.35,1.1,0),Color3.fromRGB(102,202,77));
  ↳ p.CFrame*=CFrame.Angles(0,0,i*.4) end
28     for _,p in ipairs(m:GetChildren()) do p.CanCollide=false; p.CanQuery=false end
29     return m
30 end
31 function W.build()
32     local c=workspace:WaitForChild('Laboratory_Complex'); local old=c:FindFirstChild('GWAPResearch')
33     if old then Spatial.sync(old:WaitForChild('Stations')); return old end
34     local f=Instance.new('Folder'); f.Name='GWAPResearch'; f.Parent=c
35     local stations=Instance.new('Folder'); stations.Name='Stations'; stations.Parent=f
36     local specs={
37         {'Seed bank',-1},{ 'Incubator',0},{ 'Containment',1},{ 'Bio-analyzer',2},
38         {'Spectrograph',1},{ 'Infrared scanner',2},{ 'Chemical seedlings',-1},{ 'Root analyzer',1},
39         {'Split-root tray',0},{ 'Centrifuge',2},{ 'UV chamber',1},{ 'Clinostat',-1},
40         {'Orientation scanner',0},{ 'Climate chamber',2},{ 'Porometer',1},{ 'Wet lab',-1},
41         {'Symbiosis incubator',1},{ 'Cryo-freezer',-1},{ 'Vitality scanner',0}}
```

```

42 local perFloor={}
43 for _,s in ipairs(specs) do
44   local n=(perFloor[s[2]] or 0)+1; perFloor[s[2]]=n
45   local p=part(stations,s[1],Vector3.new(3,3,5),Vector3.new(67,s[2]*16.25+1.5,32+n*17))
46   p:SetAttribute('Floor',s[2]); p:SetAttribute('Approach',Vector3.new(78,s[2]*16.25+3.25,p.Position.Z))
47   local glass=part(p,'Observation window',Vector3.new(3.05,1.4,3),p.Position+Vector3.new(0,.6,0),Color3.
    ↪ fromRGB(113,232,204),Enum.Material.Glass); glass.Transparency=.45
48   part(p,'Status light',Vector3.new(.15,.25,3),p.Position+Vector3.new(1.6,1.3,0),Color3.fromRGB(95,255,1
    ↪ 41),Enum.Material.Neon)
49   label(p,s[1]..' | '..(s[2]==-1 and 'B1' or 'L'..(s[2]+1)))
50   W.plant(p,p.Position+Vector3.new(0,1.7,0),'Reference seedling')
51 end
52 -- Extend the existing north stair geometry by one flight, preserving player controls.
53 local tower=c:FindFirstChild('StairTower_North')
54 if tower then
55   for _,p in ipairs(tower:GetChildren()) do
56     if p:IsA('BasePart') and (p.Name:match('^Flight[AB]_L6_') or p.Name=='MidLanding_L6' or
    ↪ p.Name=='Handrail_L6') then
57       local q=p:Clone(); q.Name=p.Name:gsub('L6','L7');
    ↪ q.CFrame=Spatial.frame():Inverse()*q.CFrame+Vector3.new(0,16.25,0); q.Parent=f
58     end
59   end
60   local cap=tower:FindFirstChild('T_Roof'); if cap then
    ↪ cap:SetAttribute('GWAPOriginalCanCollide',cap.CanCollide); cap.CanCollide=false;
    ↪ cap.Transparency=1; cap.CanQuery=false end
61   part(f,'Roof stair landing',Vector3.new(41.14,.75,12.86),Vector3.new(78.43,129.625,-6.43))
62 end
63 local parapet=c:FindFirstChild('Parapet_N_1')
64 if parapet then
65   parapet:SetAttribute('GWAPOriginalCanCollide',parapet.CanCollide); parapet.CanCollide=false;
    ↪ parapet.Transparency=1; parapet.CanQuery=false
66   local parapetPosition=Spatial.localPoint(parapet.Position)
67   local left=parapetPosition.X-parapet.Size.X/2; local right=parapetPosition.X+parapet.Size.X/2
68   for _,range in ipairs({{left,71},{86,right}}) do part(f,'Roof entry
    ↪ parapet',Vector3.new(range[2]-range[1],parapet.Size.Y,parapet.Size.Z),Vector3.new((range[1]+range[
    ↪ 2])/2,parapetPosition.Y,parapetPosition.Z),parapet.Color) end
69 end
70 local g=Instance.new('Model'); g.Name='RooftopGreenhouse'; g.Parent=f
71 local y=130
72 part(g,'Foundation',Vector3.new(100,.35,112),Vector3.new(78,y+.175,90),Color3.fromRGB(190,200,190))
73 for _,x in ipairs({28,128}) do for z=34,146,28 do part(g,'Steel
    ↪ upright',Vector3.new(.4,15,.4),Vector3.new(x,y+7.5,z)) end end
74 for _,x in ipairs({28,128}) do local p=part(g,'Glazed
    ↪ wall',Vector3.new(.25,14,112),Vector3.new(x,y+7.4,90),Color3.fromRGB(184,223,222),Enum.Material.Glass);
    ↪ p.Transparency=.72 end
75 for _,z in ipairs({34,146}) do
76   for _,range in ipairs({{28,70},{86,128}}) do local p=part(g,'Entry
    ↪ glass',Vector3.new(range[2]-range[1],14,.25),Vector3.new((range[1]+range[2])/2,y+7.4,z),Color3.fro
    ↪ mRGB(184,223,222),Enum.Material.Glass); p.Transparency=.72 end
77   part(g,'Door lintel',Vector3.new(100,.5,.5),Vector3.new(78,y+14.5,z))
78 end
79 for z=34,146,14 do part(g,'Roof rib',Vector3.new(100,.3,.3),Vector3.new(78,y+15,z)) end
80 local roof=part(g,'Glass roof',Vector3.new(100,.25,112),Vector3.new(78,y+15,90),Color3.fromRGB(190,235,230
    ↪ ),Enum.Material.Glass); roof.Transparency=.8
81 local sign=part(g,'Greenhouse
    ↪ sign',Vector3.new(14,1,1),Vector3.new(78,y+11,34),Color3.fromRGB(60,175,113)); label(sign,'ROOFTOP
    ↪ GREENHOUSE | Research plots')
82 for i=1,10 do
83   local x=i%2==1 and 48 or 108; local z=48+math.floor((i-1)/2)*20
84   local bed=part(stations,'Plot
    ↪ '..i,Vector3.new(22,1,12),Vector3.new(x,y+.8,z),Color3.fromRGB(91,65,44),Enum.Material.Ground)
85   bed:SetAttribute('Floor',8); bed:SetAttribute('Approach',Vector3.new(78,y+3.6,z));
    ↪ bed:SetAttribute('Planted',0)
86   label(bed,'PLOT '..i..' | Ready for planting')
87   for _,dz in ipairs({-6,6}) do part(g,'Bed border',Vector3.new(23,1.4,.4),Vector3.new(x,y+.7,z+dz),Colo
    ↪ r3.fromRGB(132,112,80),Enum.Material.Wood) end
88   local prompt=Instance.new('ProximityPrompt'); prompt.Name='PlantSeedling'; prompt.ActionText='Plant
    ↪ carried seedling'; prompt.ObjectText='Research plot '..i; prompt.KeyboardKeyCode=Enum.KeyCode.E;
    ↪ prompt.HoldDuration=1; prompt.MaxActivationDistance=12; prompt.Parent=bed
89 end
90 local frame=Spatial.frame()
91 for _,v in ipairs(f:GetDescendants()) do if v:IsA('BasePart') then v.CFrame=frame*v.CFrame end end
92 Spatial.sync(stations)
93 return f
94 end

```


95 return W

Chapter 32

S05 - LabKinds

ReplicatedStorage.LabKinds

Authority group: active. 10408 UTF-8 bytes; 173 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ReplicatedStorage.LabKinds  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Equipment catalog  
7 Future change: Add one Kind with inputs, outputs, wait duration, and display text. Keep simulation values  
  ↪ explicit.  
8 Preserve dependencies: LabEquipment consumption and treatment paths; generator Kind attributes.  
9 Acceptance cases: Unknown Kind, missing input, invalid output, full destination, and a complete valid cycle.  
10 ]=====]  
11 -- LabKinds : what each kind of basic laboratory equipment does.  
12 -- Keyed by the Kind attribute the generator writes on every item, so  
13 -- any number of copies anywhere in the building behave the same.  
14  
15 local K = {}  
16  
17 K.Items = {  
18     TRAY    = "Sample Tray",  
19     GROUND  = "Ground Sample",  
20     DRIED   = "Dried Sample",  
21     SPUN    = "Separated Sample",  
22     CULTURE = "Culture Plate",  
23     WASTE   = "Used Material",  
24     CLEAN   = "Clean Glassware",  
25 }  
26 local I = K.Items  
27  
28 local function f(n, dp) return string.format("%. " .. dp .. "f", n) end  
29 local function rnd(a, b) return a + math.random() * (b - a) end  
30  
31 K.Actions = {  
32     ["Workbench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},  
33         log = function() return "Sample laid out on the bench." end},  
34     ["Island Bench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},  
35         log = function() return "Sample laid out on the island bench." end},  
36     ["Base Cabinet Run"] = {action = "Open the cabinet", wait = 0.8,  
37         log = function() return "Consumables drawn from the cabinet." end},  
38     ["Desk"] = {action = "Write up the record", wait = 1.2,  
39         log = function() return "Notes entered in the log book." end},  
40  
41     ["Sink Unit"] = {action = "Wash glassware", wait = 2.0, produces = {I.CLEAN},  
42         log = function() return "Glassware rinsed and set to drain." end},  
43     ["Fume Hood"] = {action = "Work under extraction", wait = 2.5, uvLight = true,  
44         log = function() return "Extraction running at " .. f(rnd(0.4, 0.6), 2) .. " m/s face velocity." end},  
45  
46     ["Laboratory Mill"] = {action = "Grind the sample", wait = 2.0,  
47         consumes = I.TRAY, produces = {I.GROUND}, activeColour = Color3.fromRGB(200, 160, 60),  
48         log = function() return "Ground for two minutes; chamber cleaned afterwards." end},  
49     ["Sieve Shaker"] = {action = "Sieve the sample", wait = 2.0,  
50         log = function() return f(rnd(45, 80), 0) .. "% passed the working mesh." end},
```

```

51 ["Drying Oven"] = {action = "Dry the sample", wait = 3.5,
52     consumes = I.TRAY, produces = {I.DRIED}, activeColour = Color3.fromRGB(200, 60, 50),
53     log = function() return "Oven holding " .. f(rnd(101, 105), 1) .. " C." end},
54 ["Centrifuge"] = {action = "Spin the sample", wait = 2.5,
55     consumes = I.TRAY, produces = {I.SPUN},
56     log = function() return f(rnd(3000, 12000), 0) .. " rpm for " .. math.random(5, 20) .. " minutes."
57     ↪ end},
58 ["Incubator"] = {action = "Incubate the sample", wait = 3.0,
59     consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(255, 170, 0),
60     log = function() return "Holding " .. f(rnd(19, 31), 1) .. " C; door log updated." end},
61 ["Autoclave"] = {action = "Sterilize a load", wait = 3.0,
62     consumes = I.WASTE, activeColour = Color3.fromRGB(0, 120, 255),
63     log = function() return "Cycle complete at 121 C; indicator strip filed." end},
64 ["Waste Bin"] = {action = "Discard used material", wait = 0.8, consumes = I.WASTE,
65     log = function() return "Waste segregated for removal." end},
66 ["Analytical Balance"] = {action = "Weigh the sample", wait = 1.5,
67     log = function()
68         local w = rnd(0.4, 25)
69         return "Reading " .. f(w, w < 1 and 4 or 3) .. " g; balance levelled and zeroed."
70     end},
71 ["Microscope"] = {action = "Examine the sample", wait = 1.5,
72     log = function() return "Examined at " .. math.random(1, 4) * 10 .. "x." end},
73 ["pH Meter"] = {action = "Measure pH", wait = 1.2,
74     log = function() return "pH " .. f(rnd(4.5, 9.0), 2) .. " at " .. f(rnd(19, 23), 1) .. " C." end},
75 ["Water Bath"] = {action = "Warm the sample", wait = 1.5,
76     log = function() return "Bath steady at " .. f(rnd(30, 60), 1) .. " C." end},
77 ["Hot Plate Stirrer"] = {action = "Stir and heat", wait = 1.5,
78     log = function() return "Stirring at " .. math.random(200, 900) .. " rpm." end},
79 ["Orbital Shaker"] = {action = "Shake the sample", wait = 1.5,
80     log = function() return "Shaking at " .. math.random(80, 260) .. " rpm." end},
81 ["Desiccator"] = {action = "Cool the sample", wait = 1.8,
82     consumes = I.DRIED, produces = {I.WASTE},
83     log = function() return "Cooled over dry desiccant before the second weighing." end},
84 ["Computer Terminal"] = {action = "Enter the result", wait = 1.2,
85     log = function() return "Result recorded against the sample number." end},
86
87 ["Refrigerator"] = {action = "Load / unload sample", wait = 1.2,
88     log = function() return "Held at " .. f(rnd(2, 8), 1) .. " C." end},
89 ["Chest Freezer"] = {action = "Load / unload sample", wait = 1.2,
90     log = function() return "Held at " .. f(rnd(-25, -16), 0) .. " C." end},
91 ["Sample Cabinet"] = {action = "File the sample", wait = 1.0,
92     log = function() return "Filed by sample number." end},
93 ["Shelving Unit"] = {action = "Draw stock", wait = 0.8,
94     log = function() return "Stock drawn; reorder level checked." end},
95 ["Storage Rack"] = {action = "Draw stock", wait = 0.8,
96     log = function() return "Stock drawn from the rack." end},
97 ["Glassware Rack"] = {action = "Take clean glassware", wait = 0.8, produces = {I.CLEAN},
98     log = function() return "Clean glassware taken from the rack." end},
99 ["Trolley"] = {action = "Load the trolley", wait = 0.8,
100     log = function() return "Trolley loaded for transfer." end},
101 ["Filing Cabinet"] = {action = "Pull a file", wait = 0.8,
102     log = function() return "File retrieved." end},
103 ["Locker Bank"] = {action = "Change clothing", wait = 1.2,
104     log = function() return "Changed before entering the working area." end},
105 ["Gas Cylinder Rack"] = {action = "Check the cylinders", wait = 1.0,
106     log = function() return "Cylinders chained; regulator pressure " .. math.random(20, 200) .. " bar."
107     ↪ end},
108 ["Reagent Rack"] = {action = "Take a reagent", wait = 0.8,
109     log = function() return "Reagent taken; bottle logged back in." end},
110 ["Test Tube Rack"] = {action = "Rack the tubes", wait = 0.8,
111     log = function() return "Tubes racked and labelled." end},
112 ["Pipette Stand"] = {action = "Take a pipette", wait = 0.8,
113     log = function() return "Pipette taken; last service date in date." end},
114
115 ["Germination Chamber"] = {action = "Load a germination test", wait = 2.5,
116     consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(120, 200, 140),
117     log = function() return "Test set at " .. f(rnd(18, 25), 1) .. " C, day count started." end},
118 ["Sample Divider"] = {action = "Divide the sample", wait = 1.8,
119     consumes = I.GROUND, produces = {I.TRAY},
120     log = function() return "Split into equal working portions." end},
121 ["Decontamination Unit"] = {action = "Run decontamination", wait = 2.2, consumes = I.WASTE,
122     activeColour = Color3.fromRGB(150, 210, 255),
123     log = function() return "Cycle complete; surfaces clear." end},
124 ["Water Distiller"] = {action = "Draw distilled water", wait = 1.2,
125     log = function() return "Conductivity " .. f(rnd(0.5, 2.5), 1) .. " uS/cm." end},

```

```

125     ["Reference Sample Archive"] = {action = "File a reference sample", wait = 1.0,
126         log = function() return "Filed against the reference register." end},
127     ["Optical Analyzer"] = {action = "Run an optical reading", wait = 1.8,
128         log = function() return "Reading logged at " .. math.random(400, 800) .. " nm." end},
129     ["Electrophoresis Unit"] = {action = "Run a separation", wait = 2.0, activeColour = Color3.fromRGB(0, 120,
130         ↪ 255),
131         log = function() return "Run complete at " .. math.random(60, 150) .. " V." end},
132     ["Thermal Cycler"] = {action = "Run an amplification cycle", wait = 2.5, activeColour = Color3.fromRGB(255,
133         ↪ 170, 0),
134         log = function() return math.random(25, 40) .. " cycles complete." end},
135     ["Homogenizer"] = {action = "Homogenize the sample", wait = 1.5,
136         log = function() return "Sample homogenized at " .. math.random(8000, 24000) .. " rpm." end},
137     ["Seed Counter"] = {action = "Count the sample", wait = 1.5,
138         log = function() return math.random(200, 1200) .. " units counted." end},
139     ["Calibration Weight Set"] = {action = "Check calibration", wait = 1.0,
140         log = function() return "Balance checked against certified weights." end},
141     ["Temperature Logger"] = {action = "Read the logger", wait = 0.6,
142         log = function() return "Log downloaded; " .. f(rnd(2, 8), 1) .. " C average." end},
143     ["Plant Growth Chamber"] = {action = "Take / return plant", wait = 0.8},
144     ["Growth Hormone Mixer"] = {action = "Apply growth hormone", wait = 2.0},
145     ["GMO Injector"] = {action = "Inject GMO trait", wait = 2.4},
146     ["Nutrient Infuser"] = {action = "Infuse nutrients", wait = 1.8},
147     ["UV Growth Scanner"] = {action = "Scan plant development", wait = 1.5},
148     ["Pollination Station"] = {action = "Pollinate the plant", wait = 2.1},
149     ["Mutagenesis Chamber"] = {action = "Irradiate the sample", wait = 2.4,
150         log = function() return math.random(100, 1000) .. " Gy dose logged; dosimeter reset." end},
151     ["Hormone Treatment Bench"] = {action = "Apply hormone treatment", wait = 2.0,
152         log = function() return "Auxin/cytokinin mix dosed and logged." end},
153     -- Cooperative role stations: LabEquipment special-cases these three
154     -- kinds before ever reaching this table, so only the prompt flavour
155     -- text below (action/wait) is actually used for them.
156     ["Sample Intake"] = {action = "Log a sample", wait = 1.0},
157     ["Inspection Station"] = {action = "Inspect the batch", wait = 1.2},
158     ["Supervisor Console"] = {action = "Sign off the batch", wait = 1.5},
159
160     ["Fire Extinguisher"] = {action = "Check the extinguisher", wait = 1.0,
161         log = function() return "Gauge in the green; inspection tag current." end},
162     ["First Aid Cabinet"] = {action = "Check the first aid kit", wait = 1.0,
163         log = function() return "Contents complete." end},
164     ["Eyewash Station"] = {action = "Flush the eyewash", wait = 1.5,
165         log = function() return "Weekly flush run; water clear." end},
166 }
167
168 K.Default = {action = "Inspect", wait = 1.0,
169     log = function() return "Checked and left in order." end}
170
171 function K.forKind(kind) return K.Actions[kind] or K.Default end
172
173 return K

```

Chapter 33

S06 - Blaster

ReplicatedStorage.Modules.Blasters

Authority group: active. 9143 UTF-8 bytes; 239 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: ReplicatedStorage.Modules.Blasters  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Shared casting  
7  Future change: Preserve desktop screen conversion and mobile camera-forward behavior as separate tested paths.  
8  Preserve dependencies: Controller reticle and server manager raycasts.  
9  Acceptance cases: Mouse edges, asymmetric safe areas, near wall, long range, and invalid vectors.  
10 This revision changes documentation only in this source; executable input/aiming behavior is preserved.  
11 ]=====  
12 -- Blaster  
13 -- Shared, device-independent camera casting and client shot presentation.  
14  
15 local Debris = game:GetService("Debris")  
16 local ReplicatedStorage = game:GetService("ReplicatedStorage")  
17 local RunService = game:GetService("RunService")  
18  
19 local Blaster = {}  
20  
21 Blaster.REMOTE_FOLDER_NAME = "BlasterRemotes"  
22 Blaster.FIRE_REMOTE_NAME = "Fire"  
23 Blaster.EFFECT_REMOTE_NAME = "ReplicateShot"  
24  
25 local replicationConnection  
26  
27 local function finiteNumber(value)  
28     return typeof(value) == "number"  
29         and value == value  
30         and value > -1e7  
31         and value < 1e7  
32 end  
33  
34 function Blaster.IsFiniteVector(value)  
35     return typeof(value) == "Vector3"  
36         and finiteNumber(value.X)  
37         and finiteNumber(value.Y)  
38         and finiteNumber(value.Z)  
39 end  
40  
41 local function numberAttribute(tool, name, fallback, minimum, maximum)  
42     local value = tool and tool:GetAttribute(name)  
43     if not finiteNumber(value) then value = fallback end  
44     return math.clamp(value, minimum, maximum)  
45 end  
46  
47 function Blaster.GetWeaponConfig(tool)  
48     return {  
49         damage = numberAttribute(tool, "BlasterDamage", 25, 0, 1000),  
50         range = numberAttribute(tool, "BlasterRange", 250, 8, 1000),  
51         cooldown = numberAttribute(tool, "BlasterCooldown", 0.35, 0.06, 10),
```

```

52     pellets = math.floor(numberAttribute(tool, "BlasterPellets", 1, 1, 16)),
53     spread = numberAttribute(tool, "BlasterSpread", 0, 0, 0.35),
54     selfCostPercent = numberAttribute(tool, "BlasterSelfCostPercent", 0, 0, 25),
55     adsFov = numberAttribute(tool, "BlasterADSFOV", 55, 25, 100),
56     automatic = tool and tool:GetAttribute("BlasterAutomatic") == true or false,
57     instantHeadshot = tool and tool:GetAttribute("BlasterInstantHeadshot") == true or false,
58     allowPlayerDamage = tool and tool:GetAttribute("BlasterAllowPlayerDamage") == true or false,
59 }
60 end
61
62 function Blaster.GetCameraRay(camera, maxRange, screenPoint)
63     camera = camera or workspace.CurrentCamera
64     if not camera then return nil end
65     maxRange = math.clamp(tonumber(maxRange) or 500, 1, 2000)
66     if typeof(screenPoint) == "Vector2" then
67         -- Desktop points are expressed in CoreUISafeInsets screen space (the
68         -- mouse location minus GuiService:GetGuiInset()). ScreenPointToRay is
69         -- the matching inverse transform. ViewportPointToRay uses device-safe
70         -- coordinates instead and produces a vertical offset on PC and in the
71         -- phone simulator.
72         local ray = camera:ScreenPointToRay(screenPoint.X, screenPoint.Y)
73         if ray.Direction.Magnitude < 0.99 then return nil end
74         return ray.Origin, ray.Direction.Unit, ray.Direction.Unit * maxRange
75     end
76
77     local direction = camera.CFrame.LookVector
78     if direction.Magnitude < 0.99 then return nil end
79
80     return camera.CFrame.Position, direction.Unit, direction.Unit * maxRange
81 end
82
83 function Blaster.NewRaycastParams(ignoreList)
84     local params = RaycastParams.new()
85     params.FilterType = Enum.RaycastFilterType.Exclude
86     params.FilterDescendantsInstances = ignoreList or {}
87     params.IgnoreWater = true
88     return params
89 end
90
91 function Blaster.CastFromCamera(camera, maxRange, ignoreList, screenPoint)
92     local origin, unitDirection, castDirection = Blaster.GetCameraRay(camera, maxRange, screenPoint)
93     if not origin then return nil end
94
95     local result = workspace:Raycast(origin, castDirection, Blaster.NewRaycastParams(ignoreList))
96     return {
97         origin = origin,
98         direction = unitDirection,
99         result = result,
100         position = result and result.Position or (origin + castDirection),
101     }
102 end
103
104 -- Returns the exact centre-line solution used for both the live reticle and a
105 -- shot. The camera chooses the intended point; the muzzle ray then catches a
106 -- nearby wall or prop that physically blocks the weapon.
107 function Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
108     if not tool or not tool:IsA("Tool") then return nil end
109     local config = Blaster.GetWeaponConfig(tool)
110     local cast = Blaster.CastFromCamera(camera, config.range, ignoreList, screenPoint)
111     if not cast then return nil end
112
113     local handle = tool:FindFirstChild("Handle")
114     local muzzleOrigin = handle and handle.Position or cast.origin
115     local muzzleDelta = cast.position - muzzleOrigin
116     local muzzleResult
117     if muzzleDelta.Magnitude > 0.05 then
118         -- Extend just beyond the camera hit point. Ending exactly on a surface can
119         -- miss it because of floating-point/raycast endpoint precision.
120         muzzleResult = workspace:Raycast(
121             muzzleOrigin,
122             muzzleDelta.Unit * (muzzleDelta.Magnitude + 0.25),
123             Blaster.NewRaycastParams(ignoreList)
124         )
125     end
126
127     return {

```

```

128     cameraOrigin = cast.origin,
129     direction = cast.direction,
130     muzzleOrigin = muzzleOrigin,
131     position = muzzleResult and muzzleResult.Position or cast.position,
132     result = muzzleResult,
133     cameraResult = cast.result,
134 }
135 end
136
137 local function tracerColor(tool)
138     local value = tool and tool:GetAttribute("BlasterTracerColor")
139     return typeof(value) == "Color3" and value or Color3.fromRGB(255, 226, 140)
140 end
141
142 function Blaster.DrawTracer(origin, hitPosition, color, lifetime)
143     if not Blaster.IsFiniteVector(origin) or not Blaster.IsFiniteVector(hitPosition) then return end
144     local distance = (hitPosition - origin).Magnitude
145     if distance < 0.05 then return end
146
147     local beam = Instance.new("Part")
148     beam.Name = "BlasterTracer"
149     beam.Size = Vector3.new(0.055, 0.055, distance)
150     beam.CFrame = CFrame.lookAt((origin + hitPosition) * 0.5, hitPosition)
151     beam.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
152     beam.Material = Enum.Material.Neon
153     beam.Transparency = 0.08
154     beam.Anchored = true
155     beam.CanCollide = false
156     beam.CanTouch = false
157     beam.CanQuery = false
158     beam.CastShadow = false
159     beam.Parent = workspace
160     Debris.AddItem(beam, lifetime or 0.075)
161 end
162
163 function Blaster.DrawImpact(position, normal, color, lifetime)
164     if not Blaster.IsFiniteVector(position) or not Blaster.IsFiniteVector(normal) then return end
165
166     local impact = Instance.new("Part")
167     impact.Name = "BlasterImpact"
168     impact.Shape = Enum.PartType.Ball
169     impact.Size = Vector3.new(0.16, 0.16, 0.16)
170     impact.Position = position + normal * 0.04
171     impact.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
172     impact.Material = Enum.Material.Neon
173     impact.Anchored = true
174     impact.CanCollide = false
175     impact.CanTouch = false
176     impact.CanQuery = false
177     impact.CastShadow = false
178     impact.Parent = workspace
179     Debris.AddItem(impact, lifetime or 0.65)
180 end
181
182 local function getRemotes(timeout)
183     local folder = ReplicatedStorage.WaitForChild(Blaster.REMOTE_FOLDER_NAME, timeout or 10)
184     if not folder then return nil end
185     local fireRemote = folder.WaitForChild(Blaster.FIRE_REMOTE_NAME, timeout or 10)
186     local effectRemote = folder.WaitForChild(Blaster.EFFECT_REMOTE_NAME, timeout or 10)
187     return fireRemote, effectRemote
188 end
189
190 function Blaster.StartReplication()
191     if not RunService.IsClient() or replicationConnection then return replicationConnection end
192     local _, effectRemote = getRemotes(15)
193     if not effectRemote or not effectRemote:IsA("RemoteEvent") then return nil end
194
195     replicationConnection = effectRemote.OnClientEvent:Connect(function(payload)
196         if typeof(payload) ~= "table" then return end
197         if not Blaster.IsFiniteVector(payload.origin) or not Blaster.IsFiniteVector(payload.position) then
198             ↪ return end
199         Blaster.DrawTracer(payload.origin, payload.position, payload.color, 0.075)
200         if Blaster.IsFiniteVector(payload.normal) then
201             Blaster.DrawImpact(payload.position, payload.normal, payload.color)
202         end
203     end)
204 end

```

```
203     return replicationConnection
204 end
205
206 function Blaster.Fire(tool, camera, sequence, screenPoint)
207     assert(RunService:IsClient(), "Blaster.Fire can only be called by a client")
208     if not tool or not tool:IsA("Tool") then return nil end
209
210     local player = game:GetService("Players").LocalPlayer
211     local character = player and player.Character
212     if not character or tool.Parent ~= character then return nil end
213
214     local ignoreList = {character, tool, workspace.CurrentCamera}
215     local solution = Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
216     if not solution then return nil end
217
218     Blaster.DrawTracer(solution.muzzleOrigin, solution.position, tracerColor(tool), 0.11)
219     if solution.result then
220         Blaster.DrawImpact(solution.position, solution.result.Normal, tracerColor(tool), 0.8)
221     end
222
223     local fireRemote = getRemotes(10)
224     if fireRemote and fireRemote:IsA("RemoteEvent") then
225         fireRemote:FireServer({
226             version = 1,
227             tool = tool,
228             cameraOrigin = solution.cameraOrigin,
229             direction = solution.direction,
230             sequence = sequence,
231             clientTime = workspace:GetServerTimeNow(),
232         })
233     end
234
235     return solution
236 end
237
238 return Blaster
239
```


Chapter 34

S07 - BlasterController

ReplicatedStorage.Modules.BlasterController

Authority group: active. 35949 UTF-8 bytes; 915 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: ReplicatedStorage.Modules.BlasterController  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Input/camera/pose  
7  Future change: Fix one measured fault at a time; retain cached base transforms and deterministic update  
   ↳ ownership.  
8  Preserve dependencies: Shared casting, Tool grip metadata, HUD rectangles, rig joints.  
9  Acceptance cases: Idle/walk/recoil, mouse pitch/yaw, mobile swipe, hybrid input, ADS restore, GUI click  
   ↳ suppression.  
10 This revision changes documentation only in this source; executable input/aiming behavior is preserved.  
11 ]=====]  
12 -- BlasterController  
13 -- Local Tool controllers: ContextActionService input, mobile controls, ADS,  
14 -- touch aim damping, camera recoil, crosshair, and weapon bob.  
15  
16 local ContextActionService = game:GetService("ContextActionService")  
17 local GuiService = game:GetService("GuiService")  
18 local Players = game:GetService("Players")  
19 local ReplicatedStorage = game:GetService("ReplicatedStorage")  
20 local RunService = game:GetService("RunService")  
21 local TweenService = game:GetService("TweenService")  
22 local UserInputService = game:GetService("UserInputService")  
23  
24 local Blaster = require(script.Parent:WaitForChild("Blaster"))  
25 local localPlayer = Players.LocalPlayer  
26  
27 local BlasterController = {}  
28 BlasterController.__index = BlasterController  
29  
30 local controllers = setmetatable({}, {__mode = "k"})  
31 local nextControllerId = 0  
32  
33 local function touchControlsActive()  
34     if UserInputService.TouchEnabled then return true end  
35     local okTouchScreen, touchScreen = pcall(function()  
36         return UserInputService.TouchScreenEnabled  
37     end)  
38     if okTouchScreen and touchScreen then return true end  
39     local okPreferred, preferred = pcall(function()  
40         return UserInputService.PreferredInput  
41     end)  
42     if okPreferred and tostring(preferred):find("Touch") then return true end  
43     local okControls, controls = pcall(function()  
44         return GuiService.TouchControlsEnabled  
45     end)  
46     if okControls and controls then return true end  
47  
48     -- Studio can retain desktop input flags when a phone preset is selected  
49     -- after the client starts. Recognize the simulator's compact landscape
```

```

50     -- viewport so the weapon does not silently lose its mobile camera/buttons.
51     local camera = workspace.CurrentCamera
52     local viewport = camera and camera.ViewportSize
53     return RunService:IsStudio()
54         and viewport ~= nil
55         and viewport.Y <= 450
56         and viewport.X / math.max(1, viewport.Y) >= 1.45
57 end
58
59 local function pointInside(button, point)
60     if not button or not button.Visible then return false end
61     local p = button.AbsolutePosition
62     local s = button.AbsoluteSize
63     return point.X >= p.X and point.X <= p.X + s.X
64         and point.Y >= p.Y and point.Y <= p.Y + s.Y
65 end
66
67 -- Hardware support does not identify the active input on an emulator/hybrid.
68 local function touchAimActive()
69     local input = UserInputService:GetLastInputType()
70     if input == Enum.UserInputType.Touch then return true end
71     if input == Enum.UserInputType.MouseMovement
72         or input == Enum.UserInputType.MouseButton1
73         or input == Enum.UserInputType.MouseButton2
74         or input == Enum.UserInputType.MouseButton3
75         or input == Enum.UserInputType.MouseWheel
76         or input == Enum.UserInputType.Keyboard then
77         return false
78     end
79     return touchControlsActive()
80 end
81
82 local function rotationBetween(from, to)
83     local dot = math.clamp(from:Dot(to), -1, 1)
84     local axis = from:Cross(to)
85     if axis.Magnitude < 0.00001 then
86         if dot > 0 then return CFrame.identity end
87         axis = from:Cross(Vector3.yAxis)
88         if axis.Magnitude < 0.00001 then axis = from:Cross(Vector3.xAxis) end
89     end
90     return CFrame.fromAxisAngle(axis.Unit, math.acos(dot))
91 end
92
93 local function makeCrosshair(hudName)
94     local playerGui = localPlayer:WaitForChild("PlayerGui")
95     local old = playerGui:FindFirstChild(hudName)
96     if old then old:Destroy() end
97
98     local gui = Instance.new("ScreenGui")
99     gui.Name = hudName
100     gui.ResetOnSpawn = false
101     gui.IgnoreGuiInset = true
102     gui.ScreenInsets = Enum.ScreenInsets.None
103     gui.ClipToDeviceSafeArea = false
104     gui.DisplayOrder = 12
105     gui.Enabled = false
106     gui.Parent = playerGui
107
108     local root = Instance.new("Frame")
109     root.Name = "Crosshair"
110     root.AnchorPoint = Vector2.new(0.5, 0.5)
111     root.Position = UDim2.fromScale(0.5, 0.5)
112     root.Size = UDim2.fromOffset(30, 30)
113     root.BackgroundTransparency = 1
114     root.Parent = gui
115
116     local dot = Instance.new("Frame")
117     dot.Name = "Dot"
118     dot.AnchorPoint = Vector2.new(0.5, 0.5)
119     dot.Position = UDim2.fromScale(0.5, 0.5)
120     dot.Size = UDim2.fromOffset(6, 6)
121     dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
122     dot.BorderSizePixel = 0
123     dot.Parent = root
124     local dotCorner = Instance.new("UICorner")
125     dotCorner.CornerRadius = UDim.new(1, 0)

```

```

126     dotCorner.Parent = dot
127     local dotStroke = Instance.new("UIStroke")
128     dotStroke.Color = Color3.fromRGB(6, 15, 22)
129     dotStroke.Transparency = 0.15
130     dotStroke.Thickness = 1.5
131     dotStroke.Parent = dot
132
133     for index, data in ipairs({
134         {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
135         {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
136         {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
137         {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
138     }) do
139         local line = Instance.new("Frame")
140         line.Name = "Line" .. index
141         line.Position = data[1]
142         line.Size = data[2]
143         line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
144         line.BackgroundTransparency = 0.08
145         line.BorderSizePixel = 0
146         line.Parent = root
147     end
148
149     local status = Instance.new("TextLabel")
150     status.Name = "Status"
151     status.AnchorPoint = Vector2.new(0.5, 0)
152     status.Position = UDim2.new(0.5, 0, 0.5, 25)
153     status.Size = UDim2.fromOffset(210, 24)
154     status.BackgroundTransparency = 1
155     status.Font = Enum.Font.GothamBold
156     status.Text = ""
157     status.TextColor3 = Color3.fromRGB(220, 232, 242)
158     status.TextSize = 13
159     status.TextStrokeTransparency = 0.5
160     status.Parent = gui
161
162     return gui, root, status, dot
163 end
164
165 function BlasterController:_ensureHUD()
166     if self.hud and self.hud.Parent and self.crosshair and self.crosshair.Parent == self.hud
167         and self.reticleDot and self.reticleDot.Parent == self.crosshair then return false end
168     if self.hud then self.hud:Destroy() end
169     self.fallbackShootButton = nil
170     self.fallbackAimButton = nil
171     self.hud, self.crosshair, self.statusLabel, self.reticleDot = makeCrosshair("BlasterHUD_" .. self.id)
172     self.hud.Enabled = self.enabled
173     return true
174 end
175
176 function BlasterController.new(tool, overrides)
177     assert(RunService.IsClient(), "BlasterController must be required on the client")
178     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
179
180     nextControllerId = nextControllerId + 1
181     local self = setmetatable({}, BlasterController)
182     self.tool = tool
183     self.overrides = overrides or {}
184     self.id = nextControllerId
185     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
186     self.connections = {}
187     self.enabled = false
188     self.aiming = false
189     self.triggerHeld = false
190     self.fireLoopRunning = false
191     self.sequence = 0
192     self.nextLocalShot = 0
193     self.baseGrip = tool.Grip
194     self.recoilPitch = 0
195     self.recoilYaw = 0
196     self.touchInput = nil
197     self.touchLastPosition = nil
198     self.hiddenParts = {}
199     self.shoulderOffset = Vector3.zero
200
201     self.connections.equipped = tool.Equipped:Connect(function()

```

```

202     self:Enable()
203 end)
204 self.connections.unequipped = tool.Unequipped:Connect(function()
205     self:Disable()
206 end)
207 self.connections.destroying = tool.Destroying:Connect(function()
208     self:Destroy()
209 end)
210 self.connections.ancestry = tool.AncestryChanged:Connect(function()
211     if self.enabled and tool.Parent ~= localPlayer.Character then
212         self:Disable()
213     end
214 end)
215 -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
216 -- remains the primary input layer, while this fallback covers touch/controller layouts
217 -- that route their built-in Tool button directly to Activated.
218 self.connections.activated = tool.Activated:Connect(function()
219     if self.enabled then self:_beginFiring() end
220 end)
221 self.connections.deactivated = tool.Deactivated:Connect(function()
222     self.triggerHeld = false
223 end)
224
225 if localPlayer.Character and tool.Parent == localPlayer.Character then
226     task.defer(function() self:Enable() end)
227 end
228
229 return self
230 end
231
232 function BlasterController:_button(actionName)
233     local ok, button = pcall(function()
234         return ContextActionService:GetButton(actionName)
235     end)
236     return ok and button or nil
237 end
238
239 function BlasterController:_fallbackTouchButton(kind, title)
240     local field = kind == "Shoot" and "fallbackShootButton" or "fallbackAimButton"
241     local existing = self[field]
242     if existing and existing.Parent then return existing end
243     if not self.hud then return nil end
244
245     local button = Instance.new("TextButton")
246     button.Name = "Fallback" .. kind
247     button.AutoButtonColor = true
248     button.BackgroundColor3 = Color3.fromRGB(35, 48, 60)
249     button.BorderSizePixel = 0
250     button.Text = title
251     button.Font = Enum.Font.GothamBold
252     button.TextColor3 = Color3.fromRGB(245, 250, 255)
253     button.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
254     button.TextStrokeTransparency = 0.25
255     button.TextScaled = false
256     button.ZIndex = 20
257     button.Parent = self.hud
258     self[field] = button
259
260     if kind == "Shoot" then
261         button.InputBegan:Connect(function(input)
262             if input.UserInputType == Enum.UserInputType.Touch
263                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
264                 self:_shootAction(nil, Enum.UserInputState.Begin)
265             end
266         end)
267         button.InputEnded:Connect(function(input)
268             if input.UserInputType == Enum.UserInputType.Touch
269                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
270                 self:_shootAction(nil, Enum.UserInputState.End)
271             end
272         end)
273     else
274         button.Activated:Connect(function()
275             if self.enabled then self:SetAiming(not self.aiming) end
276         end)
277     end
end

```

```

278     return button
279 end
280
281 function BlasterController:_styleTouchButtons()
282     if not touchControlsActive() then return end
283     local camera = workspace.CurrentCamera
284     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
285     local playerGui = localPlayer:FindFirstChildOfClass("PlayerGui")
286     local touchGui = playerGui and playerGui:FindFirstChild("TouchGui")
287     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
288     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 52, 80)
289     if jump and jump.AbsoluteSize.X > 0 then
290         diameter = math.clamp(math.floor(math.min(jump.AbsoluteSize.X, jump.AbsoluteSize.Y) * 0.85), 48, 80)
291     end
292
293     -- Own the visuals/hit targets; CAS still binds desktop/controller actions.
294     local shootButton = self:_fallbackTouchButton("Shoot", "FIRE")
295     local aimButton = self:_fallbackTouchButton("Aim", "AIM")
296     for _, action in ipairs({"Shoot", "Aim"}) do
297         local casButton = self:_button(self.actionPrefix .. action)
298         if casButton then casButton.Visible = false end
299     end
300     for _, button in ipairs({shootButton, aimButton}) do
301         if button then
302             button.Visible = true
303             button.Size = UDim2.fromOffset(diameter, diameter)
304             button.TextSize = math.floor(diameter * 0.25)
305             button.BackgroundTransparency = 0.18
306             if button:IsA("ImageButton") then
307                 button.ImageTransparency = 0.08
308                 button.ImageColor3 = Color3.fromRGB(35, 48, 60)
309             end
310             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
311             corner.CornerRadius = UDim.new(1, 0)
312             corner.Parent = button
313             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
314             stroke.Color = Color3.fromRGB(220, 235, 245)
315             stroke.Transparency = 0.25
316             stroke.Thickness = 2
317             stroke.Parent = button
318             local title = button:FindFirstChild("ActionTitle")
319             if title and title:IsA("TextLabel") then
320                 title.Font = Enum.Font.GothamBold
321                 title.TextColor3 = Color3.fromRGB(245, 250, 255)
322                 title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
323                 title.TextStrokeTransparency = 0.25
324                 title.TextScaled = true
325             end
326         end
327     end
328
329     -- GUI absolute coordinates include the simulator's safe-area translation.
330     -- Anchor to Roblox's real JUMP rectangle, never an estimated viewport point.
331     local gap = math.max(12, math.floor(diameter * 0.22))
332     local jumpSize = jump and jump.AbsoluteSize or Vector2.new(diameter, diameter)
333     local jumpPoint = jump and jump.AbsolutePosition
334     or (self.hud.AbsolutePosition + self.hud.AbsoluteSize - jumpSize - Vector2.new(28, 28))
335     local shootPoint = Vector2.new(jumpPoint.X + (jumpSize.X - diameter) * 0.5, jumpPoint.Y - diameter - gap)
336     local aimPoint = Vector2.new(jumpPoint.X - diameter - gap, jumpPoint.Y + (jumpSize.Y - diameter) * 0.5)
337     local function place(button, point)
338         if not button or not button.Parent then return end
339         button.AnchorPoint = Vector2.zero
340         local origin = button.Parent.AbsolutePosition
341         button.Position = UDim2.fromOffset(
342             math.floor(point.X - origin.X + 0.5),
343             math.floor(point.Y - origin.Y + 0.5)
344         )
345     end
346     place(shootButton, shootPoint)
347     place(aimButton, aimPoint)
348     self.lastJumpPosition = jump and jump.AbsolutePosition
349 end
350
351 function BlasterController:_getAimScreenPoint(camera)
352     if not camera then return nil end
353     if touchAimActive() then

```

```

354         -- Mobile aims along the Scriptable camera's true forward vector. In the
355         -- Studio phone simulator, ViewportPointToRay(viewportCenter) is offset by
356         -- the asymmetric safe-area canvas and can land several degrees above the
357         -- visible centre marker.
358         return nil
359     end
360     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
361     -- Clicking an on-screen action must not pull the target onto that button.
362     if pointInside(self.fallbackShootButton, point) or pointInside(self.fallbackAimButton, point) then
363         return self.lastPointerAim or camera.ViewportSize * 0.5
364     end
365     self.lastPointerAim = point
366     return Vector2.new(
367         math.clamp(point.X, 0, camera.ViewportSize.X),
368         math.clamp(point.Y, 0, camera.ViewportSize.Y)
369     )
370 end
371
372 function BlasterController:updateAimReticle(camera)
373     if not self.hud or not self.crosshair or not camera then return end
374     local rayScreenPoint = self:_getAimScreenPoint(camera)
375     local screenPoint = rayScreenPoint
376     local character = localPlayer.Character
377     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
378         character,
379         self.tool,
380         camera,
381     }, rayScreenPoint)
382
383     if rayScreenPoint == nil then
384         -- The shot uses Camera.CFrame.LookVector. Project that same forward
385         -- vector with WorldToScreenPoint so the marker lands in the exact CoreUI
386         -- coordinate space used by this ScreenGui, including notches/top bars.
387         local projected = camera:WorldToScreenPoint(
388             camera.CFrame.Position + camera.CFrame.LookVector * 100
389         )
390         screenPoint = Vector2.new(projected.X, projected.Y)
391     elseif not screenPoint then
392         screenPoint = camera.ViewportSize * 0.5
393     end
394
395     if solution then
396         if self.reticledot then
397             self.reticledot.BackgroundColor3 = solution.result
398             and Color3.fromRGB(255, 224, 105)
399             or Color3.fromRGB(120, 235, 255)
400         end
401     end
402
403     -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
404     -- and begin at a negative absolute position. Counter that offset so this
405     -- marker identifies the muzzle-checked world hit on every device.
406     local origin = self.hud.AbsolutePosition
407     local center = screenPoint - origin
408     self.crosshair.Position = UDim2.fromOffset(
409         math.floor(center.X + 0.5),
410         math.floor(center.Y + 0.5)
411     )
412 end
413
414 function BlasterController:_setLocalHeadHidden(hidden)
415     local character = localPlayer.Character
416     if not character then return end
417     if hidden then
418         for _, descendant in ipairs(character:GetDescendants()) do
419             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
420                 then
421                 if self.hiddenParts[descendant] == nil then
422                     self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
423                 end
424                 descendant.LocalTransparencyModifier = 1
425             end
426         end
427     else
428         for part, transparency in pairs(self.hiddenParts) do
429             if part.Parent then part.LocalTransparencyModifier = transparency end
430         end
431     end
432 end

```

```

429         end
430         table.clear(self.hiddenParts)
431     end
432 end
433
434 function BlasterController:_beginTouchCamera()
435     if not touchAimActive() or self.mobileCameraActive then return end
436     local camera = workspace.CurrentCamera
437     local character = localPlayer.Character
438     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
439     if not camera or not humanoid then return end
440
441     local look = camera.CFrame.LookVector
442     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
443     self.touchYaw = math.atan2(-look.X, -look.Z)
444     self.savedCameraType = camera.CameraType
445     self.mobileCameraActive = true
446     humanoid.AutoRotate = false
447     camera.CameraType = Enum.CameraType.Scriptable
448     self:_setLocalHeadHidden(true)
449 end
450
451 function BlasterController:_endTouchCamera()
452     if not self.mobileCameraActive then return end
453     local camera = workspace.CurrentCamera
454     local character = localPlayer.Character
455     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
456     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
457     self.mobileCameraActive = false
458     self.touchInput = nil
459     self.touchLastPosition = nil
460     self:_setLocalHeadHidden(false)
461 end
462
463 function BlasterController:SetAiming(aiming)
464     if not self.enabled then aiming = false end
465     if self.aiming == aiming then return end
466     self.aiming = aiming
467
468     local camera = workspace.CurrentCamera
469     if not camera then return end
470     local config = Blaster.GetWeaponConfig(self.tool)
471     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
472     if self.fovTween then self.fovTween:Cancel() end
473     self.fovTween = TweenService.Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
474 ↪ Enum.EasingDirection.Out), {
475         FieldOfView = targetFov,
476     })
477     self.fovTween:Play()
478
479     if touchAimActive() then
480         -- Mobile weapons keep the camera active in hip-fire too, so the right
481         -- half of the screen always steers the reticle like a dedicated FPS.
482         self:_beginTouchCamera()
483     elseif aiming then
484         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
485         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
486     else
487         if self.savedMouseSensitivity then
488             UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
489             self.savedMouseSensitivity = nil
490         end
491     end
492
493     local character = localPlayer.Character
494     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
495     if humanoid and not self.mobileCameraActive then
496         humanoid.CameraOffset = aiming
497         and Vector3.new(1.25, 0.55, 0)
498         or self.shoulderOffset
499     end
500
501     if self.statusLabel then
502         self.statusLabel.Text = aiming and "ADS" or ""
503     end
504 end

```

```

504
505 function BlasterController:_fireOnce()
506     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
507     local config = Blaster.GetWeaponConfig(self.tool)
508     local now = os.clock()
509     if now < self.nextLocalShot then return end
510     self.nextLocalShot = now + config.cooldown
511     self.sequence = self.sequence + 1
512
513     local camera = workspace.CurrentCamera
514     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
515     local recoilScale = self.aiming and 0.55 or 1
516     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
517     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
518 end
519
520 function BlasterController:_beginFiring()
521     if os.clock() < (self.ignoreGuiFireUntil or 0) then return end
522     self.triggerHeld = true
523     if self.fireLoopRunning then return end
524     self.fireLoopRunning = true
525     task.spawn(function()
526         repeat
527             self:_fireOnce()
528             local config = Blaster.GetWeaponConfig(self.tool)
529             if not config.automatic then break end
530             task.wait(config.cooldown)
531         until not self.triggerHeld or not self.enabled
532         self.fireLoopRunning = false
533     end)
534 end
535
536 function BlasterController:_shootAction(_, inputState, inputObject)
537     if inputState == Enum.UserInputState.Begin and inputObject and inputObject.UserInputType ==
538     ↪ Enum.UserInputType.MouseButton1 then
539         local hud=localPlayer.PlayerGui:FindFirstChild("GWAPHUD")
540         local point=UserInputService:GetMouseLocation()
541         for _,object in ipairs(localPlayer.PlayerGui:GetGuiObjectsAtPosition(point.X,point.Y)) do
542             if hud and object:IsDescendantOf(hud) then
543                 self.ignoreGuiFireUntil=os.clock()+.15
544                 return Enum.ContextActionResult.Pass
545             end
546         end
547     if not self.enabled then return Enum.ContextActionResult.Pass end
548     if inputState == Enum.UserInputState.Begin and inputObject
549     and inputObject.UserInputType == Enum.UserInputType.MouseButton1 then
550         local point = UserInputService:GetMouseLocation() - GuiService:GetGuiInset()
551         if pointInside(self.fallbackAimButton, point) then return Enum.ContextActionResult.Pass end
552     end
553     if inputState == Enum.UserInputState.Begin then
554         self:_beginFiring()
555     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
556         self.triggerHeld = false
557     end
558     return Enum.ContextActionResult.Sink
559 end
560
561 function BlasterController:_aimAction(_, inputState, inputObject)
562     if not self.enabled then return Enum.ContextActionResult.Pass end
563     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
564     if isTouch then
565         if inputState == Enum.UserInputState.Begin then
566             self:SetAiming(not self.aiming)
567         end
568     elseif inputState == Enum.UserInputState.Begin then
569         self:SetAiming(true)
570     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
571         self:SetAiming(false)
572     end
573     return Enum.ContextActionResult.Sink
574 end
575
576 function BlasterController:_touchPanAction(_, inputState, inputObject)
577     if not self.enabled or not touchControlsActive() then
578         return Enum.ContextActionResult.Pass

```



```

579     end
580
581     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
582     local camera = workspace.CurrentCamera
583     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
584     local inset = GuiService:GetGuiInset()
585     point = point - inset
586
587     if inputState == Enum.UserInputState.Begin then
588         local shootButton = self.fallbackShootButton or self:_button(self.actionPrefix .. "Shoot")
589         local aimButton = self.fallbackAimButton or self:_button(self.actionPrefix .. "Aim")
590         local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
591         local jumpButton = touchGui and touchGui:FindFirstChild("JumpButton", true)
592         if point.X < viewport.X * 0.42 or pointInside(shootButton, point)
593            or pointInside(aimButton, point) or pointInside(jumpButton, point) then
594             return Enum.ContextActionResult.Pass
595         end
596         self.touchInput = inputObject
597         self:_beginTouchCamera()
598         self.touchLastPosition = point
599         return Enum.ContextActionResult.Sink
600     end
601
602     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
603     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
604         local delta = point - self.touchLastPosition
605         self.touchLastPosition = point
606         local sensitivity = self.aiming
607             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
608             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
609         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
610         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
611         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
        ↪ math.rad(78))
612         return Enum.ContextActionResult.Sink
613     end
614
615     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
616         self.touchInput = nil
617         self.touchLastPosition = nil
618         return Enum.ContextActionResult.Sink
619     end
620     return Enum.ContextActionResult.Pass
621 end
622
623 function BlasterController:_updateArmPose(targetPosition, dt)
624     local character = localPlayer.Character
625     local handle = self.tool:FindFirstChild("Handle")
626     if not character or not handle or not targetPosition then return end
627     if not self.aimJoint or not self.aimJoint.Parent then
628         local joints = {}
629         for _, joint in ipairs(character:GetDescendants()) do
630             if (joint:IsA("JointInstance") or joint:IsA("AnimationConstraint")) and joint.Part0 and joint.Part1
631                ↪ then
632                 joints[joint.Part1] = joint
633             end
634         end
635         local chain = {}
636         local part = handle
637         for _ = 1, 8 do
638             local joint = joints[part]
639             if not joint then return end
640             table.insert(chain, 1, joint)
641             if joint.Name == "Right Shoulder" or joint.Name == "RightShoulder" then
642                 self.aimJoint = joint
643                 self.aimBaseC0 = joint.C0
644                 self.aimChain = chain
645                 break
646             end
647             part = joint.Part0
648         end
649         local shoulder = self.aimJoint
650         if not shoulder or not shoulder.Part0 then return end
651         -- Rebuild from the untouched joint/grip and the current animation each frame.
652         -- This preserves R6/R15 rest rotations and avoids accumulating aim offsets.

```

```

653     local jointWorld = shoulder.Part0.CFrame * self.aimBaseC0
654     local predicted = shoulder.Part0.CFrame
655     for _, joint in ipairs(self.aimChain) do
656         if not joint.Parent then return end
657         local c0 = joint == shoulder and self.aimBaseC0 or joint.C0
658         local animation = CFrame.identity -- armed arm chain is isolated from idle/walk animation
659         predicted = predicted * c0 * animation * joint.C1:Inverse()
660     end
661     local forward = self.tool:GetAttribute("BlasterBarrelDirection")
662     if typeof(forward) ~= "Vector3" or forward.Magnitude < 0.1 then
663         -- Imported Weapons Kit meshes face +Z; procedural barrels face -Z.
664         forward = self.tool:GetAttribute("LabSidearm") and Vector3.zAxis or -Vector3.zAxis
665     end
666     local turn = CFrame.identity
667     local pivot = jointWorld.Position
668     for _ = 1, 4 do
669         local delta = targetPosition - predicted.Position
670         if delta.Magnitude < 0.1 then break end
671         local correction = rotationBetween(predicted:VectorToWorldSpace(forward.Unit), delta.Unit)
672         predicted = CFrame.new(pivot) * correction * CFrame.new(-pivot) * predicted
673         turn = correction * turn
674     end
675     local pose = shoulder.Part0.CFrame:ToObjectSpace(CFrame.new(pivot) * turn * jointWorld.Rotation)
676     self.smoothedArmPose = (self.smoothedArmPose or pose):Lerp(pose, 1 - math.exp(-math.min(dt or 1/60, .1) *
        ↪ 24))
677     pose = self.smoothedArmPose
678     if shoulder:IsA("AnimationConstraint") then
679         shoulder.Attachment0.CFrame = pose
680     else
681         shoulder.C0 = pose
682     end
683 end
684
685 function BlasterController:_render(dt)
686     if not self.enabled then return end
687     local character = localPlayer.Character
688     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
689     local root = character and character:FindFirstChild("HumanoidRootPart")
690     local head = character and character:FindFirstChild("Head")
691     local camera = workspace.CurrentCamera
692     if not humanoid or not camera then return end
693     local hudRebuilt = self:_ensureHUD()
694     local touchMode = touchControlsActive()
695     local touchAim = touchAimActive()
696     if touchAim and not self.mobileCameraActive then
697         self:_beginTouchCamera()
698     elseif not touchAim and self.mobileCameraActive then
699         self:_endTouchCamera()
700     end
701     local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
702     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
703     if touchMode and (hudRebuilt or self.lastTouchViewport ~= camera.ViewportSize
704         or (jump and jump.AbsolutePosition ~= self.lastJumpPosition)) then
705         self.lastTouchViewport = camera.ViewportSize
706         task.defer(function()
707             if self.enabled then self:_styleTouchButtons() end
708         end)
709     end
710     -- Keep the inventory grip fixed: no idle or walking weapon bob.
711
712     self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
713     self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
714     if self.mobileCameraActive and head and root then
715         self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
716         self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
717         local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
718         local focus = root.Position + Vector3.new(0, self.stableHeadHeight or 1.5, 0)
719         local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
720         local cameraResult = workspace:Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
        ↪ self.tool}))
721         local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
722         camera.CFrame = CFrame.new(position) * rotation
723         camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
724
725     elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
726         camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)

```

```

727     end
728     local config = Blaster.GetWeaponConfig(self.tool)
729     local cast = Blaster.CastFromCamera(camera, config.range, {character, self.tool, camera},
    ↪ self:_getAimScreenPoint(camera))
730     if cast then
731         self:_updateArmPose(cast.position, dt)
732     end
733     self:_updateAimReticle(camera)
734 end
735
736 local activeController
737 function BlasterController:Enable()
738     if self.enabled or self.tool.Parent ~= localPlayer.Character then return end
739     if activeController and activeController ~= self then activeController:Disable() end
740     activeController = self
741     self.enabled = true
742     self.baseGrip = self.tool.Grip
743     local rootPart = localPlayer.Character and localPlayer.Character:FindFirstChild("HumanoidRootPart")
744     local headPart = localPlayer.Character and localPlayer.Character:FindFirstChild("Head")
745     self.stableHeadHeight = rootPart and headPart and (headPart.Position.Y - rootPart.Position.Y + .12) or 1.5
746     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
    ↪ workspace.CurrentCamera.FieldOfView) or 70
747
748     self:_ensureHUD()
749     self.hud.Enabled = true
750
751     local character = localPlayer.Character
752     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
753     if humanoid then
754         self.savedAutoRotate = humanoid.AutoRotate
755         humanoid.AutoRotate = false
756         self.savedCameraOffset = humanoid.CameraOffset
757         self.shoulderOffset = Vector3.new(
758             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
759             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
760             0
761         )
762         humanoid.CameraOffset = self.shoulderOffset
763     end
764     if touchAimActive() then self:_beginTouchCamera() end
765
766     local shootName = self.actionPrefix .. "Shoot"
767     local aimName = self.actionPrefix .. "Aim"
768     local panName = self.actionPrefix .. "TouchPan"
769     local priority = Enum.ContextActionPriority.High.Value
770
771     ContextActionService.BindActionAtPriority(shootName, function(...)
772         return self:_shootAction(...)
773     end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
774     ContextActionService.BindActionAtPriority(aimName, function(...)
775         return self:_aimAction(...)
776     end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
777     ContextActionService.BindActionAtPriority(panName, function(...)
778         return self:_touchPanAction(...)
779     end, false, priority + 1, Enum.UserInputType.Touch)
780
781     ContextActionService.SetTitle(shootName, "FIRE")
782     ContextActionService.SetTitle(aimName, "AIM")
783     task.delay(0.1, function()
784         if self.enabled then self:_styleTouchButtons() end
785     end)
786
787     -- Rotate once, before the camera reads the character. Using camera heading
788     -- removes the shoulder-offset/nearby-hit feedback loop that caused oscillation.
789     self.bodyRenderName = self.actionPrefix .. "Body"
790     RunService:BindToRenderStep(self.bodyRenderName, Enum.RenderPriority.Camera.Value - 1, function()
791         if not self.enabled then return end
792         local char = localPlayer.Character
793         local hum = char and char:FindFirstChildOfClass("Humanoid")
794         local body = char and char:FindFirstChild("HumanoidRootPart")
795         local cam = workspace.CurrentCamera
796         if not hum or hum.Sit or not body or not cam then return end
797         local direction = self.mobileCameraActive and CFrame.fromOrientation(0, self.touchYaw, 0).LookVector or
    ↪ cam.CFrame.LookVector
798         local flat = Vector3.new(direction.X, 0, direction.Z)
799         if flat.Magnitude > .01 then body.CFrame = CFrame.lookAt(body.Position, body.Position + flat.Unit) end

```

```

800     end)
801     -- Animator writes Transform before PreSimulation. Reset only the armed chain,
802     -- preserving locomotion animation everywhere else.
803     self.armAnimationConnection = RunService.PreSimulation:Connect(function()
804         if not self.enabled or not self.aimChain then return end
805         for _,joint in ipairs(self.aimChain) do if joint.Parent and joint:IsA("Motor6D") then joint.Transform =
            ↪ CFrame.identity end end
806         local char=localPlayer.Character
807         if char then
808             self.torsoAnimationJoints=self.torsoAnimationJoints or {}
809             if #self.torsoAnimationJoints==0 then for _,joint in ipairs(char:GetDescendants()) do
810                 if joint:IsA("Motor6D") and (joint.Name=="Waist" or joint.Name=="RootJoint" or
            ↪ joint.Name=="Root") then table.insert(self.torsoAnimationJoints,joint) end
811             end end
812             for _,joint in ipairs(self.torsoAnimationJoints) do if joint.Parent then
            ↪ joint.Transform=CFrame.identity end end
813         end
814     end)
815     self.renderName = self.actionPrefix .. "Render"
816     RunService:BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
817         self:_render(dt)
818     end)
819 end
820
821 function BlasterController:Disable()
822     if not self.enabled then return end
823     self.triggerHeld = false
824     self:SetAiming(false)
825     self.enabled = false
826     if activeController == self then activeController = nil end
827     self.smoothedArmPose = nil
828     self.torsoAnimationJoints = nil
829     if self.armAnimationConnection then self.armAnimationConnection:Disconnect(); self.armAnimationConnection =
        ↪ nil end
830     if self.bodyRenderName then RunService:UnbindFromRenderStep(self.bodyRenderName) end
831
832     ContextActionService:UnbindAction(self.actionPrefix .. "Shoot")
833     ContextActionService:UnbindAction(self.actionPrefix .. "Aim")
834     ContextActionService:UnbindAction(self.actionPrefix .. "TouchPan")
835     if self.renderName then RunService:UnbindFromRenderStep(self.renderName) end
836     if self.hud then self.hud.Enabled = false end
837     if self.tool.Parent then self.tool.Grip = self.baseGrip end
838     if self.aimJoint and self.aimJoint.Parent then
839         if self.aimJoint:IsA("AnimationConstraint") then
840             self.aimJoint.Attachment0.CFrame = self.aimBaseC0
841         else
842             self.aimJoint.C0 = self.aimBaseC0
843         end
844     end
845     self.aimJoint, self.aimBaseC0, self.aimChain = nil, nil, nil
846
847     local camera = workspace.CurrentCamera
848     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
849     self:_endTouchCamera()
850     if self.savedMouseSensitivity then
851         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
852         self.savedMouseSensitivity = nil
853     end
854     local character = localPlayer.Character
855     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
856     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
857     self.savedAutoRotate = nil
858     if humanoid and self.savedCameraOffset then
859         humanoid.CameraOffset = self.savedCameraOffset
860     end
861     self.savedCameraOffset = nil
862     self.lastTouchViewport = nil
863 end
864
865 function BlasterController:Destroy()
866     self:Disable()
867     for _, connection in pairs(self.connections) do
868         connection:Disconnect()
869     end
870     table.clear(self.connections)
871     if self.hud then self.hud:Destroy() end

```

```
872     controllers[self.tool] = nil
873 end
874
875 function BlasterController.BindTool(tool, overrides)
876     if controllers[tool] then return controllers[tool] end
877     local controller = BlasterController.new(tool, overrides)
878     controllers[tool] = controller
879     Blaster.StartReplication()
880     return controller
881 end
882
883 function BlasterController.GetController(tool)
884     return controllers[tool]
885 end
886
887 function BlasterController.AutoBind()
888     local binder = {connections = {}}
889
890     local function consider(instance)
891         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
892             ↪ instance:GetAttribute("LabSidearm")) then
893             BlasterController.BindTool(instance)
894         end
895     end
896
897     local function watch(container)
898         if not container then return end
899         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
900         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
901     end
902
903     watch(localPlayer.WaitForChild("Backpack"))
904     watch(localPlayer.Character)
905     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
906     Blaster.StartReplication()
907
908     function binder:Destroy()
909         for _, connection in ipairs(self.connections) do connection:Disconnect() end
910         table.clear(self.connections)
911     end
912
913     return binder
914 end
915 return BlasterController
```

Chapter 35

S08 - GWAPServer

ServerScriptService.GWAPServer

Authority group: active. 32199 UTF-8 bytes; 469 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====]
2  Revision 4 engineering reference | 13 September 2026
3  Authority: ServerScriptService.GWAPServer
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5  Do not require the documentation module from gameplay or replicate its prose to clients.
6  Purpose: Mission orchestration
7  Future change: Extract mission state/reward transactions without changing existing role meanings. Introduce
   ↳ route revision cancellation consistently.
8  Preserve dependencies: All GWAP modules, LabRoles, Points, LabRound.
9  Acceptance cases: Concurrent players, damage during processing, repeated replies, respawn, departure, and
   ↳ terminal reward idempotence.
10 ]=====]
11 -- Authoritative assignments, research carriers, solo opposition and score accounting.
12 local Players=game:GetService('Players')
13 local RS=game:GetService('ReplicatedStorage')
14 local CS=game:GetService('CollectionService')
15 local SSS=game:GetService('ServerScriptService')
16 local Debris=game:GetService('Debris')
17 local Run=game:GetService('RunService')
18 local NPC=require(RS:WaitForChild('GWAPNPC'))
19 local Spatial=require(RS:WaitForChild('LabSpatial'))
20 local refreshArmed=function() end
21 local Pathfinding=game:GetService('PathfindingService')
22 local pool=Instance.new('Folder'); pool.Name='GWAPCarrierPool'; pool.Parent=game:GetService('ServerStorage')
23 local companions={}
24 local World=require(RS:WaitForChild('GWAPWorld'))
25 local scenarios=require(RS:WaitForChild('GWAPScenarios'))
26 local research=World.build()
27 local Router=require(RS:WaitForChild('GWAPRouting'))
28 local stations=Router.ensureBackups()
29 local stationInfo={}
30 for _,item in ipairs(stations:GetChildren()) do
   ↳ stationInfo[item.Name]={capability=item:GetAttribute('Capability'),floor=Router.floorOf(item)} end
31 local remote=Instance.new('RemoteEvent'); remote.Name='GWAPAction'; remote.Parent=RS
32 local bridge=Instance.new('BindableFunction'); bridge.Name='GWAPHitBridge'; bridge.Parent=SSS
33 local botsFolder=Instance.new('Folder'); botsFolder.Name='Carriers'; botsFolder.Parent=research
34 local routesFolder=Instance.new('Folder'); routesFolder.Name='Routes'; routesFolder.Parent=research
35 local random=Random.new()
36 local carriers, assignments, cooldowns, armed={}, {}, {}, {}
37 local function alive(p)
38     local c=p.Character; local h=c and c:FindFirstChildOfClass('Humanoid'); return h and h.Health>0 and
   ↳ c:FindFirstChild('HumanoidRootPart')
39 end
40 local function points(p)
41     local s=p:FindFirstChild('leaderstats'); return s and s:FindFirstChild('Points')
42 end
43 local function award(p,n,reason)
44     local v=points(p); if not v then return end
45     v.Value=math.max(0,v.Value+n); p:SetAttribute('GWAPNotice',reason); p:SetAttribute('LastPointAmount',n);
   ↳ p:SetAttribute('LastPointAward',reason)
```

```

46 end
47 local function role(p) return p and (p:GetAttribute('LabRole') or (p.Team and p.Team.Name)) end
48 local function active(a) return a and a.bot and a.bot.Parent and a.bot:GetAttribute('State')=='RUNNING' end
49 local function nearby(p,bot,d) local r=alive(p); return r and (r.Position-bot:GetPivot().Position).Magnitude<=d
↳ end
50 local function publish(a)
51     for p,bound in pairs(assignments) do if bound==a then
52         p:SetAttribute('GWAPStage',a.stage); p:SetAttribute('GWAPTotalStages',#a.route)
53         p:SetAttribute('GWAPTarget',a.target and a.target.Name or a.route[a.stage] or 'Complete');
↳ p:SetAttribute('GWAPTargetFloor',a.target and Router.floorOf(a.target) or nil);
↳ p:SetAttribute('GWAPState',a.bot:GetAttribute('State'))
54     end end
55 end
56 local function tell(a,message)
57     NPC.say(a.bot,message)
58     for p,bound in pairs(assignments) do if bound==a then p:SetAttribute('GWAPDialogue',message);
↳ p:SetAttribute('GWAPSpeaker',a.bot:GetAttribute('DisplayName') or 'Research carrier') end end
59 end
60 local function finish(a,state)
61     if not active(a) then return end
62     a.bot:SetAttribute('State',state); a.bot:SetAttribute('TaskState',state);
↳ a.bot:SetAttribute('HasSeedling',false)
63     local carried=a.bot:FindFirstChild('Mission seedling'); if carried then carried:Destroy() end
64     local route=routesFolder:FindFirstChild(a.bot.Name);if route then route:Destroy() end
65     publish(a); a.finishedAt=os.clock(); tell(a,state=='COMPLETE' and 'The plant is in the greenhouse. Research
↳ assignment complete. Thank you!' or 'Mission stopped: '..state..'. Choose another assignment when
↳ ready.')
66 end
67 local function updateHealth(bot)
68     local t=bot:FindFirstChild('HealthText',true); local fill=bot:FindFirstChild('HealthBarFill',true)
69     local h=bot:GetAttribute('Health') or 100
70     local status=bot:FindFirstChild('Status',true)
71     if status and status:IsA('TextLabel') then status.Text='RESEARCH CARRIER\n'..(bot:GetAttribute('Title') or
↳ 'Choose an assignment') end
72     if t then t.Text='LIFE '..math.ceil(h).. / 100' end
73     if fill then fill.Size=UDim2.fromScale(h/100,1) end
74 end
75 local function damage(p,bot,amount)
76     local a=carriers[bot]; if not active(a) then return end
77     local h=math.max(0,bot:GetAttribute('Health')-amount); bot:SetAttribute('Health',h); updateHealth(bot)
78     if role(p)=='Technician' and assignments[p]==a then a.disqualified[p]=true;
↳ p:SetAttribute('GWAPNotice','Protect your carrier: friendly hits cancel escort rewards.') end
79     if h==0 then
80         if assignments[p]==a and role(p)=='Laboratory Technician' then award(p,50,'+50 | Assigned research
↳ carrier stopped') end
81         finish(a,'INTERCEPTED')
82     end
83 end
84
85 local equipmentOriginal={}
86 for _,item in ipairs(stations:GetChildren()) do
87     if item:IsA('BasePart') then
88         local saved={}
89         local function remember(v) if v:IsA('BasePart') then
↳ saved[v]={v.Color,v.Material,v.Transparency,v.CanQuery} end end
90         remember(item); for _,v in ipairs(item:GetDescendants()) do remember(v) end
91         equipmentOriginal[item]=saved
92         item:SetAttribute('GWAPEquipment',true)
93     end
94 end
95 local function repair(item)
96     item:SetAttribute('Exploded',false); item:SetAttribute('FunctionalState','RUNNING')
97     for part,props in pairs(equipmentOriginal[item] or {}) do if part.Parent then part.Color=props[1];
↳ part.Material=props[2]; part.Transparency=props[3]; part.CanQuery=props[4] end end
98     for _,v in ipairs(item:GetChildren()) do if v.Name=='GWAPFire' or v.Name=='GWAPSmoke' then v:Destroy() end
↳ end
99     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=false end
100 end
101 local explode
102 explode=function(item,attacker)
103     if not equipmentOriginal[item] or item:GetAttribute('Exploded') then return end
104     item:SetAttribute('Exploded',true); item:SetAttribute('FunctionalState','DESTROYED')
105     for part in pairs(equipmentOriginal[item]) do if part.Parent then part.Color=Color3.fromRGB(49,42,38);
↳ part.Material=Enum.Material.CorrodedMetal end end
106     local fire=Instance.new('Fire'); fire.Name='GWAPFire'; fire.Size=5; fire.Heat=4; fire.Parent=item;
↳ Debris:AddItem(fire,8)

```

```

107     local smoke=Instance.new('Smoke'); smoke.Name='GWAPSmoke'; smoke.Size=4; smoke.Opacity=.3;
    ↪ smoke.Parent=item; Debris:AddItem(smoke,12)
108     local blast=Instance.new('Explosion'); blast.Position=item.Position; blast.BlastRadius=13;
    ↪ blast.BlastPressure=0; blast.DestroyJointRadiusPercent=0;
    ↪ blast.ExplosionType=Enum.ExplosionType.NoCraters
109     if attacker then blast:SetAttribute('GWAPAttacker',attacker.UserId) end
110     blast.Parent=workspace
111     for _,a in pairs(carriers) do if active(a) then
112         if (a.bot:GetPivot().Position-item.Position).Magnitude<13 then damage(attacker,a.bot,60) end
113         if a.target==item then tell(a,item.Name..' was destroyed. I will find a working equivalent.');
```

end

```

114     end end
115     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=true end
116 end
117 for item in pairs(equipmentOriginal) do
118     local prompt=Instance.new('ProximityPrompt'); prompt.Name='ResearchRepair'; prompt.ActionText='Repair
    ↪ research equipment'; prompt.ObjectText=item.Name; prompt.KeyboardKeyCode=Enum.KeyCode.R;
    ↪ prompt.HoldDuration=3; prompt.MaxActivationDistance=12; prompt.Enabled=false; prompt.Parent=item
119     -- Validate the server interaction; client prompt visibility is not authority.
120     prompt.Triggered:Connect(function(p)
121         local root=alive(p)
122         if not root or role(p)~='Technician' or not item:IsDescendantOf(workspace)
123             or item:GetAttribute('BlastRemoved') or not item:GetAttribute('Exploded')
124             or (root.Position-item.Position).Magnitude>14 then return end
125         local head=p.Character:FindFirstChild('Head') or root
126         local params=RaycastParams.new();params.FilterType=Enum.RaycastFilterType.Exclude;params.FilterDescend
    ↪ antsInstances={p.Character}
127         local hit=workspace:Raycast(head.Position,item.Position-head.Position,params)
128         if hit and hit.Instance~=item and not hit.Instance:IsDescendantOf(item) then return end
129         repair(item);p:SetAttribute('GWAPNotice',item.Name..' repaired. Ready for research.')
130     end)
131 end
132 workspace.DescendantAdded:Connect(function(v)
133     if not v:IsA('Explosion') then return end
134     local center,radius=v.Position,v.BlastRadius
135     local userId=v:GetAttribute('GWAPAttacker'); local attacker=userId and Players:GetPlayerByUserId(userId)
136     -- Deferred bounded propagation: each station can explode only once until repaired.
137     task.delay(.12,function()
138         for item in pairs(equipmentOriginal) do if item:IsDescendantOf(workspace) and not
    ↪ item:GetAttribute('BlastRemoved') and not item:GetAttribute('Exploded') and
    ↪ (item.Position-center).Magnitude<=radius+item.Size.Magnitude*.25 then
139             local delta=item.Position-center
140             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ params.FilterDescendantsInstances={v}
141             local hit=delta.Magnitude>.1 and workspace:Raycast(center,delta,params)
142             if not hit or hit.Instance==item or hit.Instance:IsDescendantOf(item) or
    ↪ hit.Instance:GetAttribute('Exploded') then explode(item,attacker) end
143         end end
144     end)
145 end)
146
147 bridge.OnInvoke=function(p,hit,damageAmount)
148     local node=hit
149     while node and node~=workspace do if equipmentOriginal[node] then explode(node,p); return true end; if
    ↪ carriers[node] then damage(p,node,damageAmount); return true end; node=node.Parent end
150     return false
151 end
152 local function routePoints(from,to)
153     local r={}; local function add(v) table.insert(r,Spatial.world(v)) end
154     from=Spatial.LocalPoint(from)
155     local f1=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
156     local f2=to:GetAttribute('Floor'); local dest=Spatial.LocalPoint(Spatial.approach(to))
157     if f1~=f2 then
158         add(Vector3.new(78,from.Y,from.Z)); add(Vector3.new(78,from.Y,-6))
159         local function flight(level,reverse)
160             local steps={Vector3.new(68.7857,level*16.25+3.25,-6)}
161             for i=1,8 do
    ↪ table.insert(steps,Vector3.new(68.7857,level*16.25+i*1.015625+3.25,-15.267857-(i-1)*4.821428))
    ↪ end
162             table.insert(steps,Vector3.new(68.7857,level*16.25+8.125+3.25,-55.28))
163             table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+3.25,-55.28))
164             for i=1,8 do table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+i*1.015625+3.25,-49.017857+(
    ↪ i-1)*4.821428)) end
165             table.insert(steps,Vector3.new(88.0714,(level+1)*16.25+3.25,-6));
    ↪ table.insert(steps,Vector3.new(78,(level+1)*16.25+3.25,-6))
166             if reverse then for i=#steps,1,-1 do add(steps[i]) end else for _,v in ipairs(steps) do add(v) end
    ↪ end

```



```

167         end
168         if f2>f1 then for level=f1,f2-1 do flight(level,false) end else for level=f1-1,f2,-1 do
            ↪ flight(level,true) end end
169     end
170     add(Vector3.new(78,dest.Y,dest.Z)); add(dest); return r
171 end
172 local function showRoute(a,route)
173     local old=routesFolder:FindFirstChild(a.bot.Name); if old then old:Destroy() end
174     local folder=Instance.new('Folder'); folder.Name=a.bot.Name; folder.Parent=routesFolder
175     for i,v in ipairs(route) do local n=Instance.new('Vector3Value'); n.Name=string.format('%03d',i);
        ↪ n.Value=v; n.Parent=folder end
176 end
177 local function go(a,target)
178     local revision=a.routeRevision or 0
179     if not Router.healthy(target) then return false end
180     local route
181     if a.needsRendezvous then
182         local from=a.bot:GetPivot().Position
183         local floor=Spatial.floor(from,3.25)
184         local localFrom=Spatial.localPoint(from)
185         local entry=Spatial.world(Vector3.new(78,floor*16.25+3.25,math.clamp(localFrom.Z,24,240)))
186         route={}
187         if (from-entry).Magnitude>.5 then
188             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
            ↪ params.FilterDescendantsInstances={a.bot,a.owner and a.owner.Character};
            ↪ params.RespectCanCollide=true
189             local blocked=workspace:Blockcast(CFrame.new(from),Vector3.new(2.5,4.4,2.5),entry-from,params)
190             if not blocked and math.abs(from.Y-entry.Y)<1 then table.insert(route,entry) else
191                 local path=Pathfinding.CreatePath({AgentRadius=1.8,AgentHeight=5,AgentCanJump=false,WaypointSp
                    ↪ acing=5})
192                 local ok=pcall(function() path:ComputeAsync(from,entry) end)
193                 if not ok or path.Status~=Enum.PathStatus.Success then tell(a,'I cannot reach the corridor from
                    ↪ here. Move to an open area and ask me to regroup.');
```

```

232         a.lastObstacleMessage=os.clock();tell(a,'The route is obstructed. Clear the way, or ask me
        ↳ to Wait and Regroup from an open corridor.')
233     end
234 end
235 for _,p in ipairs(Players:GetPlayers()) do local r=alive(p); if r and
        ↳ (r.Position-nextPos).Magnitude<2.8 then blocked=true end end
236 if not blocked then
237     local flat=Vector3.new(delta.X,0,delta.Z)
238     local previous=a.bot:GetPivot()
239     local desired=flat.Magnitude>.01 and CFrame.lookAt(nextPos,nextPos+flat) or
        ↳ CFrame.new(nextPos)*previous.Rotation
240     a.bot:PivotTo(CFrame.new(nextPos)*previous.Rotation:Lerp(desired.Rotation,1-math.exp(-dt*10)))
241 end
242 if os.clock()>a.deadline then finish(a,'EXPIRED'); return false end
243 end
244 if not active(a) then return false end
245 end
246 return true
247 end
248 local function plantIn(bed,name)
249     if bed:GetAttribute('Exploded') then return false end
250     local count=bed:GetAttribute('Planted') or 0
251     if count>=12 then return false end
252     local pos=bed.Position+Vector3.new(-8+(count%4)*5,1,-3+math.floor(count/4)*3)
253     World.plant(bed,pos,name); bed:SetAttribute('Planted',count+1)
254     local sign=bed:FindFirstChild('StationLabel'); local label=sign and sign:FindFirstChildOfClass('TextLabel')
255     if label then label.Text=bed.Name..' | '..(count+1)..' / 12 plants' end
256     return true
257 end
258 local function checkpoint(a,final)
259     for p,bound in pairs(assignments) do
260         if bound==a and role(p)=='Technician' and not a.disqualified[p] and nearby(p,a.bot,45) then
        ↳ award(p,final and 60 or 15,final and '+60 | Protected rooftop planting' or '+15 | Protected
        ↳ research checkpoint') end
261     end
262 end
263 local function runMission(a)
264     for i,originalName in ipairs(a.route) do
265         a.stage=i
266         local original=stations:FindFirstChild(originalName)
267         local info=stationInfo[originalName]
268         local capability=info and info.capability or original and original:GetAttribute('Capability') or
        ↳ (originalName:match('^Plot ') and 'Greenhouse planting' or originalName)
269         local preferredFloor=info and info.floor or original and Router.floorOf(original) or 0
270         local completed=false
271         while active(a) and not completed do
272             local target=Router.healthy(a.target) and a.target:GetAttribute('Capability')==capability and
        ↳ a.target
273             or (Router.healthy(original) and original or
        ↳ Router.choose(stations,capability,preferredFloor,a.bot:GetPivot().Position))
274             if not target then
275                 a.target=nil; publish(a)
276                 if os.clock()-(a.lastNoEquipment or 0)>8 then a.lastNoEquipment=os.clock(); tell(a,'No working
        ↳ '..capability..' is available. Repair one with R; I will resume this assignment.') end
277                 a.deadline+=1; task.wait(1); continue
278             end
279             if target~=a.target then
280                 a.target=target; publish(a)
281                 if target~=original then a.bot:SetAttribute('Reroutes',(a.bot:GetAttribute('Reroutes') or 0)+1)
        ↳ end
282                 tell(a,target~=original and ('Rerouting to '..target.Name..' on '..(Router.floorOf(target)==-1
        ↳ and 'B1' or Router.floorOf(target)==8 and 'the roof' or
        ↳ 'L'..(Router.floorOf(target)+1))..' Same research function, new route.')
283                 or 'Next stop: '..target.Name..' Stay with me.')
284             end
285             a.bot:SetAttribute('TaskState','Travel to '..target.Name)
286             local reached=go(a,target)
287             if not active(a) then return end
288             if not reached then
289                 a.needsRendezvous=true
290                 if a.paused then repeat task.wait(.25) until not active(a) or not a.paused end
291                 task.wait(.1); continue
292             end
293             tell(a,'At '..target.Name..' Processing the seedling now.')
294             a.bot:SetAttribute('TaskState','Processing: '..target.Name)
295             local seed=a.bot:FindFirstChild('Mission seedling')

```

```

296         if seed then seed:PivotTo(CFrame.new(target.Position+Vector3.new(0,2,0))) end
297         local work=0
298         while active(a) and Router.healthy(target) and work<4 do task.wait(.1); if not a.paused then
299             ↪ work+=.1 end end
300         if not active(a) then return end
301         if not Router.healthy(target) then a.needsRendezvous=true; continue end
302         if i==#a.route then
303             if plantIn(target,a.title..' | '..a.variant) then checkpoint(a,true); finish(a,'COMPLETE');
304             ↪ return end
305             a.target=nil; original=nil; task.wait(.1); continue
306         end
307         if i==1 and not seed then a.bot:SetAttribute('HasSeedling',true);
308         ↪ seed=World.plant(a.bot,a.bot:GetPivot().Position+Vector3.new(0,0,-2),'Mission seedling') end
309         if seed then seed:PivotTo(CFrame.new(a.bot:GetPivot().Position+Vector3.new(0,0,-2))) end
310         checkpoint(a,false); completed=true; a.target=nil
311     end
312     if not active(a) then return end
313 end
314 local function startMission(bot,id,p)
315     local s=scenarios[id]; local route=table.clone(s.route); table.insert(route,'Plot '..id)
316     local a={bot=bot,id=id,title=s.title,route=route,stage=1,disqualified={},owner=p,needsRendezvous=true,paused=true,deadline=os.clock()+600}
317     a.variant=s.variable..'': '..random:NextInteger(s.low,s.high)..' '..s.units
318     carriers[bot]=a; bot:SetAttribute('State','RUNNING'); bot:SetAttribute('Health',100);
319     ↪ bot:SetAttribute('Scenario',id); bot:SetAttribute('Title',s.title);
320     ↪ bot:SetAttribute('Variant',a.variant); updateHealth(bot)
321     bot.Parent=botsFolder
322     if not NPC.nearPlayer(p,bot) then bot.Parent=pool; carriers[bot]=nil; return nil end
323     return a
324 end
325 local function makeBot(index)
326     local template
327     local choices={}
328     for _,b in ipairs(CS:GetTagged('LabWorkerBot')) do if not b:GetAttribute('Seated') and b.PrimaryPart and
329         ↪ ((index%2==1)==(b:GetAttribute('Gender')=='Girl')) then table.insert(choices,b) end end
330     table.sort(choices,function(a,b) return a.Name<b.Name end);
331     ↪ template=choices[(index-1)%math.max(1,#choices)+1]
332     assert(template,'No worker template available')
333     local b=template:Clone(); b.Name='ResearchCarrier_'..index
334     for _,v in ipairs(b:GetDescendants()) do if v:IsA('LuaSourceContainer') or v:IsA('ProximityPrompt') or
335         ↪ v:IsA('Humanoid') then v:Destroy() elseif v:IsA('BasePart') then v.Anchored=true; v.CanCollide=false
336         ↪ end end
337     for _,tag in ipairs(CS:GetTags(b)) do CS:RemoveTag(b,tag) end
338     NPC.appearance(b)
339     b:SetAttribute('Dead',false); b:SetAttribute('KnockedOut',false); b:SetAttribute('GWAPCarrier',true);
340     ↪ b.Parent=botsFolder
341     local h=Instance.new('Highlight'); h.Name='ResearchOutline'; h.FillTransparency=1;
342     ↪ h.OutlineColor=Color3.fromRGB(92,244,199); h.DepthMode=Enum.HighlightDepthMode.Occluded; h.Parent=b
343     return b
344 end
345 Run.Heartbeat:Connect(function(dt) for _,bot in ipairs(botsFolder:GetChildren()) do if bot:IsA('Model') then
346     ↪ NPC.animate(bot,dt) end end end)
347 local available={}; for i=1,5 do available[i]=makeBot(i); available[i].Parent=pool;
348     ↪ available[i]:SetAttribute('State','IDLE') end
349 local function companion(p)
350     local bot=companions[p]
351     if bot and bot.Parent then return bot end
352     local reserved={}; for _,b in pairs(companions) do reserved[b]=true end
353     for _,candidate in ipairs(available) do if not reserved[candidate] and not active(carriers[candidate]) then
354         ↪ bot=candidate; break end end
355     if not bot then bot=makeBot(#available+1); table.insert(available,bot) end
356     companions[p]=bot; bot.Parent=botsFolder
357     if not NPC.nearPlayer(p,bot) then bot.Parent=pool; p:SetAttribute('GWAPDialogue','Move to an open spot and
358         ↪ choose an assignment so I can meet you.');
```

```

352     if active(assignments[p]) then p:SetAttribute('GWAPNotice','Finish your current assignment before selecting
    ↪ another. '); return end
353     local bot=companion(p)
354     local a=startMission(bot,id,p)
355     if not a then p:SetAttribute('GWAPNotice','Please move to an open spot so your carrier can appear beside
    ↪ you. '); return end
356     assignments[p]=a; p:SetAttribute('GWAPCarrier',a.bot.Name); p:SetAttribute('GWAPScenario',id);
    ↪ p:SetAttribute('GWAPVariant',a.variant); p:SetAttribute('GWAPNotice',role(p)=='Technician' and 'Escort
    ↪ your carrier. Stay within 45 studs at checkpoints.' or 'Stop your assigned carrier before rooftop
    ↪ planting. '); publish(a)
357     tell(a,'Our assignment is '..a.title..' '..scenarios[id].purpose..' First stop: '..a.route[1]..' Tell me
    ↪ READY when you want to start.')
358     a.started=true; task.spawn(function() while active(a) and a.paused do task.wait(.25);
    ↪ a.deadline=os.clock()+600 end; if active(a) then runMission(a) end end)
359 end
360 remote.OnServerEvent:Connect(function(p,action,index)
361     if typeof(action)~= 'string' or typeof(index)~= 'number' or index~=index or index%1~=0 then return end
362     if os.clock()-(cooldowns[p] or 0)<.3 then return end; cooldowns[p]=os.clock()
363     if action=='choose' and index>=1 and index<=10 then choose(p,index)
364     elseif action=='reply' then
365         local a=assignments[p]; if not active(a) then return end
366         if index==1 then a.paused=false; tell(a,'Ready. Let us continue to '..(a.route[a.stage] or 'the
    ↪ greenhouse')..'')
367         elseif index==2 then a.paused=true; tell(a,'Understood. I will wait here. Press Ready when you want to
    ↪ continue.')
368         elseif index==3 then tell(a,'We are heading to '..(a.target and a.target.Name or a.route[a.stage])..'
    ↪ '..scenarios[a.id].purpose)
369         elseif index==4 then
370             if not a.paused then tell(a,'Ask me to wait before regrouping. '); return end
371             if NPC.nearPlayer(p,a.bot) then a.needsRendezvous=true; a.routeRevision=(a.routeRevision or 0)+1;
    ↪ tell(a,'I am beside you again. Press Ready to continue.') end
372         end
373     end
374 end)
375 for _,bed in ipairs(stations:GetChildren()) do
376     local prompt=bed:FindFirstChild('PlantSeedling')
377     if prompt then prompt.Triggered:Connect(function(p)
378         local r=alive(p); if not r or (r.Position-bed.Position).Magnitude>14 then return end
379         local sample
380         for _,holder in ipairs({p.Character,p:FindFirstChildOfClass('Backpack')}) do if holder then for _,t in
    ↪ ipairs(holder:GetChildren()) do if t:IsA('Tool') and (t:GetAttribute('PlantSample') or
    ↪ t:GetAttribute('TransferSample')) then sample=t; break end end end
381         if not sample then p:SetAttribute('GWAPNotice','Carry a germinated plant sample to this plot. '); return
    ↪ end
382         if plantIn(bed,sample.Name) then sample:Destroy(); p:SetAttribute('CarriedSample','');
    ↪ p:SetAttribute('GWAPNotice','Seedling planted in '..bed.Name) else
    ↪ p:SetAttribute('GWAPNotice','This plot is full. Use another plot.') end
383     end) end
384 end
385 local function setArmed()
386     local all={};local candidates=CS:GetTagged('LabWorkerBot')
387     for _,bot in ipairs(available) do table.insert(candidates,bot) end
388     for _,bot in ipairs(candidates) do
389         if bot:IsDescendantOf(workspace) and not bot:GetAttribute('Dead') and not
    ↪ bot:GetAttribute('KnockedOut') and (bot:GetAttribute('Health') or 100)>0 then table.insert(all,bot)
390         else bot:SetAttribute('GWAPArmed',false);local gun=bot:FindFirstChild('SoloWeapon');if gun then
    ↪ gun:Destroy() end end
391     end
392     table.sort(all,function(a,b) return a.Name<b.Name end)
393     armed={}; local solo=#Players:GetPlayers()==1; local count=solo and math.floor(#all*.2) or 0
394     local selected={}; local shuffle=table.clone(all); for i=#shuffle,2,-1 do local j=random:NextInteger(1,i);
    ↪ shuffle[i],shuffle[j]=shuffle[j],shuffle[i] end
395     for i=1,count do selected[shuffle[i]]=true end
396     for _,bot in ipairs(all) do
397         local yes=selected[bot]==true; bot:SetAttribute('GWAPArmed',yes)
398         local gun=bot:FindFirstChild('SoloWeapon')
399         if yes then table.insert(armed,bot);if not gun then NPC.gun(bot,(bot:GetAttribute('BotNumber') or 1))
    ↪ end
400         elseif gun then gun:Destroy() end
401     end
402     workspace:SetAttribute('GWAPArmedCount',count); workspace:SetAttribute('GWAPBotCount',#all)
403 end
404 refreshArmed=setArmed
405 local retired={['Research SMG']=true,['Precision Rifle']=true,['Scatter Blaster']=true,['Blast
    ↪ Launcher']=true,['Pulse Bomb']=true}
406 local function watchPlayer(p)

```

```

407     local function clean(v) if v:IsA('Tool') and (v:GetAttribute('GWAPWeapon') or retired[v.Name]) then
408         ↪ v:Destroy() end end
409     local function watch(container) for _,v in ipairs(container:GetChildren()) do clean(v) end;
410     ↪ container.ChildAdded:Connect(clean) end
411     task.spawn(function() watch(p:WaitForChild('Backpack')) end)
412     local function onCharacter(c)
413         watch(c)
414         task.delay(1,function() if role(p)=='Technician' or role(p)=='Laboratory Technician' then
415             local a=assignments[p]; if active(a) then a.paused=true; tell(a,'You are back. Ask me to regroup,
416             ↪ then press Ready.') else companion(p) end
417         end end)
418     end
419     if p.Character then onCharacter(p.Character) end; p.CharacterAdded:Connect(onCharacter)
420     p:GetAttributeChangedSignal('LabRole'):Connect(function() if role(p)=='Technician' or role(p)=='Laboratory
421     ↪ Technician' then companion(p) end end)
422     task.defer(setArmed)
423 end
424 Players.PlayerAdded:Connect(watchPlayer)
425 for _,p in ipairs(Players:GetPlayers()) do watchPlayer(p) end
426 for _,bot in ipairs(CS:GetTagged('LabWorkerBot')) do NPC.appearance(bot) end
427 Players.PlayerRemoving:Connect(function(p) local a=assignments[p]; if active(a) then finish(a,'PLAYER LEFT')
428 ↪ end; local bot=companions[p]; if bot then bot.Parent=pool end; companions[p]=nil; assignments[p]=nil;
429 ↪ cooldowns[p]=nil; task.defer(setArmed) end)
430 setArmed()
431 -- Keep eligibility correct after NPC knockout/recovery without rebuilding guns every shot.
432 local lastEligible=''
433 task.spawn(function() while task.wait(5) do
434     local names={}
435     for _,bot in ipairs(CS:GetTagged('LabWorkerBot')) do if bot:IsDescendantOf(workspace) and not
436     ↪ bot:GetAttribute('Dead') and not bot:GetAttribute('KnockedOut') and (bot:GetAttribute('Health') or
437     ↪ 100)>0 then table.insert(names,bot.Name) end end
438     for _,bot in ipairs(available) do if bot:IsDescendantOf(workspace) and (bot:GetAttribute('Health') or
439     ↪ 100)>0 then table.insert(names,bot.Name) end end
440     table.sort(names);local key=tostring(#Players:GetPlayers())..' '..table.concat(names,'|')
441     if key~=lastEligible then lastEligible=key;setArmed() end
442 end end)
443 task.spawn(function()
444     while task.wait(2.5) do
445         local list=Players:GetPlayers(); if #list~=1 or workspace:GetAttribute('RoundPhase')=='BREAK' then
446         ↪ continue end
447         local p=list[1]; local root=alive(p); if not root then continue end
448         for _,bot in ipairs(armed) do
449             if not bot:IsDescendantOf(workspace) or bot:GetAttribute('Dead') or bot:GetAttribute('KnockedOut')
450             ↪ or (bot:GetAttribute('Health') or 100)<=0 then continue end
451             -- Assigned escort carriers never attack their own defender.
452             if assignments[p] and assignments[p].bot==bot and role(p)=='Technician' then continue end
453             local gun=bot:FindFirstChild('SoloWeapon'); local origin=gun and
454             ↪ (gun.Position+gun.CFrame.LookVector*(-gun.Size.Z*.5)) or
455             ↪ bot:GetPivot().Position+Vector3.new(0,1,0); local delta=root.Position-origin
456             if delta.Magnitude<4 or delta.Magnitude>85 then continue end
457             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
458             ↪ params.FilterDescendantsInstances={bot}
459             local hit=workspace:Raycast(origin,delta,params)
460             if hit and hit.Instance:IsDescendantOf(p.Character) then
461                 local spread=Vector3.new(random:NextNumber(-2.5,2.5),random:NextNumber(-1.5,1.5),random:NextNu
462                 ↪ mber(-2.5,2.5))
463                 local shot=workspace:Raycast(origin,delta+spread,params); local endpoint=shot and shot.Position
464                 ↪ or origin+delta+spread
465                 local beam=Instance.new('Part'); beam.Name='NPC shot'; beam.Anchored=true;
466                 ↪ beam.CanCollide=false; beam.CanQuery=false; beam.Material=Enum.Material.Neon;
467                 ↪ beam.Color=Color3.fromRGB(255,105,72);
468                 ↪ beam.Size=Vector3.new(.07,.07,(endpoint-origin).Magnitude);
469                 ↪ beam.CFrame=CFrame.lookAt((origin+endpoint)/2,endpoint); beam.Parent=workspace;
470                 ↪ Debris:AddItem(beam,.18)
471                 if shot and shot.Instance:IsDescendantOf(p.Character) then
472                     local v=points(p); if v then
473                         -- Existing score is an IntValue: accumulate fractional losses, never turn 1% into a
474                         ↪ minimum one-point tax.
475                         local debt=(p:GetAttribute('GWAPFractionalLoss') or 0)+math.max(0,v.Value)*.01
476                         local loss=math.floor(debt+1e-8); p:SetAttribute('GWAPFractionalLoss',debt-loss);
477                         ↪ v.Value=math.max(0,v.Value-loss)
478                         p:SetAttribute('GWAPNotice','NPC hit: -1% score');
479                         ↪ p:SetAttribute('GWAPHitAt',workspace:GetServerTimeNow())
480                     end
481                 end
482             end
483         end
484     end
485 end

```

```
459         end
460     end
461 end)
462 workspace:GetAttributeChangedSignal('RoundNumber'):Connect(function()
463     for _,a in pairs(carriers) do if active(a) then finish(a,'SHIFT ENDED') end end
464     for _,p in ipairs(Players:GetPlayers()) do p:SetAttribute('GWAPFractionalLoss',0) end
465     for item in pairs(equipmentOriginal) do repair(item) end
466     for _,bed in ipairs(stations:GetChildren()) do if bed:GetAttribute('Planted') then for _,v in
        ↪ ipairs(bed:GetChildren()) do if v:IsA('Model') then v:Destroy() end end; bed:SetAttribute('Planted',0);
        ↪ local sign=bed:FindFirstChild('StationLabel'); local label=sign and
        ↪ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=bed.Name..' | Ready for planting' end
        ↪ end end
467     setArmed()
468 end)
469 print('[GWAP] 10 scenarios, 5 research carriers, rooftop planting and solo opposition ready')
```

Chapter 36

S09 - GWAPWallDamage

ServerScriptService.GWAPWallDamage

Authority group: active. 3254 UTF-8 bytes; 55 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====[
2 Revision 4 engineering reference | 13 September 2026
3 Authority: ServerScriptService.GWAPWallDamage
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5 Do not require the documentation module from gameplay or replicate its prose to clients.
6 Purpose: Wall fixture lifecycle
7 Future change: Replace name-pattern coupling with explicit wall/fixture association when geometry is
  ↳ refactored. Define restoration.
8 Preserve dependencies: Legacy gas detonation, research stations, prompts and tags.
9 Acceptance cases: Near/outside fixture, rotated wall, two blasts, round reset, old callback after quarantine.
10 ]=====[
11 -- Remove fixtures belonging to a wall section destroyed by an explosion.
12 local CS=game:GetService('CollectionService')
13 local c=workspace:WaitForChild('Laboratory_Complex')
14 local debris=Instance.new('Folder'); debris.Name='DestroyedWallFixtures';
  ↳ debris.Parent=game:GetService('ServerStorage')
15 local fixtures={}; local recent={}
16 local function register(v)
17     if not v:IsA('BasePart') then return end
18     if v:GetAttribute('ToolKind') or v:GetAttribute('ShootablePoster') or v.Name:match('^HealthStation')
19     or CS:HasTag(v,'LabEquipment') or CS:HasTag(v,'PlantGrowthChamber') or v:GetAttribute('GWAPEquipment')
20     or (v.Parent and v.Parent.Name=='Stations') then fixtures[v]=true end
21 end
22 for _,v in ipairs(c:GetDescendants()) do register(v) end
23 c.DescendantAdded:Connect(register)
24 CS:GetInstanceAddedSignal('LabEquipment'):Connect(register)
25 workspace.DescendantAdded:Connect(function(v)
26     if v:IsA('Explosion') then table.insert(recent,{position=v.Position,radius=v.BlastRadius,time=os.clock()})
  ↳ end
27     while #recent>0 and os.clock()-recent[1].time>3 do table.remove(recent,1) end
28 end)
29 c.DescendantRemoving:Connect(function(wall)
30     if not wall:IsA('BasePart') or not (wall.Name:match('^Corridor_') or wall.Name:match('^Div_') or
  ↳ wall.Name:match('^Ext_') or wall.Name:match('^B_')) then return end
31     local cf,size=wall.CFrame,wall.Size
32     local exploded=false
33     for _,blast in ipairs(recent) do if os.clock()-blast.time<3 then
34         local localPoint=cf:PointToObjectSpace(blast.position)
35         local closest=Vector3.new(math.clamp(localPoint.X,-size.X/2,size.X/2),math.clamp(localPoint.Y,-size.Y/2),
  ↳ 2,size.Y/2),math.clamp(localPoint.Z,-size.Z/2,size.Z/2))
36         if (localPoint-closest).Magnitude<=blast.radius then exploded=true; break end
37     end end
38     if not exploded then return end
39     task.defer(function()
40         if not c.Parent then return end
41         local pad=Vector3.new(size.X<4 and 7 or 1,1,size.Z<4 and 7 or 1)
42         local half=size/2+pad
43         local count=0
44         for item in pairs(fixtures) do if item:IsDescendantOf(c) then
45             local pos=cf:PointToObjectSpace(item.Position)
```

```
46         if math.abs(pos.X)<=half.X and math.abs(pos.Y)<=half.Y and math.abs(pos.Z)<=half.Z then
47             item:SetAttribute('BlastRemoved',true); item:SetAttribute('FunctionalState','DESTROYED');
48             ↪ item:SetAttribute('Exploded',true)
49             for _,v in ipairs(item:GetDescendants()) do if v:IsA('ProximityPrompt') then v.Enabled=false
50             ↪ end end
51             for _,tag in ipairs(CS:GetTags(item)) do CS:RemoveTag(item,tag) end
52             item.Parent=debris; count+=1
53         end
54     end end
55 workspace:SetAttribute('GWAPRemovedFixtures',(workspace:GetAttribute('GWAPRemovedFixtures') or
    ↪ 0)+count)
56 end)
57 end)
```


Chapter 37

S10 - LabElevators

ServerScriptService.LabElevators

Authority group: active. 4405 UTF-8 bytes; 132 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: ServerScriptService.LabElevators  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Elevator compatibility script  
7  Future change: Treat as dormant unless real tagged elevator geometry exists. Do not claim elevator transport  
   ↪ from script presence alone.  
8  Preserve dependencies: Level metadata and any future lift models.  
9  Acceptance cases: Door interlock, floor alignment, queueing, interrupted travel, and stair fallback.  
10 ]=====]  
11 -- LabElevators : moving cars that carry whoever is standing inside.  
12 -- An anchored part does not drag a character with it, so the car  
13 -- offsets every occupant by the same delta on each step.  
14  
15 local CollectionService = game:GetService("CollectionService")  
16 local RunService        = game:GetService("RunService")  
17 local Players            = game:GetService("Players")  
18  
19 local SPEED = 45  -- studs per second  
20  
21 local function setGate(gate, open)  
22     gate.CanCollide = not open  
23     gate.Transparency = open and 0.85 or 0  
24 end  
25  
26 local function setup(model)  
27     local floorCount = model:GetAttribute("FloorCount")  
28     if not floorCount then return end  
29  
30     local car = model:WaitForChild("Car")  
31     local platform = car:FindFirstChild("Platform") or car:FindFirstChild("Platform_1")  
32     if not platform then  
33         for _, d in ipairs(car:GetDescendants()) do  
34             if d:IsA("BasePart") and d.Name:match("^Platform") then platform = d break end  
35         end  
36     end  
37     if not platform then return end  
38  
39     local gates = {}  
40     for _, g in ipairs(model:WaitForChild("Gates"):GetChildren()) do  
41         gates[g:GetAttribute("Floor")] = g  
42     end  
43  
44     local ys = {}  
45     for i = 1, floorCount do ys[i] = model:GetAttribute("FloorY_" .. i) end  
46  
47     local current, moving = 1, false  
48     for i, g in pairs(gates) do setGate(g, i == current) end  
49  
50     local carParts = {}
```

```

51     for _, d in ipairs(car:GetDescendants()) do
52         if d:IsA("BasePart") then table.insert(carParts, d) end
53     end
54
55     local function occupants()
56         local list = {}
57         local half = platform.Size * 0.5
58         local top = platform.Position.Y + half.Y
59         for _, pl in ipairs(Players:GetPlayers()) do
60             local ch = pl.Character
61             local hrp = ch and ch:FindFirstChild("HumanoidRootPart")
62             if hrp then
63                 local d = hrp.Position - platform.Position
64                 if math.abs(d.X) <= half.X + 2 and math.abs(d.Z) <= half.Z + 2
65                     and hrp.Position.Y > top - 2 and hrp.Position.Y < top + 24 then
66                     table.insert(list, hrp)
67                 end
68             end
69         end
70         return list
71     end
72
73     local function moveTo(target)
74         if moving or target == current or not ys[target] then return end
75         moving = true
76         setGate(gates[current], false)
77
78         local startY = platform.Position.Y
79         local goalY = startY + (ys[target] - ys[current])
80         local dist = math.abs(goalY - startY)
81         local dur = math.max(0.4, dist / SPEED)
82         local t0 = os.clock()
83         local lastY = startY
84
85         while true do
86             local a = math.min(1, (os.clock() - t0) / dur)
87             local newY = startY + (goalY - startY) * a
88             local dy = newY - lastY
89             if dy ~= 0 then
90                 local riders = occupants()
91                 for _, p in ipairs(carParts) do
92                     p.CFrame = p.CFrame + Vector3.new(0, dy, 0)
93                 end
94                 for _, hrp in ipairs(riders) do
95                     hrp.CFrame = hrp.CFrame + Vector3.new(0, dy, 0)
96                 end
97                 lastY = newY
98             end
99             if a >= 1 then break end
100             RunService.Heartbeat:Wait()
101         end
102
103         current = target
104         model:SetAttribute("CurrentFloor", current)
105         setGate(gates[current], true)
106         moving = false
107     end
108
109     local function addPrompt(part, floor, label, text)
110         local p = Instance.new("ProximityPrompt")
111         p.ObjectText = label
112         p.ActionText = text
113         p.HoldDuration = 0.15
114         p.MaxActivationDistance = 22
115         p.RequiresLineOfSight = false
116         p.Parent = part
117         p.Triggered:Connect(function() moveTo(floor) end)
118     end
119
120     for _, panel in ipairs(model:WaitForChild("CallPanels"):GetChildren()) do
121         addPrompt(panel, panel:GetAttribute("Floor"), "Elevator", "Call")
122     end
123     for _, btn in ipairs(car:WaitForChild("Buttons"):GetChildren()) do
124         local i = btn:GetAttribute("Floor")
125         addPrompt(btn, i, "Elevator", "Go to floor " .. (model:GetAttribute("FloorLabel_" .. i) or i))
126     end

```

```
127 end
128
129 for _, m in ipairs(CollectionService:GetTagged("Elevator")) do setup(m) end
130 CollectionService:GetInstanceAddedSignal("Elevator"):Connect(setup)
131
132 print("Elevators armed: " .. #CollectionService:GetTagged("Elevator") .. ".")
```

Chapter 38

S11 - LabEquipment

ServerScriptService.LabEquipment

Authority group: active. 182838 UTF-8 bytes; 4178 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: ServerScriptService.LabEquipment  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Equipment engine  
7  Future change: Extract inventory transactions before touching hazards or NPC motion. Preserve reservations and  
8  ↪ post-yield validation.  
9  Preserve dependencies: LabRoles assignments, LabQuizClient, tagged geometry, server hit bridge.  
9  Acceptance cases: Concurrent pickup/drop, repair during failure tick, dead player, destroyed target, canceled  
10 ↪ quiz.  
10 ]=====]  
11 -- Spatial repair: room rectangles and stair metadata remain laboratory-local.  
12 local LabSpatial=require(game:GetService('ReplicatedStorage'):WaitForChild('LabSpatial'))  
13  
14 -- LabEquipment : physical controls, equipment readouts, carried samples,  
15 -- cold-store delivery missions, and non-lethal sample recovery tools.  
16 -- All inventory and hit decisions happen on the server.  
17  
18 local ReplicatedStorage = game:GetService("ReplicatedStorage")  
19 local CollectionService = game:GetService("CollectionService")  
20 local Players           = game:GetService("Players")  
21 local TweenService      = game:GetService("TweenService")  
22 local Debris            = game:GetService("Debris")  
23 local PathfindingService = game:GetService("PathfindingService")  
24  
25 local Kinds = require(ReplicatedStorage:WaitForChild("LabKinds"))  
26 local QuizRemote = ReplicatedStorage:FindFirstChild("LabMathQuiz")  
27 if QuizRemote and not QuizRemote:IsA("RemoteEvent") then QuizRemote:Destroy(); QuizRemote = nil end  
28 if not QuizRemote then  
29     QuizRemote = Instance.new("RemoteEvent")  
30     QuizRemote.Name = "LabMathQuiz"  
31     QuizRemote.Parent = ReplicatedStorage  
32 end  
33  
34 -----  
35 -- SHARED GAME STATE  
36 -----  
37 local GameState = {  
38     repairsCompleted = 0,  
39     assignedRepairsCompleted = 0,  
40     transfersCompleted = 0,  
41     assignedTransfersCompleted = 0,  
42 }  
43  
44 local ROLE_STATIONS = {}  
45  
46 local POINTS = {  
47     MOVE_ANY = 12,  
48     MOVE_ASSIGNED_LATE = 18,
```

```

49     MOVE_ASSIGNED_ON_TIME = 30,
50     PROCESS_PLANT = 4,
51 }
52
53 local function updateScoreboard()
54     local board = CollectionService:GetTagged("BatchScoreboard")[1]
55     if not board then return end
56     local gui = board:FindFirstChildWhichIsA("BillboardGui")
57     local label = gui and gui:FindFirstChild("Text")
58     if label then
59         local rivalScore = 0
60         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
61             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
62         end
63         label.Text = string.format(
64             "PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d • assigned: %d\nRival NPC
        ↳ score: %d",
65             GameState.repairsCompleted, GameState.assignedRepairsCompleted,
66             GameState.transfersCompleted, GameState.assignedTransfersCompleted, rivalScore)
67     end
68 end
69
70 -- Per-room equipment status monitors. Each RoomMonitor panel carries a
71 -- Room attribute matching the LabEquipment parts placed in that same room,
72 -- so this just groups by that attribute and tallies running/failed/plants.
73 local function refreshRoomMonitors()
74     -- Room names ("Room 4", etc.) repeat on every level, so the key here
75     -- must include the level too -- otherwise a monitor sums up every
76     -- same-numbered room across the whole building instead of just its own.
77     local totals = {}
78     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
79         local room = equip:GetAttribute("Room")
80         local level = equip:GetAttribute("Level") or ""
81         if room then
82             local key = level .. "|" .. room
83             local t = totals[key]
84             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
85             t.count = t.count + 1
86             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
87             else t.running = t.running + 1 end
88             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
89         end
90     end
91     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
92         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
93         local label = surface and surface:FindFirstChild("Status")
94         if label then
95             local room = panel:GetAttribute("Room") or "ROOM"
96             local level = panel:GetAttribute("Level") or ""
97             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
98             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
99                 room, t.count, t.running, t.failed, t.plants)
100             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235, 170)
101         end
102     end
103 end
104
105 local function addContribution(player, amount)
106     local stats = player:FindFirstChild("leaderstats")
107     local value = stats and stats:FindFirstChild("Contributions")
108     if value then value.Value = value.Value + (amount or 1) end
109 end
110
111 local facilityNoticeAt = setmetatable({}, {__mode = "k"})
112 local function notify(player, text)
113     local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
        ↳ text:match("^CATASTROPHIC FAILURE:")
114     if facilityAlert then
115         local now = os.clock()
116         if now - (facilityNoticeAt[player] or -math.huge) < 6 then return end
117         facilityNoticeAt[player] = now
118     end
119     print("[ " .. player.Name .. " ] " .. text)
120     local gui = player:FindFirstChild("PlayerGui")
121     if not gui then return end
122     local screen = gui:FindFirstChild("LabLog")

```

```

123     if not screen then
124         screen = Instance.new("ScreenGui")
125         screen.Name = "LabLog"
126         screen.ResetOnSpawn = false
127         screen.DisplayOrder = 40
128         screen.Parent = gui
129
130         local frame = Instance.new("TextLabel")
131         frame.Name = "Line"
132         frame.Size = UDim2.new(0.48, 0, 0, 46)
133         frame.AnchorPoint = Vector2.new(0.5, 1)
134         frame.Position = UDim2.new(0.5, 0, 1, -68)
135         frame.BackgroundColor3 = Color3.fromRGB(18, 22, 28)
136         frame.BackgroundTransparency = 0.12
137         frame.TextColor3 = Color3.fromRGB(235, 238, 242)
138         frame.TextWrapped = true
139         frame.TextTruncate = Enum.TextTruncate.AtEnd
140         frame.TextSize = 13
141         frame.Font = Enum.Font.Gotham
142         frame.Parent = screen
143         local c = Instance.new("UICorner")
144         c.CornerRadius = UDim.new(0, 8)
145         c.Parent = frame
146         local sizeLimit = Instance.new("UISizeConstraint")
147         sizeLimit.MinSize = Vector2.new(260, 46)
148         sizeLimit.MaxSize = Vector2.new(420, 46)
149         sizeLimit.Parent = frame
150         local padding = Instance.new("UIPadding")
151         padding.PaddingLeft = UDim.new(0, 10)
152         padding.PaddingRight = UDim.new(0, 10)
153         padding.Parent = frame
154     end
155
156     local line = screen:FindFirstChild("Line")
157     if not line then return end
158     local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
159     screen:SetAttribute("MessageSerial", serial)
160     line.Visible = true
161     line.Text = text
162     task.delay(4, function()
163         if line.Parent and screen:GetAttribute("MessageSerial") == serial then line.Visible = false end
164     end)
165 end
166
167 local function playerRole(player)
168     local attributeRole = player:GetAttribute("LabRole")
169     if attributeRole and attributeRole ~= "" then return attributeRole end
170     if player.Team then return player.Team.Name end
171     local stats = player:FindFirstChild("leaderstats")
172     local role = stats and stats:FindFirstChild("Role")
173     return role and role.Value or "Unassigned"
174 end
175
176 local function awardPoints(player, amount, reason)
177     local stats = player:FindFirstChild("leaderstats")
178     if not stats then
179         stats = Instance.new("Folder")
180         stats.Name = "leaderstats"
181         stats.Parent = player
182     end
183     local points = stats:FindFirstChild("Points")
184     if not points then
185         points = Instance.new("IntValue")
186         points.Name = "Points"
187         points.Parent = stats
188     end
189     points.Value = points.Value + amount
190     addContribution(player, 1)
191     player:SetAttribute("LastPointAmount", amount)
192     player:SetAttribute("LastPointAward", reason)
193     notify(player, string.format("+%d POINTS - %s • TOTAL %d", amount, reason, points.Value))
194     updateScoreboard()
195     return points.Value
196 end
197
198 local function canInteract(player, part, maxDistance)

```

```

199     local character = player.Character
200     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
201     local root = character and character:FindFirstChild("HumanoidRootPart")
202     if humanoid == nil or humanoid.Health <= 0 or root == nil
203         or (root.Position - part.Position).Magnitude > maxDistance + 2 then return false end
204     local originPart = character:FindFirstChild("Head") or root
205     local direction = part.Position - originPart.Position
206     local params = RaycastParams.new()
207     params.FilterType = Enum.RaycastFilterType.Exclude
208     params.FilterDescendantsInstances = {character}
209     params.IgnoreWater = true
210     local hit = workspace:Raycast(originPart.Position, direction, params)
211     return hit == nil or hit.Instance == part or hit.Instance:IsDescendantOf(part)
212 end
213
214 -- Server-owned, one-use addition challenges. No InvokeClient yield can leave
215 -- equipment locked forever; the server expires or cancels each pending action.
216 local pendingQuizzes = {}
217 local quizSerial = 0
218 QuizRemote.OnServerEvent:Connect(function(player, id, answer)
219     local pending = pendingQuizzes[player]
220     if not pending or pending.id ~= id or pending.finished or os.clock() > pending.expires then return end
221     if answer ~= nil and (typeof(answer) ~= "number" or answer ~= answer or answer % 1 ~= 0) then return end
222     pending.answer = answer
223     pending.finished = true
224 end)
225 Players.PlayerRemoving:Connect(function(player) pendingQuizzes[player] = nil end)
226
227 local function runMathQuiz(player, reason, target, validate)
228     if pendingQuizzes[player] or not player.Parent or not validate() then return false end
229     local a, b = math.random(1, 100), math.random(1, 100)
230     local expected = a + b
231     local choices = {expected, expected + math.random(1, 9), math.max(0, expected - math.random(1, 9))}
232     for i = #choices, 2, -1 do local j = math.random(1, i); choices[i], choices[j] = choices[j], choices[i] end
233     quizSerial = quizSerial + 1
234     local pending = {id = quizSerial, expected = expected, expires = os.clock() + 60, finished = false}
235     pendingQuizzes[player] = pending
236     player:SetAttribute("MathQuizActive", true)
237     target:SetAttribute("QuizPlayerId", player.UserId)
238     QuizRemote.FireClient(player, {Id = pending.id, Question = string.format("%d + %d", a, b),
239         Choices = choices, Seconds = 60, Reason = reason})
240     while player.Parent and pendingQuizzes[player] == pending and not pending.finished
241         and os.clock() < pending.expires and target.Parent and validate() do task.wait(0.1) end
242     local correct = player.Parent ~= nil and target.Parent ~= nil and pendingQuizzes[player] == pending
243     and pending.finished and pending.answer == pending.expected and os.clock() <= pending.expires and
244         ↪ validate()
245     if pendingQuizzes[player] == pending then pendingQuizzes[player] = nil end
246     if target.Parent and target:GetAttribute("QuizPlayerId") == player.UserId then
247         ↪ target:SetAttribute("QuizPlayerId", nil) end
248     if player.Parent then
249         player:SetAttribute("MathQuizActive", false)
250         QuizRemote.FireClient(player, {Close = true, Id = pending.id})
251         if not correct then notify(player, "Task not completed. Stay near the unit and try the addition check
252             ↪ again.") end
253     end
254     return correct
255 end
256
257 -----
258 -- VISIBLE INVENTORY ITEMS
259 -- The old script collection used Tool + Handle + welded decorative
260 -- pieces. This keeps that useful pattern, but creates and validates the
261 -- tools entirely on the server instead of trusting a client script.
262 -----
263 local SAMPLE_ITEM = {
264     [Kinds.Items.TRAY] = true,
265     [Kinds.Items.GROUND] = true,
266     [Kinds.Items.DRIED] = true,
267     [Kinds.Items.SPUN] = true,
268     [Kinds.Items.CULTURE] = true,
269 }
270
271 local function addWeldedPart(tool, handle, name, size, relativeCFrame, colour, material, shape)
272     local p = Instance.new("Part")
273     p.Name = name
274     p.Size = size

```

```

272     p.CFrame = handle.CFrame * relativeCFrame
273     p.Color = colour
274     p.Material = material or Enum.Material.SmoothPlastic
275     p.Shape = shape or Enum.PartType.Block
276     p.Anchored = false
277     p.CanCollide = false
278     p.CanTouch = false
279     p.Massless = true
280     p.TopSurface = Enum.SurfaceType.Smooth
281     p.BottomSurface = Enum.SurfaceType.Smooth
282     p.Parent = tool
283     local weld = Instance.new("WeldConstraint")
284     weld.Part0 = handle
285     weld.Part1 = p
286     weld.Parent = p
287     return p
288 end
289
290 local function equipTool(player, tool)
291     task.defer(function()
292         if not tool.Parent then return end
293         local character = player.Character
294         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
295         if humanoid and humanoid.Health > 0 then humanoid:EquipTool(tool) end
296     end)
297 end
298
299 local function makeTool(player, name, options)
300     options = options or {}
301     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
302     if not backpack then return nil end
303
304     local tool = Instance.new("Tool")
305     tool.Name = name
306     tool.ToolTip = options.tooltip or "Visible laboratory item"
307     tool.RequiresHandle = true
308     tool.CanBeDropped = false
309     tool:SetAttribute("LabItemType", options.itemType or name)
310     tool:SetAttribute("LabSample", options.sample == true or SAMPLE_ITEM[name] == true)
311
312     for key, value in pairs(options.attributes or {}) do tool:SetAttribute(key, value) end
313     if options.plant and tool:GetAttribute("PlantLife") == nil then tool:SetAttribute("PlantLife", 100) end
314
315     local handle = Instance.new("Part")
316     handle.Name = "Handle"
317     handle.Size = options.plant and Vector3.new(0.95, 0.72, 0.95) or Vector3.new(1.7, 0.22, 1.35)
318     handle.Color = options.colour or Color3.fromRGB(205, 185, 138)
319     handle.Material = options.plant and Enum.Material.WoodPlanks or Enum.Material.SmoothPlastic
320     handle.Anchored = false
321     handle.CanCollide = false
322     handle.Massless = true
323     handle.TopSurface = Enum.SurfaceType.Smooth
324     handle.BottomSurface = Enum.SurfaceType.Smooth
325     handle.Parent = tool
326
327     if options.plant then
328         -- A real visible potted plant: soil, stem, leaves, and a glowing bud
329         -- are welded to the Tool handle and replicate in the character hand.
330         addWeldedPart(tool, handle, "Soil", Vector3.new(0.82, 0.14, 0.82),
331             CFrame.new(0, 0.4, 0), Color3.fromRGB(78, 52, 33), Enum.Material.Ground)
332         addWeldedPart(tool, handle, "PlantStem", Vector3.new(0.16, 1.0, 0.16),
333             CFrame.new(0, 0.96, 0), Color3.fromRGB(65, 155, 75), Enum.Material.Grass)
334         addWeldedPart(tool, handle, "LeafLeft", Vector3.new(0.78, 0.16, 0.42),
335             CFrame.new(-0.34, 1.1, 0) * CFrame.Angles(0, 0, math.rad(-22)),
336             Color3.fromRGB(70, 180, 82), Enum.Material.Grass)
337         addWeldedPart(tool, handle, "LeafRight", Vector3.new(0.78, 0.16, 0.42),
338             CFrame.new(0.34, 1.35, 0) * CFrame.Angles(0, 0, math.rad(22)),
339             Color3.fromRGB(85, 195, 92), Enum.Material.Grass)
340         addWeldedPart(tool, handle, "GrowthBud", Vector3.new(0.36, 0.36, 0.36),
341             CFrame.new(0, 1.58, 0), Color3.fromRGB(245, 185, 80), Enum.Material.Neon, Enum.PartType.Ball)
342     else
343         -- Tray rim, sample pouch, and labelled vial make ordinary samples
344         -- recognizable from a distance instead of a plain block.
345         addWeldedPart(tool, handle, "TrayRim", Vector3.new(1.82, 0.12, 1.47),
346             CFrame.new(0, 0.14, 0), Color3.fromRGB(55, 68, 78), Enum.Material.Metal)
347         addWeldedPart(tool, handle, "SeedPouch", Vector3.new(0.92, 0.32, 0.76),

```



```

348         CFrame.new(0, 0.35, 0), Color3.fromRGB(220, 193, 116), Enum.Material.Fabric)
349     addWeldedPart(tool, handle, "SampleVial", Vector3.new(0.62, 0.22, 0.22),
350         CFrame.new(0.47, 0.54, 0) * CFrame.Angles(0, 0, math.rad(90)),
351         Color3.fromRGB(115, 215, 245), Enum.Material.Glass, Enum.PartType.Cylinder)
352 end
353
354 local marker = Instance.new("BillboardGui")
355 marker.Name = "CarryLabel"
356 marker.Size = UDim2.fromOffset(150, 34)
357 marker.StudsOffsetWorldSpace = Vector3.new(0, 1.15, 0)
358 marker.MaxDistance = 8
359 marker.AlwaysOnTop = false
360 marker.Parent = handle
361 local markerText = Instance.new("TextLabel")
362 markerText.Size = UDim2.fromScale(1, 1)
363 markerText.BackgroundColor3 = Color3.fromRGB(20, 25, 30)
364 markerText.BackgroundTransparency = 0.2
365 markerText.TextColor3 = Color3.fromRGB(245, 225, 150)
366 markerText.TextScaled = true
367 markerText.Text = options.label or name
368 markerText.Font = Enum.Font.GothamBold
369 markerText.Parent = marker
370 local markerCorner = Instance.new("UICorner")
371 markerCorner.CornerRadius = UDim.new(0, 6)
372 markerCorner.Parent = markerText
373 if options.plant then
374     local lifeGui = Instance.new("BillboardGui")
375     lifeGui.Name = "PlantLifeStatus"
376     lifeGui.Size = UDim2.fromOffset(160, 46)
377     lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
378     lifeGui.MaxDistance = 18
379     lifeGui.AlwaysOnTop = false
380     lifeGui.Parent = handle
381     local lifeFrame = Instance.new("Frame")
382     lifeFrame.Size = UDim2.fromScale(1, 1)
383     lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
384     lifeFrame.BackgroundTransparency = 0.12
385     lifeFrame.BorderSizePixel = 0
386     lifeFrame.Parent = lifeGui
387     local lifeText = Instance.new("TextLabel")
388     lifeText.Name = "LifeText"
389     lifeText.Size = UDim2.new(1, 0, 0.58, 0)
390     lifeText.BackgroundTransparency = 1
391     lifeText.TextScaled = true
392     lifeText.Font = Enum.Font.GothamBold
393     lifeText.Parent = lifeFrame
394     local lifeBack = Instance.new("Frame")
395     lifeBack.Name = "LifeBarBackground"
396     lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
397     lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
398     lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
399     lifeBack.BorderSizePixel = 0
400     lifeBack.Parent = lifeFrame
401     local lifeFill = Instance.new("Frame")
402     lifeFill.Name = "LifeBarFill"
403     lifeFill.Size = UDim2.fromScale(1, 1)
404     lifeFill.BorderSizePixel = 0
405     lifeFill.Parent = lifeBack
406     local function updateLifeBar()
407         local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
408         lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
409         lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
410         local colour = life > 60 and Color3.fromRGB(75, 235, 105)
411             or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
412         lifeText.TextColor3 = colour
413         lifeFill.BackgroundColor3 = colour
414     end
415     tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLifeBar)
416     updateLifeBar()
417 end
418
419 tool.Grip = options.plant
420     and (CFrame.new(0, -0.25, -0.75) * CFrame.Angles(math.rad(-8), 0, 0))
421     or (CFrame.new(0, -0.45, -0.65) * CFrame.Angles(math.rad(-12), 0, 0))
422 tool.Parent = backpack
423 if options.autoEquip ~= false then equipTool(player, tool) end

```

```

424     return tool
425 end
426
427 local function playerContainers(player)
428     local containers = {}
429     if player.Character then table.insert(containers, player.Character) end
430     local backpack = player:FindFirstChildOfClass("Backpack")
431     if backpack then table.insert(containers, backpack) end
432     return containers
433 end
434
435 local function findTool(player, itemType)
436     for _, container in ipairs(playerContainers(player)) do
437         if container then
438             for _, child in ipairs(container:GetChildren()) do
439                 if child:IsA("Tool") and
440                     (child.Name == itemType or child:GetAttribute("LabItemType") == itemType) then
441                     return child
442                 end
443             end
444         end
445     end
446     return nil
447 end
448
449 local function findTransferSample(player)
450     for _, container in ipairs(playerContainers(player)) do
451         if container then
452             for _, child in ipairs(container:GetChildren()) do
453                 if child:IsA("Tool") and child:GetAttribute("TransferSample") then return child end
454             end
455         end
456     end
457     return nil
458 end
459
460 local function findHeldSample(player)
461     local character = player.Character
462     if not character then return nil end
463     for _, child in ipairs(character:GetChildren()) do
464         if child:IsA("Tool") and child:GetAttribute("LabSample") then return child end
465     end
466     return nil
467 end
468
469 local GROWTH_STAGE_NAME = {
470     [0] = "Seedling",
471     [1] = "Germinated",
472     [2] = "Vegetative",
473     [3] = "Budding",
474     [4] = "Flowering",
475     [5] = "Mature",
476 }
477
478 local function findPlantSample(player)
479     for _, container in ipairs(playerContainers(player)) do
480         for _, child in ipairs(container:GetChildren()) do
481             if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end
482         end
483     end
484     return nil
485 end
486
487 local function treatmentText(tool)
488     local treatments = tool:GetAttribute("Treatments") or ""
489     if treatments == "" then return "untreated" end
490     return treatments:gsub("|", ", ")
491 end
492
493 local function hasTreatment(tool, key)
494     local treatments = "|" .. (tool:GetAttribute("Treatments") or "") .. "|"
495     return treatments:find("|" .. key .. "|", 1, true) ~= nil
496 end
497
498 local function addTreatment(tool, key)
499     if hasTreatment(tool, key) then return false end

```

```

500     local treatments = tool:GetAttribute("Treatments") or ""
501     tool:SetAttribute("Treatments", treatments == "" and key or (treatments .. "|" .. key))
502     return true
503 end
504
505 local function refreshPlantVisual(tool)
506     local stage = math.clamp(tool:GetAttribute("GrowthStage") or 0, 0, 5)
507     local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
508     local stem = tool:FindFirstChild("PlantStem")
509     local left = tool:FindFirstChild("LeafLeft")
510     local right = tool:FindFirstChild("LeafRight")
511     local bud = tool:FindFirstChild("GrowthBud")
512     if stem then stem.Size = Vector3.new(0.16 + stage * 0.025, 1 + stage * 0.13, 0.16 + stage * 0.025) end
513     local lifeScale = 0.35 + life / 100 * 0.65
514     if left then left.Color = Color3.fromRGB((70 + stage * 5) * lifeScale, (165 + stage * 10) * lifeScale, 55 *
↪ lifeScale) end
515     if right then right.Color = Color3.fromRGB((75 + stage * 4) * lifeScale, (175 + stage * 10) * lifeScale, 60
↪ * lifeScale) end
516     if bud then
517         bud.Transparency = stage < 3 and 0.65 or 0
518         bud.Color = stage >= 5 and Color3.fromRGB(255, 105, 175) or Color3.fromRGB(245, 185, 80)
519     end
520     local handle = tool:FindFirstChild("Handle")
521     local marker = handle and handle:FindFirstChild("CarryLabel")
522     local label = marker and marker:FindFirstChildOfClass("TextLabel")
523     if label then
524         local destination = tool:GetAttribute("DestinationGrowthCode") or ""
525         label.Text = (tool:GetAttribute("PlantId") or "PLANT") .. " • " .. (GROWTH_STAGE_NAME[stage] or
↪ "Growing")
526         .. (destination ~= "" and (" • BONUS DROP " .. destination) or " • DROP IN GREEN R")
527     end
528 end
529
530 local function setCarryState(player, sample)
531     -- A cold-store transfer remains the main HUD objective even if the
532     -- player briefly produces or pockets another laboratory item.
533     local transfer = findTransferSample(player)
534     if transfer and sample and not sample:GetAttribute("TransferSample") then sample = transfer end
535     if sample then
536         local id = sample:GetAttribute("SampleId") or sample.Name
537         local destination = sample:GetAttribute("DestinationLabel") or
↪ sample:GetAttribute("DestinationGrowthCode")
538         player:SetAttribute("CarriedSample", id)
539         if destination and destination ~= "" then
540             player:SetAttribute("LabObjective", "Carry " .. id .. " to bonus chamber " .. destination .. ", or
↪ press E at any GREEN RUNNING cabinet with a free slot.")
541         else
542             player:SetAttribute("LabObjective", "Carry " .. sample.Name .. " to the next required machine.")
543         end
544     else
545         player:SetAttribute("CarriedSample", "")
546         player:SetAttribute("LabObjective", "")
547     end
548 end
549
550 local function destroyCarriedItem(player, tool)
551     if player:GetAttribute("CarriedSample") == (tool:GetAttribute("SampleId") or tool.Name) then
552         setCarryState(player, nil)
553     end
554     tool:Destroy()
555 end
556
557 -----
558 -- NON-LETHAL HIT TOOLS
559 -----
560 local IMPACT_STYLE = {
561     ["Foam Baton"] = {colour = Color3.fromRGB(245, 135, 55), accent = Color3.fromRGB(45, 48, 54), mode =
↪ "melee", damage = 18},
562     ["Sample Grabber"] = {colour = Color3.fromRGB(70, 165, 220), accent = Color3.fromRGB(205, 225, 235), mode =
↪ "melee", damage = 12},
563     ["Inspection Paddle"] = {colour = Color3.fromRGB(245, 205, 70), accent = Color3.fromRGB(95, 75, 30), mode =
↪ "melee", damage = 22},
564     ["Stun Blaster"] = {colour = Color3.fromRGB(90, 225, 245), accent = Color3.fromRGB(35, 55, 70), mode =
↪ "gun", damage = 28, range = 95},
565     ["Foam Dart Rifle"] = {colour = Color3.fromRGB(225, 80, 95), accent = Color3.fromRGB(45, 45, 52), mode =
↪ "gun", damage = 38, range = 125},

```

```

566     ["Basketball"] = {colour = Color3.fromRGB(225, 105, 35), accent = Color3.fromRGB(45, 28, 18), mode =
    ↳ "ball", damage = 24},
567 -- PC-only weapons, built from the official Roblox Weapons Kit meshes.
568 --
569 -- meshScale is (the kit's intended size / the raw mesh size), measured off
570 -- the kit's own MeshParts. This matters: the raw pistol mesh is 9.68 studs
571 -- long, so the scale of 1.8 used at first drew a pistol about 17 studs
572 -- long -- roughly three times the height of the character holding it.
573 --
574 -- A worker bot has 100 life, so damage is set for 2-3 body shots; a head
575 -- shot kills outright regardless. selfCostPercent still charges the
576 -- shooter 1% of their own max health per shot. pcOnly means the generic
577 -- Tool.Activated path ignores them -- they fire only through the
578 -- LabSidearmFire remote, which only the desktop client script sends.
579 ["Lab Sidearm"] = {
580     colour = Color3.fromRGB(126, 133, 145), accent = Color3.fromRGB(38, 42, 50),
581     mode = "gun", damage = 34, range = 240, selfCostPercent = 1, pcOnly = true,
582     cooldown = 0.32, pellets = 1, spread = 0,
583     meshId = "rbxassetid://2492922600", meshTexture = "rbxassetid://2492931263",
584     meshScale = 0.1407, handleSize = Vector3.new(0.20, 0.92, 1.36),
585     grip = CFrame.new(0, -0.20, -0.30) * CFrame.Angles(0, math.rad(180), 0),
586 },
587 ["Lab Shotgun"] = {
588     colour = Color3.fromRGB(122, 98, 72), accent = Color3.fromRGB(40, 34, 30),
589     mode = "gun", damage = 20, range = 110, selfCostPercent = 1, pcOnly = true,
590     cooldown = 0.85, pellets = 6, spread = 0.075,
591     meshId = "rbxassetid://2492972199", meshTexture = "rbxassetid://2492974190",
592     meshScale = 0.6666, handleSize = Vector3.new(0.31, 1.00, 4.87),
593     grip = CFrame.new(0, -0.35, -0.95) * CFrame.Angles(0, math.rad(180), 0),
594 },
595 ["Lab Rifle"] = {
596     colour = Color3.fromRGB(96, 104, 96), accent = Color3.fromRGB(34, 38, 34),
597     mode = "gun", damage = 55, range = 620, selfCostPercent = 1, pcOnly = true,
598     cooldown = 0.95, pellets = 1, spread = 0,
599     meshId = "rbxassetid://2759059561", meshTexture = "rbxassetid://2759063893",
600     meshScale = 0.6383, handleSize = Vector3.new(0.29, 1.17, 6.18),
601     grip = CFrame.new(0, -0.35, -1.20) * CFrame.Angles(0, math.rad(180), 0),
602 },
603 }
604
605 local function refreshBotHealth(bot)
606     local health = math.clamp(bot:GetAttribute("Health") or 100, 0, bot:GetAttribute("MaxHealth") or 100)
607     local maximum = bot:GetAttribute("MaxHealth") or 100
608     local text = bot:FindFirstChild("HealthText", true)
609     local fill = bot:FindFirstChild("HealthBarFill", true)
610     if text and text:IsA("TextLabel") then text.Text = string.format("LIFE %d / %d", health, maximum) end
611     if fill and fill:IsA("Frame") then
612         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
613         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
614         or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
615     end
616 end
617
618 -- How long a body stays before it is cleaned up. Bodies are the biggest
619 -- avoidable part cost in a long round, so this is deliberately short.
620 -- Declared HERE, above damageWorkerBot, because that is what reads it: a
621 -- local declared further down the file resolves to a nil global inside a
622 -- function defined above it.
623 local BODY_LIFETIME_SECONDS = 60
624
625 local function damageWorkerBot(bot, amount, attacker)
626     -- Only an already-dead worker is immune. This used to refuse damage to
627     -- anything KnockedOut, which meant a worker downed by a hazard could not
628     -- be finished off by gunfire or caught in a blast -- so a floor could
629     -- never actually be cleared.
630     if not bot or not CollectionService.HasTag(bot, "LabWorkerBot") or bot:GetAttribute("Dead") then return
    ↳ false end
631     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
632     bot:SetAttribute("Health", health)
633     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
634     refreshBotHealth(bot)
635     local flash = Instance.new("Highlight")
636     flash.FillColor = Color3.fromRGB(255, 90, 70)
637     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
638     flash.FillTransparency = 0.25
639     -- Highlights render through geometry by default, which is why a hit

```

```

640 -- worker (and any corpse that inherited the flash) glowed through walls.
641 flash.DepthMode = Enum.HighlightDepthMode.Occluded
642 flash.Parent = bot
643 Debris:AddItem(flash, 0.22)
644 if health <= 0 then
645     -- Death is permanent now: the hunt is the objective, so a worker that
646     -- goes down stays down and its body is left where it fell. Bodies are
647     -- cleared when the shift rolls over (see the RoundNumber watcher).
648     bot:SetAttribute("Dead", true)
649     bot:SetAttribute("KnockedOut", true)
650     bot:SetAttribute("TaskState", "DEAD")
651
652     -- The body left behind is a CLONE, not the worker itself. That keeps
653     -- the original model pristine, so the shift roll-over can simply move
654     -- it home and stand it back up rather than trying to undo a pile of
655     -- destructive colour and rotation edits.
656     local pivot = bot:GetPivot()
657     local corpse = bot:Clone()
658     corpse.Name = bot.Name .. "_Body"
659     for _, d in ipairs(corpse:GetDescendants()) do
660         -- A body reports no status, and must not inherit the damage-flash
661         -- Highlight -- that is what made corpses visible through walls,
662         -- because the clone's copy was never cleaned up.
663         if d:IsA("BillboardGui") or d:IsA("Highlight") or d:IsA("Sparkles")
664            or d:IsA("Fire") or d:IsA("Smoke") or d:IsA("PointLight")
665            or d.Name:match("^Chair") or d.Name:match("^Desk") or d.Name == "Monitor" then
666             d:Destroy()
667         end
668     end
669     for _, d in ipairs(corpse:GetDescendants()) do
670         if d:IsA("BasePart") then
671             d.Color = Color3.new(d.Color.R * 0.55 + 0.10, d.Color.G * 0.55 + 0.10, d.Color.B * 0.55 + 0.10)
672             d.Material = Enum.Material.SmoothPlastic
673             d.Anchored = true
674             d.CanCollide = false
675             d.CanQuery = false -- a body must not soak up later shots
676             d.CastShadow = false -- 180 shadow-casting corpses is not free
677         end
678     end
679     local corpseFolder = workspace:FindFirstChild("LabBotBodies")
680     if not corpseFolder then
681         corpseFolder = Instance.new("Folder")
682         corpseFolder.Name = "LabBotBodies"
683         corpseFolder.Parent = workspace
684     end
685     -- The clone inherits every tag, including LabWorkerBot. Left alone it
686     -- would be counted as a live worker by every GetTagged sweep and the
687     -- kill total would never be able to reach the target.
688     CollectionService:RemoveTag(corpse, "LabWorkerBot")
689     corpse.Parent = corpseFolder
690     -- Lay it down where it fell, with a random spin so a cleared room does
691     -- not read as a row of identical dropped props.
692     corpse:PivotTo(CFrame.new(pivot.Position - Vector3.new(0, 2.6, 0))
693        * CFrame.Angles(0, math.rad(math.random(0, 359)), 0)
694        * CFrame.Angles(math.rad(-90), 0, 0))
695     CollectionService:AddTag(corpse, "LabBotCorpse")
696     -- Bodies are the single biggest avoidable part cost in a long round:
697     -- 180 of them is ~1,600 extra parts. They stay long enough to read a
698     -- room as cleared, then go.
699     Debris:AddItem(corpse, BODY_LIFETIME_SECONDS)
700
701     -- Park the live worker out of the world until the next shift.
702     if not bot:GetAttribute("HomeCFrame") then bot:SetAttribute("HomeCFrame", pivot) end
703     bot:PivotTo(CFrame.new(0, -1000, 0))
704
705     local complex = workspace:FindFirstChild("Laboratory_Complex")
706     if complex then
707         complex:SetAttribute("BotsKilled", (complex:GetAttribute("BotsKilled") or 0) + 1)
708     end
709 end
710 return true
711 end
712
713 -- Walk up from whatever the ray actually hit to the tagged equipment part.
714 local function equipmentFromHit(hit)
715     local node = hit

```

```

716     while node and node ~= workspace do
717         if node:IsA("BasePart")
718             and (CollectionService:HasTag(node, "LabEquipment") or CollectionService:HasTag(node,
719                 ↳ "PlantGrowthChamber")) then
720             return node
721         end
722         node = node.Parent
723     end
724     return nil
725 end
726 -- Shoot a machine and it goes up, taking out whatever is standing near it --
727 -- including the player who pulled the trigger, if they did it from close range.
728 local function explodeEquipment(item, attacker)
729     if item:GetAttribute("Exploded") then return false end
730     item:SetAttribute("Exploded", true)
731     item:SetAttribute("FunctionalState", "DESTROYED")
732     item:SetAttribute("RepairInstruction", "Destroyed beyond repair")
733
734     local position = item.Position
735     local blast = Instance.new("Explosion")
736     blast.Position = position
737     blast.BlastRadius = 17
738     blast.BlastPressure = 0           -- the lab is anchored; pressure would do nothing
739     blast.DestroyJointRadiusPercent = 0
740     blast.ExplosionType = Enum.ExplosionType.NoCraters
741     blast.Parent = workspace
742
743     local fire = Instance.new("Fire")
744     fire.Heat = 14
745     fire.Size = 9
746     fire.Parent = item
747     Debris:AddItem(fire, 14)
748     local smoke = Instance.new("Smoke")
749     smoke.Size = 5
750     smoke.Opacity = 0.4
751     smoke.RiseVelocity = 9
752     smoke.Color = Color3.fromRGB(60, 58, 56)
753     smoke.Parent = item
754     Debris:AddItem(smoke, 18)
755
756     item.Color = Color3.fromRGB(48, 40, 36)
757     item.Material = Enum.Material.CorrodedMetal
758
759     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
760         if not bot:GetAttribute("Dead") and bot.PrimaryPart then
761             if (bot.PrimaryPart.Position - position).Magnitude < 17 then
762                 damageWorkerBot(bot, 1000, attacker)
763             end
764         end
765     end
766     for _, other in ipairs(Players:GetPlayers()) do
767         local c = other.Character
768         local h = c and c:FindFirstChildOfClass("Humanoid")
769         local hrp = c and c:FindFirstChild("HumanoidRootPart")
770         if h and hrp and h.Health > 0 then
771             local d = (hrp.Position - position).Magnitude
772             if d < 17 then h:TakeDamage(math.max(18, 75 * (1 - d / 17))) end
773         end
774     end
775     return true
776 end
777
778 -- A gas line going up is a different order of event from a machine failing:
779 -- it takes out a chunk of the room -- a couple of neighbouring machines and
780 -- part of the wall structure itself.
781 local function detonateGasPipe(pipe, attacker, atPosition)
782     if pipe:GetAttribute("Blown") then return false end
783     pipe:SetAttribute("Blown", true)
784     -- Blow at the point that was actually hit. A corridor run is one long part
785     -- whose Position is its midpoint, so using that detonated the pipe up to a
786     -- hundred studs away from where the player shot it.
787     local position = atPosition or pipe.Position
788
789     local blast = Instance.new("Explosion")
790     blast.Position = position

```

```

791     blast.BlastRadius = 30
792     blast.BlastPressure = 0
793     blast.DestroyJointRadiusPercent = 0
794     blast.ExplosionType = Enum.ExplosionType.NoCraters
795     blast.Parent = workspace
796
797     -- a running wall of fire and smoke along the rupture
798     for i = -2, 2 do
799         local flare = Instance.new("Part")
800         flare.Name = "GasFire"
801         flare.Size = Vector3.new(1, 1, 1)
802         flare.CFrame = CFrame.new(position + Vector3.new(0, 0, i * 7))
803         flare.Anchored = true
804         flare.CanCollide = false
805         flare.CanQuery = false
806         flare.CanTouch = false
807         flare.Transparency = 1
808         flare.Parent = workspace
809         local fire = Instance.new("Fire")
810         fire.Heat = 25
811         fire.Size = 24
812         fire.Parent = flare
813         local smoke = Instance.new("Smoke")
814         smoke.Size = 15
815         smoke.Opacity = 0.5
816         smoke.RiseVelocity = 14
817         smoke.Color = Color3.fromRGB(40, 38, 36)
818         smoke.Parent = flare
819         task.delay(16, function()
820             if flare.Parent then fire.Enabled = false; smoke.Enabled = false end
821         end)
822         Debris.AddItem(flare, 22)
823     end
824
825     pipe.Color = Color3.fromRGB(62, 52, 30)
826     pipe.Material = Enum.Material.CorrodedMetal
827
828     -- take two or three neighbouring machines with it, staggered so it reads
829     -- as a chain rather than one simultaneous bang
830     local nearby = {}
831     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 24)) do
832         if (CollectionService:HasTag(part, "LabEquipment") or CollectionService:HasTag(part,
833             ↪ "PlantGrowthChamber"))
834             and not part:GetAttribute("Exploded") then
835             table.insert(nearby, part)
836         end
837     end
838     for i = 1, math.min(3, #nearby) do
839         local item = nearby[i]
840         task.delay(0.18 * i, function()
841             if item.Parent then explodeEquipment(item, attacker) end
842         end)
843     end
844
845     -- and blow an actual hole in the structure
846     local blown = 0
847     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 15)) do
848         if part:IsA("BasePart") and part.CanCollide and not part:GetAttribute("GasPipe") then
849             local n = part.Name
850             if n:match("^Corridor_") or n:match("^Div_") or n:match("^Ext_") or n:match("^B_") then
851                 if blown < 5 then
852                     blown = blown + 1
853                     part:Destroy()
854                 else
855                     part.Color = Color3.fromRGB(60, 54, 50)
856                     part.Material = Enum.Material.CorrodedMetal
857                 end
858             end
859         end
860     end
861
862     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
863         if not bot:GetAttribute("Dead") and bot.PrimaryPart
864             and (bot.PrimaryPart.Position - position).Magnitude < 30 then
865             damageWorkerBot(bot, 1000, attacker)
866         end
867     end

```

```

866     end
867     for _, other in ipairs(Players:GetPlayers()) do
868         local c = other.Character
869         local h = c and c:FindFirstChildOfClass("Humanoid")
870         local hrp = c and c:FindFirstChild("HumanoidRootPart")
871         if h and hrp and h.Health > 0 then
872             local d = (hrp.Position - position).Magnitude
873             if d < 30 then h:TakeDamage(math.max(30, 140 * (1 - d / 30))) end
874         end
875     end
876     return true
877 end
878
879 -- Scorch mark, smoke puff and sparks wherever a round lands.
880 local function spawnImpact(position, normal)
881     local hole = Instance.new("Part")
882     hole.Name = "BulletHole"
883     hole.Size = Vector3.new(0.3, 0.3, 0.06)
884     hole.CFrame = CFrame.lookAt(position + normal * 0.03, position + normal * 2)
885     hole.Color = Color3.fromRGB(24, 22, 21)
886     hole.Material = Enum.Material.Slate
887     hole.Anchored = true
888     hole.CanCollide = false
889     hole.CanQuery = false
890     hole.CanTouch = false
891     hole.CastShadow = false
892     hole.Parent = workspace
893     Debris:AddItem(hole, 45)
894
895     local puff = Instance.new("Part")
896     puff.Name = "ImpactPuff"
897     puff.Size = Vector3.new(0.1, 0.1, 0.1)
898     puff.CFrame = CFrame.new(position + normal * 0.25)
899     puff.Anchored = true
900     puff.CanCollide = false
901     puff.CanQuery = false
902     puff.CanTouch = false
903     puff.Transparency = 1
904     puff.Parent = workspace
905     local smoke = Instance.new("Smoke")
906     smoke.Size = 0.7
907     smoke.RiseVelocity = 5
908     smoke.Opacity = 0.3
909     smoke.Color = Color3.fromRGB(125, 125, 125)
910     smoke.Parent = puff
911     local sparks = Instance.new("Sparkles")
912     sparks.SparkleColor = Color3.fromRGB(255, 208, 128)
913     sparks.Parent = puff
914     task.delay(0.3, function()
915         if puff.Parent then smoke.Enabled = false; sparks.Enabled = false end
916     end)
917     Debris:AddItem(puff, 3)
918 end
919
920 -- Posters keep their holes for good rather than fading like scorch marks.
921 local function punchPoster(poster, position, normal)
922     local holes = poster:GetAttribute("Holes") or 0
923     if holes >= 30 then return end
924     poster:SetAttribute("Holes", holes + 1)
925     local tear = Instance.new("Part")
926     tear.Name = "PosterTear"
927     tear.Size = Vector3.new(0.36, 0.36, 0.03)
928     tear.CFrame = CFrame.lookAt(position + normal * 0.012, position + normal * 2)
929     tear.Color = Color3.fromRGB(198, 190, 174)
930     tear.Material = Enum.Material.SmoothPlastic
931     tear.Anchored = true
932     tear.CanCollide = false
933     tear.CanQuery = false
934     tear.CanTouch = false
935     tear.CastShadow = false
936     tear.Parent = poster
937     local hole = tear:Clone()
938     hole.Name = "PosterHole"
939     hole.Size = Vector3.new(0.22, 0.22, 0.05)
940     hole.Color = Color3.fromRGB(16, 15, 14)
941     hole.CFrame = CFrame.lookAt(position + normal * 0.025, position + normal * 2)

```



```

942     hole.Parent = poster
943 end
944
945 local function workerBotFromHit(hit)
946     local model = hit and hit:FindFirstAncestorOfClass("Model")
947     return model and CollectionService:HasTag(model, "LabWorkerBot") and model or nil
948 end
949
950 local function stealHeldSample(attacker, victim)
951     local sample = findHeldSample(victim)
952     if not sample then
953         notify(attacker, victim.DisplayName .. " is not holding a sample.")
954         return false
955     end
956     if sample:GetAttribute("TransferSample") and findTransferSample(attacker) then
957         notify(attacker, "Your hands are already assigned to another cold-store sample.")
958         return false
959     end
960
961     local backpack = attacker:FindFirstChildOfClass("Backpack")
962     if not backpack then return false end
963     local id = sample:GetAttribute("SampleId") or sample.Name
964     sample.Parent = backpack
965     setCarryState(victim, nil)
966     setCarryState(attacker, sample)
967     addContribution(attacker)
968     notify(attacker, "Recovered " .. id .. " from " .. victim.DisplayName .. ". It is now in your hand.")
969     notify(victim, attacker.DisplayName .. " took " .. id .. " from your hand. Collect or recover another
    ↪ sample.")
970     equipTool(attacker, sample)
971     return true
972 end
973
974 local function armImpactTool(tool)
975     if tool:GetAttribute("HitLogicBound") then return end
976     tool:SetAttribute("HitLogicBound", true)
977     local handle = tool:FindFirstChild("Handle")
978     if not handle then return end
979     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or tool.Name]
980     if not style then return end
981
982     local activeUntil = 0
983     local nextSwing = 0
984     local hitThisSwing = {}
985
986     tool.Activated:Connect(function()
987         -- Every gun is driven by BlasterController on the owning client.
988         -- Keep Tool.Activated for melee and throwable tools only, avoiding
989         -- duplicate shots from Roblox's default Tool activation path.
990         if style.mode == "gun" then return end
991         local owner = Players:GetPlayerFromCharacter(tool.Parent)
992         if not owner or os.clock() < nextSwing then return end
993         nextSwing = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
994         hitThisSwing = {}
995         local original = handle.Color
996         handle.Color = Color3.fromRGB(255, 245, 190)
997         task.delay(0.18, function()
998             if handle.Parent then handle.Color = original end
999         end)
1000
1001         local rootPart = owner.Character and owner.Character:FindFirstChild("HumanoidRootPart")
1002         if not rootPart then return end
1003         if style.mode == "gun" then
1004             local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
1005             local direction = rootPart.CFrame.LookVector * (style.range or 100)
1006             local params = RaycastParams.new()
1007             params.FilterType = Enum.RaycastFilterType.Exclude
1008             params.FilterDescendantsInstances = {owner.Character}
1009             local result = workspace:Raycast(origin, direction, params)
1010             local hitPosition = result and result.Position or (origin + direction)
1011             local beam = Instance.new("Part")
1012             beam.Name = "FoamTracer"
1013             beam.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
1014             beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
1015             beam.Color = style.colour
1016             beam.Material = Enum.Material.Neon

```

```

1017     beam.Anchored = true
1018     beam.CanCollide = false
1019     beam.CanTouch = false
1020     beam.Parent = workspace
1021     Debris:AddItem(beam, 0.12)
1022     local bot = result and workerBotFromHit(result.Instance)
1023     if bot then damageWorkerBot(bot, style.damage or 25, owner) end
1024     return
1025 elseif style.mode == "ball" then
1026     local ball = Instance.new("Part")
1027     ball.Name = "ThrownBasketball"
1028     ball.Shape = Enum.PartType.Ball
1029     ball.Size = Vector3.new(1.35, 1.35, 1.35)
1030     ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) + rootPart.CFrame.LookVector *
        ↪ 3)
1031     ball.Color = style.colour
1032     ball.Material = Enum.Material.Rubber
1033     ball.CanCollide = true
1034     ball.Parent = workspace
1035     ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
1036     local ballHit = false
1037     ball.Touched:Connect(function(hit)
1038         if ballHit then return end
1039         local bot = workerBotFromHit(hit)
1040         if bot then ballHit = true; damageWorkerBot(bot, style.damage or 20, owner) end
1041     end)
1042     Debris:AddItem(ball, 7)
1043     return
1044 end
1045 activeUntil = os.clock() + 0.45
1046 end)
1047
1048 handle.Touched:Connect(function(hit)
1049     if os.clock() > activeUntil then return end
1050     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
1051     if not attacker then return end
1052     local targetCharacter = hit:FindFirstAncestorOfClass("Model")
1053     local workerBot = workerBotFromHit(hit)
1054     if workerBot then
1055         if hitThisSwing[workerBot] then return end
1056         hitThisSwing[workerBot] = true
1057         damageWorkerBot(workerBot, style.damage or 15, attacker)
1058         return
1059     end
1060     local victim = targetCharacter and Players:GetPlayerFromCharacter(targetCharacter)
1061     if not victim or victim == attacker or hitThisSwing[victim] then return end
1062
1063     local attackerRoot = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
1064     local victimRoot = victim.Character and victim.Character:FindFirstChild("HumanoidRootPart")
1065     local victimHumanoid = victim.Character and victim.Character:FindFirstChildOfClass("Humanoid")
1066     if not attackerRoot or not victimRoot or not victimHumanoid or victimHumanoid.Health <= 0 then return
        ↪ end
1067     if (attackerRoot.Position - victimRoot.Position).Magnitude > 9 then return end
1068     hitThisSwing[victim] = true
1069
1070     local delta = victimRoot.Position - attackerRoot.Position
1071     if delta.Magnitude > 0.1 then
1072         local direction = delta.Unit
1073         victimRoot.AssemblyLinearVelocity = victimRoot.AssemblyLinearVelocity
1074             + Vector3.new(direction.X * 9, 3, direction.Z * 9)
1075     end
1076
1077     local flash = Instance.new("Highlight")
1078     flash.Name = "SampleRecoveryHit"
1079     flash.FillColor = Color3.fromRGB(255, 210, 80)
1080     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
1081     flash.FillTransparency = 0.35
1082     flash.Parent = victim.Character
1083     Debris:AddItem(flash, 0.25)
1084     stealHeldSample(attacker, victim)
1085 end)
1086 end
1087
1088 local function findImpactTool(player, kind)
1089     for _, container in ipairs(playerContainers(player)) do
1090         if container then

```

```

1091         for _, child in ipairs(container:GetChildren()) do
1092             if child:IsA("Tool") and child:GetAttribute("ImpactToolKind") == kind then return child end
1093         end
1094     end
1095 end
1096 return nil
1097 end
1098
1099 local function makeImpactTool(player, kind)
1100     local existing = findImpactTool(player, kind)
1101     if existing then
1102         equipTool(player, existing)
1103         notify(player, kind .. " equipped. Click or tap to use it against worker NPCs or recover a held sample
1104             ↪ from another player.")
1105         return existing
1106     end
1107
1108     local style = IMPACT_STYLE[kind]
1109     if not style then return nil end
1110     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
1111     if not backpack then return nil end
1112
1113     local tool = Instance.new("Tool")
1114     tool.Name = kind
1115     tool.ToolTip = style.mode == "gun" and "Fire at the screen crosshair; hold right-click or tap AIM for ADS"
1116         or (style.mode == "ball" and "Throw a basketball at worker NPCs" or "Hit players to recover samples;
1117             ↪ hit NPCs to lower life")
1118     tool.RequiresHandle = true
1119     tool.CanBeDropped = false
1120     tool:SetAttribute("ImpactToolKind", kind)
1121     tool:SetAttribute("LabImpactTool", true)
1122     if style.mode == "gun" then
1123         tool:SetAttribute("BlasterWeapon", true)
1124         tool:SetAttribute("BlasterDamage", style.damage or 25)
1125         tool:SetAttribute("BlasterRange", style.range or 200)
1126         tool:SetAttribute("BlasterCooldown", style.cooldown or 0.55)
1127         tool:SetAttribute("BlasterPellets", style.pellets or 1)
1128         tool:SetAttribute("BlasterSpread", style.spread or 0)
1129         tool:SetAttribute("BlasterSelfCostPercent", style.selfCostPercent or 0)
1130         tool:SetAttribute("BlasterInstantHeadshot", style.pcOnly == true)
1131         tool:SetAttribute("BlasterAutomatic", style.automatic == true)
1132         tool:SetAttribute("BlasterADSFov", (style.range or 0) >= 500 and 44 or ((style.pellets or 1) > 1 and 58
1133             ↪ or 54))
1134         tool:SetAttribute("BlasterADSTouchSensitivity", 0.42)
1135         tool:SetAttribute("BlasterTracerColor", style.colour)
1136         -- Legacy marker retained for old saved tools; the legacy remote is inert.
1137         if style.pcOnly then tool:SetAttribute("LabSidearm", true) end
1138     end
1139
1140     local handle = Instance.new("Part")
1141     handle.Name = "Handle"
1142     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
1143         or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35) or Vector3.new(0.42, 3.4, 0.42))
1144     handle.Color = style.colour
1145     handle.Material = Enum.Material.SmoothPlastic
1146     handle.Anchored = false
1147     handle.CanCollide = false
1148     handle.Massless = true
1149     handle.Parent = tool
1150     if style.mode == "ball" then handle.Shape = Enum.PartType.Ball; handle.Material = Enum.Material.Rubber end
1151
1152     if style.meshId then
1153         -- Imported gun shape. A SpecialMesh is used rather than a MeshPart
1154         -- because MeshPart.MeshId cannot be assigned at runtime -- SpecialMesh
1155         -- can, so the tool builds correctly on a live server as well as in
1156         -- Studio, with no dependency on anything sitting in ServerStorage.
1157         handle.Size = style.handleSize or Vector3.new(0.20, 0.92, 1.36)
1158         local mesh = Instance.new("SpecialMesh")
1159         mesh.MeshType = Enum.MeshType.FileMesh
1160         mesh.MeshId = style.meshId
1161         if style.meshTexture then mesh.TextureId = style.meshTexture end
1162         local scale = style.meshScale or 1.6
1163         mesh.Scale = Vector3.new(scale, scale, scale)
1164         mesh.Parent = handle
1165     elseif style.mode == "gun" then
1166         addWeldedPart(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7),

```

```

1164         CFrame.new(0, 0.1, -1.65), style.colour, Enum.Material.Neon)
1165         addWeldedPart(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55),
1166         CFrame.new(0, -0.75, 0.42) * CFrame.Angles(math.rad(-18), 0, 0), style.accent,
1167         ↳ Enum.Material.Rubber)
1168     elseif kind == "Inspection Paddle" then
1169         addWeldedPart(tool, handle, "Paddle", Vector3.new(1.45, 1.65, 0.28),
1170         CFrame.new(0, 2.15, 0), style.colour, Enum.Material.SmoothPlastic)
1171     elseif kind == "Sample Grabber" then
1172         addWeldedPart(tool, handle, "LeftClaw", Vector3.new(0.25, 1.1, 0.3),
1173         CFrame.new(-0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(-28)), style.accent, Enum.Material.Metal)
1174         addWeldedPart(tool, handle, "RightClaw", Vector3.new(0.25, 1.1, 0.3),
1175         CFrame.new(0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(28)), style.accent, Enum.Material.Metal)
1176     elseif style.mode == "melee" then
1177         addWeldedPart(tool, handle, "FoamTip", Vector3.new(0.75, 1.5, 0.75),
1178         CFrame.new(0, 2, 0), style.colour, Enum.Material.Fabric)
1179     end
1180     if style.mode == "melee" then
1181         addWeldedPart(tool, handle, "Grip", Vector3.new(0.58, 1.15, 0.58),
1182         CFrame.new(0, -1.1, 0), style.accent, Enum.Material.Rubber)
1183     end
1184     tool.Grip = style.grip or (style.mode == "gun" and CFrame.new(0, -0.2, 0.6))
1185     or (style.mode == "ball" and CFrame.new(0, -0.25, -0.4) or CFrame.new(0, -1.2, 0) *
1186     ↳ CFrame.Angles(math.rad(-8), 0, 0))
1187     tool.Parent = backpack
1188     armImpactTool(tool)
1189     equipTool(player, tool)
1190     notify(player, kind .. " equipped. Click or tap to use it against worker NPCs; their LIFE bar shows damage
1191     ↳ and recovery.")
1192     return tool
1193 end
1194
1195 local function bindDispenser(part)
1196     if part:GetAttribute("DispenserBound") then return end
1197     local kind = part:GetAttribute("ToolKind")
1198     if not kind or not IMPACT_STYLE[kind] then return end
1199     part:SetAttribute("DispenserBound", true)
1200
1201     local prompt = Instance.new("ProximityPrompt")
1202     prompt.ObjectText = kind
1203     prompt.ActionText = "Equip combat tool"
1204     prompt.KeyboardKeyCode = Enum.KeyCode.E
1205     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
1206     prompt.HoldDuration = 0.25
1207     prompt.MaxActivationDistance = 10
1208     prompt.RequiresLineOfSight = false
1209     prompt.Parent = part
1210     local lastUse = {}
1211     prompt.Triggered:Connect(function(player)
1212         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
1213         if os.clock() < (lastUse[player] or 0) then return end
1214         lastUse[player] = os.clock() + 0.75
1215         makeImpactTool(player, kind)
1216     end)
1217 end
1218
1219 -----
1220 -- PC SIDEARM -- mouse-aimed, costs the shooter 1% of their life per shot
1221 --
1222 -- Fired only through this remote, which only the desktop client script
1223 -- sends, so the weapon is inert on touch and console clients.
1224 -----
1225 local sidearmRemote = ReplicatedStorage:FindFirstChild("LabSidearmFire")
1226 if not sidearmRemote then
1227     sidearmRemote = Instance.new("RemoteEvent")
1228     sidearmRemote.Name = "LabSidearmFire"
1229     sidearmRemote.Parent = ReplicatedStorage
1230 end
1231
1232 local sidearmCooldown = {}
1233 Players.PlayerRemoving:Connect(function(player) sidearmCooldown[player] = nil end)
1234
1235 sidearmRemote.OnServerEvent:Connect(function(player, aimPoint)
1236     -- Superseded by ServerBlasterManager. Keep the legacy remote inert so
1237     -- old clients and exploit traffic cannot enter the retired mouse path.
1238     if true then return end

```

```

1237     if typeof(aimPoint) ~= "Vector3" then return end
1238     local character = player.Character
1239     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1240     local root = character and character:FindFirstChild("HumanoidRootPart")
1241     if not humanoid or not root or humanoid.Health <= 0 then return end
1242
1243     -- The client only ever asks; the server decides whether the player is
1244     -- really holding a weapon, and what that weapon is allowed to do.
1245     local tool = character:FindFirstChildOfClass("Tool")
1246     if not tool or not tool:GetAttribute("LabSidearm") then return end
1247     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or ""]
1248     if not style then return end
1249
1250     local now = os.clock()
1251     if now < (sidearmCooldown[player] or 0) then return end
1252     sidearmCooldown[player] = now + (style.cooldown or 0.35)
1253
1254     local handle = tool:FindFirstChild("Handle")
1255     local origin = (handle and handle.Position or root.Position) + Vector3.new(0, 0.5, 0)
1256     local baseDelta = aimPoint - origin
1257     if baseDelta.Magnitude < 0.1 then return end
1258     local baseDir = baseDelta.Unit
1259     local range = style.range or 200
1260
1261     local params = RaycastParams.new()
1262     params.FilterType = Enum.RaycastFilterType.Exclude
1263     params.FilterDescendantsInstances = {character}
1264
1265     local killsThisShot = 0
1266     for _ = 1, (style.pellets or 1) do
1267         -- Shotgun pellets scatter around the aim line; everything else is exact.
1268         local dir = baseDir
1269         local spread = style.spread or 0
1270         if spread > 0 then
1271             dir = (baseDir + Vector3.new(
1272                 (math.random() - 0.5) * spread * 2,
1273                 (math.random() - 0.5) * spread * 2,
1274                 (math.random() - 0.5) * spread * 2)).Unit
1275         end
1276
1277         local result = workspace.Raycast(origin, dir * range, params)
1278         local hitPosition = result and result.Position or (origin + dir * range)
1279
1280         local beam = Instance.new("Part")
1281         beam.Name = "SidearmTracer"
1282         beam.Size = Vector3.new(0.07, 0.07, (hitPosition - origin).Magnitude)
1283         beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
1284         beam.Color = Color3.fromRGB(255, 226, 140)
1285         beam.Material = Enum.Material.Neon
1286         beam.Anchored = true
1287         beam.CanCollide = false
1288         beam.CanTouch = false
1289         beam.CanQuery = false
1290         beam.CastShadow = false
1291         beam.Parent = workspace
1292         Debris:AddItem(beam, 0.1)
1293
1294         if result then
1295             local instance, normal = result.Instance, result.Normal
1296             local bot = workerBotFromHit(instance)
1297             if bot and not bot:GetAttribute("Dead") then
1298                 -- A head shot kills outright, whatever the weapon.
1299                 local headShot = instance.Name == "Head"
1300                 damageWorkerBot(bot, headShot and 1000 or (style.damage or 25), player)
1301                 if bot:GetAttribute("Dead") then killsThisShot = killsThisShot + 1 end
1302             elseif instance:GetAttribute("GasPipe") then
1303                 detonateGasPipe(instance, player, hitPosition)
1304                 spawnImpact(hitPosition, normal)
1305             else
1306                 local equipment = equipmentFromHit(instance)
1307                 if equipment then
1308                     explodeEquipment(equipment, player)
1309                 elseif instance:GetAttribute("ShootablePoster") then
1310                     punchPoster(instance, hitPosition, normal)
1311                 end
1312                 spawnImpact(hitPosition, normal)

```

```

1313         end
1314     end
1315 end
1316
1317 -- Firing always costs the shooter 1% of their own maximum health,
1318 -- whether or not the shot connects with anything.
1319 humanoid:TakeDamage(humanoid.MaxHealth * (style.selfCostPercent or 1) / 100)
1320
1321 if killsThisShot > 0 then
1322     local complex = workspace:FindFirstChild("Laboratory_Complex")
1323     notify(player, string.format("Target down. %d / %d cleared.",
1324         complex and complex:GetAttribute("BotsKilled") or 0,
1325         complex and complex:GetAttribute("BotsTotal") or 0))
1326 end
1327 end)
1328
1329 -----
1330 -- BLASTER GAMEPLAY BRIDGE
1331 -- ServerBlasterManager owns validation and raycasts. This callback only
1332 -- translates a validated impact into this game's custom worker/environment
1333 -- systems, keeping those systems private to the server.
1334 -----
1335 local blasterBridge = game:GetService("ServerScriptService"):FindFirstChild("BlasterHitBridge")
1336 if not blasterBridge then
1337     blasterBridge = Instance.new("BindableFunction")
1338     blasterBridge.Name = "BlasterHitBridge"
1339     blasterBridge.Parent = game:GetService("ServerScriptService")
1340 end
1341
1342 blasterBridge.OnInvoke = function(player, instance, hitPosition, hitNormal, damage, instantHeadshot)
1343     if typeof(player) ~= "Instance" or not player:IsA("Player") then return false end
1344     if typeof(instance) ~= "Instance" or not instance:IsDescendantOf(workspace) then return false end
1345
1346     -- This receives an already-validated bullet impact; aiming/firing is unchanged.
1347     if instance:GetAttribute("BreakableWindow") then
1348         if instance:GetAttribute("WindowBroken") then return false end
1349         instance:SetAttribute("WindowBroken", true)
1350         instance.Transparency = 1
1351         instance.CanCollide, instance.CanTouch, instance.CanQuery = false, false, false
1352         spawnImpact(hitPosition, hitNormal)
1353         return true
1354     end
1355
1356     local bot = workerBotFromHit(instance)
1357     if bot then
1358         local wasDead = bot:GetAttribute("Dead") == true
1359         local handled = damageWorkerBot(bot, instantHeadshot and 1000 or damage, player)
1360         if handled and not wasDead and bot:GetAttribute("Dead") then
1361             local complex = workspace:FindFirstChild("Laboratory_Complex")
1362             notify(player, string.format("Target down. %d / %d cleared.",
1363                 complex and complex:GetAttribute("BotsKilled") or 0,
1364                 complex and complex:GetAttribute("BotsTotal") or 0))
1365         end
1366         return handled
1367     end
1368
1369     if instance:GetAttribute("GasPipe") then
1370         detonateGasPipe(instance, player, hitPosition)
1371         spawnImpact(hitPosition, hitNormal)
1372         return true
1373     end
1374
1375     local equipment = equipmentFromHit(instance)
1376     if equipment then
1377         explodeEquipment(equipment, player)
1378         spawnImpact(hitPosition, hitNormal)
1379         return true
1380     end
1381
1382     if instance:GetAttribute("ShootablePoster") then
1383         punchPoster(instance, hitPosition, hitNormal)
1384         spawnImpact(hitPosition, hitNormal)
1385         return true
1386     end
1387
1388     return false

```

```

1389 end
1390
1391 -----
1392 -- COLD-STORE SAMPLE TRANSFERS
1393 -----
1394 local COLD_STORE = {Refrigerator = true, ["Chest Freezer"] = true}
1395 local nextSampleNumber = 1000
1396
1397 local PLANT_CHAMBER_KIND = "Plant Growth Chamber"
1398
1399 local function equipmentFloorLabel(part)
1400     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
1401 end
1402
1403 local function equipmentId(part)
1404     local code = part:GetAttribute("GrowthCode")
1405     if code and code ~= "" then return "CHAMBER:" .. code end
1406     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
        ↪ " | ")
1407 end
1408
1409 local function equipmentLabel(part)
1410     local code = part:GetAttribute("GrowthCode")
1411     if code and code ~= "" then return "CHAMBER " .. code end
1412     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
1413 end
1414
1415 local function adaptiveTransferSeconds(source, destination)
1416     local distance = destination and (destination.Position - source.Position).Magnitude or 60
1417     local playerCount = math.max(1, #Players:GetPlayers())
1418     local complex = workspace:FindFirstChild("Laboratory_Complex")
1419     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
1420     return math.clamp(math.floor(180 + distance * 1.2 + math.max(0, 3 - playerCount) * 15
1421         + math.clamp(backlog / 20, 0, 30)), 180, 420)
1422 end
1423 local PLANT_PROCESS = {
1424     ["Germination Chamber"] = {
1425         key = "Germination", gain = 1,
1426         caption = "Warming seed and logging germination response",
1427     },
1428     ["Incubator"] = {
1429         key = "Incubation", gain = 1,
1430         caption = "Holding controlled temperature and humidity",
1431     },
1432     ["Growth Hormone Mixer"] = {
1433         key = "Growth Hormone", gain = 2,
1434         caption = "Dosing auxin/cytokinin growth treatment",
1435     },
1436     ["GMO Injector"] = {
1437         key = "GMO Injection", gain = 1, genome = true,
1438         caption = "Injecting and verifying a new plant trait",
1439     },
1440     ["Nutrient Infuser"] = {
1441         key = "Nutrient Infusion", gain = 1, moistureGain = 18,
1442         caption = "Balancing nitrogen, phosphorus, and potassium",
1443     },
1444     ["UV Growth Scanner"] = {
1445         repeatable = true, gain = 0,
1446         caption = "Scanning chlorophyll and canopy development",
1447     },
1448     ["Pollination Station"] = {
1449         key = "Pollination", gain = 1, minStage = 3,
1450         caption = "Applying controlled pollen and checking viability",
1451     },
1452     -- Micropropagation chain, step 1: surface-sterilize the seed coat before
1453     -- it ever reaches a growth chamber. Nothing downstream requires this, but
1454     -- a Sterilized sample gets a real (and referenced) germination boost --
1455     -- contamination is the #1 real-world cause of failed plant tissue culture.
1456     ["Decontamination Unit"] = {
1457         key = "Sterilized", gain = 0,
1458         caption = "Surface-sterilizing seed coat (dilute bleach + ethanol rinse) and sealing on sterile media",
1459     },
1460     -- Dose-response mutagenesis modeled on the reference Viola cornuta gamma
1461     -- study: germination viability falls off steeply above ~400 Gy, and by
1462     -- 1000 Gy almost nothing survives. Dose is chosen per treatment so two
1463     -- visits to the same chamber can give a different, honestly-labeled result.

```

```

1464     ["Mutagenesis Chamber"] = {
1465         key = "Mutagenesis", gain = 0, mutagenesis = true,
1466         caption = "Irradiating seed with a measured gamma dose",
1467     },
1468     -- Hormone Treatment Bench: auxin pushes rooting, cytokinin pushes
1469     -- shoot/bud growth, and a balanced ratio pushes both -- the same
1470     -- direction of effect reported for lemon seed germination trials.
1471     ["Hormone Treatment Bench"] = {
1472         key = "Hormone Treatment", gain = 1, hormone = true,
1473         caption = "Dosing a measured auxin/cytokinin treatment",
1474     },
1475 }
1476
1477 local function storageId(part)
1478     local id = part:GetAttribute("StorageId")
1479     if not id then
1480         id = table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
1481             ↪ part.Name}, "|")
1482         part:SetAttribute("StorageId", id)
1483     end
1484     return id
1485 end
1486
1487 local function storageLabel(part)
1488     local unit = part.Name:match("_(%d+)$") or "1"
1489     return string.format("%s, %s, %s (unit %s)", part:GetAttribute("Kind") or "Cold store",
1490         part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "room", unit)
1491 end
1492
1493 local function coldStoresExcept(source)
1494     local stores = {}
1495     for _, candidate in ipairs(CollectionService:GetTagged("LabEquipment")) do
1496         if candidate ~= source and COLD_STORE[candidate:GetAttribute("Kind")] then
1497             table.insert(stores, candidate)
1498         end
1499     end
1500     table.sort(stores, function(a, b) return storageId(a) < storageId(b) end)
1501     return stores
1502 end
1503
1504 -----
1505 -- PHYSICAL CONTROL BUTTON AND DATA DISPLAY
1506 -----
1507 local function makeControls(part, kind, spec)
1508     local wallSide = part:GetAttribute("WallSide")
1509     local frontAxisX = wallSide == "W" or wallSide == "E"
1510     local frontDirection = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
1511     local isGrowthChamber = kind == PLANT_CHAMBER_KIND
1512     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
1513     local timerGui = Instance.new("BillboardGui")
1514     timerGui.Name = "EquipmentStatusTimer"
1515     timerGui.Size = UDim2.fromOffset(160, 32)
1516     timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
1517     timerGui.MaxDistance = 10
1518     timerGui.AlwaysOnTop = false
1519     timerGui.Parent = part
1520     local timerLabel = Instance.new("TextLabel")
1521     timerLabel.Name = "Timer"
1522     timerLabel.Size = UDim2.fromScale(1, 1)
1523     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
1524     timerLabel.BackgroundTransparency = 0.16
1525     timerLabel.BorderSizePixel = 0
1526     timerLabel.TextScaled = true
1527     timerLabel.Font = Enum.Font.GothamBold
1528     timerLabel.Parent = timerGui
1529     local timerCorner = Instance.new("UICorner")
1530     timerCorner.CornerRadius = UDim.new(0, 8)
1531     timerCorner.Parent = timerLabel
1532
1533     -- Equipment name plate, mounted directly above the RUNNING/FAILED timer
1534     -- badge so each unit's kind is obvious at a glance from close range.
1535     local nameGui = Instance.new("BillboardGui")
1536     nameGui.Name = "EquipmentNamePlate"
1537     nameGui.Size = UDim2.fromOffset(160, 24)
1538     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
1539     nameGui.MaxDistance = 10

```



```

1539     nameGui.AlwaysOnTop = false
1540     nameGui.Parent = part
1541     local nameLabel = Instance.new("TextLabel")
1542     nameLabel.Name = "Kind"
1543     nameLabel.Size = UDim2.fromScale(1, 1)
1544     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
1545     nameLabel.BackgroundTransparency = 0.05
1546     nameLabel.BorderSizePixel = 0
1547     nameLabel.Text = kind
1548     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
1549     nameLabel.TextScaled = true
1550     nameLabel.Font = Enum.Font.SourceSansBold
1551     nameLabel.Parent = nameGui
1552     local nameCorner = Instance.new("UICorner")
1553     nameCorner.CornerRadius = UDim.new(0, 8)
1554     nameCorner.Parent = nameLabel
1555
1556     local slotGui = Instance.new("BillboardGui")
1557     slotGui.Name = "EquipmentFreeSlots"
1558     slotGui.Size = UDim2.fromOffset(150, 28)
1559     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
1560     slotGui.MaxDistance = 10
1561     slotGui.AlwaysOnTop = false
1562     slotGui.Parent = part
1563     local slotLabel = Instance.new("TextLabel")
1564     slotLabel.Name = "FreeSlots"
1565     slotLabel.Size = UDim2.fromScale(1, 1)
1566     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)
1567     slotLabel.BackgroundTransparency = 0.12
1568     slotLabel.BorderSizePixel = 0
1569     slotLabel.TextScaled = true
1570     slotLabel.Font = Enum.Font.GothamBold
1571     slotLabel.Parent = slotGui
1572     local slotCorner = Instance.new("UICorner")
1573     slotCorner.CornerRadius = UDim.new(0, 8)
1574     slotCorner.Parent = slotLabel
1575     local storedLifeGui, storedLifeText, storedLifeFill
1576     if isPlantContainer then
1577         local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
1578         storedLifeGui = Instance.new("BillboardGui")
1579         storedLifeGui.Name = "StoredPlantLifeStatus"
1580         storedLifeGui.Size = UDim2.fromOffset(140, 40)
1581         storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
1582         storedLifeGui.MaxDistance = 10
1583         storedLifeGui.AlwaysOnTop = false
1584         storedLifeGui.Parent = plantAnchor
1585         storedLifeText = Instance.new("TextLabel")
1586         storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
1587         storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
1588         storedLifeText.BackgroundTransparency = 0.16
1589         storedLifeText.BorderSizePixel = 0
1590         storedLifeText.TextScaled = true
1591         storedLifeText.Font = Enum.Font.GothamBold
1592         storedLifeText.Parent = storedLifeGui
1593         local storedLifeBack = Instance.new("Frame")
1594         storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
1595         storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
1596         storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)
1597         storedLifeBack.BorderSizePixel = 0
1598         storedLifeBack.Parent = storedLifeGui
1599         storedLifeFill = Instance.new("Frame")
1600         storedLifeFill.Size = UDim2.fromScale(1, 1)
1601         storedLifeFill.BorderSizePixel = 0
1602         storedLifeFill.Parent = storedLifeBack
1603     end
1604     local function clockText(seconds)
1605         seconds = math.max(0, math.floor(seconds or 0))
1606         return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
            ↪ seconds % 60)
1607     end
1608     local function renderTimer()
1609         local state = part:GetAttribute("FunctionalState")
1610         local destroyed = state == "DESTROYED"
1611         local failed = destroyed or state == "FAILED"
1612         local since = failed and (part:GetAttribute("FailureSince") or os.time())
1613         or (part:GetAttribute("OperationStarted") or os.time())

```

```

1614     if destroyed then
1615         timerLabel.Text = "DESTROYED -- UNFIXABLE"
1616         timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
1617     else
1618         timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. clockText(os.time() - since)
1619         timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)
1620     end
1621     if isPlantContainer and failed then
1622         local life = math.clamp(100 - math.floor(math.max(0, os.time() - since - 120) / 6), 0, 100)
1623         part:SetAttribute("PlantLife", life)
1624         if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
1625             part:SetAttribute("SampleIds", "")
1626             part:SetAttribute("SampleCount", 0)
1627             part:SetAttribute("Occupied", false)
1628             part:SetAttribute("PlantId", "")
1629             part:SetAttribute("PlantExpiredAt", os.time())
1630             for _, child in ipairs(part:GetDescendants()) do
1631                 if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then child.Transparency = 1
1632                     ↪ end
1633             end
1634         end
1635         local capacity = part:GetAttribute("Capacity") or 0
1636         local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
1637         local free = math.max(0, capacity - count)
1638         slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
1639             or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
1640         slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)
1641         -- Beacon: someone is actively carrying a plant toward THIS chamber
1642         -- (IncomingReservations is set the moment a plant is taken with this
1643         -- chamber as its destination). Make that one chamber's slot readout
1644         -- visible from across the room -- through walls too -- so "where do I
1645         -- put this plant" is answered by looking up, not by wandering every
1646         -- aisle reading labels that are otherwise only readable up close.
1647         local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
1648         if incoming and not failed then
1649             slotGui.MaxDistance = 30
1650             slotGui.AlwaysOnTop = false
1651             slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
1652             slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
1653         else
1654             slotGui.MaxDistance = 10
1655             slotGui.AlwaysOnTop = false
1656         end
1657         if storedLifeGui then
1658             local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
1659             local colour = life > 60 and Color3.fromRGB(75, 235, 105)
1660                 or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
1661             storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
1662             storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
1663             storedLifeText.TextColor3 = colour
1664             storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
1665             storedLifeFill.BackgroundColor3 = colour
1666         end
1667     end
1668     renderTimer()
1669     task.spawn(function()
1670         while part.Parent do
1671             renderTimer()
1672             task.wait(1)
1673         end
1674     end)
1675
1676     local button = Instance.new("Part")
1677     button.Name = "PushButton"
1678     button.Shape = Enum.PartType.Cylinder
1679     button.Size = isGrowthChamber and Vector3.new(0.24, 0.64, 0.64) or Vector3.new(0.2, 0.48, 0.48)
1680     local buttonSide = math.clamp((frontAxisX and part.Size.Z or part.Size.X) * 0.34, 0.25, 1.2)
1681     local buttonY = math.clamp(part.Size.Y * 0.03, -0.15, 0.45)
1682     if frontAxisX then
1683         button.CFrame = part.CFrame * CFrame.new(frontDirection * (part.Size.X / 2 + 0.16), buttonY,
1684             ↪ buttonSide)
1685     else
1686         button.CFrame = part.CFrame * CFrame.new(-buttonSide, buttonY, frontDirection * (part.Size.Z / 2 +
1687             ↪ 0.16))
1688         * CFrame.Angles(0, math.rad(90), 0)
1689     end

```

```

1687     end
1688     button.Color = Color3.fromRGB(218, 68, 58)
1689     button.Material = Enum.Material.Neon
1690     button.Anchored = true
1691     button.CanCollide = false
1692     button.CanTouch = false
1693     button.CastShadow = false
1694     button.Parent = part
1695     local restCFrame = button.CFrame
1696
1697     local prompt = Instance.new("ProximityPrompt")
1698     prompt.ObjectText = isGrowthChamber
1699         and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " • CHAMBER " ..
1700             ↪ (part:GetAttribute("GrowthCode") or "R?"))
1701             or kind
1702     prompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
1703     prompt.KeyboardKeyCode = Enum.KeyCode.E
1704     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
1705     prompt.HoldDuration = isPlantContainer and 0 or 0.2
1706     prompt.MaxActivationDistance = 8
1707     prompt.RequiresLineOfSight = true
1708     prompt.Parent = button
1709
1710     local function setReadout(status, data, colour)
1711         part:SetAttribute("LastActionStatus", tostring(status or ""))
1712         part:SetAttribute("LastActionDetail", tostring(data or ""))
1713         renderTimer()
1714     end
1715
1716     local function pressButton()
1717         button.Color = Color3.fromRGB(255, 205, 70)
1718         local down = TweenService:Create(button, TweenInfo.new(0.07, Enum.EasingStyle.Quad),
1719             {CFrame = restCFrame * CFrame.new(-0.11, 0, 0)})
1720         down:Play()
1721         down.Completed:Wait()
1722         local up = TweenService:Create(button, TweenInfo.new(0.1, Enum.EasingStyle.Quad), {CFrame =
1723             ↪ restCFrame})
1724         up:Play()
1725         task.delay(0.12, function()
1726             if button.Parent then button.Color = Color3.fromRGB(218, 68, 58) end
1727         end)
1728     end
1729
1730     return prompt, setReadout, pressButton
1731 end
1732
1733 local function runProgress(seconds, caption, setReadout)
1734     local steps = 4
1735     for step = 1, steps do
1736         task.wait(seconds / steps)
1737         setReadout("RUNNING " .. (step * 25) .. "%", caption, Color3.fromRGB(255, 205, 80))
1738     end
1739 end
1740
1741 local emergencyMovesCompleted = 0
1742 local setChamberFailure
1743 local setGenericFailure
1744
1745 local function chamberSampleList(chamber)
1746     local samples = {}
1747     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do table.insert(samples, id) end
1748     return samples
1749 end
1750
1751 local function saveChamberSamples(chamber, samples)
1752     local capacity = chamber:GetAttribute("Capacity") or 10
1753     while #samples > capacity do table.remove(samples) end
1754     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
1755     chamber:SetAttribute("SampleCount", #samples)
1756     chamber:SetAttribute("Occupied", #samples > 0)
1757     chamber:SetAttribute("PlantId", samples[1] or "")
1758 end
1759
1760 local function formatRuntime(seconds)
1761     seconds = math.max(0, math.floor(seconds or 0))
1762     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60, seconds %
1763         ↪ 60)

```

```

1761 end
1762
1763 local function recoverPlantLife(chamber, startingLife, player, plantId)
1764     startingLife = math.clamp(startingLife or 100, 0, 100)
1765     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then
1766         chamber:SetAttribute("PlantLife", startingLife)
1767         return 0
1768     end
1769     local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
1770     local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
1771     chamber:SetAttribute("RecoveryToken", token)
1772     chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
1773     chamber:SetAttribute("PlantLife", startingLife)
1774     if player then
1775         player:SetAttribute("LabObjective", string.format("%s dropped in %. LIFE %d%% → 100% in %d seconds.",
1776             plantId or "Plant", chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber), startingLife,
1777             ↪ seconds))
1778         notify(player, string.format("Plant LIFE %d%. Recovery started; full life in %d seconds.",
1779             ↪ startingLife, seconds))
1780     end
1781     task.spawn(function()
1782         for step = 1, seconds do
1783             task.wait(1)
1784             if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
1785                 or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
1786             chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step / seconds +
1787                 ↪ 0.5))
1788         end
1789         chamber:SetAttribute("PlantLife", 100)
1790         chamber:SetAttribute("RecoveryEndsAt", 0)
1791         if player and player.Parent then notify(player, (plantId or "Plant") .. " recovered to 100% LIFE.") end
1792     end)
1793     return seconds
1794 end
1795
1796 local function updateStoredSamples(chamber)
1797     local count = chamber:GetAttribute("SampleCount") or 0
1798     local stage = math.clamp(chamber:GetAttribute("GrowthStage") or 0, 0, 5)
1799     for _, child in ipairs(chamber:GetDescendants()) do
1800         if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then
1801             local visible = child:GetAttribute("SampleSlot") <= count
1802             child.Transparency = visible and (child:GetAttribute("StoredTransparency") or 0) or 1
1803             if child.Name:match("Stem") then child.Color = Color3.fromRGB(55 + stage * 4, 145 + stage * 10, 65)
1804             elseif child.Name:match("Leaf") then child.Color = Color3.fromRGB(65 + stage * 3, 160 + stage * 12,
1805                 ↪ 75) end
1806         end
1807     end
1808 end
1809
1810 local function refreshChamberTelemetry(chamber)
1811     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
1812     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
1813     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
1814     local samples = chamberSampleList(chamber)
1815     local sampleText = #samples > 0 and table.concat(samples, ", ") or "NO SAMPLES"
1816     local destination = chamber:GetAttribute("EvacuationDestination") or ""
1817     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "Unknown fault")
1818         ↪ .. (destination ~= "" and (" -> MOVE TO " .. destination) or " -> WAITING FOR SPACE")) or "SYSTEM
1819         ↪ NORMAL"
1820     local text = string.format("%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM
1821         ↪ %d% • VENT %d%\n%s\n%s",
1822         floorLabel, code, state, #samples, chamber:GetAttribute("Capacity") or 10,
1823         chamber:GetAttribute("GerminationDate") or "unknown",
1824         formatRuntime(os.time() - (chamber:GetAttribute("OperationStarted") or os.time())),
1825         chamber:GetAttribute("Temperature") or 0, chamber:GetAttribute("Humidity") or 0,
1826         chamber:GetAttribute("Ventilation") or 0, detail,
1827         state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
1828     for _, child in ipairs(chamber:GetDescendants()) do
1829         if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
1830             child.Text = text
1831             -- DESTROYED must never read as fine (green): only a genuinely
1832             -- WORKING chamber gets green, FAILED gets the usual red, and a
1833             -- permanently DESTROYED one gets an even harsher red so it never
1834             -- gets mistaken for "just needs a repair."
1835             child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
1836             or state == "FAILED" and Color3.fromRGB(255, 105, 90)
1837         end
1838     end
1839 end

```

```

1831         or Color3.fromRGB(105, 245, 160)
1832     elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
1833         child.Text = "PLANTS (" .. #samples .. "): " .. sampleText
1834     elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
1835         child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)
1836         or state == "FAILED" and Color3.fromRGB(230, 55, 45)
1837         or Color3.fromRGB(40, 165, 95)
1838         child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
1839     end
1840 end
1841 end
1842
1843 local function evacuationDestinations(source)
1844     local sameLevel, anywhere = {}, {}
1845     for _, candidate in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
1846         local count = candidate:GetAttribute("SampleCount") or 0
1847         local capacity = candidate:GetAttribute("Capacity") or 10
1848         local incoming = candidate:GetAttribute("IncomingReservations") or 0
1849         if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED" and count + incoming <
            ↳ capacity then
1850             table.insert(anywhere, candidate)
1851             if candidate:GetAttribute("Level") == source:GetAttribute("Level") then table.insert(sameLevel,
            ↳ candidate) end
1852         end
1853     end
1854     local list = #sameLevel > 0 and sameLevel or anywhere
1855     table.sort(list, function(a, b)
1856         local ac = (a:GetAttribute("SampleCount") or 0) + (a:GetAttribute("IncomingReservations") or 0)
1857         local bc = (b:GetAttribute("SampleCount") or 0) + (b:GetAttribute("IncomingReservations") or 0)
1858         if ac == bc then return (a.Position - source.Position).Magnitude < (b.Position -
            ↳ source.Position).Magnitude end
1859         return ac < bc
1860     end)
1861     return list
1862 end
1863
1864 local function repairPathFolder(player, create)
1865     local complex = workspace:FindFirstChild("Laboratory_Complex")
1866     if not complex then return nil end
1867     local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
1868     if create and not rootFolder then
1869         rootFolder = Instance.new("Folder")
1870         rootFolder.Name = "TechnicianRepairPaths"
1871         rootFolder.Parent = complex
1872     end
1873     local name = "Player_" .. player.UserId
1874     local folder = rootFolder and rootFolder:FindFirstChild(name)
1875     if create and not folder then
1876         folder = Instance.new("Folder")
1877         folder.Name = name
1878         folder.Parent = rootFolder
1879     end
1880     return folder
1881 end
1882
1883 local function clearRepairPath(player)
1884     local folder = repairPathFolder(player, false)
1885     if folder then folder:Destroy() end
1886 end
1887
1888 local function repairReward(player, equipment, assigned)
1889     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
1890     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
1891     local points = 8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0)
1892     return math.clamp(points, 8, 120), waited, distance
1893 end
1894
1895 local function createRepairPath(player, target)
1896     clearRepairPath(player)
1897     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
1898     if not characterRoot or not target then return 0, false end
1899     local folder = repairPathFolder(player, true)
1900     if not folder then return 0, false end
1901
1902     local function approachPoint(part)
1903         local side = part:GetAttribute("WallSide")

```

```

1904     local floorY = part.Position.Y - part.Size.Y / 2
1905     if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
1906     ↪ part.Position.Z) end
1907     if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
1908     ↪ part.Position.Z) end
1909     if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
1910     ↪ 2.5) end
1911     return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
1912 end
1913 local function appendNavigable(points, fromPosition, toPosition)
1914     local path = PathfindingService.CreatePath({
1915         AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
1916         AgentCanClimb = true, WaypointSpacing = 4,
1917     })
1918     local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
1919     if not computed or path.Status ~= Enum.PathStatus.Success then return false end
1920     local waypoints = path.GetWaypoints()
1921     if #waypoints < 2 then return false end
1922     for index, waypoint in ipairs(waypoints) do
1923         if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
1924     end
1925     return true
1926 end
1927
1928 local complex = workspace:FindFirstChild("Laboratory_Complex")
1929 local rise = complex and complex:GetAttribute("FloorRiseStuds")
1930 local westX = complex and complex:GetAttribute("StairWestLaneX")
1931 local eastX = complex and complex:GetAttribute("StairEastLaneX")
1932 local northZ = complex and complex:GetAttribute("StairNorthZ")
1933 local southZ = complex and complex:GetAttribute("StairSouthZ")
1934 local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
1935 local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
1936 local destination = approachPoint(target)
1937 local points = {}
1938 local routeReady = false
1939 if rise and westX and eastX and northZ and southZ then
1940     local startFloor = math.clamp(LabSpatial.floor(characterRoot.Position, 2.5), floorMin, floorMax)
1941     local targetFloor = math.clamp(LabSpatial.floor(destination, 0), floorMin, floorMax)
1942     if startFloor == targetFloor then
1943         routeReady = appendNavigable(points, characterRoot.Position, destination)
1944     else
1945         local direction = targetFloor > startFloor and 1 or -1
1946         local firstLaneX = direction > 0 and westX or eastX
1947         local stairEntry = LabSpatial.world(Vector3.new(firstLaneX, startFloor * rise, northZ))
1948         routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
1949         local floor = startFloor
1950         while routeReady and floor ~= targetFloor do
1951             local baseY = floor * rise
1952             if direction > 0 then
1953                 table.insert(points, LabSpatial.world(Vector3.new(westX, baseY + rise / 2, southZ)))
1954                 table.insert(points, LabSpatial.world(Vector3.new(eastX, baseY + rise / 2, southZ)))
1955                 table.insert(points, LabSpatial.world(Vector3.new(eastX, baseY + rise, northZ)))
1956             else
1957                 table.insert(points, LabSpatial.world(Vector3.new(eastX, baseY - rise / 2, southZ)))
1958                 table.insert(points, LabSpatial.world(Vector3.new(westX, baseY - rise / 2, southZ)))
1959                 table.insert(points, LabSpatial.world(Vector3.new(westX, baseY - rise, northZ)))
1960             end
1961             floor = floor + direction
1962             if floor == targetFloor then
1963                 table.insert(points, LabSpatial.world(Vector3.new(direction > 0 and westX or eastX, floor *
1964                 ↪ rise, northZ)))
1965             end
1966         end
1967         if routeReady then
1968             routeReady = appendNavigable(points, points[#points], destination)
1969         end
1970     end
1971 end
1972 if not routeReady then
1973     folder:Destroy()
1974     player:SetAttribute("RepairRouteReady", false)
1975     return (destination - characterRoot.Position).Magnitude, false
1976 end
1977 player:SetAttribute("RepairRouteReady", true)
1978 local distance = 0
1979 local markerCount = 0

```

```

1976     for pointIndex = 1, #points - 1 do
1977         local a, b = points[pointIndex], points[pointIndex + 1]
1978         local delta = b - a
1979         local segmentDistance = delta.Magnitude
1980         distance = distance + segmentDistance
1981         if segmentDistance > 0.1 then
1982             local direction = delta.Unit
1983             local count = math.max(1, math.ceil(segmentDistance / 4))
1984             for index = 1, count do
1985                 markerCount = markerCount + 1
1986                 if markerCount > 120 then break end
1987                 local marker = Instance.new("Part")
1988                 marker.Name = "RedRepairPath"
1989                 marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
1990                 local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
1991                 marker.CFrame = CFrame.lookAt(position, position + direction)
1992                 marker.Color = Color3.fromRGB(255, 32, 28)
1993                 marker.Material = Enum.Material.Neon
1994                 marker.Transparency = 0.12
1995                 marker.Anchored = true
1996                 marker.CanCollide = false
1997                 marker.CanTouch = false
1998                 marker.CanQuery = false
1999                 marker.Parent = folder
2000             end
2001         end
2002         if markerCount > 120 then break end
2003     end
2004     return distance, true
2005 end
2006
2007 local function clearAssignment(player)
2008     clearRepairPath(player)
2009     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
2010 ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
2011         player:SetAttribute(name, nil)
2012     end
2013 end
2014
2015 local function assignNextTask(player)
2016     clearAssignment(player)
2017     local role = playerRole(player)
2018     if role == "Technician" then
2019         local failed = {}
2020         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
2021             if equipment.Parent and equipment:GetAttribute("FunctionalState") == "FAILED" then
2022                 ↪ table.insert(failed, equipment) end
2023         end
2024         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
2025         table.sort(failed, function(a, b)
2026             if root then
2027                 local aDistance = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
2028 ↪ root.Position.Y) > 12 and 500 or 0)
2029                 local bDistance = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
2030 ↪ root.Position.Y) > 12 and 500 or 0)
2031                 if aDistance ~= bDistance then return aDistance < bDistance end
2032             end
2033             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
2034         end)
2035         local target = failed[1]
2036         if not target then
2037             local text = "All equipment is working. Repair score increases with failure time; assigned jobs
2038 ↪ also pay for travel distance."
2039             player:SetAttribute("LabObjective", text)
2040             return text
2041         end
2042         player:SetAttribute("AssignedRepairId", equipmentId(target))
2043         local distance, routeReady = createRepairPath(player, target)
2044         player:SetAttribute("AssignedRepairDistance", distance)
2045         local projected = select(1, repairReward(player, target, true))
2046         player:SetAttribute("AssignedRepairProjectedPoints", projected)
2047         local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR; route is
2048 ↪ recalculating"
2049         local text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
2050 ↪ while failed).",

```

```

2044     equipmentFloorLabel(target), equipmentLabel(target), routeInstruction, projected,
        ↪ math.floor(distance + 0.5))
2045     player:SetAttribute("LabObjective", text)
2046     return text
2047 elseif role == "Laboratory Technician" then
2048     local sources = {}
2049     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
2050         if chamber.Parent and chamber:GetAttribute("FunctionalState") == "FAILED"
2051             and (chamber:GetAttribute("SampleCount") or 0) > 0
2052             and not chamber:GetAttribute("ReservedByBot") then
2053             local destination = evacuationDestinations(chamber)[1]
2054             if destination then table.insert(sources, {source = chamber, destination = destination}) end
2055         end
2056     end
2057     table.sort(sources, function(a, b)
2058         local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
2059         local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
2060         return an < bn
2061     end)
2062     local job = sources[1]
2063     if not job then
2064         local text = "Laboratory Technician: waiting for a failed chamber with plants. Emergency moves pay
        ↪ +12; assigned on-time moves pay +30."
2065         player:SetAttribute("LabObjective", text)
2066         return text
2067     end
2068     local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
2069     local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
2070     local allowedSeconds = adaptiveTransferSeconds(job.source, job.destination)
2071     local deadline = os.time() + allowedSeconds
2072     player:SetAttribute("AssignedTransferSource", sourceCode)
2073     player:SetAttribute("AssignedTransferDestination", destinationCode)
2074     player:SetAttribute("AssignedTransferDeadline", deadline)
2075     -- Same red floor-marker route the Technician role gets to a failed
2076     -- unit, but pointed at the destination chamber the plant must be
2077     -- carried to -- so "where does this go" is never a guess.
2078     local routeReady = select(2, createRepairPath(player, job.destination))
2079     local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
        ↪ recalculating"
2080     local text = string.format("ASSIGNED +%d: %s • %s → %s • %s. Press E to take a plant. %s. Deliver in %d
        ↪ sec.",
2081         POINTS.MOVE_ASSIGNED_ON_TIME, equipmentFloorLabel(job.source), sourceCode, destinationCode,
2082         equipmentFloorLabel(job.destination), routeInstruction, allowedSeconds)
2083     player:SetAttribute("LabObjective", text)
2084     return text
2085 end
2086 local text = "Choose BLUE Technician or GREEN Laboratory Technician at a role kiosk."
2087 player:SetAttribute("LabObjective", text)
2088 return text
2089 end
2090
2091 local function chamberByCode(code)
2092     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
2093         if chamber:GetAttribute("GrowthCode") == code then return chamber end
2094     end
2095 end
2096
2097 local function takePlantFromChamber(player, chamber, setReadout)
2098     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
2099     if playerRole(player) ~= "Laboratory Technician" then
2100         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician moves plants", Color3.fromRGB(255,
        ↪ 175, 80))
2101         notify(player, "Only a Laboratory Technician can take plants. Choose the GREEN role; Technicians repair
        ↪ with R.")
2102         return
2103     end
2104     local samples = chamberSampleList(chamber)
2105     if #samples == 0 then setReadout(code .. " EMPTY", "No germinated samples available", Color3.fromRGB(255,
        ↪ 200, 90)); return end
2106     local failed = chamber:GetAttribute("FunctionalState") == "FAILED"
2107     local destination
2108     if failed then
2109         local assignedSource = player:GetAttribute("AssignedTransferSource")
2110         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
2111         if assignedSource == code and assignedDestination then
2112             local requested = chamberByCode(assignedDestination)

```



```

2113         if requested and requested:GetAttribute("FunctionalState") ~= "FAILED"
2114             and (requested:GetAttribute("SampleCount") or 0) +
2115                 ↪ (requested:GetAttribute("IncomingReservations") or 0)
2116                 < (requested:GetAttribute("Capacity") or 10) then
2117                 destination = requested
2118             end
2119         destination = destination or evacuationDestinations(chamber)[1]
2120         if not destination then
2121             setReadout(code .. " FAILED", "No working chamber has free capacity", Color3.fromRGB(255, 105,
2122                 ↪ 90))
2123             notify(player, "No safe destination currently has room. The adaptive controller will reduce
2124                 ↪ failures and free capacity.")
2125             return
2126         end
2127         -- Refresh the floor-marker route to whichever chamber this specific
2128         -- plant actually ends up assigned to (may differ from the original
2129         -- job destination if that one filled up in the meantime).
2130         createRepairPath(player, destination)
2131     end
2132     local plantId = samples[1]
2133     local stage = chamber:GetAttribute("GrowthStage") or 0
2134     local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
2135     local plantLife = chamber:GetAttribute("PlantLife") or 100
2136     if failed then
2137         plantLife = math.clamp(100 - math.floor(math.max(0, os.time() - (chamber:GetAttribute("FailureSince")
2138             ↪ or os.time()) - 120) / 6), 0, 100)
2139     end
2140     setReadout("TAKING SAMPLE", "Removing " .. plantId .. " from " .. code, Color3.fromRGB(120, 235, 170))
2141     local function canFinishTaking()
2142         return chamber.Parent ~= nil and playerRole(player) == "Laboratory Technician"
2143             and canInteract(player, chamber, 10) and not findPlantSample(player)
2144             and chamber:GetAttribute("FunctionalState") ~= "DESTROYED"
2145             and chamberSampleList(chamber)[1] == plantId
2146             and (not destination or (destination.Parent ~= nil and destination:GetAttribute("FunctionalState")
2147                 ↪ == "WORKING"
2148                 and (destination:GetAttribute("SampleCount") or 0) +
2149                     ↪ (destination:GetAttribute("IncomingReservations") or 0)
2150                     < (destination:GetAttribute("Capacity") or 10)))
2151     end
2152     if not runMathQuiz(player, "TAKE PLANT " .. plantId, chamber, canFinishTaking) then return end
2153     -- Re-read inventory after the popup; do not overwrite another transfer.
2154     samples = chamberSampleList(chamber)
2155     plantLife = math.clamp(chamber:GetAttribute("PlantLife") or 100, 0, 100)
2156     table.remove(samples, 1)
2157     local sample = makeTool(player, "Plant Sample " .. plantId, {
2158         itemType = failed and "Emergency Plant Transfer" or "Plant Sample",
2159         sample = true, plant = true, label = plantId, colour = Color3.fromRGB(133, 92, 55),
2160         attributes = {
2161             PlantSample = true, SampleId = plantId, PlantId = plantId, HomeGrowthCode = code, SourceGrowthCode
2162                 ↪ = code,
2163             GrowthStage = stage, Treatments = chamber:GetAttribute("Treatments") or "",
2164             Genome = chamber:GetAttribute("Genome") or "Wild Type", Moisture = chamber:GetAttribute("Moisture")
2165                 ↪ or 50,
2166             PlantLife = plantLife,
2167             GerminationDate = chamber:GetAttribute("GerminationDate") or "unknown",
2168             EvacuationSample = failed, DestinationGrowthCode = destinationCode, TransferStartedAt = os.time(),
2169         },
2170     })
2171     if not sample then table.insert(samples, 1, plantId); return end
2172     saveChamberSamples(chamber, samples)
2173     updateStoredSamples(chamber)
2174     refreshChamberTelemetry(chamber)
2175     refreshPlantVisual(sample)
2176     setCarryState(player, sample)
2177     task.spawn(function()
2178         while sample.Parent and (sample:GetAttribute("PlantLife") or 0) > 0 do
2179             task.wait(1)
2180             if not sample.Parent then return end
2181             if not sample:GetAttribute("QuizProtected") then
2182                 local carriedFor = os.time() - (sample:GetAttribute("TransferStartedAt") or os.time())
2183                 local life = math.max(0, (sample:GetAttribute("PlantLife") or 0) - (carriedFor > 180 and 0.2 or
2184                     ↪ 0))
2185                 sample:SetAttribute("PlantLife", life)
2186                 refreshPlantVisual(sample)
2187                 if life <= 0 then

```

```

2180         local reservedCode = sample:GetAttribute("DestinationGrowthCode") or ""
2181         local reserved = reservedCode ~= "" and chamberByCode(reservedCode) or nil
2182         if reserved then reserved:SetAttribute("IncomingReservations", math.max(0,
2183             ↪ (reserved:GetAttribute("IncomingReservations") or 1) - 1)) end
2183         player:SetAttribute("CarriedSample", "")
2184         player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
2185             ↪ another plant.")
2185         task.wait(0.5)
2186         if sample.Parent then sample:Destroy() end
2187         return
2188     end
2189 end
2190 end
2191 end)
2192 if destination then
2193     destination:SetAttribute("IncomingReservations", (destination:GetAttribute("IncomingReservations") or
2194         ↪ 0) + 1)
2194     -- Begin the bonus clock after pickup, excluding the time spent on the quiz.
2195     player:SetAttribute("AssignedTransferSource", code)
2196     player:SetAttribute("AssignedTransferDestination", destinationCode)
2197     player:SetAttribute("AssignedTransferDeadline", os.time() + adaptiveTransferSeconds(chamber,
2198         ↪ destination))
2198     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
2199     local seconds = math.max(0, deadline - os.time())
2200     player:SetAttribute("LabObjective", string.format("CARRYING %s: press E at ANY GREEN RUNNING cabinet
2201         ↪ with a FREE SLOT. BONUS target %s has %d sec left for +%d points.",
2202             plantId, destinationCode, seconds, POINTS.MOVE_ASSIGNED_ON_TIME))
2202     setReadout(code .. " FAILED", plantId .. " removed • SEND TO " .. destinationCode, Color3.fromRGB(255,
2203         ↪ 145, 75))
2203     notify(player, "ALARM: " .. code .. " failed (" .. (chamber:GetAttribute("FailureReason") or "fault")
2204         ↪ .. "). Move " .. plantId .. " to " .. destinationCode .. ".")
2204 else
2205     player:SetAttribute("LabObjective", "Treat " .. plantId .. ", then press E at any GREEN RUNNING cabinet
2206         ↪ with a FREE SLOT.")
2206     setReadout(code .. " WORKING", plantId .. " taken • " .. #samples .. "/" ..
2207         ↪ (chamber:GetAttribute("Capacity") or 0) .. " remain", Color3.fromRGB(115, 220, 160))
2207     notify(player, plantId .. " equipped in your hand. Process it or move it to a working chamber.")
2208 end
2209 end
2210
2211 local function storePlantInChamber(player, chamber, plant, setReadout)
2212     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
2213     if playerRole(player) ~= "Laboratory Technician" then
2214         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician stores plants",
2215             ↪ Color3.fromRGB(255, 175, 80))
2215         notify(player, "Only a Laboratory Technician can complete plant transfers.")
2216         return
2217     end
2218     if chamber:GetAttribute("FunctionalState") == "FAILED" then
2219         setReadout(code .. " FAILED", "Cannot drop here • find any GREEN RUNNING cabinet", Color3.fromRGB(255,
2220             ↪ 105, 90))
2220         notify(player, code .. " is failed. Carry the plant to any GREEN RUNNING cabinet with a FREE SLOT and
2221             ↪ press E.")
2221         return
2222     end
2223     local samples = chamberSampleList(chamber)
2224     local capacity = chamber:GetAttribute("Capacity") or 10
2225     if #samples >= capacity then
2226         setReadout(code .. " FULL", capacity .. "/" .. capacity .. " samples • choose another working
2227             ↪ chamber", Color3.fromRGB(255, 175, 75))
2227         return
2228     end
2229     local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
2230     local plantId = plant:GetAttribute("PlantId") or plant.Name
2231     local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
2232     local sourceCode = plant:GetAttribute("SourceGrowthCode") or plant:GetAttribute("HomeGrowthCode") or ""
2233     local assignedMove = wanted ~= "" and player:GetAttribute("AssignedTransferSource") == sourceCode
2234         and player:GetAttribute("AssignedTransferDestination") == code
2235     local assignedOnTime = assignedMove and os.time() <= (player:GetAttribute("AssignedTransferDeadline") or 0)
2236     setReadout("STORING SAMPLE", "Placing " .. plantId .. " into " .. code, Color3.fromRGB(120, 235, 170))
2237     -- Only repair and pickup use a quiz; delivery remains immediate.
2238     if not plant.Parent or not canInteract(player, chamber, 10) then return end
2239     table.insert(samples, plantId)
2240     saveChamberSamples(chamber, samples)
2241     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
2242     chamber:SetAttribute("GrowthStage", math.max(chamber:GetAttribute("GrowthStage") or 0, stage))

```

```

2243     chamber:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
2244     chamber:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
2245     chamber:SetAttribute("Moisture", plant:GetAttribute("Moisture") or chamber:GetAttribute("Moisture") or 50)
2246     local recoverySeconds = recoverPlantLife(chamber, plantLife, player, plantId)
2247     if wanted ~= "" then
2248         local reservedDestination = chamberByCode(wanted)
2249         if reservedDestination then
2250             reservedDestination:SetAttribute("IncomingReservations", math.max(0,
2251                 ↪ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
2252         end
2253         emergencyMovesCompleted = emergencyMovesCompleted + 1
2254         player:SetAttribute("EmergencyTransfers", (player:GetAttribute("EmergencyTransfers") or 0) + 1)
2255     end
2256     updateStoredSamples(chamber)
2257     refreshChamberTelemetry(chamber)
2258     destroyCarriedItem(player, plant)
2259     local points = 0
2260     local reason = "plant stored"
2261     if wanted ~= "" then
2262         GameState.transfersCompleted = GameState.transfersCompleted + 1
2263         if assignedMove then
2264             GameState.assignedTransfersCompleted = GameState.assignedTransfersCompleted + 1
2265             points = assignedOnTime and POINTS.MOVE_ASSIGNED_ON_TIME or POINTS.MOVE_ASSIGNED_LATE
2266             reason = assignedOnTime and ("assigned on-time move " .. sourceCode .. " → " .. code)
2267                 or ("assigned late move " .. sourceCode .. " → " .. code)
2268         else
2269             points = POINTS.MOVE_ANY
2270             reason = "emergency plant move " .. sourceCode .. " → " .. code
2271         end
2272     end
2273     setReadout(code .. " STORED" .. (points > 0 and (" +" .. points) or ""),
2274         plantId .. " • " .. #samples .. "/" .. capacity .. " samples", Color3.fromRGB(115, 245, 150))
2275     if points > 0 then
2276         awardPoints(player, points, reason)
2277         if recoverySeconds > 0 then
2278             task.delay(recoverySeconds, function() if player.Parent then assignNextTask(player) end end)
2279         else
2280             assignNextTask(player)
2281         end
2282     else
2283         notify(player, plantId .. " safely stored in " .. code .. ". Capacity: " .. #samples .. "/" .. capacity
2284             ↪ .. ".")
2285     end
2286 end
2287
2288 local function handleGrowthChamber(player, chamber, setReadout)
2289     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
2290     local reservedBy = chamber:GetAttribute("ReservedByBot")
2291     if reservedBy then
2292         -- Human players always have priority over test bots. Clearing the
2293         -- reservation makes one press of E immediately take or drop a plant.
2294         chamber:SetAttribute("ReservedByBot", nil)
2295     end
2296     local plant = findPlantSample(player)
2297     if plant then storePlantInChamber(player, chamber, plant, setReadout)
2298     elseif (chamber:GetAttribute("SampleCount") or 0) > 0 then takePlantFromChamber(player, chamber,
2299         ↪ setReadout)
2300     else
2301         setReadout(code .. " EMPTY", "Working chamber can receive a carried sample", Color3.fromRGB(255, 200,
2302             ↪ 90))
2303     end
2304 end
2305
2306 local GMO_TRAITS = {"Drought Resistant", "High Yield", "Salt Tolerant", "Fast Flowering", "Disease Resistant"}
2307
2308 -- Dose-response germination-viability curve, bucketed off the reference
2309 -- Viola cornuta gamma-irradiation study (Trends Sci. 2025;22(1):8705): mild
2310 -- doses barely dent germination, but survival collapses above ~600 Gy and is
2311 -- nearly gone by 1000 Gy. Each visit rolls a dose and a survival check.
2312 local MUTAGENESIS_DOSES = {
2313     {gy = 100, survivalMin = 85, survivalMax = 95},
2314     {gy = 200, survivalMin = 65, survivalMax = 85},
2315     {gy = 400, survivalMin = 45, survivalMax = 70},
2316     {gy = 600, survivalMin = 15, survivalMax = 40},
2317     {gy = 800, survivalMin = 5, survivalMax = 20},
2318     {gy = 1000, survivalMin = 0, survivalMax = 8},

```

```

2315 }
2316
2317 -- Auxin pushes rooting, cytokinin pushes shoot/bud growth, and a balanced
2318 -- ratio pushes both -- the direction of effect reported for lemon seed
2319 -- germination hormone trials, kept as a labeled tag rather than a hidden stat.
2320 local HORMONE_TREATMENTS = {
2321     {name = "Auxin-dominant (IBA-heavy)",      focus = "Root",  gain = 1},
2322     {name = "Cytokinin-dominant (BAP-heavy)",   focus = "Shoot", gain = 1},
2323     {name = "Balanced 1:1 auxin:cytokinin",     focus = "Root+Shoot", gain = 2},
2324 }
2325
2326 local function germinationPercent(player)
2327     local sown = player:GetAttribute("GermSownCount") or 0
2328     if sown == 0 then return 0 end
2329     return math.floor(((player:GetAttribute("GermNormalCount") or 0) / sown) * 100 + 0.5)
2330 end
2331
2332 local function processPlant(player, part, spec, process, setReadout)
2333     if playerRole(player) ~= "Laboratory Technician" then
2334         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician develops plants",
2335             ↪ Color3.fromRGB(255, 175, 80))
2336         notify(player, "Only a Laboratory Technician can use plant-development equipment.")
2337         return false
2338     end
2339     local plant = findPlantSample(player)
2340     if not plant then
2341         setReadout("PLANT REQUIRED", "Take one from an R-numbered chamber", Color3.fromRGB(255, 150, 85))
2342         notify(player, part:GetAttribute("Kind") .. ": take a visible plant sample from an R chamber first.")
2343         return false
2344     end
2345     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
2346     if process.minStage and stage < process.minStage then
2347         setReadout("STAGE TOO EARLY", "Needs Budding stage or later", Color3.fromRGB(255, 150, 85))
2348         notify(player, "Grow " .. (plant:GetAttribute("PlantId") or "the plant") .. " to Budding before
2349             ↪ pollination.")
2350         return false
2351     end
2352     if process.key and not process.repeatable and hasTreatment(plant, process.key) then
2353         setReadout("ALREADY APPLIED", process.key .. " is already recorded", Color3.fromRGB(255, 180, 85))
2354         notify(player, process.key .. " has already been applied to this plant. Use another machine.")
2355         return false
2356     end
2357     runProgress(spec.wait or 1.5, process.caption, setReadout)
2358
2359     -- Mutagenesis Chamber: roll a dose, then roll survival against that
2360     -- dose's real-world-modeled band. A killed sample ends here.
2361     if process.mutagenesis then
2362         local bucket = MUTAGENESIS_DOSES[math.random(1, #MUTAGENESIS_DOSES)]
2363         local survivalPercent = math.random(bucket.survivalMin, bucket.survivalMax)
2364         local survived = math.random(1, 100) <= math.max(survivalPercent, 1)
2365         plant:SetAttribute("MutagenesisDoseGy", bucket.gy)
2366         plant:SetAttribute("MutagenesisSurvivalPercent", survivalPercent)
2367         if not survived then
2368             local plantId = plant:GetAttribute("PlantId") or "sample"
2369             setReadout("SAMPLE LOST", string.format("%d Gy dose • %d%% survival odds • did not survive",
2370                 ↪ bucket.gy, survivalPercent),
2371                 ↪ Color3.fromRGB(255, 90, 80))
2372             notify(player, plantId .. " did not survive the " .. bucket.gy .. " Gy dose (" .. survivalPercent
2373                 ↪ .. "% survival odds). Take another plant.")
2374             destroyCarriedItem(player, plant)
2375             return true
2376         end
2377         addTreatment(plant, "Mutagenesis " .. bucket.gy .. "Gy")
2378         plant:SetAttribute("Genome", "Mutagenized (" .. bucket.gy .. " Gy)")
2379     end
2380
2381     -- Hormone Treatment Bench: roll a hormone mix, tag the resulting focus.
2382     if process.hormone then
2383         local mix = HORMONE_TREATMENTS[math.random(1, #HORMONE_TREATMENTS)]
2384         plant:SetAttribute("HormoneTreatment", mix.name)
2385         plant:SetAttribute("HormoneFocus", mix.focus)
2386         process = {key = process.key, gain = mix.gain, moistureGain = process.moistureGain, caption =
2387             ↪ process.caption}
2388         addTreatment(plant, "Hormone: " .. mix.name)
2389     end

```

```

2386     elseif process.key then
2387         addTreatment(plant, process.key)
2388     end
2389
2390     -- Germination Chamber: classify the outcome (AOSA normal / abnormal /
2391     -- dead), the same three buckets a real seed-testing lab reports, and
2392     -- track a running germination percentage on the Laboratory Technician.
2393     -- A prior Decontamination Unit visit (Sterilized treatment) meaningfully
2394     -- improves the odds, matching real seed-testing guidance that
2395     -- contamination is the leading cause of an abnormal or dead result.
2396     if process.key == "Germination" then
2397         local sterilized = hasTreatment(plant, "Sterilized")
2398         local irradiated = plant:GetAttribute("MutagenesisSurvivalPercent")
2399         local normalChance = sterilized and 82 or 60
2400         if irradiated then normalChance = math.floor(normalChance * (irradiated / 100)) end
2401         local roll = math.random(1, 100)
2402         local outcome
2403         if roll <= normalChance then outcome = "Normal"
2404         elseif roll <= normalChance + 25 then outcome = "Abnormal"
2405         else outcome = "Dead" end
2406         plant:SetAttribute("GerminationOutcome", outcome)
2407         player:SetAttribute("GermSownCount", (player:GetAttribute("GermSownCount") or 0) + 1)
2408         if outcome == "Normal" then
2409             player:SetAttribute("GermNormalCount", (player:GetAttribute("GermNormalCount") or 0) + 1)
2410         elseif outcome == "Abnormal" then
2411             player:SetAttribute("GermAbnormalCount", (player:GetAttribute("GermAbnormalCount") or 0) + 1)
2412             plant:SetAttribute("PlantLife", math.clamp((plant:GetAttribute("PlantLife") or 100) - 25, 10, 100))
2413         else
2414             player:SetAttribute("GermDeadCount", (player:GetAttribute("GermDeadCount") or 0) + 1)
2415             local plantId = plant:GetAttribute("PlantId") or "sample"
2416             setReadout("DEAD • AOSA", string.format("Lab germination rate now %d%%",
2417                 ↪ germinationPercent(player)), Color3.fromRGB(255, 90, 80))
2418             notify(player, plantId .. " classified DEAD under AOSA rules (" .. (sterilized and "sterilized" or
2419                 ↪ "unsterilized")
2420                 .. " seed). Your running germination rate: " .. germinationPercent(player) .. "%. Take another
2421                 ↪ plant.")
2422             destroyCarriedItem(player, plant)
2423             return true
2424         end
2425     end
2426
2427     local newStage = math.clamp(stage + (process.gain or 0), 0, 5)
2428     plant:SetAttribute("GrowthStage", newStage)
2429     plant:SetAttribute("Moisture", math.clamp((plant:GetAttribute("Moisture") or 50) + (process.moistureGain or
2430         ↪ -2), 15, 100))
2431     if process.genome then plant:SetAttribute("Genome", GMO_TRAITS[math.random(1, #GMO_TRAITS)]) end
2432     refreshPlantVisual(plant)
2433     setCarryState(player, plant)
2434     player:SetAttribute("LabObjective", "Plant " .. (plant:GetAttribute("PlantId") or "sample") .. " is "
2435         .. (GROWTH_STAGE_NAME[newStage] or "Growing") .. ". Use another treatment or return it to an empty R
2436         ↪ chamber.")
2437     if process.key then awardPoints(player, POINTS.PROCESS_PLANT, process.key .. " plant treatment") end
2438
2439     local data = string.format("%s • %s • %d%% moisture\nGenome: %s",
2440         plant:GetAttribute("PlantId") or "PLANT", GROWTH_STAGE_NAME[newStage] or "Growing",
2441         plant:GetAttribute("Moisture") or 50, plant:GetAttribute("Genome") or "Wild Type")
2442     if plant:GetAttribute("GerminationOutcome") then
2443         data = data .. string.format("\nAOSA: %s • Lab germination rate %d%%",
2444             ↪ plant:GetAttribute("GerminationOutcome"), germinationPercent(player))
2445     end
2446     if plant:GetAttribute("MutagenesisDoseGy") then
2447         data = data .. string.format("\nDose %d Gy • %d%% survival odds",
2448             ↪ plant:GetAttribute("MutagenesisDoseGy"), plant:GetAttribute("MutagenesisSurvivalPercent") or 0)
2449     end
2450     if plant:GetAttribute("HormoneTreatment") then
2451         data = data .. "\n" .. plant:GetAttribute("HormoneTreatment") .. " • " ..
2452             ↪ (plant:GetAttribute("HormoneFocus") or "")
2453     end
2454     setReadout("PLANT DATA", data, Color3.fromRGB(120, 240, 160))
2455     notify(player, data:gsub("\n", " | ") .. " | Treatments: " .. treatmentText(plant))
2456     equipTool(player, plant)
2457     return true
2458 end
2459
2460 local function createTransferSample(player, source, setReadout)
2461     local existing = findTransferSample(player)

```

```

2454     if existing then
2455         local target = existing:GetAttribute("DestinationLabel") or "the named cold store"
2456         setReadout("SAMPLE IN TRANSIT", (existing:GetAttribute("SampleId") or existing.Name) .. " -> " ..
            ↳ target,
2457             Color3.fromRGB(255, 205, 80))
2458         notify(player, "You already carry " .. (existing:GetAttribute("SampleId") or existing.Name) .. ".
            ↳ Deliver it to " .. target .. ".")
2459         equipTool(player, existing)
2460         return
2461     end
2462
2463     local destinations = coldStoresExcept(source)
2464     if #destinations == 0 then
2465         setReadout("TRANSFER UNAVAILABLE", "A second refrigerator or freezer is required.", Color3.fromRGB(255,
            ↳ 110, 100))
2466         notify(player, "This transfer needs at least two refrigerators/freezers in the generated lab.")
2467         return
2468     end
2469     -- Prefer a true refrigerator destination so the guaranteed lobby
2470     -- pair always supports the requested fridge-to-fridge play loop.
2471     local refrigeratorDestinations = {}
2472     for _, candidate in ipairs(destinations) do
2473         if candidate:GetAttribute("Kind") == "Refrigerator" then
2474             table.insert(refrigeratorDestinations, candidate)
2475         end
2476     end
2477     if #refrigeratorDestinations > 0 then destinations = refrigeratorDestinations end
2478
2479     local destination = destinations[math.random(1, #destinations)]
2480     nextSampleNumber = nextSampleNumber + 1
2481     local sampleId = string.format("SL-%04d", nextSampleNumber)
2482     local targetLabel = storageLabel(destination)
2483     local sample = makeTool(player, "Seed Sample " .. sampleId, {
2484         itemType = "Transfer Seed Sample",
2485         sample = true,
2486         label = sampleId,
2487         colour = Color3.fromRGB(120, 190, 225),
2488         attributes = {
2489             TransferSample = true,
2490             SampleId = sampleId,
2491             SourceStorageId = storageId(source),
2492             DestinationStorageId = storageId(destination),
2493             DestinationLabel = targetLabel,
2494         },
2495     })
2496     if not sample then return end
2497     setCarryState(player, sample)
2498     setReadout("DISPATCHED " .. sampleId, "DEST: " .. targetLabel, Color3.fromRGB(100, 220, 255))
2499     notify(player, "Picked up " .. sampleId .. ". Carry it visibly to " .. targetLabel .. ".")
2500 end
2501
2502 local function handleColdStore(player, part, setReadout)
2503     local sample = findTransferSample(player)
2504     if not sample then
2505         createTransferSample(player, part, setReadout)
2506         return
2507     end
2508
2509     local sampleId = sample:GetAttribute("SampleId") or sample.Name
2510     local wanted = sample:GetAttribute("DestinationStorageId")
2511     if wanted ~= storageId(part) then
2512         local destination = sample:GetAttribute("DestinationLabel") or "the assigned cold store"
2513         setReadout("WRONG STORAGE", sampleId .. " -> " .. destination, Color3.fromRGB(255, 120, 95))
2514         notify(player, sampleId .. " belongs in " .. destination .. ". It remains in your hand.")
2515         equipTool(player, sample)
2516         return
2517     end
2518
2519     destroyCarriedItem(player, sample)
2520     GameState.transfersCompleted = GameState.transfersCompleted + 1
2521     addContribution(player)
2522     updateScoreboard()
2523     setReadout("RECEIVED " .. sampleId, "TRANSFER VERIFIED | +1 contribution", Color3.fromRGB(115, 245, 150))
2524     notify(player, sampleId .. " delivered and verified. Transfer complete.")
2525 end
2526

```

```

2527 local function startCycleIndicator(part)
2528     local auto = part:GetAttribute("Auto")
2529     if not auto then return nil, function() end end
2530     local indicator = Instance.new("Part")
2531     indicator.Name = "CycleIndicator"
2532     indicator.Size = (auto == "spin") and Vector3.new(0.9, 0.15, 0.9) or Vector3.new(0.5, 0.5, 0.5)
2533     indicator.Shape = (auto == "spin") and Enum.PartType.Block or Enum.PartType.Ball
2534     indicator.Anchored = true
2535     indicator.CanCollide = false
2536     indicator.CanTouch = false
2537     indicator.CastShadow = false
2538     indicator.Material = Enum.Material.Neon
2539     indicator.Color = Color3.fromRGB(255, 196, 60)
2540     indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
2541     indicator.Parent = part.Parent
2542     local light = Instance.new("PointLight")
2543     light.Color = indicator.Color
2544     light.Brightness = 1.5
2545     light.Range = 12
2546     light.Parent = indicator
2547
2548     local running = true
2549     task.spawn(function()
2550         local t = 0
2551         while running and indicator.Parent do
2552             t = t + task.wait(0.1)
2553             if auto == "spin" then
2554                 indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
2555                     * CFrame.Angles(0, t * 5, 0)
2556             else
2557                 indicator.Transparency = 0.15 + (math.sin(t * 7) + 1) / 2 * 0.55
2558             end
2559         end
2560     end)
2561     return indicator, function()
2562         running = false
2563         if indicator.Parent then indicator:Destroy() end
2564     end
2565 end
2566
2567 local function bind(part)
2568     local kind = part:GetAttribute("Kind")
2569     if not kind or part:GetAttribute("InteractionsBound") then return end
2570     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
2571     part:SetAttribute("InteractionsBound", true)
2572     part:SetAttribute("EquipmentId", equipmentId(part))
2573     part:SetAttribute("FloorLabel", equipmentFloorLabel(part))
2574     part:SetAttribute("RepairKey", "R")
2575     if part:GetAttribute("FunctionalState") == nil then
2576         local hash = 0
2577         for i = 1, #equipmentId(part) do hash = hash + string.byte(equipmentId(part), i) end
2578         local failed = false
2579         part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
2580         part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
2581         part:SetAttribute("FailureSince", failed and os.time() or 0)
2582     end
2583     if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time()) end
2584     if COLD_STORE[kind] then storageId(part) end
2585     if isPlantContainer then
2586         updateStoredSamples(part)
2587         refreshChamberTelemetry(part)
2588     end
2589     local spec = Kinds.forKind(kind)
2590     local prompt, setReadout, pressButton = makeControls(part, kind, spec)
2591     -- Cabinet trim can occlude Roblox's prompt ray to its own push button.
2592     -- The server still enforces distance and wall line-of-sight via canInteract.
2593     prompt.RequiresLineOfSight = false
2594     local baseColour = part.Color
2595     local busy = false
2596     local powerOn = true
2597     part:SetAttribute("Powered", true)
2598     local powerLight = Instance.new("PointLight")
2599     powerLight.Color = Color3.fromRGB(120, 220, 140)
2600     powerLight.Brightness = powerOn and 2 or 0
2601     powerLight.Range = 7
2602     powerLight.Parent = part

```

```

2603
2604     local led = Instance.new("Part")
2605     led.Name = "StatusLED"
2606     led.Shape = Enum.PartType.Ball
2607     led.Size = Vector3.new(0.22, 0.22, 0.22)
2608     led.Anchored = true
2609     led.CanCollide = false
2610     led.CanTouch = false
2611     led.CastShadow = false
2612     led.Material = Enum.Material.Neon
2613     led.Color = powerOn and Color3.fromRGB(110, 230, 130) or Color3.fromRGB(64, 68, 74)
2614     local controlSide = part:GetAttribute("WallSide")
2615     if controlSide == "W" or controlSide == "E" then
2616         local direction = controlSide == "W" and 1 or -1
2617         led.CFrame = part.CFrame * CFrame.new(direction * (part.Size.X / 2 + 0.08), part.Size.Y / 2 - 0.3,
↪      part.Size.Z * 0.32)
2618     else
2619         local direction = controlSide == "S" and -1 or 1
2620         led.CFrame = part.CFrame * CFrame.new(part.Size.X * 0.32, part.Size.Y / 2 - 0.3, direction *
↪      (part.Size.Z / 2 + 0.08))
2621     end
2622     led.Parent = part
2623
2624     local repairAnchor = part:FindFirstChild("ControllerScreen") or led
2625     local floorLabel = equipmentFloorLabel(part)
2626     local unitLabel = equipmentLabel(part)
2627     local repairPrompt = Instance.new("ProximityPrompt")
2628     repairPrompt.Name = "RepairPrompt"
2629     repairPrompt.ObjectText = floorLabel .. " • " .. unitLabel
2630     repairPrompt.ActionText = "Hold R to repair"
2631     repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
2632     repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
2633     repairPrompt.HoldDuration = 3
2634     repairPrompt.MaxActivationDistance = 8
2635     repairPrompt.RequiresLineOfSight = false
2636     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
2637     repairPrompt.Parent = repairAnchor
2638
2639     part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
2640         if repairPrompt.Parent then
2641             repairPrompt.Enabled = not busy and part:GetAttribute("FunctionalState") == "FAILED"
2642         end
2643     end)
2644
2645     repairPrompt.Triggered:Connect(function(player)
2646         if playerRole(player) ~= "Technician" then
2647             setReadout("TECHNICIAN REQUIRED", "Choose the BLUE role to repair equipment", Color3.fromRGB(255,
↪      175, 80))
2648             notify(player, "Only a Technician can repair equipment. Choose the BLUE role kiosk.")
2649             return
2650         end
2651         if busy or part:GetAttribute("FunctionalState") ~= "FAILED"
2652             or not canInteract(player, part, repairPrompt.MaxActivationDistance) then return end
2653         busy = true
2654         prompt.Enabled = false
2655         repairPrompt.Enabled = false
2656         setReadout("REPAIRING", floorLabel .. " • " .. unitLabel .. " • diagnosing "
2657             .. (part:GetAttribute("FailureReason") or "equipment fault"), Color3.fromRGB(255, 205, 75))
2658         setReadout("REPAIRING", "Working on " .. floorLabel .. " • " .. unitLabel, Color3.fromRGB(120, 235,
↪      170))
2659         local function canFinishRepair()
2660             return part.Parent ~= nil and playerRole(player) == "Technician"
2661                 and part:GetAttribute("FunctionalState") == "FAILED"
2662                 and canInteract(player, part, repairPrompt.MaxActivationDistance)
2663         end
2664         if not runMathQuiz(player, "REPAIR " .. unitLabel, part, canFinishRepair) then
2665             setReadout("REPAIR NOT COMPLETED", "Hold R for 3 seconds to try another quiz", Color3.fromRGB(255,
↪      105, 90))
2666             prompt.Enabled = true
2667             repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
2668             busy = false
2669             return
2670         end
2671         local assigned = player:GetAttribute("AssignedRepairId") == equipmentId(part)
2672         local points, failedSeconds, routeDistance = repairReward(player, part, assigned)
2673         part:SetAttribute("LastRepairedAt", os.time())

```



```

2674     part:SetAttribute("LastRepairedBy", player.Name)
2675     if kind == PLANT_CHAMBER_KIND then setChamberFailure(part, false) else setGenericFailure(part, false)
2676     ⇨ end
2677     GameState.repairsCompleted = GameState.repairsCompleted + 1
2678     if assigned then GameState.assignedRepairsCompleted = GameState.assignedRepairsCompleted + 1 end
2679     local total = awardPoints(player, points, string.format("%s%s • failed %ds • route %d studs",
2680         assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
2681         ⇨ math.floor(routeDistance + 0.5)))
2682     setReadout("REPAIR COMPLETE +" .. points, floorLabel .. " • " .. unitLabel .. " WORKING • TOTAL " ..
2683         ⇨ total,
2684         Color3.fromRGB(105, 245, 155))
2685     assignNextTask(player)
2686     prompt.Enabled = true
2687     busy = false
2688 end)
2689
2690 prompt.Triggered:Connect(function(player)
2691     if busy or not canInteract(player, part, prompt.MaxActivationDistance) then return end
2692     task.spawn(pressButton)
2693     if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
2694         setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
2695             ⇨ (part:GetAttribute("FailureReason") or "equipment fault"),
2696             Color3.fromRGB(255, 105, 90))
2697         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
2698         return
2699     end
2700
2701     local station = ROLE_STATIONS[kind]
2702     if station then
2703         if not player.Team or player.Team.Name ~= station.role then
2704             setReadout("ACCESS DENIED", station.role .. " role required", Color3.fromRGB(255, 105, 90))
2705             notify(player, kind .. ": only a " .. station.role .. " can use this. Join that role at the
2706                 ⇨ entrance kiosk.")
2707             return
2708         end
2709         busy = true
2710         prompt.Enabled = false
2711         runProgress(spec.wait or 1, "Reading shared batch queue", setReadout)
2712         local msg, err = station.run(GameState)
2713         if err then
2714             setReadout("NO BATCH DATA", err, Color3.fromRGB(255, 160, 90))
2715             notify(player, kind .. ": " .. err)
2716         else
2717             addContribution(player)
2718             setReadout("INSPECTION DATA", msg, Color3.fromRGB(115, 245, 150))
2719             notify(player, kind .. " -- " .. msg)
2720             updateScoreboard()
2721         end
2722         prompt.Enabled = true
2723         busy = false
2724         return
2725     end
2726
2727     local plantProcess = PLANT_PROCESS[kind]
2728     local carriedPlant = findPlantSample(player)
2729     local shouldUsePlantStorage = isPlantContainer and (
2730         kind == PLANT_CHAMBER_KIND
2731         or not plantProcess
2732         or not carriedPlant
2733         or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
2734     )
2735     if shouldUsePlantStorage then
2736         busy = true
2737         prompt.Enabled = false
2738         handleGrowthChamber(player, part, setReadout)
2739         prompt.Enabled = true
2740         busy = false
2741         return
2742     end
2743
2744     if part:GetAttribute("FunctionalState") == "FAILED" then
2745         setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
2746             ⇨ (part:GetAttribute("FailureReason") or "equipment fault"),
2747             Color3.fromRGB(255, 105, 90))
2748         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
2749         return
2750     end

```

```

2744     end
2745
2746     if plantProcess then
2747         busy = true
2748         prompt.Enabled = false
2749         processPlant(player, part, spec, plantProcess, setReadout)
2750         prompt.Enabled = true
2751         busy = false
2752         return
2753     end
2754
2755     if COLD_STORE[kind] then
2756         busy = true
2757         prompt.Enabled = false
2758         runProgress(0.6, "Scanning cold-store inventory", setReadout)
2759         handleColdStore(player, part, setReadout)
2760         prompt.Enabled = true
2761         busy = false
2762         return
2763     end
2764
2765     local consumed
2766     if spec.consumes then
2767         consumed = findTool(player, spec.consumes)
2768         if not consumed then
2769             setReadout("MATERIAL NEEDED", spec.consumes, Color3.fromRGB(255, 160, 90))
2770             notify(player, kind .. ": needs " .. spec.consumes .. " first.")
2771             return
2772         end
2773     end
2774
2775     busy = true
2776     prompt.Enabled = false
2777     if spec.activeColour then part.Color = spec.activeColour end
2778     local uv
2779     if spec.uvLight then
2780         uv = Instance.new("PointLight")
2781         uv.Color = Color3.fromRGB(150, 210, 255)
2782         uv.Brightness = 2
2783         uv.Range = 26
2784         uv.Parent = part
2785     end
2786
2787     local _, stopIndicator = startCycleIndicator(part)
2788     runProgress(spec.wait or 1, spec.action, setReadout)
2789     stopIndicator()
2790
2791     if consumed then destroyCarriedItem(player, consumed) end
2792     for _, item in ipairs(spec.produces or {}) do
2793         local made = makeTool(player, item)
2794         if made and made:GetAttribute("LabSample") then setCarryState(player, made) end
2795     end
2796     local result = spec.log and spec.log() or "Inspection complete; equipment left ready."
2797     setReadout("INSPECTION DATA", result, Color3.fromRGB(115, 245, 150))
2798     notify(player, kind .. " -- " .. result)
2799
2800     if uv then uv:Destroy() end
2801     part.Color = baseColour
2802     prompt.Enabled = true
2803     busy = false
2804 end)
2805 end
2806
2807 -----
2808 -- FIFTY AUTONOMOUS GAMEPLAY TEST WORKERS
2809 -----
2810 local function setBotStatus(bot, message)
2811     bot:SetAttribute("TaskState", message)
2812     local status = bot:FindFirstChild("Status", true)
2813     if status and status:IsA("TextLabel") then
2814         status.Text = (bot:GetAttribute("DisplayName") or bot.Name) .. "\n" .. message
2815     end
2816 end
2817
2818 local function updateBotTestBoard()
2819     local workers = CollectionService:GetTagged("LabWorkerBot")

```

```

2820     local cycles, carrying, fallbacks, rivalScore = 0, 0, 0, 0
2821     for _, bot in ipairs(workers) do
2822         cycles = cycles + (bot:GetAttribute("CompletedCycles") or 0)
2823         fallbacks = fallbacks + (bot:GetAttribute("PathFallbacks") or 0)
2824         if (bot:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
2825         if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 end
2826     end
2827     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
2828     local failing, failedSamples = 0, 0
2829     for _, chamber in ipairs(chambers) do
2830         if chamber:GetAttribute("FunctionalState") == "FAILED" then
2831             failing = failing + 1
2832             failedSamples = failedSamples + (chamber:GetAttribute("SampleCount") or 0)
2833         end
2834     end
2835     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
2836         for _, label in ipairs(board:GetDescendants()) do
2837             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
2838                 label.Text = string.format(
2839                     "ADAPTIVE GROWTH-ROOM TEST\nWorkers: %d/50  Transfers: %d  Carrying: %d\nFAILED: %d/%d
↳ (%.0f%%)  Samples waiting: %d\nRIVAL SCORE: %d  •  Path fallbacks: %d",
2840                     #workers, cycles, carrying, failing, #chambers, failing / math.max(1, #chambers) * 100,
↳ failedSamples, rivalScore, fallbacks)
2841             end
2842         end
2843     end
2844 end
2845
2846 local function interactionPoint(part)
2847     local side = part:GetAttribute("WallSide")
2848     local bottomY = part.Position.Y - part.Size.Y / 2
2849     local point = Vector3.new(part.Position.X, bottomY + 3.25, part.Position.Z)
2850     if side == "W" then point = point + Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
2851     elseif side == "E" then point = point - Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
2852     elseif side == "N" then point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
2853     elseif side == "S" then point = point - Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
2854     else point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5) end
2855     return point
2856 end
2857
2858 -----
2859 -- OBSERVER GATE
2860 -- A worker only runs its scenario while a player is in the room with it.
2861 -- Everything else stands still. This is what keeps 180 NPCs off the
2862 -- pathfinding budget: an unobserved bot costs one cheap check a second
2863 -- instead of a continuous walk loop and a PathfindingService computation
2864 -- every few seconds.
2865 -----
2866 local FLOOR_BAND = 10 -- studs, about half a floor-to-floor height
2867 local WAKE_RADIUS = 35 -- studs, fallback for corridors and doorways, where
2868 -- neither the bot nor the player is inside any room
2869
2870 -- Room rectangles published by the generator as one packed attribute:
2871 -- "Level_1:Room 1:x1:z1:x2:z2:y;..." already in stud space.
2872 local roomsByLevel = {}
2873 do
2874     local complex = workspace:FindFirstChild("Laboratory_Complex")
2875     local packed = complex and complex:GetAttribute("RoomIndex")
2876     if typeof(packed) == "string" then
2877         for entry in packed:gmatch("[^;]+") do
2878             local level, name, x1, z1, x2, z2, y =
2879                 entry:match("^([^:]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+)$")
2880             if level then
2881                 roomsByLevel[level] = roomsByLevel[level] or {}
2882                 table.insert(roomsByLevel[level], {
2883                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
2884                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
2885                 })
2886             end
2887         end
2888     end
2889 end
2890
2891 -- Only the ~10 rooms on the bot's own floor are ever tested, not all 81.
2892 local function roomAt(levelName, position)
2893     position=LabSpatial.localPoint(position)

```

```

2894     local list = levelName and roomsByLevel[levelName]
2895     if not list then return nil end
2896     for _, r in ipairs(list) do
2897         if math.abs(position.Y - r.y) < FLOOR_BAND
2898             and position.X >= r.x1 and position.X <= r.x2
2899             and position.Z >= r.z1 and position.Z <= r.z2 then
2900             return r.name
2901         end
2902     end
2903     return nil
2904 end
2905
2906 -- The rectangle itself, for picking a patrol point inside a room.
2907 local function roomRect(levelName, roomName)
2908     for _, r in ipairs((levelName and roomsByLevel[levelName]) or {}) do
2909         if r.name == roomName then return r end
2910     end
2911     return nil
2912 end
2913
2914 local function botHasObserver(bot)
2915     local ok, pivot = pcall(function() return bot:GetPivot().Position end)
2916     if not ok then return false end
2917     local levelName = bot:GetAttribute("Level")
2918     local botRoom = roomAt(levelName, pivot)
2919     for _, player in ipairs(Players:GetPlayers()) do
2920         local character = player.Character
2921         local hrp = character and character:FindFirstChild("HumanoidRootPart")
2922         if hrp then
2923             local here = hrp.Position
2924             -- Cheapest test first: wrong floor is an instant no.
2925             if math.abs(here.Y - pivot.Y) < FLOOR_BAND then
2926                 if botRoom and roomAt(levelName, here) == botRoom then return true end
2927                 local dx, dz = here.X - pivot.X, here.Z - pivot.Z
2928                 if dx * dx + dz * dz < WAKE_RADIUS * WAKE_RADIUS then return true end
2929             end
2930         end
2931     end
2932     return false
2933 end
2934
2935 local function botMotionChecks(bot)
2936     local excluded = {bot}
2937     local corpses = workspace:FindFirstChild("LabBotBodies")
2938     if corpses then table.insert(excluded, corpses) end
2939     local cast = RaycastParams.new()
2940     cast.FilterType = Enum.RaycastFilterType.Exclude
2941     cast.FilterDescendantsInstances = excluded
2942     cast.RespectCanCollide = true
2943     local overlap = OverlapParams.new()
2944     overlap.FilterType = Enum.RaycastFilterType.Exclude
2945     overlap.FilterDescendantsInstances = excluded
2946     overlap.RespectCanCollide = true
2947     overlap.MaxParts = 1
2948     -- Square horizontal footprint encloses the arms at any yaw. The small
2949     -- floor clearance avoids treating a floor seam as a wall.
2950     local size = Vector3.new(3.4, 5.4, 3.4)
2951     local function clear(from, to)
2952         local delta = to - from
2953         for _, player in ipairs(Players:GetPlayers()) do
2954             local character = player.Character
2955             local hrp = character and character:FindFirstChild("HumanoidRootPart")
2956             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
2957             if hrp and humanoid and humanoid.Health > 0 and math.abs(hrp.Position.Y - from.Y) < 5 then
2958                 local flatDelta = Vector3.new(delta.X, 0, delta.Z)
2959                 local relative = Vector3.new(hrp.Position.X - from.X, 0, hrp.Position.Z - from.Z)
2960                 local alpha = flatDelta.Magnitude > 0.01 and math.clamp(relative:Dot(flatDelta) /
2961                     ⇨ flatDelta:Dot(flatDelta), 0, 1) or 0
2962                 local movingAway = relative.Magnitude < 3.4 and alpha < 0.001
2963                     and (relative - flatDelta).Magnitude > relative.Magnitude + 0.01
2964                 if (relative - flatDelta * alpha).Magnitude < 3.4 and not movingAway then
2965                     return false, hrp.Position
2966                 end
2967             end
2968         end
2969         local offset = Vector3.new(0, -0.2, 0)

```

```

2969     if delta.Magnitude > 0.01 then
2970         local hit = workspace.Blockcast(CFrame.new(from + offset), size, delta, cast)
2971         if hit then return false, hit.Instance.Position end
2972     end
2973     -- Blockcast does not cover pre-existing overlaps; test the destination
2974     -- too, so a door closing or another worker arriving cannot be skipped.
2975     local occupied = workspace.GetPartBoundsInBox(CFrame.new(to + offset), size, overlap)
2976     if occupied[1] then return false, occupied[1].Position end
2977     return true
2978 end
2979 local function grounded(point, currentY)
2980     local floor = workspace.Raycast(Vector3.new(point.X, currentY + 1, point.Z), Vector3.new(0, -7, 0),
2981     ↪ cast)
2982     if not floor or floor.Normal.Y < 0.65 then return nil end
2983     local y = floor.Position.Y + 3.25
2984     if math.abs(y - currentY) > 1.25 then return nil end
2985     return Vector3.new(point.X, y, point.Z)
2986 end
2987 return clear, grounded
2988 end
2989 local function walkBotSegment(bot, destination, allowDetour)
2990     if not bot.Parent or not bot.PrimaryPart or bot:GetAttribute("Dead") then return false end
2991     local clear, grounded = botMotionChecks(bot)
2992     local start = bot:GetPivot().Position
2993     local goal = Vector3.new(destination.X, start.Y, destination.Z)
2994     local direction = goal - start
2995     if direction.Magnitude < 0.15 then return true end
2996     local free, obstruction = clear(start, goal)
2997     if not free then
2998         -- Dynamic obstacles are not reliably included in Roblox's navmesh.
2999         -- Try both sides, with two corner points around the person/obstacle.
3000         -- Each leg still gets the same full-body collision tests below.
3001         if allowDetour ~= false and obstruction then
3002             local forward = direction.Unit
3003             local side = Vector3.new(-forward.Z, 0, forward.X)
3004             local centre = Vector3.new(obstruction.X, start.Y, obstruction.Z)
3005             local preferred = (bot:GetAttribute("BotNumber") or 1) % 2 == 0 and 1 or -1
3006             for _, sign in ipairs({preferred, -preferred}) do
3007                 local entry = grounded(centre - forward * 4.5 + side * sign * 4.5, start.Y)
3008                 local exitPoint = grounded(centre + forward * 4.5 + side * sign * 4.5, start.Y)
3009                 if entry and exitPoint and clear(start, entry) and clear(entry, exitPoint) and clear(exitPoint,
3010                 ↪ goal) then
3011                     bot:SetAttribute("AvoidanceDetours", (bot:GetAttribute("AvoidanceDetours") or 0) + 1)
3012                     return walkBotSegment(bot, entry, false) and walkBotSegment(bot, exitPoint, false) and
3013                     ↪ walkBotSegment(bot, destination, false)
3014                 end
3015             end
3016             end
3017             bot:SetAttribute("NavigationBlocked", true)
3018             return false
3019         end
3020         bot:SetAttribute("NavigationBlocked", false)
3021         local steps = math.max(1, math.ceil(direction.Magnitude / 0.45))
3022         for step = 1, steps do
3023             while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do task.wait(0.25)
3024             ↪ end
3025             while bot.Parent and not bot:GetAttribute("Dead") and not bot:GetAttribute("Competitor") and not
3026             ↪ botHasObserver(bot) do task.wait(0.5) end
3027             if not bot.Parent or bot:GetAttribute("Dead") then return false end
3028             local current = bot:GetPivot().Position
3029             local nextPoint = grounded(start:Lerp(goal, step / steps), current.Y)
3030             -- A player may have stepped into the path since it was planned. Stop
3031             -- before touching them; the caller replans from this actual position.
3032             if not nextPoint or not clear(current, nextPoint) then
3033                 bot:SetAttribute("NavigationBlocked", true)
3034                 return false
3035             end
3036             bot:PivotTo(CFrame.lookAt(nextPoint, nextPoint + direction.Unit))
3037             task.wait(0.06)
3038         end
3039         return true
3040     end
3041 end
3042 local function moveBotTo(bot, target)
3043     local destination = interactionPoint(target)

```

```

3040 -- A blocked leg is replanned, never replaced with a teleport/straight
3041 -- line through a wall. Radius matches the swept body, not just its torso.
3042 for attempt = 1, 3 do
3043     if not bot.Parent or bot:GetAttribute("Dead") then return false end
3044     local path = PathfindingService.CreatePath({AgentRadius = 1.7, AgentHeight = 5.8, AgentCanJump = false,
        ↳ WaypointSpacing = 5})
3045     local ok = pcall(function() path:ComputeAsync(bot:GetPivot().Position, destination) end)
3046     local arrived = ok and path.Status == Enum.PathStatus.Success
3047     if arrived then
3048         for _, waypoint in ipairs(path:GetWaypoints()) do
3049             if not walkBotSegment(bot, waypoint.Position + Vector3.new(0, 3.25, 0)) then arrived = false
                ↳ break end
            end
        end
3051     end
3052     if arrived then return true end
3053     bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
3054     task.wait(0.25 * attempt)
3055 end
3056 return false
3057 end
3058
3059 local function makeBotPlant(bot, stage)
3060     local old = bot:FindFirstChild("BotHeldPlant")
3061     if old then old:Destroy() end
3062     local held = Instance.new("Model")
3063     held.Name = "BotHeldPlant"
3064     held.Parent = bot
3065     local base = bot:GetPivot() * CFrame.new(0.85, 0.1, -0.72)
3066     local function piece(name, size, offset, colour, material)
3067         local p = Instance.new("Part")
3068         p.Name = name
3069         p.Size = size
3070         p.CFrame = base * CFrame.new(offset)
3071         p.Color = colour
3072         p.Material = material or Enum.Material.SmoothPlastic
3073         p.Anchored = true
3074         p.CanCollide = false
3075         p.CanTouch = false
3076         p.CanQuery = false
3077         p.Parent = held
3078         return p
3079     end
3080     piece("Pot", Vector3.new(0.85, 0.62, 0.85), Vector3.new(0, 0, 0), Color3.fromRGB(118, 77, 46))
3081     piece("Stem", Vector3.new(0.13, 0.75 + stage * 0.13, 0.13), Vector3.new(0, 0.65, 0), Color3.fromRGB(65,
        ↳ 165, 72))
3082     piece("LeafL", Vector3.new(0.48, 0.12, 0.3), Vector3.new(-0.24, 0.78, 0), Color3.fromRGB(75, 185, 82))
3083     piece("LeafR", Vector3.new(0.48, 0.12, 0.3), Vector3.new(0.24, 1.0, 0), Color3.fromRGB(75, 185, 82))
3084     return held
3085 end
3086
3087 local function equipmentReadout(part, top, bottom, colour)
3088     local label = part:FindFirstChild("Readout", true)
3089     if label and label:IsA("TextLabel") then
3090         label.Text = top .. "\n" .. bottom
3091         label.TextColor3 = colour or Color3.fromRGB(115, 245, 150)
3092     end
3093 end
3094
3095 local function candidatesOnLevel(levelName, predicate)
3096     local result = {}
3097     for _, item in ipairs(CollectionService.GetTagged("LabEquipment")) do
3098         if item.Parent and item:GetAttribute("Level") == levelName and predicate(item) then
3099             table.insert(result, item)
3100         end
3101     end
3102     return result
3103 end
3104
3105 local function runBotScenario(bot)
3106     -- Remember where this worker started, so a shift reset can stand it back up.
3107     if not bot:GetAttribute("HomeCFrame") then
3108         pcall(function() bot:SetAttribute("HomeCFrame", bot:GetPivot()) end)
3109     end
3110     -- Desk staff stay at their desk. They are still perfectly good targets.
3111     if bot:GetAttribute("Seated") then
3112         setBotStatus(bot, "At the desk")

```

```

3113         while bot.Parent and not bot:GetAttribute("Dead") do task.wait(5) end
3114         return
3115     end
3116
3117     while bot.Parent and not bot:GetAttribute("Dead") do
3118         -- The rival keeps running wherever it is: it is a single NPC and the
3119         -- competitive scoring loop depends on it making progress.
3120         local awake = bot:GetAttribute("Competitor") or botHasObserver(bot)
3121         if not awake then
3122             setBotStatus(bot, "Idle -- no one in the room")
3123             task.wait(1)
3124         elseif bot:GetAttribute("KnockedOut") then task.wait(1) else
3125             local levelName = bot:GetAttribute("Level")
3126             local failed = candidatesOnLevel(levelName, function(item)
3127                 return item:GetAttribute("Kind") == PLANT_CHAMBER_KIND
3128                     and item:GetAttribute("FunctionalState") == "FAILED"
3129                     and (item:GetAttribute("SampleCount") or 0) > 0
3130                     and not item:GetAttribute("ReservedByBot")
3131             end)
3132             if #failed == 0 then
3133                 -- Nothing has failed on this floor right now. The adaptive
3134                 -- difficulty tick deliberately holds the failure rate down when
3135                 -- few players are online, so this is the common case rather than
3136                 -- the rare one -- and standing frozen in front of a player who
3137                 -- just walked in is exactly what "the NPCs don't move" looked
3138                 -- like. Patrol the room instead so the floor reads as staffed.
3139                 -- Only ever runs for a bot somebody is actually watching.
3140                 setBotStatus(bot, "Checking chambers")
3141                 local patrolRoom = roomAt(levelName, bot:GetPivot().Position)
3142                 local rect = patrolRoom and roomRect(levelName, patrolRoom)
3143                 if rect then
3144                     local here = LabSpatial.localPoint(bot:GetPivot().Position)
3145                     local spanX = math.max(1, (rect.x2 - rect.x1) - 12)
3146                     local spanZ = math.max(1, (rect.z2 - rect.z1) - 12)
3147                     walkBotSegment(bot, LabSpatial.world(Vector3.new(
3148                         rect.x1 + 6 + math.random() * spanX,
3149                         here.Y,
3150                         rect.z1 + 6 + math.random() * spanZ)))
3151                     task.wait(0.5 + math.random() * 1.5)
3152                 else
3153                     task.wait(3)
3154                 end
3155             else
3156                 table.sort(failed, function(a, b)
3157                     local an = tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
3158                     local bn = tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
3159                     return an < bn
3160                 end)
3161                 -- Prefer a job in the room the worker is already standing in --
3162                 -- which, because it only wakes when a player is there, is the
3163                 -- room the player is watching. Without this a woken worker
3164                 -- usually picks a chamber elsewhere on the floor, walks out of
3165                 -- the room and stops the moment it loses its observer.
3166                 local botRoom = roomAt(levelName, bot:GetPivot().Position)
3167                 local sameRoom = {}
3168                 if botRoom then
3169                     for _, item in ipairs(failed) do
3170                         if item:GetAttribute("Room") == botRoom then table.insert(sameRoom, item) end
3171                     end
3172                 end
3173                 local pool = (#sameRoom > 0) and sameRoom or failed
3174                 local source = bot:GetAttribute("Competitor") and pool[1] or pool[math.random(1, #pool)]
3175                 local destination = evacuationDestinations(source)[1]
3176                 if not destination then
3177                     setBotStatus(bot, "Waiting for safe capacity")
3178                     task.wait(3)
3179                 else
3180                     local botId = bot.Name
3181                     source:SetAttribute("ReservedByBot", botId)
3182                     destination:SetAttribute("IncomingReservations",
3183                         ↳ (destination:GetAttribute("IncomingReservations") or 0) + 1)
3184                     source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode") or "")
3185                     refreshChamberTelemetry(source)
3186                     setBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or "alarm"))
3187                     local reachedSource = moveBotTo(bot, source)
3188                     local samples = chamberSampleList(source)

```

```

3188     local plantId = reachedSource and table.remove(samples, 1)
3189     if plantId and source:GetAttribute("ReservedByBot") == botId then
3190         saveChamberSamples(source, samples)
3191         updateStoredSamples(source)
3192         refreshChamberTelemetry(source)
3193         local stage = math.clamp(source:GetAttribute("GrowthStage") or 0, 0, 5)
3194         makeBotPlant(bot, stage)
3195         bot:SetAttribute("CarryingPlant", plantId)
3196         setBotStatus(bot, "Evacuating " .. plantId)
3197         equipmentReadout(source, (source:GetAttribute("GrowthCode") or "R?") .. " FAILED",
            ↪ "SEND " .. plantId .. " TO " .. (destination:GetAttribute("GrowthCode") or "R?"),
            ↪ Color3.fromRGB(255, 105, 90))

3198     -- Emergency transfers still receive one quick development step,
3199     -- so the same growth mechanics are continuously regression-tested.
3200     local stations = candidatesOnLevel(levelName, function(item) return
3201         ↪ PLANT_PROCESS[item:GetAttribute("Kind")] ~= nil end)
3202     if #stations > 0 then
3203         local station = stations[((bot:GetAttribute("BotNumber") or 1) - 1) % #stations +
            ↪ 1]
3204         local process = PLANT_PROCESS[station:GetAttribute("Kind")]
3205         if moveBotTo(bot, station) then
3206             stage = math.clamp(stage + (process.gain or 0), 0, 5)
3207             station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or 0)
            ↪ + 1)
3208             equipmentReadout(station, "BOT GROWTH TEST", plantId .. " • stage " .. stage,
            ↪ Color3.fromRGB(120, 235, 165))
3209         end
3210     end

3211     setBotStatus(bot, "Delivering to " .. (destination:GetAttribute("GrowthCode") or "R?"))
3212     local reachedDestination = moveBotTo(bot, destination)
3213     local destinationSamples = chamberSampleList(destination)
3214     if reachedDestination and destination:GetAttribute("FunctionalState") ~= "FAILED" and
3215         ↪ destination:GetAttribute("FunctionalState") ~= "DESTROYED" and #destinationSamples
3216         ↪ < (destination:GetAttribute("Capacity") or 10) then
3217         table.insert(destinationSamples, plantId)
3218         saveChamberSamples(destination, destinationSamples)
3219         destination:SetAttribute("GrowthStage",
            ↪ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
3220         updateStoredSamples(destination)
3221         refreshChamberTelemetry(destination)
3222         emergencyMovesCompleted = emergencyMovesCompleted + 1
3223         bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or 0) + 1)
3224         if bot:GetAttribute("Competitor") then
3225             local gained = POINTS.MOVE_ASSIGNED_ON_TIME
3226             bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) + gained)
3227             setBotStatus(bot, "+" .. gained .. " resource win")
3228             updateScoreboard()
3229         else
3230             setBotStatus(bot, "Transfer passed")
3231         end
3232     else
3233         -- Capacity changed while walking; return the sample to the source
3234         -- rather than deleting it and let the next cycle choose again.
3235         table.insert(samples, plantId)
3236         saveChamberSamples(source, samples)
3237         updateStoredSamples(source)
3238         refreshChamberTelemetry(source)
3239         setBotStatus(bot, "Destination changed; retrying")
3240     end
3241     local held = bot:FindFirstChild("BotHeldPlant")
3242     if held then held:Destroy() end
3243     bot:SetAttribute("CarryingPlant", "")
3244     end
3245     destination:SetAttribute("IncomingReservations", math.max(0,
        ↪ (destination:GetAttribute("IncomingReservations") or 1) - 1))
3246     if source:GetAttribute("ReservedByBot") == botId then source:SetAttribute("ReservedByBot",
        ↪ nil) end
3247     updateBotTestBoard()
3248     task.wait(1.5 + math.random())
3249 end
3250 end
3251 end
3252 end

```



```

3253
3254 -----
3255 -- ADAPTIVE, INDEFINITE CHAMBER FAILURE SIMULATION
3256 -----
3257 local FAILURE_REASONS = {
3258     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
3259     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
3260 }
3261 local generatedSampleSerial = 20000
3262 local previousCompletedMoves = 0
3263
3264 setChamberFailure = function(chamber, failed)
3265     if not chamber.Parent or chamber:GetAttribute("FunctionalState") == "DESTROYED" then return end
3266     if chamber:GetAttribute("QuizPlayerId") then return end
3267     if failed and ((chamber:GetAttribute("IncomingReservations") or 0) > 0
3268         or chamber:GetAttribute("ReservedByBot")) then return end
3269     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
3270     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
3271     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
3272     if failed then
3273         chamber:SetAttribute("FunctionalState", "FAILED")
3274         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
3275         chamber:SetAttribute("RepairKey", "R")
3276         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " • HOLD R
3277             ↳ TO REPAIR")
3278         chamber:SetAttribute("FailureReason", FAILURE_REASONS[math.random(1, #FAILURE_REASONS)])
3279         chamber:SetAttribute("FailureSince", os.time())
3280         chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
3281         chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
3282         chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
3283         local destination = evacuationDestinations(chamber)[1]
3284         chamber:SetAttribute("EvacuationDestination", destination and destination:GetAttribute("GrowthCode") or
3285             ↳ "")
3286         for _, player in ipairs(Players:GetPlayers()) do
3287             if playerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
3288                 ↳ assignNextTask(player) end
3289             if playerRole(player) == "Laboratory Technician" and not
3290                 ↳ player:GetAttribute("AssignedTransferSource") then assignNextTask(player) end
3291             if playerRole(player) == "Laboratory Technician" then
3292                 notify(player, "PLANT ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Press E to
3293                     ↳ remove plants and follow the assigned destination.")
3294             else
3295                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Technician:
3296                     ↳ HOLD R to repair.")
3297             end
3298         end
3299     else
3300         chamber:SetAttribute("FunctionalState", "WORKING")
3301         chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
3302         chamber:SetAttribute("FailureReason", "")
3303         chamber:SetAttribute("FailureSince", 0)
3304         chamber:SetAttribute("EvacuationDestination", "")
3305         chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
3306         chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
3307         chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
3308         chamber:SetAttribute("OperationStarted", os.time())
3309         if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) < 100
3310             ↳ then
3311             recoverPlantLife(chamber, chamber:GetAttribute("PlantLife"), nil, chamber:GetAttribute("PlantId"))
3312         end
3313     end
3314     refreshChamberTelemetry(chamber)
3315 end
3316
3317 local GENERIC_FAILURE_REASONS = {
3318     "Power relay fault", "Controller communication loss", "Pump interlock fault",
3319     "Calibration expired", "Safety sensor trip", "Motor overload",
3320 }
3321
3322 setGenericFailure = function(part, failed)
3323     if not part.Parent or part:GetAttribute("FunctionalState") == "DESTROYED" then return end
3324     if part:GetAttribute("QuizPlayerId") then return end
3325     if failed and (part:GetAttribute("IncomingReservations") or 0) > 0 then return end
3326     local floorLabel = equipmentFloorLabel(part)
3327     local unitLabel = equipmentLabel(part)
3328     if failed then

```

```

3322     part:SetAttribute("FunctionalState", "FAILED")
3323     part:SetAttribute("RepairKey", "R")
3324     part:SetAttribute("FailureReason", GENERIC_FAILURE_REASONS[math.random(1, #GENERIC_FAILURE_REASONS)])
3325     part:SetAttribute("FailureSince", os.time())
3326     part:SetAttribute("RepairInstruction", floorLabel .. " " .. unitLabel .. " " .. HOLD R")
3327     equipmentReadout(part, "FAILED • HOLD R", floorLabel .. " " .. unitLabel .. " " .. PLANTS "
3328         .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " " .. "
3329         ↪ .. part:GetAttribute("FailureReason"),
3330         Color3.fromRGB(255, 105, 90))
3331     for _, player in ipairs(Players:GetPlayers()) do
3332         if playerRole(player) == "Technician" then
3333             if not player:GetAttribute("AssignedRepairId") then assignNextTask(player) end
3334             notify(player, "REPAIR ALARM: " .. floorLabel .. " " .. unitLabel .. " FAILED. HOLD R to
3335                 ↪ repair.")
3336         end
3337     end
3338 else
3339     part:SetAttribute("FunctionalState", "WORKING")
3340     part:SetAttribute("FailureReason", "")
3341     part:SetAttribute("FailureSince", 0)
3342     part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
3343     part:SetAttribute("OperationStarted", os.time())
3344     equipmentReadout(part, "WORKING", floorLabel .. " " .. unitLabel .. " " .. PLANTS "
3345         .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " " .. E
3346         ↪ USE / INSPECT",
3347         Color3.fromRGB(110, 235, 170))
3348 end
3349 end
3350
3351 local function shuffled(list)
3352     for i = #list, 2, -1 do
3353         local j = math.random(1, i)
3354         list[i], list[j] = list[j], list[i]
3355     end
3356     return list
3357 end
3358
3359 local function adaptiveFailureTick()
3360     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
3361     local failed, working = {}, {}
3362     local failedSamples, totalSamples, workingFree = 0, 0, 0
3363     for _, chamber in ipairs(chambers) do
3364         local count = chamber:GetAttribute("SampleCount") or 0
3365         local capacity = chamber:GetAttribute("Capacity") or 10
3366         totalSamples = totalSamples + count
3367         if chamber:GetAttribute("FunctionalState") == "FAILED" then
3368             table.insert(failed, chamber)
3369             failedSamples = failedSamples + count
3370         elseif chamber:GetAttribute("FunctionalState") == "WORKING" then
3371             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 180 then
3372                 table.insert(working, chamber)
3373             end
3374             workingFree = workingFree + math.max(0, capacity - count -
3375                 ↪ (chamber:GetAttribute("IncomingReservations") or 0))
3376         end
3377     end
3378
3379     local throughput = emergencyMovesCompleted - previousCompletedMoves
3380     previousCompletedMoves = emergencyMovesCompleted
3381     -- A human-sized workload, independent of the hundreds of installed units.
3382     -- Two failed chambers per player (at most eight), one new alarm per tick.
3383     local playerCount = math.max(1, #Players:GetPlayers())
3384     local targetFailed = workingFree >= 12 and math.min(8, playerCount * 2) or 0
3385     local targetRate = targetFailed / math.max(1, #chambers)
3386     local change = math.min(1, targetFailed - #failed)
3387     if change > 0 then
3388         local eligible = {}
3389         for _, chamber in ipairs(working) do
3390             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 180
3391                 and (chamber:GetAttribute("IncomingReservations") or 0) == 0
3392                 and not chamber:GetAttribute("ReservedByBot") then table.insert(eligible, chamber) end
3393         end
3394         shuffled(eligible)
3395         for i = 1, math.min(change, #eligible) do setChamberFailure(eligible[i], true) end
3396     elseif change < 0 then
3397         table.sort(failed, function(a, b)

```

```

3394         local ac, bc = a:GetAttribute("SampleCount") or 0, b:GetAttribute("SampleCount") or 0
3395         if ac == bc then return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or
3396             ↪ 0) end
3397         return ac < bc
3398     end)
3399     for i = 1, math.min(-change, #failed) do setChamberFailure(failed[i], false) end
3400 end
3401 -- Failed telemetry continues to drift; working chambers continuously
3402 -- germinate replacement samples so the transfer game never runs out.
3403 for _, chamber in ipairs(chambers) do
3404     if chamber:GetAttribute("FunctionalState") == "FAILED" then
3405         chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
3406             ↪ math.random(-2, 2), 28, 42))
3407         chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
3408             ↪ math.random(-4, 4), 25, 65))
3409         chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
3410             ↪ math.random(-5, 3), 0, 48))
3411         local destination = evacuationDestinations(chamber)[1]
3412         chamber:SetAttribute("EvacuationDestination", destination and
3413             ↪ destination:GetAttribute("GrowthCode") or "")
3414     elseif chamber:GetAttribute("FunctionalState") == "WORKING" then
3415         local samples = chamberSampleList(chamber)
3416         local capacity = chamber:GetAttribute("Capacity") or 10
3417         local refillChance = math.clamp(0.08 + (capacity - #samples) * 0.025 + throughput * 0.002
3418             ↪ + playerCount * 0.008, 0.08, 0.46)
3419         -- Keep receiving space available for real evacuation tasks.
3420         if #samples < capacity - math.max(2, math.ceil(capacity * 0.3)) and math.random() < refillChance
3421             ↪ then
3422             generatedSampleSerial = generatedSampleSerial + 1
3423             table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
3424                 ↪ generatedSampleSerial))
3425             saveChamberSamples(chamber, samples)
3426             chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
3427             updateStoredSamples(chamber)
3428         end
3429     end
3430     refreshChamberTelemetry(chamber)
3431 end
3432 local complex = workspace:FindFirstChild("Laboratory_Complex")
3433 if complex then
3434     complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
3435     complex:SetAttribute("FailingChambers", #failed)
3436     complex:SetAttribute("FailedSampleBacklog", failedSamples)
3437     complex:SetAttribute("EmergencyMovesCompleted", emergencyMovesCompleted)
3438     complex:SetAttribute("RecentTransferThroughput", throughput)
3439     complex:SetAttribute("ActivePlayerCount", playerCount)
3440     complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples /
3441         ↪ 20), 8, 50))
3442     -- Per-FL00R budget. Multiplied across the basement plus every level
3443     -- this is what actually decides how many workers are visibly busy.
3444     complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
3445         ↪ math.floor(failedSamples / 60), 6, 20))
3446 end
3447 end
3448
3449 local function genericFailureTick()
3450     local failed, working = {}, {}
3451     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
3452         if equipment:GetAttribute("Kind") ~= PLANT_CHAMBER_KIND then
3453             if equipment:GetAttribute("FunctionalState") == "FAILED" then
3454                 table.insert(failed, equipment)
3455             elseif equipment:GetAttribute("FunctionalState") == "WORKING"
3456                 and os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 180 then
3457                 table.insert(working, equipment)
3458             end
3459         end
3460     end
3461     end
3462     end
3463     local playerCount = math.max(1, #Players:GetPlayers())
3464     local targetFailed = math.min(4, playerCount)
3465     local change = math.min(1, targetFailed - #failed)
3466     if change > 0 then
3467         shuffled(working)
3468         for i = 1, math.min(change, #working) do setGenericFailure(working[i], true) end
3469     elseif change < 0 then

```

```

3460         table.sort(failed, function(a, b) return (a:GetAttribute("FailureSince") or 0) <
↪      (b:GetAttribute("FailureSince") or 0) end)
3461     for i = 1, math.min(-change, #failed) do setGenericFailure(failed[i], false) end
3462 end
3463 end
3464
3465 local function bindPlayerTaskRouting(player)
3466     if player:GetAttribute("EquipmentTaskRoutingBound") then return end
3467     player:SetAttribute("EquipmentTaskRoutingBound", true)
3468     player:GetAttributeChangedSignal("LabRole"):Connect(function()
3469         task.delay(0.2, function()
3470             if player.Parent then assignNextTask(player) end
3471         end)
3472     end)
3473     player.CharacterAdded:Connect(function()
3474         if playerRole(player) == "Technician" and player:GetAttribute("AssignedRepairId") then
3475             task.delay(1, function() if player.Parent then assignNextTask(player) end end)
3476         end
3477     end)
3478 end
3479
3480 for _, player in ipairs(Players:GetPlayers()) do bindPlayerTaskRouting(player) end
3481 Players.PlayerAdded:Connect(bindPlayerTaskRouting)
3482 Players.PlayerRemoving:Connect(clearRepairPath)
3483
3484 for _, p in ipairs(CollectionService:GetTagged("LabEquipment")) do bind(p) end
3485 CollectionService.GetInstanceAddedSignal("LabEquipment"):Connect(bind)
3486
3487 -----
3488 -- AUTOMATIC DOORS
3489 -- Every AutoDoor panel slides up into the header void when a player's
3490 -- HumanoidRootPart comes within range, and back down once nobody is close,
3491 -- so rooms have real doors instead of a permanently open gap.
3492 -----
3493 local DOOR_OPEN_RANGE = 11
3494 local DOOR_CLOSE_RANGE = 15
3495 local DOOR_MIN_OPEN_SECONDS = 3
3496 local function bindAutoDoor(door)
3497     if door:GetAttribute("DoorBound") then return end
3498     door:SetAttribute("DoorBound", true)
3499     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
3500     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
3501     local openX = door:GetAttribute("DoorOpenX") or closedX
3502     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
3503     local doorY = door:GetAttribute("DoorY") or door.Position.Y
3504     local isOpen = false
3505     local openedAt = 0
3506     local tween
3507     local function setIndicatorColour(open)
3508         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
3509         for _, gui in ipairs(door:GetChildren()) do
3510             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then
3511                 local indicator = gui:FindFirstChild("DoorIndicator")
3512                 if indicator then indicator.BackgroundColor3 = colour end
3513             end
3514         end
3515     end
3516     local function setOpen(open)
3517         if open == isOpen then return end
3518         isOpen = open
3519         if open then openedAt = os.clock() end
3520         if tween then tween:Cancel() end
3521         local targetX = open and openX or closedX
3522         local targetZ = open and openZ or closedZ
3523         -- 0.9s ease so the slide is smooth and readable (never an instant
3524         -- snap, which is what made the old vertical slide read as flicker).
3525         tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
↪      Enum.EasingDirection.InOut),
        {CFrame = CFrame.new(targetX, doorY, targetZ)})
3526         tween:Play()
3527         door.CanCollide = not open
3528         door:SetAttribute("DoorOpen", open) -- watched by the room smoke system
3529         setIndicatorColour(open)
3530     end
3531     task.spawn(function()
3532         while door.Parent do

```

```

3534     local nearest = math.huge
3535     for _, player in ipairs(Players:GetPlayers()) do
3536         local character = player.Character
3537         local hrp = character and character:FindFirstChild("HumanoidRootPart")
3538         if hrp then
3539             -- Floors stack directly above each other at the same X/Z, so a
3540             -- flat (X/Z-only) distance check would treat a player on a
3541             -- completely different floor as "standing at the door." Only
3542             -- count players actually close in height to this door first.
3543             local verticalGap = math.abs(hrp.Position.Y - doorY)
3544             if verticalGap < 8 then
3545                 local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) - Vector3.new(closedX,
3546                     ↪ doorY, closedZ)).Magnitude
3547                 if flat < nearest then nearest = flat end
3548             end
3549         end
3550         -- Once opened, a door holds for at least DOOR_MIN_OPEN_SECONDS
3551         -- before it's allowed to close again -- this is what actually
3552         -- fixes the "opens and closes too fast" flicker, independent of
3553         -- the slide direction/speed above.
3554         if isOpen and nearest > DOOR_CLOSE_RANGE and (os.clock() - openedAt) >= DOOR_MIN_OPEN_SECONDS then
3555             setOpen(false)
3556         elseif not isOpen and nearest < DOOR_OPEN_RANGE then
3557             setOpen(true)
3558         end
3559         task.wait(0.25)
3560     end
3561 end)
3562 end
3563
3564 for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindAutoDoor(p) end
3565 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(bindAutoDoor)
3566
3567 -----
3568 -- EMERGENCY KITS
3569 -- One per room. Press E to fully restore health. When a player's health
3570 -- drops below half (and they're not mid-job), a red floor-marker route --
3571 -- the same system used for repair/transfer jobs -- guides them to the
3572 -- closest kit.
3573 -----
3574 local function bindEmergencyKit(kit)
3575     if kit:GetAttribute("KitBound") then return end
3576     kit:SetAttribute("KitBound", true)
3577     local prompt = kit:FindFirstChild("EmergencyKitPrompt")
3578     if not prompt then return end
3579     -- Health stations RECHARGE rather than being consumed. A single-use kit
3580     -- that destroyed itself meant a floor could be permanently stripped of
3581     -- health with no way to get it back.
3582     local RECHARGE_SECONDS = 25
3583     local caption
3584     local faceGui = kit:FindFirstChild("EmergencyKitFace")
3585     if faceGui then
3586         for _, d in ipairs(faceGui:GetDescendants()) do
3587             if d:IsA("TextLabel") then caption = d break end
3588         end
3589     end
3590
3591     local function setCharged(charged)
3592         kit:SetAttribute("Charged", charged)
3593         prompt.Enabled = charged
3594         kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
3595         if caption then caption.Text = charged and "HEALTH" or "CHARGING" end
3596     end
3597
3598     prompt.Triggered:Connect(function(player)
3599         if kit:GetAttribute("Charged") == false then return end
3600         local character = player.Character
3601         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3602         if not humanoid then return end
3603         setCharged(false)
3604         humanoid.Health = humanoid.MaxHealth
3605         notify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s -- there are
3606             ↪ more along every corridor.")
3607         task.delay(RECHARGE_SECONDS, function()
3608             if kit.Parent then setCharged(true) end
3609         end)
3610     end)

```

```

3608         end)
3609     end)
3610     setCharged(true)
3611 end
3612 for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindEmergencyKit(p) end
3613 CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindEmergencyKit)
3614
3615 -----
3616 -- PLAYER HEALTH BARS
3617 -- Matches the bar the worker bots carry, so a player can read their own
3618 -- and everyone else's condition without opening any menu.
3619 -----
3620 local function attachPlayerHealthBar(character)
3621     local humanoid = character:FindFirstChildOfClass("Humanoid") or character:WaitForChild("Humanoid", 8)
3622     local head = character:FindFirstChild("Head") or character:WaitForChild("Head", 8)
3623     if not humanoid or not head or head:FindFirstChild("LabHealthBar") then return end
3624
3625     local gui = Instance.new("BillboardGui")
3626     gui.Name = "LabHealthBar"
3627     gui.Size = UDim2.fromOffset(168, 44)
3628     gui.StudsOffset = Vector3.new(0, 2.6, 0)
3629     gui.MaxDistance = 90
3630     gui.AlwaysOnTop = false
3631     gui.Parent = head
3632
3633     local back = Instance.new("Frame")
3634     back.Name = "HealthBarBack"
3635     back.Size = UDim2.new(0.9, 0, 0.34, 0)
3636     back.Position = UDim2.new(0.05, 0, 0.06, 0)
3637     back.BackgroundColor3 = Color3.fromRGB(22, 26, 32)
3638     back.BackgroundTransparency = 0.2
3639     back.BorderSizePixel = 0
3640     back.Parent = gui
3641     local fill = Instance.new("Frame")
3642     fill.Name = "HealthBarFill"
3643     fill.Size = UDim2.fromScale(1, 1)
3644     fill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
3645     fill.BorderSizePixel = 0
3646     fill.Parent = back
3647     local label = Instance.new("TextLabel")
3648     label.Name = "HealthText"
3649     label.Size = UDim2.new(1, 0, 0.5, 0)
3650     label.Position = UDim2.new(0, 0, 0.48, 0)
3651     label.BackgroundTransparency = 1
3652     label.TextColor3 = Color3.fromRGB(224, 238, 248)
3653     label.TextStrokeTransparency = 0.4
3654     label.TextScaled = true
3655     label.Font = Enum.Font.GothamBold
3656     label.Parent = gui
3657
3658     local function refresh()
3659         local maximum = math.max(1, humanoid.MaxHealth)
3660         local ratio = math.clamp(humanoid.Health / maximum, 0, 1)
3661         fill.Size = UDim2.fromScale(ratio, 1)
3662         fill.BackgroundColor3 = ratio > 0.55 and Color3.fromRGB(75, 220, 95)
3663             or (ratio > 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
3664         label.Text = string.format("%d%%", math.floor(ratio * 100 + 0.5))
3665     end
3666     humanoid.HealthChanged:Connect(refresh)
3667     humanoid:GetPropertyChangedSignal("MaxHealth"):Connect(refresh)
3668     refresh()
3669 end
3670
3671 local function watchPlayerHealth(player)
3672     player.CharacterAdded:Connect(attachPlayerHealthBar)
3673     if player.Character then task.spawn(attachPlayerHealthBar, player.Character) end
3674 end
3675 Players.PlayerAdded:Connect(watchPlayerHealth)
3676 for _, player in ipairs(Players:GetPlayers()) do watchPlayerHealth(player) end
3677
3678 -----
3679 -- SECRET BASEMENT VENT
3680 -- Disguised as part of the lobby's own exterior wall. Holding E on it
3681 -- drops the player straight into the basement corridor.
3682 -----
3683 local function bindSecretBasementPanel(panel)

```

```

3684     if panel:GetAttribute("SecretBound") then return end
3685     panel:SetAttribute("SecretBound", true)
3686     local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")
3687     if not prompt then return end
3688     prompt.Triggered:Connect(function(player)
3689         local character = player.Character
3690         local hrp = character and character:FindFirstChild("HumanoidRootPart")
3691         if not hrp then return end
3692         local humanoid = character:FindFirstChildOfClass("Humanoid")
3693         if not humanoid or humanoid.Health <= 0 or (hrp.Position - panel.Position).Magnitude > 12 then return
3694         ↪ end
3695         local floorPoint = Vector3.new(
3696             panel:GetAttribute("SecretDestX") or hrp.Position.X,
3697             panel:GetAttribute("SecretDestY") or hrp.Position.Y,
3698             panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
3699         local params = RaycastParams.new()
3700         params.FilterType = Enum.RaycastFilterType.Exclude
3701         params.FilterDescendantsInstances = {character}
3702         params.RespectCanCollide = true
3703         local floorHit = workspace:Raycast(floorPoint + Vector3.new(0, 7, 0), Vector3.new(0, -12, 0), params)
3704         if not floorHit or floorHit.Normal.Y < 0.7 then return end
3705         local feetOffset = humanoid.HipHeight + hrp.Size.Y / 2
3706         for _, name in ipairs({"LeftFoot", "RightFoot", "Left Leg", "Right Leg"}) do
3707             local foot = character:FindFirstChild(name)
3708             if foot and foot:IsA("BasePart") then
3709                 feetOffset = math.max(feetOffset, hrp.Position.Y - (foot.Position.Y - foot.Size.Y / 2))
3710             end
3711         end
3712         hrp.CFrame = CFrame.new(floorHit.Position + Vector3.new(0, feetOffset + 0.35, 0))
3713         hrp.AssemblyLinearVelocity, hrp.AssemblyAngularVelocity = Vector3.zero, Vector3.zero
3714         humanoid:ChangeState(Enum.HumanoidStateType.GettingUp)
3715         notify(player, "The vent panel swings open -- you drop into the basement.")
3716     end)
3717     end
3718     for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindSecretBasementPanel(p) end
3719     CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindSecretBasementPanel)
3720     local function nearestEmergencyKit(position)
3721         local best, bestDist
3722         for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
3723             if kit.Parent then
3724                 local d = (kit.Position - position).Magnitude
3725                 if not bestDist or d < bestDist then best, bestDist = kit, d end
3726             end
3727         end
3728         return best
3729     end
3730     local healthPathActive = {}
3731     task.spawn(function()
3732         while true do
3733             task.wait(3)
3734             for _, player in ipairs(Players:GetPlayers()) do
3735                 local character = player.Character
3736                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3737                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
3738                 if humanoid and hrp and humanoid.Health > 0 then
3739                     local low = humanoid.Health < humanoid.MaxHealth * 0.5
3740                     local busy = player:GetAttribute("AssignedRepairId") or
3741                     ↪ player:GetAttribute("AssignedTransferSource")
3742                     if low and not busy and not healthPathActive[player] then
3743                         local kit = nearestEmergencyKit(hrp.Position)
3744                         if kit then
3745                             createRepairPath(player, kit)
3746                             healthPathActive[player] = true
3747                             notify(player, "Health low: follow the RED floor lights to the nearest emergency kit.")
3748                         end
3749                     elseif (not low or busy) and healthPathActive[player] then
3750                         healthPathActive[player] = false
3751                         if not busy then clearRepairPath(player) end
3752                     end
3753                 end
3754             end
3755         end
3756     end)
3757 
```

```

3758 -----
3759 -- EQUIPMENT HAZARDS
3760 -- A malfunctioning (FAILED) unit is genuinely dangerous to linger next to:
3761 -- electrical faults, steam/humidity blowouts, chemical spills and coolant
3762 -- (freon) leaks all drain health while a player stays in range.
3763 -----
3764 local HAZARD_BY_KIND = {
3765     ["Germination Chamber"] = {"STEAM LEAK", 3},
3766     ["Incubator"] = {"STEAM LEAK", 3},
3767     ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
3768     ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
3769     ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
3770     ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
3771     ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
3772     ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
3773     ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},
3774     ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
3775     ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
3776 }
3777 local HAZARD_RANGE = 9
3778 local hazardWarnedAt = {}
3779
3780 -- A unit that's been left FAILED too long can catastrophically blow up,
3781 -- catch fire, or arc -- and unlike an ordinary failure, this one can never
3782 -- be repaired. It keeps draining nearby health (worse than before) and its
3783 -- StatusLED/timer read DESTROYED permanently.
3784 --
3785 -- Two-stage sequence: warnEquipmentCatastrophe() runs a 2-second siren +
3786 -- strobe so anyone nearby has time to clear out, THEN destroyEquipment()
3787 -- fires the actual explosion/fire/arc/leak effects.
3788 local DESTRUCTION_KIND = {
3789     ["Germination Chamber"] = "steam",
3790     ["Incubator"] = "steam",
3791     ["Decontamination Unit"] = "gas",
3792     ["Mutagenesis Chamber"] = "electrical",
3793     ["Hormone Treatment Bench"] = "liquid",
3794     ["Plant Growth Chamber"] = "gas",
3795     ["Growth Hormone Mixer"] = "liquid",
3796     ["GMO Injector"] = "electrical",
3797     ["Nutrient Infuser"] = "gas",
3798     ["UV Growth Scanner"] = "electrical",
3799     ["Pollination Station"] = "electrical",
3800 }
3801
3802 local function scatterDebris(part)
3803     for i = 1, math.random(4, 7) do
3804         local chunk = Instance.new("Part")
3805         chunk.Name = "Debris"
3806         chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
3807         chunk.Color = part.Color
3808         chunk.Material = Enum.Material.Metal
3809         chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2, math.random(-3,
3810             ↪ 3) / 2)
3811         chunk.Anchored = false
3812         chunk.CanCollide = true
3813         chunk.Parent = part.Parent
3814         local av = Instance.new("AngularVelocity")
3815         av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
3816         av.Parent = chunk
3817         local bv = Instance.new("BodyVelocity")
3818         bv.MaxForce = Vector3.new(4000, 4000, 4000)
3819         bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
3820         bv.Parent = chunk
3821         Debris:AddItem(bv, 0.25)
3822         Debris:AddItem(chunk, 8)
3823     end
3824 end
3825
3826 local destroyEquipment
3827
3828 local function warnEquipmentCatastrophe(part, floorLabel, unitLabel)
3829     if part:GetAttribute("AboutToBlow") then return end
3830     part:SetAttribute("AboutToBlow", true)
3831     local beacon = Instance.new("PointLight")
3832     beacon.Color = Color3.fromRGB(255, 40, 30)
3833     beacon.Brightness = 4

```



```

3833 beacon.Range = 16
3834 beacon.Parent = part
3835 local siren = Instance.new("Sound")
3836 siren.Name = "SirenWarning"
3837 siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
3838 siren.Looped = true
3839 siren.Volume = 3
3840 siren.PlaybackSpeed = 1.6
3841 siren.Parent = part
3842 siren:Play()
3843 for _, player in ipairs(Players:GetPlayers()) do
3844     notify(player, "ALARM: " .. floorLabel .. " * " .. unitLabel .. " is about to fail catastrophically --
        ↳ MOVE AWAY NOW!")
3845 end
3846 task.spawn(function()
3847     local elapsed = 0
3848     while elapsed < 2 and part.Parent do
3849         beacon.Enabled = not beacon.Enabled
3850         task.wait(0.2)
3851         elapsed = elapsed + 0.2
3852     end
3853 end)
3854 task.delay(2, function()
3855     siren:Stop()
3856     siren:Destroy()
3857     beacon:Destroy()
3858     if part.Parent then destroyEquipment(part) end
3859 end)
3860 end
3861
3862 destroyEquipment = function(part)
3863     local floorLabel = equipmentFloorLabel(part)
3864     local unitLabel = equipmentLabel(part)
3865     local causes = {"exploded", "caught fire", "arced and shorted out"}
3866     local cause = causes[math.random(1, #causes)]
3867     part:SetAttribute("FunctionalState", "DESTROYED")
3868     part:SetAttribute("AboutToBlow", nil)
3869     part:SetAttribute("RepairInstruction", floorLabel .. " * " .. unitLabel .. " * DESTROYED, UNFIXABLE")
3870     equipmentReadout(part, "DESTROYED", floorLabel .. " * " .. unitLabel .. " * cause .. " -- permanently
        ↳ offline",
3871         Color3.fromRGB(150, 40, 35))
3872     for _, ledPart in ipairs(part:GetDescendants()) do
3873         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
            ↳ Enum.Material.SmoothPlastic end
3874     end
3875     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
3876
3877     local bang = Instance.new("Sound")
3878     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
3879     bang.Volume = 4
3880     bang.PlaybackSpeed = 0.8
3881     bang.Parent = part
3882     bang:Play()
3883     Debris:AddItem(bang, 4)
3884
3885     local explosion = Instance.new("Explosion")
3886     explosion.Position = part.Position
3887     explosion.BlastRadius = 8
3888     explosion.BlastPressure = 0
3889     explosion.ExplosionType = Enum.ExplosionType.NoCraters
3890     explosion.DestroyJointRadiusPercent = 0
3891     explosion.Parent = workspace
3892
3893     local kindOfDestruction = DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
3894
3895     local fire = Instance.new("Fire")
3896     fire.Size = 6
3897     fire.Heat = 9
3898     fire.Color = Color3.fromRGB(255, 140, 40)
3899     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
3900     fire.Parent = part
3901     Debris:AddItem(fire, 16)
3902
3903     local sparkles = Instance.new("Sparkles")
3904     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
3905     sparkles.Parent = part

```

```

3906     Debris:AddItem(sparkles, 3)
3907
3908     local smoke = Instance.new("Smoke")
3909     smoke.Size = 8
3910     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
3911     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
3912     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
3913     or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
3914     or Color3.fromRGB(50, 50, 55)
3915     smoke.Parent = part
3916     Debris:AddItem(smoke, 20)
3917
3918     if kindOfDestruction == "liquid" then
3919         local puddle = Instance.new("Part")
3920         puddle.Name = "LeakPuddle"
3921         puddle.Shape = Enum.PartType.Cylinder
3922         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
3923         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05, part.Position.Z)
3924             * CFrame.Angles(0, 0, math.rad(90))
3925         puddle.Color = Color3.fromRGB(120, 90, 40)
3926         puddle.Material = Enum.Material.Neon
3927         puddle.Transparency = 0.35
3928         puddle.Anchored = true
3929         puddle.CanCollide = false
3930         puddle.Parent = part.Parent
3931         Debris:AddItem(puddle, 25)
3932     end
3933
3934     scatterDebris(part)
3935
3936     for _, player in ipairs(Players:GetPlayers()) do
3937         notify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause .. ". It
            ↳ cannot be repaired.")
3938     end
3939 end
3940
3941 task.spawn(function()
3942     while true do
3943         task.wait(1)
3944         local hazards = {}
3945         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
3946             local state = equip.Parent and equip:GetAttribute("FunctionalState")
3947             if state == "FAILED" then
3948                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
3949                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
3950                 -- Five minutes to reach the job before a rare catastrophic risk;
3951                 -- a unit under a player's addition check cannot self-destruct.
3952                 local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
3953                 if failedFor > 300 and not equip:GetAttribute("QuizPlayerId") and math.random(1, 600) == 1 then
3954                     warnEquipmentCatastrophe(equip, equipmentFloorLabel(equip), equipmentLabel(equip))
3955                 end
3956             elseif state == "DESTROYED" then
3957                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
3958                 if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
                    ↳ info[2] + 2}) end
3959             end
3960         end
3961         if #hazards > 0 then
3962             for _, player in ipairs(Players:GetPlayers()) do
3963                 local character = player.Character
3964                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3965                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
3966                 if humanoid and hrp and humanoid.Health > 0 then
3967                     local worst
3968                     for _, hazard in ipairs(hazards) do
3969                         local d = (hazard.part.Position - hrp.Position).Magnitude
3970                         if d < HAZARD_RANGE and hazard.part:GetAttribute("QuizPlayerId") ~= player.UserId
3971                             and (not worst or hazard.dps > worst.dps) then worst = hazard end
3972                     end
3973                     if worst then
3974                         humanoid:TakeDamage(worst.dps)
3975                         local last = hazardWarnedAt[player] or 0
3976                         if os.clock() - last > 2 then
3977                             hazardWarnedAt[player] = os.clock()
3978                             notify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
                                ↳ worst.label, math.floor(humanoid.Health)))

```

```

3979         end
3980     end
3981 end
3982     end
3983 end
3984 end
3985 end)
3986
3987 for _, p in ipairs(CollectionService:GetTagged("LabToolDispenser")) do bindDispenser(p) end
3988 CollectionService:GetInstanceAddedSignal("LabToolDispenser"):Connect(bindDispenser)
3989
3990 local testBots = CollectionService:GetTagged("LabWorkerBot")
3991 table.sort(testBots, function(a, b) return (a:GetAttribute("BotNumber") or 0) < (b:GetAttribute("BotNumber") or
↪ 0) end)
3992 for index, bot in ipairs(testBots) do
3993     -- Stagger WITHIN each floor, not across the whole building. Staggering on
3994     -- the global index at 0.32s a bot meant the top floor did not begin its
3995     -- loop until nearly a minute after the server started, so the upper
3996     -- levels looked completely frozen to anyone who walked up there early.
3997     -- Keyed off the per-floor slot, every floor is fully awake in ~7s.
3998     -- Only bots someone is standing next to will actually compute a path now,
3999     -- so there is no thundering herd to spread out and the stagger can be
4000     -- short. Every floor is ready ~1.6s in instead of ~58s.
4001     local slot = bot:GetAttribute("LevelSlot") or index
4002     task.delay(slot * 0.08, function()
4003         if bot.Parent then runBotScenario(bot) end
4004     end)
4005 end
4006
4007 -- Bodies stay put for the whole shift. LabRound stamps RoundNumber on
4008 -- Workspace when the shift rolls over; that is the cue to clear them away and
4009 -- put every worker back on its feet for a fresh hunt.
4010 local function resetHunt()
4011     local bodies = workspace:FindFirstChild("LabBotBodies")
4012     if bodies then bodies:ClearAllChildren() end
4013
4014     local complex = workspace:FindFirstChild("Laboratory_Complex")
4015     if complex then complex:SetAttribute("BotsKilled", 0) end
4016
4017     local restaged = 0
4018     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
4019         if bot:GetAttribute("Dead") then
4020             local home = bot:GetAttribute("HomeCFrame")
4021             if typeof(home) == "CFrame" then bot:PivotTo(home) end
4022             bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
4023             bot:SetAttribute("KnockedOut", false)
4024             bot:SetAttribute("Dead", false)
4025             bot:SetAttribute("TaskState", "Waiting for plant task")
4026             refreshBotHealth(bot)
4027             -- its old scenario loop exited on death, so start a fresh one
4028             task.spawn(runBotScenario, bot)
4029             restaged = restaged + 1
4030         end
4031     end
4032     if restaged > 0 then
4033         print("Shift reset: cleared bodies and restaged " .. restaged .. " workers.")
4034     end
4035 end
4036 workspace:GetAttributeChangedSignal("RoundNumber"):Connect(resetHunt)
4037
4038 -----
4039 -- ROOM SMOKE
4040 -- Small rooms slowly fill with smoke. Opening a door vents it. Capped so
4041 -- there are never more than a handful of smoke volumes alive at once.
4042 -----
4043 local MAX_SMOKY_ROOMS = 4
4044 local smokeByRoom = {}
4045
4046 local function clearRoomSmoke(key)
4047     local folder = smokeByRoom[key]
4048     if not folder then return end
4049     smokeByRoom[key] = nil
4050     for _, d in ipairs(folder:GetDescendants()) do
4051         if d:IsA("Smoke") then d.Enabled = false end
4052     end
4053     Debris:AddItem(folder, 5) -- let what is already in the air disperse

```

```

4054 end
4055
4056 local function fillRoomWithSmoke(levelName, rect)
4057     local key = levelName .. "/" .. rect.name
4058     if smokeByRoom[key] then return end
4059     local folder = Instance.new("Folder")
4060     folder.Name = "RoomSmoke"
4061     folder:SetAttribute("RoomKey", key)
4062     folder.Parent = workspace
4063     for i = 1, 5 do
4064         local emitter = Instance.new("Part")
4065         emitter.Name = "SmokeSource"
4066         emitter.Size = Vector3.new(1, 1, 1)
4067         emitter.Anchored = true
4068         emitter.CanCollide = false
4069         emitter.CanQuery = false
4070         emitter.CanTouch = false
4071         emitter.Transparency = 1
4072         emitter.CFrame = LabSpatial.frame() * CFrame.new(
4073             rect.x1 + (rect.x2 - rect.x1) * (i - 0.5) / 5,
4074             rect.y + 3,
4075             rect.z1 + (rect.z2 - rect.z1) * ((i * 2) % 5 + 0.5) / 5)
4076         emitter.Parent = folder
4077         local smoke = Instance.new("Smoke")
4078         smoke.Size = 11
4079         smoke.Opacity = 0.32
4080         smoke.RiseVelocity = 2
4081         smoke.Color = Color3.fromRGB(150, 150, 152)
4082         smoke.Parent = emitter
4083     end
4084     smokeByRoom[key] = folder
4085 end
4086
4087 local function ventFromDoor(door)
4088     local pos = LabSpatial.localPoint(door.Position)
4089     for key in pairs(smokeByRoom) do
4090         local levelName, roomName = key:match("^(.)/(.)$")
4091         local rect = roomRect(levelName, roomName)
4092         -- a door on the room's own boundary counts, hence the small margin
4093         if rect and math.abs(pos.Y - rect.y) < FLOOR_BAND
4094             and pos.X >= rect.x1 - 8 and pos.X <= rect.x2 + 8
4095             and pos.Z >= rect.z1 - 8 and pos.Z <= rect.z2 + 8 then
4096             clearRoomSmoke(key)
4097         end
4098     end
4099 end
4100
4101 local function watchDoor(door)
4102     door:GetAttributeChangedSignal("DoorOpen"):Connect(function()
4103         if door:GetAttribute("DoorOpen") then ventFromDoor(door) end
4104     end)
4105 end
4106 for _, door in ipairs(CollectionService:GetTagged("AutoDoor")) do watchDoor(door) end
4107 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(watchDoor)
4108
4109 task.spawn(function()
4110     while true do
4111         task.wait(45)
4112         local live = 0
4113         for _ in pairs(smokeByRoom) do live = live + 1 end
4114         if live < MAX_SMOKY_ROOMS then
4115             local levels = {}
4116             for levelName in pairs(roomsByLevel) do table.insert(levels, levelName) end
4117             if #levels > 0 then
4118                 local levelName = levels[math.random(1, #levels)]
4119                 -- only the small east bays; the long west room is not a "small room"
4120                 local small = {}
4121                 for _, r in ipairs(roomsByLevel[levelName]) do
4122                     if (r.x2 - r.x1) < 90 and (r.z2 - r.z1) < 90 then table.insert(small, r) end
4123                 end
4124                 if #small > 0 then
4125                     fillRoomWithSmoke(levelName, small[math.random(1, #small)])
4126                 end
4127             end
4128         end
4129     end
4130 end

```

```

4130 end)
4131 task.spawn(function()
4132     while task.wait(2) do updateBotTestBoard() end
4133 end)
4134 task.spawn(function()
4135     while task.wait(2) do refreshRoomMonitors() end
4136 end)
4137 task.spawn(function()
4138     task.wait(3)
4139     while true do
4140         adaptiveFailureTick()
4141         genericFailureTick()
4142         task.wait(45)
4143     end
4144 end)
4145
4146 print("Equipment armed: " .. #CollectionService:GetTagged("LabEquipment")
4147     .. " items; recovery tool pads: " .. #CollectionService:GetTagged("LabToolDispenser") .. ".")
4148
4149 -----
4150 -- LAB STAFF NPCs
4151 -----
4152 local function bindNPC(part)
4153     if part:GetAttribute("TalkBound") then return end
4154     part:SetAttribute("TalkBound", true)
4155     local tipsStr = part:GetAttribute("Tips") or ""
4156     local tips = {}
4157     for t in tipsStr:gmatch("[^|]+") do table.insert(tips, t) end
4158     local label = part:GetAttribute("NPCName") or part:GetAttribute("Role") or "Lab Staff"
4159
4160     local prompt = Instance.new("ProximityPrompt")
4161     prompt.ObjectText = label
4162     prompt.ActionText = "Talk"
4163     prompt.HoldDuration = 0.3
4164     prompt.MaxActivationDistance = 10
4165     prompt.RequiresLineOfSight = false
4166     prompt.Parent = part
4167
4168     prompt.Triggered:Connect(function(player)
4169         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
4170         if #tips == 0 then return end
4171         notify(player, label .. ": " .. tips[math.random(1, #tips)])
4172     end)
4173 end
4174
4175 for _, p in ipairs(CollectionService:GetTagged("LabNPC")) do bindNPC(p) end
4176 CollectionService:GetInstanceAddedSignal("LabNPC"):Connect(bindNPC)
4177
4178 print("Lab staff NPCs armed: " .. #CollectionService:GetTagged("LabNPC") .. ".")

```

Chapter 39

S12 - LabRoles

ServerScriptService.LabRoles

Authority group: active. 19385 UTF-8 bytes; 431 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ServerScriptService.LabRoles  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Roles and outfits  
7 Future change: Keep exactly two roles unless commissioned otherwise; centralize role-to-outfit and permission  
↪ mapping.  
8 Preserve dependencies: GWAPServer LabRole observer and original equipment role checks.  
9 Acceptance cases: Both kiosks, respawn, role change during mission, and accessory/rig variations.  
10 ]=====]  
11 -- LabRoles : role kiosks, teams, per-player leaderstats, a persistent  
12 -- HUD, and a visible appearance change (tinted uniform + hard hat) so  
13 -- everyone can see at a glance who is playing which role.  
14  
15 local Players          = game:GetService("Players")  
16 local Teams            = game:GetService("Teams")  
17 local CollectionService = game:GetService("CollectionService")  
18  
19 local ROLE_PAINT = {  
20     Technician = Color3.fromRGB(60, 120, 200),  
21     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),  
22 }  
23  
24 local ROLE_HINT = {  
25     Unassigned = "Choose BLUE Technician or GREEN Laboratory Technician.",  
26     Technician = "Follow RED floor lights. HOLD R at the failed unit. Score grows with failure time and  
↪ assigned distance.",  
27     ["Laboratory Technician"] = "E takes a plant sample. Carry it to any GREEN RUNNING cabinet marked FREE SLOT  
↪ and press E again to revive/drop it.",  
28 }  
29  
30 local function roleEquipmentId(part)  
31     local code = part:GetAttribute("GrowthCode")  
32     if code and code ~= "" then return "CHAMBER:" .. code end  
33     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},  
↪ " |")  
34 end  
35  
36 local function roleFloor(part)  
37     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"  
38 end  
39  
40 local function roleTransferSeconds(source, destination)  
41     local distance = destination and (destination.Position - source.Position).Magnitude or 60  
42     local playerCount = math.max(1, #Players:GetPlayers())  
43     local complex = workspace:FindFirstChild("Laboratory_Complex")  
44     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0  
45     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10  
46         + math.clamp(backlog / 20, 0, 25)), 60, 180)  
47 end
```

```

48
49 local function roleEquipmentLabel(part)
50     local code = part:GetAttribute("GrowthCode")
51     if code and code ~= "" then return "CHAMBER " .. code end
52     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
53 end
54
55 local function roleDestinations(source)
56     local choices = {}
57     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
58         local count = chamber:GetAttribute("SampleCount") or 0
59         local reservations = chamber:GetAttribute("IncomingReservations") or 0
60         if chamber ~= source and chamber:GetAttribute("FunctionalState") ~= "FAILED"
61             and count + reservations < (chamber:GetAttribute("Capacity") or 10) then table.insert(choices,
62                 ⇐ chamber) end
63     end
64     table.sort(choices, function(a, b)
65         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
66         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
67         if sameA ~= sameB then return sameA < sameB end
68         return (a:GetAttribute("SampleCount") or 0) < (b:GetAttribute("SampleCount") or 0)
69     end)
70     return choices
71 end
72
73 local function clearRoleAssignment(player)
74     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
75         ⇐ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
76         player:SetAttribute(name, nil)
77     end
78 end
79
80 local function assignRoleObjective(player, roleName)
81     clearRoleAssignment(player)
82     if roleName == "Technician" then
83         local failed = {}
84         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
85             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment) end
86         end
87         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
88         table.sort(failed, function(a, b)
89             if root then return (a.Position - root.Position).Magnitude < (b.Position - root.Position).Magnitude
90                 ⇐ end
91             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
92         end)
93         local target = failed[1]
94         if not target then return "All equipment is working. Repair score grows with failure time and assigned
95             ⇐ distance." end
96         player:SetAttribute("AssignedRepairId", roleEquipmentId(target))
97         local distance = root and (target.Position - root.Position).Magnitude or 0
98         local waited = math.max(0, os.time() - (target:GetAttribute("FailureSince") or os.time()))
99         local projected = math.clamp(20 + math.floor(waited / 8) + math.floor(distance / 10), 8, 120)
100         player:SetAttribute("AssignedRepairDistance", distance)
101         player:SetAttribute("AssignedRepairProjectedPoints", projected)
102         return string.format("REPAIR JOB: %s • %s. Follow RED floor lights; HOLD R. About +%d points.",
103             roleFloor(target), roleEquipmentLabel(target), projected)
104     end
105 end
106
107 local jobs = {}
108 for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
109     if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0) > 0
110         and not source:GetAttribute("ReservedByBot") then
111         local destination = roleDestinations(source)[1]
112         if destination then table.insert(jobs, {source = source, destination = destination}) end
113     end
114 end
115 table.sort(jobs, function(a, b)
116     local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
117     local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
118     return an < bn
119 end)
120 local job = jobs[1]
121 if not job then return "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
122     ⇐ +30." end
123 local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
124 local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"

```

```

119     player:SetAttribute("AssignedTransferSource", sourceCode)
120     player:SetAttribute("AssignedTransferDestination", destinationCode)
121     local allowedSeconds = roleTransferSeconds(job.source, job.destination)
122     player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
123     return "ASSIGNED +30: " .. roleFloor(job.source) .. " * " .. sourceCode .. " → " .. destinationCode
124     .. " * " .. roleFloor(job.destination) .. ". PRESS E at source, deliver in " .. allowedSeconds .. "
        ↪ seconds."
125 end
126
127 -----
128 -- per-player HUD (role, contributions, current objective)
129 -----
130     local function buildHud(player)
131         local playerGui = player:WaitForChild("PlayerGui")
132         local duplicate = playerGui:FindFirstChild("LiveLabHUD")
133         if duplicate then duplicate:Destroy() end
134         local gui = Instance.new("ScreenGui")
135         gui.Name = "RoleHUD"
136         gui.ResetOnSpawn = false
137         gui.DisplayOrder = 20
138         gui.Parent = playerGui
139
140         local frame = Instance.new("Frame")
141         frame.Size = UDim2.fromOffset(238, 90)
142         frame.Position = UDim2.fromOffset(8, 50)
143         frame.BackgroundColor3 = Color3.fromRGB(18, 20, 26)
144         frame.BackgroundTransparency = 0.14
145         frame.Parent = gui
146         local corner = Instance.new("UICorner"); corner.CornerRadius = UDim.new(0, 8); corner.Parent = frame
147
148         local roleLabel = Instance.new("TextLabel")
149         roleLabel.Name = "RoleLabel"
150         roleLabel.Size = UDim2.new(1, -14, 0, 16)
151         roleLabel.Position = UDim2.new(0, 8, 0, 4)
152         roleLabel.BackgroundTransparency = 1
153         roleLabel.TextXAlignment = Enum.TextXAlignment.Left
154         roleLabel.Font = Enum.Font.GothamBold
155         roleLabel.TextSize = 13
156         roleLabel.TextColor3 = Color3.fromRGB(240, 242, 246)
157         roleLabel.Text = "Role: Unassigned"
158         roleLabel.Parent = frame
159
160         local contribLabel = Instance.new("TextLabel")
161         contribLabel.Name = "ContribLabel"
162         contribLabel.Size = UDim2.new(1, -14, 0, 13)
163         contribLabel.Position = UDim2.new(0, 8, 0, 20)
164         contribLabel.BackgroundTransparency = 1
165         contribLabel.TextXAlignment = Enum.TextXAlignment.Left
166         contribLabel.Font = Enum.Font.Gotham
167         contribLabel.TextSize = 10
168         contribLabel.TextTruncate = Enum.TextTruncate.AtEnd
169         contribLabel.TextYAlignment = Enum.TextYAlignment.Top
170         contribLabel.TextColor3 = Color3.fromRGB(200, 205, 212)
171         contribLabel.Text = "POINTS 0"
172         contribLabel.Parent = frame
173
174         local carryLabel = Instance.new("TextLabel")
175         carryLabel.Name = "CarryLabel"
176         carryLabel.Size = UDim2.new(1, -14, 0, 13)
177         carryLabel.Position = UDim2.new(0, 8, 0, 33)
178         carryLabel.BackgroundTransparency = 1
179         carryLabel.TextXAlignment = Enum.TextXAlignment.Left
180         carryLabel.Font = Enum.Font.GothamBold
181         carryLabel.TextSize = 10
182         carryLabel.TextColor3 = Color3.fromRGB(115, 215, 245)
183         carryLabel.Text = "Carrying: nothing"
184         carryLabel.Parent = frame
185
186         local targetsLabel = Instance.new("TextLabel")
187         targetsLabel.Name = "TargetsLabel"
188         targetsLabel.Size = UDim2.new(1, -14, 0, 13)
189         targetsLabel.Position = UDim2.new(0, 8, 0, 46)
190         targetsLabel.BackgroundTransparency = 1
191         targetsLabel.TextXAlignment = Enum.TextXAlignment.Left
192         targetsLabel.Font = Enum.Font.GothamBold
193         targetsLabel.TextSize = 10

```



```

194     targetsLabel.TextColor3 = Color3.fromRGB(255, 205, 120)
195     targetsLabel.Text = "TARGETS 0 / 0"
196     targetsLabel.Parent = frame
197
198     local hintLabel = Instance.new("TextLabel")
199     hintLabel.Name = "HintLabel"
200     hintLabel.Size = UDim2.new(1, -14, 0, 25)
201     hintLabel.Position = UDim2.new(0, 8, 0, 61)
202     hintLabel.BackgroundTransparency = 1
203     hintLabel.TextXAlignment = Enum.TextXAlignment.Left
204     hintLabel.TextYAlignment = Enum.TextYAlignment.Top
205     hintLabel.TextWrapped = true
206     hintLabel.Font = Enum.Font.Gotham
207     hintLabel.TextSize = 9
208     hintLabel.TextTruncate = Enum.TextTruncate.AtEnd
209     hintLabel.TextColor3 = Color3.fromRGB(255, 220, 150)
210     hintLabel.Text = ROLE_HINT.Unassigned
211     hintLabel.Parent = frame
212
213     return gui
214 end
215
216 local function refreshHud(player, roleName)
217     local gui = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
218     if not gui then return end
219     local frame = gui:FindFirstChildOfClass("Frame")
220     if not frame then return end
221     frame.RoleLabel.Text = "Role: " .. roleName
222     local carried = player:GetAttribute("CarriedSample")
223     local objective = player:GetAttribute("LabObjective")
224     frame.CarryLabel.Text = (carried and carried ~= "") and ("Carrying: " .. carried) or "Carrying: nothing"
225     local hint = (objective and objective ~= "") and objective or (ROLE_HINT[roleName] or ROLE_HINT.Unassigned)
226     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
227     if roleName == "Laboratory Technician" and deadline > 0 then
228         hint = hint .. "\nTIME LEFT FOR +30: " .. math.max(0, deadline - os.time()) .. " seconds"
229     end
230     frame.HintLabel.Text = hint
231     local stats = player:FindFirstChild("leaderstats")
232     local points = stats and stats:FindFirstChild("Points")
233     local lastAmount = player:GetAttribute("LastPointAmount") or 0
234     local lastReason = player:GetAttribute("LastPointAward") or ""
235     frame.ContribLabel.Text = string.format("POINTS %d%s",
236         points and points.Value or 0, lastAmount > 0 and (" • LAST +" .. lastAmount .. " " .. lastReason)
237         ↳ or "")
238
239 end
240
241 -----
242 -- Visible appearance change: a welded lab uniform sits above avatar
243 -- clothing, so layered clothes cannot hide the selected role.
244 -----
245 local function wearablePart(uniform, bodyPart, name, size, offset, colour, material)
246     local p = Instance.new("Part")
247     p.Name = name
248     p.Size = size
249     p.CFrame = bodyPart.CFrame * offset
250     p.Color = colour
251     p.Material = material or Enum.Material.Fabric
252     p.Anchored = false
253     p.CanCollide = false
254     p.CanTouch = false
255     p.Massless = true
256     p.CastShadow = false
257     p.TopSurface = Enum.SurfaceType.Smooth
258     p.BottomSurface = Enum.SurfaceType.Smooth
259     p.Parent = uniform
260     local weld = Instance.new("WeldConstraint")
261     weld.Part0 = bodyPart
262     weld.Part1 = p
263     weld.Parent = p
264     return p
265 end
266
267 local function applyAppearance(character, roleName)
268     local paint = ROLE_PAINT[roleName]
269     if not paint then return end
270     local oldUniform = character:FindFirstChild("RoleUniform")

```

```

269     if oldUniform then oldUniform:Destroy() end
270     local oldHelmet = character:FindFirstChild("RoleHelmet")
271     if oldHelmet then oldHelmet:Destroy() end
272
273     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
274     local head = character:FindFirstChild("Head")
275     if not torso or not head then return end
276
277     local uniform = Instance.new("Model")
278     uniform.Name = "RoleUniform"
279     uniform:SetAttribute("Role", roleName)
280     uniform.Parent = character
281
282     wearablePart(uniform, torso, "LabCoat", Vector3.new(torso.Size.X * 1.08, torso.Size.Y * 0.94, torso.Size.Z
↵ * 1.1),
283         CFrame.new(), Color3.fromRGB(238, 241, 244), Enum.Material.Fabric)
284     wearablePart(uniform, torso, "RolePanel", Vector3.new(torso.Size.X * 0.82, torso.Size.Y * 0.55, 0.14),
285         CFrame.new(0, 0.08, -(torso.Size.Z / 2 + 0.12)), paint, Enum.Material.SmoothPlastic)
286     wearablePart(uniform, torso, "RoleBelt", Vector3.new(torso.Size.X * 1.12, 0.18, torso.Size.Z * 1.14),
287         CFrame.new(0, -torso.Size.Y * 0.34, 0), paint, Enum.Material.SmoothPlastic)
288     wearablePart(uniform, torso, "RoleBadge", Vector3.new(0.36, 0.46, 0.16),
289         CFrame.new(-torso.Size.X * 0.28, torso.Size.Y * 0.22, -(torso.Size.Z / 2 + 0.18)),
290         Color3.fromRGB(255, 235, 120), Enum.Material.Neon)
291
292     local helmet = wearablePart(uniform, head, "RoleHelmet", Vector3.new(1.82, 0.72, 1.82),
293         CFrame.new(0, 0.72, 0), paint, Enum.Material.SmoothPlastic)
294     helmet.Shape = Enum.PartType.Ball
295     wearablePart(uniform, head, "HelmetBand", Vector3.new(1.9, 0.16, 1.9),
296         CFrame.new(0, 0.52, 0), Color3.fromRGB(245, 245, 245), Enum.Material.SmoothPlastic)
297
298     for _, limbName in ipairs({"LeftUpperArm", "RightUpperArm", "Left Arm", "Right Arm"}) do
299         local limb = character:FindFirstChild(limbName)
300         if limb then
301             wearablePart(uniform, limb, limbName:gsub(" ", "") .. "RoleBand",
302                 Vector3.new(limb.Size.X * 1.12, math.min(0.24, limb.Size.Y * 0.18), limb.Size.Z * 1.12),
303                 CFrame.new(0, -limb.Size.Y * 0.26, 0), paint, Enum.Material.SmoothPlastic)
304         end
305     end
306 end
307
308 local setupDone = {}
309 local function setupPlayer(player)
310     if setupDone[player] then return end
311     setupDone[player] = true
312     if player:GetAttribute("CarriedSample") == nil then player:SetAttribute("CarriedSample", "") end
313     if player:GetAttribute("LabObjective") == nil then player:SetAttribute("LabObjective", "") end
314     local stats = player:FindFirstChild("leaderstats") or Instance.new("Folder")
315     stats.Name = "leaderstats"
316     stats.Parent = player
317
318     local role = stats:FindFirstChild("Role") or Instance.new("StringValue")
319     role.Name = "Role"
320     if role.Value == "" then role.Value = "Unassigned" end
321     role.Parent = stats
322
323     local contrib = stats:FindFirstChild("Contributions") or Instance.new("IntValue")
324     contrib.Name = "Contributions"
325     contrib.Parent = stats
326     local points = stats:FindFirstChild("Points") or Instance.new("IntValue")
327     points.Name = "Points"
328     points.Parent = stats
329
330     local oldHud = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
331     if oldHud then oldHud:Destroy() end
332     buildHud(player)
333     role:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
334     contrib:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
335     points:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
336     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshHud(player, role.Value) end)
337     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshHud(player, role.Value) end)
338     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshHud(player, role.Value) end)
339     refreshHud(player, role.Value)
340
341     -- Health must never regenerate passively: it only comes back through an
342     -- Emergency Kit. Roblox's default character ships a "Health" script that
343     -- does exactly the auto-regen we don't want, so remove it every spawn.

```

```

344     local function stripAutoHealthRegen(character)
345         local healthScript = character:FindFirstChild("Health")
346         if healthScript then healthScript:Destroy() end
347     end
348     player.CharacterAdded:Connect(function(character)
349         if role.Value ~= "Unassigned" then
350             applyAppearance(character, role.Value)
351         end
352         stripAutoHealthRegen(character)
353     end)
354     if player.Character then
355         if role.Value ~= "Unassigned" then applyAppearance(player.Character, role.Value) end
356         stripAutoHealthRegen(player.Character)
357     end
358     task.spawn(function()
359         while player.Parent do
360             if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshHud(player, role.Value)
361                 ↪ end
362                 task.wait(1)
363             end
364         end
365     end)
366     Players.PlayerAdded:Connect(setupPlayer)
367     for _, player in ipairs(Players:GetPlayers()) do task.spawn(setupPlayer, player) end
368
369     local function bindKiosk(part)
370         local roleName = part:GetAttribute("Role")
371         if not roleName then return end
372         if part:GetAttribute("RoleKioskBound") then return end
373         part:SetAttribute("RoleKioskBound", true)
374
375         local button = Instance.new("Part")
376         button.Name = "RoleSelectButton"
377         button.Shape = Enum.PartType.Cylinder
378         button.Size = Vector3.new(0.28, 1.05, 1.05)
379         button.CFrame = part.CFrame * CFrame.new(0, -part.Size.Y * 0.32, -(part.Size.Z / 2 + 0.32))
380             * CFrame.Angles(0, math.rad(90), 0)
381         button.Color = ROLE_PAINT[roleName]
382         button.Material = Enum.Material.Neon
383         button.Anchored = true
384         button.CanCollide = false
385         button.Parent = part
386
387         local prompt = Instance.new("ProximityPrompt")
388         prompt.ObjectText = roleName .. " Kiosk"
389         prompt.ActionText = "Select " .. roleName
390         prompt.KeyboardKeyCode = Enum.KeyCode.E
391         prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
392         prompt.HoldDuration = 0
393         prompt.MaxActivationDistance = 30
394         prompt.RequiresLineOfSight = false
395         prompt.Parent = button
396
397         local click = Instance.new("ClickDetector")
398         click.MaxActivationDistance = 30
399         click.Parent = button
400         local debounce = {}
401         local function selectRole(player)
402             if debounce[player] and os.clock() < debounce[player] then return end
403             local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
404             if not root or (root.Position - part.Position).Magnitude > 36 then return end
405             debounce[player] = os.clock() + 1
406             setupPlayer(player)
407             local team = Teams:FindFirstChild(roleName)
408             if not team then return end
409             player.Team = team
410             player.Neutral = false
411             player:SetAttribute("LabRole", roleName)
412             local stats = player:FindFirstChild("leaderstats")
413             if stats and stats:FindFirstChild("Role") then
414                 stats.Role.Value = roleName
415             end
416             player:SetAttribute("LabObjective", assignRoleObjective(player, roleName))
417             if player.Character then applyAppearance(player.Character, roleName) end
418             local oldColour = button.Color

```

```
419         button.Color = Color3.fromRGB(255, 255, 255)
420         task.delay(0.35, function() if button.Parent then button.Color = oldColour end end)
421         refreshHud(player, roleName)
422     end
423
424     prompt.Triggered:Connect(selectRole)
425     click.MouseClick:Connect(selectRole)
426 end
427
428 for _, p in ipairs(CollectionService:GetTagged("RoleKiosk")) do bindKiosk(p) end
429 CollectionService:GetInstanceAddedSignal("RoleKiosk"):Connect(bindKiosk)
430
431 print("Role kiosks armed: " .. #CollectionService:GetTagged("RoleKiosk") .. ".")
```

Chapter 40

S13 - LabRound

ServerScriptService.LabRound

Authority group: active. 3591 UTF-8 bytes; 94 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: ServerScriptService.LabRound  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Round lifecycle  
7  Future change: Make reset ordering explicit with one round transition contract and subsystem callbacks.  
8  Preserve dependencies: Points, fractional debt, carrier termination, research repair, plot clearing.  
9  Acceptance cases: Join during break, end during processing, disconnected winner, and reset after destruction.  
10 ]=====  
11 -- LabRound: round timer, scoring cycle and end-of-round resolution.  
12 -- Round length is COMPUTED from the built environment, not hard-coded.  
13 local Players = game:GetService("Players")  
14 local CollectionService = game:GetService("CollectionService")  
15  
16 local INTERMISSION = 20  
17  
18 local function computeRoundSeconds()  
19     local eq = CollectionService:GetTagged("LabEquipment")  
20     local pts = {}  
21     for _, e in ipairs(eq) do  
22         local ok, p = pcall(function() return e:GetPivot().Position end)  
23         if ok then table.insert(pts, p) end  
24     end  
25     if #pts < 2 then return 480, 0, 0 end  
26     local n, sum, samples = #pts, 0, 2000  
27     for _ = 1, samples do  
28         sum = sum + (pts[math.random(n)] - pts[math.random(n)]).Magnitude  
29     end  
30     local mean = sum / samples  
31     local WALK_SPEED = 16 -- studs/sec, Roblox default  
32     local ROUTE_FACTOR = 1.5 -- corridors/doors vs straight line  
33     local VERTICAL = 2.5 -- averaged elevator/stair cost  
34     local ACTION = 4.0 -- HOLD R repair / E transfer  
35     local TARGET_TASKS = 25 -- tasks a competent player should fit in a round  
36     local perTask = (mean / WALK_SPEED) * ROUTE_FACTOR + VERTICAL + ACTION  
37     local secs = math.floor((perTask * TARGET_TASKS) / 30 + 0.5) * 30  
38     return math.clamp(secs, 300, 900), mean, perTask  
39 end  
40  
41 local ROUND_SECONDS, meanDist, perTask = computeRoundSeconds()  
42 print(string.format(  
43     "[LabRound] units=%d meanDist=%.0f studs perTask=%.1fs -> round=%ds (%.1f min)",  
44     #CollectionService:GetTagged("LabEquipment"), meanDist, perTask, ROUND_SECONDS, ROUND_SECONDS / 60))  
45  
46 workspace:SetAttribute("RoundLength", ROUND_SECONDS)  
47  
48 local function pointsOf(player)  
49     local stats = player:FindFirstChild("leaderstats")  
50     local p = stats and stats:FindFirstChild("Points")  
51     return p and p.Value or 0, p
```

```
52 end
53
54 local function resetScores()
55     for _, player in ipairs(Players:GetPlayers()) do
56         local _, p = pointsOf(player)
57         if p then p.Value = 0 end
58         player:SetAttribute("RoundStreakBest", 0)
59     end
60 end
61
62 local function resolveRound()
63     local best, bestScore = nil, -1
64     for _, player in ipairs(Players:GetPlayers()) do
65         local v = pointsOf(player)
66         if v > bestScore then best, bestScore = player, v end
67         local pb = player:GetAttribute("PersonalBest") or 0
68         if v > pb then player:SetAttribute("PersonalBest", v) end
69         player:SetAttribute("LastRoundScore", v)
70     end
71     workspace:SetAttribute("LastWinnerName", best and best.Name or "")
72     workspace:SetAttribute("LastWinnerScore", bestScore > 0 and bestScore or 0)
73 end
74
75 task.spawn(function()
76     local round = 0
77     while true do
78         round += 1
79         resetScores()
80         workspace:SetAttribute("RoundNumber", round)
81         workspace:SetAttribute("RoundPhase", "PLAY")
82         workspace:SetAttribute("RoundEndsAt", os.time() + ROUND_SECONDS)
83         task.wait(ROUND_SECONDS)
84
85         resolveRound()
86         workspace:SetAttribute("RoundPhase", "BREAK")
87         workspace:SetAttribute("RoundEndsAt", os.time() + INTERMISSION)
88         task.wait(INTERMISSION)
89     end
90 end)
91
92 Players.PlayerAdded:Connect(function(player)
93     player:SetAttribute("PersonalBest", 0)
94 end)
```

Chapter 41

S14 - LabTechnicians

ServerScriptService.LabTechnicians

Authority group: active. 7915 UTF-8 bytes; 210 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ServerScriptService.LabTechnicians  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Legacy worker service  
7 Future change: Check which templates actually have Humanoids before extending its path loop. Avoid competing  
  ↪ with LabEquipment worker motion.  
8 Preserve dependencies: LabWorkerBot tags and template structure.  
9 Acceptance cases: No double movement, no idle thread leak, valid target assignment, and bounded retry.  
10 ]=====  
11 -- LabTechnicians: 15 human technicians who panic, tire, call for help and die.  
12 -- They CANNOT repair anything: they can only locate faults and beg the player.  
13 local CollectionService = game:GetService("CollectionService")  
14 local Players = game:GetService("Players")  
15  
16 -----  
17 -- AUDIO: paste asset IDs you OWN or that are royalty-free.  
18 -- Left empty deliberately: every free "scream"/"help me" asset on the  
19 -- Creator Store is ripped from a commercial game, which risks having  
20 -- your experience moderated. Bubbles work without any audio.  
21 -----  
22 local AUDIO = {  
23     scream_f = "",    -- e.g. "rbxassetid://123456789"  
24     scream_m = "",  
25     help     = "",  
26     follow   = "",  
27 }  
28  
29 local PANIC_RADIUS   = 26  
30 local HELP_COOLDOWN   = 9  
31 local DAMAGE_PER_SEC = 2.5  
32 local DAMAGE_RADIUS  = 11  
33 local HEALTH_FLOOR   = 12  
34 local RESPAWN_DELAY  = 25  
35  
36 local HELP_LINES = {  
37     "Help! %s is failing!", "Please help me, I can't repair this!",  
38     "Follow me - %s is critical!", "Come with me, hurry!",  
39     "I'm only a technician, I need you!", "Someone repair %s, please!",  
40 }  
41 local TIRED_LINES = {  
42     "I need a break...", "I can't keep running like this.",  
43     "My legs are gone. Give me a second.", "Too many alarms today...",  
44 }  
45  
46 local function bubbleOf(rig)  
47     local head = rig:FindFirstChild("Head")  
48     if not head then return nil end  
49     local bb = head:FindFirstChild("SpeechBubble")  
50     if bb then return bb:FindFirstChildOfClass("TextLabel"), bb end
```

```

51     bb = Instance.new("BillboardGui")
52     bb.Name = "SpeechBubble"
53     bb.Size = UDim2.new(0, 220, 0, 42)
54     bb.StudsOffset = Vector3.new(0, 4.2, 0)
55     bb.AlwaysOnTop = true
56     bb.MaxDistance = 90
57     bb.Enabled = false
58     bb.Parent = head
59     local f = Instance.new("Frame")
60     f.Size = UDim2.new(1, 0, 1, 0); f.BackgroundColor3 = Color3.fromRGB(18, 22, 24)
61     f.BackgroundTransparency = 0.15; f.BorderSizePixel = 0; f.Parent = bb
62     local c = Instance.new("UICorner"); c.CornerRadius = UDim.new(0, 8); c.Parent = f
63     local st = Instance.new("UIStroke"); st.Color = Color3.fromRGB(255, 120, 100); st.Thickness = 1.5;
64     ↪ st.Parent = f
65     local t = Instance.new("TextLabel")
66     t.Size = UDim2.new(1, -12, 1, -6); t.Position = UDim2.new(0, 6, 0, 3)
67     t.BackgroundTransparency = 1; t.Font = Enum.Font.GothamMedium; t.TextSize = 13
68     t.TextColor3 = Color3.fromRGB(240, 245, 245); t.TextWrapped = true; t.Parent = f
69     return t, bb
70 end
71 local function say(rig, text, colour)
72     local label, bb = bubbleOf(rig)
73     if not label then return end
74     label.Text = text
75     local stroke = bb:FindFirstChildOfClass("Frame"):FindFirstChildOfClass("UIStroke")
76     if stroke and colour then stroke.Color = colour end
77     bb.Enabled = true
78     task.delay(4.5, function() if bb and bb.Parent then bb.Enabled = false end end)
79 end
80
81 local function playSound(rig, key)
82     local id = AUDIO[key]
83     if not id or id == "" then return end
84     local head = rig:FindFirstChild("Head")
85     if not head then return end
86     local snd = Instance.new("Sound")
87     snd.SoundId = id; snd.Volume = 1; snd.RollOffMaxDistance = 90
88     snd.Parent = head; snd:Play()
89     snd.Ended:Connect(function() snd:Destroy() end)
90     task.delay(8, function() if snd and snd.Parent then snd:Destroy() end end)
91 end
92
93 local function labelOf(equip)
94     return equip:GetAttribute("GrowthCode") or equip:GetAttribute("Kind") or equip.Name
95 end
96
97 local function nearestFault(pos)
98     local best, bestD = nil, PANIC_RADIUS
99     for _, e in ipairs(CollectionService:GetTagged("LabEquipment")) do
100         if e:GetAttribute("FunctionalState") == "FAILED" then
101             local ok, p = pcall(function() return e:GetPivot().Position end)
102             if ok then
103                 local d = (p - pos).Magnitude
104                 if d < bestD then best, bestD = e, d end
105             end
106         end
107     end
108     return best, bestD
109 end
110
111 local function setStatus(rig, text, colour)
112     local head = rig:FindFirstChild("Head")
113     local tag = head and head:FindFirstChild("NameTag")
114     local st = tag and tag:FindFirstChild("Status")
115     if st then st.Text = text; if colour then st.TextColor3 = colour end end
116 end
117
118 local function reviveLater(rig, spawnPivot)
119     task.delay(RESPAWN_DELAY, function()
120         if not rig or not rig.Parent then return end
121         local hum = rig:FindFirstChildOfClass("Humanoid")
122         if not hum then return end
123         rig:PivotTo(spawnPivot)
124         hum.Health = hum.MaxHealth
125         rig:SetAttribute("Fatigue", 0)

```



```

126     rig:SetAttribute("Panic", false)
127     rig:SetAttribute("NeedsHelp", false)
128     rig:SetAttribute("KnockedOut", false)
129     setStatus(rig, "Lab Technician", Color3.fromRGB(150, 200, 210))
130     say(rig, "I'm back. Please keep the chambers alive.", Color3.fromRGB(120, 220, 180))
131 end)
132 end
133
134 local function runTechnician(rig)
135     local hum = rig:FindFirstChildOfClass("Humanoid")
136     if not hum then return end
137     local home = rig:GetPivot()
138     local lastHelp, baseSpeed = 0, hum.WalkSpeed
139     local female = rig:GetAttribute("Gender") == "F"
140
141     hum.Died:Connect(function()
142         rig:SetAttribute("KnockedOut", true)
143         setStatus(rig, "COLLAPSED", Color3.fromRGB(255, 90, 80))
144         say(rig, "I can't... breathe...", Color3.fromRGB(255, 90, 80))
145         playSound(rig, female and "scream_f" or "scream_m")
146         reviveLater(rig, home)
147     end)
148
149     task.spawn(function()
150         while rig.Parent and hum.Health > 0 do
151             local dt = 1.0
152             local pos = rig:GetPivot().Position
153             local fault, dist = nearestFault(pos)
154             local fatigue = rig:GetAttribute("Fatigue") or 0
155
156             if fault then
157                 -- PANIC: exaggerated reaction, damage over time, calls for help
158                 rig:SetAttribute("Panic", true)
159                 rig:SetAttribute("NeedsHelp", true)
160                 if dist <= DAMAGE_RADIUS then
161                     hum.Health = math.max(HEALTH_FLOOR, hum.Health - DAMAGE_PER_SEC * dt)
162                 end
163                 fatigue = math.min(100, fatigue + 2)
164                 setStatus(rig, "PANIC - needs a repair tech!", Color3.fromRGB(255, 120, 100))
165
166                 -- flailing: jump and jerk toward/away from the hazard
167                 if math.random() < 0.5 then hum.Jump = true end
168                 local away = (pos - fault:GetPivot().Position).Unit * 6
169                 hum.MoveTo(pos + Vector3.new(away.X, 0, away.Z))
170
171                 if os.clock() - lastHelp > HELP_COOLDOWN then
172                     lastHelp = os.clock()
173                     local line = HELP_LINES[math.random(#HELP_LINES)]
174                     say(rig, string.format(line, labelOf(fault)), Color3.fromRGB(255, 120, 100))
175                     playSound(rig, math.random() < 0.5 and "help" or "follow")
176                 end
177             else
178                 rig:SetAttribute("Panic", false)
179                 rig:SetAttribute("NeedsHelp", false)
180                 fatigue = math.max(0, fatigue - 1.5)
181                 if hum.Health < hum.MaxHealth then
182                     hum.Health = math.min(hum.MaxHealth, hum.Health + 4)
183                 end
184             end
185
186             -- FATIGUE: slows them down, and they complain
187             rig:SetAttribute("Fatigue", fatigue)
188             hum.WalkSpeed = baseSpeed * (1 - 0.55 * (fatigue / 100))
189             if fatigue > 70 then
190                 setStatus(rig, "EXHAUSTED", Color3.fromRGB(255, 200, 90))
191                 if math.random() < 0.12 then
192                     say(rig, TIRED_LINES[math.random(#TIRED_LINES)], Color3.fromRGB(255, 200, 90))
193                 end
194             elseif not fault then
195                 setStatus(rig, "Lab Technician", Color3.fromRGB(150, 200, 210))
196             end
197
198             task.wait(dt)
199         end
200     end)
201 end

```

```
202
203 for _, rig in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
204     task.spawn(runTechnician, rig)
205 end
206 CollectionService.GetInstanceAddedSignal("LabWorkerBot"):Connect(function(rig)
207     task.wait(0.5); runTechnician(rig)
208 end)
209
210 print("[LabTechnicians] " .. #CollectionService:GetTagged("LabWorkerBot") .. " technicians online")
```

Chapter 42

S15 - LoungeSystems

ServerScriptService.LoungeSystems

Authority group: active. 5872 UTF-8 bytes; 172 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ServerScriptService.LoungeSystems  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Lounge server behavior  
7 Future change: Keep lounge interactions isolated from laboratory scoring and navigation. Profile actual cost  
  ↳ before deleting scenery.  
8 Preserve dependencies: LoungeClient and generated lounge objects.  
9 Acceptance cases: Missing lounge, occupied seat, interaction range, and cleanup on departure.  
10 ]=====  
11 -- LoungeSystems: the hidden social floor. Nothing here touches lab gameplay.  
12 local Players = game:GetService("Players")  
13 local TweenService = game:GetService("TweenService")  
14 local TextService=game:GetService("TextService")  
15 local noteCooldown={}  
16  
17 local L = workspace:WaitForChild("Lounge")  
18 local hatch = workspace:WaitForChild("LoungeHatch")  
19 local juke = L.DJ.Jukebox  
20 local sound = juke:WaitForChild("JukeSound")  
21 local npText = juke.NowPlaying.Frame.Text  
22 local noteBody = L.MessageWall.Notes.Frame.Body  
23  
24 local ARRIVE = L.ArrivalPad.Position + Vector3.new(0, 4, 0)  
25 local RETURN_T0 = hatch.Position + Vector3.new(0, 5, 0)  
26  
27 ----- travel  
28 local function move(player, where, inLounge)  
29     local char = player.Character  
30     local hrp = char and char:FindFirstChild("HumanoidRootPart")  
31     if not hrp then return end  
32     hrp.CFrame = CFrame.new(where)  
33     player:SetAttribute("InLounge", inLounge)  
34 end  
35  
36 hatch.HatchPrompt.Triggered:Connect(function(player)  
37     move(player, ARRIVE, true)  
38 end)  
39 L.ExitPad.ExitPrompt.Triggered:Connect(function(player)  
40     move(player, RETURN_T0, false)  
41 end)  
42  
43 -- if someone respawns while flagged, clear the flag  
44 Players.PlayerAdded:Connect(function(p)  
45     p.CharacterAdded:Connect(function()  
46         if p:GetAttribute("InLounge") then p:SetAttribute("InLounge", false) end  
47     end)  
48 end)  
49
```

```

50 ----- jukebox
51 local queue, current, skips = {}, nil, {}
52
53 local function refreshPanel()
54     local lines = {"JUKEBOX"}
55     table.insert(lines, current and ("now playing: " .. current.name) or "nothing playing")
56     table.insert(lines, "queued: " .. #queue)
57     table.insert(lines, "")
58     table.insert(lines, "say /play <audio id>")
59     table.insert(lines, "hold E to vote skip")
60     npText.Text = table.concat(lines, "\n")
61 end
62
63 local function playNext()
64     skips = {}
65     local nxt = table.remove(queue, 1)
66     if not nxt then
67         current = nil
68         sound:Stop()
69         refreshPanel()
70         return
71     end
72     current = nxt
73     sound.SoundId = "rbxassetid://" .. nxt.id
74     sound.TimePosition = 0
75     sound:Play()
76     refreshPanel()
77 end
78
79 sound.Ended:Connect(playNext)
80 sound.Stopped:Connect(function()
81     if current and not sound.IsPlaying then
82         task.wait(0.5)
83         if not sound.IsPlaying then playNext() end
84     end
85 end)
86
87 juke.JukePrompt.Triggered:Connect(function(player)
88     if not current then return end
89     skips[player.UserId] = true
90     local n = 0
91     for _ in pairs(skips) do n = n + 1 end
92     local need = math.max(1, math.floor(#Players:GetPlayers() / 2))
93     if n >= need then
94         playNext()
95     else
96         npText.Text = string.format("JUKEBOX\nskip votes %d / %d", n, need)
97         task.delay(2, refreshPanel)
98     end
99 end)
100
101 ----- the wall
102 local notes = {}
103 local function refreshWall()
104     if #notes == 0 then noteBody.Text = "(no messages yet)" return end
105     local lines = {}
106     for i = #notes, math.max(1, #notes - 7), -1 do
107         table.insert(lines, notes[i])
108     end
109     noteBody.Text = table.concat(lines, "\n")
110 end
111
112 ----- chat commands
113 local function onChat(player, msg)
114     if not player:GetAttribute("InLounge") then return end
115     local lower = string.lower(msg)
116     local id = string.match(lower, "^/play%s+(%d+)")
117     if id then
118         if #queue >= 12 then return end
119         table.insert(queue, {id = id, name = player.Name .. "'s pick (" .. id .. ")"})
120         if not current then playNext() else refreshPanel() end
121         return
122     end
123     if lower == "/stop" then
124         queue = {}; current = nil; sound:Stop(); refreshPanel(); return
125     end

```

```

126     local note = string.match(msg, "^/note%s+(.+)")
127     if note then
128         if os.clock()-(noteCooldown[player] or -math.huge)<3 then return end
129         noteCooldown[player]=os.clock()
130         note=string.sub(note,1,90)
131         -- Shared signs must use the broadcast-filtered result, never raw chat text.
132         local ok,filtered=pcall(function()
133             return TextService:FilterStringAsync(note,player.UserId):GetNonChatStringForBroadcastAsync()
134         end)
135         if not ok or not player.Parent or not player:GetAttribute('InLounge') then return end
136         table.insert(notes,player.Name..":  "..filtered)
137         if #notes > 40 then table.remove(notes, 1) end
138         refreshWall()
139     end
140 end
141
142 local function hook(player) player.Chatted:Connect(function(m) onChat(player, m) end) end
143 for _, p in ipairs(Players:GetPlayers()) do hook(p) end
144 Players.PlayerAdded:Connect(hook)
145
146 ----- dance floor
147 local PALETTE = {
148     Color3.fromRGB(255, 80, 120), Color3.fromRGB(90, 200, 255),
149     Color3.fromRGB(255, 200, 90), Color3.fromRGB(150, 110, 255),
150     Color3.fromRGB(90, 255, 170),
151 }
152 task.spawn(function()
153     local tiles = L.DanceFloor:GetChildren()
154     local step = 0
155     while true do
156         step = step + 1
157         local beat = sound.IsPlaying and 0.35 or 0.9
158         for _, t in ipairs(tiles) do
159             local r = t:GetAttribute("Row") or 0
160             local c = t:GetAttribute("Col") or 0
161             local idx = ((r + c + step) % #PALETTE) + 1
162             TweenService:Create(t, TweenInfo.new(beat * 0.8), {Color = PALETTE[idx]}):Play()
163         end
164         task.wait(beat)
165     end
166 end)
167
168 refreshPanel()
169 refreshWall()
170 print("[LoungeSystems] hidden lounge online")
171
172 Players.PlayerRemoving:Connect(function(player) noteCooldown[player]=nil;skips[player.UserId]=nil end)

```

Chapter 43

S16 - ServerBlasterManager

ServerScriptService.ServerBlasterManager

Authority group: active. 8483 UTF-8 bytes; 195 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: ServerScriptService.ServerBlasterManager  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Authoritative shooting  
7 Future change: Add weapon configuration only through validated server-side attributes; keep sequence and rate  
  ↳ limits.  
8 Preserve dependencies: Blaster configuration, GWAPHitBridge, legacy impact bridge.  
9 Acceptance cases: Spoofed tool, out-of-order packet, impossible camera origin, wall obstruction, and correct  
  ↳ target damage.  
10 This revision changes documentation only in this source; executable input/aiming behavior is preserved.  
11 ]=====]  
12 -- ServerBlasterManager  
13 -- Authoritative validation, obstacle recasting, damage, and shot replication.  
14  
15 local Players = game:GetService("Players")  
16 local ReplicatedStorage = game:GetService("ReplicatedStorage")  
17 local ServerScriptService = game:GetService("ServerScriptService")  
18  
19 local modules = ReplicatedStorage:WaitForChild("Modules")  
20 local Blaster = require(modules:WaitForChild("Blaster"))  
21  
22 local remoteFolder = ReplicatedStorage:FindFirstChild(Blaster.REMOTE_FOLDER_NAME)  
23 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end  
24 if not remoteFolder then  
25     remoteFolder = Instance.new("Folder")  
26     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME  
27     remoteFolder.Parent = ReplicatedStorage  
28 end  
29  
30 local function getRemote(name)  
31     local remote = remoteFolder:FindFirstChild(name)  
32     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end  
33     if not remote then  
34         remote = Instance.new("RemoteEvent")  
35         remote.Name = name  
36         remote.Parent = remoteFolder  
37     end  
38     return remote  
39 end  
40  
41 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)  
42 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)  
43  
44 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")  
45 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end  
46 if not hitBridge then  
47     hitBridge = Instance.new("BindableFunction")  
48     hitBridge.Name = "BlasterHitBridge"
```

```

49     hitBridge.Parent = ServerScriptService
50 end
51
52 -- Roblox's default third-person zoom can legitimately place the camera well
53 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
54 -- zoom distances and the over-shoulder hip-fire camera.
55 local MAX_CAMERA_OFFSET = 40
56 local RATE_TOLERANCE = 0.88
57 local nextShotByPlayer = {}
58 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
59
60 local function findHumanoidFromHit(instance)
61     local node = instance
62     while node and node ~= workspace do
63         if node:IsA("Model") then
64             local humanoid = node:FindFirstChildOfClass("Humanoid")
65             if humanoid then return humanoid, node end
66         end
67         node = node.Parent
68     end
69     return nil
70 end
71
72 local function randomSpread(direction, spread)
73     if spread <= 0 then return direction end
74     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
75     local right = direction:Cross(reference).Unit
76     local up = right:Cross(direction).Unit
77     local radius = math.sqrt(math.random()) * spread
78     local angle = math.random() * math.pi * 2
79     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
80 end
81
82 local function broadcastShot(shooter, origin, position, normal, color)
83     local payload = {
84         origin = origin,
85         position = position,
86         normal = normal,
87         color = color,
88     }
89     for _, other in ipairs(Players:GetPlayers()) do
90         if other ~= shooter then effectRemote:FireClient(other, payload) end
91     end
92 end
93
94
95 local function researchHit(player, result, damage)
96     local researchBridge = ServerScriptService:FindFirstChild("GWAPHitBridge")
97     if not researchBridge then return false end
98     local ok, handled = pcall(function() return researchBridge:Invoke(player, result.Instance, damage) end)
99     return ok and handled == true
100 end
101
102 local function bridgeHit(player, result, damage, instantHeadshot)
103     if not result then return false end
104     if researchHit(player, result, instantHeadshot and result.Instance.Name == "Head" and 1000 or damage) then
105         ↪ return true end
106     -- BindableFunction callbacks are write-only. LabEquipment binds this during
107     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
108     local ok, response = pcall(function()
109         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
110             instantHeadshot and result.Instance.Name == "Head")
111     end)
112     return ok and response == true
113 end
114
115 local function genericDamage(player, result, config)
116     local humanoid, model = findHumanoidFromHit(result.Instance)
117     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
118     local victimPlayer = Players:GetPlayerFromCharacter(model)
119     if victimPlayer and not config.allowPlayerDamage then return end
120
121     local damage = config.damage
122     if result.Instance.Name == "Head" then
123         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
124     end

```

```

124     humanoid:TakeDamage(damage)
125 end
126
127 local function validSequence(tool, sequence)
128     if typeof(sequence) ~= "number" or sequence ~= sequence % 1 ~= 0 then return false end
129     if sequence < 1 or sequence > 2147483647 then return false end
130     local previous = lastSequenceByTool[tool]
131     if previous and sequence <= previous then return false end
132     lastSequenceByTool[tool] = sequence
133     return true
134 end
135
136 local function handleFire(player, packet)
137     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
138     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
139         ↪ return end
140
141     local character = player.Character
142     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
143     local head = character and character:FindFirstChild("Head")
144     local root = character and character:FindFirstChild("HumanoidRootPart")
145     local tool = packet.tool
146     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
147     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
148     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
149
150     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
151     local directionMagnitude = packet.direction.Magnitude
152     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
153     local aimDirection = packet.direction.Unit
154     if not validSequence(tool, packet.sequence) then return end
155
156     local config = Blaster.GetWeaponConfig(tool)
157     local now = os.clock()
158     if now < (nextShotByPlayer[player] or 0) then return end
159     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
160
161     local params = Blaster.NewRaycastParams({character, tool})
162     local handle = tool:FindFirstChild("Handle")
163     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
164     local tracerColor = tool:GetAttribute("BlasterTracerColor")
165     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
166
167     for _ = 1, config.pellets do
168         local pelletDirection = randomSpread(aimDirection, config.spread)
169         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
170         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
171             ↪ config.range)
172         local muzzleDelta = aimPoint - muzzleOrigin
173         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection
174         local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
175         local result = workspace:Raycast(muzzleOrigin, castDirection * castLength, params)
176         local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
177
178         broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
179         if result then
180             local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
181             if not handled then genericDamage(player, result, config) end
182         end
183     end
184
185     if config.selfCostPercent > 0 and humanoid.Health > 0 then
186         humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
187     end
188 end
189
190 fireRemote.OnServerEvent:Connect(handleFire)
191 Players.PlayerRemoving:Connect(function(player)
192     nextShotByPlayer[player] = nil
193 end)
194
195 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")

```


Chapter 44

S25 - LabHUD

StarterPlayer.StarterPlayerScripts.LabHUD

Authority group: active. 12852 UTF-8 bytes; 140 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====]
2  Revision 4 engineering reference | 13 September 2026
3  Authority: StarterPlayer.StarterPlayerScripts.LabHUD
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5  Do not require the documentation module from gameplay or replicate its prose to clients.
6  Purpose: Grouped mission display
7  Future change: Centralize notice priority and reserved screen regions. Preserve original backpack and combat
   ↳ controls.
8  Preserve dependencies: GWAP attributes, original notices, quiz modal, joystick/jump bounds.
9  Acceptance cases: Small portrait/landscape, long text, two-digit stages, chat open, quiz open, and
   ↳ keyboard/gamepad focus.
10 ]=====]
11 -- Two grouped panels. Native inventory and movement/fire controls remain available.
12 local Players=game:GetService('Players')
13 local RS=game:GetService('ReplicatedStorage')
14 local Run=game:GetService('RunService')
15 local StarterGui=game:GetService('StarterGui')
16 local prompts=game:GetService('ProximityPromptService')
17 local p=Players.LocalPlayer; local pg=p:WaitForChild('PlayerGui')
18 local scenarios=require(RS:WaitForChild('GWAPScenarios')); local remote=RS:WaitForChild('GWAPAction')
19 local research=workspace:WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch')
20 local gui=Instance.new('ScreenGui'); gui.Name='GWAPHUD'; gui.ResetOnSpawn=false; gui.IgnoreGuiInset=false;
   ↳ gui.DisplayOrder=25; gui.Parent=pg
21 pcall(function() StarterGui:SetCoreGuiEnabled(Enum.CoreGuiType.Health,false) end)
22 local white=Color3.fromRGB(239,247,243)
23 local function box(name,parent)
24     local f=Instance.new('Frame'); f.Name=name; f.BackgroundColor3=Color3.fromRGB(23,31,37);
   ↳ f.BackgroundTransparency=.12; f.BorderSizePixel=0; f.Parent=parent or gui;
   ↳ Instance.new('UICorner',f).CornerRadius=UDim.new(0,8); return f
25 end
26 local function label(parent,name,size)
27     local t=Instance.new('TextLabel'); t.Name=name; t.BackgroundColor3=white; t.Font=Enum.Font.GothamMedium;
   ↳ t.TextColor3=white; t.TextSize=size or 13; t.TextWrapped=true; t.TextTruncate=Enum.TextTruncate.AtEnd;
   ↳ t.TextXAlignment=Enum.TextXAlignment.Left; t.TextYAlignment=Enum.TextYAlignment.Center;
   ↳ t.Parent=parent; return t
28 end
29 local function button(parent,name,callback)
30     local t=Instance.new('TextButton'); t.Name=name; t.Text=name; t.Font=Enum.Font.GothamBold;
   ↳ t.TextColor3=white; t.TextSize=11; t.BackgroundColor3=Color3.fromRGB(63,73,91); t.BorderSizePixel=0;
   ↳ t.Parent=parent; Instance.new('UICorner',t).CornerRadius=UDim.new(0,5); t.Activated:Connect(callback);
   ↳ return t
31 end
32 local strip=box('Round and player'); strip.AnchorPoint=Vector2.new(.5,0)
33 local teams=label(strip,'Team scores',13); teams.TextXAlignment=Enum.TextXAlignment.Center
34 local timer=label(strip,'Timer',18); timer.TextXAlignment=Enum.TextXAlignment.Center
35 local personal=label(strip,'Player',12); personal.TextXAlignment=Enum.TextXAlignment.Right
36 local health=box('Health',strip); health.BackgroundColor3=Color3.fromRGB(54,66,61)
37 local fill=box('Fill',health); fill.BackgroundColor3=Color3.fromRGB(100,235,68); fill.BackgroundTransparency=0;
   ↳ fill.Size=UDim2.fromScale(1,1)
38 local panel=box('Mission and conversation')
```

```

39 local heading=label(panel,'Heading',13); heading.Font=Enum.Font.GothamBold
40 local target=label(panel,'Destination',12); target.TextColor3=Color3.fromRGB(151,238,204)
41 local dialogue=label(panel,'Carrier message',12); dialogue.TextYAlignment=Enum.TextYAlignment.Top
42 local note=label(panel,'Status',10); note.TextColor3=Color3.fromRGB(216,202,158);
  ↳ note.TextTruncate=Enum.TextTruncate.AtEnd; note.TextWrapped=false
43 local menu=box('Scenario selection'); menu.AnchorPoint=Vector2.new(.5,.5); menu.Visible=false; menu.ZIndex=10
44 local function toggle(value) menu.Visible=value; panel.Visible=not value; strip.Visible=not value;
  ↳ prompts.Enabled=not value end
45 local selectButton=button(panel,'Jobs',function() toggle(true) end)
46 local buttons={}
47 for i,name in ipairs({'Ready','Wait','Status','Regroup'}) do buttons[i]=button(panel,name,function()
  ↳ remote:FireServer('reply',i) end) end
48 local menuHeading=label(menu,'Select assignment',16); menuHeading.Text='CHOOSE A RESEARCH ASSIGNMENT';
  ↳ menuHeading.Position=UDim2.fromOffset(12,6); menuHeading.Size=UDim2.new(1,-58,0,28)
49 local close=button(menu,'X',function() toggle(false) end); close.Position=UDim2.new(1,-38,0,6);
  ↳ close.Size=UDim2.fromOffset(30,28)
50 local list=Instance.new('ScrollingFrame'); list.Name='Assignments'; list.Position=UDim2.fromOffset(10,42);
  ↳ list.Size=UDim2.new(1,-20,1,-52); list.BackgroundTransparency=1; list.BorderSizePixel=0;
  ↳ list.ScrollBarThickness=5; list.CanvasSize=UDim2.fromOffset(0,11*46); list.Parent=menu
51 local ids={}; for i=1,10 do ids[i]=i end; local random=Random.new(); for i=10,2,-1 do local
  ↳ j=random:NextInteger(1,i); ids[i],ids[j]=ids[j],ids[i] end
52 for row,id in ipairs(ids) do local b=button(list,scenarios[id].title,function() remote:FireServer('choose',id);
  ↳ toggle(false) end); b.Position=UDim2.fromOffset(0,(row-1)*46); b.Size=UDim2.new(1,-8,0,40); b.TextSize=13
  ↳ end
53 local randomButton=button(list,'Random assignment',function()
  ↳ remote:FireServer('choose',random:NextInteger(1,10)); toggle(false) end);
  ↳ randomButton.Position=UDim2.fromOffset(0,460); randomButton.Size=UDim2.new(1,-8,0,40)
54 local trail=Instance.new('Folder'); trail.Name='LocalMissionTrail'; trail.Parent=workspace
55 local dots={}
56 for i=1,90 do local dot=Instance.new('Part'); dot.Name='Direction'; dot.Size=Vector3.new(.16,.12,1.6);
  ↳ dot.Anchored=true; dot.CanCollide=false; dot.CanQuery=false; dot.CanTouch=false;
  ↳ dot.Material=Enum.Material.Neon; dot.Color=Color3.fromRGB(103,255,198); dot.Transparency=1;
  ↳ dot.Parent=trail; dots[i]=dot end
57 local routeCache=setmetatable({}, {__mode='k'})
58 local function showTrail(bot)
59   for _,v in ipairs(dots) do v.Transparency=1 end
60   local route=bot and research.Routes:FindFirstChild(bot.Name); local root=p.Character and
  ↳ p.Character:FindFirstChild('HumanoidRootPart')
61   if not bot or not route or not root or bot:GetAttribute('State')~='RUNNING' then return end
62   local record=routeCache[route]
63   if not record then record={}; routeCache[route]=record; route.ChildAdded:Connect(function() record.nodes=nil
  ↳ end); route.ChildRemoved:Connect(function() record.nodes=nil end) end
64   local nodes=record.nodes
65   if not nodes then nodes=route:GetChildren(); table.sort(nodes,function(a,b) return a.Name<b.Name
  ↳ end); record.nodes=nodes end
66   local previous=bot:GetPivot().Position; local n=0
67   for j=bot:GetAttribute('RouteIndex') or 1,#nodes do
68     local to=nodes[j].Value; local delta=to-previous
69     if delta.Magnitude>.1 then for d=0,delta.Magnitude,4 do
70       local pos=previous+delta.Unit*d-Vector3.new(0,2.9,0)
71       if (pos-root.Position).Magnitude<110 and math.abs(pos.Y-root.Position.Y)<10 then n+=1; if n>#dots
  ↳ then return end; dots[n].CFRAME=CFRAME.lookAt(pos,pos+delta); dots[n].Transparency=.12 end
72     end end; previous=to
73   end
74 end
75 prompts.PromptShown:Connect(function(prompt)
76   local viewport=workspace.CurrentCamera.ViewportSize
77   prompt.UIOffset=Vector2.new(viewport.X<1000 and 70 or 0,0)
78 end)
79 local billboards=setmetatable({}, {__mode='k'})
80 local function watchBot(bot)
81   if billboards[bot] then return end
82   local items={}; for _,v in ipairs(bot:GetDescendants()) do if v:IsA('BillboardGui') then
  ↳ table.insert(items,v) end end; billboards[bot]=items
83   bot.DescendantAdded:Connect(function(v) if v:IsA('BillboardGui') then table.insert(items,v) end end)
84 end
85 local cs=game.GetService('CollectionService')
86 for _,bot in ipairs(cs:GetTagged('LabWorkerBot')) do watchBot(bot) end
87 for _,bot in ipairs(research.Carriers:GetChildren()) do watchBot(bot) end
88 research.Carriers.ChildAdded:Connect(watchBot)
89 local elapsed=0
90 Run.RenderStepped:Connect(function(dt)
91   elapsed+=dt; if elapsed<.15 then return end; elapsed=0
92   local v=workspace.CurrentCamera.ViewportSize; local mobile=v.X<1000 or v.Y<500
93   local width=math.min(mobile and 180 or 290,v.X*.25); local height=mobile and 150 or 218
94   local scale=math.min(1,(v.Y-95)/height); scale=math.max(.6,scale)

```

```

95     local touch=pg:FindFirstChild('TouchGui'); local thumb=touch and
    ↪ touch:FindFirstChild('ThumbstickStart',true)
96     if mobile and thumb and thumb.Visible then
    ↪     scale=math.min(scale,math.max(.6,(thumb.AbsolutePosition.Y-48-8)/height)) end
97     local ui=panel:FindFirstChildOfClass('UIScale') or Instance.new('UIScale',panel); ui.Scale=scale
98     panel.Position=UDim2.fromOffset(8,48); panel.Size=UDim2.fromOffset(width/scale,height)
99     local w=width/scale
100    heading.Position=UDim2.fromOffset(8,6); heading.Size=UDim2.fromOffset(w-59,mobile and 25 or 36);
    ↪ heading.TextSize=mobile and 11 or 15
101    selectButton.Position=UDim2.fromOffset(w-46,7); selectButton.Size=UDim2.fromOffset(38,24)
102    target.Position=UDim2.fromOffset(8,mobile and 33 or 46); target.Size=UDim2.fromOffset(w-16,mobile and 26 or
    ↪ 34); target.TextSize=mobile and 10 or 13
103    dialogue.Position=UDim2.fromOffset(8,mobile and 61 or 84); dialogue.Size=UDim2.fromOffset(w-16,mobile and
    ↪ 45 or 69); dialogue.TextSize=mobile and 10 or 13
104    for i,b in ipairs(buttons) do b.Position=UDim2.fromOffset(8+(i-1)*(w-16)/4,height-43);
    ↪ b.Size=UDim2.fromOffset((w-20)/4-2,23); b.TextSize=mobile and 9 or 11 end
105    note.Position=UDim2.fromOffset(8,height-18); note.Size=UDim2.fromOffset(w-16,14); note.TextSize=mobile and
    ↪ 9 or 11
106    local sw=math.min(460,v.X-26); strip.Position=UDim2.new(.5,0,0,3); strip.Size=UDim2.fromOffset(sw,38)
107    teams.Position=UDim2.fromOffset(7,3); teams.Size=UDim2.fromOffset(sw*.35,30); teams.TextSize=mobile and 10
    ↪ or 13
108    timer.Position=UDim2.fromOffset(sw*.38,0); timer.Size=UDim2.fromOffset(sw*.19,35)
109    personal.Position=UDim2.fromOffset(sw*.59,0); personal.Size=UDim2.fromOffset(sw*.39-8,23);
    ↪ personal.TextSize=mobile and 10 or 12
110    health.Position=UDim2.fromOffset(sw*.61,26); health.Size=UDim2.fromOffset(sw*.35,6)
111    menu.Position=UDim2.fromScale(.5,.5);
    ↪ menu.Size=UDim2.fromOffset(math.min(430,v.X-30),math.max(150,math.min(520,v.Y-80)))
112    local own=p.Character and p.Character:FindFirstChild('LabHealthBar',true); if own then own.Enabled=false
    ↪ end
113    for _,name in ipairs({'RoleHUD','LabLog'}) do local old=pg:FindFirstChild(name); if old then
    ↪ old.Enabled=false end end
114    local sums={Technician=0,['Laboratory Technician']=0}
115    for _,player in ipairs(Players:GetPlayers()) do local role=player:GetAttribute('LabRole'); local
    ↪ stats=player:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); if
    ↪ sums[role] then sums[role]+=value and value.Value or 0 end end
116    teams.Text='PROTECT '..sums.Technician..' | STOP '..sums['Laboratory Technician']
117    local remaining=math.max(0,(workspace:GetAttribute('RoundEndsAt') or os.time()-os.time()));
    ↪ timer.Text=string.format('%d:%02d',remaining//60,remaining%60)
118    local stats=p:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); local
    ↪ hum=p.Character and p.Character:FindFirstChildOfClass('Humanoid')
119    personal.Text=(value and value.Value or 0)..' pts | '..math.ceil(hum and hum.Health or 0)..' HP';
    ↪ fill.Size=UDim2.fromScale(hum and math.clamp(hum.Health/math.max(1,hum.MaxHealth),0,1) or 0,1)
120    local id=p:GetAttribute('GWAPScenario'); local s=id and scenarios[id]; local
    ↪ name=p:GetAttribute('GWAPCarrier'); local bot=name and research.Carriers:FindFirstChild(name)
121    heading.Text=s and s.title or 'RESEARCH ASSIGNMENT'
122    local floor=p:GetAttribute('GWAPTargetFloor'); local location=floor and (floor==8 and 'R00F' or floor==1
    ↪ and 'B1' or 'L'..(floor+1)) or ''
123    target.Text=s and ((p:GetAttribute('GWAPTarget') or '')..' '..location) or 'Choose a role kiosk, then tap
    ↪ Jobs.'
124    if bot and s then target.Text.=' | NPC '..math.ceil(bot:GetAttribute('Health') or 100)..' HP' end
125    dialogue.Text=p:GetAttribute('GWAPDialogue') or 'Your carrier will meet you nearby and wait for your Ready
    ↪ reply.'
126    local carried=p:GetAttribute('CarriedSample') or ''; local notice=p:GetAttribute('GWAPNotice') or ''
127    note.Text=carried~='' and 'Carrying: '..carried or notice
128    local cam=workspace.CurrentCamera; local params=RaycastParams.new();
    ↪ params.FilterType=Enum.RaycastFilterType.Exclude; params.FilterDescendantsInstances={p.Character}
129    local aim=workspace:Raycast(cam.CFrame.Position,cam.CFrame.LookVector*80,params)
130    for npc,items in pairs(billboards) do
131        local projected,onScreen=cam:WorldToViewportPoint(npc:GetPivot().Position+Vector3.new(0,5,0))
132        local show=npc.Parent and npc~=bot and onScreen and projected.Y>120 and (aim and
    ↪ aim.Instance:IsDescendantOf(npc)) and (npc:GetPivot().Position-cam.CFrame.Position).Magnitude<65
133        for _,billboard in ipairs(items) do if billboard.Parent then billboard.Enabled=show==true and
    ↪ billboard.Name~='MissionSpeech'; billboard.MaxDistance=65 end end
134    end
135    for _,station in ipairs(research.Stations:GetChildren()) do
136        local sign=station:FindFirstChild('StationLabel')
137        if sign then sign.Enabled=aim~=nil and (aim.Instance==station or aim.Instance:IsDescendantOf(station))
    ↪ and (station.Position-cam.CFrame.Position).Magnitude<40 end
138    end
139    showTrail(bot)
140 end)

```

Chapter 45

S26 - LabQuizClient

StarterPlayer.StarterPlayerScripts.LabQuizClient

Authority group: active. 5319 UTF-8 bytes; 138 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====[
2 Revision 4 engineering reference | 13 September 2026
3 Authority: StarterPlayer.StarterPlayerScripts.LabQuizClient
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.
5 Do not require the documentation module from gameplay or replicate its prose to clients.
6 Purpose: Quiz UI
7 Future change: Use an accessible modal with touch and keyboard choices and a clear timeout. Display server
  ↳ challenge data only.
8 Preserve dependencies: LabEquipment pendingQuizzes and MathQuizActive.
9 Acceptance cases: Wrong/late/replayed reply, cancel, death, target destruction, and overlap with GWAP HUD.
10 ]=====]
11
12 -- Addition popup: click/tap one of three answers. No phone keyboard or
13 -- changes to the player's camera, aiming, or movement controls are required.
14 local ReplicatedStorage = game:GetService("ReplicatedStorage")
15 local Players = game:GetService("Players")
16 local player = Players.LocalPlayer
17 local remote = ReplicatedStorage:WaitForChild("LabMathQuiz")
18 local playerGui = player:WaitForChild("PlayerGui")
19 local old = playerGui:FindFirstChild("LabMathConsole")
20 if old then old:Destroy() end
21
22 local gui = Instance.new("ScreenGui")
23 gui.Name = "LabMathConsole"
24 gui.ResetOnSpawn = false
25 gui.IgnoreGuiInset = false
26 gui.DisplayOrder = 60
27 pcall(function() gui.ScreenInsets = Enum.ScreenInsets.DeviceSafeInsets end)
28 gui.Parent = playerGui
29
30 local panel = Instance.new("Frame")
31 panel.Name = "AdditionQuiz"
32 panel.AnchorPoint = Vector2.new(0.5, 0.5)
33 panel.Position = UDim2.fromScale(0.5, 0.46)
34 panel.Size = UDim2.new(1, -28, 0, 238)
35 panel.BackgroundColor3 = Color3.fromRGB(14, 24, 30)
36 panel.BorderSizePixel = 0
37 panel.Active = true
38 panel.Visible = false
39 panel.Parent = gui
40 local sizeLimit = Instance.new("UISizeConstraint")
41 sizeLimit.MaxSize = Vector2.new(440, 238)
42 sizeLimit.Parent = panel
43 local corner = Instance.new("UICorner")
44 corner.CornerRadius = UDim.new(0, 14)
45 corner.Parent = panel
46 local stroke = Instance.new("UIStroke")
47 stroke.Color = Color3.fromRGB(95, 225, 170)
48 stroke.Thickness = 2
49 stroke.Parent = panel
```

```

50
51 local function label(name, y, height, size, colour)
52     local item = Instance.new("TextLabel")
53     item.Name = name
54     item.Size = UDim2.new(1, -28, 0, height)
55     item.Position = UDim2.fromOffset(14, y)
56     item.BackgroundTransparency = 1
57     item.TextColor3 = colour
58     item.TextSize = size
59     item.Font = Enum.Font.GothamBold
60     item.TextWrapped = true
61     item.Parent = panel
62     return item
63 end
64 local title = label("Title", 10, 26, 18, Color3.fromRGB(110, 240, 185))
65 title.Size = UDim2.new(1, -70, 0, 26)
66 title.Text = "ADDITION CHECK"
67 local reason = label("Task", 40, 34, 13, Color3.fromRGB(195, 213, 223))
68 local question = label("Question", 76, 45, 34, Color3.fromRGB(250, 250, 250))
69 local status = label("Status", 190, 34, 13, Color3.fromRGB(255, 210, 100))
70 local cancel = Instance.new("TextButton")
71 cancel.Name = "Cancel"
72 cancel.Position = UDim2.new(1, -42, 0, 6)
73 cancel.Size = UDim2.fromOffset(36, 36)
74 cancel.BackgroundTransparency = 1
75 cancel.Text = "x"
76 cancel.TextColor3 = Color3.fromRGB(220, 228, 232)
77 cancel.TextSize = 28
78 cancel.Modal = true -- Let the pointer select an answer even in first-person view.
79 cancel.Parent = panel
80
81 local activeId
82 local submitted = false
83 local buttons = {}
84 local function answer(value)
85     if not activeId or submitted then return end
86     submitted = true
87     status.Text = "Checking answer..."
88     remote.FireServer(activeId, value)
89 end
90 for i = 1, 3 do
91     local button = Instance.new("TextButton")
92     button.Name = "Answer" .. i
93     button.Position = UDim2.new((i - 1) / 3, 10, 0, 128)
94     button.Size = UDim2.new(1 / 3, -14, 0, 56)
95     button.BackgroundColor3 = Color3.fromRGB(50, 146, 112)
96     button.TextColor3 = Color3.fromRGB(255, 255, 255)
97     button.Font = Enum.Font.GothamBold
98     button.TextSize = 26
99     button.Parent = panel
100     local rounding = Instance.new("UICorner")
101     rounding.CornerRadius = UDim.new(0, 9)
102     rounding.Parent = button
103     button.Activated:Connect(function() answer(button:GetAttribute("Answer")) end)
104     buttons[i] = button
105 end
106 cancel.Activated:Connect(function() answer(nil) end)
107
108 remote.OnClientEvent:Connect(function(payload)
109     if typeof(payload) ~= "table" then return end
110     if payload.Close then
111         if payload.Id == activeId then panel.Visible = false; activeId = nil end
112         return
113     end
114     activeId = payload.Id
115     submitted = false
116     reason.Text = tostring(payload.Reason or "Repair or collect a plant")
117     question.Text = tostring(payload.Question or "?") .. " = ?"
118     for i, button in ipairs(buttons) do
119         local value = payload.Choices and payload.Choices[i]
120         button.Text = tostring(value or "?")
121         button:SetAttribute("Answer", value)
122     end
123     panel.Visible = true
124     local thisId = activeId
125     local expires = os.clock() + (tonumber(payload.Seconds) or 60)

```

```
126     task.spawn(function()
127         while activeId == thisId and panel.Visible do
128             local left = math.max(0, math.ceil(expires - os.clock()))
129             if not submitted then status.Text = "Tap the correct answer • " .. left .. "s • × cancels" end
130             if left <= 0 then
131                 if not submitted then answer(nil) end
132                 panel.Visible = false
133                 break
134             end
135             task.wait(0.2)
136         end
137     end)
138 end)
```

Chapter 46

S27 - LabSidearmClient

StarterPlayer.StarterPlayerScripts.LabSidearmClient

Authority group: active. 2409 UTF-8 bytes; 54 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Revision 4 engineering reference | 13 September 2026  
3  Authority: StarterPlayer.StarterPlayerScripts.LabSidearmClient  
4  Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5  Do not require the documentation module from gameplay or replicate its prose to clients.  
6  Purpose: Tool bootstrap  
7  Future change: Keep one controller instance per equipped original Tool and one active camera owner.  
8  Preserve dependencies: BlasterController AutoBind/Enable/Disable.  
9  Acceptance cases: Rapid equip, respawn, backpack replacement, duplicate script prevention.  
10 This revision changes documentation only in this source; executable input/aiming behavior is preserved.  
11 ]=====]  
12 local Players          = game:GetService("Players")  
13 local ReplicatedStorage = game:GetService("ReplicatedStorage")  
14  
15 local player = Players.LocalPlayer  
16  
17 -- The hunt counter belongs inside RoleHUD; a second top-centre panel covered  
18 -- the crosshair and duplicated information on phones.  
19 task.spawn(function()  
20     local complex = workspace:WaitForChild("Laboratory_Complex", 60)  
21     if not complex then return end  
22     local playerGui = player:WaitForChild("PlayerGui", 30)  
23     if not playerGui then return end  
24     local roleGui = playerGui:WaitForChild("RoleHUD", 30)  
25     local frame = roleGui and roleGui:FindFirstChildOfClass("Frame")  
26     local label = frame and frame:FindFirstChild("TargetsLabel")  
27     if not label then return end  
28  
29     local function refresh()  
30         local killed = complex:GetAttribute("BotsKilled") or 0  
31         local total = complex:GetAttribute("BotsTotal") or 0  
32         if total > 0 and killed >= total then  
33             label.Text = "LAB CLEARED • " .. killed .. " / " .. total  
34             label.TextColor3 = Color3.fromRGB(125, 255, 155)  
35         else  
36             label.Text = string.format("TARGETS %d / %d", killed, total)  
37             label.TextColor3 = Color3.fromRGB(255, 205, 120)  
38         end  
39     end  
40     complex:GetAttributeChangedSignal("BotsKilled"):Connect(refresh)  
41     complex:GetAttributeChangedSignal("BotsTotal"):Connect(refresh)  
42     refresh()  
43 end)  
44  
45 -- One lightweight bootstrap watches standard Tools as they move between the  
46 -- Backpack and Character. All device input/camera behavior lives in the module.  
47 local modules = ReplicatedStorage:WaitForChild("Modules", 30)  
48 if not modules then  
49     warn("Blaster modules were not installed")  
50     return  
51 end
```

```
52
53 local BlasterController = require(modules.WaitForChild("BlasterController"))
54 BlasterController.AutoBind()
```


Chapter 47

S28 - LoungeClient

StarterPlayer.StarterPlayerScripts.LoungeClient

Authority group: active. 3247 UTF-8 bytes; 85 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====  
2 Revision 4 engineering reference | 13 September 2026  
3 Authority: StarterPlayer.StarterPlayerScripts.LoungeClient  
4 Full design, defect history and extension specifications: ServerStorage.GameDocumentation.  
5 Do not require the documentation module from gameplay or replicate its prose to clients.  
6 Purpose: Lounge presentation  
7 Future change: Add local effects through lifecycle-managed connections and restore prior camera/UI state on  
  ↪ exit.  
8 Preserve dependencies: LoungeSystems and player character lifecycle.  
9 Acceptance cases: Enter/exit, respawn inside, missing props, and device input.  
10 ]=====]  
11 -- LoungeClient: while you are downstairs, the lab stops shouting at you.  
12 local Players = game:GetService("Players")  
13 local player = Players.LocalPlayer  
14 local pg = player:WaitForChild("PlayerGui")  
15  
16 local KEEP = {LabHUD = true, LoungeUI = true}  
17 local hidden = {}  
18  
19 local ui = Instance.new("ScreenGui")  
20 ui.Name = "LoungeUI"; ui.ResetOnSpawn = false; ui.IgnoreGuiInset = true  
21 ui.Enabled = false; ui.Parent = pg  
22  
23 local card = Instance.new("Frame")  
24 card.Size = UDim2.new(0, 300, 0, 128)  
25 card.Position = UDim2.new(0.5, 0, 1, -22)  
26 card.AnchorPoint = Vector2.new(0.5, 1)  
27 card.BackgroundColor3 = Color3.fromRGB(20, 18, 26)  
28 card.BackgroundTransparency = 0.12  
29 card.BorderSizePixel = 0; card.Parent = ui  
30 local c = Instance.new("UICorner"); c.CornerRadius = UDim.new(0, 12); c.Parent = card  
31 local st = Instance.new("UIStroke"); st.Color = Color3.fromRGB(150, 110, 200)  
32 st.Thickness = 1.5; st.Transparency = 0.3; st.Parent = card  
33  
34 local head = Instance.new("TextLabel")  
35 head.Size = UDim2.new(1, 0, 0, 26); head.Position = UDim2.new(0, 0, 0, 8)  
36 head.BackgroundTransparency = 1; head.Font = Enum.Font.GothamBold; head.TextSize = 15  
37 head.TextColor3 = Color3.fromRGB(210, 170, 255); head.Text = "OFF SHIFT - no alarms down here"  
38 head.Parent = card  
39  
40 local body = Instance.new("TextLabel")  
41 body.Size = UDim2.new(1, -22, 1, -40); body.Position = UDim2.new(0, 11, 0, 32)  
42 body.BackgroundTransparency = 1; body.Font = Enum.Font.Gotham; body.TextSize = 13  
43 body.TextColor3 = Color3.fromRGB(215, 218, 224)  
44 body.TextXAlignment = Enum.TextXAlignment.Left; body.TextYAlignment = Enum.TextYAlignment.Top  
45 body.TextWrapped = true  
46 body.Text = "/play <audio id> queue a track\n/note <message> post on the wall\n/nstop clear the queue\n/nHold E  
  ↪ at the jukebox to vote skip\n/nSit down - click a sofa or stool"  
47 body.Parent = card  
48
```

```
49 local function apply(inLounge)
50     ui.Enabled = inLounge
51     if inLounge then
52         for _, g in ipairs(pg:GetChildren()) do
53             if g:IsA("ScreenGui") and not KEEP[g.Name] and g.Enabled then
54                 hidden[g] = true
55                 g.Enabled = false
56             end
57         end
58     else
59         for g in pairs(hidden) do
60             if g and g.Parent then g.Enabled = true end
61         end
62         hidden = {}
63     end
64 end
65
66 player:GetAttributeChangedSignal("InLounge"):Connect(function()
67     apply(player:GetAttribute("InLounge") == true)
68 end)
69
70 -- catch anything the lab spawns while we are downstairs
71 task.spawn(function()
72     while true do
73         task.wait(1)
74         if player:GetAttribute("InLounge") then
75             for _, g in ipairs(pg:GetChildren()) do
76                 if g:IsA("ScreenGui") and not KEEP[g.Name] and g.Enabled then
77                     hidden[g] = true
78                     g.Enabled = false
79                 end
80             end
81         end
82     end
83 end)
84
85 apply(player:GetAttribute("InLounge") == true)
```

Chapter 48

S21 - LabSpatial

ReplicatedStorage.LabSpatial

Authority group: active. 1994 UTF-8 bytes; 39 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 -- Laboratory coordinate contract (revision 4).
2 -- Generator coordinates are local to the lab, never assumed to be world coordinates.
3 -- Roof_1 is an existing stable landmark; no new world geometry or player controls.
4 -- ApproachPoint attachments follow individual stations and whole-building moves.
5 local S = {}
6 local BASE_ROOF = CFrame.new(82.28571428571429, 129.375, 133.71428571428572)
7 function S.frame()
8     local complex = workspace:FindFirstChild('Laboratory_Complex')
9     local roof = complex and complex:FindFirstChild('Roof_1')
10    return roof and roof.CFrame * BASE_ROOF:Inverse() or CFrame.identity
11 end
12 function S.world(point) return S.frame():PointToWorldSpace(point) end
13 function S.localPoint(point) return S.frame():PointToObjectSpace(point) end
14 function S.vector(vector) return S.frame():VectorToWorldSpace(vector) end
15 function S.floor(point, standingOffset)
16     return math.clamp(math.floor((S.localPoint(point).Y-(standingOffset or 0))/16.25+.5),-1,8)
17 end
18 function S.setApproach(station, localPoint)
19     station:SetAttribute('ApproachLocal',localPoint)
20     local marker=station:FindFirstChild('ApproachPoint')
21     if not marker then marker=Instance.new('Attachment');marker.Name='ApproachPoint';marker.Parent=station end
22     marker.Position=station.CFrame:PointToObjectSpace(S.world(localPoint))
23     station:SetAttribute('Approach',marker.WorldPosition)
24     return marker.WorldPosition
25 end
26 function S.approach(station)
27     local marker=station:FindFirstChild('ApproachPoint')
28     if marker and marker:IsA('Attachment') then return marker.WorldPosition end
29     local point=station:GetAttribute('ApproachLocal') or station:GetAttribute('Approach')
30     return point and S.setApproach(station,point) or station.Position
31 end
32 function S.sync(stations)
33     for _,station in ipairs(stations:GetChildren()) do
34         if station:IsA('BasePart') and station:GetAttribute('Approach') then
35             station:SetAttribute('Approach',S.approach(station))
36         end
37     end
38 end
39 return S
```

Part III

Complete archived Studio source

Chapter 49

S17 - LabElevators

ServerStorage.BeforeBasementAndQuiz_20260912.LabElevators

Authority group: archive. 3761 UTF-8 bytes; 123 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LabElevators : moving cars that carry whoever is standing inside.
2 -- An anchored part does not drag a character with it, so the car
3 -- offsets every occupant by the same delta on each step.
4
5 local CollectionService = game:GetService("CollectionService")
6 local RunService        = game:GetService("RunService")
7 local Players            = game:GetService("Players")
8
9 local SPEED = 45  -- studs per second
10
11 local function setGate(gate, open)
12     gate.CanCollide = not open
13     gate.Transparency = open and 0.85 or 0
14 end
15
16 local function setup(model)
17     local floorCount = model:GetAttribute("FloorCount")
18     if not floorCount then return end
19
20     local car = model:WaitForChild("Car")
21     local platform = car:FindFirstChild("Platform") or car:FindFirstChild("Platform_1")
22     if not platform then
23         for _, d in ipairs(car:GetDescendants()) do
24             if d:IsA("BasePart") and d.Name:match("^Platform") then platform = d break end
25         end
26     end
27     if not platform then return end
28
29     local gates = {}
30     for _, g in ipairs(model:WaitForChild("Gates"):GetChildren()) do
31         gates[g:GetAttribute("Floor")] = g
32     end
33
34     local ys = {}
35     for i = 1, floorCount do ys[i] = model:GetAttribute("FloorY_" .. i) end
36
37     local current, moving = 1, false
38     for i, g in pairs(gates) do setGate(g, i == current) end
39
40     local carParts = {}
41     for _, d in ipairs(car:GetDescendants()) do
42         if d:IsA("BasePart") then table.insert(carParts, d) end
43     end
44
45     local function occupants()
46         local list = {}
47         local half = platform.Size * 0.5
48         local top = platform.Position.Y + half.Y
```

```

49     for _, pl in ipairs(Players:GetPlayers()) do
50         local ch = pl.Character
51         local hrp = ch and ch:FindFirstChild("HumanoidRootPart")
52         if hrp then
53             local d = hrp.Position - platform.Position
54             if math.abs(d.X) <= half.X + 2 and math.abs(d.Z) <= half.Z + 2
55                 and hrp.Position.Y > top - 2 and hrp.Position.Y < top + 24 then
56                 table.insert(list, hrp)
57             end
58         end
59     end
60     return list
61 end
62
63 local function moveTo(target)
64     if moving or target == current or not ys[target] then return end
65     moving = true
66     setGate(gates[current], false)
67
68     local startY = platform.Position.Y
69     local goalY = startY + (ys[target] - ys[current])
70     local dist = math.abs(goalY - startY)
71     local dur = math.max(0.4, dist / SPEED)
72     local t0 = os.clock()
73     local lastY = startY
74
75     while true do
76         local a = math.min(1, (os.clock() - t0) / dur)
77         local newY = startY + (goalY - startY) * a
78         local dy = newY - lastY
79         if dy ~= 0 then
80             local riders = occupants()
81             for _, p in ipairs(carParts) do
82                 p.CFrame = p.CFrame + Vector3.new(0, dy, 0)
83             end
84             for _, hrp in ipairs(riders) do
85                 hrp.CFrame = hrp.CFrame + Vector3.new(0, dy, 0)
86             end
87             lastY = newY
88         end
89         if a >= 1 then break end
90         RunService.Heartbeat:Wait()
91     end
92
93     current = target
94     model:SetAttribute("CurrentFloor", current)
95     setGate(gates[current], true)
96     moving = false
97 end
98
99 local function addPrompt(part, floor, label, text)
100     local p = Instance.new("ProximityPrompt")
101     p.ObjectText = label
102     p.ActionText = text
103     p.HoldDuration = 0.15
104     p.MaxActivationDistance = 22
105     p.RequiresLineOfSight = false
106     p.Parent = part
107     p.Triggered:Connect(function() moveTo(floor) end)
108 end
109
110 for _, panel in ipairs(model:WaitForChild("CallPanels"):GetChildren()) do
111     addPrompt(panel, panel:GetAttribute("Floor"), "Elevator", "Call")
112 end
113 for _, btn in ipairs(car:WaitForChild("Buttons"):GetChildren()) do
114     local i = btn:GetAttribute("Floor")
115     addPrompt(btn, i, "Elevator", "Go to floor " .. (model:GetAttribute("FloorLabel_" .. i) or i))
116 end
117 end
118
119 for _, m in ipairs(CollectionService:GetTagged("Elevator")) do setup(m) end
120 CollectionService.GetInstanceAddedSignal("Elevator"):Connect(setup)
121
122 print("Elevators armed: " .. #CollectionService:GetTagged("Elevator") .. ".")

```

Chapter 50

S18 - LabEquipment

ServerStorage.BeforeBasementAndQuiz_20260912.LabEquipment

Authority group: archive. 175504 UTF-8 bytes; 4068 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LabEquipment : physical controls, equipment readouts, carried samples,
2 -- cold-store delivery missions, and non-lethal sample recovery tools.
3 -- All inventory and hit decisions happen on the server.
4
5 local ReplicatedStorage = game:GetService("ReplicatedStorage")
6 local CollectionService = game:GetService("CollectionService")
7 local Players           = game:GetService("Players")
8 local TweenService      = game:GetService("TweenService")
9 local Debris            = game:GetService("Debris")
10 local PathfindingService = game:GetService("PathfindingService")
11
12 local Kinds = require(ReplicatedStorage:WaitForChild("LabKinds"))
13 local QuizRemote = ReplicatedStorage:FindFirstChild("LabMathQuiz")
14 if not QuizRemote then
15     QuizRemote = Instance.new("RemoteFunction")
16     QuizRemote.Name = "LabMathQuiz"
17     QuizRemote.Parent = ReplicatedStorage
18 end
19
20 -----
21 -- SHARED GAME STATE
22 -----
23 local GameState = {
24     repairsCompleted = 0,
25     assignedRepairsCompleted = 0,
26     transfersCompleted = 0,
27     assignedTransfersCompleted = 0,
28 }
29
30 local ROLE_STATIONS = {}
31
32 local POINTS = {
33     MOVE_ANY = 12,
34     MOVE_ASSIGNED_LATE = 18,
35     MOVE_ASSIGNED_ON_TIME = 30,
36     PROCESS_PLANT = 4,
37 }
38
39 local function updateScoreboard()
40     local board = CollectionService:GetTagged("BatchScoreboard")[1]
41     if not board then return end
42     local gui = board:FindFirstChildWhichIsA("BillboardGui")
43     local label = gui and gui:FindFirstChild("Text")
44     if label then
45         local rivalScore = 0
46         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
47             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
```

```

48     end
49     label.Text = string.format(
50         "PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d • assigned: %d\nRival NPC
        ↳ score: %d",
51         GameState.repairsCompleted, GameState.assignedRepairsCompleted,
52         GameState.transfersCompleted, GameState.assignedTransfersCompleted, rivalScore)
53     end
54 end
55
56 -- Per-room equipment status monitors. Each RoomMonitor panel carries a
57 -- Room attribute matching the LabEquipment parts placed in that same room,
58 -- so this just groups by that attribute and tallies running/failed/plants.
59 local function refreshRoomMonitors()
60     -- Room names ("Room 4", etc.) repeat on every level, so the key here
61     -- must include the level too -- otherwise a monitor sums up every
62     -- same-numbered room across the whole building instead of just its own.
63     local totals = {}
64     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
65         local room = equip:GetAttribute("Room")
66         local level = equip:GetAttribute("Level") or ""
67         if room then
68             local key = level .. "|" .. room
69             local t = totals[key]
70             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
71             t.count = t.count + 1
72             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
73             else t.running = t.running + 1 end
74             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
75         end
76     end
77     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
78         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
79         local label = surface and surface:FindFirstChild("Status")
80         if label then
81             local room = panel:GetAttribute("Room") or "ROOM"
82             local level = panel:GetAttribute("Level") or ""
83             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
84             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
85                 room, t.count, t.running, t.failed, t.plants)
86             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235, 170)
87         end
88     end
89 end
90
91 local function addContribution(player, amount)
92     local stats = player:FindFirstChild("leaderstats")
93     local value = stats and stats:FindFirstChild("Contributions")
94     if value then value.Value = value.Value + (amount or 1) end
95 end
96
97 local facilityNoticeAt = setmetatable({}, {__mode = "k"})
98 local function notify(player, text)
99     local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
        ↳ text:match("^CATASTROPHIC FAILURE:")
100     if facilityAlert then
101         local now = os.clock()
102         if now - (facilityNoticeAt[player] or -math.huge) < 6 then return end
103         facilityNoticeAt[player] = now
104     end
105     print("[ " .. player.Name .. " ] " .. text)
106     local gui = player:FindFirstChild("PlayerGui")
107     if not gui then return end
108     local screen = gui:FindFirstChild("LabLog")
109     if not screen then
110         screen = Instance.new("ScreenGui")
111         screen.Name = "LabLog"
112         screen.ResetOnSpawn = false
113         screen.DisplayOrder = 40
114         screen.Parent = gui
115     end
116     local frame = Instance.new("TextLabel")
117     frame.Name = "Line"
118     frame.Size = UDim2.new(0.48, 0, 0, 46)
119     frame.AnchorPoint = Vector2.new(0.5, 1)
120     frame.Position = UDim2.new(0.5, 0, 1, -68)
121     frame.BackgroundColor3 = Color3.fromRGB(18, 22, 28)

```



```

122     frame.BackgroundTransparency = 0.12
123     frame.TextColor3 = Color3.fromRGB(235, 238, 242)
124     frame.TextWrapped = true
125     frame.TextTruncate = Enum.TextTruncate.AtEnd
126     frame.TextSize = 13
127     frame.Font = Enum.Font.Gotham
128     frame.Parent = screen
129     local c = Instance.new("UICorner")
130     c.CornerRadius = UDim.new(0, 8)
131     c.Parent = frame
132     local sizeLimit = Instance.new("UISizeConstraint")
133     sizeLimit.MinSize = Vector2.new(260, 46)
134     sizeLimit.MaxSize = Vector2.new(420, 46)
135     sizeLimit.Parent = frame
136     local padding = Instance.new("UIPadding")
137     padding.PaddingLeft = UDim.new(0, 10)
138     padding.PaddingRight = UDim.new(0, 10)
139     padding.Parent = frame
140 end
141
142     local line = screen:FindFirstChild("Line")
143     if not line then return end
144     local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
145     screen:SetAttribute("MessageSerial", serial)
146     line.Visible = true
147     line.Text = text
148     task.delay(4, function()
149         if line.Parent and screen:GetAttribute("MessageSerial") == serial then line.Visible = false end
150     end)
151 end
152
153 local function playerRole(player)
154     local attributeRole = player:GetAttribute("LabRole")
155     if attributeRole and attributeRole ~= "" then return attributeRole end
156     if player.Team then return player.Team.Name end
157     local stats = player:FindFirstChild("leaderstats")
158     local role = stats and stats:FindFirstChild("Role")
159     return role and role.Value or "Unassigned"
160 end
161
162 local function awardPoints(player, amount, reason)
163     local stats = player:FindFirstChild("leaderstats")
164     if not stats then
165         stats = Instance.new("Folder")
166         stats.Name = "leaderstats"
167         stats.Parent = player
168     end
169     local points = stats:FindFirstChild("Points")
170     if not points then
171         points = Instance.new("IntValue")
172         points.Name = "Points"
173         points.Parent = stats
174     end
175     points.Value = points.Value + amount
176     addContribution(player, 1)
177     player:SetAttribute("LastPointAmount", amount)
178     player:SetAttribute("LastPointAward", reason)
179     notify(player, string.format("+%d POINTS - %s • TOTAL %d", amount, reason, points.Value))
180     updateScoreboard()
181     return points.Value
182 end
183
184 local function canInteract(player, part, maxDistance)
185     local character = player.Character
186     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
187     local root = character and character:FindFirstChild("HumanoidRootPart")
188     if humanoid == nil or humanoid.Health <= 0 or root == nil
189         or (root.Position - part.Position).Magnitude > maxDistance + 2 then return false end
190     local originPart = character:FindFirstChild("Head") or root
191     local direction = part.Position - originPart.Position
192     local params = RaycastParams.new()
193     params.FilterType = Enum.RaycastFilterType.Exclude
194     params.FilterDescendantsInstances = {character}
195     params.IgnoreWater = true
196     local hit = workspace:Raycast(originPart.Position, direction, params)
197     return hit == nil or hit.Instance == part or hit.Instance:IsDescendantOf(part)

```

```

198 end
199
200 -- Math-quiz gate removed by request: every task it used to guard now
201 -- proceeds immediately. Kept as a pass-through function (rather than
202 -- deleting every call site) so nothing else in this file has to change.
203 local function runMathQuiz(player, reason)
204     return true
205 end
206
207 local function runMathQuiz_DISABLED(player, reason)
208     if player:GetAttribute("MathQuizActive") then return false end
209     local digits = math.random(2, 7)
210     local maximum = math.min(9999999, 10 ^ digits - 1)
211     local a, b = math.random(1, maximum), math.random(1, maximum)
212     local subtract = math.random() < 0.5
213     if subtract and b > a then a, b = b, a end
214     local expected = subtract and (a - b) or (a + b)
215     local question = string.format("%d %s %d", a, subtract and "-" or "+", b)
216     local seconds = 10 + digits * 2
217     local previousObjective = player:GetAttribute("LabObjective")
218     player:SetAttribute("MathQuizActive", true)
219     player:SetAttribute("LabObjective", string.format("MATH CHECK: %s = ? Enter it in the console within %d
↪ seconds.", question, seconds))
220     local started = os.clock()
221     local ok, response = pcall(function()
222         return QuizRemote:InvokeClient(player, {Question = question, Seconds = seconds, Reason = reason})
223     end)
224     player:SetAttribute("MathQuizActive", false)
225     local correct = ok and os.clock() - started <= seconds + 1 and tonumber(response) == expected
226     if correct then
227         player:SetAttribute("LabObjective", previousObjective)
228         notify(player, "Math check correct. Task authorized.")
229         return true
230     end
231     player:SetAttribute("LabObjective", "Math check failed or timed out. Repeat the equipment action to try
↪ again.")
232     notify(player, "Math check failed or timed out. Try the task again.")
233     return false
234 end
235
236 -----
237 -- VISIBLE INVENTORY ITEMS
238 -- The old script collection used Tool + Handle + welded decorative
239 -- pieces. This keeps that useful pattern, but creates and validates the
240 -- tools entirely on the server instead of trusting a client script.
241 -----
242 local SAMPLE_ITEM = {
243     [Kinds.Items.TRAY] = true,
244     [Kinds.Items.GROUND] = true,
245     [Kinds.Items.DRIED] = true,
246     [Kinds.Items.SPUN] = true,
247     [Kinds.Items.CULTURE] = true,
248 }
249
250 local function addWeldedPart(tool, handle, name, size, relativeCFrame, colour, material, shape)
251     local p = Instance.new("Part")
252     p.Name = name
253     p.Size = size
254     p.CFrame = handle.CFrame * relativeCFrame
255     p.Color = colour
256     p.Material = material or Enum.Material.SmoothPlastic
257     p.Shape = shape or Enum.PartType.Block
258     p.Anchored = false
259     p.CanCollide = false
260     p.CanTouch = false
261     p.Massless = true
262     p.TopSurface = Enum.SurfaceType.Smooth
263     p.BottomSurface = Enum.SurfaceType.Smooth
264     p.Parent = tool
265     local weld = Instance.new("WeldConstraint")
266     weld.Part0 = handle
267     weld.Part1 = p
268     weld.Parent = p
269     return p
270 end
271

```

```

272 local function equipTool(player, tool)
273     task.defer(function()
274         if not tool.Parent then return end
275         local character = player.Character
276         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
277         if humanoid and humanoid.Health > 0 then humanoid:EquipTool(tool) end
278     end)
279 end
280
281 local function makeTool(player, name, options)
282     options = options or {}
283     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
284     if not backpack then return nil end
285
286     local tool = Instance.new("Tool")
287     tool.Name = name
288     tool.ToolTip = options.tooltip or "Visible laboratory item"
289     tool.RequiresHandle = true
290     tool.CanBeDropped = false
291     tool:SetAttribute("LabItemType", options.itemType or name)
292     tool:SetAttribute("LabSample", options.sample == true or SAMPLE_ITEM[name] == true)
293
294     for key, value in pairs(options.attributes or {}) do tool:SetAttribute(key, value) end
295     if options.plant and tool:GetAttribute("PlantLife") == nil then tool:SetAttribute("PlantLife", 100) end
296
297     local handle = Instance.new("Part")
298     handle.Name = "Handle"
299     handle.Size = options.plant and Vector3.new(0.95, 0.72, 0.95) or Vector3.new(1.7, 0.22, 1.35)
300     handle.Color = options.colour or Color3.fromRGB(205, 185, 138)
301     handle.Material = options.plant and Enum.Material.WoodPlanks or Enum.Material.SmoothPlastic
302     handle.Anchored = false
303     handle.CanCollide = false
304     handle.Massless = true
305     handle.TopSurface = Enum.SurfaceType.Smooth
306     handle.BottomSurface = Enum.SurfaceType.Smooth
307     handle.Parent = tool
308
309     if options.plant then
310         -- A real visible potted plant: soil, stem, leaves, and a glowing bud
311         -- are welded to the Tool handle and replicate in the character hand.
312         addWeldedPart(tool, handle, "Soil", Vector3.new(0.82, 0.14, 0.82),
313             CFrame.new(0, 0.4, 0), Color3.fromRGB(78, 52, 33), Enum.Material.Ground)
314         addWeldedPart(tool, handle, "PlantStem", Vector3.new(0.16, 1.0, 0.16),
315             CFrame.new(0, 0.96, 0), Color3.fromRGB(65, 155, 75), Enum.Material.Grass)
316         addWeldedPart(tool, handle, "LeafLeft", Vector3.new(0.78, 0.16, 0.42),
317             CFrame.new(-0.34, 1.1, 0) * CFrame.Angles(0, 0, math.rad(-22)),
318             Color3.fromRGB(70, 180, 82), Enum.Material.Grass)
319         addWeldedPart(tool, handle, "LeafRight", Vector3.new(0.78, 0.16, 0.42),
320             CFrame.new(0.34, 1.35, 0) * CFrame.Angles(0, 0, math.rad(22)),
321             Color3.fromRGB(85, 195, 92), Enum.Material.Grass)
322         addWeldedPart(tool, handle, "GrowthBud", Vector3.new(0.36, 0.36, 0.36),
323             CFrame.new(0, 1.58, 0), Color3.fromRGB(245, 185, 80), Enum.Material.Neon, Enum.PartType.Ball)
324     else
325         -- Tray rim, sample pouch, and labelled vial make ordinary samples
326         -- recognizable from a distance instead of a plain block.
327         addWeldedPart(tool, handle, "TrayRim", Vector3.new(1.82, 0.12, 1.47),
328             CFrame.new(0, 0.14, 0), Color3.fromRGB(55, 68, 78), Enum.Material.Metal)
329         addWeldedPart(tool, handle, "SeedPouch", Vector3.new(0.92, 0.32, 0.76),
330             CFrame.new(0, 0.35, 0), Color3.fromRGB(220, 193, 116), Enum.Material.Fabric)
331         addWeldedPart(tool, handle, "SampleVial", Vector3.new(0.62, 0.22, 0.22),
332             CFrame.new(0.47, 0.54, 0) * CFrame.Angles(0, 0, math.rad(90)),
333             Color3.fromRGB(115, 215, 245), Enum.Material.Glass, Enum.PartType.Cylinder)
334     end
335
336     local marker = Instance.new("BillboardGui")
337     marker.Name = "CarryLabel"
338     marker.Size = UDim2.fromOffset(150, 34)
339     marker.StudsOffsetWorldSpace = Vector3.new(0, 1.15, 0)
340     marker.MaxDistance = 8
341     marker.AlwaysOnTop = false
342     marker.Parent = handle
343     local markerText = Instance.new("TextLabel")
344     markerText.Size = UDim2.fromScale(1, 1)
345     markerText.BackgroundColor3 = Color3.fromRGB(20, 25, 30)
346     markerText.BackgroundTransparency = 0.2
347     markerText.TextColor3 = Color3.fromRGB(245, 225, 150)

```

```

348     markerText.TextScaled = true
349     markerText.Text = options.label or name
350     markerText.Font = Enum.Font.GothamBold
351     markerText.Parent = marker
352     local markerCorner = Instance.new("UICorner")
353     markerCorner.CornerRadius = UDim.new(0, 6)
354     markerCorner.Parent = markerText
355     if options.plant then
356         local lifeGui = Instance.new("BillboardGui")
357         lifeGui.Name = "PlantLifeStatus"
358         lifeGui.Size = UDim2.fromOffset(160, 46)
359         lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
360         lifeGui.MaxDistance = 18
361         lifeGui.AlwaysOnTop = false
362         lifeGui.Parent = handle
363         local lifeFrame = Instance.new("Frame")
364         lifeFrame.Size = UDim2.fromScale(1, 1)
365         lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
366         lifeFrame.BackgroundTransparency = 0.12
367         lifeFrame.BorderSizePixel = 0
368         lifeFrame.Parent = lifeGui
369         local lifeText = Instance.new("TextLabel")
370         lifeText.Name = "LifeText"
371         lifeText.Size = UDim2.new(1, 0, 0.58, 0)
372         lifeText.BackgroundTransparency = 1
373         lifeText.TextScaled = true
374         lifeText.Font = Enum.Font.GothamBold
375         lifeText.Parent = lifeFrame
376         local lifeBack = Instance.new("Frame")
377         lifeBack.Name = "LifeBarBackground"
378         lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
379         lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
380         lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
381         lifeBack.BorderSizePixel = 0
382         lifeBack.Parent = lifeFrame
383         local lifeFill = Instance.new("Frame")
384         lifeFill.Name = "LifeBarFill"
385         lifeFill.Size = UDim2.fromScale(1, 1)
386         lifeFill.BorderSizePixel = 0
387         lifeFill.Parent = lifeBack
388         local function updateLifeBar()
389             local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
390             lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
391             lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
392             local colour = life > 60 and Color3.fromRGB(75, 235, 105)
393             or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
394             lifeText.TextColor3 = colour
395             lifeFill.BackgroundColor3 = colour
396         end
397         tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLifeBar)
398         updateLifeBar()
399     end
400
401     tool.Grip = options.plant
402     and (CFrame.new(0, -0.25, -0.75) * CFrame.Angles(math.rad(-8), 0, 0))
403     or (CFrame.new(0, -0.45, -0.65) * CFrame.Angles(math.rad(-12), 0, 0))
404     tool.Parent = backpack
405     if options.autoEquip ~= false then equipTool(player, tool) end
406     return tool
407 end
408
409 local function playerContainers(player)
410     local containers = {}
411     if player.Character then table.insert(containers, player.Character) end
412     local backpack = player:FindFirstChildOfClass("Backpack")
413     if backpack then table.insert(containers, backpack) end
414     return containers
415 end
416
417 local function findTool(player, itemType)
418     for _, container in ipairs(playerContainers(player)) do
419         if container then
420             for _, child in ipairs(container:GetChildren()) do
421                 if child:IsA("Tool") and
422                     (child.Name == itemType or child:GetAttribute("LabItemType") == itemType) then
423                     return child

```

```

424         end
425     end
426 end
427 end
428 return nil
429 end
430
431 local function findTransferSample(player)
432     for _, container in ipairs(playerContainers(player)) do
433         if container then
434             for _, child in ipairs(container:GetChildren()) do
435                 if child:IsA("Tool") and child:GetAttribute("TransferSample") then return child end
436             end
437         end
438     end
439     return nil
440 end
441
442 local function findHeldSample(player)
443     local character = player.Character
444     if not character then return nil end
445     for _, child in ipairs(character:GetChildren()) do
446         if child:IsA("Tool") and child:GetAttribute("LabSample") then return child end
447     end
448     return nil
449 end
450
451 local GROWTH_STAGE_NAME = {
452     [0] = "Seedling",
453     [1] = "Germinated",
454     [2] = "Vegetative",
455     [3] = "Budding",
456     [4] = "Flowering",
457     [5] = "Mature",
458 }
459
460 local function findPlantSample(player)
461     for _, container in ipairs(playerContainers(player)) do
462         for _, child in ipairs(container:GetChildren()) do
463             if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end
464         end
465     end
466     return nil
467 end
468
469 local function treatmentText(tool)
470     local treatments = tool:GetAttribute("Treatments") or ""
471     if treatments == "" then return "untreated" end
472     return treatments:gsub("|", ", ")
473 end
474
475 local function hasTreatment(tool, key)
476     local treatments = "|" .. (tool:GetAttribute("Treatments") or "") .. "|"
477     return treatments:find("|" .. key .. "|", 1, true) ~= nil
478 end
479
480 local function addTreatment(tool, key)
481     if hasTreatment(tool, key) then return false end
482     local treatments = tool:GetAttribute("Treatments") or ""
483     tool:SetAttribute("Treatments", treatments == "" and key or (treatments .. "|" .. key))
484     return true
485 end
486
487 local function refreshPlantVisual(tool)
488     local stage = math.clamp(tool:GetAttribute("GrowthStage") or 0, 0, 5)
489     local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
490     local stem = tool:FindFirstChild("PlantStem")
491     local left = tool:FindFirstChild("LeafLeft")
492     local right = tool:FindFirstChild("LeafRight")
493     local bud = tool:FindFirstChild("GrowthBud")
494     if stem then stem.Size = Vector3.new(0.16 + stage * 0.025, 1 + stage * 0.13, 0.16 + stage * 0.025) end
495     local lifeScale = 0.35 + life / 100 * 0.65
496     if left then left.Color = Color3.fromRGB((70 + stage * 5) * lifeScale, (165 + stage * 10) * lifeScale, 55 *
↵ lifeScale) end
497     if right then right.Color = Color3.fromRGB((75 + stage * 4) * lifeScale, (175 + stage * 10) * lifeScale, 60
↵ * lifeScale) end

```

```

498     if bud then
499         bud.Transparency = stage < 3 and 0.65 or 0
500         bud.Color = stage >= 5 and Color3.fromRGB(255, 105, 175) or Color3.fromRGB(245, 185, 80)
501     end
502     local handle = tool:FindFirstChild("Handle")
503     local marker = handle and handle:FindFirstChild("CarryLabel")
504     local label = marker and marker:FindFirstChildOfClass("TextLabel")
505     if label then
506         local destination = tool:GetAttribute("DestinationGrowthCode") or ""
507         label.Text = (tool:GetAttribute("PlantId") or "PLANT") .. " • " .. (GROWTH_STAGE_NAME[stage] or
            ↳ "Growing")
508         .. (destination ~= "" and (" • BONUS DROP " .. destination) or " • DROP IN GREEN R")
509     end
510 end
511
512 local function setCarryState(player, sample)
513     -- A cold-store transfer remains the main HUD objective even if the
514     -- player briefly produces or pockets another laboratory item.
515     local transfer = findTransferSample(player)
516     if transfer and sample and not sample:GetAttribute("TransferSample") then sample = transfer end
517     if sample then
518         local id = sample:GetAttribute("SampleId") or sample.Name
519         local destination = sample:GetAttribute("DestinationLabel") or
            ↳ sample:GetAttribute("DestinationGrowthCode")
520         player:SetAttribute("CarriedSample", id)
521         if destination and destination ~= "" then
522             player:SetAttribute("LabObjective", "Carry " .. id .. " to bonus chamber " .. destination .. ", or
                ↳ press E at any GREEN RUNNING cabinet with a free slot.")
523         else
524             player:SetAttribute("LabObjective", "Carry " .. sample.Name .. " to the next required machine.")
525         end
526     else
527         player:SetAttribute("CarriedSample", "")
528         player:SetAttribute("LabObjective", "")
529     end
530 end
531
532 local function destroyCarriedItem(player, tool)
533     if player:GetAttribute("CarriedSample") == (tool:GetAttribute("SampleId") or tool.Name) then
534         setCarryState(player, nil)
535     end
536     tool:Destroy()
537 end
538
539 -----
540 -- NON-LETHAL HIT TOOLS
541 -----
542 local IMPACT_STYLE = {
543     ["Foam Baton"] = {colour = Color3.fromRGB(245, 135, 55), accent = Color3.fromRGB(45, 48, 54), mode =
        ↳ "melee", damage = 18},
544     ["Sample Grabber"] = {colour = Color3.fromRGB(70, 165, 220), accent = Color3.fromRGB(205, 225, 235), mode =
        ↳ "melee", damage = 12},
545     ["Inspection Paddle"] = {colour = Color3.fromRGB(245, 205, 70), accent = Color3.fromRGB(95, 75, 30), mode =
        ↳ "melee", damage = 22},
546     ["Stun Blaster"] = {colour = Color3.fromRGB(90, 225, 245), accent = Color3.fromRGB(35, 55, 70), mode =
        ↳ "gun", damage = 28, range = 95},
547     ["Foam Dart Rifle"] = {colour = Color3.fromRGB(225, 80, 95), accent = Color3.fromRGB(45, 45, 52), mode =
        ↳ "gun", damage = 38, range = 125},
548     ["Basketball"] = {colour = Color3.fromRGB(225, 105, 35), accent = Color3.fromRGB(45, 28, 18), mode =
        ↳ "ball", damage = 24},
549     -- PC-only weapons, built from the official Roblox Weapons Kit meshes.
550     --
551     -- meshScale is (the kit's intended size / the raw mesh size), measured off
552     -- the kit's own MeshParts. This matters: the raw pistol mesh is 9.68 studs
553     -- long, so the scale of 1.8 used at first drew a pistol about 17 studs
554     -- long -- roughly three times the height of the character holding it.
555     --
556     -- A worker bot has 100 life, so damage is set for 2-3 body shots; a head
557     -- shot kills outright regardless. selfCostPercent still charges the
558     -- shooter 1% of their own max health per shot. pcOnly means the generic
559     -- Tool.Activated path ignores them -- they fire only through the
560     -- LabSidearmFire remote, which only the desktop client script sends.
561     ["Lab Sidearm"] = {
562         colour = Color3.fromRGB(126, 133, 145), accent = Color3.fromRGB(38, 42, 50),
563         mode = "gun", damage = 34, range = 240, selfCostPercent = 1, pcOnly = true,
564         cooldown = 0.32, pellets = 1, spread = 0,

```

```

565     meshId = "rbxassetid://2492922600", meshTexture = "rbxassetid://2492931263",
566     meshScale = 0.1407, handleSize = Vector3.new(0.20, 0.92, 1.36),
567     grip = CFrame.new(0, -0.20, -0.30) * CFrame.Angles(0, math.rad(180), 0),
568 },
569 ["Lab Shotgun"] = {
570     colour = Color3.fromRGB(122, 98, 72), accent = Color3.fromRGB(40, 34, 30),
571     mode = "gun", damage = 20, range = 110, selfCostPercent = 1, pcOnly = true,
572     cooldown = 0.85, pellets = 6, spread = 0.075,
573     meshId = "rbxassetid://2492972199", meshTexture = "rbxassetid://2492974190",
574     meshScale = 0.6666, handleSize = Vector3.new(0.31, 1.00, 4.87),
575     grip = CFrame.new(0, -0.35, -0.95) * CFrame.Angles(0, math.rad(180), 0),
576 },
577 ["Lab Rifle"] = {
578     colour = Color3.fromRGB(96, 104, 96), accent = Color3.fromRGB(34, 38, 34),
579     mode = "gun", damage = 55, range = 620, selfCostPercent = 1, pcOnly = true,
580     cooldown = 0.95, pellets = 1, spread = 0,
581     meshId = "rbxassetid://2759059561", meshTexture = "rbxassetid://2759063893",
582     meshScale = 0.6383, handleSize = Vector3.new(0.29, 1.17, 6.18),
583     grip = CFrame.new(0, -0.35, -1.20) * CFrame.Angles(0, math.rad(180), 0),
584 },
585 }
586
587 local function refreshBotHealth(bot)
588     local health = math.clamp(bot:GetAttribute("Health") or 100, 0, bot:GetAttribute("MaxHealth") or 100)
589     local maximum = bot:GetAttribute("MaxHealth") or 100
590     local text = bot:FindFirstChild("HealthText", true)
591     local fill = bot:FindFirstChild("HealthBarFill", true)
592     if text and text:IsA("TextLabel") then text.Text = string.format("LIFE %d / %d", health, maximum) end
593     if fill and fill:IsA("Frame") then
594         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
595         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
596             or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
597     end
598 end
599
600 -- How long a body stays before it is cleaned up. Bodies are the biggest
601 -- avoidable part cost in a long round, so this is deliberately short.
602 -- Declared HERE, above damageWorkerBot, because that is what reads it: a
603 -- local declared further down the file resolves to a nil global inside a
604 -- function defined above it.
605 local BODY_LIFETIME_SECONDS = 60
606
607 local function damageWorkerBot(bot, amount, attacker)
608     -- Only an already-dead worker is immune. This used to refuse damage to
609     -- anything KnockedOut, which meant a worker downed by a hazard could not
610     -- be finished off by gunfire or caught in a blast -- so a floor could
611     -- never actually be cleared.
612     if not bot or not CollectionService.HasTag(bot, "LabWorkerBot") or bot:GetAttribute("Dead") then return
613     ↪ false end
614     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
615     bot:SetAttribute("Health", health)
616     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
617     refreshBotHealth(bot)
618     local flash = Instance.new("Highlight")
619     flash.FillColor = Color3.fromRGB(255, 90, 70)
620     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
621     flash.FillTransparency = 0.25
622     -- Highlights render through geometry by default, which is why a hit
623     -- worker (and any corpse that inherited the flash) glowed through walls.
624     flash.DepthMode = Enum.HighlightDepthMode.Occluded
625     flash.Parent = bot
626     Debris.AddItem(flash, 0.22)
627     if health <= 0 then
628         -- Death is permanent now: the hunt is the objective, so a worker that
629         -- goes down stays down and its body is left where it fell. Bodies are
630         -- cleared when the shift rolls over (see the RoundNumber watcher).
631         bot:SetAttribute("Dead", true)
632         bot:SetAttribute("KnockedOut", true)
633         bot:SetAttribute("TaskState", "DEAD")
634
635         -- The body left behind is a CLONE, not the worker itself. That keeps
636         -- the original model pristine, so the shift roll-over can simply move
637         -- it home and stand it back up rather than trying to undo a pile of
638         -- destructive colour and rotation edits.
639         local pivot = bot:GetPivot()
640         local corpse = bot:Clone()

```

```

640     corpse.Name = bot.Name .. "_Body"
641     for _, d in ipairs(corpse:GetDescendants()) do
642         -- A body reports no status, and must not inherit the damage-flash
643         -- Highlight -- that is what made corpses visible through walls,
644         -- because the clone's copy was never cleaned up.
645         if d:IsA("BillboardGui") or d:IsA("Highlight") or d:IsA("Sparkles")
646            or d:IsA("Fire") or d:IsA("Smoke") or d:IsA("PointLight")
647            or d.Name:match("^Chair") or d.Name:match("^Desk") or d.Name == "Monitor" then
648             d:Destroy()
649         end
650     end
651     for _, d in ipairs(corpse:GetDescendants()) do
652         if d:IsA("BasePart") then
653             d.Color = Color3.new(d.Color.R * 0.55 + 0.10, d.Color.G * 0.55 + 0.10, d.Color.B * 0.55 + 0.10)
654             d.Material = Enum.Material.SmoothPlastic
655             d.Anchored = true
656             d.CanCollide = false
657             d.CanQuery = false -- a body must not soak up later shots
658             d.CastShadow = false -- 180 shadow-casting corpses is not free
659         end
660     end
661     local corpseFolder = workspace:FindFirstChild("LabBotBodies")
662     if not corpseFolder then
663         corpseFolder = Instance.new("Folder")
664         corpseFolder.Name = "LabBotBodies"
665         corpseFolder.Parent = workspace
666     end
667     -- The clone inherits every tag, including LabWorkerBot. Left alone it
668     -- would be counted as a live worker by every GetTagged sweep and the
669     -- kill total would never be able to reach the target.
670     CollectionService:RemoveTag(corpse, "LabWorkerBot")
671     corpse.Parent = corpseFolder
672     -- Lay it down where it fell, with a random spin so a cleared room does
673     -- not read as a row of identical dropped props.
674     corpse:PivotTo(CFrame.new(pivot.Position - Vector3.new(0, 2.6, 0))
675         * CFrame.Angles(0, math.rad(math.random(0, 359)), 0)
676         * CFrame.Angles(math.rad(-90), 0, 0))
677     CollectionService:AddTag(corpse, "LabBotCorpse")
678     -- Bodies are the single biggest avoidable part cost in a long round:
679     -- 180 of them is ~1,600 extra parts. They stay long enough to read a
680     -- room as cleared, then go.
681     Debris:AddItem(corpse, BODY_LIFETIME_SECONDS)
682
683     -- Park the live worker out of the world until the next shift.
684     if not bot:GetAttribute("HomeCFrame") then bot:SetAttribute("HomeCFrame", pivot) end
685     bot:PivotTo(CFrame.new(0, -1000, 0))
686
687     local complex = workspace:FindFirstChild("Laboratory_Complex")
688     if complex then
689         complex:SetAttribute("BotsKilled", (complex:GetAttribute("BotsKilled") or 0) + 1)
690     end
691 end
692 return true
693 end
694
695 -- Walk up from whatever the ray actually hit to the tagged equipment part.
696 local function equipmentFromHit(hit)
697     local node = hit
698     while node and node ~= workspace do
699         if node:IsA("BasePart")
700            and (CollectionService:HasTag(node, "LabEquipment") or CollectionService:HasTag(node,
701                ↪ "PlantGrowthChamber")) then
702             return node
703         end
704         node = node.Parent
705     end
706     return nil
707 end
708
709 -- Shoot a machine and it goes up, taking out whatever is standing near it --
710 -- including the player who pulled the trigger, if they did it from close range.
711 local function explodeEquipment(item, attacker)
712     if item:GetAttribute("Exploded") then return false end
713     item:SetAttribute("Exploded", true)
714     item:SetAttribute("FunctionalState", "DESTROYED")
715     item:SetAttribute("RepairInstruction", "Destroyed beyond repair")

```



```

715
716     local position = item.Position
717     local blast = Instance.new("Explosion")
718     blast.Position = position
719     blast.BlastRadius = 17
720     blast.BlastPressure = 0          -- the lab is anchored; pressure would do nothing
721     blast.DestroyJointRadiusPercent = 0
722     blast.ExplosionType = Enum.ExplosionType.NoCraters
723     blast.Parent = workspace
724
725     local fire = Instance.new("Fire")
726     fire.Heat = 14
727     fire.Size = 9
728     fire.Parent = item
729     Debris:AddItem(fire, 14)
730     local smoke = Instance.new("Smoke")
731     smoke.Size = 5
732     smoke.Opacity = 0.4
733     smoke.RiseVelocity = 9
734     smoke.Color = Color3.fromRGB(60, 58, 56)
735     smoke.Parent = item
736     Debris:AddItem(smoke, 18)
737
738     item.Color = Color3.fromRGB(48, 40, 36)
739     item.Material = Enum.Material.CorrodedMetal
740
741     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
742         if not bot:GetAttribute("Dead") and bot.PrimaryPart then
743             if (bot.PrimaryPart.Position - position).Magnitude < 17 then
744                 damageWorkerBot(bot, 1000, attacker)
745             end
746         end
747     end
748     for _, other in ipairs(Players:GetPlayers()) do
749         local c = other.Character
750         local h = c and c:FindFirstChildOfClass("Humanoid")
751         local hrp = c and c:FindFirstChild("HumanoidRootPart")
752         if h and hrp and h.Health > 0 then
753             local d = (hrp.Position - position).Magnitude
754             if d < 17 then h:TakeDamage(math.max(18, 75 * (1 - d / 17))) end
755         end
756     end
757     return true
758 end
759
760 -- A gas line going up is a different order of event from a machine failing:
761 -- it takes out a chunk of the room -- a couple of neighbouring machines and
762 -- part of the wall structure itself.
763 local function detonateGasPipe(pipe, attacker, atPosition)
764     if pipe:GetAttribute("Blown") then return false end
765     pipe:SetAttribute("Blown", true)
766     -- Blow at the point that was actually hit. A corridor run is one long part
767     -- whose Position is its midpoint, so using that detonated the pipe up to a
768     -- hundred studs away from where the player shot it.
769     local position = atPosition or pipe.Position
770
771     local blast = Instance.new("Explosion")
772     blast.Position = position
773     blast.BlastRadius = 30
774     blast.BlastPressure = 0
775     blast.DestroyJointRadiusPercent = 0
776     blast.ExplosionType = Enum.ExplosionType.NoCraters
777     blast.Parent = workspace
778
779     -- a running wall of fire and smoke along the rupture
780     for i = -2, 2 do
781         local flare = Instance.new("Part")
782         flare.Name = "GasFire"
783         flare.Size = Vector3.new(1, 1, 1)
784         flare.CFrame = CFrame.new(position + Vector3.new(0, 0, i * 7))
785         flare.Anchored = true
786         flare.CanCollide = false
787         flare.CanQuery = false
788         flare.CanTouch = false
789         flare.Transparency = 1
790         flare.Parent = workspace

```

```

791     local fire = Instance.new("Fire")
792     fire.Heat = 25
793     fire.Size = 24
794     fire.Parent = flare
795     local smoke = Instance.new("Smoke")
796     smoke.Size = 15
797     smoke.Opacity = 0.5
798     smoke.RiseVelocity = 14
799     smoke.Color = Color3.fromRGB(40, 38, 36)
800     smoke.Parent = flare
801     task.delay(16, function()
802         if flare.Parent then fire.Enabled = false; smoke.Enabled = false end
803     end)
804     Debris:AddItem(flare, 22)
805 end
806
807 pipe.Color = Color3.fromRGB(62, 52, 30)
808 pipe.Material = Enum.Material.CorrodedMetal
809
810 -- take two or three neighbouring machines with it, staggered so it reads
811 -- as a chain rather than one simultaneous bang
812 local nearby = {}
813 for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 24)) do
814     if (CollectionService:HasTag(part, "LabEquipment") or CollectionService:HasTag(part,
815         ↪ "PlantGrowthChamber"))
816         and not part:GetAttribute("Exploded") then
817         table.insert(nearby, part)
818     end
819 end
820 for i = 1, math.min(3, #nearby) do
821     local item = nearby[i]
822     task.delay(0.18 * i, function()
823         if item.Parent then explodeEquipment(item, attacker) end
824     end)
825 end
826
827 -- and blow an actual hole in the structure
828 local blown = 0
829 for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 15)) do
830     if part:IsA("BasePart") and part.CanCollide and not part:GetAttribute("GasPipe") then
831         local n = part.Name
832         if n:match("^Corridor_") or n:match("^Div_") or n:match("^Ext_") or n:match("^B_") then
833             if blown < 5 then
834                 blown = blown + 1
835                 part:Destroy()
836             else
837                 part.Color = Color3.fromRGB(60, 54, 50)
838                 part.Material = Enum.Material.CorrodedMetal
839             end
840         end
841     end
842 end
843
844 for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
845     if not bot:GetAttribute("Dead") and bot.PrimaryPart
846         and (bot.PrimaryPart.Position - position).Magnitude < 30 then
847         damageWorkerBot(bot, 1000, attacker)
848     end
849 end
850
851 for _, other in ipairs(Players:GetPlayers()) do
852     local c = other.Character
853     local h = c and c:FindFirstChildOfClass("Humanoid")
854     local hrp = c and c:FindFirstChild("HumanoidRootPart")
855     if h and hrp and h.Health > 0 then
856         local d = (hrp.Position - position).Magnitude
857         if d < 30 then h:TakeDamage(math.max(30, 140 * (1 - d / 30))) end
858     end
859 end
860
861 return true
862 end
863
864 -- Scorch mark, smoke puff and sparks wherever a round lands.
865 local function spawnImpact(position, normal)
866     local hole = Instance.new("Part")
867     hole.Name = "BulletHole"
868     hole.Size = Vector3.new(0.3, 0.3, 0.06)

```

```

866     hole.CFrame = CFrame.lookAt(position + normal * 0.03, position + normal * 2)
867     hole.Color = Color3.fromRGB(24, 22, 21)
868     hole.Material = Enum.Material.Slate
869     hole.Anchored = true
870     hole.CanCollide = false
871     hole.CanQuery = false
872     hole.CanTouch = false
873     hole.CastShadow = false
874     hole.Parent = workspace
875     Debris:AddItem(hole, 45)
876
877     local puff = Instance.new("Part")
878     puff.Name = "ImpactPuff"
879     puff.Size = Vector3.new(0.1, 0.1, 0.1)
880     puff.CFrame = CFrame.new(position + normal * 0.25)
881     puff.Anchored = true
882     puff.CanCollide = false
883     puff.CanQuery = false
884     puff.CanTouch = false
885     puff.Transparency = 1
886     puff.Parent = workspace
887     local smoke = Instance.new("Smoke")
888     smoke.Size = 0.7
889     smoke.RiseVelocity = 5
890     smoke.Opacity = 0.3
891     smoke.Color = Color3.fromRGB(125, 125, 125)
892     smoke.Parent = puff
893     local sparks = Instance.new("Sparkles")
894     sparks.SparkleColor = Color3.fromRGB(255, 208, 128)
895     sparks.Parent = puff
896     task.delay(0.3, function()
897         if puff.Parent then smoke.Enabled = false; sparks.Enabled = false end
898     end)
899     Debris:AddItem(puff, 3)
900 end
901
902 -- Posters keep their holes for good rather than fading like scorch marks.
903 local function punchPoster(poster, position, normal)
904     local holes = poster:GetAttribute("Holes") or 0
905     if holes >= 30 then return end
906     poster:SetAttribute("Holes", holes + 1)
907     local tear = Instance.new("Part")
908     tear.Name = "PosterTear"
909     tear.Size = Vector3.new(0.36, 0.36, 0.03)
910     tear.CFrame = CFrame.lookAt(position + normal * 0.012, position + normal * 2)
911     tear.Color = Color3.fromRGB(198, 190, 174)
912     tear.Material = Enum.Material.SmoothPlastic
913     tear.Anchored = true
914     tear.CanCollide = false
915     tear.CanQuery = false
916     tear.CanTouch = false
917     tear.CastShadow = false
918     tear.Parent = poster
919     local hole = tear:Clone()
920     hole.Name = "PosterHole"
921     hole.Size = Vector3.new(0.22, 0.22, 0.05)
922     hole.Color = Color3.fromRGB(16, 15, 14)
923     hole.CFrame = CFrame.lookAt(position + normal * 0.025, position + normal * 2)
924     hole.Parent = poster
925 end
926
927 local function workerBotFromHit(hit)
928     local model = hit and hit:FindFirstAncestorOfClass("Model")
929     return model and CollectionService.HasTag(model, "LabWorkerBot") and model or nil
930 end
931
932 local function stealHeldSample(attacker, victim)
933     local sample = findHeldSample(victim)
934     if not sample then
935         notify(attacker, victim.DisplayName .. " is not holding a sample.")
936         return false
937     end
938     if sample:GetAttribute("TransferSample") and findTransferSample(attacker) then
939         notify(attacker, "Your hands are already assigned to another cold-store sample.")
940         return false
941     end

```

```

942
943     local backpack = attacker:FindFirstChildOfClass("Backpack")
944     if not backpack then return false end
945     local id = sample:GetAttribute("SampleId") or sample.Name
946     sample.Parent = backpack
947     setCarryState(victim, nil)
948     setCarryState(attacker, sample)
949     addContribution(attacker)
950     notify(attacker, "Recovered " .. id .. " from " .. victim.DisplayName .. ". It is now in your hand.")
951     notify(victim, attacker.DisplayName .. " took " .. id .. " from your hand. Collect or recover another
↪ sample.")
952     equipTool(attacker, sample)
953     return true
954 end
955
956 local function armImpactTool(tool)
957     if tool:GetAttribute("HitLogicBound") then return end
958     tool:SetAttribute("HitLogicBound", true)
959     local handle = tool:FindFirstChild("Handle")
960     if not handle then return end
961     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or tool.Name]
962     if not style then return end
963
964     local activeUntil = 0
965     local nextSwing = 0
966     local hitThisSwing = {}
967
968     tool.Activated:Connect(function()
969         -- Every gun is driven by BlasterController on the owning client.
970         -- Keep Tool.Activated for melee and throwable tools only, avoiding
971         -- duplicate shots from Roblox's default Tool activation path.
972         if style.mode == "gun" then return end
973         local owner = Players:GetPlayerFromCharacter(tool.Parent)
974         if not owner or os.clock() < nextSwing then return end
975         nextSwing = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
976         hitThisSwing = {}
977         local original = handle.Color
978         handle.Color = Color3.fromRGB(255, 245, 190)
979         task.delay(0.18, function()
980             if handle.Parent then handle.Color = original end
981         end)
982
983         local rootPart = owner.Character and owner.Character:FindFirstChild("HumanoidRootPart")
984         if not rootPart then return end
985         if style.mode == "gun" then
986             local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
987             local direction = rootPart.CFrame.LookVector * (style.range or 100)
988             local params = RaycastParams.new()
989             params.FilterType = Enum.RaycastFilterType.Exclude
990             params.FilterDescendantsInstances = {owner.Character}
991             local result = workspace:Raycast(origin, direction, params)
992             local hitPosition = result and result.Position or (origin + direction)
993             local beam = Instance.new("Part")
994             beam.Name = "FoamTracer"
995             beam.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
996             beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
997             beam.Color = style.colour
998             beam.Material = Enum.Material.Neon
999             beam.Anchored = true
1000             beam.CanCollide = false
1001             beam.CanTouch = false
1002             beam.Parent = workspace
1003             Debris:AddItem(beam, 0.12)
1004             local bot = result and workerBotFromHit(result.Instance)
1005             if bot then damageWorkerBot(bot, style.damage or 25, owner) end
1006             return
1007         elseif style.mode == "ball" then
1008             local ball = Instance.new("Part")
1009             ball.Name = "ThrownBasketball"
1010             ball.Shape = Enum.PartType.Ball
1011             ball.Size = Vector3.new(1.35, 1.35, 1.35)
1012             ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) + rootPart.CFrame.LookVector *
↪ 3)
1013             ball.Color = style.colour
1014             ball.Material = Enum.Material.Rubber
1015             ball.CanCollide = true

```

```

1016     ball.Parent = workspace
1017     ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
1018     local ballHit = false
1019     ball.Touched:Connect(function(hit)
1020         if ballHit then return end
1021         local bot = workerBotFromHit(hit)
1022         if bot then ballHit = true; damageWorkerBot(bot, style.damage or 20, owner) end
1023     end)
1024     Debris:AddItem(ball, 7)
1025     return
1026 end
1027 activeUntil = os.clock() + 0.45
1028 end)
1029
1030 handle.Touched:Connect(function(hit)
1031     if os.clock() > activeUntil then return end
1032     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
1033     if not attacker then return end
1034     local targetCharacter = hit:FindFirstAncestorOfClass("Model")
1035     local workerBot = workerBotFromHit(hit)
1036     if workerBot then
1037         if hitThisSwing[workerBot] then return end
1038         hitThisSwing[workerBot] = true
1039         damageWorkerBot(workerBot, style.damage or 15, attacker)
1040         return
1041     end
1042     local victim = targetCharacter and Players:GetPlayerFromCharacter(targetCharacter)
1043     if not victim or victim == attacker or hitThisSwing[victim] then return end
1044
1045     local attackerRoot = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
1046     local victimRoot = victim.Character and victim.Character:FindFirstChild("HumanoidRootPart")
1047     local victimHumanoid = victim.Character and victim.Character:FindFirstChildOfClass("Humanoid")
1048     if not attackerRoot or not victimRoot or not victimHumanoid or victimHumanoid.Health <= 0 then return
1049     ↪ end
1050     if (attackerRoot.Position - victimRoot.Position).Magnitude > 9 then return end
1051     hitThisSwing[victim] = true
1052
1053     local delta = victimRoot.Position - attackerRoot.Position
1054     if delta.Magnitude > 0.1 then
1055         local direction = delta.Unit
1056         victimRoot.AssemblyLinearVelocity = victimRoot.AssemblyLinearVelocity
1057         + Vector3.new(direction.X * 9, 3, direction.Z * 9)
1058     end
1059
1060     local flash = Instance.new("Highlight")
1061     flash.Name = "SampleRecoveryHit"
1062     flash.FillColor = Color3.fromRGB(255, 210, 80)
1063     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
1064     flash.FillTransparency = 0.35
1065     flash.Parent = victim.Character
1066     Debris:AddItem(flash, 0.25)
1067     stealHeldSample(attacker, victim)
1068 end)
1069 end
1070
1071 local function findImpactTool(player, kind)
1072     for _, container in ipairs(playerContainers(player)) do
1073         if container then
1074             for _, child in ipairs(container:GetChildren()) do
1075                 if child:IsA("Tool") and child:GetAttribute("ImpactToolKind") == kind then return child end
1076             end
1077         end
1078     end
1079     return nil
1080 end
1081
1082 local function makeImpactTool(player, kind)
1083     local existing = findImpactTool(player, kind)
1084     if existing then
1085         equipTool(player, existing)
1086         notify(player, kind .. " equipped. Click or tap to use it against worker NPCs or recover a held sample
1087         ↪ from another player.")
1088         return existing
1089     end
1090
1091     local style = IMPACT_STYLE[kind]

```

```

1090     if not style then return nil end
1091     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
1092     if not backpack then return nil end
1093
1094     local tool = Instance.new("Tool")
1095     tool.Name = kind
1096     tool.ToolTip = style.mode == "gun" and "Fire at the screen crosshair; hold right-click or tap AIM for ADS"
1097         or (style.mode == "ball" and "Throw a basketball at worker NPCs" or "Hit players to recover samples;
1098             ↳ hit NPCs to lower life")
1099     tool.RequiresHandle = true
1100     tool.CanBeDropped = false
1101     tool:SetAttribute("ImpactToolKind", kind)
1102     tool:SetAttribute("LabImpactTool", true)
1103     if style.mode == "gun" then
1104         tool:SetAttribute("BlasterWeapon", true)
1105         tool:SetAttribute("BlasterDamage", style.damage or 25)
1106         tool:SetAttribute("BlasterRange", style.range or 200)
1107         tool:SetAttribute("BlasterCooldown", style.cooldown or 0.55)
1108         tool:SetAttribute("BlasterPellets", style.pellets or 1)
1109         tool:SetAttribute("BlasterSpread", style.spread or 0)
1110         tool:SetAttribute("BlasterSelfCostPercent", style.selfCostPercent or 0)
1111         tool:SetAttribute("BlasterInstantHeadshot", style.pcOnly == true)
1112         tool:SetAttribute("BlasterAutomatic", style.automatic == true)
1113         tool:SetAttribute("BlasterADSFov", (style.range or 0) >= 500 and 44 or ((style.pellets or 1) > 1 and 58
1114             ↳ or 54))
1115         tool:SetAttribute("BlasterADSTouchSensitivity", 0.42)
1116         tool:SetAttribute("BlasterTracerColor", style.colour)
1117         -- Legacy marker retained for old saved tools; the legacy remote is inert.
1118         if style.pcOnly then tool:SetAttribute("LabSidearm", true) end
1119     end
1120
1121     local handle = Instance.new("Part")
1122     handle.Name = "Handle"
1123     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
1124         or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35) or Vector3.new(0.42, 3.4, 0.42))
1125     handle.Color = style.colour
1126     handle.Material = Enum.Material.SmoothPlastic
1127     handle.Anchored = false
1128     handle.CanCollide = false
1129     handle.Massless = true
1130     handle.Parent = tool
1131     if style.mode == "ball" then handle.Shape = Enum.PartType.Ball; handle.Material = Enum.Material.Rubber end
1132
1133     if style.meshId then
1134         -- Imported gun shape. A SpecialMesh is used rather than a MeshPart
1135         -- because MeshPart.MeshId cannot be assigned at runtime -- SpecialMesh
1136         -- can, so the tool builds correctly on a live server as well as in
1137         -- Studio, with no dependency on anything sitting in ServerStorage.
1138         handle.Size = style.handleSize or Vector3.new(0.20, 0.92, 1.36)
1139         local mesh = Instance.new("SpecialMesh")
1140         mesh.MeshType = Enum.MeshType.FileMesh
1141         mesh.MeshId = style.meshId
1142         if style.meshTexture then mesh.TextureId = style.meshTexture end
1143         local scale = style.meshScale or 1.6
1144         mesh.Scale = Vector3.new(scale, scale, scale)
1145         mesh.Parent = handle
1146     elseif style.mode == "gun" then
1147         addWeldedPart(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7),
1148             CFrame.new(0, 0.1, -1.65), style.colour, Enum.Material.Neon)
1149         addWeldedPart(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55),
1150             CFrame.new(0, -0.75, 0.42) * CFrame.Angles(math.rad(-18), 0, 0), style.accent,
1151             ↳ Enum.Material.Rubber)
1152     elseif kind == "Inspection Paddle" then
1153         addWeldedPart(tool, handle, "Paddle", Vector3.new(1.45, 1.65, 0.28),
1154             CFrame.new(0, 2.15, 0), style.colour, Enum.Material.SmoothPlastic)
1155     elseif kind == "Sample Grabber" then
1156         addWeldedPart(tool, handle, "LeftClaw", Vector3.new(0.25, 1.1, 0.3),
1157             CFrame.new(-0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(-28)), style.accent, Enum.Material.Metal)
1158         addWeldedPart(tool, handle, "RightClaw", Vector3.new(0.25, 1.1, 0.3),
1159             CFrame.new(0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(28)), style.accent, Enum.Material.Metal)
1160     elseif style.mode == "melee" then
1161         addWeldedPart(tool, handle, "FoamTip", Vector3.new(0.75, 1.5, 0.75),
1162             CFrame.new(0, 2, 0), style.colour, Enum.Material.Fabric)
1163     end
1164     if style.mode == "melee" then
1165         addWeldedPart(tool, handle, "Grip", Vector3.new(0.58, 1.15, 0.58),

```

```

1163         CFrame.new(0, -1.1, 0), style.accent, Enum.Material.Rubber)
1164     end
1165
1166     tool.Grip = style.grip or (style.mode == "gun" and CFrame.new(0, -0.2, 0.6))
1167     or (style.mode == "ball" and CFrame.new(0, -0.25, -0.4) or CFrame.new(0, -1.2, 0) *
        ↪ CFrame.Angles(math.rad(-8), 0, 0))
1168     tool.Parent = backpack
1169     armImpactTool(tool)
1170     equipTool(player, tool)
1171     notify(player, kind .. " equipped. Click or tap to use it against worker NPCs; their LIFE bar shows damage
        ↪ and recovery.")
1172     return tool
1173 end
1174
1175 local function bindDispenser(part)
1176     if part:GetAttribute("DispenserBound") then return end
1177     local kind = part:GetAttribute("ToolKind")
1178     if not kind or not IMPACT_STYLE[kind] then return end
1179     part:SetAttribute("DispenserBound", true)
1180
1181     local prompt = Instance.new("ProximityPrompt")
1182     prompt.ObjectText = kind
1183     prompt.ActionText = "Equip combat tool"
1184     prompt.KeyboardKeyCode = Enum.KeyCode.E
1185     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
1186     prompt.HoldDuration = 0.25
1187     prompt.MaxActivationDistance = 10
1188     prompt.RequiresLineOfSight = false
1189     prompt.Parent = part
1190     local lastUse = {}
1191     prompt.Triggered:Connect(function(player)
1192         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
1193         if os.clock() < (lastUse[player] or 0) then return end
1194         lastUse[player] = os.clock() + 0.75
1195         makeImpactTool(player, kind)
1196     end)
1197 end
1198
1199 -----
1200 -- PC SIDEARM -- mouse-aimed, costs the shooter 1% of their life per shot
1201 --
1202 -- Fired only through this remote, which only the desktop client script
1203 -- sends, so the weapon is inert on touch and console clients.
1204 -----
1205 local sidearmRemote = ReplicatedStorage:FindFirstChild("LabSidearmFire")
1206 if not sidearmRemote then
1207     sidearmRemote = Instance.new("RemoteEvent")
1208     sidearmRemote.Name = "LabSidearmFire"
1209     sidearmRemote.Parent = ReplicatedStorage
1210 end
1211
1212 local sidearmCooldown = {}
1213 Players.PlayerRemoving:Connect(function(player) sidearmCooldown[player] = nil end)
1214
1215 sidearmRemote.OnServerEvent:Connect(function(player, aimPoint)
1216     -- Superseded by ServerBlasterManager. Keep the legacy remote inert so
1217     -- old clients and exploit traffic cannot enter the retired mouse path.
1218     if true then return end
1219     if typeof(aimPoint) ~= "Vector3" then return end
1220     local character = player.Character
1221     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1222     local root = character and character:FindFirstChild("HumanoidRootPart")
1223     if not humanoid or not root or humanoid.Health <= 0 then return end
1224
1225     -- The client only ever asks; the server decides whether the player is
1226     -- really holding a weapon, and what that weapon is allowed to do.
1227     local tool = character:FindFirstChildOfClass("Tool")
1228     if not tool or not tool:GetAttribute("LabSidearm") then return end
1229     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or ""]
1230     if not style then return end
1231
1232     local now = os.clock()
1233     if now < (sidearmCooldown[player] or 0) then return end
1234     sidearmCooldown[player] = now + (style.cooldown or 0.35)
1235
1236     local handle = tool:FindFirstChild("Handle")

```

```

1237     local origin = (handle and handle.Position or root.Position) + Vector3.new(0, 0.5, 0)
1238     local baseDelta = aimPoint - origin
1239     if baseDelta.Magnitude < 0.1 then return end
1240     local baseDir = baseDelta.Unit
1241     local range = style.range or 200
1242
1243     local params = RaycastParams.new()
1244     params.FilterType = Enum.RaycastFilterType.Exclude
1245     params.FilterDescendantsInstances = {character}
1246
1247     local killsThisShot = 0
1248     for _ = 1, (style.pellets or 1) do
1249         -- Shotgun pellets scatter around the aim line; everything else is exact.
1250         local dir = baseDir
1251         local spread = style.spread or 0
1252         if spread > 0 then
1253             dir = (baseDir + Vector3.new(
1254                 (math.random() - 0.5) * spread * 2,
1255                 (math.random() - 0.5) * spread * 2,
1256                 (math.random() - 0.5) * spread * 2)).Unit
1257         end
1258
1259         local result = workspace.Raycast(origin, dir * range, params)
1260         local hitPosition = result and result.Position or (origin + dir * range)
1261
1262         local beam = Instance.new("Part")
1263         beam.Name = "SidearmTracer"
1264         beam.Size = Vector3.new(0.07, 0.07, (hitPosition - origin).Magnitude)
1265         beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
1266         beam.Color = Color3.fromRGB(255, 226, 140)
1267         beam.Material = Enum.Material.Neon
1268         beam.Anchored = true
1269         beam.CanCollide = false
1270         beam.CanTouch = false
1271         beam.CanQuery = false
1272         beam.CastShadow = false
1273         beam.Parent = workspace
1274         Debris:AddItem(beam, 0.1)
1275
1276         if result then
1277             local instance, normal = result.Instance, result.Normal
1278             local bot = workerBotFromHit(instance)
1279             if bot and not bot:GetAttribute("Dead") then
1280                 -- A head shot kills outright, whatever the weapon.
1281                 local headShot = instance.Name == "Head"
1282                 damageWorkerBot(bot, headShot and 1000 or (style.damage or 25), player)
1283                 if bot:GetAttribute("Dead") then killsThisShot = killsThisShot + 1 end
1284             elseif instance:GetAttribute("GasPipe") then
1285                 detonateGasPipe(instance, player, hitPosition)
1286                 spawnImpact(hitPosition, normal)
1287             else
1288                 local equipment = equipmentFromHit(instance)
1289                 if equipment then
1290                     explodeEquipment(equipment, player)
1291                 elseif instance:GetAttribute("ShootablePoster") then
1292                     punchPoster(instance, hitPosition, normal)
1293                 end
1294                 spawnImpact(hitPosition, normal)
1295             end
1296         end
1297     end
1298
1299     -- Firing always costs the shooter 1% of their own maximum health,
1300     -- whether or not the shot connects with anything.
1301     humanoid:TakeDamage(humanoid.MaxHealth * (style.selfCostPercent or 1) / 100)
1302
1303     if killsThisShot > 0 then
1304         local complex = workspace.FindFirstChild("Laboratory_Complex")
1305         notify(player, string.format("Target down. %d / %d cleared.",
1306             complex and complex:GetAttribute("BotsKilled") or 0,
1307             complex and complex:GetAttribute("BotsTotal") or 0))
1308     end
1309 end)
1310
1311 -----
1312 -- BLASTER GAMEPLAY BRIDGE

```



```

1313 -- ServerBlasterManager owns validation and raycasts. This callback only
1314 -- translates a validated impact into this game's custom worker/environment
1315 -- systems, keeping those systems private to the server.
1316 -----
1317 local blasterBridge = game:GetService("ServerScriptService"):FindFirstChild("BlasterHitBridge")
1318 if not blasterBridge then
1319     blasterBridge = Instance.new("BindableFunction")
1320     blasterBridge.Name = "BlasterHitBridge"
1321     blasterBridge.Parent = game:GetService("ServerScriptService")
1322 end
1323
1324 blasterBridge.OnInvoke = function(player, instance, hitPosition, hitNormal, damage, instantHeadshot)
1325     if typeof(player) ~= "Instance" or not player:IsA("Player") then return false end
1326     if typeof(instance) ~= "Instance" or not instance:IsDescendantOf(workspace) then return false end
1327
1328     local bot = workerBotFromHit(instance)
1329     if bot then
1330         local wasDead = bot:GetAttribute("Dead") == true
1331         local handled = damageWorkerBot(bot, instantHeadshot and 1000 or damage, player)
1332         if handled and not wasDead and bot:GetAttribute("Dead") then
1333             local complex = workspace:FindFirstChild("Laboratory_Complex")
1334             notify(player, string.format("Target down. %d / %d cleared.",
1335                 complex and complex:GetAttribute("BotsKilled") or 0,
1336                 complex and complex:GetAttribute("BotsTotal") or 0))
1337         end
1338         return handled
1339     end
1340
1341     if instance:GetAttribute("GasPipe") then
1342         detonateGasPipe(instance, player, hitPosition)
1343         spawnImpact(hitPosition, hitNormal)
1344         return true
1345     end
1346
1347     local equipment = equipmentFromHit(instance)
1348     if equipment then
1349         explodeEquipment(equipment, player)
1350         spawnImpact(hitPosition, hitNormal)
1351         return true
1352     end
1353
1354     if instance:GetAttribute("ShootablePoster") then
1355         punchPoster(instance, hitPosition, hitNormal)
1356         spawnImpact(hitPosition, hitNormal)
1357         return true
1358     end
1359
1360     return false
1361 end
1362
1363 -----
1364 -- COLD-STORE SAMPLE TRANSFERS
1365 -----
1366 local COLD_STORE = {Refrigerator = true, ["Chest Freezer"] = true}
1367 local nextSampleNumber = 1000
1368
1369 local PLANT_CHAMBER_KIND = "Plant Growth Chamber"
1370
1371 local function equipmentFloorLabel(part)
1372     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
1373 end
1374
1375 local function equipmentId(part)
1376     local code = part:GetAttribute("GrowthCode")
1377     if code and code ~= "" then return "CHAMBER:" .. code end
1378     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
1379         ↪ " | ")
1380 end
1381
1382 local function equipmentLabel(part)
1383     local code = part:GetAttribute("GrowthCode")
1384     if code and code ~= "" then return "CHAMBER " .. code end
1385     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
1386 end
1387
1388 local function adaptiveTransferSeconds(source, destination)

```

```

1388     local distance = destination and (destination.Position - source.Position).Magnitude or 60
1389     local playerCount = math.max(1, #Players:GetPlayers())
1390     local complex = workspace:FindFirstChild("Laboratory_Complex")
1391     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
1392     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
1393         + math.clamp(backlog / 20, 0, 25)), 60, 180)
1394 end
1395 local PLANT_PROCESS = {
1396     ["Germination Chamber"] = {
1397         key = "Germination", gain = 1,
1398         caption = "Warming seed and logging germination response",
1399     },
1400     ["Incubator"] = {
1401         key = "Incubation", gain = 1,
1402         caption = "Holding controlled temperature and humidity",
1403     },
1404     ["Growth Hormone Mixer"] = {
1405         key = "Growth Hormone", gain = 2,
1406         caption = "Dosing auxin/cytokinin growth treatment",
1407     },
1408     ["GMO Injector"] = {
1409         key = "GMO Injection", gain = 1, genome = true,
1410         caption = "Injecting and verifying a new plant trait",
1411     },
1412     ["Nutrient Infuser"] = {
1413         key = "Nutrient Infusion", gain = 1, moistureGain = 18,
1414         caption = "Balancing nitrogen, phosphorus, and potassium",
1415     },
1416     ["UV Growth Scanner"] = {
1417         repeatable = true, gain = 0,
1418         caption = "Scanning chlorophyll and canopy development",
1419     },
1420     ["Pollination Station"] = {
1421         key = "Pollination", gain = 1, minStage = 3,
1422         caption = "Applying controlled pollen and checking viability",
1423     },
1424     -- Micropropagation chain, step 1: surface-sterilize the seed coat before
1425     -- it ever reaches a growth chamber. Nothing downstream requires this, but
1426     -- a Sterilized sample gets a real (and referenced) germination boost --
1427     -- contamination is the #1 real-world cause of failed plant tissue culture.
1428     ["Decontamination Unit"] = {
1429         key = "Sterilized", gain = 0,
1430         caption = "Surface-sterilizing seed coat (dilute bleach + ethanol rinse) and sealing on sterile media",
1431     },
1432     -- Dose-response mutagenesis modeled on the reference Viola cornuta gamma
1433     -- study: germination viability falls off steeply above ~400 Gy, and by
1434     -- 1000 Gy almost nothing survives. Dose is chosen per treatment so two
1435     -- visits to the same chamber can give a different, honestly-labeled result.
1436     ["Mutagenesis Chamber"] = {
1437         key = "Mutagenesis", gain = 0, mutagenesis = true,
1438         caption = "Irradiating seed with a measured gamma dose",
1439     },
1440     -- Hormone Treatment Bench: auxin pushes rooting, cytokinin pushes
1441     -- shoot/bud growth, and a balanced ratio pushes both -- the same
1442     -- direction of effect reported for lemon seed germination trials.
1443     ["Hormone Treatment Bench"] = {
1444         key = "Hormone Treatment", gain = 1, hormone = true,
1445         caption = "Dosing a measured auxin/cytokinin treatment",
1446     },
1447 }
1448
1449 local function storageId(part)
1450     local id = part:GetAttribute("StorageId")
1451     if not id then
1452         id = table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
1453             ↪ part.Name}, "|")
1454         part:SetAttribute("StorageId", id)
1455     end
1456     return id
1457 end
1458
1459 local function storageLabel(part)
1460     local unit = part.Name:match("(%d+)$") or "1"
1461     return string.format("%s, %s, %s (unit %s)", part:GetAttribute("Kind") or "Cold store",
1462         part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "room", unit)
1463 end

```

```

1463
1464 local function coldStoresExcept(source)
1465     local stores = {}
1466     for _, candidate in ipairs(CollectionService.GetTagged("LabEquipment")) do
1467         if candidate ~= source and COLD_STORE[candidate:GetAttribute("Kind")] then
1468             table.insert(stores, candidate)
1469         end
1470     end
1471     table.sort(stores, function(a, b) return storageId(a) < storageId(b) end)
1472     return stores
1473 end
1474
1475 -----
1476 -- PHYSICAL CONTROL BUTTON AND DATA DISPLAY
1477 -----
1478 local function makeControls(part, kind, spec)
1479     local wallSide = part:GetAttribute("WallSide")
1480     local frontAxisX = wallSide == "W" or wallSide == "E"
1481     local frontDirection = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
1482     local isGrowthChamber = kind == PLANT_CHAMBER_KIND
1483     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
1484     local timerGui = Instance.new("BillboardGui")
1485     timerGui.Name = "EquipmentStatusTimer"
1486     timerGui.Size = UDim2.fromOffset(160, 32)
1487     timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
1488     timerGui.MaxDistance = 10
1489     timerGui.AlwaysOnTop = false
1490     timerGui.Parent = part
1491     local timerLabel = Instance.new("TextLabel")
1492     timerLabel.Name = "Timer"
1493     timerLabel.Size = UDim2.fromScale(1, 1)
1494     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
1495     timerLabel.BackgroundTransparency = 0.16
1496     timerLabel.BorderSizePixel = 0
1497     timerLabel.TextScaled = true
1498     timerLabel.Font = Enum.Font.GothamBold
1499     timerLabel.Parent = timerGui
1500     local timerCorner = Instance.new("UICorner")
1501     timerCorner.CornerRadius = UDim.new(0, 8)
1502     timerCorner.Parent = timerLabel
1503
1504     -- Equipment name plate, mounted directly above the RUNNING/FAILED timer
1505     -- badge so each unit's kind is obvious at a glance from close range.
1506     local nameGui = Instance.new("BillboardGui")
1507     nameGui.Name = "EquipmentNamePlate"
1508     nameGui.Size = UDim2.fromOffset(160, 24)
1509     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
1510     nameGui.MaxDistance = 10
1511     nameGui.AlwaysOnTop = false
1512     nameGui.Parent = part
1513     local nameLabel = Instance.new("TextLabel")
1514     nameLabel.Name = "Kind"
1515     nameLabel.Size = UDim2.fromScale(1, 1)
1516     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
1517     nameLabel.BackgroundTransparency = 0.05
1518     nameLabel.BorderSizePixel = 0
1519     nameLabel.Text = kind
1520     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
1521     nameLabel.TextScaled = true
1522     nameLabel.Font = Enum.Font.SourceSansBold
1523     nameLabel.Parent = nameGui
1524     local nameCorner = Instance.new("UICorner")
1525     nameCorner.CornerRadius = UDim.new(0, 8)
1526     nameCorner.Parent = nameLabel
1527
1528     local slotGui = Instance.new("BillboardGui")
1529     slotGui.Name = "EquipmentFreeSlots"
1530     slotGui.Size = UDim2.fromOffset(150, 28)
1531     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
1532     slotGui.MaxDistance = 10
1533     slotGui.AlwaysOnTop = false
1534     slotGui.Parent = part
1535     local slotLabel = Instance.new("TextLabel")
1536     slotLabel.Name = "FreeSlots"
1537     slotLabel.Size = UDim2.fromScale(1, 1)
1538     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)

```

```

1539 slotLabel.BackgroundTransparency = 0.12
1540 slotLabel.BorderSizePixel = 0
1541 slotLabel.TextScaled = true
1542 slotLabel.Font = Enum.Font.GothamBold
1543 slotLabel.Parent = slotGui
1544 local slotCorner = Instance.new("UICorner")
1545 slotCorner.CornerRadius = UDim.new(0, 8)
1546 slotCorner.Parent = slotLabel
1547 local storedLifeGui, storedLifeText, storedLifeFill
1548 if isPlantContainer then
1549     local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
1550     storedLifeGui = Instance.new("BillboardGui")
1551     storedLifeGui.Name = "StoredPlantLifeStatus"
1552     storedLifeGui.Size = UDim2.fromOffset(140, 40)
1553     storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
1554     storedLifeGui.MaxDistance = 10
1555     storedLifeGui.AlwaysOnTop = false
1556     storedLifeGui.Parent = plantAnchor
1557     storedLifeText = Instance.new("TextLabel")
1558     storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
1559     storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
1560     storedLifeText.BackgroundTransparency = 0.16
1561     storedLifeText.BorderSizePixel = 0
1562     storedLifeText.TextScaled = true
1563     storedLifeText.Font = Enum.Font.GothamBold
1564     storedLifeText.Parent = storedLifeGui
1565     local storedLifeBack = Instance.new("Frame")
1566     storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
1567     storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
1568     storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)
1569     storedLifeBack.BorderSizePixel = 0
1570     storedLifeBack.Parent = storedLifeGui
1571     storedLifeFill = Instance.new("Frame")
1572     storedLifeFill.Size = UDim2.fromScale(1, 1)
1573     storedLifeFill.BorderSizePixel = 0
1574     storedLifeFill.Parent = storedLifeBack
1575 end
1576 local function clockText(seconds)
1577     seconds = math.max(0, math.floor(seconds or 0))
1578     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
        ↪ seconds % 60)
1579 end
1580 local function renderTimer()
1581     local state = part:GetAttribute("FunctionalState")
1582     local destroyed = state == "DESTROYED"
1583     local failed = destroyed or state == "FAILED"
1584     local since = failed and (part:GetAttribute("FailureSince") or os.time())
1585     or (part:GetAttribute("OperationStarted") or os.time())
1586     if destroyed then
1587         timerLabel.Text = "DESTROYED -- UNFIXABLE"
1588         timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
1589     else
1590         timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. clockText(os.time() - since)
1591         timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)
1592     end
1593     if isPlantContainer and failed then
1594         local life = math.clamp(100 - math.floor((os.time() - since) * 1.5), 0, 100)
1595         part:SetAttribute("PlantLife", life)
1596         if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
1597             part:SetAttribute("SampleIds", "")
1598             part:SetAttribute("SampleCount", 0)
1599             part:SetAttribute("Occupied", false)
1600             part:SetAttribute("PlantId", "")
1601             part:SetAttribute("PlantExpiredAt", os.time())
1602             for _, child in ipairs(part:GetDescendants()) do
1603                 if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then child.Transparency = 1
                    ↪ end
1604             end
1605         end
1606     end
1607     local capacity = part:GetAttribute("Capacity") or 0
1608     local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
1609     local free = math.max(0, capacity - count)
1610     slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
1611     or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
1612     slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)

```

```

1613 -- Beacon: someone is actively carrying a plant toward THIS chamber
1614 -- (IncomingReservations is set the moment a plant is taken with this
1615 -- chamber as its destination). Make that one chamber's slot readout
1616 -- visible from across the room -- through walls too -- so "where do I
1617 -- put this plant" is answered by looking up, not by wandering every
1618 -- aisle reading labels that are otherwise only readable up close.
1619 local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
1620 if incoming and not failed then
1621     slotGui.MaxDistance = 30
1622     slotGui.AlwaysOnTop = false
1623     slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
1624     slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
1625 else
1626     slotGui.MaxDistance = 10
1627     slotGui.AlwaysOnTop = false
1628 end
1629 if storedLifeGui then
1630     local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
1631     local colour = life > 60 and Color3.fromRGB(75, 235, 105)
1632     or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
1633     storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
1634     storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
1635     storedLifeText.TextColor3 = colour
1636     storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
1637     storedLifeFill.BackgroundColor3 = colour
1638 end
1639 end
1640 renderTimer()
1641 task.spawn(function()
1642     while part.Parent do
1643         renderTimer()
1644         task.wait(1)
1645     end
1646 end)
1647
1648 local button = Instance.new("Part")
1649 button.Name = "PushButton"
1650 button.Shape = Enum.PartType.Cylinder
1651 button.Size = isGrowthChamber and Vector3.new(0.24, 0.64, 0.64) or Vector3.new(0.2, 0.48, 0.48)
1652 local buttonSide = math.clamp((frontAxisX and part.Size.Z or part.Size.X) * 0.34, 0.25, 1.2)
1653 local buttonY = math.clamp(part.Size.Y * 0.03, -0.15, 0.45)
1654 if frontAxisX then
1655     button.CFrame = part.CFrame * CFrame.new(frontDirection * (part.Size.X / 2 + 0.16), buttonY,
1656     ↪ buttonSide)
1657 else
1658     button.CFrame = part.CFrame * CFrame.new(-buttonSide, buttonY, frontDirection * (part.Size.Z / 2 +
1659     ↪ 0.16))
1660     * CFrame.Angles(0, math.rad(90), 0)
1661 end
1662 button.Color = Color3.fromRGB(218, 68, 58)
1663 button.Material = Enum.Material.Neon
1664 button.Anchored = true
1665 button.CanCollide = false
1666 button.CanTouch = false
1667 button.CastShadow = false
1668 button.Parent = part
1669 local restCFrame = button.CFrame
1670
1671 local prompt = Instance.new("ProximityPrompt")
1672 prompt.ObjectText = isGrowthChamber
1673 and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " • CHAMBER " ..
1674     ↪ (part:GetAttribute("GrowthCode") or "R?"))
1675 or kind
1676 prompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
1677 prompt.KeyboardKeyCode = Enum.KeyCode.E
1678 prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
1679 prompt.HoldDuration = isPlantContainer and 0 or 0.2
1680 prompt.MaxActivationDistance = 8
1681 prompt.RequiresLineOfSight = true
1682 prompt.Parent = button
1683
1684 local function setReadout(status, data, colour)
1685     part:SetAttribute("LastActionStatus", tostring(status or ""))
1686     part:SetAttribute("LastActionDetail", tostring(data or ""))
1687     renderTimer()
1688 end

```

```

1686
1687     local function pressButton()
1688         button.Color = Color3.fromRGB(255, 205, 70)
1689         local down = TweenService:Create(button, TweenInfo.new(0.07, Enum.EasingStyle.Quad),
1690             {CFrame = restCFrame * CFrame.new(-0.11, 0, 0)})
1691         down:Play()
1692         down.Completed:Wait()
1693         local up = TweenService:Create(button, TweenInfo.new(0.1, Enum.EasingStyle.Quad), {CFrame =
1694             ↪ restCFrame})
1695         up:Play()
1696         task.delay(0.12, function()
1697             if button.Parent then button.Color = Color3.fromRGB(218, 68, 58) end
1698         end)
1699     end
1700     return prompt, setReadout, pressButton
1701 end
1702
1703 local function runProgress(seconds, caption, setReadout)
1704     local steps = 4
1705     for step = 1, steps do
1706         task.wait(seconds / steps)
1707         setReadout("RUNNING " .. (step * 25) .. "%", caption, Color3.fromRGB(255, 205, 80))
1708     end
1709 end
1710
1711 local emergencyMovesCompleted = 0
1712 local setChamberFailure
1713 local setGenericFailure
1714
1715 local function chamberSampleList(chamber)
1716     local samples = {}
1717     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do table.insert(samples, id) end
1718     return samples
1719 end
1720
1721 local function saveChamberSamples(chamber, samples)
1722     local capacity = chamber:GetAttribute("Capacity") or 10
1723     while #samples > capacity do table.remove(samples) end
1724     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
1725     chamber:SetAttribute("SampleCount", #samples)
1726     chamber:SetAttribute("Occupied", #samples > 0)
1727     chamber:SetAttribute("PlantId", samples[1] or "")
1728 end
1729
1730 local function formatRuntime(seconds)
1731     seconds = math.max(0, math.floor(seconds or 0))
1732     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60, seconds %
1733         ↪ 60)
1734 end
1735
1736 local function recoverPlantLife(chamber, startingLife, player, plantId)
1737     startingLife = math.clamp(startingLife or 100, 0, 100)
1738     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then
1739         chamber:SetAttribute("PlantLife", startingLife)
1740         return 0
1741     end
1742     local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
1743     local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
1744     chamber:SetAttribute("RecoveryToken", token)
1745     chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
1746     chamber:SetAttribute("PlantLife", startingLife)
1747     if player then
1748         player:SetAttribute("LabObjective", string.format("%s dropped in %. LIFE %d% → 100% in %d seconds.",
1749             plantId or "Plant", chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber), startingLife,
1750             ↪ seconds))
1751         notify(player, string.format("Plant LIFE %d%. Recovery started; full life in %d seconds.",
1752             ↪ startingLife, seconds))
1753     end
1754     task.spawn(function()
1755         for step = 1, seconds do
1756             task.wait(1)
1757             if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
1758                 or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
1759             chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step / seconds +
1760                 ↪ 0.5))
1761         end
1762     end)
1763 end

```

```

1757     end
1758     chamber:SetAttribute("PlantLife", 100)
1759     chamber:SetAttribute("RecoveryEndsAt", 0)
1760     if player and player.Parent then notify(player, (plantId or "Plant") .. " recovered to 100% LIFE.") end
1761 end
1762 return seconds
1763 end
1764
1765 local function updateStoredSamples(chamber)
1766     local count = chamber:GetAttribute("SampleCount") or 0
1767     local stage = math.clamp(chamber:GetAttribute("GrowthStage") or 0, 0, 5)
1768     for _, child in ipairs(chamber:GetDescendants()) do
1769         if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then
1770             local visible = child:GetAttribute("SampleSlot") <= count
1771             child.Transparency = visible and (child:GetAttribute("StoredTransparency") or 0) or 1
1772             if child.Name:match("Stem") then child.Color = Color3.fromRGB(55 + stage * 4, 145 + stage * 10, 65)
1773             elseif child.Name:match("Leaf") then child.Color = Color3.fromRGB(65 + stage * 3, 160 + stage * 12,
1774                 ↪ 75) end
1775         end
1776     end
1777 end
1778
1779 local function refreshChamberTelemetry(chamber)
1780     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
1781     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
1782     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
1783     local samples = chamberSampleList(chamber)
1784     local sampleText = #samples > 0 and table.concat(samples, ", ") or "NO SAMPLES"
1785     local destination = chamber:GetAttribute("EvacuationDestination") or ""
1786     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "Unknown fault")
1787         ↪ .. (destination ~= "" and (" -> MOVE TO " .. destination) or " -> WAITING FOR SPACE")) or "SYSTEM
1788         ↪ NORMAL"
1789     local text = string.format("%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM
1790         ↪ %d%% • VENT %d%%\n%s\n%s",
1791         floorLabel, code, state, #samples, chamber:GetAttribute("Capacity") or 10,
1792         chamber:GetAttribute("GerminationDate") or "unknown",
1793         formatRuntime(os.time() - (chamber:GetAttribute("OperationStarted") or os.time()),
1794         chamber:GetAttribute("Temperature") or 0, chamber:GetAttribute("Humidity") or 0,
1795         chamber:GetAttribute("Ventilation") or 0, detail,
1796         state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
1797     for _, child in ipairs(chamber:GetDescendants()) do
1798         if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
1799             child.Text = text
1800             -- DESTROYED must never read as fine (green): only a genuinely
1801             -- WORKING chamber gets green, FAILED gets the usual red, and a
1802             -- permanently DESTROYED one gets an even harsher red so it never
1803             -- gets mistaken for "just needs a repair."
1804             child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
1805             or state == "FAILED" and Color3.fromRGB(255, 105, 90)
1806             or Color3.fromRGB(105, 245, 160)
1807         elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
1808             child.Text = "PLANTS (" .. #samples .. "): " .. sampleText
1809         elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
1810             child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)
1811             or state == "FAILED" and Color3.fromRGB(230, 55, 45)
1812             or Color3.fromRGB(40, 165, 95)
1813             child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
1814         end
1815     end
1816 end
1817
1818 local function evacuationDestinations(source)
1819     local sameLevel, anywhere = {}, {}
1820     for _, candidate in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
1821         local count = candidate:GetAttribute("SampleCount") or 0
1822         local capacity = candidate:GetAttribute("Capacity") or 10
1823         local incoming = candidate:GetAttribute("IncomingReservations") or 0
1824         if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED" and count + incoming <
1825             ↪ capacity then
1826             table.insert(anywhere, candidate)
1827             if candidate:GetAttribute("Level") == source:GetAttribute("Level") then table.insert(sameLevel,
1828                 ↪ candidate) end
1829         end
1830     end
1831 end
1832
1833 local list = #sameLevel > 0 and sameLevel or anywhere
1834 table.sort(list, function(a, b)

```

```

1828     local ac = (a:GetAttribute("SampleCount") or 0) + (a:GetAttribute("IncomingReservations") or 0)
1829     local bc = (b:GetAttribute("SampleCount") or 0) + (b:GetAttribute("IncomingReservations") or 0)
1830     if ac == bc then return (a.Position - source.Position).Magnitude < (b.Position -
        ↪ source.Position).Magnitude end
1831     return ac < bc
1832 end)
1833 return list
1834 end
1835
1836 local function repairPathFolder(player, create)
1837     local complex = workspace:FindFirstChild("Laboratory_Complex")
1838     if not complex then return nil end
1839     local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
1840     if create and not rootFolder then
1841         rootFolder = Instance.new("Folder")
1842         rootFolder.Name = "TechnicianRepairPaths"
1843         rootFolder.Parent = complex
1844     end
1845     local name = "Player_" .. player.UserId
1846     local folder = rootFolder and rootFolder:FindFirstChild(name)
1847     if create and not folder then
1848         folder = Instance.new("Folder")
1849         folder.Name = name
1850         folder.Parent = rootFolder
1851     end
1852     return folder
1853 end
1854
1855 local function clearRepairPath(player)
1856     local folder = repairPathFolder(player, false)
1857     if folder then folder:Destroy() end
1858 end
1859
1860 local function repairReward(player, equipment, assigned)
1861     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
1862     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
1863     local points = 8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0)
1864     return math.clamp(points, 8, 120), waited, distance
1865 end
1866
1867 local function createRepairPath(player, target)
1868     clearRepairPath(player)
1869     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
1870     if not characterRoot or not target then return 0, false end
1871     local folder = repairPathFolder(player, true)
1872     if not folder then return 0, false end
1873
1874     local function approachPoint(part)
1875         local side = part:GetAttribute("WallSide")
1876         local floorY = part.Position.Y - part.Size.Y / 2
1877         if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
            ↪ part.Position.Z) end
1878         if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
            ↪ part.Position.Z) end
1879         if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
            ↪ 2.5) end
1880         return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
1881     end
1882     local function appendNavigable(points, fromPosition, toPosition)
1883         local path = PathfindingService:CreatePath({
1884             AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
1885             AgentCanClimb = true, WaypointSpacing = 4,
1886         })
1887         local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
1888         if not computed or path.Status ~= Enum.PathStatus.Success then return false end
1889         local waypoints = path:GetWaypoints()
1890         if #waypoints < 2 then return false end
1891         for index, waypoint in ipairs(waypoints) do
1892             if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
1893         end
1894         return true
1895     end
1896
1897     local complex = workspace:FindFirstChild("Laboratory_Complex")
1898     local rise = complex and complex:GetAttribute("FloorRiseStuds")
1899     local westX = complex and complex:GetAttribute("StairWestLaneX")

```



```

1900     local eastX = complex and complex:GetAttribute("StairEastLaneX")
1901     local northZ = complex and complex:GetAttribute("StairNorthZ")
1902     local southZ = complex and complex:GetAttribute("StairSouthZ")
1903     local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
1904     local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
1905     local destination = approachPoint(target)
1906     local points = {}
1907     local routeReady = false
1908     if rise and westX and eastX and northZ and southZ then
1909         local startFloor = math.clamp(math.floor((characterRoot.Position.Y - 2.5) / rise + 0.5), floorMin,
            ↪ floorMax)
1910         local targetFloor = math.clamp(math.floor(destination.Y / rise + 0.5), floorMin, floorMax)
1911         if startFloor == targetFloor then
1912             routeReady = appendNavigable(points, characterRoot.Position, destination)
1913         else
1914             local direction = targetFloor > startFloor and 1 or -1
1915             local firstLaneX = direction > 0 and westX or eastX
1916             local stairEntry = Vector3.new(firstLaneX, startFloor * rise, northZ)
1917             routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
1918             local floor = startFloor
1919             while routeReady and floor ~= targetFloor do
1920                 local baseY = floor * rise
1921                 if direction > 0 then
1922                     table.insert(points, Vector3.new(westX, baseY + rise / 2, southZ))
1923                     table.insert(points, Vector3.new(eastX, baseY + rise / 2, southZ))
1924                     table.insert(points, Vector3.new(eastX, baseY + rise, northZ))
1925                 else
1926                     table.insert(points, Vector3.new(eastX, baseY - rise / 2, southZ))
1927                     table.insert(points, Vector3.new(westX, baseY - rise / 2, southZ))
1928                     table.insert(points, Vector3.new(westX, baseY - rise, northZ))
1929                 end
1930                 floor = floor + direction
1931                 if floor ~= targetFloor then
1932                     table.insert(points, Vector3.new(direction > 0 and westX or eastX, floor * rise, northZ))
1933                 end
1934             end
1935             if routeReady then
1936                 routeReady = appendNavigable(points, points[#points], destination)
1937             end
1938         end
1939     end
1940     if not routeReady then
1941         folder:Destroy()
1942         player:SetAttribute("RepairRouteReady", false)
1943         return (destination - characterRoot.Position).Magnitude, false
1944     end
1945     player:SetAttribute("RepairRouteReady", true)
1946     local distance = 0
1947     local markerCount = 0
1948     for pointIndex = 1, #points - 1 do
1949         local a, b = points[pointIndex], points[pointIndex + 1]
1950         local delta = b - a
1951         local segmentDistance = delta.Magnitude
1952         distance = distance + segmentDistance
1953         if segmentDistance > 0.1 then
1954             local direction = delta.Unit
1955             local count = math.max(1, math.ceil(segmentDistance / 4))
1956             for index = 1, count do
1957                 markerCount = markerCount + 1
1958                 if markerCount > 120 then break end
1959                 local marker = Instance.new("Part")
1960                 marker.Name = "RedRepairPath"
1961                 marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
1962                 local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
1963                 marker.CFrame = CFrame.lookAt(position, position + direction)
1964                 marker.Color = Color3.fromRGB(255, 32, 28)
1965                 marker.Material = Enum.Material.Neon
1966                 marker.Transparency = 0.12
1967                 marker.Anchored = true
1968                 marker.CanCollide = false
1969                 marker.CanTouch = false
1970                 marker.CanQuery = false
1971                 marker.Parent = folder
1972             end
1973         end
1974         if markerCount > 120 then break end

```

```

1975     end
1976     return distance, true
1977 end
1978
1979 local function clearAssignment(player)
1980     clearRepairPath(player)
1981     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
1982         ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
1983         player:SetAttribute(name, nil)
1984     end
1985 end
1986
1987 local function assignNextTask(player)
1988     clearAssignment(player)
1989     local role = playerRole(player)
1990     if role == "Technician" then
1991         local failed = {}
1992         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
1993             if equipment.Parent and equipment:GetAttribute("FunctionalState") == "FAILED" then
1994                 ↪ table.insert(failed, equipment) end
1995         end
1996         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
1997         table.sort(failed, function(a, b)
1998             if root then
1999                 local aDistance = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
2000                     ↪ root.Position.Y) > 12 and 500 or 0)
2001                 local bDistance = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
2002                     ↪ root.Position.Y) > 12 and 500 or 0)
2003                 if aDistance ~= bDistance then return aDistance < bDistance end
2004             end
2005             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
2006         end)
2007         local target = failed[1]
2008         if not target then
2009             local text = "All equipment is working. Repair score increases with failure time; assigned jobs
2010                 ↪ also pay for travel distance."
2011             player:SetAttribute("LabObjective", text)
2012             return text
2013         end
2014         player:SetAttribute("AssignedRepairId", equipmentId(target))
2015         local distance, routeReady = createRepairPath(player, target)
2016         player:SetAttribute("AssignedRepairDistance", distance)
2017         local projected = select(1, repairReward(player, target, true))
2018         player:SetAttribute("AssignedRepairProjectedPoints", projected)
2019         local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR; route is
2020             ↪ recalculating"
2021         local text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
2022             ↪ while failed).",
2023             equipmentFloorLabel(target), equipmentLabel(target), routeInstruction, projected,
2024             ↪ math.floor(distance + 0.5))
2025         player:SetAttribute("LabObjective", text)
2026         return text
2027     elseif role == "Laboratory Technician" then
2028         local sources = {}
2029         for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
2030             if chamber.Parent and chamber:GetAttribute("FunctionalState") == "FAILED"
2031                 and (chamber:GetAttribute("SampleCount") or 0) > 0
2032                 and not chamber:GetAttribute("ReservedByBot") then
2033                 local destination = evacuationDestinations(chamber)[1]
2034                 if destination then table.insert(sources, {source = chamber, destination = destination}) end
2035             end
2036         end
2037         table.sort(sources, function(a, b)
2038             local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
2039             local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
2040             return an < bn
2041         end)
2042         local job = sources[1]
2043         if not job then
2044             local text = "Laboratory Technician: waiting for a failed chamber with plants. Emergency moves pay
2045                 ↪ +12; assigned on-time moves pay +30."
2046             player:SetAttribute("LabObjective", text)
2047             return text
2048         end
2049         local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
2050         local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"

```

```

2042     local allowedSeconds = adaptiveTransferSeconds(job.source, job.destination)
2043     local deadline = os.time() + allowedSeconds
2044     player:SetAttribute("AssignedTransferSource", sourceCode)
2045     player:SetAttribute("AssignedTransferDestination", destinationCode)
2046     player:SetAttribute("AssignedTransferDeadline", deadline)
2047     -- Same red floor-marker route the Technician role gets to a failed
2048     -- unit, but pointed at the destination chamber the plant must be
2049     -- carried to -- so "where does this go" is never a guess.
2050     local routeReady = select(2, createRepairPath(player, job.destination))
2051     local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
↪ recalculating"
2052     local text = string.format("ASSIGNED +%d: %s • %s → %s • %s. Press E to take a plant. %s. Deliver in %d
↪ sec.",
2053         POINTS.MOVE_ASSIGNED_ON_TIME, equipmentFloorLabel(job.source), sourceCode, destinationCode,
2054         equipmentFloorLabel(job.destination), routeInstruction, allowedSeconds)
2055     player:SetAttribute("LabObjective", text)
2056     return text
2057 end
2058 local text = "Choose BLUE Technician or GREEN Laboratory Technician at a role kiosk."
2059 player:SetAttribute("LabObjective", text)
2060 return text
2061 end
2062
2063 local function chamberByCode(code)
2064     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
2065         if chamber:GetAttribute("GrowthCode") == code then return chamber end
2066     end
2067 end
2068
2069 local function takePlantFromChamber(player, chamber, setReadout)
2070     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
2071     if playerRole(player) ~= "Laboratory Technician" then
2072         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician moves plants", Color3.fromRGB(255,
↪ 175, 80))
2073         notify(player, "Only a Laboratory Technician can take plants. Choose the GREEN role; Technicians repair
↪ with R.")
2074         return
2075     end
2076     local samples = chamberSampleList(chamber)
2077     if #samples == 0 then setReadout(code .. " EMPTY", "No germinated samples available", Color3.fromRGB(255,
↪ 200, 90)); return end
2078     local failed = chamber:GetAttribute("FunctionalState") == "FAILED"
2079     local destination
2080     if failed then
2081         local assignedSource = player:GetAttribute("AssignedTransferSource")
2082         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
2083         if assignedSource == code and assignedDestination then
2084             local requested = chamberByCode(assignedDestination)
2085             if requested and requested:GetAttribute("FunctionalState") ~= "FAILED"
2086                 and (requested:GetAttribute("SampleCount") or 0) +
↪ (requested:GetAttribute("IncomingReservations") or 0)
2087                 < (requested:GetAttribute("Capacity") or 10) then
2088                 destination = requested
2089             end
2090         end
2091         destination = destination or evacuationDestinations(chamber)[1]
2092     end
2093     if not destination then
2094         setReadout(code .. " FAILED", "No working chamber has free capacity", Color3.fromRGB(255, 105,
↪ 90))
2095         notify(player, "No safe destination currently has room. The adaptive controller will reduce
↪ failures and free capacity.")
2096         return
2097     end
2098     -- Refresh the floor-marker route to whichever chamber this specific
2099     -- plant actually ends up assigned to (may differ from the original
2100     -- job destination if that one filled up in the meantime).
2101     createRepairPath(player, destination)
2102 end
2103 local plantId = samples[1]
2104 local stage = chamber:GetAttribute("GrowthStage") or 0
2105 local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
2106 local plantLife = chamber:GetAttribute("PlantLife") or 100
2107 if failed then
2108     plantLife = math.clamp(100 - math.floor((os.time() - (chamber:GetAttribute("FailureSince") or
↪ os.time())) * 1.5), 0, 100)
2109 end
2110 end

```

```

2109   setReadout("TAKING SAMPLE", "Removing " .. plantId .. " from " .. code, Color3.fromRGB(120, 235, 170))
2110   if not runMathQuiz(player, "TAKE PLANT " .. plantId .. " FROM " .. code) then return end
2111   table.remove(samples, 1)
2112   local sample = makeTool(player, "Plant Sample " .. plantId, {
2113     itemType = failed and "Emergency Plant Transfer" or "Plant Sample",
2114     sample = true, plant = true, label = plantId, colour = Color3.fromRGB(133, 92, 55),
2115     attributes = {
2116       PlantSample = true, SampleId = plantId, PlantId = plantId, HomeGrowthCode = code, SourceGrowthCode
2117       ⇨ = code,
2118       GrowthStage = stage, Treatments = chamber:GetAttribute("Treatments") or "",
2119       Genome = chamber:GetAttribute("Genome") or "Wild Type", Moisture = chamber:GetAttribute("Moisture")
2120       ⇨ or 50,
2121       PlantLife = plantLife,
2122       GerminationDate = chamber:GetAttribute("GerminationDate") or "unknown",
2123       EvacuationSample = failed, DestinationGrowthCode = destinationCode, TransferStartedAt = os.time(),
2124     },
2125   })
2126   if not sample then table.insert(samples, 1, plantId); return end
2127   saveChamberSamples(chamber, samples)
2128   updateStoredSamples(chamber)
2129   refreshChamberTelemetry(chamber)
2130   refreshPlantVisual(sample)
2131   setCarryState(player, sample)
2132   task.spawn(function()
2133     while sample.Parent and (sample:GetAttribute("PlantLife") or 0) > 0 do
2134       task.wait(1)
2135       if not sample.Parent then return end
2136       if not sample:GetAttribute("QuizProtected") then
2137         local life = math.max(0, (sample:GetAttribute("PlantLife") or 0) - 2)
2138         sample:SetAttribute("PlantLife", life)
2139         refreshPlantVisual(sample)
2140         if life <= 0 then
2141           local reservedCode = sample:GetAttribute("DestinationGrowthCode") or ""
2142           local reserved = reservedCode ~= "" and chamberByCode(reservedCode) or nil
2143           if reserved then reserved:SetAttribute("IncomingReservations", math.max(0,
2144             ⇨ (reserved:GetAttribute("IncomingReservations") or 1) - 1)) end
2145           player:SetAttribute("CarriedSample", "")
2146           player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
2147             ⇨ another plant.")
2148           task.wait(0.5)
2149           if sample.Parent then sample:Destroy() end
2150           return
2151         end
2152       end
2153     end
2154   end)
2155   if destination then
2156     destination:SetAttribute("IncomingReservations", (destination:GetAttribute("IncomingReservations") or
2157       ⇨ 0) + 1)
2158     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
2159     local seconds = math.max(0, deadline - os.time())
2160     player:SetAttribute("LabObjective", string.format("CARRYING %s: press E at ANY GREEN RUNNING cabinet
2161       ⇨ with a FREE SLOT. BONUS target %s has %d sec left for +%d points.",
2162         plantId, destinationCode, seconds, POINTS.MOVE_ASSIGNED_ON_TIME))
2163     setReadout(code .. " FAILED", plantId .. " removed • SEND TO " .. destinationCode, Color3.fromRGB(255,
2164       ⇨ 145, 75))
2165     notify(player, "ALARM: " .. code .. " failed (" .. (chamber:GetAttribute("FailureReason") or "fault")
2166       ⇨ .. "). Move " .. plantId .. " to " .. destinationCode .. ".")
2167   else
2168     player:SetAttribute("LabObjective", "Treat " .. plantId .. ", then press E at any GREEN RUNNING cabinet
2169       ⇨ with a FREE SLOT.")
2170     setReadout(code .. " WORKING", plantId .. " taken • " .. #samples .. "/" ..
2171       ⇨ (chamber:GetAttribute("Capacity") or 0) .. " remain", Color3.fromRGB(115, 220, 160))
2172     notify(player, plantId .. " equipped in your hand. Process it or move it to a working chamber.")
2173   end
2174 end
2175
2176 local function storePlantInChamber(player, chamber, plant, setReadout)
2177   local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
2178   if playerRole(player) ~= "Laboratory Technician" then
2179     setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician stores plants",
2180       ⇨ Color3.fromRGB(255, 175, 80))
2181     notify(player, "Only a Laboratory Technician can complete plant transfers.")
2182     return
2183   end
2184   if chamber:GetAttribute("FunctionalState") == "FAILED" then

```

```

2174     setReadout(code .. " FAILED", "Cannot drop here • find any GREEN RUNNING cabinet", Color3.fromRGB(255,
↪ 105, 90))
2175     notify(player, code .. " is failed. Carry the plant to any GREEN RUNNING cabinet with a FREE SLOT and
↪ press E.")
2176     return
2177 end
2178 local samples = chamberSampleList(chamber)
2179 local capacity = chamber:GetAttribute("Capacity") or 10
2180 if #samples >= capacity then
2181     setReadout(code .. " FULL", capacity .. "/" .. capacity .. " samples • choose another working
↪ chamber", Color3.fromRGB(255, 175, 75))
2182     return
2183 end
2184 local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
2185 local plantId = plant:GetAttribute("PlantId") or plant.Name
2186 local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
2187 local sourceCode = plant:GetAttribute("SourceGrowthCode") or plant:GetAttribute("HomeGrowthCode") or ""
2188 local assignedMove = wanted ~= "" and player:GetAttribute("AssignedTransferSource") == sourceCode
2189     and player:GetAttribute("AssignedTransferDestination") == code
2190 local assignedOnTime = assignedMove and os.time() <= (player:GetAttribute("AssignedTransferDeadline") or 0)
2191 plant:SetAttribute("QuizProtected", true)
2192 setReadout("STORING SAMPLE", "Placing " .. plantId .. " into " .. code, Color3.fromRGB(120, 235, 170))
2193 local passedQuiz = runMathQuiz(player, "REVIVE " .. plantId .. " IN " .. code)
2194 plant:SetAttribute("QuizProtected", nil)
2195 if not passedQuiz or not plant.Parent then return end
2196 table.insert(samples, plantId)
2197 saveChamberSamples(chamber, samples)
2198 local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
2199 chamber:SetAttribute("GrowthStage", math.max(chamber:GetAttribute("GrowthStage") or 0, stage))
2200 chamber:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
2201 chamber:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
2202 chamber:SetAttribute("Moisture", plant:GetAttribute("Moisture") or chamber:GetAttribute("Moisture") or 50)
2203 local recoverySeconds = recoverPlantLife(chamber, plantLife, player, plantId)
2204 if wanted ~= "" then
2205     local reservedDestination = chamberByCode(wanted)
2206     if reservedDestination then
2207         reservedDestination:SetAttribute("IncomingReservations", math.max(0,
↪ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
2208     end
2209     emergencyMovesCompleted = emergencyMovesCompleted + 1
2210     player:SetAttribute("EmergencyTransfers", (player:GetAttribute("EmergencyTransfers") or 0) + 1)
2211 end
2212 updateStoredSamples(chamber)
2213 refreshChamberTelemetry(chamber)
2214 destroyCarriedItem(player, plant)
2215 local points = 0
2216 local reason = "plant stored"
2217 if wanted ~= "" then
2218     GameState.transfersCompleted = GameState.transfersCompleted + 1
2219     if assignedMove then
2220         GameState.assignedTransfersCompleted = GameState.assignedTransfersCompleted + 1
2221         points = assignedOnTime and POINTS.MOVE_ASSIGNED_ON_TIME or POINTS.MOVE_ASSIGNED_LATE
2222         reason = assignedOnTime and ("assigned on-time move " .. sourceCode .. " → " .. code)
2223         or ("assigned late move " .. sourceCode .. " → " .. code)
2224     else
2225         points = POINTS.MOVE_ANY
2226         reason = "emergency plant move " .. sourceCode .. " → " .. code
2227     end
2228 end
2229 setReadout(code .. " STORED" .. (points > 0 and (" +" .. points) or ""),
2230     plantId .. " • " .. #samples .. "/" .. capacity .. " samples", Color3.fromRGB(115, 245, 150))
2231 if points > 0 then
2232     awardPoints(player, points, reason)
2233     if recoverySeconds > 0 then
2234         task.delay(recoverySeconds, function() if player.Parent then assignNextTask(player) end end)
2235     else
2236         assignNextTask(player)
2237     end
2238 else
2239     notify(player, plantId .. " safely stored in " .. code .. ". Capacity: " .. #samples .. "/" .. capacity
↪ .. ".")
2240 end
2241 end
2242
2243 local function handleGrowthChamber(player, chamber, setReadout)
2244     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)

```

```

2245     local reservedBy = chamber:GetAttribute("ReservedByBot")
2246     if reservedBy then
2247         -- Human players always have priority over test bots. Clearing the
2248         -- reservation makes one press of E immediately take or drop a plant.
2249         chamber:SetAttribute("ReservedByBot", nil)
2250     end
2251     local plant = findPlantSample(player)
2252     if plant then storePlantInChamber(player, chamber, plant, setReadout)
2253     elseif (chamber:GetAttribute("SampleCount") or 0) > 0 then takePlantFromChamber(player, chamber,
        ↪ setReadout)
2254     else
2255         setReadout(code .. " EMPTY", "Working chamber can receive a carried sample", Color3.fromRGB(255, 200,
        ↪ 90))
2256     end
2257 end
2258
2259 local GMO_TRAITS = {"Drought Resistant", "High Yield", "Salt Tolerant", "Fast Flowering", "Disease Resistant"}
2260
2261 -- Dose-response germination-viability curve, bucketed off the reference
2262 -- Viola cornuta gamma-irradiation study (Trends Sci. 2025;22(1):8705): mild
2263 -- doses barely dent germination, but survival collapses above ~600 Gy and is
2264 -- nearly gone by 1000 Gy. Each visit rolls a dose and a survival check.
2265 local MUTAGENESIS_DOSES = {
2266     {gy = 100, survivalMin = 85, survivalMax = 95},
2267     {gy = 200, survivalMin = 65, survivalMax = 85},
2268     {gy = 400, survivalMin = 45, survivalMax = 70},
2269     {gy = 600, survivalMin = 15, survivalMax = 40},
2270     {gy = 800, survivalMin = 5, survivalMax = 20},
2271     {gy = 1000, survivalMin = 0, survivalMax = 8},
2272 }
2273
2274 -- Auxin pushes rooting, cytokinin pushes shoot/bud growth, and a balanced
2275 -- ratio pushes both -- the direction of effect reported for lemon seed
2276 -- germination hormone trials, kept as a labeled tag rather than a hidden stat.
2277 local HORMONE_TREATMENTS = {
2278     {name = "Auxin-dominant (IBA-heavy)", focus = "Root", gain = 1},
2279     {name = "Cytokinin-dominant (BAP-heavy)", focus = "Shoot", gain = 1},
2280     {name = "Balanced 1:1 auxin:cytokinin", focus = "Root+Shoot", gain = 2},
2281 }
2282
2283 local function germinationPercent(player)
2284     local sown = player:GetAttribute("GermSownCount") or 0
2285     if sown == 0 then return 0 end
2286     return math.floor(((player:GetAttribute("GermNormalCount") or 0) / sown) * 100 + 0.5)
2287 end
2288
2289 local function processPlant(player, part, spec, process, setReadout)
2290     if playerRole(player) ~= "Laboratory Technician" then
2291         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician develops plants",
        ↪ Color3.fromRGB(255, 175, 80))
2292         notify(player, "Only a Laboratory Technician can use plant-development equipment.")
2293         return false
2294     end
2295     local plant = findPlantSample(player)
2296     if not plant then
2297         setReadout("PLANT REQUIRED", "Take one from an R-numbered chamber", Color3.fromRGB(255, 150, 85))
2298         notify(player, part:GetAttribute("Kind") .. ": take a visible plant sample from an R chamber first.")
2299         return false
2300     end
2301
2302     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
2303     if process.minStage and stage < process.minStage then
2304         setReadout("STAGE TOO EARLY", "Needs Budding stage or later", Color3.fromRGB(255, 150, 85))
2305         notify(player, "Grow " .. (plant:GetAttribute("PlantId") or "the plant") .. " to Budding before
        ↪ pollination.")
2306         return false
2307     end
2308     if process.key and not process.repeatable and hasTreatment(plant, process.key) then
2309         setReadout("ALREADY APPLIED", process.key .. " is already recorded", Color3.fromRGB(255, 180, 85))
2310         notify(player, process.key .. " has already been applied to this plant. Use another machine.")
2311         return false
2312     end
2313
2314     runProgress(spec.wait or 1.5, process.caption, setReadout)
2315
2316     -- Mutagenesis Chamber: roll a dose, then roll survival against that

```

```

2317 -- dose's real-world-modeled band. A killed sample ends here.
2318 if process.mutagenesis then
2319     local bucket = MUTAGENESIS_DOSES[math.random(1, #MUTAGENESIS_DOSES)]
2320     local survivalPercent = math.random(bucket.survivalMin, bucket.survivalMax)
2321     local survived = math.random(1, 100) <= math.max(survivalPercent, 1)
2322     plant:SetAttribute("MutagenesisDoseGy", bucket.gy)
2323     plant:SetAttribute("MutagenesisSurvivalPercent", survivalPercent)
2324     if not survived then
2325         local plantId = plant:GetAttribute("PlantId") or "sample"
2326         setReadout("SAMPLE LOST", string.format("%d Gy dose • %d%% survival odds • did not survive",
2327             ↳ bucket.gy, survivalPercent),
2328             Color3.fromRGB(255, 90, 80))
2329         notify(player, plantId .. " did not survive the " .. bucket.gy .. " Gy dose (" .. survivalPercent
2330             ↳ .. "% survival odds). Take another plant.")
2331         destroyCarriedItem(player, plant)
2332         return true
2333     end
2334     addTreatment(plant, "Mutagenesis " .. bucket.gy .. "Gy")
2335     plant:SetAttribute("Genome", "Mutagenized (" .. bucket.gy .. " Gy)")
2336 end
2337
2338 -- Hormone Treatment Bench: roll a hormone mix, tag the resulting focus.
2339 if process.hormone then
2340     local mix = HORMONE_TREATMENTS[math.random(1, #HORMONE_TREATMENTS)]
2341     plant:SetAttribute("HormoneTreatment", mix.name)
2342     plant:SetAttribute("HormoneFocus", mix.focus)
2343     process = {key = process.key, gain = mix.gain, moistureGain = process.moistureGain, caption =
2344         ↳ process.caption}
2345     addTreatment(plant, "Hormone: " .. mix.name)
2346 elseif process.key then
2347     addTreatment(plant, process.key)
2348 end
2349
2350 -- Germination Chamber: classify the outcome (AOSA normal / abnormal /
2351 -- dead), the same three buckets a real seed-testing lab reports, and
2352 -- track a running germination percentage on the Laboratory Technician.
2353 -- A prior Decontamination Unit visit (Sterilized treatment) meaningfully
2354 -- improves the odds, matching real seed-testing guidance that
2355 -- contamination is the leading cause of an abnormal or dead result.
2356 if process.key == "Germination" then
2357     local sterilized = hasTreatment(plant, "Sterilized")
2358     local irradiated = plant:GetAttribute("MutagenesisSurvivalPercent")
2359     local normalChance = sterilized and 82 or 60
2360     if irradiated then normalChance = math.floor(normalChance * (irradiated / 100)) end
2361     local roll = math.random(1, 100)
2362     local outcome
2363     if roll <= normalChance then outcome = "Normal"
2364     elseif roll <= normalChance + 25 then outcome = "Abnormal"
2365     else outcome = "Dead" end
2366     plant:SetAttribute("GerminationOutcome", outcome)
2367     player:SetAttribute("GermSownCount", (player:GetAttribute("GermSownCount") or 0) + 1)
2368     if outcome == "Normal" then
2369         player:SetAttribute("GermNormalCount", (player:GetAttribute("GermNormalCount") or 0) + 1)
2370     elseif outcome == "Abnormal" then
2371         player:SetAttribute("GermAbnormalCount", (player:GetAttribute("GermAbnormalCount") or 0) + 1)
2372         plant:SetAttribute("PlantLife", math.clamp((plant:GetAttribute("PlantLife") or 100) - 25, 10, 100))
2373     else
2374         player:SetAttribute("GermDeadCount", (player:GetAttribute("GermDeadCount") or 0) + 1)
2375         local plantId = plant:GetAttribute("PlantId") or "sample"
2376         setReadout("DEAD • AOSA", string.format("Lab germination rate now %d%%",
2377             ↳ germinationPercent(player)), Color3.fromRGB(255, 90, 80))
2378         notify(player, plantId .. " classified DEAD under AOSA rules (" .. (sterilized and "sterilized" or
2379             ↳ "unsterilized")
2380             ↳ .. " seed). Your running germination rate: " .. germinationPercent(player) .. "%. Take another
2381             ↳ plant.")
2382         destroyCarriedItem(player, plant)
2383         return true
2384     end
2385 end
2386
2387 local newStage = math.clamp(stage + (process.gain or 0), 0, 5)
2388 plant:SetAttribute("GrowthStage", newStage)
2389 plant:SetAttribute("Moisture", math.clamp((plant:GetAttribute("Moisture") or 50) + (process.moistureGain or
2390     ↳ -2), 15, 100))
2391 if process.genome then plant:SetAttribute("Genome", GMO_TRAITS[math.random(1, #GMO_TRAITS)]) end
2392 refreshPlantVisual(plant)

```

```

2386 setCarryState(player, plant)
2387 player:SetAttribute("LabObjective", "Plant " .. (plant:GetAttribute("PlantId") or "sample") .. " is "
2388 .. (GROWTH_STAGE_NAME[newStage] or "Growing") .. ". Use another treatment or return it to an empty R
    ⇐ chamber.")
2389 if process.key then awardPoints(player, POINTS.PROCESS_PLANT, process.key .. " plant treatment") end
2390
2391 local data = string.format("%s • %s • %d%% moisture\nGenome: %s",
2392     plant:GetAttribute("PlantId") or "PLANT", GROWTH_STAGE_NAME[newStage] or "Growing",
2393     plant:GetAttribute("Moisture") or 50, plant:GetAttribute("Genome") or "Wild Type")
2394 if plant:GetAttribute("GerminationOutcome") then
2395     data = data .. string.format("\nAOSA: %s • Lab germination rate %d%%",
    ⇐ plant:GetAttribute("GerminationOutcome"), germinationPercent(player))
2396 end
2397 if plant:GetAttribute("MutagenesisDoseGy") then
2398     data = data .. string.format("\nDose %d Gy • %d%% survival odds",
    ⇐ plant:GetAttribute("MutagenesisDoseGy"), plant:GetAttribute("MutagenesisSurvivalPercent") or 0)
2399 end
2400 if plant:GetAttribute("HormoneTreatment") then
2401     data = data .. "\n" .. plant:GetAttribute("HormoneTreatment") .. " • " ..
    ⇐ (plant:GetAttribute("HormoneFocus") or "")
2402 end
2403 setReadout("PLANT DATA", data, Color3.fromRGB(120, 240, 160))
2404 notify(player, data:gsub("\n", " | ") .. " | Treatments: " .. treatmentText(plant))
2405 equipTool(player, plant)
2406 return true
2407 end
2408
2409 local function createTransferSample(player, source, setReadout)
2410     local existing = findTransferSample(player)
2411     if existing then
2412         local target = existing:GetAttribute("DestinationLabel") or "the named cold store"
2413         setReadout("SAMPLE IN TRANSIT", (existing:GetAttribute("SampleId") or existing.Name) .. " -> " ..
    ⇐ target,
2414             Color3.fromRGB(255, 205, 80))
2415         notify(player, "You already carry " .. (existing:GetAttribute("SampleId") or existing.Name) .. ".
    ⇐ Deliver it to " .. target .. ".")
2416         equipTool(player, existing)
2417         return
2418     end
2419
2420     local destinations = coldStoresExcept(source)
2421     if #destinations == 0 then
2422         setReadout("TRANSFER UNAVAILABLE", "A second refrigerator or freezer is required.", Color3.fromRGB(255,
    ⇐ 110, 100))
2423         notify(player, "This transfer needs at least two refrigerators/freezers in the generated lab.")
2424         return
2425     end
2426     -- Prefer a true refrigerator destination so the guaranteed lobby
2427     -- pair always supports the requested fridge-to-fridge play loop.
2428     local refrigeratorDestinations = {}
2429     for _, candidate in ipairs(destinations) do
2430         if candidate:GetAttribute("Kind") == "Refrigerator" then
2431             table.insert(refrigeratorDestinations, candidate)
2432         end
2433     end
2434     if #refrigeratorDestinations > 0 then destinations = refrigeratorDestinations end
2435
2436     local destination = destinations[math.random(1, #destinations)]
2437     nextSampleNumber = nextSampleNumber + 1
2438     local sampleId = string.format("SL-%04d", nextSampleNumber)
2439     local targetLabel = storageLabel(destination)
2440     local sample = makeTool(player, "Seed Sample " .. sampleId, {
2441         itemType = "Transfer Seed Sample",
2442         sample = true,
2443         label = sampleId,
2444         colour = Color3.fromRGB(120, 190, 225),
2445         attributes = {
2446             TransferSample = true,
2447             SampleId = sampleId,
2448             SourceStorageId = storageId(source),
2449             DestinationStorageId = storageId(destination),
2450             DestinationLabel = targetLabel,
2451         },
2452     })
2453     if not sample then return end
2454     setCarryState(player, sample)

```



```

2455     setReadout("DISPATCHED " .. sampleId, "DEST: " .. targetLabel, Color3.fromRGB(100, 220, 255))
2456     notify(player, "Picked up " .. sampleId .. ". Carry it visibly to " .. targetLabel .. ".")
2457 end
2458
2459 local function handleColdStore(player, part, setReadout)
2460     local sample = findTransferSample(player)
2461     if not sample then
2462         createTransferSample(player, part, setReadout)
2463         return
2464     end
2465
2466     local sampleId = sample:GetAttribute("SampleId") or sample.Name
2467     local wanted = sample:GetAttribute("DestinationStorageId")
2468     if wanted ~= storageId(part) then
2469         local destination = sample:GetAttribute("DestinationLabel") or "the assigned cold store"
2470         setReadout("WRONG STORAGE", sampleId .. " -> " .. destination, Color3.fromRGB(255, 120, 95))
2471         notify(player, sampleId .. " belongs in " .. destination .. ". It remains in your hand.")
2472         equipTool(player, sample)
2473         return
2474     end
2475
2476     destroyCarriedItem(player, sample)
2477     GameState.transfersCompleted = GameState.transfersCompleted + 1
2478     addContribution(player)
2479     updateScoreboard()
2480     setReadout("RECEIVED " .. sampleId, "TRANSFER VERIFIED | +1 contribution", Color3.fromRGB(115, 245, 150))
2481     notify(player, sampleId .. " delivered and verified. Transfer complete.")
2482 end
2483
2484 local function startCycleIndicator(part)
2485     local auto = part:GetAttribute("Auto")
2486     if not auto then return nil, function() end end
2487     local indicator = Instance.new("Part")
2488     indicator.Name = "CycleIndicator"
2489     indicator.Size = (auto == "spin") and Vector3.new(0.9, 0.15, 0.9) or Vector3.new(0.5, 0.5, 0.5)
2490     indicator.Shape = (auto == "spin") and Enum.PartType.Block or Enum.PartType.Ball
2491     indicator.Anchored = true
2492     indicator.CanCollide = false
2493     indicator.CanTouch = false
2494     indicator.CastShadow = false
2495     indicator.Material = Enum.Material.Neon
2496     indicator.Color = Color3.fromRGB(255, 196, 60)
2497     indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
2498     indicator.Parent = part.Parent
2499     local light = Instance.new("PointLight")
2500     light.Color = indicator.Color
2501     light.Brightness = 1.5
2502     light.Range = 12
2503     light.Parent = indicator
2504
2505     local running = true
2506     task.spawn(function()
2507         local t = 0
2508         while running and indicator.Parent do
2509             t = t + task.wait(0.1)
2510             if auto == "spin" then
2511                 indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
2512                     * CFrame.Angles(0, t * 5, 0)
2513             else
2514                 indicator.Transparency = 0.15 + (math.sin(t * 7) + 1) / 2 * 0.55
2515             end
2516         end
2517     end)
2518     return indicator, function()
2519         running = false
2520         if indicator.Parent then indicator:Destroy() end
2521     end
2522 end
2523
2524 local function bind(part)
2525     local kind = part:GetAttribute("Kind")
2526     if not kind or part:GetAttribute("InteractionsBound") then return end
2527     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
2528     part:SetAttribute("InteractionsBound", true)
2529     part:SetAttribute("EquipmentId", equipmentId(part))
2530     part:SetAttribute("FloorLabel", equipmentFloorLabel(part))

```

```

2531     part:SetAttribute("RepairKey", "R")
2532     if part:GetAttribute("FunctionalState") == nil then
2533         local hash = 0
2534         for i = 1, #equipmentId(part) do hash = hash + string.byte(equipmentId(part), i) end
2535         local failed = kind ~= PLANT_CHAMBER_KIND and hash % 7 == 0
2536         part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
2537         part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
2538         part:SetAttribute("FailureSince", failed and os.time() or 0)
2539     end
2540     if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time()) end
2541     if COLD_STORE[kind] then storageId(part) end
2542     if isPlantContainer then
2543         updateStoredSamples(part)
2544         refreshChamberTelemetry(part)
2545     end
2546     local spec = Kinds.forKind(kind)
2547     local prompt, setReadout, pressButton = makeControls(part, kind, spec)
2548     local baseColour = part.Color
2549     local busy = false
2550     local powerOn = true
2551     part:SetAttribute("Powered", true)
2552     local powerLight = Instance.new("PointLight")
2553     powerLight.Color = Color3.fromRGB(120, 220, 140)
2554     powerLight.Brightness = powerOn and 2 or 0
2555     powerLight.Range = 7
2556     powerLight.Parent = part
2557
2558     local led = Instance.new("Part")
2559     led.Name = "StatusLED"
2560     led.Shape = Enum.PartType.Ball
2561     led.Size = Vector3.new(0.22, 0.22, 0.22)
2562     led.Anchored = true
2563     led.CanCollide = false
2564     led.CanTouch = false
2565     led.CastShadow = false
2566     led.Material = Enum.Material.Neon
2567     led.Color = powerOn and Color3.fromRGB(110, 230, 130) or Color3.fromRGB(64, 68, 74)
2568     local controlSide = part:GetAttribute("WallSide")
2569     if controlSide == "W" or controlSide == "E" then
2570         local direction = controlSide == "W" and 1 or -1
2571         led.CFrame = part.CFrame * CFrame.new(direction * (part.Size.X / 2 + 0.08), part.Size.Y / 2 - 0.3,
            ↪ part.Size.Z * 0.32)
2572     else
2573         local direction = controlSide == "S" and -1 or 1
2574         led.CFrame = part.CFrame * CFrame.new(part.Size.X * 0.32, part.Size.Y / 2 - 0.3, direction *
            ↪ (part.Size.Z / 2 + 0.08))
2575     end
2576     led.Parent = part
2577
2578     local repairAnchor = part:FindFirstChild("ControllerScreen") or led
2579     local floorLabel = equipmentFloorLabel(part)
2580     local unitLabel = equipmentLabel(part)
2581     local repairPrompt = Instance.new("ProximityPrompt")
2582     repairPrompt.Name = "RepairPrompt"
2583     repairPrompt.ObjectText = floorLabel .. " • " .. unitLabel
2584     repairPrompt.ActionText = "Hold R to repair"
2585     repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
2586     repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
2587     repairPrompt.HoldDuration = 3
2588     repairPrompt.MaxActivationDistance = 8
2589     repairPrompt.RequiresLineOfSight = true
2590     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
2591     repairPrompt.Parent = repairAnchor
2592
2593     part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
2594         if repairPrompt.Parent then
2595             repairPrompt.Enabled = not busy and part:GetAttribute("FunctionalState") == "FAILED"
2596         end
2597     end)
2598
2599     repairPrompt.Triggered:Connect(function(player)
2600         if playerRole(player) ~= "Technician" then
2601             setReadout("TECHNICIAN REQUIRED", "Choose the BLUE role to repair equipment", Color3.fromRGB(255,
                ↪ 175, 80))
2602             notify(player, "Only a Technician can repair equipment. Choose the BLUE role kiosk.")
2603             return

```

```

2604     end
2605     if busy or part:GetAttribute("FunctionalState") ~= "FAILED"
2606         or not canInteract(player, part, repairPrompt.MaxActivationDistance) then return end
2607     busy = true
2608     prompt.Enabled = false
2609     repairPrompt.Enabled = false
2610     setReadout("REPAIRING", floorLabel .. " " * " .. unitLabel .. " " * diagnosing "
2611         .. (part:GetAttribute("FailureReason") or "equipment fault"), Color3.fromRGB(255, 205, 75))
2612     setReadout("REPAIRING", "Working on " .. floorLabel .. " " * " .. unitLabel, Color3.fromRGB(120, 235,
2613         ↪ 170))
2614     if not runMathQuiz(player, "REPAIR " .. floorLabel .. " " * " .. unitLabel) then
2615         setReadout("REPAIR NOT COMPLETED", "Hold R for 3 seconds to try another quiz", Color3.fromRGB(255,
2616             ↪ 105, 90))
2617         prompt.Enabled = true
2618         repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
2619         busy = false
2620         return
2621     end
2622     local assigned = player:GetAttribute("AssignedRepairId") == equipmentId(part)
2623     local points, failedSeconds, routeDistance = repairReward(player, part, assigned)
2624     part:SetAttribute("LastRepairedAt", os.time())
2625     part:SetAttribute("LastRepairedBy", player.Name)
2626     if kind == PLANT_CHAMBER_KIND then setChamberFailure(part, false) else setGenericFailure(part, false)
2627     ↪ end
2628     GameState.repairsCompleted = GameState.repairsCompleted + 1
2629     if assigned then GameState.assignedRepairsCompleted = GameState.assignedRepairsCompleted + 1 end
2630     local total = awardPoints(player, points, string.format("%s%s * failed %ds * route %d studs",
2631         assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
2632         ↪ math.floor(routeDistance + 0.5)))
2633     setReadout("REPAIR COMPLETE +" .. points, floorLabel .. " " * " .. unitLabel .. " WORKING * TOTAL " ..
2634         ↪ total,
2635         Color3.fromRGB(105, 245, 155))
2636     assignNextTask(player)
2637     prompt.Enabled = true
2638     busy = false
2639 end)
2640
2641 prompt.Triggered:Connect(function(player)
2642     if busy or not canInteract(player, part, prompt.MaxActivationDistance) then return end
2643     task.spawn(pressButton)
2644     if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
2645         setReadout("FAILED * HOLD R", equipmentFloorLabel(part) .. " " * " ..
2646             ↪ (part:GetAttribute("FailureReason") or "equipment fault"),
2647             Color3.fromRGB(255, 105, 90))
2648         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
2649         return
2650     end
2651
2652     local station = ROLE_STATIONS[kind]
2653     if station then
2654         if not player.Team or player.Team.Name ~= station.role then
2655             setReadout("ACCESS DENIED", station.role .. " role required", Color3.fromRGB(255, 105, 90))
2656             notify(player, kind .. ": only a " .. station.role .. " can use this. Join that role at the
2657             ↪ entrance kiosk.")
2658             return
2659         end
2660         busy = true
2661         prompt.Enabled = false
2662         runProgress(spec.wait or 1, "Reading shared batch queue", setReadout)
2663         local msg, err = station.run(GameState)
2664         if err then
2665             setReadout("NO BATCH DATA", err, Color3.fromRGB(255, 160, 90))
2666             notify(player, kind .. ": " .. err)
2667         else
2668             addContribution(player)
2669             setReadout("INSPECTION DATA", msg, Color3.fromRGB(115, 245, 150))
2670             notify(player, kind .. " -- " .. msg)
2671             updateScoreboard()
2672         end
2673         prompt.Enabled = true
2674         busy = false
2675         return
2676     end
2677
2678     local plantProcess = PLANT_PROCESS[kind]
2679     local carriedPlant = findPlantSample(player)

```

```

2673     local shouldUsePlantStorage = isPlantContainer and (
2674         kind == PLANT_CHAMBER_KIND
2675         or not plantProcess
2676         or not carriedPlant
2677         or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
2678     )
2679     if shouldUsePlantStorage then
2680         busy = true
2681         prompt.Enabled = false
2682         handleGrowthChamber(player, part, setReadout)
2683         prompt.Enabled = true
2684         busy = false
2685         return
2686     end
2687
2688     if part:GetAttribute("FunctionalState") == "FAILED" then
2689         setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
2690             ↳ (part:GetAttribute("FailureReason") or "equipment fault"),
2691             Color3.fromRGB(255, 105, 90))
2692         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
2693         return
2694     end
2695
2696     if plantProcess then
2697         busy = true
2698         prompt.Enabled = false
2699         processPlant(player, part, spec, plantProcess, setReadout)
2700         prompt.Enabled = true
2701         busy = false
2702         return
2703     end
2704
2705     if COLD_STORE[kind] then
2706         busy = true
2707         prompt.Enabled = false
2708         runProgress(0.6, "Scanning cold-store inventory", setReadout)
2709         handleColdStore(player, part, setReadout)
2710         prompt.Enabled = true
2711         busy = false
2712         return
2713     end
2714
2715     local consumed
2716     if spec.consumes then
2717         consumed = findTool(player, spec.consumes)
2718         if not consumed then
2719             setReadout("MATERIAL NEEDED", spec.consumes, Color3.fromRGB(255, 160, 90))
2720             notify(player, kind .. ": needs " .. spec.consumes .. " first.")
2721             return
2722         end
2723     end
2724
2725     busy = true
2726     prompt.Enabled = false
2727     if spec.activeColour then part.Color = spec.activeColour end
2728     local uv
2729     if spec.uvLight then
2730         uv = Instance.new("PointLight")
2731         uv.Color = Color3.fromRGB(150, 210, 255)
2732         uv.Brightness = 2
2733         uv.Range = 26
2734         uv.Parent = part
2735     end
2736
2737     local _, stopIndicator = startCycleIndicator(part)
2738     runProgress(spec.wait or 1, spec.action, setReadout)
2739     stopIndicator()
2740
2741     if consumed then destroyCarriedItem(player, consumed) end
2742     for _, item in ipairs(spec.produces or {}) do
2743         local made = makeTool(player, item)
2744         if made and made:GetAttribute("LabSample") then setCarryState(player, made) end
2745     end
2746     local result = spec.log and spec.log() or "Inspection complete; equipment left ready."
2747     setReadout("INSPECTION DATA", result, Color3.fromRGB(115, 245, 150))
2748     notify(player, kind .. " -- " .. result)

```

```

2748
2749     if uv then uv:Destroy() end
2750     part.Color = baseColour
2751     prompt.Enabled = true
2752     busy = false
2753 end)
2754 end
2755
2756 -----
2757 -- FIFTY AUTONOMOUS GAMEPLAY TEST WORKERS
2758 -----
2759 local function setBotStatus(bot, message)
2760     bot:SetAttribute("TaskState", message)
2761     local status = bot:FindFirstChild("Status", true)
2762     if status and status:IsA("TextLabel") then
2763         status.Text = bot:GetAttribute("Competitor") and ("RIVAL NPC\n" .. message)
2764         or string.format("BOT %02d\n%s", bot:GetAttribute("BotNumber") or 0, message)
2765     end
2766 end
2767
2768 local function updateBotTestBoard()
2769     local workers = CollectionService:GetTagged("LabWorkerBot")
2770     local cycles, carrying, fallbacks, rivalScore = 0, 0, 0, 0
2771     for _, bot in ipairs(workers) do
2772         cycles = cycles + (bot:GetAttribute("CompletedCycles") or 0)
2773         fallbacks = fallbacks + (bot:GetAttribute("PathFallbacks") or 0)
2774         if (bot:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
2775         if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 end
2776     end
2777     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
2778     local failing, failedSamples = 0, 0
2779     for _, chamber in ipairs(chambers) do
2780         if chamber:GetAttribute("FunctionalState") == "FAILED" then
2781             failing = failing + 1
2782             failedSamples = failedSamples + (chamber:GetAttribute("SampleCount") or 0)
2783         end
2784     end
2785     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
2786         for _, label in ipairs(board:GetDescendants()) do
2787             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
2788                 label.Text = string.format(
2789                     "ADAPTIVE GROWTH-ROOM TEST\nWorkers: %d/50 Transfers: %d Carrying: %d\nFAILED: %d/%d
2790                     ↳ (%.0f%%) Samples waiting: %d\nRIVAL SCORE: %d • Path fallbacks: %d",
2791                     #workers, cycles, carrying, failing, #chambers, failing / math.max(1, #chambers) * 100,
2792                     ↳ failedSamples, rivalScore, fallbacks)
2793             end
2794         end
2795     end
2796 end
2797
2798 local function interactionPoint(part)
2799     local side = part:GetAttribute("WallSide")
2800     local bottomY = part.Position.Y - part.Size.Y / 2
2801     local point = Vector3.new(part.Position.X, bottomY + 3.25, part.Position.Z)
2802     if side == "W" then point = point + Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
2803     elseif side == "E" then point = point - Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
2804     elseif side == "N" then point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
2805     elseif side == "S" then point = point - Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
2806     else point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5) end
2807     return point
2808 end
2809
2810 -----
2811 -- OBSERVER GATE
2812 -- A worker only runs its scenario while a player is in the room with it.
2813 -- Everything else stands still. This is what keeps 180 NPCs off the
2814 -- pathfinding budget: an unobserved bot costs one cheap check a second
2815 -- instead of a continuous walk loop and a PathfindingService computation
2816 -- every few seconds.
2817 -----
2818 local FLOOR_BAND = 10 -- studs, about half a floor-to-floor height
2819 local WAKE_RADIUS = 35 -- studs, fallback for corridors and doorways, where
2820 -- neither the bot nor the player is inside any room
2821
2822 -- Room rectangles published by the generator as one packed attribute:
2823 -- "Level_1:Room 1:x1:z1:x2:z2:y;..." already in stud space.

```

```

2822 local roomsByLevel = {}
2823 do
2824     local complex = workspace:FindFirstChild("Laboratory_Complex")
2825     local packed = complex and complex:GetAttribute("RoomIndex")
2826     if typeof(packed) == "string" then
2827         for entry in packed:gmatch("[^;]+") do
2828             local level, name, x1, z1, x2, z2, y =
2829                 entry:match("^([^:]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+)$")
2830             if level then
2831                 roomsByLevel[level] = roomsByLevel[level] or {}
2832                 table.insert(roomsByLevel[level], {
2833                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
2834                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
2835                 })
2836             end
2837         end
2838     end
2839 end
2840
2841 -- Only the ~10 rooms on the bot's own floor are ever tested, not all 81.
2842 local function roomAt(levelName, position)
2843     local list = levelName and roomsByLevel[levelName]
2844     if not list then return nil end
2845     for _, r in ipairs(list) do
2846         if math.abs(position.Y - r.y) < FLOOR_BAND
2847             and position.X >= r.x1 and position.X <= r.x2
2848             and position.Z >= r.z1 and position.Z <= r.z2 then
2849             return r.name
2850         end
2851     end
2852     return nil
2853 end
2854
2855 -- The rectangle itself, for picking a patrol point inside a room.
2856 local function roomRect(levelName, roomName)
2857     for _, r in ipairs((levelName and roomsByLevel[levelName]) or {}) do
2858         if r.name == roomName then return r end
2859     end
2860     return nil
2861 end
2862
2863 local function botHasObserver(bot)
2864     local ok, pivot = pcall(function() return bot:GetPivot().Position end)
2865     if not ok then return false end
2866     local levelName = bot:GetAttribute("Level")
2867     local botRoom = roomAt(levelName, pivot)
2868     for _, player in ipairs(Players:GetPlayers()) do
2869         local character = player.Character
2870         local hrp = character and character:FindFirstChild("HumanoidRootPart")
2871         if hrp then
2872             local here = hrp.Position
2873             -- Cheapest test first: wrong floor is an instant no.
2874             if math.abs(here.Y - pivot.Y) < FLOOR_BAND then
2875                 if botRoom and roomAt(levelName, here) == botRoom then return true end
2876                 local dx, dz = here.X - pivot.X, here.Z - pivot.Z
2877                 if dx * dx + dz * dz < WAKE_RADIUS * WAKE_RADIUS then return true end
2878             end
2879         end
2880     end
2881     return false
2882 end
2883
2884 local function walkBotSegment(bot, destination)
2885     if not bot.Parent or not bot.PrimaryPart then return false end
2886     local startPivot = bot:GetPivot()
2887     local start = startPivot.Position
2888     local heldOffsets = {}
2889     local held = bot:FindFirstChild("BotHeldPlant")
2890     if held then
2891         for _, piece in ipairs(held:GetDescendants()) do
2892             if piece:IsA("BasePart") then heldOffsets[piece] = startPivot:ToObjectSpace(piece.CFrame) end
2893         end
2894     end
2895     -- Refuse to walk a leg that passes through solid geometry. RespectCanCollide
2896     -- means only real obstructions count -- walls, machines and closed doors --
2897     -- while posters, wall cabinets and other workers (all CanCollide off) are

```

```

2898 -- ignored. Without this the walk simply lerped in a straight line and slid
2899 -- through walls and equipment whenever pathfinding gave a coarse waypoint.
2900 local blockParams = RaycastParams.new()
2901 blockParams.FilterType = Enum.RaycastFilterType.Exclude
2902 blockParams.FilterDescendantsInstances = {bot, workspace:FindFirstChild("LabBotBodies")}
2903 blockParams.RespectCanCollide = true
2904 local flat = Vector3.new(destination.X, start.Y, destination.Z)
2905 if (flat - start).Magnitude > 0.5 then
2906     local blocked = workspace:Raycast(start, flat - start, blockParams)
2907     if blocked then return false end
2908 end
2909
2910 local flatDistance = (flat - start).Magnitude
2911 local steps = math.max(1, math.ceil(flatDistance / 1.25))
2912 for step = 1, steps do
2913     if not bot.Parent then return false end
2914     if bot:GetAttribute("Dead") then return false end
2915     while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do task.wait(0.25)
2916         ↪ end
2917     -- Stop dead the moment the room empties, and resume from the same spot
2918     -- when someone comes back. Without this a bot that was already walking
2919     -- would finish its whole segment after the player had left.
2920     while bot.Parent and not bot:GetAttribute("Competitor") and not botHasObserver(bot) do
2921         task.wait(0.5)
2922     end
2923     if not bot.Parent then return false end
2924     local alpha = step / steps
2925     local position = start:Lerp(destination, alpha)
2926     local flatLook = Vector3.new(destination.X, position.Y, destination.Z)
2927     local pose = (flatLook - position).Magnitude > 0.05
2928     and CFrame.lookAt(position, flatLook) or CFrame.new(position)
2929     -- A subtle sway makes the anchored test rig visibly walk instead of
2930     -- silently teleporting between test stations.
2931     pose = pose * CFrame.Angles(0, 0, math.sin(step * 0.9) * 0.025)
2932     bot:PivotTo(pose)
2933     for piece, offset in pairs(heldOffsets) do
2934         if piece.Parent then piece.CFrame = pose * offset end
2935     end
2936     local complex = workspace:FindFirstChild("Laboratory_Complex")
2937     local throughput = complex and (complex:GetAttribute("RecentTransferThroughput") or 0) or 0
2938     local speedScale = 1 + math.min(10, #Players:GetPlayers()) * 0.05 + math.clamp(throughput, 0, 20) *
2939         ↪ 0.01
2940     task.wait(math.clamp(0.075 / speedScale, 0.035, 0.09))
2941 end
2942
2943 local function moveBotTo(bot, target)
2944     local destination = interactionPoint(target)
2945     local path = PathfindingService:CreatePath({
2946         AgentRadius = 1.2,
2947         AgentHeight = 5.8,
2948         AgentCanJump = true,
2949         WaypointSpacing = 5,
2950     })
2951     local ok = pcall(function()
2952         path:ComputeAsync(bot:GetPivot().Position, destination)
2953     end)
2954     if ok and path.Status == Enum.PathStatus.Success then
2955         for _, waypoint in ipairs(path:GetWaypoints()) do
2956             if not walkBotSegment(bot, waypoint.Position + Vector3.new(0, 3.0, 0)) then return false end
2957         end
2958         return true
2959     end
2960     -- A failed navigation calculation used to fall back to a straight line to
2961     -- the destination, which is exactly how a worker ends up strolling through
2962     -- a wall. Give up on this leg instead and let the scenario pick again.
2963     bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
2964     return false
2965 end
2966
2967 local function makeBotPlant(bot, stage)
2968     local old = bot:FindFirstChild("BotHeldPlant")
2969     if old then old:Destroy() end
2970     local held = Instance.new("Model")
2971     held.Name = "BotHeldPlant"

```

```

2972     held.Parent = bot
2973     local base = bot:GetPivot() * CFrame.new(1.45, 0.1, -0.72)
2974     local function piece(name, size, offset, colour, material)
2975         local p = Instance.new("Part")
2976         p.Name = name
2977         p.Size = size
2978         p.CFrame = base * CFrame.new(offset)
2979         p.Color = colour
2980         p.Material = material or Enum.Material.SmoothPlastic
2981         p.Anchored = true
2982         p.CanCollide = false
2983         p.CanTouch = false
2984         p.CanQuery = false
2985         p.Parent = held
2986         return p
2987     end
2988     piece("Pot", Vector3.new(0.85, 0.62, 0.85), Vector3.new(0, 0, 0), Color3.fromRGB(118, 77, 46))
2989     piece("Stem", Vector3.new(0.13, 0.75 + stage * 0.13, 0.13), Vector3.new(0, 0.65, 0), Color3.fromRGB(65,
↵ 165, 72))
2990     piece("LeafL", Vector3.new(0.48, 0.12, 0.3), Vector3.new(-0.24, 0.78, 0), Color3.fromRGB(75, 185, 82))
2991     piece("LeafR", Vector3.new(0.48, 0.12, 0.3), Vector3.new(0.24, 1.0, 0), Color3.fromRGB(75, 185, 82))
2992     return held
2993 end
2994
2995 local function equipmentReadout(part, top, bottom, colour)
2996     local label = part:FindFirstChild("Readout", true)
2997     if label and label:IsA("TextLabel") then
2998         label.Text = top .. "\n" .. bottom
2999         label.TextColor3 = colour or Color3.fromRGB(115, 245, 150)
3000     end
3001 end
3002
3003 local function candidatesOnLevel(levelName, predicate)
3004     local result = {}
3005     for _, item in ipairs(CollectionService:GetTagged("LabEquipment")) do
3006         if item.Parent and item:GetAttribute("Level") == levelName and predicate(item) then
3007             table.insert(result, item)
3008         end
3009     end
3010     return result
3011 end
3012
3013 local function runBotScenario(bot)
3014     -- Remember where this worker started, so a shift reset can stand it back up.
3015     if not bot:GetAttribute("HomeCFrame") then
3016         pcall(function() bot:SetAttribute("HomeCFrame", bot:GetPivot()) end)
3017     end
3018     -- Desk staff stay at their desk. They are still perfectly good targets.
3019     if bot:GetAttribute("Seated") then
3020         setBotStatus(bot, "At the desk")
3021         while bot.Parent and not bot:GetAttribute("Dead") do task.wait(5) end
3022         return
3023     end
3024
3025     while bot.Parent and not bot:GetAttribute("Dead") do
3026         -- The rival keeps running wherever it is: it is a single NPC and the
3027         -- competitive scoring loop depends on it making progress.
3028         local awake = bot:GetAttribute("Competitor") or botHasObserver(bot)
3029         if not awake then
3030             setBotStatus(bot, "Idle -- no one in the room")
3031             task.wait(1)
3032         elseif bot:GetAttribute("KnockedOut") then task.wait(1) else
3033             local levelName = bot:GetAttribute("Level")
3034             local failed = candidatesOnLevel(levelName, function(item)
3035                 return item:GetAttribute("Kind") == PLANT_CHAMBER_KIND
3036                     and item:GetAttribute("FunctionalState") == "FAILED"
3037                     and (item:GetAttribute("SampleCount") or 0) > 0
3038                     and not item:GetAttribute("ReservedByBot")
3039             end)
3040             if #failed == 0 then
3041                 -- Nothing has failed on this floor right now. The adaptive
3042                 -- difficulty tick deliberately holds the failure rate down when
3043                 -- few players are online, so this is the common case rather than
3044                 -- the rare one -- and standing frozen in front of a player who
3045                 -- just walked in is exactly what "the NPCs don't move" looked
3046                 -- like. Patrol the room instead so the floor reads as staffed.

```



```

3047     -- Only ever runs for a bot somebody is actually watching.
3048     setBotStatus(bot, "Checking chambers")
3049     local patrolRoom = roomAt(levelName, bot:GetPivot().Position)
3050     local rect = patrolRoom and roomRect(levelName, patrolRoom)
3051     if rect then
3052         local here = bot:GetPivot().Position
3053         local spanX = math.max(1, (rect.x2 - rect.x1) - 12)
3054         local spanZ = math.max(1, (rect.z2 - rect.z1) - 12)
3055         walkBotSegment(bot, Vector3.new(
3056             rect.x1 + 6 + math.random() * spanX,
3057             here.Y,
3058             rect.z1 + 6 + math.random() * spanZ))
3059         task.wait(0.5 + math.random() * 1.5)
3060     else
3061         task.wait(3)
3062     end
3063 else
3064     table.sort(failed, function(a, b)
3065         local an = tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
3066         local bn = tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
3067         return an < bn
3068     end)
3069     -- Prefer a job in the room the worker is already standing in --
3070     -- which, because it only wakes when a player is there, is the
3071     -- room the player is watching. Without this a woken worker
3072     -- usually picks a chamber elsewhere on the floor, walks out of
3073     -- the room and stops the moment it loses its observer.
3074     local botRoom = roomAt(levelName, bot:GetPivot().Position)
3075     local sameRoom = {}
3076     if botRoom then
3077         for _, item in ipairs(failed) do
3078             if item:GetAttribute("Room") == botRoom then table.insert(sameRoom, item) end
3079         end
3080     end
3081     local pool = (#sameRoom > 0) and sameRoom or failed
3082     local source = bot:GetAttribute("Competitor") and pool[1] or pool[math.random(1, #pool)]
3083     local destination = evacuationDestinations(source)[1]
3084     if not destination then
3085         setBotStatus(bot, "Waiting for safe capacity")
3086         task.wait(3)
3087     else
3088         local botId = bot.Name
3089         source:SetAttribute("ReservedByBot", botId)
3090         destination:SetAttribute("IncomingReservations",
3091             ↳ (destination:GetAttribute("IncomingReservations") or 0) + 1)
3092         source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode") or "")
3093         refreshChamberTelemetry(source)
3094         setBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or "alarm"))
3095         moveBotTo(bot, source)
3096         local samples = chamberSampleList(source)
3097         local plantId = table.remove(samples, 1)
3098         if plantId and source:GetAttribute("ReservedByBot") == botId then
3099             saveChamberSamples(source, samples)
3100             updateStoredSamples(source)
3101             refreshChamberTelemetry(source)
3102             local stage = math.clamp(source:GetAttribute("GrowthStage") or 0, 0, 5)
3103             makeBotPlant(bot, stage)
3104             bot:SetAttribute("CarryingPlant", plantId)
3105             setBotStatus(bot, "Evacuating " .. plantId)
3106             equipmentReadout(source, (source:GetAttribute("GrowthCode") or "R?") .. " FAILED",
3107                 ↳ "SEND " .. plantId .. " TO " .. (destination:GetAttribute("GrowthCode") or "R?"),
3108                 ↳ Color3.fromRGB(255, 105, 90))
3109
3110             -- Emergency transfers still receive one quick development step,
3111             -- so the same growth mechanics are continuously regression-tested.
3112             local stations = candidatesOnLevel(levelName, function(item) return
3113                 ↳ PLANT_PROCESS[item:GetAttribute("Kind")] ~= nil end)
3114             if #stations > 0 then
3115                 local station = stations[((bot:GetAttribute("BotNumber") or 1) - 1) % #stations +
3116                     ↳ 1]
3117                 local process = PLANT_PROCESS[station:GetAttribute("Kind")]
3118                 moveBotTo(bot, station)
3119                 stage = math.clamp(stage + (process.gain or 0), 0, 5)
3120                 station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or 0) + 1)
3121                 equipmentReadout(station, "BOT GROWTH TEST", plantId .. " • stage " .. stage,
3122                     ↳ Color3.fromRGB(120, 235, 165))

```

```

3117         end
3118
3119         setBotStatus(bot, "Delivering to " .. (destination:GetAttribute("GrowthCode") or "R?"))
3120         moveBotTo(bot, destination)
3121         local destinationSamples = chamberSampleList(destination)
3122         if destination:GetAttribute("FunctionalState") ~= "FAILED" and #destinationSamples <
            ↳ (destination:GetAttribute("Capacity") or 10) then
3123             table.insert(destinationSamples, plantId)
3124             saveChamberSamples(destination, destinationSamples)
3125             destination:SetAttribute("GrowthStage",
            ↳ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
3126             updateStoredSamples(destination)
3127             refreshChamberTelemetry(destination)
3128             emergencyMovesCompleted = emergencyMovesCompleted + 1
3129             bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or 0) + 1)
3130             if bot:GetAttribute("Competitor") then
3131                 local gained = POINTS.MOVE_ASSIGNED_ON_TIME
3132                 bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) + gained)
3133                 setBotStatus(bot, "+" .. gained .. " resource win")
3134                 updateScoreboard()
3135             else
3136                 setBotStatus(bot, "Transfer passed")
3137             end
3138         else
3139             -- Capacity changed while walking; return the sample to the source
3140             -- rather than deleting it and let the next cycle choose again.
3141             table.insert(samples, plantId)
3142             saveChamberSamples(source, samples)
3143             updateStoredSamples(source)
3144             refreshChamberTelemetry(source)
3145             setBotStatus(bot, "Destination changed; retrying")
3146         end
3147         local held = bot:FindFirstChild("BotHeldPlant")
3148         if held then held:Destroy() end
3149         bot:SetAttribute("CarryingPlant", "")
3150     end
3151     destination:SetAttribute("IncomingReservations", math.max(0,
            ↳ (destination:GetAttribute("IncomingReservations") or 1) - 1))
3152     if source:GetAttribute("ReservedByBot") == botId then source:SetAttribute("ReservedByBot",
            ↳ nil) end
3153     updateBotTestBoard()
3154     task.wait(1.5 + math.random())
3155 end
3156     end
3157 end
3158 end
3159 end
3160
3161 -----
3162 -- ADAPTIVE, INDEFINITE CHAMBER FAILURE SIMULATION
3163 -----
3164 local FAILURE_REASONS = {
3165     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
3166     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
3167 }
3168 local generatedSampleSerial = 20000
3169 local previousCompletedMoves = 0
3170
3171 setChamberFailure = function(chamber, failed)
3172     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
3173     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
3174     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
3175     if failed then
3176         chamber:SetAttribute("FunctionalState", "FAILED")
3177         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
3178         chamber:SetAttribute("RepairKey", "R")
3179         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " • HOLD R
            ↳ TO REPAIR")
3180         chamber:SetAttribute("FailureReason", FAILURE_REASONS[math.random(1, #FAILURE_REASONS)])
3181         chamber:SetAttribute("FailureSince", os.time())
3182         chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
3183         chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
3184         chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
3185         local destination = evacuationDestinations(chamber)[1]
3186         chamber:SetAttribute("EvacuationDestination", destination and destination:GetAttribute("GrowthCode") or
            ↳ "")

```

```

3187     for _, player in ipairs(Players:GetPlayers()) do
3188         if playerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
3189             ↪ assignNextTask(player) end
3190         if playerRole(player) == "Laboratory Technician" and not
3191             ↪ player:GetAttribute("AssignedTransferSource") then assignNextTask(player) end
3192         if playerRole(player) == "Laboratory Technician" then
3193             notify(player, "PLANT ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Press E to
3194                 ↪ remove plants and follow the assigned destination.")
3195         else
3196             notify(player, "REPAIR ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Technician:
3197                 ↪ HOLD R to repair.")
3198         end
3199     end
3200 else
3201     chamber:SetAttribute("FunctionalState", "WORKING")
3202     chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
3203     chamber:SetAttribute("FailureReason", "")
3204     chamber:SetAttribute("FailureSince", 0)
3205     chamber:SetAttribute("EvacuationDestination", "")
3206     chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
3207     chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
3208     chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
3209     chamber:SetAttribute("OperationStarted", os.time())
3210     if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) < 100
3211         ↪ then
3212         recoverPlantLife(chamber, chamber:GetAttribute("PlantLife"), nil, chamber:GetAttribute("PlantId"))
3213     end
3214 end
3215 refreshChamberTelemetry(chamber)
3216 end
3217
3218 local GENERIC_FAILURE_REASONS = {
3219     "Power relay fault", "Controllor communication loss", "Pump interlock fault",
3220     "Calibration expired", "Safety sensor trip", "Motor overload",
3221 }
3222
3223 setGenericFailure = function(part, failed)
3224     local floorLabel = equipmentFloorLabel(part)
3225     local unitLabel = equipmentLabel(part)
3226     if failed then
3227         part:SetAttribute("FunctionalState", "FAILED")
3228         part:SetAttribute("RepairKey", "R")
3229         part:SetAttribute("FailureReason", GENERIC_FAILURE_REASONS[math.random(1, #GENERIC_FAILURE_REASONS)])
3230         part:SetAttribute("FailureSince", os.time())
3231         part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • HOLD R")
3232         equipmentReadout(part, "FAILED • HOLD R", floorLabel .. " • " .. unitLabel .. " • PLANTS "
3233             .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • "
3234             ↪ .. part:GetAttribute("FailureReason"),
3235             Color3.fromRGB(255, 105, 90))
3236         for _, player in ipairs(Players:GetPlayers()) do
3237             if playerRole(player) == "Technician" then
3238                 if not player:GetAttribute("AssignedRepairId") then assignNextTask(player) end
3239                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • " .. unitLabel .. " FAILED. HOLD R to
3240                     ↪ repair.")
3241             end
3242         end
3243     else
3244         part:SetAttribute("FunctionalState", "WORKING")
3245         part:SetAttribute("FailureReason", "")
3246         part:SetAttribute("FailureSince", 0)
3247         part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
3248         part:SetAttribute("OperationStarted", os.time())
3249         equipmentReadout(part, "WORKING", floorLabel .. " • " .. unitLabel .. " • PLANTS "
3250             .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • E
3251             ↪ USE / INSPECT",
3252             Color3.fromRGB(110, 235, 170))
3253     end
3254 end
3255
3256 local function shuffled(list)
3257     for i = #list, 2, -1 do
3258         local j = math.random(1, i)
3259         list[i], list[j] = list[j], list[i]
3260     end
3261     return list
3262 end

```

```

3255
3256 local function adaptiveFailureTick()
3257     local chambers = CollectionService.GetTagged("PlantGrowthChamber")
3258     local failed, working = {}, {}
3259     local failedSamples, totalSamples, workingFree = 0, 0, 0
3260     for _, chamber in ipairs(chambers) do
3261         local count = chamber:GetAttribute("SampleCount") or 0
3262         local capacity = chamber:GetAttribute("Capacity") or 10
3263         totalSamples = totalSamples + count
3264         if chamber:GetAttribute("FunctionalState") == "FAILED" then
3265             table.insert(failed, chamber)
3266             failedSamples = failedSamples + count
3267         else
3268             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then
3269                 table.insert(working, chamber)
3270             end
3271             workingFree = workingFree + math.max(0, capacity - count -
3272                 ↪ (chamber:GetAttribute("IncomingReservations") or 0))
3273         end
3274     end
3275     local throughput = emergencyMovesCompleted - previousCompletedMoves
3276     previousCompletedMoves = emergencyMovesCompleted
3277     local backlogRatio = failedSamples / math.max(1, totalSamples)
3278     -- Sixty percent is the operating centre. Fast rescues allow a few more
3279     -- alarms; a large backlog or exhausted safe capacity automatically eases
3280     -- the load so 50 NPCs plus real players can keep the loop moving forever.
3281     local playerCount = math.max(1, #Players:GetPlayers())
3282     local speedAdjustment = math.clamp((throughput - 5) * 0.003, -0.03, 0.08)
3283     local playerAdjustment = math.clamp((playerCount - 1) * 0.012, 0, 0.10)
3284     local backlogAdjustment = -math.clamp((backlogRatio - 0.60) * 0.08, -0.04, 0.04)
3285     local capacityAdjustment = workingFree < 20 and -0.12 or 0
3286     local targetRate = math.clamp(0.60 + speedAdjustment + playerAdjustment + backlogAdjustment +
3287         ↪ capacityAdjustment, 0.42, 0.76)
3288     local targetFailed = math.floor(#chambers * targetRate + 0.5)
3289     local maxChange = math.clamp(2 + playerCount * 2, 4, 12)
3290     local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
3291     if change > 0 then
3292         local eligible = {}
3293         for _, chamber in ipairs(working) do
3294             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then table.insert(eligible,
3295                 ↪ chamber) end
3296         end
3297         shuffled(eligible)
3298         for i = 1, math.min(change, #eligible) do setChamberFailure(eligible[i], true) end
3299     elseif change < 0 then
3300         table.sort(failed, function(a, b)
3301             local ac, bc = a:GetAttribute("SampleCount") or 0, b:GetAttribute("SampleCount") or 0
3302             if ac == bc then return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or
3303                 ↪ 0) end
3304             return ac < bc
3305         end)
3306         for i = 1, math.min(-change, #failed) do setChamberFailure(failed[i], false) end
3307     end
3308     -- Failed telemetry continues to drift; working chambers continuously
3309     -- germinate replacement samples so the transfer game never runs out.
3310     for _, chamber in ipairs(chambers) do
3311         if chamber:GetAttribute("FunctionalState") == "FAILED" then
3312             chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
3313                 ↪ math.random(-2, 2), 28, 42))
3314             chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
3315                 ↪ math.random(-4, 4), 25, 65))
3316             chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
3317                 ↪ math.random(-5, 3), 0, 48))
3318             local destination = evacuationDestinations(chamber)[1]
3319             chamber:SetAttribute("EvacuationDestination", destination and
3320                 ↪ destination:GetAttribute("GrowthCode") or "")
3321         else
3322             local samples = chamberSampleList(chamber)
3323             local capacity = chamber:GetAttribute("Capacity") or 10
3324             local refillChance = math.clamp(0.08 + (capacity - #samples) * 0.025 + throughput * 0.002
3325                 ↪ + playerCount * 0.008, 0.08, 0.46)
3326             if #samples < capacity and math.random() < refillChance then
3327                 generatedSampleSerial = generatedSampleSerial + 1

```

```

3322         table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
3323         ↪ generatedSampleSerial))
3323         saveChamberSamples(chamber, samples)
3324         chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
3325         updateStoredSamples(chamber)
3326     end
3327 end
3328 refreshChamberTelemetry(chamber)
3329 end
3330 local complex = workspace:FindFirstChild("Laboratory_Complex")
3331 if complex then
3332     complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
3333     complex:SetAttribute("FailingChambers", #failed)
3334     complex:SetAttribute("FailedSampleBacklog", failedSamples)
3335     complex:SetAttribute("EmergencyMovesCompleted", emergencyMovesCompleted)
3336     complex:SetAttribute("RecentTransferThroughput", throughput)
3337     complex:SetAttribute("ActivePlayerCount", playerCount)
3338     complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples /
3339     ↪ 20), 8, 50))
3339     -- Per-FL00R budget. Multiplied across the basement plus every level
3340     -- this is what actually decides how many workers are visibly busy.
3341     complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
3342     ↪ math.floor(failedSamples / 60), 6, 20))
3342 end
3343 end
3344
3345 local function genericFailureTick()
3346     local failed, working = {}, {}
3347     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
3348         if equipment:GetAttribute("Kind") ~= PLANT_CHAMBER_KIND then
3349             if equipment:GetAttribute("FunctionalState") == "FAILED" then
3350                 table.insert(failed, equipment)
3351             elseif os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 45 then
3352                 table.insert(working, equipment)
3353             end
3354         end
3355     end
3356     local playerCount = math.max(1, #Players:GetPlayers())
3357     local targetRate = math.clamp(0.12 + playerCount * 0.018, 0.14, 0.28)
3358     local targetFailed = math.max(2, math.floor((#failed + #working) * targetRate + 0.5))
3359     local maxChange = math.clamp(1 + math.ceil(playerCount / 2), 2, 6)
3360     local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
3361     if change > 0 then
3362         shuffled(working)
3363         for i = 1, math.min(change, #working) do setGenericFailure(working[i], true) end
3364     elseif change < 0 then
3365         table.sort(failed, function(a, b) return (a:GetAttribute("FailureSince") or 0) <
3366         ↪ (b:GetAttribute("FailureSince") or 0) end)
3367         for i = 1, math.min(-change, #failed) do setGenericFailure(failed[i], false) end
3368     end
3369 end
3370
3371 local function bindPlayerTaskRouting(player)
3372     if player:GetAttribute("EquipmentTaskRoutingBound") then return end
3373     player:SetAttribute("EquipmentTaskRoutingBound", true)
3374     player:GetAttributeChangedSignal("LabRole"):Connect(function()
3375         task.delay(0.2, function()
3376             if player.Parent then assignNextTask(player) end
3377         end)
3378     end)
3379     player.CharacterAdded:Connect(function()
3380         if playerRole(player) == "Technician" and player:GetAttribute("AssignedRepairId") then
3381             task.delay(1, function() if player.Parent then assignNextTask(player) end end)
3382         end
3383     end)
3384 end
3385
3386 for _, player in ipairs(Players:GetPlayers()) do bindPlayerTaskRouting(player) end
3387 Players.PlayerAdded:Connect(bindPlayerTaskRouting)
3388 Players.PlayerRemoving:Connect(clearRepairPath)
3389
3390 for _, p in ipairs(CollectionService:GetTagged("LabEquipment")) do bind(p) end
3391 CollectionService.GetInstanceAddedSignal("LabEquipment"):Connect(bind)
3392
3393 -----
3394 -- AUTOMATIC DOORS

```

```

3394 -- Every AutoDoor panel slides up into the header void when a player's
3395 -- HumanoidRootPart comes within range, and back down once nobody is close,
3396 -- so rooms have real doors instead of a permanently open gap.
3397 -----
3398 local DOOR_OPEN_RANGE = 11
3399 local DOOR_CLOSE_RANGE = 15
3400 local DOOR_MIN_OPEN_SECONDS = 3
3401 local function bindAutoDoor(door)
3402     if door:GetAttribute("DoorBound") then return end
3403     door:SetAttribute("DoorBound", true)
3404     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
3405     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
3406     local openX = door:GetAttribute("DoorOpenX") or closedX
3407     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
3408     local doorY = door:GetAttribute("DoorY") or door.Position.Y
3409     local isOpen = false
3410     local openedAt = 0
3411     local tween
3412     local function setIndicatorColour(open)
3413         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
3414         for _, gui in ipairs(door:GetChildren()) do
3415             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then
3416                 local indicator = gui:FindFirstChild("DoorIndicator")
3417                 if indicator then indicator.BackgroundColor3 = colour end
3418             end
3419         end
3420     end
3421     local function setOpen(open)
3422         if open == isOpen then return end
3423         isOpen = open
3424         if open then openedAt = os.clock() end
3425         if tween then tween:Cancel() end
3426         local targetX = open and openX or closedX
3427         local targetZ = open and openZ or closedZ
3428         -- 0.9s ease so the slide is smooth and readable (never an instant
3429         -- snap, which is what made the old vertical slide read as flicker).
3430         tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
3431             Enum.EasingDirection.InOut),
3432             {CFrame = CFrame.new(targetX, doorY, targetZ)})
3433         tween:Play()
3434         door.CanCollide = not open
3435         door:SetAttribute("DoorOpen", open) -- watched by the room smoke system
3436         setIndicatorColour(open)
3437     end
3438     task.spawn(function()
3439         while door.Parent do
3440             local nearest = math.huge
3441             for _, player in ipairs(Players:GetPlayers()) do
3442                 local character = player.Character
3443                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
3444                 if hrp then
3445                     -- Floors stack directly above each other at the same X/Z, so a
3446                     -- flat (X/Z-only) distance check would treat a player on a
3447                     -- completely different floor as "standing at the door." Only
3448                     -- count players actually close in height to this door first.
3449                     local verticalGap = math.abs(hrp.Position.Y - doorY)
3450                     if verticalGap < 8 then
3451                         local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) - Vector3.new(closedX,
3452                             doorY, closedZ)).Magnitude
3453                         if flat < nearest then nearest = flat end
3454                     end
3455                 end
3456             end
3457             -- Once opened, a door holds for at least DOOR_MIN_OPEN_SECONDS
3458             -- before it's allowed to close again -- this is what actually
3459             -- fixes the "opens and closes too fast" flicker, independent of
3460             -- the slide direction/speed above.
3461             if isOpen and nearest > DOOR_CLOSE_RANGE and (os.clock() - openedAt) >= DOOR_MIN_OPEN_SECONDS then
3462                 setOpen(false)
3463             elseif not isOpen and nearest < DOOR_OPEN_RANGE then
3464                 setOpen(true)
3465             end
3466             task.wait(0.25)
3467         end
3468     end)
3469 end

```

```

3468
3469 for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindAutoDoor(p) end
3470 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(bindAutoDoor)
3471
3472 -----
3473 -- EMERGENCY KITS
3474 -- One per room. Press E to fully restore health. When a player's health
3475 -- drops below half (and they're not mid-job), a red floor-marker route --
3476 -- the same system used for repair/transfer jobs -- guides them to the
3477 -- closest kit.
3478 -----
3479 local function bindEmergencyKit(kit)
3480     if kit:GetAttribute("KitBound") then return end
3481     kit:SetAttribute("KitBound", true)
3482     local prompt = kit:FindFirstChild("EmergencyKitPrompt")
3483     if not prompt then return end
3484     -- Health stations RECHARGE rather than being consumed. A single-use kit
3485     -- that destroyed itself meant a floor could be permanently stripped of
3486     -- health with no way to get it back.
3487     local RECHARGE_SECONDS = 25
3488     local caption
3489     local faceGui = kit:FindFirstChild("EmergencyKitFace")
3490     if faceGui then
3491         for _, d in ipairs(faceGui:GetDescendants()) do
3492             if d:IsA("TextLabel") then caption = d break end
3493         end
3494     end
3495
3496     local function setCharged(charged)
3497         kit:SetAttribute("Charged", charged)
3498         prompt.Enabled = charged
3499         kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
3500         if caption then caption.Text = charged and "HEALTH" or "CHARGING" end
3501     end
3502
3503     prompt.Triggered:Connect(function(player)
3504         if kit:GetAttribute("Charged") == false then return end
3505         local character = player.Character
3506         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3507         if not humanoid then return end
3508         setCharged(false)
3509         humanoid.Health = humanoid.MaxHealth
3510         notify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s -- there are
3511         ↳ more along every corridor.")
3512         task.delay(RECHARGE_SECONDS, function()
3513             if kit.Parent then setCharged(true) end
3514         end)
3515         setCharged(true)
3516     end)
3517 for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindEmergencyKit(p) end
3518 CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindEmergencyKit)
3519
3520 -----
3521 -- PLAYER HEALTH BARS
3522 -- Matches the bar the worker bots carry, so a player can read their own
3523 -- and everyone else's condition without opening any menu.
3524 -----
3525 local function attachPlayerHealthBar(character)
3526     local humanoid = character:FindFirstChildOfClass("Humanoid") or character:WaitForChild("Humanoid", 8)
3527     local head = character:FindFirstChild("Head") or character:WaitForChild("Head", 8)
3528     if not humanoid or not head or head:FindFirstChild("LabHealthBar") then return end
3529
3530     local gui = Instance.new("BillboardGui")
3531     gui.Name = "LabHealthBar"
3532     gui.Size = UDim2.fromOffset(168, 44)
3533     gui.StudsOffset = Vector3.new(0, 2.6, 0)
3534     gui.MaxDistance = 90
3535     gui.AlwaysOnTop = false
3536     gui.Parent = head
3537
3538     local back = Instance.new("Frame")
3539     back.Name = "HealthBarBack"
3540     back.Size = UDim2.new(0.9, 0, 0.34, 0)
3541     back.Position = UDim2.new(0.05, 0, 0.06, 0)
3542     back.BackgroundColor3 = Color3.fromRGB(22, 26, 32)

```

```

3543     back.BackgroundTransparency = 0.2
3544     back.BorderSizePixel = 0
3545     back.Parent = gui
3546     local fill = Instance.new("Frame")
3547     fill.Name = "HealthBarFill"
3548     fill.Size = UDim2.fromScale(1, 1)
3549     fill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
3550     fill.BorderSizePixel = 0
3551     fill.Parent = back
3552     local label = Instance.new("TextLabel")
3553     label.Name = "HealthText"
3554     label.Size = UDim2.new(1, 0, 0.5, 0)
3555     label.Position = UDim2.new(0, 0, 0.48, 0)
3556     label.BackgroundTransparency = 1
3557     label.TextColor3 = Color3.fromRGB(224, 238, 248)
3558     label.TextStrokeTransparency = 0.4
3559     label.TextScaled = true
3560     label.Font = Enum.Font.GothamBold
3561     label.Parent = gui
3562
3563     local function refresh()
3564         local maximum = math.max(1, humanoid.MaxHealth)
3565         local ratio = math.clamp(humanoid.Health / maximum, 0, 1)
3566         fill.Size = UDim2.fromScale(ratio, 1)
3567         fill.BackgroundColor3 = ratio > 0.55 and Color3.fromRGB(75, 220, 95)
3568             or (ratio > 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
3569         label.Text = string.format("%d%%", math.floor(ratio * 100 + 0.5))
3570     end
3571     humanoid.HealthChanged:Connect(refresh)
3572     humanoid:GetPropertyChangedSignal("MaxHealth"):Connect(refresh)
3573     refresh()
3574 end
3575
3576 local function watchPlayerHealth(player)
3577     player.CharacterAdded:Connect(attachPlayerHealthBar)
3578     if player.Character then task.spawn(attachPlayerHealthBar, player.Character) end
3579 end
3580 Players.PlayerAdded:Connect(watchPlayerHealth)
3581 for _, player in ipairs(Players:GetPlayers()) do watchPlayerHealth(player) end
3582
3583 -----
3584 -- SECRET BASEMENT VENT
3585 -- Disguised as part of the lobby's own exterior wall. Holding E on it
3586 -- drops the player straight into the basement corridor.
3587 -----
3588 local function bindSecretBasementPanel(panel)
3589     if panel:GetAttribute("SecretBound") then return end
3590     panel:SetAttribute("SecretBound", true)
3591     local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")
3592     if not prompt then return end
3593     prompt.Triggered:Connect(function(player)
3594         local character = player.Character
3595         local hrp = character and character:FindFirstChild("HumanoidRootPart")
3596         if not hrp then return end
3597         local dest = Vector3.new(
3598             panel:GetAttribute("SecretDestX") or hrp.Position.X,
3599             panel:GetAttribute("SecretDestY") or hrp.Position.Y,
3600             panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
3601         hrp.CFrame = CFrame.new(dest)
3602         notify(player, "The vent panel swings open -- you drop into the basement.")
3603     end)
3604 end
3605 for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindSecretBasementPanel(p) end
3606 CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindSecretBasementPanel)
3607
3608 local function nearestEmergencyKit(position)
3609     local best, bestDist
3610     for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
3611         if kit.Parent then
3612             local d = (kit.Position - position).Magnitude
3613             if not bestDist or d < bestDist then best, bestDist = kit, d end
3614         end
3615     end
3616     return best
3617 end
3618

```



```

3619 local healthPathActive = {}
3620 task.spawn(function()
3621     while true do
3622         task.wait(3)
3623         for _, player in ipairs(Players:GetPlayers()) do
3624             local character = player.Character
3625             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3626             local hrp = character and character:FindFirstChild("HumanoidRootPart")
3627             if humanoid and hrp and humanoid.Health > 0 then
3628                 local low = humanoid.Health < humanoid.MaxHealth * 0.5
3629                 local busy = player:GetAttribute("AssignedRepairId") or
3630                     ⇐ player:GetAttribute("AssignedTransferSource")
3631                 if low and not busy and not healthPathActive[player] then
3632                     local kit = nearestEmergencyKit(hrp.Position)
3633                     if kit then
3634                         createRepairPath(player, kit)
3635                         healthPathActive[player] = true
3636                         notify(player, "Health low: follow the RED floor lights to the nearest emergency kit.")
3637                     end
3638                 elseif (not low or busy) and healthPathActive[player] then
3639                     healthPathActive[player] = false
3640                     if not busy then clearRepairPath(player) end
3641                 end
3642             end
3643         end
3644     end)
3645
3646 -----
3647 -- EQUIPMENT HAZARDS
3648 -- A malfunctioning (FAILED) unit is genuinely dangerous to linger next to:
3649 -- electrical faults, steam/humidity blowouts, chemical spills and coolant
3650 -- (freon) leaks all drain health while a player stays in range.
3651 -----
3652 local HAZARD_BY_KIND = {
3653     ["Germination Chamber"] = {"STEAM LEAK", 3},
3654     ["Incubator"] = {"STEAM LEAK", 3},
3655     ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
3656     ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
3657     ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
3658     ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
3659     ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
3660     ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
3661     ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},
3662     ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
3663     ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
3664 }
3665 local HAZARD_RANGE = 9
3666 local hazardWarnedAt = {}
3667
3668 -- A unit that's been left FAILED too long can catastrophically blow up,
3669 -- catch fire, or arc -- and unlike an ordinary failure, this one can never
3670 -- be repaired. It keeps draining nearby health (worse than before) and its
3671 -- StatusLED/timer read DESTROYED permanently.
3672 --
3673 -- Two-stage sequence: warnEquipmentCatastrophe() runs a 2-second siren +
3674 -- strobe so anyone nearby has time to clear out, THEN destroyEquipment()
3675 -- fires the actual explosion/fire/arc/leak effects.
3676 local DESTRUCTION_KIND = {
3677     ["Germination Chamber"] = "steam",
3678     ["Incubator"] = "steam",
3679     ["Decontamination Unit"] = "gas",
3680     ["Mutagenesis Chamber"] = "electrical",
3681     ["Hormone Treatment Bench"] = "liquid",
3682     ["Plant Growth Chamber"] = "gas",
3683     ["Growth Hormone Mixer"] = "liquid",
3684     ["GMO Injector"] = "electrical",
3685     ["Nutrient Infuser"] = "gas",
3686     ["UV Growth Scanner"] = "electrical",
3687     ["Pollination Station"] = "electrical",
3688 }
3689
3690 local function scatterDebris(part)
3691     for i = 1, math.random(4, 7) do
3692         local chunk = Instance.new("Part")
3693         chunk.Name = "Debris"

```

```

3694     chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
3695     chunk.Color = part.Color
3696     chunk.Material = Enum.Material.Metal
3697     chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2, math.random(-3,
↵ 3) / 2)
3698     chunk.Anchored = false
3699     chunk.CanCollide = true
3700     chunk.Parent = part.Parent
3701     local av = Instance.new("AngularVelocity")
3702     av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
3703     av.Parent = chunk
3704     local bv = Instance.new("BodyVelocity")
3705     bv.MaxForce = Vector3.new(4000, 4000, 4000)
3706     bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
3707     bv.Parent = chunk
3708     Debris:AddItem(bv, 0.25)
3709     Debris:AddItem(chunk, 8)
3710 end
3711 end
3712
3713 local destroyEquipment
3714
3715 local function warnEquipmentCatastrophe(part, floorLabel, unitLabel)
3716     if part:GetAttribute("AboutToBlow") then return end
3717     part:SetAttribute("AboutToBlow", true)
3718     local beacon = Instance.new("PointLight")
3719     beacon.Color = Color3.fromRGB(255, 40, 30)
3720     beacon.Brightness = 4
3721     beacon.Range = 16
3722     beacon.Parent = part
3723     local siren = Instance.new("Sound")
3724     siren.Name = "SirenWarning"
3725     siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
3726     siren.Looped = true
3727     siren.Volume = 3
3728     siren.PlaybackSpeed = 1.6
3729     siren.Parent = part
3730     siren:Play()
3731     for _, player in ipairs(Players:GetPlayers()) do
3732         notify(player, "ALARM: " .. floorLabel .. " " * " .. unitLabel .. " is about to fail catastrophically --
↵ MOVE AWAY NOW!")
3733     end
3734     task.spawn(function()
3735         local elapsed = 0
3736         while elapsed < 2 and part.Parent do
3737             beacon.Enabled = not beacon.Enabled
3738             task.wait(0.2)
3739             elapsed = elapsed + 0.2
3740         end
3741     end)
3742     task.delay(2, function()
3743         siren:Stop()
3744         siren:Destroy()
3745         beacon:Destroy()
3746         if part.Parent then destroyEquipment(part) end
3747     end)
3748 end
3749
3750 destroyEquipment = function(part)
3751     local floorLabel = equipmentFloorLabel(part)
3752     local unitLabel = equipmentLabel(part)
3753     local causes = {"exploded", "caught fire", "arced and shorted out"}
3754     local cause = causes[math.random(1, #causes)]
3755     part:SetAttribute("FunctionalState", "DESTROYED")
3756     part:SetAttribute("AboutToBlow", nil)
3757     part:SetAttribute("RepairInstruction", floorLabel .. " " * " .. unitLabel .. " • DESTROYED, UNFIXABLE")
3758     equipmentReadout(part, "DESTROYED", floorLabel .. " " * " .. unitLabel .. " " .. cause .. " -- permanently
↵ offline",
3759         Color3.fromRGB(150, 40, 35))
3760     for _, ledPart in ipairs(part:GetDescendants()) do
3761         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
↵ Enum.Material.SmoothPlastic end
3762     end
3763     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
3764
3765     local bang = Instance.new("Sound")

```

```

3766     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
3767     bang.Volume = 4
3768     bang.PlaybackSpeed = 0.8
3769     bang.Parent = part
3770     bang:Play()
3771     Debris:AddItem(bang, 4)
3772
3773     local explosion = Instance.new("Explosion")
3774     explosion.Position = part.Position
3775     explosion.BlastRadius = 8
3776     explosion.BlastPressure = 0
3777     explosion.ExplosionType = Enum.ExplosionType.NoCraters
3778     explosion.DestroyJointRadiusPercent = 0
3779     explosion.Parent = workspace
3780
3781     local kindOfDestruction = DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
3782
3783     local fire = Instance.new("Fire")
3784     fire.Size = 6
3785     fire.Heat = 9
3786     fire.Color = Color3.fromRGB(255, 140, 40)
3787     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
3788     fire.Parent = part
3789     Debris:AddItem(fire, 16)
3790
3791     local sparkles = Instance.new("Sparkles")
3792     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
3793     sparkles.Parent = part
3794     Debris:AddItem(sparkles, 3)
3795
3796     local smoke = Instance.new("Smoke")
3797     smoke.Size = 8
3798     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
3799     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
3800     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
3801     or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
3802     or Color3.fromRGB(50, 50, 55)
3803     smoke.Parent = part
3804     Debris:AddItem(smoke, 20)
3805
3806     if kindOfDestruction == "liquid" then
3807         local puddle = Instance.new("Part")
3808         puddle.Name = "LeakPuddle"
3809         puddle.Shape = Enum.PartType.Cylinder
3810         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
3811         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05, part.Position.Z)
3812             * CFrame.Angles(0, 0, math.rad(90))
3813         puddle.Color = Color3.fromRGB(120, 90, 40)
3814         puddle.Material = Enum.Material.Neon
3815         puddle.Transparency = 0.35
3816         puddle.Anchored = true
3817         puddle.CanCollide = false
3818         puddle.Parent = part.Parent
3819         Debris:AddItem(puddle, 25)
3820     end
3821
3822     scatterDebris(part)
3823
3824     for _, player in ipairs(Players:GetPlayers()) do
3825         notify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause .. ". It
↪ cannot be repaired.")
3826     end
3827 end
3828
3829 task.spawn(function()
3830     while true do
3831         task.wait(1)
3832         local hazards = {}
3833         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
3834             local state = equip.Parent and equip:GetAttribute("FunctionalState")
3835             if state == "FAILED" then
3836                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
3837                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
3838                 -- The longer a unit stays failed, the likelier it blows up for
3839                 -- good. Roughly a 1-in-6 chance per second once it's been down
3840                 -- 45+ seconds -- gives Technicians a real reason to hurry. The

```

```

3841         -- 2-second siren warning (above) still gives an out.
3842         local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
3843         if failedFor > 45 and math.random(1, 6) == 1 then
3844             warnEquipmentCatastrophe(equip, equipmentFloorLabel(equip), equipmentLabel(equip))
3845         end
3846     elseif state == "DESTROYED" then
3847         local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
3848         if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
            ↪ info[2] + 2}) end
3849     end
3850 end
3851 if #hazards > 0 then
3852     for _, player in ipairs(Players:GetPlayers()) do
3853         local character = player.Character
3854         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3855         local hrp = character and character:FindFirstChild("HumanoidRootPart")
3856         if humanoid and hrp and humanoid.Health > 0 then
3857             local worst
3858             for _, hazard in ipairs(hazards) do
3859                 local d = (hazard.part.Position - hrp.Position).Magnitude
3860                 if d < HAZARD_RANGE and (not worst or hazard.dps > worst.dps) then worst = hazard end
3861             end
3862             if worst then
3863                 humanoid:TakeDamage(worst.dps)
3864                 local last = hazardWarnedAt[player] or 0
3865                 if os.clock() - last > 2 then
3866                     hazardWarnedAt[player] = os.clock()
3867                     notify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
                        ↪ worst.label, math.floor(humanoid.Health)))
3868                 end
3869             end
3870         end
3871     end
3872 end
3873 end
3874 end)
3875
3876 for _, p in ipairs(CollectionService:GetTagged("LabToolDispenser")) do bindDispenser(p) end
3877 CollectionService:GetInstanceAddedSignal("LabToolDispenser"):Connect(bindDispenser)
3878
3879 local testBots = CollectionService:GetTagged("LabWorkerBot")
3880 table.sort(testBots, function(a, b) return (a:GetAttribute("BotNumber") or 0) < (b:GetAttribute("BotNumber") or
    ↪ 0) end)
3881 for index, bot in ipairs(testBots) do
3882     -- Stagger WITHIN each floor, not across the whole building. Staggering on
3883     -- the global index at 0.32s a bot meant the top floor did not begin its
3884     -- loop until nearly a minute after the server started, so the upper
3885     -- levels looked completely frozen to anyone who walked up there early.
3886     -- Keyed off the per-floor slot, every floor is fully awake in ~7s.
3887     -- Only bots someone is standing next to will actually compute a path now,
3888     -- so there is no thundering herd to spread out and the stagger can be
3889     -- short. Every floor is ready ~1.6s in instead of ~58s.
3890     local slot = bot:GetAttribute("LevelSlot") or index
3891     task.delay(slot * 0.08, function()
3892         if bot.Parent then runBotScenario(bot) end
3893     end)
3894 end
3895
3896 -- Bodies stay put for the whole shift. LabRound stamps RoundNumber on
3897 -- Workspace when the shift rolls over; that is the cue to clear them away and
3898 -- put every worker back on its feet for a fresh hunt.
3899 local function resetHunt()
3900     local bodies = workspace:FindFirstChild("LabBotBodies")
3901     if bodies then bodies:ClearAllChildren() end
3902
3903     local complex = workspace:FindFirstChild("Laboratory_Complex")
3904     if complex then complex:SetAttribute("BotsKilled", 0) end
3905
3906     local restaged = 0
3907     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
3908         if bot:GetAttribute("Dead") then
3909             local home = bot:GetAttribute("HomeCFrame")
3910             if typeof(home) == "CFrame" then bot:PivotTo(home) end
3911             bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
3912             bot:SetAttribute("KnockedOut", false)
3913             bot:SetAttribute("Dead", false)

```

```

3914         bot:SetAttribute("TaskState", "Waiting for plant task")
3915         refreshBotHealth(bot)
3916         -- its old scenario loop exited on death, so start a fresh one
3917         task.spawn(runBotScenario, bot)
3918         restaged = restaged + 1
3919     end
3920 end
3921 if restaged > 0 then
3922     print("Shift reset: cleared bodies and restaged " .. restaged .. " workers.")
3923 end
3924 end
3925 workspace:GetAttributeChangedSignal("RoundNumber"):Connect(resetHunt)
3926
3927 -----
3928 -- ROOM SMOKE
3929 -- Small rooms slowly fill with smoke. Opening a door vents it. Capped so
3930 -- there are never more than a handful of smoke volumes alive at once.
3931 -----
3932 local MAX_SMOKY_ROOMS = 4
3933 local smokeByRoom = {}
3934
3935 local function clearRoomSmoke(key)
3936     local folder = smokeByRoom[key]
3937     if not folder then return end
3938     smokeByRoom[key] = nil
3939     for _, d in ipairs(folder:GetDescendants()) do
3940         if d:IsA("Smoke") then d.Enabled = false end
3941     end
3942     Debris:AddItem(folder, 5) -- let what is already in the air disperse
3943 end
3944
3945 local function fillRoomWithSmoke(levelName, rect)
3946     local key = levelName .. "/" .. rect.name
3947     if smokeByRoom[key] then return end
3948     local folder = Instance.new("Folder")
3949     folder.Name = "RoomSmoke"
3950     folder:SetAttribute("RoomKey", key)
3951     folder.Parent = workspace
3952     for i = 1, 5 do
3953         local emitter = Instance.new("Part")
3954         emitter.Name = "SmokeSource"
3955         emitter.Size = Vector3.new(1, 1, 1)
3956         emitter.Anchored = true
3957         emitter.CanCollide = false
3958         emitter.CanQuery = false
3959         emitter.CanTouch = false
3960         emitter.Transparency = 1
3961         emitter.CFrame = CFrame.new(
3962             rect.x1 + (rect.x2 - rect.x1) * (i - 0.5) / 5,
3963             rect.y + 3,
3964             rect.z1 + (rect.z2 - rect.z1) * ((i * 2) % 5 + 0.5) / 5)
3965         emitter.Parent = folder
3966         local smoke = Instance.new("Smoke")
3967         smoke.Size = 11
3968         smoke.Opacity = 0.32
3969         smoke.RiseVelocity = 2
3970         smoke.Color = Color3.fromRGB(150, 150, 152)
3971         smoke.Parent = emitter
3972     end
3973     smokeByRoom[key] = folder
3974 end
3975
3976 local function ventFromDoor(door)
3977     local pos = door.Position
3978     for key in pairs(smokeByRoom) do
3979         local levelName, roomName = key:match("^(.)/(.+)$")
3980         local rect = roomRect(levelName, roomName)
3981         -- a door on the room's own boundary counts, hence the small margin
3982         if rect and math.abs(pos.Y - rect.y) < FLOOR_BAND
3983             and pos.X >= rect.x1 - 8 and pos.X <= rect.x2 + 8
3984             and pos.Z >= rect.z1 - 8 and pos.Z <= rect.z2 + 8 then
3985             clearRoomSmoke(key)
3986         end
3987     end
3988 end
3989

```

```

3990 local function watchDoor(door)
3991     door:GetAttributeChangedSignal("DoorOpen"):Connect(function()
3992         if door:GetAttribute("DoorOpen") then ventFromDoor(door) end
3993     end)
3994 end
3995 for _, door in ipairs(CollectionService:GetTagged("AutoDoor")) do watchDoor(door) end
3996 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(watchDoor)
3997
3998 task.spawn(function()
3999     while true do
4000         task.wait(45)
4001         local live = 0
4002         for _ in pairs(smokeByRoom) do live = live + 1 end
4003         if live < MAX_SMOKY_ROOMS then
4004             local levels = {}
4005             for levelName in pairs(roomsByLevel) do table.insert(levels, levelName) end
4006             if #levels > 0 then
4007                 local levelName = levels[math.random(1, #levels)]
4008                 -- only the small east bays; the long west room is not a "small room"
4009                 local small = {}
4010                 for _, r in ipairs(roomsByLevel[levelName]) do
4011                     if (r.x2 - r.x1) < 90 and (r.z2 - r.z1) < 90 then table.insert(small, r) end
4012                 end
4013                 if #small > 0 then
4014                     fillRoomWithSmoke(levelName, small[math.random(1, #small)])
4015                 end
4016             end
4017         end
4018     end
4019 end)
4020 task.spawn(function()
4021     while task.wait(2) do updateBotTestBoard() end
4022 end)
4023 task.spawn(function()
4024     while task.wait(2) do refreshRoomMonitors() end
4025 end)
4026 task.spawn(function()
4027     task.wait(3)
4028     while true do
4029         adaptiveFailureTick()
4030         genericFailureTick()
4031         task.wait(math.clamp(15 - #Players:GetPlayers() * 1.25, 6, 14))
4032     end
4033 end)
4034
4035 print("Equipment armed: " .. #CollectionService:GetTagged("LabEquipment")
4036     .. " items; recovery tool pads: " .. #CollectionService:GetTagged("LabToolDispenser") .. ".")
4037
4038 -----
4039 -- LAB STAFF NPCs
4040 -----
4041 local function bindNPC(part)
4042     if part:GetAttribute("TalkBound") then return end
4043     part:SetAttribute("TalkBound", true)
4044     local tipsStr = part:GetAttribute("Tips") or ""
4045     local tips = {}
4046     for t in tipsStr:gmatch("[^|]+") do table.insert(tips, t) end
4047     local label = part:GetAttribute("NPCName") or part:GetAttribute("Role") or "Lab Staff"
4048
4049     local prompt = Instance.new("ProximityPrompt")
4050     prompt.ObjectText = label
4051     prompt.ActionText = "Talk"
4052     prompt.HoldDuration = 0.3
4053     prompt.MaxActivationDistance = 10
4054     prompt.RequiresLineOfSight = false
4055     prompt.Parent = part
4056
4057     prompt.Triggered:Connect(function(player)
4058         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
4059         if #tips == 0 then return end
4060         notify(player, label .. ": " .. tips[math.random(1, #tips)])
4061     end)
4062 end
4063
4064 for _, p in ipairs(CollectionService:GetTagged("LabNPC")) do bindNPC(p) end
4065 CollectionService:GetInstanceAddedSignal("LabNPC"):Connect(bindNPC)

```

4066

4067 `print("Lab staff NPCs armed: " .. #CollectionService:GetTagged("LabNPC") .. ".")`

Chapter 51

S19 - LabRoles

ServerStorage.BeforeBasementAndQuiz_20260912.LabRoles

Authority group: archive. 18757 UTF-8 bytes; 422 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LabRoles : role kiosks, teams, per-player leaderstats, a persistent
2 -- HUD, and a visible appearance change (tinted uniform + hard hat) so
3 -- everyone can see at a glance who is playing which role.
4
5 local Players          = game:GetService("Players")
6 local Teams           = game:GetService("Teams")
7 local CollectionService = game:GetService("CollectionService")
8
9 local ROLE_PAINT = {
10     Technician = Color3.fromRGB(60, 120, 200),
11     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),
12 }
13
14 local ROLE_HINT = {
15     Unassigned = "Choose BLUE Technician or GREEN Laboratory Technician.",
16     Technician = "Follow RED floor lights. HOLD R at the failed unit. Score grows with failure time and
17     ↪ assigned distance.",
18     ["Laboratory Technician"] = "E takes a plant sample. Carry it to any GREEN RUNNING cabinet marked FREE SLOT
19     ↪ and press E again to revive/drop it.",
20 }
21
22 local function roleEquipmentId(part)
23     local code = part:GetAttribute("GrowthCode")
24     if code and code ~= "" then return "CHAMBER:" .. code end
25     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
26     ↪ " | ")
27 end
28
29 local function roleFloor(part)
30     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
31 end
32
33 local function roleTransferSeconds(source, destination)
34     local distance = destination and (destination.Position - source.Position).Magnitude or 60
35     local playerCount = math.max(1, #Players:GetPlayers())
36     local complex = workspace:FindFirstChild("Laboratory_Complex")
37     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
38     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
39     + math.clamp(backlog / 20, 0, 25)), 60, 180)
40 end
41
42 local function roleEquipmentLabel(part)
43     local code = part:GetAttribute("GrowthCode")
44     if code and code ~= "" then return "CHAMBER:" .. code end
45     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
46 end
```



```

45 local function roleDestinations(source)
46     local choices = {}
47     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
48         local count = chamber:GetAttribute("SampleCount") or 0
49         local reservations = chamber:GetAttribute("IncomingReservations") or 0
50         if chamber ~= source and chamber:GetAttribute("FunctionalState") ~= "FAILED"
51             and count + reservations < (chamber:GetAttribute("Capacity") or 10) then table.insert(choices,
52             ⇨ chamber) end
53     end
54     table.sort(choices, function(a, b)
55         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
56         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
57         if sameA ~= sameB then return sameA < sameB end
58         return (a:GetAttribute("SampleCount") or 0) < (b:GetAttribute("SampleCount") or 0)
59     end)
60     return choices
61 end
62 local function clearRoleAssignment(player)
63     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
64     ⇨ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
65         player:SetAttribute(name, nil)
66     end
67 end
68 local function assignRoleObjective(player, roleName)
69     clearRoleAssignment(player)
70     if roleName == "Technician" then
71         local failed = {}
72         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
73             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment) end
74         end
75         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
76         table.sort(failed, function(a, b)
77             if root then return (a.Position - root.Position).Magnitude < (b.Position - root.Position).Magnitude
78             ⇨ end
79             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
80         end)
81         local target = failed[1]
82         if not target then return "All equipment is working. Repair score grows with failure time and assigned
83         ⇨ distance." end
84         player:SetAttribute("AssignedRepairId", roleEquipmentId(target))
85         local distance = root and (target.Position - root.Position).Magnitude or 0
86         local waited = math.max(0, os.time() - (target:GetAttribute("FailureSince") or os.time()))
87         local projected = math.clamp(20 + math.floor(waited / 8) + math.floor(distance / 10), 8, 120)
88         player:SetAttribute("AssignedRepairDistance", distance)
89         player:SetAttribute("AssignedRepairProjectedPoints", projected)
90         return string.format("REPAIR JOB: %s * %s. Follow RED floor lights; HOLD R. About +%d points.",
91         ⇨ roleFloor(target), roleEquipmentLabel(target), projected)
92     end
93 end
94 local jobs = {}
95 for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
96     if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0) > 0
97     and not source:GetAttribute("ReservedByBot") then
98         local destination = roleDestinations(source)[1]
99         if destination then table.insert(jobs, {source = source, destination = destination}) end
100     end
101 end
102 table.sort(jobs, function(a, b)
103     local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
104     local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
105     return an < bn
106 end)
107 local job = jobs[1]
108 if not job then return "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
109 ⇨ +30." end
110 local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
111 local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
112 player:SetAttribute("AssignedTransferSource", sourceCode)
113 player:SetAttribute("AssignedTransferDestination", destinationCode)
114 local allowedSeconds = roleTransferSeconds(job.source, job.destination)
115 player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
116 return "ASSIGNED +30: " .. roleFloor(job.source) .. " * " .. sourceCode .. " ⇨ " .. destinationCode
117 .. " * " .. roleFloor(job.destination) .. ". PRESS E at source, deliver in " .. allowedSeconds .. "
118 ⇨ seconds."

```

```

115 end
116
117 -----
118 -- per-player HUD (role, contributions, current objective)
119 -----
120     local function buildHud(player)
121         local playerGui = player:WaitForChild("PlayerGui")
122         local duplicate = playerGui:FindFirstChild("LiveLabHUD")
123         if duplicate then duplicate:Destroy() end
124         local gui = Instance.new("ScreenGui")
125         gui.Name = "RoleHUD"
126         gui.ResetOnSpawn = false
127         gui.DisplayOrder = 20
128         gui.Parent = playerGui
129
130         local frame = Instance.new("Frame")
131         frame.Size = UDim2.fromOffset(238, 90)
132         frame.Position = UDim2.fromOffset(8, 50)
133         frame.BackgroundColor3 = Color3.fromRGB(18, 20, 26)
134         frame.BackgroundTransparency = 0.14
135         frame.Parent = gui
136         local corner = Instance.new("UICorner"); corner.CornerRadius = UDim.new(0, 8); corner.Parent = frame
137
138         local roleLabel = Instance.new("TextLabel")
139         roleLabel.Name = "RoleLabel"
140         roleLabel.Size = UDim2.new(1, -14, 0, 16)
141         roleLabel.Position = UDim2.new(0, 8, 0, 4)
142         roleLabel.BackgroundTransparency = 1
143         roleLabel.TextXAlignment = Enum.TextXAlignment.Left
144         roleLabel.Font = Enum.Font.GothamBold
145         roleLabel.TextSize = 13
146         roleLabel.TextColor3 = Color3.fromRGB(240, 242, 246)
147         roleLabel.Text = "Role: Unassigned"
148         roleLabel.Parent = frame
149
150         local contribLabel = Instance.new("TextLabel")
151         contribLabel.Name = "ContribLabel"
152         contribLabel.Size = UDim2.new(1, -14, 0, 13)
153         contribLabel.Position = UDim2.new(0, 8, 0, 20)
154         contribLabel.BackgroundTransparency = 1
155         contribLabel.TextXAlignment = Enum.TextXAlignment.Left
156         contribLabel.Font = Enum.Font.Gotham
157         contribLabel.TextSize = 10
158         contribLabel.TextTruncate = Enum.TextTruncate.AtEnd
159         contribLabel.TextYAlignment = Enum.TextYAlignment.Top
160         contribLabel.TextColor3 = Color3.fromRGB(200, 205, 212)
161         contribLabel.Text = "POINTS 0"
162         contribLabel.Parent = frame
163
164         local carryLabel = Instance.new("TextLabel")
165         carryLabel.Name = "CarryLabel"
166         carryLabel.Size = UDim2.new(1, -14, 0, 13)
167         carryLabel.Position = UDim2.new(0, 8, 0, 33)
168         carryLabel.BackgroundTransparency = 1
169         carryLabel.TextXAlignment = Enum.TextXAlignment.Left
170         carryLabel.Font = Enum.Font.GothamBold
171         carryLabel.TextSize = 10
172         carryLabel.TextColor3 = Color3.fromRGB(115, 215, 245)
173         carryLabel.Text = "Carrying: nothing"
174         carryLabel.Parent = frame
175
176         local targetsLabel = Instance.new("TextLabel")
177         targetsLabel.Name = "TargetsLabel"
178         targetsLabel.Size = UDim2.new(1, -14, 0, 13)
179         targetsLabel.Position = UDim2.new(0, 8, 0, 46)
180         targetsLabel.BackgroundTransparency = 1
181         targetsLabel.TextXAlignment = Enum.TextXAlignment.Left
182         targetsLabel.Font = Enum.Font.GothamBold
183         targetsLabel.TextSize = 10
184         targetsLabel.TextColor3 = Color3.fromRGB(255, 205, 120)
185         targetsLabel.Text = "TARGETS 0 / 0"
186         targetsLabel.Parent = frame
187
188         local hintLabel = Instance.new("TextLabel")
189         hintLabel.Name = "HintLabel"
190         hintLabel.Size = UDim2.new(1, -14, 0, 25)

```

```

191     hintLabel.Position = UDim2.new(0, 8, 0, 61)
192     hintLabel.BackgroundTransparency = 1
193     hintLabel.TextXAlignment = Enum.TextXAlignment.Left
194     hintLabel.TextYAlignment = Enum.TextYAlignment.Top
195     hintLabel.TextWrapped = true
196     hintLabel.Font = Enum.Font.Gotham
197     hintLabel.TextSize = 9
198     hintLabel.TextTruncate = Enum.TextTruncate.AtEnd
199     hintLabel.TextColor3 = Color3.fromRGB(255, 220, 150)
200     hintLabel.Text = ROLE_HINT.Unassigned
201     hintLabel.Parent = frame
202
203     return gui
204 end
205
206 local function refreshHud(player, roleName)
207     local gui = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
208     if not gui then return end
209     local frame = gui:FindFirstChildOfClass("Frame")
210     if not frame then return end
211     frame.RoleLabel.Text = "Role: " .. roleName
212     local carried = player:GetAttribute("CarriedSample")
213     local objective = player:GetAttribute("LabObjective")
214     frame.CarryLabel.Text = (carried and carried ~= "") and ("Carrying: " .. carried) or "Carrying: nothing"
215     local hint = (objective and objective ~= "") and objective or (ROLE_HINT[roleName] or ROLE_HINT.Unassigned)
216     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
217     if roleName == "Laboratory Technician" and deadline > 0 then
218         hint = hint .. "\nTIME LEFT FOR +30: " .. math.max(0, deadline - os.time()) .. " seconds"
219     end
220     frame.HintLabel.Text = hint
221     local stats = player:FindFirstChild("leaderstats")
222     local points = stats and stats:FindFirstChild("Points")
223     local lastAmount = player:GetAttribute("LastPointAmount") or 0
224     local lastReason = player:GetAttribute("LastPointAward") or ""
225     frame.ContribLabel.Text = string.format("POINTS %d%s",
226         points and points.Value or 0, lastAmount > 0 and (" • LAST +" .. lastAmount .. " " .. lastReason)
227         ↪ or "")
228
229     end
230
231     -----
232     -- Visible appearance change: a welded lab uniform sits above avatar
233     -- clothing, so layered clothes cannot hide the selected role.
234     -----
235 local function wearablePart(uniform, bodyPart, name, size, offset, colour, material)
236     local p = Instance.new("Part")
237     p.Name = name
238     p.Size = size
239     p.CFrame = bodyPart.CFrame * offset
240     p.Color = colour
241     p.Material = material or Enum.Material.Fabric
242     p.Anchored = false
243     p.CanCollide = false
244     p.CanTouch = false
245     p.Massless = true
246     p.CastShadow = false
247     p.TopSurface = Enum.SurfaceType.Smooth
248     p.BottomSurface = Enum.SurfaceType.Smooth
249     p.Parent = uniform
250     local weld = Instance.new("WeldConstraint")
251     weld.Part0 = bodyPart
252     weld.Part1 = p
253     weld.Parent = p
254     return p
255 end
256
257 local function applyAppearance(character, roleName)
258     local paint = ROLE_PAINT[roleName]
259     if not paint then return end
260     local oldUniform = character:FindFirstChild("RoleUniform")
261     if oldUniform then oldUniform:Destroy() end
262     local oldHelmet = character:FindFirstChild("RoleHelmet")
263     if oldHelmet then oldHelmet:Destroy() end
264
265     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
266     local head = character:FindFirstChild("Head")
267     if not torso or not head then return end

```

```

266
267     local uniform = Instance.new("Model")
268     uniform.Name = "RoleUniform"
269     uniform:SetAttribute("Role", roleName)
270     uniform.Parent = character
271
272     wearablePart(uniform, torso, "LabCoat", Vector3.new(torso.Size.X * 1.08, torso.Size.Y * 0.94, torso.Size.Z
↵ * 1.1),
273         CFrame.new(), Color3.fromRGB(238, 241, 244), Enum.Material.Fabric)
274     wearablePart(uniform, torso, "RolePanel", Vector3.new(torso.Size.X * 0.82, torso.Size.Y * 0.55, 0.14),
275         CFrame.new(0, 0.08, -(torso.Size.Z / 2 + 0.12)), paint, Enum.Material.SmoothPlastic)
276     wearablePart(uniform, torso, "RoleBelt", Vector3.new(torso.Size.X * 1.12, 0.18, torso.Size.Z * 1.14),
277         CFrame.new(0, -torso.Size.Y * 0.34, 0), paint, Enum.Material.SmoothPlastic)
278     wearablePart(uniform, torso, "RoleBadge", Vector3.new(0.36, 0.46, 0.16),
279         CFrame.new(-torso.Size.X * 0.28, torso.Size.Y * 0.22, -(torso.Size.Z / 2 + 0.18)),
280         Color3.fromRGB(255, 235, 120), Enum.Material.Neon)
281
282     local helmet = wearablePart(uniform, head, "RoleHelmet", Vector3.new(1.82, 0.72, 1.82),
283         CFrame.new(0, 0.72, 0), paint, Enum.Material.SmoothPlastic)
284     helmet.Shape = Enum.PartType.Ball
285     wearablePart(uniform, head, "HelmetBand", Vector3.new(1.9, 0.16, 1.9),
286         CFrame.new(0, 0.52, 0), Color3.fromRGB(245, 245, 245), Enum.Material.SmoothPlastic)
287
288     for _, limbName in ipairs({"LeftUpperArm", "RightUpperArm", "Left Arm", "Right Arm"}) do
289         local limb = character:FindFirstChild(limbName)
290         if limb then
291             wearablePart(uniform, limb, limbName:gsub(" ", "") .. "RoleBand",
292                 Vector3.new(limb.Size.X * 1.12, math.min(0.24, limb.Size.Y * 0.18), limb.Size.Z * 1.12),
293                 CFrame.new(0, -limb.Size.Y * 0.26, 0), paint, Enum.Material.SmoothPlastic)
294         end
295     end
296 end
297
298 local setupDone = {}
299 local function setupPlayer(player)
300     if setupDone[player] then return end
301     setupDone[player] = true
302     if player:GetAttribute("CarriedSample") == nil then player:SetAttribute("CarriedSample", "") end
303     if player:GetAttribute("LabObjective") == nil then player:SetAttribute("LabObjective", "") end
304     local stats = player:FindFirstChild("leaderstats") or Instance.new("Folder")
305     stats.Name = "leaderstats"
306     stats.Parent = player
307
308     local role = stats:FindFirstChild("Role") or Instance.new("StringValue")
309     role.Name = "Role"
310     if role.Value == "" then role.Value = "Unassigned" end
311     role.Parent = stats
312
313     local contrib = stats:FindFirstChild("Contributions") or Instance.new("IntValue")
314     contrib.Name = "Contributions"
315     contrib.Parent = stats
316     local points = stats:FindFirstChild("Points") or Instance.new("IntValue")
317     points.Name = "Points"
318     points.Parent = stats
319
320     local oldHud = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
321     if oldHud then oldHud:Destroy() end
322     buildHud(player)
323     role:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
324     contrib:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
325     points:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
326     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshHud(player, role.Value) end)
327     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshHud(player, role.Value) end)
328     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshHud(player, role.Value) end)
329     refreshHud(player, role.Value)
330
331     -- Health must never regenerate passively: it only comes back through an
332     -- Emergency Kit. Roblox's default character ships a "Health" script that
333     -- does exactly the auto-regen we don't want, so remove it every spawn.
334     local function stripAutoHealthRegen(character)
335         local healthScript = character:FindFirstChild("Health")
336         if healthScript then healthScript:Destroy() end
337     end
338     player.CharacterAdded:Connect(function(character)
339         if role.Value ~= "Unassigned" then
340             applyAppearance(character, role.Value)

```

```

341     end
342     stripAutoHealthRegen(character)
343 end)
344 if player.Character then
345     if role.Value ~= "Unassigned" then applyAppearance(player.Character, role.Value) end
346     stripAutoHealthRegen(player.Character)
347 end
348 task.spawn(function()
349     while player.Parent do
350         if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshHud(player, role.Value)
351             ↪ end
352         task.wait(1)
353     end
354 end)
355 end
356 Players.PlayerAdded:Connect(setupPlayer)
357 for _, player in ipairs(Players:GetPlayers()) do task.spawn(setupPlayer, player) end
358
359 local function bindKiosk(part)
360     local roleName = part:GetAttribute("Role")
361     if not roleName then return end
362     if part:GetAttribute("RoleKioskBound") then return end
363     part:SetAttribute("RoleKioskBound", true)
364
365     local button = Instance.new("Part")
366     button.Name = "RoleSelectButton"
367     button.Shape = Enum.PartType.Cylinder
368     button.Size = Vector3.new(0.28, 1.05, 1.05)
369     button.CFrame = part.CFrame * CFrame.new(0, -part.Size.Y * 0.32, -(part.Size.Z / 2 + 0.32))
370         * CFrame.Angles(0, math.rad(90), 0)
371     button.Color = ROLE_PAINT[roleName]
372     button.Material = Enum.Material.Neon
373     button.Anchored = true
374     button.CanCollide = false
375     button.Parent = part
376
377     local prompt = Instance.new("ProximityPrompt")
378     prompt.ObjectText = roleName .. " Kiosk"
379     prompt.ActionText = "Select " .. roleName
380     prompt.KeyboardKeyCode = Enum.KeyCode.E
381     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
382     prompt.HoldDuration = 0
383     prompt.MaxActivationDistance = 30
384     prompt.RequiresLineOfSight = false
385     prompt.Parent = button
386
387     local click = Instance.new("ClickDetector")
388     click.MaxActivationDistance = 30
389     click.Parent = button
390     local debounce = {}
391     local function selectRole(player)
392         if debounce[player] and os.clock() < debounce[player] then return end
393         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
394         if not root or (root.Position - part.Position).Magnitude > 36 then return end
395         debounce[player] = os.clock() + 1
396         setupPlayer(player)
397         local team = Teams:FindFirstChild(roleName)
398         if not team then return end
399         player.Team = team
400         player.Neutral = false
401         player:SetAttribute("LabRole", roleName)
402         local stats = player:FindFirstChild("leaderstats")
403         if stats and stats:FindFirstChild("Role") then
404             stats.Role.Value = roleName
405         end
406         player:SetAttribute("LabObjective", assignRoleObjective(player, roleName))
407         if player.Character then applyAppearance(player.Character, roleName) end
408         local oldColour = button.Color
409         button.Color = Color3.fromRGB(255, 255, 255)
410         task.delay(0.35, function() if button.Parent then button.Color = oldColour end end)
411         refreshHud(player, roleName)
412     end
413
414     prompt.Triggered:Connect(selectRole)
415     click.MouseClick:Connect(selectRole)

```

```
416 end
417
418 for _, p in ipairs(CollectionService:GetTagged("RoleKiosk")) do bindKiosk(p) end
419 CollectionService:GetInstanceAddedSignal("RoleKiosk"):Connect(bindKiosk)
420
421 print("Role kiosks armed: " .. #CollectionService:GetTagged("RoleKiosk") .. ".")
```

Chapter 52

S20 - LabRound

ServerStorage.BeforeBasementAndQuiz_20260912.LabRound

Authority group: archive. 2954 UTF-8 bytes; 85 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LabRound: round timer, scoring cycle and end-of-round resolution.
2 -- Round length is COMPUTED from the built environment, not hard-coded.
3 local Players = game:GetService("Players")
4 local CollectionService = game:GetService("CollectionService")
5
6 local INTERMISSION = 20
7
8 local function computeRoundSeconds()
9     local eq = CollectionService:GetTagged("LabEquipment")
10     local pts = {}
11     for _, e in ipairs(eq) do
12         local ok, p = pcall(function() return e:GetPivot().Position end)
13         if ok then table.insert(pts, p) end
14     end
15     if #pts < 2 then return 480, 0, 0 end
16     local n, sum, samples = #pts, 0, 2000
17     for _ = 1, samples do
18         sum = sum + (pts[math.random(n)] - pts[math.random(n)]).Magnitude
19     end
20     local mean = sum / samples
21     local WALK_SPEED = 16 -- studs/sec, Roblox default
22     local ROUTE_FACTOR = 1.5 -- corridors/doors vs straight line
23     local VERTICAL = 2.5 -- averaged elevator/stair cost
24     local ACTION = 4.0 -- HOLD R repair / E transfer
25     local TARGET_TASKS = 25 -- tasks a competent player should fit in a round
26     local perTask = (mean / WALK_SPEED) * ROUTE_FACTOR + VERTICAL + ACTION
27     local secs = math.floor((perTask * TARGET_TASKS) / 30 + 0.5) * 30
28     return math.clamp(secs, 300, 900), mean, perTask
29 end
30
31 local ROUND_SECONDS, meanDist, perTask = computeRoundSeconds()
32 print(string.format(
33     "[LabRound] units=%d meanDist=%.0f studs perTask=%.1fs -> round=%ds (%.1f min)",
34     #CollectionService:GetTagged("LabEquipment"), meanDist, perTask, ROUND_SECONDS, ROUND_SECONDS / 60))
35
36 workspace:SetAttribute("RoundLength", ROUND_SECONDS)
37
38 local function pointsOf(player)
39     local stats = player:FindFirstChild("leaderstats")
40     local p = stats and stats:FindFirstChild("Points")
41     return p and p.Value or 0, p
42 end
43
44 local function resetScores()
45     for _, player in ipairs(Players:GetPlayers()) do
46         local _, p = pointsOf(player)
47         if p then p.Value = 0 end
48         player:SetAttribute("RoundStreakBest", 0)
```

```
49     end
50 end
51
52 local function resolveRound()
53     local best, bestScore = nil, -1
54     for _, player in ipairs(Players:GetPlayers()) do
55         local v = pointsOf(player)
56         if v > bestScore then best, bestScore = player, v end
57         local pb = player:GetAttribute("PersonalBest") or 0
58         if v > pb then player:SetAttribute("PersonalBest", v) end
59         player:SetAttribute("LastRoundScore", v)
60     end
61     workspace:SetAttribute("LastWinnerName", best and best.Name or "")
62     workspace:SetAttribute("LastWinnerScore", bestScore > 0 and bestScore or 0)
63 end
64
65 task.spawn(function()
66     local round = 0
67     while true do
68         round += 1
69         resetScores()
70         workspace:SetAttribute("RoundNumber", round)
71         workspace:SetAttribute("RoundPhase", "PLAY")
72         workspace:SetAttribute("RoundEndsAt", os.time() + ROUND_SECONDS)
73         task.wait(ROUND_SECONDS)
74
75         resolveRound()
76         workspace:SetAttribute("RoundPhase", "BREAK")
77         workspace:SetAttribute("RoundEndsAt", os.time() + INTERMISSION)
78         task.wait(INTERMISSION)
79     end
80 end)
81
82 Players.PlayerAdded:Connect(function(player)
83     player:SetAttribute("PersonalBest", 0)
84 end)
```


Chapter 53

S21 - LabTechnicians

ServerStorage.BeforeBasementAndQuiz_20260912.LabTechnicians

Authority group: archive. 7271 UTF-8 bytes; 201 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LabTechnicians: 15 human technicians who panic, tire, call for help and die.
2 -- They CANNOT repair anything: they can only locate faults and beg the player.
3 local CollectionService = game:GetService("CollectionService")
4 local Players = game:GetService("Players")
5
6 -----
7 -- AUDIO: paste asset IDs you OWN or that are royalty-free.
8 -- Left empty deliberately: every free "scream"/"help me" asset on the
9 -- Creator Store is ripped from a commercial game, which risks having
10 -- your experience moderated. Bubbles work without any audio.
11 -----
12 local AUDIO = {
13     scream_f = "", -- e.g. "rbxassetid://123456789"
14     scream_m = "",
15     help     = "",
16     follow   = "",
17 }
18
19 local PANIC_RADIUS   = 26
20 local HELP_COOLDOWN   = 9
21 local DAMAGE_PER_SEC = 2.5
22 local DAMAGE_RADIUS  = 11
23 local HEALTH_FLOOR   = 12
24 local RESPAWN_DELAY  = 25
25
26 local HELP_LINES = {
27     "Help! %s is failing!", "Please help me, I can't repair this!",
28     "Follow me - %s is critical!", "Come with me, hurry!",
29     "I'm only a technician, I need you!", "Someone repair %s, please!",
30 }
31 local TIRED_LINES = {
32     "I need a break...", "I can't keep running like this.",
33     "My legs are gone. Give me a second.", "Too many alarms today...",
34 }
35
36 local function bubbleOf(rig)
37     local head = rig:FindFirstChild("Head")
38     if not head then return nil end
39     local bb = head:FindFirstChild("SpeechBubble")
40     if bb then return bb:FindFirstChildOfClass("TextLabel"), bb end
41     bb = Instance.new("BillboardGui")
42     bb.Name = "SpeechBubble"
43     bb.Size = UDim2.new(0, 220, 0, 42)
44     bb.StudsOffset = Vector3.new(0, 4.2, 0)
45     bb.AlwaysOnTop = true
46     bb.MaxDistance = 90
47     bb.Enabled = false
48     bb.Parent = head
```

```

49     local f = Instance.new("Frame")
50     f.Size = UDim2.new(1, 0, 1, 0); f.BackgroundColor3 = Color3.fromRGB(18, 22, 24)
51     f.BackgroundTransparency = 0.15; f.BorderSizePixel = 0; f.Parent = bb
52     local c = Instance.new("UICorner"); c.CornerRadius = UDim.new(0, 8); c.Parent = f
53     local st = Instance.new("UIStroke"); st.Color = Color3.fromRGB(255, 120, 100); st.Thickness = 1.5;
54     ↪ st.Parent = f
55     local t = Instance.new("TextLabel")
56     t.Size = UDim2.new(1, -12, 1, -6); t.Position = UDim2.new(0, 6, 0, 3)
57     t.BackgroundTransparency = 1; t.Font = Enum.Font.GothamMedium; t.TextSize = 13
58     t.TextColor3 = Color3.fromRGB(240, 245, 245); t.TextWrapped = true; t.Parent = f
59     return t, bb
60 end
61 local function say(rig, text, colour)
62     local label, bb = bubbleOf(rig)
63     if not label then return end
64     label.Text = text
65     local stroke = bb:FindFirstChildOfClass("Frame"):FindFirstChildOfClass("UIStroke")
66     if stroke and colour then stroke.Color = colour end
67     bb.Enabled = true
68     task.delay(4.5, function() if bb and bb.Parent then bb.Enabled = false end end)
69 end
70
71 local function playSound(rig, key)
72     local id = AUDIO[key]
73     if not id or id == "" then return end
74     local head = rig:FindFirstChild("Head")
75     if not head then return end
76     local snd = Instance.new("Sound")
77     snd.SoundId = id; snd.Volume = 1; snd.RollOffMaxDistance = 90
78     snd.Parent = head; snd:Play()
79     snd.Ended:Connect(function() snd:Destroy() end)
80     task.delay(8, function() if snd and snd.Parent then snd:Destroy() end end)
81 end
82
83 local function labelOf(equip)
84     return equip:GetAttribute("GrowthCode") or equip:GetAttribute("Kind") or equip.Name
85 end
86
87 local function nearestFault(pos)
88     local best, bestD = nil, PANIC_RADIUS
89     for _, e in ipairs(CollectionService:GetTagged("LabEquipment")) do
90         if e:GetAttribute("FunctionalState") == "FAILED" then
91             local ok, p = pcall(function() return e:GetPivot().Position end)
92             if ok then
93                 local d = (p - pos).Magnitude
94                 if d < bestD then best, bestD = e, d end
95             end
96         end
97     end
98     return best, bestD
99 end
100
101 local function setStatus(rig, text, colour)
102     local head = rig:FindFirstChild("Head")
103     local tag = head and head:FindFirstChild("NameTag")
104     local st = tag and tag:FindFirstChild("Status")
105     if st then st.Text = text; if colour then st.TextColor3 = colour end end
106 end
107
108 local function reviveLater(rig, spawnPivot)
109     task.delay(RESPAWN_DELAY, function()
110         if not rig or not rig.Parent then return end
111         local hum = rig:FindFirstChildOfClass("Humanoid")
112         if not hum then return end
113         rig:PivotTo(spawnPivot)
114         hum.Health = hum.MaxHealth
115         rig:SetAttribute("Fatigue", 0)
116         rig:SetAttribute("Panic", false)
117         rig:SetAttribute("NeedsHelp", false)
118         rig:SetAttribute("KnockedOut", false)
119         setStatus(rig, "Lab Technician", Color3.fromRGB(150, 200, 210))
120         say(rig, "I'm back. Please keep the chambers alive.", Color3.fromRGB(120, 220, 180))
121     end)
122 end
123

```

```

124 local function runTechnician(rig)
125     local hum = rig:FindFirstChildOfClass("Humanoid")
126     if not hum then return end
127     local home = rig:GetPivot()
128     local lastHelp, baseSpeed = 0, hum.WalkSpeed
129     local female = rig:GetAttribute("Gender") == "F"
130
131     hum.Died:Connect(function()
132         rig:SetAttribute("KnockedOut", true)
133         setStatus(rig, "COLLAPSED", Color3.fromRGB(255, 90, 80))
134         say(rig, "I can't... breathe...", Color3.fromRGB(255, 90, 80))
135         playSound(rig, female and "scream_f" or "scream_m")
136         reviveLater(rig, home)
137     end)
138
139     task.spawn(function()
140         while rig.Parent and hum.Health > 0 do
141             local dt = 1.0
142             local pos = rig:GetPivot().Position
143             local fault, dist = nearestFault(pos)
144             local fatigue = rig:GetAttribute("Fatigue") or 0
145
146             if fault then
147                 -- PANIC: exaggerated reaction, damage over time, calls for help
148                 rig:SetAttribute("Panic", true)
149                 rig:SetAttribute("NeedsHelp", true)
150                 if dist <= DAMAGE_RADIUS then
151                     hum.Health = math.max(HEALTH_FLOOR, hum.Health - DAMAGE_PER_SEC * dt)
152                 end
153                 fatigue = math.min(100, fatigue + 2)
154                 setStatus(rig, "PANIC - needs a repair tech!", Color3.fromRGB(255, 120, 100))
155
156                 -- flailing: jump and jerk toward/away from the hazard
157                 if math.random() < 0.5 then hum.Jump = true end
158                 local away = (pos - fault:GetPivot().Position).Unit * 6
159                 hum.MoveTo(pos + Vector3.new(away.X, 0, away.Z))
160
161                 if os.clock() - lastHelp > HELP_COOLDOWN then
162                     lastHelp = os.clock()
163                     local line = HELP_LINES[math.random(#HELP_LINES)]
164                     say(rig, string.format(line, labelOf(fault)), Color3.fromRGB(255, 120, 100))
165                     playSound(rig, math.random() < 0.5 and "help" or "follow")
166                 end
167             else
168                 rig:SetAttribute("Panic", false)
169                 rig:SetAttribute("NeedsHelp", false)
170                 fatigue = math.max(0, fatigue - 1.5)
171                 if hum.Health < hum.MaxHealth then
172                     hum.Health = math.min(hum.MaxHealth, hum.Health + 4)
173                 end
174             end
175
176             -- FATIGUE: slows them down, and they complain
177             rig:SetAttribute("Fatigue", fatigue)
178             hum.WalkSpeed = baseSpeed * (1 - 0.55 * (fatigue / 100))
179             if fatigue > 70 then
180                 setStatus(rig, "EXHAUSTED", Color3.fromRGB(255, 200, 90))
181                 if math.random() < 0.12 then
182                     say(rig, TIRED_LINES[math.random(#TIRED_LINES)], Color3.fromRGB(255, 200, 90))
183                 end
184             elseif not fault then
185                 setStatus(rig, "Lab Technician", Color3.fromRGB(150, 200, 210))
186             end
187
188             task.wait(dt)
189         end
190     end)
191 end
192
193 for _, rig in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
194     task.spawn(runTechnician, rig)
195 end
196 CollectionService.GetInstanceAddedSignal("LabWorkerBot"):Connect(function(rig)
197     task.wait(0.5); runTechnician(rig)
198 end)
199

```

```
200 print("[LabTechnicians] " .. #CollectionService:GetTagged("LabWorkerBot") .. " technicians online")
```

Chapter 54

S22 - LoungeSystems

ServerStorage.BeforeBasementAndQuiz_20260912.LoungeSystems

Authority group: archive. 4605 UTF-8 bytes; 152 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- LoungeSystems: the hidden social floor. Nothing here touches lab gameplay.
2 local Players = game:GetService("Players")
3 local TweenService = game:GetService("TweenService")
4
5 local L = workspace:WaitForChild("Lounge")
6 local hatch = workspace:WaitForChild("LoungeHatch")
7 local juke = L.DJ.Jukebox
8 local sound = juke:WaitForChild("JukeSound")
9 local npText = juke.NowPlaying.Frame.Text
10 local noteBody = L.MessageWall.Notes.Frame.Body
11
12 local ARRIVE = L.ArrivalPad.Position + Vector3.new(0, 4, 0)
13 local RETURN_T0 = hatch.Position + Vector3.new(0, 5, 0)
14
15 ----- travel
16 local function move(player, where, inLounge)
17     local char = player.Character
18     local hrp = char and char:FindFirstChild("HumanoidRootPart")
19     if not hrp then return end
20     hrp.CFrame = CFrame.new(where)
21     player:SetAttribute("InLounge", inLounge)
22 end
23
24 hatch.HatchPrompt.Triggered:Connect(function(player)
25     move(player, ARRIVE, true)
26 end)
27 L.ExitPad.ExitPrompt.Triggered:Connect(function(player)
28     move(player, RETURN_T0, false)
29 end)
30
31 -- if someone respawns while flagged, clear the flag
32 Players.PlayerAdded:Connect(function(p)
33     p.CharacterAdded:Connect(function()
34         if p:GetAttribute("InLounge") then p:SetAttribute("InLounge", false) end
35     end)
36 end)
37
38 ----- jukebox
39 local queue, current, skips = {}, nil, {}
40
41 local function refreshPanel()
42     local lines = {"JUKEBOX"}
43     table.insert(lines, current and ("now playing: " .. current.name) or "nothing playing")
44     table.insert(lines, "queued: " .. #queue)
45     table.insert(lines, "")
46     table.insert(lines, "say /play <audio id>")
47     table.insert(lines, "hold E to vote skip")
48 end
```

```

48     npText.Text = table.concat(lines, "\n")
49 end
50
51 local function playNext()
52     skips = {}
53     local nxt = table.remove(queue, 1)
54     if not nxt then
55         current = nil
56         sound:Stop()
57         refreshPanel()
58         return
59     end
60     current = nxt
61     sound.SoundId = "rbxassetid://" .. nxt.id
62     sound.TimePosition = 0
63     sound:Play()
64     refreshPanel()
65 end
66
67 sound.Ended:Connect(playNext)
68 sound.Stopped:Connect(function()
69     if current and not sound.IsPlaying then
70         task.wait(0.5)
71         if not sound.IsPlaying then playNext() end
72     end
73 end)
74
75 juke.JukePrompt.Triggered:Connect(function(player)
76     if not current then return end
77     skips[player.UserId] = true
78     local n = 0
79     for _ in pairs(skips) do n = n + 1 end
80     local need = math.max(1, math.floor(#Players:GetPlayers() / 2))
81     if n >= need then
82         playNext()
83     else
84         npText.Text = string.format("JUKEBOX\nskip votes %d / %d", n, need)
85         task.delay(2, refreshPanel)
86     end
87 end)
88
89 ----- the wall
90 local notes = {}
91 local function refreshWall()
92     if #notes == 0 then noteBody.Text = "(no messages yet)" return end
93     local lines = {}
94     for i = #notes, math.max(1, #notes - 7), -1 do
95         table.insert(lines, notes[i])
96     end
97     noteBody.Text = table.concat(lines, "\n")
98 end
99
100 ----- chat commands
101 local function onChat(player, msg)
102     if not player:GetAttribute("InLounge") then return end
103     local lower = string.lower(msg)
104     local id = string.match(lower, "^/play%s+(%d+)")
105     if id then
106         if #queue >= 12 then return end
107         table.insert(queue, {id = id, name = player.Name .. "'s pick (" .. id .. ")"})
108         if not current then playNext() else refreshPanel() end
109         return
110     end
111     if lower == "/stop" then
112         queue = {}; current = nil; sound:Stop(); refreshPanel(); return
113     end
114     local note = string.match(msg, "^/note%s+(.+)")
115     if note then
116         note = string.sub(note, 1, 90)
117         table.insert(notes, player.Name .. ": " .. note)
118         if #notes > 40 then table.remove(notes, 1) end
119         refreshWall()
120     end
121 end
122
123 local function hook(player) player.Chatted:Connect(function(m) onChat(player, m) end) end

```

```
124 for _, p in ipairs(Players:GetPlayers()) do hook(p) end
125 Players.PlayerAdded:Connect(hook)
126
127 ----- dance floor
128 local PALETTE = {
129     Color3.fromRGB(255, 80, 120), Color3.fromRGB(90, 200, 255),
130     Color3.fromRGB(255, 200, 90), Color3.fromRGB(150, 110, 255),
131     Color3.fromRGB(90, 255, 170),
132 }
133 task.spawn(function()
134     local tiles = L.DanceFloor:GetChildren()
135     local step = 0
136     while true do
137         step = step + 1
138         local beat = sound.IsPlaying and 0.35 or 0.9
139         for _, t in ipairs(tiles) do
140             local r = t:GetAttribute("Row") or 0
141             local c = t:GetAttribute("Col") or 0
142             local idx = ((r + c + step) % #PALETTE) + 1
143             TweenService:Create(t, TweenInfo.new(beat * 0.8), {Color = PALETTE[idx]}):Play()
144         end
145         task.wait(beat)
146     end
147 end)
148
149 refreshPanel()
150 refreshWall()
151 print("[LoungeSystems] hidden lounge online")
```

Chapter 55

S23 - ServerBlasterManager

ServerStorage.BeforeBasementAndQuiz_20260912.ServerBlasterManager

Authority group: archive. 7275 UTF-8 bytes; 175 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- ServerBlasterManager
2 -- Authoritative validation, obstacle recasting, damage, and shot replication.
3
4 local Players = game:GetService("Players")
5 local ReplicatedStorage = game:GetService("ReplicatedStorage")
6 local ServerScriptService = game:GetService("ServerScriptService")
7
8 local modules = ReplicatedStorage:WaitForChild("Modules")
9 local Blaster = require(modules:WaitForChild("Blaster"))
10
11 local remoteFolder = ReplicatedStorage:FindFirstChild(Blaster.REMOTE_FOLDER_NAME)
12 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end
13 if not remoteFolder then
14     remoteFolder = Instance.new("Folder")
15     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME
16     remoteFolder.Parent = ReplicatedStorage
17 end
18
19 local function getRemote(name)
20     local remote = remoteFolder:FindFirstChild(name)
21     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end
22     if not remote then
23         remote = Instance.new("RemoteEvent")
24         remote.Name = name
25         remote.Parent = remoteFolder
26     end
27     return remote
28 end
29
30 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)
31 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)
32
33 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")
34 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end
35 if not hitBridge then
36     hitBridge = Instance.new("BindableFunction")
37     hitBridge.Name = "BlasterHitBridge"
38     hitBridge.Parent = ServerScriptService
39 end
40
41 -- Roblox's default third-person zoom can legitimately place the camera well
42 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
43 -- zoom distances and the over-shoulder hip-fire camera.
44 local MAX_CAMERA_OFFSET = 40
45 local RATE_TOLERANCE = 0.88
46 local nextShotByPlayer = {}
47 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
```



```

48
49 local function findHumanoidFromHit(instance)
50     local node = instance
51     while node and node ~= workspace do
52         if node:IsA("Model") then
53             local humanoid = node:FindFirstChildOfClass("Humanoid")
54             if humanoid then return humanoid, node end
55         end
56         node = node.Parent
57     end
58     return nil
59 end
60
61 local function randomSpread(direction, spread)
62     if spread <= 0 then return direction end
63     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
64     local right = direction:Cross(reference).Unit
65     local up = right:Cross(direction).Unit
66     local radius = math.sqrt(math.random()) * spread
67     local angle = math.random() * math.pi * 2
68     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
69 end
70
71 local function broadcastShot(shooter, origin, position, normal, color)
72     local payload = {
73         origin = origin,
74         position = position,
75         normal = normal,
76         color = color,
77     }
78     for _, other in ipairs(Players:GetPlayers()) do
79         if other ~= shooter then effectRemote:FireClient(other, payload) end
80     end
81 end
82
83 local function bridgeHit(player, result, damage, instantHeadshot)
84     if not result then return false end
85     -- BindableFunction callbacks are write-only. LabEquipment binds this during
86     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
87     local ok, response = pcall(function()
88         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
89             instantHeadshot and result.Instance.Name == "Head")
90     end)
91     return ok and response == true
92 end
93
94 local function genericDamage(player, result, config)
95     local humanoid, model = findHumanoidFromHit(result.Instance)
96     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
97     local victimPlayer = Players:GetPlayerFromCharacter(model)
98     if victimPlayer and not config.allowPlayerDamage then return end
99
100     local damage = config.damage
101     if result.Instance.Name == "Head" then
102         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
103     end
104     humanoid:TakeDamage(damage)
105 end
106
107 local function validSequence(tool, sequence)
108     if typeof(sequence) ~= "number" or sequence ~= sequence or sequence % 1 ~= 0 then return false end
109     if sequence < 1 or sequence > 2147483647 then return false end
110     local previous = lastSequenceByTool[tool]
111     if previous and sequence <= previous then return false end
112     lastSequenceByTool[tool] = sequence
113     return true
114 end
115
116 local function handleFire(player, packet)
117     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
118     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
119         ↪ return end
120
121     local character = player.Character
122     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
123     local head = character and character:FindFirstChild("Head")

```

```

123     local root = character and character:FindFirstChild("HumanoidRootPart")
124     local tool = packet.tool
125     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
126     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
127     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
128
129     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
130     local directionMagnitude = packet.direction.Magnitude
131     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
132     local aimDirection = packet.direction.Unit
133     if not validSequence(tool, packet.sequence) then return end
134
135     local config = Blaster.GetWeaponConfig(tool)
136     local now = os.clock()
137     if now < (nextShotByPlayer[player] or 0) then return end
138     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
139
140     local params = Blaster.NewRaycastParams({character, tool})
141     local handle = tool:FindFirstChild("Handle")
142     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
143     local tracerColor = tool:GetAttribute("BlasterTracerColor")
144     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
145
146     for _ = 1, config.pellets do
147         local pelletDirection = randomSpread(aimDirection, config.spread)
148         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
149         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
            ↪ config.range)
150         local muzzleDelta = aimPoint - muzzleOrigin
151         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection
152         local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
153         local result = workspace:Raycast(muzzleOrigin, castDirection * castLength, params)
154         local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
155
156         broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
157         if result then
158             local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
159             if not handled then genericDamage(player, result, config) end
160         end
161     end
162
163     if config.selfCostPercent > 0 and humanoid.Health > 0 then
164         humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
165     end
166 end
167
168 fireRemote.OnServerEvent:Connect(handleFire)
169 Players.PlayerRemoving:Connect(function(player)
170     nextShotByPlayer[player] = nil
171 end)
172
173 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")
174
175

```

Chapter 56

S24 - BlasterController_ BeforeMouseArmFix_20260912

ServerStorage.BlasterController_BeforeMouseArmFix_20260912

Authority group: archive. 26610 UTF-8 bytes; 730 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

HISTORICAL BACKUP. This container is in ServerStorage and is not an active runtime script. Do not install it as an update without a deliberate comparison.

```
1 -- BlasterController
2 -- Local Tool controller: ContextActionService input, mobile controls, ADS,
3 -- touch aim damping, camera recoil, crosshair, and weapon bob.
4
5 local ContextActionService = game:GetService("ContextActionService")
6 local GuiService = game:GetService("GuiService")
7 local Players = game:GetService("Players")
8 local ReplicatedStorage = game:GetService("ReplicatedStorage")
9 local RunService = game:GetService("RunService")
10 local TweenService = game:GetService("TweenService")
11 local UserInputService = game:GetService("UserInputService")
12
13 local Blaster = require(script.Parent:WaitForChild("Blaster"))
14 local localPlayer = Players.LocalPlayer
15
16 local BlasterController = {}
17 BlasterController.__index = BlasterController
18
19 local controllers = setmetatable({}, {__mode = "k"})
20 local nextControllerId = 0
21
22 local function touchControlsActive()
23     if UserInputService.TouchEnabled then return true end
24     local okTouchScreen, touchScreen = pcall(function()
25         return UserInputService.TouchScreenEnabled
26     end)
27     if okTouchScreen and touchScreen then return true end
28     local okPreferred, preferred = pcall(function()
29         return UserInputService.PreferredInput
30     end)
31     if okPreferred and tostring(preferred):find("Touch") then return true end
32     local okControls, controls = pcall(function()
33         return GuiService.TouchControlsEnabled
34     end)
35     if okControls and controls then return true end
36
37     -- Studio can retain desktop input flags when a phone preset is selected
38     -- after the client starts. Recognize the simulator's compact landscape
39     -- viewport so the weapon does not silently lose its mobile camera/buttons.
40     local camera = workspace.CurrentCamera
41     local viewport = camera and camera.ViewportSize
42     return RunService:IsStudio()
43         and viewport ~= nil
44         and viewport.Y <= 450
```

```

45         and viewport.X / math.max(1, viewport.Y) >= 1.45
46     end
47
48     local function pointInside(button, point)
49         if not button or not button.Visible then return false end
50         local p = button.AbsolutePosition
51         local s = button.AbsoluteSize
52         return point.X >= p.X and point.X <= p.X + s.X
53             and point.Y >= p.Y and point.Y <= p.Y + s.Y
54     end
55
56     local function makeCrosshair()
57         local playerGui = localPlayer:WaitForChild("PlayerGui")
58         local old = playerGui:FindFirstChild("BlasterHUD")
59         if old then old:Destroy() end
60
61         local gui = Instance.new("ScreenGui")
62         gui.Name = "BlasterHUD"
63         gui.ResetOnSpawn = false
64         gui.IgnoreGuiInset = true
65         gui.ScreenInsets = Enum.ScreenInsets.None
66         gui.ClipToDeviceSafeArea = false
67         gui.DisplayOrder = 12
68         gui.Enabled = false
69         gui.Parent = playerGui
70
71         local root = Instance.new("Frame")
72         root.Name = "Crosshair"
73         root.AnchorPoint = Vector2.new(0.5, 0.5)
74         root.Position = UDim2.fromScale(0.5, 0.5)
75         root.Size = UDim2.fromOffset(30, 30)
76         root.BackgroundTransparency = 1
77         root.Parent = gui
78
79         local dot = Instance.new("Frame")
80         dot.Name = "Dot"
81         dot.AnchorPoint = Vector2.new(0.5, 0.5)
82         dot.Position = UDim2.fromScale(0.5, 0.5)
83         dot.Size = UDim2.fromOffset(6, 6)
84         dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
85         dot.BorderSizePixel = 0
86         dot.Parent = root
87         local dotCorner = Instance.new("UICorner")
88         dotCorner.CornerRadius = UDim.new(1, 0)
89         dotCorner.Parent = dot
90         local dotStroke = Instance.new("UIStroke")
91         dotStroke.Color = Color3.fromRGB(6, 15, 22)
92         dotStroke.Transparency = 0.15
93         dotStroke.Thickness = 1.5
94         dotStroke.Parent = dot
95
96         for index, data in ipairs({
97             {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
98             {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
99             {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
100            {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
101        }) do
102            local line = Instance.new("Frame")
103            line.Name = "Line" .. index
104            line.Position = data[1]
105            line.Size = data[2]
106            line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
107            line.BackgroundTransparency = 0.08
108            line.BorderSizePixel = 0
109            line.Parent = root
110        end
111
112        local status = Instance.new("TextLabel")
113        status.Name = "Status"
114        status.AnchorPoint = Vector2.new(0.5, 0)
115        status.Position = UDim2.new(0.5, 0, 0.5, 25)
116        status.Size = UDim2.fromOffset(210, 24)
117        status.BackgroundTransparency = 1
118        status.Font = Enum.Font.GothamBold
119        status.Text = ""
120        status.TextColor3 = Color3.fromRGB(220, 232, 242)

```

```

121     status.TextSize = 13
122     status.TextStrokeTransparency = 0.5
123     status.Parent = gui
124
125     return gui, root, status, dot
126 end
127
128 function BlasterController.new(tool, overrides)
129     assert(RunService.IsClient(), "BlasterController must be required on the client")
130     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
131
132     nextControllerId = nextControllerId + 1
133     local self = setmetatable({}, BlasterController)
134     self.tool = tool
135     self.overrides = overrides or {}
136     self.id = nextControllerId
137     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
138     self.connections = {}
139     self.enabled = false
140     self.aiming = false
141     self.triggerHeld = false
142     self.fireLoopRunning = false
143     self.sequence = 0
144     self.nextLocalShot = 0
145     self.baseGrip = tool.Grip
146     self.recoilPitch = 0
147     self.recoilYaw = 0
148     self.touchInput = nil
149     self.touchLastPosition = nil
150     self.hiddenParts = {}
151     self.shoulderOffset = Vector3.zero
152
153     self.connections.equipped = tool.Equipped:Connect(function()
154         self:Enable()
155     end)
156     self.connections.unequipped = tool.Unequipped:Connect(function()
157         self:Disable()
158     end)
159     self.connections.destroying = tool.Destroying:Connect(function()
160         self:Destroy()
161     end)
162     self.connections.ancestry = tool.AncestryChanged:Connect(function()
163         if self.enabled and tool.Parent ~= localPlayer.Character then
164             self:Disable()
165         end
166     end)
167     -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
168     -- remains the primary input layer, while this fallback covers touch/controller layouts
169     -- that route their built-in Tool button directly to Activated.
170     self.connections.activated = tool.Activated:Connect(function()
171         if self.enabled then self:_beginFiring() end
172     end)
173     self.connections.deactivated = tool.Deactivated:Connect(function()
174         self.triggerHeld = false
175     end)
176
177     if localPlayer.Character and tool.Parent == localPlayer.Character then
178         task.defer(function() self:Enable() end)
179     end
180
181     return self
182 end
183
184 function BlasterController:_button(actionName)
185     local ok, button = pcall(function()
186         return ContextActionService:GetButton(actionName)
187     end)
188     return ok and button or nil
189 end
190
191 function BlasterController:_fallbackTouchButton(kind, title)
192     local field = kind == "Shoot" and "fallbackShootButton" or "fallbackAimButton"
193     local existing = self[field]
194     if existing and existing.Parent then return existing end
195     if not self.hud then return nil end
196

```

```

197     local button = Instance.new("TextButton")
198     button.Name = "Fallback" .. kind
199     button.AutoButtonColor = true
200     button.BackgroundColor3 = Color3.fromRGB(35, 48, 60)
201     button.BorderSizePixel = 0
202     button.Text = title
203     button.Font = Enum.Font.GothamBold
204     button.TextColor3 = Color3.fromRGB(245, 250, 255)
205     button.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
206     button.TextStrokeTransparency = 0.25
207     button.TextScaled = true
208     button.ZIndex = 20
209     button.Parent = self.hud
210     self[field] = button
211
212     if kind == "Shoot" then
213         button.InputBegan:Connect(function(input)
214             if input.UserInputType == Enum.UserInputType.Touch
215                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
216                 self:_shootAction(nil, Enum.UserInputState.Begin)
217             end
218         end)
219         button.InputEnded:Connect(function(input)
220             if input.UserInputType == Enum.UserInputType.Touch
221                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
222                 self:_shootAction(nil, Enum.UserInputState.End)
223             end
224         end)
225     else
226         button.Activated:Connect(function()
227             if self.enabled then self:SetAiming(not self.aiming) end
228         end)
229     end
230     return button
231 end
232
233 function BlasterController:_styleTouchButtons()
234     if not touchControlsActive() then return end
235     local camera = workspace.CurrentCamera
236     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
237     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 64, 94)
238
239     local shootButton = self:_button(self.actionPrefix .. "Shoot")
240     or self:_fallbackTouchButton("Shoot", "FIRE")
241     local aimButton = self:_button(self.actionPrefix .. "Aim")
242     or self:_fallbackTouchButton("Aim", "AIM")
243     for _, button in ipairs({shootButton, aimButton}) do
244         if button then
245             button.Visible = true
246             button.Size = UDim2.fromOffset(diameter, diameter)
247             button.BackgroundTransparency = 0.18
248             button.ImageTransparency = 0.08
249             button.ImageColor3 = Color3.fromRGB(35, 48, 60)
250             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
251             corner.CornerRadius = UDim.new(1, 0)
252             corner.Parent = button
253             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
254             stroke.Color = Color3.fromRGB(220, 235, 245)
255             stroke.Transparency = 0.25
256             stroke.Thickness = 2
257             stroke.Parent = button
258             local title = button:FindFirstChild("ActionTitle")
259             if title and title:IsA("TextLabel") then
260                 title.Font = Enum.Font.GothamBold
261                 title.TextColor3 = Color3.fromRGB(245, 250, 255)
262                 title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
263                 title.TextStrokeTransparency = 0.25
264                 title.TextScaled = true
265             end
266         end
267     end
268
269     -- ContextActionGui uses a cropped/offset canvas in Studio's phone simulator.
270     -- Place the actual buttons in viewport-pixel space so their hit targets stay
271     -- fully visible on notched phones as well as large tablets.
272     local edge = math.max(24, math.floor(diameter * 0.3))

```

```

273     local gap = math.max(16, math.floor(diameter * 0.25))
274     -- Keep FIRE above Roblox's jump control and AIM to its left. A full button-height
275     -- bottom reserve prevents both actions from being clipped by Android navigation
276     -- and Studio's simulated safe-area canvas.
277     local bottomY = viewport.Y - diameter - math.max(82, edge)
278     local shootPoint = Vector2.new(
279         viewport.X - diameter - edge,
280         math.max(64, bottomY - diameter - gap)
281     )
282     local aimPoint = Vector2.new(
283         viewport.X - diameter * 2 - gap - edge,
284         math.max(64, bottomY)
285     )
286     local function place(button, point)
287         if not button or not button.Parent then return end
288         button.AnchorPoint = Vector2.zero
289         local origin = button.Parent.AbsolutePosition
290         button.Position = UDim2.fromOffset(
291             math.floor(point.X - origin.X + 0.5),
292             math.floor(point.Y - origin.Y + 0.5)
293         )
294     end
295     place(shootButton, shootPoint)
296     place(aimButton, aimPoint)
297 end
298
299 function BlasterController:_getAimScreenPoint(camera)
300     if not camera then return nil end
301     if touchControlsActive() then
302         -- Mobile aims along the Scriptable camera's true forward vector. In the
303         -- Studio phone simulator, ViewportPointToRay(viewportCenter) is offset by
304         -- the asymmetric safe-area canvas and can land several degrees above the
305         -- visible centre marker.
306         return nil
307     end
308     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
309     return Vector2.new(
310         math.clamp(point.X, 0, camera.ViewportSize.X),
311         math.clamp(point.Y, 0, camera.ViewportSize.Y)
312     )
313 end
314
315 function BlasterController:_updateAimReticle(camera)
316     if not self.hud or not self.crosshair or not camera then return end
317     local rayScreenPoint = self:_getAimScreenPoint(camera)
318     local screenPoint = rayScreenPoint
319     local character = localPlayer.Character
320     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
321         character,
322         self.tool,
323         camera,
324     }, rayScreenPoint)
325
326     if touchControlsActive() then
327         -- The shot uses Camera.CFrame.LookVector. Project that same forward
328         -- vector with WorldToScreenPoint so the marker lands in the exact CoreUI
329         -- coordinate space used by this ScreenGui, including notches/top bars.
330         local projected = camera:WorldToScreenPoint(
331             camera.CFrame.Position + camera.CFrame.LookVector * 100
332         )
333         screenPoint = Vector2.new(projected.X, projected.Y)
334     elseif not screenPoint then
335         screenPoint = camera.ViewportSize * 0.5
336     end
337
338     if solution then
339         if self.reticledot then
340             self.reticledot.BackgroundColor3 = solution.result
341             and Color3.fromRGB(255, 224, 105)
342             or Color3.fromRGB(120, 235, 255)
343         end
344     end
345
346     -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
347     -- and begin at a negative absolute position. Counter that offset so this
348     -- marker identifies the muzzle-checked world hit on every device.

```

```

349     local origin = self.hud.AbsolutePosition
350     local center = screenPoint - origin
351     self.crosshair.Position = UDim2.fromOffset(
352         math.floor(center.X + 0.5),
353         math.floor(center.Y + 0.5)
354     )
355 end
356
357 function BlasterController:_setLocalHeadHidden(hidden)
358     local character = localPlayer.Character
359     if not character then return end
360     if hidden then
361         for _, descendant in ipairs(character:GetDescendants()) do
362             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
363                 ↪ then
364                 if self.hiddenParts[descendant] == nil then
365                     self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
366                 end
367                 descendant.LocalTransparencyModifier = 1
368             end
369         end
370     else
371         for part, transparency in pairs(self.hiddenParts) do
372             if part.Parent then part.LocalTransparencyModifier = transparency end
373         end
374         table.clear(self.hiddenParts)
375     end
376 end
377
378 function BlasterController:_beginTouchCamera()
379     if not touchControlsActive() or self.mobileCameraActive then return end
380     local camera = workspace.CurrentCamera
381     local character = localPlayer.Character
382     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
383     if not camera or not humanoid then return end
384
385     local look = camera.CFrame.LookVector
386     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
387     self.touchYaw = math.atan2(-look.X, -look.Z)
388     self.savedCameraType = camera.CameraType
389     self.savedAutoRotate = humanoid.AutoRotate
390     self.mobileCameraActive = true
391     humanoid.AutoRotate = false
392     camera.CameraType = Enum.CameraType.Scriptable
393     self:_setLocalHeadHidden(true)
394 end
395
396 function BlasterController:_endTouchCamera()
397     if not self.mobileCameraActive then return end
398     local camera = workspace.CurrentCamera
399     local character = localPlayer.Character
400     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
401     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
402     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
403     self.mobileCameraActive = false
404     self.touchInput = nil
405     self.touchLastPosition = nil
406     self:_setLocalHeadHidden(false)
407 end
408
409 function BlasterController:SetAiming(aiming)
410     if not self.enabled then aiming = false end
411     if self.aiming == aiming then return end
412     self.aiming = aiming
413
414     local camera = workspace.CurrentCamera
415     if not camera then return end
416     local config = Blaster.GetWeaponConfig(self.tool)
417     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
418     if self.fovTween then self.fovTween:Cancel() end
419     self.fovTween = TweenService:Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
420         ↪ Enum.EasingDirection.Out), {
421         FieldOfView = targetFov,
422     })
423     self.fovTween:Play()
424 end

```



```

423     if touchControlsActive() then
424         -- Mobile weapons keep the camera active in hip-fire too, so the right
425         -- half of the screen always steers the reticle like a dedicated FPS.
426         self:_beginTouchCamera()
427     elseif aiming then
428         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
429         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
430     else
431         if self.savedMouseSensitivity then
432             UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
433             self.savedMouseSensitivity = nil
434         end
435     end
436
437     local character = localPlayer.Character
438     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
439     if humanoid and not self.mobileCameraActive then
440         humanoid.CameraOffset = aiming
441             and Vector3.new(1.25, 0.55, 0)
442             or self.shoulderOffset
443     end
444
445     if self.statusLabel then
446         self.statusLabel.Text = aiming and "ADS" or ""
447     end
448 end
449
450 function BlasterController:_fireOnce()
451     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
452     local config = Blaster.GetWeaponConfig(self.tool)
453     local now = os.clock()
454     if now < self.nextLocalShot then return end
455     self.nextLocalShot = now + config.cooldown
456     self.sequence = self.sequence + 1
457
458     local camera = workspace.CurrentCamera
459     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
460     local recoilScale = self.aiming and 0.55 or 1
461     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
462     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
463 end
464
465 function BlasterController:_beginFiring()
466     self.triggerHeld = true
467     if self.fireLoopRunning then return end
468     self.fireLoopRunning = true
469     task.spawn(function()
470         repeat
471             self:_fireOnce()
472             local config = Blaster.GetWeaponConfig(self.tool)
473             if not config.automatic then break end
474             task.wait(config.cooldown)
475         until not self.triggerHeld or not self.enabled
476         self.fireLoopRunning = false
477     end)
478 end
479
480 function BlasterController:_shootAction(_, inputState)
481     if not self.enabled then return Enum.ContextActionResult.Pass end
482     if inputState == Enum.UserInputState.Begin then
483         self:_beginFiring()
484     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
485         self.triggerHeld = false
486     end
487     return Enum.ContextActionResult.Sink
488 end
489
490 function BlasterController:_aimAction(_, inputState, inputObject)
491     if not self.enabled then return Enum.ContextActionResult.Pass end
492     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
493     if isTouch then
494         if inputState == Enum.UserInputState.Begin then
495             self:SetAiming(not self.aiming)
496         end
497     elseif inputState == Enum.UserInputState.Begin then
498         self:SetAiming(true)

```

```

499     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
500         self:SetAiming(false)
501     end
502     return Enum.ContextActionResult.Sink
503 end
504
505 function BlasterController:_touchPanAction(_, inputState, inputObject)
506     if not self.enabled or not touchControlsActive() then
507         return Enum.ContextActionResult.Pass
508     end
509
510     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
511     local camera = workspace.CurrentCamera
512     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
513     local inset = GuiService:GetGuiInset()
514     point = point - inset
515
516     if inputState == Enum.UserInputState.Begin then
517         local shootButton = self:_button(self.actionPrefix .. "Shoot") or self.fallbackShootButton
518         local aimButton = self:_button(self.actionPrefix .. "Aim") or self.fallbackAimButton
519         if point.X < viewport.X * 0.42 or pointInside(shootButton, point) or pointInside(aimButton, point) then
520             return Enum.ContextActionResult.Pass
521         end
522         self.touchInput = inputObject
523         self.touchLastPosition = point
524         return Enum.ContextActionResult.Sink
525     end
526
527     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
528     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
529         local delta = point - self.touchLastPosition
530         self.touchLastPosition = point
531         local sensitivity = self.aiming
532             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
533             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
534         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
535         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
536         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
537             ⇨ math.rad(78))
538         return Enum.ContextActionResult.Sink
539     end
540
541     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
542         self.touchInput = nil
543         self.touchLastPosition = nil
544         return Enum.ContextActionResult.Sink
545     end
546     return Enum.ContextActionResult.Pass
547 end
548
549 function BlasterController:_render(dt)
550     if not self.enabled then return end
551     local character = localPlayer.Character
552     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
553     local root = character and character:FindFirstChild("HumanoidRootPart")
554     local head = character and character:FindFirstChild("Head")
555     local camera = workspace.CurrentCamera
556     if not humanoid or not camera then return end
557     local touchMode = touchControlsActive()
558     if touchMode and not self.mobileCameraActive then
559         self:_beginTouchCamera()
560     elseif not touchMode and self.mobileCameraActive then
561         self:_endTouchCamera()
562     end
563     if touchMode and self.lastTouchViewport ~= camera.ViewportSize then
564         self.lastTouchViewport = camera.ViewportSize
565         task.defer(function()
566             if self.enabled then self:_styleTouchButtons() end
567         end)
568     end
569     local moving = math.clamp(humanoid.MoveDirection.Magnitude, 0, 1)
570     local speed = self.aiming and 7 or 9.5
571     local amount = self.aiming and 0.012 or 0.035
572     local clock = os.clock() * speed
573     local bob = CFrame.new(math.sin(clock) * amount * moving, math.abs(math.cos(clock)) * amount * moving, 0)
574         * CFrame.Angles(0, 0, math.sin(clock) * math.rad(0.35) * moving)

```

```

574     self.tool.Grip = self.baseGrip * bob
575
576     self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
577     self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
578     if self.mobileCameraActive and head and root then
579         self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
580         self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
581         local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
582         local focus = head.Position + Vector3.new(0, 0.12, 0)
583         local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
584         local cameraResult = workspace.Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
585             ↪ self.tool}))
586         local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
587         camera.CFrame = CFrame.new(position) * rotation
588         camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
589         local flatLook = Vector3.new(rotation.LookVector.X, 0, rotation.LookVector.Z)
590         if flatLook.Magnitude > 0.01 then
591             root.CFrame = CFrame.lookAt(root.Position, root.Position + flatLook.Unit)
592         end
593     elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
594         camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)
595     end
596     self:_updateAimReticle(camera)
597 end
598
599 function BlasterController:Enable()
600     if self.enabled then return end
601     self.enabled = true
602     self.baseGrip = self.tool.Grip
603     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
604         ↪ workspace.CurrentCamera.FieldOfView) or 70
605
606     if not self.hud or not self.hud.Parent then
607         self.hud, self.crosshair, self.statusLabel, self.reticuleDot = makeCrosshair()
608     end
609     self.hud.Enabled = true
610
611     local character = localPlayer.Character
612     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
613     if humanoid then
614         self.savedCameraOffset = humanoid.CameraOffset
615         self.shoulderOffset = Vector3.new(
616             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
617             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
618             0
619         )
620         humanoid.CameraOffset = self.shoulderOffset
621     end
622
623     if touchControlsActive() then self:_beginTouchCamera() end
624
625     local shootName = self.actionPrefix .. "Shoot"
626     local aimName = self.actionPrefix .. "Aim"
627     local panName = self.actionPrefix .. "TouchPan"
628     local priority = Enum.ContextActionPriority.High.Value
629
630     ContextActionService.BindActionAtPriority(shootName, function(...)
631         return self:_shootAction(...)
632     end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
633     ContextActionService.BindActionAtPriority(aimName, function(...)
634         return self:_aimAction(...)
635     end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
636     ContextActionService.BindActionAtPriority(panName, function(...)
637         return self:_touchPanAction(...)
638     end, false, priority + 1, Enum.UserInputType.Touch)
639
640     ContextActionService.SetTitle(shootName, "FIRE")
641     ContextActionService.SetTitle(aimName, "AIM")
642     task.delay(0.1, function()
643         if self.enabled then self:_styleTouchButtons() end
644     end)
645
646     self.renderName = self.actionPrefix .. "Render"
647     RunService.BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
648         self:_render(dt)
649     end)
650 end

```

```

648
649 function BlasterController:Disable()
650     if not self.enabled then return end
651     self.triggerHeld = false
652     self:SetAiming(false)
653     self.enabled = false
654
655     ContextActionService:UnbindAction(self.actionPrefix .. "Shoot")
656     ContextActionService:UnbindAction(self.actionPrefix .. "Aim")
657     ContextActionService:UnbindAction(self.actionPrefix .. "TouchPan")
658     if self.renderName then RunService:UnbindFromRenderStep(self.renderName) end
659     if self.hud then self.hud.Enabled = false end
660     if self.tool.Parent then self.tool.Grip = self.baseGrip end
661
662     local camera = workspace.CurrentCamera
663     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
664     self:_endTouchCamera()
665     if self.savedMouseSensitivity then
666         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
667         self.savedMouseSensitivity = nil
668     end
669     local character = localPlayer.Character
670     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
671     if humanoid and self.savedCameraOffset then
672         humanoid.CameraOffset = self.savedCameraOffset
673     end
674     self.savedCameraOffset = nil
675     self.lastTouchViewport = nil
676 end
677
678 function BlasterController:Destroy()
679     self:Disable()
680     for _, connection in pairs(self.connections) do
681         connection:Disconnect()
682     end
683     table.clear(self.connections)
684     if self.hud then self.hud:Destroy() end
685     controllers[self.tool] = nil
686 end
687
688 function BlasterController.BindTool(tool, overrides)
689     if controllers[tool] then return controllers[tool] end
690     local controller = BlasterController.new(tool, overrides)
691     controllers[tool] = controller
692     Blaster.StartReplication()
693     return controller
694 end
695
696 function BlasterController.GetController(tool)
697     return controllers[tool]
698 end
699
700 function BlasterController.AutoBind()
701     local binder = {connections = {}}
702
703     local function consider(instance)
704         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
705             ⇨ instance:GetAttribute("LabSidearm")) then
706             BlasterController.BindTool(instance)
707         end
708     end
709
710     local function watch(container)
711         if not container then return end
712         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
713         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
714     end
715
716     watch(localPlayer.WaitForChild("Backpack"))
717     watch(localPlayer.Character)
718     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
719     Blaster.StartReplication()
720
721     function binder:Destroy()
722         for _, connection in ipairs(self.connections) do connection:Disconnect() end
723         table.clear(self.connections)

```

```
723     end
724
725     return binder
726 end
727
728 return BlasterController
729
```

Part IV

Complete disabled Studio generator

Chapter 57

S29 - SeedLabGenerator_DISABLED_doNotEnable

Workspace.SeedLabGenerator_DISABLED_doNotEnable

Authority group: disabled. 524207 UTF-8 bytes; 11886 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

DISABLED GENERATOR. Retained as evidence. Enabling or executing it can rebuild/replace the world. It differs from the loose generator.

```
1  =====
2  -- LABORATORY COMPLEX -- PROCEDURAL BUILDING GENERATOR
3  -- Basement + 3 upper floors + roof.
4  -- Rooms are numbered. Equipment is generated from a catalogue of
5  -- basic laboratory items and packed against the walls automatically.
6  -- Vertical circulation: one stair tower + three working elevators.
7  --
8  -- HOW TO USE
9  --   Roblox Studio -> View -> Command Bar -> paste -> Enter.
10 --   Edit-mode Command Bar use installs the permanent runtime scripts.
11 --   Running from a server Script during Play also works for that session
12 --   through the built-in live fallback near the end of this file.
13  =====
14
15 local Workspace          = game:GetService("Workspace")
16 local ServerScriptService = game:GetService("ServerScriptService")
17 local ReplicatedStorage  = game:GetService("ReplicatedStorage")
18 local CollectionService  = game:GetService("CollectionService")
19 local Lighting           = game:GetService("Lighting")
20 local RunService         = game:GetService("RunService")
21 local StarterPlayer      = game:GetService("StarterPlayer")
22
23 -----
24 -- SCALE
25 -----
26 local STUD              = 28    -- cm per stud at 1:1
27 -- The building's footprint (SCALE) is independent of equipment size
28 -- (FIT_SCALE below): equipment is always drawn at its literal real-world
29 -- size, and only the ROOM PLAN gets stretched by SCALE before converting
30 -- to studs. So a bigger SCALE spreads the same fixed-size equipment
31 -- further apart -- exactly how you make a floor feel roomier instead of
32 -- crowded, without equipment growing at all.
33 local SCALE              = 3.6   -- horizontal size of the plan vs. true 1:1 (3x the previous pass)
34 local HEIGHT_SCALE      = 1.4   -- vertical size vs. true 1:1 -- kept modest on purpose: this is what
35                                -- was making the building read as an 8-story warehouse before, and
36                                -- fixing "crowded" doesn't need taller ceilings, just more floor room
37 -- Equipment stays at TRUE (1:1) human scale regardless of how big the
38 -- rooms are blown up, so a balance or a fridge reads as roughly the
39 -- size it should be next to a normal-sized player -- not a giant box.
40 local FIT_SCALE         = 1
41 local FIT_HEIGHT        = 1
42
43 -----
44 -- PLAN DIMENSIONS (centimetres)
```

```

45 -----
46 local WALL_H = 300
47 local SLAB = 25
48 local F2F = WALL_H + SLAB
49 local EXT_T = 30
50 local INT_T = 16
51 local DOOR_W = 90
52 local DOOR_H = 210
53 local WIDE_W = 150 -- double doors
54 local WIN_SILL = 90
55 local WIN_HEAD = 220
56
57 local COUNTER_H = 90 -- equipment cm
58 -- Overall footprint and corridor position follow the reference floor
59 -- plans (one long room down the west side, individually-doored rooms
60 -- off a corridor to the east): roughly 12.8 m x 20.8 m per floor.
61 local W, L = 1280, 2080
62 local CORR_X1, CORR_X2 = 500, 720
63
64 local RNG_SEED = 20260907
65
66 -----
67 -- LIGHTING
68 -----
69 local BUILD_LIGHTS = true
70 local LIGHT_SPACING = 40 -- in studs, against the now-true-scale building
71 local LIGHT_RANGE = 16 -- was tuned for a 24x-oversized building; way too
72 -- big now, which is most of why everything blew out white
73 local LIGHT_BRIGHTNESS = 0.6
74 local CONFIGURE_LIGHTING = true
75
76 -----
77 local C = {
78   slab = Color3.fromRGB(224, 227, 226),
79   basement = Color3.fromRGB(120, 122, 125),
80   ext = Color3.fromRGB(232, 232, 228),
81   int = Color3.fromRGB(244, 244, 246),
82   corridor = Color3.fromRGB(226, 231, 236),
83   basewall = Color3.fromRGB(105, 110, 116),
84   pillar = Color3.fromRGB(88, 92, 96),
85   stair = Color3.fromRGB(168, 170, 172),
86   roof = Color3.fromRGB(148, 150, 152),
87   glass = Color3.fromRGB(180, 214, 230),
88   door = Color3.fromRGB(120, 132, 146),
89   shaft = Color3.fromRGB(150, 154, 160),
90   car = Color3.fromRGB(206, 210, 214),
91 }
92
93 -- Stop a previous play-test runtime before creating replacement tagged
94 -- objects. Otherwise its CollectionService listeners can bind the new
95 -- equipment, then disappear when replaced, leaving dead interaction prompts.
96 for _, runtimeName in ipairs({"LabEquipment", "LabElevators", "LabRoles", "LabWorkflow", "SeedLabWorkflow",
97   ↳ "ServerBlasterManager"}) do
98   local previous = ServerScriptService:FindFirstChild(runtimeName)
99   if previous then previous:Destroy() end
100 end
101 for _, runtimeName in ipairs({"LabKinds", "LabFlow", "ISTA_Flow", "LabMathQuiz"}) do
102   local previous = ReplicatedStorage:FindFirstChild(runtimeName)
103   if previous then previous:Destroy() end
104 end
105 local previousModules = ReplicatedStorage:FindFirstChild("Modules")
106 if previousModules and previousModules:IsA("Folder") then
107   for _, moduleName in ipairs({"Blaster", "BlasterController"}) do
108     local previous = previousModules:FindFirstChild(moduleName)
109     if previous then previous:Destroy() end
110   end
111 end
112 for _, clientName in ipairs({"LabQuizClient", "LabSidearmClient", "LabHUD"}) do
113   local previousClient = StarterPlayer.StarterPlayerScripts:FindFirstChild(clientName)
114   if previousClient then previousClient:Destroy() end
115 end
116
117 -- Work that this generator does NOT own but which lives inside
118 -- Laboratory_Complex is carried across a rebuild instead of being wiped
119 -- with it. Rebuilding used to destroy the whole model outright, which
120 -- silently took hand-built content -- the site exterior and the office

```



```

120 -- fit-out -- with it every single time the generator was re-run.
121 -- Anything listed here, or any child carrying a PreserveOnRebuild
122 -- attribute, is detached first and re-attached to the fresh model.
123 -- SiteWorks and OfficeFitout used to be carried across a rebuild. They were
124 -- built for an earlier, differently-sized building, and measured against the
125 -- current one they no longer fit: NONE of the 5,541 office parts landed inside
126 -- the building footprint, and the exterior ran straight through the corridors.
127 -- Neither a translation nor a rotation lines them up (the room layout changed
128 -- between the two builds), so they are regenerated from the current W and L
129 -- instead. Carrying the old geometry would only reintroduce the overlap.
130 local SUPERSEDED_ON_REBUILD = {
131     SiteWorks = true,
132     OfficeFitout = true,
133 }
134
135 local PRESERVE_ON_REBUILD = {
136     -- add hand-built folder names here to carry them across a rebuild
137 }
138
139 local old = Workspace:FindFirstChild("Laboratory_Complex")
140 local preserved = {}
141 if old then
142     for _, child in ipairs(old:GetChildren()) do
143         if SUPERSEDED_ON_REBUILD[child.Name] then
144             child:Destroy() -- old geometry for a building that no longer exists
145         elseif PRESERVE_ON_REBUILD[child.Name] or child:GetAttribute("PreserveOnRebuild") then
146             child.Parent = Workspace -- park it outside the model being destroyed
147             table.insert(preserved, child)
148         end
149     end
150     old:Destroy()
151 end
152
153 local root = Instance.new("Model")
154 root.Name = "Laboratory_Complex"
155 root.Parent = Workspace
156
157 for _, child in ipairs(preserved) do
158     child:SetAttribute("PreserveOnRebuild", true) -- so it survives every later rebuild too
159     child.Parent = root
160 end
161 if #preserved > 0 then
162     print("Carried " .. #preserved .. " hand-built folder(s) across the rebuild.")
163 end
164
165 -----
166 -- UNITS
167 -----
168 local function X(v) return v * SCALE / STUD end -- plan cm -> studs, horizontal
169 local function Y(v) return v * HEIGHT_SCALE / STUD end -- plan cm -> studs, vertical
170 local function E(v) return v * FIT_SCALE / STUD end -- fit cm -> studs, horizontal
171 local function EH(v) return v * FIT_HEIGHT / STUD end -- fit cm -> studs, vertical
172 local function P(v) return v * FIT_SCALE / SCALE end -- fit cm -> plan cm
173
174 -----
175 -- DETERMINISTIC RANDOM (so a rebuild gives the same layout)
176 -----
177 local function newRng(seed)
178     local s = seed % 2147483647
179     if s <= 0 then s = s + 2147483646 end
180     local r = {}
181     function r.next() s = (s * 16807) % 2147483647; return s / 2147483647 end
182     function r.int(a, b) return a + math.floor(r.next() * (b - a + 1)) end
183     function r.pick(t) return t[r.int(1, #t)] end
184     function r.chance(p) return r.next() < p end
185     return r
186 end
187
188 -----
189 -- PRIMITIVES
190 -----
191 local function newPart(name, size, cframe, colour, material, parent)
192     local p = Instance.new("Part")
193     p.Name = name
194     p.Size = size
195     p.CFrame = cframe

```

```

196     p.Color = colour or C.int
197     p.Material = material or Enum.Material.SmoothPlastic
198     p.Anchored = true
199     p.TopSurface = Enum.SurfaceType.Smooth
200     p.BottomSurface = Enum.SurfaceType.Smooth
201     p.Parent = parent or root
202     return p
203 end
204
205 local function slab(parent, name, x1, z1, x2, z2, topY, thickness, colour, material)
206     thickness = thickness or SLAB
207     return newPart(name,
208         Vector3.new(X(math.abs(x2 - x1)), Y(thickness), X(math.abs(z2 - z1))),
209         CFrame.new(X((x1 + x2) / 2), Y(topY - thickness / 2), X((z1 + z2) / 2)),
210         colour or C.slab, material or Enum.Material.Concrete, parent)
211 end
212
213 -----
214 -- WALLS WITH OPENINGS
215 -----
216 local function wall(parent, name, x1, z1, x2, z2, thickness, baseY, height, openings, colour, material)
217     height = height or WALL_H
218     local alongX = (z1 == z2)
219     local len = alongX and math.abs(x2 - x1) or math.abs(z2 - z1)
220     local sgn = alongX and (x2 >= x1 and 1 or -1) or (z2 >= z1 and 1 or -1)
221     local n = 0
222
223     local function seg(s, e, y0, y1)
224         if e - s < 0.5 or y1 - y0 < 0.5 then return end
225         n = n + 1
226         local mid = (s + e) / 2
227         local px = alongX and (x1 + sgn * mid) or x1
228         local pz = alongX and z1 or (z1 + sgn * mid)
229         local size = alongX
230             and Vector3.new(X(e - s), Y(y1 - y0), X(thickness))
231             or Vector3.new(X(thickness), Y(y1 - y0), X(e - s))
232         newPart(name .. "-" .. n, size,
233             CFrame.new(X(px), Y(baseY + (y0 + y1) / 2), X(pz)),
234             colour or C.int, material or Enum.Material.Plaster, parent)
235     end
236
237     local ops = {}
238     for _, o in ipairs(openings or {}) do
239         table.insert(ops, {at = o.at, width = o.width, sill = o.sill or 0,
240             head = o.head or DOOR_H, glass = o.glass, leaf = o.leaf})
241     end
242     table.sort(ops, function(a, b) return a.at < b.at end)
243
244     local cursor = 0
245     for _, o in ipairs(ops) do
246         local a, b = math.max(0, o.at), math.min(len, o.at + o.width)
247         if a > cursor then seg(cursor, a, 0, height) end
248         if o.sill > 0 then seg(a, b, 0, o.sill) end
249         if o.head < height then seg(a, b, o.head, height) end
250
251         local mid = (a + b) / 2
252         local px = alongX and (x1 + sgn * mid) or x1
253         local pz = alongX and z1 or (z1 + sgn * mid)
254
255         if o.glass then
256             local size = alongX
257                 and Vector3.new(X(b - a), Y(o.head - o.sill), X(4))
258                 or Vector3.new(X(4), Y(o.head - o.sill), X(b - a))
259             local g = newPart(name .. "_glass" .. n, size,
260                 CFrame.new(X(px), Y(baseY + (o.sill + o.head) / 2), X(pz)),
261                 C.glass, Enum.Material.Glass, parent)
262             g.Transparency = 0.55
263         elseif o.leaf then
264             -- Decorative door casing: two flat leaves hinged against the wall
265             -- beside the reveal (CanCollide off, purely visual framing).
266             local leafW = (b - a) * 0.45
267             local off = (b - a) / 2 + leafW / 2
268             local sideOff = thickness / 2 + 6
269             for k = -1, 1, 2 do
270                 local size, lx, lz
271                 if alongX then

```

```

272         size = Vector3.new(X(leafW), Y(o.head - 8), X(7))
273         lx, lz = px + k * off, pz + sideOff
274     else
275         size = Vector3.new(X(7), Y(o.head - 8), X(leafW))
276         lx, lz = px + sideOff, pz + k * off
277     end
278     local d = newPart(name .. "_leaf" .. n .. "_" .. (k + 2), size,
279         CFrame.new(X(lx), Y(baseY + (o.head - 8) / 2 + 4), X(lz)),
280         C.door, Enum.Material.WoodPlanks, parent)
281     d.CanCollide = false
282 end
283
284 -- The actual functional door: a solid panel filling the opening
285 -- itself. It starts closed (CanCollide true) and is tagged
286 -- AutoDoor so the runtime slides it up into the header void when
287 -- a player approaches, and back down once they've passed through.
288 local doorHeight = math.max(o.head - o.sill - 6, 20)
289 local doorThickness = math.max(thickness * 0.6, 5)
290 local doorSize
291 if alongX then
292     doorSize = Vector3.new(X(b - a - 6), Y(doorHeight), X(doorThickness))
293 else
294     doorSize = Vector3.new(X(doorThickness), Y(doorHeight), X(b - a - 6))
295 end
296 local closedY = Y(baseY + o.sill + doorHeight / 2 + 3)
297 -- Automatic doors get their own industrial blue/metal look (never
298 -- the same colour/material as the wall or the decorative casing)
299 -- plus black-and-yellow hazard striping along the leading edge and
300 -- an "AUTOMATIC DOOR" surface label, so they read as a functional
301 -- door rather than a wall panel even before they start moving.
302 local autoDoor = newPart(name .. "_autodoor" .. n, doorSize,
303     CFrame.new(X(px), closedY, X(pz)), Color3.fromRGB(58, 104, 158), Enum.Material.Metal, parent)
304 autoDoor.CanCollide = true
305 -- Doors slide sideways into the wall pocket beside the opening
306 -- (left-to-right along the wall's own run), not up/down -- a
307 -- vertical slide read as flicker when it snapped through the
308 -- header void. The slide distance is the door's own width so it
309 -- tucks fully out of the opening.
310 -- IMPORTANT: these are stored in STUDS (via X()), matching the
311 -- part's actual CFrame units -- storing raw plan-cm values here
312 -- (as an earlier pass did) made the door's "closed"/"open"
313 -- targets read as almost-zero stud offsets, so the proximity
314 -- check never found a player "close enough" and the door never
315 -- moved at all.
316 autoDoor:SetAttribute("DoorAxis", alongX and "X" or "Z")
317 local closedXStud, closedZStud = X(px), X(pz)
318 autoDoor:SetAttribute("DoorClosedX", closedXStud)
319 autoDoor:SetAttribute("DoorClosedZ", closedZStud)
320 local slideDistStud = X(b - a) * 0.96
321 if alongX then
322     autoDoor:SetAttribute("DoorOpenX", closedXStud + slideDistStud)
323     autoDoor:SetAttribute("DoorOpenZ", closedZStud)
324 else
325     autoDoor:SetAttribute("DoorOpenX", closedXStud)
326     autoDoor:SetAttribute("DoorOpenZ", closedZStud + slideDistStud)
327 end
328 autoDoor:SetAttribute("DoorY", closedY)
329 autoDoor:SetAttribute("DoorOpen", false)
330 CollectionService.AddTag(autoDoor, "AutoDoor")
331
332 -- Hazard stripes, the "AUTOMATIC DOOR" label, and the open/closed
333 -- indicator light are all drawn as SurfaceGui elements parented to
334 -- the door part itself (rather than separate 3D parts), so they
335 -- track the door perfectly when the runtime tweens it open/closed.
336 local labelFaces = alongX and {Enum.NormalId.Front, Enum.NormalId.Back}
337     or {Enum.NormalId.Left, Enum.NormalId.Right}
338 for _, normal in ipairs(labelFaces) do
339     local labelGui = Instance.new("SurfaceGui")
340     labelGui.Name = "AutoDoorLabel"
341     labelGui.Face = normal
342     labelGui.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
343     labelGui.PixelsPerStud = 60
344     labelGui.AlwaysOnTop = false
345     labelGui.Parent = autoDoor
346
347     local topStripe = Instance.new("Frame")

```

```

348         topStripe.Name = "HazardStripeTop"
349         topStripe.Size = UDim2.new(1, 0, 0.07, 0)
350         topStripe.Position = UDim2.new(0, 0, 0, 0)
351         topStripe.BackgroundColor3 = Color3.fromRGB(235, 195, 35)
352         topStripe.BorderSizePixel = 0
353         topStripe.Parent = labelGui
354         local bottomStripe = topStripe:Clone()
355         bottomStripe.Name = "HazardStripeBottom"
356         bottomStripe.Position = UDim2.new(0, 0, 0.93, 0)
357         bottomStripe.Parent = labelGui
358
359         local labelText = Instance.new("TextLabel")
360         labelText.Name = "Caption"
361         labelText.Size = UDim2.new(0.86, 0, 0.24, 0)
362         labelText.Position = UDim2.new(0.07, 0, 0.13, 0)
363         labelText.BackgroundColor3 = Color3.fromRGB(235, 195, 35)
364         labelText.BackgroundTransparency = 0.05
365         labelText.BorderSizePixel = 0
366         labelText.Text = "AUTOMATIC DOOR"
367         labelText.TextColor3 = Color3.fromRGB(20, 20, 20)
368         labelText.TextScaled = true
369         labelText.Font = Enum.Font.SourceSansBold
370         labelText.Parent = labelGui
371
372         local indicator = Instance.new("Frame")
373         indicator.Name = "DoorIndicator"
374         indicator.Size = UDim2.new(0.16, 0, 0.06, 0)
375         indicator.Position = UDim2.new(0.42, 0, 0.42, 0)
376         indicator.BackgroundColor3 = Color3.fromRGB(210, 40, 40)
377         indicator.BorderSizePixel = 0
378         indicator.Parent = labelGui
379     end
380 end
381     cursor = math.max(cursor, b)
382 end
383 if cursor < len then seg(cursor, len, 0, height) end
384 end
385
386 -----
387 -- ROOM SCHEDULE -- numbered rooms
388 --
389 -- Matches the shape of the reference floor plans: one long room runs
390 -- the full length of the west side, and a cluster of individually
391 -- doored rooms line the east side of a central corridor. Room COUNT
392 -- varies by level, following the reference drawings: 10 on the ground
393 -- floor, 9 on the (identical) upper floors, 8 in the basement.
394 -----
395
396 -- Build the room list with every opening it owns.
397 -- namePrefix - nil for "Room 1".. "Room N", or e.g. "B" for basement
398 --              "Room B1".. "Room BN"
399 -- rightCount - how many individually-doored rooms line the east side
400 -- hasExterior - false suppresses exterior windows/exit doors (basement)
401 local function buildRoomSchedule(namePrefix, rightCount, hasExterior)
402     if hasExterior == nil then hasExterior = true end
403     local rooms, n = {}, 0
404
405     local function add(sideOfPlan, x1, x2, band)
406         n = n + 1
407         local label = namePrefix and (namePrefix .. n) or tostring(n)
408         local r = {
409             number = n, name = "Room " .. label, plan = sideOfPlan,
410             x1 = x1, x2 = x2, z1 = band[1], z2 = band[2],
411             open = {N = {}, S = {}, E = {}, W = {}},
412         }
413         rooms[n] = r
414     end
415     return r
416 end
417
418 add("L", 0, CORR_X1, {0, L})
419 local firstRight = n + 1
420 for i = 1, rightCount do
421     add("R", CORR_X2, W, {L * (i - 1) / rightCount, L * i / rightCount})
422 end
423
424 -- doors

```

```

424   for _, r in ipairs(rooms) do
425       local depth = r.z2 - r.z1
426       local corridorSide = (r.plan == "L") and "E" or "W"
427
428       -- corridor doors: one, or two for a deep room
429       if depth <= 300 then
430           table.insert(r.open[corridorSide], {at = depth / 2 - D00R_W / 2, width = D00R_W, leaf = true})
431       else
432           table.insert(r.open[corridorSide], {at = depth * 0.28 - D00R_W / 2, width = D00R_W, leaf = true})
433           table.insert(r.open[corridorSide], {at = depth * 0.72 - WIDE_W / 2, width = WIDE_W, leaf = true})
434       end
435
436       -- exterior wall: windows, plus an exit door on the deeper rooms
437       -- (skipped below grade, in the basement)
438       if hasExterior then
439           local extSide = (r.plan == "L") and "W" or "E"
440           if depth >= 300 then
441               table.insert(r.open[extSide], {at = depth * 0.12 - D00R_W / 2, width = D00R_W, head = 230, leaf
442               ↪ = true})
443               table.insert(r.open[extSide], {at = depth * 0.45 - 90, width = 180, sill = WIN_SILL, head =
444               ↪ WIN_HEAD, glass = true})
445               table.insert(r.open[extSide], {at = depth * 0.78 - 90, width = 180, sill = WIN_SILL, head =
446               ↪ WIN_HEAD, glass = true})
447           else
448               table.insert(r.open[extSide], {at = depth * 0.5 - 55, width = 110, sill = WIN_SILL, head =
449               ↪ WIN_HEAD, glass = true})
450           end
451       end
452       r.extDoorDepth = depth
453   end
454
455   -- interconnecting doors between consecutive east-side rooms
456   -- (the west side is a single room, so it needs none)
457   local function link(a, b)
458       local at = (a.x2 - a.x1) * 0.32 - D00R_W / 2
459       local op = {at = at, width = D00R_W, leaf = true}
460       table.insert(a.open.S, op)
461       table.insert(b.open.N, {at = at, width = D00R_W, leaf = true})
462   end
463   for i = firstRight, n - 1 do link(rooms[i], rooms[i + 1]) end
464
465   -- The building shell defines its north/south doors and windows separately
466   -- from the room schedule. Copy those openings into the rooms that touch
467   -- each end wall so wall packing can never put a cabinet across one.
468   if hasExterior then
469       local northOpenings = {
470           {at = CORR_X1 + 20, width = 160},
471           {at = W * 0.12, width = 140, glass = true},
472           {at = W * 0.77, width = 140, glass = true},
473       }
474       local southOpenings = {
475           {at = CORR_X1 + 25, width = WIDE_W},
476           {at = W * 0.12, width = 90},
477           {at = W * 0.20, width = 150, glass = true},
478           {at = W * 0.50, width = 150, glass = true},
479           {at = W * 0.80, width = 90},
480       }
481       local function copyEndOpenings(room, side, openings)
482           for _, opening in ipairs(openings) do
483               local left = math.max(room.x1, opening.at)
484               local right = math.min(room.x2, opening.at + opening.width)
485               if right > left then
486                   table.insert(room.open[side], {
487                       at = left - room.x1,
488                       width = right - left,
489                       glass = opening.glass,
490                   })
491               end
492           end
493       end
494       for _, room in ipairs(rooms) do
495           if room.z1 == 0 then copyEndOpenings(room, "N", northOpenings) end
496           if room.z2 == L then copyEndOpenings(room, "S", southOpenings) end
497       end
498   end
499 end

```

```

496     return rooms, firstRight
497 end
498
499 local GROUND_ROOMS    = buildRoomSchedule(nil, 9, true)    -- "Plan parter": 10 rooms
500 local TYPICAL_ROOMS    = buildRoomSchedule(nil, 8, true)    -- "Plan etaj": 9 rooms, reused on floors 2 and 3
501 local BASEMENT_ROOMS  = buildRoomSchedule("B", 7, false)   -- "Plan demisol": 8 rooms, no exterior windows
502
503 -----
504 -- SIGNAGE AND LIGHT
505 -----
506 local function sign(parent, text, xcm, zcm, baseY, px, maxDist)
507     local a = newPart("Sign", Vector3.new(1, 1, 1),
508         CFrame.new(X(xcm), Y(baseY + WALL_H * 0.72), X(zcm)), C.int, Enum.Material.SmoothPlastic, parent)
509     a.Transparency = 1
510     a.CanCollide = false
511     local bb = Instance.new("BillboardGui")
512     bb.Size = UDim2.new(0, px or 300, 0, (px or 300) * 0.32)
513     -- World labels only appear when the player is close. Long-range,
514     -- always-on-top labels stack over one another in a furnished room.
515     bb.MaxDistance = math.min(maxDist or 18, 22)
516     bb.AlwaysOnTop = false
517     bb.Parent = a
518     local t = Instance.new("TextLabel")
519     t.Size = UDim2.new(1, 0, 1, 0)
520     t.BackgroundTransparency = 1
521     t.Text = text
522     t.TextColor3 = Color3.fromRGB(35, 40, 48)
523     t.TextStrokeTransparency = 0.55
524     t.TextStrokeColor3 = Color3.new(1, 1, 1)
525     t.TextScaled = true
526     t.Font = Enum.Font.GothamBold
527     t.Parent = bb
528     return a
529 end
530
531 -- A stationary lab-staff mannequin, built from primitive parts at true
532 -- human scale (independent of the building's SCALE/HEIGHT_SCALE, same
533 -- as equipment) so it always reads as a person standing in the room.
534 -- Tagged "LabNPC" for the runtime script to arm with a "Talk" prompt.
535 local function buildNPC(parent, npcName, role, xcm, zcm, y0, badgeColour, tips)
536     -- Medical/lab-coat outfit: white coat and trousers for every NPC,
537     -- with a small coloured badge and a red cross on the chest so it
538     -- reads as clinical/lab staff rather than a random passer-by.
539     local skin = Color3.fromRGB(224, 189, 154)
540     local coat = Color3.fromRGB(246, 247, 249)
541     local trouser = Color3.fromRGB(210, 213, 217)
542     local baseY = Y(y0)
543
544     local legL = newPart(npcName .. "_LegL", Vector3.new(0.8, 2.3, 0.8),
545         CFrame.new(X(xcm) - 0.42, baseY + 1.15, X(zcm)), trouser, Enum.Material.Fabric, parent)
546     local legR = newPart(npcName .. "_LegR", Vector3.new(0.8, 2.3, 0.8),
547         CFrame.new(X(xcm) + 0.42, baseY + 1.15, X(zcm)), trouser, Enum.Material.Fabric, parent)
548     local torso = newPart(npcName .. "_Torso", Vector3.new(2, 2.1, 1.1),
549         CFrame.new(X(xcm), baseY + 3.35, X(zcm)), coat, Enum.Material.Fabric, parent)
550     local armL = newPart(npcName .. "_ArmL", Vector3.new(0.65, 2, 0.65),
551         CFrame.new(X(xcm) - 1.35, baseY + 3.3, X(zcm)), coat, Enum.Material.Fabric, parent)
552     local armR = newPart(npcName .. "_ArmR", Vector3.new(0.65, 2, 0.65),
553         CFrame.new(X(xcm) + 1.35, baseY + 3.3, X(zcm)), coat, Enum.Material.Fabric, parent)
554     local head = newPart(npcName .. "_Head", Vector3.new(1.25, 1.25, 1.25),
555         CFrame.new(X(xcm), baseY + 4.95, X(zcm)), skin, Enum.Material.SmoothPlastic, parent)
556     head.Shape = Enum.PartType.Ball
557
558     -- badge (role colour) and a small red cross, both worn on the chest
559     local badge = newPart(npcName .. "_Badge", Vector3.new(0.32, 0.42, 0.06),
560         CFrame.new(X(xcm) - 0.55, baseY + 3.55, X(zcm) - 0.58), badgeColour, Enum.Material.SmoothPlastic,
561         ↳ parent)
562     local crossV = newPart(npcName .. "_CrossV", Vector3.new(0.1, 0.32, 0.06),
563         CFrame.new(X(xcm) + 0.4, baseY + 3.45, X(zcm) - 0.58), Color3.fromRGB(200, 40, 40),
564         ↳ Enum.Material.SmoothPlastic, parent)
565     local crossH = newPart(npcName .. "_CrossH", Vector3.new(0.32, 0.1, 0.06),
566         CFrame.new(X(xcm) + 0.4, baseY + 3.45, X(zcm) - 0.58), Color3.fromRGB(200, 40, 40),
567         ↳ Enum.Material.SmoothPlastic, parent)
568
569     for _, p in ipairs({legL, legR, torso, armL, armR, head, badge, crossV, crossH}) do
570         p.CanCollide = false
571         p.CastShadow = true
572     end
573 end

```

```

569     end
570
571     CollectionService.AddTag(torso, "LabNPC")
572     torso:SetAttribute("NPCName", npcName)
573     torso:SetAttribute("Role", role)
574     torso:SetAttribute("Tips", table.concat(tips, "|"))
575
576     local bb = Instance.new("BillboardGui")
577     bb.Size = UDim2.new(0, 170, 0, 40)
578     bb.StudsOffset = Vector3.new(0, 3.2, 0)
579     bb.MaxDistance = 14
580     bb.AlwaysOnTop = false
581     bb.Parent = torso
582     local lbl = Instance.new("TextLabel")
583     lbl.Size = UDim2.new(1, 0, 1, 0)
584     lbl.BackgroundTransparency = 1
585     lbl.TextColor3 = Color3.fromRGB(255, 255, 255)
586     lbl.TextStrokeTransparency = 0.4
587     lbl.TextScaled = true
588     lbl.Font = Enum.Font.GothamBold
589     lbl.Text = npcName .. "\n" .. role
590     lbl.Parent = bb
591
592     return torso
593 end
594
595 -- Lightweight, server-driven worker models. They stay anchored and are
596 -- moved along navigation waypoints by the runtime, which lets fifty test
597 -- workers operate without the physics cost of fifty full player rigs.
598 -- Desk, chair and monitor for a seated worker. Built as separate furniture
599 -- rather than as part of the bot, so that killing the worker leaves the
600 -- workstation standing instead of tipping a desk over with the body.
601 local function buildWorkstation(parent, seatCF, name)
602     local metal = Color3.fromRGB(58, 62, 70)
603     local function piece(pieceName, size, offset, colour, material)
604         local part = newPart(name .. "_" .. pieceName, size, seatCF * offset, colour, material, parent)
605         part.CanCollide = false -- never block a walking route
606         part.CanTouch = false
607         return part
608     end
609     -- the bot faces -Z, so the desk sits at negative Z and the chair behind it
610     piece("ChairSeat", Vector3.new(1.7, 0.2, 1.7), CFrame.new(0, -1.10, 0.40), metal, Enum.Material.Metal)
611     piece("ChairBack", Vector3.new(1.7, 1.5, 0.2), CFrame.new(0, -0.35, 0.95), metal, Enum.Material.Metal)
612     piece("ChairStem", Vector3.new(0.25, 1.3, 0.25), CFrame.new(0, -1.78, 0.40), Color3.fromRGB(48, 52, 58),
613         Enum.Material.Metal)
614     piece("DeskTop", Vector3.new(3.4, 0.18, 1.9), CFrame.new(0, -0.45, -1.90), Color3.fromRGB(198, 188, 170),
615         Enum.Material.WoodPlanks)
616     piece("DeskLegL", Vector3.new(0.2, 2.0, 0.2), CFrame.new(-1.50, -1.45, -1.90), metal, Enum.Material.Metal)
617     piece("DeskLegR", Vector3.new(0.2, 2.0, 0.2), CFrame.new(1.50, -1.45, -1.90), metal, Enum.Material.Metal)
618     piece("Monitor", Vector3.new(1.5, 0.95, 0.12), CFrame.new(0, 0.22, -2.35), Color3.fromRGB(28, 32, 38),
619         Enum.Material.SmoothPlastic)
620     piece("MonitorStand", Vector3.new(0.35, 0.5, 0.3), CFrame.new(0, -0.28, -2.35), metal, Enum.Material.Metal)
621 end
622
623 local function buildWorkerBot(parent, botNumber, xcm, zcm, y0, seated)
624     local model = Instance.new("Model")
625     local isCompetitor = botNumber == 14
626     model.Name = isCompetitor and "RivalScientist" or string.format("WorkerBot_%02d", botNumber)
627     model.Parent = parent
628     local roleNames = {"Technician", "Laboratory Technician"}
629     local roleColours = {
630         Color3.fromRGB(68, 135, 220),
631         Color3.fromRGB(75, 175, 105),
632     }
633     local roleIndex = ((botNumber - 1) % #roleNames) + 1
634     local baseY = Y(y0)
635     local coat = Color3.fromRGB(240, 244, 246)
636     local skin = Color3.fromRGB(220, 182, 145)
637     local centre = Vector3.new(X(xcm), baseY, X(zcm))
638     local function botPart(name, size, offset, colour, material)
639         local part = newPart(name, size, CFrame.new(centre + offset), colour, material, model)
640         part.CanCollide = false
641         part.CanTouch = false
642         -- Queryable on purpose. These were CanQuery = false back when nothing
643         -- ever needed to hit-test a worker, which meant every shot fired at one
644         -- passed straight through it into the wall behind. A worker has to be

```

```

642     -- raycast-visible to be a target.
643     part.CanQuery = true
644     return part
645 end
646
647 local torso, head
648 local trouser = Color3.fromRGB(62, 72, 86)
649 if seated then
650     -- Desk pose: thighs forward, shins down, body dropped to chair height.
651     -- The whole model faces -Z, which is also its LookVector, so the caller
652     -- can simply PivotTo(CFrame.lookAt(seat, desk)) to sit it at a desk.
653     torso = botPart("Torso", Vector3.new(1.8, 2.0, 1.0), Vector3.new(0, 2.45, 0), coat,
654         ↳ Enum.Material.Fabric)
655     botPart("LegL", Vector3.new(0.7, 0.7, 1.7), Vector3.new(-0.42, 1.55, -0.75), trouser,
656         ↳ Enum.Material.Fabric)
657     botPart("LegR", Vector3.new(0.7, 0.7, 1.7), Vector3.new(0.42, 1.55, -0.75), trouser,
658         ↳ Enum.Material.Fabric)
659     botPart("ShinL", Vector3.new(0.7, 1.45, 0.7), Vector3.new(-0.42, 0.72, -1.5), trouser,
660         ↳ Enum.Material.Fabric)
661     botPart("ShinR", Vector3.new(0.7, 1.45, 0.7), Vector3.new(0.42, 0.72, -1.5), trouser,
662         ↳ Enum.Material.Fabric)
663     botPart("ArmL", Vector3.new(0.58, 0.58, 1.6), Vector3.new(-1.05, 2.6, -0.7), coat,
664         ↳ Enum.Material.Fabric)
665     botPart("ArmR", Vector3.new(0.58, 0.58, 1.6), Vector3.new(1.05, 2.6, -0.7), coat, Enum.Material.Fabric)
666     head = botPart("Head", Vector3.new(1.1, 1.1, 1.1), Vector3.new(0, 3.95, 0), skin,
667         ↳ Enum.Material.SmoothPlastic)
668     botPart("RoleBadge", Vector3.new(0.38, 0.42, 0.08), Vector3.new(-0.5, 2.68, -0.54),
669         ↳ roleColours[roleIndex], Enum.Material.Neon)
670 else
671     torso = botPart("Torso", Vector3.new(1.8, 2.0, 1.0), Vector3.new(0, 3.25, 0), coat,
672         ↳ Enum.Material.Fabric)
673     botPart("LegL", Vector3.new(0.7, 2.2, 0.7), Vector3.new(-0.42, 1.1, 0), trouser, Enum.Material.Fabric)
674     botPart("LegR", Vector3.new(0.7, 2.2, 0.7), Vector3.new(0.42, 1.1, 0), trouser, Enum.Material.Fabric)
675     botPart("ArmL", Vector3.new(0.58, 1.9, 0.58), Vector3.new(-1.18, 3.2, 0), coat, Enum.Material.Fabric)
676     botPart("ArmR", Vector3.new(0.58, 1.9, 0.58), Vector3.new(1.18, 3.2, 0), coat, Enum.Material.Fabric)
677     head = botPart("Head", Vector3.new(1.1, 1.1, 1.1), Vector3.new(0, 4.75, 0), skin,
678         ↳ Enum.Material.SmoothPlastic)
679     botPart("RoleBadge", Vector3.new(0.38, 0.42, 0.08), Vector3.new(-0.5, 3.48, -0.54),
680         ↳ roleColours[roleIndex], Enum.Material.Neon)
681 end
682 head.Shape = Enum.PartType.Ball
683
684 model.PrimaryPart = torso
685 model:SetAttribute("BotNumber", botNumber)
686 model:SetAttribute("Role", roleNames[roleIndex])
687 model:SetAttribute("Level", y0 == -F2F and "Basement" or ("Level_" .. (math.floor(y0 / F2F) + 1)))
688 model:SetAttribute("TaskState", "Waiting for plant task")
689 model:SetAttribute("CompletedCycles", 0)
690 model:SetAttribute("Competitor", isCompetitor)
691 model:SetAttribute("Score", 0)
692 model:SetAttribute("Health", 100)
693 model:SetAttribute("MaxHealth", 100)
694 model:SetAttribute("KnockedOut", false)
695 CollectionService.AddTag(model, "LabWorkerBot")
696
697 local gui = Instance.new("BillboardGui")
698 gui.Name = "BotStatus"
699 gui.Size = UDim2.fromOffset(172, 64)
700 gui.StudsOffset = Vector3.new(0, 3.0, 0)
701 -- Was 12 studs, which meant you had to stand almost on top of a bot to
702 -- see anything. The health bar is only useful if it reads across a room.
703 gui.MaxDistance = 70
704 gui.AlwaysOnTop = false
705 gui.Parent = head
706 local text = Instance.new("TextLabel")
707 text.Name = "Status"
708 text.Size = UDim2.fromScale(1, 0.5)
709 text.BackgroundTransparency = 1
710 text.TextColor3 = Color3.fromRGB(235, 245, 255)
711 text.TextStrokeTransparency = 0.35
712 text.TextScaled = true
713 text.Font = Enum.Font.GothamBold
714 text.Text = isCompetitor and "RIVAL NPC\nCompeting for resources"
715 or string.format("BOT %02d\n%s", botNumber, roleNames[roleIndex])
716 text.Parent = gui
717
718
719

```



```

707 -- Health bar under the caption. refreshBotHealth() has always looked for
708 -- children named HealthBarFill and HealthText and updated them -- but
709 -- nothing ever created them, so it searched, found nothing and silently
710 -- did nothing. A damaged bot never showed any damage.
711 local barBack = Instance.new("Frame")
712 barBack.Name = "HealthBarBack"
713 barBack.Size = UDim2.new(0.88, 0, 0.18, 0)
714 barBack.Position = UDim2.new(0.06, 0, 0.54, 0)
715 barBack.BackgroundColor3 = Color3.fromRGB(22, 26, 32)
716 barBack.BackgroundTransparency = 0.2
717 barBack.BorderSizePixel = 0
718 barBack.Parent = gui
719 local barFill = Instance.new("Frame")
720 barFill.Name = "HealthBarFill"
721 barFill.Size = UDim2.fromScale(1, 1)
722 barFill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
723 barFill.BorderSizePixel = 0
724 barFill.Parent = barBack
725 local healthText = Instance.new("TextLabel")
726 healthText.Name = "HealthText"
727 healthText.Size = UDim2.new(1, 0, 0.26, 0)
728 healthText.Position = UDim2.new(0, 0, 0.74, 0)
729 healthText.BackgroundTransparency = 1
730 healthText.TextColor3 = Color3.fromRGB(214, 232, 244)
731 healthText.TextStrokeTransparency = 0.4
732 healthText.TextScaled = true
733 healthText.Font = Enum.Font.Gotham
734 healthText.Text = "LIFE 100 / 100"
735 healthText.Parent = gui
736
737 return model
738 end
739
740 local function lamp(parent, xcm, zcm, baseY, brightness)
741 if not BUILD_LIGHTS then return end
742 local p = newPart("Luminaire", Vector3.new(X(120), Y(8), X(30)),
743 CFrames.new(X(xcm), Y(baseY + WALL_H - 14), X(zcm)),
744 Color3.fromRGB(250, 250, 244), Enum.Material.SmoothPlastic, parent)
745 p.CanCollide = false
746 p.CastShadow = false
747 local l = Instance.new("PointLight")
748 l.Range = LIGHT_RANGE
749 l.Brightness = (brightness or 1) * LIGHT_BRIGHTNESS
750 l.Color = Color3.fromRGB(255, 250, 236)
751 l.Shadows = false
752 l.Parent = p
753 end
754
755 local function lightGrid(parent, x1, z1, x2, z2, baseY, brightness)
756 local nx = math.max(1, math.floor(X(math.abs(x2 - x1)) / LIGHT_SPACING + 0.5))
757 local nz = math.max(1, math.floor(X(math.abs(z2 - z1)) / LIGHT_SPACING + 0.5))
758 for i = 1, nx do
759 for j = 1, nz do
760 lamp(parent, x1 + (x2 - x1) * ((i - 0.5) / nx), z1 + (z2 - z1) * ((j - 0.5) / nz), baseY,
761 ↪ brightness)
762 end
763 end
764 end
765
766 --=====
767 -- ELEVATOR CORES (three, set into the corridor so a walking route
768 -- always stays clear alongside each shaft)
769 --=====
770 -- Elevators removed by request: the building is stairs-only now (a tower
771 -- at each end of the corridor -- see buildStairTower below -- both running
772 -- basement to roof). Left as an empty list rather than deleting all the
773 -- downstream machinery: with zero entries here, no shafts get built, no
774 -- holes get cut in any floor slab, and the elevator control script simply
775 -- finds nothing tagged "Elevator" and does nothing.
776 local ELEVATORS = {}
777
778 -- The clear opening each shaft needs through a slab: the inner car
779 -- area only, so the slab still carries the shaft walls.
780 local ELEV_HOLES = {}
781 for _, e in ipairs(ELEVATORS) do
782 table.insert(ELEV_HOLES, {

```

```

782     x1 = (e.face == "X+") and (e.x1 + INT_T / 2) or e.x1,
783     x2 = (e.face == "X+") and e.x2 or (e.x2 - INT_T / 2),
784     z1 = e.z1 + INT_T / 2,
785     z2 = e.z2 - INT_T / 2,
786 })
787 end
788
789 -- Slab with rectangular holes punched through it (holes must not
790 -- overlap in z, which the elevator bays do not).
791 local function slabWithHoles(parent, name, x1, z1, x2, z2, topY, thickness, holes, colour, material)
792     local list = {}
793     for _, h in ipairs(holes or {}) do table.insert(list, h) end
794     table.sort(list, function(a, b) return a.z1 < b.z1 end)
795     local k = 0
796     local function rect(ax, az, bx, bz)
797         if bx - ax < 1 or bz - az < 1 then return end
798         k = k + 1
799         slab(parent, name .. "_" .. k, ax, az, bx, bz, topY, thickness, colour, material)
800     end
801     local cursor = z1
802     for _, h in ipairs(list) do
803         if h.z1 > cursor then rect(x1, cursor, x2, h.z1) end
804         if h.x1 > x1 then rect(x1, h.z1, h.x1, h.z2) end
805         if h.x2 < x2 then rect(h.x2, h.z1, x2, h.z2) end
806         cursor = math.max(cursor, h.z2)
807     end
808     if cursor < z2 then rect(x1, cursor, x2, z2) end
809 end
810
811 -----
812 -- LEVEL SHELL
813 -----
814 local LEVELS = {
815     {index = 0, label = "1", name = "Level_1", rooms = GROUND_ROOMS},
816     {index = 1, label = "2", name = "Level_2", rooms = TYPICAL_ROOMS},
817     {index = 2, label = "3", name = "Level_3", rooms = TYPICAL_ROOMS},
818 }
819 -- Five more floors on top of the original three (eight total). Each one
820 -- reuses the same typical room layout but gets its own RNG seed for
821 -- fitOutRoom (seeded off the level index further down), so the equipment
822 -- arrangement differs floor to floor even though the room shapes repeat.
823 for i = 3, 7 do
824     table.insert(LEVELS, {index = i, label = tostring(i + 1), name = "Level_" .. (i + 1), rooms =
825         ↳ TYPICAL_ROOMS})
826 end
827
828 local function offsetOps(list, base, extra)
829     local out = {}
830     for _, o in ipairs(list) do
831         table.insert(out, {at = base + o.at, width = o.width, sill = o.sill,
832             head = o.head, glass = o.glass, leaf = o.leaf})
833     end
834     for _, o in ipairs(extra or {}) do table.insert(out, o) end
835     return out
836 end
837
838 local function buildLevelShell(levelIndex, levelName, levelLabel, rooms)
839     local y = levelIndex * F2F
840     local m = Instance.new("Model")
841     m.Name = levelName
842     m.Parent = root
843
844     slabWithHoles(m, "FloorSlab", 0, 0, W, L, y, SLAB, ELEV_HOLES, C.slab, Enum.Material.SmoothPlastic)
845
846     -- corridor walls, assembled from every room's own door list
847     local wOps, eOps = {}, {}
848     for _, r in ipairs(rooms) do
849         if r.plan == "L" then
850             for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(wOps, o) end
851         else
852             for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(eOps, o) end
853         end
854     end
855     wall(m, "Corridor_West", CORR_X1, 0, CORR_X1, L, INT_T, y, WALL_H, wOps, C.int)
856     wall(m, "Corridor_East", CORR_X2, 0, CORR_X2, L, INT_T, y, WALL_H, eOps, C.int)

```

```

857 -- exterior long walls
858 local extW, extE = {}, {}
859 for _, r in ipairs(rooms) do
860   if r.plan == "L" then
861     for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(extW, o) end
862   else
863     for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(extE, o) end
864   end
865 end
866 wall(m, "Ext_West", EXT_T / 2, 0, EXT_T / 2, L, EXT_T, y, WALL_H, extW, C.ext, Enum.Material.Concrete)
867 wall(m, "Ext_East", W - EXT_T / 2, 0, W - EXT_T / 2, L, EXT_T, y, WALL_H, extE, C.ext,
  ↪ Enum.Material.Concrete)

868
869 -- exterior end walls
870 wall(m, "Ext_North", 0, EXT_T / 2, W, EXT_T / 2, EXT_T, y, WALL_H, {
871   {at = CORR_X1 + 20, width = 160, head = 240},
872   {at = W * 0.12, width = 140, sill = WIN_SILL, head = WIN_HEAD, glass = true},
873   {at = W * 0.77, width = 140, sill = WIN_SILL, head = WIN_HEAD, glass = true},
874 }, C.ext, Enum.Material.Concrete)
875
876 wall(m, "Ext_South", 0, L - EXT_T / 2, W, L - EXT_T / 2, EXT_T, y, WALL_H, {
877   {at = CORR_X1 + 25, width = WIDE_W, head = 240, leaf = true},
878   {at = W * 0.12, width = 90, head = 230, leaf = true},
879   {at = W * 0.20, width = 150, sill = WIN_SILL, head = WIN_HEAD, glass = true},
880   {at = W * 0.50, width = 150, sill = WIN_SILL, head = WIN_HEAD, glass = true},
881   {at = W * 0.80, width = 90, head = 230, leaf = true},
882 }, C.ext, Enum.Material.Concrete)
883
884 -- dividers, carrying the interconnecting doors
885 for i, r in ipairs(rooms) do
886   local nxt = rooms[i + 1]
887   if nxt and nxt.plan == r.plan and nxt.z1 == r.z2 then
888     wall(m, "Div_" .. i, r.x1, r.z2, r.x2, r.z2, INT_T, y, WALL_H,
889       offsetOps(r.open.S, r.x1), C.int)
890   end
891 end
892
893 -- signage and lighting
894 for _, r in ipairs(rooms) do
895   local sx = (r.plan == "L") and (CORR_X1 - 45) or (CORR_X2 + 45)
896   sign(m, r.name, sx, (r.z1 + r.z2) / 2, y, 220, 55)
897   lightGrid(m, r.x1, r.z1, r.x2, r.z2, y, 1)
898 end
899 lightGrid(m, CORR_X1, 0, CORR_X2, L, y, 0.8)
900 sign(m, "Floor " .. levelLabel, (CORR_X1 + CORR_X2) / 2, 120, y, 300, 90)
901
902 return m
903 end
904
905 =====
906 -- BASEMENT -- driven by BASEMENT_ROOMS, same shape as "Plan demisol":
907 -- one long room down the west side, 7 rooms off the corridor to the
908 -- east, solid (below-grade) exterior walls with no windows.
909 =====
910 local function buildBasement()
911   local y = -F2F
912   local m = Instance.new("Model")
913   m.Name = "Basement"
914   m.Parent = root
915
916   slab(m, "BasementSlab", 0, 0, W, L, y, 40, C.basement, Enum.Material.Concrete)
917
918   wall(m, "B_West", EXT_T, 0, EXT_T, L, 40, y, WALL_H, {}, C.basewall, Enum.Material.Concrete)
919   wall(m, "B_East", W - EXT_T, 0, W - EXT_T, L, 40, y, WALL_H, {}, C.basewall, Enum.Material.Concrete)
920   wall(m, "B_North", 0, EXT_T, W, EXT_T, 40, y, WALL_H,
921     {{at = CORR_X1 + 20, width = 160, head = 240}}, C.basewall, Enum.Material.Concrete)
922   wall(m, "B_South", 0, L - EXT_T, W, L - EXT_T, 40, y, WALL_H, {}, C.basewall, Enum.Material.Concrete)
923
924   -- corridor walls, doors gathered from every basement room's own list
925   local wOps, eOps = {}, {}
926   for _, r in ipairs(BASEMENT_ROOMS) do
927     if r.plan == "L" then
928       for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(wOps, o) end
929     else
930       for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(eOps, o) end
931     end

```

```

932     end
933     wall(m, "B_Corr_West", CORR_X1, 0, CORR_X1, L, INT_T, y, WALL_H, wOps, C.basewall)
934     wall(m, "B_Corr_East", CORR_X2, 0, CORR_X2, L, INT_T, y, WALL_H, eOps, C.basewall)
935
936     -- dividers between consecutive east-side bays
937     for i, r in ipairs(BASEMENT_ROOMS) do
938         local nxt = BASEMENT_ROOMS[i + 1]
939         if nxt and nxt.plan == r.plan and nxt.z1 == r.z2 then
940             wall(m, "B_Div_" .. i, r.x1, r.z2, r.x2, r.z2, INT_T, y, WALL_H,
941                 offsetOps(r.open.S, r.x1), C.basewall)
942         end
943     end
944
945     for _, x in ipairs({150, 250, W - 250, W - 150}) do
946         for z = 250, L - 200, 350 do
947             newPart("Pillar", Vector3.new(X(45), Y(WALL_H), X(45)),
948                 CFrame.new(X(x), Y(y + WALL_H / 2), X(z)), C.pillar, Enum.Material.Concrete, m)
949         end
950     end
951
952     for _, r in ipairs(BASEMENT_ROOMS) do
953         local sx = (r.plan == "L") and (CORR_X1 - 45) or (CORR_X2 + 45)
954         sign(m, r.name, sx, (r.z1 + r.z2) / 2, y, 200, 55)
955         lightGrid(m, r.x1, r.z1, r.x2, r.z2, y, 0.6)
956     end
957     lightGrid(m, CORR_X1, 0, CORR_X2, L, y, 0.5)
958     sign(m, "Basement", (CORR_X1 + CORR_X2) / 2, 120, y, 300, 90)
959     return m, BASEMENT_ROOMS
960 end
961
962 -----
963 -- STAIR TOWER
964 -----
965 -- centred on the corridor, which sits at CORR_X1..CORR_X2. Two towers now,
966 -- one attached beyond each end of the building (north, before z=0, and
967 -- south, beyond z=L) -- stairs "on each opposite part of the hallway" per
968 -- request, instead of a single stairwell everyone has to walk the whole
969 -- corridor to reach. Both run the full basement-to-roof height.
970 local TX1, TX2 = CORR_X1 - 50, CORR_X2 + 50
971 local TXM = (TX1 + TX2) / 2
972
973 local function buildStairTower(bottomLevel, topLevel, southEnd)
974     local m = Instance.new("Model")
975     m.Name = southEnd and "StairTower_South" or "StairTower_North"
976     m.Parent = root
977     local bottomY = bottomLevel * F2F
978     local shellH = (topLevel - bottomLevel) * F2F + WALL_H
979
980     -- All the tower's Z coordinates are measured outward from the building
981     -- edge it attaches to (0 = flush with the building). "dir" flips the
982     -- sign so the exact same layout mirrors onto the south end: outerZ is
983     -- the far exterior wall (carries the entrance door), edgeZ is flush
984     -- with the building's own end wall (and its existing WIDE_W doorway,
985     -- on the south end -- see Ext_South), flightNearZ/flightFarZ are the
986     -- switchback landing points in between. wall()/slab() both take their
987     -- span via math.abs() of the two endpoints, so the sign of "dir" alone
988     -- is enough to mirror every call below without reordering anything.
989     local edgeZ = southEnd and L or 0
990     local dir = southEnd and 1 or -1
991     local outerZ = edgeZ + dir * 460
992     local flightNearZ = edgeZ + dir * 100
993     local flightFarZ = edgeZ + dir * 400
994
995     wall(m, "T_West", TX1, edgeZ, TX1, outerZ, EXT_T, bottomY, shellH, {}, C.ext, Enum.Material.Concrete)
996     wall(m, "T_East", TX2, edgeZ, TX2, outerZ, EXT_T, bottomY, shellH, {}, C.ext, Enum.Material.Concrete)
997     wall(m, southEnd and "T_South" or "T_North", TX1, outerZ, TX2, outerZ, EXT_T, bottomY, shellH,
998         {{at = 105, width = 110, head = 235, leaf = true}}, C.ext, Enum.Material.Concrete)
999     slab(m, "T_Roof", TX1 - EXT_T, edgeZ, TX2 + EXT_T, outerZ + dir * EXT_T, bottomY + shellH + SLAB, SLAB,
1000         C.roof)
1001     slab(m, "T_Apron", TX1 - 60, outerZ, TX2 + 60, outerZ + dir * 260, 0, 20, C.slab)
1002
1003     local halfRise = Y(F2F) / 2
1004     local STEPS = math.max(8, math.ceil(halfRise / 1.4))
1005
1006     local function flight(name, x1, x2, zStart, zEnd, yStart, yEnd)
1007         local dz, dy = (zEnd - zStart) / STEPS, (yEnd - yStart) / STEPS

```

```

1007     for i = 1, STEPS do
1008         local z0, z1 = zStart + dz * (i - 1), zStart + dz * i
1009         local top = yStart + dy * i
1010         local a, b = math.min(z0, z1), math.max(z0, z1)
1011         newPart(name .. "_t" .. i, Vector3.new(X(x2 - x1), Y(18), X(math.abs(dz))),
1012             CFrame.new(X((x1 + x2) / 2), Y(top - 9), X((a + b) / 2)), C.stair, Enum.Material.Concrete, m)
1013         newPart(name .. "_r" .. i, Vector3.new(X(x2 - x1), Y(math.abs(dy) + 4), X(math.abs(dz))),
1014             CFrame.new(X((x1 + x2) / 2), Y(top - 18 - (math.abs(dy) + 4) / 2), X((a + b) / 2)),
1015             C.stair, Enum.Material.Concrete, m)
1016     end
1017 end
1018
1019 for lvl = bottomLevel, topLevel do
1020     local y = lvl * F2F
1021     slab(m, "Landing_L" .. lvl, TX1, flightNearZ, TX2, edgeZ, y, 15, C.stair)
1022     if lvl < topLevel then
1023         local half = y + F2F / 2
1024         flight("FlightA_L" .. lvl, TX1 + 10, TXM, flightNearZ, flightFarZ, y, half)
1025         slab(m, "MidLanding_L" .. lvl, TX1, outerZ, TX2, flightFarZ, half, 15, C.stair)
1026         flight("FlightB_L" .. lvl, TXM, TX2 - 10, flightFarZ, flightNearZ, half, y + F2F)
1027         newPart("Handrail_L" .. lvl, Vector3.new(X(8), Y(100), X(math.abs(flightNearZ - flightFarZ))),
1028             CFrame.new(X(TXM), Y(y + F2F / 2 + 50), X((flightFarZ + flightNearZ) / 2)),
1029             Color3.fromRGB(120, 128, 136), Enum.Material.Metal, m).CanCollide = false
1030     end
1031     lamp(m, (TX1 + TX2) / 2, (flightFarZ + flightNearZ) / 2, y, 0.8)
1032 end
1033 return m
1034 end
1035
1036 -----
1037 -- ELEVATOR BUILDER
1038 -----
1039 local function buildElevator(spec, bottomLevel, topLevel)
1040     local m = Instance.new("Model")
1041     m.Name = "Elevator_" .. spec.id
1042     m.Parent = root
1043
1044     local bottomY = bottomLevel * F2F
1045     local shellH = (topLevel - bottomLevel) * F2F + WALL_H
1046     local openAtX2 = (spec.face == "X+") -- open face is x2 (or x1)
1047     local backX = openAtX2 and spec.x1 or spec.x2
1048     local frontX = openAtX2 and spec.x2 or spec.x1
1049
1050     -- shaft: back wall plus both sides, open on the corridor face
1051     wall(m, "Shaft_Back", backX, spec.z1, backX, spec.z2, INT_T, bottomY, shellH, {}, C.shaft,
1052         Enum.Material.Metal)
1053     wall(m, "Shaft_S1", spec.x1, spec.z1, spec.x2, spec.z1, INT_T, bottomY, shellH, {}, C.shaft,
1054         Enum.Material.Metal)
1055     wall(m, "Shaft_S2", spec.x1, spec.z2, spec.x2, spec.z2, INT_T, bottomY, shellH, {}, C.shaft,
1056         Enum.Material.Metal)
1057     slab(m, "Shaft_Roof", spec.x1, spec.z1, spec.x2, spec.z2, bottomY + shellH + SLAB, SLAB, C.shaft)
1058
1059     local innerZ1, innerZ2 = spec.z1 + INT_T / 2, spec.z2 - INT_T / 2
1060     local innerX1 = openAtX2 and (spec.x1 + INT_T / 2) or spec.x1
1061     local innerX2 = openAtX2 and spec.x2 or (spec.x2 - INT_T / 2)
1062
1063     local floors = {}
1064     for lvl = bottomLevel, topLevel do table.insert(floors, lvl) end
1065
1066     m:SetAttribute("FloorCount", #floors)
1067     m:SetAttribute("Face", spec.face)
1068     for i, lvl in ipairs(floors) do
1069         m:SetAttribute("FloorY_" .. i, Y(lvl * F2F))
1070         m:SetAttribute("FloorLabel_" .. i, lvl < 0 and "B" or tostring(lvl + 1))
1071     end
1072
1073     -- landing gate + call button at every floor
1074     local gates = Instance.new("Folder"); gates.Name = "Gates"; gates.Parent = m
1075     local calls = Instance.new("Folder"); calls.Name = "CallPanels"; calls.Parent = m
1076     for i, lvl in ipairs(floors) do
1077         local y = lvl * F2F
1078         local g = newPart("Gate_" .. i,
1079             Vector3.new(X(8), Y(DOOR_H + 20), X(innerZ2 - innerZ1)),
1080             CFrame.new(X(frontX), Y(y + (DOOR_H + 20) / 2), X((innerZ1 + innerZ2) / 2)),
1081             Color3.fromRGB(110, 120, 132), Enum.Material.Metal, gates)
1082         g:SetAttribute("Floor", i)
1083     end
1084 end

```

```

1080
1081     local panel = newPart("Call_" .. i, Vector3.new(X(6), Y(40), X(30)),
1082         CFrame.new(X(frontX + (openAtX2 and 12 or -12)), Y(y + 120), X(innerZ1 - 25)),
1083         Color3.fromRGB(60, 66, 74), Enum.Material.Metal, calls)
1084     panel:SetAttribute("Floor", i)
1085     panel:SetAttribute("ElevatorId", spec.id)
1086 end
1087
1088 -- the car
1089 local car = Instance.new("Model"); car.Name = "Car"; car.Parent = m
1090 local startY = bottomLevel * F2F
1091 local cw = innerX2 - innerX1
1092 local cd = innerZ2 - innerZ1
1093
1094 slab(car, "Platform", innerX1, innerZ1, innerX2, innerZ2, startY + 10, 10, C.car, Enum.Material.Metal)
1095 local function carWall(name, x1, z1, x2, z2)
1096     wall(car, name, x1, z1, x2, z2, 10, startY + 10, 250, {}, C.car, Enum.Material.Metal)
1097 end
1098 carWall("Car_Back", backX + (openAtX2 and 12 or -12), innerZ1, backX + (openAtX2 and 12 or -12), innerZ2)
1099 carWall("Car_S1", innerX1, innerZ1 + 6, innerX2, innerZ1 + 6)
1100 carWall("Car_S2", innerX1, innerZ2 - 6, innerX2, innerZ2 - 6)
1101 slab(car, "Car_Roof", innerX1, innerZ1, innerX2, innerZ2, startY + 270, 10, C.car, Enum.Material.Metal)
1102 local carLight = newPart("Car_Light", Vector3.new(X(40), Y(6), X(40)),
1103     CFrame.new(X((innerX1 + innerX2) / 2), Y(startY + 250), X((innerZ1 + innerZ2) / 2)),
1104     Color3.fromRGB(255, 255, 245), Enum.Material.Neon, car)
1105 carLight.CanCollide = false
1106 local cl = Instance.new("PointLight"); cl.Range = 30; cl.Brightness = 1.2; cl.Parent = carLight
1107
1108 -- in-car buttons, one per floor, on the back wall
1109 local buttons = Instance.new("Folder"); buttons.Name = "Buttons"; buttons.Parent = car
1110 for i, lvl in ipairs(floors) do
1111     local t = (i - 0.5) / #floors
1112     local b = newPart("Btn_" .. i, Vector3.new(X(8), Y(26), X(16)),
1113         CFrame.new(X(backX + (openAtX2 and 22 or -22)), Y(startY + 130),
1114             X(innerZ1 + (innerZ2 - innerZ1) * t)),
1115         Color3.fromRGB(230, 200, 90), Enum.Material.Neon, buttons)
1116     b.CanCollide = false
1117     b:SetAttribute("Floor", i)
1118 end
1119
1120 m:SetAttribute("CurrentFloor", 1)
1121 CollectionService.AddTag(m, "Elevator")
1122 return m
1123 end
1124
1125 =====
1126 -- BASIC LABORATORY EQUIPMENT CATALOGUE
1127 -- Nothing here is tied to a particular room or process. The fit-out
1128 -- for every room is generated from this list.
1129 =====
1130 local COL = {
1131     bench   = Color3.fromRGB(178, 182, 186),
1132     steel   = Color3.fromRGB(196, 200, 205),
1133     white   = Color3.fromRGB(236, 238, 241),
1134     dark    = Color3.fromRGB(72, 78, 86),
1135     timber  = Color3.fromRGB(166, 142, 112),
1136     cold    = Color3.fromRGB(198, 216, 226),
1137     warm    = Color3.fromRGB(206, 178, 140),
1138     safety  = Color3.fromRGB(190, 62, 52),
1139     green   = Color3.fromRGB(78, 138, 96),
1140     glassy  = Color3.fromRGB(198, 220, 230),
1141 }
1142
1143 -- class: "run" = long fitted furniture, takes the best wall
1144 -- "floor" = free-standing unit against a wall
1145 -- "top" = sits on a bench run
1146 -- "duty" = safety kit, one of each per room
1147 -- shape: how the item is actually built out of primitive parts, so it
1148 -- reads as real equipment rather than a plain box (see buildKind below)
1149 local CATALOGUE = {
1150     -- Room contents are now limited to equipment that actually acts on a
1151     -- germinated plant sample (matches an entry in PLANT_PROCESS, or is the
1152     -- separately-placed Plant Growth Chamber). Generic lab furniture,
1153     -- benchtop instruments and safety kit that never touch a plant sample
1154     -- were removed by request so every object in a room is something that
1155     -- moves a plant's status forward.

```

```

1156 {kind = "Germination Chamber", class = "floor", w = 95, d = 80, h = 195, col = COL.cold, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance", auto = "pulse"},
1157 {kind = "Incubator", class = "floor", w = 90, d = 75, h = 180, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance"},
1158 {kind = "Decontamination Unit", class = "floor", w = 75, d = 60, h = 150, col = COL.white, mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "appliance", auto = "pulse"},
1159 {kind = "Mutagenesis Chamber", class = "floor", w = 90, d = 80, h = 200, col = COL.safety, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "pulse"},
1160 {kind = "Hormone Treatment Bench", class = "floor", w = 80, d = 65, h = 175, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "spin"},
1161
1162 -- Smaller "run" variants of the same five kinds so bench-run islands
1163 -- (the middle of a room's floor, not just its walls) also fill up with
1164 -- functional equipment -- this is most of what gets the building well
1165 -- past a thousand units total.
1166 {kind = "Germination Chamber", class = "run", w = 68, d = 55, h = 195, col = COL.cold, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance", auto = "pulse"},
1167 {kind = "Incubator", class = "run", w = 64, d = 52, h = 180, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance"},
1168 {kind = "Decontamination Unit", class = "run", w = 58, d = 46, h = 150, col = COL.white, mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "appliance", auto = "pulse"},
1169 {kind = "Mutagenesis Chamber", class = "run", w = 64, d = 55, h = 200, col = COL.safety, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "pulse"},
1170 {kind = "Hormone Treatment Bench", class = "run", w = 58, d = 48, h = 175, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "spin"},
1171
1172 -- Pure decoration: potted plants, flower planters, and lab storage
1173 -- furniture (drawer units / filing cabinets). None of these are tagged
1174 -- as functional equipment (no status LED, no failure/hazard behaviour)
1175 -- -- they exist purely to fill every remaining sliver of wall and floor
1176 -- space so the building reads as a real, lived-in facility rather than
1177 -- rows of machines with bare walls and floor between them.
1178 {kind = "Potted Plant", class = "decor", w = 42, d = 42, h = 62, col = Color3.fromRGB(150, 108, 70), mat =
    ↳ Enum.Material.SmoothPlastic, weight = 5, shape = "planter"},
1179 {kind = "Flowering Planter", class = "decor", w = 55, d = 32, h = 40, col = Color3.fromRGB(168, 120, 78),
    ↳ mat = Enum.Material.SmoothPlastic, weight = 4, shape = "planter", flowering = true},
1180 {kind = "Storage Drawers", class = "decor", w = 62, d = 45, h = 95, col = Color3.fromRGB(96, 102, 110), mat
    ↳ = Enum.Material.Metal, weight = 4, shape = "drawers"},
1181 {kind = "Filing Cabinet", class = "decor", w = 48, d = 52, h = 130, col = Color3.fromRGB(72, 78, 86), mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "drawers"},
1182 {kind = "Supply Shelf", class = "decor", w = 90, d = 38, h = 175, col = Color3.fromRGB(120, 126, 130), mat
    ↳ = Enum.Material.Metal, weight = 3, shape = "rack"},
1183 }
1184
1185 -- Scientific machines share one readable, refrigerator-like design language.
1186 -- Desks and benches remain furniture; benchtop instruments and safety kit
1187 -- keep their own real-world proportions (a Microscope should never look like
1188 -- a walk-in freezer). Only true free-standing floor equipment -- growth
1189 -- chambers, ovens, storage cabinets and the like -- get the industrial
1190 -- cabinet treatment, and it's sized for an industrial facility: wide, tall
1191 -- units with real walking clearance around them. Width/height variants
1192 -- still communicate capacity.
1193 local CABINET_SIZES = {
1194     {w = 100, d = 90, h = 220, capacity = 6},
1195     {w = 115, d = 95, h = 235, capacity = 8},
1196     {w = 130, d = 100, h = 250, capacity = 10},
1197     {w = 145, d = 105, h = 265, capacity = 14},
1198 }
1199 for _, entry in ipairs(CATALOGUE) do
1200     if entry.class == "floor" then
1201         local hash = 0
1202         for i = 1, #entry.kind do hash = hash + string.byte(entry.kind, i) end
1203         local cabinet = CABINET_SIZES[hash % #CABINET_SIZES + 1]
1204         entry.w, entry.d, entry.h = cabinet.w, cabinet.d, cabinet.h
1205         entry.capacity = cabinet.capacity
1206         entry.shape = "appliance"
1207         entry.plantCabinet = true
1208     end
1209 end
1210
1211 local BY_CLASS = {run = {}, floor = {}, top = {}, duty = {}, decor = {}}
1212 for _, e in ipairs(CATALOGUE) do
1213     table.insert(BY_CLASS[e.class], e)
1214 end
1215
1216 local function weightedPick(list, rng)

```

```

1217     local total = 0
1218     for _, e in ipairs(list) do total = total + (e.weight or 1) end
1219     local roll = rng.next() * total
1220     for _, e in ipairs(list) do
1221         roll = roll - (e.weight or 1)
1222         if roll <= 0 then return e end
1223     end
1224     return list[#list]
1225 end
1226
1227 -----
1228 -- AUTOMATIC WALL PACKING
1229 -----
1230 local WALL_GAP   = 16 -- plan cm of clear floor between a wall face and a cabinet
1231 local ALONG_GAP  = 26 -- plan cm between neighbours -- enough of an aisle gap that each
1232                      -- cabinet's status text doesn't overlap its neighbour's
1233 local CORNER     = 100 -- plan cm kept clear at each corner (must exceed
1234                      -- the deepest unit so adjacent walls cannot clash)
1235 local DOOR_CLEAR = 55 -- plan cm kept clear either side of a door
1236
1237 -- Free intervals along one wall of a room, in that wall's local
1238 -- coordinates, with door swings excluded.
1239 local function freeSpans(length, openings)
1240     local blocked = {}
1241     for _, o in ipairs(openings or {}) do
1242         -- Windows need clear floor space just as doors do. The smaller window
1243         -- margin preserves capacity while keeping every cabinet out of glazing.
1244         local clear = o.glass and 30 or DOOR_CLEAR
1245         table.insert(blocked, {o.at - clear, o.at + o.width + clear})
1246     end
1247     table.sort(blocked, function(a, b) return a[1] < b[1] end)
1248
1249     local spans, cursor = {}, CORNER
1250     for _, b in ipairs(blocked) do
1251         if b[1] > cursor then table.insert(spans, {cursor, math.min(b[1], length - CORNER)}) end
1252         cursor = math.max(cursor, b[2])
1253     end
1254     if cursor < length - CORNER then table.insert(spans, {cursor, length - CORNER}) end
1255
1256     local out = {}
1257     for _, s in ipairs(spans) do
1258         if s[2] - s[1] > 20 then table.insert(out, s) end
1259     end
1260     return out
1261 end
1262
1263 local function takeSpan(spans, need)
1264     for _, s in ipairs(spans) do
1265         if s[2] - s[1] >= need then
1266             local at = s[1] + need / 2
1267             s[1] = s[1] + need + ALONG_GAP
1268             return at
1269         end
1270     end
1271     return nil
1272 end
1273
1274 -- Same as takeSpan but with a caller-chosen gap. Decoration (plants,
1275 -- drawers, ...) doesn't carry a status readout that neighbours could
1276 -- overlap, so it can pack into slivers real equipment leaves behind with
1277 -- only a token gap between pieces.
1278 local function takeSpanGap(spans, need, gap)
1279     for _, s in ipairs(spans) do
1280         if s[2] - s[1] >= need then
1281             local at = s[1] + need / 2
1282             s[1] = s[1] + need + gap
1283             return at
1284         end
1285     end
1286     return nil
1287 end
1288
1289 local function spansTotal(spans)
1290     local t = 0
1291     for _, s in ipairs(spans) do t = t + (s[2] - s[1]) end
1292     return t

```



```

1293 end
1294
1295 =====
1296 -- EQUIPMENT SHAPES
1297 -- Every catalogue entry is built from 1-4 primitive parts instead of a
1298 -- single box, so it reads as the kind of thing it is: a bench with
1299 -- legs and a top, an appliance with a door and a handle, a rack with
1300 -- shelves, an instrument with a small control panel. Everything here
1301 -- is axis-aligned (the generator never yaws a part), so offsets are
1302 -- plain world-space vectors added to a centre position.
1303 =====
1304 local function buildKind(parent, entry, pos, size, wallSide)
1305     local shape = entry.shape or "box"
1306     local dark = Color3.fromRGB(36, 38, 44)
1307     -- The accent/panel colour always contrasts AGAINST the body colour,
1308     -- not just darkens it: a body that's already dark (COL.dark items
1309     -- like the hot plate stirrer, the pH meter, ...) used to get an
1310     -- accent that was darker still, so the door/panel disappeared into
1311     -- the body and the whole thing read as a plain black box.
1312     local lum = entry.col.R * 0.299 + entry.col.G * 0.587 + entry.col.B * 0.114
1313     local accent
1314     if lum > 0.45 then
1315         accent = Color3.fromRGB(
1316             math.max(0, entry.col.R * 255 - 45),
1317             math.max(0, entry.col.G * 255 - 45),
1318             math.max(0, entry.col.B * 255 - 45))
1319     else
1320         accent = Color3.fromRGB(
1321             math.min(255, entry.col.R * 255 + 80),
1322             math.min(255, entry.col.G * 255 + 80),
1323             math.min(255, entry.col.B * 255 + 80))
1324     end
1325     local panelColour = lum > 0.45 and dark or Color3.fromRGB(168, 172, 178)
1326
1327     -- Every sub-part of an assembly is parented to the piece's own main
1328     -- part (not the room model) so the runtime script can reliably find
1329     -- "this instance's own door/panel/light" with FindFirstChild instead
1330     -- of guessing among same-named siblings from every other item.
1331     if shape == "appliance" then
1332         -- Every appliance-class unit is built as a real germination-cabinet
1333         -- style enclosure -- insulated shell, glass front with frame trim,
1334         -- interior shelving/lighting, a rear vent, a controller screen and
1335         -- caster wheels -- the same construction language as the R1-R200
1336         -- growth chambers, never a plain box with a door bolted on. Colour
1337         -- (drawn from the catalogue entry, which already varies per kind)
1338         -- is what gives each equipment TYPE its own tailored look: a cold
1339         -- germination unit reads pale and clinical, a mutagenesis chamber
1340         -- reads hazard-yellow, and so on -- hospital-white insulated shells,
1341         -- industrial metal frame/trim, and lab glass fronts throughout.
1342         local frontOnX, direction
1343         if wallSide == "W" or wallSide == "E" then
1344             frontOnX = true
1345             direction = (wallSide == "W") and 1 or -1
1346         else
1347             frontOnX = false
1348             direction = (wallSide == "S") and -1 or 1
1349         end
1350
1351         local shellColour = lum > 0.55 and Color3.fromRGB(226, 231, 229)
1352             or lum > 0.3 and Color3.fromRGB(196, 202, 201)
1353             or Color3.fromRGB(54, 58, 62)
1354         local frameColour = Color3.fromRGB(56, 62, 66)
1355         local glassColour = Color3.fromRGB(
1356             math.min(255, entry.col.R * 255 * 0.5 + 150),
1357             math.min(255, entry.col.G * 255 * 0.5 + 155),
1358             math.min(255, entry.col.B * 255 * 0.5 + 155))
1359
1360         -- Backing volume only (matches the growth-chamber reference): a
1361         -- transparent placeholder that all the real, opaque panels below
1362         -- attach to, sized/positioned exactly as the caller expects so the
1363         -- runtime's LED/prompt/status binding keeps working unmodified.
1364         local body = newPart("Body", size, CFrame.new(pos), Color3.fromRGB(205, 216, 212), Enum.Material.Glass,
1365             ↪ parent)
1366         body.Transparency = 0.9
1367
1368         local innerWidth = frontOnX and size.Z or size.X

```

```

1368     local innerDepth = frontOnX and size.X or size.Z
1369     local shellT = math.max(math.min(size.X, size.Z) * 0.05, 0.12)
1370     local function interiorOffset(across, up, fromFront)
1371         if frontOnX then return CFrame.new(direction * (innerDepth / 2 - fromFront), up, across) end
1372         return CFrame.new(across, up, direction * (innerDepth / 2 - fromFront))
1373     end
1374
1375     -- Opaque insulated back/side/top/floor panels -- only the front
1376     -- stays glazed, matching a real controlled-environment cabinet.
1377     if frontOnX then
1378         newPart("InsulatedBack", Vector3.new(shellT, size.Y * 0.96, innerWidth),
1379             body.CFrame * CFrame.new(-direction * (innerDepth / 2 - shellT / 2), 0, 0), shellColour,
1380             Enum.Material.Metal, body)
1381         for _, sideSign in ipairs({-1, 1}) do
1382             newPart("InsulatedSide", Vector3.new(innerDepth, size.Y * 0.96, shellT),
1383                 body.CFrame * CFrame.new(0, 0, sideSign * (innerWidth / 2 - shellT / 2)), shellColour,
1384                 Enum.Material.Metal, body)
1385         end
1386     else
1387         newPart("InsulatedBack", Vector3.new(innerWidth, size.Y * 0.96, shellT),
1388             body.CFrame * CFrame.new(0, 0, -direction * (innerDepth / 2 - shellT / 2)), shellColour,
1389             Enum.Material.Metal, body)
1390         for _, sideSign in ipairs({-1, 1}) do
1391             newPart("InsulatedSide", Vector3.new(shellT, size.Y * 0.96, innerDepth),
1392                 body.CFrame * CFrame.new(sideSign * (innerWidth / 2 - shellT / 2), 0, 0), shellColour,
1393                 Enum.Material.Metal, body)
1394         end
1395     end
1396     local capSize = frontOnX and Vector3.new(innerDepth, shellT, innerWidth) or Vector3.new(innerWidth,
1397         ↪ shellT, innerDepth)
1398     newPart("InsulatedTop", capSize, body.CFrame * CFrame.new(0, size.Y / 2 - shellT / 2, 0), shellColour,
1399         ↪ Enum.Material.Metal, body)
1400     newPart("InsulatedFloor", capSize, body.CFrame * CFrame.new(0, -size.Y / 2 + shellT / 2, 0),
1401         ↪ shellColour, Enum.Material.Metal, body)
1402
1403     -- Glass front door, framed in metal on all four edges.
1404     local frontDist = innerDepth / 2 + 0.07
1405     local doorSize = frontOnX and Vector3.new(shellT * 1.2, size.Y * 0.82, innerWidth * 0.86)
1406         or Vector3.new(innerWidth * 0.86, size.Y * 0.82, shellT * 1.2)
1407     local doorCFrame = frontOnX and (body.CFrame * CFrame.new(direction * frontDist, 0, 0))
1408         or (body.CFrame * CFrame.new(0, 0, direction * frontDist))
1409     local door = newPart("Door", doorSize, doorCFrame, glassColour, Enum.Material.Glass, body)
1410     door.Transparency = entry.plantCabinet and 0.34 or 0.5
1411     door.CanCollide = false
1412     if frontOnX then
1413         local fx = direction * frontDist
1414         newPart("GlassFrameLeft", Vector3.new(0.18, size.Y * 0.94, 0.2), body.CFrame * CFrame.new(fx, 0,
1415             ↪ -innerWidth * 0.46), frameColour, Enum.Material.Metal, body).CanCollide = false
1416         newPart("GlassFrameRight", Vector3.new(0.18, size.Y * 0.94, 0.2), body.CFrame * CFrame.new(fx, 0,
1417             ↪ innerWidth * 0.46), frameColour, Enum.Material.Metal, body).CanCollide = false
1418         newPart("GlassFrameTop", Vector3.new(0.18, 0.22, innerWidth * 0.94), body.CFrame * CFrame.new(fx,
1419             ↪ size.Y * 0.46, 0), frameColour, Enum.Material.Metal, body).CanCollide = false
1420         newPart("GlassFrameBottom", Vector3.new(0.18, 0.22, innerWidth * 0.94), body.CFrame *
1421             ↪ CFrame.new(fx, -size.Y * 0.46, 0), frameColour, Enum.Material.Metal, body).CanCollide = false
1422     else
1423         local fz = direction * frontDist
1424         newPart("GlassFrameLeft", Vector3.new(0.2, size.Y * 0.94, 0.18), body.CFrame *
1425             ↪ CFrame.new(-innerWidth * 0.46, 0, fz), frameColour, Enum.Material.Metal, body).CanCollide =
1426             ↪ false
1427         newPart("GlassFrameRight", Vector3.new(0.2, size.Y * 0.94, 0.18), body.CFrame *
1428             ↪ CFrame.new(innerWidth * 0.46, 0, fz), frameColour, Enum.Material.Metal, body).CanCollide =
1429             ↪ false
1430         newPart("GlassFrameTop", Vector3.new(innerWidth * 0.94, 0.22, 0.18), body.CFrame * CFrame.new(0,
1431             ↪ size.Y * 0.46, fz), frameColour, Enum.Material.Metal, body).CanCollide = false
1432         newPart("GlassFrameBottom", Vector3.new(innerWidth * 0.94, 0.22, 0.18), body.CFrame * CFrame.new(0,
1433             ↪ -size.Y * 0.46, fz), frameColour, Enum.Material.Metal, body).CanCollide = false
1434     end
1435     local handleSize = frontOnX and Vector3.new(0.05, size.Y * 0.22, 0.05) or Vector3.new(size.Y * 0.22,
1436         ↪ 0.05, 0.05)
1437     local handle = newPart("Handle", handleSize,
1438         doorCFrame * (frontOnX and CFrame.new(direction * 0.05, 0, innerWidth * 0.3) or
1439             ↪ CFrame.new(innerWidth * 0.3, 0, direction * 0.05)),
1440         dark, Enum.Material.Metal, body)
1441     handle.CanCollide = false
1442
1443     -- Interior tier shelving + a light strip per tier, exactly like the

```

```

1425 -- growth-chamber reference, on every appliance -- not just the ones
1426 -- that hold plant samples -- so nothing reads as an empty shell.
1427 local tiers = entry.plantCabinet and 3 or 2
1428 for tier = 0, tiers - 1 do
1429     local shelfY = -size.Y * 0.32 + tier * (size.Y * 0.6 / math.max(1, tiers - 1))
1430     local shelfSize = frontOnX
1431         and Vector3.new(innerDepth * 0.72, 0.08, innerWidth * 0.84)
1432         or Vector3.new(innerWidth * 0.84, 0.08, innerDepth * 0.72)
1433     newPart("Shelf_" .. (tier + 1), shelfSize, body.CFrame * CFrame.new(0, shelfY - 0.24, 0),
1434         Color3.fromRGB(150, 156, 156), Enum.Material.Metal, body).CanCollide = false
1435     local lightSize = frontOnX and Vector3.new(0.1, 0.08, innerWidth * 0.72) or Vector3.new(innerWidth
1436         ↳ * 0.72, 0.08, 0.1)
1437     newPart("Light_" .. (tier + 1), lightSize,
1438         body.CFrame * interiorOffset(0, shelfY + size.Y * 0.1, innerDepth * 0.44),
1439         Color3.fromRGB(255, 248, 222), Enum.Material.Neon, body).CanCollide = false
1440 end
1441
1442 -- Rear ventilation grille + a raised controller housing with a
1443 -- status screen -- the runtime anchors the repair prompt/LED here
1444 -- when present, exactly as it does on the growth chambers.
1445 local ventSize = frontOnX and Vector3.new(0.1, size.Y * 0.22, innerWidth * 0.42) or
1446     ↳ Vector3.new(innerWidth * 0.42, size.Y * 0.22, 0.1)
1447 newPart("VentilationGrille", ventSize, body.CFrame * interiorOffset(0, size.Y * 0.18, innerDepth *
1448     ↳ 0.94),
1449     Color3.fromRGB(68, 74, 78), Enum.Material.DiamondPlate, body).CanCollide = false
1450 local controllerSize = frontOnX and Vector3.new(innerDepth * 0.86, 0.6, innerWidth * 0.92) or
1451     ↳ Vector3.new(innerWidth * 0.92, 0.6, innerDepth * 0.86)
1452 local controller = newPart("ClimateController", controllerSize, body.CFrame * CFrame.new(0, size.Y *
1453     ↳ 0.5 + 0.3, 0),
1454     Color3.fromRGB(222, 226, 224), Enum.Material.Metal, body)
1455 controller.CanCollide = false
1456 newPart("ControllerScreen",
1457     frontOnX and Vector3.new(0.1, 0.4, innerWidth * 0.56) or Vector3.new(innerWidth * 0.56, 0.4, 0.1),
1458     body.CFrame * interiorOffset(0, size.Y * 0.5 + 0.3, -0.16), accent, Enum.Material.Neon,
1459     ↳ body).CanCollide = false
1460
1461 -- Small caster wheels, like a real mobile lab cabinet.
1462 for _, across in ipairs({-innerWidth * 0.38, innerWidth * 0.38}) do
1463     for _, depthSign in ipairs({-1, 1}) do
1464         local wheelOffset
1465         if frontOnX then wheelOffset = CFrame.new(depthSign * innerDepth * 0.32, -size.Y / 2 - 0.2,
1466             ↳ across)
1467         else wheelOffset = CFrame.new(across, -size.Y / 2 - 0.2, depthSign * innerDepth * 0.32) end
1468         local wheel = newPart("CasterWheel", Vector3.new(0.34, 0.34, 0.2), body.CFrame * wheelOffset,
1469             Color3.fromRGB(28, 30, 32), Enum.Material.Rubber, body)
1470         wheel.Shape = Enum.PartType.Cylinder
1471         wheel.CanCollide = false
1472     end
1473 end
1474
1475 -- Power hookup: a wall junction box and conduit on the BACK face
1476 -- (opposite the door), running down to a floor-level cable stub
1477 -- toward the wall it's packed against, so every appliance reads as
1478 -- actually wired in rather than just placed in the room.
1479 local backDirX = frontOnX and -direction or 0
1480 local backDirZ = frontOnX and 0 or -direction
1481 local backX = pos.X + backDirX * size.X / 2
1482 local backZ = pos.Z + backDirZ * size.Z / 2
1483 local boxSize = Vector3.new(0.42, 0.42, 0.42)
1484 local junction = newPart("PowerJunctionBox", boxSize,
1485     CFrame.new(backX + backDirX * boxSize.X * 0.45, pos.Y + size.Y * 0.3, backZ + backDirZ * boxSize.Z
1486     ↳ * 0.45),
1487     Color3.fromRGB(56, 58, 62), Enum.Material.Metal, body)
1488 junction.CanCollide = false
1489 junction.CastShadow = false
1490 local conduitH = math.max(size.Y * 0.38, 0.5)
1491 local conduit = newPart("PowerConduit", Vector3.new(0.14, conduitH, 0.14),
1492     CFrame.new(backX + backDirX * 0.14, pos.Y + size.Y * 0.3 - conduitH / 2, backZ + backDirZ * 0.14),
1493     Color3.fromRGB(26, 28, 31), Enum.Material.Metal, body)
1494 conduit.CanCollide = false
1495 conduit.CastShadow = false
1496 local cableLen = 0.85
1497 local cableSize = frontOnX and Vector3.new(cableLen, 0.08, 0.08) or Vector3.new(0.08, 0.08, cableLen)
1498 local cable = newPart("PowerCable", cableSize,
1499     CFrame.new(backX + backDirX * (0.14 + cableLen / 2), pos.Y - size.Y / 2 + 0.12, backZ + backDirZ *
1500     ↳ (0.14 + cableLen / 2)),

```

```

1492         Color3.fromRGB(18, 18, 19), Enum.Material.SmoothPlastic, body)
1493     cable.CanCollide = false
1494     cable.CastShadow = false
1495     return body
1496
1497 elseif shape == "bench" then
1498     local topH = math.max(size.Y * 0.14, 0.15)
1499     local top = newPart("Top", Vector3.new(size.X, topH, size.Z),
1500         CFrame.new(pos.X, pos.Y + size.Y / 2 - topH / 2, pos.Z), entry.col, entry.mat, parent)
1501     local legT = math.min(size.X, size.Z) * 0.07
1502     local legH = math.max(size.Y - topH, 0.2)
1503     for _, s in ipairs({{1, 1}, {1, -1}, {-1, 1}, {-1, -1}}) do
1504         local leg = newPart("Leg", Vector3.new(legT, legH, legT),
1505             CFrame.new(pos.X + s[1] * (size.X / 2 - legT), pos.Y - size.Y / 2 + legH / 2,
1506                 pos.Z + s[2] * (size.Z / 2 - legT)),
1507             dark, Enum.Material.Metal, top)
1508         leg.CanCollide = false
1509     end
1510     return top
1511
1512 elseif shape == "instrument" then
1513     local bodyH = size.Y * 0.55
1514     local body = newPart("Body", Vector3.new(size.X, bodyH, size.Z),
1515         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH / 2, pos.Z), entry.col, entry.mat, parent)
1516     local panelH = math.max(size.Y - bodyH, 0.1)
1517     local panel = newPart("Panel", Vector3.new(size.X * 0.72, panelH, size.Z * 0.55),
1518         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH + panelH / 2, pos.Z - size.Z * 0.12),
1519         panelColour, Enum.Material.SmoothPlastic, body)
1520     panel.CanCollide = false
1521     -- a small glowing indicator strip so even a tiny instrument reads
1522     -- as "a device" at a glance, not a plain block
1523     local led = newPart("IndicatorStrip", Vector3.new(size.X * 0.5, math.max(panelH * 0.18, 0.05), size.Z *
1524         ⇨ 0.05),
1525         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH + panelH * 0.78, pos.Z - size.Z * 0.12 + size.Z *
1526         ⇨ 0.28),
1527         accent, Enum.Material.Neon, body)
1528     led.CanCollide = false
1529     led.CastShadow = false
1530     return body
1531
1532 elseif shape == "rack" then
1533     local backT = math.max(size.Z * 0.14, 0.1)
1534     local back = newPart("Back", Vector3.new(size.X, size.Y, backT),
1535         CFrame.new(pos.X, pos.Y, pos.Z - size.Z / 2 + backT / 2), entry.col, entry.mat, parent)
1536     local shelfH = math.max(size.Y * 0.06, 0.08)
1537     for i = 1, 3 do
1538         local sy = pos.Y - size.Y / 2 + size.Y * (i / 4)
1539         local shelf = newPart("Shelf" .. i, Vector3.new(size.X, shelfH, size.Z),
1540             CFrame.new(pos.X, sy, pos.Z), entry.col, entry.mat, back)
1541         shelf.CanCollide = false
1542     end
1543     return back
1544
1545 elseif shape == "planter" then
1546     -- A potted plant / flower box: a pot or trough with a soil cap and
1547     -- several stem+leaf clusters -- pure decoration, no door/handle.
1548     local potH = size.Y * 0.42
1549     local pot = newPart("Pot", Vector3.new(size.X, potH, size.Z),
1550         CFrame.new(pos.X, pos.Y - size.Y / 2 + potH / 2, pos.Z), entry.col, entry.mat, parent)
1551     local soil = newPart("Soil", Vector3.new(size.X * 0.86, potH * 0.22, size.Z * 0.86),
1552         CFrame.new(pos.X, pos.Y - size.Y / 2 + potH * 0.94, pos.Z), Color3.fromRGB(58, 42, 30),
1553         ⇨ Enum.Material.Ground, pot)
1554     soil.CanCollide = false
1555     local clusters = entry.flowering and 4 or 3
1556     for i = 1, clusters do
1557         local ang = (i / clusters) * math.pi * 2 + (pos.X % 1)
1558         local rx = math.cos(ang) * size.X * 0.22
1559         local rz = math.sin(ang) * size.Z * 0.22
1560         local stemH = size.Y * (0.4 + (i % 3) * 0.06)
1561         local stem = newPart("Stem_" .. i, Vector3.new(size.X * 0.05, stemH, size.Z * 0.05),
1562             CFrame.new(pos.X + rx, pos.Y - size.Y / 2 + potH + stemH / 2, pos.Z + rz),
1563             Color3.fromRGB(58, 130, 66), Enum.Material.Grass, pot)
1564         stem.CanCollide = false
1565         local leafColour = entry.flowering
1566         and ({Color3.fromRGB(230, 90, 130), Color3.fromRGB(240, 200, 70), Color3.fromRGB(210, 100,
1567             ⇨ 210)})[(i - 1) % 3 + 1]

```

```

1564         or Color3.fromRGB(72, 168, 84)
1565         local leaf = newPart("Leaf_" .. i, Vector3.new(size.X * 0.3, size.Y * 0.16, size.Z * 0.3),
1566             CFrame.new(pos.X + rx, pos.Y - size.Y / 2 + potH + stemH, pos.Z + rz),
1567             leafColour, entry.flowering and Enum.Material.Neon or Enum.Material.Grass, pot)
1568         leaf.CanCollide = false
1569         leaf.Shape = Enum.PartType.Ball
1570     end
1571     return pot
1572
1573 elseif shape == "drawers" then
1574     -- Lab storage drawers / filing cabinet: a metal carcass with 3-4
1575     -- stacked drawer fronts and handles, so it reads as furniture rather
1576     -- than another appliance.
1577     local body = newPart("Body", size, CFrame.new(pos), entry.col, entry.mat, parent)
1578     local tiers = 3 + (math.floor(size.Y * 10) % 2)
1579     local frontOnX = wallSide == "W" or wallSide == "E"
1580     local frontDir = (wallSide == "W" or wallSide == "N") and 1 or -1
1581     for tier = 1, tiers do
1582         local fy = -size.Y / 2 + (tier - 0.5) * (size.Y / tiers)
1583         local frontSize = frontOnX
1584             and Vector3.new(size.X * 0.06, size.Y / tiers * 0.82, size.Z * 0.86)
1585             or Vector3.new(size.X * 0.86, size.Y / tiers * 0.82, size.Z * 0.06)
1586         local frontCFrame = frontOnX
1587             and CFrame.new(pos.X + frontDir * size.X / 2 * 0.97, pos.Y + fy, pos.Z)
1588             or CFrame.new(pos.X, pos.Y + fy, pos.Z + frontDir * size.Z / 2 * 0.97)
1589         local front = newPart("Drawer_" .. tier, frontSize, frontCFrame, accent, Enum.Material.Metal, body)
1590         front.CanCollide = false
1591         local handleSize = frontOnX and Vector3.new(0.06, 0.06, size.Z * 0.4) or Vector3.new(size.X * 0.4,
1592             ↪ 0.06, 0.06)
1593         local handle = newPart("DrawerHandle_" .. tier, handleSize,
1594             frontCFrame * CFrame.new(frontOnX and 0.05 or 0, 0, 0) * (frontOnX and CFrame.new(0, 0, 0) or
1595             ↪ CFrame.new(0, 0, 0.05)),
1596             dark, Enum.Material.Metal, body)
1597         handle.CanCollide = false
1598     end
1599     return body
1600
1601 else -- "box": last-resort fallback, not used by anything in the
1602     -- catalogue any more, but kept so a future entry without a
1603     -- matching shape still gets a plate instead of a bare cube.
1604     local body = newPart("Body", size, CFrame.new(pos), entry.col, entry.mat, parent)
1605     local capH = math.min(size.Y * 0.08, 0.15)
1606     if capH > 0.03 then
1607         local cap = newPart("Cap", Vector3.new(size.X * 0.94, capH, size.Z * 0.94),
1608             CFrame.new(pos.X, pos.Y + size.Y / 2 + capH / 2, pos.Z), accent, entry.mat, body)
1609         cap.CanCollide = false
1610     end
1611     return body
1612 end
1613
1614 local function populateEquipmentCabinet(body, entry, size, wallSide, sampleCount, capacity)
1615     if not entry.plantCabinet or entry.kind == "Plant Growth Chamber" then return end
1616     local frontOnX = wallSide == "W" or wallSide == "E"
1617     local width = frontOnX and size.Z or size.X
1618     local depth = frontOnX and size.X or size.Z
1619     local shelfCount = capacity >= 9 and 4 or (capacity >= 6 and 3 or 2)
1620     for tier = 1, shelfCount do
1621         local y = -size.Y * 0.34 + (tier - 1) * (size.Y * 0.68 / math.max(1, shelfCount - 1))
1622         local shelfSize = frontOnX and Vector3.new(depth * 0.72, 0.1, width * 0.84)
1623             or Vector3.new(width * 0.84, 0.1, depth * 0.72)
1624         local shelf = newPart("EquipmentShelf_" .. tier, shelfSize, body.CFrame * CFrame.new(0, y, 0),
1625             Color3.fromRGB(138, 145, 147), Enum.Material.Metal, body)
1626         shelf.CanCollide = false
1627     end
1628     for slot = 1, capacity do
1629         local tier = math.floor((slot - 1) / 3)
1630         local column = (slot - 1) % 3
1631         local across = (column - 1) * width * 0.25
1632         local up = -size.Y * 0.34 + tier * (size.Y * 0.68 / math.max(1, shelfCount - 1))
1633         local offset = frontOnX and Vector3.new(0, up + 0.25, across) or Vector3.new(across, up + 0.25, 0)
1634         local base = body.CFrame * CFrame.new(offset)
1635         local pot = newPart("EquipmentPlant_" .. slot .. "_Tray", Vector3.new(0.58, 0.28, 0.52), base,
1636             Color3.fromRGB(126, 77, 45), Enum.Material.SmoothPlastic, body)
1637         local stem = newPart("EquipmentPlant_" .. slot .. "_Stem", Vector3.new(0.09, 0.48, 0.09),
1638             base * CFrame.new(0, 0.35, 0), Color3.fromRGB(58, 158, 70), Enum.Material.SmoothPlastic, body)

```

```

1638     local leaf = newPart("EquipmentPlant_" .. slot .. "_Leaf", Vector3.new(0.42, 0.1, 0.25),
1639       base * CFrame.new(0.17, 0.52, 0), Color3.fromRGB(72, 184, 82), Enum.Material.SmoothPlastic, body)
1640   for _, plantPart in ipairs({pot, stem, leaf}) do
1641     plantPart.CanCollide = false
1642     plantPart.CanTouch = false
1643     plantPart:SetAttribute("SampleSlot", slot)
1644     plantPart:SetAttribute("StoredTransparency", 0)
1645     if slot > sampleCount then plantPart.Transparency = 1 end
1646   end
1647 end
1648 end
1649
1650 -----
1651 -- ROOM FIT-OUT
1652 -----
1653 local EQUIP_TAG = "LabEquipment"
1654 local placedCount = 0
1655 local addGrowthCodeLabel
1656
1657 local function fitOutRoom(parent, room, levelY, levelName, rng, growthCodes, plantStationKind)
1658   local sides = {"W", "E", "N", "S"}
1659   local lenOf = {
1660     W = room.z2 - room.z1, E = room.z2 - room.z1,
1661     N = room.x2 - room.x1, S = room.x2 - room.x1,
1662   }
1663   local spans = {}
1664   for _, s in ipairs(sides) do
1665     spans[s] = freeSpans(lenOf[s], room.open[s])
1666   end
1667
1668   local benchRuns = {}
1669   local index = 0
1670   local function wallFaceInset(side)
1671     local exterior = (side == "W" and room.x1 == 0)
1672       or (side == "E" and room.x2 == W)
1673       or (side == "N" and room.z1 == 0)
1674       or (side == "S" and room.z2 == L)
1675     return (exterior and EXT_T or INT_T / 2) + WALL_GAP
1676   end
1677
1678   local function spawn(entry, side, at, lift, liftInset)
1679     index = index + 1
1680     placedCount = placedCount + 1
1681     local inset = liftInset or (wallFaceInset(side) + P(entry.d) / 2)
1682     local px, pz, size
1683     if side == "W" then
1684       px, pz = room.x1 + inset, room.z1 + at
1685       size = Vector3.new(E(entry.d), EH(entry.h), E(entry.w))
1686     elseif side == "E" then
1687       px, pz = room.x2 - inset, room.z1 + at
1688       size = Vector3.new(E(entry.d), EH(entry.h), E(entry.w))
1689     elseif side == "N" then
1690       px, pz = room.x1 + at, room.z1 + inset
1691       size = Vector3.new(E(entry.w), EH(entry.h), E(entry.d))
1692     else
1693       px, pz = room.x1 + at, room.z2 - inset
1694       size = Vector3.new(E(entry.w), EH(entry.h), E(entry.d))
1695     end
1696     local baseY = Y(levelY) + (lift and EH(COUNTER_H) or 0)
1697     local pos = Vector3.new(X(px), baseY + size.Y / 2, X(pz))
1698     local p = buildKind(parent, entry, pos, size, side)
1699     p.Name = entry.kind:gsub("%s", "") .. "_" .. room.number .. "_" .. index
1700     p:SetAttribute("Kind", entry.kind)
1701     p:SetAttribute("Room", room.name)
1702     p:SetAttribute("Level", levelName)
1703     p:SetAttribute("WallSide", side)
1704     -- Every placed unit -- including the densely-packed "run" duplicates
1705     -- added to fill out walls -- is tagged as real, functional equipment
1706     -- so it gets the same status LED, timer/nameplate, hazard behaviour,
1707     -- and catastrophic-failure system as everything else. (Previously
1708     -- "run"-class units were left untagged, which is why they looked
1709     -- like bare, undetailed boxes -- none of the runtime binding that
1710     -- adds those visuals ever ran on them.) Pure decoration (potted
1711     -- plants, drawers, ...) is the one exception -- it's not equipment,
1712     -- so it gets its own lightweight tag instead of the full status/
1713     -- hazard/failure system.

```

```

1714     if entry.class == "decor" then
1715         CollectionService:AddTag(p, "LabDecor")
1716     else
1717         CollectionService:AddTag(p, EQUIP_TAG)
1718     end
1719     if entry.plantCabinet then
1720         local capacity = entry.capacity or 6
1721         local sampleCount = math.clamp(1 + ((room.number * 3 + index * 5) % capacity), 1, capacity)
1722         local sampleIds = {}
1723         for sampleIndex = 1, sampleCount do
1724             table.insert(sampleIds, string.format("%s-P%02d", entry.kind:gsub("%s", ""):sub(1, 6):upper(),
1725                 ↪ sampleIndex))
1726         end
1727         p:SetAttribute("Capacity", capacity)
1728         p:SetAttribute("SampleCount", sampleCount)
1729         p:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
1730         p:SetAttribute("Occupied", sampleCount > 0)
1731         populateEquipmentCabinet(p, entry, size, side, sampleCount, capacity)
1732     end
1733     if entry.auto then
1734         p:SetAttribute("Auto", entry.auto)
1735         CollectionService:AddTag(p, "Automatable")
1736     end
1737     -- Equipment names and instructions are presented on the two large
1738     -- floor-level panels created by the runtime; no overhead label is used.
1739     return p
1740 end
1741
1742 -- 1. fitted runs on the longest available walls
1743 local order = {}
1744 for _, s in ipairs(sides) do table.insert(order, s) end
1745 table.sort(order, function(a, b) return spansTotal(spans[a]) > spansTotal(spans[b]) end)
1746
1747 -- Room status monitor: reserved FIRST, on the roomiest wall, so it never
1748 -- gets squeezed into a leftover corner sliver by equipment placed later.
1749 -- Large panel, mounted at eye level so it reads from across the room.
1750 do
1751     local monitorWidth = 220
1752     local mSide, mAt
1753     for _, side in ipairs(order) do
1754         local total = spansTotal(spans[side])
1755         local at = (total >= P(monitorWidth)) and takeSpan(spans[side], P(monitorWidth)) or nil
1756         if at then mSide, mAt = side, at; break end
1757     end
1758     if not mSide then
1759         -- Even the roomiest wall is tight: shrink to fit rather than
1760         -- falling back to a corner placement.
1761         for _, side in ipairs(order) do
1762             local total = spansTotal(spans[side])
1763             if total > 40 then
1764                 local fitWidth = math.min(monitorWidth, (total - 4) * SCALE)
1765                 local at = takeSpan(spans[side], P(fitWidth))
1766                 if at then monitorWidth, mSide, mAt = fitWidth, side, at; break end
1767             end
1768         end
1769     end
1770     if mSide then
1771         local inset = wallFaceInset(mSide)
1772         local px, pz
1773         if mSide == "W" then px, pz = room.x1 + inset, room.z1 + mAt
1774         elseif mSide == "E" then px, pz = room.x2 - inset, room.z1 + mAt
1775         elseif mSide == "N" then px, pz = room.x1 + mAt, room.z1 + inset
1776         else px, pz = room.x1 + mAt, room.z2 - inset end
1777         local mOnX = (mSide == "W" or mSide == "E")
1778         local panelSize = mOnX and Vector3.new(0.2, EH(135), E(monitorWidth))
1779             or Vector3.new(E(monitorWidth), EH(135), 0.2)
1780         local baseY = Y(levelY) + EH(175)
1781         local mPos = Vector3.new(X(px), baseY + panelSize.Y / 2, X(pz))
1782         local panel = newPart("RoomMonitor_" .. room.number, panelSize, CFrame.new(mPos),
1783             Color3.fromRGB(22, 26, 30), Enum.Material.Metal, parent)
1784         panel.CanCollide = false
1785         panel:SetAttribute("Room", room.name)
1786         panel:SetAttribute("Level", levelName)
1787         CollectionService:AddTag(panel, "RoomMonitor")
1788         -- Slim backlit frame so the panel reads as a mounted screen, not a wall patch.

```

```

1789     local frameDepth = mOnX and Vector3.new(0.05, panelSize.Y + 0.5, panelSize.Z + 0.5)
1790     or Vector3.new(panelSize.X + 0.5, panelSize.Y + 0.5, 0.05)
1791     local frameOffset = (mSide == "W") and CFrame.new(-0.14, 0, 0)
1792     or (mSide == "E") and CFrame.new(0.14, 0, 0)
1793     or (mSide == "N") and CFrame.new(0, 0, -0.14)
1794     or CFrame.new(0, 0, 0.14)
1795     local frame = newPart("RoomMonitor_" .. room.number .. "_Frame", frameDepth,
1796     panel.CFrame * frameOffset, Color3.fromRGB(235, 200, 60), Enum.Material.Metal, parent)
1797     frame.CanCollide = false
1798     local face = (mSide == "W") and Enum.NormalId.Right
1799     or (mSide == "E") and Enum.NormalId.Left
1800     or (mSide == "N") and Enum.NormalId.Back
1801     or Enum.NormalId.Front
1802     local surface = Instance.new("SurfaceGui")
1803     surface.Name = "RoomMonitorSurface"
1804     surface.Face = face
1805     surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
1806     surface.PixelsPerStud = 60
1807     surface.AlwaysOnTop = false
1808     surface.Parent = panel
1809     local header = Instance.new("TextLabel")
1810     header.Name = "Header"
1811     header.Size = UDim2.new(1, 0, 0.22, 0)
1812     header.Position = UDim2.new(0, 0, 0, 0)
1813     header.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
1814     header.BorderSizePixel = 0
1815     header.TextColor3 = Color3.fromRGB(20, 20, 20)
1816     header.TextScaled = true
1817     header.Font = Enum.Font.SourceSansBold
1818     header.Text = "ROOM MONITOR"
1819     header.Parent = surface
1820     local label = Instance.new("TextLabel")
1821     label.Name = "Status"
1822     label.Size = UDim2.new(1, 0, 0.78, 0)
1823     label.Position = UDim2.new(0, 0, 0.22, 0)
1824     label.BackgroundColor3 = Color3.fromRGB(14, 18, 20)
1825     label.BackgroundTransparency = 0.05
1826     label.BorderSizePixel = 0
1827     label.TextColor3 = Color3.fromRGB(120, 235, 170)
1828     label.TextScaled = true
1829     label.TextWrapped = true
1830     label.Font = Enum.Font.Code
1831     label.Text = room.name .. "\nSTATUS LOADING..."
1832     label.Parent = surface
1833 end
1834 end
1835
1836 -- Emergency Kit: a LIMITED, single-use supply (about 1 room in 3, not
1837 -- every room), avatar-sized, reserved right after the monitor so it
1838 -- always gets real wall space too. Using one destroys it permanently --
1839 -- health only comes back by finding another kit. The interaction and
1840 -- healing behaviour is wired at runtime (bindEmergencyKit); this just
1841 -- builds the physical cabinet.
1842 if room.number % 3 == 0 then
1843     local kitWidth, kitDepth, kitHeight = 62, 30, 154 -- cm, ~ a standing avatar's footprint/height
1844     local kSide, kAt
1845     for _, side in ipairs(order) do
1846         local at = takeSpan(spans[side], P(kitWidth))
1847         if at then kSide, kAt = side, at; break end
1848     end
1849     if kSide then
1850         local inset = wallFaceInset(kSide) + P(kitDepth) / 2
1851         local px, pz
1852         if kSide == "W" then px, pz = room.x1 + inset, room.z1 + kAt
1853         elseif kSide == "E" then px, pz = room.x2 - inset, room.z1 + kAt
1854         elseif kSide == "N" then px, pz = room.x1 + kAt, room.z1 + inset
1855         else px, pz = room.x1 + kAt, room.z2 - inset end
1856         local kOnX = (kSide == "W" or kSide == "E")
1857         local cabinetSize = kOnX and Vector3.new(E(kitDepth), EH(kitHeight), E(kitWidth))
1858         or Vector3.new(E(kitWidth), EH(kitHeight), E(kitDepth))
1859         local baseY = Y(levelY)
1860         local cabinet = newPart("EmergencyKit_" .. room.number, cabinetSize,
1861         CFrame.new(X(px), baseY + cabinetSize.Y / 2, X(pz)), Color3.fromRGB(225, 40, 35),
1862         Enum.Material.SmoothPlastic, parent)
1863         cabinet.CanCollide = true
1864         cabinet:SetAttribute("Room", room.name)

```



```

1864     cabinet:SetAttribute("Level", levelName)
1865     cabinet:SetAttribute("WallSide", kSide)
1866     CollectionService.AddTag(cabinet, "EmergencyKit")
1867
1868     -- White cross + label on the door face so it reads as a first-aid
1869     -- cabinet, not a generic red box.
1870     local face = (kSide == "W") and Enum.NormalId.Right
1871                 or (kSide == "E") and Enum.NormalId.Left
1872                 or (kSide == "N") and Enum.NormalId.Back
1873                 or Enum.NormalId.Front
1874     local surface = Instance.new("SurfaceGui")
1875     surface.Name = "EmergencyKitFace"
1876     surface.Face = face
1877     surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
1878     surface.PixelsPerStud = 60
1879     surface.AlwaysOnTop = false
1880     surface.Parent = cabinet
1881     local crossV = Instance.new("Frame")
1882     crossV.Size = UDim2.new(0.22, 0, 0.55, 0)
1883     crossV.Position = UDim2.new(0.39, 0, 0.12, 0)
1884     crossV.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1885     crossV.BorderSizePixel = 0
1886     crossV.Parent = surface
1887     local crossH = Instance.new("Frame")
1888     crossH.Size = UDim2.new(0.55, 0, 0.22, 0)
1889     crossH.Position = UDim2.new(0.12, 0, 0.28, 0)
1890     crossH.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1891     crossH.BorderSizePixel = 0
1892     crossH.Parent = surface
1893     local kitLabel = Instance.new("TextLabel")
1894     kitLabel.Size = UDim2.new(1, 0, 0.22, 0)
1895     kitLabel.Position = UDim2.new(0, 0, 0.72, 0)
1896     kitLabel.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1897     kitLabel.BackgroundTransparency = 0.05
1898     kitLabel.BorderSizePixel = 0
1899     kitLabel.Text = "EMERGENCY KIT"
1900     kitLabel.TextColor3 = Color3.fromRGB(200, 30, 25)
1901     kitLabel.TextScaled = true
1902     kitLabel.Font = Enum.Font.SourceSansBold
1903     kitLabel.Parent = surface
1904
1905     local prompt = Instance.new("ProximityPrompt")
1906     prompt.Name = "EmergencyKitPrompt"
1907     prompt.ActionText = "Use Emergency Kit"
1908     prompt.ObjectText = "Restore health"
1909     prompt.KeyboardKeyCode = Enum.KeyCode.E
1910     prompt.MaxActivationDistance = 9
1911     prompt.RequiresLineOfSight = false
1912     prompt.Parent = cabinet
1913 end
1914 end
1915
1916 -- Reserve wall space for this room's share of R1-R200 before normal
1917 -- furniture is packed. A shuffled wall order keeps the chambers spread
1918 -- around the building, but every one remains fridge-like and wall-backed.
1919 for codeIndex, code in ipairs(growthCodes or {}) do
1920     local sizeVariants = {
1921         {w = 100, d = 90, h = 220, capacity = 6},
1922         {w = 115, d = 95, h = 235, capacity = 9},
1923         {w = 130, d = 100, h = 250, capacity = 12},
1924         {w = 145, d = 105, h = 265, capacity = 16},
1925     }
1926     local variant = sizeVariants[((tonumber(code:match("%d+")) or codeIndex) - 1) % #sizeVariants + 1]
1927     local entry = {
1928         kind = "Plant Growth Chamber",
1929         label = code,
1930         w = variant.w, d = variant.d, h = variant.h,
1931         col = Color3.fromRGB(215, 220, 218),
1932         mat = Enum.Material.Metal,
1933         shape = "appliance",
1934     }
1935     local chamber
1936     local firstSide = rng.int(1, #order)
1937     -- Placement ladder. The preferred size is tried first on every wall;
1938     -- if the room's walls are already busy (the monitor, the emergency
1939     -- kit and the guaranteed plant station are all reserved ahead of the

```

```

1940 -- chambers) it steps down through the smaller cabinets, and only as a
1941 -- last resort tightens the neighbour gap. Without this ladder a code
1942 -- that found no span was silently dropped, which is exactly why the
1943 -- narrow east-side rooms kept landing 1-2 chambers under their share
1944 -- and the building missed its 50-per-floor target.
1945 local fallbackOrder = {variant}
1946 local smaller = {}
1947 for _, alt in ipairs(sizeVariants) do
1948   if alt ~= variant then table.insert(smaller, alt) end
1949 end
1950 table.sort(smaller, function(a, b) return a.w < b.w end)
1951 for _, alt in ipairs(smaller) do table.insert(fallbackOrder, alt) end
1952 -- Bottom of the ladder: a compact under-counter cabinet that exists
1953 -- only to rescue the very last code in a room whose walls are down to
1954 -- slivers. It is never picked as a room's preferred size, so it does
1955 -- not change how the building normally looks.
1956 table.insert(fallbackOrder, {w = 85, d = 78, h = 196, capacity = 5})
1957
1958 for _, gap in ipairs({ALONG_GAP, 10}) do
1959   for _, candidate in ipairs(fallbackOrder) do
1960     for try = 0, #order - 1 do
1961       local side = order[((firstSide + try - 1) % #order) + 1]
1962       local at = takeSpanGap(spans[side], P(candidate.w), gap)
1963       if at then
1964         variant = candidate
1965         entry.w, entry.d, entry.h = candidate.w, candidate.d, candidate.h
1966         chamber = spawn(entry, side, at, false)
1967         break
1968       end
1969     end
1970     if chamber then break end
1971   end
1972   if chamber then break end
1973 end
1974 if chamber then
1975   local codeNumber = tonumber(code:match("%d+")) or codeIndex
1976   local capacity = variant.capacity
1977   local sampleCount = math.clamp(1 + ((codeNumber * 7) % capacity), 1, capacity)
1978   local sampleIds = {}
1979   for sampleIndex = 1, sampleCount do
1980     table.insert(sampleIds, string.format("%s-S%02d", code, sampleIndex))
1981   end
1982   local initiallyFailed = ((codeNumber - 1) % 10) < 6
1983   local floorLabel = levelName == "Basement" and "BASEMENT"
1984   or ("FLOOR " .. (levelName:match("%d+") or levelName))
1985   local failureReasons = {
1986     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
1987     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
1988   }
1989   local temperature = initiallyFailed and (31 + codeNumber % 8) or (22 + codeNumber % 4)
1990   local humidity = initiallyFailed and (38 + codeNumber % 19) or (67 + codeNumber % 14)
1991   local ventilation = initiallyFailed and (12 + codeNumber % 24) or (62 + codeNumber % 29)
1992   chamber.Name = "GrowthChamber_" .. code
1993   chamber:SetAttribute("GrowthCode", code)
1994   chamber:SetAttribute("FloorLabel", floorLabel)
1995   chamber:SetAttribute("Capacity", capacity)
1996   chamber:SetAttribute("SampleCount", sampleCount)
1997   chamber:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
1998   chamber:SetAttribute("Occupied", sampleCount > 0)
1999   chamber:SetAttribute("PlantId", sampleIds[1] or code)
2000   chamber:SetAttribute("GrowthStage", 0)
2001   chamber:SetAttribute("Treatments", "")
2002   chamber:SetAttribute("Genome", "Wild Type")
2003   chamber:SetAttribute("PlantLife", 100)
2004   chamber:SetAttribute("RecoveryUntil", 0)
2005   chamber:SetAttribute("Moisture", humidity)
2006   chamber:SetAttribute("Temperature", temperature)
2007   chamber:SetAttribute("Humidity", humidity)
2008   chamber:SetAttribute("Ventilation", ventilation)
2009   chamber:SetAttribute("OperationStarted", os.time() - (codeNumber * 173) % 10800)
2010   chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M", os.time() - (codeNumber % 6 + 1)
2011     ↪ * 86400))
2012   chamber:SetAttribute("FunctionalState", initiallyFailed and "FAILED" or "WORKING")
2013   chamber:SetAttribute("RepairKey", "R")
2014   chamber:SetAttribute("RepairInstruction", initiallyFailed
2015     and ("Go to " .. floorLabel .. " " • CHAMBER " .. code .. " " • HOLD R TO REPAIR"))

```

```

2015         or "Unit working • E TAKE / DROP • R REPAIR")
2016 chamber:SetAttribute("FailureReason", initiallyFailed and failureReasons[((codeNumber - 1) %
↳ #failureReasons) + 1] or "")
2017 chamber:SetAttribute("FailureSince", initiallyFailed and os.time() or 0)
2018 chamber:SetAttribute("EvacuationDestination", "")
2019 CollectionService:AddTag(chamber, "PlantGrowthChamber")
2020 -- Reference-inspired controlled-environment cabinet: insulated
2021 -- body, glass door, bright tier lighting, tray shelves, rear vents,
2022 -- header controller and caster wheels. The translucent body is only
2023 -- a backing volume; the metal frame gives it a refrigerator profile.
2024 chamber.Material = Enum.Material.Glass
2025 chamber.Color = Color3.fromRGB(205, 216, 212)
2026 chamber.Transparency = 0.88
2027 local wallSide = chamber:GetAttribute("WallSide")
2028 local frontOnX = wallSide == "W" or wallSide == "E"
2029 local direction = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
2030 local frameColour = Color3.fromRGB(62, 69, 72)
2031 local innerWidth = frontOnX and chamber.Size.Z or chamber.Size.X
2032 local innerDepth = frontOnX and chamber.Size.X or chamber.Size.Z
2033 local shellColour = codeNumber % 5 == 0 and Color3.fromRGB(43, 48, 50)
2034         or (codeNumber % 3 == 0 and Color3.fromRGB(198, 203, 202) or Color3.fromRGB(232, 234, 231))
2035 local function interiorOffset(across, up, fromFront)
2036     if frontOnX then return CFrame.new(direction * (innerDepth / 2 - fromFront), up, across) end
2037     return CFrame.new(across, up, direction * (innerDepth / 2 - fromFront))
2038 end
2039 local frameParts = {}
2040 local shellThickness = 0.18
2041 -- Opaque insulated rear, side, top and floor panels leave only
2042 -- the front door transparent, matching real growth cabinets.
2043 if frontOnX then
2044     table.insert(frameParts, newPart("InsulatedBack",
2045         Vector3.new(shellThickness, chamber.Size.Y * 0.96, innerWidth),
2046         chamber.CFrame * CFrame.new(-direction * (innerDepth / 2 - shellThickness / 2), 0, 0),
2047         shellColour, Enum.Material.Metal, chamber))
2048     for _, sideSign in ipairs({-1, 1}) do
2049         table.insert(frameParts, newPart("InsulatedSide",
2050             Vector3.new(innerDepth, chamber.Size.Y * 0.96, shellThickness),
2051             chamber.CFrame * CFrame.new(0, 0, sideSign * (innerWidth / 2 - shellThickness / 2)),
2052             shellColour, Enum.Material.Metal, chamber))
2053     end
2054     table.insert(frameParts, newPart("InsulatedTop", Vector3.new(innerDepth, shellThickness,
2055         ↳ innerWidth),
2056         chamber.CFrame * CFrame.new(0, chamber.Size.Y / 2 - shellThickness / 2, 0),
2057         shellColour, Enum.Material.Metal, chamber))
2058     table.insert(frameParts, newPart("InsulatedFloor", Vector3.new(innerDepth, shellThickness,
2059         ↳ innerWidth),
2060         chamber.CFrame * CFrame.new(0, -chamber.Size.Y / 2 + shellThickness / 2, 0),
2061         shellColour, Enum.Material.Metal, chamber))
2062 else
2063     table.insert(frameParts, newPart("InsulatedBack",
2064         Vector3.new(innerWidth, chamber.Size.Y * 0.96, shellThickness),
2065         chamber.CFrame * CFrame.new(0, 0, -direction * (innerDepth / 2 - shellThickness / 2)),
2066         shellColour, Enum.Material.Metal, chamber))
2067     for _, sideSign in ipairs({-1, 1}) do
2068         table.insert(frameParts, newPart("InsulatedSide",
2069             Vector3.new(shellThickness, chamber.Size.Y * 0.96, innerDepth),
2070             chamber.CFrame * CFrame.new(sideSign * (innerWidth / 2 - shellThickness / 2), 0, 0),
2071             shellColour, Enum.Material.Metal, chamber))
2072     end
2073     table.insert(frameParts, newPart("InsulatedTop", Vector3.new(innerWidth, shellThickness,
2074         ↳ innerDepth),
2075         chamber.CFrame * CFrame.new(0, chamber.Size.Y / 2 - shellThickness / 2, 0),
2076         shellColour, Enum.Material.Metal, chamber))
2077     table.insert(frameParts, newPart("InsulatedFloor", Vector3.new(innerWidth, shellThickness,
2078         ↳ innerDepth),
2079         chamber.CFrame * CFrame.new(0, -chamber.Size.Y / 2 + shellThickness / 2, 0),
2080         shellColour, Enum.Material.Metal, chamber))
2081 end
2082 local frontDistance = innerDepth / 2 + 0.08
2083 if frontOnX then
2084     local frontX = direction * frontDistance
2085     table.insert(frameParts, newPart("GlassFrameLeft", Vector3.new(0.2, chamber.Size.Y * 0.94,
2086         ↳ 0.22), chamber.CFrame * CFrame.new(frontX, 0, -chamber.Size.Z * 0.46), frameColour,
2087         ↳ Enum.Material.Metal, chamber))

```

```

2082     table.insert(frameParts, newPart("GlassFrameRight", Vector3.new(0.2, chamber.Size.Y * 0.94,
    ↳ 0.22), chamber.CFrame * CFrame.new(frontX, 0, chamber.Size.Z * 0.46), frameColour,
    ↳ Enum.Material.Metal, chamber))
2083     table.insert(frameParts, newPart("GlassFrameTop", Vector3.new(0.2, 0.24, chamber.Size.Z *
    ↳ 0.94), chamber.CFrame * CFrame.new(frontX, chamber.Size.Y * 0.47, 0), frameColour,
    ↳ Enum.Material.Metal, chamber))
2084     table.insert(frameParts, newPart("GlassFrameBottom", Vector3.new(0.2, 0.24, chamber.Size.Z *
    ↳ 0.94), chamber.CFrame * CFrame.new(frontX, -chamber.Size.Y * 0.47, 0), frameColour,
    ↳ Enum.Material.Metal, chamber))
2085 else
2086     local frontZ = direction * frontDistance
2087     table.insert(frameParts, newPart("GlassFrameLeft", Vector3.new(0.22, chamber.Size.Y * 0.94,
    ↳ 0.2), chamber.CFrame * CFrame.new(-chamber.Size.X * 0.46, 0, frontZ), frameColour,
    ↳ Enum.Material.Metal, chamber))
2088     table.insert(frameParts, newPart("GlassFrameRight", Vector3.new(0.22, chamber.Size.Y * 0.94,
    ↳ 0.2), chamber.CFrame * CFrame.new(chamber.Size.X * 0.46, 0, frontZ), frameColour,
    ↳ Enum.Material.Metal, chamber))
2089     table.insert(frameParts, newPart("GlassFrameTop", Vector3.new(chamber.Size.X * 0.94, 0.24,
    ↳ 0.2), chamber.CFrame * CFrame.new(0, chamber.Size.Y * 0.47, frontZ), frameColour,
    ↳ Enum.Material.Metal, chamber))
2090     table.insert(frameParts, newPart("GlassFrameBottom", Vector3.new(chamber.Size.X * 0.94, 0.24,
    ↳ 0.2), chamber.CFrame * CFrame.new(0, -chamber.Size.Y * 0.47, frontZ), frameColour,
    ↳ Enum.Material.Metal, chamber))
2091 end
2092
2093 -- Four ventilated shelves and four white grow-light strips.
2094 for tier = 0, 3 do
2095     local shelfY = -chamber.Size.Y * 0.35 + tier * chamber.Size.Y * 0.22
2096     local shelfSize = frontOnX
2097         and Vector3.new(innerDepth * 0.72, 0.1, innerWidth * 0.86)
2098         or Vector3.new(innerWidth * 0.86, 0.1, innerDepth * 0.72)
2099     local shelf = newPart("PlantShelf_" .. (tier + 1), shelfSize,
2100         chamber.CFrame * CFrame.new(0, shelfY - 0.28, 0), Color3.fromRGB(128, 135, 136),
2101         ↳ Enum.Material.Metal, chamber)
2102     local lightSize = frontOnX and Vector3.new(0.14, 0.1, innerWidth * 0.75) or
2103         ↳ Vector3.new(innerWidth * 0.75, 0.1, 0.14)
2104     local light = newPart("GrowLight_" .. (tier + 1), lightSize,
2105         chamber.CFrame * interiorOffset(0, shelfY + chamber.Size.Y * 0.13, innerDepth * 0.45),
2106         Color3.fromRGB(255, 248, 222), Enum.Material.Neon, chamber)
2107     table.insert(frameParts, shelf); table.insert(frameParts, light)
2108 end
2109
2110 -- Rear ventilation grille and a raised controller housing.
2111 local ventSize = frontOnX and Vector3.new(0.12, chamber.Size.Y * 0.24, innerWidth * 0.45) or
2112     ↳ Vector3.new(innerWidth * 0.45, chamber.Size.Y * 0.24, 0.12)
2113 local vent = newPart("VentilationGrille", ventSize,
2114     chamber.CFrame * interiorOffset(0, chamber.Size.Y * 0.2, innerDepth * 0.92), Color3.fromRGB(70,
2115     ↳ 78, 82), Enum.Material.DiamondPlate, chamber)
2116 table.insert(frameParts, vent)
2117 local controllerSize = frontOnX and Vector3.new(innerDepth * 0.9, 0.72, innerWidth * 0.94) or
2118     ↳ Vector3.new(innerWidth * 0.94, 0.72, innerDepth * 0.9)
2119 local controller = newPart("ClimateController", controllerSize,
2120     chamber.CFrame * CFrame.new(0, chamber.Size.Y * 0.5 + 0.34, 0), Color3.fromRGB(226, 229, 228),
2121     ↳ Enum.Material.Metal, chamber)
2122 controller.CanCollide = false
2123 local controllerFace = newPart("ControllerScreen",
2124     frontOnX and Vector3.new(0.12, 0.46, innerWidth * 0.58) or Vector3.new(innerWidth * 0.58, 0.46,
2125     ↳ 0.12),
2126     chamber.CFrame * interiorOffset(0, chamber.Size.Y * 0.5 + 0.34, -0.18),
2127     initiallyFailed and Color3.fromRGB(220, 55, 45) or Color3.fromRGB(40, 150, 95),
2128     ↳ Enum.Material.Neon, chamber)
2129 controllerFace.CanCollide = false
2130 controllerFace:SetAttribute("ChamberStatusLamp", true)
2131
2132 -- Small caster wheels like the mobile cabinets in the references.
2133 for _, across in ipairs({-innerWidth * 0.38, innerWidth * 0.38}) do
2134     for _, depthSign in ipairs({-1, 1}) do
2135         local wheelOffset
2136         if frontOnX then wheelOffset = CFrame.new(depthSign * innerDepth * 0.32, -chamber.Size.Y /
2137             ↳ 2 - 0.23, across)
2138         else wheelOffset = CFrame.new(across, -chamber.Size.Y / 2 - 0.23, depthSign * innerDepth *
2139             ↳ 0.32) end
2140         local wheel = newPart("CasterWheel", Vector3.new(0.38, 0.38, 0.22), chamber.CFrame *
2141             ↳ wheelOffset,
2142             Color3.fromRGB(28, 30, 32), Enum.Material.Rubber, chamber)
2143         wheel.Shape = Enum.PartType.Cylinder

```

```

2133         wheel.CanCollide = false
2134     end
2135 end
2136
2137 local door = chamber:FindFirstChild("Door")
2138 if door then
2139     door.Material = Enum.Material.Glass
2140     door.Color = Color3.fromRGB(205, 232, 225)
2141     door.Transparency = 0.38
2142     addGrowthCodeLabel(door, code)
2143 end
2144 for _, framePart in ipairs(frameParts) do
2145     framePart.CanCollide = false
2146     framePart.CanTouch = false
2147 end
2148
2149 -- Individually addressable germinated-seed trays. Capacity follows
2150 -- physical cabinet size: 4, 6, 8, or 12 plants.
2151 for slot = 1, capacity do
2152     local tier = math.floor((slot - 1) / 3)
2153     local column = (slot - 1) % 3
2154     local across = (column - 1) * innerWidth * 0.27
2155     if slot == 10 then across = 0 end
2156     local up = -chamber.Size.Y * 0.35 + tier * chamber.Size.Y * 0.22
2157     local plantBase = chamber.CFrame * interiorOffset(across, up, 0.48)
2158     local slotName = string.format("ChamberSample%02d_", slot)
2159     local pot = newPart(slotName .. "Tray", Vector3.new(0.64, 0.32, 0.56), plantBase,
2160         Color3.fromRGB(128, 75, 43), Enum.Material.SmoothPlastic, chamber)
2161     local stem = newPart(slotName .. "Stem", Vector3.new(0.1, 0.55, 0.1),
2162         plantBase * CFrame.new(0, 0.4, 0), Color3.fromRGB(55, 155, 68),
2163         Enum.Material.SmoothPlastic, chamber)
2164     local leafL = newPart(slotName .. "LeafL", Vector3.new(0.36, 0.09, 0.22),
2165         plantBase * CFrame.new(-0.18, 0.53, 0), Color3.fromRGB(72, 184, 82),
2166         Enum.Material.SmoothPlastic, chamber)
2167     local leafR = newPart(slotName .. "LeafR", Vector3.new(0.36, 0.09, 0.22),
2168         plantBase * CFrame.new(0.18, 0.68, 0), Color3.fromRGB(72, 184, 82),
2169         Enum.Material.SmoothPlastic, chamber)
2170     for _, plantPart in ipairs({pot, stem, leafL, leafR}) do
2171         plantPart.CanCollide = false
2172         plantPart.CanTouch = false
2173         plantPart:SetAttribute("SampleSlot", slot)
2174         plantPart:SetAttribute("StoredTransparency", 0)
2175         if slot > sampleCount then plantPart.Transparency = 1 end
2176     end
2177 end
2178
2179 end
2180
2181 -- Every level receives the complete five-machine plant-development
2182 -- route, distributed across separate rooms and packed against walls.
2183 -- This guarantees that players and all 50 bots can finish a full cycle.
2184 if plantStationKind then
2185     local stationColours = {
2186         ["Growth Hormone Mixer"] = Color3.fromRGB(100, 190, 110),
2187         ["GMO Injector"] = Color3.fromRGB(170, 100, 215),
2188         ["Nutrient Infuser"] = Color3.fromRGB(210, 165, 65),
2189         ["UV Growth Scanner"] = Color3.fromRGB(70, 145, 225),
2190         ["Pollination Station"] = Color3.fromRGB(225, 125, 170),
2191     }
2192     local entry = {
2193         kind = plantStationKind,
2194         w = 115, d = 95, h = 235, capacity = 8,
2195         col = stationColours[plantStationKind] or COL.green,
2196         mat = Enum.Material.Metal,
2197         shape = "appliance",
2198         plantCabinet = true,
2199         auto = "pulse",
2200     }
2201     local firstSide = rng.int(1, #order)
2202     for try = 0, #order - 1 do
2203         local side = order[((firstSide + try - 1) % #order) + 1]
2204         local at = takeSpan(spans[side], P(entry.w))
2205         if at then spawn(entry, side, at, false); break end
2206     end
2207 end
2208
2209 end

```

```

2206
2207     local area = (room.x2 - room.x1) * (room.z2 - room.z1)
2208     -- Every wall gets its own island run, and each run keeps packing units
2209     -- for as long as its wall physically has room -- not a fixed rep count,
2210     -- so no wall is left with a leftover gap a unit could still have filled.
2211     -- A short streak of consecutive misses (picked entry just didn't fit
2212     -- the last sliver) is what actually ends a wall's run, not a target.
2213     if #BY_CLASS.run > 0 then
2214         for _, side in ipairs(order) do
2215             local guard, misses = 0, 0
2216             while guard < 200 and misses < 6 do
2217                 guard = guard + 1
2218                 local entry = weightedPick(BY_CLASS.run, rng)
2219                 local need = P(entry.w)
2220                 local at = takeSpan(spans[side], need)
2221                 if at then
2222                     misses = 0
2223                     spawn(entry, side, at, false)
2224                     table.insert(benchRuns, {
2225                         side = side,
2226                         inset = wallFaceInset(side) + P(entry.d) / 2,
2227                         spans = {{at - need / 2 + 12, at + need / 2 - 12}},
2228                         height = entry.h,
2229                     })
2230                 else
2231                     misses = misses + 1
2232                 end
2233             end
2234         end
2235     end
2236
2237     -- 2. free-standing units -- fills whatever wall space the island runs
2238     -- above didn't claim, again running until every wall genuinely can't
2239     -- take another unit rather than stopping at an area-based headcount.
2240     if #BY_CLASS.floor > 0 then
2241         local guard, misses = 0, 0
2242         while guard < 3000 and misses < 24 do
2243             guard = guard + 1
2244             local entry = weightedPick(BY_CLASS.floor, rng)
2245             local side = order[rng.int(1, #order)]
2246             local at = takeSpan(spans[side], P(entry.w))
2247             if at then
2248                 misses = 0
2249                 spawn(entry, side, at, false)
2250             else
2251                 misses = misses + 1
2252             end
2253         end
2254     end
2255
2256     -- 2b. decoration pass -- potted plants, flower planters, and lab
2257     -- storage furniture pack into whatever slivers are still left on every
2258     -- wall once real equipment can no longer fit, using a much smaller gap
2259     -- since decor carries no status readout for a neighbour to overlap.
2260     -- This is what actually gets a room to "no free wall space" rather
2261     -- than just "no free space equipment can use."
2262     if #BY_CLASS.decor > 0 then
2263         for _, side in ipairs(order) do
2264             local guard, misses = 0, 0
2265             while guard < 300 and misses < 8 do
2266                 guard = guard + 1
2267                 local entry = weightedPick(BY_CLASS.decor, rng)
2268                 local at = takeSpanGap(spans[side], P(entry.w), 6)
2269                 if at then
2270                     misses = 0
2271                     spawn(entry, side, at, false)
2272                 else
2273                     misses = misses + 1
2274                 end
2275             end
2276         end
2277     end
2278
2279     -- 3. instruments on the bench runs
2280     if #BY_CLASS.top > 0 then
2281         for _, run in ipairs(benchRuns) do

```

```

2282         local slots = rng.int(1, 3)
2283         for _ = 1, slots do
2284             local entry = weightedPick(BY_CLASS.top, rng)
2285             local at = takeSpan(run.spans, P(entry.w))
2286             if at then spawn(entry, run.side, at, true, run.inset) end
2287         end
2288     end
2289 end
2290
2291 -- 4. safety kit, one of each, wherever it still fits
2292 for _, entry in ipairs(BY_CLASS.duty) do
2293     for _, side in ipairs(order) do
2294         local at = takeSpan(spans[side], P(entry.w))
2295         if at then spawn(entry, side, at, false); break end
2296     end
2297 end
2298
2299 -- (Room status monitor is reserved earlier, right after `order` is
2300 -- computed, so it always claims prime wall space instead of a leftover
2301 -- corner sliver – see the block above, right after the wall-side sort.)
2302 end
2303
2304 -----
2305 -- R1-R200 PLANT GROWTH CHAMBERS
2306 -- Fifty variable-size growing chambers are placed on every level. Codes are
2307 -- printed on the door itself, so labels never float across the room.
2308 -----
2309 addGrowthCodeLabel = function(door, code)
2310     local labelFaces = (door.Size.X < door.Size.Z)
2311     and {Enum.NormalId.Left, Enum.NormalId.Right}
2312     or {Enum.NormalId.Front, Enum.NormalId.Back}
2313     for _, normal in ipairs(labelFaces) do
2314         local surface = Instance.new("SurfaceGui")
2315         surface.Name = "GrowthCodeLabel"
2316         surface.Face = normal
2317         surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2318         surface.PixelsPerStud = 70
2319         surface.AlwaysOnTop = false
2320         surface.Parent = door
2321
2322         local label = Instance.new("TextLabel")
2323         label.Size = UDim2.new(0.72, 0, 0.3, 0)
2324         label.Position = UDim2.new(0.14, 0, 0.08, 0)
2325         label.BackgroundColor3 = Color3.fromRGB(20, 44, 34)
2326         label.BackgroundTransparency = 0.08
2327         label.BorderSizePixel = 0
2328         label.Text = code
2329         label.TextColor3 = Color3.fromRGB(170, 255, 185)
2330         label.TextScaled = true
2331         label.Font = Enum.Font.GothamBold
2332         label.Parent = surface
2333
2334         local instruction = Instance.new("TextLabel")
2335         instruction.Size = UDim2.new(0.82, 0, 0.18, 0)
2336         instruction.Position = UDim2.new(0.09, 0, 0.72, 0)
2337         instruction.BackgroundTransparency = 1
2338         instruction.Text = "E TAKE / DROP • R REPAIR"
2339         instruction.TextColor3 = Color3.fromRGB(225, 235, 225)
2340         instruction.TextScaled = true
2341         instruction.Font = Enum.Font.Gotham
2342         instruction.Parent = surface
2343     end
2344 end
2345
2346 -- Usable wall a room can actually back a chamber against: each of the four
2347 -- walls minus the corner keep-outs at both of its ends.
2348 local function growthWallWeight(room)
2349     local acrossX = math.max(0, (room.x2 - room.x1) - 2 * CORNER)
2350     local acrossZ = math.max(0, (room.z2 - room.z1) - 2 * CORNER)
2351     return math.max(acrossX * 2 + acrossZ * 2, 1)
2352 end
2353
2354 -- The 50 R-codes a floor owns are shared out in proportion to that usable
2355 -- wall, not evenly. The plan is deliberately lopsided -- one long room runs
2356 -- the entire west side while the east rooms are short bays -- so an even
2357 -- split kept asking the narrow bays for more chambers than their walls could

```

```

2358 -- physically hold (the overflow was silently dropped) while the long west
2359 -- room sat half empty. Largest-remainder rounding keeps each floor's total
2360 -- at exactly 50 and the codes contiguous.
2361 local function growthCodesForRoom(firstCode, roomIndex, rooms)
2362     local weights, total = {}, 0
2363     for i, r in ipairs(rooms) do
2364         weights[i] = growthWallWeight(r)
2365         total = total + weights[i]
2366     end
2367
2368     local counts, assigned, remainders = {}, 0, {}
2369     for i = 1, #rooms do
2370         local exact = 50 * weights[i] / total
2371         counts[i] = math.floor(exact)
2372         assigned = assigned + counts[i]
2373         table.insert(remainders, {index = i, frac = exact - counts[i]})
2374     end
2375     table.sort(remainders, function(a, b)
2376         if a.frac == b.frac then return a.index < b.index end
2377         return a.frac > b.frac
2378     end)
2379     for i = 1, 50 - assigned do
2380         local pick = remainders[i]
2381         if not pick then break end
2382         counts[pick.index] = counts[pick.index] + 1
2383     end
2384
2385     local before = 0
2386     for i = 1, roomIndex - 1 do before = before + counts[i] end
2387     local codes = {}
2388     for i = 1, counts[roomIndex] do
2389         table.insert(codes, "R" .. (firstCode + before + i - 1))
2390     end
2391     return codes
2392 end
2393
2394 -----
2395 -- BUILD
2396 -----
2397 local BOTTOM_LEVEL = -1
2398 local TOP_LEVEL = LEVELS[#LEVELS].index
2399
2400 local basementModel, basementRooms = buildBasement()
2401 local levelModels = {}
2402
2403 local GUARANTEED_PLANT_STATIONS = {
2404     "Growth Hormone Mixer",
2405     "GMO Injector",
2406     "Nutrient Infuser",
2407     "UV Growth Scanner",
2408     "Pollination Station",
2409 }
2410 for _, lv in ipairs(LEVELS) do
2411     local m = buildLevelShell(lv.index, lv.name, lv.label, lv.rooms)
2412     levelModels[lv.index] = m
2413     local folder = Instance.new("Folder")
2414     folder.Name = "Equipment"
2415     folder.Parent = m
2416     for roomIndex, r in ipairs(lv.rooms) do
2417         -- The ground floor's first room is the entrance lobby: it gets a
2418         -- full equipment fit-out just like every other room, but the mid-
2419         -- room strip where the role-selection kiosks/spawn/chevrons live
2420         -- (roughly z 150-460 in the room's own coordinates) is reserved as
2421         -- a keep-clear zone first, the same way a door or window reserves
2422         -- clearance, so wall equipment never grows over the kiosks.
2423         local isEntranceLobby = (lv.index == 0 and roomIndex == 1)
2424         if isEntranceLobby then
2425             table.insert(r.open.W, {at = 150, width = 310})
2426             table.insert(r.open.E, {at = 150, width = 310})
2427         end
2428         fitOutRoom(folder, r, lv.index * F2F, lv.name,
2429             newRng(RNG_SEED + lv.index * 1000 + r.number),
2430             growthCodesForRoom(1 + lv.index * 50, roomIndex, lv.rooms),
2431             GUARANTEED_PLANT_STATIONS[roomIndex])
2432     end
2433 end

```



```

2434
2435 do
2436     local folder = Instance.new("Folder")
2437     folder.Name = "Equipment"
2438     folder.Parent = basementModel
2439     for roomIndex, r in ipairs(basementRooms) do
2440         -- Basement's code block starts right after every above-ground
2441         -- level's own 50-code block, whatever the level count is (used to
2442         -- be a hard-coded 151, back when there were only 3 levels above
2443         -- it -- with more levels that collided with a level's own codes).
2444         fitOutRoom(folder, r, BOTTOM_LEVEL * F2F, "Basement",
2445             newRng(RNG_SEED + 7777 + r.number),
2446             growthCodesForRoom(1 + #LEVELS * 50, roomIndex, basementRooms),
2447             GUARANTEED_PLANT_STATIONS[roomIndex])
2448     end
2449 end
2450
2451 -- Visible test scientists are distributed over EVERY level -- basement
2452 -- included -- at a fixed count per floor, rather than a fixed total split
2453 -- across just the original four. That keeps bot density per floor roughly
2454 -- constant as floors are added or removed instead of spreading the same
2455 -- headcount thinner across a bigger building. BOTS_PER_LEVEL is kept
2456 -- moderate (not the 50/floor first asked for) because with basement + all
2457 -- LEVELS that would put ~950 concurrent pathfinding NPCs on one server --
2458 -- a real lag/timeout risk. This keeps the total in the low hundreds while
2459 -- still reading as far denser per floor than before. The runtime assigns
2460 -- each one the same loop a player performs: answer a failed-chamber alarm,
2461 -- visibly carry one germinated plant through a growth process, and rescue
2462 -- it into a working R chamber with capacity.
2463 -----
2464 -- CORRIDOR FIXTURES -- health stations and sidearm cabinets
2465 --
2466 -- These hang on the corridor walls rather than inside rooms on purpose:
2467 -- room wall space is already fully budgeted (monitor, kit, plant station,
2468 -- the floor's share of R-code chambers, then furniture), so putting these
2469 -- in rooms would push chambers back under the 50-per-floor target. The
2470 -- corridor is circulation space nobody was using. They are CanCollide off
2471 -- so they can never block a doorway or an NPC's path.
2472 -----
2473 -- Ten of each support pickup per floor gives players a nearby recovery or
2474 -- weapon option without putting a cabinet in a doorway.
2475     local HEALTH_STATIONS_PER_FLOOR = 10
2476     local WEAPON_STATIONS_PER_FLOOR = 10
2477
2478 -- Weapon kinds the corridor cabinets hand out, one per cabinet per floor.
2479 -- Declared here because the cabinets are built long before IMPACT_STYLE (which
2480 -- defines the weapons themselves) appears further down the file.
2481 local PC_WEAPONS = {"Lab Sidearm", "Lab Shotgun", "Lab Rifle"}
2482
2483 -- Poster copy. Drawn procedurally from Frames and TextLabels -- no imported
2484 -- decals, so nothing here depends on an uploaded image asset.
2485 local POSTER_TITLES = {
2486     {title = "GERMINATION RATE BY CULTIVAR", seed = 3, colour = Color3.fromRGB(46, 122, 84),
2487      body = "n = 400 seeds per lot, 4 replicates of 100. Counts at day 4 and day 8. Rolled paper substrate,
2488      ↪ 20/30 C alternating."},
2489     {title = "SEED VIABILITY - TETRAZOLIUM", seed = 5, colour = Color3.fromRGB(150, 52, 52),
2490      body = "1% TZ solution, 30 C for 4 h in darkness. Embryo staining scored 1-5. Confirms viability where
2491      ↪ germination is dormant."},
2492     {title = "MOISTURE CONTENT & STORAGE LIFE", seed = 2, colour = Color3.fromRGB(52, 96, 150),
2493      body = "Oven method, 103 C for 17 h. Each 1% drop in moisture roughly doubles safe storage life within the
2494      ↪ working range."},
2495     {title = "ISTA SAMPLING PROTOCOL", seed = 7, colour = Color3.fromRGB(96, 74, 148),
2496      body = "Primary samples drawn from the lot, combined, then reduced by successive halving to the submitted
2497      ↪ sample."},
2498     {title = "THERMAL TIME MODEL", seed = 4, colour = Color3.fromRGB(150, 108, 40),
2499      body = "Base temperature Tb = 4.2 C. Accumulated degree-days predict time to 50% emergence across the
2500      ↪ tested range."},
2501     {title = "GMO EVENT DETECTION - qPCR", seed = 6, colour = Color3.fromRGB(38, 108, 130),
2502      body = "Event-specific primers against the junction region. Reference gene normalisation, LOD 0.05%
2503      ↪ w/w."},
2504     {title = "HORMONE PRIMING RESPONSE", seed = 8, colour = Color3.fromRGB(120, 60, 110),
2505      body = "Gibberellin soak breaks dormancy in stratification-requiring lots. Dose response measured over 5
2506      ↪ concentrations."},
2507     {title = "PATHOGEN SCREENING WORKFLOW", seed = 9, colour = Color3.fromRGB(140, 84, 36),
2508      body = "Blotter incubation 7 days, 22 C, 12 h NUV. Fungal structures identified under stereo and compound
2509      ↪ microscopy."},

```

```

2502 }
2503
2504 local function buildCorridorFixtures(parent, levelY, levelName, rooms)
2505     if not parent then return end
2506
2507     -- Build one shared wall-space allocator for every corridor fixture. Door
2508     -- openings are excluded with the same generous clearance used by room
2509     -- equipment, and the end margins keep pickups away from entrance glazing.
2510     local function corridorOpenings(onWest)
2511         local openings = {}
2512         for _, room in ipairs(rooms or {}) do
2513             local belongs = (onWest and room.plan == "L") or ((not onWest) and room.plan == "R")
2514             if belongs then
2515                 local side = onWest and "E" or "W"
2516                 for _, opening in ipairs(room.open[side] or {}) do
2517                     table.insert(openings, {at = room.z1 + opening.at, width = opening.width})
2518                 end
2519             end
2520         end
2521         return openings
2522     end
2523
2524     local available = {
2525         west = freeSpans(L, corridorOpenings(true)),
2526         east = freeSpans(L, corridorOpenings(false)),
2527     }
2528
2529     -- Taking the centre of the longest remaining span distributes fixtures
2530     -- along the full corridor instead of clustering them near one stairwell.
2531     local function takeCentred(spans, needed)
2532         local bestIndex, bestLength
2533         for index, span in ipairs(spans) do
2534             local length = span[2] - span[1]
2535             if length >= needed and (not bestLength or length > bestLength) then
2536                 bestIndex, bestLength = index, length
2537             end
2538         end
2539         if not bestIndex then return nil end
2540
2541         local span = table.remove(spans, bestIndex)
2542         local centre = (span[1] + span[2]) / 2
2543         local half = needed / 2
2544         local separation = 18
2545         if centre - half - separation > span[1] then
2546             table.insert(spans, {span[1], centre - half - separation})
2547         end
2548         if centre + half + separation < span[2] then
2549             table.insert(spans, {centre + half + separation, span[2]})
2550         end
2551         return centre
2552     end
2553
2554     local function reserveCorridorSpot(preferWest, width)
2555         local first = preferWest and "west" or "east"
2556         local second = preferWest and "east" or "west"
2557         local at = takeCentred(available[first], P(width))
2558         if at then return first == "west", at end
2559         at = takeCentred(available[second], P(width))
2560         if at then return second == "west", at end
2561         error("No door-safe corridor wall space remains for " .. levelName)
2562     end
2563
2564     for i = 1, HEALTH_STATIONS_PER_FLOOR do
2565         -- Alternate walls so you can always see one from anywhere in the run.
2566         local depth, width, height = 18, 55, 70
2567         local onWest, pz = reserveCorridorSpot(i % 2 == 1, width)
2568         local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2569
2570         local station = newPart("HealthStation_" .. levelName .. "_" .. i,
2571             Vector3.new(E(depth), EH(height), E(width)),
2572             CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(110), X(pz)),
2573             Color3.fromRGB(225, 40, 35), Enum.Material.SmoothPlastic, parent)
2574         station.CanCollide = false
2575         station:SetAttribute("Level", levelName)
2576         CollectionService:AddTag(station, "EmergencyKit")
2577     end

```

```

2578     local face = Instance.new("SurfaceGui")
2579     face.Name = "EmergencyKitFace"
2580     face.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left
2581     face.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2582     face.PixelsPerStud = 60
2583     face.AlwaysOnTop = false
2584     face.Parent = station
2585     local crossV = Instance.new("Frame")
2586     crossV.Size = UDim2.new(0.2, 0, 0.5, 0)
2587     crossV.Position = UDim2.new(0.4, 0, 0.1, 0)
2588     crossV.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2589     crossV.BorderSizePixel = 0
2590     crossV.Parent = face
2591     local crossH = Instance.new("Frame")
2592     crossH.Size = UDim2.new(0.5, 0, 0.2, 0)
2593     crossH.Position = UDim2.new(0.25, 0, 0.25, 0)
2594     crossH.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2595     crossH.BorderSizePixel = 0
2596     crossH.Parent = face
2597     local caption = Instance.new("TextLabel")
2598     caption.Name = "KitCaption"
2599     caption.Size = UDim2.new(1, 0, 0.26, 0)
2600     caption.Position = UDim2.new(0, 0, 0.7, 0)
2601     caption.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2602     caption.BackgroundTransparency = 0.05
2603     caption.BorderSizePixel = 0
2604     caption.Text = "HEALTH"
2605     caption.TextColor3 = Color3.fromRGB(200, 30, 25)
2606     caption.TextScaled = true
2607     caption.Font = Enum.Font.SourceSansBold
2608     caption.Parent = face
2609
2610     local prompt = Instance.new("ProximityPrompt")
2611     prompt.Name = "EmergencyKitPrompt"
2612     prompt.ActionText = "Use Health Station"
2613     prompt.ObjectText = "Restore health"
2614     prompt.KeyboardKeyCode = Enum.KeyCode.E
2615     prompt.MaxActivationDistance = 10
2616     prompt.RequiresLineOfSight = false
2617     prompt.Parent = station
2618 end
2619
2620 -- Ten weapon cabinets per floor, distributed across both corridor walls.
2621 for i = 1, WEAPON_STATIONS_PER_FLOOR do
2622     local depth, width, height = 22, 70, 120
2623     local onWest, pz = reserveCorridorSpot(i % 2 == 0, width)
2624     local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2625     local cabinet = newPart("SidearmCabinet_" .. levelName .. "_" .. i,
2626         Vector3.new(E(depth), EH(height), E(width)),
2627         CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(80), X(pz)),
2628         Color3.fromRGB(52, 58, 68), Enum.Material.Metal, parent)
2629     cabinet.CanCollide = false
2630     cabinet:SetAttribute("Level", levelName)
2631     -- One weapon type per cabinet, so every floor offers all three.
2632     local weaponKind = PC_WEAPONS[(i - 1) % #PC_WEAPONS] + 1]
2633     cabinet:SetAttribute("ToolKind", weaponKind)
2634     CollectionService:AddTag(cabinet, "LabToolDispenser")
2635
2636     local face = Instance.new("SurfaceGui")
2637     face.Name = "SidearmCabinetFace"
2638     face.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left
2639     face.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2640     face.PixelsPerStud = 60
2641     face.AlwaysOnTop = false
2642     face.Parent = cabinet
2643     local caption = Instance.new("TextLabel")
2644     caption.Size = UDim2.new(0.9, 0, 0.3, 0)
2645     caption.Position = UDim2.new(0.05, 0, 0.12, 0)
2646     caption.BackgroundColor3 = Color3.fromRGB(235, 195, 35)
2647     caption.BackgroundTransparency = 0.05
2648     caption.BorderSizePixel = 0
2649     caption.Text = (weaponKind:gsub("^Lab ", "")):upper()
2650     caption.TextColor3 = Color3.fromRGB(25, 25, 25)
2651     caption.TextScaled = true
2652     caption.Font = Enum.Font.SourceSansBold
2653     caption.Parent = face

```

```

2654     local sub = Instance.new("TextLabel")
2655     sub.Size = UDim2.new(0.9, 0, 0.4, 0)
2656     sub.Position = UDim2.new(0.05, 0, 0.48, 0)
2657     sub.BackgroundTransparency = 1
2658     sub.Text = "PC • MOBILE • TABLET\nHold E / tap to equip\nAim at the target marker"
2659     sub.TextColor3 = Color3.fromRGB(235, 240, 245)
2660     sub.TextScaled = true
2661     sub.Font = Enum.Font.Gotham
2662     sub.Parent = face
2663 end
2664
2665 -- Gas service pipes. Deliberately obvious, and very dangerous: a round
2666 -- into one takes out a chunk of the room. CanCollide is off so they never
2667 -- block a walking route, but CanQuery stays on so they can be shot.
2668 for i, px in ipairs({CORR_X1 + 20, CORR_X2 - 20}) do
2669     local pipe = newPart("GasPipe_" .. levelName .. "_" .. i,
2670         Vector3.new(E(16), EH(16), X(L - 240)),
2671         CFrame.new(X(px), Y(levelY) + Y(WALL_H) - EH(18), X(L / 2)),
2672         Color3.fromRGB(216, 176, 42), Enum.Material.Metal, parent)
2673     pipe.CanCollide = false
2674     pipe:SetAttribute("GasPipe", true)
2675     pipe:SetAttribute("Level", levelName)
2676     CollectionService.AddTag(pipe, "GasPipe")
2677
2678     -- hazard collars every few metres so it reads as a gas line
2679     for band = 1, 6 do
2680         local collar = newPart("GasPipeBand_" .. levelName .. "_" .. i .. "_" .. band,
2681             Vector3.new(E(19), EH(19), X(26)),
2682             CFrame.new(X(px), Y(levelY) + Y(WALL_H) - EH(18), X(L * band / 7)),
2683             Color3.fromRGB(30, 30, 30), Enum.Material.SmoothPlastic, parent)
2684         collar.CanCollide = false
2685         collar.CastShadow = false
2686     end
2687 end
2688
2689 -- Scientific posters. Every one is a real part with CanQuery left on, so
2690 -- the weapon raycast can hit it and punch a hole that stays put.
2691 for i = 1, #POSTER_TITLES do
2692     local depth, width, height = 6, 118, 84
2693     local onWest, pz = reserveCorridorSpot(i % 2 == 0, width)
2694     local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2695
2696     local poster = newPart("Poster_" .. levelName .. "_" .. i,
2697         Vector3.new(E(depth), EH(height), E(width)),
2698         CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(120), X(pz)),
2699         Color3.fromRGB(242, 240, 233), Enum.Material.SmoothPlastic, parent)
2700     poster.CanCollide = false
2701     poster:SetAttribute("ShootablePoster", true)
2702     poster:SetAttribute("Holes", 0)
2703     poster:SetAttribute("Level", levelName)
2704     CollectionService.AddTag(poster, "LabPoster")
2705
2706     local art = Instance.new("SurfaceGui")
2707     art.Name = "PosterFace"
2708     art.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left
2709     art.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2710     art.PixelsPerStud = 55
2711     art.AlwaysOnTop = false
2712     art.Parent = poster
2713
2714     local entry = POSTER_TITLES[i]
2715     local band = Instance.new("Frame")
2716     band.Size = UDim2.new(1, 0, 0.2, 0)
2717     band.BackgroundColor3 = entry.colour
2718     band.BorderSizePixel = 0
2719     band.Parent = art
2720     local title = Instance.new("TextLabel")
2721     title.Size = UDim2.new(0.92, 0, 0.8, 0)
2722     title.Position = UDim2.new(0.04, 0, 0.1, 0)
2723     title.BackgroundTransparency = 1
2724     title.Text = entry.title
2725     title.TextColor3 = Color3.fromRGB(255, 255, 255)
2726     title.TextScaled = true
2727     title.Font = Enum.Font.GothamBold
2728     title.Parent = band
2729

```

```

2730 -- A little chart so it reads as a research poster rather than a sign.
2731 local plot = Instance.new("Frame")
2732 plot.Size = UDim2.new(0.44, 0, 0.5, 0)
2733 plot.Position = UDim2.new(0.04, 0, 0.28, 0)
2734 plot.BackgroundColor3 = Color3.fromRGB(250, 250, 246)
2735 plot.BorderSizePixel = 0
2736 plot.Parent = art
2737 for barIndex = 1, 5 do
2738     local h = 0.2 + ((entry.seed * barIndex) % 7) / 9
2739     local bar = Instance.new("Frame")
2740     bar.Size = UDim2.new(0.14, 0, h, 0)
2741     bar.Position = UDim2.new(0.05 + (barIndex - 1) * 0.19, 0, 1 - h, 0)
2742     bar.BackgroundColor3 = entry.colour
2743     bar.BorderSizePixel = 0
2744     bar.Parent = plot
2745 end
2746
2747 local body = Instance.new("TextLabel")
2748 body.Size = UDim2.new(0.44, 0, 0.5, 0)
2749 body.Position = UDim2.new(0.52, 0, 0.28, 0)
2750 body.BackgroundTransparency = 1
2751 body.Text = entry.body
2752 body.TextColor3 = Color3.fromRGB(48, 52, 58)
2753 body.TextScaled = true
2754 body.TextWrapped = true
2755 body.Font = Enum.Font.Gotham
2756 body.TextXAlignment = Enum.TextXAlignment.Left
2757 body.Parent = art
2758
2759 local footer = Instance.new("TextLabel")
2760 footer.Size = UDim2.new(0.92, 0, 0.12, 0)
2761 footer.Position = UDim2.new(0.04, 0, 0.84, 0)
2762 footer.BackgroundTransparency = 1
2763 footer.Text = "SEED TESTING LABORATORY - " .. levelName:upper()
2764 footer.TextColor3 = Color3.fromRGB(120, 126, 134)
2765 footer.TextScaled = true
2766 footer.Font = Enum.Font.Gotham
2767 footer.TextXAlignment = Enum.TextXAlignment.Left
2768 footer.Parent = art
2769 end
2770 end
2771
2772 buildCorridorFixtures(basementModel, BOTTOM_LEVEL * F2F, "Basement", BASEMENT_ROOMS)
2773 for _, lv in ipairs(LEVELS) do
2774     buildCorridorFixtures(levelModels[lv.index], lv.index * F2F, lv.name, lv.rooms)
2775 end
2776
2777 -- Publish every room's rectangle, already converted to stud space, so the
2778 -- runtime can answer "is this player in the same room as this bot?" exactly.
2779 -- A plain radius test cannot do that here: the west room runs the full 267
2780 -- studs of the plan, so any radius wide enough to cover it also reaches
2781 -- straight through the corridor wall into the rooms opposite.
2782 do
2783     local entries = {}
2784     local function addRooms(levelName, rooms, levelY)
2785         for _, r in ipairs(rooms) do
2786             table.insert(entries, string.format("%s:%s:%.1f:%.1f:%.1f:%.1f",
2787                 levelName, r.name, X(r.x1), X(r.z1), X(r.x2), X(r.z2), Y(levelY)))
2788         end
2789     end
2790     addRooms("Basement", basementRooms, BOTTOM_LEVEL * F2F)
2791     for _, lv in ipairs(LEVELS) do
2792         addRooms(lv.name, lv.rooms, lv.index * F2F)
2793     end
2794     root:SetAttribute("RoomIndex", table.concat(entries, ";"))
2795 end
2796
2797 local BOTS_PER_LEVEL = 20
2798 local workerFolder = Instance.new("Folder")
2799 workerFolder.Name = "WorkerTestNPCs"
2800 workerFolder.Parent = root
2801 local botNumber = 0
2802 local botLevelSpecs = {{y = BOTTOM_LEVEL * F2F, count = BOTS_PER_LEVEL,
2803     name = "Basement", rooms = basementRooms}}
2804 for _, lv in ipairs(LEVELS) do
2805     table.insert(botLevelSpecs, {y = lv.index * F2F, count = BOTS_PER_LEVEL,

```

```

2806         name = lv.name, rooms = lv.rooms}})
2807 end
2808
2809 -- Some staff sit at a workstation instead of patrolling. Seats are placed
2810 -- from the ROOM SCHEDULE, not from the preserved OfficeFitout: that fit-out
2811 -- was built for an earlier version of the building and its geometry now sits
2812 -- outside the current footprint entirely, so seating from it would put staff
2813 -- in mid-air outside the walls.
2814 local SEATED_PER_LEVEL = 5
2815 for _, levelSpec in ipairs(botLevelSpecs) do
2816     local seatedThisLevel = 0
2817     for slot = 1, levelSpec.count do
2818         botNumber = botNumber + 1
2819         local xcm = (slot % 2 == 0) and 260 or 1040
2820         local zcm = 190 + ((slot - 1) * 137) % 1690
2821         -- skip index 1: that is the long west room, which is circulation space
2822         local seatRoom
2823         if seatedThisLevel < SEATED_PER_LEVEL and levelSpec.rooms then
2824             seatRoom = levelSpec.rooms[seatedThisLevel + 2]
2825         end
2826         local botModel = buildWorkerBot(workerFolder, botNumber, xcm, zcm, levelSpec.y, seatRoom ~= nil)
2827         -- The bot's index WITHIN ITS OWN FLOOR (1..BOTS_PER_LEVEL). The runtime
2828         -- throttles how many workers are busy per floor using this. It used to
2829         -- throttle on the global BotNumber instead, which counts straight
2830         -- through every floor (basement 1-20, Level_1 21-40, ... Level_8
2831         -- 161-180) and was compared against a cap of at most 50 -- so every
2832         -- bot above 50 sat in permanent "Adaptive standby" and floors 3 and up
2833         -- never moved at all.
2834         if botModel then
2835             botModel:SetAttribute("LevelSlot", slot)
2836             if seatRoom then
2837                 seatedThisLevel = seatedThisLevel + 1
2838                 botModel:SetAttribute("Seated", true)
2839                 botModel:SetAttribute("Room", seatRoom.name)
2840                 -- Sit a third of the way in from the room's west wall, facing
2841                 -- the middle of the room. The seated pose is built facing -Z,
2842                 -- which is the model's LookVector, so lookAt orients it.
2843                 local seatHeight = Y(levelSpec.y) + 2.45
2844                 local centreX = (seatRoom.x1 + seatRoom.x2) / 2
2845                 local centreZ = (seatRoom.z1 + seatRoom.z2) / 2
2846                 local seatX = seatRoom.x1 + (seatRoom.x2 - seatRoom.x1) * 0.3
2847                 local seatCF = CFrame.lookAt(
2848                     Vector3.new(X(seatX), seatHeight, X(centreZ)),
2849                     Vector3.new(X(centreX), seatHeight, X(centreZ)))
2850                 botModel:PivotTo(seatCF)
2851                 buildWorkstation(workerFolder, seatCF, botModel.Name)
2852             end
2853         end
2854     end
2855 end
2856
2857 -- Hunt bookkeeping: how many targets exist, and how many are down. The HUD
2858 -- and the kill feed both read these off the model.
2859 root:SetAttribute("BotsTotal", botNumber)
2860 root:SetAttribute("BotsKilled", 0)
2861
2862 -- Wall-mounted QA board: the autonomous workers update this during play,
2863 -- making the scenario test visible instead of hiding it in server output.
2864 -- Mounted on the CORRIDOR wall (never inside a room), well above any door's
2865 -- head height, so equipment placed against a room's own walls can never end
2866 -- up in front of it.
2867 local botTestBoard = newPart("BotTestBoard", Vector3.new(X(6), Y(70), X(220)),
2868     CFrame.new(X(CORR_X1 + 4), Y(250), X(520)), Color3.fromRGB(20, 28, 32), Enum.Material.Metal, root)
2869 botTestBoard.CanCollide = false
2870 CollectionService:AddTag(botTestBoard, "BotTestBoard")
2871 for _, face in ipairs({Enum.NormalId.Left, Enum.NormalId.Right}) do
2872     local gui = Instance.new("SurfaceGui")
2873     gui.Face = face
2874     gui.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2875     gui.PixelsPerStud = 60
2876     gui.AlwaysOnTop = false
2877     gui.Parent = botTestBoard
2878     local label = Instance.new("TextLabel")
2879     label.Name = "BotTestStatus"
2880     label.Size = UDim2.fromScale(1, 1)
2881     label.BackgroundColor3 = Color3.fromRGB(12, 24, 28)

```

```

2882     label.BackgroundTransparency = 0.08
2883     label.BorderSizePixel = 0
2884     label.TextColor3 = Color3.fromRGB(120, 235, 170)
2885     label.TextScaled = true
2886     label.TextWrapped = true
2887     label.Font = Enum.Font.Code
2888     label.Text = "NPC GAMEPLAY TESTS\n" .. botNumber .. " workers ready\nCycles passed: 0\nPlants in transit:
    ↪ 0"
2889     label.Parent = gui
2890 end
2891
2892 -- roof, punched for the shafts
2893 local topY = TOP_LEVEL * F2F + WALL_H + SLAB
2894 slabWithHoles(root, "Roof", -EXT_T, -EXT_T, W + EXT_T, L + EXT_T, topY, SLAB, ELEV_HOLES, C.roof)
2895 wall(root, "Parapet_W", 0, -EXT_T, 0, L + EXT_T, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2896 wall(root, "Parapet_E", W, -EXT_T, W, L + EXT_T, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2897 wall(root, "Parapet_N", -EXT_T, 0, W + EXT_T, 0, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2898 wall(root, "Parapet_S", -EXT_T, L, W + EXT_T, L, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2899
2900 buildStairTower(BOTTOM_LEVEL, TOP_LEVEL, false)
2901 buildStairTower(BOTTOM_LEVEL, TOP_LEVEL, true)
2902 -- Runtime route guidance uses these exact stair-lane coordinates. Keeping
2903 -- them on the generated model avoids hard-coded world positions inside the
2904 -- server scripts and guarantees an upper-floor route follows both flights.
2905 root:SetAttribute("FloorRiseStuds", Y(F2F))
2906 root:SetAttribute("StairWestLaneX", X((TX1 + 10 + TXM) / 2))
2907 root:SetAttribute("StairEastLaneX", X((TXM + TX2 - 10) / 2))
2908 -- These describe the north tower's switchback specifically (the runtime's
2909 -- manual floor-by-floor fallback route always walks through that one
2910 -- tower) -- same numeric values buildStairTower(..., false) itself used.
2911 root:SetAttribute("StairNorthZ", X(-100))
2912 root:SetAttribute("StairSouthZ", X(-400))
2913 -- So the fallback router (below) can clamp to however many floors this
2914 -- building actually has instead of a hard-coded 3-floors-plus-basement
2915 -- assumption from when there were only 3 floors.
2916 root:SetAttribute("FloorMinLevel", BOTTOM_LEVEL)
2917 root:SetAttribute("FloorMaxLevel", TOP_LEVEL)
2918 for _, spec in ipairs(ELEVATORS) do buildElevator(spec, BOTTOM_LEVEL, TOP_LEVEL) end
2919
2920 slab(root, "Apron_East", W, 100, W + 400, L * 0.94, 0, 20, C.slab)
2921 slab(root, "Apron_West", -400, 100, 0, L * 0.94, 0, 20, C.slab)
2922 slab(root, "Apron_South", 60, L, W * 0.9, L + 350, 0, 20, C.slab)
2923
2924 local pad = 700
2925 slab(root, "SitePad", -pad, -pad - 500, W + pad, L + pad, -2, 30,
2926     Color3.fromRGB(122, 136, 112), Enum.Material.Grass)
2927
2928 =====
2929 -- SITE WORKS
2930 --
2931 -- Regenerated from W and L every build, so it always fits the building
2932 -- that was just generated. Everything here stays strictly OUTSIDE the
2933 -- footprint x[0..W] z[0..L] -- the only things south of the building are
2934 -- the forecourt and entrance canopy, which belong there.
2935 =====
2936 do
2937     local site = Instance.new("Folder")
2938     site.Name = "SiteWorks"
2939     site.Parent = root
2940
2941     local rng = newRng(RNG_SEED + 4242)
2942     local FENCE_X1, FENCE_X2 = -pad + 90, W + pad - 90
2943     local FENCE_Z1, FENCE_Z2 = -pad - 400, L + pad - 90
2944     local fenceGreen = Color3.fromRGB(46, 104, 58)
2945     local GATE_X1, GATE_X2 = 460, 740 -- lines up with the south entrance
2946
2947     -- perimeter fence, with a gap on the south run for the gate
2948     wall(site, "Fence_S", FENCE_X1, FENCE_Z2, FENCE_X2, FENCE_Z2, 14, 0, 190,
2949         {{at = GATE_X1 - FENCE_X1, width = GATE_X2 - GATE_X1, head = 190}}, fenceGreen, Enum.Material.Metal)
2950     wall(site, "Fence_N", FENCE_X1, FENCE_Z1, FENCE_X2, FENCE_Z1, 14, 0, 190, {}, fenceGreen,
2951         ↪ Enum.Material.Metal)
2952     wall(site, "Fence_W", FENCE_X1, FENCE_Z1, FENCE_X1, FENCE_Z2, 14, 0, 190, {}, fenceGreen,
2953         ↪ Enum.Material.Metal)
2954     wall(site, "Fence_E", FENCE_X2, FENCE_Z1, FENCE_X2, FENCE_Z2, 14, 0, 190, {}, fenceGreen,
2955         ↪ Enum.Material.Metal)
2956
2957
2958

```

```

2954 -- barred gate across the opening
2955 for i = 0, 11 do
2956     local gx = GATE_X1 + (GATE_X2 - GATE_X1) * i / 11
2957     newPart("GateBar_" .. i, Vector3.new(X(9), Y(180), X(9)),
2958         CFrame.new(X(gx), Y(90), X(FENCE_Z2)), Color3.fromRGB(28, 34, 30), Enum.Material.Metal, site)
2959 end
2960 newPart("GateRail", Vector3.new(X(GATE_X2 - GATE_X1), Y(12), X(12)),
2961     CFrame.new(X((GATE_X1 + GATE_X2) / 2), Y(174), X(FENCE_Z2)), Color3.fromRGB(28, 34, 30),
2962     ↪ Enum.Material.Metal, site)
2963
2964 -- entrance canopy, on posts, over the south doors
2965 slab(site, "Canopy", 450, L, 750, L + 210, 270, 16, Color3.fromRGB(206, 210, 214), Enum.Material.Metal)
2966 for _, cx in ipairs({465, 735}) do
2967     newPart("CanopyPost", Vector3.new(X(16), Y(254), X(16)),
2968         CFrame.new(X(cx), Y(127), X(L + 195)), Color3.fromRGB(150, 155, 160), Enum.Material.Metal, site)
2969 end
2970
2971 -- planters either side of the door
2972 for i, px in ipairs({390, 810}) do
2973     newPart("Planter_" .. i, Vector3.new(X(120), Y(70), X(120)),
2974         CFrame.new(X(px), Y(35), X(L + 90)), Color3.fromRGB(176, 168, 150), Enum.Material.Concrete, site)
2975     newPart("PlanterSoil_" .. i, Vector3.new(X(104), Y(16), X(104)),
2976         CFrame.new(X(px), Y(72), X(L + 90)), Color3.fromRGB(84, 66, 48), Enum.Material.Ground, site)
2977     for f = 1, 4 do
2978         newPart("PlanterShrub_" .. i .. "_" .. f, Vector3.new(X(34), Y(46), X(34)),
2979             CFrame.new(X(px + rng.int(-30, 30)), Y(100), X(L + 90 + rng.int(-30, 30))),
2980             Color3.fromRGB(64, 122, 62), Enum.Material.Grass, site)
2981     end
2982 end
2983
2984 -- flagpole
2985 newPart("FlagPole", Vector3.new(X(10), Y(560), X(10)),
2986     CFrame.new(X(900), Y(280), X(L + 210)), Color3.fromRGB(214, 216, 220), Enum.Material.Metal, site)
2987 newPart("Flag", Vector3.new(X(4), Y(90), X(150)),
2988     CFrame.new(X(900), Y(500), X(L + 285)), Color3.fromRGB(200, 208, 216), Enum.Material.Fabric, site)
2989
2990 -- parking bays and cars on the south forecourt
2991 local function buildCar(name, cx, cz, colour)
2992     newPart(name .. "_Body", Vector3.new(X(170), Y(80), X(360)),
2993         CFrame.new(X(cx), Y(60), X(cz)), colour, Enum.Material.Metal, site)
2994     newPart(name .. "_Cabin", Vector3.new(X(150), Y(60), X(170)),
2995         CFrame.new(X(cx), Y(126), X(cz + 10)), Color3.fromRGB(80, 96, 112), Enum.Material.Glass, site)
2996     for _, wx in ipairs({-80, 80}) do
2997         for _, wz in ipairs({-115, 115}) do
2998             local wheel = newPart(name .. "_Wheel", Vector3.new(X(26), Y(58), X(58)),
2999                 CFrame.new(X(cx + wx), Y(28), X(cz + wz)), Color3.fromRGB(30, 30, 32),
3000                 ↪ Enum.Material.Rubber, site)
3001             wheel.Shape = Enum.PartType.Cylinder
3002         end
3003     end
3004 end
3005
3006 local carColours = {
3007     Color3.fromRGB(180, 60, 55), Color3.fromRGB(60, 80, 150),
3008     Color3.fromRGB(220, 220, 224), Color3.fromRGB(70, 74, 80),
3009 }
3010
3011 for bay = 0, 5 do
3012     local bx = 700 + bay * 80
3013     newPart("BayLine_" .. bay, Vector3.new(X(6), Y(4), X(200)),
3014         CFrame.new(X(bx), Y(2), X(L + 220)), Color3.fromRGB(238, 238, 232), Enum.Material.SmoothPlastic,
3015         ↪ site)
3016     if bay < 4 then
3017         buildCar("Car_" .. bay, bx + 40, L + 215, carColours[bay + 1])
3018     end
3019 end
3020
3021 -- greenhouses on the west apron
3022 for g = 1, 3 do
3023     local gz = 260 + (g - 1) * 620
3024     slab(site, "GreenhouseBase_" .. g, -370, gz, -70, gz + 460, 4, 12, Color3.fromRGB(150, 146, 138),
3025         ↪ Enum.Material.Concrete)
3026     for _, corner in ipairs({{-370, gz}, {-70, gz}, {-370, gz + 460}, {-70, gz + 460}}) do
3027         newPart("GreenhouseFrame_" .. g, Vector3.new(X(14), Y(230), X(14)),
3028             CFrame.new(X(corner[1]), Y(115), X(corner[2])), Color3.fromRGB(196, 200, 204),
3029             ↪ Enum.Material.Metal, site)
3030     end
3031     local glassSides = {

```



```

3025     {-370, gz, -70, gz}, {-370, gz + 460, -70, gz + 460},
3026     {-370, gz, -370, gz + 460}, {-70, gz, -70, gz + 460},
3027 }
3028 for si, side in ipairs(glassSides) do
3029     local gl = wall(site, "GreenhouseGlass_" .. g .. "_" .. si,
3030         side[1], side[2], side[3], side[4], 8, 0, 210, {}, Color3.fromRGB(186, 216, 224),
3031         Enum.Material.Glass)
3032 end
3033 local roof = slab(site, "GreenhouseRoof_" .. g, -370, gz, -70, gz + 460, 240, 10,
3034     Color3.fromRGB(196, 222, 230), Enum.Material.Glass)
3035 roof.Transparency = 0.5
3036 -- crop benches inside
3037 for b = 0, 2 do
3038     newPart("GreenhouseBench_" .. g .. "_" .. b, Vector3.new(X(70), Y(20), X(400)),
3039         CFrame.new(X(-330 + b * 100), Y(80), X(gz + 230)), Color3.fromRGB(96, 138, 84),
3040         Enum.Material.Grass, site)
3041 end
3042 -- trial crop plots on the east apron
3043 for plot = 0, 1 do
3044     local px = W + 90 + plot * 170
3045     slab(site, "TrialPlot_" .. plot, px, 220, px + 140, L * 0.9, 6, 14,
3046         Color3.fromRGB(96, 74, 54), Enum.Material.Ground)
3047     for row = 0, 9 do
3048         newPart("CropRow_" .. plot .. "_" .. row, Vector3.new(X(120), Y(26), X(70)),
3049             CFrame.new(X(px + 70), Y(20), X(300 + row * 160)),
3050             Color3.fromRGB(84, 142, 70), Enum.Material.Grass, site)
3051     end
3052 end
3053
3054 -- street trees, kept out of the building, the aprons and the forecourt
3055 local function buildTree(name, tx, tz)
3056     newPart(name .. "_Trunk", Vector3.new(X(30), Y(240), X(30)),
3057         CFrame.new(X(tx), Y(120), X(tz)), Color3.fromRGB(94, 68, 46), Enum.Material.Wood, site)
3058     for c = 1, 3 do
3059         local crown = newPart(name .. "_Crown", Vector3.new(X(150 - c * 22), Y(150 - c * 22), X(150 - c *
3060             22)),
3061             CFrame.new(X(tx + rng.int(-20, 20)), Y(250 + c * 42), X(tz + rng.int(-20, 20))),
3062             Color3.fromRGB(52, 104 + c * 8, 50), Enum.Material.Grass, site)
3063         crown.Shape = Enum.PartType.Ball
3064     end
3065 end
3066 local n = 0
3067 for i = 0, 7 do -- west strip
3068     n = n + 1; buildTree("Tree_" .. n, -560 + rng.int(-60, 60), -200 + i * 330)
3069 end
3070 for i = 0, 7 do -- east strip
3071     n = n + 1; buildTree("Tree_" .. n, W + 560 + rng.int(-60, 60), -200 + i * 330)
3072 end
3073 for i = 0, 5 do -- north band, in front of the building
3074     n = n + 1; buildTree("Tree_" .. n, -300 + i * 380, -560 + rng.int(-80, 80))
3075 end
3076 for i = 0, 5 do -- south band, beyond the forecourt
3077     n = n + 1; buildTree("Tree_" .. n, -300 + i * 380, L + 520 + rng.int(-60, 60))
3078 end
3079 end
3080
3081 --=====
3082 -- OFFICE FIT-OUT
3083 -- A desk and a shelf in each room, placed from the room rectangle so they
3084 -- sit in the clear middle of the floor rather than fighting the equipment
3085 -- packed against the walls.
3086 --=====
3087 do
3088     local office = Instance.new("Folder")
3089     office.Name = "OfficeFitout"
3090     office.Parent = root
3091
3092     local function fitOutOffice(rooms, levelY, levelName)
3093         for index, r in ipairs(rooms) do
3094             local width, depth = r.x2 - r.x1, r.z2 - r.z1
3095             if width > 200 and depth > 200 then
3096                 local seatHeight = Y(levelY) + 2.45
3097                 -- desk, facing across the room
3098                 local deskX = r.x1 + width * 0.62

```

```

3098         local deskZ = r.z1 + depth * 0.34
3099         local cf = CFrame.lookAt(
3100             Vector3.new(X(deskX), seatHeight, X(deskZ)),
3101             Vector3.new(X(r.x1 + width * 0.5), seatHeight, X(deskZ)))
3102         buildWorkstation(office, cf, levelName .. "_" .. index)
3103
3104         -- a shelf unit against the clear part of the floor
3105         local shelfX = r.x1 + width * 0.62
3106         local shelfZ = r.z1 + depth * 0.72
3107         newPart("Shelf_" .. levelName .. "_" .. index, Vector3.new(E(90), EH(180), E(38)),
3108             CFrame.new(X(shelfX), Y(levelY) + EH(90), X(shelfZ)),
3109             Color3.fromRGB(150, 142, 128), Enum.Material.WoodPlanks, office).CanCollide = false
3110         for shelfLevel = 1, 3 do
3111             newPart("ShelfBooks_" .. levelName .. "_" .. index .. "_" .. shelfLevel,
3112                 Vector3.new(E(78), EH(26), E(28)),
3113                 CFrame.new(X(shelfX), Y(levelY) + EH(38 + shelfLevel * 42), X(shelfZ)),
3114                 Color3.fromRGB(120 + shelfLevel * 22, 90, 70), Enum.Material.SmoothPlastic,
3115                 office).CanCollide = false
3116         end
3117     end
3118 end
3119
3120 fitOutOffice(basementRooms, BOTTOM_LEVEL * F2F, "Basement")
3121 for _, lv in ipairs(LEVELS) do
3122     fitOutOffice(lv.rooms, lv.index * F2F, lv.name)
3123 end
3124 end
3125
3126 local oldSpawn = Workspace:FindFirstChild("LabSpawn")
3127 if oldSpawn then oldSpawn:Destroy() end
3128 -- neutral start: right inside the entrance, facing the three role
3129 -- kiosks, so choosing a role is the very first thing a new player does
3130 local sp = Instance.new("SpawnLocation")
3131 sp.Name = "LabSpawn"
3132 sp.Size = Vector3.new(X(360), Y(30), X(120))
3133 -- SpawnLocation's default facing is -Z; the kiosks sit at a HIGHER z
3134 -- than the spawn, so without a 180-degree turn a new player spawns
3135 -- with their back to them and never sees the panels. Face +Z instead.
3136 sp.CFrame = CFrame.new(X(GROUND_ROOMS[1].x1 + 250), Y(20), X(90)) * CFrame.Angles(0, math.rad(180), 0)
3137 sp.Anchored = true
3138 sp.Color = Color3.fromRGB(120, 170, 120)
3139 sp.Material = Enum.Material.Concrete
3140 sp.Parent = Workspace
3141
3142 =====
3143 -- TEAMS AND ROLE STATIONS
3144 -- Two competitive gameplay roles: Technicians repair equipment, while
3145 -- Laboratory Technicians move and process germinated plant samples.
3146 =====
3147 local Teams = game:GetService("Teams")
3148
3149 local ROLES = {
3150     {name = "Technician", team = BrickColor.new("Bright blue"), paint = Color3.fromRGB(60, 120, 200)},
3151     {name = "Laboratory Technician", team = BrickColor.new("Lime green"), paint = Color3.fromRGB(90, 170, 90)},
3152 }
3153
3154 for _, obsoleteName in ipairs({"Inspector", "Supervisor"}) do
3155     local obsolete = Teams:FindFirstChild(obsoleteName)
3156     if obsolete then obsolete:Destroy() end
3157 end
3158
3159 for _, role in ipairs(ROLES) do
3160     local old = Teams:FindFirstChild(role.name)
3161     if old then old:Destroy() end
3162     local t = Instance.new("Team")
3163     t.Name = role.name
3164     t.TeamColor = role.team
3165     t.AutoAssignable = false
3166     t.Parent = Teams
3167 end
3168
3169 do
3170     -- everything below lives in the long west room of the ground floor
3171     local lobby = GROUND_ROOMS[1]
3172     local y0 = 0

```

```

3173     local colX = {lobby.x1 + 130, lobby.x1 + 250, lobby.x1 + 370}
3174
3175     local function prop(name, xcm, zcm, w, d, h, colour, shape)
3176         local isEquipmentCabinet = (shape or "box") ~= "rack"
3177         if isEquipmentCabinet then w, d, h = 96, 74, 205 end
3178         local size = Vector3.new(E(w), EH(h), E(d))
3179         local pos = Vector3.new(X(xcm), Y(y0) + size.Y / 2, X(zcm))
3180         local entry = {kind = name, col = colour, mat = Enum.Material.Metal,
3181             shape = isEquipmentCabinet and "appliance" or (shape or "box"), plantCabinet = isEquipmentCabinet,
3182             ↪ capacity = 8}
3183         local p = buildKind(root, entry, pos, size)
3184         p.Name = name:gsub("%s", "")
3185         if isEquipmentCabinet then
3186             local sampleCount = 4
3187             local sampleIds = {}
3188             for i = 1, sampleCount do table.insert(sampleIds, string.format("%s-P%02d", p.Name:sub(1,
3189                 ↪ 6):upper(), i)) end
3190             p:SetAttribute("Capacity", entry.capacity)
3191             p:SetAttribute("SampleCount", sampleCount)
3192             p:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
3193             populateEquipmentCabinet(p, entry, size, nil, sampleCount, entry.capacity)
3194         end
3195         return p
3196     end
3197
3198     -- a glowing floor chevron pointing from a kiosk toward its station
3199     local function chevron(xcm, zcm, colour)
3200         local w, len, thick = 60, 40, 7
3201         for _, side in ipairs({-1, 1}) do
3202             local arm = newPart("ArrowMark", Vector3.new(X(thick), Y(4), X(len)),
3203                 CFrame.new(X(xcm + side * w / 4), Y(y0) + Y(2), X(zcm)) * CFrame.Angles(0, math.rad(side * 35),
3204                 ↪ 0),
3205                 colour, Enum.Material.Neon, root)
3206             arm.CanCollide = false
3207             arm.CastShadow = false
3208         end
3209     end
3210
3211     -- role panels: two big, unmistakable glowing panels right in
3212     -- front of the spawn point -- touch one to become that role, which
3213     -- immediately changes your character's appearance.
3214     for i, role in ipairs(ROLES) do
3215         local panelSize = Vector3.new(E(100), EH(190), E(14))
3216         local panelPos = Vector3.new(X(colX[i]), Y(y0) + panelSize.Y / 2, X(220))
3217         local frame = newPart(role.name .. "_PanelFrame", panelSize,
3218             CFrame.new(panelPos), Color3.fromRGB(30, 32, 36), Enum.Material.Metal, root)
3219         frame.CanCollide = true
3220         local face = newPart(role.name .. "_PanelFace", Vector3.new(panelSize.X * 0.86, panelSize.Y * 0.82,
3221             ↪ panelSize.Z * 0.3),
3222             CFrame.new(panelPos.X, panelPos.Y + panelSize.Y * 0.04, panelPos.Z + panelSize.Z / 2 * 0.9),
3223             role.paint, Enum.Material.Neon, root)
3224         face.CanCollide = false
3225         face.CastShadow = false
3226         local p = frame
3227         p:SetAttribute("Role", role.name)
3228         CollectionService:AddTag(p, "RoleKiosk")
3229
3230         -- Printed directly on the panel instead of floating in the room.
3231         -- Front + back faces keep it readable regardless of panel direction.
3232         for _, normal in ipairs({Enum.NormalId.Front, Enum.NormalId.Back}) do
3233             local surface = Instance.new("SurfaceGui")
3234             surface.Name = "RoleLabel"
3235             surface.Face = normal
3236             surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
3237             surface.PixelsPerStud = 55
3238             surface.AlwaysOnTop = false
3239             surface.Parent = face
3240             local lbl = Instance.new("TextLabel")
3241             lbl.Size = UDim2.fromScale(1, 1)
3242             lbl.BackgroundTransparency = 1
3243             lbl.Text = role.name:upper() .. "\nPRESS E"
3244             lbl.TextColor3 = Color3.fromRGB(255, 255, 255)
3245             lbl.TextStrokeTransparency = 0.55
3246             lbl.TextScaled = true
3247             lbl.Font = Enum.Font.GothamBold
3248             lbl.Parent = surface

```

```

3245     end
3246
3247     local oldSp = Workspace:FindFirstChild(role.name .. "Spawn")
3248     if oldSp then oldSp:Destroy() end
3249     local rsp = Instance.new("SpawnLocation")
3250     rsp.Name = role.name .. "Spawn"
3251     rsp.Neutral = false
3252     rsp.TeamColor = role.team
3253     rsp.Size = Vector3.new(X(140), Y(20), X(140))
3254     rsp.CFrame = CFrame.new(X(colX[i]), Y(10), X(340))
3255     rsp.Anchored = true
3256     rsp.CanCollide = true
3257     rsp.Color = role.paint
3258     rsp.Material = Enum.Material.SmoothPlastic
3259     rsp.Parent = Workspace
3260
3261     -- a trail of chevrons from this kiosk's lane down to its station
3262     for _, zcm in ipairs({300, 360, 420}) do
3263         chevron(colX[i], zcm, role.paint)
3264     end
3265 end
3266
3267 -- scoreboard, updated live by LabEquipment on every station use
3268 local board = newPart("BatchScoreboard", Vector3.new(X(10), Y(150), X(280)),
3269     CFrame.new(X(lobby.x1 + 250), Y(220), X(650)), Color3.fromRGB(30, 32, 36), Enum.Material.Metal, root)
3270 board.CanCollide = false
3271 CollectionService:AddTag(board, "BatchScoreboard")
3272 local bb = Instance.new("BillboardGui")
3273 bb.Size = UDim2.new(0, 380, 0, 110)
3274 bb.MaxDistance = 18
3275 bb.AlwaysOnTop = false
3276 bb.Parent = board
3277 local scoreText = Instance.new("TextLabel")
3278 scoreText.Name = "Text"
3279 scoreText.Size = UDim2.new(1, 0, 1, 0)
3280 scoreText.BackgroundTransparency = 1
3281 scoreText.TextColor3 = Color3.fromRGB(255, 240, 210)
3282 scoreText.TextScaled = true
3283 scoreText.Font = Enum.Font.GothamBold
3284 scoreText.Text = "PLAYER TEAM SCORE\nRepairs: 0 • assigned: 0\nPlant moves: 0 • assigned: 0\nRival NPC
↪ score: 0"
3285 scoreText.Parent = bb
3286
3287 -- a wall-mounted "how to play" board, right by the entrance
3288 local howBoard = newPart("HowToPlay", Vector3.new(X(6), Y(200), X(320)),
3289     CFrame.new(X(EXT_T + 4), Y(200), X(150)), Color3.fromRGB(24, 26, 30), Enum.Material.Metal, root)
3290 howBoard.CanCollide = false
3291 local howBb = Instance.new("BillboardGui")
3292 howBb.Size = UDim2.new(0, 340, 0, 150)
3293 howBb.MaxDistance = 14
3294 howBb.AlwaysOnTop = false
3295 howBb.Parent = howBoard
3296 local howText = Instance.new("TextLabel")
3297 howText.Size = UDim2.new(1, 0, 1, 0)
3298 howText.BackgroundTransparency = 1
3299 howText.TextColor3 = Color3.fromRGB(235, 238, 242)
3300 howText.TextScaled = true
3301 howText.TextXAlignment = Enum.TextXAlignment.Left
3302 howText.TextWrapped = true
3303 howText.Font = Enum.Font.Gotham
3304 howText.Text = table.concat({
3305     "HOW TO PLAY",
3306     "1. BLUE Technician: follow RED floor lights and HOLD R at the failed unit.",
3307     "2. Repair points grow with failure time and assigned travel distance.",
3308     "3. GREEN Laboratory Technician: E takes a plant; E at a working chamber drops it.",
3309     "4. Rescue damaged plants and wait 3-7 seconds for LIFE to return to 100%.",
3310 }, "\n")
3311 howText.Parent = howBb
3312
3313 -- SECRET BASEMENT ENTRANCE: a disguised maintenance vent flush with the
3314 -- lobby's own exterior wall, finished in the same concrete as the wall
3315 -- around it so it reads as part of the wall, not a door. It sits inside
3316 -- the kiosk area's reserved buffer zone (z 150-460 on this room's W/E
3317 -- walls, blocked off from the normal equipment fit-out earlier), so it
3318 -- can never end up buried behind an equipment cabinet. A nearby sign
3319 -- carries the actual instructions -- this is meant to be found, just

```

```

3320 -- not obvious at a glance.
3321 local secretPanel = newPart("MaintenanceVentPanel", Vector3.new(X(6), Y(140), X(90)),
3322     CFrame.new(X(EXT_T + 3), Y(y0) + Y(95), X(400)), C.ext, Enum.Material.Concrete, root)
3323 secretPanel.CanCollide = false
3324 CollectionService:AddTag(secretPanel, "SecretBasementPanel")
3325 local grille = newPart("VentGrille", Vector3.new(X(2), Y(60), X(60)),
3326     CFrame.new(X(EXT_T + 6), Y(y0) + Y(95), X(400)), Color3.fromRGB(70, 74, 78),
3327     ↳ Enum.Material.DiamondPlate, root)
3328 grille.CanCollide = false
3329 grille.CastShadow = false
3330 local secretPrompt = Instance.new("ProximityPrompt")
3331 secretPrompt.ObjectText = "Maintenance Vent"
3332 secretPrompt.ActionText = "Pry open"
3333 secretPrompt.KeyboardKeyCode = Enum.KeyCode.E
3334 secretPrompt.HoldDuration = 1.5
3335 secretPrompt.MaxActivationDistance = 8
3336 secretPrompt.RequiresLineOfSight = false
3337 secretPrompt.Parent = secretPanel
3338 -- Drop point: open basement corridor floor, well clear of any equipment.
3339 secretPanel:SetAttribute("SecretDestX", X((CORR_X1 + CORR_X2) / 2))
3340 secretPanel:SetAttribute("SecretDestY", Y(BOTTOM_LEVEL * F2F) + 3.5)
3341 secretPanel:SetAttribute("SecretDestZ", X(200))
3342 sign(root, "MAINTENANCE ACCESS\nA basement vent panel is set into the west wall\nnear the role kiosks. Hold
3343     ↳ E on it to pry it open.",
3344     EXT_T + 55, 400, y0, 260, 20)
3345 end
3346
3347 --=====
3348 -- RUNTIME SCRIPTS
3349 --=====
3350 local FLOW_SOURCE = {[==[
3351 -- LabKinds : what each kind of basic laboratory equipment does.
3352 -- Keyed by the Kind attribute the generator writes on every item, so
3353 -- any number of copies anywhere in the building behave the same.
3354
3355 local K = {}
3356
3357 K.Items = {
3358     TRAY    = "Sample Tray",
3359     GROUND  = "Ground Sample",
3360     DRIED   = "Dried Sample",
3361     SPUN    = "Separated Sample",
3362     CULTURE = "Culture Plate",
3363     WASTE   = "Used Material",
3364     CLEAN   = "Clean Glassware",
3365 }
3366 local I = K.Items
3367
3368 local function f(n, dp) return string.format("%. " .. dp .. "f", n) end
3369 local function rnd(a, b) return a + math.random() * (b - a) end
3370
3371 K.Actions = {
3372     ["Workbench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},
3373         log = function() return "Sample laid out on the bench." end},
3374     ["Island Bench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},
3375         log = function() return "Sample laid out on the island bench." end},
3376     ["Base Cabinet Run"] = {action = "Open the cabinet", wait = 0.8,
3377         log = function() return "Consumables drawn from the cabinet." end},
3378     ["Desk"] = {action = "Write up the record", wait = 1.2,
3379         log = function() return "Notes entered in the log book." end},
3380
3381     ["Sink Unit"] = {action = "Wash glassware", wait = 2.0, produces = {I.CLEAN},
3382         log = function() return "Glassware rinsed and set to drain." end},
3383     ["Fume Hood"] = {action = "Work under extraction", wait = 2.5, uvLight = true,
3384         log = function() return "Extraction running at " .. f(rnd(0.4, 0.6), 2) .. " m/s face velocity." end},
3385
3386     ["Laboratory Mill"] = {action = "Grind the sample", wait = 2.0,
3387         consumes = I.TRAY, produces = {I.GROUND}, activeColour = Color3.fromRGB(200, 160, 60),
3388         log = function() return "Ground for two minutes; chamber cleaned afterwards." end},
3389     ["Sieve Shaker"] = {action = "Sieve the sample", wait = 2.0,
3390         log = function() return f(rnd(45, 80), 0) .. "% passed the working mesh." end},
3391     ["Drying Oven"] = {action = "Dry the sample", wait = 3.5,
3392         consumes = I.TRAY, produces = {I.DRIED}, activeColour = Color3.fromRGB(200, 60, 50),
3393         log = function() return "Oven holding " .. f(rnd(101, 105), 1) .. " C." end},
3394     ["Centrifuge"] = {action = "Spin the sample", wait = 2.5,

```

```

3394     consumes = I.TRAY, produces = {I.SPUN},
3395     log = function() return f(rnd(3000, 12000), 0) .. " rpm for " .. math.random(5, 20) .. " minutes."
    ↪ end},
3396 ["Incubator"] = {action = "Incubate the sample", wait = 3.0,
3397     consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(255, 170, 0),
3398     log = function() return "Holding " .. f(rnd(19, 31), 1) .. " C; door log updated." end},
3399 ["Autoclave"] = {action = "Sterilize a load", wait = 3.0,
3400     consumes = I.WASTE, activeColour = Color3.fromRGB(0, 120, 255),
3401     log = function() return "Cycle complete at 121 C; indicator strip filed." end},
3402 ["Waste Bin"] = {action = "Discard used material", wait = 0.8, consumes = I.WASTE,
3403     log = function() return "Waste segregated for removal." end},
3404
3405 ["Analytical Balance"] = {action = "Weigh the sample", wait = 1.5,
3406     log = function()
3407         local w = rnd(0.4, 25)
3408         return "Reading " .. f(w, w < 1 and 4 or 3) .. " g; balance levelled and zeroed."
3409     end},
3410 ["Microscope"] = {action = "Examine the sample", wait = 1.5,
3411     log = function() return "Examined at " .. math.random(1, 4) * 10 .. "x." end},
3412 ["pH Meter"] = {action = "Measure pH", wait = 1.2,
3413     log = function() return "pH " .. f(rnd(4.5, 9.0), 2) .. " at " .. f(rnd(19, 23), 1) .. " C." end},
3414 ["Water Bath"] = {action = "Warm the sample", wait = 1.5,
3415     log = function() return "Bath steady at " .. f(rnd(30, 60), 1) .. " C." end},
3416 ["Hot Plate Stirrer"] = {action = "Stir and heat", wait = 1.5,
3417     log = function() return "Stirring at " .. math.random(200, 900) .. " rpm." end},
3418 ["Orbital Shaker"] = {action = "Shake the sample", wait = 1.5,
3419     log = function() return "Shaking at " .. math.random(80, 260) .. " rpm." end},
3420 ["Desiccator"] = {action = "Cool the sample", wait = 1.8,
3421     consumes = I.DRIED, produces = {I.WASTE},
3422     log = function() return "Cooled over dry desiccant before the second weighing." end},
3423 ["Computer Terminal"] = {action = "Enter the result", wait = 1.2,
3424     log = function() return "Result recorded against the sample number." end},
3425
3426 ["Refrigerator"] = {action = "Load / unload sample", wait = 1.2,
3427     log = function() return "Held at " .. f(rnd(2, 8), 1) .. " C." end},
3428 ["Chest Freezer"] = {action = "Load / unload sample", wait = 1.2,
3429     log = function() return "Held at " .. f(rnd(-25, -16), 0) .. " C." end},
3430 ["Sample Cabinet"] = {action = "File the sample", wait = 1.0,
3431     log = function() return "Filed by sample number." end},
3432 ["Shelving Unit"] = {action = "Draw stock", wait = 0.8,
3433     log = function() return "Stock drawn; reorder level checked." end},
3434 ["Storage Rack"] = {action = "Draw stock", wait = 0.8,
3435     log = function() return "Stock drawn from the rack." end},
3436 ["Glassware Rack"] = {action = "Take clean glassware", wait = 0.8, produces = {I.CLEAN},
3437     log = function() return "Clean glassware taken from the rack." end},
3438 ["Trolley"] = {action = "Load the trolley", wait = 0.8,
3439     log = function() return "Trolley loaded for transfer." end},
3440 ["Filing Cabinet"] = {action = "Pull a file", wait = 0.8,
3441     log = function() return "File retrieved." end},
3442 ["Locker Bank"] = {action = "Change clothing", wait = 1.2,
3443     log = function() return "Changed before entering the working area." end},
3444 ["Gas Cylinder Rack"] = {action = "Check the cylinders", wait = 1.0,
3445     log = function() return "Cylinders chained; regulator pressure " .. math.random(20, 200) .. " bar."
    ↪ end},
3446 ["Reagent Rack"] = {action = "Take a reagent", wait = 0.8,
3447     log = function() return "Reagent taken; bottle logged back in." end},
3448 ["Test Tube Rack"] = {action = "Rack the tubes", wait = 0.8,
3449     log = function() return "Tubes racked and labelled." end},
3450 ["Pipette Stand"] = {action = "Take a pipette", wait = 0.8,
3451     log = function() return "Pipette taken; last service date in date." end},
3452
3453 ["Germination Chamber"] = {action = "Load a germination test", wait = 2.5,
3454     consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(120, 200, 140),
3455     log = function() return "Test set at " .. f(rnd(18, 25), 1) .. " C, day count started." end},
3456 ["Sample Divider"] = {action = "Divide the sample", wait = 1.8,
3457     consumes = I.GROUND, produces = {I.TRAY},
3458     log = function() return "Split into equal working portions." end},
3459 ["Decontamination Unit"] = {action = "Run decontamination", wait = 2.2, consumes = I.WASTE,
3460     activeColour = Color3.fromRGB(150, 210, 255),
3461     log = function() return "Cycle complete; surfaces clear." end},
3462 ["Water Distiller"] = {action = "Draw distilled water", wait = 1.2,
3463     log = function() return "Conductivity " .. f(rnd(0.5, 2.5), 1) .. " uS/cm." end},
3464 ["Reference Sample Archive"] = {action = "File a reference sample", wait = 1.0,
3465     log = function() return "Filed against the reference register." end},
3466 ["Optical Analyzer"] = {action = "Run an optical reading", wait = 1.8,
3467     log = function() return "Reading logged at " .. math.random(400, 800) .. " nm." end},

```

```

3468     ["Electrophoresis Unit"] = {action = "Run a separation", wait = 2.0, activeColour = Color3.fromRGB(0, 120,
↪ 255),
3469         log = function() return "Run complete at " .. math.random(60, 150) .. " V." end},
3470     ["Thermal Cycler"] = {action = "Run an amplification cycle", wait = 2.5, activeColour = Color3.fromRGB(255,
↪ 170, 0),
3471         log = function() return math.random(25, 40) .. " cycles complete." end},
3472     ["Homogenizer"] = {action = "Homogenize the sample", wait = 1.5,
3473         log = function() return "Sample homogenized at " .. math.random(8000, 24000) .. " rpm." end},
3474     ["Seed Counter"] = {action = "Count the sample", wait = 1.5,
3475         log = function() return math.random(200, 1200) .. " units counted." end},
3476     ["Calibration Weight Set"] = {action = "Check calibration", wait = 1.0,
3477         log = function() return "Balance checked against certified weights." end},
3478     ["Temperature Logger"] = {action = "Read the logger", wait = 0.6,
3479         log = function() return "Log downloaded; " .. f(rnd(2, 8), 1) .. " C average." end},
3480
3481     ["Plant Growth Chamber"] = {action = "Take / return plant", wait = 0.8},
3482     ["Growth Hormone Mixer"] = {action = "Apply growth hormone", wait = 2.0},
3483     ["GMO Injector"] = {action = "Inject GMO trait", wait = 2.4},
3484     ["Nutrient Infuser"] = {action = "Infuse nutrients", wait = 1.8},
3485     ["UV Growth Scanner"] = {action = "Scan plant development", wait = 1.5},
3486     ["Pollination Station"] = {action = "Pollinate the plant", wait = 2.1},
3487     ["Mutagenesis Chamber"] = {action = "Irradiate the sample", wait = 2.4,
3488         log = function() return math.random(100, 1000) .. " Gy dose logged; dosimeter reset." end},
3489     ["Hormone Treatment Bench"] = {action = "Apply hormone treatment", wait = 2.0,
3490         log = function() return "Auxin/cytokinin mix dosed and logged." end},
3491
3492     -- Cooperative role stations: LabEquipment special-cases these three
3493     -- kinds before ever reaching this table, so only the prompt flavour
3494     -- text below (action/wait) is actually used for them.
3495     ["Sample Intake"] = {action = "Log a sample", wait = 1.0},
3496     ["Inspection Station"] = {action = "Inspect the batch", wait = 1.2},
3497     ["Supervisor Console"] = {action = "Sign off the batch", wait = 1.5},
3498
3499     ["Fire Extinguisher"] = {action = "Check the extinguisher", wait = 1.0,
3500         log = function() return "Gauge in the green; inspection tag current." end},
3501     ["First Aid Cabinet"] = {action = "Check the first aid kit", wait = 1.0,
3502         log = function() return "Contents complete." end},
3503     ["Eyewash Station"] = {action = "Flush the eyewash", wait = 1.5,
3504         log = function() return "Weekly flush run; water clear." end},
3505 }
3506
3507 K.Default = {action = "Inspect", wait = 1.0,
3508     log = function() return "Checked and left in order." end}
3509
3510 function K.forKind(kind) return K.Actions[kind] or K.Default end
3511
3512 return K
3513 ]==]
3514
3515 local EQUIP_SCRIPT_SOURCE = [=[
3516 -- LabEquipment : physical controls, equipment readouts, carried samples,
3517 -- cold-store delivery missions, and non-lethal sample recovery tools.
3518 -- All inventory and hit decisions happen on the server.
3519
3520 local ReplicatedStorage = game:GetService("ReplicatedStorage")
3521 local CollectionService = game:GetService("CollectionService")
3522 local Players = game:GetService("Players")
3523 local TweenService = game:GetService("TweenService")
3524 local Debris = game:GetService("Debris")
3525 local PathfindingService = game:GetService("PathfindingService")
3526
3527 local Kinds = require(ReplicatedStorage:WaitForChild("LabKinds"))
3528 local QuizRemote = ReplicatedStorage:FindFirstChild("LabMathQuiz")
3529 if not QuizRemote then
3530     QuizRemote = Instance.new("RemoteFunction")
3531     QuizRemote.Name = "LabMathQuiz"
3532     QuizRemote.Parent = ReplicatedStorage
3533 end
3534
3535 -----
3536 -- SHARED GAME STATE
3537 -----
3538 local GameState = {
3539     repairsCompleted = 0,
3540     assignedRepairsCompleted = 0,
3541     transfersCompleted = 0,

```

```

3542     assignedTransfersCompleted = 0,
3543 }
3544
3545 local ROLE_STATIONS = {}
3546
3547 local POINTS = {
3548     MOVE_ANY = 12,
3549     MOVE_ASSIGNED_LATE = 18,
3550     MOVE_ASSIGNED_ON_TIME = 30,
3551     PROCESS_PLANT = 4,
3552 }
3553
3554 local function updateScoreboard()
3555     local board = CollectionService:GetTagged("BatchScoreboard")[1]
3556     if not board then return end
3557     local gui = board:FindFirstChildWhichIsA("BillboardGui")
3558     local label = gui and gui:FindFirstChild("Text")
3559     if label then
3560         local rivalScore = 0
3561         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
3562             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
3563         end
3564         label.Text = string.format(
3565             "PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d • assigned: %d\nRival NPC
↵ score: %d",
3566             GameState.repairsCompleted, GameState.assignedRepairsCompleted,
3567             GameState.transfersCompleted, GameState.assignedTransfersCompleted, rivalScore)
3568     end
3569 end
3570
3571 -- Per-room equipment status monitors. Each RoomMonitor panel carries a
3572 -- Room attribute matching the LabEquipment parts placed in that same room,
3573 -- so this just groups by that attribute and tallies running/failed/plants.
3574 local function refreshRoomMonitors()
3575     -- Room names ("Room 4", etc.) repeat on every level, so the key here
3576     -- must include the level too -- otherwise a monitor sums up every
3577     -- same-numbered room across the whole building instead of just its own.
3578     local totals = {}
3579     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
3580         local room = equip:GetAttribute("Room")
3581         local level = equip:GetAttribute("Level") or ""
3582         if room then
3583             local key = level .. "|" .. room
3584             local t = totals[key]
3585             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
3586             t.count = t.count + 1
3587             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
3588             else t.running = t.running + 1 end
3589             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
3590         end
3591     end
3592     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
3593         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
3594         local label = surface and surface:FindFirstChild("Status")
3595         if label then
3596             local room = panel:GetAttribute("Room") or "ROOM"
3597             local level = panel:GetAttribute("Level") or ""
3598             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
3599             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
3600                 room, t.count, t.running, t.failed, t.plants)
3601             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235, 170)
3602         end
3603     end
3604 end
3605
3606 local function addContribution(player, amount)
3607     local stats = player:FindFirstChild("leaderstats")
3608     local value = stats and stats:FindFirstChild("Contributions")
3609     if value then value.Value = value.Value + (amount or 1) end
3610 end
3611
3612 local facilityNoticeAt = setmetatable({}, {__mode = "k"})
3613 local function notify(player, text)
3614     local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
↵ text:match("^CATASTROPHIC FAILURE:")
3615     if facilityAlert then

```



```

3616         local now = os.clock()
3617         if now - (facilityNoticeAt[player] or -math.huge) < 6 then return end
3618         facilityNoticeAt[player] = now
3619     end
3620     print("[ " .. player.Name .. " ] " .. text)
3621     local gui = player:FindFirstChild("PlayerGui")
3622     if not gui then return end
3623     local screen = gui:FindFirstChild("LabLog")
3624     if not screen then
3625         screen = Instance.new("ScreenGui")
3626         screen.Name = "LabLog"
3627         screen.ResetOnSpawn = false
3628         screen.DisplayOrder = 40
3629         screen.Parent = gui
3630
3631         local frame = Instance.new("TextLabel")
3632         frame.Name = "Line"
3633         frame.Size = UDim2.new(0.48, 0, 0, 46)
3634         frame.AnchorPoint = Vector2.new(0.5, 1)
3635         frame.Position = UDim2.new(0.5, 0, 1, -68)
3636         frame.BackgroundColor3 = Color3.fromRGB(18, 22, 28)
3637         frame.BackgroundTransparency = 0.12
3638         frame.TextColor3 = Color3.fromRGB(235, 238, 242)
3639         frame.TextWrapped = true
3640         frame.TextTruncate = Enum.TextTruncate.AtEnd
3641         frame.TextSize = 13
3642         frame.Font = Enum.Font.Gotham
3643         frame.Parent = screen
3644         local c = Instance.new("UICorner")
3645         c.CornerRadius = UDim.new(0, 8)
3646         c.Parent = frame
3647         local sizeLimit = Instance.new("UISizeConstraint")
3648         sizeLimit.MinSize = Vector2.new(260, 46)
3649         sizeLimit.MaxSize = Vector2.new(420, 46)
3650         sizeLimit.Parent = frame
3651         local padding = Instance.new("UIPadding")
3652         padding.PaddingLeft = UDim.new(0, 10)
3653         padding.PaddingRight = UDim.new(0, 10)
3654         padding.Parent = frame
3655     end
3656
3657     local line = screen:FindFirstChild("Line")
3658     if not line then return end
3659     local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
3660     screen:SetAttribute("MessageSerial", serial)
3661     line.Visible = true
3662     line.Text = text
3663     task.delay(4, function()
3664         if line.Parent and screen:GetAttribute("MessageSerial") == serial then line.Visible = false end
3665     end)
3666 end
3667
3668 local function playerRole(player)
3669     local attributeRole = player:GetAttribute("LabRole")
3670     if attributeRole and attributeRole ~= "" then return attributeRole end
3671     if player.Team then return player.Team.Name end
3672     local stats = player:FindFirstChild("leaderstats")
3673     local role = stats and stats:FindFirstChild("Role")
3674     return role and role.Value or "Unassigned"
3675 end
3676
3677 local function awardPoints(player, amount, reason)
3678     local stats = player:FindFirstChild("leaderstats")
3679     if not stats then
3680         stats = Instance.new("Folder")
3681         stats.Name = "leaderstats"
3682         stats.Parent = player
3683     end
3684     local points = stats:FindFirstChild("Points")
3685     if not points then
3686         points = Instance.new("IntValue")
3687         points.Name = "Points"
3688         points.Parent = stats
3689     end
3690     points.Value = points.Value + amount
3691     addContribution(player, 1)

```

```

3692     player:SetAttribute("LastPointAmount", amount)
3693     player:SetAttribute("LastPointAward", reason)
3694     notify(player, string.format("+%d POINTS - %s • TOTAL %d", amount, reason, points.Value))
3695     updateScoreboard()
3696     return points.Value
3697 end
3698
3699 local function canInteract(player, part, maxDistance)
3700     local character = player.Character
3701     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3702     local root = character and character:FindFirstChild("HumanoidRootPart")
3703     if humanoid == nil or humanoid.Health <= 0 or root == nil
3704         or (root.Position - part.Position).Magnitude > maxDistance + 2 then return false end
3705     local originPart = character:FindFirstChild("Head") or root
3706     local direction = part.Position - originPart.Position
3707     local params = RaycastParams.new()
3708     params.FilterType = Enum.RaycastFilterType.Exclude
3709     params.FilterDescendantsInstances = {character}
3710     params.IgnoreWater = true
3711     local hit = workspace:Raycast(originPart.Position, direction, params)
3712     return hit == nil or hit.Instance == part or hit.Instance:IsDescendantOf(part)
3713 end
3714
3715 -- Math-quiz gate removed by request: every task it used to guard now
3716 -- proceeds immediately. Kept as a pass-through function (rather than
3717 -- deleting every call site) so nothing else in this file has to change.
3718 local function runMathQuiz(player, reason)
3719     return true
3720 end
3721
3722 local function runMathQuiz_DISABLED(player, reason)
3723     if player:GetAttribute("MathQuizActive") then return false end
3724     local digits = math.random(2, 7)
3725     local maximum = math.min(9999999, 10 ^ digits - 1)
3726     local a, b = math.random(1, maximum), math.random(1, maximum)
3727     local subtract = math.random() < 0.5
3728     if subtract and b > a then a, b = b, a end
3729     local expected = subtract and (a - b) or (a + b)
3730     local question = string.format("%d %s %d", a, subtract and "-" or "+", b)
3731     local seconds = 10 + digits * 2
3732     local previousObjective = player:GetAttribute("LabObjective")
3733     player:SetAttribute("MathQuizActive", true)
3734     player:SetAttribute("LabObjective", string.format("MATH CHECK: %s = ? Enter it in the console within %d
↵ seconds.", question, seconds))
3735     local started = os.clock()
3736     local ok, response = pcall(function()
3737         return QuizRemote:InvokeClient(player, {Question = question, Seconds = seconds, Reason = reason})
3738     end)
3739     player:SetAttribute("MathQuizActive", false)
3740     local correct = ok and os.clock() - started <= seconds + 1 and tonumber(response) == expected
3741     if correct then
3742         player:SetAttribute("LabObjective", previousObjective)
3743         notify(player, "Math check correct. Task authorized.")
3744         return true
3745     end
3746     player:SetAttribute("LabObjective", "Math check failed or timed out. Repeat the equipment action to try
↵ again.")
3747     notify(player, "Math check failed or timed out. Try the task again.")
3748     return false
3749 end
3750
3751 -----
3752 -- VISIBLE INVENTORY ITEMS
3753 -- The old script collection used Tool + Handle + welded decorative
3754 -- pieces. This keeps that useful pattern, but creates and validates the
3755 -- tools entirely on the server instead of trusting a client script.
3756 -----
3757 local SAMPLE_ITEM = {
3758     [Kinds.Items.TRAY] = true,
3759     [Kinds.Items.GROUND] = true,
3760     [Kinds.Items.DRIED] = true,
3761     [Kinds.Items.SPUN] = true,
3762     [Kinds.Items.CULTURE] = true,
3763 }
3764
3765 local function addWeldedPart(tool, handle, name, size, relativeCFrame, colour, material, shape)

```

```

3766     local p = Instance.new("Part")
3767     p.Name = name
3768     p.Size = size
3769     p.CFrame = handle.CFrame * relativeCFrame
3770     p.Color = colour
3771     p.Material = material or Enum.Material.SmoothPlastic
3772     p.Shape = shape or Enum.PartType.Block
3773     p.Anchored = false
3774     p.CanCollide = false
3775     p.CanTouch = false
3776     p.Massless = true
3777     p.TopSurface = Enum.SurfaceType.Smooth
3778     p.BottomSurface = Enum.SurfaceType.Smooth
3779     p.Parent = tool
3780     local weld = Instance.new("WeldConstraint")
3781     weld.Part0 = handle
3782     weld.Part1 = p
3783     weld.Parent = p
3784     return p
3785 end
3786
3787 local function equipTool(player, tool)
3788     task.defer(function()
3789         if not tool.Parent then return end
3790         local character = player.Character
3791         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3792         if humanoid and humanoid.Health > 0 then humanoid:EquipTool(tool) end
3793     end)
3794 end
3795
3796 local function makeTool(player, name, options)
3797     options = options or {}
3798     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
3799     if not backpack then return nil end
3800
3801     local tool = Instance.new("Tool")
3802     tool.Name = name
3803     tool.ToolTip = options.tooltip or "Visible laboratory item"
3804     tool.RequiresHandle = true
3805     tool.CanBeDropped = false
3806     tool:SetAttribute("LabItemType", options.itemType or name)
3807     tool:SetAttribute("LabSample", options.sample == true or SAMPLE_ITEM[name] == true)
3808
3809     for key, value in pairs(options.attributes or {}) do tool:SetAttribute(key, value) end
3810     if options.plant and tool:GetAttribute("PlantLife") == nil then tool:SetAttribute("PlantLife", 100) end
3811
3812     local handle = Instance.new("Part")
3813     handle.Name = "Handle"
3814     handle.Size = options.plant and Vector3.new(0.95, 0.72, 0.95) or Vector3.new(1.7, 0.22, 1.35)
3815     handle.Color = options.colour or Color3.fromRGB(205, 185, 138)
3816     handle.Material = options.plant and Enum.Material.WoodPlanks or Enum.Material.SmoothPlastic
3817     handle.Anchored = false
3818     handle.CanCollide = false
3819     handle.Massless = true
3820     handle.TopSurface = Enum.SurfaceType.Smooth
3821     handle.BottomSurface = Enum.SurfaceType.Smooth
3822     handle.Parent = tool
3823
3824     if options.plant then
3825         -- A real visible potted plant: soil, stem, leaves, and a glowing bud
3826         -- are welded to the Tool handle and replicate in the character hand.
3827         addWeldedPart(tool, handle, "Soil", Vector3.new(0.82, 0.14, 0.82),
3828             CFrame.new(0, 0.4, 0), Color3.fromRGB(78, 52, 33), Enum.Material.Ground)
3829         addWeldedPart(tool, handle, "PlantStem", Vector3.new(0.16, 1.0, 0.16),
3830             CFrame.new(0, 0.96, 0), Color3.fromRGB(65, 155, 75), Enum.Material.Grass)
3831         addWeldedPart(tool, handle, "LeafLeft", Vector3.new(0.78, 0.16, 0.42),
3832             CFrame.new(-0.34, 1.1, 0) * CFrame.Angles(0, 0, math.rad(-22)),
3833             Color3.fromRGB(70, 180, 82), Enum.Material.Grass)
3834         addWeldedPart(tool, handle, "LeafRight", Vector3.new(0.78, 0.16, 0.42),
3835             CFrame.new(0.34, 1.35, 0) * CFrame.Angles(0, 0, math.rad(22)),
3836             Color3.fromRGB(85, 195, 92), Enum.Material.Grass)
3837         addWeldedPart(tool, handle, "GrowthBud", Vector3.new(0.36, 0.36, 0.36),
3838             CFrame.new(0, 1.58, 0), Color3.fromRGB(245, 185, 80), Enum.Material.Neon, Enum.PartType.Ball)
3839     else
3840         -- Tray rim, sample pouch, and labelled vial make ordinary samples
3841         -- recognizable from a distance instead of a plain block.

```

```

3842         addWeldedPart(tool, handle, "TrayRim", Vector3.new(1.82, 0.12, 1.47),
3843             CFrame.new(0, 0.14, 0), Color3.fromRGB(55, 68, 78), Enum.Material.Metal)
3844         addWeldedPart(tool, handle, "SeedPouch", Vector3.new(0.92, 0.32, 0.76),
3845             CFrame.new(0, 0.35, 0), Color3.fromRGB(220, 193, 116), Enum.Material.Fabric)
3846         addWeldedPart(tool, handle, "SampleVial", Vector3.new(0.62, 0.22, 0.22),
3847             CFrame.new(0.47, 0.54, 0) * CFrame.Angles(0, 0, math.rad(90)),
3848             Color3.fromRGB(115, 215, 245), Enum.Material.Glass, Enum.PartType.Cylinder)
3849     end
3850
3851     local marker = Instance.new("BillboardGui")
3852     marker.Name = "CarryLabel"
3853     marker.Size = UDim2.fromOffset(150, 34)
3854     marker.StudsOffsetWorldSpace = Vector3.new(0, 1.15, 0)
3855     marker.MaxDistance = 8
3856     marker.AlwaysOnTop = false
3857     marker.Parent = handle
3858     local markerText = Instance.new("TextLabel")
3859     markerText.Size = UDim2.fromScale(1, 1)
3860     markerText.BackgroundColor3 = Color3.fromRGB(20, 25, 30)
3861     markerText.BackgroundTransparency = 0.2
3862     markerText.TextColor3 = Color3.fromRGB(245, 225, 150)
3863     markerText.TextScaled = true
3864     markerText.Text = options.Label or name
3865     markerText.Font = Enum.Font.GothamBold
3866     markerText.Parent = marker
3867     local markerCorner = Instance.new("UICorner")
3868     markerCorner.CornerRadius = UDim.new(0, 6)
3869     markerCorner.Parent = markerText
3870     if options.plant then
3871         local lifeGui = Instance.new("BillboardGui")
3872         lifeGui.Name = "PlantLifeStatus"
3873         lifeGui.Size = UDim2.fromOffset(160, 46)
3874         lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
3875         lifeGui.MaxDistance = 18
3876         lifeGui.AlwaysOnTop = false
3877         lifeGui.Parent = handle
3878         local lifeFrame = Instance.new("Frame")
3879         lifeFrame.Size = UDim2.fromScale(1, 1)
3880         lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
3881         lifeFrame.BackgroundTransparency = 0.12
3882         lifeFrame.BorderSizePixel = 0
3883         lifeFrame.Parent = lifeGui
3884         local lifeText = Instance.new("TextLabel")
3885         lifeText.Name = "LifeText"
3886         lifeText.Size = UDim2.new(1, 0, 0.58, 0)
3887         lifeText.BackgroundTransparency = 1
3888         lifeText.TextScaled = true
3889         lifeText.Font = Enum.Font.GothamBold
3890         lifeText.Parent = lifeFrame
3891         local lifeBack = Instance.new("Frame")
3892         lifeBack.Name = "LifeBarBackground"
3893         lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
3894         lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
3895         lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
3896         lifeBack.BorderSizePixel = 0
3897         lifeBack.Parent = lifeFrame
3898         local lifeFill = Instance.new("Frame")
3899         lifeFill.Name = "LifeBarFill"
3900         lifeFill.Size = UDim2.fromScale(1, 1)
3901         lifeFill.BorderSizePixel = 0
3902         lifeFill.Parent = lifeBack
3903         local function updateLifeBar()
3904             local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
3905             lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
3906             lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
3907             local colour = life > 60 and Color3.fromRGB(75, 235, 105)
3908                 or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
3909             lifeText.TextColor3 = colour
3910             lifeFill.BackgroundColor3 = colour
3911         end
3912         tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLifeBar)
3913         updateLifeBar()
3914     end
3915
3916     tool.Grip = options.plant
3917     and (CFrame.new(0, -0.25, -0.75) * CFrame.Angles(math.rad(-8), 0, 0))

```

```

3918         or (CFrame.new(0, -0.45, -0.65) * CFrame.Angles(math.rad(-12), 0, 0))
3919     tool.Parent = backpack
3920     if options.autoEquip ~= false then equipTool(player, tool) end
3921     return tool
3922 end
3923
3924 local function playerContainers(player)
3925     local containers = {}
3926     if player.Character then table.insert(containers, player.Character) end
3927     local backpack = player:FindFirstChildOfClass("Backpack")
3928     if backpack then table.insert(containers, backpack) end
3929     return containers
3930 end
3931
3932 local function findTool(player, itemType)
3933     for _, container in ipairs(playerContainers(player)) do
3934         if container then
3935             for _, child in ipairs(container:GetChildren()) do
3936                 if child:IsA("Tool") and
3937                     (child.Name == itemType or child:GetAttribute("LabItemType") == itemType) then
3938                     return child
3939                 end
3940             end
3941         end
3942     end
3943     return nil
3944 end
3945
3946 local function findTransferSample(player)
3947     for _, container in ipairs(playerContainers(player)) do
3948         if container then
3949             for _, child in ipairs(container:GetChildren()) do
3950                 if child:IsA("Tool") and child:GetAttribute("TransferSample") then return child end
3951             end
3952         end
3953     end
3954     return nil
3955 end
3956
3957 local function findHeldSample(player)
3958     local character = player.Character
3959     if not character then return nil end
3960     for _, child in ipairs(character:GetChildren()) do
3961         if child:IsA("Tool") and child:GetAttribute("LabSample") then return child end
3962     end
3963     return nil
3964 end
3965
3966 local GROWTH_STAGE_NAME = {
3967     [0] = "Seedling",
3968     [1] = "Germinated",
3969     [2] = "Vegetative",
3970     [3] = "Budding",
3971     [4] = "Flowering",
3972     [5] = "Mature",
3973 }
3974
3975 local function findPlantSample(player)
3976     for _, container in ipairs(playerContainers(player)) do
3977         for _, child in ipairs(container:GetChildren()) do
3978             if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end
3979         end
3980     end
3981     return nil
3982 end
3983
3984 local function treatmentText(tool)
3985     local treatments = tool:GetAttribute("Treatments") or ""
3986     if treatments == "" then return "untreated" end
3987     return treatments:gsub("|", ", ")
3988 end
3989
3990 local function hasTreatment(tool, key)
3991     local treatments = "|" .. (tool:GetAttribute("Treatments") or "") .. "|"
3992     return treatments:find("|" .. key .. "|" , 1, true) ~= nil
3993 end

```

```

3994
3995 local function addTreatment(tool, key)
3996     if hasTreatment(tool, key) then return false end
3997     local treatments = tool:GetAttribute("Treatments") or ""
3998     tool:SetAttribute("Treatments", treatments == "" and key or (treatments .. "|" .. key))
3999     return true
4000 end
4001
4002 local function refreshPlantVisual(tool)
4003     local stage = math.clamp(tool:GetAttribute("GrowthStage") or 0, 0, 5)
4004     local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
4005     local stem = tool:FindFirstChild("PlantStem")
4006     local left = tool:FindFirstChild("LeafLeft")
4007     local right = tool:FindFirstChild("LeafRight")
4008     local bud = tool:FindFirstChild("GrowthBud")
4009     if stem then stem.Size = Vector3.new(0.16 + stage * 0.025, 1 + stage * 0.13, 0.16 + stage * 0.025) end
4010     local lifeScale = 0.35 + life / 100 * 0.65
4011     if left then left.Color = Color3.fromRGB((70 + stage * 5) * lifeScale, (165 + stage * 10) * lifeScale, 55 *
↳ lifeScale) end
4012     if right then right.Color = Color3.fromRGB((75 + stage * 4) * lifeScale, (175 + stage * 10) * lifeScale, 60
↳ * lifeScale) end
4013     if bud then
4014         bud.Transparency = stage < 3 and 0.65 or 0
4015         bud.Color = stage >= 5 and Color3.fromRGB(255, 105, 175) or Color3.fromRGB(245, 185, 80)
4016     end
4017     local handle = tool:FindFirstChild("Handle")
4018     local marker = handle and handle:FindFirstChild("CarryLabel")
4019     local label = marker and marker:FindFirstChildOfClass("TextLabel")
4020     if label then
4021         local destination = tool:GetAttribute("DestinationGrowthCode") or ""
4022         label.Text = (tool:GetAttribute("PlantId") or "PLANT") .. " • " .. (GROWTH_STAGE_NAME[stage] or
↳ "Growing")
4023         .. (destination ~= "" and (" • BONUS DROP " .. destination) or " • DROP IN GREEN R")
4024     end
4025 end
4026
4027 local function setCarryState(player, sample)
4028     -- A cold-store transfer remains the main HUD objective even if the
4029     -- player briefly produces or pockets another laboratory item.
4030     local transfer = findTransferSample(player)
4031     if transfer and sample and not sample:GetAttribute("TransferSample") then sample = transfer end
4032     if sample then
4033         local id = sample:GetAttribute("SampleId") or sample.Name
4034         local destination = sample:GetAttribute("DestinationLabel") or
↳ sample:GetAttribute("DestinationGrowthCode")
4035         player:SetAttribute("CarriedSample", id)
4036         if destination and destination ~= "" then
4037             player:SetAttribute("LabObjective", "Carry " .. id .. " to bonus chamber " .. destination .. ", or
↳ press E at any GREEN RUNNING cabinet with a free slot.")
4038         else
4039             player:SetAttribute("LabObjective", "Carry " .. sample.Name .. " to the next required machine.")
4040         end
4041     else
4042         player:SetAttribute("CarriedSample", "")
4043         player:SetAttribute("LabObjective", "")
4044     end
4045 end
4046
4047 local function destroyCarriedItem(player, tool)
4048     if player:GetAttribute("CarriedSample") == (tool:GetAttribute("SampleId") or tool.Name) then
4049         setCarryState(player, nil)
4050     end
4051     tool:Destroy()
4052 end
4053
4054 -----
4055 -- NON-LETHAL HIT TOOLS
4056 -----
4057 local IMPACT_STYLE = {
4058     ["Foam Baton"] = {colour = Color3.fromRGB(245, 135, 55), accent = Color3.fromRGB(45, 48, 54), mode =
↳ "melee", damage = 18},
4059     ["Sample Grabber"] = {colour = Color3.fromRGB(70, 165, 220), accent = Color3.fromRGB(205, 225, 235), mode =
↳ "melee", damage = 12},
4060     ["Inspection Paddle"] = {colour = Color3.fromRGB(245, 205, 70), accent = Color3.fromRGB(95, 75, 30), mode =
↳ "melee", damage = 22},

```

```

4061 ["Stun Blaster"] = {colour = Color3.fromRGB(90, 225, 245), accent = Color3.fromRGB(35, 55, 70), mode =
↳ "gun", damage = 28, range = 95},
4062 ["Foam Dart Rifle"] = {colour = Color3.fromRGB(225, 80, 95), accent = Color3.fromRGB(45, 45, 52), mode =
↳ "gun", damage = 38, range = 125},
4063 ["Basketball"] = {colour = Color3.fromRGB(225, 105, 35), accent = Color3.fromRGB(45, 28, 18), mode =
↳ "ball", damage = 24},
4064 -- PC-only weapons, built from the official Roblox Weapons Kit meshes.
4065 --
4066 -- meshScale is (the kit's intended size / the raw mesh size), measured off
4067 -- the kit's own MeshParts. This matters: the raw pistol mesh is 9.68 studs
4068 -- long, so the scale of 1.8 used at first drew a pistol about 17 studs
4069 -- long -- roughly three times the height of the character holding it.
4070 --
4071 -- A worker bot has 100 life, so damage is set for 2-3 body shots; a head
4072 -- shot kills outright regardless. selfCostPercent still charges the
4073 -- shooter 1% of their own max health per shot. pcOnly means the generic
4074 -- Tool.Activated path ignores them -- they fire only through the
4075 -- LabSidearmFire remote, which only the desktop client script sends.
4076 ["Lab Sidearm"] = {
4077     colour = Color3.fromRGB(126, 133, 145), accent = Color3.fromRGB(38, 42, 50),
4078     mode = "gun", damage = 34, range = 240, selfCostPercent = 1, pcOnly = true,
4079     cooldown = 0.32, pellets = 1, spread = 0,
4080     meshId = "rbxassetid://2492922600", meshTexture = "rbxassetid://2492931263",
4081     meshScale = 0.1407, handleSize = Vector3.new(0.20, 0.92, 1.36),
4082     grip = CFrame.new(0, -0.20, -0.30) * CFrame.Angles(0, math.rad(180), 0),
4083 },
4084 ["Lab Shotgun"] = {
4085     colour = Color3.fromRGB(122, 98, 72), accent = Color3.fromRGB(40, 34, 30),
4086     mode = "gun", damage = 20, range = 110, selfCostPercent = 1, pcOnly = true,
4087     cooldown = 0.85, pellets = 6, spread = 0.075,
4088     meshId = "rbxassetid://2492972199", meshTexture = "rbxassetid://2492974190",
4089     meshScale = 0.6666, handleSize = Vector3.new(0.31, 1.00, 4.87),
4090     grip = CFrame.new(0, -0.35, -0.95) * CFrame.Angles(0, math.rad(180), 0),
4091 },
4092 ["Lab Rifle"] = {
4093     colour = Color3.fromRGB(96, 104, 96), accent = Color3.fromRGB(34, 38, 34),
4094     mode = "gun", damage = 55, range = 620, selfCostPercent = 1, pcOnly = true,
4095     cooldown = 0.95, pellets = 1, spread = 0,
4096     meshId = "rbxassetid://2759059561", meshTexture = "rbxassetid://2759063893",
4097     meshScale = 0.6383, handleSize = Vector3.new(0.29, 1.17, 6.18),
4098     grip = CFrame.new(0, -0.35, -1.20) * CFrame.Angles(0, math.rad(180), 0),
4099 },
4100 }
4101
4102 local function refreshBotHealth(bot)
4103     local health = math.clamp(bot:GetAttribute("Health") or 100, 0, bot:GetAttribute("MaxHealth") or 100)
4104     local maximum = bot:GetAttribute("MaxHealth") or 100
4105     local text = bot:FindFirstChild("HealthText", true)
4106     local fill = bot:FindFirstChild("HealthBarFill", true)
4107     if text and text:IsA("TextLabel") then text.Text = string.format("LIFE %d / %d", health, maximum) end
4108     if fill and fill:IsA("Frame") then
4109         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
4110         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
4111         or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
4112     end
4113 end
4114
4115 -- How long a body stays before it is cleaned up. Bodies are the biggest
4116 -- avoidable part cost in a long round, so this is deliberately short.
4117 -- Declared HERE, above damageWorkerBot, because that is what reads it: a
4118 -- local declared further down the file resolves to a nil global inside a
4119 -- function defined above it.
4120 local BODY_LIFETIME_SECONDS = 60
4121
4122 local function damageWorkerBot(bot, amount, attacker)
4123     -- Only an already-dead worker is immune. This used to refuse damage to
4124     -- anything KnockedOut, which meant a worker downed by a hazard could not
4125     -- be finished off by gunfire or caught in a blast -- so a floor could
4126     -- never actually be cleared.
4127     if not bot or not CollectionService.HasTag(bot, "LabWorkerBot") or bot:GetAttribute("Dead") then return
↳ false end
4128     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
4129     bot:SetAttribute("Health", health)
4130     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
4131     refreshBotHealth(bot)
4132     local flash = Instance.new("Highlight")

```

```

4133 flash.FillColor = Color3.fromRGB(255, 90, 70)
4134 flash.OutlineColor = Color3.fromRGB(255, 255, 255)
4135 flash.FillTransparency = 0.25
4136 -- Highlights render through geometry by default, which is why a hit
4137 -- worker (and any corpse that inherited the flash) glowed through walls.
4138 flash.DepthMode = Enum.HighlightDepthMode.Occluded
4139 flash.Parent = bot
4140 Debris:AddItem(flash, 0.22)
4141 if health <= 0 then
4142     -- Death is permanent now: the hunt is the objective, so a worker that
4143     -- goes down stays down and its body is left where it fell. Bodies are
4144     -- cleared when the shift rolls over (see the RoundNumber watcher).
4145     bot:SetAttribute("Dead", true)
4146     bot:SetAttribute("KnockedOut", true)
4147     bot:SetAttribute("TaskState", "DEAD")
4148
4149     -- The body left behind is a CLONE, not the worker itself. That keeps
4150     -- the original model pristine, so the shift roll-over can simply move
4151     -- it home and stand it back up rather than trying to undo a pile of
4152     -- destructive colour and rotation edits.
4153     local pivot = bot:GetPivot()
4154     local corpse = bot:Clone()
4155     corpse.Name = bot.Name .. "_Body"
4156     for _, d in ipairs(corpse:GetDescendants()) do
4157         -- A body reports no status, and must not inherit the damage-flash
4158         -- Highlight -- that is what made corpses visible through walls,
4159         -- because the clone's copy was never cleaned up.
4160         if d:IsA("BillboardGui") or d:IsA("Highlight") or d:IsA("Sparkles")
4161            or d:IsA("Fire") or d:IsA("Smoke") or d:IsA("PointLight")
4162            or d.Name:match("^Chair") or d.Name:match("^Desk") or d.Name == "Monitor" then
4163             d:Destroy()
4164         end
4165     end
4166     for _, d in ipairs(corpse:GetDescendants()) do
4167         if d:IsA("BasePart") then
4168             d.Color = Color3.new(d.Color.R * 0.55 + 0.10, d.Color.G * 0.55 + 0.10, d.Color.B * 0.55 + 0.10)
4169             d.Material = Enum.Material.SmoothPlastic
4170             d.Anchored = true
4171             d.CanCollide = false
4172             d.CanQuery = false -- a body must not soak up later shots
4173             d.CastShadow = false -- 180 shadow-casting corpses is not free
4174         end
4175     end
4176     local corpseFolder = workspace:FindFirstChild("LabBotBodies")
4177     if not corpseFolder then
4178         corpseFolder = Instance.new("Folder")
4179         corpseFolder.Name = "LabBotBodies"
4180         corpseFolder.Parent = workspace
4181     end
4182     -- The clone inherits every tag, including LabWorkerBot. Left alone it
4183     -- would be counted as a live worker by every GetTagged sweep and the
4184     -- kill total would never be able to reach the target.
4185     CollectionService:RemoveTag(corpse, "LabWorkerBot")
4186     corpse.Parent = corpseFolder
4187     -- Lay it down where it fell, with a random spin so a cleared room does
4188     -- not read as a row of identical dropped props.
4189     corpse:PivotTo(CFrame.new(pivot.Position - Vector3.new(0, 2.6, 0))
4190         * CFrame.Angles(0, math.rad(math.random(0, 359)), 0)
4191         * CFrame.Angles(math.rad(-90), 0, 0))
4192     CollectionService:AddTag(corpse, "LabBotCorpse")
4193     -- Bodies are the single biggest avoidable part cost in a long round:
4194     -- 180 of them is ~1,600 extra parts. They stay long enough to read a
4195     -- room as cleared, then go.
4196     Debris:AddItem(corpse, BODY_LIFETIME_SECONDS)
4197
4198     -- Park the live worker out of the world until the next shift.
4199     if not bot:GetAttribute("HomeCFrame") then bot:SetAttribute("HomeCFrame", pivot) end
4200     bot:PivotTo(CFrame.new(0, -1000, 0))
4201
4202     local complex = workspace:FindFirstChild("Laboratory_Complex")
4203     if complex then
4204         complex:SetAttribute("BotsKilled", (complex:GetAttribute("BotsKilled") or 0) + 1)
4205     end
4206 end
4207 return true
4208 end

```



```

4209
4210 -- Walk up from whatever the ray actually hit to the tagged equipment part.
4211 local function equipmentFromHit(hit)
4212     local node = hit
4213     while node and node ~= workspace do
4214         if node:IsA("BasePart")
4215             and (CollectionService:HasTag(node, "LabEquipment") or CollectionService:HasTag(node,
4216                 ↳ "PlantGrowthChamber")) then
4217                 return node
4218             end
4219             node = node.Parent
4220     end
4221     return nil
4222 end
4223 -- Shoot a machine and it goes up, taking out whatever is standing near it --
4224 -- including the player who pulled the trigger, if they did it from close range.
4225 local function explodeEquipment(item, attacker)
4226     if item:GetAttribute("Exploded") then return false end
4227     item:SetAttribute("Exploded", true)
4228     item:SetAttribute("FunctionalState", "DESTROYED")
4229     item:SetAttribute("RepairInstruction", "Destroyed beyond repair")
4230
4231     local position = item.Position
4232     local blast = Instance.new("Explosion")
4233     blast.Position = position
4234     blast.BlastRadius = 17
4235     blast.BlastPressure = 0 -- the lab is anchored; pressure would do nothing
4236     blast.DestroyJointRadiusPercent = 0
4237     blast.ExplosionType = Enum.ExplosionType.NoCraters
4238     blast.Parent = workspace
4239
4240     local fire = Instance.new("Fire")
4241     fire.Heat = 14
4242     fire.Size = 9
4243     fire.Parent = item
4244     Debris:AddItem(fire, 14)
4245     local smoke = Instance.new("Smoke")
4246     smoke.Size = 5
4247     smoke.Opacity = 0.4
4248     smoke.RiseVelocity = 9
4249     smoke.Color = Color3.fromRGB(60, 58, 56)
4250     smoke.Parent = item
4251     Debris:AddItem(smoke, 18)
4252
4253     item.Color = Color3.fromRGB(48, 40, 36)
4254     item.Material = Enum.Material.CorrodedMetal
4255
4256     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
4257         if not bot:GetAttribute("Dead") and bot.PrimaryPart then
4258             if (bot.PrimaryPart.Position - position).Magnitude < 17 then
4259                 damageWorkerBot(bot, 1000, attacker)
4260             end
4261         end
4262     end
4263     for _, other in ipairs(Players:GetPlayers()) do
4264         local c = other.Character
4265         local h = c and c:FindFirstChildOfClass("Humanoid")
4266         local hrp = c and c:FindFirstChild("HumanoidRootPart")
4267         if h and hrp and h.Health > 0 then
4268             local d = (hrp.Position - position).Magnitude
4269             if d < 17 then h:TakeDamage(math.max(18, 75 * (1 - d / 17))) end
4270         end
4271     end
4272     return true
4273 end
4274
4275 -- A gas line going up is a different order of event from a machine failing:
4276 -- it takes out a chunk of the room -- a couple of neighbouring machines and
4277 -- part of the wall structure itself.
4278 local function detonateGasPipe(pipe, attacker, atPosition)
4279     if pipe:GetAttribute("Blown") then return false end
4280     pipe:SetAttribute("Blown", true)
4281     -- Blow at the point that was actually hit. A corridor run is one long part
4282     -- whose Position is its midpoint, so using that detonated the pipe up to a
4283     -- hundred studs away from where the player shot it.

```

```

4284     local position = atPosition or pipe.Position
4285
4286     local blast = Instance.new("Explosion")
4287     blast.Position = position
4288     blast.BlastRadius = 30
4289     blast.BlastPressure = 0
4290     blast.DestroyJointRadiusPercent = 0
4291     blast.ExplosionType = Enum.ExplosionType.NoCraters
4292     blast.Parent = workspace
4293
4294     -- a running wall of fire and smoke along the rupture
4295     for i = -2, 2 do
4296         local flare = Instance.new("Part")
4297         flare.Name = "GasFire"
4298         flare.Size = Vector3.new(1, 1, 1)
4299         flare.CFrame = CFrame.new(position + Vector3.new(0, 0, i * 7))
4300         flare.Anchored = true
4301         flare.CanCollide = false
4302         flare.CanQuery = false
4303         flare.CanTouch = false
4304         flare.Transparency = 1
4305         flare.Parent = workspace
4306         local fire = Instance.new("Fire")
4307         fire.Heat = 25
4308         fire.Size = 24
4309         fire.Parent = flare
4310         local smoke = Instance.new("Smoke")
4311         smoke.Size = 15
4312         smoke.Opacity = 0.5
4313         smoke.RiseVelocity = 14
4314         smoke.Color = Color3.fromRGB(40, 38, 36)
4315         smoke.Parent = flare
4316         task.delay(16, function()
4317             if flare.Parent then fire.Enabled = false; smoke.Enabled = false end
4318         end)
4319         Debris:AddItem(flare, 22)
4320     end
4321
4322     pipe.Color = Color3.fromRGB(62, 52, 30)
4323     pipe.Material = Enum.Material.CorrodedMetal
4324
4325     -- take two or three neighbouring machines with it, staggered so it reads
4326     -- as a chain rather than one simultaneous bang
4327     local nearby = {}
4328     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 24)) do
4329         if (CollectionService:HasTag(part, "LabEquipment") or CollectionService:HasTag(part,
4330             ↪ "PlantGrowthChamber"))
4331             and not part:GetAttribute("Exploded") then
4332             table.insert(nearby, part)
4333         end
4334     end
4335     for i = 1, math.min(3, #nearby) do
4336         local item = nearby[i]
4337         task.delay(0.18 * i, function()
4338             if item.Parent then explodeEquipment(item, attacker) end
4339         end)
4340     end
4341
4342     -- and blow an actual hole in the structure
4343     local blown = 0
4344     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 15)) do
4345         if part:IsA("BasePart") and part.CanCollide and not part:GetAttribute("GasPipe") then
4346             local n = part.Name
4347             if n:match("^Corridor_") or n:match("^Div_") or n:match("^Ext_") or n:match("^B_") then
4348                 if blown < 5 then
4349                     blown = blown + 1
4350                     part:Destroy()
4351                 else
4352                     part.Color = Color3.fromRGB(60, 54, 50)
4353                     part.Material = Enum.Material.CorrodedMetal
4354                 end
4355             end
4356         end
4357     end
4358     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do

```

```

4359         if not bot:GetAttribute("Dead") and bot.PrimaryPart
4360             and (bot.PrimaryPart.Position - position).Magnitude < 30 then
4361             damageWorkerBot(bot, 1000, attacker)
4362         end
4363     end
4364     for _, other in ipairs(Players:GetPlayers()) do
4365         local c = other.Character
4366         local h = c and c:FindFirstChildOfClass("Humanoid")
4367         local hrp = c and c:FindFirstChild("HumanoidRootPart")
4368         if h and hrp and h.Health > 0 then
4369             local d = (hrp.Position - position).Magnitude
4370             if d < 30 then h:TakeDamage(math.max(30, 140 * (1 - d / 30))) end
4371         end
4372     end
4373     return true
4374 end
4375
4376 -- Scorch mark, smoke puff and sparks wherever a round lands.
4377 local function spawnImpact(position, normal)
4378     local hole = Instance.new("Part")
4379     hole.Name = "BulletHole"
4380     hole.Size = Vector3.new(0.3, 0.3, 0.06)
4381     hole.CFrame = CFrame.lookAt(position + normal * 0.03, position + normal * 2)
4382     hole.Color = Color3.fromRGB(24, 22, 21)
4383     hole.Material = Enum.Material.Slate
4384     hole.Anchored = true
4385     hole.CanCollide = false
4386     hole.CanQuery = false
4387     hole.CanTouch = false
4388     hole.CastShadow = false
4389     hole.Parent = workspace
4390     Debris:AddItem(hole, 45)
4391
4392     local puff = Instance.new("Part")
4393     puff.Name = "ImpactPuff"
4394     puff.Size = Vector3.new(0.1, 0.1, 0.1)
4395     puff.CFrame = CFrame.new(position + normal * 0.25)
4396     puff.Anchored = true
4397     puff.CanCollide = false
4398     puff.CanQuery = false
4399     puff.CanTouch = false
4400     puff.Transparency = 1
4401     puff.Parent = workspace
4402     local smoke = Instance.new("Smoke")
4403     smoke.Size = 0.7
4404     smoke.RiseVelocity = 5
4405     smoke.Opacity = 0.3
4406     smoke.Color = Color3.fromRGB(125, 125, 125)
4407     smoke.Parent = puff
4408     local sparks = Instance.new("Sparkles")
4409     sparks.SparkleColor = Color3.fromRGB(255, 208, 128)
4410     sparks.Parent = puff
4411     task.delay(0.3, function()
4412         if puff.Parent then smoke.Enabled = false; sparks.Enabled = false end
4413     end)
4414     Debris:AddItem(puff, 3)
4415 end
4416
4417 -- Posters keep their holes for good rather than fading like scorch marks.
4418 local function punchPoster(poster, position, normal)
4419     local holes = poster:GetAttribute("Holes") or 0
4420     if holes >= 30 then return end
4421     poster:SetAttribute("Holes", holes + 1)
4422     local tear = Instance.new("Part")
4423     tear.Name = "PosterTear"
4424     tear.Size = Vector3.new(0.36, 0.36, 0.03)
4425     tear.CFrame = CFrame.lookAt(position + normal * 0.012, position + normal * 2)
4426     tear.Color = Color3.fromRGB(198, 190, 174)
4427     tear.Material = Enum.Material.SmoothPlastic
4428     tear.Anchored = true
4429     tear.CanCollide = false
4430     tear.CanQuery = false
4431     tear.CanTouch = false
4432     tear.CastShadow = false
4433     tear.Parent = poster
4434     local hole = tear:Clone()

```

```

4435     hole.Name = "PosterHole"
4436     hole.Size = Vector3.new(0.22, 0.22, 0.05)
4437     hole.Color = Color3.fromRGB(16, 15, 14)
4438     hole.CFrame = CFrame.lookAt(position + normal * 0.025, position + normal * 2)
4439     hole.Parent = poster
4440 end
4441
4442 local function workerBotFromHit(hit)
4443     local model = hit and hit:FindFirstAncestorOfClass("Model")
4444     return model and CollectionService.HasTag(model, "LabWorkerBot") and model or nil
4445 end
4446
4447 local function stealHeldSample(attacker, victim)
4448     local sample = findHeldSample(victim)
4449     if not sample then
4450         notify(attacker, victim.DisplayName .. " is not holding a sample.")
4451         return false
4452     end
4453     if sample:GetAttribute("TransferSample") and findTransferSample(attacker) then
4454         notify(attacker, "Your hands are already assigned to another cold-store sample.")
4455         return false
4456     end
4457
4458     local backpack = attacker:FindFirstChildOfClass("Backpack")
4459     if not backpack then return false end
4460     local id = sample:GetAttribute("SampleId") or sample.Name
4461     sample.Parent = backpack
4462     setCarryState(victim, nil)
4463     setCarryState(attacker, sample)
4464     addContribution(attacker)
4465     notify(attacker, "Recovered " .. id .. " from " .. victim.DisplayName .. ". It is now in your hand.")
4466     notify(victim, attacker.DisplayName .. " took " .. id .. " from your hand. Collect or recover another
↪ sample.")
4467     equipTool(attacker, sample)
4468     return true
4469 end
4470
4471 local function armImpactTool(tool)
4472     if tool:GetAttribute("HitLogicBound") then return end
4473     tool:SetAttribute("HitLogicBound", true)
4474     local handle = tool:FindFirstChild("Handle")
4475     if not handle then return end
4476     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or tool.Name]
4477     if not style then return end
4478
4479     local activeUntil = 0
4480     local nextSwing = 0
4481     local hitThisSwing = {}
4482
4483     tool.Activated:Connect(function()
4484         -- Every gun is driven by BlasterController on the owning client.
4485         -- Keep Tool.Activated for melee and throwable tools only, avoiding
4486         -- duplicate shots from Roblox's default Tool activation path.
4487         if style.mode == "gun" then return end
4488         local owner = Players:GetPlayerFromCharacter(tool.Parent)
4489         if not owner or os.clock() < nextSwing then return end
4490         nextSwing = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
4491         hitThisSwing = {}
4492         local original = handle.Color
4493         handle.Color = Color3.fromRGB(255, 245, 190)
4494         task.delay(0.18, function()
4495             if handle.Parent then handle.Color = original end
4496         end)
4497
4498         local rootPart = owner.Character and owner.Character:FindFirstChild("HumanoidRootPart")
4499         if not rootPart then return end
4500         if style.mode == "gun" then
4501             local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
4502             local direction = rootPart.CFrame.LookVector * (style.range or 100)
4503             local params = RaycastParams.new()
4504             params.FilterType = Enum.RaycastFilterType.Exclude
4505             params.FilterDescendantsInstances = {owner.Character}
4506             local result = workspace.Raycast(origin, direction, params)
4507             local hitPosition = result and result.Position or (origin + direction)
4508             local beam = Instance.new("Part")
4509             beam.Name = "FoamTracer"

```

```

4510     beam.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
4511     beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
4512     beam.Color = style.colour
4513     beam.Material = Enum.Material.Neon
4514     beam.Anchored = true
4515     beam.CanCollide = false
4516     beam.CanTouch = false
4517     beam.Parent = workspace
4518     Debris:AddItem(beam, 0.12)
4519     local bot = result and workerBotFromHit(result.Instance)
4520     if bot then damageWorkerBot(bot, style.damage or 25, owner) end
4521     return
4522 elseif style.mode == "ball" then
4523     local ball = Instance.new("Part")
4524     ball.Name = "ThrownBasketball"
4525     ball.Shape = Enum.PartType.Ball
4526     ball.Size = Vector3.new(1.35, 1.35, 1.35)
4527     ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) + rootPart.CFrame.LookVector *
        ↪ 3)
4528     ball.Color = style.colour
4529     ball.Material = Enum.Material.Rubber
4530     ball.CanCollide = true
4531     ball.Parent = workspace
4532     ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
4533     local ballHit = false
4534     ball.Touched:Connect(function(hit)
4535         if ballHit then return end
4536         local bot = workerBotFromHit(hit)
4537         if bot then ballHit = true; damageWorkerBot(bot, style.damage or 20, owner) end
4538     end)
4539     Debris:AddItem(ball, 7)
4540     return
4541 end
4542 activeUntil = os.clock() + 0.45
4543 end)
4544
4545 handle.Touched:Connect(function(hit)
4546     if os.clock() > activeUntil then return end
4547     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
4548     if not attacker then return end
4549     local targetCharacter = hit:FindFirstAncestorOfClass("Model")
4550     local workerBot = workerBotFromHit(hit)
4551     if workerBot then
4552         if hitThisSwing[workerBot] then return end
4553         hitThisSwing[workerBot] = true
4554         damageWorkerBot(workerBot, style.damage or 15, attacker)
4555         return
4556     end
4557     local victim = targetCharacter and Players:GetPlayerFromCharacter(targetCharacter)
4558     if not victim or victim == attacker or hitThisSwing[victim] then return end
4559
4560     local attackerRoot = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
4561     local victimRoot = victim.Character and victim.Character:FindFirstChild("HumanoidRootPart")
4562     local victimHumanoid = victim.Character and victim.Character:FindFirstChildOfClass("Humanoid")
4563     if not attackerRoot or not victimRoot or not victimHumanoid or victimHumanoid.Health <= 0 then return
        ↪ end
4564     if (attackerRoot.Position - victimRoot.Position).Magnitude > 9 then return end
4565     hitThisSwing[victim] = true
4566
4567     local delta = victimRoot.Position - attackerRoot.Position
4568     if delta.Magnitude > 0.1 then
4569         local direction = delta.Unit
4570         victimRoot.AssemblyLinearVelocity = victimRoot.AssemblyLinearVelocity
4571             + Vector3.new(direction.X * 9, 3, direction.Z * 9)
4572     end
4573
4574     local flash = Instance.new("Highlight")
4575     flash.Name = "SampleRecoveryHit"
4576     flash.FillColor = Color3.fromRGB(255, 210, 80)
4577     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
4578     flash.FillTransparency = 0.35
4579     flash.Parent = victim.Character
4580     Debris:AddItem(flash, 0.25)
4581     stealHeldSample(attacker, victim)
4582 end)
4583 end

```

```

4584
4585 local function findImpactTool(player, kind)
4586     for _, container in ipairs(playerContainers(player)) do
4587         if container then
4588             for _, child in ipairs(container:GetChildren()) do
4589                 if child:IsA("Tool") and child:GetAttribute("ImpactToolKind") == kind then return child end
4590             end
4591         end
4592     end
4593     return nil
4594 end
4595
4596 local function makeImpactTool(player, kind)
4597     local existing = findImpactTool(player, kind)
4598     if existing then
4599         equipTool(player, existing)
4600         notify(player, kind .. " equipped. Click or tap to use it against worker NPCs or recover a held sample
4601             ↪ from another player.")
4602         return existing
4603     end
4604
4605     local style = IMPACT_STYLE[kind]
4606     if not style then return nil end
4607     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
4608     if not backpack then return nil end
4609
4610     local tool = Instance.new("Tool")
4611     tool.Name = kind
4612     tool.ToolTip = style.mode == "gun" and "Fire at the screen crosshair; hold right-click or tap AIM for ADS"
4613         or (style.mode == "ball" and "Throw a basketball at worker NPCs" or "Hit players to recover samples;
4614             ↪ hit NPCs to lower life")
4615     tool.RequiresHandle = true
4616     tool.CanBeDropped = false
4617     tool:SetAttribute("ImpactToolKind", kind)
4618     tool:SetAttribute("LabImpactTool", true)
4619     if style.mode == "gun" then
4620         tool:SetAttribute("BlasterWeapon", true)
4621         tool:SetAttribute("BlasterDamage", style.damage or 25)
4622         tool:SetAttribute("BlasterRange", style.range or 200)
4623         tool:SetAttribute("BlasterCooldown", style.cooldown or 0.55)
4624         tool:SetAttribute("BlasterPellets", style.pellets or 1)
4625         tool:SetAttribute("BlasterSpread", style.spread or 0)
4626         tool:SetAttribute("BlasterSelfCostPercent", style.selfCostPercent or 0)
4627         tool:SetAttribute("BlasterInstantHeadshot", style.pcOnly == true)
4628         tool:SetAttribute("BlasterAutomatic", style.automatic == true)
4629         tool:SetAttribute("BlasterADSFov", (style.range or 0) >= 500 and 44 or ((style.pellets or 1) > 1 and 58
4630             ↪ or 54))
4631         tool:SetAttribute("BlasterADSTouchSensitivity", 0.42)
4632         tool:SetAttribute("BlasterTracerColor", style.colour)
4633         -- Legacy marker retained for old saved tools; the legacy remote is inert.
4634         if style.pcOnly then tool:SetAttribute("LabSidearm", true) end
4635     end
4636
4637     local handle = Instance.new("Part")
4638     handle.Name = "Handle"
4639     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
4640         or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35) or Vector3.new(0.42, 3.4, 0.42))
4641     handle.Color = style.colour
4642     handle.Material = Enum.Material.SmoothPlastic
4643     handle.Anchored = false
4644     handle.CanCollide = false
4645     handle.Massless = true
4646     handle.Parent = tool
4647     if style.mode == "ball" then handle.Shape = Enum.PartType.Ball; handle.Material = Enum.Material.Rubber end
4648
4649     if style.meshId then
4650         -- Imported gun shape. A SpecialMesh is used rather than a MeshPart
4651         -- because MeshPart.MeshId cannot be assigned at runtime -- SpecialMesh
4652         -- can, so the tool builds correctly on a live server as well as in
4653         -- Studio, with no dependency on anything sitting in ServerStorage.
4654         handle.Size = style.handleSize or Vector3.new(0.20, 0.92, 1.36)
4655         local mesh = Instance.new("SpecialMesh")
4656         mesh.MeshType = Enum.MeshType.FileMesh
4657         mesh.MeshId = style.meshId
4658         if style.meshTexture then mesh.TextureId = style.meshTexture end
4659         local scale = style.meshScale or 1.6

```

```

4657     mesh.Scale = Vector3.new(scale, scale, scale)
4658     mesh.Parent = handle
4659     elseif style.mode == "gun" then
4660         addWeldedPart(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7),
4661             CFrame.new(0, 0.1, -1.65), style.colour, Enum.Material.Neon)
4662         addWeldedPart(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55),
4663             CFrame.new(0, -0.75, 0.42) * CFrame.Angles(math.rad(-18), 0, 0), style.accent,
4664             Enum.Material.Rubber)
4665     elseif kind == "Inspection Paddle" then
4666         addWeldedPart(tool, handle, "Paddle", Vector3.new(1.45, 1.65, 0.28),
4667             CFrame.new(0, 2.15, 0), style.colour, Enum.Material.SmoothPlastic)
4668     elseif kind == "Sample Grabber" then
4669         addWeldedPart(tool, handle, "LeftClaw", Vector3.new(0.25, 1.1, 0.3),
4670             CFrame.new(-0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(-28)), style.accent, Enum.Material.Metal)
4671         addWeldedPart(tool, handle, "RightClaw", Vector3.new(0.25, 1.1, 0.3),
4672             CFrame.new(0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(28)), style.accent, Enum.Material.Metal)
4673     elseif style.mode == "melee" then
4674         addWeldedPart(tool, handle, "FoamTip", Vector3.new(0.75, 1.5, 0.75),
4675             CFrame.new(0, 2, 0), style.colour, Enum.Material.Fabric)
4676     end
4677     if style.mode == "melee" then
4678         addWeldedPart(tool, handle, "Grip", Vector3.new(0.58, 1.15, 0.58),
4679             CFrame.new(0, -1.1, 0), style.accent, Enum.Material.Rubber)
4680     end
4681     tool.Grip = style.grip or (style.mode == "gun" and CFrame.new(0, -0.2, 0.6))
4682     or (style.mode == "ball" and CFrame.new(0, -0.25, -0.4) or CFrame.new(0, -1.2, 0) *
4683         CFrame.Angles(math.rad(-8), 0, 0))
4684     tool.Parent = backpack
4685     armImpactTool(tool)
4686     equipTool(player, tool)
4687     notify(player, kind .. " equipped. Click or tap to use it against worker NPCs; their LIFE bar shows damage
4688     and recovery.")
4689     return tool
4690 end
4691
4692 local function bindDispenser(part)
4693     if part:GetAttribute("DispenserBound") then return end
4694     local kind = part:GetAttribute("ToolKind")
4695     if not kind or not IMPACT_STYLE[kind] then return end
4696     part:SetAttribute("DispenserBound", true)
4697
4698     local prompt = Instance.new("ProximityPrompt")
4699     prompt.ObjectText = kind
4700     prompt.ActionText = "Equip combat tool"
4701     prompt.KeyboardKeyCode = Enum.KeyCode.E
4702     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
4703     prompt.HoldDuration = 0.25
4704     prompt.MaxActivationDistance = 10
4705     prompt.RequiresLineOfSight = false
4706     prompt.Parent = part
4707     local lastUse = {}
4708     prompt.Triggered:Connect(function(player)
4709         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
4710         if os.clock() < (lastUse[player] or 0) then return end
4711         lastUse[player] = os.clock() + 0.75
4712         makeImpactTool(player, kind)
4713     end)
4714 end
4715
4716 -----
4717 -- PC SIDEARM -- mouse-aimed, costs the shooter 1% of their life per shot
4718 --
4719 -- Fired only through this remote, which only the desktop client script
4720 -- sends, so the weapon is inert on touch and console clients.
4721 -----
4722 local sidearmRemote = ReplicatedStorage:FindFirstChild("LabSidearmFire")
4723 if not sidearmRemote then
4724     sidearmRemote = Instance.new("RemoteEvent")
4725     sidearmRemote.Name = "LabSidearmFire"
4726     sidearmRemote.Parent = ReplicatedStorage
4727 end
4728
4729 local sidearmCooldown = {}
4730 Players.PlayerRemoving:Connect(function(player) sidearmCooldown[player] = nil end)
4731

```

```

4730 sidearmRemote.OnServerEvent:Connect(function(player, aimPoint)
4731     -- Superseded by ServerBlasterManager. Keep the legacy remote inert so
4732     -- old clients and exploit traffic cannot enter the retired mouse path.
4733     if true then return end
4734     if typeof(aimPoint) ~= "Vector3" then return end
4735     local character = player.Character
4736     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
4737     local root = character and character:FindFirstChild("HumanoidRootPart")
4738     if not humanoid or not root or humanoid.Health <= 0 then return end
4739
4740     -- The client only ever asks; the server decides whether the player is
4741     -- really holding a weapon, and what that weapon is allowed to do.
4742     local tool = character:FindFirstChildOfClass("Tool")
4743     if not tool or not tool:GetAttribute("LabSidearm") then return end
4744     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or ""]
4745     if not style then return end
4746
4747     local now = os.clock()
4748     if now < (sidearmCooldown[player] or 0) then return end
4749     sidearmCooldown[player] = now + (style.cooldown or 0.35)
4750
4751     local handle = tool:FindFirstChild("Handle")
4752     local origin = (handle and handle.Position or root.Position) + Vector3.new(0, 0.5, 0)
4753     local baseDelta = aimPoint - origin
4754     if baseDelta.Magnitude < 0.1 then return end
4755     local baseDir = baseDelta.Unit
4756     local range = style.range or 200
4757
4758     local params = RaycastParams.new()
4759     params.FilterType = Enum.RaycastFilterType.Exclude
4760     params.FilterDescendantsInstances = {character}
4761
4762     local killsThisShot = 0
4763     for _ = 1, (style.pellets or 1) do
4764         -- Shotgun pellets scatter around the aim line; everything else is exact.
4765         local dir = baseDir
4766         local spread = style.spread or 0
4767         if spread > 0 then
4768             dir = (baseDir + Vector3.new(
4769                 (math.random() - 0.5) * spread * 2,
4770                 (math.random() - 0.5) * spread * 2,
4771                 (math.random() - 0.5) * spread * 2)).Unit
4772         end
4773
4774         local result = workspace.Raycast(origin, dir * range, params)
4775         local hitPosition = result and result.Position or (origin + dir * range)
4776
4777         local beam = Instance.new("Part")
4778         beam.Name = "SidearmTracer"
4779         beam.Size = Vector3.new(0.07, 0.07, (hitPosition - origin).Magnitude)
4780         beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
4781         beam.Color = Color3.fromRGB(255, 226, 140)
4782         beam.Material = Enum.Material.Neon
4783         beam.Anchored = true
4784         beam.CanCollide = false
4785         beam.CanTouch = false
4786         beam.CanQuery = false
4787         beam.CastShadow = false
4788         beam.Parent = workspace
4789         Debris:AddItem(beam, 0.1)
4790
4791         if result then
4792             local instance, normal = result.Instance, result.Normal
4793             local bot = workerBotFromHit(instance)
4794             if bot and not bot:GetAttribute("Dead") then
4795                 -- A head shot kills outright, whatever the weapon.
4796                 local headShot = instance.Name == "Head"
4797                 damageWorkerBot(bot, headShot and 1000 or (style.damage or 25), player)
4798                 if bot:GetAttribute("Dead") then killsThisShot = killsThisShot + 1 end
4799             elseif instance:GetAttribute("GasPipe") then
4800                 detonateGasPipe(instance, player, hitPosition)
4801                 spawnImpact(hitPosition, normal)
4802             else
4803                 local equipment = equipmentFromHit(instance)
4804                 if equipment then
4805                     explodeEquipment(equipment, player)

```



```

4806         elseif instance:GetAttribute("ShootablePoster") then
4807             punchPoster(instance, hitPosition, normal)
4808         end
4809         spawnImpact(hitPosition, normal)
4810     end
4811 end
4812 end
4813
4814 -- Firing always costs the shooter 1% of their own maximum health,
4815 -- whether or not the shot connects with anything.
4816 humanoid:TakeDamage(humanoid.MaxHealth * (style.selfCostPercent or 1) / 100)
4817
4818 if killsThisShot > 0 then
4819     local complex = workspace:FindFirstChild("Laboratory_Complex")
4820     notify(player, string.format("Target down. %d / %d cleared.",
4821         complex and complex:GetAttribute("BotsKilled") or 0,
4822         complex and complex:GetAttribute("BotsTotal") or 0))
4823 end
4824 end)
4825
4826 -----
4827 -- BLASTER GAMEPLAY BRIDGE
4828 -- ServerBlasterManager owns validation and raycasts. This callback only
4829 -- translates a validated impact into this game's custom worker/environment
4830 -- systems, keeping those systems private to the server.
4831 -----
4832 local blasterBridge = game:GetService("ServerScriptService"):FindFirstChild("BlasterHitBridge")
4833 if not blasterBridge then
4834     blasterBridge = Instance.new("BindableFunction")
4835     blasterBridge.Name = "BlasterHitBridge"
4836     blasterBridge.Parent = game:GetService("ServerScriptService")
4837 end
4838
4839 blasterBridge.OnInvoke = function(player, instance, hitPosition, hitNormal, damage, instantHeadshot)
4840     if typeof(player) ~= "Instance" or not player:IsA("Player") then return false end
4841     if typeof(instance) ~= "Instance" or not instance:IsDescendantOf(workspace) then return false end
4842
4843     local bot = workerBotFromHit(instance)
4844     if bot then
4845         local wasDead = bot:GetAttribute("Dead") == true
4846         local handled = damageWorkerBot(bot, instantHeadshot and 1000 or damage, player)
4847         if handled and not wasDead and bot:GetAttribute("Dead") then
4848             local complex = workspace:FindFirstChild("Laboratory_Complex")
4849             notify(player, string.format("Target down. %d / %d cleared.",
4850                 complex and complex:GetAttribute("BotsKilled") or 0,
4851                 complex and complex:GetAttribute("BotsTotal") or 0))
4852         end
4853         return handled
4854     end
4855
4856     if instance:GetAttribute("GasPipe") then
4857         detonateGasPipe(instance, player, hitPosition)
4858         spawnImpact(hitPosition, hitNormal)
4859         return true
4860     end
4861
4862     local equipment = equipmentFromHit(instance)
4863     if equipment then
4864         explodeEquipment(equipment, player)
4865         spawnImpact(hitPosition, hitNormal)
4866         return true
4867     end
4868
4869     if instance:GetAttribute("ShootablePoster") then
4870         punchPoster(instance, hitPosition, hitNormal)
4871         spawnImpact(hitPosition, hitNormal)
4872         return true
4873     end
4874
4875     return false
4876 end
4877
4878 -----
4879 -- COLD-STORE SAMPLE TRANSFERS
4880 -----
4881 local COLD_STORE = {Refrigerator = true, ["Chest Freezer"] = true}

```

```

4882 local nextSampleNumber = 1000
4883
4884 local PLANT_CHAMBER_KIND = "Plant Growth Chamber"
4885
4886 local function equipmentFloorLabel(part)
4887     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
4888 end
4889
4890 local function equipmentId(part)
4891     local code = part:GetAttribute("GrowthCode")
4892     if code and code ~= "" then return "CHAMBER:" .. code end
4893     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
4894         ↵ " | ")
4895 end
4896
4897 local function equipmentLabel(part)
4898     local code = part:GetAttribute("GrowthCode")
4899     if code and code ~= "" then return "CHAMBER:" .. code end
4900     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
4901 end
4902
4903 local function adaptiveTransferSeconds(source, destination)
4904     local distance = destination and (destination.Position - source.Position).Magnitude or 60
4905     local playerCount = math.max(1, #Players:GetPlayers())
4906     local complex = workspace:FindFirstChild("Laboratory_Complex")
4907     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
4908     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
4909         + math.clamp(backlog / 20, 0, 25)), 60, 180)
4910 end
4911
4912 local PLANT_PROCESS = {
4913     ["Germination Chamber"] = {
4914         key = "Germination", gain = 1,
4915         caption = "Warming seed and logging germination response",
4916     },
4917     ["Incubator"] = {
4918         key = "Incubation", gain = 1,
4919         caption = "Holding controlled temperature and humidity",
4920     },
4921     ["Growth Hormone Mixer"] = {
4922         key = "Growth Hormone", gain = 2,
4923         caption = "Dosing auxin/cytokinin growth treatment",
4924     },
4925     ["GMO Injector"] = {
4926         key = "GMO Injection", gain = 1, genome = true,
4927         caption = "Injecting and verifying a new plant trait",
4928     },
4929     ["Nutrient Infuser"] = {
4930         key = "Nutrient Infusion", gain = 1, moistureGain = 18,
4931         caption = "Balancing nitrogen, phosphorus, and potassium",
4932     },
4933     ["UV Growth Scanner"] = {
4934         repeatable = true, gain = 0,
4935         caption = "Scanning chlorophyll and canopy development",
4936     },
4937     ["Pollination Station"] = {
4938         key = "Pollination", gain = 1, minStage = 3,
4939         caption = "Applying controlled pollen and checking viability",
4940     },
4941     -- Micropropagation chain, step 1: surface-sterilize the seed coat before
4942     -- it ever reaches a growth chamber. Nothing downstream requires this, but
4943     -- a Sterilized sample gets a real (and referenced) germination boost --
4944     -- contamination is the #1 real-world cause of failed plant tissue culture.
4945     ["Decontamination Unit"] = {
4946         key = "Sterilized", gain = 0,
4947         caption = "Surface-sterilizing seed coat (dilute bleach + ethanol rinse) and sealing on sterile media",
4948     },
4949     -- Dose-response mutagenesis modeled on the reference Viola cornuta gamma
4950     -- study: germination viability falls off steeply above ~400 Gy, and by
4951     -- 1000 Gy almost nothing survives. Dose is chosen per treatment so two
4952     -- visits to the same chamber can give a different, honestly-labeled result.
4953     ["Mutagenesis Chamber"] = {
4954         key = "Mutagenesis", gain = 0, mutagenesis = true,
4955         caption = "Irradiating seed with a measured gamma dose",
4956     },
4957     -- Hormone Treatment Bench: auxin pushes rooting, cytokinin pushes
4958     -- shoot/bud growth, and a balanced ratio pushes both -- the same

```

```

4957 -- direction of effect reported for lemon seed germination trials.
4958 ["Hormone Treatment Bench"] = {
4959     key = "Hormone Treatment", gain = 1, hormone = true,
4960     caption = "Dosing a measured auxin/cytokinin treatment",
4961 },
4962 }
4963
4964 local function storageId(part)
4965     local id = part:GetAttribute("StorageId")
4966     if not id then
4967         id = table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
4968             ↪ part.Name}, "|")
4969         part:SetAttribute("StorageId", id)
4970     end
4971     return id
4972 end
4973
4974 local function storageLabel(part)
4975     local unit = part.Name:match("_(%d+)$") or "1"
4976     return string.format("%s, %s, %s (unit %s)", part:GetAttribute("Kind") or "Cold store",
4977         part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "room", unit)
4978 end
4979
4980 local function coldStoresExcept(source)
4981     local stores = {}
4982     for _, candidate in ipairs(CollectionService:GetTagged("LabEquipment")) do
4983         if candidate ~= source and COLD_STORE[candidate:GetAttribute("Kind")] then
4984             table.insert(stores, candidate)
4985         end
4986     end
4987     table.sort(stores, function(a, b) return storageId(a) < storageId(b) end)
4988     return stores
4989 end
4990
4991 -----
4992 -- PHYSICAL CONTROL BUTTON AND DATA DISPLAY
4993 -----
4994 local function makeControls(part, kind, spec)
4995     local wallSide = part:GetAttribute("WallSide")
4996     local frontAxisX = wallSide == "W" or wallSide == "E"
4997     local frontDirection = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
4998     local isGrowthChamber = kind == PLANT_CHAMBER_KIND
4999     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
5000     local timerGui = Instance.new("BillboardGui")
5001     timerGui.Name = "EquipmentStatusTimer"
5002     timerGui.Size = UDim2.fromOffset(160, 32)
5003     timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
5004     timerGui.MaxDistance = 10
5005     timerGui.AlwaysOnTop = false
5006     timerGui.Parent = part
5007     local timerLabel = Instance.new("TextLabel")
5008     timerLabel.Name = "Timer"
5009     timerLabel.Size = UDim2.fromScale(1, 1)
5010     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
5011     timerLabel.BackgroundTransparency = 0.16
5012     timerLabel.BorderSizePixel = 0
5013     timerLabel.TextScaled = true
5014     timerLabel.Font = Enum.Font.GothamBold
5015     timerLabel.Parent = timerGui
5016     local timerCorner = Instance.new("UICorner")
5017     timerCorner.CornerRadius = UDim.new(0, 8)
5018     timerCorner.Parent = timerLabel
5019
5020     -- Equipment name plate, mounted directly above the RUNNING/FAILED timer
5021     -- badge so each unit's kind is obvious at a glance from close range.
5022     local nameGui = Instance.new("BillboardGui")
5023     nameGui.Name = "EquipmentNamePlate"
5024     nameGui.Size = UDim2.fromOffset(160, 24)
5025     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
5026     nameGui.MaxDistance = 10
5027     nameGui.AlwaysOnTop = false
5028     nameGui.Parent = part
5029     local nameLabel = Instance.new("TextLabel")
5030     nameLabel.Name = "Kind"
5031     nameLabel.Size = UDim2.fromScale(1, 1)
5032     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)

```

```

5032     nameLabel.BackgroundTransparency = 0.05
5033     nameLabel.BorderSizePixel = 0
5034     nameLabel.Text = kind
5035     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
5036     nameLabel.TextScaled = true
5037     nameLabel.Font = Enum.Font.SourceSansBold
5038     nameLabel.Parent = nameGui
5039     local nameCorner = Instance.new("UICorner")
5040     nameCorner.CornerRadius = UDim.new(0, 8)
5041     nameCorner.Parent = nameLabel
5042
5043     local slotGui = Instance.new("BillboardGui")
5044     slotGui.Name = "EquipmentFreeSlots"
5045     slotGui.Size = UDim2.fromOffset(150, 28)
5046     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
5047     slotGui.MaxDistance = 10
5048     slotGui.AlwaysOnTop = false
5049     slotGui.Parent = part
5050     local slotLabel = Instance.new("TextLabel")
5051     slotLabel.Name = "FreeSlots"
5052     slotLabel.Size = UDim2.fromScale(1, 1)
5053     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)
5054     slotLabel.BackgroundTransparency = 0.12
5055     slotLabel.BorderSizePixel = 0
5056     slotLabel.TextScaled = true
5057     slotLabel.Font = Enum.Font.GothamBold
5058     slotLabel.Parent = slotGui
5059     local slotCorner = Instance.new("UICorner")
5060     slotCorner.CornerRadius = UDim.new(0, 8)
5061     slotCorner.Parent = slotLabel
5062     local storedLifeGui, storedLifeText, storedLifeFill
5063     if isPlantContainer then
5064         local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
5065         storedLifeGui = Instance.new("BillboardGui")
5066         storedLifeGui.Name = "StoredPlantLifeStatus"
5067         storedLifeGui.Size = UDim2.fromOffset(140, 40)
5068         storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
5069         storedLifeGui.MaxDistance = 10
5070         storedLifeGui.AlwaysOnTop = false
5071         storedLifeGui.Parent = plantAnchor
5072         storedLifeText = Instance.new("TextLabel")
5073         storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
5074         storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
5075         storedLifeText.BackgroundTransparency = 0.16
5076         storedLifeText.BorderSizePixel = 0
5077         storedLifeText.TextScaled = true
5078         storedLifeText.Font = Enum.Font.GothamBold
5079         storedLifeText.Parent = storedLifeGui
5080         local storedLifeBack = Instance.new("Frame")
5081         storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
5082         storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
5083         storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)
5084         storedLifeBack.BorderSizePixel = 0
5085         storedLifeBack.Parent = storedLifeGui
5086         storedLifeFill = Instance.new("Frame")
5087         storedLifeFill.Size = UDim2.fromScale(1, 1)
5088         storedLifeFill.BorderSizePixel = 0
5089         storedLifeFill.Parent = storedLifeBack
5090     end
5091     local function clockText(seconds)
5092         seconds = math.max(0, math.floor(seconds or 0))
5093         return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
5094             ↪ seconds % 60)
5095     end
5096     local function renderTimer()
5097         local state = part:GetAttribute("FunctionalState")
5098         local destroyed = state == "DESTROYED"
5099         local failed = destroyed or state == "FAILED"
5100         local since = failed and (part:GetAttribute("FailureSince") or os.time())
5101             or (part:GetAttribute("OperationStarted") or os.time())
5102         if destroyed then
5103             timerLabel.Text = "DESTROYED -- UNFIXABLE"
5104             timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
5105         else
5106             timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. clockText(os.time() - since)
5107             timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)

```

```

5107     end
5108     if isPlantContainer and failed then
5109         local life = math.clamp(100 - math.floor((os.time() - since) * 1.5), 0, 100)
5110         part:SetAttribute("PlantLife", life)
5111         if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
5112             part:SetAttribute("SampleIds", "")
5113             part:SetAttribute("SampleCount", 0)
5114             part:SetAttribute("Occupied", false)
5115             part:SetAttribute("PlantId", "")
5116             part:SetAttribute("PlantExpiredAt", os.time())
5117             for _, child in ipairs(part:GetDescendants()) do
5118                 if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then child.Transparency = 1
5119                 ↪ end
5120             end
5121         end
5122         local capacity = part:GetAttribute("Capacity") or 0
5123         local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
5124         local free = math.max(0, capacity - count)
5125         slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
5126             or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
5127         slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)
5128         -- Beacon: someone is actively carrying a plant toward THIS chamber
5129         -- (IncomingReservations is set the moment a plant is taken with this
5130         -- chamber as its destination). Make that one chamber's slot readout
5131         -- visible from across the room -- through walls too -- so "where do I
5132         -- put this plant" is answered by looking up, not by wandering every
5133         -- aisle reading labels that are otherwise only readable up close.
5134         local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
5135         if incoming and not failed then
5136             slotGui.MaxDistance = 30
5137             slotGui.AlwaysOnTop = false
5138             slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
5139             slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
5140         else
5141             slotGui.MaxDistance = 10
5142             slotGui.AlwaysOnTop = false
5143         end
5144         if storedLifeGui then
5145             local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
5146             local colour = life > 60 and Color3.fromRGB(75, 235, 105)
5147                 or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
5148             storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
5149             storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
5150             storedLifeText.TextColor3 = colour
5151             storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
5152             storedLifeFill.BackgroundColor3 = colour
5153         end
5154     end
5155     renderTimer()
5156     task.spawn(function()
5157         while part.Parent do
5158             renderTimer()
5159             task.wait(1)
5160         end
5161     end)
5162
5163     local button = Instance.new("Part")
5164     button.Name = "PushButton"
5165     button.Shape = Enum.PartType.Cylinder
5166     button.Size = isGrowthChamber and Vector3.new(0.24, 0.64, 0.64) or Vector3.new(0.2, 0.48, 0.48)
5167     local buttonSide = math.clamp((frontAxisX and part.Size.Z or part.Size.X) * 0.34, 0.25, 1.2)
5168     local buttonY = math.clamp(part.Size.Y * 0.03, -0.15, 0.45)
5169     if frontAxisX then
5170         button.CFrame = part.CFrame * CFrame.new(frontDirection * (part.Size.X / 2 + 0.16), buttonY,
5171             ↪ buttonSide)
5172     else
5173         button.CFrame = part.CFrame * CFrame.new(-buttonSide, buttonY, frontDirection * (part.Size.Z / 2 +
5174             ↪ 0.16))
5175         * CFrame.Angles(0, math.rad(90), 0)
5176     end
5177     button.Color = Color3.fromRGB(218, 68, 58)
5178     button.Material = Enum.Material.Neon
5179     button.Anchored = true
5180     button.CanCollide = false
5181     button.CanTouch = false

```

```

5180     button.CastShadow = false
5181     button.Parent = part
5182     local restCFrame = button.CFrame
5183
5184     local prompt = Instance.new("ProximityPrompt")
5185     prompt.ObjectText = isGrowthChamber
5186         and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " • CHAMBER " ..
5187             ↪ (part:GetAttribute("GrowthCode") or "R?"))
5188         or kind
5189     prompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
5190     prompt.KeyboardKeyCode = Enum.KeyCode.E
5191     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
5192     prompt.HoldDuration = isPlantContainer and 0 or 0.2
5193     prompt.MaxActivationDistance = 8
5194     prompt.RequiresLineOfSight = true
5195     prompt.Parent = button
5196
5197     local function setReadout(status, data, colour)
5198         part:SetAttribute("LastActionStatus", tostring(status or ""))
5199         part:SetAttribute("LastActionDetail", tostring(data or ""))
5200         renderTimer()
5201     end
5202
5203     local function pressButton()
5204         button.Color = Color3.fromRGB(255, 205, 70)
5205         local down = TweenService:Create(button, TweenInfo.new(0.07, Enum.EasingStyle.Quad),
5206             {CFrame = restCFrame * CFrame.new(-0.11, 0, 0)})
5207         down:Play()
5208         down.Completed:Wait()
5209         local up = TweenService:Create(button, TweenInfo.new(0.1, Enum.EasingStyle.Quad), {CFrame =
5210             ↪ restCFrame})
5211         up:Play()
5212         task.delay(0.12, function()
5213             if button.Parent then button.Color = Color3.fromRGB(218, 68, 58) end
5214         end)
5215     end
5216     end
5217
5218     return prompt, setReadout, pressButton
5219 end
5220
5221 local function runProgress(seconds, caption, setReadout)
5222     local steps = 4
5223     for step = 1, steps do
5224         task.wait(seconds / steps)
5225         setReadout("RUNNING " .. (step * 25) .. "%", caption, Color3.fromRGB(255, 205, 80))
5226     end
5227 end
5228
5229 local emergencyMovesCompleted = 0
5230 local setChamberFailure
5231 local setGenericFailure
5232
5233 local function chamberSampleList(chamber)
5234     local samples = {}
5235     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do table.insert(samples, id) end
5236     return samples
5237 end
5238
5239 local function saveChamberSamples(chamber, samples)
5240     local capacity = chamber:GetAttribute("Capacity") or 10
5241     while #samples > capacity do table.remove(samples) end
5242     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
5243     chamber:SetAttribute("SampleCount", #samples)
5244     chamber:SetAttribute("Occupied", #samples > 0)
5245     chamber:SetAttribute("PlantId", samples[1] or "")
5246 end
5247
5248 local function formatRuntime(seconds)
5249     seconds = math.max(0, math.floor(seconds or 0))
5250     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60, seconds %
5251         ↪ 60)
5252 end
5253
5254 local function recoverPlantLife(chamber, startingLife, player, plantId)
5255     startingLife = math.clamp(startingLife or 100, 0, 100)
5256     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then

```

```

5253     chamber:SetAttribute("PlantLife", startingLife)
5254     return 0
5255 end
5256 local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
5257 local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
5258 chamber:SetAttribute("RecoveryToken", token)
5259 chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
5260 chamber:SetAttribute("PlantLife", startingLife)
5261 if player then
5262     player:SetAttribute("LabObjective", string.format("%s dropped in %s. LIFE %d%% → 100%% in %d seconds.",
5263         plantId or "Plant", chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber), startingLife,
5264         ↪ seconds))
5265     notify(player, string.format("Plant LIFE %d%%. Recovery started; full life in %d seconds.",
5266         ↪ startingLife, seconds))
5267 end
5268 task.spawn(function()
5269     for step = 1, seconds do
5270         task.wait(1)
5271         if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
5272             or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
5273         chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step / seconds +
5274             ↪ 0.5))
5275     end
5276     chamber:SetAttribute("PlantLife", 100)
5277     chamber:SetAttribute("RecoveryEndsAt", 0)
5278     if player and player.Parent then notify(player, (plantId or "Plant") .. " recovered to 100% LIFE.") end
5279 end)
5280 return seconds
5281 end
5282
5283 local function updateStoredSamples(chamber)
5284     local count = chamber:GetAttribute("SampleCount") or 0
5285     local stage = math.clamp(chamber:GetAttribute("GrowthStage") or 0, 0, 5)
5286     for _, child in ipairs(chamber:GetDescendants()) do
5287         if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then
5288             local visible = child:GetAttribute("SampleSlot") <= count
5289             child.Transparency = visible and (child:GetAttribute("StoredTransparency") or 0) or 1
5290             if child.Name:match("Stem") then child.Color = Color3.fromRGB(55 + stage * 4, 145 + stage * 10, 65)
5291             elseif child.Name:match("Leaf") then child.Color = Color3.fromRGB(65 + stage * 3, 160 + stage * 12,
5292                 ↪ 75) end
5293         end
5294     end
5295 end
5296
5297 local function refreshChamberTelemetry(chamber)
5298     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
5299     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5300     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
5301     local samples = chamberSampleList(chamber)
5302     local sampleText = #samples > 0 and table.concat(samples, ", ") or "NO SAMPLES"
5303     local destination = chamber:GetAttribute("EvacuationDestination") or ""
5304     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "Unknown fault")
5305         .. (destination ~= "" and (" → MOVE TO " .. destination) or " → WAITING FOR SPACE")) or "SYSTEM
5306         ↪ NORMAL"
5307     local text = string.format("%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM
5308         ↪ %d% • VENT %d%\n%s\n",
5309         floorLabel, code, state, #samples, chamber:GetAttribute("Capacity") or 10,
5310         chamber:GetAttribute("GerminationDate") or "unknown",
5311         formatRuntime(os.time() - (chamber:GetAttribute("OperationStarted") or os.time())),
5312         chamber:GetAttribute("Temperature") or 0, chamber:GetAttribute("Humidity") or 0,
5313         chamber:GetAttribute("Ventilation") or 0, detail,
5314         state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
5315     for _, child in ipairs(chamber:GetDescendants()) do
5316         if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
5317             child.Text = text
5318             -- DESTROYED must never read as fine (green): only a genuinely
5319             -- WORKING chamber gets green, FAILED gets the usual red, and a
5320             -- permanently DESTROYED one gets an even harsher red so it never
5321             -- gets mistaken for "just needs a repair."
5322             child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
5323             or state == "FAILED" and Color3.fromRGB(255, 105, 90)
5324             or Color3.fromRGB(105, 245, 160)
5325         elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
5326             child.Text = "PLANTS ( " .. #samples .. " ): " .. sampleText
5327         elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
5328             child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)

```

```

5323         or state == "FAILED" and Color3.fromRGB(230, 55, 45)
5324         or Color3.fromRGB(40, 165, 95)
5325         child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
5326     end
5327 end
5328 end
5329
5330 local function evacuationDestinations(source)
5331     local sameLevel, anywhere = {}, {}
5332     for _, candidate in ipairs(CollectionService.GetTagged("PlantGrowthChamber")) do
5333         local count = candidate:GetAttribute("SampleCount") or 0
5334         local capacity = candidate:GetAttribute("Capacity") or 10
5335         local incoming = candidate:GetAttribute("IncomingReservations") or 0
5336         if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED" and count + incoming <
5337             ↪ capacity then
5338             table.insert(anywhere, candidate)
5339             if candidate:GetAttribute("Level") == source:GetAttribute("Level") then table.insert(sameLevel,
5340                 ↪ candidate) end
5341         end
5342     end
5343     local list = #sameLevel > 0 and sameLevel or anywhere
5344     table.sort(list, function(a, b)
5345         local ac = (a:GetAttribute("SampleCount") or 0) + (a:GetAttribute("IncomingReservations") or 0)
5346         local bc = (b:GetAttribute("SampleCount") or 0) + (b:GetAttribute("IncomingReservations") or 0)
5347         if ac == bc then return (a.Position - source.Position).Magnitude < (b.Position -
5348             ↪ source.Position).Magnitude end
5349         return ac < bc
5350     end)
5351     return list
5352 end
5353
5354 local function repairPathFolder(player, create)
5355     local complex = workspace:FindFirstChild("Laboratory_Complex")
5356     if not complex then return nil end
5357     local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
5358     if create and not rootFolder then
5359         rootFolder = Instance.new("Folder")
5360         rootFolder.Name = "TechnicianRepairPaths"
5361         rootFolder.Parent = complex
5362     end
5363     local name = "Player_" .. player.UserId
5364     local folder = rootFolder and rootFolder:FindFirstChild(name)
5365     if create and not folder then
5366         folder = Instance.new("Folder")
5367         folder.Name = name
5368         folder.Parent = rootFolder
5369     end
5370     return folder
5371 end
5372
5373 local function clearRepairPath(player)
5374     local folder = repairPathFolder(player, false)
5375     if folder then folder:Destroy() end
5376 end
5377
5378 local function repairReward(player, equipment, assigned)
5379     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
5380     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
5381     local points = 8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0)
5382     return math.clamp(points, 8, 120), waited, distance
5383 end
5384
5385 local function createRepairPath(player, target)
5386     clearRepairPath(player)
5387     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
5388     if not characterRoot or not target then return 0, false end
5389     local folder = repairPathFolder(player, true)
5390     if not folder then return 0, false end
5391
5392     local function approachPoint(part)
5393         local side = part:GetAttribute("WallSide")
5394         local floorY = part.Position.Y - part.Size.Y / 2
5395         if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
5396             ↪ part.Position.Z) end
5397         if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
5398             ↪ part.Position.Z) end

```



```

5394         if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
↪ 2.5) end
5395     return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
5396 end
5397 local function appendNavigable(points, fromPosition, toPosition)
5398     local path = PathfindingService.CreatePath({
5399         AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
5400         AgentCanClimb = true, WaypointSpacing = 4,
5401     })
5402     local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
5403     if not computed or path.Status ~= Enum.PathStatus.Success then return false end
5404     local waypoints = path:GetWaypoints()
5405     if #waypoints < 2 then return false end
5406     for index, waypoint in ipairs(waypoints) do
5407         if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
5408     end
5409     return true
5410 end
5411
5412 local complex = workspace:FindFirstChild("Laboratory_Complex")
5413 local rise = complex and complex:GetAttribute("FloorRiseStuds")
5414 local westX = complex and complex:GetAttribute("StairWestLaneX")
5415 local eastX = complex and complex:GetAttribute("StairEastLaneX")
5416 local northZ = complex and complex:GetAttribute("StairNorthZ")
5417 local southZ = complex and complex:GetAttribute("StairSouthZ")
5418 local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
5419 local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
5420 local destination = approachPoint(target)
5421 local points = {}
5422 local routeReady = false
5423 if rise and westX and eastX and northZ and southZ then
5424     local startFloor = math.clamp(math.floor((characterRoot.Position.Y - 2.5) / rise + 0.5), floorMin,
↪ floorMax)
5425     local targetFloor = math.clamp(math.floor(destination.Y / rise + 0.5), floorMin, floorMax)
5426     if startFloor == targetFloor then
5427         routeReady = appendNavigable(points, characterRoot.Position, destination)
5428     else
5429         local direction = targetFloor > startFloor and 1 or -1
5430         local firstLaneX = direction > 0 and westX or eastX
5431         local stairEntry = Vector3.new(firstLaneX, startFloor * rise, northZ)
5432         routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
5433         local floor = startFloor
5434         while routeReady and floor ~= targetFloor do
5435             local baseY = floor * rise
5436             if direction > 0 then
5437                 table.insert(points, Vector3.new(westX, baseY + rise / 2, southZ))
5438                 table.insert(points, Vector3.new(eastX, baseY + rise / 2, southZ))
5439                 table.insert(points, Vector3.new(eastX, baseY + rise, northZ))
5440             else
5441                 table.insert(points, Vector3.new(eastX, baseY - rise / 2, southZ))
5442                 table.insert(points, Vector3.new(westX, baseY - rise / 2, southZ))
5443                 table.insert(points, Vector3.new(westX, baseY - rise, northZ))
5444             end
5445             floor = floor + direction
5446             if floor == targetFloor then
5447                 table.insert(points, Vector3.new(direction > 0 and westX or eastX, floor * rise, northZ))
5448             end
5449         end
5450         if routeReady then
5451             routeReady = appendNavigable(points, points[#points], destination)
5452         end
5453     end
5454 end
5455 if not routeReady then
5456     folder:Destroy()
5457     player:SetAttribute("RepairRouteReady", false)
5458     return (destination - characterRoot.Position).Magnitude, false
5459 end
5460 player:SetAttribute("RepairRouteReady", true)
5461 local distance = 0
5462 local markerCount = 0
5463 for pointIndex = 1, #points - 1 do
5464     local a, b = points[pointIndex], points[pointIndex + 1]
5465     local delta = b - a
5466     local segmentDistance = delta.Magnitude
5467     distance = distance + segmentDistance

```

```

5468     if segmentDistance > 0.1 then
5469         local direction = delta.Unit
5470         local count = math.max(1, math.ceil(segmentDistance / 4))
5471         for index = 1, count do
5472             markerCount = markerCount + 1
5473             if markerCount > 120 then break end
5474             local marker = Instance.new("Part")
5475             marker.Name = "RedRepairPath"
5476             marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
5477             local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
5478             marker.CFrame = CFrame.lookAt(position, position + direction)
5479             marker.Color = Color3.fromRGB(255, 32, 28)
5480             marker.Material = Enum.Material.Neon
5481             marker.Transparency = 0.12
5482             marker.Anchored = true
5483             marker.CanCollide = false
5484             marker.CanTouch = false
5485             marker.CanQuery = false
5486             marker.Parent = folder
5487         end
5488     end
5489     if markerCount > 120 then break end
5490 end
5491 return distance, true
5492 end
5493
5494 local function clearAssignment(player)
5495     clearRepairPath(player)
5496     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
5497 ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
5498         player:SetAttribute(name, nil)
5499     end
5500 end
5501
5502 local function assignNextTask(player)
5503     clearAssignment(player)
5504     local role = playerRole(player)
5505     if role == "Technician" then
5506         local failed = {}
5507         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
5508             if equipment.Parent and equipment:GetAttribute("FunctionalState") == "FAILED" then
5509                 ↪ table.insert(failed, equipment) end
5510         end
5511         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
5512         table.sort(failed, function(a, b)
5513             if root then
5514                 local aDistance = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
5515 ↪ root.Position.Y) > 12 and 500 or 0)
5516                 local bDistance = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
5517 ↪ root.Position.Y) > 12 and 500 or 0)
5518                 if aDistance ~= bDistance then return aDistance < bDistance end
5519             end
5520             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
5521         end)
5522         local target = failed[1]
5523         if not target then
5524             local text = "All equipment is working. Repair score increases with failure time; assigned jobs
5525 ↪ also pay for travel distance."
5526             player:SetAttribute("LabObjective", text)
5527             return text
5528         end
5529         player:SetAttribute("AssignedRepairId", equipmentId(target))
5530         local distance, routeReady = createRepairPath(player, target)
5531         player:SetAttribute("AssignedRepairDistance", distance)
5532         local projected = select(1, repairReward(player, target, true))
5533         player:SetAttribute("AssignedRepairProjectedPoints", projected)
5534         local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR; route is
5535 ↪ recalculating"
5536         local text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
5537 ↪ while failed).",
5538             equipmentFloorLabel(target), equipmentLabel(target), routeInstruction, projected,
5539             ↪ math.floor(distance + 0.5))
5540         player:SetAttribute("LabObjective", text)
5541         return text
5542     elseif role == "Laboratory Technician" then
5543         local sources = {}

```

```

5536     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
5537         if chamber.Parent and chamber:GetAttribute("FunctionalState") == "FAILED"
5538             and (chamber:GetAttribute("SampleCount") or 0) > 0
5539             and not chamber:GetAttribute("ReservedByBot") then
5540             local destination = evacuationDestinations[chamber][1]
5541             if destination then table.insert(sources, {source = chamber, destination = destination}) end
5542         end
5543     end
5544     table.sort(sources, function(a, b)
5545         local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
5546         local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
5547         return an < bn
5548     end)
5549     local job = sources[1]
5550     if not job then
5551         local text = "Laboratory Technician: waiting for a failed chamber with plants. Emergency moves pay
5552             ↳ +12; assigned on-time moves pay +30."
5553         player:SetAttribute("LabObjective", text)
5554         return text
5555     end
5556     local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
5557     local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
5558     local allowedSeconds = adaptiveTransferSeconds(job.source, job.destination)
5559     local deadline = os.time() + allowedSeconds
5560     player:SetAttribute("AssignedTransferSource", sourceCode)
5561     player:SetAttribute("AssignedTransferDestination", destinationCode)
5562     player:SetAttribute("AssignedTransferDeadline", deadline)
5563     -- Same red floor-marker route the Technician role gets to a failed
5564     -- unit, but pointed at the destination chamber the plant must be
5565     -- carried to -- so "where does this go" is never a guess.
5566     local routeReady = select(2, createRepairPath(player, job.destination))
5567     local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
5568         ↳ recalculating"
5569     local text = string.format("ASSIGNED +%d: %s • %s → %s • %s. Press E to take a plant. %s. Deliver in %d
5570         ↳ sec.",
5571         POINTS.MOVE_ASSIGNED_ON_TIME, equipmentFloorLabel(job.source), sourceCode, destinationCode,
5572         equipmentFloorLabel(job.destination), routeInstruction, allowedSeconds)
5573     player:SetAttribute("LabObjective", text)
5574     return text
5575 end
5576
5577 local function chamberByCode(code)
5578     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
5579         if chamber:GetAttribute("GrowthCode") == code then return chamber end
5580     end
5581 end
5582
5583 local function takePlantFromChamber(player, chamber, setReadout)
5584     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5585     if playerRole(player) ~= "Laboratory Technician" then
5586         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician moves plants", Color3.fromRGB(255,
5587             ↳ 175, 80))
5588         notify(player, "Only a Laboratory Technician can take plants. Choose the GREEN role; Technicians repair
5589             ↳ with R.")
5590         return
5591     end
5592     local samples = chamberSampleList(chamber)
5593     if #samples == 0 then setReadout(code .. " EMPTY", "No germinated samples available", Color3.fromRGB(255,
5594         ↳ 200, 90)); return end
5595     local failed = chamber:GetAttribute("FunctionalState") == "FAILED"
5596     local destination
5597     if failed then
5598         local assignedSource = player:GetAttribute("AssignedTransferSource")
5599         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
5600         if assignedSource == code and assignedDestination then
5601             local requested = chamberByCode(assignedDestination)
5602             if requested and requested:GetAttribute("FunctionalState") ~= "FAILED"
5603                 and (requested:GetAttribute("SampleCount") or 0) +
5604                 ↳ (requested:GetAttribute("IncomingReservations") or 0)
5605                 < (requested:GetAttribute("Capacity") or 10) then
5606                 destination = requested
5607             end
5608         end
5609     end

```

```

5605     end
5606     destination = destination or evacuationDestinations(chamber)[1]
5607     if not destination then
5608         setReadout(code .. "   FAILED", "No working chamber has free capacity", Color3.fromRGB(255, 105,
5609             ↪ 90))
5609         notify(player, "No safe destination currently has room. The adaptive controller will reduce
5610             ↪ failures and free capacity.")
5610         return
5611     end
5612     -- Refresh the floor-marker route to whichever chamber this specific
5613     -- plant actually ends up assigned to (may differ from the original
5614     -- job destination if that one filled up in the meantime).
5615     createRepairPath(player, destination)
5616 end
5617 local plantId = samples[1]
5618 local stage = chamber:GetAttribute("GrowthStage") or 0
5619 local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
5620 local plantLife = chamber:GetAttribute("PlantLife") or 100
5621 if failed then
5622     plantLife = math.clamp(100 - math.floor((os.time() - (chamber:GetAttribute("FailureSince") or
5623         ↪ os.time())) * 1.5), 0, 100)
5624 end
5625 setReadout("TAKING SAMPLE", "Removing " .. plantId .. " from " .. code, Color3.fromRGB(120, 235, 170))
5626 if not runMathQuiz(player, "TAKE PLANT " .. plantId .. " FROM " .. code) then return end
5627 table.remove(samples, 1)
5628 local sample = makeTool(player, "Plant Sample " .. plantId, {
5629     itemType = failed and "Emergency Plant Transfer" or "Plant Sample",
5630     sample = true, plant = true, label = plantId, colour = Color3.fromRGB(133, 92, 55),
5631     attributes = {
5632         PlantSample = true, SampleId = plantId, PlantId = plantId, HomeGrowthCode = code, SourceGrowthCode
5633             ↪ = code,
5634         GrowthStage = stage, Treatments = chamber:GetAttribute("Treatments") or "",
5635         Genome = chamber:GetAttribute("Genome") or "Wild Type", Moisture = chamber:GetAttribute("Moisture")
5636             ↪ or 50,
5637         PlantLife = plantLife,
5638         GerminationDate = chamber:GetAttribute("GerminationDate") or "unknown",
5639         EvacuationSample = failed, DestinationGrowthCode = destinationCode, TransferStartedAt = os.time(),
5640     },
5641 })
5642 if not sample then table.insert(samples, 1, plantId); return end
5643 saveChamberSamples(chamber, samples)
5644 updateStoredSamples(chamber)
5645 refreshChamberTelemetry(chamber)
5646 refreshPlantVisual(sample)
5647 setCarryState(player, sample)
5648 task.spawn(function()
5649     while sample.Parent and (sample:GetAttribute("PlantLife") or 0) > 0 do
5650         task.wait(1)
5651         if not sample.Parent then return end
5652         if not sample:GetAttribute("QuizProtected") then
5653             local life = math.max(0, (sample:GetAttribute("PlantLife") or 0) - 2)
5654             sample:SetAttribute("PlantLife", life)
5655             refreshPlantVisual(sample)
5656             if life <= 0 then
5657                 local reservedCode = sample:GetAttribute("DestinationGrowthCode") or ""
5658                 local reserved = reservedCode ~= "" and chamberByCode(reservedCode) or nil
5659                 if reserved then reserved:SetAttribute("IncomingReservations", math.max(0,
5660                     ↪ (reserved:GetAttribute("IncomingReservations") or 1) - 1)) end
5661                 player:SetAttribute("CarriedSample", "")
5662                 player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
5663                     ↪ another plant.")
5664                 task.wait(0.5)
5665                 if sample.Parent then sample:Destroy() end
5666                 return
5667             end
5668         end
5669     end
5670 end)
5671 if destination then
5672     destination:SetAttribute("IncomingReservations", (destination:GetAttribute("IncomingReservations") or
5673         ↪ 0) + 1)
5674     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
5675     local seconds = math.max(0, deadline - os.time())
5676     player:SetAttribute("LabObjective", string.format("CARRYING %s: press E at ANY GREEN RUNNING cabinet
5677         ↪ with a FREE SLOT. BONUS target %s has %d sec left for +%d points.",
5678         plantId, destinationCode, seconds, POINTS.MOVE_ASSIGNED_ON_TIME))

```

```

5672     setReadout(code .. " FAILED", plantId .. " removed • SEND TO " .. destinationCode, Color3.fromRGB(255,
5673     ↪ 145, 75))
5673     notify(player, "ALARM: " .. code .. " failed (" .. (chamber:GetAttribute("FailureReason") or "fault")
5674     ↪ .. "). Move " .. plantId .. " to " .. destinationCode .. ".")
5674 else
5675     player:SetAttribute("LabObjective", "Treat " .. plantId .. ", then press E at any GREEN RUNNING cabinet
5676     ↪ with a FREE SLOT.")
5676     setReadout(code .. " WORKING", plantId .. " taken • " .. #samples .. "/" ..
5677     ↪ (chamber:GetAttribute("Capacity") or 0) .. " remain", Color3.fromRGB(115, 220, 160))
5677     notify(player, plantId .. " equipped in your hand. Process it or move it to a working chamber.")
5678 end
5679 end
5680
5681 local function storePlantInChamber(player, chamber, plant, setReadout)
5682     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5683     if playerRole(player) ~= "Laboratory Technician" then
5684         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician stores plants",
5685         ↪ Color3.fromRGB(255, 175, 80))
5686         notify(player, "Only a Laboratory Technician can complete plant transfers.")
5687         return
5688     end
5688     if chamber:GetAttribute("FunctionalState") == "FAILED" then
5689         setReadout(code .. " FAILED", "Cannot drop here • find any GREEN RUNNING cabinet", Color3.fromRGB(255,
5690         ↪ 105, 90))
5690         notify(player, code .. " is failed. Carry the plant to any GREEN RUNNING cabinet with a FREE SLOT and
5691         ↪ press E.")
5691         return
5692     end
5692     local samples = chamberSampleList(chamber)
5693     local capacity = chamber:GetAttribute("Capacity") or 10
5694     if #samples >= capacity then
5695         setReadout(code .. " FULL", capacity .. "/" .. capacity .. " samples • choose another working
5696         ↪ chamber", Color3.fromRGB(255, 175, 75))
5697         return
5698     end
5699     local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
5700     local plantId = plant:GetAttribute("PlantId") or plant.Name
5701     local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
5702     local sourceCode = plant:GetAttribute("SourceGrowthCode") or plant:GetAttribute("HomeGrowthCode") or ""
5703     local assignedMove = wanted ~= "" and player:GetAttribute("AssignedTransferSource") == sourceCode
5704     and player:GetAttribute("AssignedTransferDestination") == code
5705     local assignedOnTime = assignedMove and os.time() <= (player:GetAttribute("AssignedTransferDeadline") or 0)
5706     plant:SetAttribute("QuizProtected", true)
5707     setReadout("STORING SAMPLE", "Placing " .. plantId .. " into " .. code, Color3.fromRGB(120, 235, 170))
5708     local passedQuiz = runMathQuiz(player, "REVIVE " .. plantId .. " IN " .. code)
5709     plant:SetAttribute("QuizProtected", nil)
5710     if not passedQuiz or not plant.Parent then return end
5711     table.insert(samples, plantId)
5712     saveChamberSamples(chamber, samples)
5713     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
5714     chamber:SetAttribute("GrowthStage", math.max(chamber:GetAttribute("GrowthStage") or 0, stage))
5715     chamber:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
5716     chamber:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
5717     chamber:SetAttribute("Moisture", plant:GetAttribute("Moisture") or chamber:GetAttribute("Moisture") or 50)
5718     local recoverySeconds = recoverPlantLife(chamber, plantLife, player, plantId)
5719     if wanted ~= "" then
5720         local reservedDestination = chamberByCode(wanted)
5721         if reservedDestination then
5722             reservedDestination:SetAttribute("IncomingReservations", math.max(0,
5723             ↪ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
5724         end
5725     end
5726     emergencyMovesCompleted = emergencyMovesCompleted + 1
5727     player:SetAttribute("EmergencyTransfers", (player:GetAttribute("EmergencyTransfers") or 0) + 1)
5728 end
5729 updateStoredSamples(chamber)
5730 refreshChamberTelemetry(chamber)
5731 destroyCarriedItem(player, plant)
5732 local points = 0
5733 local reason = "plant stored"
5734 if wanted ~= "" then
5735     GameState.transfersCompleted = GameState.transfersCompleted + 1
5736     if assignedMove then
5737         GameState.assignedTransfersCompleted = GameState.assignedTransfersCompleted + 1
5738         points = assignedOnTime and POINTS.MOVE_ASSIGNED_ON_TIME or POINTS.MOVE_ASSIGNED_LATE
5739         reason = assignedOnTime and ("assigned on-time move " .. sourceCode .. " → " .. code)
5740         or ("assigned late move " .. sourceCode .. " → " .. code)

```

```

5739     else
5740         points = POINTS.MOVE_ANY
5741         reason = "emergency plant move " .. sourceCode .. " → " .. code
5742     end
5743 end
5744 setReadout(code .. " STORED" .. (points > 0 and (" + " .. points) or ""),
5745     plantId .. " • " .. #samples .. "/" .. capacity .. " samples", Color3.fromRGB(115, 245, 150))
5746 if points > 0 then
5747     awardPoints(player, points, reason)
5748     if recoverySeconds > 0 then
5749         task.delay(recoverySeconds, function() if player.Parent then assignNextTask(player) end end)
5750     else
5751         assignNextTask(player)
5752     end
5753 else
5754     notify(player, plantId .. " safely stored in " .. code .. ". Capacity: " .. #samples .. "/" .. capacity
5755         ↪ .. ".")
5756 end
5757 end
5758 local function handleGrowthChamber(player, chamber, setReadout)
5759     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5760     local reservedBy = chamber:GetAttribute("ReservedByBot")
5761     if reservedBy then
5762         -- Human players always have priority over test bots. Clearing the
5763         -- reservation makes one press of E immediately take or drop a plant.
5764         chamber:SetAttribute("ReservedByBot", nil)
5765     end
5766     local plant = findPlantSample(player)
5767     if plant then storePlantInChamber(player, chamber, plant, setReadout)
5768     elseif (chamber:GetAttribute("SampleCount") or 0) > 0 then takePlantFromChamber(player, chamber,
5769         ↪ setReadout)
5770     else
5771         setReadout(code .. " EMPTY", "Working chamber can receive a carried sample", Color3.fromRGB(255, 200,
5772             ↪ 90))
5773     end
5774 end
5775 local GMO_TRAITS = {"Drought Resistant", "High Yield", "Salt Tolerant", "Fast Flowering", "Disease Resistant"}
5776 -- Dose-response germination-viability curve, bucketed off the reference
5777 -- Viola cornuta gamma-irradiation study (Trends Sci. 2025;22(1):8705): mild
5778 -- doses barely dent germination, but survival collapses above ~600 Gy and is
5779 -- nearly gone by 1000 Gy. Each visit rolls a dose and a survival check.
5780 local MUTAGENESIS_DOSES = {
5781     {gy = 100, survivalMin = 85, survivalMax = 95},
5782     {gy = 200, survivalMin = 65, survivalMax = 85},
5783     {gy = 400, survivalMin = 45, survivalMax = 70},
5784     {gy = 600, survivalMin = 15, survivalMax = 40},
5785     {gy = 800, survivalMin = 5, survivalMax = 20},
5786     {gy = 1000, survivalMin = 0, survivalMax = 8},
5787 }
5788 -- Auxin pushes rooting, cytokinin pushes shoot/bud growth, and a balanced
5789 -- ratio pushes both -- the direction of effect reported for lemon seed
5790 -- germination hormone trials, kept as a labeled tag rather than a hidden stat.
5791 local HORMONE_TREATMENTS = {
5792     {name = "Auxin-dominant (IBA-heavy)", focus = "Root", gain = 1},
5793     {name = "Cytokinin-dominant (BAP-heavy)", focus = "Shoot", gain = 1},
5794     {name = "Balanced 1:1 auxin:cytokinin", focus = "Root+Shoot", gain = 2},
5795 }
5796 }
5797 local function germinationPercent(player)
5798     local sown = player:GetAttribute("GermSownCount") or 0
5799     if sown == 0 then return 0 end
5800     return math.floor(((player:GetAttribute("GermNormalCount") or 0) / sown) * 100 + 0.5)
5801 end
5802 end
5803 local function processPlant(player, part, spec, process, setReadout)
5804     if playerRole(player) ~= "Laboratory Technician" then
5805         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician develops plants",
5806             ↪ Color3.fromRGB(255, 175, 80))
5807         notify(player, "Only a Laboratory Technician can use plant-development equipment.")
5808         return false
5809     end
5810     local plant = findPlantSample(player)

```

```

5811     if not plant then
5812         setReadout("PLANT REQUIRED", "Take one from an R-numbered chamber", Color3.fromRGB(255, 150, 85))
5813         notify(player, part:GetAttribute("Kind") .. ": take a visible plant sample from an R chamber first.")
5814         return false
5815     end
5816
5817     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
5818     if process.minStage and stage < process.minStage then
5819         setReadout("STAGE TOO EARLY", "Needs Budding stage or later", Color3.fromRGB(255, 150, 85))
5820         notify(player, "Grow " .. (plant:GetAttribute("PlantId") or "the plant") .. " to Budding before
5821             ↪ pollination.")
5822         return false
5823     end
5824     if process.key and not process.repeatable and hasTreatment(plant, process.key) then
5825         setReadout("ALREADY APPLIED", process.key .. " is already recorded", Color3.fromRGB(255, 180, 85))
5826         notify(player, process.key .. " has already been applied to this plant. Use another machine.")
5827         return false
5828     end
5829     runProgress(spec.wait or 1.5, process.caption, setReadout)
5830
5831     -- Mutagenesis Chamber: roll a dose, then roll survival against that
5832     -- dose's real-world-modeled band. A killed sample ends here.
5833     if process.mutagenesis then
5834         local bucket = MUTAGENESIS_DOSES[math.random(1, #MUTAGENESIS_DOSES)]
5835         local survivalPercent = math.random(bucket.survivalMin, bucket.survivalMax)
5836         local survived = math.random(1, 100) <= math.max(survivalPercent, 1)
5837         plant:SetAttribute("MutagenesisDoseGy", bucket.gy)
5838         plant:SetAttribute("MutagenesisSurvivalPercent", survivalPercent)
5839         if not survived then
5840             local plantId = plant:GetAttribute("PlantId") or "sample"
5841             setReadout("SAMPLE LOST", string.format("%d Gy dose • %d%% survival odds • did not survive",
5842                 ↪ bucket.gy, survivalPercent),
5843                 Color3.fromRGB(255, 90, 80))
5844             notify(player, plantId .. " did not survive the " .. bucket.gy .. " Gy dose (" .. survivalPercent
5845                 ↪ .. "% survival odds). Take another plant.")
5846             destroyCarriedItem(player, plant)
5847             return true
5848         end
5849         addTreatment(plant, "Mutagenesis " .. bucket.gy .. "Gy")
5850         plant:SetAttribute("Genome", "Mutagenized (" .. bucket.gy .. " Gy)")
5851     end
5852
5853     -- Hormone Treatment Bench: roll a hormone mix, tag the resulting focus.
5854     if process.hormone then
5855         local mix = HORMONE_TREATMENTS[math.random(1, #HORMONE_TREATMENTS)]
5856         plant:SetAttribute("HormoneTreatment", mix.name)
5857         plant:SetAttribute("HormoneFocus", mix.focus)
5858         process = {key = process.key, gain = mix.gain, moistureGain = process.moistureGain, caption =
5859             ↪ process.caption}
5860         addTreatment(plant, "Hormone: " .. mix.name)
5861     elseif process.key then
5862         addTreatment(plant, process.key)
5863     end
5864
5865     -- Germination Chamber: classify the outcome (AOSA normal / abnormal /
5866     -- dead), the same three buckets a real seed-testing lab reports, and
5867     -- track a running germination percentage on the Laboratory Technician.
5868     -- A prior Decontamination Unit visit (Sterilized treatment) meaningfully
5869     -- improves the odds, matching real seed-testing guidance that
5870     -- contamination is the leading cause of an abnormal or dead result.
5871     if process.key == "Germination" then
5872         local sterilized = hasTreatment(plant, "Sterilized")
5873         local irradiated = plant:GetAttribute("MutagenesisSurvivalPercent")
5874         local normalChance = sterilized and 82 or 60
5875         if irradiated then normalChance = math.floor(normalChance * (irradiated / 100)) end
5876         local roll = math.random(1, 100)
5877         local outcome
5878         if roll <= normalChance then outcome = "Normal"
5879         elseif roll <= normalChance + 25 then outcome = "Abnormal"
5880         else outcome = "Dead" end
5881         plant:SetAttribute("GerminationOutcome", outcome)
5882         player:SetAttribute("GermSownCount", (player:GetAttribute("GermSownCount") or 0) + 1)
5883         if outcome == "Normal" then
5884             player:SetAttribute("GermNormalCount", (player:GetAttribute("GermNormalCount") or 0) + 1)
5885         elseif outcome == "Abnormal" then

```

```

5883     player:SetAttribute("GermAbnormalCount", (player:GetAttribute("GermAbnormalCount") or 0) + 1)
5884     plant:SetAttribute("PlantLife", math.clamp((plant:GetAttribute("PlantLife") or 100) - 25, 10, 100))
5885     else
5886         player:SetAttribute("GermDeadCount", (player:GetAttribute("GermDeadCount") or 0) + 1)
5887         local plantId = plant:GetAttribute("PlantId") or "sample"
5888         setReadout("DEAD • AOSA", string.format("Lab germination rate now %d%%",
5889             ↪ germinationPercent(player)), Color3.fromRGB(255, 90, 80))
5889         notify(player, plantId .. " classified DEAD under AOSA rules (" .. (sterilized and "sterilized" or
5890             ↪ "unsterilized")
5891             .. " seed). Your running germination rate: " .. germinationPercent(player) .. "%. Take another
5892             ↪ plant.")
5891         destroyCarriedItem(player, plant)
5892         return true
5893     end
5894 end
5895
5896 local newStage = math.clamp(stage + (process.gain or 0), 0, 5)
5897 plant:SetAttribute("GrowthStage", newStage)
5898 plant:SetAttribute("Moisture", math.clamp((plant:GetAttribute("Moisture") or 50) + (process.moistureGain or
5899     ↪ -2), 15, 100))
5899 if process.genome then plant:SetAttribute("Genome", GMO_TRAITS[math.random(1, #GMO_TRAITS)]) end
5900 refreshPlantVisual(plant)
5901 setCarryState(player, plant)
5902 player:SetAttribute("LabObjective", "Plant " .. (plant:GetAttribute("PlantId") or "sample") .. " is "
5903     .. (GROWTH_STAGE_NAME[newStage] or "Growing") .. ". Use another treatment or return it to an empty R
5904     ↪ chamber.")
5904 if process.key then awardPoints(player, POINTS.PROCESS_PLANT, process.key .. " plant treatment") end
5905
5906 local data = string.format("%s • %s • %d%% moisture\nGenome: %s",
5907     plant:GetAttribute("PlantId") or "PLANT", GROWTH_STAGE_NAME[newStage] or "Growing",
5908     plant:GetAttribute("Moisture") or 50, plant:GetAttribute("Genome") or "Wild Type")
5909 if plant:GetAttribute("GerminationOutcome") then
5910     data = data .. string.format("\nAOSA: %s • Lab germination rate %d%%",
5911         ↪ plant:GetAttribute("GerminationOutcome"), germinationPercent(player))
5912 end
5912 if plant:GetAttribute("MutagenesisDoseGy") then
5913     data = data .. string.format("\nDose %d Gy • %d%% survival odds",
5914         ↪ plant:GetAttribute("MutagenesisDoseGy"), plant:GetAttribute("MutagenesisSurvivalPercent") or 0)
5915 end
5915 if plant:GetAttribute("HormoneTreatment") then
5916     data = data .. "\n" .. plant:GetAttribute("HormoneTreatment") .. " • " ..
5917         ↪ (plant:GetAttribute("HormoneFocus") or "")
5918 end
5918 setReadout("PLANT DATA", data, Color3.fromRGB(120, 240, 160))
5919 notify(player, data:gsub("\n", " | ") .. " | Treatments: " .. treatmentText(plant))
5920 equipTool(player, plant)
5921 return true
5922 end
5923
5924 local function createTransferSample(player, source, setReadout)
5925     local existing = findTransferSample(player)
5926     if existing then
5927         local target = existing:GetAttribute("DestinationLabel") or "the named cold store"
5928         setReadout("SAMPLE IN TRANSIT", (existing:GetAttribute("SampleId") or existing.Name) .. " -> " ..
5929             ↪ target,
5930             ↪ Color3.fromRGB(255, 205, 80))
5929         notify(player, "You already carry " .. (existing:GetAttribute("SampleId") or existing.Name) .. ".
5930             ↪ Deliver it to " .. target .. ".")
5931         equipTool(player, existing)
5932         return
5933     end
5934
5935     local destinations = coldStoresExcept(source)
5936     if #destinations == 0 then
5937         setReadout("TRANSFER UNAVAILABLE", "A second refrigerator or freezer is required.", Color3.fromRGB(255,
5938             ↪ 110, 100))
5938         notify(player, "This transfer needs at least two refrigerators/freezers in the generated lab.")
5939         return
5940     end
5941     -- Prefer a true refrigerator destination so the guaranteed lobby
5942     -- pair always supports the requested fridge-to-fridge play loop.
5943     local refrigeratorDestinations = {}
5944     for _, candidate in ipairs(destinations) do
5945         if candidate:GetAttribute("Kind") == "Refrigerator" then
5946             table.insert(refrigeratorDestinations, candidate)
5947         end
5948     end

```



```

5948     end
5949     if #refrigeratorDestinations > 0 then destinations = refrigeratorDestinations end
5950
5951     local destination = destinations[math.random(1, #destinations)]
5952     nextSampleNumber = nextSampleNumber + 1
5953     local sampleId = string.format("SL-%04d", nextSampleNumber)
5954     local targetLabel = storageLabel(destination)
5955     local sample = makeTool(player, "Seed Sample " .. sampleId, {
5956         itemType = "Transfer Seed Sample",
5957         sample = true,
5958         label = sampleId,
5959         colour = Color3.fromRGB(120, 190, 225),
5960         attributes = {
5961             TransferSample = true,
5962             SampleId = sampleId,
5963             SourceStorageId = storageId(source),
5964             DestinationStorageId = storageId(destination),
5965             DestinationLabel = targetLabel,
5966         },
5967     })
5968     if not sample then return end
5969     setCarryState(player, sample)
5970     setReadout("DISPATCHED " .. sampleId, "DEST: " .. targetLabel, Color3.fromRGB(100, 220, 255))
5971     notify(player, "Picked up " .. sampleId .. ". Carry it visibly to " .. targetLabel .. ".")
5972 end
5973
5974 local function handleColdStore(player, part, setReadout)
5975     local sample = findTransferSample(player)
5976     if not sample then
5977         createTransferSample(player, part, setReadout)
5978         return
5979     end
5980
5981     local sampleId = sample:GetAttribute("SampleId") or sample.Name
5982     local wanted = sample:GetAttribute("DestinationStorageId")
5983     if wanted ~= storageId(part) then
5984         local destination = sample:GetAttribute("DestinationLabel") or "the assigned cold store"
5985         setReadout("WRONG STORAGE", sampleId .. " -> " .. destination, Color3.fromRGB(255, 120, 95))
5986         notify(player, sampleId .. " belongs in " .. destination .. ". It remains in your hand.")
5987         equipTool(player, sample)
5988         return
5989     end
5990
5991     destroyCarriedItem(player, sample)
5992     GameState.transfersCompleted = GameState.transfersCompleted + 1
5993     addContribution(player)
5994     updateScoreboard()
5995     setReadout("RECEIVED " .. sampleId, "TRANSFER VERIFIED | +1 contribution", Color3.fromRGB(115, 245, 150))
5996     notify(player, sampleId .. " delivered and verified. Transfer complete.")
5997 end
5998
5999 local function startCycleIndicator(part)
6000     local auto = part:GetAttribute("Auto")
6001     if not auto then return nil, function() end end
6002     local indicator = Instance.new("Part")
6003     indicator.Name = "CycleIndicator"
6004     indicator.Size = (auto == "spin") and Vector3.new(0.9, 0.15, 0.9) or Vector3.new(0.5, 0.5, 0.5)
6005     indicator.Shape = (auto == "spin") and Enum.PartType.Block or Enum.PartType.Ball
6006     indicator.Anchored = true
6007     indicator.CanCollide = false
6008     indicator.CanTouch = false
6009     indicator.CastShadow = false
6010     indicator.Material = Enum.Material.Neon
6011     indicator.Color = Color3.fromRGB(255, 196, 60)
6012     indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
6013     indicator.Parent = part.Parent
6014     local light = Instance.new("PointLight")
6015     light.Color = indicator.Color
6016     light.Brightness = 1.5
6017     light.Range = 12
6018     light.Parent = indicator
6019
6020     local running = true
6021     task.spawn(function()
6022         local t = 0
6023         while running and indicator.Parent do

```

```

6024         t = t + task.wait(0.1)
6025         if auto == "spin" then
6026             indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
6027                 * CFrame.Angles(0, t * 5, 0)
6028         else
6029             indicator.Transparency = 0.15 + (math.sin(t * 7) + 1) / 2 * 0.55
6030         end
6031     end
6032 end)
6033 return indicator, function()
6034     running = false
6035     if indicator.Parent then indicator:Destroy() end
6036 end
6037 end
6038
6039 local function bind(part)
6040     local kind = part:GetAttribute("Kind")
6041     if not kind or part:GetAttribute("InteractionsBound") then return end
6042     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
6043     part:SetAttribute("InteractionsBound", true)
6044     part:SetAttribute("EquipmentId", equipmentId(part))
6045     part:SetAttribute("FloorLabel", equipmentFloorLabel(part))
6046     part:SetAttribute("RepairKey", "R")
6047     if part:GetAttribute("FunctionalState") == nil then
6048         local hash = 0
6049         for i = 1, #equipmentId(part) do hash = hash + string.byte(equipmentId(part), i) end
6050         local failed = kind ~= PLANT_CHAMBER_KIND and hash % 7 == 0
6051         part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
6052         part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
6053         part:SetAttribute("FailureSince", failed and os.time() or 0)
6054     end
6055     if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time()) end
6056     if COLD_STORE[kind] then storageId(part) end
6057     if isPlantContainer then
6058         updateStoredSamples(part)
6059         refreshChamberTelemetry(part)
6060     end
6061     local spec = Kinds.forKind(kind)
6062     local prompt, setReadout, pressButton = makeControls(part, kind, spec)
6063     local baseColour = part.Color
6064     local busy = false
6065     local powerOn = true
6066     part:SetAttribute("Powered", true)
6067     local powerLight = Instance.new("PointLight")
6068     powerLight.Color = Color3.fromRGB(120, 220, 140)
6069     powerLight.Brightness = powerOn and 2 or 0
6070     powerLight.Range = 7
6071     powerLight.Parent = part
6072
6073     local led = Instance.new("Part")
6074     led.Name = "StatusLED"
6075     led.Shape = Enum.PartType.Ball
6076     led.Size = Vector3.new(0.22, 0.22, 0.22)
6077     led.Anchored = true
6078     led.CanCollide = false
6079     led.CanTouch = false
6080     led.CastShadow = false
6081     led.Material = Enum.Material.Neon
6082     led.Color = powerOn and Color3.fromRGB(110, 230, 130) or Color3.fromRGB(64, 68, 74)
6083     local controlSide = part:GetAttribute("WallSide")
6084     if controlSide == "W" or controlSide == "E" then
6085         local direction = controlSide == "W" and 1 or -1
6086         led.CFrame = part.CFrame * CFrame.new(direction * (part.Size.X / 2 + 0.08), part.Size.Y / 2 - 0.3,
        ↪ part.Size.Z * 0.32)
6087     else
6088         local direction = controlSide == "S" and -1 or 1
6089         led.CFrame = part.CFrame * CFrame.new(part.Size.X * 0.32, part.Size.Y / 2 - 0.3, direction *
        ↪ (part.Size.Z / 2 + 0.08))
6090     end
6091     led.Parent = part
6092
6093     local repairAnchor = part:FindFirstChild("ControllerScreen") or led
6094     local floorLabel = equipmentFloorLabel(part)
6095     local unitLabel = equipmentLabel(part)
6096     local repairPrompt = Instance.new("ProximityPrompt")
6097     repairPrompt.Name = "RepairPrompt"

```

```

6098 repairPrompt.ObjectText = floorLabel .. " • " .. unitLabel
6099 repairPrompt.ActionText = "Hold R to repair"
6100 repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
6101 repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
6102 repairPrompt.HoldDuration = 3
6103 repairPrompt.MaxActivationDistance = 8
6104 repairPrompt.RequiresLineOfSight = true
6105 repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
6106 repairPrompt.Parent = repairAnchor
6107
6108 part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
6109     if repairPrompt.Parent then
6110         repairPrompt.Enabled = not busy and part:GetAttribute("FunctionalState") == "FAILED"
6111     end
6112 end)
6113
6114 repairPrompt.Triggered:Connect(function(player)
6115     if playerRole(player) ~= "Technician" then
6116         setReadout("TECHNICIAN REQUIRED", "Choose the BLUE role to repair equipment", Color3.fromRGB(255,
        ↪ 175, 80))
6117         notify(player, "Only a Technician can repair equipment. Choose the BLUE role kiosk.")
6118         return
6119     end
6120     if busy or part:GetAttribute("FunctionalState") ~= "FAILED"
6121         or not canInteract(player, part, repairPrompt.MaxActivationDistance) then return end
6122     busy = true
6123     prompt.Enabled = false
6124     repairPrompt.Enabled = false
6125     setReadout("REPAIRING", floorLabel .. " • " .. unitLabel .. " • diagnosing "
6126         .. (part:GetAttribute("FailureReason") or "equipment fault"), Color3.fromRGB(255, 205, 75))
6127     setReadout("REPAIRING", "Working on " .. floorLabel .. " • " .. unitLabel, Color3.fromRGB(120, 235,
        ↪ 170))
6128     if not runMathQuiz(player, "REPAIR " .. floorLabel .. " • " .. unitLabel) then
6129         setReadout("REPAIR NOT COMPLETED", "Hold R for 3 seconds to try another quiz", Color3.fromRGB(255,
        ↪ 105, 90))
6130         prompt.Enabled = true
6131         repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
6132         busy = false
6133         return
6134     end
6135     local assigned = player:GetAttribute("AssignedRepairId") == equipmentId(part)
6136     local points, failedSeconds, routeDistance = repairReward(player, part, assigned)
6137     part:SetAttribute("LastRepairedAt", os.time())
6138     part:SetAttribute("LastRepairedBy", player.Name)
6139     if kind == PLANT_CHAMBER_KIND then setChamberFailure(part, false) else setGenericFailure(part, false)
        ↪ end
6140     GameState.repairsCompleted = GameState.repairsCompleted + 1
6141     if assigned then GameState.assignedRepairsCompleted = GameState.assignedRepairsCompleted + 1 end
6142     local total = awardPoints(player, points, string.format("%s • failed %ds • route %d studs",
6143         assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
        ↪ math.floor(routeDistance + 0.5)))
6144     setReadout("REPAIR COMPLETE +" .. points, floorLabel .. " • " .. unitLabel .. " WORKING • TOTAL " ..
        ↪ total,
        ↪ Color3.fromRGB(105, 245, 155))
6145     assignNextTask(player)
6146     prompt.Enabled = true
6147     busy = false
6148 end)
6149
6150
6151 prompt.Triggered:Connect(function(player)
6152     if busy or not canInteract(player, part, prompt.MaxActivationDistance) then return end
6153     task.spawn(pressButton)
6154     if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
6155         setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
        ↪ (part:GetAttribute("FailureReason") or "equipment fault"),
        ↪ Color3.fromRGB(255, 105, 90))
6156         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
6157         return
6158     end
6159 end)
6160
6161 local station = ROLE_STATIONS[kind]
6162 if station then
6163     if not player.Team or player.Team.Name ~= station.role then
6164         setReadout("ACCESS DENIED", station.role .. " role required", Color3.fromRGB(255, 105, 90))
6165         notify(player, kind .. ": only a " .. station.role .. " can use this. Join that role at the
        ↪ entrance kiosk.")

```

```

6166         return
6167     end
6168     busy = true
6169     prompt.Enabled = false
6170     runProgress(spec.wait or 1, "Reading shared batch queue", setReadout)
6171     local msg, err = station.run(GameState)
6172     if err then
6173         setReadout("NO BATCH DATA", err, Color3.fromRGB(255, 160, 90))
6174         notify(player, kind .. ": " .. err)
6175     else
6176         addContribution(player)
6177         setReadout("INSPECTION DATA", msg, Color3.fromRGB(115, 245, 150))
6178         notify(player, kind .. " -- " .. msg)
6179         updateScoreboard()
6180     end
6181     prompt.Enabled = true
6182     busy = false
6183     return
6184 end
6185
6186 local plantProcess = PLANT_PROCESS[kind]
6187 local carriedPlant = findPlantSample(player)
6188 local shouldUsePlantStorage = isPlantContainer and (
6189     kind == PLANT_CHAMBER_KIND
6190     or not plantProcess
6191     or not carriedPlant
6192     or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
6193 )
6194 if shouldUsePlantStorage then
6195     busy = true
6196     prompt.Enabled = false
6197     handleGrowthChamber(player, part, setReadout)
6198     prompt.Enabled = true
6199     busy = false
6200     return
6201 end
6202
6203 if part:GetAttribute("FunctionalState") == "FAILED" then
6204     setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
6205         ↳ (part:GetAttribute("FailureReason") or "equipment fault"),
6206         Color3.fromRGB(255, 105, 90))
6207     notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
6208     return
6209 end
6210
6211 if plantProcess then
6212     busy = true
6213     prompt.Enabled = false
6214     processPlant(player, part, spec, plantProcess, setReadout)
6215     prompt.Enabled = true
6216     busy = false
6217     return
6218 end
6219
6220 if COLD_STORE[kind] then
6221     busy = true
6222     prompt.Enabled = false
6223     runProgress(0.6, "Scanning cold-store inventory", setReadout)
6224     handleColdStore(player, part, setReadout)
6225     prompt.Enabled = true
6226     busy = false
6227     return
6228 end
6229
6230 local consumed
6231 if spec.consumes then
6232     consumed = findTool(player, spec.consumes)
6233     if not consumed then
6234         setReadout("MATERIAL NEEDED", spec.consumes, Color3.fromRGB(255, 160, 90))
6235         notify(player, kind .. ": needs " .. spec.consumes .. " first.")
6236         return
6237     end
6238 end
6239
6240 busy = true
6241 prompt.Enabled = false

```

```

6241     if spec.activeColour then part.Color = spec.activeColour end
6242     local uv
6243     if spec.uvLight then
6244         uv = Instance.new("PointLight")
6245         uv.Color = Color3.fromRGB(150, 210, 255)
6246         uv.Brightness = 2
6247         uv.Range = 26
6248         uv.Parent = part
6249     end
6250
6251     local _, stopIndicator = startCycleIndicator(part)
6252     runProgress(spec.wait or 1, spec.action, setReadout)
6253     stopIndicator()
6254
6255     if consumed then destroyCarriedItem(player, consumed) end
6256     for _, item in ipairs(spec.produces or {}) do
6257         local made = makeTool(player, item)
6258         if made and made:GetAttribute("LabSample") then setCarryState(player, made) end
6259     end
6260     local result = spec.log and spec.log() or "Inspection complete; equipment left ready."
6261     setReadout("INSPECTION DATA", result, Color3.fromRGB(115, 245, 150))
6262     notify(player, kind .. " -- " .. result)
6263
6264     if uv then uv:Destroy() end
6265     part.Color = baseColour
6266     prompt.Enabled = true
6267     busy = false
6268 end)
6269 end
6270
6271 -----
6272 -- FIFTY AUTONOMOUS GAMEPLAY TEST WORKERS
6273 -----
6274 local function setBotStatus(bot, message)
6275     bot:SetAttribute("TaskState", message)
6276     local status = bot:FindFirstChild("Status", true)
6277     if status and status:IsA("TextLabel") then
6278         status.Text = bot:GetAttribute("Competitor") and ("RIVAL NPC\n" .. message)
6279         or string.format("BOT %02d\n%s", bot:GetAttribute("BotNumber") or 0, message)
6280     end
6281 end
6282
6283 local function updateBotTestBoard()
6284     local workers = CollectionService:GetTagged("LabWorkerBot")
6285     local cycles, carrying, fallbacks, rivalScore = 0, 0, 0, 0
6286     for _, bot in ipairs(workers) do
6287         cycles = cycles + (bot:GetAttribute("CompletedCycles") or 0)
6288         fallbacks = fallbacks + (bot:GetAttribute("PathFallbacks") or 0)
6289         if (bot:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
6290         if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 end
6291     end
6292     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
6293     local failing, failedSamples = 0, 0
6294     for _, chamber in ipairs(chambers) do
6295         if chamber:GetAttribute("FunctionalState") == "FAILED" then
6296             failing = failing + 1
6297             failedSamples = failedSamples + (chamber:GetAttribute("SampleCount") or 0)
6298         end
6299     end
6300     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
6301         for _, label in ipairs(board:GetDescendants()) do
6302             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
6303                 label.Text = string.format(
6304                     "ADAPTIVE GROWTH-ROOM TEST\nWorkers: %d/50 Transfers: %d Carrying: %d\nFAILED: %d/%d
6305                     ↳ (%.0f%%) Samples waiting: %d\nRIVAL SCORE: %d • Path fallbacks: %d",
6306                     #workers, cycles, carrying, failing, #chambers, failing / math.max(1, #chambers) * 100,
6307                     ↳ failedSamples, rivalScore, fallbacks)
6308             end
6309         end
6310     end
6311     local function interactionPoint(part)
6312         local side = part:GetAttribute("WallSide")
6313         local bottomY = part.Position.Y - part.Size.Y / 2
6314         local point = Vector3.new(part.Position.X, bottomY + 3.25, part.Position.Z)

```

```

6315     if side == "W" then point = point + Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
6316     elseif side == "E" then point = point - Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
6317     elseif side == "N" then point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
6318     elseif side == "S" then point = point - Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
6319     else point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5) end
6320     return point
6321 end
6322
6323 -----
6324 -- OBSERVER GATE
6325 -- A worker only runs its scenario while a player is in the room with it.
6326 -- Everything else stands still. This is what keeps 180 NPCs off the
6327 -- pathfinding budget: an unobserved bot costs one cheap check a second
6328 -- instead of a continuous walk loop and a PathfindingService computation
6329 -- every few seconds.
6330 -----
6331 local FLOOR_BAND = 10    -- studs, about half a floor-to-floor height
6332 local WAKE_RADIUS = 35   -- studs, fallback for corridors and doorways, where
6333                          -- neither the bot nor the player is inside any room
6334
6335 -- Room rectangles published by the generator as one packed attribute:
6336 -- "Level_1:Room 1:x1:z1:x2:z2:y;..." already in stud space.
6337 local roomsByLevel = {}
6338 do
6339     local complex = workspace:FindFirstChild("Laboratory_Complex")
6340     local packed = complex and complex:GetAttribute("RoomIndex")
6341     if typeof(packed) == "string" then
6342         for entry in packed:gmatch("[^;]+") do
6343             local level, name, x1, z1, x2, z2, y =
6344                 entry:match("^([^:]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+):([[:]]+)$")
6345             if level then
6346                 roomsByLevel[level] = roomsByLevel[level] or {}
6347                 table.insert(roomsByLevel[level], {
6348                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
6349                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
6350                 })
6351             end
6352         end
6353     end
6354 end
6355
6356 -- Only the ~10 rooms on the bot's own floor are ever tested, not all 81.
6357 local function roomAt(levelName, position)
6358     local list = levelName and roomsByLevel[levelName]
6359     if not list then return nil end
6360     for _, r in ipairs(list) do
6361         if math.abs(position.Y - r.y) < FLOOR_BAND
6362             and position.X >= r.x1 and position.X <= r.x2
6363             and position.Z >= r.z1 and position.Z <= r.z2 then
6364             return r.name
6365         end
6366     end
6367     return nil
6368 end
6369
6370 -- The rectangle itself, for picking a patrol point inside a room.
6371 local function roomRect(levelName, roomName)
6372     for _, r in ipairs((levelName and roomsByLevel[levelName]) or {}) do
6373         if r.name == roomName then return r end
6374     end
6375     return nil
6376 end
6377
6378 local function botHasObserver(bot)
6379     local ok, pivot = pcall(function() return bot:GetPivot().Position end)
6380     if not ok then return false end
6381     local levelName = bot:GetAttribute("Level")
6382     local botRoom = roomAt(levelName, pivot)
6383     for _, player in ipairs(Players:GetPlayers()) do
6384         local character = player.Character
6385         local hrp = character and character:FindFirstChild("HumanoidRootPart")
6386         if hrp then
6387             local here = hrp.Position
6388             -- Cheapest test first: wrong floor is an instant no.
6389             if math.abs(here.Y - pivot.Y) < FLOOR_BAND then
6390                 if botRoom and roomAt(levelName, here) == botRoom then return true end

```

```

6391         local dx, dz = here.X - pivot.X, here.Z - pivot.Z
6392         if dx * dx + dz * dz < WAKE_RADIUS * WAKE_RADIUS then return true end
6393     end
6394 end
6395 end
6396 return false
6397 end
6398
6399 local function walkBotSegment(bot, destination)
6400     if not bot.Parent or not bot.PrimaryPart then return false end
6401     local startPivot = bot:GetPivot()
6402     local start = startPivot.Position
6403     local heldOffsets = {}
6404     local held = bot:FindFirstChild("BotHeldPlant")
6405     if held then
6406         for _, piece in ipairs(held:GetDescendants()) do
6407             if piece:IsA("BasePart") then heldOffsets[piece] = startPivot:ToObjectSpace(piece.CFrame) end
6408         end
6409     end
6410     -- Refuse to walk a leg that passes through solid geometry. RespectCanCollide
6411     -- means only real obstructions count -- walls, machines and closed doors --
6412     -- while posters, wall cabinets and other workers (all CanCollide off) are
6413     -- ignored. Without this the walk simply lered in a straight line and slid
6414     -- through walls and equipment whenever pathfinding gave a coarse waypoint.
6415     local blockParams = RaycastParams.new()
6416     blockParams.FilterType = Enum.RaycastFilterType.Exclude
6417     blockParams.FilterDescendantsInstances = {bot, workspace:FindFirstChild("LabBotBodies")}
6418     blockParams.RespectCanCollide = true
6419     local flat = Vector3.new(destination.X, start.Y, destination.Z)
6420     if (flat - start).Magnitude > 0.5 then
6421         local blocked = workspace:Raycast(start, flat - start, blockParams)
6422         if blocked then return false end
6423     end
6424
6425     local flatDistance = (flat - start).Magnitude
6426     local steps = math.max(1, math.ceil(flatDistance / 1.25))
6427     for step = 1, steps do
6428         if not bot.Parent then return false end
6429         if bot:GetAttribute("Dead") then return false end
6430         while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do task.wait(0.25)
6431             ↪ end
6432         -- Stop dead the moment the room empties, and resume from the same spot
6433         -- when someone comes back. Without this a bot that was already walking
6434         -- would finish its whole segment after the player had left.
6435         while bot.Parent and not bot:GetAttribute("Competitor") and not botHasObserver(bot) do
6436             task.wait(0.5)
6437         end
6438         if not bot.Parent then return false end
6439         local alpha = step / steps
6440         local position = start:Lerp(destination, alpha)
6441         local flatLook = Vector3.new(destination.X, position.Y, destination.Z)
6442         local pose = (flatLook - position).Magnitude > 0.05
6443             and CFrame.lookAt(position, flatLook) or CFrame.new(position)
6444         -- A subtle sway makes the anchored test rig visibly walk instead of
6445         -- silently teleporting between test stations.
6446         pose = pose * CFrame.Angles(0, 0, math.sin(step * 0.9) * 0.025)
6447         bot:PivotTo(pose)
6448         for piece, offset in pairs(heldOffsets) do
6449             if piece.Parent then piece.CFrame = pose * offset end
6450         end
6451         local complex = workspace:FindFirstChild("Laboratory_Complex")
6452         local throughput = complex and (complex:GetAttribute("RecentTransferThroughput") or 0) or 0
6453         local speedScale = 1 + math.min(10, #Players:GetPlayers()) * 0.05 + math.clamp(throughput, 0, 20) *
6454             ↪ 0.01
6455         task.wait(math.clamp(0.075 / speedScale, 0.035, 0.09))
6456     end
6457     return true
6458 end
6459
6460 local function moveBotTo(bot, target)
6461     local destination = interactionPoint(target)
6462     local path = PathfindingService:CreatePath({
6463         AgentRadius = 1.2,
6464         AgentHeight = 5.8,
6465         AgentCanJump = true,
6466         WaypointSpacing = 5,

```

```

6465     })
6466     local ok = pcall(function()
6467         path:ComputeAsync(bot:GetPivot().Position, destination)
6468     end)
6469     if ok and path.Status == Enum.PathStatus.Success then
6470         for _, waypoint in ipairs(path:GetWaypoints()) do
6471             if not walkBotSegment(bot, waypoint.Position + Vector3.new(0, 3.0, 0)) then return false end
6472         end
6473         return true
6474     end
6475     -- A failed navigation calculation used to fall back to a straight line to
6476     -- the destination, which is exactly how a worker ends up strolling through
6477     -- a wall. Give up on this leg instead and let the scenario pick again.
6478     bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
6479     return false
6480 end
6481
6482 local function makeBotPlant(bot, stage)
6483     local old = bot:FindFirstChild("BotHeldPlant")
6484     if old then old:Destroy() end
6485     local held = Instance.new("Model")
6486     held.Name = "BotHeldPlant"
6487     held.Parent = bot
6488     local base = bot:GetPivot() * CFrame.new(1.45, 0.1, -0.72)
6489     local function piece(name, size, offset, colour, material)
6490         local p = Instance.new("Part")
6491         p.Name = name
6492         p.Size = size
6493         p.CFrame = base * CFrame.new(offset)
6494         p.Color = colour
6495         p.Material = material or Enum.Material.SmoothPlastic
6496         p.Anchored = true
6497         p.CanCollide = false
6498         p.CanTouch = false
6499         p.CanQuery = false
6500         p.Parent = held
6501         return p
6502     end
6503     piece("Pot", Vector3.new(0.85, 0.62, 0.85), Vector3.new(0, 0, 0), Color3.fromRGB(118, 77, 46))
6504     piece("Stem", Vector3.new(0.13, 0.75 + stage * 0.13, 0.13), Vector3.new(0, 0.65, 0), Color3.fromRGB(65,
6505         ↪ 165, 72))
6506     piece("LeafL", Vector3.new(0.48, 0.12, 0.3), Vector3.new(-0.24, 0.78, 0), Color3.fromRGB(75, 185, 82))
6507     piece("LeafR", Vector3.new(0.48, 0.12, 0.3), Vector3.new(0.24, 1.0, 0), Color3.fromRGB(75, 185, 82))
6508     return held
6509 end
6510
6511 local function equipmentReadout(part, top, bottom, colour)
6512     local label = part:FindFirstChild("Readout", true)
6513     if label and label:IsA("TextLabel") then
6514         label.Text = top .. "\n" .. bottom
6515         label.TextColor3 = colour or Color3.fromRGB(115, 245, 150)
6516     end
6517 end
6518
6519 local function candidatesOnLevel(levelName, predicate)
6520     local result = {}
6521     for _, item in ipairs(CollectionService:GetTagged("LabEquipment")) do
6522         if item.Parent and item:GetAttribute("Level") == levelName and predicate(item) then
6523             table.insert(result, item)
6524         end
6525     end
6526     return result
6527 end
6528
6529 local function runBotScenario(bot)
6530     -- Remember where this worker started, so a shift reset can stand it back up.
6531     if not bot:GetAttribute("HomeCFrame") then
6532         pcall(function() bot:SetAttribute("HomeCFrame", bot:GetPivot()) end)
6533     end
6534     -- Desk staff stay at their desk. They are still perfectly good targets.
6535     if bot:GetAttribute("Seated") then
6536         setBotStatus(bot, "At the desk")
6537         while bot.Parent and not bot:GetAttribute("Dead") do task.wait(5) end
6538         return
6539     end

```



```

6540 while bot.Parent and not bot:GetAttribute("Dead") do
6541     -- The rival keeps running wherever it is: it is a single NPC and the
6542     -- competitive scoring loop depends on it making progress.
6543     local awake = bot:GetAttribute("Competitor") or botHasObserver(bot)
6544     if not awake then
6545         setBotStatus(bot, "Idle -- no one in the room")
6546         task.wait(1)
6547     elseif bot:GetAttribute("KnockedOut") then task.wait(1) else
6548         local levelName = bot:GetAttribute("Level")
6549         local failed = candidatesOnLevel(levelName, function(item)
6550             return item:GetAttribute("Kind") == PLANT_CHAMBER_KIND
6551                 and item:GetAttribute("FunctionalState") == "FAILED"
6552                 and (item:GetAttribute("SampleCount") or 0) > 0
6553                 and not item:GetAttribute("ReservedByBot")
6554         end)
6555         if #failed == 0 then
6556             -- Nothing has failed on this floor right now. The adaptive
6557             -- difficulty tick deliberately holds the failure rate down when
6558             -- few players are online, so this is the common case rather than
6559             -- the rare one -- and standing frozen in front of a player who
6560             -- just walked in is exactly what "the NPCs don't move" looked
6561             -- like. Patrol the room instead so the floor reads as staffed.
6562             -- Only ever runs for a bot somebody is actually watching.
6563             setBotStatus(bot, "Checking chambers")
6564             local patrolRoom = roomAt(levelName, bot:GetPivot().Position)
6565             local rect = patrolRoom and roomRect(levelName, patrolRoom)
6566             if rect then
6567                 local here = bot:GetPivot().Position
6568                 local spanX = math.max(1, (rect.x2 - rect.x1) - 12)
6569                 local spanZ = math.max(1, (rect.z2 - rect.z1) - 12)
6570                 walkBotSegment(bot, Vector3.new(
6571                     rect.x1 + 6 + math.random() * spanX,
6572                     here.Y,
6573                     rect.z1 + 6 + math.random() * spanZ))
6574                 task.wait(0.5 + math.random() * 1.5)
6575             else
6576                 task.wait(3)
6577             end
6578         else
6579             table.sort(failed, function(a, b)
6580                 local an = tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
6581                 local bn = tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
6582                 return an < bn
6583             end)
6584             -- Prefer a job in the room the worker is already standing in --
6585             -- which, because it only wakes when a player is there, is the
6586             -- room the player is watching. Without this a woken worker
6587             -- usually picks a chamber elsewhere on the floor, walks out of
6588             -- the room and stops the moment it loses its observer.
6589             local botRoom = roomAt(levelName, bot:GetPivot().Position)
6590             local sameRoom = {}
6591             if botRoom then
6592                 for _, item in ipairs(failed) do
6593                     if item:GetAttribute("Room") == botRoom then table.insert(sameRoom, item) end
6594                 end
6595             end
6596             local pool = (#sameRoom > 0) and sameRoom or failed
6597             local source = bot:GetAttribute("Competitor") and pool[1] or pool[math.random(1, #pool)]
6598             local destination = evacuationDestinations(source)[1]
6599             if not destination then
6600                 setBotStatus(bot, "Waiting for safe capacity")
6601                 task.wait(3)
6602             else
6603                 local botId = bot.Name
6604                 source:SetAttribute("ReservedByBot", botId)
6605                 destination:SetAttribute("IncomingReservations",
6606                     ↳ (destination:GetAttribute("IncomingReservations") or 0) + 1)
6607                 source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode") or "")
6608                 refreshChamberTelemetry(source)
6609                 setBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or "alarm"))
6610                 moveBotTo(bot, source)
6611                 local samples = chamberSampleList(source)
6612                 local plantId = table.remove(samples, 1)
6613                 if plantId and source:GetAttribute("ReservedByBot") == botId then
6614                     saveChamberSamples(source, samples)
6615                     updateStoredSamples(source)
6616                 end
6617             end
6618         end
6619     end
6620 end

```

```

6615         refreshChamberTelemetry(source)
6616         local stage = math.clamp(source:GetAttribute("GrowthStage") or 0, 0, 5)
6617         makeBotPlant(bot, stage)
6618         bot:SetAttribute("CarryingPlant", plantId)
6619         setBotStatus(bot, "Evacuating " .. plantId)
6620         equipmentReadout(source, (source:GetAttribute("GrowthCode") or "R?") .. " FAILED",
        ↪ "SEND " .. plantId .. " TO " .. (destination:GetAttribute("GrowthCode") or "R?"),
        ↪ Color3.fromRGB(255, 105, 90))

6621
6622         -- Emergency transfers still receive one quick development step,
6623         -- so the same growth mechanics are continuously regression-tested.
6624         local stations = candidatesOnLevel(levelName, function(item) return
        ↪ PLANT_PROCESS[item:GetAttribute("Kind")] ~= nil end)
6625         if #stations > 0 then
6626             local station = stations[((bot:GetAttribute("BotNumber") or 1) - 1) % #stations +
        ↪ 1]
6627             local process = PLANT_PROCESS[station:GetAttribute("Kind")]
6628             moveBotTo(bot, station)
6629             stage = math.clamp(stage + (process.gain or 0), 0, 5)
6630             station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or 0) + 1)
6631             equipmentReadout(station, "BOT GROWTH TEST", plantId .. " • stage " .. stage,
        ↪ Color3.fromRGB(120, 235, 165))
6632         end
6633
6634         setBotStatus(bot, "Delivering to " .. (destination:GetAttribute("GrowthCode") or "R?"))
6635         moveBotTo(bot, destination)
6636         local destinationSamples = chamberSampleList(destination)
6637         if destination:GetAttribute("FunctionalState") ~= "FAILED" and #destinationSamples <
        ↪ (destination:GetAttribute("Capacity") or 10) then
6638             table.insert(destinationSamples, plantId)
6639             saveChamberSamples(destination, destinationSamples)
6640             destination:SetAttribute("GrowthStage",
        ↪ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
6641             updateStoredSamples(destination)
6642             refreshChamberTelemetry(destination)
6643             emergencyMovesCompleted = emergencyMovesCompleted + 1
6644             bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or 0) + 1)
6645             if bot:GetAttribute("Competitor") then
6646                 local gained = POINTS.MOVE_ASSIGNED_ON_TIME
6647                 bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) + gained)
6648                 setBotStatus(bot, "+" .. gained .. " resource win")
6649                 updateScoreboard()
6650             else
6651                 setBotStatus(bot, "Transfer passed")
6652             end
6653         else
6654             -- Capacity changed while walking; return the sample to the source
6655             -- rather than deleting it and let the next cycle choose again.
6656             table.insert(samples, plantId)
6657             saveChamberSamples(source, samples)
6658             updateStoredSamples(source)
6659             refreshChamberTelemetry(source)
6660             setBotStatus(bot, "Destination changed; retrying")
6661         end
6662         local held = bot:FindFirstChild("BotHeldPlant")
6663         if held then held:Destroy() end
6664         bot:SetAttribute("CarryingPlant", "")
6665     end
6666     destination:SetAttribute("IncomingReservations", math.max(0,
        ↪ (destination:GetAttribute("IncomingReservations") or 1) - 1))
6667     if source:GetAttribute("ReservedByBot") == botId then source:SetAttribute("ReservedByBot",
        ↪ nil) end
6668     updateBotTestBoard()
6669     task.wait(1.5 + math.random())
6670 end
6671 end
6672 end
6673 end
6674 end
6675
6676 -----
6677 -- ADAPTIVE, INDEFINITE CHAMBER FAILURE SIMULATION
6678 -----
6679 local FAILURE_REASONS = {
6680     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
6681     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",

```

```

6682 }
6683 local generatedSampleSerial = 20000
6684 local previousCompletedMoves = 0
6685
6686 setChamberFailure = function(chamber, failed)
6687     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
6688     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
6689     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
6690     if failed then
6691         chamber:SetAttribute("FunctionalState", "FAILED")
6692         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
6693         chamber:SetAttribute("RepairKey", "R")
6694         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " • HOLD R
        ↪ TO REPAIR")
6695         chamber:SetAttribute("FailureReason", FAILURE_REASONS[math.random(1, #FAILURE_REASONS)])
6696         chamber:SetAttribute("FailureSince", os.time())
6697         chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
6698         chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
6699         chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
6700         local destination = evacuationDestinations(chamber)[1]
6701         chamber:SetAttribute("EvacuationDestination", destination and destination:GetAttribute("GrowthCode") or
        ↪ "")
6702         for _, player in ipairs(Players:GetPlayers()) do
6703             if playerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
6704                 ↪ assignNextTask(player) end
6705             if playerRole(player) == "Laboratory Technician" and not
        ↪ player:GetAttribute("AssignedTransferSource") then assignNextTask(player) end
6706             if playerRole(player) == "Laboratory Technician" then
6707                 notify(player, "PLANT ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Press E to
        ↪ remove plants and follow the assigned destination.")
6708             else
6709                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Technician:
        ↪ HOLD R to repair.")
6710             end
6711         end
6712     else
6713         chamber:SetAttribute("FunctionalState", "WORKING")
6714         chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
6715         chamber:SetAttribute("FailureReason", "")
6716         chamber:SetAttribute("FailureSince", 0)
6717         chamber:SetAttribute("EvacuationDestination", "")
6718         chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
6719         chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
6720         chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
6721         chamber:SetAttribute("OperationStarted", os.time())
6722         if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) < 100
        ↪ then
6723             recoverPlantLife(chamber, chamber:GetAttribute("PlantLife"), nil, chamber:GetAttribute("PlantId"))
6724         end
6725     end
6726     refreshChamberTelemetry(chamber)
6727 end
6728
6729 local GENERIC_FAILURE_REASONS = {
6730     "Power relay fault", "Controllor communication loss", "Pump interlock fault",
6731     "Calibration expired", "Safety sensor trip", "Motor overload",
6732 }
6733
6734 setGenericFailure = function(part, failed)
6735     local floorLabel = equipmentFloorLabel(part)
6736     local unitLabel = equipmentLabel(part)
6737     if failed then
6738         part:SetAttribute("FunctionalState", "FAILED")
6739         part:SetAttribute("RepairKey", "R")
6740         part:SetAttribute("FailureReason", GENERIC_FAILURE_REASONS[math.random(1, #GENERIC_FAILURE_REASONS)])
6741         part:SetAttribute("FailureSince", os.time())
6742         part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • HOLD R")
6743         equipmentReadout(part, "FAILED • HOLD R", floorLabel .. " • " .. unitLabel .. " • PLANTS "
        ↪ .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • "
        ↪ .. part:GetAttribute("FailureReason"),
6744             Color3.fromRGB(255, 105, 90))
6745         for _, player in ipairs(Players:GetPlayers()) do
6746             if playerRole(player) == "Technician" then
6747                 if not player:GetAttribute("AssignedRepairId") then assignNextTask(player) end
6748                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • " .. unitLabel .. " FAILED. HOLD R to
        ↪ repair.")

```

```

6749         end
6750     end
6751     else
6752         part:SetAttribute("FunctionalState", "WORKING")
6753         part:SetAttribute("FailureReason", "")
6754         part:SetAttribute("FailureSince", 0)
6755         part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
6756         part:SetAttribute("OperationStarted", os.time())
6757         equipmentReadout(part, "WORKING", floorLabel .. " • " .. unitLabel .. " • PLANTS "
6758             .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • E
        ↪ USE / INSPECT",
6759             Color3.fromRGB(110, 235, 170))
6760     end
6761 end
6762
6763 local function shuffled(list)
6764     for i = #list, 2, -1 do
6765         local j = math.random(1, i)
6766         list[i], list[j] = list[j], list[i]
6767     end
6768     return list
6769 end
6770
6771 local function adaptiveFailureTick()
6772     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
6773     local failed, working = {}, {}
6774     local failedSamples, totalSamples, workingFree = 0, 0, 0
6775     for _, chamber in ipairs(chambers) do
6776         local count = chamber:GetAttribute("SampleCount") or 0
6777         local capacity = chamber:GetAttribute("Capacity") or 10
6778         totalSamples = totalSamples + count
6779         if chamber:GetAttribute("FunctionalState") == "FAILED" then
6780             table.insert(failed, chamber)
6781             failedSamples = failedSamples + count
6782         else
6783             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then
6784                 table.insert(working, chamber)
6785             end
6786             workingFree = workingFree + math.max(0, capacity - count -
        ↪ (chamber:GetAttribute("IncomingReservations") or 0))
6787         end
6788     end
6789
6790     local throughput = emergencyMovesCompleted - previousCompletedMoves
6791     previousCompletedMoves = emergencyMovesCompleted
6792     local backlogRatio = failedSamples / math.max(1, totalSamples)
6793     -- Sixty percent is the operating centre. Fast rescues allow a few more
6794     -- alarms; a large backlog or exhausted safe capacity automatically eases
6795     -- the load so 50 NPCs plus real players can keep the loop moving forever.
6796     local playerCount = math.max(1, #Players:GetPlayers())
6797     local speedAdjustment = math.clamp((throughput - 5) * 0.003, -0.03, 0.08)
6798     local playerAdjustment = math.clamp((playerCount - 1) * 0.012, 0, 0.10)
6799     local backlogAdjustment = -math.clamp((backlogRatio - 0.60) * 0.08, -0.04, 0.04)
6800     local capacityAdjustment = workingFree < 20 and -0.12 or 0
6801     local targetRate = math.clamp(0.60 + speedAdjustment + playerAdjustment + backlogAdjustment +
        ↪ capacityAdjustment, 0.42, 0.76)
6802     local targetFailed = math.floor(#chambers * targetRate + 0.5)
6803     local maxChange = math.clamp(2 + playerCount * 2, 4, 12)
6804     local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
6805     if change > 0 then
6806         local eligible = {}
6807         for _, chamber in ipairs(working) do
6808             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then table.insert(eligible,
        ↪ chamber) end
6809         end
6810         shuffled(eligible)
6811         for i = 1, math.min(change, #eligible) do setChamberFailure(eligible[i], true) end
6812     elseif change < 0 then
6813         table.sort(failed, function(a, b)
6814             local ac, bc = a:GetAttribute("SampleCount") or 0, b:GetAttribute("SampleCount") or 0
6815             if ac == bc then return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or
        ↪ 0) end
6816             return ac < bc
6817         end)
6818         for i = 1, math.min(-change, #failed) do setChamberFailure(failed[i], false) end
6819     end

```

```

6820
6821 -- Failed telemetry continues to drift; working chambers continuously
6822 -- germinate replacement samples so the transfer game never runs out.
6823 for _, chamber in ipairs(chambers) do
6824     if chamber:GetAttribute("FunctionalState") == "FAILED" then
6825         chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
6826             ↪ math.random(-2, 2), 28, 42))
6827         chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
6828             ↪ math.random(-4, 4), 25, 65))
6829         chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
6830             ↪ math.random(-5, 3), 0, 48))
6831         local destination = evacuationDestinations(chamber)[1]
6832         chamber:SetAttribute("EvacuationDestination", destination and
6833             ↪ destination:GetAttribute("GrowthCode") or "")
6834     else
6835         local samples = chamberSampleList(chamber)
6836         local capacity = chamber:GetAttribute("Capacity") or 10
6837         local refillChance = math.clamp(0.08 + (capacity - #samples) * 0.025 + throughput * 0.002
6838             ↪ + playerCount * 0.008, 0.08, 0.46)
6839         if #samples < capacity and math.random() < refillChance then
6840             generatedSampleSerial = generatedSampleSerial + 1
6841             table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
6842                 ↪ generatedSampleSerial))
6843             saveChamberSamples(chamber, samples)
6844             chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
6845             updateStoredSamples(chamber)
6846         end
6847     end
6848     refreshChamberTelemetry(chamber)
6849 end
6850 local complex = workspace:FindFirstChild("Laboratory_Complex")
6851 if complex then
6852     complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
6853     complex:SetAttribute("FailingChambers", #failed)
6854     complex:SetAttribute("FailedSampleBacklog", failedSamples)
6855     complex:SetAttribute("EmergencyMovesCompleted", emergencyMovesCompleted)
6856     complex:SetAttribute("RecentTransferThroughput", throughput)
6857     complex:SetAttribute("ActivePlayerCount", playerCount)
6858     complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples /
6859         ↪ 20), 8, 50))
6860     -- Per-FL00R budget. Multiplied across the basement plus every level
6861     -- this is what actually decides how many workers are visibly busy.
6862     complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
6863         ↪ math.floor(failedSamples / 60), 6, 20))
6864 end
6865 end
6866
6867 local function genericFailureTick()
6868     local failed, working = {}, {}
6869     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
6870         if equipment:GetAttribute("Kind") ~= PLANT_CHAMBER_KIND then
6871             if equipment:GetAttribute("FunctionalState") == "FAILED" then
6872                 table.insert(failed, equipment)
6873             elseif os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 45 then
6874                 table.insert(working, equipment)
6875             end
6876         end
6877     end
6878     end
6879     local playerCount = math.max(1, #Players:GetPlayers())
6880     local targetRate = math.clamp(0.12 + playerCount * 0.018, 0.14, 0.28)
6881     local targetFailed = math.max(2, math.floor((#failed + #working) * targetRate + 0.5))
6882     local maxChange = math.clamp(1 + math.ceil(playerCount / 2), 2, 6)
6883     local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
6884     if change > 0 then
6885         shuffled(working)
6886         for i = 1, math.min(change, #working) do setGenericFailure(working[i], true) end
6887     elseif change < 0 then
6888         table.sort(failed, function(a, b) return (a:GetAttribute("FailureSince") or 0) <
6889             ↪ (b:GetAttribute("FailureSince") or 0) end)
6890         for i = 1, math.min(-change, #failed) do setGenericFailure(failed[i], false) end
6891     end
6892 end
6893 end
6894
6895 local function bindPlayerTaskRouting(player)
6896     if player:GetAttribute("EquipmentTaskRoutingBound") then return end
6897     player:SetAttribute("EquipmentTaskRoutingBound", true)

```

```

6888     player:GetAttributeChangedSignal("LabRole"):Connect(function()
6889         task.delay(0.2, function()
6890             if player.Parent then assignNextTask(player) end
6891         end)
6892     end)
6893     player.CharacterAdded:Connect(function()
6894         if playerRole(player) == "Technician" and player:GetAttribute("AssignedRepairId") then
6895             task.delay(1, function() if player.Parent then assignNextTask(player) end end)
6896         end
6897     end)
6898 end
6899
6900 for _, player in ipairs(Players:GetPlayers()) do bindPlayerTaskRouting(player) end
6901 Players.PlayerAdded:Connect(bindPlayerTaskRouting)
6902 Players.PlayerRemoving:Connect(clearRepairPath)
6903
6904 for _, p in ipairs(CollectionService:GetTagged("LabEquipment")) do bind(p) end
6905 CollectionService:GetInstanceAddedSignal("LabEquipment"):Connect(bind)
6906
6907 -----
6908 -- AUTOMATIC DOORS
6909 -- Every AutoDoor panel slides up into the header void when a player's
6910 -- HumanoidRootPart comes within range, and back down once nobody is close,
6911 -- so rooms have real doors instead of a permanently open gap.
6912 -----
6913 local DOOR_OPEN_RANGE = 11
6914 local DOOR_CLOSE_RANGE = 15
6915 local DOOR_MIN_OPEN_SECONDS = 3
6916 local function bindAutoDoor(door)
6917     if door:GetAttribute("DoorBound") then return end
6918     door:SetAttribute("DoorBound", true)
6919     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
6920     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
6921     local openX = door:GetAttribute("DoorOpenX") or closedX
6922     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
6923     local doorY = door:GetAttribute("DoorY") or door.Position.Y
6924     local isOpen = false
6925     local openedAt = 0
6926     local tween
6927     local function setIndicatorColour(open)
6928         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
6929         for _, gui in ipairs(door:GetChildren()) do
6930             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then
6931                 local indicator = gui:FindFirstChild("DoorIndicator")
6932                 if indicator then indicator.BackgroundColor3 = colour end
6933             end
6934         end
6935     end
6936     local function setOpen(open)
6937         if open == isOpen then return end
6938         isOpen = open
6939         if open then openedAt = os.clock() end
6940         if tween then tween:Cancel() end
6941         local targetX = open and openX or closedX
6942         local targetZ = open and openZ or closedZ
6943         -- 0.9s ease so the slide is smooth and readable (never an instant
6944         -- snap, which is what made the old vertical slide read as flicker).
6945         tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
6946             Enum.EasingDirection.InOut),
6947             {CFrame = CFrame.new(targetX, doorY, targetZ)})
6948         tween:Play()
6949         door.CanCollide = not open
6950         door:SetAttribute("DoorOpen", open) -- watched by the room smoke system
6951         setIndicatorColour(open)
6952     end
6953     task.spawn(function()
6954         while door.Parent do
6955             local nearest = math.huge
6956             for _, player in ipairs(Players:GetPlayers()) do
6957                 local character = player.Character
6958                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
6959                 if hrp then
6960                     -- Floors stack directly above each other at the same X/Z, so a
6961                     -- flat (X/Z-only) distance check would treat a player on a
6962                     -- completely different floor as "standing at the door." Only
6963                     -- count players actually close in height to this door first.

```

```

6963         local verticalGap = math.abs(hrp.Position.Y - doorY)
6964         if verticalGap < 8 then
6965             local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) - Vector3.new(closedX,
6966                 ↪ doorY, closedZ)).Magnitude
6967             if flat < nearest then nearest = flat end
6968         end
6969     end
6970     -- Once opened, a door holds for at least DOOR_MIN_OPEN_SECONDS
6971     -- before it's allowed to close again -- this is what actually
6972     -- fixes the "opens and closes too fast" flicker, independent of
6973     -- the slide direction/speed above.
6974     if isOpen and nearest > DOOR_CLOSE_RANGE and (os.clock() - openedAt) >= DOOR_MIN_OPEN_SECONDS then
6975         setOpen(false)
6976     elseif not isOpen and nearest < DOOR_OPEN_RANGE then
6977         setOpen(true)
6978     end
6979     task.wait(0.25)
6980 end
6981 end)
6982 end
6983
6984 for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindAutoDoor(p) end
6985 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(bindAutoDoor)
6986
6987 -----
6988 -- EMERGENCY KITS
6989 -- One per room. Press E to fully restore health. When a player's health
6990 -- drops below half (and they're not mid-job), a red floor-marker route --
6991 -- the same system used for repair/transfer jobs -- guides them to the
6992 -- closest kit.
6993 -----
6994 local function bindEmergencyKit(kit)
6995     if kit:GetAttribute("KitBound") then return end
6996     kit:SetAttribute("KitBound", true)
6997     local prompt = kit:FindFirstChild("EmergencyKitPrompt")
6998     if not prompt then return end
6999     -- Health stations RECHARGE rather than being consumed. A single-use kit
7000     -- that destroyed itself meant a floor could be permanently stripped of
7001     -- health with no way to get it back.
7002     local RECHARGE_SECONDS = 25
7003     local caption
7004     local faceGui = kit:FindFirstChild("EmergencyKitFace")
7005     if faceGui then
7006         for _, d in ipairs(faceGui:GetDescendants()) do
7007             if d:IsA("TextLabel") then caption = d break end
7008         end
7009     end
7010
7011     local function setCharged(charged)
7012         kit:SetAttribute("Charged", charged)
7013         prompt.Enabled = charged
7014         kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
7015         if caption then caption.Text = charged and "HEALTH" or "CHARGING" end
7016     end
7017
7018     prompt.Triggered:Connect(function(player)
7019         if kit:GetAttribute("Charged") == false then return end
7020         local character = player.Character
7021         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7022         if not humanoid then return end
7023         setCharged(false)
7024         humanoid.Health = humanoid.MaxHealth
7025         notify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s -- there are
7026             ↪ more along every corridor.")
7027         task.delay(RECHARGE_SECONDS, function()
7028             if kit.Parent then setCharged(true) end
7029         end)
7030     end)
7031     setCharged(true)
7032 end
7033 for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindEmergencyKit(p) end
7034 CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindEmergencyKit)
7035
7036 -----
7037 -- PLAYER HEALTH BARS

```

```

7037 -- Matches the bar the worker bots carry, so a player can read their own
7038 -- and everyone else's condition without opening any menu.
7039 -----
7040 local function attachPlayerHealthBar(character)
7041     local humanoid = character:FindFirstChildOfClass("Humanoid") or character:WaitForChild("Humanoid", 8)
7042     local head = character:FindFirstChild("Head") or character:WaitForChild("Head", 8)
7043     if not humanoid or not head or head:FindFirstChild("LabHealthBar") then return end
7044
7045     local gui = Instance.new("BillboardGui")
7046     gui.Name = "LabHealthBar"
7047     gui.Size = UDim2.fromOffset(168, 44)
7048     gui.StudsOffset = Vector3.new(0, 2.6, 0)
7049     gui.MaxDistance = 90
7050     gui.AlwaysOnTop = false
7051     gui.Parent = head
7052
7053     local back = Instance.new("Frame")
7054     back.Name = "HealthBarBack"
7055     back.Size = UDim2.new(0.9, 0, 0.34, 0)
7056     back.Position = UDim2.new(0.05, 0, 0.06, 0)
7057     back.BackgroundColor3 = Color3.fromRGB(22, 26, 32)
7058     back.BackgroundTransparency = 0.2
7059     back.BorderSizePixel = 0
7060     back.Parent = gui
7061     local fill = Instance.new("Frame")
7062     fill.Name = "HealthBarFill"
7063     fill.Size = UDim2.fromScale(1, 1)
7064     fill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
7065     fill.BorderSizePixel = 0
7066     fill.Parent = back
7067     local label = Instance.new("TextLabel")
7068     label.Name = "HealthText"
7069     label.Size = UDim2.new(1, 0, 0.5, 0)
7070     label.Position = UDim2.new(0, 0, 0.48, 0)
7071     label.BackgroundTransparency = 1
7072     label.TextColor3 = Color3.fromRGB(224, 238, 248)
7073     label.TextStrokeTransparency = 0.4
7074     label.TextScaled = true
7075     label.Font = Enum.Font.GothamBold
7076     label.Parent = gui
7077
7078     local function refresh()
7079         local maximum = math.max(1, humanoid.MaxHealth)
7080         local ratio = math.clamp(humanoid.Health / maximum, 0, 1)
7081         fill.Size = UDim2.fromScale(ratio, 1)
7082         fill.BackgroundColor3 = ratio > 0.55 and Color3.fromRGB(75, 220, 95)
7083             or (ratio > 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
7084         label.Text = string.format("%d%%", math.floor(ratio * 100 + 0.5))
7085     end
7086     humanoid.HealthChanged:Connect(refresh)
7087     humanoid:GetPropertyChangedSignal("MaxHealth"):Connect(refresh)
7088     refresh()
7089 end
7090
7091 local function watchPlayerHealth(player)
7092     player.CharacterAdded:Connect(attachPlayerHealthBar)
7093     if player.Character then task.spawn(attachPlayerHealthBar, player.Character) end
7094 end
7095 Players.PlayerAdded:Connect(watchPlayerHealth)
7096 for _, player in ipairs(Players:GetPlayers()) do watchPlayerHealth(player) end
7097
7098 -----
7099 -- SECRET BASEMENT VENT
7100 -- Disguised as part of the lobby's own exterior wall. Holding E on it
7101 -- drops the player straight into the basement corridor.
7102 -----
7103 local function bindSecretBasementPanel(panel)
7104     if panel:GetAttribute("SecretBound") then return end
7105     panel:SetAttribute("SecretBound", true)
7106     local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")
7107     if not prompt then return end
7108     prompt.Triggered:Connect(function(player)
7109         local character = player.Character
7110         local hrp = character and character:FindFirstChild("HumanoidRootPart")
7111         if not hrp then return end
7112         local dest = Vector3.new(

```



```

7113         panel:GetAttribute("SecretDestX") or hrp.Position.X,
7114         panel:GetAttribute("SecretDestY") or hrp.Position.Y,
7115         panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
7116         hrp.CFrame = CFrame.new(dest)
7117         notify(player, "The vent panel swings open -- you drop into the basement.")
7118     end)
7119 end
7120 for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindSecretBasementPanel(p) end
7121 CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindSecretBasementPanel)
7122
7123 local function nearestEmergencyKit(position)
7124     local best, bestDist
7125     for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
7126         if kit.Parent then
7127             local d = (kit.Position - position).Magnitude
7128             if not bestDist or d < bestDist then best, bestDist = kit, d end
7129         end
7130     end
7131     return best
7132 end
7133
7134 local healthPathActive = {}
7135 task.spawn(function()
7136     while true do
7137         task.wait(3)
7138         for _, player in ipairs(Players:GetPlayers()) do
7139             local character = player.Character
7140             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7141             local hrp = character and character:FindFirstChild("HumanoidRootPart")
7142             if humanoid and hrp and humanoid.Health > 0 then
7143                 local low = humanoid.Health < humanoid.MaxHealth * 0.5
7144                 local busy = player:GetAttribute("AssignedRepairId") or
7145                     ⇨ player:GetAttribute("AssignedTransferSource")
7146                 if low and not busy and not healthPathActive[player] then
7147                     local kit = nearestEmergencyKit(hrp.Position)
7148                     if kit then
7149                         createRepairPath(player, kit)
7150                         healthPathActive[player] = true
7151                         notify(player, "Health low: follow the RED floor lights to the nearest emergency kit.")
7152                     end
7153                 elseif (not low or busy) and healthPathActive[player] then
7154                     healthPathActive[player] = false
7155                     if not busy then clearRepairPath(player) end
7156                 end
7157             end
7158         end
7159     end)
7160
7161     -----
7162     -- EQUIPMENT HAZARDS
7163     -- A malfunctioning (FAILED) unit is genuinely dangerous to linger next to:
7164     -- electrical faults, steam/humidity blowouts, chemical spills and coolant
7165     -- (freon) leaks all drain health while a player stays in range.
7166     -----
7167     local HAZARD_BY_KIND = {
7168         ["Germination Chamber"] = {"STEAM LEAK", 3},
7169         ["Incubator"] = {"STEAM LEAK", 3},
7170         ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
7171         ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
7172         ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
7173         ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
7174         ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
7175         ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
7176         ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},
7177         ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
7178         ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
7179     }
7180     local HAZARD_RANGE = 9
7181     local hazardWarnedAt = {}
7182
7183     -- A unit that's been left FAILED too long can catastrophically blow up,
7184     -- catch fire, or arc -- and unlike an ordinary failure, this one can never
7185     -- be repaired. It keeps draining nearby health (worse than before) and its
7186     -- StatusLED/timer read DESTROYED permanently.
7187     --

```

```

7188 -- Two-stage sequence: warnEquipmentCatastrophe() runs a 2-second siren +
7189 -- strobe so anyone nearby has time to clear out, THEN destroyEquipment()
7190 -- fires the actual explosion/fire/arc/leak effects.
7191 local DESTRUCTION_KIND = {
7192     ["Germination Chamber"] = "steam",
7193     ["Incubator"] = "steam",
7194     ["Decontamination Unit"] = "gas",
7195     ["Mutagenesis Chamber"] = "electrical",
7196     ["Hormone Treatment Bench"] = "liquid",
7197     ["Plant Growth Chamber"] = "gas",
7198     ["Growth Hormone Mixer"] = "liquid",
7199     ["GMO Injector"] = "electrical",
7200     ["Nutrient Infuser"] = "gas",
7201     ["UV Growth Scanner"] = "electrical",
7202     ["Pollination Station"] = "electrical",
7203 }
7204
7205 local function scatterDebris(part)
7206     for i = 1, math.random(4, 7) do
7207         local chunk = Instance.new("Part")
7208         chunk.Name = "Debris"
7209         chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
7210         chunk.Color = part.Color
7211         chunk.Material = Enum.Material.Metal
7212         chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2, math.random(-3,
7213             ↳ 3) / 2)
7214         chunk.Anchored = false
7215         chunk.CanCollide = true
7216         chunk.Parent = part.Parent
7217         local av = Instance.new("AngularVelocity")
7218         av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
7219         av.Parent = chunk
7220         local bv = Instance.new("BodyVelocity")
7221         bv.MaxForce = Vector3.new(4000, 4000, 4000)
7222         bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
7223         bv.Parent = chunk
7224         Debris:AddItem(bv, 0.25)
7225         Debris:AddItem(chunk, 8)
7226     end
7227 end
7228
7229 local destroyEquipment
7230
7231 local function warnEquipmentCatastrophe(part, floorLabel, unitLabel)
7232     if part:GetAttribute("AboutToBlow") then return end
7233     part:SetAttribute("AboutToBlow", true)
7234     local beacon = Instance.new("PointLight")
7235     beacon.Color = Color3.fromRGB(255, 40, 30)
7236     beacon.Brightness = 4
7237     beacon.Range = 16
7238     beacon.Parent = part
7239     local siren = Instance.new("Sound")
7240     siren.Name = "SirenWarning"
7241     siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
7242     siren.Looped = true
7243     siren.Volume = 3
7244     siren.PlaybackSpeed = 1.6
7245     siren.Parent = part
7246     siren:Play()
7247     for _, player in ipairs(Players:GetPlayers()) do
7248         notify(player, "ALARM: " .. floorLabel .. " • " .. unitLabel .. " is about to fail catastrophically --
7249             ↳ MOVE AWAY NOW!")
7250     end
7251     task.spawn(function()
7252         local elapsed = 0
7253         while elapsed < 2 and part.Parent do
7254             beacon.Enabled = not beacon.Enabled
7255             task.wait(0.2)
7256             elapsed = elapsed + 0.2
7257         end
7258     end)
7259     task.delay(2, function()
7260         siren:Stop()
7261         siren:Destroy()
7262         beacon:Destroy()
7263         if part.Parent then destroyEquipment(part) end
7264     end)

```

```

7262     end)
7263 end
7264
7265 destroyEquipment = function(part)
7266     local floorLabel = equipmentFloorLabel(part)
7267     local unitLabel = equipmentLabel(part)
7268     local causes = {"exploded", "caught fire", "arced and shorted out"}
7269     local cause = causes[math.random(1, #causes)]
7270     part:SetAttribute("FunctionalState", "DESTROYED")
7271     part:SetAttribute("AboutToBlow", nil)
7272     part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • DESTROYED, UNFIXABLE")
7273     equipmentReadout(part, "DESTROYED", floorLabel .. " • " .. unitLabel .. " • " .. cause .. " -- permanently
    ↪ offline",
7274         Color3.fromRGB(150, 40, 35))
7275     for _, ledPart in ipairs(part:GetDescendants()) do
7276         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
    ↪ Enum.Material.SmoothPlastic end
7277     end
7278     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
7279
7280     local bang = Instance.new("Sound")
7281     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
7282     bang.Volume = 4
7283     bang.PlaybackSpeed = 0.8
7284     bang.Parent = part
7285     bang:Play()
7286     Debris:AddItem(bang, 4)
7287
7288     local explosion = Instance.new("Explosion")
7289     explosion.Position = part.Position
7290     explosion.BlastRadius = 8
7291     explosion.BlastPressure = 0
7292     explosion.ExplosionType = Enum.ExplosionType.NoCraters
7293     explosion.DestroyJointRadiusPercent = 0
7294     explosion.Parent = workspace
7295
7296     local kindOfDestruction = DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
7297
7298     local fire = Instance.new("Fire")
7299     fire.Size = 6
7300     fire.Heat = 9
7301     fire.Color = Color3.fromRGB(255, 140, 40)
7302     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
7303     fire.Parent = part
7304     Debris:AddItem(fire, 16)
7305
7306     local sparkles = Instance.new("Sparkles")
7307     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
7308     sparkles.Parent = part
7309     Debris:AddItem(sparkles, 3)
7310
7311     local smoke = Instance.new("Smoke")
7312     smoke.Size = 8
7313     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
7314     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
7315     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
7316         or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
7317         or Color3.fromRGB(50, 50, 55)
7318     smoke.Parent = part
7319     Debris:AddItem(smoke, 20)
7320
7321     if kindOfDestruction == "liquid" then
7322         local puddle = Instance.new("Part")
7323         puddle.Name = "LeakPuddle"
7324         puddle.Shape = Enum.PartType.Cylinder
7325         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
7326         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05, part.Position.Z)
7327             * CFrame.Angles(0, 0, math.rad(90))
7328         puddle.Color = Color3.fromRGB(120, 90, 40)
7329         puddle.Material = Enum.Material.Neon
7330         puddle.Transparency = 0.35
7331         puddle.Anchored = true
7332         puddle.CanCollide = false
7333         puddle.Parent = part.Parent
7334         Debris:AddItem(puddle, 25)
7335     end

```

```

7336
7337     scatterDebris(part)
7338
7339     for _, player in ipairs(Players:GetPlayers()) do
7340         notify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause .. ". It
        ↳ cannot be repaired.")
7341     end
7342 end
7343
7344 task.spawn(function()
7345     while true do
7346         task.wait(1)
7347         local hazards = {}
7348         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
7349             local state = equip.Parent and equip:GetAttribute("FunctionalState")
7350             if state == "FAILED" then
7351                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
7352                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
7353                 -- The longer a unit stays failed, the likelier it blows up for
7354                 -- good. Roughly a 1-in-6 chance per second once it's been down
7355                 -- 45+ seconds -- gives Technicians a real reason to hurry. The
7356                 -- 2-second siren warning (above) still gives an out.
7357                 local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
7358                 if failedFor > 45 and math.random(1, 6) == 1 then
7359                     warnEquipmentCatastrophe(equip, equipmentFloorLabel(equip), equipmentLabel(equip))
7360                 end
7361             elseif state == "DESTROYED" then
7362                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
7363                 if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
        ↳ info[2] + 2}) end
7364             end
7365         end
7366         if #hazards > 0 then
7367             for _, player in ipairs(Players:GetPlayers()) do
7368                 local character = player.Character
7369                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7370                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
7371                 if humanoid and hrp and humanoid.Health > 0 then
7372                     local worst
7373                     for _, hazard in ipairs(hazards) do
7374                         local d = (hazard.part.Position - hrp.Position).Magnitude
7375                         if d < HAZARD_RANGE and (not worst or hazard.dps > worst.dps) then worst = hazard end
7376                     end
7377                     if worst then
7378                         humanoid:TakeDamage(worst.dps)
7379                         local last = hazardWarnedAt[player] or 0
7380                         if os.clock() - last > 2 then
7381                             hazardWarnedAt[player] = os.clock()
7382                             notify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
        ↳ worst.label, math.floor(humanoid.Health)))
7383                         end
7384                     end
7385                 end
7386             end
7387         end
7388     end
7389 end)
7390
7391 for _, p in ipairs(CollectionService:GetTagged("LabToolDispenser")) do bindDispenser(p) end
7392 CollectionService:GetInstanceAddedSignal("LabToolDispenser"):Connect(bindDispenser)
7393
7394 local testBots = CollectionService:GetTagged("LabWorkerBot")
7395 table.sort(testBots, function(a, b) return (a:GetAttribute("BotNumber") or 0) < (b:GetAttribute("BotNumber") or
        ↳ 0) end)
7396 for index, bot in ipairs(testBots) do
7397     -- Stagger WITHIN each floor, not across the whole building. Staggering on
7398     -- the global index at 0.32s a bot meant the top floor did not begin its
7399     -- loop until nearly a minute after the server started, so the upper
7400     -- levels looked completely frozen to anyone who walked up there early.
7401     -- Keyed off the per-floor slot, every floor is fully awake in ~7s.
7402     -- Only bots someone is standing next to will actually compute a path now,
7403     -- so there is no thundering herd to spread out and the stagger can be
7404     -- short. Every floor is ready ~1.6s in instead of ~58s.
7405     local slot = bot:GetAttribute("LevelSlot") or index
7406     task.delay(slot * 0.08, function()
7407         if bot.Parent then runBotScenario(bot) end
7408     end)
7409 end

```

```

7408     end)
7409 end
7410
7411 -- Bodies stay put for the whole shift. LabRound stamps RoundNumber on
7412 -- Workspace when the shift rolls over; that is the cue to clear them away and
7413 -- put every worker back on its feet for a fresh hunt.
7414 local function resetHunt()
7415     local bodies = workspace:FindFirstChild("LabBotBodies")
7416     if bodies then bodies:ClearAllChildren() end
7417
7418     local complex = workspace:FindFirstChild("Laboratory_Complex")
7419     if complex then complex:SetAttribute("BotsKilled", 0) end
7420
7421     local restaged = 0
7422     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
7423         if bot:GetAttribute("Dead") then
7424             local home = bot:GetAttribute("HomeCFrame")
7425             if typeof(home) == "CFrame" then bot:PivotTo(home) end
7426             bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
7427             bot:SetAttribute("KnockedOut", false)
7428             bot:SetAttribute("Dead", false)
7429             bot:SetAttribute("TaskState", "Waiting for plant task")
7430             refreshBotHealth(bot)
7431             -- its old scenario loop exited on death, so start a fresh one
7432             task.spawn(runBotScenario, bot)
7433             restaged = restaged + 1
7434         end
7435     end
7436     if restaged > 0 then
7437         print("Shift reset: cleared bodies and restaged " .. restaged .. " workers.")
7438     end
7439 end
7440 workspace:GetAttributeChangedSignal("RoundNumber"):Connect(resetHunt)
7441
7442 -----
7443 -- ROOM SMOKE
7444 -- Small rooms slowly fill with smoke. Opening a door vents it. Capped so
7445 -- there are never more than a handful of smoke volumes alive at once.
7446 -----
7447 local MAX_SMOKY_ROOMS = 4
7448 local smokeByRoom = {}
7449
7450 local function clearRoomSmoke(key)
7451     local folder = smokeByRoom[key]
7452     if not folder then return end
7453     smokeByRoom[key] = nil
7454     for _, d in ipairs(folder:GetDescendants()) do
7455         if d:IsA("Smoke") then d.Enabled = false end
7456     end
7457     Debris:AddItem(folder, 5) -- let what is already in the air disperse
7458 end
7459
7460 local function fillRoomWithSmoke(levelName, rect)
7461     local key = levelName .. "/" .. rect.name
7462     if smokeByRoom[key] then return end
7463     local folder = Instance.new("Folder")
7464     folder.Name = "RoomSmoke"
7465     folder:SetAttribute("RoomKey", key)
7466     folder.Parent = workspace
7467     for i = 1, 5 do
7468         local emitter = Instance.new("Part")
7469         emitter.Name = "SmokeSource"
7470         emitter.Size = Vector3.new(1, 1, 1)
7471         emitter.Anchored = true
7472         emitter.CanCollide = false
7473         emitter.CanQuery = false
7474         emitter.CanTouch = false
7475         emitter.Transparency = 1
7476         emitter.CFrame = CFrame.new(
7477             rect.x1 + (rect.x2 - rect.x1) * (i - 0.5) / 5,
7478             rect.y + 3,
7479             rect.z1 + (rect.z2 - rect.z1) * ((i * 2) % 5 + 0.5) / 5)
7480         emitter.Parent = folder
7481         local smoke = Instance.new("Smoke")
7482         smoke.Size = 11
7483         smoke.Opacity = 0.32

```

```

7484     smoke.RiseVelocity = 2
7485     smoke.Color = Color3.fromRGB(150, 150, 152)
7486     smoke.Parent = emitter
7487     end
7488     smokeByRoom[key] = folder
7489 end
7490
7491 local function ventFromDoor(door)
7492     local pos = door.Position
7493     for key in pairs(smokeByRoom) do
7494         local levelName, roomName = key:match("^(.-)/(.)$")
7495         local rect = roomRect(levelName, roomName)
7496         -- a door on the room's own boundary counts, hence the small margin
7497         if rect and math.abs(pos.Y - rect.y) < FLOOR_BAND
7498             and pos.X >= rect.x1 - 8 and pos.X <= rect.x2 + 8
7499             and pos.Z >= rect.z1 - 8 and pos.Z <= rect.z2 + 8 then
7500             clearRoomSmoke(key)
7501         end
7502     end
7503 end
7504
7505 local function watchDoor(door)
7506     door:GetAttributeChangedSignal("DoorOpen"):Connect(function()
7507         if door:GetAttribute("DoorOpen") then ventFromDoor(door) end
7508     end)
7509 end
7510 for _, door in ipairs(CollectionService:GetTagged("AutoDoor")) do watchDoor(door) end
7511 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(watchDoor)
7512
7513 task.spawn(function()
7514     while true do
7515         task.wait(45)
7516         local live = 0
7517         for _ in pairs(smokeByRoom) do live = live + 1 end
7518         if live < MAX_SMOKY_ROOMS then
7519             local levels = {}
7520             for levelName in pairs(roomsByLevel) do table.insert(levels, levelName) end
7521             if #levels > 0 then
7522                 local levelName = levels[math.random(1, #levels)]
7523                 -- only the small east bays; the long west room is not a "small room"
7524                 local small = {}
7525                 for _, r in ipairs(roomsByLevel[levelName]) do
7526                     if (r.x2 - r.x1) < 90 and (r.z2 - r.z1) < 90 then table.insert(small, r) end
7527                 end
7528                 if #small > 0 then
7529                     fillRoomWithSmoke(levelName, small[math.random(1, #small)])
7530                 end
7531             end
7532         end
7533     end
7534 end)
7535 task.spawn(function()
7536     while task.wait(2) do updateBotTestBoard() end
7537 end)
7538 task.spawn(function()
7539     while task.wait(2) do refreshRoomMonitors() end
7540 end)
7541 task.spawn(function()
7542     task.wait(3)
7543     while true do
7544         adaptiveFailureTick()
7545         genericFailureTick()
7546         task.wait(math.clamp(15 - #Players:GetPlayers() * 1.25, 6, 14))
7547     end
7548 end)
7549
7550 print("Equipment armed: " .. #CollectionService:GetTagged("LabEquipment")
7551     .. " items; recovery tool pads: " .. #CollectionService:GetTagged("LabToolDispenser") .. ".")
7552
7553 -----
7554 -- LAB STAFF NPCs
7555 -----
7556 local function bindNPC(part)
7557     if part:GetAttribute("TalkBound") then return end
7558     part:SetAttribute("TalkBound", true)
7559     local tipsStr = part:GetAttribute("Tips") or ""

```

```

7560     local tips = {}
7561     for t in tipsStr:gmatch("[^|]+") do table.insert(tips, t) end
7562     local label = part:GetAttribute("NPCName") or part:GetAttribute("Role") or "Lab Staff"
7563
7564     local prompt = Instance.new("ProximityPrompt")
7565     prompt.ObjectText = label
7566     prompt.ActionText = "Talk"
7567     prompt.HoldDuration = 0.3
7568     prompt.MaxActivationDistance = 10
7569     prompt.RequiresLineOfSight = false
7570     prompt.Parent = part
7571
7572     prompt.Triggered:Connect(function(player)
7573         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
7574         if #tips == 0 then return end
7575         notify(player, label .. ": " .. tips[math.random(1, #tips)])
7576     end)
7577 end
7578
7579 for _, p in ipairs(CollectionService:GetTagged("LabNPC")) do bindNPC(p) end
7580 CollectionService:GetInstanceAddedSignal("LabNPC"):Connect(bindNPC)
7581
7582 print("Lab staff NPCs armed: " .. #CollectionService:GetTagged("LabNPC") .. ".")
7583 ]==]
7584
7585 local BLASTER_SOURCE = [====[
7586 -- Blaster
7587 -- Shared, device-independent camera casting and client shot presentation.
7588
7589 local Debris = game:GetService("Debris")
7590 local ReplicatedStorage = game:GetService("ReplicatedStorage")
7591 local RunService = game:GetService("RunService")
7592
7593 local Blaster = {}
7594
7595 Blaster.REMOTE_FOLDER_NAME = "BlasterRemotes"
7596 Blaster.FIRE_REMOTE_NAME = "Fire"
7597 Blaster.EFFECT_REMOTE_NAME = "ReplicateShot"
7598
7599 local replicationConnection
7600
7601 local function finiteNumber(value)
7602     return typeof(value) == "number"
7603         and value == value
7604         and value > -1e7
7605         and value < 1e7
7606 end
7607
7608 function Blaster.IsFiniteVector(value)
7609     return typeof(value) == "Vector3"
7610         and finiteNumber(value.X)
7611         and finiteNumber(value.Y)
7612         and finiteNumber(value.Z)
7613 end
7614
7615 local function numberAttribute(tool, name, fallback, minimum, maximum)
7616     local value = tool and tool:GetAttribute(name)
7617     if not finiteNumber(value) then value = fallback end
7618     return math.clamp(value, minimum, maximum)
7619 end
7620
7621 function Blaster.GetWeaponConfig(tool)
7622     return {
7623         damage = numberAttribute(tool, "BlasterDamage", 25, 0, 1000),
7624         range = numberAttribute(tool, "BlasterRange", 250, 8, 1000),
7625         cooldown = numberAttribute(tool, "BlasterCooldown", 0.35, 0.06, 10),
7626         pellets = math.floor(numberAttribute(tool, "BlasterPellets", 1, 1, 16)),
7627         spread = numberAttribute(tool, "BlasterSpread", 0, 0, 0.35),
7628         selfCostPercent = numberAttribute(tool, "BlasterSelfCostPercent", 0, 0, 25),
7629         adsFov = numberAttribute(tool, "BlasterADSFOV", 55, 25, 100),
7630         automatic = tool and tool:GetAttribute("BlasterAutomatic") == true or false,
7631         instantHeadshot = tool and tool:GetAttribute("BlasterInstantHeadshot") == true or false,
7632         allowPlayerDamage = tool and tool:GetAttribute("BlasterAllowPlayerDamage") == true or false,
7633     }
7634 end
7635

```

```

7636 function Blaster.GetCameraRay(camera, maxRange, screenPoint)
7637     camera = camera or workspace.CurrentCamera
7638     if not camera then return nil end
7639     maxRange = math.clamp(tonumber(maxRange) or 500, 1, 2000)
7640     if typeof(screenPoint) == "Vector2" then
7641         local ray = camera.ViewportPointToRay(screenPoint.X, screenPoint.Y)
7642         if ray.Direction.Magnitude < 0.99 then return nil end
7643         return ray.Origin, ray.Direction.Unit, ray.Direction.Unit * maxRange
7644     end
7645
7646     local direction = camera.CFrame.LookVector
7647     if direction.Magnitude < 0.99 then return nil end
7648
7649     return camera.CFrame.Position, direction.Unit, direction.Unit * maxRange
7650 end
7651
7652 function Blaster.NewRaycastParams(ignoreList)
7653     local params = RaycastParams.new()
7654     params.FilterType = Enum.RaycastFilterType.Exclude
7655     params.FilterDescendantsInstances = ignoreList or {}
7656     params.IgnoreWater = true
7657     return params
7658 end
7659
7660 function Blaster.CastFromCamera(camera, maxRange, ignoreList, screenPoint)
7661     local origin, unitDirection, castDirection = Blaster.GetCameraRay(camera, maxRange, screenPoint)
7662     if not origin then return nil end
7663
7664     local result = workspace.Raycast(origin, castDirection, Blaster.NewRaycastParams(ignoreList))
7665     return {
7666         origin = origin,
7667         direction = unitDirection,
7668         result = result,
7669         position = result and result.Position or (origin + castDirection),
7670     }
7671 end
7672
7673 -- Returns the exact centre-line solution used for both the live reticle and a
7674 -- shot. The camera chooses the intended point; the muzzle ray then catches a
7675 -- nearby wall or prop that physically blocks the weapon.
7676 function Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
7677     if not tool or not tool:IsA("Tool") then return nil end
7678     local config = Blaster.GetWeaponConfig(tool)
7679     local cast = Blaster.CastFromCamera(camera, config.range, ignoreList, screenPoint)
7680     if not cast then return nil end
7681
7682     local handle = tool:FindFirstChild("Handle")
7683     local muzzleOrigin = handle and handle.Position or cast.origin
7684     local muzzleDelta = cast.position - muzzleOrigin
7685     local muzzleResult
7686     if muzzleDelta.Magnitude > 0.05 then
7687         -- Extend just beyond the camera hit point. Ending exactly on a surface can
7688         -- miss it because of floating-point/raycast endpoint precision.
7689         muzzleResult = workspace.Raycast(
7690             muzzleOrigin,
7691             muzzleDelta.Unit * (muzzleDelta.Magnitude + 0.25),
7692             Blaster.NewRaycastParams(ignoreList)
7693         )
7694     end
7695
7696     return {
7697         cameraOrigin = cast.origin,
7698         direction = cast.direction,
7699         muzzleOrigin = muzzleOrigin,
7700         position = muzzleResult and muzzleResult.Position or cast.position,
7701         result = muzzleResult,
7702         cameraResult = cast.result,
7703     }
7704 end
7705
7706 local function tracerColor(tool)
7707     local value = tool and tool:GetAttribute("BlasterTracerColor")
7708     return typeof(value) == "Color3" and value or Color3.fromRGB(255, 226, 140)
7709 end
7710
7711 function Blaster.DrawTracer(origin, hitPosition, color, lifetime)

```



```

7712     if not Blaster.IsFiniteVector(origin) or not Blaster.IsFiniteVector(hitPosition) then return end
7713     local distance = (hitPosition - origin).Magnitude
7714     if distance < 0.05 then return end
7715
7716     local beam = Instance.new("Part")
7717     beam.Name = "BlasterTracer"
7718     beam.Size = Vector3.new(0.055, 0.055, distance)
7719     beam.CFrame = CFrame.lookAt((origin + hitPosition) * 0.5, hitPosition)
7720     beam.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
7721     beam.Material = Enum.Material.Neon
7722     beam.Transparency = 0.08
7723     beam.Anchored = true
7724     beam.CanCollide = false
7725     beam.CanTouch = false
7726     beam.CanQuery = false
7727     beam.CastShadow = false
7728     beam.Parent = workspace
7729     Debris:AddItem(beam, lifetime or 0.075)
7730 end
7731
7732 function Blaster.DrawImpact(position, normal, color, lifetime)
7733     if not Blaster.IsFiniteVector(position) or not Blaster.IsFiniteVector(normal) then return end
7734
7735     local impact = Instance.new("Part")
7736     impact.Name = "BlasterImpact"
7737     impact.Shape = Enum.PartType.Ball
7738     impact.Size = Vector3.new(0.16, 0.16, 0.16)
7739     impact.Position = position + normal * 0.04
7740     impact.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
7741     impact.Material = Enum.Material.Neon
7742     impact.Anchored = true
7743     impact.CanCollide = false
7744     impact.CanTouch = false
7745     impact.CanQuery = false
7746     impact.CastShadow = false
7747     impact.Parent = workspace
7748     Debris:AddItem(impact, lifetime or 0.65)
7749 end
7750
7751 local function getRemotes(timeout)
7752     local folder = ReplicatedStorage:WaitForChild(Blaster.REMOTE_FOLDER_NAME, timeout or 10)
7753     if not folder then return nil end
7754     local fireRemote = folder:WaitForChild(Blaster.FIRE_REMOTE_NAME, timeout or 10)
7755     local effectRemote = folder:WaitForChild(Blaster.EFFECT_REMOTE_NAME, timeout or 10)
7756     return fireRemote, effectRemote
7757 end
7758
7759 function Blaster.StartReplication()
7760     if not RunService:IsClient() or replicationConnection then return replicationConnection end
7761     local _, effectRemote = getRemotes(15)
7762     if not effectRemote or not effectRemote:IsA("RemoteEvent") then return nil end
7763
7764     replicationConnection = effectRemote.OnClientEvent:Connect(function(payload)
7765         if typeof(payload) ~= "table" then return end
7766         if not Blaster.IsFiniteVector(payload.origin) or not Blaster.IsFiniteVector(payload.position) then
7767             ↵ return end
7768         Blaster.DrawTracer(payload.origin, payload.position, payload.color, 0.075)
7769         if Blaster.IsFiniteVector(payload.normal) then
7770             Blaster.DrawImpact(payload.position, payload.normal, payload.color)
7771         end
7772     end)
7773     return replicationConnection
7774 end
7775
7776 function Blaster.Fire(tool, camera, sequence, screenPoint)
7777     assert(RunService:IsClient(), "Blaster.Fire can only be called by a client")
7778     if not tool or not tool:IsA("Tool") then return nil end
7779
7780     local player = game:GetService("Players").LocalPlayer
7781     local character = player and player.Character
7782     if not character or tool.Parent ~= character then return nil end
7783
7784     local ignoreList = {character, tool, workspace.CurrentCamera}
7785     local solution = Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
7786     if not solution then return nil end

```

```

7787     Blaster.DrawTracer(solution.muzzleOrigin, solution.position, tracerColor(tool), 0.11)
7788     if solution.result then
7789         Blaster.DrawImpact(solution.position, solution.result.Normal, tracerColor(tool), 0.8)
7790     end
7791
7792     local fireRemote = getRemotes(10)
7793     if fireRemote and fireRemote:IsA("RemoteEvent") then
7794         fireRemote:FireServer({
7795             version = 1,
7796             tool = tool,
7797             cameraOrigin = solution.cameraOrigin,
7798             direction = solution.direction,
7799             sequence = sequence,
7800             clientTime = workspace:GetServerTimeNow(),
7801         })
7802     end
7803
7804     return solution
7805 end
7806
7807 return Blaster
7808
7809 ]====]
7810
7811 local BLASTER_CONTROLLER_SOURCE = [====[
7812 -- BlasterController
7813 -- Local Tool controller: ContextActionService input, mobile controls, ADS,
7814 -- touch aim damping, camera recoil, crosshair, and weapon bob.
7815
7816 local ContextActionService = game:GetService("ContextActionService")
7817 local GuiService = game:GetService("GuiService")
7818 local Players = game:GetService("Players")
7819 local ReplicatedStorage = game:GetService("ReplicatedStorage")
7820 local RunService = game:GetService("RunService")
7821 local TweenService = game:GetService("TweenService")
7822 local UserInputService = game:GetService("UserInputService")
7823
7824 local Blaster = require(script.Parent:WaitForChild("Blaster"))
7825 local localPlayer = Players.LocalPlayer
7826
7827 local BlasterController = {}
7828 BlasterController.__index = BlasterController
7829
7830 local controllers = setmetatable({}, {__mode = "k"})
7831 local nextControllerId = 0
7832
7833 local function pointInside(button, point)
7834     if not button or not button.Visible then return false end
7835     local p = button.AbsolutePosition
7836     local s = button.AbsoluteSize
7837     return point.X >= p.X and point.X <= p.X + s.X
7838         and point.Y >= p.Y and point.Y <= p.Y + s.Y
7839 end
7840
7841 local function makeCrosshair()
7842     local playerGui = localPlayer:WaitForChild("PlayerGui")
7843     local old = playerGui:FindFirstChild("BlasterHUD")
7844     if old then old:Destroy() end
7845
7846     local gui = Instance.new("ScreenGui")
7847     gui.Name = "BlasterHUD"
7848     gui.ResetOnSpawn = false
7849     gui.IgnoreGuiInset = true
7850     gui.ScreenInsets = Enum.ScreenInsets.None
7851     gui.ClipToDeviceSafeArea = false
7852     gui.DisplayOrder = 12
7853     gui.Enabled = false
7854     gui.Parent = playerGui
7855
7856     local root = Instance.new("Frame")
7857     root.Name = "Crosshair"
7858     root.AnchorPoint = Vector2.new(0.5, 0.5)
7859     root.Position = UDim2.fromScale(0.5, 0.5)
7860     root.Size = UDim2.fromOffset(30, 30)
7861     root.BackgroundTransparency = 1
7862     root.Parent = gui

```

```

7863
7864     local dot = Instance.new("Frame")
7865     dot.Name = "Dot"
7866     dot.AnchorPoint = Vector2.new(0.5, 0.5)
7867     dot.Position = UDim2.fromScale(0.5, 0.5)
7868     dot.Size = UDim2.fromOffset(6, 6)
7869     dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
7870     dot.BorderSizePixel = 0
7871     dot.Parent = root
7872     local dotCorner = Instance.new("UICorner")
7873     dotCorner.CornerRadius = UDim.new(1, 0)
7874     dotCorner.Parent = dot
7875     local dotStroke = Instance.new("UIStroke")
7876     dotStroke.Color = Color3.fromRGB(6, 15, 22)
7877     dotStroke.Transparency = 0.15
7878     dotStroke.Thickness = 1.5
7879     dotStroke.Parent = dot
7880
7881     for index, data in ipairs({
7882         {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
7883         {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
7884         {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
7885         {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
7886     }) do
7887         local line = Instance.new("Frame")
7888         line.Name = "Line" .. index
7889         line.Position = data[1]
7890         line.Size = data[2]
7891         line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
7892         line.BackgroundTransparency = 0.08
7893         line.BorderSizePixel = 0
7894         line.Parent = root
7895     end
7896
7897     local status = Instance.new("TextLabel")
7898     status.Name = "Status"
7899     status.AnchorPoint = Vector2.new(0.5, 0)
7900     status.Position = UDim2.new(0.5, 0, 0.5, 25)
7901     status.Size = UDim2.fromOffset(210, 24)
7902     status.BackgroundTransparency = 1
7903     status.Font = Enum.Font.GothamBold
7904     status.Text = ""
7905     status.TextColor3 = Color3.fromRGB(220, 232, 242)
7906     status.TextSize = 13
7907     status.TextStrokeTransparency = 0.5
7908     status.Parent = gui
7909
7910     return gui, root, status, dot
7911 end
7912
7913 function BlasterController.new(tool, overrides)
7914     assert(RunService.IsClient(), "BlasterController must be required on the client")
7915     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
7916
7917     nextControllerId = nextControllerId + 1
7918     local self = setmetatable({}, BlasterController)
7919     self.tool = tool
7920     self.overrides = overrides or {}
7921     self.id = nextControllerId
7922     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
7923     self.connections = {}
7924     self.enabled = false
7925     self.aiming = false
7926     self.triggerHeld = false
7927     self.fireLoopRunning = false
7928     self.sequence = 0
7929     self.nextLocalShot = 0
7930     self.baseGrip = tool.Grip
7931     self.recoilPitch = 0
7932     self.recoilYaw = 0
7933     self.touchInput = nil
7934     self.touchLastPosition = nil
7935     self.hiddenParts = {}
7936     self.shoulderOffset = Vector3.zero
7937
7938     self.connections.equipped = tool.Equipped:Connect(function()

```

```

7939         self:Enable()
7940     end)
7941     self.connections.unequipped = tool.Unequipped:Connect(function()
7942         self:Disable()
7943     end)
7944     self.connections.destroying = tool.Destroying:Connect(function()
7945         self:Destroy()
7946     end)
7947     self.connections.ancestry = tool.AncestryChanged:Connect(function()
7948         if self.enabled and tool.Parent ~= localPlayer.Character then
7949             self:Disable()
7950         end
7951     end)
7952     -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
7953     -- remains the primary input layer, while this fallback covers touch/controller layouts
7954     -- that route their built-in Tool button directly to Activated.
7955     self.connections.activated = tool.Activated:Connect(function()
7956         if self.enabled then self:_beginFiring() end
7957     end)
7958     self.connections.deactivated = tool.Deactivated:Connect(function()
7959         self.triggerHeld = false
7960     end)
7961
7962     if localPlayer.Character and tool.Parent == localPlayer.Character then
7963         task.defer(function() self:Enable() end)
7964     end
7965
7966     return self
7967 end
7968
7969 function BlasterController:_button(actionName)
7970     local ok, button = pcall(function()
7971         return ContextActionService:GetButton(actionName)
7972     end)
7973     return ok and button or nil
7974 end
7975
7976 function BlasterController:_styleTouchButtons()
7977     if not UserInputService.TouchEnabled then return end
7978     local camera = workspace.CurrentCamera
7979     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
7980     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 64, 94)
7981
7982     local shootButton = self:_button(self.actionPrefix .. "Shoot")
7983     local aimButton = self:_button(self.actionPrefix .. "Aim")
7984     for _, button in ipairs({shootButton, aimButton}) do
7985         if button then
7986             button.Size = UDim2.fromOffset(diameter, diameter)
7987             button.BackgroundTransparency = 0.18
7988             button.ImageTransparency = 0.08
7989             button.ImageColor3 = Color3.fromRGB(35, 48, 60)
7990             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
7991             corner.CornerRadius = UDim.new(1, 0)
7992             corner.Parent = button
7993             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
7994             stroke.Color = Color3.fromRGB(220, 235, 245)
7995             stroke.Transparency = 0.25
7996             stroke.Thickness = 2
7997             stroke.Parent = button
7998             local title = button:FindFirstChild("ActionTitle")
7999             if title and title:IsA("TextLabel") then
8000                 title.Font = Enum.Font.GothamBold
8001                 title.TextColor3 = Color3.fromRGB(245, 250, 255)
8002                 title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
8003                 title.TextStrokeTransparency = 0.25
8004                 title.TextScaled = true
8005             end
8006         end
8007     end
8008
8009     -- ContextActionGui uses a cropped/offset canvas in Studio's phone simulator.
8010     -- Place the actual buttons in viewport-pixel space so their hit targets stay
8011     -- fully visible on notched phones as well as large tablets.
8012     local edge = math.max(24, math.floor(diameter * 0.3))
8013     local gap = math.max(16, math.floor(diameter * 0.25))
8014     -- Keep FIRE above Roblox's jump control and AIM to its left. A full button-height

```

```

8015 -- bottom reserve prevents both actions from being clipped by Android navigation
8016 -- and Studio's simulated safe-area canvas.
8017 local bottomY = viewport.Y - diameter - math.max(82, edge)
8018 local shootPoint = Vector2.new(
8019     viewport.X - diameter - edge,
8020     math.max(64, bottomY - diameter - gap)
8021 )
8022 local aimPoint = Vector2.new(
8023     viewport.X - diameter * 2 - gap - edge,
8024     math.max(64, bottomY)
8025 )
8026 local function place(button, point)
8027     if not button or not button.Parent then return end
8028     button.AnchorPoint = Vector2.zero
8029     local origin = button.Parent.AbsolutePosition
8030     button.Position = UDim2.fromOffset(
8031         math.floor(point.X - origin.X + 0.5),
8032         math.floor(point.Y - origin.Y + 0.5)
8033     )
8034 end
8035 place(shootButton, shootPoint)
8036 place(aimButton, aimPoint)
8037 end
8038
8039 function BlasterController:_getAimScreenPoint(camera)
8040     if not camera then return nil end
8041     if UserInputService.TouchEnabled then
8042         return camera.ViewportSize * 0.5
8043     end
8044     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
8045     return Vector2.new(
8046         math.clamp(point.X, 0, camera.ViewportSize.X),
8047         math.clamp(point.Y, 0, camera.ViewportSize.Y)
8048     )
8049 end
8050
8051 function BlasterController:_updateAimReticle(camera)
8052     if not self.hud or not self.crosshair or not camera then return end
8053     local screenPoint = self:_getAimScreenPoint(camera) or camera.ViewportSize * 0.5
8054     local character = localPlayer.Character
8055     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
8056         character,
8057         self.tool,
8058         camera,
8059     }, screenPoint)
8060     if solution then
8061         local projected, visible = camera:WorldToViewportPoint(solution.position)
8062         if visible and projected.Z > 0 then
8063             screenPoint = Vector2.new(projected.X, projected.Y)
8064         end
8065         if self.reticledot then
8066             self.reticledot.BackgroundColor3 = solution.result
8067             and Color3.fromRGB(255, 224, 105)
8068             or Color3.fromRGB(120, 235, 255)
8069         end
8070     end
8071 end
8072
8073 -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
8074 -- and begin at a negative absolute position. Counter that offset so this
8075 -- marker identifies the muzzle-checked world hit on every device.
8076 local origin = self.hud.AbsolutePosition
8077 local center = screenPoint - origin
8078 self.crosshair.Position = UDim2.fromOffset(
8079     math.floor(center.X + 0.5),
8080     math.floor(center.Y + 0.5)
8081 )
8082 end
8083
8084 function BlasterController:_setLocalHeadHidden(hidden)
8085     local character = localPlayer.Character
8086     if not character then return end
8087     if hidden then
8088         for _, descendant in ipairs(character:GetDescendants()) do
8089             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
8090                 then
8091                 if self.hiddenParts[descendant] == nil then

```

```

8090         self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
8091     end
8092     descendant.LocalTransparencyModifier = 1
8093 end
8094 end
8095 else
8096     for part, transparency in pairs(self.hiddenParts) do
8097         if part.Parent then part.LocalTransparencyModifier = transparency end
8098     end
8099     table.clear(self.hiddenParts)
8100 end
8101 end
8102
8103 function BlasterController:_beginTouchCamera()
8104     if not UserInputService.TouchEnabled or self.mobileCameraActive then return end
8105     local camera = workspace.CurrentCamera
8106     local character = localPlayer.Character
8107     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8108     if not camera or not humanoid then return end
8109
8110     local look = camera.CFrame.LookVector
8111     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
8112     self.touchYaw = math.atan2(-look.X, -look.Z)
8113     self.savedCameraType = camera.CameraType
8114     self.savedAutoRotate = humanoid.AutoRotate
8115     self.mobileCameraActive = true
8116     humanoid.AutoRotate = false
8117     camera.CameraType = Enum.CameraType.Scriptable
8118     self:_setLocalHeadHidden(true)
8119 end
8120
8121 function BlasterController:_endTouchCamera()
8122     if not self.mobileCameraActive then return end
8123     local camera = workspace.CurrentCamera
8124     local character = localPlayer.Character
8125     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8126     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
8127     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
8128     self.mobileCameraActive = false
8129     self.touchInput = nil
8130     self.touchLastPosition = nil
8131     self:_setLocalHeadHidden(false)
8132 end
8133
8134 function BlasterController:SetAiming(aiming)
8135     if not self.enabled then aiming = false end
8136     if self.aiming == aiming then return end
8137     self.aiming = aiming
8138
8139     local camera = workspace.CurrentCamera
8140     if not camera then return end
8141     local config = Blaster.GetWeaponConfig(self.tool)
8142     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
8143     if self.fovTween then self.fovTween:Cancel() end
8144     self.fovTween = TweenService:Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
8145         ↪ Enum.EasingDirection.Out), {
8146         FieldOfView = targetFov,
8147     })
8148     self.fovTween:Play()
8149
8150     if UserInputService.TouchEnabled then
8151         -- Mobile weapons keep the camera active in hip-fire too, so the right
8152         -- half of the screen always steers the reticle like a dedicated FPS.
8153         self:_beginTouchCamera()
8154     elseif aiming then
8155         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
8156         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
8157     else
8158         if self.savedMouseSensitivity then
8159             UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
8160             self.savedMouseSensitivity = nil
8161         end
8162     end
8163
8164     local character = localPlayer.Character
8165     local humanoid = character and character:FindFirstChildOfClass("Humanoid")

```

```

8165     if humanoid and not self.mobileCameraActive then
8166         humanoid.CameraOffset = aiming
8167             and Vector3.new(1.25, 0.55, 0)
8168         or self.shoulderOffset
8169     end
8170
8171     if self.statusLabel then
8172         self.statusLabel.Text = aiming and "ADS" or ""
8173     end
8174 end
8175
8176 function BlasterController:_fireOnce()
8177     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
8178     local config = Blaster.GetWeaponConfig(self.tool)
8179     local now = os.clock()
8180     if now < self.nextLocalShot then return end
8181     self.nextLocalShot = now + config.cooldown
8182     self.sequence = self.sequence + 1
8183
8184     local camera = workspace.CurrentCamera
8185     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
8186     local recoilScale = self.aiming and 0.55 or 1
8187     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
8188     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
8189 end
8190
8191 function BlasterController:_beginFiring()
8192     self.triggerHeld = true
8193     if self.fireLoopRunning then return end
8194     self.fireLoopRunning = true
8195     task.spawn(function()
8196         repeat
8197             self:_fireOnce()
8198             local config = Blaster.GetWeaponConfig(self.tool)
8199             if not config.automatic then break end
8200             task.wait(config.cooldown)
8201         until not self.triggerHeld or not self.enabled
8202         self.fireLoopRunning = false
8203     end)
8204 end
8205
8206 function BlasterController:_shootAction(_, inputState)
8207     if not self.enabled then return Enum.ContextActionResult.Pass end
8208     if inputState == Enum.UserInputState.Begin then
8209         self:_beginFiring()
8210     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8211         self.triggerHeld = false
8212     end
8213     return Enum.ContextActionResult.Sink
8214 end
8215
8216 function BlasterController:_aimAction(_, inputState, inputObject)
8217     if not self.enabled then return Enum.ContextActionResult.Pass end
8218     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
8219     if isTouch then
8220         if inputState == Enum.UserInputState.Begin then
8221             self:SetAiming(not self.aiming)
8222         end
8223     elseif inputState == Enum.UserInputState.Begin then
8224         self:SetAiming(true)
8225     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8226         self:SetAiming(false)
8227     end
8228     return Enum.ContextActionResult.Sink
8229 end
8230
8231 function BlasterController:_touchPanAction(_, inputState, inputObject)
8232     if not self.enabled or not UserInputService.TouchEnabled then
8233         return Enum.ContextActionResult.Pass
8234     end
8235
8236     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
8237     local camera = workspace.CurrentCamera
8238     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
8239     local inset = GuiService.GetGuiInset()
8240     point = point - inset

```

```

8241
8242     if inputState == Enum.UserInputState.Begin then
8243         local shootButton = self:_button(self.actionPrefix .. "Shoot")
8244         local aimButton = self:_button(self.actionPrefix .. "Aim")
8245         if point.X < viewport.X * 0.42 or pointInside(shootButton, point) or pointInside(aimButton, point) then
8246             return Enum.ContextActionResult.Pass
8247         end
8248         self.touchInput = inputObject
8249         self.touchLastPosition = point
8250         return Enum.ContextActionResult.Sink
8251     end
8252
8253     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
8254     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
8255         local delta = point - self.touchLastPosition
8256         self.touchLastPosition = point
8257         local sensitivity = self.aiming
8258             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
8259             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
8260         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
8261         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
8262         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
            ↪ math.rad(78))
8263         return Enum.ContextActionResult.Sink
8264     end
8265
8266     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8267         self.touchInput = nil
8268         self.touchLastPosition = nil
8269         return Enum.ContextActionResult.Sink
8270     end
8271     return Enum.ContextActionResult.Pass
8272 end
8273
8274 function BlasterController:_render(dt)
8275     if not self.enabled then return end
8276     local character = localPlayer.Character
8277     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8278     local root = character and character:FindFirstChild("HumanoidRootPart")
8279     local head = character and character:FindFirstChild("Head")
8280     local camera = workspace.CurrentCamera
8281     if not humanoid or not camera then return end
8282     local moving = math.clamp(humanoid.MoveDirection.Magnitude, 0, 1)
8283     local speed = self.aiming and 7 or 9.5
8284     local amount = self.aiming and 0.012 or 0.035
8285     local clock = os.clock() * speed
8286     local bob = CFrame.new(math.sin(clock) * amount * moving, math.abs(math.cos(clock)) * amount * moving, 0)
8287         * CFrame.Angles(0, 0, math.sin(clock) * math.rad(0.35) * moving)
8288     self.tool.Grip = self.baseGrip * bob
8289
8290     self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
8291     self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
8292     if self.mobileCameraActive and head and root then
8293         self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
8294         self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
8295         local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
8296         local focus = head.Position + Vector3.new(0, 0.12, 0)
8297         local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
8298         local cameraResult = workspace:Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
            ↪ self.tool}))
8299         local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
8300         camera.CFrame = CFrame.new(position) * rotation
8301         camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
8302         local flatLook = Vector3.new(rotation.LookVector.X, 0, rotation.LookVector.Z)
8303         if flatLook.Magnitude > 0.01 then
8304             root.CFrame = CFrame.lookAt(root.Position, root.Position + flatLook.Unit)
8305         end
8306     elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
8307         camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)
8308     end
8309     self:_updateAimReticle(camera)
8310 end
8311
8312 function BlasterController:Enable()
8313     if self.enabled then return end
8314     self.enabled = true

```



```

8315     self.baseGrip = self.tool.Grip
8316     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
↪ workspace.CurrentCamera.FieldOfView) or 70
8317
8318     if not self.hud or not self.hud.Parent then
8319         self.hud, self.crosshair, self.statusLabel, self.reticleDot = makeCrosshair()
8320     end
8321     self.hud.Enabled = true
8322
8323     local character = localPlayer.Character
8324     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8325     if humanoid then
8326         self.savedCameraOffset = humanoid.CameraOffset
8327         self.shoulderOffset = Vector3.new(
8328             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
8329             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
8330             0
8331         )
8332         humanoid.CameraOffset = self.shoulderOffset
8333     end
8334     if UserInputService.TouchEnabled then self:_beginTouchCamera() end
8335
8336     local shootName = self.actionPrefix .. "Shoot"
8337     local aimName = self.actionPrefix .. "Aim"
8338     local panName = self.actionPrefix .. "TouchPan"
8339     local priority = Enum.ContextActionPriority.High.Value
8340
8341     ContextActionService.BindActionAtPriority(shootName, function(...)
8342         return self:_shootAction(...)
8343     end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
8344     ContextActionService.BindActionAtPriority(aimName, function(...)
8345         return self:_aimAction(...)
8346     end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
8347     ContextActionService.BindActionAtPriority(panName, function(...)
8348         return self:_touchPanAction(...)
8349     end, false, priority + 1, Enum.UserInputType.Touch)
8350
8351     ContextActionService.SetTitle(shootName, "FIRE")
8352     ContextActionService.SetTitle(aimName, "AIM")
8353     task.defer(function()
8354         if self.enabled then self:_styleTouchButtons() end
8355     end)
8356
8357     self.renderName = self.actionPrefix .. "Render"
8358     RunService.BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
8359         self:_render(dt)
8360     end)
8361 end
8362
8363 function BlasterController:Disable()
8364     if not self.enabled then return end
8365     self.triggerHeld = false
8366     self:SetAiming(false)
8367     self.enabled = false
8368
8369     ContextActionService.UnbindAction(self.actionPrefix .. "Shoot")
8370     ContextActionService.UnbindAction(self.actionPrefix .. "Aim")
8371     ContextActionService.UnbindAction(self.actionPrefix .. "TouchPan")
8372     if self.renderName then RunService.UnbindFromRenderStep(self.renderName) end
8373     if self.hud then self.hud.Enabled = false end
8374     if self.tool.Parent then self.tool.Grip = self.baseGrip end
8375
8376     local camera = workspace.CurrentCamera
8377     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
8378     self:_endTouchCamera()
8379     if self.savedMouseSensitivity then
8380         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
8381         self.savedMouseSensitivity = nil
8382     end
8383     local character = localPlayer.Character
8384     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8385     if humanoid and self.savedCameraOffset then
8386         humanoid.CameraOffset = self.savedCameraOffset
8387     end
8388     self.savedCameraOffset = nil
8389 end

```

```

8390
8391 function BlasterController:Destroy()
8392     self:Disable()
8393     for _, connection in pairs(self.connections) do
8394         connection:Disconnect()
8395     end
8396     table.clear(self.connections)
8397     if self.hud then self.hud:Destroy() end
8398     controllers[self.tool] = nil
8399 end
8400
8401 function BlasterController.BindTool(tool, overrides)
8402     if controllers[tool] then return controllers[tool] end
8403     local controller = BlasterController.new(tool, overrides)
8404     controllers[tool] = controller
8405     Blaster.StartReplication()
8406     return controller
8407 end
8408
8409 function BlasterController.GetController(tool)
8410     return controllers[tool]
8411 end
8412
8413 function BlasterController.AutoBind()
8414     local binder = {connections = {}}
8415
8416     local function consider(instance)
8417         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
8418             ⇐ instance:GetAttribute("LabSidearm")) then
8419             BlasterController.BindTool(instance)
8420         end
8421     end
8422
8423     local function watch(container)
8424         if not container then return end
8425         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
8426         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
8427     end
8428
8429     watch(localPlayer.WaitForChild("Backpack"))
8430     watch(localPlayer.Character)
8431     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
8432     Blaster.StartReplication()
8433
8434     function binder:Destroy()
8435         for _, connection in ipairs(self.connections) do connection:Disconnect() end
8436         table.clear(self.connections)
8437     end
8438
8439     return binder
8440 end
8441
8442 return BlasterController
8443 ]====]
8444
8445 local SERVER_BLASTER_MANAGER_SOURCE = [====[
8446 -- ServerBlasterManager
8447 -- Authoritative validation, obstacle recasting, damage, and shot replication.
8448
8449 local Players = game:GetService("Players")
8450 local ReplicatedStorage = game:GetService("ReplicatedStorage")
8451 local ServerScriptService = game:GetService("ServerScriptService")
8452
8453 local modules = ReplicatedStorage.WaitForChild("Modules")
8454 local Blaster = require(modules.WaitForChild("Blaster"))
8455
8456 local remoteFolder = ReplicatedStorage.FindFirstChild(Blaster.REMOTE_FOLDER_NAME)
8457 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end
8458 if not remoteFolder then
8459     remoteFolder = Instance.new("Folder")
8460     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME
8461     remoteFolder.Parent = ReplicatedStorage
8462 end
8463
8464 local function getRemote(name)

```

```

8465     local remote = remoteFolder:FindFirstChild(name)
8466     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end
8467     if not remote then
8468         remote = Instance.new("RemoteEvent")
8469         remote.Name = name
8470         remote.Parent = remoteFolder
8471     end
8472     return remote
8473 end
8474
8475 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)
8476 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)
8477
8478 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")
8479 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end
8480 if not hitBridge then
8481     hitBridge = Instance.new("BindableFunction")
8482     hitBridge.Name = "BlasterHitBridge"
8483     hitBridge.Parent = ServerScriptService
8484 end
8485
8486 -- Roblox's default third-person zoom can legitimately place the camera well
8487 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
8488 -- zoom distances and the over-shoulder hip-fire camera.
8489 local MAX_CAMERA_OFFSET = 40
8490 local RATE_TOLERANCE = 0.88
8491 local nextShotByPlayer = {}
8492 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
8493
8494 local function findHumanoidFromHit(instance)
8495     local node = instance
8496     while node and node ~= workspace do
8497         if node:IsA("Model") then
8498             local humanoid = node:FindFirstChildOfClass("Humanoid")
8499             if humanoid then return humanoid, node end
8500         end
8501         node = node.Parent
8502     end
8503     return nil
8504 end
8505
8506 local function randomSpread(direction, spread)
8507     if spread <= 0 then return direction end
8508     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
8509     local right = direction:Cross(reference).Unit
8510     local up = right:Cross(direction).Unit
8511     local radius = math.sqrt(math.random()) * spread
8512     local angle = math.random() * math.pi * 2
8513     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
8514 end
8515
8516 local function broadcastShot(shooter, origin, position, normal, color)
8517     local payload = {
8518         origin = origin,
8519         position = position,
8520         normal = normal,
8521         color = color,
8522     }
8523     for _, other in ipairs(Players:GetPlayers()) do
8524         if other ~= shooter then effectRemote:FireClient(other, payload) end
8525     end
8526 end
8527
8528 local function bridgeHit(player, result, damage, instantHeadshot)
8529     if not result then return false end
8530     -- BindableFunction callbacks are write-only. LabEquipment binds this during
8531     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
8532     local ok, response = pcall(function()
8533         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
8534             instantHeadshot and result.Instance.Name == "Head")
8535     end)
8536     return ok and response == true
8537 end
8538
8539 local function genericDamage(player, result, config)
8540     local humanoid, model = findHumanoidFromHit(result.Instance)

```

```

8541     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
8542     local victimPlayer = Players:GetPlayerFromCharacter(model)
8543     if victimPlayer and not config.allowPlayerDamage then return end
8544
8545     local damage = config.damage
8546     if result.Instance.Name == "Head" then
8547         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
8548     end
8549     humanoid:TakeDamage(damage)
8550 end
8551
8552 local function validSequence(tool, sequence)
8553     if typeof(sequence) ~= "number" or sequence ~= sequence or sequence % 1 ~= 0 then return false end
8554     if sequence < 1 or sequence > 2147483647 then return false end
8555     local previous = lastSequenceByTool[tool]
8556     if previous and sequence <= previous then return false end
8557     lastSequenceByTool[tool] = sequence
8558     return true
8559 end
8560
8561 local function handleFire(player, packet)
8562     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
8563     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
8564         ↪ return end
8565
8566     local character = player.Character
8567     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8568     local head = character and character:FindFirstChild("Head")
8569     local root = character and character:FindFirstChild("HumanoidRootPart")
8570     local tool = packet.tool
8571     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
8572     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
8573     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
8574
8575     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
8576     local directionMagnitude = packet.direction.Magnitude
8577     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
8578     local aimDirection = packet.direction.Unit
8579     if not validSequence(tool, packet.sequence) then return end
8580
8581     local config = Blaster.GetWeaponConfig(tool)
8582     local now = os.clock()
8583     if now < (nextShotByPlayer[player] or 0) then return end
8584     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
8585
8586     local params = Blaster.NewRaycastParams({character, tool})
8587     local handle = tool:FindFirstChild("Handle")
8588     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
8589     local tracerColor = tool:GetAttribute("BlasterTracerColor")
8590     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
8591
8592     for _ = 1, config.pellets do
8593         local pelletDirection = randomSpread(aimDirection, config.spread)
8594         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
8595         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
8596             ↪ config.range)
8597         local muzzleDelta = aimPoint - muzzleOrigin
8598         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection
8599         local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
8600         local result = workspace:Raycast(muzzleOrigin, castDirection * castLength, params)
8601         local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
8602
8603         broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
8604         if result then
8605             local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
8606             if not handled then genericDamage(player, result, config) end
8607         end
8608     end
8609
8610     if config.selfCostPercent > 0 and humanoid.Health > 0 then
8611         humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
8612     end
8613 end
8614
8615 fireRemote.OnServerEvent:Connect(handleFire)
8616 Players.PlayerRemoving:Connect(function(player)

```

```

8615     nextShotByPlayer[player] = nil
8616 end)
8617
8618 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")
8619
8620 ]====]
8621
8622 -- Compact responsive score/timer overlay. Touch devices get only the live
8623 -- essentials; the top-three board remains available on larger displays.
8624 local LAB_HUD_SOURCE = {[
8625     local Players = game:GetService("Players")
8626     local TweenService = game:GetService("TweenService")
8627     local UIS = game:GetService("UserInputService")
8628     local RunService = game:GetService("RunService")
8629
8630     local player = Players.LocalPlayer
8631     local camera = workspace.CurrentCamera
8632     local compact = UIS.TouchEnabled or (camera and camera.ViewportSize.X < 900)
8633     local S = compact and 0.78 or 0.9
8634
8635     local INK = Color3.fromRGB(232, 238, 236)
8636     local PANEL = Color3.fromRGB(18, 22, 24)
8637     local GOOD = Color3.fromRGB(96, 226, 150)
8638     local WARN = Color3.fromRGB(255, 186, 74)
8639     local HOT = Color3.fromRGB(255, 96, 84)
8640
8641     local old = player:WaitForChild("PlayerGui"):FindFirstChild("LabHUD")
8642     if old then old:Destroy() end
8643     local gui = Instance.new("ScreenGui")
8644     gui.Name = "LabHUD"
8645     gui.ResetOnSpawn = false
8646     gui.IgnoreGuiInset = false
8647     gui.DisplayOrder = 10
8648     gui.ZIndexBehavior = Enum.ZIndexBehavior.Sibling
8649     gui.Parent = player.PlayerGui
8650
8651     local function panel(parent, size, pos, anchor)
8652         local f = Instance.new("Frame")
8653         f.Size, f.Position, f.AnchorPoint = size, pos, anchor or Vector2.zero
8654         f.BackgroundColor3 = PANEL
8655         f.BackgroundTransparency = 0.14
8656         f.BorderSizePixel = 0
8657         f.Parent = parent
8658         local c = Instance.new("UICorner")
8659         c.CornerRadius = UDim.new(0, 8)
8660         c.Parent = f
8661         local stroke = Instance.new("UIStroke")
8662         stroke.Color = Color3.fromRGB(90, 100, 104)
8663         stroke.Transparency = 0.58
8664         stroke.Parent = f
8665         return f
8666     end
8667
8668     local function label(parent, text, size, pos, textSize, colour, align)
8669         local t = Instance.new("TextLabel")
8670         t.Size, t.Position = size, pos
8671         t.BackgroundTransparency = 1
8672         t.Font = Enum.Font.GothamBold
8673         t.Text = text
8674         t.TextSize = textSize * S
8675         t.TextColor3 = colour or INK
8676         t.TextXAlignment = align or Enum.TextXAlignment.Center
8677         t.TextTruncate = Enum.TextTruncate.AtEnd
8678         t.Parent = parent
8679         return t
8680     end
8681
8682     local roundBox = panel(gui, UDim2.fromOffset(230 * S, 52 * S), UDim2.new(0.5, 0, 0, 6), Vector2.new(0.5,
8683 ↪ 0))
8684     local roundName = label(roundBox, "SHIFT 1", UDim2.new(1, 0, 0, 16 * S), UDim2.fromOffset(0, 3), 11,
8685 ↪ Color3.fromRGB(150, 160, 165))
8686     local clock = label(roundBox, "--:--", UDim2.new(1, 0, 0, 24 * S), UDim2.fromOffset(0, 17 * S), 21)
8687     local barBg = Instance.new("Frame")
8688     barBg.Size = UDim2.new(1, -18, 0, 4)
8689     barBg.Position = UDim2.new(0, 9, 1, -8)
8690     barBg.BackgroundColor3 = Color3.fromRGB(52, 58, 62)

```

```

8689 barBg.BorderSizePixel = 0
8690 barBg.Parent = roundBox
8691 local barCorner = Instance.new("UICorner")
8692 barCorner.CornerRadius = UDim.new(1, 0)
8693 barCorner.Parent = barBg
8694 local bar = Instance.new("Frame")
8695 bar.Size = UDim2.fromScale(1, 1)
8696 bar.BackgroundColor3 = G00D
8697 bar.BorderSizePixel = 0
8698 bar.Parent = barBg
8699 local fillCorner = Instance.new("UICorner")
8700 fillCorner.CornerRadius = UDim.new(1, 0)
8701 fillCorner.Parent = bar
8702
8703 local scoreBox = panel(gui, UDim2.fromOffset(140 * S, 52 * S), UDim2.new(1, -8, 0, 6), Vector2.new(1, 0))
8704 label(scoreBox, "SCORE", UDim2.new(1, -10, 0, 14 * S), UDim2.fromOffset(0, 3), 10, Color3.fromRGB(150, 160,
↵ 165), Enum.TextXAlignment.Right)
8705 local scoreText = label(scoreBox, "0", UDim2.new(1, -10, 0, 23 * S), UDim2.fromOffset(0, 16 * S), 22, INK,
↵ Enum.TextXAlignment.Right)
8706 local pbText = label(scoreBox, "BEST 0", UDim2.new(1, -10, 0, 11 * S), UDim2.new(0, 0, 1, -13 * S), 9,
↵ Color3.fromRGB(140, 150, 155), Enum.TextXAlignment.Right)
8707 local streakText = label(scoreBox, "", UDim2.new(0.55, 0, 0, 16 * S), UDim2.fromOffset(7, 18 * S), 11,
↵ WARN, Enum.TextXAlignment.Left)
8708
8709 local popups = Instance.new("Frame")
8710 popups.Size = UDim2.fromOffset(190 * S, 150 * S)
8711 popups.Position = UDim2.new(1, -8, 0, 54 * S)
8712 popups.AnchorPoint = Vector2.new(1, 0)
8713 popups.BackgroundTransparency = 1
8714 popups.Parent = gui
8715 local popupLayout = Instance.new("UIListLayout")
8716 popupLayout.HorizontalAlignment = Enum.HorizontalAlignment.Right
8717 popupLayout.Padding = UDim.new(0, 3)
8718 popupLayout.Parent = popups
8719
8720 local board = panel(gui, UDim2.fromOffset(170 * S, 82 * S), UDim2.new(1, -8, 0, 64 * S), Vector2.new(1, 0))
8721 board.Visible = not compact
8722 label(board, "TOP TECHNICIANS", UDim2.new(1, 0, 0, 14 * S), UDim2.fromOffset(0, 4), 10, Color3.fromRGB(150,
↵ 160, 165))
8723 local rows = {}
8724 for i = 1, 3 do
8725     rows[i] = label(board, "", UDim2.new(1, -14, 0, 17 * S), UDim2.fromOffset(7, (16 + i * 17) * S), 11,
↵ INK, Enum.TextXAlignment.Left)
8726     rows[i].Font = Enum.Font.Gotham
8727 end
8728
8729 local shown, target = 0, 0
8730 local function pointsValue()
8731     local stats = player:FindFirstChild("leaderstats")
8732     local points = stats and stats:FindFirstChild("Points")
8733     return points and points.Value or 0
8734 end
8735
8736 local streak, lastAward = 0, 0
8737 local function popup(amount, reason)
8738     local row = panel(popups, UDim2.fromOffset(184 * S, 25 * S), UDim2.fromOffset(0, 0), Vector2.zero)
8739     local amountLabel = label(row, "+" .. amount, UDim2.new(0, 45 * S, 1, 0), UDim2.fromOffset(6, 0), 13,
↵ G00D, Enum.TextXAlignment.Left)
8740     local reasonLabel = label(row, tostring(reason), UDim2.new(1, -53 * S, 1, 0), UDim2.fromOffset(49 * S,
↵ 0), 9, Color3.fromRGB(190, 200, 205), Enum.TextXAlignment.Left)
8741     reasonLabel.Font = Enum.Font.Gotham
8742     task.delay(2.2, function()
8743         if not row.Parent then return end
8744         TweenService:Create(row, TweenInfo.new(0.3), {BackgroundTransparency = 1}):Play()
8745         TweenService:Create(amountLabel, TweenInfo.new(0.3), {TextTransparency = 1}):Play()
8746         TweenService:Create(reasonLabel, TweenInfo.new(0.3), {TextTransparency = 1}):Play()
8747         task.wait(0.32)
8748         row:Destroy()
8749     end)
8750 end
8751
8752 player:GetAttributeChangedSignal("LastPointAward"):Connect(function()
8753     local amount = player:GetAttribute("LastPointAmount") or 0
8754     if amount <= 0 then return end
8755     local now = os.clock()
8756     streak = (now - lastAward <= 14) and streak + 1 or 1

```

```

8757     lastAward = now
8758     popup(amount, player:GetAttribute("LastPointAward") or "Task complete")
8759     scoreText.TextColor3 = GOOD
8760     TweenService:Create(scoreText, TweenInfo.new(0.5), {TextColor3 = INK}):Play()
8761     if streak >= 2 then streakText.Text = "x" .. streak end
8762 end)
8763
8764 local acc = 0
8765 RunService.RenderStepped:Connect(function(dt)
8766     target = pointsValue()
8767     if shown ~= target then
8768         shown += (target - shown) * math.min(dt * 8, 1)
8769         if math.abs(target - shown) < 0.6 then shown = target end
8770         scoreText.Text = tostring(math.floor(shown + 0.5))
8771     end
8772     if streak >= 2 and os.clock() - lastAward > 14 then streak = 0; streakText.Text = "" end
8773
8774     local endsAt = workspace:GetAttribute("RoundEndsAt")
8775     local length = workspace:GetAttribute("RoundLength") or 480
8776     local phase = workspace:GetAttribute("RoundPhase") or "PLAY"
8777     if endsAt then
8778         local left = math.max(0, endsAt - os.time())
8779         clock.Text = string.format("%d:%02d", left // 60, left % 60)
8780         if phase == "BREAK" then
8781             roundName.Text = "SHIFT OVER"
8782             bar.BackgroundColor3 = Color3.fromRGB(120, 130, 140)
8783             bar.Size = UDim2.fromScale(1, 1)
8784         else
8785             roundName.Text = "SHIFT " .. tostring(workspace:GetAttribute("RoundNumber") or 1)
8786             bar.Size = UDim2.new(math.clamp(left / length, 0, 1), 0, 1, 0)
8787             bar.BackgroundColor3 = left <= 30 and HOT or left <= 90 and WARN or GOOD
8788             clock.TextColor3 = (left <= 10 and left > 0 and left % 2 == 0) and HOT or INK
8789         end
8790     end
8791
8792     acc += dt
8793     if acc >= 1.5 then
8794         acc = 0
8795         local list = {}
8796         for _, pl in ipairs(Players:GetPlayers()) do
8797             local stats = pl:FindFirstChild("leaderstats")
8798             local points = stats and stats:FindFirstChild("Points")
8799             table.insert(list, {name = pl.Name, value = points and points.Value or 0})
8800         end
8801         table.sort(list, function(a, b) return a.value > b.value end)
8802         for i = 1, 3 do
8803             local entry = list[i]
8804             rows[i].Text = entry and string.format("%d. %s %d", i, entry.name, entry.value) or ""
8805             rows[i].TextColor3 = entry and entry.name == player.Name and GOOD or INK
8806         end
8807         pbText.Text = "BEST " .. tostring(player:GetAttribute("PersonalBest") or 0)
8808     end
8809 end)
8810 ]==]
8811
8812 -- Lightweight client bootstrap. Input, crosshair, camera, bob and ADS all
8813 -- live in BlasterController; this script updates the shared compact HUD and binds
8814 -- every standard Tool carrying the BlasterWeapon attribute.
8815 local SIDEARM_CLIENT_SOURCE = [=[
8816 local Players      = game:GetService("Players")
8817 local ReplicatedStorage = game:GetService("ReplicatedStorage")
8818
8819 local player = Players.LocalPlayer
8820
8821 -- The hunt counter belongs inside RoleHUD; a second top-centre panel covered
8822 -- the crosshair and duplicated information on phones.
8823 task.spawn(function()
8824     local complex = workspace:WaitForChild("Laboratory_Complex", 60)
8825     if not complex then return end
8826     local playerGui = player:WaitForChild("PlayerGui", 30)
8827     if not playerGui then return end
8828     local roleGui = playerGui:WaitForChild("RoleHUD", 30)
8829     local frame = roleGui and roleGui:FindFirstChildOfClass("Frame")
8830     local label = frame and frame:FindFirstChild("TargetsLabel")
8831     if not label then return end
8832

```

```

8833     local function refresh()
8834         local killed = complex:GetAttribute("BotsKilled") or 0
8835         local total = complex:GetAttribute("BotsTotal") or 0
8836         if total > 0 and killed >= total then
8837             label.Text = "LAB CLEARED • " .. killed .. " / " .. total
8838             label.TextColor3 = Color3.fromRGB(125, 255, 155)
8839         else
8840             label.Text = string.format("TARGETS %d / %d", killed, total)
8841             label.TextColor3 = Color3.fromRGB(255, 205, 120)
8842         end
8843     end
8844     complex:GetAttributeChangedSignal("BotsKilled"):Connect(refresh)
8845     complex:GetAttributeChangedSignal("BotsTotal"):Connect(refresh)
8846     refresh()
8847 end)
8848
8849 -- One lightweight bootstrap watches standard Tools as they move between the
8850 -- Backpack and Character. All device input/camera behavior lives in the module.
8851 local modules = ReplicatedStorage:WaitForChild("Modules", 30)
8852 if not modules then
8853     warn("Blaster modules were not installed")
8854     return
8855 end
8856
8857 local BlasterController = require(modules:WaitForChild("BlasterController"))
8858 BlasterController.AutoBind()
8859 ]==]
8860
8861 local ELEVATOR_SCRIPT_SOURCE = [=[
8862 -- LabElevators : moving cars that carry whoever is standing inside.
8863 -- An anchored part does not drag a character with it, so the car
8864 -- offsets every occupant by the same delta on each step.
8865
8866 local CollectionService = game:GetService("CollectionService")
8867 local RunService        = game:GetService("RunService")
8868 local Players            = game:GetService("Players")
8869
8870 local SPEED = 45 -- studs per second
8871
8872 local function setGate(gate, open)
8873     gate.CanCollide = not open
8874     gate.Transparency = open and 0.85 or 0
8875 end
8876
8877 local function setup(model)
8878     local floorCount = model:GetAttribute("FloorCount")
8879     if not floorCount then return end
8880
8881     local car = model:WaitForChild("Car")
8882     local platform = car:FindFirstChild("Platform") or car:FindFirstChild("Platform_1")
8883     if not platform then
8884         for _, d in ipairs(car:GetDescendants()) do
8885             if d:IsA("BasePart") and d.Name:match("^Platform") then platform = d break end
8886         end
8887     end
8888     if not platform then return end
8889
8890     local gates = {}
8891     for _, g in ipairs(model:WaitForChild("Gates"):GetChildren()) do
8892         gates[g:GetAttribute("Floor")] = g
8893     end
8894
8895     local ys = {}
8896     for i = 1, floorCount do ys[i] = model:GetAttribute("FloorY_" .. i) end
8897
8898     local current, moving = 1, false
8899     for i, g in pairs(gates) do setGate(g, i == current) end
8900
8901     local carParts = {}
8902     for _, d in ipairs(car:GetDescendants()) do
8903         if d:IsA("BasePart") then table.insert(carParts, d) end
8904     end
8905
8906     local function occupants()
8907         local list = {}
8908         local half = platform.Size * 0.5

```



```

8909     local top = platform.Position.Y + half.Y
8910     for _, pl in ipairs(Players:GetPlayers()) do
8911         local ch = pl.Character
8912         local hrp = ch and ch:FindFirstChild("HumanoidRootPart")
8913         if hrp then
8914             local d = hrp.Position - platform.Position
8915             if math.abs(d.X) <= half.X + 2 and math.abs(d.Z) <= half.Z + 2
8916                 and hrp.Position.Y > top - 2 and hrp.Position.Y < top + 24 then
8917                 table.insert(list, hrp)
8918             end
8919         end
8920     end
8921     return list
8922 end
8923
8924 local function moveTo(target)
8925     if moving or target == current or not ys[target] then return end
8926     moving = true
8927     setGate(gates[current], false)
8928
8929     local startY = platform.Position.Y
8930     local goalY = startY + (ys[target] - ys[current])
8931     local dist = math.abs(goalY - startY)
8932     local dur = math.max(0.4, dist / SPEED)
8933     local t0 = os.clock()
8934     local lastY = startY
8935
8936     while true do
8937         local a = math.min(1, (os.clock() - t0) / dur)
8938         local newY = startY + (goalY - startY) * a
8939         local dy = newY - lastY
8940         if dy ~= 0 then
8941             local riders = occupants()
8942             for _, p in ipairs(carParts) do
8943                 p.CFrame = p.CFrame + Vector3.new(0, dy, 0)
8944             end
8945             for _, hrp in ipairs(riders) do
8946                 hrp.CFrame = hrp.CFrame + Vector3.new(0, dy, 0)
8947             end
8948             lastY = newY
8949         end
8950         if a >= 1 then break end
8951         RunService.Heartbeat:Wait()
8952     end
8953
8954     current = target
8955     model:SetAttribute("CurrentFloor", current)
8956     setGate(gates[current], true)
8957     moving = false
8958 end
8959
8960 local function addPrompt(part, floor, label, text)
8961     local p = Instance.new("ProximityPrompt")
8962     p.ObjectText = label
8963     p.ActionText = text
8964     p.HoldDuration = 0.15
8965     p.MaxActivationDistance = 22
8966     p.RequiresLineOfSight = false
8967     p.Parent = part
8968     p.Triggered:Connect(function() moveTo(floor) end)
8969 end
8970
8971 for _, panel in ipairs(model:WaitForChild("CallPanels"):GetChildren()) do
8972     addPrompt(panel, panel:GetAttribute("Floor"), "Elevator", "Call")
8973 end
8974 for _, btn in ipairs(car:WaitForChild("Buttons"):GetChildren()) do
8975     local i = btn:GetAttribute("Floor")
8976     addPrompt(btn, i, "Elevator", "Go to floor " .. (model:GetAttribute("FloorLabel_" .. i) or i))
8977 end
8978 end
8979
8980 for _, m in ipairs(CollectionService:GetTagged("Elevator")) do setup(m) end
8981 CollectionService:GetInstanceAddedSignal("Elevator"):Connect(setup)
8982
8983 print("Elevators armed: " .. #CollectionService:GetTagged("Elevator") .. ".")
8984 ]==]

```

```

8985
8986 local ROLES_SCRIPT_SOURCE = {[
8987 -- LabRoles : role kiosks, teams, per-player leaderstats, a persistent
8988 -- HUD, and a visible appearance change (tinted uniform + hard hat) so
8989 -- everyone can see at a glance who is playing which role.
8990
8991 local Players          = game:GetService("Players")
8992 local Teams            = game:GetService("Teams")
8993 local CollectionService = game:GetService("CollectionService")
8994
8995 local ROLE_PAINT = {
8996     Technician = Color3.fromRGB(60, 120, 200),
8997     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),
8998 }
8999
9000 local ROLE_HINT = {
9001     Unassigned = "Choose BLUE Technician or GREEN Laboratory Technician.",
9002     Technician = "Follow RED floor lights. HOLD R at the failed unit. Score grows with failure time and
9003     ↪ assigned distance.",
9004     ["Laboratory Technician"] = "E takes a plant sample. Carry it to any GREEN RUNNING cabinet marked FREE SLOT
9005     ↪ and press E again to revive/drop it.",
9006 }
9007
9008 local function roleEquipmentId(part)
9009     local code = part:GetAttribute("GrowthCode")
9010     if code and code ~= "" then return "CHAMBER:" .. code end
9011     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
9012     ↪ " | ")
9013 end
9014
9015 local function roleFloor(part)
9016     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
9017 end
9018
9019 local function roleTransferSeconds(source, destination)
9020     local distance = destination and (destination.Position - source.Position).Magnitude or 60
9021     local playerCount = math.max(1, #Players:GetPlayers())
9022     local complex = workspace:FindFirstChild("Laboratory_Complex")
9023     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
9024     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
9025     + math.clamp(backlog / 20, 0, 25)), 60, 180)
9026 end
9027
9028 local function roleEquipmentLabel(part)
9029     local code = part:GetAttribute("GrowthCode")
9030     if code and code ~= "" then return "CHAMBER: " .. code end
9031     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
9032 end
9033
9034 local function roleDestinations(source)
9035     local choices = {}
9036     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
9037         local count = chamber:GetAttribute("SampleCount") or 0
9038         local reservations = chamber:GetAttribute("IncomingReservations") or 0
9039         if chamber ~= source and chamber:GetAttribute("FunctionalState") ~= "FAILED"
9040             and count + reservations < (chamber:GetAttribute("Capacity") or 10) then table.insert(choices,
9041             ↪ chamber) end
9042     end
9043     table.sort(choices, function(a, b)
9044         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9045         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9046         if sameA ~= sameB then return sameA < sameB end
9047         return (a:GetAttribute("SampleCount") or 0) < (b:GetAttribute("SampleCount") or 0)
9048     end)
9049     return choices
9050 end
9051
9052 local function clearRoleAssignment(player)
9053     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
9054     ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
9055         player:SetAttribute(name, nil)
9056     end
9057 end
9058
9059 local function assignRoleObjective(player, roleName)
9060     clearRoleAssignment(player)

```

```

9056     if roleName == "Technician" then
9057         local failed = {}
9058         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
9059             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment) end
9060         end
9061         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
9062         table.sort(failed, function(a, b)
9063             if root then return (a.Position - root.Position).Magnitude < (b.Position - root.Position).Magnitude
9064                 ↪ end
9065             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
9066         end)
9067         local target = failed[1]
9068         if not target then return "All equipment is working. Repair score grows with failure time and assigned
9069             ↪ distance." end
9070         player:SetAttribute("AssignedRepairId", roleEquipmentId(target))
9071         local distance = root and (target.Position - root.Position).Magnitude or 0
9072         local waited = math.max(0, os.time() - (target:GetAttribute("FailureSince") or os.time()))
9073         local projected = math.clamp(20 + math.floor(waited / 8) + math.floor(distance / 10), 8, 120)
9074         player:SetAttribute("AssignedRepairDistance", distance)
9075         player:SetAttribute("AssignedRepairProjectedPoints", projected)
9076         return string.format("REPAIR JOB: %s • %s. Follow RED floor lights; HOLD R. About +%d points.",
9077             roleFloor(target), roleEquipmentLabel(target), projected)
9078     end
9079
9080     local jobs = {}
9081     for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
9082         if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0) > 0
9083             and not source:GetAttribute("ReservedByBot") then
9084             local destination = roleDestinations(source)[1]
9085             if destination then table.insert(jobs, {source = source, destination = destination}) end
9086         end
9087     end
9088     table.sort(jobs, function(a, b)
9089         local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
9090         local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
9091         return an < bn
9092     end)
9093     local job = jobs[1]
9094     if not job then return "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
9095         ↪ +30." end
9096     local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
9097     local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
9098     player:SetAttribute("AssignedTransferSource", sourceCode)
9099     player:SetAttribute("AssignedTransferDestination", destinationCode)
9100     local allowedSeconds = roleTransferSeconds(job.source, job.destination)
9101     player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
9102     return "ASSIGNED +30: " .. roleFloor(job.source) .. " • " .. sourceCode .. " → " .. destinationCode
9103     .. " • " .. roleFloor(job.destination) .. ". PRESS E at source, deliver in " .. allowedSeconds .. "
9104     ↪ seconds."
9105 end
9106
9107 -----
9108 -- per-player HUD (role, contributions, current objective)
9109 -----
9110 local function buildHud(player)
9111     local playerGui = player:WaitForChild("PlayerGui")
9112     local duplicate = playerGui:FindFirstChild("LiveLabHUD")
9113     if duplicate then duplicate:Destroy() end
9114     local gui = Instance.new("ScreenGui")
9115     gui.Name = "RoleHUD"
9116     gui.ResetOnSpawn = false
9117     gui.DisplayOrder = 20
9118     gui.Parent = playerGui
9119
9120     local frame = Instance.new("Frame")
9121     frame.Size = UDim2.fromOffset(238, 90)
9122     frame.Position = UDim2.fromOffset(8, 50)
9123     frame.BackgroundColor3 = Color3.fromRGB(18, 20, 26)
9124     frame.BackgroundTransparency = 0.14
9125     frame.Parent = gui
9126     local corner = Instance.new("UICorner"); corner.CornerRadius = UDim.new(0, 8); corner.Parent = frame
9127
9128     local roleLabel = Instance.new("TextLabel")
9129     roleLabel.Name = "RoleLabel"
9130     roleLabel.Size = UDim2.new(1, -14, 0, 16)
9131     roleLabel.Position = UDim2.new(0, 8, 0, 4)

```

```

9128     roleLabel.BackgroundTransparency = 1
9129     roleLabel.TextXAlignment = Enum.TextXAlignment.Left
9130     roleLabel.Font = Enum.Font.GothamBold
9131     roleLabel.TextSize = 13
9132     roleLabel.TextColor3 = Color3.fromRGB(240, 242, 246)
9133     roleLabel.Text = "Role: Unassigned"
9134     roleLabel.Parent = frame
9135
9136     local contribLabel = Instance.new("TextLabel")
9137     contribLabel.Name = "ContribLabel"
9138     contribLabel.Size = UDim2.new(1, -14, 0, 13)
9139     contribLabel.Position = UDim2.new(0, 8, 0, 20)
9140     contribLabel.BackgroundTransparency = 1
9141     contribLabel.TextXAlignment = Enum.TextXAlignment.Left
9142     contribLabel.Font = Enum.Font.Gotham
9143     contribLabel.TextSize = 10
9144     contribLabel.TextTruncate = Enum.TextTruncate.AtEnd
9145     contribLabel.TextYAlignment = Enum.TextYAlignment.Top
9146     contribLabel.TextColor3 = Color3.fromRGB(200, 205, 212)
9147     contribLabel.Text = "POINTS 0"
9148     contribLabel.Parent = frame
9149
9150     local carryLabel = Instance.new("TextLabel")
9151     carryLabel.Name = "CarryLabel"
9152     carryLabel.Size = UDim2.new(1, -14, 0, 13)
9153     carryLabel.Position = UDim2.new(0, 8, 0, 33)
9154     carryLabel.BackgroundTransparency = 1
9155     carryLabel.TextXAlignment = Enum.TextXAlignment.Left
9156     carryLabel.Font = Enum.Font.GothamBold
9157     carryLabel.TextSize = 10
9158     carryLabel.TextColor3 = Color3.fromRGB(115, 215, 245)
9159     carryLabel.Text = "Carrying: nothing"
9160     carryLabel.Parent = frame
9161
9162     local targetsLabel = Instance.new("TextLabel")
9163     targetsLabel.Name = "TargetsLabel"
9164     targetsLabel.Size = UDim2.new(1, -14, 0, 13)
9165     targetsLabel.Position = UDim2.new(0, 8, 0, 46)
9166     targetsLabel.BackgroundTransparency = 1
9167     targetsLabel.TextXAlignment = Enum.TextXAlignment.Left
9168     targetsLabel.Font = Enum.Font.GothamBold
9169     targetsLabel.TextSize = 10
9170     targetsLabel.TextColor3 = Color3.fromRGB(255, 205, 120)
9171     targetsLabel.Text = "TARGETS 0 / 0"
9172     targetsLabel.Parent = frame
9173
9174     local hintLabel = Instance.new("TextLabel")
9175     hintLabel.Name = "HintLabel"
9176     hintLabel.Size = UDim2.new(1, -14, 0, 25)
9177     hintLabel.Position = UDim2.new(0, 8, 0, 61)
9178     hintLabel.BackgroundTransparency = 1
9179     hintLabel.TextXAlignment = Enum.TextXAlignment.Left
9180     hintLabel.TextYAlignment = Enum.TextYAlignment.Top
9181     hintLabel.TextWrapped = true
9182     hintLabel.Font = Enum.Font.Gotham
9183     hintLabel.TextSize = 9
9184     hintLabel.TextTruncate = Enum.TextTruncate.AtEnd
9185     hintLabel.TextColor3 = Color3.fromRGB(255, 220, 150)
9186     hintLabel.Text = ROLE_HINT.Unassigned
9187     hintLabel.Parent = frame
9188
9189     return gui
9190 end
9191
9192 local function refreshHud(player, roleName)
9193     local gui = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
9194     if not gui then return end
9195     local frame = gui:FindFirstChildOfClass("Frame")
9196     if not frame then return end
9197     frame.RoleLabel.Text = "Role: " .. roleName
9198     local carried = player:GetAttribute("CarriedSample")
9199     local objective = player:GetAttribute("LabObjective")
9200     frame.CarryLabel.Text = (carried and carried ~= "") and ("Carrying: " .. carried) or "Carrying: nothing"
9201     local hint = (objective and objective ~= "") and objective or (ROLE_HINT[roleName] or ROLE_HINT.Unassigned)
9202     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
9203     if roleName == "Laboratory Technician" and deadline > 0 then

```

```

9204     hint = hint .. "\nTIME LEFT FOR +30: " .. math.max(0, deadline - os.time()) .. " seconds"
9205 end
9206 frame.HintLabel.Text = hint
9207 local stats = player:FindFirstChild("leaderstats")
9208 local points = stats and stats:FindFirstChild("Points")
9209 local lastAmount = player:GetAttribute("LastPointAmount") or 0
9210 local lastReason = player:GetAttribute("LastPointAward") or ""
9211     frame.ContribLabel.Text = string.format("POINTS %d%s",
9212         points and points.Value or 0, lastAmount > 0 and (" • LAST +" .. lastAmount .. " " .. lastReason)
9213         ↪ or "")
9214 end
9215 -----
9216 -- Visible appearance change: a welded lab uniform sits above avatar
9217 -- clothing, so layered clothes cannot hide the selected role.
9218 -----
9219 local function wearablePart(uniform, bodyPart, name, size, offset, colour, material)
9220     local p = Instance.new("Part")
9221     p.Name = name
9222     p.Size = size
9223     p.CFrame = bodyPart.CFrame * offset
9224     p.Color = colour
9225     p.Material = material or Enum.Material.Fabric
9226     p.Anchored = false
9227     p.CanCollide = false
9228     p.CanTouch = false
9229     p.Massless = true
9230     p.CastShadow = false
9231     p.TopSurface = Enum.SurfaceType.Smooth
9232     p.BottomSurface = Enum.SurfaceType.Smooth
9233     p.Parent = uniform
9234     local weld = Instance.new("WeldConstraint")
9235     weld.Part0 = bodyPart
9236     weld.Part1 = p
9237     weld.Parent = p
9238     return p
9239 end
9240
9241 local function applyAppearance(character, roleName)
9242     local paint = ROLE_PAINT[roleName]
9243     if not paint then return end
9244     local oldUniform = character:FindFirstChild("RoleUniform")
9245     if oldUniform then oldUniform:Destroy() end
9246     local oldHelmet = character:FindFirstChild("RoleHelmet")
9247     if oldHelmet then oldHelmet:Destroy() end
9248
9249     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
9250     local head = character:FindFirstChild("Head")
9251     if not torso or not head then return end
9252
9253     local uniform = Instance.new("Model")
9254     uniform.Name = "RoleUniform"
9255     uniform:SetAttribute("Role", roleName)
9256     uniform.Parent = character
9257
9258     wearablePart(uniform, torso, "LabCoat", Vector3.new(torso.Size.X * 1.08, torso.Size.Y * 0.94, torso.Size.Z
9259     ↪ * 1.1),
9260         CFrame.new(), Color3.fromRGB(238, 241, 244), Enum.Material.Fabric)
9261     wearablePart(uniform, torso, "RolePanel", Vector3.new(torso.Size.X * 0.82, torso.Size.Y * 0.55, 0.14),
9262         CFrame.new(0, 0.08, -(torso.Size.Z / 2 + 0.12)), paint, Enum.Material.SmoothPlastic)
9263     wearablePart(uniform, torso, "RoleBelt", Vector3.new(torso.Size.X * 1.12, 0.18, torso.Size.Z * 1.14),
9264         CFrame.new(0, -torso.Size.Y * 0.34, 0), paint, Enum.Material.SmoothPlastic)
9265     wearablePart(uniform, torso, "RoleBadge", Vector3.new(0.36, 0.46, 0.16),
9266         CFrame.new(-torso.Size.X * 0.28, torso.Size.Y * 0.22, -(torso.Size.Z / 2 + 0.18)),
9267         Color3.fromRGB(255, 235, 120), Enum.Material.Neon)
9268
9269     local helmet = wearablePart(uniform, head, "RoleHelmet", Vector3.new(1.82, 0.72, 1.82),
9270         CFrame.new(0, 0.72, 0), paint, Enum.Material.SmoothPlastic)
9271     helmet.Shape = Enum.PartType.Ball
9272     wearablePart(uniform, head, "HelmetBand", Vector3.new(1.9, 0.16, 1.9),
9273         CFrame.new(0, 0.52, 0), Color3.fromRGB(245, 245, 245), Enum.Material.SmoothPlastic)
9274
9275     for _, limbName in ipairs({"LeftUpperArm", "RightUpperArm", "Left Arm", "Right Arm"}) do
9276         local limb = character:FindFirstChild(limbName)
9277         if limb then
9278             wearablePart(uniform, limb, limbName:gsub(" ", "") .. "RoleBand",

```

```

9278         Vector3.new(limb.Size.X * 1.12, math.min(0.24, limb.Size.Y * 0.18), limb.Size.Z * 1.12),
9279         CFrame.new(0, -limb.Size.Y * 0.26, 0), paint, Enum.Material.SmoothPlastic)
9280     end
9281 end
9282 end
9283
9284 local setupDone = {}
9285 local function setupPlayer(player)
9286     if setupDone[player] then return end
9287     setupDone[player] = true
9288     if player:GetAttribute("CarriedSample") == nil then player:SetAttribute("CarriedSample", "") end
9289     if player:GetAttribute("LabObjective") == nil then player:SetAttribute("LabObjective", "") end
9290     local stats = player:FindFirstChild("leaderstats") or Instance.new("Folder")
9291     stats.Name = "leaderstats"
9292     stats.Parent = player
9293
9294     local role = stats:FindFirstChild("Role") or Instance.new("StringValue")
9295     role.Name = "Role"
9296     if role.Value == "" then role.Value = "Unassigned" end
9297     role.Parent = stats
9298
9299     local contrib = stats:FindFirstChild("Contributions") or Instance.new("IntValue")
9300     contrib.Name = "Contributions"
9301     contrib.Parent = stats
9302     local points = stats:FindFirstChild("Points") or Instance.new("IntValue")
9303     points.Name = "Points"
9304     points.Parent = stats
9305
9306     local oldHud = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
9307     if oldHud then oldHud:Destroy() end
9308     buildHud(player)
9309     role:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9310     contrib:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9311     points:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9312     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshHud(player, role.Value) end)
9313     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshHud(player, role.Value) end)
9314     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshHud(player, role.Value) end)
9315     refreshHud(player, role.Value)
9316
9317     -- Health must never regenerate passively: it only comes back through an
9318     -- Emergency Kit. Roblox's default character ships a "Health" script that
9319     -- does exactly the auto-regen we don't want, so remove it every spawn.
9320     local function stripAutoHealthRegen(character)
9321         local healthScript = character:FindFirstChild("Health")
9322         if healthScript then healthScript:Destroy() end
9323     end
9324     player.CharacterAdded:Connect(function(character)
9325         if role.Value ~= "Unassigned" then
9326             applyAppearance(character, role.Value)
9327         end
9328         stripAutoHealthRegen(character)
9329     end)
9330     if player.Character then
9331         if role.Value ~= "Unassigned" then applyAppearance(player.Character, role.Value) end
9332         stripAutoHealthRegen(player.Character)
9333     end
9334     task.spawn(function()
9335         while player.Parent do
9336             if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshHud(player, role.Value)
9337                 ↪ end
9338             task.wait(1)
9339         end
9340     end)
9341 end
9342
9343 Players.PlayerAdded:Connect(setupPlayer)
9344 for _, player in ipairs(Players:GetPlayers()) do task.spawn(setupPlayer, player) end
9345
9346 local function bindKiosk(part)
9347     local roleName = part:GetAttribute("Role")
9348     if not roleName then return end
9349     if part:GetAttribute("RoleKioskBound") then return end
9350     part:SetAttribute("RoleKioskBound", true)
9351
9352     local button = Instance.new("Part")
9353     button.Name = "RoleSelectButton"

```

```

9353 button.Shape = Enum.PartType.Cylinder
9354 button.Size = Vector3.new(0.28, 1.05, 1.05)
9355 button.CFrame = part.CFrame * CFrame.new(0, -part.Size.Y * 0.32, -(part.Size.Z / 2 + 0.32))
9356     * CFrame.Angles(0, math.rad(90), 0)
9357 button.Color = ROLE_PAINT[roleName]
9358 button.Material = Enum.Material.Neon
9359 button.Anchored = true
9360 button.CanCollide = false
9361 button.Parent = part
9362
9363 local prompt = Instance.new("ProximityPrompt")
9364 prompt.ObjectText = roleName .. " Kiosk"
9365 prompt.ActionText = "Select " .. roleName
9366 prompt.KeyboardKeyCode = Enum.KeyCode.E
9367 prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
9368 prompt.HoldDuration = 0
9369 prompt.MaxActivationDistance = 30
9370 prompt.RequiresLineOfSight = false
9371 prompt.Parent = button
9372
9373 local click = Instance.new("ClickDetector")
9374 click.MaxActivationDistance = 30
9375 click.Parent = button
9376 local debounce = {}
9377 local function selectRole(player)
9378     if debounce[player] and os.clock() < debounce[player] then return end
9379     local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
9380     if not root or (root.Position - part.Position).Magnitude > 36 then return end
9381     debounce[player] = os.clock() + 1
9382     setupPlayer(player)
9383     local team = Teams:FindFirstChild(roleName)
9384     if not team then return end
9385     player.Team = team
9386     player.Neutral = false
9387     player:SetAttribute("LabRole", roleName)
9388     local stats = player:FindFirstChild("leaderstats")
9389     if stats and stats:FindFirstChild("Role") then
9390         stats.Role.Value = roleName
9391     end
9392     player:SetAttribute("LabObjective", assignRoleObjective(player, roleName))
9393     if player.Character then applyAppearance(player.Character, roleName) end
9394     local oldColour = button.Color
9395     button.Color = Color3.fromRGB(255, 255, 255)
9396     task.delay(0.35, function() if button.Parent then button.Color = oldColour end end)
9397     refreshHud(player, roleName)
9398 end
9399
9400 prompt.Triggered:Connect(selectRole)
9401 click.MouseClick:Connect(selectRole)
9402 end
9403
9404 for _, p in ipairs(CollectionService:GetTagged("RoleKiosk")) do bindKiosk(p) end
9405 CollectionService:GetInstanceAddedSignal("RoleKiosk"):Connect(bindKiosk)
9406
9407 print("Role kiosks armed: " .. #CollectionService:GetTagged("RoleKiosk") .. ".")
9408 ]==]
9409
9410 local QUIZ_CLIENT_SOURCE = [==[
9411 -- LabQuizClient: a compact in-game console for server-checked math tasks.
9412 local ReplicatedStorage = game:GetService("ReplicatedStorage")
9413 local Players = game:GetService("Players")
9414 local player = Players.LocalPlayer
9415 local remote = ReplicatedStorage:WaitForChild("LabMathQuiz")
9416
9417 local gui = Instance.new("ScreenGui")
9418 gui.Name = "LabMathConsole"
9419 gui.ResetOnSpawn = false
9420 gui.IgnoreGuiInset = true
9421 gui.DisplayOrder = 40
9422 gui.Parent = player:WaitForChild("PlayerGui")
9423
9424 local panel = Instance.new("Frame")
9425 panel.Name = "Console"
9426 panel.AnchorPoint = Vector2.new(0.5, 0.5)
9427 panel.Position = UDim2.fromScale(0.5, 0.46)
9428 panel.Size = UDim2.fromOffset(520, 285)

```

```
9429 panel.BackgroundColor3 = Color3.fromRGB(10, 18, 22)
9430 panel.BackgroundTransparency = 0.04
9431 panel.BorderSizePixel = 0
9432 panel.Visible = false
9433 panel.Parent = gui
9434 local corner = Instance.new("UICorner")
9435 corner.CornerRadius = UDim.new(0, 12)
9436 corner.Parent = panel
9437 local stroke = Instance.new("UIStroke")
9438 stroke.Color = Color3.fromRGB(80, 230, 165)
9439 stroke.Thickness = 3
9440 stroke.Parent = panel
9441
9442 local title = Instance.new("TextLabel")
9443 title.Size = UDim2.new(1, -28, 0, 54)
9444 title.Position = UDim2.fromOffset(14, 10)
9445 title.BackgroundTransparency = 1
9446 title.TextColor3 = Color3.fromRGB(100, 245, 180)
9447 title.TextSize = 23
9448 title.Font = Enum.Font.GothamBold
9449 title.TextWrapped = true
9450 title.Parent = panel
9451
9452 local question = Instance.new("TextLabel")
9453 question.Size = UDim2.new(1, -28, 0, 72)
9454 question.Position = UDim2.fromOffset(14, 66)
9455 question.BackgroundTransparency = 1
9456 question.TextColor3 = Color3.fromRGB(245, 248, 250)
9457 question.TextSize = 38
9458 question.Font = Enum.Font.Code
9459 question.Parent = panel
9460
9461 local answer = Instance.new("TextBox")
9462 answer.Size = UDim2.new(1, -164, 0, 58)
9463 answer.Position = UDim2.fromOffset(18, 146)
9464 answer.BackgroundColor3 = Color3.fromRGB(235, 240, 242)
9465 answer.TextColor3 = Color3.fromRGB(18, 25, 28)
9466 answer.PlaceholderText = "TYPE ANSWER"
9467 answer.ClearTextOnFocus = true
9468 answer.Text = ""
9469 answer.TextSize = 26
9470 answer.Font = Enum.Font.Code
9471 answer.Parent = panel
9472 local answerCorner = Instance.new("UICorner")
9473 answerCorner.CornerRadius = UDim.new(0, 8)
9474 answerCorner.Parent = answer
9475
9476 local submit = Instance.new("TextButton")
9477 submit.Size = UDim2.fromOffset(122, 58)
9478 submit.Position = UDim2.new(1, -140, 0, 146)
9479 submit.BackgroundColor3 = Color3.fromRGB(65, 205, 135)
9480 submit.TextColor3 = Color3.fromRGB(8, 28, 18)
9481 submit.Text = "SUBMIT"
9482 submit.TextSize = 20
9483 submit.Font = Enum.Font.GothamBold
9484 submit.Parent = panel
9485 local submitCorner = Instance.new("UICorner")
9486 submitCorner.CornerRadius = UDim.new(0, 8)
9487 submitCorner.Parent = submit
9488
9489 local countdown = Instance.new("TextLabel")
9490 countdown.Size = UDim2.new(1, -28, 0, 52)
9491 countdown.Position = UDim2.fromOffset(14, 216)
9492 countdown.BackgroundTransparency = 1
9493 countdown.TextColor3 = Color3.fromRGB(255, 200, 75)
9494 countdown.TextSize = 20
9495 countdown.Font = Enum.Font.GothamBold
9496 countdown.Parent = panel
9497
9498 local active = false
9499 remote.OnClientInvoke = function(payload)
9500     if active then return nil end
9501     active = true
9502     local seconds = math.max(10, tonumber(payload.Seconds) or 10)
9503     local endAt = os.clock() + seconds
9504     local finished = false
```



```

9505     local response
9506     title.Text = "LAB MATH CHECK • " .. tostring(payload.Reason or "TASK")
9507     question.Text = tostring(payload.Question or "?") .. " = ?"
9508     answer.Text = ""
9509     panel.Visible = true
9510     answer:CaptureFocus()
9511     local function trySubmit()
9512         local value = tonumber(answer.Text)
9513         if value ~= nil then response = value; finished = true end
9514     end
9515     local buttonConnection = submit.Activated:Connect(trySubmit)
9516     local focusConnection = answer.FocusLost:Connect(function(enterPressed)
9517         if enterPressed then trySubmit() end
9518     end)
9519     while not finished and os.clock() < endAt do
9520         local remaining = math.max(0, math.ceil(endAt - os.clock()))
9521         countdown.Text = "Enter the answer and press SUBMIT • " .. remaining .. " seconds"
9522         task.wait(0.05)
9523     end
9524     buttonConnection:Disconnect()
9525     focusConnection:Disconnect()
9526     panel.Visible = false
9527     active = false
9528     return response
9529 end
9530 ]==]
9531
9532 -- Studio only permits Script.Source writes from trusted authoring contexts.
9533 -- If the generator itself is executed by a normal server Script during a
9534 -- play test, those writes fail. This compact live fallback binds the core
9535 -- gameplay directly so role prompts, held plants, recovery tools and worker
9536 -- tests still function in that exact situation.
9537 local function startLiveRuntimeFallback()
9538     local Players = game:GetService("Players")
9539     local TweenService = game:GetService("TweenService")
9540     local Debris = game:GetService("Debris")
9541     local PathfindingService = game:GetService("PathfindingService")
9542     local liveBound = {}
9543     local liveEmergencyMoves = 0
9544     local liveSampleSerial = 40000
9545     local liveRepairs, liveAssignedRepairs, liveTransfers, liveAssignedTransfers = 0, 0, 0, 0
9546     local LIVE_POINTS = {MOVE_ANY = 12, MOVE_LATE = 18, MOVE_ON_TIME = 30, PROCESS = 4}
9547     local liveAssignTask
9548
9549     local function livePlayerRole(player)
9550         local role = player:GetAttribute("LabRole")
9551         if role and role ~= "" then return role end
9552         return player.Team and player.Team.Name or "Unassigned"
9553     end
9554
9555     local function liveEquipmentId(part)
9556         local code = part:GetAttribute("GrowthCode")
9557         if code and code ~= "" then return "CHAMBER:" .. code end
9558         return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
9559             ⇐ part.Name}, "|")
9559     end
9560
9561     local function liveFloor(part)
9562         return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
9563     end
9564
9565     local function liveTransferSeconds(source, destination)
9566         local distance = destination and (destination.Position - source.Position).Magnitude or 60
9567         local playerCount = math.max(1, #Players:GetPlayers())
9568         local complex = workspace:FindFirstChild("Laboratory_Complex")
9569         local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
9570         return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
9571             + math.clamp(backlog / 20, 0, 25)), 60, 180)
9572     end
9573
9574     local function liveEquipmentLabel(part)
9575         local code = part:GetAttribute("GrowthCode")
9576         if code and code ~= "" then return "CHAMBER " .. code end
9577         return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
9578     end
9579

```

```

9580     local function refreshLiveScoreboard()
9581         local rivalScore = 0
9582         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
9583             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
9584         end
9585         for _, board in ipairs(CollectionService:GetTagged("BatchScoreboard")) do
9586             local label = board:FindFirstChild("Text", true)
9587             if label and label:IsA("TextLabel") then
9588                 label.Text = string.format("PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d •
↳ assigned: %d\nRival NPC score: %d",
liveRepairs, liveAssignedRepairs, liveTransfers, liveAssignedTransfers, rivalScore)
9589             end
9590         end
9591     end
9592 end
9593
9594 local function refreshLiveRoomMonitors()
9595     -- Room names repeat on every level, so key by level+room, not room alone.
9596     local totals = {}
9597     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
9598         local room = equip:GetAttribute("Room")
9599         local level = equip:GetAttribute("Level") or ""
9600         if room then
9601             local key = level .. "|" .. room
9602             local t = totals[key]
9603             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
9604             t.count = t.count + 1
9605             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
9606             else t.running = t.running + 1 end
9607             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
9608         end
9609     end
9610     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
9611         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
9612         local label = surface and surface:FindFirstChild("Status")
9613         if label then
9614             local room = panel:GetAttribute("Room") or "ROOM"
9615             local level = panel:GetAttribute("Level") or ""
9616             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
9617             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
room, t.count, t.running, t.failed, t.plants)
9618             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235,
↳ 170)
9619         end
9620     end
9621 end
9622 end
9623
9624 local function liveAward(player, amount, reason)
9625     local stats = player:FindFirstChild("leaderstats")
9626     if not stats then stats = Instance.new("Folder"); stats.Name = "leaderstats"; stats.Parent = player end
9627     local points = stats:FindFirstChild("Points")
9628     if not points then points = Instance.new("IntValue"); points.Name = "Points"; points.Parent = stats end
9629     points.Value = points.Value + amount
9630     player:SetAttribute("LastPointAmount", amount)
9631     player:SetAttribute("LastPointAward", reason)
9632     refreshLiveScoreboard()
9633     return points.Value
9634 end
9635
9636 -- Math-quiz gate removed by request: pass straight through.
9637 local function runLiveMathQuiz(player, reason)
9638     return true
9639 end
9640 local plantProcesses = {
9641     ["Germination Chamber"] = {name = "Germination", gain = 1},
9642     Incubator = {name = "Incubation", gain = 1},
9643     ["Growth Hormone Mixer"] = {name = "Growth Hormone", gain = 2},
9644     ["GMO Injector"] = {name = "GMO Injection", gain = 1},
9645     ["Nutrient Infuser"] = {name = "Nutrient Infusion", gain = 1},
9646     ["UV Growth Scanner"] = {name = "UV Scan", gain = 0},
9647     ["Pollination Station"] = {name = "Pollination", gain = 1},
9648 }
9649 local function findPlant(player)
9650     for _, container in ipairs({player.Character, player:FindFirstChildOfClass("Backpack")}) do
9651         if container then
9652             for _, child in ipairs(container:GetChildren()) do
9653                 if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end

```

```

9654         end
9655     end
9656 end
9657 end
9658
9659 local function liveChamberSamples(chamber)
9660     local samples = {}
9661     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do
9662         table.insert(samples, id)
9663     end
9664     return samples
9665 end
9666
9667 local function saveLiveChamber(chamber, samples)
9668     local capacity = chamber:GetAttribute("Capacity") or 10
9669     while #samples > capacity do table.remove(samples) end
9670     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
9671     chamber:SetAttribute("SampleCount", #samples)
9672     chamber:SetAttribute("Occupied", #samples > 0)
9673     chamber:SetAttribute("PlantId", samples[1] or "")
9674 end
9675
9676 local function equip(player, tool)
9677     local humanoid = player.Character and player.Character:FindFirstChildOfClass("Humanoid")
9678     if humanoid and tool then humanoid:EquipTool(tool) end
9679 end
9680
9681 local function weldPiece(tool, handle, name, size, offset, colour)
9682     local piece = Instance.new("Part")
9683     piece.Name = name
9684     piece.Size = size
9685     piece.CFrame = handle.CFrame * CFrame.new(offset)
9686     piece.Color = colour
9687     piece.Material = Enum.Material.SmoothPlastic
9688     piece.CanCollide = false
9689     piece.Massless = true
9690     piece.Parent = tool
9691     local weld = Instance.new("WeldConstraint")
9692     weld.Part0 = handle
9693     weld.Part1 = piece
9694     weld.Parent = piece
9695     return piece
9696 end
9697
9698 local function makePlantTool(player, chamber, requestedPlantId, destinationCode)
9699     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 3)
9700     if not backpack then return nil end
9701     local code = chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber)
9702     local plantId = requestedPlantId or chamber:GetAttribute("PlantId") or code
9703     local tool = Instance.new("Tool")
9704     tool.Name = "Plant Sample " .. plantId
9705     tool.ToolTip = "Visible germinated plant from " .. code
9706     tool.RequiresHandle = true
9707     tool.CanBeDropped = false
9708     tool:SetAttribute("LabSample", true)
9709     tool:SetAttribute("PlantSample", true)
9710     tool:SetAttribute("PlantId", plantId)
9711     tool:SetAttribute("SampleId", plantId)
9712     tool:SetAttribute("OriginGrowthCode", code)
9713     tool:SetAttribute("SourceGrowthCode", code)
9714     tool:SetAttribute("DestinationGrowthCode", destinationCode or "")
9715     tool:SetAttribute("TransferStartedAt", os.time())
9716     tool:SetAttribute("GrowthStage", chamber:GetAttribute("GrowthStage") or 0)
9717     tool:SetAttribute("Treatments", chamber:GetAttribute("Treatments") or "")
9718     tool:SetAttribute("Genome", chamber:GetAttribute("Genome") or "Wild Type")
9719     tool:SetAttribute("Moisture", chamber:GetAttribute("Moisture") or 50)
9720     local plantLife = chamber:GetAttribute("PlantLife") or 100
9721     if chamber:GetAttribute("FunctionalState") == "FAILED" then
9722         plantLife = math.clamp(100 - math.floor((os.time() - (chamber:GetAttribute("FailureSince") or
9723             ↪ os.time())) * 1.5), 0, 100)
9724     end
9725     tool:SetAttribute("PlantLife", plantLife)
9726     local handle = Instance.new("Part")
9727     handle.Name = "Handle"
9728     handle.Size = Vector3.new(1.05, 0.8, 1.05)
9729     handle.Color = Color3.fromRGB(116, 75, 45)

```

```

9729     handle.Material = Enum.Material.SmoothPlastic
9730     handle.CanCollide = false
9731     handle.Massless = true
9732     handle.Parent = tool
9733     weldPiece(tool, handle, "PlantStem", Vector3.new(0.15, 1.25, 0.15), Vector3.new(0, 0.9, 0),
↪     Color3.fromRGB(58, 160, 70))
9734     weldPiece(tool, handle, "LeafL", Vector3.new(0.7, 0.15, 0.4), Vector3.new(-0.32, 1.15, 0),
↪     Color3.fromRGB(73, 185, 84))
9735     weldPiece(tool, handle, "LeafR", Vector3.new(0.7, 0.15, 0.4), Vector3.new(0.32, 1.42, 0),
↪     Color3.fromRGB(73, 185, 84))
9736     local lifeGui = Instance.new("BillboardGui")
9737     lifeGui.Name = "PlantLifeStatus"
9738     lifeGui.Size = UDim2.fromOffset(160, 46)
9739     lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
9740     lifeGui.MaxDistance = 18
9741     lifeGui.AlwaysOnTop = false
9742     lifeGui.Parent = handle
9743     local lifeFrame = Instance.new("Frame")
9744     lifeFrame.Size = UDim2.fromScale(1, 1)
9745     lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
9746     lifeFrame.BackgroundTransparency = 0.12
9747     lifeFrame.BorderSizePixel = 0
9748     lifeFrame.Parent = lifeGui
9749     local lifeText = Instance.new("TextLabel")
9750     lifeText.Size = UDim2.new(1, 0, 0.58, 0)
9751     lifeText.BackgroundTransparency = 1
9752     lifeText.TextScaled = true
9753     lifeText.Font = Enum.Font.GothamBold
9754     lifeText.Parent = lifeFrame
9755     local lifeBack = Instance.new("Frame")
9756     lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
9757     lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
9758     lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
9759     lifeBack.BorderSizePixel = 0
9760     lifeBack.Parent = lifeFrame
9761     local lifeFill = Instance.new("Frame")
9762     lifeFill.Name = "LifeBarFill"
9763     lifeFill.BorderSizePixel = 0
9764     lifeFill.Parent = lifeBack
9765     local function updateLife()
9766         local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
9767         local colour = life > 60 and Color3.fromRGB(75, 235, 105)
9768             or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
9769         lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
9770         lifeText.TextColor3 = colour
9771         lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
9772         lifeFill.BackgroundColor3 = colour
9773     end
9774     tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLife)
9775     updateLife()
9776     tool.Grip = CFrame.new(0, -0.65, -0.25) * CFrame.Angles(math.rad(-10), 0, 0)
9777     tool.Parent = backpack
9778     equip(player, tool)
9779     task.spawn(function()
9780         while tool.Parent and (tool:GetAttribute("PlantLife") or 0) > 0 do
9781             task.wait(1)
9782             if not tool.Parent then return end
9783             if not tool:GetAttribute("QuizProtected") then
9784                 local life = math.max(0, (tool:GetAttribute("PlantLife") or 0) - 2)
9785                 tool:SetAttribute("PlantLife", life)
9786                 if life <= 0 then
9787                     local reservedCode = tool:GetAttribute("DestinationGrowthCode") or ""
9788                     for _, candidate in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
9789                         if candidate:GetAttribute("GrowthCode") == reservedCode then
9790                             candidate:SetAttribute("IncomingReservations", math.max(0,
↪                             (candidate:GetAttribute("IncomingReservations") or 1) - 1))
9791                             break
9792                         end
9793                     end
9794                     player:SetAttribute("CarriedSample", "")
9795                     player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
↪                     another plant.")
9796                     task.wait(0.5)
9797                     if tool.Parent then tool:Destroy() end
9798                     return
9799                 end
9800             end
9801         end
9802     end)

```

```

9800         end
9801     end
9802 end)
9803     return tool
9804 end
9805
9806 local function showChamberPlant(chamber, visible)
9807     local count = visible and (chamber:GetAttribute("SampleCount") or 0) or 0
9808     for _, child in ipairs(chamber:GetDescendants()) do
9809         if child:IsA("BasePart") and (child:GetAttribute("SampleSlot") or
9810             ↪ child.Name:match("^ChamberPlant")) then
9811             local slot = child:GetAttribute("SampleSlot") or 1
9812             child.Transparency = slot <= count and (child:GetAttribute("StoredTransparency") or 0) or 1
9813         end
9814     end
9815
9816 local function liveRuntimeText(seconds)
9817     seconds = math.max(0, math.floor(seconds or 0))
9818     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
9819         ↪ seconds % 60)
9820 end
9821
9822 local function recoverLivePlantLife(chamber, startingLife, player, plantId)
9823     startingLife = math.clamp(startingLife or 100, 0, 100)
9824     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then
9825         chamber:SetAttribute("PlantLife", startingLife)
9826         return 0
9827     end
9828     local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
9829     local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
9830     chamber:SetAttribute("RecoveryToken", token)
9831     chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
9832     chamber:SetAttribute("PlantLife", startingLife)
9833     if player then
9834         player:SetAttribute("LabObjective", string.format("%s dropped in %s. LIFE %d% → 100% in %d
9835             ↪ seconds.",
9836                 plantId or "Plant", chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber),
9837                 ↪ startingLife, seconds))
9838     end
9839     task.spawn(function()
9840         for step = 1, seconds do
9841             task.wait(1)
9842             if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
9843                 or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
9844             chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step /
9845                 ↪ seconds + 0.5))
9846         end
9847         chamber:SetAttribute("PlantLife", 100)
9848         chamber:SetAttribute("RecoveryEndsAt", 0)
9849         if player and player.Parent then player:SetAttribute("LabObjective", (plantId or "Plant") .. "
9850             ↪ recovered to 100% LIFE.") end
9851     end)
9852     return seconds
9853 end
9854
9855 local function refreshLiveTelemetry(chamber)
9856     local samples = liveChamberSamples(chamber)
9857     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
9858     local code = chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber)
9859     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN
9860     ↪ FLOOR"
9861     local destination = chamber:GetAttribute("EvacuationDestination") or ""
9862     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "FAULT")
9863         .. (destination ~= "" and (" • EVACUATE TO " .. destination) or "")) or "SYSTEM NORMAL"
9864     local text = string.format(
9865         "%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM %d% • VENT
9866         ↪ %d%\n%s\n%s",
9867         floorLabel, code, state, #samples,
9868         chamber:GetAttribute("Capacity") or 0,
9869         chamber:GetAttribute("GerminationDate") or "pending",
9870         liveRuntimeText(os.time() - (chamber:GetAttribute("OperationStarted") or os.time())),
9871         chamber:GetAttribute("Temperature") or 23, chamber:GetAttribute("Humidity") or 72,
9872         chamber:GetAttribute("Ventilation") or 75, detail,
9873         state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
9874     for _, child in ipairs(chamber:GetDescendants()) do

```

```

9868     if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
9869         child.Text = text
9870         child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
9871             or state == "FAILED" and Color3.fromRGB(255, 120, 105)
9872             or Color3.fromRGB(130, 245, 165)
9873     elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
9874         child.Text = "PLANTS (" .. #samples .. "): "
9875             .. (#samples > 0 and table.concat(samples, ", ") or "NO SAMPLES")
9876     elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
9877         child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)
9878             or state == "FAILED" and Color3.fromRGB(230, 55, 45)
9879             or Color3.fromRGB(40, 165, 95)
9880         child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
9881     end
9882 end
9883 end
9884
9885 local function liveDestinations(source)
9886     local choices = {}
9887     for _, candidate in ipairs(CollectionService.GetTagged("PlantGrowthChamber")) do
9888         local count = candidate:GetAttribute("SampleCount") or 0
9889         local capacity = candidate:GetAttribute("Capacity") or 10
9890         local reservations = candidate:GetAttribute("IncomingReservations") or 0
9891         if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED"
9892             and count + reservations < capacity then
9893             table.insert(choices, candidate)
9894         end
9895     end
9896     table.sort(choices, function(a, b)
9897         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9898         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9899         if sameA ~= sameB then return sameA < sameB end
9900         local ac = a:GetAttribute("SampleCount") or 0
9901         local bc = b:GetAttribute("SampleCount") or 0
9902         if ac ~= bc then return ac < bc end
9903         return (a:GetAttribute("GrowthCode") or "") < (b:GetAttribute("GrowthCode") or "")
9904     end)
9905     return choices
9906 end
9907
9908 local function liveRepairPathFolder(player, create)
9909     local complex = workspace:FindFirstChild("Laboratory_Complex")
9910     if not complex then return nil end
9911     local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
9912     if create and not rootFolder then
9913         rootFolder = Instance.new("Folder")
9914         rootFolder.Name = "TechnicianRepairPaths"
9915         rootFolder.Parent = complex
9916     end
9917     local folder = rootFolder and rootFolder:FindFirstChild("Player_" .. player.UserId)
9918     if create and not folder then
9919         folder = Instance.new("Folder")
9920         folder.Name = "Player_" .. player.UserId
9921         folder.Parent = rootFolder
9922     end
9923     return folder
9924 end
9925
9926 local function clearLiveRepairPath(player)
9927     local folder = liveRepairPathFolder(player, false)
9928     if folder then folder:Destroy() end
9929 end
9930
9931 local function liveRepairReward(player, equipment, assigned)
9932     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
9933     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
9934     return math.clamp(8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0), 8,
9935         ↪ 120), waited, distance
9936
9937 end
9938
9939 local function createLiveRepairPath(player, target)
9940     clearLiveRepairPath(player)
9941     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
9942     if not characterRoot or not target then return 0, false end
9943     local folder = liveRepairPathFolder(player, true)
9944     if not folder then return 0, false end

```

```

9943     local function approachPoint(part)
9944         local side = part:GetAttribute("WallSide")
9945         local floorY = part.Position.Y - part.Size.Y / 2
9946         if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
↪ part.Position.Z) end
9947         if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
↪ part.Position.Z) end
9948         if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
↪ 2.5) end
9949         return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
9950     end
9951     local function appendNavigable(points, fromPosition, toPosition)
9952         local path = PathfindingService:CreatePath({
9953             AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
9954             AgentCanClimb = true, WaypointSpacing = 4,
9955         })
9956         local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
9957         if not computed or path.Status ~= Enum.PathStatus.Success then return false end
9958         local waypoints = path:GetWaypoints()
9959         if #waypoints < 2 then return false end
9960         for index, waypoint in ipairs(waypoints) do
9961             if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
9962         end
9963         return true
9964     end
9965
9966     local complex = workspace:FindFirstChild("Laboratory_Complex")
9967     local rise = complex and complex:GetAttribute("FloorRiseStuds")
9968     local westX = complex and complex:GetAttribute("StairWestLaneX")
9969     local eastX = complex and complex:GetAttribute("StairEastLaneX")
9970     local northZ = complex and complex:GetAttribute("StairNorthZ")
9971     local southZ = complex and complex:GetAttribute("StairSouthZ")
9972     local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
9973     local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
9974     local destination = approachPoint(target)
9975     local points = {}
9976     local routeReady = false
9977     if rise and westX and eastX and northZ and southZ then
9978         local startFloor = math.clamp(math.floor((characterRoot.Position.Y - 2.5) / rise + 0.5), floorMin,
↪ floorMax)
9979         local targetFloor = math.clamp(math.floor(destination.Y / rise + 0.5), floorMin, floorMax)
9980         if startFloor == targetFloor then
9981             routeReady = appendNavigable(points, characterRoot.Position, destination)
9982         else
9983             local direction = targetFloor > startFloor and 1 or -1
9984             local firstLaneX = direction > 0 and westX or eastX
9985             local stairEntry = Vector3.new(firstLaneX, startFloor * rise, northZ)
9986             routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
9987             local floor = startFloor
9988             while routeReady and floor ~= targetFloor do
9989                 local baseY = floor * rise
9990                 if direction > 0 then
9991                     table.insert(points, Vector3.new(westX, baseY + rise / 2, southZ))
9992                     table.insert(points, Vector3.new(eastX, baseY + rise / 2, southZ))
9993                     table.insert(points, Vector3.new(eastX, baseY + rise, northZ))
9994                 else
9995                     table.insert(points, Vector3.new(eastX, baseY - rise / 2, southZ))
9996                     table.insert(points, Vector3.new(westX, baseY - rise / 2, southZ))
9997                     table.insert(points, Vector3.new(westX, baseY - rise, northZ))
9998                 end
9999                 floor = floor + direction
10000             if floor ~= targetFloor then
10001                 table.insert(points, Vector3.new(direction > 0 and westX or eastX, floor * rise,
↪ northZ))
10002             end
10003         end
10004         if routeReady then routeReady = appendNavigable(points, points[#points], destination) end
10005     end
10006 end
10007 if not routeReady then
10008     folder:Destroy()
10009     player:SetAttribute("RepairRouteReady", false)
10010     return (destination - characterRoot.Position).Magnitude, false
10011 end
10012 player:SetAttribute("RepairRouteReady", true)
10013 local distance, markerCount = 0, 0

```

```

10014     for pointIndex = 1, #points - 1 do
10015         local a, b = points[pointIndex], points[pointIndex + 1]
10016         local delta = b - a
10017         local segmentDistance = delta.Magnitude
10018         distance = distance + segmentDistance
10019         if segmentDistance > 0.1 then
10020             local direction = delta.Unit
10021             local count = math.max(1, math.ceil(segmentDistance / 4))
10022             for index = 1, count do
10023                 markerCount = markerCount + 1
10024                 if markerCount > 120 then break end
10025                 local marker = Instance.new("Part")
10026                 marker.Name = "RedRepairPath"
10027                 marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
10028                 local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
10029                 marker.CFrame = CFrame.lookAt(position, position + direction)
10030                 marker.Color = Color3.fromRGB(255, 32, 28)
10031                 marker.Material = Enum.Material.Neon
10032                 marker.Transparency = 0.12
10033                 marker.Anchored = true
10034                 marker.CanCollide = false
10035                 marker.CanTouch = false
10036                 marker.CanQuery = false
10037                 marker.Parent = folder
10038             end
10039         end
10040         if markerCount > 120 then break end
10041     end
10042     return distance, true
10043 end
10044
10045 liveAssignTask = function(player)
10046     clearLiveRepairPath(player)
10047     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
10048 ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
10049         player:SetAttribute(name, nil)
10050     end
10051     if livePlayerRole(player) == "Technician" then
10052         local failed = {}
10053         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
10054             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment)
10055             ↪ end
10056         end
10057         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
10058         table.sort(failed, function(a, b)
10059             if root then
10060                 local ad = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
10061 ↪ root.Position.Y) > 12 and 500 or 0)
10062                 local bd = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
10063 ↪ root.Position.Y) > 12 and 500 or 0)
10064                 if ad ~= bd then return ad < bd end
10065             end
10066             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
10067         end)
10068         local target = failed[1]
10069         local text = "All equipment is working. Repair score grows with failure time; assigned jobs also
10070 ↪ pay for travel distance."
10071         if target then
10072             player:SetAttribute("AssignedRepairId", liveEquipmentId(target))
10073             local distance, routeReady = createLiveRepairPath(player, target)
10074             player:SetAttribute("AssignedRepairDistance", distance)
10075             local projected = select(1, liveRepairReward(player, target, true))
10076             player:SetAttribute("AssignedRepairProjectedPoints", projected)
10077             local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR;
10078 ↪ route is recalculating"
10079             text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
10080 ↪ while failed).",
10081                 liveFloor(target), liveEquipmentLabel(target), routeInstruction, projected,
10082                 ↪ math.floor(distance + 0.5))
10083         end
10084         player:SetAttribute("LabObjective", text)
10085         return text
10086     end
10087     local jobs = {}
10088     for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do

```



```

10081         if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0)
10082             ↪ > 0
10083             and not source:GetAttribute("ReservedByBot") then
10084                 local destination = liveDestinations[source][1]
10085                 if destination then table.insert(jobs, {source = source, destination = destination}) end
10086             end
10087         end
10088         table.sort(jobs, function(a, b)
10089             return (tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
10090                 < (tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
10091         end)
10092         local job = jobs[1]
10093         if not job then
10094             local text = "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
10095                 ↪ +30."
10096             player:SetAttribute("LabObjective", text)
10097             return text
10098         end
10099         local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
10100         local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
10101         player:SetAttribute("AssignedTransferSource", sourceCode)
10102         player:SetAttribute("AssignedTransferDestination", destinationCode)
10103         local allowedSeconds = liveTransferSeconds(job.source, job.destination)
10104         player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
10105         -- Same red floor-marker route the Technician role gets, pointed at
10106         -- the destination chamber so it's obvious where the plant goes.
10107         local routeReady = select(2, createLiveRepairPath(player, job.destination))
10108         local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
10109             ↪ recalculating"
10110         local text = "ASSIGNED +30: " .. liveFloor(job.source) .. " • " .. sourceCode .. " → " ..
10111             ↪ destinationCode
10112             .. " • " .. liveFloor(job.destination) .. ". PRESS E; " .. routeInstruction .. ". Deliver in " ..
10113             ↪ allowedSeconds .. " seconds."
10114         player:SetAttribute("LabObjective", text)
10115         return text
10116     end
10117
10118     local setLiveFailure
10119     local setLiveGenericFailure
10120
10121     local function controlCFrame(part, yOffset, standOff)
10122         local side = part:GetAttribute("WallSide")
10123         if side == "W" then return part.CFrame * CFrame.new(part.Size.X / 2 + standOff, yOffset, 0), true end
10124         if side == "E" then return part.CFrame * CFrame.new(-part.Size.X / 2 - standOff, yOffset, 0), true end
10125         if side == "S" then return part.CFrame * CFrame.new(0, yOffset, -part.Size.Z / 2 - standOff), false end
10126         return part.CFrame * CFrame.new(0, yOffset, part.Size.Z / 2 + standOff), false
10127     end
10128
10129     local function bindLiveEquipment(part)
10130         if liveBound[part] or not part:IsA("BasePart") then return end
10131         liveBound[part] = true
10132         local kind = part:GetAttribute("Kind") or part.Name
10133         local isGrowthChamber = kind == "Plant Growth Chamber"
10134         local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
10135         part:SetAttribute("EquipmentId", liveEquipmentId(part))
10136         part:SetAttribute("FloorLabel", liveFloor(part))
10137         part:SetAttribute("RepairKey", "R")
10138         if part:GetAttribute("FunctionalState") == nil then
10139             local hash = 0
10140             for i = 1, #liveEquipmentId(part) do hash = hash + string.byte(liveEquipmentId(part), i) end
10141             local failed = not isGrowthChamber and hash % 7 == 0
10142             part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
10143             part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
10144             part:SetAttribute("FailureSince", failed and os.time() or 0)
10145         end
10146         if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time())
10147             ↪ end
10148         local display = part
10149         local readout = {Text = "", TextColor3 = Color3.fromRGB(115, 235, 165)}
10150         local timerGui = Instance.new("BillboardGui")
10151         timerGui.Name = "EquipmentStatusTimer"
10152         timerGui.Size = UDim2.fromOffset(160, 32)
10153         timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
10154         timerGui.MaxDistance = 10
10155         timerGui.AlwaysOnTop = false
10156         timerGui.Parent = part

```

```

10151     local timerLabel = Instance.new("TextLabel")
10152     timerLabel.Name = "Timer"
10153     timerLabel.Size = UDim2.fromScale(1, 1)
10154     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
10155     timerLabel.BackgroundTransparency = 0.16
10156     timerLabel.BorderSizePixel = 0
10157     timerLabel.TextScaled = true
10158     timerLabel.Font = Enum.Font.GothamBold
10159     timerLabel.Parent = timerGui
10160     local timerCorner = Instance.new("UICorner")
10161     timerCorner.CornerRadius = UDim.new(0, 8)
10162     timerCorner.Parent = timerLabel
10163
10164     local nameGui = Instance.new("BillboardGui")
10165     nameGui.Name = "EquipmentNamePlate"
10166     nameGui.Size = UDim2.fromOffset(160, 24)
10167     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
10168     nameGui.MaxDistance = 10
10169     nameGui.AlwaysOnTop = false
10170     nameGui.Parent = part
10171     local nameLabel = Instance.new("TextLabel")
10172     nameLabel.Name = "Kind"
10173     nameLabel.Size = UDim2.fromScale(1, 1)
10174     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
10175     nameLabel.BackgroundTransparency = 0.05
10176     nameLabel.BorderSizePixel = 0
10177     nameLabel.Text = kind
10178     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
10179     nameLabel.TextScaled = true
10180     nameLabel.Font = Enum.Font.SourceSansBold
10181     nameLabel.Parent = nameGui
10182     local nameCorner = Instance.new("UICorner")
10183     nameCorner.CornerRadius = UDim.new(0, 8)
10184     nameCorner.Parent = nameLabel
10185
10186     local slotGui = Instance.new("BillboardGui")
10187     slotGui.Name = "EquipmentFreeSlots"
10188     slotGui.Size = UDim2.fromOffset(150, 28)
10189     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
10190     slotGui.MaxDistance = 10
10191     slotGui.AlwaysOnTop = false
10192     slotGui.Parent = part
10193     local slotLabel = Instance.new("TextLabel")
10194     slotLabel.Name = "FreeSlots"
10195     slotLabel.Size = UDim2.fromScale(1, 1)
10196     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)
10197     slotLabel.BackgroundTransparency = 0.12
10198     slotLabel.BorderSizePixel = 0
10199     slotLabel.TextScaled = true
10200     slotLabel.Font = Enum.Font.GothamBold
10201     slotLabel.Parent = slotGui
10202     local slotCorner = Instance.new("UICorner")
10203     slotCorner.CornerRadius = UDim.new(0, 8)
10204     slotCorner.Parent = slotLabel
10205     local storedLifeGui, storedLifeText, storedLifeFill
10206     if isPlantContainer then
10207         local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
10208         storedLifeGui = Instance.new("BillboardGui")
10209         storedLifeGui.Name = "StoredPlantLifeStatus"
10210         storedLifeGui.Size = UDim2.fromOffset(140, 40)
10211         storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
10212         storedLifeGui.MaxDistance = 10
10213         storedLifeGui.AlwaysOnTop = false
10214         storedLifeGui.Parent = plantAnchor
10215         storedLifeText = Instance.new("TextLabel")
10216         storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
10217         storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
10218         storedLifeText.BackgroundTransparency = 0.16
10219         storedLifeText.BorderSizePixel = 0
10220         storedLifeText.TextScaled = true
10221         storedLifeText.Font = Enum.Font.GothamBold
10222         storedLifeText.Parent = storedLifeGui
10223         local storedLifeBack = Instance.new("Frame")
10224         storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
10225         storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
10226         storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)

```

```

10227     storedLifeBack.BorderSizePixel = 0
10228     storedLifeBack.Parent = storedLifeGui
10229     storedLifeFill = Instance.new("Frame")
10230     storedLifeFill.Size = UDim2.fromScale(1, 1)
10231     storedLifeFill.BorderSizePixel = 0
10232     storedLifeFill.Parent = storedLifeBack
10233 end
10234 local function liveClockText(seconds)
10235     seconds = math.max(0, math.floor(seconds or 0))
10236     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
        ↪ seconds % 60)
10237 end
10238 local function renderLiveTimer()
10239     local state = part:GetAttribute("FunctionalState")
10240     local destroyed = state == "DESTROYED"
10241     local failed = destroyed or state == "FAILED"
10242     local since = failed and (part:GetAttribute("FailureSince") or os.time())
        or (part:GetAttribute("OperationStarted") or os.time())
10243     if destroyed then
10244         timerLabel.Text = "DESTROYED -- UNFIXABLE"
10245         timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
10246     else
10247         timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. liveClockText(os.time() - since)
10248         timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)
10249     end
10250     if isPlantContainer and failed then
10251         local life = math.clamp(100 - math.floor((os.time() - since) * 1.5), 0, 100)
10252         part:SetAttribute("PlantLife", life)
10253         if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
10254             part:SetAttribute("SampleIds", "")
10255             part:SetAttribute("SampleCount", 0)
10256             part:SetAttribute("Occupied", false)
10257             part:SetAttribute("PlantId", "")
10258             part:SetAttribute("PlantExpiredAt", os.time())
10259             showChamberPlant(part, false)
10260         end
10261     end
10262     local capacity = part:GetAttribute("Capacity") or 0
10263     local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
10264     local free = math.max(0, capacity - count)
10265     slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
        or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
10266     slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)
10267     local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
10268     if incoming and not failed then
10269         slotGui.MaxDistance = 30
10270         slotGui.AlwaysOnTop = false
10271         slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
10272         slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
10273     else
10274         slotGui.MaxDistance = 10
10275         slotGui.AlwaysOnTop = false
10276     end
10277     if storedLifeGui then
10278         local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
10279         local colour = life > 60 and Color3.fromRGB(75, 235, 105)
            or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
10280         storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
10281         storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
10282         storedLifeText.TextColor3 = colour
10283         storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
10284         storedLifeFill.BackgroundColor3 = colour
10285     end
10286 end
10287 renderLiveTimer()
10288 task.spawn(function()
10289     while part.Parent do renderLiveTimer(); task.wait(1) end
10290 end)
10291 local button = Instance.new("Part")
10292 button.Name = "PushButton"
10293 button.Shape = Enum.PartType.Ball
10294 button.Size = isGrowthChamber and Vector3.new(0.62, 0.62, 0.62) or Vector3.new(0.42, 0.42, 0.42)
10295 button.CFrame = select(1, controlCFrame(part, 0, 0.22))
10296 button.Color = Color3.fromRGB(220, 65, 55)
10297 button.Material = Enum.Material.Neon

```

```

10302     button.Anchored = true
10303     button.CanCollide = false
10304     button.Parent = part
10305     local usePrompt = Instance.new("ProximityPrompt")
10306     usePrompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
10307     usePrompt.ObjectText = isPlantContainer
10308         and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " " * " " ..
10309             ⇨ liveEquipmentLabel(part))
10309         or kind
10310     usePrompt.KeyboardKeyCode = Enum.KeyCode.E
10311     usePrompt.MaxActivationDistance = 8
10312     usePrompt.RequiresLineOfSight = true
10313     usePrompt.Parent = button
10314
10315     local led = Instance.new("Part")
10316     led.Name = "StatusLED"
10317     led.Shape = Enum.PartType.Ball
10318     led.Size = Vector3.new(0.28, 0.28, 0.28)
10319     led.CFrame = select(1, controlCFrame(part, -0.55, 0.24))
10320     led.Color = Color3.fromRGB(95, 240, 125)
10321     led.Material = Enum.Material.Neon
10322     led.Anchored = true
10323     led.CanCollide = false
10324     led.Parent = part
10325     local powered = true
10326     part:SetAttribute("Powered", true)
10327     local repairBusy = false
10328
10329     local floorLabel = liveFloor(part)
10330     local unitLabel = liveEquipmentLabel(part)
10331     local repairPrompt = Instance.new("ProximityPrompt")
10332     repairPrompt.Name = "RepairPrompt"
10333     repairPrompt.ActionText = "Hold R to repair"
10334     repairPrompt.ObjectText = floorLabel .. " " * " " .. unitLabel
10335     repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
10336     repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
10337     repairPrompt.HoldDuration = 3
10338     repairPrompt.MaxActivationDistance = 8
10339     repairPrompt.RequiresLineOfSight = true
10340     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
10341     repairPrompt.Parent = part:FindFirstChild("ControllerScreen") or display
10342
10343     part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
10344         if repairPrompt.Parent then repairPrompt.Enabled = not repairBusy and
10345             ⇨ part:GetAttribute("FunctionalState") == "FAILED" end
10346     end)
10347     repairPrompt.Triggered:Connect(function(player)
10348         if livePlayerRole(player) ~= "Technician" then
10349             readout.Text = "TECHNICIAN REQUIRED\nChoose the BLUE role kiosk"
10350             return
10351         end
10352         if repairBusy or part:GetAttribute("FunctionalState") ~= "FAILED" then return end
10353         repairBusy = true
10354         repairPrompt.Enabled = false
10355         readout.Text = "REPAIRING\n" .. floorLabel .. " " * " " .. unitLabel
10356         if not runLiveMathQuiz(player, "REPAIR " .. floorLabel .. " " * " " .. unitLabel) then
10357             readout.Text = "REPAIR NOT COMPLETED\nHold R for 3 seconds to try again"
10358             repairBusy = false
10359             repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
10360             return
10361         end
10362         local assigned = player:GetAttribute("AssignedRepairId") == liveEquipmentId(part)
10363         local points, failedSeconds, routeDistance = liveRepairReward(player, part, assigned)
10364         part:SetAttribute("LastRepairedAt", os.time())
10365         part:SetAttribute("LastRepairedBy", player.Name)
10366         if isGrowthChamber then setLiveFailure(part, false) else setLiveGenericFailure(part, false) end
10367         liveRepairs = liveRepairs + 1
10368         if assigned then liveAssignedRepairs = liveAssignedRepairs + 1 end
10369         local total = liveAward(player, points, string.format("%s%s • failed %ds • route %d studs",
10370             assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
10371             ⇨ math.floor(routeDistance + 0.5)))
10372         readout.Text = "REPAIR COMPLETE +" .. points .. "\n" .. floorLabel .. " " * " " .. unitLabel ..
10373             ⇨ "\nPLANTS "
10374         .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. "
10375             ⇨ • TOTAL " .. total
10376         liveAssignTask(player)

```

```

10373         repairBusy = false
10374     end)
10375
10376     usePrompt.Triggered:Connect(function(player)
10377         button.Color = Color3.fromRGB(255, 205, 65)
10378         task.delay(0.2, function() if button.Parent then button.Color = Color3.fromRGB(220, 65, 55) end
10379         ↪ end)
10379     if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
10380         readout.Text = "FAILED • HOLD R\n" .. liveFloor(part) .. " • " ..
10381         ↪ (part:GetAttribute("FailureReason") or "fault")
10382         return
10383     end
10384     local carriedPlant = findPlant(player)
10385     local plantProcess = plantProcesses[kind]
10386     local shouldUsePlantStorage = isPlantContainer and (
10387         isGrowthChamber
10388         or not plantProcess
10389         or not carriedPlant
10390         or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
10391     )
10392     if shouldUsePlantStorage then
10393         if livePlayerRole(player) ~= "Laboratory Technician" then
10394             readout.Text = "LAB ROLE REQUIRED\nGREEN kiosk moves plants"
10395             return
10396         end
10397         -- Players override test-bot reservations so E always performs the
10398         -- obvious take/drop action when they are standing at the cabinet.
10399         if part:GetAttribute("ReservedByBot") then part:SetAttribute("ReservedByBot", nil) end
10400         local plant = carriedPlant
10401         local code = part:GetAttribute("GrowthCode") or liveEquipmentLabel(part)
10402         local samples = liveChamberSamples(part)
10403         local capacity = part:GetAttribute("Capacity") or 10
10404         local failed = part:GetAttribute("FunctionalState") == "FAILED"
10405         if plant then
10406             local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
10407             if failed then
10408                 readout.Text = code .. " FAILED\nCannot store plants here"
10409                 player:SetAttribute("LabObjective", code .. " is FAILED. Find any GREEN RUNNING cabinet
10410                 ↪ marked FREE SLOT and press E to drop the plant.")
10411             elseif #samples + (part:GetAttribute("IncomingReservations") or 0) >= capacity then
10412                 readout.Text = code .. " FULL\nChoose another working chamber"
10413             else
10414                 local plantId = plant:GetAttribute("PlantId") or plant.Name
10415                 local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
10416                 local sourceCode = plant:GetAttribute("SourceGrowthCode") or
10417                 ↪ plant:GetAttribute("OriginGrowthCode") or ""
10418                 local assigned = wanted ~= "" and player:GetAttribute("AssignedTransferSource") ==
10419                 ↪ sourceCode
10420                 and player:GetAttribute("AssignedTransferDestination") == code
10421                 local onTime = assigned and os.time() <=
10422                 ↪ (player:GetAttribute("AssignedTransferDeadline") or 0)
10423                 plant:SetAttribute("QuizProtected", true)
10424                 readout.Text = "STORING SAMPLE\nPlacing " .. plantId .. " into " .. code
10425                 local passedQuiz = runLiveMathQuiz(player, "REVIVE " .. plantId .. " IN " .. code)
10426                 plant:SetAttribute("QuizProtected", nil)
10427                 if not passedQuiz or not plant.Parent then return end
10428                 table.insert(samples, plantId)
10429                 saveLiveChamber(part, samples)
10430                 part:SetAttribute("GrowthStage", math.max(part:GetAttribute("GrowthStage") or 0,
10431                 ↪ plant:GetAttribute("GrowthStage") or 0))
10432                 part:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
10433                 part:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
10434                 local recoverySeconds = recoverLivePlantLife(part, plantLife, player, plantId)
10435                 if wanted ~= "" then
10436                     for _, reservedDestination in
10437                     ↪ ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
10438                         if reservedDestination:GetAttribute("GrowthCode") == wanted then
10439                             reservedDestination:SetAttribute("IncomingReservations", math.max(0,
10440                             ↪ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
10441                             break
10442                         end
10443                     end
10444                 end
10445                 liveEmergencyMoves = liveEmergencyMoves + 1
10446             end
10447             showChamberPlant(part, true)
10448             refreshLiveTelemetry(part)
10449         end

```

```

10440     player:SetAttribute("CarriedSample", "")
10441     plant:Destroy()
10442     if wanted ~= "" then
10443         liveTransfers = liveTransfers + 1
10444         if assigned then liveAssignedTransfers = liveAssignedTransfers + 1 end
10445         local points = assigned and (onTime and LIVE_POINTS.MOVE_ON_TIME or
            ↪ LIVE_POINTS.MOVE_LATE) or LIVE_POINTS.MOVE_ANY
10446         local total = liveAward(player, points, assigned and (onTime and "assigned on-time
            ↪ plant move" or "assigned late plant move") or "emergency plant move")
10447         readout.Text = string.format("%s STORED +%d\n%s • %d/%d • TOTAL %d", code, points,
            ↪ plantId, #samples, capacity, total)
10448         if recoverySeconds > 0 then
10449             task.delay(recoverySeconds, function() if player.Parent then
                ↪ liveAssignTask(player) end end)
10450         else
10451             liveAssignTask(player)
10452         end
10453     else
10454         readout.Text = string.format("%s STORED\n%s • %d/%d samples", code, plantId,
            ↪ #samples, capacity)
10455         player:SetAttribute("LabObjective", "Take the plant to growth equipment or another
            ↪ chamber.")
10456     end
10457 end
10458 elseif #samples > 0 then
10459     local destination
10460     if failed and player:GetAttribute("AssignedTransferSource") == code then
10461         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
10462         for _, candidate in ipairs(liveDestinations(part)) do
10463             if candidate:GetAttribute("GrowthCode") == assignedDestination then destination =
                ↪ candidate break end
10464         end
10465     end
10466     if failed then destination = destination or liveDestinations(part)[1] end
10467     if failed and not destination then
10468         readout.Text = code .. " FAILED\nNo working chamber has space"
10469         player:SetAttribute("LabObjective", "Working chambers are full. Wait for NPC transfers
            ↪ or an automatic recovery.")
10470         return
10471     end
10472     if failed and destination then createLiveRepairPath(player, destination) end
10473     local plantId = samples[1]
10474     local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
10475     readout.Text = "TAKING SAMPLE\n" .. plantId .. " from " .. code
10476     if not runLiveMathQuiz(player, "TAKE PLANT " .. plantId .. " FROM " .. code) then return
        ↪ end
10477     table.remove(samples, 1)
10478     local made = makePlantTool(player, part, plantId, destinationCode)
10479     if made then
10480         saveLiveChamber(part, samples)
10481         showChamberPlant(part, true)
10482         if destination then
10483             destination:SetAttribute("IncomingReservations",
                ↪ (destination:GetAttribute("IncomingReservations") or 0) + 1)
10484             part:SetAttribute("EvacuationDestination", destinationCode)
10485             player:SetAttribute("LabObjective", "CARRYING " .. plantId .. ": press E at ANY
                ↪ GREEN RUNNING cabinet marked FREE SLOT. BONUS target: " .. destinationCode ..
                ↪ ".")
10486             readout.Text = code .. " FAILED\nSEND " .. plantId .. " TO " .. destinationCode
10487         else
10488             player:SetAttribute("LabObjective", "Use growth equipment on " .. plantId .. ",
                ↪ then return it to a working chamber.")
10489             readout.Text = string.format("%s SAMPLE OUT\n%s • %d/%d remain", code, plantId,
                ↪ #samples, capacity)
10490         end
10491         player:SetAttribute("CarriedSample", plantId)
10492         refreshLiveTelemetry(part)
10493     else
10494         table.insert(samples, 1, plantId)
10495     end
10496 else
10497     readout.Text = code .. (failed and " FAILED + EMPTY\nAlarm awaiting refill" or "
        ↪ EMPTY\nBring a plant")
10498 end
10499 elseif part:GetAttribute("FunctionalState") == "FAILED" then

```

```

10500         readout.Text = "FAILED • HOLD R\n" .. liveFloor(part) .. " • " ..
        ↪ (part:GetAttribute("FailureReason") or "fault")
10501     return
10502 elseif plantProcess then
10503     if livePlayerRole(player) ~= "Laboratory Technician" then
10504         readout.Text = "LAB ROLE REQUIRED\nGREEN kiosk develops plants"
10505         return
10506     end
10507     local plant = findPlant(player)
10508     if not plant then readout.Text = "PLANT REQUIRED\nTake one from an R-numbered chamber"; return
        ↪ end
10509     local process = plantProcess
10510     local stage = math.clamp((plant:GetAttribute("GrowthStage") or 0) + process.gain, 0, 5)
10511     plant:SetAttribute("GrowthStage", stage)
10512     local treatments = plant:GetAttribute("Treatments") or ""
10513     if not string.find(treatments, process.name, 1, true) then
10514         plant:SetAttribute("Treatments", treatments == "" and process.name or (treatments .. ", "
        ↪ .. process.name))
10515     end
10516     local total = liveAward(player, LIVE_POINTS.PROCESS, process.name .. " plant treatment")
10517     readout.Text = kind .. "\nSTATUS WORKING • PLANTS " .. (part:GetAttribute("SampleCount") or 0)
        ↪ .. "/"
        .. (part:GetAttribute("Capacity") or 0) .. "\nPLANT DATA +" .. LIVE_POINTS.PROCESS .. " • "
10518     .. (plant:GetAttribute("PlantId") or "Plant") .. " • stage " .. stage .. " • TOTAL " ..
10519     ↪ total
10520     equip(player, plant)
10521 else
10522     part:SetAttribute("Inspections", (part:GetAttribute("Inspections") or 0) + 1)
10523     readout.Text = string.format("%s • %s\nSTATUS WORKING • POWER ON\nPLANTS %d/%d\nInspection
        ↪ normal",
10524         liveFloor(part), kind, part:GetAttribute("SampleCount") or 0, part:GetAttribute("Capacity")
        ↪ or 0)
10525     end
10526 end)
10527 end
10528
10529 local roleColours = {
10530     Technician = Color3.fromRGB(60, 120, 200),
10531     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),
10532 }
10533 local function dressCharacter(character, role)
10534     for _, old in ipairs(character:GetChildren()) do
10535         if old.Name:match("^LiveLabUniform") then old:Destroy() end
10536     end
10537     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
10538     if not torso then return end
10539     local panel = Instance.new("Part")
10540     panel.Name = "LiveLabUniform_Coat"
10541     panel.Size = torso.Size + Vector3.new(0.18, 0.25, 0.18)
10542     panel.CFrame = torso.CFrame
10543     panel.Color = Color3.fromRGB(240, 244, 247)
10544     panel.Material = Enum.Material.Fabric
10545     panel.CanCollide = false
10546     panel.Massless = true
10547     panel.Parent = character
10548     local weld = Instance.new("WeldConstraint")
10549     weld.Part0 = torso; weld.Part1 = panel; weld.Parent = panel
10550     local roleColour = roleColours[role] or Color3.fromRGB(150, 160, 170)
10551     local badge = weldPiece(character, panel, "LiveLabUniform_Badge", Vector3.new(0.4, 0.5, 0.08),
        ↪ Vector3.new(-0.52, 0.25, -panel.Size.Z / 2), roleColour)
10552     badge.Name = "LiveLabUniform_Badge"
10553     local head = character:FindFirstChild("Head")
10554     if head then
10555         local helmet = weldPiece(character, head, "LiveLabUniform_Helmet", Vector3.new(1.82, 0.7, 1.82),
        ↪ Vector3.new(0, 0.72, 0), roleColour)
10556         helmet.Shape = Enum.PartType.Ball
10557     end
10558 end
10559
10560 local function refreshLiveHud(player)
10561     local playerGui = player:FindFirstChild("PlayerGui")
10562     if not playerGui then return end
10563     local gui = playerGui:FindFirstChild("LiveLabHUD")
10564     local label = gui and gui:FindFirstChild("Objective")
10565     if label and label:IsA("TextLabel") then
10566         local role = player:GetAttribute("LabRole") or "Choose a role"

```

```

10567         local carried = player:GetAttribute("CarriedSample") or ""
10568         local objective = player:GetAttribute("LabObjective") or ""
10569         local stats = player:FindFirstChild("leaderstats")
10570         local points = stats and stats:FindFirstChild("Points")
10571         local lastAmount = player:GetAttribute("LastPointAmount") or 0
10572         local lastReason = player:GetAttribute("LastPointAward") or ""
10573         local timer = ""
10574         if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then
10575             timer = " • " .. math.max(0, player:GetAttribute("AssignedTransferDeadline") - os.time()) .. "
                ↪ sec"
10576         end
10577         label.Text = string.format("%s%s • POINTS %d%s\n%s%s", role,
10578             carried ~= "" and (" • HAND: " .. carried) or "",
10579             points and points.Value or 0,
10580             lastAmount > 0 and (" • LAST + " .. lastAmount .. " " .. lastReason) or "",
10581             objective ~= "" and objective or "Press E at BLUE or GREEN role kiosk.", timer)
10582     end
10583 end
10584
10585 local function buildLiveHud(player)
10586     local playerGui = player:WaitForChild("PlayerGui", 5)
10587     if not playerGui then return end
10588     if playerGui:FindFirstChild("RoleHUD") then
10589         local duplicate = playerGui:FindFirstChild("LiveLabHUD")
10590         if duplicate then duplicate:Destroy() end
10591         return
10592     end
10593     if playerGui:FindFirstChild("LiveLabHUD") then refreshLiveHud(player); return end
10594     local gui = Instance.new("ScreenGui")
10595     gui.Name = "LiveLabHUD"
10596     gui.ResetOnSpawn = false
10597     gui.Parent = playerGui
10598     local label = Instance.new("TextLabel")
10599     label.Name = "Objective"
10600     label.Size = UDim2.new(0.48, 0, 0, 62)
10601     label.Position = UDim2.new(0.02, 0, 0, 54)
10602     label.BackgroundColor3 = Color3.fromRGB(10, 18, 22)
10603     label.BackgroundTransparency = 0.18
10604     label.BorderSizePixel = 0
10605     label.TextColor3 = Color3.fromRGB(225, 245, 232)
10606     label.TextWrapped = true
10607     label.TextScaled = false
10608     label.TextSize = 12
10609     label.TextTruncate = Enum.TextTruncate.AtEnd
10610     label.Font = Enum.Font.GothamBold
10611     label.Parent = gui
10612     local corner = Instance.new("UICorner")
10613     corner.CornerRadius = UDim.new(0, 8)
10614     corner.Parent = label
10615     local constraint = Instance.new("UISizeConstraint")
10616     constraint.MinSize = Vector2.new(260, 62)
10617     constraint.MaxSize = Vector2.new(360, 62)
10618     constraint.Parent = label
10619     refreshLiveHud(player)
10620 end
10621
10622 for _, kiosk in ipairs(CollectionService:GetTagged("RoleKiosk")) do
10623     local prompt = Instance.new("ProximityPrompt")
10624     prompt.ActionText = "Select role"
10625     prompt.ObjectText = kiosk:GetAttribute("Role") or "Lab role"
10626     prompt.KeyboardKeyCode = Enum.KeyCode.E
10627     prompt.MaxActivationDistance = 14
10628     prompt.RequiresLineOfSight = false
10629     prompt.Parent = kiosk
10630     prompt.Triggered:Connect(function(player)
10631         local role = kiosk:GetAttribute("Role")
10632         local team = role and Teams:FindFirstChild(role)
10633         if team then player.Team = team; player.Neutral = false end
10634         player:SetAttribute("LabRole", role)
10635         liveAssignTask(player)
10636         if player.Character then dressCharacter(player.Character, role) end
10637         refreshLiveHud(player)
10638     end)
10639 end
10640 local function preserveLiveUniform(player)
10641     if player:GetAttribute("LiveUniformConnected") then return end

```



```

10642     player:SetAttribute("LiveUniformConnected", true)
10643     local stats = player:FindFirstChild("leaderstats")
10644     if not stats then stats = Instance.new("Folder"); stats.Name = "leaderstats"; stats.Parent = player end
10645     local points = stats:FindFirstChild("Points")
10646     if not points then points = Instance.new("IntValue"); points.Name = "Points"; points.Parent = stats end
10647     task.spawn(buildLiveHud, player)
10648     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshLiveHud(player) end)
10649     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshLiveHud(player) end)
10650     player:GetAttributeChangedSignal("LabRole"):Connect(function() refreshLiveHud(player) end)
10651     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshLiveHud(player) end)
10652     points:GetPropertyChangedSignal("Value"):Connect(function() refreshLiveHud(player) end)
10653     player.CharacterAdded:Connect(function(character)
10654         local role = player:GetAttribute("LabRole")
10655         if role then task.wait(0.35); dressCharacter(character, role) end
10656         local healthScript = character:FindFirstChild("Health")
10657         if healthScript then healthScript:Destroy() end
10658     end)
10659     if player.Character then
10660         local healthScript = player.Character:FindFirstChild("Health")
10661         if healthScript then healthScript:Destroy() end
10662     end
10663     task.spawn(function()
10664         while player.Parent do
10665             if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshLiveHud(player) end
10666             task.wait(1)
10667         end
10668     end)
10669 end
10670 for _, player in ipairs(Players:GetPlayers()) do preserveLiveUniform(player) end
10671 Players.PlayerAdded:Connect(preserveLiveUniform)
10672
10673 local LIVE_IMPACT = {
10674     ["Foam Baton"] = {mode = "melee", damage = 18, colour = Color3.fromRGB(245, 135, 55)},
10675     ["Sample Grabber"] = {mode = "melee", damage = 12, colour = Color3.fromRGB(70, 165, 220)},
10676     ["Inspection Paddle"] = {mode = "melee", damage = 22, colour = Color3.fromRGB(245, 205, 70)},
10677     ["Stun Blaster"] = {mode = "gun", damage = 28, range = 95, colour = Color3.fromRGB(90, 225, 245)},
10678     ["Foam Dart Rifle"] = {mode = "gun", damage = 38, range = 125, colour = Color3.fromRGB(225, 80, 95)},
10679     ["Basketball"] = {mode = "ball", damage = 24, colour = Color3.fromRGB(225, 105, 35)},
10680 }
10681
10682 local function liveBotFromHit(hit)
10683     local model = hit and hit:FindFirstAncestorOfClass("Model")
10684     return model and CollectionService.HasTag(model, "LabWorkerBot") and model or nil
10685 end
10686
10687 local function refreshLiveBotHealth(bot)
10688     local maximum = bot:GetAttribute("MaxHealth") or 100
10689     local health = math.clamp(bot:GetAttribute("Health") or maximum, 0, maximum)
10690     local label = bot:FindFirstChild("HealthText", true)
10691     local fill = bot:FindFirstChild("HealthBarFill", true)
10692     if label and label:IsA("TextLabel") then label.Text = string.format("LIFE %d / %d", health, maximum)
10693     ↪ end
10694     if fill and fill:IsA("Frame") then
10695         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
10696         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
10697         or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
10698     end
10699 end
10700
10701 local function damageLiveBot(bot, amount, attacker)
10702     if not bot or bot:GetAttribute("KnockedOut") then return end
10703     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
10704     bot:SetAttribute("Health", health)
10705     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
10706     refreshLiveBotHealth(bot)
10707     local flash = Instance.new("Highlight")
10708     flash.FillColor = Color3.fromRGB(255, 90, 70)
10709     flash.FillTransparency = 0.25
10710     flash.Parent = bot
10711     Debris:AddItem(flash, 0.22)
10712     if health <= 0 then
10713         bot:SetAttribute("KnockedOut", true)
10714         bot:SetAttribute("TaskState", "KNOCKED OUT")
10715         local status = bot:FindFirstChild("Status", true)
10716         if status and status:IsA("TextLabel") then status.Text = bot.Name .. "\nKNOCKED OUT" end
10717         task.delay(8, function()

```

```

10717         if bot.Parent then
10718             bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
10719             bot:SetAttribute("KnockedOut", false)
10720             refreshLiveBotHealth(bot)
10721         end
10722     end)
10723 end
10724 end
10725
10726 local function stealLiveSample(attacker, victim)
10727     local stolen = findPlant(victim)
10728     local backpack = attacker:FindFirstChildOfClass("Backpack")
10729     if not stolen or stolen.Parent ~= victim.Character or not backpack then return end
10730     stolen.Parent = backpack
10731     victim:SetAttribute("CarriedSample", "")
10732     attacker:SetAttribute("CarriedSample", stolen:GetAttribute("PlantId") or stolen.Name)
10733     local destination = stolen:GetAttribute("DestinationGrowthCode") or ""
10734     attacker:SetAttribute("LabObjective", destination ~= "" and ("Stolen sample must be rescued to " ..
        ↪ destination .. ".")
10735         or "Use growth equipment, then return the sample to a working chamber.")
10736     equip(attacker, stolen)
10737 end
10738
10739 local function grantImpactTool(player, kind)
10740     local style = LIVE_IMPACT[kind]
10741     if not style then return end
10742     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 3)
10743     if not backpack then return end
10744     local existing = backpack:FindFirstChild(kind) or (player.Character and
        ↪ player.Character:FindFirstChild(kind))
10745     if existing then equip(player, existing); return end
10746     local tool = Instance.new("Tool")
10747     tool.Name = kind
10748     tool.RequiresHandle = true
10749     tool.CanBeDropped = false
10750     tool.ToolTip = style.mode == "gun" and "Fire at worker NPCs"
10751         or (style.mode == "ball" and "Throw at worker NPCs" or "Hit NPCs or recover another player's held
        ↪ sample")
10752     tool:SetAttribute("LabImpactTool", true)
10753     tool:SetAttribute("ImpactToolKind", kind)
10754     local handle = Instance.new("Part")
10755     handle.Name = "Handle"
10756     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
10757         or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35)
10758         or (kind == "Inspection Paddle" and Vector3.new(1.2, 3.2, 0.35) or Vector3.new(0.5, 3.4, 0.5)))
10759     handle.Color = style.colour
10760     handle.Material = style.mode == "ball" and Enum.Material.Rubber or Enum.Material.SmoothPlastic
10761     handle.CanCollide = false
10762     handle.Massless = true
10763     handle.Parent = tool
10764     if style.mode == "ball" then
10765         handle.Shape = Enum.PartType.Ball
10766     elseif style.mode == "gun" then
10767         weldPiece(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7), Vector3.new(0, 0.1, -1.65),
        ↪ style.colour)
10768         weldPiece(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55), Vector3.new(0, -0.75, 0.42),
        ↪ Color3.fromRGB(42, 45, 52))
10769     end
10770
10771     local activeUntil, nextUse = 0, 0
10772     local hitThisUse = {}
10773     tool.Activated:Connect(function()
10774         local attacker = Players:GetPlayerFromCharacter(tool.Parent)
10775         if not attacker or os.clock() < nextUse then return end
10776         nextUse = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
10777         hitThisUse = {}
10778         local rootPart = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
10779         if not rootPart then return end
10780         if style.mode == "gun" then
10781             local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
10782             local direction = rootPart.CFrame.LookVector * (style.range or 100)
10783             local params = RaycastParams.new()
10784             params.FilterType = Enum.RaycastFilterType.Exclude
10785             params.FilterDescendantsInstances = {attacker.Character}
10786             local result = workspace.Raycast(origin, direction, params)
10787             local hitPosition = result and result.Position or origin + direction

```

```

10788         local tracer = Instance.new("Part")
10789         tracer.Name = "FoamTracer"
10790         tracer.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
10791         tracer.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
10792         tracer.Color = style.colour
10793         tracer.Material = Enum.Material.Neon
10794         tracer.Anchored = true
10795         tracer.CanCollide = false
10796         tracer.Parent = workspace
10797         Debris:AddItem(tracer, 0.12)
10798         local bot = result and liveBotFromHit(result.Instance)
10799         if bot then damageLiveBot(bot, style.damage, attacker) end
10800     elseif style.mode == "ball" then
10801         local ball = Instance.new("Part")
10802         ball.Name = "ThrownBasketball"
10803         ball.Shape = Enum.PartType.Ball
10804         ball.Size = Vector3.new(1.35, 1.35, 1.35)
10805         ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) +
            ↪ rootPart.CFrame.LookVector * 3)
10806         ball.Color = style.colour
10807         ball.Material = Enum.Material.Rubber
10808         ball.CanCollide = true
10809         ball.Parent = workspace
10810         ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
10811         local used = false
10812         ball.Touched:Connect(function(hit)
10813             if used then return end
10814             local bot = liveBotFromHit(hit)
10815             if bot then used = true; damageLiveBot(bot, style.damage, attacker) end
10816         end)
10817         Debris:AddItem(ball, 7)
10818     else
10819         activeUntil = os.clock() + 0.45
10820     end
10821 end)
10822 handle.Touched:Connect(function(hit)
10823     if style.mode ~= "melee" or os.clock() > activeUntil then return end
10824     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
10825     if not attacker then return end
10826     local bot = liveBotFromHit(hit)
10827     if bot then
10828         if hitThisUse[bot] then return end
10829         hitThisUse[bot] = true
10830         damageLiveBot(bot, style.damage, attacker)
10831         return
10832     end
10833     local victim = Players:GetPlayerFromCharacter(hit:FindFirstAncestorOfClass("Model"))
10834     if not victim or attacker == victim or hitThisUse[victim] then return end
10835     hitThisUse[victim] = true
10836     stealLiveSample(attacker, victim)
10837 end)
10838 tool.Parent = backpack
10839 equip(player, tool)
10840 end
10841
10842 for _, padPart in ipairs(CollectionService:GetTagged("LabToolDispenser")) do
10843     local prompt = Instance.new("ProximityPrompt")
10844     prompt.ActionText = "Pick up and equip"
10845     prompt.ObjectText = padPart:GetAttribute("ToolKind") or "Recovery tool"
10846     prompt.KeyboardKeyCode = Enum.KeyCode.E
10847     prompt.MaxActivationDistance = 12
10848     prompt.RequiresLineOfSight = false
10849     prompt.Parent = padPart
10850     prompt.Triggered:Connect(function(player) grantImpactTool(player, padPart:GetAttribute("ToolKind"))
        ↪ end)
10851 end
10852
10853 for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do bindLiveEquipment(equipment) end
10854
10855 -- Automatic doors, live-fallback copy: same open-near / close-away
10856 -- behaviour as the installed runtime.
10857 local LIVE_DOOR_OPEN_RANGE = 11
10858 local LIVE_DOOR_CLOSE_RANGE = 15
10859 local LIVE_DOOR_MIN_OPEN_SECONDS = 3
10860 local function bindLiveAutoDoor(door)
10861     if door:GetAttribute("DoorBound") then return end

```

```

10862     door:SetAttribute("DoorBound", true)
10863     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
10864     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
10865     local openX = door:GetAttribute("DoorOpenX") or closedX
10866     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
10867     local doorY = door:GetAttribute("DoorY") or door.Position.Y
10868     local isOpen = false
10869     local openedAt = 0
10870     local tween
10871     local function setIndicatorColour(open)
10872         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
10873         for _, gui in ipairs(door:GetChildren()) do
10874             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then
10875                 local indicator = gui:FindFirstChild("DoorIndicator")
10876                 if indicator then indicator.BackgroundColor3 = colour end
10877             end
10878         end
10879     end
10880     local function setOpen(open)
10881         if open == isOpen then return end
10882         isOpen = open
10883         if open then openedAt = os.clock() end
10884         if tween then tween:Cancel() end
10885         local targetX = open and openX or closedX
10886         local targetZ = open and openZ or closedZ
10887         tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
10888             ↪ Enum.EasingDirection.InOut),
10889             {CFrame.new(targetX, doorY, targetZ)})
10890         tween:Play()
10891         door.CanCollide = not open
10892         setIndicatorColour(open)
10893     end
10894     task.spawn(function()
10895         while door.Parent do
10896             local nearest = math.huge
10897             for _, player in ipairs(Players:GetPlayers()) do
10898                 local character = player.Character
10899                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
10900                 if hrp then
10901                     -- Same fix as the installed runtime: floors stack at the
10902                     -- same X/Z, so require the player be close in height to
10903                     -- this door before the flat distance counts at all.
10904                     local verticalGap = math.abs(hrp.Position.Y - doorY)
10905                     if verticalGap < 8 then
10906                         local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) -
10907                             ↪ Vector3.new(closedX, doorY, closedZ)).Magnitude
10908                         if flat < nearest then nearest = flat end
10909                     end
10910                 end
10911             end
10912             -- Minimum 3-second open hold before the door is allowed to
10913             -- close again -- fixes the open/close-too-fast flicker.
10914             if isOpen and nearest > LIVE_DOOR_CLOSE_RANGE and (os.clock() - openedAt) >=
10915                 ↪ LIVE_DOOR_MIN_OPEN_SECONDS then
10916                 setOpen(false)
10917             elseif not isOpen and nearest < LIVE_DOOR_OPEN_RANGE then
10918                 setOpen(true)
10919             end
10920             task.wait(0.25)
10921         end
10922     end)
10923     for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindLiveAutoDoor(p) end
10924     CollectionService.GetInstanceAddedSignal("AutoDoor"):Connect(bindLiveAutoDoor)
10925
10926     local liveFacilityNoticeAt = setmetatable({}, {__mode = "k"})
10927     local function liveNotify(player, text)
10928         local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
10929             ↪ text:match("^CATASTROPHIC FAILURE:")
10930         if facilityAlert then
10931             local now = os.clock()
10932             if now - (liveFacilityNoticeAt[player] or -math.huge) < 6 then return end
10933             liveFacilityNoticeAt[player] = now
10934         end
10935         print("[ " .. player.Name .. " ] " .. text)
10936         local gui = player:FindFirstChild("PlayerGui")

```

```

10934     if not gui then return end
10935     local screen = gui:FindFirstChild("LabLog")
10936     if not screen then
10937         screen = Instance.new("ScreenGui")
10938         screen.Name = "LabLog"
10939         screen.ResetOnSpawn = false
10940         screen.Parent = gui
10941         local label = Instance.new("TextLabel")
10942         label.Name = "Line"
10943         label.Size = UDim2.new(0.48, 0, 0, 46)
10944         label.Position = UDim2.new(0.5, 0, 1, -68)
10945         label.AnchorPoint = Vector2.new(0.5, 1)
10946         label.BackgroundTransparency = 0.3
10947         label.BackgroundColor3 = Color3.fromRGB(10, 10, 10)
10948         label.TextColor3 = Color3.fromRGB(255, 210, 90)
10949         label.TextScaled = false
10950         label.TextWrapped = true
10951         label.TextTruncate = Enum.TextTruncate.AtEnd
10952         label.TextSize = 13
10953         label.Font = Enum.Font.GothamBold
10954         label.Parent = screen
10955         local constraint = Instance.new("UISizeConstraint")
10956         constraint.MinSize = Vector2.new(260, 46)
10957         constraint.MaxSize = Vector2.new(420, 46)
10958         constraint.Parent = label
10959     end
10960     local label = screen:FindFirstChild("Line")
10961     if label then
10962         local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
10963         screen:SetAttribute("MessageSerial", serial)
10964         label.Text = text
10965         label.Visible = true
10966         task.delay(4, function()
10967             if label.Parent and screen:GetAttribute("MessageSerial") == serial then label.Visible =
10968                 ↪ false end
10969         end)
10970     end
10971
10972     -- EMERGENCY KITS: one per room, press E to fully restore health. Live-
10973     -- fallback copy of the same behaviour as the installed runtime.
10974     local function bindLiveEmergencyKit(kit)
10975         if kit:GetAttribute("KitBound") then return end
10976         kit:SetAttribute("KitBound", true)
10977         local prompt = kit:FindFirstChild("EmergencyKitPrompt")
10978         if not prompt then return end
10979         -- Recharges rather than being consumed, same as the installed copy.
10980         local RECHARGE_SECONDS = 25
10981         local function setCharged(charged)
10982             kit:SetAttribute("Charged", charged)
10983             prompt.Enabled = charged
10984             kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
10985         end
10986         prompt.Triggered:Connect(function(player)
10987             if kit:GetAttribute("Charged") == false then return end
10988             local character = player.Character
10989             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
10990             if not humanoid then return end
10991             setCharged(false)
10992             humanoid.Health = humanoid.MaxHealth
10993             liveNotify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s.")
10994             task.delay(RECHARGE_SECONDS, function()
10995                 if kit.Parent then setCharged(true) end
10996             end)
10997         end)
10998         setCharged(true)
10999     end
11000     for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindLiveEmergencyKit(p) end
11001     CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindLiveEmergencyKit)
11002
11003     -- SECRET BASEMENT VENT: live-fallback copy of the same disguised panel
11004     -- behaviour -- hold E on it to drop straight into the basement corridor.
11005     local function bindLiveSecretBasementPanel(panel)
11006         if panel:GetAttribute("SecretBound") then return end
11007         panel:SetAttribute("SecretBound", true)
11008         local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")

```

```

11009         if not prompt then return end
11010         prompt.Triggered:Connect(function(player)
11011             local character = player.Character
11012             local hrp = character and character:FindFirstChild("HumanoidRootPart")
11013             if not hrp then return end
11014             local dest = Vector3.new(
11015                 panel:GetAttribute("SecretDestX") or hrp.Position.X,
11016                 panel:GetAttribute("SecretDestY") or hrp.Position.Y,
11017                 panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
11018             hrp.CFrame = CFrame.new(dest)
11019             liveNotify(player, "The vent panel swings open -- you drop into the basement.")
11020         end)
11021     end
11022     for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindLiveSecretBasementPanel(p)
11023     ↪ end
11024     CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindLiveSecretBasementPanel)
11025
11026     local function liveNearestEmergencyKit(position)
11027         local best, bestDist
11028         for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
11029             if kit.Parent then
11030                 local d = (kit.Position - position).Magnitude
11031                 if not bestDist or d < bestDist then best, bestDist = kit, d end
11032             end
11033         end
11034         return best
11035     end
11036
11037     local liveHealthPathActive = {}
11038     task.spawn(function()
11039         while true do
11040             task.wait(3)
11041             for _, player in ipairs(Players:GetPlayers()) do
11042                 local character = player.Character
11043                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11044                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
11045                 if humanoid and hrp and humanoid.Health > 0 then
11046                     local low = humanoid.Health < humanoid.MaxHealth * 0.5
11047                     local busy = player:GetAttribute("AssignedRepairId") or
11048                         ↪ player:GetAttribute("AssignedTransferSource")
11049                     if low and not busy and not liveHealthPathActive[player] then
11050                         local kit = liveNearestEmergencyKit(hrp.Position)
11051                         if kit then
11052                             createLiveRepairPath(player, kit)
11053                             liveHealthPathActive[player] = true
11054                             liveNotify(player, "Health low: follow the RED floor lights to the nearest
11055                             ↪ emergency kit.")
11056                         end
11057                     elseif (not low or busy) and liveHealthPathActive[player] then
11058                         liveHealthPathActive[player] = false
11059                         if not busy then clearLiveRepairPath(player) end
11060                     end
11061                 end
11062             end
11063         end
11064     end)
11065
11066     -- EQUIPMENT HAZARDS: a FAILED unit drains health in nearby players --
11067     -- electrical faults, steam/humidity blowouts, chemical spills, coolant
11068     -- (freon) leaks. Live-fallback copy of the installed runtime's version.
11069     local LIVE_HAZARD_BY_KIND = {
11070         ["Germination Chamber"] = {"STEAM LEAK", 3},
11071         ["Incubator"] = {"STEAM LEAK", 3},
11072         ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
11073         ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
11074         ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
11075         ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
11076         ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
11077         ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
11078         ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},
11079         ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
11080         ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
11081     }
11082     local LIVE_HAZARD_RANGE = 9
11083     local liveHazardWarnedAt = {}

```

```

11082     local LIVE_DESTRUCTION_KIND = {
11083         ["Germination Chamber"] = "steam",
11084         ["Incubator"] = "steam",
11085         ["Decontamination Unit"] = "gas",
11086         ["Mutagenesis Chamber"] = "electrical",
11087         ["Hormone Treatment Bench"] = "liquid",
11088         ["Plant Growth Chamber"] = "gas",
11089         ["Growth Hormone Mixer"] = "liquid",
11090         ["GMO Injector"] = "electrical",
11091         ["Nutrient Infuser"] = "gas",
11092         ["UV Growth Scanner"] = "electrical",
11093         ["Pollination Station"] = "electrical",
11094     }
11095
11096     local function liveScatterDebris(part)
11097         for i = 1, math.random(4, 7) do
11098             local chunk = Instance.new("Part")
11099             chunk.Name = "Debris"
11100             chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
11101             chunk.Color = part.Color
11102             chunk.Material = Enum.Material.Metal
11103             chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2,
11104                 ⇨ math.random(-3, 3) / 2)
11105             chunk.Anchored = false
11106             chunk.CanCollide = true
11107             chunk.Parent = part.Parent
11108             local av = Instance.new("AngularVelocity")
11109             av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
11110             av.Parent = chunk
11111             local bv = Instance.new("BodyVelocity")
11112             bv.MaxForce = Vector3.new(4000, 4000, 4000)
11113             bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
11114             bv.Parent = chunk
11115             Debris:AddItem(bv, 0.25)
11116             Debris:AddItem(chunk, 8)
11117         end
11118     end
11119
11120     -- Same catastrophic-failure escalation as the installed runtime: a unit
11121     -- left FAILED too long can permanently blow up / catch fire / arc, with
11122     -- kind-dependent smoke/liquid/gas effects and flying debris.
11123     local destroyLiveEquipment
11124
11125     local function warnLiveEquipmentCatastrophe(part, floorLabel, unitLabel)
11126         if part:GetAttribute("AboutToBlow") then return end
11127         part:SetAttribute("AboutToBlow", true)
11128         local beacon = Instance.new("PointLight")
11129         beacon.Color = Color3.fromRGB(255, 40, 30)
11130         beacon.Brightness = 4
11131         beacon.Range = 16
11132         beacon.Parent = part
11133         local siren = Instance.new("Sound")
11134         siren.Name = "SirenWarning"
11135         siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
11136         siren.Looped = true
11137         siren.Volume = 3
11138         siren.PlaybackSpeed = 1.6
11139         siren.Parent = part
11140         siren:Play()
11141         for _, player in ipairs(Players:GetPlayers()) do
11142             liveNotify(player, "ALARM: " .. floorLabel .. " • " .. unitLabel .. " is about to fail
11143                 ⇨ catastrophically -- MOVE AWAY NOW!")
11144         end
11145         task.spawn(function()
11146             local elapsed = 0
11147             while elapsed < 2 and part.Parent do
11148                 beacon.Enabled = not beacon.Enabled
11149                 task.wait(0.2)
11150                 elapsed = elapsed + 0.2
11151             end
11152         end)
11153         task.delay(2, function()
11154             siren:Stop()
11155             siren:Destroy()
11156             beacon:Destroy()
11157             if part.Parent then destroyLiveEquipment(part) end
11158         end)
11159     end

```

```

11156     end)
11157 end
11158
11159 destroyLiveEquipment = function(part)
11160     local floorLabel = liveFloor(part)
11161     local unitLabel = liveEquipmentLabel(part)
11162     local causes = {"exploded", "caught fire", "arced and shorted out"}
11163     local cause = causes[math.random(1, #causes)]
11164     part:SetAttribute("FunctionalState", "DESTROYED")
11165     part:SetAttribute("AboutToBlow", nil)
11166     part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • DESTROYED, UNFIXABLE")
11167     for _, ledPart in ipairs(part:GetDescendants()) do
11168         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
            ↳ Enum.Material.SmoothPlastic end
11169     end
11170     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
11171
11172     local bang = Instance.new("Sound")
11173     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
11174     bang.Volume = 4
11175     bang.PlaybackSpeed = 0.8
11176     bang.Parent = part
11177     bang:Play()
11178     Debris:AddItem(bang, 4)
11179
11180     local explosion = Instance.new("Explosion")
11181     explosion.Position = part.Position
11182     explosion.BlastRadius = 8
11183     explosion.BlastPressure = 0
11184     explosion.ExplosionType = Enum.ExplosionType.NoCraters
11185     explosion.DestroyJointRadiusPercent = 0
11186     explosion.Parent = workspace
11187
11188     local kindOfDestruction = LIVE_DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
11189
11190     local fire = Instance.new("Fire")
11191     fire.Size = 6
11192     fire.Heat = 9
11193     fire.Color = Color3.fromRGB(255, 140, 40)
11194     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
11195     fire.Parent = part
11196     Debris:AddItem(fire, 16)
11197
11198     local sparkles = Instance.new("Sparkles")
11199     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
11200     sparkles.Parent = part
11201     Debris:AddItem(sparkles, 3)
11202
11203     local smoke = Instance.new("Smoke")
11204     smoke.Size = 8
11205     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
11206     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
11207     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
11208         or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
11209         or Color3.fromRGB(50, 50, 55)
11210     smoke.Parent = part
11211     Debris:AddItem(smoke, 20)
11212
11213     if kindOfDestruction == "liquid" then
11214         local puddle = Instance.new("Part")
11215         puddle.Name = "LeakPuddle"
11216         puddle.Shape = Enum.PartType.Cylinder
11217         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
11218         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05,
            ↳ part.Position.Z)
            ↳ * CFrame.Angles(0, 0, math.rad(90))
11219         puddle.Color = Color3.fromRGB(120, 90, 40)
11220         puddle.Material = Enum.Material.Neon
11221         puddle.Transparency = 0.35
11222         puddle.Anchored = true
11223         puddle.CanCollide = false
11224         puddle.Parent = part.Parent
11225         Debris:AddItem(puddle, 25)
11226     end
11227
11228     liveScatterDebris(part)
11229

```



```

11230
11231     for _, player in ipairs(Players:GetPlayers()) do
11232         liveNotify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause ..
            ↳ ". It cannot be repaired.")
11233     end
11234 end
11235
11236 task.spawn(function()
11237     while true do
11238         task.wait(1)
11239         local hazards = {}
11240         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
11241             local state = equip.Parent and equip:GetAttribute("FunctionalState")
11242             if state == "FAILED" then
11243                 local info = LIVE_HAZARD_BY_KIND[equip:GetAttribute("Kind")]
11244                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
11245                 local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
11246                 if failedFor > 45 and math.random(1, 6) == 1 then
11247                     warnLiveEquipmentCatastrophe(equip, liveFloor(equip), liveEquipmentLabel(equip))
11248                 end
11249             elseif state == "DESTROYED" then
11250                 local info = LIVE_HAZARD_BY_KIND[equip:GetAttribute("Kind")]
11251                 if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
                    ↳ info[2] + 2}) end
11252             end
11253         end
11254         if #hazards > 0 then
11255             for _, player in ipairs(Players:GetPlayers()) do
11256                 local character = player.Character
11257                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11258                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
11259                 if humanoid and hrp and humanoid.Health > 0 then
11260                     local worst
11261                     for _, hazard in ipairs(hazards) do
11262                         local d = (hazard.part.Position - hrp.Position).Magnitude
11263                         if d < LIVE_HAZARD_RANGE and (not worst or hazard.dps > worst.dps) then worst =
                            ↳ hazard end
11264                     end
11265                     if worst then
11266                         humanoid:TakeDamage(worst.dps)
11267                         local last = liveHazardWarnedAt[player] or 0
11268                         if os.clock() - last > 2 then
11269                             liveHazardWarnedAt[player] = os.clock()
11270                             liveNotify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
                                ↳ worst.label, math.floor(humanoid.Health)))
11271                         end
11272                     end
11273                 end
11274             end
11275         end
11276     end
11277 end)
11278
11279 -- One straight-line leg of a walk. Used directly only as a last resort;
11280 -- normally it's called once per pathfinding waypoint, so each leg is
11281 -- short enough that it never needs to cut through a wall.
11282 local function walkBotSegment(bot, destination)
11283     local startPivot = bot:GetPivot()
11284     local from = startPivot.Position
11285     local to = Vector3.new(destination.X, from.Y, destination.Z)
11286     local heldOffsets = {}
11287     local held = bot:FindFirstChild("BotHeldPlant")
11288     if held then
11289         for _, piece in ipairs(held:GetDescendants()) do
11290             if piece:IsA("BasePart") then heldOffsets[piece] = startPivot.ToObjectSpace(piece.CFrame) end
11291         end
11292     end
11293     local steps = math.max(1, math.ceil((to - from).Magnitude / 1.8))
11294     for step = 1, steps do
11295         if not bot.Parent then return false end
11296         if bot:GetAttribute("Dead") then return false end
11297         while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do task.wait(0.25)
            ↳ end
11298             local p = from:Lerp(to, step / steps)
11299             local pose = CFrame.lookAt(p, Vector3.new(to.X, p.Y, to.Z) + Vector3.new(0.01, 0, 0.01))
11300             bot:PivotTo(pose)

```

```

11301         for piece, offset in pairs(heldOffsets) do
11302             if piece.Parent then piece.CFrame = pose * offset end
11303         end
11304         local complex = workspace:FindFirstChild("Laboratory_Complex")
11305         local throughput = complex and (complex:GetAttribute("RecentTransferThroughput") or 0) or 0
11306         local speedScale = 1 + math.min(10, #Players:GetPlayers()) * 0.05 + math.clamp(throughput, 0, 20) *
            ↳ 0.01
11307         task.wait(math.clamp(0.08 / speedScale, 0.035, 0.1))
11308     end
11309     return true
11310 end
11311
11312 -- Real pathfinding: walk actual corridor waypoints instead of a straight
11313 -- line through everything in between (which is what made bots visibly
11314 -- clip through walls). Falls back to the old straight-line walk only if
11315 -- the navmesh calculation itself fails.
11316 local function botMove(bot, target)
11317     local destination = target.Position
11318     local path = PathfindingService:CreatePath({
11319         AgentRadius = 1.2,
11320         AgentHeight = 5.8,
11321         AgentCanJump = true,
11322         WaypointSpacing = 5,
11323     })
11324     local ok = pcall(function()
11325         path:ComputeAsync(bot:GetPivot().Position, destination)
11326     end)
11327     if ok and path.Status == Enum.PathStatus.Success then
11328         for _, waypoint in ipairs(path:GetWaypoints()) do
11329             if not walkBotSegment(bot, waypoint.Position) then return end
11330         end
11331         return
11332     end
11333     bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
11334     walkBotSegment(bot, destination)
11335 end
11336 local function liveBotPlant(bot, visible)
11337     local oldHeld = bot:FindFirstChild("BotHeldPlant")
11338     if oldHeld then oldHeld:Destroy() end
11339     if not visible then return end
11340     local held = Instance.new("Model")
11341     held.Name = "BotHeldPlant"
11342     held.Parent = bot
11343     local base = bot:GetPivot() * CFrame.new(1.45, 0.1, -0.7)
11344     for _, data in ipairs({
11345         {"Pot", Vector3.new(0.82, 0.62, 0.82), Vector3.new(0, 0, 0), Color3.fromRGB(116, 75, 45)},
11346         {"Stem", Vector3.new(0.13, 1.0, 0.13), Vector3.new(0, 0.72, 0), Color3.fromRGB(58, 160, 70)},
11347         {"LeafL", Vector3.new(0.55, 0.13, 0.34), Vector3.new(-0.25, 0.9, 0), Color3.fromRGB(73, 185, 84)},
11348         {"LeafR", Vector3.new(0.55, 0.13, 0.34), Vector3.new(0.25, 1.12, 0), Color3.fromRGB(73, 185, 84)},
11349     }) do
11350         local piece = Instance.new("Part")
11351         piece.Name = data[1]
11352         piece.Size = data[2]
11353         piece.CFrame = base * CFrame.new(data[3])
11354         piece.Color = data[4]
11355         piece.Material = Enum.Material.SmoothPlastic
11356         piece.Anchored = true
11357         piece.CanCollide = false
11358         piece.CanTouch = false
11359         piece.Parent = held
11360     end
11361 end
11362 local function refreshLiveBotBoard()
11363     local workers = CollectionService:GetTagged("LabWorkerBot")
11364     local cycles, carrying, knockedOut, rivalScore = 0, 0, 0, 0
11365     for _, worker in ipairs(workers) do
11366         cycles = cycles + (worker:GetAttribute("CompletedCycles") or 0)
11367         if (worker:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
11368         if worker:GetAttribute("KnockedOut") then knockedOut = knockedOut + 1 end
11369         if worker:GetAttribute("Competitor") then rivalScore = worker:GetAttribute("Score") or 0 end
11370     end
11371     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
11372     local failed, failedSamples, totalSamples = 0, 0, 0
11373     for _, chamber in ipairs(chambers) do
11374         local count = chamber:GetAttribute("SampleCount") or 0
11375         totalSamples = totalSamples + count

```

```

11376         if chamber:GetAttribute("FunctionalState") == "FAILED" then
11377             failed = failed + 1
11378             failedSamples = failedSamples + count
11379         end
11380     end
11381     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
11382         for _, label in ipairs(board:GetDescendants()) do
11383             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
11384                 label.Text = string.format(
11385                     "NPC GAMEPLAY TESTS\nWorkers: %d / 50 • knocked out: %d\nRescue cycles: %d • plants
↪ in transit: %d\nFailed chambers: %d / %d • failed samples: %d\nStored samples: %d
↪ • RIVAL SCORE: %d",
11386                     #workers, knockedOut, cycles, carrying, failed, #chambers, failedSamples, totalSamples,
↪ rivalScore)
11387             end
11388         end
11389     end
11390 end
11391
11392 local function setLiveBotStatus(bot, message)
11393     bot:SetAttribute("TaskState", message)
11394     local status = bot:FindFirstChild("Status", true)
11395     if status and status:IsA("TextLabel") then status.Text = bot.Name .. "\n" .. message end
11396 end
11397
11398 -- Same room-occupancy gate as the installed script: a worker only runs
11399 -- while a player is in the room with it.
11400 local liveFloorBand, liveWakeRadius = 10, 35
11401 local liveRoomsByLevel = {}
11402 do
11403     local complex = workspace:FindFirstChild("Laboratory_Complex")
11404     local packed = complex and complex:GetAttribute("RoomIndex")
11405     if typeof(packed) == "string" then
11406         for entry in packed:gmatch("[^;]+") do
11407             local level, name, x1, z1, x2, z2, y =
11408                 entry:match("^([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+)$")
11409             if level then
11410                 liveRoomsByLevel[level] = liveRoomsByLevel[level] or {}
11411                 table.insert(liveRoomsByLevel[level], {
11412                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
11413                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
11414                 })
11415             end
11416         end
11417     end
11418 end
11419
11420 local function liveRoomAt(levelName, position)
11421     local list = levelName and liveRoomsByLevel[levelName]
11422     if not list then return nil end
11423     for _, r in ipairs(list) do
11424         if math.abs(position.Y - r.y) < liveFloorBand
11425             and position.X >= r.x1 and position.X <= r.x2
11426             and position.Z >= r.z1 and position.Z <= r.z2 then
11427             return r.name
11428         end
11429     end
11430     return nil
11431 end
11432
11433 local function liveBotHasObserver(bot)
11434     local ok, pivot = pcall(function() return bot:GetPivot().Position end)
11435     if not ok then return false end
11436     local levelName = bot:GetAttribute("Level")
11437     local botRoom = liveRoomAt(levelName, pivot)
11438     for _, player in ipairs(Players:GetPlayers()) do
11439         local character = player.Character
11440         local hrp = character and character:FindFirstChild("HumanoidRootPart")
11441         if hrp then
11442             local here = hrp.Position
11443             if math.abs(here.Y - pivot.Y) < liveFloorBand then
11444                 if botRoom and liveRoomAt(levelName, here) == botRoom then return true end
11445                 local dx, dz = here.X - pivot.X, here.Z - pivot.Z
11446                 if dx * dx + dz * dz < liveWakeRadius * liveWakeRadius then return true end
11447             end
11448         end
11449     end
11450 end

```

```

11449     end
11450     return false
11451 end
11452
11453 local bots = CollectionService:GetTagged("LabWorkerBot")
11454 for index, bot in ipairs(bots) do
11455     refreshLiveBotHealth(bot)
11456     -- Sleeping bots are cheap, so there is no herd to spread out.
11457     local startSlot = bot:GetAttribute("LevelSlot") or index
11458     task.delay(startSlot * 0.08, function()
11459         while bot.Parent do
11460             local awake = bot:GetAttribute("Competitor") or liveBotHasObserver(bot)
11461             if not awake then
11462                 setLiveBotStatus(bot, "Idle -- no one in the room")
11463                 task.wait(1)
11464             elseif bot:GetAttribute("KnockedOut") then
11465                 task.wait(1)
11466             else
11467                 local levelName = bot:GetAttribute("Level")
11468                 local failed, stations = {}, {}
11469                 for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
11470                     if equipment:GetAttribute("Level") == levelName then
11471                         if equipment:GetAttribute("Kind") == "Plant Growth Chamber"
11472                             and equipment:GetAttribute("FunctionalState") == "FAILED"
11473                             and (equipment:GetAttribute("SampleCount") or 0) > 0
11474                             and not equipment:GetAttribute("ReservedByBot") then
11475                             table.insert(failed, equipment)
11476                         end
11477                         if plantProcesses[equipment:GetAttribute("Kind")] then table.insert(stations,
11478                             ↪ equipment) end
11479                     end
11480                 end
11481                 if #failed == 0 then
11482                     setLiveBotStatus(bot, "Monitoring chamber alarms")
11483                 else
11484                     table.sort(failed, function(a, b)
11485                         return (tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
11486                             < (tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
11487                     end)
11488                     local source = bot:GetAttribute("Competitor") and failed[1] or failed[math.random(1,
11489                         ↪ #failed)]
11490                     local destination = liveDestinations[source][1]
11491                     if not destination then
11492                         setLiveBotStatus(bot, "Waiting for safe capacity")
11493                     else
11494                         source:SetAttribute("ReservedByBot", bot.Name)
11495                         destination:SetAttribute("IncomingReservations",
11496                             ↪ (destination:GetAttribute("IncomingReservations") or 0) + 1)
11497                         source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode")
11498                             ↪ or "")
11499                         refreshLiveTelemetry(source)
11500                         setLiveBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or
11501                             ↪ "alarm"))
11502                         botMove(bot, source)
11503                         local sourceSamples = liveChamberSamples(source)
11504                         local plantId = table.remove(sourceSamples, 1)
11505                         if plantId and source:GetAttribute("ReservedByBot") == bot.Name then
11506                             saveLiveChamber(source, sourceSamples)
11507                             showChamberPlant(source, true)
11508                             refreshLiveTelemetry(source)
11509                             bot:SetAttribute("CarryingPlant", plantId)
11510                             liveBotPlant(bot, true)
11511                             setLiveBotStatus(bot, "Evacuating " .. plantId)
11512                         end
11513                         local stage = source:GetAttribute("GrowthStage") or 0
11514                         if #stations > 0 then
11515                             local station = stations[((bot:GetAttribute("BotNumber") or index) - 1) %
11516                                 ↪ #stations + 1]
11517                             botMove(bot, station)
11518                             local process = plantProcesses[station:GetAttribute("Kind")]
11519                             stage = math.clamp(stage + (process.gain or 0), 0, 5)
11520                             station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or
11521                                 ↪ 0) + 1)
11522                         end
11523                     end
11524                 end
11525             end
11526         end
11527     end)
11528 end

```

```

11517         setLiveBotStatus(bot, "Delivering to " ..
11518             ↪ (destination:GetAttribute("GrowthCode") or "working chamber"))
11519         botMove(bot, destination)
11520         local destinationSamples = liveChamberSamples(destination)
11521         if destination:GetAttribute("FunctionalState") ~= "FAILED"
11522             and #destinationSamples < (destination:GetAttribute("Capacity") or 10) then
11523             table.insert(destinationSamples, plantId)
11524             saveLiveChamber(destination, destinationSamples)
11525             destination:SetAttribute("GrowthStage",
11526                 ↪ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
11527             showChamberPlant(destination, true)
11528             refreshLiveTelemetry(destination)
11529             liveEmergencyMoves = liveEmergencyMoves + 1
11530             bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or
11531                 ↪ 0) + 1)
11532             if bot:GetAttribute("Competitor") then
11533                 bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) +
11534                     ↪ LIVE_POINTS.MOVE_ON_TIME)
11535                 setLiveBotStatus(bot, "+" .. LIVE_POINTS.MOVE_ON_TIME .. " resource
11536                     ↪ win")
11537                 refreshLiveScoreboard()
11538             else
11539                 setLiveBotStatus(bot, "Rescue cycle passed")
11540             end
11541         else
11542             table.insert(sourceSamples, plantId)
11543             saveLiveChamber(source, sourceSamples)
11544             showChamberPlant(source, true)
11545             refreshLiveTelemetry(source)
11546             setLiveBotStatus(bot, "Destination changed; retrying")
11547         end
11548         bot:SetAttribute("CarryingPlant", "")
11549         liveBotPlant(bot, false)
11550     end
11551     destination:SetAttribute("IncomingReservations", math.max(0,
11552         ↪ (destination:GetAttribute("IncomingReservations") or 1) - 1))
11553     if source:GetAttribute("ReservedByBot") == bot.Name then
11554         ↪ source:SetAttribute("ReservedByBot", nil) end
11555     end
11556 end
11557 end
11558 end
11559 refreshLiveBotBoard()
11560 task.wait(1.5 + math.random())
11561 end
11562 end
11563 end
11564
11565 local LIVE_FAILURE_REASONS = {
11566     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
11567     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
11568 }
11569 local previousLiveMoves = 0
11570 setLiveFailure = function(chamber, failed)
11571     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
11572     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
11573     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN"
11574     ↪ "FLOOR"
11575     if failed then
11576         chamber:SetAttribute("FunctionalState", "FAILED")
11577         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
11578         chamber:SetAttribute("RepairKey", "R")
11579         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " •
11580             ↪ HOLD R TO REPAIR")
11581         chamber:SetAttribute("FailureReason", LIVE_FAILURE_REASONS[math.random(1, #LIVE_FAILURE_REASONS)])
11582         chamber:SetAttribute("FailureSince", os.time())
11583         chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
11584         chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
11585         chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
11586         local destination = liveDestinations(chamber)[1]
11587         chamber:SetAttribute("EvacuationDestination", destination and
11588             ↪ destination:GetAttribute("GrowthCode") or "")
11589         for _, player in ipairs(Players:GetPlayers()) do
11590             if livePlayerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
11591                 ↪ liveAssignTask(player) end
11592             if livePlayerRole(player) == "Laboratory Technician" and not
11593                 ↪ player:GetAttribute("AssignedTransferSource") then liveAssignTask(player) end

```

```

11581     end
11582 else
11583     chamber:SetAttribute("FunctionalState", "WORKING")
11584     chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
11585     chamber:SetAttribute("FailureReason", "")
11586     chamber:SetAttribute("FailureSince", 0)
11587     chamber:SetAttribute("EvacuationDestination", "")
11588     chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
11589     chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
11590     chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
11591     chamber:SetAttribute("OperationStarted", os.time())
11592     if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) <
        ⇨ 100 then
11593         recoverLivePlantLife(chamber, chamber:GetAttribute("PlantLife"), nil,
            ⇨ chamber:GetAttribute("PlantId"))
11594     end
11595 end
11596 refreshLiveTelemetry(chamber)
11597 end
11598
11599 local LIVE_GENERIC_REASONS = {"Power relay fault", "Controller link lost", "Pump interlock fault",
    ⇨ "Calibration expired", "Motor overload"}
11600 setLiveGenericFailure = function(part, failed)
11601     if failed then
11602         part:SetAttribute("FunctionalState", "FAILED")
11603         part:SetAttribute("FailureReason", LIVE_GENERIC_REASONS[math.random(1, #LIVE_GENERIC_REASONS)])
11604         part:SetAttribute("FailureSince", os.time())
11605         part:SetAttribute("RepairInstruction", liveFloor(part) .. " • " .. liveEquipmentLabel(part) .. " • "
            ⇨ HOLD R")
11606     else
11607         part:SetAttribute("FunctionalState", "WORKING")
11608         part:SetAttribute("FailureReason", "")
11609         part:SetAttribute("FailureSince", 0)
11610         part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
11611         part:SetAttribute("OperationStarted", os.time())
11612     end
11613     local readout = part:FindFirstChild("Readout", true)
11614     if readout and readout:IsA("TextLabel") then
11615         readout.Text = (failed and "FAILED • HOLD R\n" or "WORKING\n") .. liveFloor(part) .. " • " ..
            ⇨ liveEquipmentLabel(part)
11616         .. "\nPLANTS " .. (part:GetAttribute("SampleCount") or 0) .. "/" ..
            ⇨ (part:GetAttribute("Capacity") or 0)
11617         readout.TextColor3 = failed and Color3.fromRGB(255, 105, 90) or Color3.fromRGB(115, 235, 165)
11618     end
11619     if failed then
11620         for _, player in ipairs(Players:GetPlayers()) do
11621             if livePlayerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
11622                 ⇨ liveAssignTask(player) end
11623         end
11624     end
11625 end
11626
11627 local function liveAdaptiveFailureTick()
11628     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
11629     local failed, working = {}, {}
11630     local failedSamples, totalSamples, workingFree = 0, 0, 0
11631     for _, chamber in ipairs(chambers) do
11632         local count = chamber:GetAttribute("SampleCount") or 0
11633         local capacity = chamber:GetAttribute("Capacity") or 10
11634         totalSamples = totalSamples + count
11635         if chamber:GetAttribute("FunctionalState") == "FAILED" then
11636             table.insert(failed, chamber)
11637             failedSamples = failedSamples + count
11638         else
11639             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then
11640                 table.insert(working, chamber)
11641             end
11642             workingFree = workingFree + math.max(0, capacity - count -
                ⇨ (chamber:GetAttribute("IncomingReservations") or 0))
11643         end
11644     end
11645     local throughput = liveEmergencyMoves - previousLiveMoves
11646     previousLiveMoves = liveEmergencyMoves
11647     local backlogRatio = failedSamples / math.max(1, totalSamples)
11648     local playerCount = math.max(1, #Players:GetPlayers())
11649     local targetRate = 0.60 + math.clamp((throughput - 5) * 0.003, -0.03, 0.08)

```

```

11649         + math.clamp((playerCount - 1) * 0.012, 0, 0.10)
11650         - math.clamp((backlogRatio - 0.60) * 0.08, -0.04, 0.04)
11651     if workingFree < 20 then targetRate = targetRate - 0.12 end
11652     targetRate = math.clamp(targetRate, 0.42, 0.76)
11653     local targetFailed = math.floor(#chambers * targetRate + 0.5)
11654     local maxChange = math.clamp(2 + playerCount * 2, 4, 12)
11655     local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
11656     if change > 0 then
11657         for _ = 1, change do
11658             if #working == 0 then break end
11659             local chosen = table.remove(working, math.random(1, #working))
11660             setLiveFailure(chosen, true)
11661         end
11662     elseif change < 0 then
11663         table.sort(failed, function(a, b) return (a:GetAttribute("SampleCount") or 0) <
11664             ⇨ (b:GetAttribute("SampleCount") or 0) end)
11665         for _ = 1, -change do
11666             if #failed == 0 then break end
11667             setLiveFailure(table.remove(failed, 1), false)
11668         end
11669     end
11670     local currentFailed = 0
11671     for _, chamber in ipairs(chambers) do
11672         if chamber:GetAttribute("FunctionalState") == "FAILED" then
11673             currentFailed = currentFailed + 1
11674             chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
11675                 ⇨ math.random(-2, 2), 28, 42))
11676             chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
11677                 ⇨ math.random(-4, 4), 25, 65))
11678             chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
11679                 ⇨ math.random(-5, 3), 0, 48))
11680             local destination = liveDestinations[chamber][1]
11681             chamber:SetAttribute("EvacuationDestination", destination and
11682                 ⇨ destination:GetAttribute("GrowthCode") or "")
11683         else
11684             local samples = liveChamberSamples(chamber)
11685             local capacity = chamber:GetAttribute("Capacity") or 10
11686             local refillChance = math.clamp(0.10 + (capacity - #samples) * 0.022 + throughput * 0.002
11687                 + playerCount * 0.008, 0.10, 0.46)
11688             if #samples < capacity and math.random() < refillChance then
11689                 liveSampleSerial = liveSampleSerial + 1
11690                 table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
11691                     ⇨ liveSampleSerial))
11692                 saveLiveChamber(chamber, samples)
11693                 chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
11694                 showChamberPlant(chamber, true)
11695             end
11696         end
11697         refreshLiveTelemetry(chamber)
11698     end
11699     local complex = workspace:FindFirstChild("Laboratory_Complex")
11700     if complex then
11701         complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
11702         complex:SetAttribute("FailingChambers", currentFailed)
11703         complex:SetAttribute("FailedSampleBacklog", failedSamples)
11704         complex:SetAttribute("EmergencyMovesCompleted", liveEmergencyMoves)
11705         complex:SetAttribute("RecentTransferThroughput", throughput)
11706         complex:SetAttribute("ActivePlayerCount", playerCount)
11707         complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples
11708             ⇨ / 20), 8, 50))
11709         -- Per-FL00R budget. Multiplied across the basement plus every level
11710         -- this is what actually decides how many workers are visibly busy.
11711         complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
11712             ⇨ math.floor(failedSamples / 60), 6, 20))
11713     end
11714 end
11715
11716 local function liveGenericFailureTick()
11717     local failed, working = {}, {}
11718     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
11719         if equipment:GetAttribute("Kind") ~= "Plant Growth Chamber" then
11720             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment)
11721             elseif os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 45 then
11722                 ⇨ table.insert(working, equipment) end
11723         end
11724     end
11725 end

```

```

11716     end
11717     local playerCount = math.max(1, #Players:GetPlayers())
11718     local targetRate = math.clamp(0.12 + playerCount * 0.018, 0.14, 0.28)
11719     local target = math.max(2, math.floor((#failed + #working) * targetRate + 0.5))
11720     local maxChange = math.clamp(1 + math.ceil(playerCount / 2), 2, 6)
11721     local change = math.clamp(target - #failed, -maxChange, maxChange)
11722     for _ = 1, math.max(0, change) do
11723         if #working == 0 then break end
11724         setLiveGenericFailure(table.remove(working, math.random(1, #working)), true)
11725     end
11726     for _ = 1, math.max(0, -change) do
11727         if #failed == 0 then break end
11728         setLiveGenericFailure(table.remove(failed, 1), false)
11729     end
11730 end
11731
11732 task.spawn(function()
11733     while task.wait(2) do refreshLiveBotBoard() end
11734 end)
11735 task.spawn(function()
11736     while task.wait(2) do refreshLiveRoomMonitors() end
11737 end)
11738 task.spawn(function()
11739     task.wait(3)
11740     while true do
11741         liveAdaptiveFailureTick()
11742         liveGenericFailureTick()
11743         task.wait(math.clamp(15 - #Players:GetPlayers() * 1.25, 6, 14))
11744     end
11745 end)
11746 Players.PlayerRemoving:Connect(clearLiveRepairPath)
11747 warn("Seed Lab live fallback armed: normal Script.Source installation was unavailable, but E/F gameplay is
↪ active for this play session.")
11748 end
11749
11750 do
11751     for _, n in ipairs({"LabKinds"}) do
11752         local e = ReplicatedStorage:FindFirstChild(n); if e then e:Destroy() end
11753     end
11754     for _, n in ipairs({"LabEquipment", "LabElevators", "LabRoles", "LabWorkflow", "SeedLabWorkflow",
↪ "ServerBlasterManager"}) do
11755         local e = ServerScriptService:FindFirstChild(n); if e then e:Destroy() end
11756     end
11757     local modules = ReplicatedStorage:FindFirstChild("Modules")
11758     if modules and not modules:IsA("Folder") then modules:Destroy(); modules = nil end
11759     if not modules then
11760         modules = Instance.new("Folder")
11761         modules.Name = "Modules"
11762         modules.Parent = ReplicatedStorage
11763     end
11764     for _, moduleName in ipairs({"Blaster", "BlasterController"}) do
11765         local oldModule = modules:FindFirstChild(moduleName)
11766         if oldModule then oldModule:Destroy() end
11767     end
11768     -- Math-quiz console removed by request: destroy any old copy and never
11769     -- reinstall it.
11770     for _, clientName in ipairs({"LabQuizClient", "LabSidearmClient", "LabHUD"}) do
11771         local oldClient = StarterPlayer.StarterPlayerScripts:FindFirstChild(clientName)
11772         if oldClient then oldClient:Destroy() end
11773     end
11774     local oldFlow = ReplicatedStorage:FindFirstChild("LabFlow"); if oldFlow then oldFlow:Destroy() end
11775     local oldFlow2 = ReplicatedStorage:FindFirstChild("ISTA_Flow"); if oldFlow2 then oldFlow2:Destroy() end
11776
11777     local mod = Instance.new("ModuleScript"); mod.Name = "LabKinds"
11778     local s1 = Instance.new("Script"); s1.Name = "LabEquipment"
11779     local s2 = Instance.new("Script"); s2.Name = "LabElevators"
11780     local s3 = Instance.new("Script"); s3.Name = "LabRoles"
11781     local s4 = Instance.new("LocalScript"); s4.Name = "LabSidearmClient"
11782     local s5 = Instance.new("LocalScript"); s5.Name = "LabHUD"
11783     local blasterModule = Instance.new("ModuleScript"); blasterModule.Name = "Blaster"
11784     local controllerModule = Instance.new("ModuleScript"); controllerModule.Name = "BlasterController"
11785     local serverBlaster = Instance.new("Script"); serverBlaster.Name = "ServerBlasterManager"
11786
11787     local ok = pcall(function()
11788         mod.Source = FLOW_SOURCE
11789         s1.Source = EQUIP_SCRIPT_SOURCE

```



```

11790     s2.Source = ELEVATOR_SCRIPT_SOURCE
11791     s3.Source = ROLES_SCRIPT_SOURCE
11792     s4.Source = SIDEARM_CLIENT_SOURCE
11793     s5.Source = LAB_HUD_SOURCE
11794     blasterModule.Source = BLASTER_SOURCE
11795     controllerModule.Source = BLASTER_CONTROLLER_SOURCE
11796     serverBlaster.Source = SERVER_BLASTER_MANAGER_SOURCE
11797 end)
11798
11799 if ok then
11800     mod.Parent = ReplicatedStorage
11801     blasterModule.Parent = modules
11802     controllerModule.Parent = modules
11803     serverBlaster.Parent = ServerScriptService
11804     s1.Parent = ServerScriptService
11805     s2.Parent = ServerScriptService
11806     s3.Parent = ServerScriptService
11807     s4.Parent = StarterPlayer.StarterPlayerScripts
11808     s5.Parent = StarterPlayer.StarterPlayerScripts
11809     print("Installed lab runtime plus cross-platform Blaster, BlasterController and ServerBlasterManager.")
11810 else
11811     mod:Destroy(); s1:Destroy(); s2:Destroy(); s3:Destroy(); s4:Destroy(); s5:Destroy()
11812     blasterModule:Destroy(); controllerModule:Destroy(); serverBlaster:Destroy()
11813     if RunService.IsRunning() then
11814         startLiveRuntimeFallback()
11815     else
11816         warn("Could not write script source from this context. Run the file from the Studio Command Bar.")
11817     end
11818 end
11819 end
11820
11821 -----
11822 -- LOAD TIME
11823 -----
11824 -- The place is ~77,000 parts. Without streaming, a joining client has to
11825 -- download and build every one of them before it can move -- that is the
11826 -- bulk of the join wait. With streaming on, the server sends only what is
11827 -- near the player and fills the rest in as they walk, which is the single
11828 -- biggest thing available here for load time. These are place properties,
11829 -- so they are saved with the place and only settable from Edit.
11830 do
11831     local ok = pcall(function() Workspace.StreamingEnabled = true end)
11832     if ok then
11833         pcall(function() Workspace.StreamingMinRadius = 160 end)
11834         pcall(function() Workspace.StreamingTargetRadius = 512 end)
11835         pcall(function()
11836             Workspace.StreamingIntegrityMode = Enum.StreamingIntegrityMode.MinimumRadiusPause
11837         end)
11838         print("Streaming enabled for faster joins.")
11839     else
11840         warn("Could not enable streaming from this context; set Workspace.StreamingEnabled by hand.")
11841     end
11842 end
11843
11844 -----
11845 -- LIGHTING SERVICE
11846 -----
11847 if CONFIGURE_LIGHTING then
11848     local ok = pcall(function() Lighting.Technology = Enum.Technology.ShadowMap end)
11849     if not ok then warn("Set Lighting.Technology to ShadowMap or Future by hand.") end
11850     Lighting.GlobalShadows = true
11851     Lighting.Brightness = 1.3
11852     Lighting.ExposureCompensation = -0.35
11853     Lighting.Ambient = Color3.fromRGB(58, 60, 66)
11854     Lighting.OutdoorAmbient = Color3.fromRGB(92, 96, 104)
11855     Lighting.EnvironmentDiffuseScale = 0.35
11856     Lighting.EnvironmentSpecularScale = 0.25
11857     Lighting.ClockTime = 13.5
11858     Lighting.GeographicLatitude = 45
11859     for _, fx in ipairs(Lighting:GetChildren()) do
11860         if fx:IsA("BloomEffect") then
11861             fx.Intensity = math.min(fx.Intensity, 0.4)
11862             fx.Threshold = math.max(fx.Threshold, 1.2)
11863         end
11864     end
11865 end

```

```
11866
11867 local lamps, doors = 0, 0
11868 for _, d in ipairs(root:GetDescendants()) do
11869     if d:IsA("PointLight") then lamps = lamps + 1 end
11870     if d.Name:find("_leaf") then doors = doors + 1 end
11871 end
11872 local growthChamberCount = #CollectionService:GetTagged("PlantGrowthChamber")
11873 local workerBotCount = #CollectionService:GetTagged("LabWorkerBot")
11874 local expectedChambers = (#LEVELS + 1) * 50
11875 local expectedBots = (#LEVELS + 1) * BOTS_PER_LEVEL
11876 if growthChamberCount ~= expectedChambers then
11877     warn("BUILD CHECK FAILED: expected " .. expectedChambers .. " plant growth chambers, created " ..
        ↪ growthChamberCount .. ".")
11878 end
11879 if workerBotCount ~= expectedBots then
11880     warn("BUILD CHECK FAILED: expected " .. expectedBots .. " worker test NPCs, created " .. workerBotCount ..
        ↪ ".")
11881 end
11882 print(string.format(
11883     "Built: %.0f x %.0f studs -- %d floors (ground floor %d rooms, floors 2-%d %d rooms each), basement %d
        ↪ rooms -- %d equipment items including %d plant growth chambers, %d worker test NPCs, %d door leaves,
        ↪ stairs-only (no elevators), %d lights, %d parts.",
11884     X(W), X(L), #LEVELS, #GROUND_ROOMS, #LEVELS, #TYPICAL_ROOMS, #BASEMENT_ROOMS, placedCount,
        ↪ growthChamberCount,
11885     workerBotCount, doors, lamps, #root:GetDescendants()))
```

Part V

Complete historical workspace source

Chapter 58

W01 - SeedLabGenerator.lua

SeedLabGenerator.lua

Authority group: workspace. 552155 UTF-8 bytes; 12398 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1  =====
2  -- LABORATORY COMPLEX -- PROCEDURAL BUILDING GENERATOR
3  -- Basement + 3 upper floors + roof.
4  -- Rooms are numbered. Equipment is generated from a catalogue of
5  -- basic laboratory items and packed against the walls automatically.
6  -- Vertical circulation: one stair tower + three working elevators.
7  --
8  -- HOW TO USE
9  --   Roblox Studio -> View -> Command Bar -> paste -> Enter.
10 --   Edit-mode Command Bar use installs the permanent runtime scripts.
11 --   Running from a server Script during Play also works for that session
12 --   through the built-in live fallback near the end of this file.
13 =====
14
15 local Workspace          = game:GetService("Workspace")
16 local ServerScriptService = game:GetService("ServerScriptService")
17 local ReplicatedStorage  = game:GetService("ReplicatedStorage")
18 local CollectionService  = game:GetService("CollectionService")
19 local Lighting           = game:GetService("Lighting")
20 local RunService         = game:GetService("RunService")
21 local StarterPlayer      = game:GetService("StarterPlayer")
22
23 -----
24 -- SCALE
25 -----
26 local STUD              = 28    -- cm per stud at 1:1
27 -- The building's footprint (SCALE) is independent of equipment size
28 -- (FIT_SCALE below): equipment is always drawn at its literal real-world
29 -- size, and only the ROOM PLAN gets stretched by SCALE before converting
30 -- to studs. So a bigger SCALE spreads the same fixed-size equipment
31 -- further apart -- exactly how you make a floor feel roomier instead of
32 -- crowded, without equipment growing at all.
33 local SCALE             = 3.6    -- horizontal size of the plan vs. true 1:1 (3x the previous pass)
34 local HEIGHT_SCALE      = 1.4    -- vertical size vs. true 1:1 -- kept modest on purpose: this is what
35 -- was making the building read as an 8-story warehouse before, and
36 -- fixing "crowded" doesn't need taller ceilings, just more floor room
37 -- Equipment stays at TRUE (1:1) human scale regardless of how big the
38 -- rooms are blown up, so a balance or a fridge reads as roughly the
39 -- size it should be next to a normal-sized player -- not a giant box.
40 local FIT_SCALE         = 1
41 local FIT_HEIGHT        = 1
42
43 -----
44 -- PLAN DIMENSIONS (centimetres)
45 -----
46 local WALL_H           = 300
```

```

47 local SLAB      = 25
48 local F2F       = WALL_H + SLAB
49 local EXT_T     = 30
50 local INT_T     = 16
51 local DOOR_W    = 90
52 local DOOR_H    = 210
53 local WIDE_W    = 150      -- double doors
54 local WIN_SILL  = 90
55 local WIN_HEAD  = 220
56
57 local COUNTER_H = 90      -- equipment cm
58 -- Overall footprint and corridor position follow the reference floor
59 -- plans (one long room down the west side, individually-doored rooms
60 -- off a corridor to the east): roughly 12.8 m x 20.8 m per floor.
61 local W, L      = 1280, 2080
62 local CORR_X1, CORR_X2 = 500, 720
63
64 local RNG_SEED = 20260907
65
66 -----
67 -- LIGHTING
68 -----
69 local BUILD_LIGHTS      = true
70 local LIGHT_SPACING     = 40  -- in studs, against the now-true-scale building
71 local LIGHT_RANGE       = 16  -- was tuned for a 24x-oversized building; way too
72                               -- big now, which is most of why everything blew out white
73 local LIGHT_BRIGHTNESS = 0.6
74 local CONFIGURE_LIGHTING = true
75
76 -----
77 local C = {
78   slab      = Color3.fromRGB(224, 227, 226),
79   basement = Color3.fromRGB(120, 122, 125),
80   ext       = Color3.fromRGB(232, 232, 228),
81   int       = Color3.fromRGB(244, 244, 246),
82   corridor  = Color3.fromRGB(226, 231, 236),
83   basewall  = Color3.fromRGB(105, 110, 116),
84   pillar    = Color3.fromRGB(88, 92, 96),
85   stair     = Color3.fromRGB(168, 170, 172),
86   roof      = Color3.fromRGB(148, 150, 152),
87   glass     = Color3.fromRGB(180, 214, 230),
88   door      = Color3.fromRGB(120, 132, 146),
89   shaft     = Color3.fromRGB(150, 154, 160),
90   car       = Color3.fromRGB(206, 210, 214),
91 }
92
93 -- Stop a previous play-test runtime before creating replacement tagged
94 -- objects. Otherwise its CollectionService listeners can bind the new
95 -- equipment, then disappear when replaced, leaving dead interaction prompts.
96 for _, runtimeName in ipairs({"LabEquipment", "LabElevators", "LabRoles", "LabWorkflow", "SeedLabWorkflow",
97   ↳ "ServerBlasterManager"}) do
98   local previous = ServerScriptService:FindFirstChild(runtimeName)
99   if previous then previous:Destroy() end
100 end
101 for _, runtimeName in ipairs({"LabKinds", "LabFlow", "ISTA_Flow", "LabMathQuiz"}) do
102   local previous = ReplicatedStorage:FindFirstChild(runtimeName)
103   if previous then previous:Destroy() end
104 end
105 local previousModules = ReplicatedStorage:FindFirstChild("Modules")
106 if previousModules and previousModules:IsA("Folder") then
107   for _, moduleName in ipairs({"Blaster", "BlasterController"}) do
108     local previous = previousModules:FindFirstChild(moduleName)
109     if previous then previous:Destroy() end
110   end
111 end
112 for _, clientName in ipairs({"LabQuizClient", "LabSidearmClient", "LabHUD"}) do
113   local previousClient = StarterPlayer.StarterPlayerScripts:FindFirstChild(clientName)
114   if previousClient then previousClient:Destroy() end
115 end
116
117 -- Work that this generator does NOT own but which lives inside
118 -- Laboratory_Complex is carried across a rebuild instead of being wiped
119 -- with it. Rebuilding used to destroy the whole model outright, which
120 -- silently took hand-built content -- the site exterior and the office
121 -- fit-out -- with it every single time the generator was re-run.
122 -- Anything listed here, or any child carrying a PreserveOnRebuild

```

```

122 -- attribute, is detached first and re-attached to the fresh model.
123 -- SiteWorks and OfficeFitout used to be carried across a rebuild. They were
124 -- built for an earlier, differently-sized building, and measured against the
125 -- current one they no longer fit: NONE of the 5,541 office parts landed inside
126 -- the building footprint, and the exterior ran straight through the corridors.
127 -- Neither a translation nor a rotation lines them up (the room layout changed
128 -- between the two builds), so they are regenerated from the current W and L
129 -- instead. Carrying the old geometry would only reintroduce the overlap.
130 local SUPERSEDED_ON_REBUILD = {
131     SiteWorks = true,
132     OfficeFitout = true,
133 }
134
135 local PRESERVE_ON_REBUILD = {
136     -- add hand-built folder names here to carry them across a rebuild
137 }
138
139 local old = Workspace:FindFirstChild("Laboratory_Complex")
140 local preserved = {}
141 if old then
142     for _, child in ipairs(old:GetChildren()) do
143         if SUPERSEDED_ON_REBUILD[child.Name] then
144             child:Destroy() -- old geometry for a building that no longer exists
145         elseif PRESERVE_ON_REBUILD[child.Name] or child:GetAttribute("PreserveOnRebuild") then
146             child.Parent = Workspace -- park it outside the model being destroyed
147             table.insert(preserved, child)
148         end
149     end
150     old:Destroy()
151 end
152
153 local root = Instance.new("Model")
154 root.Name = "Laboratory_Complex"
155 root.Parent = Workspace
156
157 for _, child in ipairs(preserved) do
158     child:SetAttribute("PreserveOnRebuild", true) -- so it survives every later rebuild too
159     child.Parent = root
160 end
161 if #preserved > 0 then
162     print("Carried " .. #preserved .. " hand-built folder(s) across the rebuild.")
163 end
164
165 -----
166 -- UNITS
167 -----
168 local function X(v) return v * SCALE / STUD end -- plan cm -> studs, horizontal
169 local function Y(v) return v * HEIGHT_SCALE / STUD end -- plan cm -> studs, vertical
170 local function E(v) return v * FIT_SCALE / STUD end -- fit cm -> studs, horizontal
171 local function EH(v) return v * FIT_HEIGHT / STUD end -- fit cm -> studs, vertical
172 local function P(v) return v * FIT_SCALE / SCALE end -- fit cm -> plan cm
173
174 -----
175 -- DETERMINISTIC RANDOM (so a rebuild gives the same layout)
176 -----
177 local function newRng(seed)
178     local s = seed % 2147483647
179     if s <= 0 then s = s + 2147483646 end
180     local r = {}
181     function r.next() s = (s * 16807) % 2147483647; return s / 2147483646 end
182     function r.int(a, b) return a + math.floor(r.next() * (b - a + 1)) end
183     function r.pick(t) return t[r.int(1, #t)] end
184     function r.chance(p) return r.next() < p end
185     return r
186 end
187
188 -----
189 -- PRIMITIVES
190 -----
191 local function newPart(name, size, cframe, colour, material, parent)
192     local p = Instance.new("Part")
193     p.Name = name
194     p.Size = size
195     p.CFrame = cframe
196     p.Color = colour or C.int
197     p.Material = material or Enum.Material.SmoothPlastic

```

```

198     p.Anchored = true
199     p.TopSurface = Enum.SurfaceType.Smooth
200     p.BottomSurface = Enum.SurfaceType.Smooth
201     p.Parent = parent or root
202     return p
203 end
204
205 local function slab(parent, name, x1, z1, x2, z2, topY, thickness, colour, material)
206     thickness = thickness or SLAB
207     return newPart(name,
208         Vector3.new(X(math.abs(x2 - x1)), Y(thickness), X(math.abs(z2 - z1))),
209         CFrame.new(X((x1 + x2) / 2), Y(topY - thickness / 2), X((z1 + z2) / 2)),
210         colour or C.slab, material or Enum.Material.Concrete, parent)
211 end
212
213 -----
214 -- WALLS WITH OPENINGS
215 -----
216 local function wall(parent, name, x1, z1, x2, z2, thickness, baseY, height, openings, colour, material)
217     height = height or WALL_H
218     local alongX = (z1 == z2)
219     local len = alongX and math.abs(x2 - x1) or math.abs(z2 - z1)
220     local sgn = alongX and (x2 >= x1 and 1 or -1) or (z2 >= z1 and 1 or -1)
221     local n = 0
222
223     local function seg(s, e, y0, y1)
224         if e - s < 0.5 or y1 - y0 < 0.5 then return end
225         n = n + 1
226         local mid = (s + e) / 2
227         local px = alongX and (x1 + sgn * mid) or x1
228         local pz = alongX and z1 or (z1 + sgn * mid)
229         local size = alongX
230             and Vector3.new(X(e - s), Y(y1 - y0), X(thickness))
231             or Vector3.new(X(thickness), Y(y1 - y0), X(e - s))
232         newPart(name .. "_" .. n, size,
233             CFrame.new(X(px), Y(baseY + (y0 + y1) / 2), X(pz)),
234             colour or C.int, material or Enum.Material.Plaster, parent)
235     end
236
237     local ops = {}
238     for _, o in ipairs(openings or {}) do
239         table.insert(ops, {at = o.at, width = o.width, sill = o.sill or 0,
240             head = o.head or DOOR_H, glass = o.glass, leaf = o.leaf, mainEntrance = o.mainEntrance})
241     end
242     table.sort(ops, function(a, b) return a.at < b.at end)
243
244     local cursor = 0
245     for _, o in ipairs(ops) do
246         local a, b = math.max(cursor, 0, o.at), math.min(len, o.at + o.width)
247         if b <= a then continue end -- Never create backwards/zero-width reveals.
248         if a > cursor then seg(cursor, a, 0, height) end
249         if o.sill > 0 then seg(a, b, 0, o.sill) end
250         if o.head < height then seg(a, b, o.head, height) end
251
252         local mid = (a + b) / 2
253         local px = alongX and (x1 + sgn * mid) or x1
254         local pz = alongX and z1 or (z1 + sgn * mid)
255
256         if o.glass then
257             local size = alongX
258                 and Vector3.new(X(b - a), Y(o.head - o.sill), X(4))
259                 or Vector3.new(X(4), Y(o.head - o.sill), X(b - a))
260             local g = newPart(name .. "_glass" .. n, size,
261                 CFrame.new(X(px), Y(baseY + (o.sill + o.head) / 2), X(pz)),
262                 C.glass, Enum.Material.Glass, parent)
263             g.Transparency = 0.55
264             g:SetAttribute("BreakableWindow", true)
265             CollectionService.AddTag(g, "LabBreakableWindow")
266         elseif o.leaf then
267             -- The surrounding structural wall already frames the door; no freestanding leaves.
268
269             -- The actual functional door: a solid panel filling the opening
270             -- itself. It starts closed (CanCollide true) and is tagged
271             -- AutoDoor so the runtime slides it up into the header void when
272             -- a player approaches, and back down once they've passed through.
273             local doorHeight = math.max(o.head - o.sill - 6, 20)

```

```

274     local doorThickness = math.max(thickness * 0.6, 5)
275     local doorSize
276     if alongX then
277         doorSize = Vector3.new(X(b - a - 6), Y(doorHeight), X(doorThickness))
278     else
279         doorSize = Vector3.new(X(doorThickness), Y(doorHeight), X(b - a - 6))
280     end
281     local closedY = Y(baseY + o.sill + doorHeight / 2 + 3)
282     -- Automatic doors get their own industrial blue/metal look (never
283     -- the same colour/material as the wall or the decorative casing)
284     -- plus black-and-yellow hazard striping along the leading edge and
285     -- an "AUTOMATIC DOOR" surface label, so they read as a functional
286     -- door rather than a wall panel even before they start moving.
287     local autoDoor = newPart(name .. "_autodoor" .. n, doorSize,
288         CFrame.new(X(px), closedY, X(pz)), Color3.fromRGB(58, 104, 158), Enum.Material.Metal, parent)
289     autoDoor.CanCollide = true
290     autoDoor:SetAttribute("MainEntrance", o.mainEntrance == true)
291     -- Doors slide sideways into the wall pocket beside the opening
292     -- (left-to-right along the wall's own run), not up/down -- a
293     -- vertical slide read as flicker when it snapped through the
294     -- header void. The slide distance is the door's own width so it
295     -- tucks fully out of the opening.
296     -- IMPORTANT: these are stored in STUDS (via X()), matching the
297     -- part's actual CFrame units -- storing raw plan-cm values here
298     -- (as an earlier pass did) made the door's "closed"/"open"
299     -- targets read as almost-zero stud offsets, so the proximity
300     -- check never found a player "close enough" and the door never
301     -- moved at all.
302     autoDoor:SetAttribute("DoorAxis", alongX and "X" or "Z")
303     local closedXStud, closedZStud = X(px), X(pz)
304     autoDoor:SetAttribute("DoorClosedX", closedXStud)
305     autoDoor:SetAttribute("DoorClosedZ", closedZStud)
306     local slideDistStud = X(b - a) * 0.96
307     if alongX then
308         autoDoor:SetAttribute("DoorOpenX", closedXStud + slideDistStud)
309         autoDoor:SetAttribute("DoorOpenZ", closedZStud)
310     else
311         autoDoor:SetAttribute("DoorOpenX", closedXStud)
312         autoDoor:SetAttribute("DoorOpenZ", closedZStud + slideDistStud)
313     end
314     autoDoor:SetAttribute("DoorY", closedY)
315     autoDoor:SetAttribute("DoorOpen", false)
316     CollectionService.AddTag(autoDoor, "AutoDoor")
317
318     -- Hazard stripes, the "AUTOMATIC DOOR" label, and the open/closed
319     -- indicator light are all drawn as SurfaceGui elements parented to
320     -- the door part itself (rather than separate 3D parts), so they
321     -- track the door perfectly when the runtime tweens it open/closed.
322     local labelFaces = alongX and {Enum.NormalId.Front, Enum.NormalId.Back}
323     or {Enum.NormalId.Left, Enum.NormalId.Right}
324     for _, normal in ipairs(labelFaces) do
325         local labelGui = Instance.new("SurfaceGui")
326         labelGui.Name = "AutoDoorLabel"
327         labelGui.Face = normal
328         labelGui.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
329         labelGui.PixelsPerStud = 60
330         labelGui.AlwaysOnTop = false
331         labelGui.Parent = autoDoor
332
333         local topStripe = Instance.new("Frame")
334         topStripe.Name = "HazardStripeTop"
335         topStripe.Size = UDim2.new(1, 0, 0.07, 0)
336         topStripe.Position = UDim2.new(0, 0, 0, 0)
337         topStripe.BackgroundColor3 = Color3.fromRGB(235, 195, 35)
338         topStripe.BorderSizePixel = 0
339         topStripe.Parent = labelGui
340         local bottomStripe = topStripe:Clone()
341         bottomStripe.Name = "HazardStripeBottom"
342         bottomStripe.Position = UDim2.new(0, 0, 0.93, 0)
343         bottomStripe.Parent = labelGui
344
345         local labelText = Instance.new("TextLabel")
346         labelText.Name = "Caption"
347         labelText.Size = UDim2.new(0.86, 0, 0.24, 0)
348         labelText.Position = UDim2.new(0.07, 0, 0.13, 0)
349         labelText.BackgroundColor3 = Color3.fromRGB(235, 195, 35)

```



```

350         labelText.BackgroundTransparency = 0.05
351         labelText.BorderSizePixel = 0
352         labelText.Text = o.mainEntrance and "ENTRANCE" or "AUTOMATIC DOOR"
353         labelText.TextColor3 = Color3.fromRGB(20, 20, 20)
354         labelText.TextScaled = true
355         labelText.Font = Enum.Font.SourceSansBold
356         labelText.Parent = labelGui
357
358         local indicator = Instance.new("Frame")
359         indicator.Name = "DoorIndicator"
360         indicator.Size = UDim2.new(0.16, 0, 0.06, 0)
361         indicator.Position = UDim2.new(0.42, 0, 0.42, 0)
362         indicator.BackgroundColor3 = Color3.fromRGB(210, 40, 40)
363         indicator.BorderSizePixel = 0
364         indicator.Parent = labelGui
365     end
366 end
367     cursor = math.max(cursor, b)
368 end
369     if cursor < len then seg(cursor, len, 0, height) end
370 end
371
372 -----
373 -- ROOM SCHEDULE -- numbered rooms
374 --
375 -- Matches the shape of the reference floor plans: one long room runs
376 -- the full length of the west side, and a cluster of individually
377 -- doored rooms line the east side of a central corridor. Room COUNT
378 -- varies by level, following the reference drawings: 10 on the ground
379 -- floor, 9 on the (identical) upper floors, 8 in the basement.
380 -----
381
382 -- Build the room list with every opening it owns.
383 -- namePrefix - nil for "Room 1".. "Room N", or e.g. "B" for basement
384 --             "Room B1".. "Room BN"
385 -- rightCount - how many individually-doored rooms line the east side
386 -- hasExterior - false suppresses exterior windows/exit doors (basement)
387 local function buildRoomSchedule(namePrefix, rightCount, hasExterior)
388     if hasExterior == nil then hasExterior = true end
389     local rooms, n = {}, 0
390
391     local function add(sideOfPlan, x1, x2, band)
392         n = n + 1
393         local label = namePrefix and (namePrefix .. n) or tostring(n)
394         local r = {
395             number = n, name = "Room " .. label, plan = sideOfPlan,
396             x1 = x1, x2 = x2, z1 = band[1], z2 = band[2],
397             open = {N = {}, S = {}, E = {}, W = {}},
398         }
399         rooms[n] = r
400         return r
401     end
402
403     add("L", 0, CORR_X1, {0, L})
404     local firstRight = n + 1
405     for i = 1, rightCount do
406         add("R", CORR_X2, W, {L * (i - 1) / rightCount, L * i / rightCount})
407     end
408
409     -- doors
410     for _, r in ipairs(rooms) do
411         local depth = r.z2 - r.z1
412         local corridorSide = (r.plan == "L") and "E" or "W"
413
414         -- corridor doors: one, or two for a deep room
415         if depth <= 300 then
416             table.insert(r.open[corridorSide], {at = depth / 2 - D00R_W / 2, width = D00R_W, leaf = true})
417         else
418             table.insert(r.open[corridorSide], {at = depth * 0.28 - D00R_W / 2, width = D00R_W, leaf = true})
419             table.insert(r.open[corridorSide], {at = depth * 0.72 - WIDE_W / 2, width = WIDE_W, leaf = true})
420         end
421
422         -- exterior wall: windows, plus an exit door on the deeper rooms
423         -- (skipped below grade, in the basement)
424         if hasExterior then
425             local extSide = (r.plan == "L") and "W" or "E"

```

```

426         if depth >= 300 then
427             table.insert(r.open[extSide], {at = depth * 0.12 - D00R_W / 2, width = D00R_W, sill = 20, head
428                 ↪ = 230, glass = true})
429             table.insert(r.open[extSide], {at = depth * 0.45 - 90, width = 180, sill = WIN_SILL, head =
430                 ↪ WIN_HEAD, glass = true})
431             table.insert(r.open[extSide], {at = depth * 0.78 - 90, width = 180, sill = WIN_SILL, head =
432                 ↪ WIN_HEAD, glass = true})
433         else
434             table.insert(r.open[extSide], {at = depth * 0.5 - 55, width = 110, sill = WIN_SILL, head =
435                 ↪ WIN_HEAD, glass = true})
436         end
437     end
438     r.extDoorDepth = depth
439 end
440
441 -- interconnecting doors between consecutive east-side rooms
442 -- (the west side is a single room, so it needs none)
443 local function link(a, b)
444     local at = (a.x2 - a.x1) * 0.32 - D00R_W / 2
445     local op = {at = at, width = D00R_W, leaf = true}
446     table.insert(a.open.S, op)
447     table.insert(b.open.N, {at = at, width = D00R_W, leaf = true})
448 end
449 for i = firstRight, n - 1 do link(rooms[i], rooms[i + 1]) end
450
451 -- The building shell defines its north/south doors and windows separately
452 -- from the room schedule. Copy those openings into the rooms that touch
453 -- each end wall so wall packing can never put a cabinet across one.
454 if hasExterior then
455     local northOpenings = {
456         {at = CORR_X1 + 20, width = 160},
457         {at = W * 0.12, width = 140, glass = true},
458         {at = W * 0.77, width = 140, glass = true},
459     }
460     local southOpenings = {
461         {at = CORR_X1 + 25, width = WIDE_W},
462         {at = W * 0.12, width = 90},
463         {at = W * 0.20, width = 150, glass = true},
464         {at = W * 0.50, width = 150, glass = true},
465         {at = W * 0.80, width = 90},
466     }
467     local function copyEndOpenings(room, side, openings)
468         for _, opening in ipairs(openings) do
469             local left = math.max(room.x1, opening.at)
470             local right = math.min(room.x2, opening.at + opening.width)
471             if right > left then
472                 table.insert(room.open[side], {
473                     at = left - room.x1,
474                     width = right - left,
475                     glass = opening.glass,
476                 })
477             end
478         end
479     end
480     for _, room in ipairs(rooms) do
481         if room.z1 == 0 then copyEndOpenings(room, "N", northOpenings) end
482         if room.z2 == L then copyEndOpenings(room, "S", southOpenings) end
483     end
484 end
485
486 return rooms, firstRight
487 end
488
489 local GROUND_ROOMS = buildRoomSchedule(nil, 9, true) -- "Plan parter": 10 rooms
490 local TYPICAL_ROOMS = buildRoomSchedule(nil, 8, true) -- "Plan etaj": 9 rooms, reused on floors 2 and 3
491 local BASEMENT_ROOMS = buildRoomSchedule("B", 7, false) -- "Plan demisol": 8 rooms, no exterior windows
492
493 -----
494 -- SIGNAGE AND LIGHT
495 -----
496 local function sign(parent, text, xcm, zcm, baseY, px, maxDist)
497     local a = newPart("Sign", Vector3.new(1, 1, 1),
498         CFrame.new(X(xcm), Y(baseY + WALL_H * 0.72), X(zcm)), C.int, Enum.Material.SmoothPlastic, parent)
499     a.Transparency = 1
500     a.CanCollide = false
501     local bb = Instance.new("BillboardGui")

```

```

498     bb.Size = UDim2.new(0, px or 300, 0, (px or 300) * 0.32)
499     -- World labels only appear when the player is close. Long-range,
500     -- always-on-top labels stack over one another in a furnished room.
501     bb.MaxDistance = math.min(maxDist or 18, 22)
502     bb.AlwaysOnTop = false
503     bb.Parent = a
504     local t = Instance.new("TextLabel")
505     t.Size = UDim2.new(1, 0, 1, 0)
506     t.BackgroundTransparency = 1
507     t.Text = text
508     t.TextColor3 = Color3.fromRGB(35, 40, 48)
509     t.TextStrokeTransparency = 0.55
510     t.TextStrokeColor3 = Color3.new(1, 1, 1)
511     t.TextScaled = true
512     t.Font = Enum.Font.GothamBold
513     t.Parent = bb
514     return a
515 end
516
517 -- A stationary lab-staff mannequin, built from primitive parts at true
518 -- human scale (independent of the building's SCALE/HEIGHT_SCALE, same
519 -- as equipment) so it always reads as a person standing in the room.
520 -- Tagged "LabNPC" for the runtime script to arm with a "Talk" prompt.
521 local function buildNPC(parent, npcName, role, xcm, zcm, y0, badgeColour, tips)
522     -- Medical/lab-coat outfit: white coat and trousers for every NPC,
523     -- with a small coloured badge and a red cross on the chest so it
524     -- reads as clinical/lab staff rather than a random passer-by.
525     local skin = Color3.fromRGB(224, 189, 154)
526     local coat = Color3.fromRGB(246, 247, 249)
527     local trouser = Color3.fromRGB(210, 213, 217)
528     local baseY = Y(y0)
529
530     local legL = newPart(npcName .. "_LegL", Vector3.new(0.8, 2.3, 0.8),
531         CFrame.new(X(xcm) - 0.42, baseY + 1.15, X(zcm)), trouser, Enum.Material.Fabric, parent)
532     local legR = newPart(npcName .. "_LegR", Vector3.new(0.8, 2.3, 0.8),
533         CFrame.new(X(xcm) + 0.42, baseY + 1.15, X(zcm)), trouser, Enum.Material.Fabric, parent)
534     local torso = newPart(npcName .. "_Torso", Vector3.new(2, 2.1, 1.1),
535         CFrame.new(X(xcm), baseY + 3.35, X(zcm)), coat, Enum.Material.Fabric, parent)
536     local armL = newPart(npcName .. "_ArmL", Vector3.new(0.65, 2, 0.65),
537         CFrame.new(X(xcm) - 1.35, baseY + 3.3, X(zcm)), coat, Enum.Material.Fabric, parent)
538     local armR = newPart(npcName .. "_ArmR", Vector3.new(0.65, 2, 0.65),
539         CFrame.new(X(xcm) + 1.35, baseY + 3.3, X(zcm)), coat, Enum.Material.Fabric, parent)
540     local head = newPart(npcName .. "_Head", Vector3.new(1.25, 1.25, 1.25),
541         CFrame.new(X(xcm), baseY + 4.95, X(zcm)), skin, Enum.Material.SmoothPlastic, parent)
542     head.Shape = Enum.PartType.Ball
543
544     -- badge (role colour) and a small red cross, both worn on the chest
545     local badge = newPart(npcName .. "_Badge", Vector3.new(0.32, 0.42, 0.06),
546         CFrame.new(X(xcm) - 0.55, baseY + 3.55, X(zcm) - 0.58), badgeColour, Enum.Material.SmoothPlastic,
547         ↪ parent)
548     local crossV = newPart(npcName .. "_CrossV", Vector3.new(0.1, 0.32, 0.06),
549         CFrame.new(X(xcm) + 0.4, baseY + 3.45, X(zcm) - 0.58), Color3.fromRGB(200, 40, 40),
550         ↪ Enum.Material.SmoothPlastic, parent)
551     local crossH = newPart(npcName .. "_CrossH", Vector3.new(0.32, 0.1, 0.06),
552         CFrame.new(X(xcm) + 0.4, baseY + 3.45, X(zcm) - 0.58), Color3.fromRGB(200, 40, 40),
553         ↪ Enum.Material.SmoothPlastic, parent)
554
555     for _, p in ipairs({legL, legR, torso, armL, armR, head, badge, crossV, crossH}) do
556         p.CanCollide = false
557         p.CastShadow = true
558     end
559
560     CollectionService.AddTag(torso, "LabNPC")
561     torso:SetAttribute("NPCName", npcName)
562     torso:SetAttribute("Role", role)
563     torso:SetAttribute("Tips", table.concat(tips, "|"))
564
565     local bb = Instance.new("BillboardGui")
566     bb.Size = UDim2.new(0, 170, 0, 40)
567     bb.StudsOffset = Vector3.new(0, 3.2, 0)
568     bb.MaxDistance = 14
569     bb.AlwaysOnTop = false
570     bb.Parent = torso
571     local lbl = Instance.new("TextLabel")
572     lbl.Size = UDim2.new(1, 0, 1, 0)
573     lbl.BackgroundTransparency = 1

```

```

571     lbl.TextColor3 = Color3.fromRGB(255, 255, 255)
572     lbl.TextStrokeTransparency = 0.4
573     lbl.TextScaled = true
574     lbl.Font = Enum.Font.GothamBold
575     lbl.Text = npcName .. "\n" .. role
576     lbl.Parent = bb
577
578     return torso
579 end
580
581 -- Lightweight, server-driven worker models. They stay anchored and are
582 -- moved along navigation waypoints by the runtime, which lets fifty test
583 -- workers operate without the physics cost of fifty full player rigs.
584 -- Desk, chair and monitor for a seated worker. Built as separate furniture
585 -- rather than as part of the bot, so that killing the worker leaves the
586 -- workstation standing instead of tipping a desk over with the body.
587 local function buildWorkstation(parent, seatCF, name)
588     local metal = Color3.fromRGB(58, 62, 70)
589     local function piece(pieceName, size, offset, colour, material)
590         local part = newPart(name .. "_" .. pieceName, size, seatCF * offset, colour, material, parent)
591         part.CanCollide = false -- never block a walking route
592         part.CanTouch = false
593         return part
594     end
595     -- the bot faces -Z, so the desk sits at negative Z and the chair behind it
596     piece("ChairSeat", Vector3.new(1.7, 0.2, 1.7), CFrame.new(0, -1.10, 0.40), metal, Enum.Material.Metal)
597     piece("ChairBack", Vector3.new(1.7, 1.5, 0.2), CFrame.new(0, -0.35, 0.95), metal, Enum.Material.Metal)
598     piece("ChairStem", Vector3.new(0.25, 1.3, 0.25), CFrame.new(0, -1.78, 0.40), Color3.fromRGB(48, 52, 58),
599         ⇨ Enum.Material.Metal)
600     piece("DeskTop", Vector3.new(3.4, 0.18, 1.9), CFrame.new(0, -0.45, -1.90), Color3.fromRGB(198, 188, 170),
601         ⇨ Enum.Material.WoodPlanks)
602     piece("DeskLegL", Vector3.new(0.2, 2.0, 0.2), CFrame.new(-1.50, -1.45, -1.90), metal, Enum.Material.Metal)
603     piece("DeskLegR", Vector3.new(0.2, 2.0, 0.2), CFrame.new(1.50, -1.45, -1.90), metal, Enum.Material.Metal)
604     piece("Monitor", Vector3.new(1.5, 0.95, 0.12), CFrame.new(0, 0.22, -2.35), Color3.fromRGB(28, 32, 38),
605         ⇨ Enum.Material.SmoothPlastic)
606     piece("MonitorStand", Vector3.new(0.35, 0.5, 0.3), CFrame.new(0, -0.28, -2.35), metal, Enum.Material.Metal)
607 end
608
609 local function buildWorkerBot(parent, botNumber, xcm, zcm, y0, seated)
610     local model = Instance.new("Model")
611     local isCompetitor = botNumber == 14
612     model.Name = isCompetitor and "RivalScientist" or string.format("WorkerBot_%02d", botNumber)
613     model.Parent = parent
614     local roleNames = {"Technician", "Laboratory Technician"}
615     local roleColours = {
616         Color3.fromRGB(68, 135, 220),
617         Color3.fromRGB(75, 175, 105),
618     }
619     local roleIndex = ((botNumber - 1) % #roleNames) + 1
620     local baseY = Y(y0)
621     -- Stable identities: 15 given names x 6 surnames for each gender. The
622     -- 180 workers therefore have 180 different names, exactly 90 of each.
623     local isGirl = botNumber % 2 == 0
624     local identity = math.floor((botNumber - 1) / 2)
625     local givenNames = isGirl
626         and {"Ana", "Ioana", "Maria", "Elena", "Andreea", "Alexandra", "Daria", "Irina", "Sofia", "Teodora",
627             ⇨ "Raluca", "Cristina", "Gabriela", "Bianca", "Mihaela"}
628         or {"Andrei", "Mihai", "Alexandru", "Radu", "Stefan", "Vlad", "Matei", "Gabriel", "David", "Tudor",
629             ⇨ "Razvan", "Ionut", "Cristian", "Darius", "Florin"}
630     local surnames = {"Popescu", "Ionescu", "Dumitrescu", "Stan", "Marin", "Georgescu"}
631     local displayName = givenNames[identity % #givenNames + 1] .. " " .. surnames[math.floor(identity /
632         ⇨ #givenNames) % #surnames + 1]
633     local coats = {Color3.fromRGB(28, 31, 39), Color3.fromRGB(44, 60, 87), Color3.fromRGB(87, 40, 49),
634         ⇨ Color3.fromRGB(49, 75, 65), Color3.fromRGB(78, 59, 92), Color3.fromRGB(178, 157, 121)}
635     local skins = {Color3.fromRGB(241, 207, 174), Color3.fromRGB(225, 183, 143), Color3.fromRGB(201, 155, 115),
636         ⇨ Color3.fromRGB(170, 123, 88), Color3.fromRGB(233, 195, 159)}
637     local hairs = {Color3.fromRGB(74, 40, 20), Color3.fromRGB(121, 70, 30), Color3.fromRGB(31, 25, 23),
638         ⇨ Color3.fromRGB(193, 153, 75), Color3.fromRGB(137, 57, 29)}
639     local coat = coats[identity % #coats + 1]
640     local skin = skins[math.floor(identity / 3) % #skins + 1]
641     local hair = hairs[math.floor(identity / 2) % #hairs + 1]
642     local shirt = Color3.fromHSV((botNumber * 0.618034) % 1, 0.48, 0.82)
643     local centre = Vector3.new(X(xcm), baseY, X(zcm))
644     local function botPart(name, size, offset, colour, material)
645         local part = newPart(name, size, CFrame.new(centre + offset), colour, material, model)
646         part.CanCollide = false

```

```

638     part.CanTouch = false
639     -- Queryable on purpose. These were CanQuery = false back when nothing
640     -- ever needed to hit-test a worker, which meant every shot fired at one
641     -- passed straight through it into the wall behind. A worker has to be
642     -- raycast-visible to be a target.
643     part.CanQuery = true
644     return part
645 end
646
647 local torso, head
648 local trouser = Color3.fromRGB(62, 72, 86)
649 if seated then
650     -- Desk pose: thighs forward, shins down, body dropped to chair height.
651     -- The whole model faces -Z, which is also its LookVector, so the caller
652     -- can simply PivotTo(CFrame.lookAt(seat, desk)) to sit it at a desk.
653     torso = botPart("Torso", Vector3.new(1.8, 2.0, 1.0), Vector3.new(0, 2.45, 0), coat,
654         ↳ Enum.Material.Fabric)
655     botPart("LegL", Vector3.new(0.7, 0.7, 1.7), Vector3.new(-0.42, 1.55, -0.75), trouser,
656         ↳ Enum.Material.Fabric)
657     botPart("LegR", Vector3.new(0.7, 0.7, 1.7), Vector3.new(0.42, 1.55, -0.75), trouser,
658         ↳ Enum.Material.Fabric)
659     botPart("ShinL", Vector3.new(0.7, 1.45, 0.7), Vector3.new(-0.42, 0.72, -1.5), trouser,
660         ↳ Enum.Material.Fabric)
661     botPart("ShinR", Vector3.new(0.7, 1.45, 0.7), Vector3.new(0.42, 0.72, -1.5), trouser,
662         ↳ Enum.Material.Fabric)
663     botPart("Arml", Vector3.new(0.58, 0.58, 1.6), Vector3.new(-1.05, 2.6, -0.7), coat,
664         ↳ Enum.Material.Fabric)
665     botPart("ArmR", Vector3.new(0.58, 0.58, 1.6), Vector3.new(1.05, 2.6, -0.7), coat, Enum.Material.Fabric)
666     head = botPart("Head", Vector3.new(1.1, 1.1, 1.1), Vector3.new(0, 3.95, 0), skin,
667         ↳ Enum.Material.SmoothPlastic)
668     botPart("RoleBadge", Vector3.new(0.38, 0.42, 0.08), Vector3.new(-0.5, 2.68, -0.54),
669         ↳ roleColours[roleIndex], Enum.Material.Neon)
670 else
671     torso = botPart("Torso", Vector3.new(1.8, 2.0, 1.0), Vector3.new(0, 3.25, 0), coat,
672         ↳ Enum.Material.Fabric)
673     botPart("LegL", Vector3.new(0.7, 2.2, 0.7), Vector3.new(-0.42, 1.1, 0), trouser, Enum.Material.Fabric)
674     botPart("LegR", Vector3.new(0.7, 2.2, 0.7), Vector3.new(0.42, 1.1, 0), trouser, Enum.Material.Fabric)
675     botPart("Arml", Vector3.new(0.58, 1.9, 0.58), Vector3.new(-1.18, 3.2, 0), coat, Enum.Material.Fabric)
676     botPart("ArmR", Vector3.new(0.58, 1.9, 0.58), Vector3.new(1.18, 3.2, 0), coat, Enum.Material.Fabric)
677     head = botPart("Head", Vector3.new(1.1, 1.1, 1.1), Vector3.new(0, 4.75, 0), skin,
678         ↳ Enum.Material.SmoothPlastic)
679     botPart("RoleBadge", Vector3.new(0.38, 0.42, 0.08), Vector3.new(-0.5, 3.48, -0.54),
680         ↳ roleColours[roleIndex], Enum.Material.Neon)
681 end
682
683 -- Block-avatar styling, like the player: hair, smiling face, an open
684 -- jacket over an individual shirt, jeans and white-soled trainers.
685 local headY = seated and 3.95 or 4.75
686 local chestY = seated and 2.45 or 3.25
687 botPart("Shirt", Vector3.new(0.80, 1.68, 0.06), Vector3.new(0, chestY, -0.535), shirt,
688     ↳ Enum.Material.Fabric)
689 botPart("JacketZip", Vector3.new(0.055, 1.65, 0.07), Vector3.new(0, chestY, -0.575), Color3.fromRGB(175,
690     ↳ 181, 184), Enum.Material.Metal)
691 for _, side in ipairs({-1, 1}) do
692     local shoeZ = seated and -1.60 or -0.10
693     botPart("Trainer" .. side, Vector3.new(0.75, 0.30, 0.95), Vector3.new(side * 0.42, 0.16, shoeZ), coat,
694         ↳ Enum.Material.Fabric)
695     botPart("Sole" .. side, Vector3.new(0.77, 0.10, 0.97), Vector3.new(side * 0.42, 0.055, shoeZ),
696         ↳ Color3.fromRGB(235, 236, 228), Enum.Material.SmoothPlastic)
697 end
698 botPart("HairCrown", Vector3.new(1.20, 0.32, 1.16), Vector3.new(0, headY + 0.51, 0.04), hair,
699     ↳ Enum.Material.SmoothPlastic)
700 botPart("HairFringe", Vector3.new(0.76, 0.36, 0.19), Vector3.new((identity % 2 == 0) and -0.20 or 0.20,
701     ↳ headY + 0.34, -0.53), hair, Enum.Material.SmoothPlastic)
702 if isGirl then
703     local style = identity % 3
704     botPart("HairBack", Vector3.new(1.14, style == 0 and 1.55 or 0.94, 0.24), Vector3.new(0, headY - (style
705         ↳ == 0 and 0.25 or 0.02), 0.55), hair, Enum.Material.SmoothPlastic)
706     if style == 1 then
707         local pony = botPart("Ponytail", Vector3.new(0.46, 1.25, 0.56), Vector3.new(0, headY - 0.08, 0.88),
708             ↳ hair, Enum.Material.SmoothPlastic)
709         pony.Shape = Enum.PartType.Ball
710     elseif style == 2 then
711         local bun = botPart("HairBun", Vector3.new(0.65, 0.65, 0.60), Vector3.new(0, headY + 0.48, 0.64),
712             ↳ hair, Enum.Material.SmoothPlastic)
713         bun.Shape = Enum.PartType.Ball
714     end
715 end

```

```

694     else
695         botPart("HairBack", Vector3.new(1.16, 0.58, 0.22), Vector3.new(0, headY + 0.16, 0.54), hair,
        ↪ Enum.Material.SmoothPlastic)
696         if identity % 3 == 0 then
697             botPart("HairQuiff", Vector3.new(0.42, 0.22, 0.68), Vector3.new(0.17, headY + 0.75, -0.10), hair,
            ↪ Enum.Material.SmoothPlastic)
698         end
699     end
700     local face = Instance.new("Decal")
701     face.Name = "FriendlyFace"
702     face.Texture = "rbxasset://textures/face.png"
703     face.Face = Enum.NormalId.Front
704     face.Parent = head
705     -- A single compact hull prevents players walking through the worker. Its
706     -- navmesh modifier lets the worker plan from inside its own hull; movement
707     -- still explicitly checks all other workers and players before every step.
708     local hull = botPart("WorkerCollisionHull", Vector3.new(2.95, seated and 3.6 or 4.9, seated and 2.5 or
        ↪ 1.70),
709         Vector3.new(0, seated and 2.1 or 2.7, seated and -0.50 or 0), coat, Enum.Material.SmoothPlastic)
710     hull.Transparency = 1
711     hull.CanCollide = true
712     hull.CastShadow = false
713     local navigation = Instance.new("PathfindingModifier")
714     navigation.Name = "DynamicWorkerObstacle"
715     navigation.PassThrough = true
716     navigation.Parent = hull
717
718     model.PrimaryPart = torso
719     model:SetAttribute("BotNumber", botNumber)
720     model:SetAttribute("DisplayName", displayName)
721     model:SetAttribute("Gender", isGirl and "Girl" or "Boy")
722     model:SetAttribute("AppearanceId", botNumber)
723     model:SetAttribute("Role", roleNames[roleIndex])
724     model:SetAttribute("Level", y0 == -F2F and "Basement" or ("Level_" .. (math.floor(y0 / F2F) + 1)))
725     model:SetAttribute("TaskState", "Waiting for plant task")
726     model:SetAttribute("CompletedCycles", 0)
727     model:SetAttribute("Competitor", isCompetitor)
728     model:SetAttribute("Score", 0)
729     model:SetAttribute("Health", 100)
730     model:SetAttribute("MaxHealth", 100)
731     model:SetAttribute("KnockedOut", false)
732     CollectionService.AddTag(model, "LabWorkerBot")
733
734     local gui = Instance.new("BillboardGui")
735     gui.Name = "BotStatus"
736     gui.Size = UDim2.fromOffset(172, 64)
737     gui.StudsOffset = Vector3.new(0, 3.0, 0)
738     -- Was 12 studs, which meant you had to stand almost on top of a bot to
739     -- see anything. The health bar is only useful if it reads across a room.
740     gui.MaxDistance = 70
741     gui.AlwaysOnTop = false
742     gui.Parent = head
743     local text = Instance.new("TextLabel")
744     text.Name = "Status"
745     text.Size = UDim2.fromScale(1, 0.5)
746     text.BackgroundTransparency = 1
747     text.TextColor3 = Color3.fromRGB(235, 245, 255)
748     text.TextStrokeTransparency = 0.35
749     text.TextScaled = true
750     text.Font = Enum.Font.GothamBold
751     text.Text = displayName .. "\n" .. (isCompetitor and "Rival scientist" or roleNames[roleIndex])
752     text.Parent = gui
753
754     -- Health bar under the caption. refreshBotHealth() has always looked for
755     -- children named HealthBarFill and HealthText and updated them -- but
756     -- nothing ever created them, so it searched, found nothing and silently
757     -- did nothing. A damaged bot never showed any damage.
758     local barBack = Instance.new("Frame")
759     barBack.Name = "HealthBarBack"
760     barBack.Size = UDim2.new(0.88, 0, 0.18, 0)
761     barBack.Position = UDim2.new(0.06, 0, 0.54, 0)
762     barBack.BackgroundColor3 = Color3.fromRGB(22, 26, 32)
763     barBack.BackgroundTransparency = 0.2
764     barBack.BorderSizePixel = 0
765     barBack.Parent = gui
766     local barFill = Instance.new("Frame")

```

```

767     barFill.Name = "HealthBarFill"
768     barFill.Size = UDim2.fromScale(1, 1)
769     barFill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
770     barFill.BorderSizePixel = 0
771     barFill.Parent = barBack
772     local healthText = Instance.new("TextLabel")
773     healthText.Name = "HealthText"
774     healthText.Size = UDim2.new(1, 0, 0.26, 0)
775     healthText.Position = UDim2.new(0, 0, 0.74, 0)
776     healthText.BackgroundTransparency = 1
777     healthText.TextColor3 = Color3.fromRGB(214, 232, 244)
778     healthText.TextStrokeTransparency = 0.4
779     healthText.TextScaled = true
780     healthText.Font = Enum.Font.Gotham
781     healthText.Text = "LIFE 100 / 100"
782     healthText.Parent = gui
783
784     return model
785 end
786
787 local function lamp(parent, xcm, zcm, baseY, brightness)
788     if not BUILD_LIGHTS then return end
789     local p = newPart("Luminaire", Vector3.new(X(120), Y(8), X(30)),
790         CFrame.new(X(xcm), Y(baseY + WALL_H - 14), X(zcm)),
791         Color3.fromRGB(250, 250, 244), Enum.Material.SmoothPlastic, parent)
792     p.CanCollide = false
793     p.CastShadow = false
794     local l = Instance.new("PointLight")
795     l.Range = LIGHT_RANGE
796     l.Brightness = (brightness or 1) * LIGHT_BRIGHTNESS
797     l.Color = Color3.fromRGB(255, 250, 236)
798     l.Shadows = false
799     l.Parent = p
800 end
801
802 local function lightGrid(parent, x1, z1, x2, z2, baseY, brightness)
803     local nx = math.max(1, math.floor(X(math.abs(x2 - x1)) / LIGHT_SPACING + 0.5))
804     local nz = math.max(1, math.floor(X(math.abs(z2 - z1)) / LIGHT_SPACING + 0.5))
805     for i = 1, nx do
806         for j = 1, nz do
807             lamp(parent, x1 + (x2 - x1) * ((i - 0.5) / nx), z1 + (z2 - z1) * ((j - 0.5) / nz), baseY,
808                 ↪ brightness)
809         end
810     end
811 end
812
813 --=====
814 -- ELEVATOR CORES (three, set into the corridor so a walking route
815 -- always stays clear alongside each shaft)
816 --=====
817 -- Elevators removed by request: the building is stairs-only now (a tower
818 -- at each end of the corridor -- see buildStairTower below -- both running
819 -- basement to roof). Left as an empty list rather than deleting all the
820 -- downstream machinery: with zero entries here, no shafts get built, no
821 -- holes get cut in any floor slab, and the elevator control script simply
822 -- finds nothing tagged "Elevator" and does nothing.
823 local ELEVATORS = {}
824
825 -- The clear opening each shaft needs through a slab: the inner car
826 -- area only, so the slab still carries the shaft walls.
827 local ELEV_HOLES = {}
828 for _, e in ipairs(ELEVATORS) do
829     table.insert(ELEV_HOLES, {
830         x1 = (e.face == "X+") and (e.x1 + INT_T / 2) or e.x1,
831         x2 = (e.face == "X+") and e.x2 or (e.x2 - INT_T / 2),
832         z1 = e.z1 + INT_T / 2,
833         z2 = e.z2 - INT_T / 2,
834     })
835 end
836
837 -- Slab with rectangular holes punched through it (holes must not
838 -- overlap in z, which the elevator bays do not).
839 local function slabWithHoles(parent, name, x1, z1, x2, z2, topY, thickness, holes, colour, material)
840     local list = {}
841     for _, h in ipairs(holes or {}) do table.insert(list, h) end
842     table.sort(list, function(a, b) return a.z1 < b.z1 end)

```

```

842     local k = 0
843     local function rect(ax, az, bx, bz)
844         if bx - ax < 1 or bz - az < 1 then return end
845         k = k + 1
846         slab(parent, name .. "_" .. k, ax, az, bx, bz, topY, thickness, colour, material)
847     end
848     local cursor = z1
849     for _, h in ipairs(list) do
850         if h.z1 > cursor then rect(x1, cursor, x2, h.z1) end
851         if h.x1 > x1 then rect(x1, h.z1, h.x1, h.z2) end
852         if h.x2 < x2 then rect(h.x2, h.z1, x2, h.z2) end
853         cursor = math.max(cursor, h.z2)
854     end
855     if cursor < z2 then rect(x1, cursor, x2, z2) end
856 end
857
858 -----
859 -- LEVEL SHELL
860 -----
861 local LEVELS = {
862     {index = 0, label = "1", name = "Level_1", rooms = GROUND_ROOMS},
863     {index = 1, label = "2", name = "Level_2", rooms = TYPICAL_ROOMS},
864     {index = 2, label = "3", name = "Level_3", rooms = TYPICAL_ROOMS},
865 }
866 -- Five more floors on top of the original three (eight total). Each one
867 -- reuses the same typical room layout but gets its own RNG seed for
868 -- fitOutRoom (seeded off the level index further down), so the equipment
869 -- arrangement differs floor to floor even though the room shapes repeat.
870 for i = 3, 7 do
871     table.insert(LEVELS, {index = i, label = tostring(i + 1), name = "Level_" .. (i + 1), rooms =
872         ↳ TYPICAL_ROOMS})
873 end
874
875 local function offsetOps(list, base, extra)
876     local out = {}
877     for _, o in ipairs(list) do
878         table.insert(out, {at = base + o.at, width = o.width, sill = o.sill,
879             head = o.head, glass = o.glass, leaf = o.leaf, mainEntrance = o.mainEntrance})
880     end
881     for _, o in ipairs(extra or {}) do table.insert(out, o) end
882     return out
883 end
884
885 local function buildLevelShell(levelIndex, levelName, levelLabel, rooms)
886     local y = levelIndex * F2F
887     local m = Instance.new("Model")
888     m.Name = levelName
889     m.Parent = root
890
891     slabWithHoles(m, "FloorSlab", 0, 0, W, L, y, SLAB, ELEV_HOLES, C.slab, Enum.Material.SmoothPlastic)
892
893     -- corridor walls, assembled from every room's own door list
894     local wOps, eOps = {}, {}
895     for _, r in ipairs(rooms) do
896         if r.plan == "L" then
897             for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(wOps, o) end
898         else
899             for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(eOps, o) end
900         end
901     end
902     wall(m, "Corridor_West", CORR_X1, 0, CORR_X1, L, INT_T, y, WALL_H, wOps, C.int)
903     wall(m, "Corridor_East", CORR_X2, 0, CORR_X2, L, INT_T, y, WALL_H, eOps, C.int)
904
905     -- exterior long walls
906     local extW, extE = {}, {}
907     for _, r in ipairs(rooms) do
908         if r.plan == "L" then
909             for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(extW, o) end
910         else
911             for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(extE, o) end
912         end
913     end
914
915     -- Only the ground-floor west lobby has a normal exterior entrance.
916     if levelIndex == 0 and extW[1] then
917         extW[1].glass, extW[1].leaf, extW[1].sill, extW[1].mainEntrance = false, true, 0, true
918     end

```



```

917 wall(m, "Ext_West", EXT_T / 2, 0, EXT_T / 2, L, EXT_T, y, WALL_H, extW, C.ext, Enum.Material.Concrete)
918 wall(m, "Ext_East", W - EXT_T / 2, 0, W - EXT_T / 2, L, EXT_T, y, WALL_H, extE, C.ext,
    ↪ Enum.Material.Concrete)
919
920 -- exterior end walls
921 wall(m, "Ext_North", 0, EXT_T / 2, W, EXT_T / 2, EXT_T, y, WALL_H, {
922     {at = CORR_X1 + 20, width = 160, head = 240},
923     {at = W * 0.12, width = 140, sill = WIN_SILL, head = WIN_HEAD, glass = true},
924     {at = W * 0.77, width = 140, sill = WIN_SILL, head = WIN_HEAD, glass = true},
925 }, C.ext, Enum.Material.Concrete)
926
927 wall(m, "Ext_South", 0, L - EXT_T / 2, W, L - EXT_T / 2, EXT_T, y, WALL_H, {
928     {at = CORR_X1 + 25, width = WIDE_W, head = 240, leaf = true},
929     {at = W * 0.12, width = 90, sill = 20, head = 230, glass = true},
930     {at = W * 0.20, width = 150, sill = WIN_SILL, head = WIN_HEAD, glass = true},
931     {at = W * 0.80, width = 90, sill = 20, head = 230, glass = true},
932 }, C.ext, Enum.Material.Concrete)
933
934 -- dividers, carrying the interconnecting doors
935 for i, r in ipairs(rooms) do
936     local nxt = rooms[i + 1]
937     if nxt and nxt.plan == r.plan and nxt.z1 == r.z2 then
938         wall(m, "Div_" .. i, r.x1, r.z2, r.x2, r.z2, INT_T, y, WALL_H,
939             r.open.S, C.int)
940     end
941 end
942
943 -- signage and lighting
944 for _, r in ipairs(rooms) do
945     local sx = (r.plan == "L") and (CORR_X1 - 45) or (CORR_X2 + 45)
946     sign(m, r.name, sx, (r.z1 + r.z2) / 2, y, 220, 55)
947     lightGrid(m, r.x1, r.z1, r.x2, r.z2, y, 1)
948 end
949 lightGrid(m, CORR_X1, 0, CORR_X2, L, y, 0.8)
950 sign(m, "Floor " .. levelLabel, (CORR_X1 + CORR_X2) / 2, 120, y, 300, 90)
951
952 return m
953 end
954
955 -----
956 -- BASEMENT -- driven by BASEMENT_ROOMS, same shape as "Plan demisol":
957 -- one long room down the west side, 7 rooms off the corridor to the
958 -- east, solid (below-grade) exterior walls with no windows.
959 -----
960 local function buildBasement()
961     local y = -F2F
962     local m = Instance.new("Model")
963     m.Name = "Basement"
964     m.Parent = root
965
966     slab(m, "BasementSlab", 0, 0, W, L, y, 40, C.basement, Enum.Material.Concrete)
967
968     wall(m, "B_West", EXT_T, 0, EXT_T, L, 40, y, WALL_H, {}, C.basewall, Enum.Material.Concrete)
969     wall(m, "B_East", W - EXT_T, 0, W - EXT_T, L, 40, y, WALL_H, {}, C.basewall, Enum.Material.Concrete)
970     wall(m, "B_North", 0, EXT_T, W, EXT_T, 40, y, WALL_H,
971         {{at = CORR_X1 + 20, width = 160, head = 240}}, C.basewall, Enum.Material.Concrete)
972     wall(m, "B_South", 0, L - EXT_T, W, L - EXT_T, 40, y, WALL_H,
973         {{at = CORR_X1 + 20, width = 160, head = 240}}, C.basewall, Enum.Material.Concrete)
974
975     -- corridor walls, doors gathered from every basement room's own list
976     local wOps, eOps = {}, {}
977     for _, r in ipairs(BASEMENT_ROOMS) do
978         if r.plan == "L" then
979             for _, o in ipairs(offsetOps(r.open.E, r.z1)) do table.insert(wOps, o) end
980         else
981             for _, o in ipairs(offsetOps(r.open.W, r.z1)) do table.insert(eOps, o) end
982         end
983     end
984     wall(m, "B_Corr_West", CORR_X1, 0, CORR_X1, L, INT_T, y, WALL_H, wOps, C.basewall)
985     wall(m, "B_Corr_East", CORR_X2, 0, CORR_X2, L, INT_T, y, WALL_H, eOps, C.basewall)
986
987     -- dividers between consecutive east-side bays
988     for i, r in ipairs(BASEMENT_ROOMS) do
989         local nxt = BASEMENT_ROOMS[i + 1]
990         if nxt and nxt.plan == r.plan and nxt.z1 == r.z2 then
991             wall(m, "B_Div_" .. i, r.x1, r.z2, r.x2, r.z2, INT_T, y, WALL_H,

```

```

992         r.open.S, C.basewall)
993     end
994 end
995
996 for _, x in ipairs({150, 250, W - 250, W - 150}) do
997     for z = 250, L - 200, 350 do
998         newPart("Pillar", Vector3.new(X(45), Y(WALL_H), X(45)),
999             CFrame.new(X(x), Y(y + WALL_H / 2), X(z)), C.pillar, Enum.Material.Concrete, m)
1000     end
1001 end
1002
1003 for _, r in ipairs(BASEMENT_ROOMS) do
1004     local sx = (r.plan == "L") and (CORR_X1 - 45) or (CORR_X2 + 45)
1005     sign(m, r.name, sx, (r.z1 + r.z2) / 2, y, 200, 55)
1006     lightGrid(m, r.x1, r.z1, r.x2, r.z2, y, 0.6)
1007 end
1008 lightGrid(m, CORR_X1, 0, CORR_X2, L, y, 0.5)
1009 sign(m, "Basement", (CORR_X1 + CORR_X2) / 2, 120, y, 300, 90)
1010 return m, BASEMENT_ROOMS
1011 end
1012
1013 =====
1014 -- STAIR TOWER
1015 =====
1016 -- centred on the corridor, which sits at CORR_X1..CORR_X2. Two towers now,
1017 -- one attached beyond each end of the building (north, before z=0, and
1018 -- south, beyond z=L) -- stairs "on each opposite part of the hallway" per
1019 -- request, instead of a single stairwell everyone has to walk the whole
1020 -- corridor to reach. Both run the full basement-to-roof height.
1021 local TX1, TX2 = CORR_X1 - 50, CORR_X2 + 50
1022 local TXM = (TX1 + TX2) / 2
1023
1024 local function buildStairTower(bottomLevel, topLevel, southEnd)
1025     local m = Instance.new("Model")
1026     m.Name = southEnd and "StairTower_South" or "StairTower_North"
1027     m.Parent = root
1028     local bottomY = bottomLevel * F2F
1029     local shellH = (topLevel - bottomLevel) * F2F + WALL_H
1030
1031     -- All the tower's Z coordinates are measured outward from the building
1032     -- edge it attaches to (0 = flush with the building). "dir" flips the
1033     -- sign so the exact same layout mirrors onto the south end: outerZ is
1034     -- the far exterior wall (carries the entrance door), edgeZ is flush
1035     -- with the building's own end wall (and its existing WIDE_W doorway,
1036     -- on the south end -- see Ext_South), flightNearZ/flightFarZ are the
1037     -- switchback landing points in between. wall()/slab() both take their
1038     -- span via math.abs() of the two endpoints, so the sign of "dir" alone
1039     -- is enough to mirror every call below without reordering anything.
1040     local edgeZ = southEnd and L or 0
1041     local dir = southEnd and 1 or -1
1042     local outerZ = edgeZ + dir * 460
1043     local flightNearZ = edgeZ + dir * 100
1044     local flightFarZ = edgeZ + dir * 400
1045
1046     wall(m, "T_West", TX1, edgeZ, TX1, outerZ, EXT_T, bottomY, shellH, {}, C.ext, Enum.Material.Concrete)
1047     wall(m, "T_East", TX2, edgeZ, TX2, outerZ, EXT_T, bottomY, shellH, {}, C.ext, Enum.Material.Concrete)
1048     wall(m, southEnd and "T_South" or "T_North", TX1, outerZ, TX2, outerZ, EXT_T, bottomY, shellH,
1049         {{at = 105, width = 110, sill = -bottomY + 20, head = -bottomY + 235, glass = true}}, C.ext,
1050         Enum.Material.Concrete)
1051     slab(m, "T_Roof", TX1 - EXT_T, edgeZ, TX2 + EXT_T, outerZ + dir * EXT_T, bottomY + shellH + SLAB, SLAB,
1052         C.roof)
1053     slab(m, "T_Apron", TX1 - 60, outerZ, TX2 + 60, outerZ + dir * 260, 0, 20, C.slab)
1054
1055     local halfRise = Y(F2F) / 2
1056     local STEPS = math.max(8, math.ceil(halfRise / 1.4))
1057
1058     local function flight(name, x1, x2, zStart, zEnd, yStart, yEnd)
1059         local dz, dy = (zEnd - zStart) / STEPS, (yEnd - yStart) / STEPS
1060         for i = 1, STEPS do
1061             local z0, z1 = zStart + dz * (i - 1), zStart + dz * i
1062             local top = yStart + dy * i
1063             local a, b = math.min(z0, z1), math.max(z0, z1)
1064             newPart(name .. "_t" .. i, Vector3.new(X(x2 - x1), Y(18), X(math.abs(dz))),
1065                 CFrame.new(X((x1 + x2) / 2), Y(top - 9), X((a + b) / 2)), C.stair, Enum.Material.Concrete, m)
1066             newPart(name .. "_r" .. i, Vector3.new(X(x2 - x1), Y(math.abs(dy) + 4), X(math.abs(dz))),
1067                 CFrame.new(X((x1 + x2) / 2), Y(top - 18 - (math.abs(dy) + 4) / 2), X((a + b) / 2)),

```

```

1066         C.stair, Enum.Material.Concrete, m)
1067     end
1068 end
1069
1070 for lvl = bottomLevel, topLevel do
1071     local y = lvl * F2F
1072     slab(m, "Landing_L" .. lvl, TX1, flightNearZ, TX2, edgeZ, y, 15, C.stair)
1073     if lvl < topLevel then
1074         local half = y + F2F / 2
1075         flight("FlightA_L" .. lvl, TX1 + 10, TXM, flightNearZ, flightFarZ, y, half)
1076         slab(m, "MidLanding_L" .. lvl, TX1, outerZ, TX2, flightFarZ, half, 15, C.stair)
1077         flight("FlightB_L" .. lvl, TXM, TX2 - 10, flightFarZ, flightNearZ, half, y + F2F)
1078         newPart("Handrail_L" .. lvl, Vector3.new(X(8), Y(100), X(math.abs(flightNearZ - flightFarZ))),
1079             CFrame.new(X(TXM), Y(y + F2F / 2 + 50), X((flightFarZ + flightNearZ) / 2)),
1080             Color3.fromRGB(120, 128, 136), Enum.Material.Metal, m).CanCollide = false
1081     end
1082     -- Put a compact light over the level landing, never across a flight.
1083     local light = newPart("StairLandingLight_L" .. lvl, Vector3.new(2.5, 0.18, 0.65),
1084         CFrame.new(X(TX1 + 22), Y(y + WALL_H - 15), X(edgeZ + dir * 45)),
1085         Color3.fromRGB(245, 242, 225), Enum.Material.Neon, m)
1086     light.CanCollide, light.CanTouch, light.CanQuery = false, false, false
1087     local glow = Instance.new("PointLight")
1088     glow.Range, glow.Brightness, glow.Parent = 24, 0.8, light
1089 end
1090 return m
1091 end
1092
1093 =====
1094 -- ELEVATOR BUILDER
1095 =====
1096 local function buildElevator(spec, bottomLevel, topLevel)
1097     local m = Instance.new("Model")
1098     m.Name = "Elevator_" .. spec.id
1099     m.Parent = root
1100
1101     local bottomY = bottomLevel * F2F
1102     local shellH = (topLevel - bottomLevel) * F2F + WALL_H
1103     local openAtX2 = (spec.face == "X+") -- open face is x2 (or x1)
1104     local backX = openAtX2 and spec.x1 or spec.x2
1105     local frontX = openAtX2 and spec.x2 or spec.x1
1106
1107     -- shaft: back wall plus both sides, open on the corridor face
1108     wall(m, "Shaft_Back", backX, spec.z1, backX, spec.z2, INT_T, bottomY, shellH, {}, C.shaft,
1109         Enum.Material.Metal)
1110     wall(m, "Shaft_S1", spec.x1, spec.z1, spec.x2, spec.z1, INT_T, bottomY, shellH, {}, C.shaft,
1111         Enum.Material.Metal)
1112     wall(m, "Shaft_S2", spec.x1, spec.z2, spec.x2, spec.z2, INT_T, bottomY, shellH, {}, C.shaft,
1113         Enum.Material.Metal)
1114     slab(m, "Shaft_Roof", spec.x1, spec.z1, spec.x2, spec.z2, bottomY + shellH + SLAB, SLAB, C.shaft)
1115
1116     local innerZ1, innerZ2 = spec.z1 + INT_T / 2, spec.z2 - INT_T / 2
1117     local innerX1 = openAtX2 and (spec.x1 + INT_T / 2) or spec.x1
1118     local innerX2 = openAtX2 and spec.x2 or (spec.x2 - INT_T / 2)
1119
1120     local floors = {}
1121     for lvl = bottomLevel, topLevel do table.insert(floors, lvl) end
1122
1123     m:SetAttribute("FloorCount", #floors)
1124     m:SetAttribute("Face", spec.face)
1125     for i, lvl in ipairs(floors) do
1126         m:SetAttribute("FloorY_" .. i, Y(lvl * F2F))
1127         m:SetAttribute("FloorLabel_" .. i, lvl < 0 and "B" or tostring(lvl + 1))
1128     end
1129
1130     -- landing gate + call button at every floor
1131     local gates = Instance.new("Folder"); gates.Name = "Gates"; gates.Parent = m
1132     local calls = Instance.new("Folder"); calls.Name = "CallPanels"; calls.Parent = m
1133     for i, lvl in ipairs(floors) do
1134         local y = lvl * F2F
1135         local g = newPart("Gate_" .. i,
1136             Vector3.new(X(8), Y(DOOR_H + 20), X(innerZ2 - innerZ1)),
1137             CFrame.new(X(frontX), Y(y + (DOOR_H + 20) / 2), X((innerZ1 + innerZ2) / 2)),
1138             Color3.fromRGB(110, 120, 132), Enum.Material.Metal, gates)
1139         g:SetAttribute("Floor", i)
1140
1141         local panel = newPart("Call_" .. i, Vector3.new(X(6), Y(40), X(30)),

```

```

1139         CFrame.new(X(frontX + (openAtX2 and 12 or -12)), Y(y + 120), X(innerZ1 - 25)),
1140         Color3.fromRGB(60, 66, 74), Enum.Material.Metal, calls)
1141     panel:SetAttribute("Floor", i)
1142     panel:SetAttribute("ElevatorId", spec.id)
1143 end
1144
1145 -- the car
1146 local car = Instance.new("Model"); car.Name = "Car"; car.Parent = m
1147 local startY = bottomLevel * F2F
1148 local cw = innerX2 - innerX1
1149 local cd = innerZ2 - innerZ1
1150
1151 slab(car, "Platform", innerX1, innerZ1, innerX2, innerZ2, startY + 10, 10, C.car, Enum.Material.Metal)
1152 local function carWall(name, x1, z1, x2, z2)
1153     wall(car, name, x1, z1, x2, z2, 10, startY + 10, 250, {}, C.car, Enum.Material.Metal)
1154 end
1155 carWall("Car_Back", backX + (openAtX2 and 12 or -12), innerZ1, backX + (openAtX2 and 12 or -12), innerZ2)
1156 carWall("Car_S1", innerX1, innerZ1 + 6, innerX2, innerZ1 + 6)
1157 carWall("Car_S2", innerX1, innerZ2 - 6, innerX2, innerZ2 - 6)
1158 slab(car, "Car_Roof", innerX1, innerZ1, innerX2, innerZ2, startY + 270, 10, C.car, Enum.Material.Metal)
1159 local carLight = newPart("Car_Light", Vector3.new(X(40), Y(6), X(40)),
1160     CFrame.new(X((innerX1 + innerX2) / 2), Y(startY + 250), X((innerZ1 + innerZ2) / 2)),
1161     Color3.fromRGB(255, 255, 245), Enum.Material.Neon, car)
1162 carLight.CanCollide = false
1163 local cl = Instance.new("PointLight"); cl.Range = 30; cl.Brightness = 1.2; cl.Parent = carLight
1164
1165 -- in-car buttons, one per floor, on the back wall
1166 local buttons = Instance.new("Folder"); buttons.Name = "Buttons"; buttons.Parent = car
1167 for i, lvl in ipairs(floors) do
1168     local t = (i - 0.5) / #floors
1169     local b = newPart("Btn_" .. i, Vector3.new(X(8), Y(26), X(16)),
1170         CFrame.new(X(backX + (openAtX2 and 22 or -22)), Y(startY + 130),
1171             X(innerZ1 + (innerZ2 - innerZ1) * t)),
1172         Color3.fromRGB(230, 200, 90), Enum.Material.Neon, buttons)
1173     b.CanCollide = false
1174     b:SetAttribute("Floor", i)
1175 end
1176
1177 m:SetAttribute("CurrentFloor", 1)
1178 CollectionService.AddTag(m, "Elevator")
1179 return m
1180 end
1181
1182 =====
1183 -- BASIC LABORATORY EQUIPMENT CATALOGUE
1184 -- Nothing here is tied to a particular room or process. The fit-out
1185 -- for every room is generated from this list.
1186 =====
1187 local COL = {
1188     bench = Color3.fromRGB(178, 182, 186),
1189     steel = Color3.fromRGB(196, 200, 205),
1190     white = Color3.fromRGB(236, 238, 241),
1191     dark = Color3.fromRGB(72, 78, 86),
1192     timber = Color3.fromRGB(166, 142, 112),
1193     cold = Color3.fromRGB(198, 216, 226),
1194     warm = Color3.fromRGB(206, 178, 140),
1195     safety = Color3.fromRGB(190, 62, 52),
1196     green = Color3.fromRGB(78, 138, 96),
1197     glassy = Color3.fromRGB(198, 220, 230),
1198 }
1199
1200 -- class: "run" = long fitted furniture, takes the best wall
1201 -- "floor" = free-standing unit against a wall
1202 -- "top" = sits on a bench run
1203 -- "duty" = safety kit, one of each per room
1204 -- shape: how the item is actually built out of primitive parts, so it
1205 -- reads as real equipment rather than a plain box (see buildKind below)
1206 local CATALOGUE = {
1207     -- Room contents are now limited to equipment that actually acts on a
1208     -- germinated plant sample (matches an entry in PLANT_PROCESS, or is the
1209     -- separately-placed Plant Growth Chamber). Generic lab furniture,
1210     -- benchtop instruments and safety kit that never touch a plant sample
1211     -- were removed by request so every object in a room is something that
1212     -- moves a plant's status forward.
1213     {kind = "Germination Chamber", class = "floor", w = 95, d = 80, h = 195, col = COL.cold, mat =
1214         Enum.Material.Metal, weight = 4, shape = "appliance", auto = "pulse"},

```

```

1214 {kind = "Incubator", class = "floor", w = 90, d = 75, h = 180, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance"},
1215 {kind = "Decontamination Unit", class = "floor", w = 75, d = 60, h = 150, col = COL.white, mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "appliance", auto = "pulse"},
1216 {kind = "Mutagenesis Chamber", class = "floor", w = 90, d = 80, h = 200, col = COL.safety, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "pulse"},
1217 {kind = "Hormone Treatment Bench", class = "floor", w = 80, d = 65, h = 175, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "spin"},
1218
1219 -- Smaller "run" variants of the same five kinds so bench-run islands
1220 -- (the middle of a room's floor, not just its walls) also fill up with
1221 -- functional equipment -- this is most of what gets the building well
1222 -- past a thousand units total.
1223 {kind = "Germination Chamber", class = "run", w = 68, d = 55, h = 195, col = COL.cold, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance", auto = "pulse"},
1224 {kind = "Incubator", class = "run", w = 64, d = 52, h = 180, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "appliance"},
1225 {kind = "Decontamination Unit", class = "run", w = 58, d = 46, h = 150, col = COL.white, mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "appliance", auto = "pulse"},
1226 {kind = "Mutagenesis Chamber", class = "run", w = 64, d = 55, h = 200, col = COL.safety, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "pulse"},
1227 {kind = "Hormone Treatment Bench", class = "run", w = 58, d = 48, h = 175, col = COL.green, mat =
    ↳ Enum.Material.Metal, weight = 2, shape = "appliance", auto = "spin"},
1228
1229 -- Pure decoration: potted plants, flower planters, and lab storage
1230 -- furniture (drawer units / filing cabinets). None of these are tagged
1231 -- as functional equipment (no status LED, no failure/hazard behaviour)
1232 -- -- they exist purely to fill every remaining sliver of wall and floor
1233 -- space so the building reads as a real, lived-in facility rather than
1234 -- rows of machines with bare walls and floor between them.
1235 {kind = "Potted Plant", class = "decor", w = 42, d = 42, h = 62, col = Color3.fromRGB(150, 108, 70), mat =
    ↳ Enum.Material.SmoothPlastic, weight = 5, shape = "planter"},
1236 {kind = "Flowering Planter", class = "decor", w = 55, d = 32, h = 40, col = Color3.fromRGB(168, 120, 78),
    ↳ mat = Enum.Material.SmoothPlastic, weight = 4, shape = "planter", flowering = true},
1237 {kind = "Storage Drawers", class = "decor", w = 62, d = 45, h = 95, col = Color3.fromRGB(96, 102, 110), mat =
    ↳ Enum.Material.Metal, weight = 4, shape = "drawers"},
1238 {kind = "Filing Cabinet", class = "decor", w = 48, d = 52, h = 130, col = Color3.fromRGB(72, 78, 86), mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "drawers"},
1239 {kind = "Supply Shelf", class = "decor", w = 90, d = 38, h = 175, col = Color3.fromRGB(120, 126, 130), mat =
    ↳ Enum.Material.Metal, weight = 3, shape = "rack"},
1240 }
1241
1242 -- Scientific machines share one readable, refrigerator-like design language.
1243 -- Desks and benches remain furniture; benchtop instruments and safety kit
1244 -- keep their own real-world proportions (a Microscope should never look like
1245 -- a walk-in freezer). Only true free-standing floor equipment -- growth
1246 -- chambers, ovens, storage cabinets and the like -- get the industrial
1247 -- cabinet treatment, and it's sized for an industrial facility: wide, tall
1248 -- units with real walking clearance around them. Width/height variants
1249 -- still communicate capacity.
1250 local CABINET_SIZES = {
1251     {w = 100, d = 90, h = 220, capacity = 6},
1252     {w = 115, d = 95, h = 235, capacity = 8},
1253     {w = 130, d = 100, h = 250, capacity = 10},
1254     {w = 145, d = 105, h = 265, capacity = 14},
1255 }
1256 for _, entry in ipairs(CATALOGUE) do
1257     if entry.class == "floor" then
1258         local hash = 0
1259         for i = 1, #entry.kind do hash = hash + string.byte(entry.kind, i) end
1260         local cabinet = CABINET_SIZES[hash % #CABINET_SIZES + 1]
1261         entry.w, entry.d, entry.h = cabinet.w, cabinet.d, cabinet.h
1262         entry.capacity = cabinet.capacity
1263         entry.shape = "appliance"
1264         entry.plantCabinet = true
1265     end
1266 end
1267
1268 local BY_CLASS = {run = {}, floor = {}, top = {}, duty = {}, decor = {}}
1269 for _, e in ipairs(CATALOGUE) do
1270     table.insert(BY_CLASS[e.class], e)
1271 end
1272
1273 local function weightedPick(list, rng)
1274     local total = 0
1275     for _, e in ipairs(list) do total = total + (e.weight or 1) end

```

```

1276     local roll = rng.next() * total
1277     for _, e in ipairs(list) do
1278         roll = roll - (e.weight or 1)
1279         if roll <= 0 then return e end
1280     end
1281     return list[#list]
1282 end
1283
1284 -----
1285 -- AUTOMATIC WALL PACKING
1286 -----
1287 local WALL_GAP = 16 -- plan cm of clear floor between a wall face and a cabinet
1288 local ALONG_GAP = 26 -- plan cm between neighbours -- enough of an aisle gap that each
1289                       -- cabinet's status text doesn't overlap its neighbour's
1290 local CORNER = 100 -- plan cm kept clear at each corner (must exceed
1291                   -- the deepest unit so adjacent walls cannot clash)
1292 local DOOR_CLEAR = 55 -- plan cm kept clear either side of a door
1293
1294 -- Free intervals along one wall of a room, in that wall's local
1295 -- coordinates, with door swings excluded.
1296 local function freeSpans(length, openings)
1297     local blocked = {}
1298     for _, o in ipairs(openings or {}) do
1299         -- Windows need clear floor space just as doors do. The smaller window
1300         -- margin preserves capacity while keeping every cabinet out of glazing.
1301         local clear = o.glass and 30 or DOOR_CLEAR
1302         table.insert(blocked, {o.at - clear, o.at + o.width + clear})
1303     end
1304     table.sort(blocked, function(a, b) return a[1] < b[1] end)
1305
1306     local spans, cursor = {}, CORNER
1307     for _, b in ipairs(blocked) do
1308         if b[1] > cursor then table.insert(spans, {cursor, math.min(b[1], length - CORNER)}) end
1309         cursor = math.max(cursor, b[2])
1310     end
1311     if cursor < length - CORNER then table.insert(spans, {cursor, length - CORNER}) end
1312
1313     local out = {}
1314     for _, s in ipairs(spans) do
1315         if s[2] - s[1] > 20 then table.insert(out, s) end
1316     end
1317     return out
1318 end
1319
1320 local function takeSpan(spans, need)
1321     for _, s in ipairs(spans) do
1322         if s[2] - s[1] >= need then
1323             local at = s[1] + need / 2
1324             s[1] = s[1] + need + ALONG_GAP
1325             return at
1326         end
1327     end
1328     return nil
1329 end
1330
1331 -- Same as takeSpan but with a caller-chosen gap. Decoration (plants,
1332 -- drawers, ...) doesn't carry a status readout that neighbours could
1333 -- overlap, so it can pack into slivers real equipment leaves behind with
1334 -- only a token gap between pieces.
1335 local function takeSpanGap(spans, need, gap)
1336     for _, s in ipairs(spans) do
1337         if s[2] - s[1] >= need then
1338             local at = s[1] + need / 2
1339             s[1] = s[1] + need + gap
1340             return at
1341         end
1342     end
1343     return nil
1344 end
1345
1346 local function spansTotal(spans)
1347     local t = 0
1348     for _, s in ipairs(spans) do t = t + (s[2] - s[1]) end
1349     return t
1350 end
1351

```

```

1352 -----
1353 -- EQUIPMENT SHAPES
1354 -- Every catalogue entry is built from 1-4 primitive parts instead of a
1355 -- single box, so it reads as the kind of thing it is: a bench with
1356 -- legs and a top, an appliance with a door and a handle, a rack with
1357 -- shelves, an instrument with a small control panel. Everything here
1358 -- is axis-aligned (the generator never yaws a part), so offsets are
1359 -- plain world-space vectors added to a centre position.
1360 -----
1361 local function buildKind(parent, entry, pos, size, wallSide)
1362     local shape = entry.shape or "box"
1363     local dark = Color3.fromRGB(36, 38, 44)
1364     -- The accent/panel colour always contrasts AGAINST the body colour,
1365     -- not just darkens it: a body that's already dark (COL.dark items
1366     -- like the hot plate stirrer, the pH meter, ...) used to get an
1367     -- accent that was darker still, so the door/panel disappeared into
1368     -- the body and the whole thing read as a plain black box.
1369     local lum = entry.col.R * 0.299 + entry.col.G * 0.587 + entry.col.B * 0.114
1370     local accent
1371     if lum > 0.45 then
1372         accent = Color3.fromRGB(
1373             math.max(0, entry.col.R * 255 - 45),
1374             math.max(0, entry.col.G * 255 - 45),
1375             math.max(0, entry.col.B * 255 - 45))
1376     else
1377         accent = Color3.fromRGB(
1378             math.min(255, entry.col.R * 255 + 80),
1379             math.min(255, entry.col.G * 255 + 80),
1380             math.min(255, entry.col.B * 255 + 80))
1381     end
1382     local panelColour = lum > 0.45 and dark or Color3.fromRGB(168, 172, 178)
1383
1384     -- Every sub-part of an assembly is parented to the piece's own main
1385     -- part (not the room model) so the runtime script can reliably find
1386     -- "this instance's own door/panel/light" with FindFirstChild instead
1387     -- of guessing among same-named siblings from every other item.
1388     if shape == "appliance" then
1389         -- Every appliance-class unit is built as a real germination-cabinet
1390         -- style enclosure -- insulated shell, glass front with frame trim,
1391         -- interior shelving/lighting, a rear vent, a controller screen and
1392         -- caster wheels -- the same construction language as the R1-R200
1393         -- growth chambers, never a plain box with a door bolted on. Colour
1394         -- (drawn from the catalogue entry, which already varies per kind)
1395         -- is what gives each equipment TYPE its own tailored look: a cold
1396         -- germination unit reads pale and clinical, a mutagenesis chamber
1397         -- reads hazard-yellow, and so on -- hospital-white insulated shells,
1398         -- industrial metal frame/trim, and lab glass fronts throughout.
1399         local frontOnX, direction
1400         if wallSide == "W" or wallSide == "E" then
1401             frontOnX = true
1402             direction = (wallSide == "W") and 1 or -1
1403         else
1404             frontOnX = false
1405             direction = (wallSide == "S") and -1 or 1
1406         end
1407
1408         local shellColour = lum > 0.55 and Color3.fromRGB(226, 231, 229)
1409             or lum > 0.3 and Color3.fromRGB(196, 202, 201)
1410             or Color3.fromRGB(54, 58, 62)
1411         local frameColour = Color3.fromRGB(56, 62, 66)
1412         local glassColour = Color3.fromRGB(
1413             math.min(255, entry.col.R * 255 * 0.5 + 150),
1414             math.min(255, entry.col.G * 255 * 0.5 + 155),
1415             math.min(255, entry.col.B * 255 * 0.5 + 155))
1416
1417         -- Backing volume only (matches the growth-chamber reference): a
1418         -- transparent placeholder that all the real, opaque panels below
1419         -- attach to, sized/positioned exactly as the caller expects so the
1420         -- runtime's LED/prompt/status binding keeps working unmodified.
1421         local body = newPart("Body", size, CFrame.new(pos), Color3.fromRGB(205, 216, 212), Enum.Material.Glass,
1422             ↳ parent)
1423         body.Transparency = 0.9
1424
1425         local innerWidth = frontOnX and size.Z or size.X
1426         local innerDepth = frontOnX and size.X or size.Z
1427         local shellLT = math.max(math.min(size.X, size.Z) * 0.05, 0.12)

```

```

1427     local function interiorOffset(across, up, fromFront)
1428         if frontOnX then return CFrame.new(direction * (innerDepth / 2 - fromFront), up, across) end
1429         return CFrame.new(across, up, direction * (innerDepth / 2 - fromFront))
1430     end
1431
1432     -- Opaque insulated back/side/top/floor panels -- only the front
1433     -- stays glazed, matching a real controlled-environment cabinet.
1434     if frontOnX then
1435         newPart("InsulatedBack", Vector3.new(shellT, size.Y * 0.96, innerWidth),
1436             body.CFrame * CFrame.new(-direction * (innerDepth / 2 - shellT / 2), 0, 0), shellColour,
1437             ↳ Enum.Material.Metal, body)
1438         for _, sideSign in ipairs({-1, 1}) do
1439             newPart("InsulatedSide", Vector3.new(innerDepth, size.Y * 0.96, shellT),
1440                 body.CFrame * CFrame.new(0, 0, sideSign * (innerWidth / 2 - shellT / 2)), shellColour,
1441                 ↳ Enum.Material.Metal, body)
1442         end
1443     else
1444         newPart("InsulatedBack", Vector3.new(innerWidth, size.Y * 0.96, shellT),
1445             body.CFrame * CFrame.new(0, 0, -direction * (innerDepth / 2 - shellT / 2)), shellColour,
1446             ↳ Enum.Material.Metal, body)
1447         for _, sideSign in ipairs({-1, 1}) do
1448             newPart("InsulatedSide", Vector3.new(shellT, size.Y * 0.96, innerDepth),
1449                 body.CFrame * CFrame.new(sideSign * (innerWidth / 2 - shellT / 2), 0, 0), shellColour,
1450                 ↳ Enum.Material.Metal, body)
1451         end
1452     end
1453     local capSize = frontOnX and Vector3.new(innerDepth, shellT, innerWidth) or Vector3.new(innerWidth,
1454         ↳ shellT, innerDepth)
1455     newPart("InsulatedTop", capSize, body.CFrame * CFrame.new(0, size.Y / 2 - shellT / 2, 0), shellColour,
1456         ↳ Enum.Material.Metal, body)
1457     newPart("InsulatedFloor", capSize, body.CFrame * CFrame.new(0, -size.Y / 2 + shellT / 2, 0),
1458         ↳ shellColour, Enum.Material.Metal, body)
1459
1460     -- Glass front door, framed in metal on all four edges.
1461     local frontDist = innerDepth / 2 + 0.07
1462     local doorSize = frontOnX and Vector3.new(shellT * 1.2, size.Y * 0.82, innerWidth * 0.86)
1463         or Vector3.new(innerWidth * 0.86, size.Y * 0.82, shellT * 1.2)
1464     local doorCFrame = frontOnX and (body.CFrame * CFrame.new(direction * frontDist, 0, 0))
1465         or (body.CFrame * CFrame.new(0, 0, direction * frontDist))
1466     local door = newPart("Door", doorSize, doorCFrame, glassColour, Enum.Material.Glass, body)
1467     door.Transparency = entry.plantCabinet and 0.34 or 0.5
1468     door.CanCollide = false
1469     if frontOnX then
1470         local fx = direction * frontDist
1471         newPart("GlassFrameLeft", Vector3.new(0.18, size.Y * 0.94, 0.2), body.CFrame * CFrame.new(fx, 0,
1472             ↳ -innerWidth * 0.46), frameColour, Enum.Material.Metal, body).CanCollide = false
1473         newPart("GlassFrameRight", Vector3.new(0.18, size.Y * 0.94, 0.2), body.CFrame * CFrame.new(fx, 0,
1474             ↳ innerWidth * 0.46), frameColour, Enum.Material.Metal, body).CanCollide = false
1475         newPart("GlassFrameTop", Vector3.new(0.18, 0.22, innerWidth * 0.94), body.CFrame * CFrame.new(fx,
1476             ↳ size.Y * 0.46, 0), frameColour, Enum.Material.Metal, body).CanCollide = false
1477         newPart("GlassFrameBottom", Vector3.new(0.18, 0.22, innerWidth * 0.94), body.CFrame *
1478             ↳ CFrame.new(fx, -size.Y * 0.46, 0), frameColour, Enum.Material.Metal, body).CanCollide = false
1479     else
1480         local fz = direction * frontDist
1481         newPart("GlassFrameLeft", Vector3.new(0.2, size.Y * 0.94, 0.18), body.CFrame *
1482             ↳ CFrame.new(-innerWidth * 0.46, 0, fz), frameColour, Enum.Material.Metal, body).CanCollide =
1483             ↳ false
1484         newPart("GlassFrameRight", Vector3.new(0.2, size.Y * 0.94, 0.18), body.CFrame *
1485             ↳ CFrame.new(innerWidth * 0.46, 0, fz), frameColour, Enum.Material.Metal, body).CanCollide =
1486             ↳ false
1487         newPart("GlassFrameTop", Vector3.new(innerWidth * 0.94, 0.22, 0.18), body.CFrame * CFrame.new(0,
1488             ↳ size.Y * 0.46, fz), frameColour, Enum.Material.Metal, body).CanCollide = false
1489         newPart("GlassFrameBottom", Vector3.new(innerWidth * 0.94, 0.22, 0.18), body.CFrame * CFrame.new(0,
1490             ↳ -size.Y * 0.46, fz), frameColour, Enum.Material.Metal, body).CanCollide = false
1491     end
1492     local handleSize = frontOnX and Vector3.new(0.05, size.Y * 0.22, 0.05) or Vector3.new(size.Y * 0.22,
1493         ↳ 0.05, 0.05)
1494     local handle = newPart("Handle", handleSize,
1495         doorCFrame * (frontOnX and CFrame.new(direction * 0.05, 0, innerWidth * 0.3) or
1496             ↳ CFrame.new(innerWidth * 0.3, 0, direction * 0.05)),
1497         dark, Enum.Material.Metal, body)
1498     handle.CanCollide = false
1499
1500     -- Interior tier shelving + a light strip per tier, exactly like the
1501     -- growth-chamber reference, on every appliance -- not just the ones
1502     -- that hold plant samples -- so nothing reads as an empty shell.

```



```

1484     local tiers = entry.plantCabinet and 3 or 2
1485     for tier = 0, tiers - 1 do
1486         local shelfY = -size.Y * 0.32 + tier * (size.Y * 0.6 / math.max(1, tiers - 1))
1487         local shelfSize = frontOnX
1488             and Vector3.new(innerDepth * 0.72, 0.08, innerWidth * 0.84)
1489             or Vector3.new(innerWidth * 0.84, 0.08, innerDepth * 0.72)
1490         newPart("Shelf_" .. (tier + 1), shelfSize, body.CFrame * CFrame.new(0, shelfY - 0.24, 0),
1491             Color3.fromRGB(150, 156, 156), Enum.Material.Metal, body).CanCollide = false
1492         local lightSize = frontOnX and Vector3.new(0.1, 0.08, innerWidth * 0.72) or Vector3.new(innerWidth
1493             ↳ * 0.72, 0.08, 0.1)
1494         newPart("Light_" .. (tier + 1), lightSize,
1495             body.CFrame * interiorOffset(0, shelfY + size.Y * 0.1, innerDepth * 0.44),
1496             Color3.fromRGB(255, 248, 222), Enum.Material.Neon, body).CanCollide = false
1497     end
1498
1499     -- Rear ventilation grille + a raised controller housing with a
1500     -- status screen -- the runtime anchors the repair prompt/LED here
1501     -- when present, exactly as it does on the growth chambers.
1502     local ventSize = frontOnX and Vector3.new(0.1, size.Y * 0.22, innerWidth * 0.42) or
1503         ↳ Vector3.new(innerWidth * 0.42, size.Y * 0.22, 0.1)
1504     newPart("VentilationGrille", ventSize, body.CFrame * interiorOffset(0, size.Y * 0.18, innerDepth *
1505         ↳ 0.94),
1506         Color3.fromRGB(68, 74, 78), Enum.Material.DiamondPlate, body).CanCollide = false
1507     local controllerSize = frontOnX and Vector3.new(innerDepth * 0.86, 0.6, innerWidth * 0.92) or
1508         ↳ Vector3.new(innerWidth * 0.92, 0.6, innerDepth * 0.86)
1509     local controller = newPart("ClimateController", controllerSize, body.CFrame * CFrame.new(0, size.Y *
1510         ↳ 0.5 + 0.3, 0),
1511         Color3.fromRGB(222, 226, 224), Enum.Material.Metal, body)
1512     controller.CanCollide = false
1513     newPart("ControllerScreen",
1514         frontOnX and Vector3.new(0.1, 0.4, innerWidth * 0.56) or Vector3.new(innerWidth * 0.56, 0.4, 0.1),
1515         body.CFrame * interiorOffset(0, size.Y * 0.5 + 0.3, -0.16), accent, Enum.Material.Neon,
1516         ↳ body).CanCollide = false
1517
1518     -- Small caster wheels, like a real mobile lab cabinet.
1519     for _, across in ipairs({-innerWidth * 0.38, innerWidth * 0.38}) do
1520         for _, depthSign in ipairs({-1, 1}) do
1521             local wheelOffset
1522             if frontOnX then wheelOffset = CFrame.new(depthSign * innerDepth * 0.32, -size.Y / 2 - 0.2,
1523                 ↳ across)
1524             else wheelOffset = CFrame.new(across, -size.Y / 2 - 0.2, depthSign * innerDepth * 0.32) end
1525             local wheel = newPart("CasterWheel", Vector3.new(0.34, 0.34, 0.2), body.CFrame * wheelOffset,
1526                 Color3.fromRGB(28, 30, 32), Enum.Material.Rubber, body)
1527             wheel.Shape = Enum.PartType.Cylinder
1528             wheel.CanCollide = false
1529         end
1530     end
1531
1532     -- Power hookup: a wall junction box and conduit on the BACK face
1533     -- (opposite the door), running down to a floor-level cable stub
1534     -- toward the wall it's packed against, so every appliance reads as
1535     -- actually wired in rather than just placed in the room.
1536     local backDirX = frontOnX and -direction or 0
1537     local backDirZ = frontOnX and 0 or -direction
1538     local backX = pos.X + backDirX * size.X / 2
1539     local backZ = pos.Z + backDirZ * size.Z / 2
1540     local boxSize = Vector3.new(0.42, 0.42, 0.42)
1541     local junction = newPart("PowerJunctionBox", boxSize,
1542         CFrame.new(backX + backDirX * boxSize.X * 0.45, pos.Y + size.Y * 0.3, backZ + backDirZ * boxSize.Z
1543             ↳ * 0.45),
1544         Color3.fromRGB(56, 58, 62), Enum.Material.Metal, body)
1545     junction.CanCollide = false
1546     junction.CastShadow = false
1547     local conduitH = math.max(size.Y * 0.38, 0.5)
1548     local conduit = newPart("PowerConduit", Vector3.new(0.14, conduitH, 0.14),
1549         CFrame.new(backX + backDirX * 0.14, pos.Y + size.Y * 0.3 - conduitH / 2, backZ + backDirZ * 0.14),
1550         Color3.fromRGB(26, 28, 31), Enum.Material.Metal, body)
1551     conduit.CanCollide = false
1552     conduit.CastShadow = false
1553     local cableLen = 0.85
1554     local cableSize = frontOnX and Vector3.new(cableLen, 0.08, 0.08) or Vector3.new(0.08, 0.08, cableLen)
1555     local cable = newPart("PowerCable", cableSize,
1556         CFrame.new(backX + backDirX * (0.14 + cableLen / 2), pos.Y - size.Y / 2 + 0.12, backZ + backDirZ *
1557             ↳ (0.14 + cableLen / 2)),
1558         Color3.fromRGB(18, 18, 19), Enum.Material.SmoothPlastic, body)
1559     cable.CanCollide = false

```

```

1551     cable.CastShadow = false
1552     return body
1553
1554 elseif shape == "bench" then
1555     local topH = math.max(size.Y * 0.14, 0.15)
1556     local top = newPart("Top", Vector3.new(size.X, topH, size.Z),
1557         CFrame.new(pos.X, pos.Y + size.Y / 2 - topH / 2, pos.Z), entry.col, entry.mat, parent)
1558     local legT = math.min(size.X, size.Z) * 0.07
1559     local legH = math.max(size.Y - topH, 0.2)
1560     for _, s in ipairs({{1, 1}, {1, -1}, {-1, 1}, {-1, -1}}) do
1561         local leg = newPart("Leg", Vector3.new(legT, legH, legT),
1562             CFrame.new(pos.X + s[1] * (size.X / 2 - legT), pos.Y - size.Y / 2 + legH / 2,
1563                 pos.Z + s[2] * (size.Z / 2 - legT)),
1564             dark, Enum.Material.Metal, top)
1565         leg.CanCollide = false
1566     end
1567     return top
1568
1569 elseif shape == "instrument" then
1570     local bodyH = size.Y * 0.55
1571     local body = newPart("Body", Vector3.new(size.X, bodyH, size.Z),
1572         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH / 2, pos.Z), entry.col, entry.mat, parent)
1573     local panelH = math.max(size.Y - bodyH, 0.1)
1574     local panel = newPart("Panel", Vector3.new(size.X * 0.72, panelH, size.Z * 0.55),
1575         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH + panelH / 2, pos.Z - size.Z * 0.12),
1576         panelColour, Enum.Material.SmoothPlastic, body)
1577     panel.CanCollide = false
1578     -- a small glowing indicator strip so even a tiny instrument reads
1579     -- as "a device" at a glance, not a plain block
1580     local led = newPart("IndicatorStrip", Vector3.new(size.X * 0.5, math.max(panelH * 0.18, 0.05), size.Z *
1581         ↪ 0.05),
1582         CFrame.new(pos.X, pos.Y - size.Y / 2 + bodyH + panelH * 0.78, pos.Z - size.Z * 0.12 + size.Z *
1583         ↪ 0.28),
1584         accent, Enum.Material.Neon, body)
1585     led.CanCollide = false
1586     led.CastShadow = false
1587     return body
1588
1589 elseif shape == "rack" then
1590     local backT = math.max(size.Z * 0.14, 0.1)
1591     local back = newPart("Back", Vector3.new(size.X, size.Y, backT),
1592         CFrame.new(pos.X, pos.Y, pos.Z - size.Z / 2 + backT / 2), entry.col, entry.mat, parent)
1593     local shelfH = math.max(size.Y * 0.06, 0.08)
1594     for i = 1, 3 do
1595         local sy = pos.Y - size.Y / 2 + size.Y * (i / 4)
1596         local shelf = newPart("Shelf" .. i, Vector3.new(size.X, shelfH, size.Z),
1597             CFrame.new(pos.X, sy, pos.Z), entry.col, entry.mat, back)
1598         shelf.CanCollide = false
1599     end
1600     return back
1601
1602 elseif shape == "planter" then
1603     -- A potted plant / flower box: a pot or trough with a soil cap and
1604     -- several stem+leaf clusters -- pure decoration, no door/handle.
1605     local potH = size.Y * 0.42
1606     local pot = newPart("Pot", Vector3.new(size.X, potH, size.Z),
1607         CFrame.new(pos.X, pos.Y - size.Y / 2 + potH / 2, pos.Z), entry.col, entry.mat, parent)
1608     local soil = newPart("Soil", Vector3.new(size.X * 0.86, potH * 0.22, size.Z * 0.86),
1609         CFrame.new(pos.X, pos.Y - size.Y / 2 + potH * 0.94, pos.Z), Color3.fromRGB(58, 42, 30),
1610         ↪ Enum.Material.Ground, pot)
1611     soil.CanCollide = false
1612     local clusters = entry.flowering and 4 or 3
1613     for i = 1, clusters do
1614         local ang = (i / clusters) * math.pi * 2 + (pos.X % 1)
1615         local rx = math.cos(ang) * size.X * 0.22
1616         local rz = math.sin(ang) * size.Z * 0.22
1617         local stemH = size.Y * (0.4 + (i % 3) * 0.06)
1618         local stem = newPart("Stem_" .. i, Vector3.new(size.X * 0.05, stemH, size.Z * 0.05),
1619             CFrame.new(pos.X + rx, pos.Y - size.Y / 2 + potH + stemH / 2, pos.Z + rz),
1620             Color3.fromRGB(58, 130, 66), Enum.Material.Grass, pot)
1621         stem.CanCollide = false
1622         local leafColour = entry.flowering
1623             and ({Color3.fromRGB(230, 90, 130), Color3.fromRGB(240, 200, 70), Color3.fromRGB(210, 100,
1624                 ↪ 210)})[(i - 1) % 3 + 1]
1625             or Color3.fromRGB(72, 168, 84)
1626         local leaf = newPart("Leaf_" .. i, Vector3.new(size.X * 0.3, size.Y * 0.16, size.Z * 0.3),

```

```

1623         CFrame.new(pos.X + rx, pos.Y - size.Y / 2 + potH + stemH, pos.Z + rz),
1624         leafColour, entry.flowering and Enum.Material.Neon or Enum.Material.Grass, pot)
1625     leaf.CanCollide = false
1626     leaf.Shape = Enum.PartType.Ball
1627 end
1628 return pot
1629
1630 elseif shape == "drawers" then
1631     -- Lab storage drawers / filing cabinet: a metal carcass with 3-4
1632     -- stacked drawer fronts and handles, so it reads as furniture rather
1633     -- than another appliance.
1634     local body = newPart("Body", size, CFrame.new(pos), entry.col, entry.mat, parent)
1635     local tiers = 3 + (math.floor(size.Y * 10) % 2)
1636     local frontOnX = wallSide == "W" or wallSide == "E"
1637     local frontDir = (wallSide == "W" or wallSide == "N") and 1 or -1
1638     for tier = 1, tiers do
1639         local fy = -size.Y / 2 + (tier - 0.5) * (size.Y / tiers)
1640         local frontSize = frontOnX
1641             and Vector3.new(size.X * 0.06, size.Y / tiers * 0.82, size.Z * 0.86)
1642             or Vector3.new(size.X * 0.86, size.Y / tiers * 0.82, size.Z * 0.06)
1643         local frontCFrame = frontOnX
1644             and CFrame.new(pos.X + frontDir * size.X / 2 * 0.97, pos.Y + fy, pos.Z)
1645             or CFrame.new(pos.X, pos.Y + fy, pos.Z + frontDir * size.Z / 2 * 0.97)
1646         local front = newPart("Drawer_" .. tier, frontSize, frontCFrame, accent, Enum.Material.Metal, body)
1647         front.CanCollide = false
1648         local handleSize = frontOnX and Vector3.new(0.06, 0.06, size.Z * 0.4) or Vector3.new(size.X * 0.4,
1649             ↪ 0.06, 0.06)
1649         local handle = newPart("DrawerHandle_" .. tier, handleSize,
1650             frontCFrame * CFrame.new(frontOnX and 0.05 or 0, 0, 0) * (frontOnX and CFrame.new(0, 0, 0) or
1651             ↪ CFrame.new(0, 0, 0.05)),
1652             dark, Enum.Material.Metal, body)
1653         handle.CanCollide = false
1654     end
1655     return body
1656
1657 else -- "box": last-resort fallback, not used by anything in the
1658     -- catalogue any more, but kept so a future entry without a
1659     -- matching shape still gets a plate instead of a bare cube.
1660     local body = newPart("Body", size, CFrame.new(pos), entry.col, entry.mat, parent)
1661     local capH = math.min(size.Y * 0.08, 0.15)
1662     if capH > 0.03 then
1663         local cap = newPart("Cap", Vector3.new(size.X * 0.94, capH, size.Z * 0.94),
1664             CFrame.new(pos.X, pos.Y + size.Y / 2 + capH / 2, pos.Z), accent, entry.mat, body)
1665         cap.CanCollide = false
1666     end
1667     return body
1668 end
1669
1670 local function populateEquipmentCabinet(body, entry, size, wallSide, sampleCount, capacity)
1671     if not entry.plantCabinet or entry.kind == "Plant Growth Chamber" then return end
1672     local frontOnX = wallSide == "W" or wallSide == "E"
1673     local width = frontOnX and size.Z or size.X
1674     local depth = frontOnX and size.X or size.Z
1675     local shelfCount = capacity >= 9 and 4 or (capacity >= 6 and 3 or 2)
1676     for tier = 1, shelfCount do
1677         local y = -size.Y * 0.34 + (tier - 1) * (size.Y * 0.68 / math.max(1, shelfCount - 1))
1678         local shelfSize = frontOnX and Vector3.new(depth * 0.72, 0.1, width * 0.84)
1679             or Vector3.new(width * 0.84, 0.1, depth * 0.72)
1680         local shelf = newPart("EquipmentShelf_" .. tier, shelfSize, body.CFrame * CFrame.new(0, y, 0),
1681             Color3.fromRGB(138, 145, 147), Enum.Material.Metal, body)
1682         shelf.CanCollide = false
1683     end
1684     for slot = 1, capacity do
1685         local tier = math.floor((slot - 1) / 3)
1686         local column = (slot - 1) % 3
1687         local across = (column - 1) * width * 0.25
1688         local up = -size.Y * 0.34 + tier * (size.Y * 0.68 / math.max(1, shelfCount - 1))
1689         local offset = frontOnX and Vector3.new(0, up + 0.25, across) or Vector3.new(across, up + 0.25, 0)
1690         local base = body.CFrame * CFrame.new(offset)
1691         local pot = newPart("EquipmentPlant_" .. slot .. "_Tray", Vector3.new(0.58, 0.28, 0.52), base,
1692             Color3.fromRGB(126, 77, 45), Enum.Material.SmoothPlastic, body)
1693         local stem = newPart("EquipmentPlant_" .. slot .. "_Stem", Vector3.new(0.09, 0.48, 0.09),
1694             base * CFrame.new(0, 0.35, 0), Color3.fromRGB(58, 158, 70), Enum.Material.SmoothPlastic, body)
1695         local leaf = newPart("EquipmentPlant_" .. slot .. "_Leaf", Vector3.new(0.42, 0.1, 0.25),
1696             base * CFrame.new(0.17, 0.52, 0), Color3.fromRGB(72, 184, 82), Enum.Material.SmoothPlastic, body)

```

```

1697     for _, plantPart in ipairs({pot, stem, leaf}) do
1698         plantPart.CanCollide = false
1699         plantPart.CanTouch = false
1700         plantPart:SetAttribute("SampleSlot", slot)
1701         plantPart:SetAttribute("StoredTransparency", 0)
1702         if slot > sampleCount then plantPart.Transparency = 1 end
1703     end
1704 end
1705 end
1706
1707 -----
1708 -- ROOM FIT-OUT
1709 -----
1710 local EQUIP_TAG = "LabEquipment"
1711 local placedCount = 0
1712 local addGrowthCodeLabel
1713
1714 local function fitOutRoom(parent, room, levelY, levelName, rng, growthCodes, plantStationKind)
1715     local sides = {"W", "E", "N", "S"}
1716     local lenOf = {
1717         W = room.z2 - room.z1, E = room.z2 - room.z1,
1718         N = room.x2 - room.x1, S = room.x2 - room.x1,
1719     }
1720     local spans = {}
1721     for _, s in ipairs(sides) do
1722         spans[s] = freeSpans[lenOf[s], room.open[s]]
1723     end
1724
1725     local benchRuns = {}
1726     local index = 0
1727     local function wallFaceInset(side)
1728         local exterior = (side == "W" and room.x1 == 0)
1729         or (side == "E" and room.x2 == W)
1730         or (side == "N" and room.z1 == 0)
1731         or (side == "S" and room.z2 == L)
1732         return (exterior and EXT_T or INT_T / 2) + WALL_GAP
1733     end
1734
1735     local function spawn(entry, side, at, lift, liftInset)
1736         index = index + 1
1737         placedCount = placedCount + 1
1738         local inset = liftInset or (wallFaceInset(side) + P(entry.d) / 2)
1739         local px, pz, size
1740         if side == "W" then
1741             px, pz = room.x1 + inset, room.z1 + at
1742             size = Vector3.new(E(entry.d), EH(entry.h), E(entry.w))
1743         elseif side == "E" then
1744             px, pz = room.x2 - inset, room.z1 + at
1745             size = Vector3.new(E(entry.d), EH(entry.h), E(entry.w))
1746         elseif side == "N" then
1747             px, pz = room.x1 + at, room.z1 + inset
1748             size = Vector3.new(E(entry.w), EH(entry.h), E(entry.d))
1749         else
1750             px, pz = room.x1 + at, room.z2 - inset
1751             size = Vector3.new(E(entry.w), EH(entry.h), E(entry.d))
1752         end
1753         local baseY = Y(levelY) + (lift and EH(COUNTER_H) or 0)
1754         local pos = Vector3.new(X(px), baseY + size.Y / 2, X(pz))
1755         local p = buildKind(parent, entry, pos, size, side)
1756         p.Name = entry.kind:gsub("%s", "") .. "_" .. room.number .. "_" .. index
1757         p:SetAttribute("Kind", entry.kind)
1758         p:SetAttribute("Room", room.name)
1759         p:SetAttribute("Level", levelName)
1760         p:SetAttribute("WallSide", side)
1761         -- Every placed unit -- including the densely-packed "run" duplicates
1762         -- added to fill out walls -- is tagged as real, functional equipment
1763         -- so it gets the same status LED, timer/nameplate, hazard behaviour,
1764         -- and catastrophic-failure system as everything else. (Previously
1765         -- "run"-class units were left untagged, which is why they looked
1766         -- like bare, undetailed boxes -- none of the runtime binding that
1767         -- adds those visuals ever ran on them.) Pure decoration (potted
1768         -- plants, drawers, ...) is the one exception -- it's not equipment,
1769         -- so it gets its own lightweight tag instead of the full status/
1770         -- hazard/failure system.
1771         if entry.class == "decor" then
1772             CollectionService:AddTag(p, "LabDecor")

```

```

1773     else
1774         CollectionService:AddTag(p, EQUIP_TAG)
1775     end
1776     if entry.plantCabinet then
1777         local capacity = entry.capacity or 6
1778         local sampleCount = math.clamp(1 + ((room.number * 3 + index * 5) % capacity), 1, capacity)
1779         local sampleIds = {}
1780         for sampleIndex = 1, sampleCount do
1781             table.insert(sampleIds, string.format("%s-P%02d", entry.kind:gsub("%s", ""):sub(1, 6):upper(),
1782                 ⇐ sampleIndex))
1783         end
1784         p:SetAttribute("Capacity", capacity)
1785         p:SetAttribute("SampleCount", sampleCount)
1786         p:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
1787         p:SetAttribute("Occupied", sampleCount > 0)
1788         populateEquipmentCabinet(p, entry, size, side, sampleCount, capacity)
1789     end
1790     if entry.auto then
1791         p:SetAttribute("Auto", entry.auto)
1792         CollectionService:AddTag(p, "Automatable")
1793     end
1794     -- Equipment names and instructions are presented on the two large
1795     -- floor-level panels created by the runtime; no overhead label is used.
1796     return p
1797 end
1798
1799 -- 1. fitted runs on the longest available walls
1800 local order = {}
1801 for _, s in ipairs(sides) do table.insert(order, s) end
1802 table.sort(order, function(a, b) return spansTotal(spans[a]) > spansTotal(spans[b]) end)
1803
1804 -- Room status monitor: reserved FIRST, on the roomiest wall, so it never
1805 -- gets squeezed into a leftover corner sliver by equipment placed later.
1806 -- Large panel, mounted at eye level so it reads from across the room.
1807 do
1808     local monitorWidth = 220
1809     local mSide, mAt
1810     for _, side in ipairs(order) do
1811         local total = spansTotal(spans[side])
1812         local at = (total >= P(monitorWidth)) and takeSpan(spans[side], P(monitorWidth)) or nil
1813         if at then mSide, mAt = side, at; break end
1814     end
1815     if not mSide then
1816         -- Even the roomiest wall is tight: shrink to fit rather than
1817         -- falling back to a corner placement.
1818         for _, side in ipairs(order) do
1819             local total = spansTotal(spans[side])
1820             if total > 40 then
1821                 local fitWidth = math.min(monitorWidth, (total - 4) * SCALE)
1822                 local at = takeSpan(spans[side], P(fitWidth))
1823                 if at then monitorWidth, mSide, mAt = fitWidth, side, at; break end
1824             end
1825         end
1826     end
1827     if mSide then
1828         local inset = wallFaceInset(mSide)
1829         local px, pz
1830         if mSide == "W" then px, pz = room.x1 + inset, room.z1 + mAt
1831         elseif mSide == "E" then px, pz = room.x2 - inset, room.z1 + mAt
1832         elseif mSide == "N" then px, pz = room.x1 + mAt, room.z1 + inset
1833         else px, pz = room.x1 + mAt, room.z2 - inset end
1834         local mOnX = (mSide == "W" or mSide == "E")
1835         local panelSize = mOnX and Vector3.new(0.2, EH(135), E(monitorWidth))
1836             or Vector3.new(E(monitorWidth), EH(135), 0.2)
1837         local baseY = Y(levelY) + EH(175)
1838         local mPos = Vector3.new(X(px), baseY + panelSize.Y / 2, X(pz))
1839         local panel = newPart("RoomMonitor_" .. room.number, panelSize, CFrame.new(mPos),
1840             Color3.fromRGB(22, 26, 30), Enum.Material.Metal, parent)
1841         panel.CanCollide = false
1842         panel:SetAttribute("Room", room.name)
1843         panel:SetAttribute("Level", levelName)
1844         CollectionService:AddTag(panel, "RoomMonitor")
1845         -- Slim backlit frame so the panel reads as a mounted screen, not a wall patch.
1846         local frameDepth = mOnX and Vector3.new(0.05, panelSize.Y + 0.5, panelSize.Z + 0.5)
1847             or Vector3.new(panelSize.X + 0.5, panelSize.Y + 0.5, 0.05)

```

```

1848     local frameOffset = (mSide == "W") and CFrame.new(-0.14, 0, 0)
1849         or (mSide == "E") and CFrame.new(0.14, 0, 0)
1850         or (mSide == "N") and CFrame.new(0, 0, -0.14)
1851         or CFrame.new(0, 0, 0.14)
1852     local frame = newPart("RoomMonitor_" .. room.number .. "_Frame", frameDepth,
1853         panel.CFrame * frameOffset, Color3.fromRGB(235, 200, 60), Enum.Material.Metal, parent)
1854     frame.CanCollide = false
1855     local face = (mSide == "W") and Enum.NormalId.Right
1856         or (mSide == "E") and Enum.NormalId.Left
1857         or (mSide == "N") and Enum.NormalId.Back
1858         or Enum.NormalId.Front
1859     local surface = Instance.new("SurfaceGui")
1860     surface.Name = "RoomMonitorSurface"
1861     surface.Face = face
1862     surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
1863     surface.PixelsPerStud = 60
1864     surface.AlwaysOnTop = false
1865     surface.Parent = panel
1866     local header = Instance.new("TextLabel")
1867     header.Name = "Header"
1868     header.Size = UDim2.new(1, 0, 0.22, 0)
1869     header.Position = UDim2.new(0, 0, 0, 0)
1870     header.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
1871     header.BorderSizePixel = 0
1872     header.TextColor3 = Color3.fromRGB(20, 20, 20)
1873     header.TextScaled = true
1874     header.Font = Enum.Font.SourceSansBold
1875     header.Text = "ROOM MONITOR"
1876     header.Parent = surface
1877     local label = Instance.new("TextLabel")
1878     label.Name = "Status"
1879     label.Size = UDim2.new(1, 0, 0.78, 0)
1880     label.Position = UDim2.new(0, 0, 0.22, 0)
1881     label.BackgroundColor3 = Color3.fromRGB(14, 18, 20)
1882     label.BackgroundTransparency = 0.05
1883     label.BorderSizePixel = 0
1884     label.TextColor3 = Color3.fromRGB(120, 235, 170)
1885     label.TextScaled = true
1886     label.TextWrapped = true
1887     label.Font = Enum.Font.Code
1888     label.Text = room.name .. "\nSTATUS LOADING..."
1889     label.Parent = surface
1890 end
1891 end
1892
1893 -- Emergency Kit: a LIMITED, single-use supply (about 1 room in 3, not
1894 -- every room), avatar-sized, reserved right after the monitor so it
1895 -- always gets real wall space too. Using one destroys it permanently --
1896 -- health only comes back by finding another kit. The interaction and
1897 -- healing behaviour is wired at runtime (bindEmergencyKit); this just
1898 -- builds the physical cabinet.
1899 if room.number % 3 == 0 then
1900     local kitWidth, kitDepth, kitHeight = 62, 30, 154 -- cm, ~ a standing avatar's footprint/height
1901     local kSide, kAt
1902     for _, side in ipairs(order) do
1903         local at = takeSpan(spans[side], P(kitWidth))
1904         if at then kSide, kAt = side, at; break end
1905     end
1906     if kSide then
1907         local inset = wallFaceInset(kSide) + P(kitDepth) / 2
1908         local px, pz
1909         if kSide == "W" then px, pz = room.x1 + inset, room.z1 + kAt
1910         elseif kSide == "E" then px, pz = room.x2 - inset, room.z1 + kAt
1911         elseif kSide == "N" then px, pz = room.x1 + kAt, room.z1 + inset
1912         else px, pz = room.x1 + kAt, room.z2 - inset end
1913         local kOnX = (kSide == "W" or kSide == "E")
1914         local cabinetSize = kOnX and Vector3.new(E(kitDepth), EH(kitHeight), E(kitWidth))
1915             or Vector3.new(E(kitWidth), EH(kitHeight), E(kitDepth))
1916         local baseY = Y(levelY)
1917         local cabinet = newPart("EmergencyKit_" .. room.number, cabinetSize,
1918             CFrame.new(X(px), baseY + cabinetSize.Y / 2, X(pz)), Color3.fromRGB(225, 40, 35),
1919             Enum.Material.SmoothPlastic, parent)
1920         cabinet.CanCollide = true
1921         cabinet:SetAttribute("Room", room.name)
1922         cabinet:SetAttribute("Level", levelName)
1923         cabinet:SetAttribute("WallSide", kSide)

```

```

1923     CollectionService:AddTag(cabinet, "EmergencyKit")
1924
1925     -- White cross + label on the door face so it reads as a first-aid
1926     -- cabinet, not a generic red box.
1927     local face = (kSide == "W") and Enum.NormalId.Right
1928         or (kSide == "E") and Enum.NormalId.Left
1929         or (kSide == "N") and Enum.NormalId.Back
1930         or Enum.NormalId.Front
1931     local surface = Instance.new("SurfaceGui")
1932     surface.Name = "EmergencyKitFace"
1933     surface.Face = face
1934     surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
1935     surface.PixelsPerStud = 60
1936     surface.AlwaysOnTop = false
1937     surface.Parent = cabinet
1938     local crossV = Instance.new("Frame")
1939     crossV.Size = UDim2.new(0.22, 0, 0.55, 0)
1940     crossV.Position = UDim2.new(0.39, 0, 0.12, 0)
1941     crossV.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1942     crossV.BorderSizePixel = 0
1943     crossV.Parent = surface
1944     local crossH = Instance.new("Frame")
1945     crossH.Size = UDim2.new(0.55, 0, 0.22, 0)
1946     crossH.Position = UDim2.new(0.12, 0, 0.28, 0)
1947     crossH.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1948     crossH.BorderSizePixel = 0
1949     crossH.Parent = surface
1950     local kitLabel = Instance.new("TextLabel")
1951     kitLabel.Size = UDim2.new(1, 0, 0.22, 0)
1952     kitLabel.Position = UDim2.new(0, 0, 0.72, 0)
1953     kitLabel.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1954     kitLabel.BackgroundTransparency = 0.05
1955     kitLabel.BorderSizePixel = 0
1956     kitLabel.Text = "EMERGENCY KIT"
1957     kitLabel.TextColor3 = Color3.fromRGB(200, 30, 25)
1958     kitLabel.TextScaled = true
1959     kitLabel.Font = Enum.Font.SourceSansBold
1960     kitLabel.Parent = surface
1961
1962     local prompt = Instance.new("ProximityPrompt")
1963     prompt.Name = "EmergencyKitPrompt"
1964     prompt.ActionText = "Use Emergency Kit"
1965     prompt.ObjectText = "Restore health"
1966     prompt.KeyboardKeyCode = Enum.KeyCode.E
1967     prompt.MaxActivationDistance = 9
1968     prompt.RequiresLineOfSight = false
1969     prompt.Parent = cabinet
1970 end
1971 end
1972
1973 -- Reserve wall space for this room's share of R1-R200 before normal
1974 -- furniture is packed. A shuffled wall order keeps the chambers spread
1975 -- around the building, but every one remains fridge-like and wall-backed.
1976 for codeIndex, code in ipairs(growthCodes or {}) do
1977     local sizeVariants = {
1978         {w = 100, d = 90, h = 220, capacity = 6},
1979         {w = 115, d = 95, h = 235, capacity = 9},
1980         {w = 130, d = 100, h = 250, capacity = 12},
1981         {w = 145, d = 105, h = 265, capacity = 16},
1982     }
1983     local variant = sizeVariants[((tonumber(code:match("%d+")) or codeIndex) - 1) % #sizeVariants + 1]
1984     local entry = {
1985         kind = "Plant Growth Chamber",
1986         label = code,
1987         w = variant.w, d = variant.d, h = variant.h,
1988         col = Color3.fromRGB(215, 220, 218),
1989         mat = Enum.Material.Metal,
1990         shape = "appliance",
1991     }
1992     local chamber
1993     local firstSide = rng.int(1, #order)
1994     -- Placement ladder. The preferred size is tried first on every wall;
1995     -- if the room's walls are already busy (the monitor, the emergency
1996     -- kit and the guaranteed plant station are all reserved ahead of the
1997     -- chambers) it steps down through the smaller cabinets, and only as a
1998     -- last resort tightens the neighbour gap. Without this ladder a code

```

```

1999 -- that found no span was silently dropped, which is exactly why the
2000 -- narrow east-side rooms kept landing 1-2 chambers under their share
2001 -- and the building missed its 50-per-floor target.
2002 local fallbackOrder = {variant}
2003 local smaller = {}
2004 for _, alt in ipairs(sizeVariants) do
2005     if alt ~= variant then table.insert(smaller, alt) end
2006 end
2007 table.sort(smaller, function(a, b) return a.w < b.w end)
2008 for _, alt in ipairs(smaller) do table.insert(fallbackOrder, alt) end
2009 -- Bottom of the ladder: a compact under-counter cabinet that exists
2010 -- only to rescue the very last code in a room whose walls are down to
2011 -- slivers. It is never picked as a room's preferred size, so it does
2012 -- not change how the building normally looks.
2013 table.insert(fallbackOrder, {w = 85, d = 78, h = 196, capacity = 5})
2014
2015 for _, gap in ipairs({ALONG_GAP, 10}) do
2016     for _, candidate in ipairs(fallbackOrder) do
2017         for try = 0, #order - 1 do
2018             local side = order[((firstSide + try - 1) % #order) + 1]
2019             local at = takeSpanGap(spans[side], P(candidate.w), gap)
2020             if at then
2021                 variant = candidate
2022                 entry.w, entry.d, entry.h = candidate.w, candidate.d, candidate.h
2023                 chamber = spawn(entry, side, at, false)
2024                 break
2025             end
2026         end
2027         if chamber then break end
2028     end
2029     if chamber then break end
2030 end
2031 if chamber then
2032     local codeNumber = tonumber(code:match("%d+")) or codeIndex
2033     local capacity = variant.capacity
2034     local sampleCount = math.clamp(1 + ((codeNumber * 7) % capacity), 1, capacity)
2035     local sampleIds = {}
2036     for sampleIndex = 1, sampleCount do
2037         table.insert(sampleIds, string.format("%s-S%02d", code, sampleIndex))
2038     end
2039     -- Start safe; the runtime introduces a small, player-scaled workload.
2040     local initiallyFailed = false
2041     local floorLabel = levelName == "Basement" and "BASEMENT"
2042     or ("FLOOR " .. (levelName:match("%d+") or levelName))
2043     local failureReasons = {
2044         "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
2045         "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
2046     }
2047     local temperature = initiallyFailed and (31 + codeNumber % 8) or (22 + codeNumber % 4)
2048     local humidity = initiallyFailed and (38 + codeNumber % 19) or (67 + codeNumber % 14)
2049     local ventilation = initiallyFailed and (12 + codeNumber % 24) or (62 + codeNumber % 29)
2050     chamber.Name = "GrowthChamber_" .. code
2051     chamber:SetAttribute("GrowthCode", code)
2052     chamber:SetAttribute("FloorLabel", floorLabel)
2053     chamber:SetAttribute("Capacity", capacity)
2054     chamber:SetAttribute("SampleCount", sampleCount)
2055     chamber:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
2056     chamber:SetAttribute("Occupied", sampleCount > 0)
2057     chamber:SetAttribute("PlantId", sampleIds[1] or code)
2058     chamber:SetAttribute("GrowthStage", 0)
2059     chamber:SetAttribute("Treatments", "")
2060     chamber:SetAttribute("Genome", "Wild Type")
2061     chamber:SetAttribute("PlantLife", 100)
2062     chamber:SetAttribute("RecoveryUntil", 0)
2063     chamber:SetAttribute("Moisture", humidity)
2064     chamber:SetAttribute("Temperature", temperature)
2065     chamber:SetAttribute("Humidity", humidity)
2066     chamber:SetAttribute("Ventilation", ventilation)
2067     chamber:SetAttribute("OperationStarted", os.time() - (codeNumber * 173) % 10800)
2068     chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M", os.time() - (codeNumber % 6 + 1)
2069     ↪ * 86400))
2069     chamber:SetAttribute("FunctionalState", initiallyFailed and "FAILED" or "WORKING")
2070     chamber:SetAttribute("RepairKey", "R")
2071     chamber:SetAttribute("RepairInstruction", initiallyFailed
2072     and ("Go to " .. floorLabel .. " " * CHAMBER .. code .. " " * HOLD R TO REPAIR")
2073     or "Unit working • E TAKE / DROP • R REPAIR")

```



```

2074     chamber:SetAttribute("FailureReason", initiallyFailed and failureReasons[((codeNumber - 1) %
↪ #failureReasons) + 1] or "")
2075     chamber:SetAttribute("FailureSince", initiallyFailed and os.time() or 0)
2076     chamber:SetAttribute("EvacuationDestination", "")
2077     CollectionService:AddTag(chamber, "PlantGrowthChamber")
2078     -- Reference-inspired controlled-environment cabinet: insulated
2079     -- body, glass door, bright tier lighting, tray shelves, rear vents,
2080     -- header controller and caster wheels. The translucent body is only
2081     -- a backing volume; the metal frame gives it a refrigerator profile.
2082     chamber.Material = Enum.Material.Glass
2083     chamber.Color = Color3.fromRGB(205, 216, 212)
2084     chamber.Transparency = 0.88
2085     local wallSide = chamber:GetAttribute("WallSide")
2086     local frontOnX = wallSide == "W" or wallSide == "E"
2087     local direction = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
2088     local frameColour = Color3.fromRGB(62, 69, 72)
2089     local innerWidth = frontOnX and chamber.Size.Z or chamber.Size.X
2090     local innerDepth = frontOnX and chamber.Size.X or chamber.Size.Z
2091     local shellColour = codeNumber % 5 == 0 and Color3.fromRGB(43, 48, 50)
2092     or (codeNumber % 3 == 0 and Color3.fromRGB(198, 203, 202) or Color3.fromRGB(232, 234, 231))
2093     local function interiorOffset(across, up, fromFront)
2094         if frontOnX then return CFrame.new(direction * (innerDepth / 2 - fromFront), up, across) end
2095         return CFrame.new(across, up, direction * (innerDepth / 2 - fromFront))
2096     end
2097     local frameParts = {}
2098     local shellThickness = 0.18
2099     -- Opaque insulated rear, side, top and floor panels leave only
2100     -- the front door transparent, matching real growth cabinets.
2101     if frontOnX then
2102         table.insert(frameParts, newPart("InsulatedBack",
2103             Vector3.new(shellThickness, chamber.Size.Y * 0.96, innerWidth),
2104             chamber.CFrame * CFrame.new(-direction * (innerDepth / 2 - shellThickness / 2), 0, 0),
2105             shellColour, Enum.Material.Metal, chamber))
2106         for _, sideSign in ipairs({-1, 1}) do
2107             table.insert(frameParts, newPart("InsulatedSide",
2108                 Vector3.new(innerDepth, chamber.Size.Y * 0.96, shellThickness),
2109                 chamber.CFrame * CFrame.new(0, 0, sideSign * (innerWidth / 2 - shellThickness / 2)),
2110                 shellColour, Enum.Material.Metal, chamber))
2111         end
2112         table.insert(frameParts, newPart("InsulatedTop", Vector3.new(innerDepth, shellThickness,
2113             ↪ innerWidth),
2114             chamber.CFrame * CFrame.new(0, chamber.Size.Y / 2 - shellThickness / 2, 0),
2115             shellColour, Enum.Material.Metal, chamber))
2116         table.insert(frameParts, newPart("InsulatedFloor", Vector3.new(innerDepth, shellThickness,
2117             ↪ innerWidth),
2118             chamber.CFrame * CFrame.new(0, -chamber.Size.Y / 2 + shellThickness / 2, 0),
2119             shellColour, Enum.Material.Metal, chamber))
2120     else
2121         table.insert(frameParts, newPart("InsulatedBack",
2122             Vector3.new(innerWidth, chamber.Size.Y * 0.96, shellThickness),
2123             chamber.CFrame * CFrame.new(0, 0, -direction * (innerDepth / 2 - shellThickness / 2)),
2124             shellColour, Enum.Material.Metal, chamber))
2125         for _, sideSign in ipairs({-1, 1}) do
2126             table.insert(frameParts, newPart("InsulatedSide",
2127                 Vector3.new(shellThickness, chamber.Size.Y * 0.96, innerDepth),
2128                 chamber.CFrame * CFrame.new(sideSign * (innerWidth / 2 - shellThickness / 2), 0, 0),
2129                 shellColour, Enum.Material.Metal, chamber))
2130         end
2131         table.insert(frameParts, newPart("InsulatedTop", Vector3.new(innerWidth, shellThickness,
2132             ↪ innerDepth),
2133             chamber.CFrame * CFrame.new(0, chamber.Size.Y / 2 - shellThickness / 2, 0),
2134             shellColour, Enum.Material.Metal, chamber))
2135         table.insert(frameParts, newPart("InsulatedFloor", Vector3.new(innerWidth, shellThickness,
2136             ↪ innerDepth),
2137             chamber.CFrame * CFrame.new(0, -chamber.Size.Y / 2 + shellThickness / 2, 0),
2138             shellColour, Enum.Material.Metal, chamber))
2139     end
2140     local frontDistance = innerDepth / 2 + 0.08
2141     if frontOnX then
2142         local frontX = direction * frontDistance
2143         table.insert(frameParts, newPart("GlassFrameLeft", Vector3.new(0.2, chamber.Size.Y * 0.94,
2144             ↪ 0.22), chamber.CFrame * CFrame.new(frontX, 0, -chamber.Size.Z * 0.46), frameColour,
2145             ↪ Enum.Material.Metal, chamber))
2146         table.insert(frameParts, newPart("GlassFrameRight", Vector3.new(0.2, chamber.Size.Y * 0.94,
2147             ↪ 0.22), chamber.CFrame * CFrame.new(frontX, 0, chamber.Size.Z * 0.46), frameColour,
2148             ↪ Enum.Material.Metal, chamber))

```

```

2141         table.insert(frameParts, newPart("GlassFrameTop", Vector3.new(0.2, 0.24, chamber.Size.Z *
↳ 0.94), chamber.CFrame * CFrame.new(frontX, chamber.Size.Y * 0.47, 0), frameColour,
↳ Enum.Material.Metal, chamber))
2142         table.insert(frameParts, newPart("GlassFrameBottom", Vector3.new(0.2, 0.24, chamber.Size.Z *
↳ 0.94), chamber.CFrame * CFrame.new(frontX, -chamber.Size.Y * 0.47, 0), frameColour,
↳ Enum.Material.Metal, chamber))
2143     else
2144         local frontZ = direction * frontDistance
2145         table.insert(frameParts, newPart("GlassFrameLeft", Vector3.new(0.22, chamber.Size.Y * 0.94,
↳ 0.2), chamber.CFrame * CFrame.new(-chamber.Size.X * 0.46, 0, frontZ), frameColour,
↳ Enum.Material.Metal, chamber))
2146         table.insert(frameParts, newPart("GlassFrameRight", Vector3.new(0.22, chamber.Size.Y * 0.94,
↳ 0.2), chamber.CFrame * CFrame.new(chamber.Size.X * 0.46, 0, frontZ), frameColour,
↳ Enum.Material.Metal, chamber))
2147         table.insert(frameParts, newPart("GlassFrameTop", Vector3.new(chamber.Size.X * 0.94, 0.24,
↳ 0.2), chamber.CFrame * CFrame.new(0, chamber.Size.Y * 0.47, frontZ), frameColour,
↳ Enum.Material.Metal, chamber))
2148         table.insert(frameParts, newPart("GlassFrameBottom", Vector3.new(chamber.Size.X * 0.94, 0.24,
↳ 0.2), chamber.CFrame * CFrame.new(0, -chamber.Size.Y * 0.47, frontZ), frameColour,
↳ Enum.Material.Metal, chamber))
2149     end
2150
2151     -- Four ventilated shelves and four white grow-light strips.
2152     for tier = 0, 3 do
2153         local shelfY = -chamber.Size.Y * 0.35 + tier * chamber.Size.Y * 0.22
2154         local shelfSize = frontOnX
2155             and Vector3.new(innerDepth * 0.72, 0.1, innerWidth * 0.86)
2156             or Vector3.new(innerWidth * 0.86, 0.1, innerDepth * 0.72)
2157         local shelf = newPart("PlantShelf_" .. (tier + 1), shelfSize,
2158             chamber.CFrame * CFrame.new(0, shelfY - 0.28, 0), Color3.fromRGB(128, 135, 136),
2159             ↳ Enum.Material.Metal, chamber)
2160         local lightSize = frontOnX and Vector3.new(0.14, 0.1, innerWidth * 0.75) or
2161             ↳ Vector3.new(innerWidth * 0.75, 0.1, 0.14)
2162         local light = newPart("GrowLight_" .. (tier + 1), lightSize,
2163             chamber.CFrame * interiorOffset(0, shelfY + chamber.Size.Y * 0.13, innerDepth * 0.45),
2164             Color3.fromRGB(255, 248, 222), Enum.Material.Neon, chamber)
2165         table.insert(frameParts, shelf); table.insert(frameParts, light)
2166     end
2167
2168     -- Rear ventilation grille and a raised controller housing.
2169     local ventSize = frontOnX and Vector3.new(0.12, chamber.Size.Y * 0.24, innerWidth * 0.45) or
2170         ↳ Vector3.new(innerWidth * 0.45, chamber.Size.Y * 0.24, 0.12)
2171     local vent = newPart("VentilationGrille", ventSize,
2172         chamber.CFrame * interiorOffset(0, chamber.Size.Y * 0.2, innerDepth * 0.92), Color3.fromRGB(70,
2173         ↳ 78, 82), Enum.Material.DiamondPlate, chamber)
2174     table.insert(frameParts, vent)
2175     local controllerSize = frontOnX and Vector3.new(innerDepth * 0.9, 0.72, innerWidth * 0.94) or
2176         ↳ Vector3.new(innerWidth * 0.94, 0.72, innerDepth * 0.9)
2177     local controller = newPart("ClimateController", controllerSize,
2178         chamber.CFrame * CFrame.new(0, chamber.Size.Y * 0.5 + 0.34, 0), Color3.fromRGB(226, 229, 228),
2179         ↳ Enum.Material.Metal, chamber)
2180     controller.CanCollide = false
2181     local controllerFace = newPart("ControllerScreen",
2182         frontOnX and Vector3.new(0.12, 0.46, innerWidth * 0.58) or Vector3.new(innerWidth * 0.58, 0.46,
2183         ↳ 0.12),
2184         chamber.CFrame * interiorOffset(0, chamber.Size.Y * 0.5 + 0.34, -0.18),
2185         initiallyFailed and Color3.fromRGB(220, 55, 45) or Color3.fromRGB(40, 150, 95),
2186         ↳ Enum.Material.Neon, chamber)
2187     controllerFace.CanCollide = false
2188     controllerFace:SetAttribute("ChamberStatusLamp", true)
2189
2190     -- Small caster wheels like the mobile cabinets in the references.
2191     for _, across in ipairs({-innerWidth * 0.38, innerWidth * 0.38}) do
2192         for _, depthSign in ipairs({-1, 1}) do
2193             local wheelOffset
2194             if frontOnX then wheelOffset = CFrame.new(depthSign * innerDepth * 0.32, -chamber.Size.Y /
2195                 ↳ 2 - 0.23, across)
2196             else wheelOffset = CFrame.new(across, -chamber.Size.Y / 2 - 0.23, depthSign * innerDepth *
2197                 ↳ 0.32) end
2198             local wheel = newPart("CasterWheel", Vector3.new(0.38, 0.38, 0.22), chamber.CFrame *
2199                 ↳ wheelOffset,
2200                 Color3.fromRGB(28, 30, 32), Enum.Material.Rubber, chamber)
2201             wheel.Shape = Enum.PartType.Cylinder
2202             wheel.CanCollide = false
2203         end
2204     end
2205 end

```

```

2194
2195     local door = chamber:FindFirstChild("Door")
2196     if door then
2197         door.Material = Enum.Material.Glass
2198         door.Color = Color3.fromRGB(205, 232, 225)
2199         door.Transparency = 0.38
2200         addGrowthCodeLabel(door, code)
2201     end
2202     for _, framePart in ipairs(frameParts) do
2203         framePart.CanCollide = false
2204         framePart.CanTouch = false
2205     end
2206
2207     -- Individually addressable germinated-seed trays. Capacity follows
2208     -- physical cabinet size: 4, 6, 8, or 12 plants.
2209     for slot = 1, capacity do
2210         local tier = math.floor((slot - 1) / 3)
2211         local column = (slot - 1) % 3
2212         local across = (column - 1) * innerWidth * 0.27
2213         if slot == 10 then across = 0 end
2214         local up = -chamber.Size.Y * 0.35 + tier * chamber.Size.Y * 0.22
2215         local plantBase = chamber.CFrame * interiorOffset(across, up, 0.48)
2216         local slotName = string.format("ChamberSample_%02d_", slot)
2217         local pot = newPart(slotName .. "Tray", Vector3.new(0.64, 0.32, 0.56), plantBase,
2218             Color3.fromRGB(128, 75, 43), Enum.Material.SmoothPlastic, chamber)
2219         local stem = newPart(slotName .. "Stem", Vector3.new(0.1, 0.55, 0.1),
2220             plantBase * CFrame.new(0, 0.4, 0), Color3.fromRGB(55, 155, 68),
2221             Enum.Material.SmoothPlastic, chamber)
2222         local leafL = newPart(slotName .. "LeafL", Vector3.new(0.36, 0.09, 0.22),
2223             plantBase * CFrame.new(-0.18, 0.53, 0), Color3.fromRGB(72, 184, 82),
2224             Enum.Material.SmoothPlastic, chamber)
2225         local leafR = newPart(slotName .. "LeafR", Vector3.new(0.36, 0.09, 0.22),
2226             plantBase * CFrame.new(0.18, 0.68, 0), Color3.fromRGB(72, 184, 82),
2227             Enum.Material.SmoothPlastic, chamber)
2228         for _, plantPart in ipairs({pot, stem, leafL, leafR}) do
2229             plantPart.CanCollide = false
2230             plantPart.CanTouch = false
2231             plantPart:SetAttribute("SampleSlot", slot)
2232             plantPart:SetAttribute("StoredTransparency", 0)
2233             if slot > sampleCount then plantPart.Transparency = 1 end
2234         end
2235     end
2236
2237     end
2238 end
2239
2240 -- Every level receives the complete five-machine plant-development
2241 -- route, distributed across separate rooms and packed against walls.
2242 -- This guarantees that players and all 50 bots can finish a full cycle.
2243 if plantStationKind then
2244     local stationColours = {
2245         ["Growth Hormone Mixer"] = Color3.fromRGB(100, 190, 110),
2246         ["GMO Injector"] = Color3.fromRGB(170, 100, 215),
2247         ["Nutrient Infuser"] = Color3.fromRGB(210, 165, 65),
2248         ["UV Growth Scanner"] = Color3.fromRGB(70, 145, 225),
2249         ["Pollination Station"] = Color3.fromRGB(225, 125, 170),
2250     }
2251     local entry = {
2252         kind = plantStationKind,
2253         w = 115, d = 95, h = 235, capacity = 8,
2254         col = stationColours[plantStationKind] or COL.green,
2255         mat = Enum.Material.Metal,
2256         shape = "appliance",
2257         plantCabinet = true,
2258         auto = "pulse",
2259     }
2260     local firstSide = rng.int(1, #order)
2261     for try = 0, #order - 1 do
2262         local side = order[((firstSide + try - 1) % #order) + 1]
2263         local at = takeSpan(spans[side], P(entry.w))
2264         if at then spawn(entry, side, at, false); break end
2265     end
2266 end
2267
2268 local area = (room.x2 - room.x1) * (room.z2 - room.z1)
2269 -- Every wall gets its own island run, and each run keeps packing units

```

```

2267 -- for as long as its wall physically has room -- not a fixed rep count,
2268 -- so no wall is left with a leftover gap a unit could still have filled.
2269 -- A short streak of consecutive misses (picked entry just didn't fit
2270 -- the last sliver) is what actually ends a wall's run, not a target.
2271 if #BY_CLASS.run > 0 then
2272     for _, side in ipairs(order) do
2273         local guard, misses = 0, 0
2274         while guard < 200 and misses < 6 do
2275             guard = guard + 1
2276             local entry = weightedPick(BY_CLASS.run, rng)
2277             local need = P(entry.w)
2278             local at = takeSpan(spans[side], need)
2279             if at then
2280                 misses = 0
2281                 spawn(entry, side, at, false)
2282                 table.insert(benchRuns, {
2283                     side = side,
2284                     inset = wallFaceInset(side) + P(entry.d) / 2,
2285                     spans = {{at - need / 2 + 12, at + need / 2 - 12}},
2286                     height = entry.h,
2287                 })
2288             else
2289                 misses = misses + 1
2290             end
2291         end
2292     end
2293 end
2294
2295 -- 2. free-standing units -- fills whatever wall space the island runs
2296 -- above didn't claim, again running until every wall genuinely can't
2297 -- take another unit rather than stopping at an area-based headcount.
2298 if #BY_CLASS.floor > 0 then
2299     local guard, misses = 0, 0
2300     while guard < 3000 and misses < 24 do
2301         guard = guard + 1
2302         local entry = weightedPick(BY_CLASS.floor, rng)
2303         local side = order[rng.int(1, #order)]
2304         local at = takeSpan(spans[side], P(entry.w))
2305         if at then
2306             misses = 0
2307             spawn(entry, side, at, false)
2308         else
2309             misses = misses + 1
2310         end
2311     end
2312 end
2313
2314 -- 2b. decoration pass -- potted plants, flower planters, and lab
2315 -- storage furniture pack into whatever slivers are still left on every
2316 -- wall once real equipment can no longer fit, using a much smaller gap
2317 -- since decor carries no status readout for a neighbour to overlap.
2318 -- This is what actually gets a room to "no free wall space" rather
2319 -- than just "no free space equipment can use."
2320 if #BY_CLASS.decor > 0 then
2321     for _, side in ipairs(order) do
2322         local guard, misses = 0, 0
2323         while guard < 300 and misses < 8 do
2324             guard = guard + 1
2325             local entry = weightedPick(BY_CLASS.decor, rng)
2326             local at = takeSpanGap(spans[side], P(entry.w), 6)
2327             if at then
2328                 misses = 0
2329                 spawn(entry, side, at, false)
2330             else
2331                 misses = misses + 1
2332             end
2333         end
2334     end
2335 end
2336
2337 -- 3. instruments on the bench runs
2338 if #BY_CLASS.top > 0 then
2339     for _, run in ipairs(benchRuns) do
2340         local slots = rng.int(1, 3)
2341         for _ = 1, slots do
2342             local entry = weightedPick(BY_CLASS.top, rng)

```

```

2343         local at = takeSpan(run.spans, P(entry.w))
2344         if at then spawn(entry, run.side, at, true, run.inset) end
2345     end
2346 end
2347
2348
2349 -- 4. safety kit, one of each, wherever it still fits
2350 for _, entry in ipairs(BY_CLASS.duty) do
2351     for _, side in ipairs(order) do
2352         local at = takeSpan(spans[side], P(entry.w))
2353         if at then spawn(entry, side, at, false); break end
2354     end
2355 end
2356
2357 -- (Room status monitor is reserved earlier, right after `order` is
2358 -- computed, so it always claims prime wall space instead of a leftover
2359 -- corner sliver – see the block above, right after the wall-side sort.)
2360 end
2361
2362 -----
2363 -- R1-R200 PLANT GROWTH CHAMBERS
2364 -- Fifty variable-size growing chambers are placed on every level. Codes are
2365 -- printed on the door itself, so labels never float across the room.
2366 -----
2367 addGrowthCodeLabel = function(door, code)
2368     local labelFaces = (door.Size.X < door.Size.Z)
2369         and {Enum.NormalId.Left, Enum.NormalId.Right}
2370         or {Enum.NormalId.Front, Enum.NormalId.Back}
2371     for _, normal in ipairs(labelFaces) do
2372         local surface = Instance.new("SurfaceGui")
2373         surface.Name = "GrowthCodeLabel"
2374         surface.Face = normal
2375         surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2376         surface.PixelsPerStud = 70
2377         surface.AlwaysOnTop = false
2378         surface.Parent = door
2379
2380         local label = Instance.new("TextLabel")
2381         label.Size = UDim2.new(0.72, 0, 0.3, 0)
2382         label.Position = UDim2.new(0.14, 0, 0.08, 0)
2383         label.BackgroundColor3 = Color3.fromRGB(20, 44, 34)
2384         label.BackgroundTransparency = 0.08
2385         label.BorderSizePixel = 0
2386         label.Text = code
2387         label.TextColor3 = Color3.fromRGB(170, 255, 185)
2388         label.TextScaled = true
2389         label.Font = Enum.Font.GothamBold
2390         label.Parent = surface
2391
2392         local instruction = Instance.new("TextLabel")
2393         instruction.Size = UDim2.new(0.82, 0, 0.18, 0)
2394         instruction.Position = UDim2.new(0.09, 0, 0.72, 0)
2395         instruction.BackgroundTransparency = 1
2396         instruction.Text = "E TAKE / DROP • R REPAIR"
2397         instruction.TextColor3 = Color3.fromRGB(225, 235, 225)
2398         instruction.TextScaled = true
2399         instruction.Font = Enum.Font.Gotham
2400         instruction.Parent = surface
2401     end
2402 end
2403
2404 -- Usable wall a room can actually back a chamber against: each of the four
2405 -- walls minus the corner keep-outs at both of its ends.
2406 local function growthWallWeight(room)
2407     local acrossX = math.max(0, (room.x2 - room.x1) - 2 * CORNER)
2408     local acrossZ = math.max(0, (room.z2 - room.z1) - 2 * CORNER)
2409     return math.max(acrossX * 2 + acrossZ * 2, 1)
2410 end
2411
2412 -- The 50 R-codes a floor owns are shared out in proportion to that usable
2413 -- wall, not evenly. The plan is deliberately lopsided -- one long room runs
2414 -- the entire west side while the east rooms are short bays -- so an even
2415 -- split kept asking the narrow bays for more chambers than their walls could
2416 -- physically hold (the overflow was silently dropped) while the long west
2417 -- room sat half empty. Largest-remainder rounding keeps each floor's total
2418 -- at exactly 50 and the codes contiguous.

```

```

2419 local function growthCodesForRoom(firstCode, roomIndex, rooms)
2420     local weights, total = {}, 0
2421     for i, r in ipairs(rooms) do
2422         weights[i] = growthWallWeight(r)
2423         total = total + weights[i]
2424     end
2425
2426     local counts, assigned, remainders = {}, 0, {}
2427     for i = 1, #rooms do
2428         local exact = 50 * weights[i] / total
2429         counts[i] = math.floor(exact)
2430         assigned = assigned + counts[i]
2431         table.insert(remainders, {index = i, frac = exact - counts[i]})
2432     end
2433     table.sort(remainders, function(a, b)
2434         if a.frac == b.frac then return a.index < b.index end
2435         return a.frac > b.frac
2436     end)
2437     for i = 1, 50 - assigned do
2438         local pick = remainders[i]
2439         if not pick then break end
2440         counts[pick.index] = counts[pick.index] + 1
2441     end
2442
2443     local before = 0
2444     for i = 1, roomIndex - 1 do before = before + counts[i] end
2445     local codes = {}
2446     for i = 1, counts[roomIndex] do
2447         table.insert(codes, "R" .. (firstCode + before + i - 1))
2448     end
2449     return codes
2450 end
2451
2452 -----
2453 -- BUILD
2454 -----
2455 local BOTTOM_LEVEL = -1
2456 local TOP_LEVEL = LEVELS[#LEVELS].index
2457
2458 local basementModel, basementRooms = buildBasement()
2459 local levelModels = {}
2460
2461 local GUARANTEED_PLANT_STATIONS = {
2462     "Growth Hormone Mixer",
2463     "GMO Injector",
2464     "Nutrient Infuser",
2465     "UV Growth Scanner",
2466     "Pollination Station",
2467 }
2468 for _, lv in ipairs(LEVELS) do
2469     local m = buildLevelShell(lv.index, lv.name, lv.label, lv.rooms)
2470     levelModels[lv.index] = m
2471     local folder = Instance.new("Folder")
2472     folder.Name = "Equipment"
2473     folder.Parent = m
2474     for roomIndex, r in ipairs(lv.rooms) do
2475         -- The ground floor's first room is the entrance lobby: it gets a
2476         -- full equipment fit-out just like every other room, but the mid-
2477         -- room strip where the role-selection kiosks/spawn/chevrons live
2478         -- (roughly z 150-460 in the room's own coordinates) is reserved as
2479         -- a keep-clear zone first, the same way a door or window reserves
2480         -- clearance, so wall equipment never grows over the kiosks.
2481         local isEntranceLobby = (lv.index == 0 and roomIndex == 1)
2482         if isEntranceLobby then
2483             table.insert(r.open.W, {at = 150, width = 310})
2484             table.insert(r.open.E, {at = 150, width = 310})
2485         end
2486         fitOutRoom(folder, r, lv.index * F2F, lv.name,
2487             newRng(RNG_SEED + lv.index * 1000 + r.number),
2488             growthCodesForRoom(1 + lv.index * 50, roomIndex, lv.rooms),
2489             GUARANTEED_PLANT_STATIONS[roomIndex])
2490     end
2491 end
2492
2493 do
2494     local folder = Instance.new("Folder")

```

```

2495     folder.Name = "Equipment"
2496     folder.Parent = basementModel
2497     for roomIndex, r in ipairs(basementRooms) do
2498         -- Basement's code block starts right after every above-ground
2499         -- level's own 50-code block, whatever the level count is (used to
2500         -- be a hard-coded 151, back when there were only 3 levels above
2501         -- it -- with more levels that collided with a level's own codes).
2502         fitOutRoom(folder, r, BOTTOM_LEVEL * F2F, "Basement",
2503             newRng(RNG_SEED + 7777 + r.number),
2504             growthCodesForRoom(1 + #LEVELS * 50, roomIndex, basementRooms),
2505             GUARANTEED_PLANT_STATIONS[roomIndex])
2506     end
2507 end
2508
2509 -- Visible test scientists are distributed over EVERY level -- basement
2510 -- included -- at a fixed count per floor, rather than a fixed total split
2511 -- across just the original four. That keeps bot density per floor roughly
2512 -- constant as floors are added or removed instead of spreading the same
2513 -- headcount thinner across a bigger building. BOTS_PER_LEVEL is kept
2514 -- moderate (not the 50/floor first asked for) because with basement + all
2515 -- LEVELS that would put ~950 concurrent pathfinding NPCs on one server --
2516 -- a real lag/timeout risk. This keeps the total in the low hundreds while
2517 -- still reading as far denser per floor than before. The runtime assigns
2518 -- each one the same loop a player performs: answer a failed-chamber alarm,
2519 -- visibly carry one germinated plant through a growth process, and rescue
2520 -- it into a working R chamber with capacity.
2521 -----
2522 -- CORRIDOR FIXTURES -- health stations and sidearm cabinets
2523 --
2524 -- These hang on the corridor walls rather than inside rooms on purpose:
2525 -- room wall space is already fully budgeted (monitor, kit, plant station,
2526 -- the floor's share of R-code chambers, then furniture), so putting these
2527 -- in rooms would push chambers back under the 50-per-floor target. The
2528 -- corridor is circulation space nobody was using. They are CanCollide off
2529 -- so they can never block a doorway or an NPC's path.
2530 -----
2531 -- Ten of each support pickup per floor gives players a nearby recovery or
2532 -- weapon option without putting a cabinet in a doorway.
2533 local HEALTH_STATIONS_PER_FLOOR = 10
2534 local WEAPON_STATIONS_PER_FLOOR = 10
2535
2536 -- Weapon kinds the corridor cabinets hand out, one per cabinet per floor.
2537 -- Declared here because the cabinets are built long before IMPACT_STYLE (which
2538 -- defines the weapons themselves) appears further down the file.
2539 local PC_WEAPONS = {"Lab Sidearm", "Lab Shotgun", "Lab Rifle"}
2540
2541 -- Poster copy. Drawn procedurally from Frames and TextLabels -- no imported
2542 -- decals, so nothing here depends on an uploaded image asset.
2543 local POSTER_TITLES = {
2544     {title = "GERMINATION RATE BY CULTIVAR", seed = 3, colour = Color3.fromRGB(46, 122, 84),
2545      body = "n = 400 seeds per lot, 4 replicates of 100. Counts at day 4 and day 8. Rolled paper substrate,
2546      ↪ 20/30 C alternating."},
2547     {title = "SEED VIABILITY - TETRAZOLIUM", seed = 5, colour = Color3.fromRGB(150, 52, 52),
2548      body = "1% TZ solution, 30 C for 4 h in darkness. Embryo staining scored 1-5. Confirms viability where
2549      ↪ germination is dormant."},
2550     {title = "MOISTURE CONTENT & STORAGE LIFE", seed = 2, colour = Color3.fromRGB(52, 96, 150),
2551      body = "Oven method, 103 C for 17 h. Each 1% drop in moisture roughly doubles safe storage life within the
2552      ↪ working range."},
2553     {title = "ISTA SAMPLING PROTOCOL", seed = 7, colour = Color3.fromRGB(96, 74, 148),
2554      body = "Primary samples drawn from the lot, combined, then reduced by successive halving to the submitted
2555      ↪ sample."},
2556     {title = "THERMAL TIME MODEL", seed = 4, colour = Color3.fromRGB(150, 108, 40),
2557      body = "Base temperature Tb = 4.2 C. Accumulated degree-days predict time to 50% emergence across the
2558      ↪ tested range."},
2559     {title = "GMO EVENT DETECTION - qPCR", seed = 6, colour = Color3.fromRGB(38, 108, 130),
2560      body = "Event-specific primers against the junction region. Reference gene normalisation, LOD 0.05%
2561      ↪ w/w."},
2562     {title = "HORMONE PRIMING RESPONSE", seed = 8, colour = Color3.fromRGB(120, 60, 110),
2563      body = "Gibberellin soak breaks dormancy in stratification-requiring lots. Dose response measured over 5
2564      ↪ concentrations."},
2565     {title = "PATHOGEN SCREENING WORKFLOW", seed = 9, colour = Color3.fromRGB(140, 84, 36),
2566      body = "Blotter incubation 7 days, 22 C, 12 h NUV. Fungal structures identified under stereo and compound
2567      ↪ microscopy."},
2568 }
2569
2570 }
2571
2572 local function buildCorridorFixtures(parent, levelY, levelName, rooms)

```

```

2563     if not parent then return end
2564
2565     -- Build one shared wall-space allocator for every corridor fixture. Door
2566     -- openings are excluded with the same generous clearance used by room
2567     -- equipment, and the end margins keep pickups away from entrance glazing.
2568     local function corridorOpenings(onWest)
2569         local openings = {}
2570         for _, room in ipairs(rooms or {}) do
2571             local belongs = (onWest and room.plan == "L") or ((not onWest) and room.plan == "R")
2572             if belongs then
2573                 local side = onWest and "E" or "W"
2574                 for _, opening in ipairs(room.open[side] or {}) do
2575                     table.insert(openings, {at = room.z1 + opening.at, width = opening.width})
2576                 end
2577             end
2578         end
2579         return openings
2580     end
2581
2582     local available = {
2583         west = freeSpans(L, corridorOpenings(true)),
2584         east = freeSpans(L, corridorOpenings(false)),
2585     }
2586
2587     -- Taking the centre of the longest remaining span distributes fixtures
2588     -- along the full corridor instead of clustering them near one stairwell.
2589     local function takeCentred(spans, needed)
2590         local bestIndex, bestLength
2591         for index, span in ipairs(spans) do
2592             local length = span[2] - span[1]
2593             if length >= needed and (not bestLength or length > bestLength) then
2594                 bestIndex, bestLength = index, length
2595             end
2596         end
2597         if not bestIndex then return nil end
2598
2599         local span = table.remove(spans, bestIndex)
2600         local centre = (span[1] + span[2]) / 2
2601         local half = needed / 2
2602         local separation = 18
2603         if centre - half - separation > span[1] then
2604             table.insert(spans, {span[1], centre - half - separation})
2605         end
2606         if centre + half + separation < span[2] then
2607             table.insert(spans, {centre + half + separation, span[2]})
2608         end
2609         return centre
2610     end
2611
2612     local function reserveCorridorSpot(preferWest, width)
2613         local first = preferWest and "west" or "east"
2614         local second = preferWest and "east" or "west"
2615         local at = takeCentred(available[first], P(width))
2616         if at then return first == "west", at end
2617         at = takeCentred(available[second], P(width))
2618         if at then return second == "west", at end
2619         error("No door-safe corridor wall space remains for " .. levelName)
2620     end
2621
2622     for i = 1, HEALTH_STATIONS_PER_FLOOR do
2623         -- Alternate walls so you can always see one from anywhere in the run.
2624         local depth, width, height = 18, 55, 70
2625         local onWest, pz = reserveCorridorSpot(i % 2 == 1, width)
2626         local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2627
2628         local station = newPart("HealthStation_" .. levelName .. "_" .. i,
2629             Vector3.new(E(depth), EH(height), E(width)),
2630             CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(110), X(pz)),
2631             Color3.fromRGB(225, 40, 35), Enum.Material.SmoothPlastic, parent)
2632         station.CanCollide = false
2633         station:SetAttribute("Level", levelName)
2634         CollectionService.AddTag(station, "EmergencyKit")
2635
2636         local face = Instance.new("SurfaceGui")
2637         face.Name = "EmergencyKitFace"
2638         face.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left

```



```

2639     face.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2640     face.PixelsPerStud = 60
2641     face.AlwaysOnTop = false
2642     face.Parent = station
2643     local crossV = Instance.new("Frame")
2644     crossV.Size = UDim2.new(0.2, 0, 0.5, 0)
2645     crossV.Position = UDim2.new(0.4, 0, 0.1, 0)
2646     crossV.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2647     crossV.BorderSizePixel = 0
2648     crossV.Parent = face
2649     local crossH = Instance.new("Frame")
2650     crossH.Size = UDim2.new(0.5, 0, 0.2, 0)
2651     crossH.Position = UDim2.new(0.25, 0, 0.25, 0)
2652     crossH.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2653     crossH.BorderSizePixel = 0
2654     crossH.Parent = face
2655     local caption = Instance.new("TextLabel")
2656     caption.Name = "KitCaption"
2657     caption.Size = UDim2.new(1, 0, 0.26, 0)
2658     caption.Position = UDim2.new(0, 0, 0.7, 0)
2659     caption.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
2660     caption.BackgroundTransparency = 0.05
2661     caption.BorderSizePixel = 0
2662     caption.Text = "HEALTH"
2663     caption.TextColor3 = Color3.fromRGB(200, 30, 25)
2664     caption.TextScaled = true
2665     caption.Font = Enum.Font.SourceSansBold
2666     caption.Parent = face
2667
2668     local prompt = Instance.new("ProximityPrompt")
2669     prompt.Name = "EmergencyKitPrompt"
2670     prompt.ActionText = "Use Health Station"
2671     prompt.ObjectText = "Restore health"
2672     prompt.KeyboardKeyCode = Enum.KeyCode.E
2673     prompt.MaxActivationDistance = 10
2674     prompt.RequiresLineOfSight = false
2675     prompt.Parent = station
2676 end
2677
2678 -- Ten weapon cabinets per floor, distributed across both corridor walls.
2679 for i = 1, WEAPON_STATIONS_PER_FLOOR do
2680     local depth, width, height = 22, 70, 120
2681     local onWest, pz = reserveCorridorSpot(i % 2 == 0, width)
2682     local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2683     local cabinet = newPart("SidearmCabinet_" .. levelName .. "_" .. i,
2684         Vector3.new(E(depth), EH(height), E(width)),
2685         CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(80), X(pz)),
2686         Color3.fromRGB(52, 58, 68), Enum.Material.Metal, parent)
2687     cabinet.CanCollide = false
2688     cabinet:SetAttribute("Level", levelName)
2689     -- One weapon type per cabinet, so every floor offers all three.
2690     local weaponKind = PC_WEAPONS[((i - 1) % #PC_WEAPONS) + 1]
2691     cabinet:SetAttribute("ToolKind", weaponKind)
2692     CollectionService.AddTag(cabinet, "LabToolDispenser")
2693
2694     local face = Instance.new("SurfaceGui")
2695     face.Name = "SidearmCabinetFace"
2696     face.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left
2697     face.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2698     face.PixelsPerStud = 60
2699     face.AlwaysOnTop = false
2700     face.Parent = cabinet
2701     local caption = Instance.new("TextLabel")
2702     caption.Size = UDim2.new(0.9, 0, 0.3, 0)
2703     caption.Position = UDim2.new(0.05, 0, 0.12, 0)
2704     caption.BackgroundColor3 = Color3.fromRGB(235, 195, 35)
2705     caption.BackgroundTransparency = 0.05
2706     caption.BorderSizePixel = 0
2707     caption.Text = (weaponKind:gsub("^Lab ", "")):upper()
2708     caption.TextColor3 = Color3.fromRGB(25, 25, 25)
2709     caption.TextScaled = true
2710     caption.Font = Enum.Font.SourceSansBold
2711     caption.Parent = face
2712     local sub = Instance.new("TextLabel")
2713     sub.Size = UDim2.new(0.9, 0, 0.4, 0)
2714     sub.Position = UDim2.new(0.05, 0, 0.48, 0)

```

```

2715     sub.BackgroundTransparency = 1
2716     sub.Text = "PC • MOBILE • TABLET\nHold E / tap to equip\nAim at the target marker"
2717     sub.TextColor3 = Color3.fromRGB(235, 240, 245)
2718     sub.TextScaled = true
2719     sub.Font = Enum.Font.Gotham
2720     sub.Parent = face
2721 end
2722
2723 -- Gas service pipes. Deliberately obvious, and very dangerous: a round
2724 -- into one takes out a chunk of the room. CanCollide is off so they never
2725 -- block a walking route, but CanQuery stays on so they can be shot.
2726 for i, px in ipairs({CORR_X1 + 20, CORR_X2 - 20}) do
2727     local pipe = newPart("GasPipe_" .. levelName .. "_" .. i,
2728         Vector3.new(E(16), EH(16), X(L - 240)),
2729         CFrame.new(X(px), Y(levelY) + Y(WALL_H) - EH(18), X(L / 2)),
2730         Color3.fromRGB(216, 176, 42), Enum.Material.Metal, parent)
2731     pipe.CanCollide = false
2732     pipe:SetAttribute("GasPipe", true)
2733     pipe:SetAttribute("Level", levelName)
2734     CollectionService:AddTag(pipe, "GasPipe")
2735
2736     -- hazard collars every few metres so it reads as a gas line
2737     for band = 1, 6 do
2738         local collar = newPart("GasPipeBand_" .. levelName .. "_" .. i .. "_" .. band,
2739             Vector3.new(E(19), EH(19), X(26)),
2740             CFrame.new(X(px), Y(levelY) + Y(WALL_H) - EH(18), X(L * band / 7)),
2741             Color3.fromRGB(30, 30, 30), Enum.Material.SmoothPlastic, parent)
2742         collar.CanCollide = false
2743         collar.CastShadow = false
2744     end
2745 end
2746
2747 -- Scientific posters. Every one is a real part with CanQuery left on, so
2748 -- the weapon raycast can hit it and punch a hole that stays put.
2749 for i = 1, #POSTER_TITLES do
2750     local depth, width, height = 6, 118, 84
2751     local onWest, pz = reserveCorridorSpot(i % 2 == 0, width)
2752     local px = onWest and (CORR_X1 + INT_T / 2 + depth / 2) or (CORR_X2 - INT_T / 2 - depth / 2)
2753
2754     local poster = newPart("Poster_" .. levelName .. "_" .. i,
2755         Vector3.new(E(depth), EH(height), E(width)),
2756         CFrame.new(X(px), Y(levelY) + EH(height) / 2 + EH(120), X(pz)),
2757         Color3.fromRGB(242, 240, 233), Enum.Material.SmoothPlastic, parent)
2758     poster.CanCollide = false
2759     poster:SetAttribute("ShootablePoster", true)
2760     poster:SetAttribute("Holes", 0)
2761     poster:SetAttribute("Level", levelName)
2762     CollectionService:AddTag(poster, "LabPoster")
2763
2764     local art = Instance.new("SurfaceGui")
2765     art.Name = "PosterFace"
2766     art.Face = onWest and Enum.NormalId.Right or Enum.NormalId.Left
2767     art.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
2768     art.PixelsPerStud = 55
2769     art.AlwaysOnTop = false
2770     art.Parent = poster
2771
2772     local entry = POSTER_TITLES[i]
2773     local band = Instance.new("Frame")
2774     band.Size = UDim2.new(1, 0, 0.2, 0)
2775     band.BackgroundColor3 = entry.colour
2776     band.BorderSizePixel = 0
2777     band.Parent = art
2778     local title = Instance.new("TextLabel")
2779     title.Size = UDim2.new(0.92, 0, 0.8, 0)
2780     title.Position = UDim2.new(0.04, 0, 0.1, 0)
2781     title.BackgroundTransparency = 1
2782     title.Text = entry.title
2783     title.TextColor3 = Color3.fromRGB(255, 255, 255)
2784     title.TextScaled = true
2785     title.Font = Enum.Font.GothamBold
2786     title.Parent = band
2787
2788     -- A little chart so it reads as a research poster rather than a sign.
2789     local plot = Instance.new("Frame")
2790     plot.Size = UDim2.new(0.44, 0, 0.5, 0)

```

```

2791     plot.Position = UDim2.new(0.04, 0, 0.28, 0)
2792     plot.BackgroundColor3 = Color3.fromRGB(250, 250, 246)
2793     plot.BorderSizePixel = 0
2794     plot.Parent = art
2795     for barIndex = 1, 5 do
2796         local h = 0.2 + ((entry.seed * barIndex) % 7) / 9
2797         local bar = Instance.new("Frame")
2798         bar.Size = UDim2.new(0.14, 0, h, 0)
2799         bar.Position = UDim2.new(0.05 + (barIndex - 1) * 0.19, 0, 1 - h, 0)
2800         bar.BackgroundColor3 = entry.colour
2801         bar.BorderSizePixel = 0
2802         bar.Parent = plot
2803     end
2804
2805     local body = Instance.new("TextLabel")
2806     body.Size = UDim2.new(0.44, 0, 0.5, 0)
2807     body.Position = UDim2.new(0.52, 0, 0.28, 0)
2808     body.BackgroundTransparency = 1
2809     body.Text = entry.body
2810     body.TextColor3 = Color3.fromRGB(48, 52, 58)
2811     body.TextScaled = true
2812     body.TextWrapped = true
2813     body.Font = Enum.Font.Gotham
2814     body.TextXAlignment = Enum.TextXAlignment.Left
2815     body.Parent = art
2816
2817     local footer = Instance.new("TextLabel")
2818     footer.Size = UDim2.new(0.92, 0, 0.12, 0)
2819     footer.Position = UDim2.new(0.04, 0, 0.84, 0)
2820     footer.BackgroundTransparency = 1
2821     footer.Text = "SEED TESTING LABORATORY - " .. levelName:upper()
2822     footer.TextColor3 = Color3.fromRGB(120, 126, 134)
2823     footer.TextScaled = true
2824     footer.Font = Enum.Font.Gotham
2825     footer.TextXAlignment = Enum.TextXAlignment.Left
2826     footer.Parent = art
2827 end
2828 end
2829
2830 buildCorridorFixtures(basementModel, BOTTOM_LEVEL * F2F, "Basement", BASEMENT_ROOMS)
2831 for _, lv in ipairs(LEVELS) do
2832     buildCorridorFixtures(levelModels[lv.index], lv.index * F2F, lv.name, lv.rooms)
2833 end
2834
2835 -- Publish every room's rectangle, already converted to stud space, so the
2836 -- runtime can answer "is this player in the same room as this bot?" exactly.
2837 -- A plain radius test cannot do that here: the west room runs the full 267
2838 -- studs of the plan, so any radius wide enough to cover it also reaches
2839 -- straight through the corridor wall into the rooms opposite.
2840 do
2841     local entries = {}
2842     local function addRooms(levelName, rooms, levelY)
2843         for _, r in ipairs(rooms) do
2844             table.insert(entries, string.format("%s:%s:%.1f:%.1f:%.1f:%.1f",
2845                 levelName, r.name, X(r.x1), X(r.z1), X(r.x2), X(r.z2), Y(levelY)))
2846         end
2847     end
2848     addRooms("Basement", basementRooms, BOTTOM_LEVEL * F2F)
2849     for _, lv in ipairs(LEVELS) do
2850         addRooms(lv.name, lv.rooms, lv.index * F2F)
2851     end
2852     root:SetAttribute("RoomIndex", table.concat(entries, ";"))
2853 end
2854
2855 local BOTS_PER_LEVEL = 20
2856 local workerFolder = Instance.new("Folder")
2857 workerFolder.Name = "WorkerTestNPCs"
2858 workerFolder.Parent = root
2859 local botNumber = 0
2860 local botLevelSpecs = {{y = BOTTOM_LEVEL * F2F, count = BOTS_PER_LEVEL,
2861     name = "Basement", rooms = basementRooms}}
2862 for _, lv in ipairs(LEVELS) do
2863     table.insert(botLevelSpecs, {y = lv.index * F2F, count = BOTS_PER_LEVEL,
2864         name = lv.name, rooms = lv.rooms})
2865 end
2866

```

```

2867 -- Some staff sit at a workstation instead of patrolling. Seats are placed
2868 -- from the ROOM SCHEDULE, not from the preserved OfficeFitout: that fit-out
2869 -- was built for an earlier version of the building and its geometry now sits
2870 -- outside the current footprint entirely, so seating from it would put staff
2871 -- in mid-air outside the walls.
2872 local SEATED_PER_LEVEL = 5
2873 for _, levelSpec in ipairs(botLevelSpecs) do
2874     local seatedThisLevel = 0
2875     for slot = 1, levelSpec.count do
2876         botNumber = botNumber + 1
2877         local xcm = (slot % 2 == 0) and 260 or 1040
2878         local zcm = 190 + ((slot - 1) * 137) % 1690
2879         -- skip index 1: that is the long west room, which is circulation space
2880         local seatRoom
2881         if seatedThisLevel < SEATED_PER_LEVEL and levelSpec.rooms then
2882             seatRoom = levelSpec.rooms[seatedThisLevel + 2]
2883         end
2884         local botModel = buildWorkerBot(workerFolder, botNumber, xcm, zcm, levelSpec.y, seatRoom ~= nil)
2885         -- The bot's index WITHIN ITS OWN FLOOR (1..BOTS_PER_LEVEL). The runtime
2886         -- throttles how many workers are busy per floor using this. It used to
2887         -- throttle on the global BotNumber instead, which counts straight
2888         -- through every floor (basement 1-20, Level_1 21-40, ... Level_8
2889         -- 161-180) and was compared against a cap of at most 50 -- so every
2890         -- bot above 50 sat in permanent "Adaptive standby" and floors 3 and up
2891         -- never moved at all.
2892         if botModel then
2893             botModel:SetAttribute("LevelSlot", slot)
2894             if seatRoom then
2895                 seatedThisLevel = seatedThisLevel + 1
2896                 botModel:SetAttribute("Seated", true)
2897                 botModel:SetAttribute("Room", seatRoom.name)
2898                 -- Sit a third of the way in from the room's west wall, facing
2899                 -- the middle of the room. The seated pose is built facing -Z,
2900                 -- which is the model's LookVector, so lookAt orients it.
2901                 local seatHeight = Y(levelSpec.y) + 2.45
2902                 local centreX = (seatRoom.x1 + seatRoom.x2) / 2
2903                 local centreZ = (seatRoom.z1 + seatRoom.z2) / 2
2904                 local seatX = seatRoom.x1 + (seatRoom.x2 - seatRoom.x1) * 0.3
2905                 local seatCF = CFrame.lookAt(
2906                     Vector3.new(X(seatX), seatHeight, X(centreZ)),
2907                     Vector3.new(X(centreX), seatHeight, X(centreZ)))
2908                 botModel:PivotTo(seatCF)
2909                 buildWorkstation(workerFolder, seatCF, botModel.Name)
2910             end
2911         end
2912     end
2913 end
2914
2915 -- Hunt bookkeeping: how many targets exist, and how many are down. The HUD
2916 -- and the kill feed both read these off the model.
2917 root:SetAttribute("BotsTotal", botNumber)
2918 root:SetAttribute("BotsKilled", 0)
2919
2920 -- Wall-mounted QA board: the autonomous workers update this during play,
2921 -- making the scenario test visible instead of hiding it in server output.
2922 -- Mounted on the CORRIDOR wall (never inside a room), well above any door's
2923 -- head height, so equipment placed against a room's own walls can never end
2924 -- up in front of it.
2925 -- Oversized floating NPC-test board removed; status remains in the compact HUD.
2926
2927 -- roof, punched for the shafts
2928 local topY = TOP_LEVEL * F2F + WALL_H + SLAB
2929 slabWithHoles(root, "Roof", -EXT_T, -EXT_T, W + EXT_T, L + EXT_T, topY, SLAB, ELEV_HOLES, C.roof)
2930 wall(root, "Parapet_W", 0, -EXT_T, 0, L + EXT_T, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2931 wall(root, "Parapet_E", W, -EXT_T, W, L + EXT_T, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2932 wall(root, "Parapet_N", -EXT_T, 0, W + EXT_T, 0, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2933 wall(root, "Parapet_S", -EXT_T, L, W + EXT_T, L, EXT_T, topY, 110, {}, C.ext, Enum.Material.Concrete)
2934
2935 buildStairTower(BOTTOM_LEVEL, TOP_LEVEL, false)
2936 buildStairTower(BOTTOM_LEVEL, TOP_LEVEL, true)
2937 -- Runtime route guidance uses these exact stair-lane coordinates. Keeping
2938 -- them on the generated model avoids hard-coded world positions inside the
2939 -- server scripts and guarantees an upper-floor route follows both flights.
2940 root:SetAttribute("FloorRiseStuds", Y(F2F))
2941 root:SetAttribute("StairWestLaneX", X((TX1 + 10 + TXM) / 2))
2942 root:SetAttribute("StairEastLaneX", X((TXM + TX2 - 10) / 2))

```

```

2943 -- These describe the north tower's switchback specifically (the runtime's
2944 -- manual floor-by-floor fallback route always walks through that one
2945 -- tower) -- same numeric values buildStairTower(..., false) itself used.
2946 root:SetAttribute("StairNorthZ", X(-100))
2947 root:SetAttribute("StairSouthZ", X(-400))
2948 -- So the fallback router (below) can clamp to however many floors this
2949 -- building actually has instead of a hard-coded 3-floors-plus-basement
2950 -- assumption from when there were only 3 floors.
2951 root:SetAttribute("FloorMinLevel", BOTTOM_LEVEL)
2952 root:SetAttribute("FloorMaxLevel", TOP_LEVEL)
2953 for _, spec in ipairs(ELEVATORS) do buildElevator(spec, BOTTOM_LEVEL, TOP_LEVEL) end
2954
2955 slab(root, "Apron_East", W, 100, W + 400, L * 0.94, 0, 20, C.slab)
2956 slab(root, "Apron_West", -400, 100, 0, L * 0.94, 0, 20, C.slab)
2957 slabWithHoles(root, "Apron_South", 60, L, W * 0.9, L + 350, 0, 20,
2958     {{x1 = TX1, x2 = TX2, z1 = L, z2 = L + 350}}, C.slab)
2959
2960 local pad = 700
2961 -- Soil must surround the foundations, not fill the basement and stair flights.
2962 slabWithHoles(root, "SitePad", -pad, -pad - 500, W + pad, L + pad, -2, 30, {
2963     {x1 = TX1 - EXT_T, x2 = TX2 + EXT_T, z1 = -460 - EXT_T, z2 = 0},
2964     {x1 = 0, x2 = W, z1 = 0, z2 = L},
2965     {x1 = TX1 - EXT_T, x2 = TX2 + EXT_T, z1 = L, z2 = L + 460 + EXT_T},
2966 },
2967     Color3.fromRGB(122, 136, 112), Enum.Material.Grass)
2968
2969 -- The template Baseplate is 16 studs thick: it used to fill the entire
2970 -- basement. Keep its exterior surface and store the original for recovery.
2971 do
2972     local storage = game:GetService("ServerStorage")
2973     local base = Workspace:FindFirstChild("Baseplate") or storage:FindFirstChild("LaboratoryOriginalBaseplate")
2974     if base and base:IsA("BasePart") and base.CFrame.Rotation == CFrame.identity then
2975         local oldGround = Workspace:FindFirstChild("LaboratoryFoundationGround")
2976         if oldGround then oldGround:Destroy() end
2977         local ground = Instance.new("Folder")
2978         ground.Name, ground.Parent = "LaboratoryFoundationGround", Workspace
2979         local minX, maxX = base.Position.X - base.Size.X / 2, base.Position.X + base.Size.X / 2
2980         local minZ, maxZ = base.Position.Z - base.Size.Z / 2, base.Position.Z + base.Size.Z / 2
2981         local hx1, hx2, hz1, hz2 = X(-EXT_T), X(W + EXT_T), X(-460 - EXT_T), X(L + 460 + EXT_T)
2982         local function outside(name, x1, z1, x2, z2)
2983             if x2 <= x1 or z2 <= z1 then return end
2984             local piece = base:Clone()
2985             piece.Name = name
2986             piece.Size = Vector3.new(x2 - x1, base.Size.Y, z2 - z1)
2987             piece.CFrame = CFrame.new((x1 + x2) / 2, base.Position.Y, (z1 + z2) / 2)
2988             piece.Parent = ground
2989         end
2990         outside("West", minX, minZ, hx1, maxZ)
2991         outside("East", hx2, minZ, maxX, maxZ)
2992         outside("North", hx1, minZ, hx2, hz1)
2993         outside("South", hx1, hz2, hx2, maxZ)
2994         base.Name, base.Parent = "LaboratoryOriginalBaseplate", storage
2995     end
2996 end
2997
2998 -----
2999 -- SITE WORKS
3000 --
3001 -- Regenerated from W and L every build, so it always fits the building
3002 -- that was just generated. Everything here stays strictly OUTSIDE the
3003 -- footprint x[0..W] z[0..L] -- the only things south of the building are
3004 -- the forecourt and entrance canopy, which belong there.
3005 -----
3006 do
3007     local site = Instance.new("Folder")
3008     site.Name = "SiteWorks"
3009     site.Parent = root
3010
3011     local rng = newRng(RNG_SEED + 4242)
3012     local FENCE_X1, FENCE_X2 = -pad + 90, W + pad - 90
3013     local FENCE_Z1, FENCE_Z2 = -pad - 400, L + pad - 90
3014     local fenceGreen = Color3.fromRGB(46, 104, 58)
3015     local GATE_X1, GATE_X2 = 460, 740 -- lines up with the south entrance
3016
3017     -- perimeter fence, with a gap on the south run for the gate
3018     wall(site, "Fence_S", FENCE_X1, FENCE_Z2, FENCE_X2, FENCE_Z2, 14, 0, 190,

```

```

3019     {{at = GATE_X1 - FENCE_X1, width = GATE_X2 - GATE_X1, head = 190}}, fenceGreen, Enum.Material.Metal)
3020 wall(site, "Fence_N", FENCE_X1, FENCE_Z1, FENCE_X2, FENCE_Z1, 14, 0, 190, {}, fenceGreen,
    ↪ Enum.Material.Metal)
3021 wall(site, "Fence_W", FENCE_X1, FENCE_Z1, FENCE_X1, FENCE_Z2, 14, 0, 190, {}, fenceGreen,
    ↪ Enum.Material.Metal)
3022 wall(site, "Fence_E", FENCE_X2, FENCE_Z1, FENCE_X2, FENCE_Z2, 14, 0, 190, {}, fenceGreen,
    ↪ Enum.Material.Metal)
3023
3024 -- barred gate across the opening
3025 for i = 0, 11 do
3026     local gx = GATE_X1 + (GATE_X2 - GATE_X1) * i / 11
3027     newPart("GateBar_" .. i, Vector3.new(X(9), Y(180), X(9)),
3028         CFrame.new(X(gx), Y(90), X(FENCE_Z2)), Color3.fromRGB(28, 34, 30), Enum.Material.Metal, site)
3029 end
3030 newPart("GateRail", Vector3.new(X(GATE_X2 - GATE_X1), Y(12), X(12)),
3031     CFrame.new(X((GATE_X1 + GATE_X2) / 2), Y(174), X(FENCE_Z2)), Color3.fromRGB(28, 34, 30),
    ↪ Enum.Material.Metal, site)
3032
3033 -- Small canopy at the single west-lobby entrance, clear of both stair towers.
3034 slab(site, "Canopy", -160, 170, 0, 330, 270, 16, Color3.fromRGB(206, 210, 214), Enum.Material.Metal)
3035 for _, cz in ipairs({180, 320}) do
3036     newPart("CanopyPost", Vector3.new(X(16), Y(254), X(16)),
3037         CFrame.new(X(-145), Y(127), X(cz)), Color3.fromRGB(150, 155, 160), Enum.Material.Metal, site)
3038 end
3039
3040 -- planters either side of the door
3041 for i, px in ipairs({390, 810}) do
3042     newPart("Planter_" .. i, Vector3.new(X(120), Y(70), X(120)),
3043         CFrame.new(X(px), Y(35), X(L + 90)), Color3.fromRGB(176, 168, 150), Enum.Material.Concrete, site)
3044     newPart("PlanterSoil_" .. i, Vector3.new(X(104), Y(16), X(104)),
3045         CFrame.new(X(px), Y(72), X(L + 90)), Color3.fromRGB(84, 66, 48), Enum.Material.Ground, site)
3046     for f = 1, 4 do
3047         newPart("PlanterShrub_" .. i .. "_" .. f, Vector3.new(X(34), Y(46), X(34)),
3048             CFrame.new(X(px + rng.int(-30, 30)), Y(100), X(L + 90 + rng.int(-30, 30))),
3049             Color3.fromRGB(64, 122, 62), Enum.Material.Grass, site)
3050     end
3051 end
3052
3053 -- flagpole
3054 newPart("FlagPole", Vector3.new(X(10), Y(560), X(10)),
3055     CFrame.new(X(900), Y(280), X(L + 210)), Color3.fromRGB(214, 216, 220), Enum.Material.Metal, site)
3056 newPart("Flag", Vector3.new(X(4), Y(90), X(150)),
3057     CFrame.new(X(900), Y(500), X(L + 285)), Color3.fromRGB(200, 208, 216), Enum.Material.Fabric, site)
3058
3059 -- parking bays and cars on the south forecourt
3060 local function buildCar(name, cx, cz, colour)
3061     newPart(name .. "_Body", Vector3.new(X(170), Y(80), X(360)),
3062         CFrame.new(X(cx), Y(60), X(cz)), colour, Enum.Material.Metal, site)
3063     newPart(name .. "_Cabin", Vector3.new(X(150), Y(60), X(170)),
3064         CFrame.new(X(cx), Y(126), X(cz + 10)), Color3.fromRGB(80, 96, 112), Enum.Material.Glass, site)
3065     for _, wx in ipairs({-80, 80}) do
3066         for _, wz in ipairs({-115, 115}) do
3067             local wheel = newPart(name .. "_Wheel", Vector3.new(X(26), Y(58), X(58)),
3068                 CFrame.new(X(cx + wx), Y(28), X(cz + wz)), Color3.fromRGB(30, 30, 32),
3069                 ↪ Enum.Material.Rubber, site)
3070             wheel.Shape = Enum.PartType.Cylinder
3071         end
3072     end
3073 end
3074 local carColours = {
3075     Color3.fromRGB(180, 60, 55), Color3.fromRGB(60, 80, 150),
3076     Color3.fromRGB(220, 220, 224), Color3.fromRGB(70, 74, 80),
3077 }
3078 for bay = 0, 4 do
3079     local bx = 900 + bay * 220 -- outside the south stair tower; full car-width bays
3080     newPart("BayLine_" .. bay, Vector3.new(X(6), Y(4), X(200)),
3081         CFrame.new(X(bx), Y(2), X(L + 220)), Color3.fromRGB(238, 238, 232), Enum.Material.SmoothPlastic,
3082         ↪ site)
3083     if bay < 4 then
3084         buildCar("Car_" .. bay, bx + 110, L + 215, carColours[bay + 1])
3085     end
3086 end
3087
3088 -- greenhouses on the west apron
3089 for g = 1, 3 do
3090     local gz = 260 + (g - 1) * 620

```

```

3089     slab(site, "GreenhouseBase_" .. g, -370, gz, -70, gz + 460, 4, 12, Color3.fromRGB(150, 146, 138),
    ↪ Enum.Material.Concrete)
3090   for _, corner in ipairs({{-370, gz}, {-70, gz}, {-370, gz + 460}, {-70, gz + 460}}) do
3091     newPart("GreenhouseFrame_" .. g, Vector3.new(X(14), Y(230), X(14)),
3092       CFrame.new(X(corner[1]), Y(115), X(corner[2])), Color3.fromRGB(196, 200, 204),
    ↪ Enum.Material.Metal, site)
3093   end
3094   local glassSides = {
3095     {-370, gz, -70, gz}, {-370, gz + 460, -70, gz + 460},
3096     {-370, gz, -370, gz + 460}, {-70, gz, -70, gz + 460},
3097   }
3098   for si, side in ipairs(glassSides) do
3099     local gl = wall(site, "GreenhouseGlass_" .. g .. "_" .. si,
3100       side[1], side[2], side[3], side[4], 8, 0, 210, {}, Color3.fromRGB(186, 216, 224),
    ↪ Enum.Material.Glass)
3101   end
3102   local roof = slab(site, "GreenhouseRoof_" .. g, -370, gz, -70, gz + 460, 240, 10,
3103     Color3.fromRGB(196, 222, 230), Enum.Material.Glass)
3104   roof.Transparency = 0.5
3105   -- crop benches inside
3106   for b = 0, 2 do
3107     newPart("GreenhouseBench_" .. g .. "_" .. b, Vector3.new(X(70), Y(20), X(400)),
3108       CFrame.new(X(-330 + b * 100), Y(80), X(gz + 230)), Color3.fromRGB(96, 138, 84),
    ↪ Enum.Material.Grass, site)
3109   end
3110 end
3111
3112 -- trial crop plots on the east apron
3113 for plot = 0, 1 do
3114   local px = W + 90 + plot * 170
3115   slab(site, "TrialPlot_" .. plot, px, 220, px + 140, L * 0.9, 6, 14,
3116     Color3.fromRGB(96, 74, 54), Enum.Material.Ground)
3117   for row = 0, 9 do
3118     newPart("CropRow_" .. plot .. "_" .. row, Vector3.new(X(120), Y(26), X(70)),
3119       CFrame.new(X(px + 70), Y(20), X(300 + row * 160)),
3120       Color3.fromRGB(84, 142, 70), Enum.Material.Grass, site)
3121   end
3122 end
3123
3124 -- street trees, kept out of the building, the aprons and the forecourt
3125 local function buildTree(name, tx, tz)
3126   newPart(name .. "_Trunk", Vector3.new(X(30), Y(240), X(30)),
3127     CFrame.new(X(tx), Y(120), X(tz)), Color3.fromRGB(94, 68, 46), Enum.Material.Wood, site)
3128   for c = 1, 3 do
3129     local crown = newPart(name .. "_Crown", Vector3.new(X(150 - c * 22), Y(150 - c * 22), X(150 - c *
    ↪ 22)),
3130       CFrame.new(X(tx + rng.int(-20, 20)), Y(250 + c * 42), X(tz + rng.int(-20, 20))),
3131       Color3.fromRGB(52, 104 + c * 8, 50), Enum.Material.Grass, site)
3132     crown.Shape = Enum.PartType.Ball
3133   end
3134 end
3135 local n = 0
3136 for i = 0, 7 do -- west strip
3137   n = n + 1; buildTree("Tree_" .. n, -560 + rng.int(-60, 60), -200 + i * 330)
3138 end
3139 for i = 0, 7 do -- east strip
3140   n = n + 1; buildTree("Tree_" .. n, W + 560 + rng.int(-60, 60), -200 + i * 330)
3141 end
3142 for i = 0, 5 do -- north band, in front of the building
3143   n = n + 1; buildTree("Tree_" .. n, -300 + i * 380, -560 + rng.int(-80, 80))
3144 end
3145 for i = 0, 5 do -- south band, beyond the forecourt
3146   n = n + 1; buildTree("Tree_" .. n, -300 + i * 380, L + 520 + rng.int(-60, 60))
3147 end
3148 end
3149
3150 =====
3151 -- OFFICE FIT-OUT
3152 -- A desk and a shelf in each room, placed from the room rectangle so they
3153 -- sit in the clear middle of the floor rather than fighting the equipment
3154 -- packed against the walls.
3155 =====
3156 do
3157   local office = Instance.new("Folder")
3158   office.Name = "OfficeFitout"
3159   office.Parent = root

```

```

3160
3161     local function fitOutOffice(rooms, levelY, levelName)
3162         for index, r in ipairs(rooms) do
3163             local width, depth = r.x2 - r.x1, r.z2 - r.z1
3164             if width > 200 and depth > 200 then
3165                 local seatHeight = Y(levelY) + 2.45
3166                 -- desk, facing across the room
3167                 local deskX = r.x1 + width * 0.62
3168                 local deskZ = r.z1 + depth * 0.34
3169                 local cf = CFrame.lookAt(
3170                     Vector3.new(X(deskX), seatHeight, X(deskZ)),
3171                     Vector3.new(X(r.x1 + width * 0.5), seatHeight, X(deskZ)))
3172                 buildWorkstation(office, cf, levelName .. "_" .. index)
3173
3174                 -- a shelf unit against the clear part of the floor
3175                 local shelfX = r.x1 + width * 0.62
3176                 local shelfZ = r.z1 + depth * 0.72
3177                 newPart("Shelf_" .. levelName .. "_" .. index, Vector3.new(E(90), EH(180), E(38)),
3178                     CFrame.new(X(shelfX), Y(levelY) + EH(90), X(shelfZ)),
3179                     Color3.fromRGB(150, 142, 128), Enum.Material.WoodPlanks, office).CanCollide = false
3180                 for shelfLevel = 1, 3 do
3181                     newPart("ShelfBooks_" .. levelName .. "_" .. index .. "_" .. shelfLevel,
3182                         Vector3.new(E(78), EH(26), E(28)),
3183                         CFrame.new(X(shelfX), Y(levelY) + EH(38 + shelfLevel * 42), X(shelfZ)),
3184                         Color3.fromRGB(120 + shelfLevel * 22, 90, 70), Enum.Material.SmoothPlastic,
3185                         office).CanCollide = false
3186                 end
3187             end
3188         end
3189
3190         fitOutOffice(basementRooms, BOTTOM_LEVEL * F2F, "Basement")
3191         for _, lv in ipairs(LEVELS) do
3192             fitOutOffice(lv.rooms, lv.index * F2F, lv.name)
3193         end
3194     end
3195
3196     local oldSpawn = Workspace:FindFirstChild("LabSpawn")
3197     if oldSpawn then oldSpawn:Destroy() end
3198     -- neutral start: right inside the entrance, facing the three role
3199     -- kiosks, so choosing a role is the very first thing a new player does
3200     local sp = Instance.new("SpawnLocation")
3201     sp.Name = "LabSpawn"
3202     sp.Size = Vector3.new(6, 0.1, 6)
3203     -- SpawnLocation's default facing is -Z; the kiosks sit at a HIGHER z
3204     -- than the spawn, so without a 180-degree turn a new player spawns
3205     -- with their back to them and never sees the panels. Face +Z instead.
3206     sp.CFrame = CFrame.new(X(GROUND_ROOMS[1].x1 + 250), -0.1, X(90)) * CFrame.Angles(0, math.rad(180), 0)
3207     sp.Anchored = true
3208     sp.Transparency, sp.CanCollide, sp.CanTouch, sp.CanQuery = 1, false, false, false
3209     sp.Duration = 0
3210     sp.Color = Color3.fromRGB(120, 170, 120)
3211     sp.Material = Enum.Material.Concrete
3212     sp.Parent = Workspace
3213
3214     =====
3215     -- TEAMS AND ROLE STATIONS
3216     -- Two competitive gameplay roles: Technicians repair equipment, while
3217     -- Laboratory Technicians move and process germinated plant samples.
3218     =====
3219     local Teams = game:GetService("Teams")
3220
3221     local ROLES = {
3222         {name = "Technician", team = BrickColor.new("Bright blue"), paint = Color3.fromRGB(60, 120, 200)},
3223         {name = "Laboratory Technician", team = BrickColor.new("Lime green"), paint = Color3.fromRGB(90, 170, 90)},
3224     }
3225
3226     for _, obsoleteName in ipairs({"Inspector", "Supervisor"}) do
3227         local obsolete = Teams:FindFirstChild(obsoleteName)
3228         if obsolete then obsolete:Destroy() end
3229     end
3230
3231     for _, role in ipairs(ROLES) do
3232         local old = Teams:FindFirstChild(role.name)
3233         if old then old:Destroy() end
3234         local t = Instance.new("Team")

```



```

3235     t.Name = role.name
3236     t.TeamColor = role.team
3237     t.AutoAssignable = false
3238     t.Parent = Teams
3239 end
3240
3241 do
3242     -- everything below lives in the long west room of the ground floor
3243     local lobby = GROUND_ROOMS[1]
3244     local y0 = 0
3245     local colX = {lobby.x1 + 130, lobby.x1 + 250, lobby.x1 + 370}
3246
3247     local function prop(name, xcm, zcm, w, d, h, colour, shape)
3248         local isEquipmentCabinet = (shape or "box") ~= "rack"
3249         if isEquipmentCabinet then w, d, h = 96, 74, 205 end
3250         local size = Vector3.new(E(w), EH(h), E(d))
3251         local pos = Vector3.new(X(xcm), Y(y0) + size.Y / 2, X(zcm))
3252         local entry = {kind = name, col = colour, mat = Enum.Material.Metal,
3253             shape = isEquipmentCabinet and "appliance" or (shape or "box"), plantCabinet = isEquipmentCabinet,
3254             ↪ capacity = 8}
3255         local p = buildKind(root, entry, pos, size)
3256         p.Name = name:gsub("%s", "")
3257         if isEquipmentCabinet then
3258             local sampleCount = 4
3259             local sampleIds = {}
3260             for i = 1, sampleCount do table.insert(sampleIds, string.format("%s-P%02d", p.Name:sub(1,
3261                 ↪ 6):upper(), i)) end
3262             p:SetAttribute("Capacity", entry.capacity)
3263             p:SetAttribute("SampleCount", sampleCount)
3264             p:SetAttribute("SampleIds", table.concat(sampleIds, "|"))
3265             populateEquipmentCabinet(p, entry, size, nil, sampleCount, entry.capacity)
3266         end
3267         return p
3268     end
3269
3270     -- a glowing floor chevron pointing from a kiosk toward its station
3271     local function chevron(xcm, zcm, colour)
3272         local w, len, thick = 60, 40, 7
3273         for _, side in ipairs({-1, 1}) do
3274             local arm = newPart("ArrowMark", Vector3.new(X(thick), Y(4), X(len)),
3275                 CFrame.new(X(xcm + side * w / 4), Y(y0) + Y(2), X(zcm)) * CFrame.Angles(0, math.rad(side * 35),
3276                 ↪ 0),
3277                 colour, Enum.Material.Neon, root)
3278             arm.CanCollide = false
3279             arm.CastShadow = false
3280         end
3281     end
3282
3283     -- role panels: two big, unmistakable glowing panels right in
3284     -- front of the spawn point -- touch one to become that role, which
3285     -- immediately changes your character's appearance.
3286     for i, role in ipairs(ROLES) do
3287         local panelSize = Vector3.new(E(100), EH(190), E(14))
3288         local panelPos = Vector3.new(X(colX[i]), Y(y0) + panelSize.Y / 2, X(220))
3289         local frame = newPart(role.name .. "_PanelFrame", panelSize,
3290             CFrame.new(panelPos), Color3.fromRGB(30, 32, 36), Enum.Material.Metal, root)
3291         frame.CanCollide = true
3292         local face = newPart(role.name .. "_PanelFace", Vector3.new(panelSize.X * 0.86, panelSize.Y * 0.82,
3293             ↪ panelSize.Z * 0.3),
3294             CFrame.new(panelPos.X, panelPos.Y + panelSize.Y * 0.04, panelPos.Z + panelSize.Z / 2 * 0.9),
3295             role.paint, Enum.Material.Neon, root)
3296         face.CanCollide = false
3297         face.CastShadow = false
3298         local p = frame
3299         p:SetAttribute("Role", role.name)
3300         CollectionService:AddTag(p, "RoleKiosk")
3301
3302         -- Printed directly on the panel instead of floating in the room.
3303         -- Front + back faces keep it readable regardless of panel direction.
3304         for _, normal in ipairs({Enum.NormalId.Front, Enum.NormalId.Back}) do
3305             local surface = Instance.new("SurfaceGui")
3306             surface.Name = "RoleLabel"
3307             surface.Face = normal
3308             surface.SizingMode = Enum.SurfaceGuiSizingMode.PixelsPerStud
3309             surface.PixelsPerStud = 55
3310             surface.AlwaysOnTop = false

```

```

3307         surface.Parent = face
3308         local lbl = Instance.new("TextLabel")
3309         lbl.Size = UDim2.fromScale(1, 1)
3310         lbl.BackgroundTransparency = 1
3311         lbl.Text = role.name:upper() .. "\nPRESS E"
3312         lbl.TextColor3 = Color3.fromRGB(255, 255, 255)
3313         lbl.TextStrokeTransparency = 0.55
3314         lbl.TextScaled = true
3315         lbl.Font = Enum.Font.GothamBold
3316         lbl.Parent = surface
3317     end
3318
3319     local oldSp = Workspace:FindFirstChild(role.name .. "Spawn")
3320     if oldSp then oldSp:Destroy() end
3321     local rsp = Instance.new("SpawnLocation")
3322     rsp.Name = role.name .. "Spawn"
3323     rsp.Neutral = false
3324     rsp.TeamColor = role.team
3325     rsp.Size = Vector3.new(6, 0.1, 6)
3326     rsp.CFrame = CFrame.new(X(colX[i]), -0.1, X(340))
3327     rsp.Anchored = true
3328     rsp.CanCollide = false
3329     rsp.CanTouch = false
3330     rsp.CanQuery = false
3331     rsp.Transparency = 1
3332     rsp.Duration = 0
3333     rsp.Color = role.paint
3334     rsp.Material = Enum.Material.SmoothPlastic
3335     rsp.Parent = Workspace
3336
3337     -- Role selection is shown on the kiosk, not on raised pads/floor arrows.
3338 end
3339
3340 -- Floating scoreboard/how-to slabs removed to keep the lobby and doors clear.
3341
3342 -- SECRET BASEMENT ENTRANCE: a disguised maintenance vent flush with the
3343 -- lobby's own exterior wall, finished in the same concrete as the wall
3344 -- around it so it reads as part of the wall, not a door. It sits inside
3345 -- the kiosk area's reserved buffer zone (z 150-460 on this room's W/E
3346 -- walls, blocked off from the normal equipment fit-out earlier), so it
3347 -- can never end up buried behind an equipment cabinet. A nearby sign
3348 -- carries the actual instructions -- this is meant to be found, just
3349 -- not obvious at a glance.
3350 local secretPanel = newPart("MaintenanceVentPanel", Vector3.new(X(6), Y(140), X(90)),
3351     CFrame.new(X(EXT_T + 3), Y(y0) + Y(95), X(400)), C.ext, Enum.Material.Concrete, root)
3352 secretPanel.CanCollide = false
3353 CollectionService.AddTag(secretPanel, "SecretBasementPanel")
3354 local grille = newPart("VentGrille", Vector3.new(X(2), Y(60), X(60)),
3355     CFrame.new(X(EXT_T + 6), Y(y0) + Y(95), X(400)), Color3.fromRGB(70, 74, 78),
3356     Enum.Material.DiamondPlate, root)
3357 grille.CanCollide = false
3358 grille.CastShadow = false
3359 local secretPrompt = Instance.new("ProximityPrompt")
3360 secretPrompt.ObjectText = "Maintenance Vent"
3361 secretPrompt.ActionText = "Pry open"
3362 secretPrompt.KeyboardKeyCode = Enum.KeyCode.E
3363 secretPrompt.HoldDuration = 1.5
3364 secretPrompt.MaxActivationDistance = 8
3365 secretPrompt.RequiresLineOfSight = false
3366 secretPrompt.Parent = secretPanel
3367 -- Drop point: open basement corridor floor, well clear of any equipment.
3368 secretPanel:SetAttribute("SecretDestX", X((CORR_X1 + CORR_X2) / 2))
3369 secretPanel:SetAttribute("SecretDestY", Y(BOTTOM_LEVEL * F2F)) -- floor surface; runtime fits the avatar
3370 secretPanel:SetAttribute("SecretDestZ", X(200))
3371
3372 sign(root, "MAINTENANCE ACCESS\nA basement vent panel is set into the west wall\nnear the role kiosks. Hold
3373     ↳ E on it to pry it open.",
3374     EXT_T + 55, 400, y0, 260, 20)
3375 end
3376
3377 --=====
3378 -- RUNTIME SCRIPTS
3379 --=====
3380 -- Check spawn clearance only after furniture, walls and kiosks all exist.
3381 -- Movement collision checks cannot rescue a worker initially embedded in a wall.
3382 do

```

```

3381     local relocated, blocked = 0, 0
3382     for _, bot in ipairs(workerFolder:GetChildren()) do
3383         if bot:IsA("Model") and bot.PrimaryPart and not bot:GetAttribute("Seated") then
3384             local original = bot:GetPivot()
3385             local overlap = OverlapParams.new()
3386             overlap.FilterType = Enum.RaycastFilterType.Exclude
3387             overlap.FilterDescendantsInstances = {bot}
3388             overlap.RespectCanCollide, overlap.MaxParts = true, 1
3389             local ray = RaycastParams.new()
3390             ray.FilterType = Enum.RaycastFilterType.Exclude
3391             ray.FilterDescendantsInstances = {bot}
3392             ray.RespectCanCollide = true
3393             local function clear(point)
3394                 return #Workspace:GetPartBoundsInBox(CFrame.new(point - Vector3.new(0, 0.2, 0)),
3395                     ↪ Vector3.new(3.6, 5.4, 3.6), overlap) == 0
3396             end
3397             if not clear(original.Position) then
3398                 local found
3399                 for radius = 4, 28, 4 do
3400                     for step = 0, 15 do
3401                         local angle = step * math.pi / 8
3402                         local point = original.Position + Vector3.new(math.cos(angle) * radius, 0,
3403                             ↪ math.sin(angle) * radius)
3404                         local floor = Workspace:Raycast(point + Vector3.new(0, 1, 0), Vector3.new(0, -6, 0),
3405                             ↪ ray)
3406                         if floor and floor.Normal.Y > 0.8 and math.abs(floor.Position.Y + 3.25 -
3407                             ↪ original.Position.Y) < 0.3 then
3408                             point = Vector3.new(point.X, floor.Position.Y + 3.25, point.Z)
3409                             if clear(point) then found = point; break end
3410                         end
3411                     end
3412                     if found then break end
3413                 end
3414                 if found then bot:PivotTo(CFrame.new(found) * original.Rotation); relocated += 1
3415                 else blocked += 1 end
3416             end
3417         end
3418     end
3419     root:SetAttribute("WorkerSpawnsRelocated", relocated)
3420     root:SetAttribute("WorkerSpawnsBlocked", blocked)
3421 end
3422
3423 local FLOW_SOURCE = {[==[
3424 -- LabKinds : what each kind of basic laboratory equipment does.
3425 -- Keyed by the Kind attribute the generator writes on every item, so
3426 -- any number of copies anywhere in the building behave the same.
3427
3428 local K = {}
3429
3430 K.Items = {
3431     TRAY    = "Sample Tray",
3432     GROUND  = "Ground Sample",
3433     DRIED   = "Dried Sample",
3434     SPUN    = "Separated Sample",
3435     CULTURE = "Culture Plate",
3436     WASTE   = "Used Material",
3437     CLEAN   = "Clean Glassware",
3438 }
3439 local I = K.Items
3440
3441 local function f(n, dp) return string.format("%. " .. dp .. "f", n) end
3442 local function rnd(a, b) return a + math.random() * (b - a) end
3443
3444 K.Actions = {
3445     ["Workbench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},
3446         log = function() return "Sample laid out on the bench." end},
3447     ["Island Bench"] = {action = "Set out a sample", wait = 1.2, produces = {I.TRAY},
3448         log = function() return "Sample laid out on the island bench." end},
3449     ["Base Cabinet Run"] = {action = "Open the cabinet", wait = 0.8,
3450         log = function() return "Consumables drawn from the cabinet." end},
3451     ["Desk"] = {action = "Write up the record", wait = 1.2,
3452         log = function() return "Notes entered in the log book." end},
3453     ["Sink Unit"] = {action = "Wash glassware", wait = 2.0, produces = {I.CLEAN},
3454         log = function() return "Glassware rinsed and set to drain." end},
3455     ["Fume Hood"] = {action = "Work under extraction", wait = 2.5, uvLight = true,

```

```

3453     log = function() return "Extraction running at " .. f(rnd(0.4, 0.6), 2) .. " m/s face velocity." end},
3454
3455     ["Laboratory Mill"] = {action = "Grind the sample", wait = 2.0,
3456       consumes = I.TRAY, produces = {I.GROUND}, activeColour = Color3.fromRGB(200, 160, 60),
3457       log = function() return "Ground for two minutes; chamber cleaned afterwards." end},
3458     ["Sieve Shaker"] = {action = "Sieve the sample", wait = 2.0,
3459       log = function() return f(rnd(45, 80), 0) .. "% passed the working mesh." end},
3460     ["Drying Oven"] = {action = "Dry the sample", wait = 3.5,
3461       consumes = I.TRAY, produces = {I.DRIED}, activeColour = Color3.fromRGB(200, 60, 50),
3462       log = function() return "Oven holding " .. f(rnd(101, 105), 1) .. " C." end},
3463     ["Centrifuge"] = {action = "Spin the sample", wait = 2.5,
3464       consumes = I.TRAY, produces = {I.SPUN},
3465       log = function() return f(rnd(3000, 12000), 0) .. " rpm for " .. math.random(5, 20) .. " minutes."
3466         ↵ end},
3467     ["Incubator"] = {action = "Incubate the sample", wait = 3.0,
3468       consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(255, 170, 0),
3469       log = function() return "Holding " .. f(rnd(19, 31), 1) .. " C; door log updated." end},
3470     ["Autoclave"] = {action = "Sterilize a load", wait = 3.0,
3471       consumes = I.WASTE, activeColour = Color3.fromRGB(0, 120, 255),
3472       log = function() return "Cycle complete at 121 C; indicator strip filed." end},
3473     ["Waste Bin"] = {action = "Discard used material", wait = 0.8, consumes = I.WASTE,
3474       log = function() return "Waste segregated for removal." end},
3475
3476     ["Analytical Balance"] = {action = "Weigh the sample", wait = 1.5,
3477       log = function()
3478         local w = rnd(0.4, 25)
3479         return "Reading " .. f(w, w < 1 and 4 or 3) .. " g; balance levelled and zeroed."
3480       end},
3481     ["Microscope"] = {action = "Examine the sample", wait = 1.5,
3482       log = function() return "Examined at " .. math.random(1, 4) * 10 .. "x." end},
3483     ["pH Meter"] = {action = "Measure pH", wait = 1.2,
3484       log = function() return "pH " .. f(rnd(4.5, 9.0), 2) .. " at " .. f(rnd(19, 23), 1) .. " C." end},
3485     ["Water Bath"] = {action = "Warm the sample", wait = 1.5,
3486       log = function() return "Bath steady at " .. f(rnd(30, 60), 1) .. " C." end},
3487     ["Hot Plate Stirrer"] = {action = "Stir and heat", wait = 1.5,
3488       log = function() return "Stirring at " .. math.random(200, 900) .. " rpm." end},
3489     ["Orbital Shaker"] = {action = "Shake the sample", wait = 1.5,
3490       log = function() return "Shaking at " .. math.random(80, 260) .. " rpm." end},
3491     ["Desiccator"] = {action = "Cool the sample", wait = 1.8,
3492       consumes = I.DRIED, produces = {I.WASTE},
3493       log = function() return "Cooled over dry desiccant before the second weighing." end},
3494     ["Computer Terminal"] = {action = "Enter the result", wait = 1.2,
3495       log = function() return "Result recorded against the sample number." end},
3496
3497     ["Refrigerator"] = {action = "Load / unload sample", wait = 1.2,
3498       log = function() return "Held at " .. f(rnd(2, 8), 1) .. " C." end},
3499     ["Chest Freezer"] = {action = "Load / unload sample", wait = 1.2,
3500       log = function() return "Held at " .. f(rnd(-25, -16), 0) .. " C." end},
3501     ["Sample Cabinet"] = {action = "File the sample", wait = 1.0,
3502       log = function() return "Filed by sample number." end},
3503     ["Shelving Unit"] = {action = "Draw stock", wait = 0.8,
3504       log = function() return "Stock drawn; reorder level checked." end},
3505     ["Storage Rack"] = {action = "Draw stock", wait = 0.8,
3506       log = function() return "Stock drawn from the rack." end},
3507     ["Glassware Rack"] = {action = "Take clean glassware", wait = 0.8, produces = {I.CLEAN},
3508       log = function() return "Clean glassware taken from the rack." end},
3509     ["Trolley"] = {action = "Load the trolley", wait = 0.8,
3510       log = function() return "Trolley loaded for transfer." end},
3511     ["Filing Cabinet"] = {action = "Pull a file", wait = 0.8,
3512       log = function() return "File retrieved." end},
3513     ["Locker Bank"] = {action = "Change clothing", wait = 1.2,
3514       log = function() return "Changed before entering the working area." end},
3515     ["Gas Cylinder Rack"] = {action = "Check the cylinders", wait = 1.0,
3516       log = function() return "Cylinders chained; regulator pressure " .. math.random(20, 200) .. " bar."
3517         ↵ end},
3518     ["Reagent Rack"] = {action = "Take a reagent", wait = 0.8,
3519       log = function() return "Reagent taken; bottle logged back in." end},
3520     ["Test Tube Rack"] = {action = "Rack the tubes", wait = 0.8,
3521       log = function() return "Tubes racked and labelled." end},
3522     ["Pipette Stand"] = {action = "Take a pipette", wait = 0.8,
3523       log = function() return "Pipette taken; last service date in date." end},
3524
3525     ["Germination Chamber"] = {action = "Load a germination test", wait = 2.5,
3526       consumes = I.TRAY, produces = {I.CULTURE}, activeColour = Color3.fromRGB(120, 200, 140),
3527       log = function() return "Test set at " .. f(rnd(18, 25), 1) .. " C, day count started." end},
3528     ["Sample Divider"] = {action = "Divide the sample", wait = 1.8,

```

```

3527     consumes = I.GROUND, produces = {I.TRAY},
3528     log = function() return "Split into equal working portions." end},
3529     ["Decontamination Unit"] = {action = "Run decontamination", wait = 2.2, consumes = I.WASTE,
3530     activeColour = Color3.fromRGB(150, 210, 255),
3531     log = function() return "Cycle complete; surfaces clear." end},
3532     ["Water Distiller"] = {action = "Draw distilled water", wait = 1.2,
3533     log = function() return "Conductivity " .. f(rnd(0.5, 2.5), 1) .. " uS/cm." end},
3534     ["Reference Sample Archive"] = {action = "File a reference sample", wait = 1.0,
3535     log = function() return "Filed against the reference register." end},
3536     ["Optical Analyzer"] = {action = "Run an optical reading", wait = 1.8,
3537     log = function() return "Reading logged at " .. math.random(400, 800) .. " nm." end},
3538     ["Electrophoresis Unit"] = {action = "Run a separation", wait = 2.0, activeColour = Color3.fromRGB(0, 120,
3539     ↪ 255),
3540     log = function() return "Run complete at " .. math.random(60, 150) .. " V." end},
3541     ["Thermal Cycler"] = {action = "Run an amplification cycle", wait = 2.5, activeColour = Color3.fromRGB(255,
3542     ↪ 170, 0),
3543     log = function() return math.random(25, 40) .. " cycles complete." end},
3544     ["Homogenizer"] = {action = "Homogenize the sample", wait = 1.5,
3545     log = function() return "Sample homogenized at " .. math.random(8000, 24000) .. " rpm." end},
3546     ["Seed Counter"] = {action = "Count the sample", wait = 1.5,
3547     log = function() return math.random(200, 1200) .. " units counted." end},
3548     ["Calibration Weight Set"] = {action = "Check calibration", wait = 1.0,
3549     log = function() return "Balance checked against certified weights." end},
3550     ["Temperature Logger"] = {action = "Read the logger", wait = 0.6,
3551     log = function() return "Log downloaded; " .. f(rnd(2, 8), 1) .. " C average." end},
3552     ["Plant Growth Chamber"] = {action = "Take / return plant", wait = 0.8},
3553     ["Growth Hormone Mixer"] = {action = "Apply growth hormone", wait = 2.0},
3554     ["GMO Injector"] = {action = "Inject GMO trait", wait = 2.4},
3555     ["Nutrient Infuser"] = {action = "Infuse nutrients", wait = 1.8},
3556     ["UV Growth Scanner"] = {action = "Scan plant development", wait = 1.5},
3557     ["Pollination Station"] = {action = "Pollinate the plant", wait = 2.1},
3558     ["Mutagenesis Chamber"] = {action = "Irradiate the sample", wait = 2.4,
3559     log = function() return math.random(100, 1000) .. " Gy dose logged; dosimeter reset." end},
3560     ["Hormone Treatment Bench"] = {action = "Apply hormone treatment", wait = 2.0,
3561     log = function() return "Auxin/cytokinin mix dosed and logged." end},
3562     -- Cooperative role stations: LabEquipment special-cases these three
3563     -- kinds before ever reaching this table, so only the prompt flavour
3564     -- text below (action/wait) is actually used for them.
3565     ["Sample Intake"] = {action = "Log a sample", wait = 1.0},
3566     ["Inspection Station"] = {action = "Inspect the batch", wait = 1.2},
3567     ["Supervisor Console"] = {action = "Sign off the batch", wait = 1.5},
3568     ["Fire Extinguisher"] = {action = "Check the extinguisher", wait = 1.0,
3569     log = function() return "Gauge in the green; inspection tag current." end},
3570     ["First Aid Cabinet"] = {action = "Check the first aid kit", wait = 1.0,
3571     log = function() return "Contents complete." end},
3572     ["Eyewash Station"] = {action = "Flush the eyewash", wait = 1.5,
3573     log = function() return "Weekly flush run; water clear." end},
3574 }
3575 }
3576
3577 K.Default = {action = "Inspect", wait = 1.0,
3578     log = function() return "Checked and left in order." end}
3579
3580 function K.forKind(kind) return K.Actions[kind] or K.Default end
3581
3582 return K
3583 ]==]
3584
3585 local EQUIP_SCRIPT_SOURCE = [=[
3586 -- LabEquipment : physical controls, equipment readouts, carried samples,
3587 -- cold-store delivery missions, and non-lethal sample recovery tools.
3588 -- All inventory and hit decisions happen on the server.
3589
3590 local ReplicatedStorage = game:GetService("ReplicatedStorage")
3591 local CollectionService = game:GetService("CollectionService")
3592 local Players = game:GetService("Players")
3593 local TweenService = game:GetService("TweenService")
3594 local Debris = game:GetService("Debris")
3595 local PathfindingService = game:GetService("PathfindingService")
3596
3597 local Kinds = require(ReplicatedStorage:WaitForChild("LabKinds"))
3598 local QuizRemote = ReplicatedStorage:FindFirstChild("LabMathQuiz")
3599 if QuizRemote and not QuizRemote:IsA("RemoteEvent") then QuizRemote:Destroy(); QuizRemote = nil end
3600 if not QuizRemote then

```

```

3601     QuizRemote = Instance.new("RemoteEvent")
3602     QuizRemote.Name = "LabMathQuiz"
3603     QuizRemote.Parent = ReplicatedStorage
3604 end
3605
3606 -----
3607 -- SHARED GAME STATE
3608 -----
3609 local GameState = {
3610     repairsCompleted = 0,
3611     assignedRepairsCompleted = 0,
3612     transfersCompleted = 0,
3613     assignedTransfersCompleted = 0,
3614 }
3615
3616 local ROLE_STATIONS = {}
3617
3618 local POINTS = {
3619     MOVE_ANY = 12,
3620     MOVE_ASSIGNED_LATE = 18,
3621     MOVE_ASSIGNED_ON_TIME = 30,
3622     PROCESS_PLANT = 4,
3623 }
3624
3625 local function updateScoreboard()
3626     local board = CollectionService:GetTagged("BatchScoreboard")[1]
3627     if not board then return end
3628     local gui = board:FindFirstChildWhichIsA("BillboardGui")
3629     local label = gui and gui:FindFirstChild("Text")
3630     if label then
3631         local rivalScore = 0
3632         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
3633             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
3634         end
3635         label.Text = string.format(
3636             "PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d • assigned: %d\nRival NPC
↪ score: %d",
3637             GameState.repairsCompleted, GameState.assignedRepairsCompleted,
3638             GameState.transfersCompleted, GameState.assignedTransfersCompleted, rivalScore)
3639     end
3640 end
3641
3642 -- Per-room equipment status monitors. Each RoomMonitor panel carries a
3643 -- Room attribute matching the LabEquipment parts placed in that same room,
3644 -- so this just groups by that attribute and tallies running/failed/plants.
3645 local function refreshRoomMonitors()
3646     -- Room names ("Room 4", etc.) repeat on every level, so the key here
3647     -- must include the level too -- otherwise a monitor sums up every
3648     -- same-numbered room across the whole building instead of just its own.
3649     local totals = {}
3650     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
3651         local room = equip:GetAttribute("Room")
3652         local level = equip:GetAttribute("Level") or ""
3653         if room then
3654             local key = level .. "|" .. room
3655             local t = totals[key]
3656             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
3657             t.count = t.count + 1
3658             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
3659             else t.running = t.running + 1 end
3660             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
3661         end
3662     end
3663     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
3664         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
3665         local label = surface and surface:FindFirstChild("Status")
3666         if label then
3667             local room = panel:GetAttribute("Room") or "ROOM"
3668             local level = panel:GetAttribute("Level") or ""
3669             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
3670             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
3671                 room, t.count, t.running, t.failed, t.plants)
3672             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235, 170)
3673         end
3674     end
3675 end

```

```

3676
3677 local function addContribution(player, amount)
3678     local stats = player:FindFirstChild("Leaderstats")
3679     local value = stats and stats:FindFirstChild("Contributions")
3680     if value then value.Value = value.Value + (amount or 1) end
3681 end
3682
3683 local facilityNoticeAt = setmetatable({}, {__mode = "k"})
3684 local function notify(player, text)
3685     local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
3686     ↪ text:match("^CATASTROPHIC FAILURE:")
3687     if facilityAlert then
3688         local now = os.clock()
3689         if now - (facilityNoticeAt[player] or -math.huge) < 6 then return end
3690         facilityNoticeAt[player] = now
3691     end
3692     print("[ " .. player.Name .. " ] " .. text)
3693     local gui = player:FindFirstChild("PlayerGui")
3694     if not gui then return end
3695     local screen = gui:FindFirstChild("LabLog")
3696     if not screen then
3697         screen = Instance.new("ScreenGui")
3698         screen.Name = "LabLog"
3699         screen.ResetOnSpawn = false
3700         screen.DisplayOrder = 40
3701         screen.Parent = gui
3702
3703         local frame = Instance.new("TextLabel")
3704         frame.Name = "Line"
3705         frame.Size = UDim2.new(0.48, 0, 0, 46)
3706         frame.AnchorPoint = Vector2.new(0.5, 1)
3707         frame.Position = UDim2.new(0.5, 0, 1, -68)
3708         frame.BackgroundColor3 = Color3.fromRGB(18, 22, 28)
3709         frame.BackgroundTransparency = 0.12
3710         frame.TextColor3 = Color3.fromRGB(235, 238, 242)
3711         frame.TextWrapped = true
3712         frame.TextTruncate = Enum.TextTruncate.AtEnd
3713         frame.TextSize = 13
3714         frame.Font = Enum.Font.Gotham
3715         frame.Parent = screen
3716         local c = Instance.new("UICorner")
3717         c.CornerRadius = UDim.new(0, 8)
3718         c.Parent = frame
3719         local sizeLimit = Instance.new("UISizeConstraint")
3720         sizeLimit.MinSize = Vector2.new(260, 46)
3721         sizeLimit.MaxSize = Vector2.new(420, 46)
3722         sizeLimit.Parent = frame
3723         local padding = Instance.new("UIPadding")
3724         padding.PaddingLeft = UDim.new(0, 10)
3725         padding.PaddingRight = UDim.new(0, 10)
3726         padding.Parent = frame
3727     end
3728     local line = screen:FindFirstChild("Line")
3729     if not line then return end
3730     local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
3731     screen:SetAttribute("MessageSerial", serial)
3732     line.Visible = true
3733     line.Text = text
3734     task.delay(4, function()
3735         if line.Parent and screen:GetAttribute("MessageSerial") == serial then line.Visible = false end
3736     end)
3737 end
3738
3739 local function playerRole(player)
3740     local attributeRole = player:GetAttribute("LabRole")
3741     if attributeRole and attributeRole ~= "" then return attributeRole end
3742     if player.Team then return player.Team.Name end
3743     local stats = player:FindFirstChild("leaderstats")
3744     local role = stats and stats:FindFirstChild("Role")
3745     return role and role.Value or "Unassigned"
3746 end
3747
3748 local function awardPoints(player, amount, reason)
3749     local stats = player:FindFirstChild("leaderstats")
3750     if not stats then

```

```

3751     stats = Instance.new("Folder")
3752     stats.Name = "leaderstats"
3753     stats.Parent = player
3754 end
3755 local points = stats:FindFirstChild("Points")
3756 if not points then
3757     points = Instance.new("IntValue")
3758     points.Name = "Points"
3759     points.Parent = stats
3760 end
3761 points.Value = points.Value + amount
3762 addContribution(player, 1)
3763 player:SetAttribute("LastPointAmount", amount)
3764 player:SetAttribute("LastPointAward", reason)
3765 notify(player, string.format("+%d POINTS - %s • TOTAL %d", amount, reason, points.Value))
3766 updateScoreboard()
3767 return points.Value
3768 end
3769
3770 local function canInteract(player, part, maxDistance)
3771     local character = player.Character
3772     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3773     local root = character and character:FindFirstChild("HumanoidRootPart")
3774     if humanoid == nil or humanoid.Health <= 0 or root == nil
3775         or (root.Position - part.Position).Magnitude > maxDistance + 2 then return false end
3776     local originPart = character:FindFirstChild("Head") or root
3777     local direction = part.Position - originPart.Position
3778     local params = RaycastParams.new()
3779     params.FilterType = Enum.RaycastFilterType.Exclude
3780     params.FilterDescendantsInstances = {character}
3781     params.IgnoreWater = true
3782     local hit = workspace:Raycast(originPart.Position, direction, params)
3783     return hit == nil or hit.Instance == part or hit.Instance:IsDescendantOf(part)
3784 end
3785
3786 -- Server-owned, one-use addition challenges. No InvokeClient yield can leave
3787 -- equipment locked forever; the server expires or cancels each pending action.
3788 local pendingQuizzes = {}
3789 local quizSerial = 0
3790 QuizRemote.OnServerEvent:Connect(function(player, id, answer)
3791     local pending = pendingQuizzes[player]
3792     if not pending or pending.id ~= id or pending.finished or os.clock() > pending.expires then return end
3793     if answer ~= nil and (typeof(answer) ~= "number" or answer ~= answer or answer % 1 ~= 0) then return end
3794     pending.answer = answer
3795     pending.finished = true
3796 end)
3797 Players.PlayerRemoving:Connect(function(player) pendingQuizzes[player] = nil end)
3798
3799 local function runMathQuiz(player, reason, target, validate)
3800     if pendingQuizzes[player] or not player.Parent or not validate() then return false end
3801     local a, b = math.random(1, 100), math.random(1, 100)
3802     local expected = a + b
3803     local choices = {expected, expected + math.random(1, 9), math.max(0, expected - math.random(1, 9))}
3804     for i = #choices, 2, -1 do local j = math.random(1, i); choices[i], choices[j] = choices[j], choices[i] end
3805     quizSerial = quizSerial + 1
3806     local pending = {id = quizSerial, expected = expected, expires = os.clock() + 60, finished = false}
3807     pendingQuizzes[player] = pending
3808     player:SetAttribute("MathQuizActive", true)
3809     target:SetAttribute("QuizPlayerId", player.UserId)
3810     QuizRemote.FireClient(player, {Id = pending.id, Question = string.format("%d + %d", a, b),
3811         Choices = choices, Seconds = 60, Reason = reason})
3812     while player.Parent and pendingQuizzes[player] == pending and not pending.finished
3813         and os.clock() < pending.expires and target.Parent and validate() do task.wait(0.1) end
3814     local correct = player.Parent ~= nil and target.Parent ~= nil and pendingQuizzes[player] == pending
3815         and pending.finished and pending.answer == pending.expected and os.clock() <= pending.expires and
3816         ↪ validate()
3817     if pendingQuizzes[player] == pending then pendingQuizzes[player] = nil end
3818     if target.Parent and target:GetAttribute("QuizPlayerId") == player.UserId then
3819         ↪ target:SetAttribute("QuizPlayerId", nil) end
3820     if player.Parent then
3821         player:SetAttribute("MathQuizActive", false)
3822         QuizRemote.FireClient(player, {Close = true, Id = pending.id})
3823         if not correct then notify(player, "Task not completed. Stay near the unit and try the addition check
        ↪ again.") end
3824     end
3825     return correct

```



```

3824 end
3825
3826 -----
3827 -- VISIBLE INVENTORY ITEMS
3828 -- The old script collection used Tool + Handle + welded decorative
3829 -- pieces. This keeps that useful pattern, but creates and validates the
3830 -- tools entirely on the server instead of trusting a client script.
3831 -----
3832 local SAMPLE_ITEM = {
3833     [Kinds.Items.TRAY] = true,
3834     [Kinds.Items.GROUND] = true,
3835     [Kinds.Items.DRIED] = true,
3836     [Kinds.Items.SPUN] = true,
3837     [Kinds.Items.CULTURE] = true,
3838 }
3839
3840 local function addWeldedPart(tool, handle, name, size, relativeCFrame, colour, material, shape)
3841     local p = Instance.new("Part")
3842     p.Name = name
3843     p.Size = size
3844     p.CFrame = handle.CFrame * relativeCFrame
3845     p.Color = colour
3846     p.Material = material or Enum.Material.SmoothPlastic
3847     p.Shape = shape or Enum.PartType.Block
3848     p.Anchored = false
3849     p.CanCollide = false
3850     p.CanTouch = false
3851     p.Massless = true
3852     p.TopSurface = Enum.SurfaceType.Smooth
3853     p.BottomSurface = Enum.SurfaceType.Smooth
3854     p.Parent = tool
3855     local weld = Instance.new("WeldConstraint")
3856     weld.Part0 = handle
3857     weld.Part1 = p
3858     weld.Parent = p
3859     return p
3860 end
3861
3862 local function equipTool(player, tool)
3863     task.defer(function()
3864         if not tool.Parent then return end
3865         local character = player.Character
3866         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
3867         if humanoid and humanoid.Health > 0 then humanoid:EquipTool(tool) end
3868     end)
3869 end
3870
3871 local function makeTool(player, name, options)
3872     options = options or {}
3873     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
3874     if not backpack then return nil end
3875
3876     local tool = Instance.new("Tool")
3877     tool.Name = name
3878     tool.ToolTip = options.tooltip or "Visible laboratory item"
3879     tool.RequiresHandle = true
3880     tool.CanBeDropped = false
3881     tool:SetAttribute("LabItemType", options.itemType or name)
3882     tool:SetAttribute("LabSample", options.sample == true or SAMPLE_ITEM[name] == true)
3883
3884     for key, value in pairs(options.attributes or {}) do tool:SetAttribute(key, value) end
3885     if options.plant and tool:GetAttribute("PlantLife") == nil then tool:SetAttribute("PlantLife", 100) end
3886
3887     local handle = Instance.new("Part")
3888     handle.Name = "Handle"
3889     handle.Size = options.plant and Vector3.new(0.95, 0.72, 0.95) or Vector3.new(1.7, 0.22, 1.35)
3890     handle.Color = options.colour or Color3.fromRGB(205, 185, 138)
3891     handle.Material = options.plant and Enum.Material.WoodPlanks or Enum.Material.SmoothPlastic
3892     handle.Anchored = false
3893     handle.CanCollide = false
3894     handle.Massless = true
3895     handle.TopSurface = Enum.SurfaceType.Smooth
3896     handle.BottomSurface = Enum.SurfaceType.Smooth
3897     handle.Parent = tool
3898
3899     if options.plant then

```

```

3900 -- A real visible potted plant: soil, stem, leaves, and a glowing bud
3901 -- are welded to the Tool handle and replicate in the character hand.
3902 addWeldedPart(tool, handle, "Soil", Vector3.new(0.82, 0.14, 0.82),
3903     CFrame.new(0, 0.4, 0), Color3.fromRGB(78, 52, 33), Enum.Material.Ground)
3904 addWeldedPart(tool, handle, "PlantStem", Vector3.new(0.16, 1.0, 0.16),
3905     CFrame.new(0, 0.96, 0), Color3.fromRGB(65, 155, 75), Enum.Material.Grass)
3906 addWeldedPart(tool, handle, "LeafLeft", Vector3.new(0.78, 0.16, 0.42),
3907     CFrame.new(-0.34, 1.1, 0) * CFrame.Angles(0, 0, math.rad(-22)),
3908     Color3.fromRGB(70, 180, 82), Enum.Material.Grass)
3909 addWeldedPart(tool, handle, "LeafRight", Vector3.new(0.78, 0.16, 0.42),
3910     CFrame.new(0.34, 1.35, 0) * CFrame.Angles(0, 0, math.rad(22)),
3911     Color3.fromRGB(85, 195, 92), Enum.Material.Grass)
3912 addWeldedPart(tool, handle, "GrowthBud", Vector3.new(0.36, 0.36, 0.36),
3913     CFrame.new(0, 1.58, 0), Color3.fromRGB(245, 185, 80), Enum.Material.Neon, Enum.PartType.Ball)
3914 else
3915 -- Tray rim, sample pouch, and labelled vial make ordinary samples
3916 -- recognizable from a distance instead of a plain block.
3917 addWeldedPart(tool, handle, "TrayRim", Vector3.new(1.82, 0.12, 1.47),
3918     CFrame.new(0, 0.14, 0), Color3.fromRGB(55, 68, 78), Enum.Material.Metal)
3919 addWeldedPart(tool, handle, "SeedPouch", Vector3.new(0.92, 0.32, 0.76),
3920     CFrame.new(0, 0.35, 0), Color3.fromRGB(220, 193, 116), Enum.Material.Fabric)
3921 addWeldedPart(tool, handle, "SampleVial", Vector3.new(0.62, 0.22, 0.22),
3922     CFrame.new(0.47, 0.54, 0) * CFrame.Angles(0, 0, math.rad(90)),
3923     Color3.fromRGB(115, 215, 245), Enum.Material.Glass, Enum.PartType.Cylinder)
3924 end
3925
3926 local marker = Instance.new("BillboardGui")
3927 marker.Name = "CarryLabel"
3928 marker.Size = UDim2.fromOffset(150, 34)
3929 marker.StudsOffsetWorldSpace = Vector3.new(0, 1.15, 0)
3930 marker.MaxDistance = 8
3931 marker.AlwaysOnTop = false
3932 marker.Parent = handle
3933 local markerText = Instance.new("TextLabel")
3934 markerText.Size = UDim2.fromScale(1, 1)
3935 markerText.BackgroundColor3 = Color3.fromRGB(20, 25, 30)
3936 markerText.BackgroundTransparency = 0.2
3937 markerText.TextColor3 = Color3.fromRGB(245, 225, 150)
3938 markerText.TextScaled = true
3939 markerText.Text = options.label or name
3940 markerText.Font = Enum.Font.GothamBold
3941 markerText.Parent = marker
3942 local markerCorner = Instance.new("UICorner")
3943 markerCorner.CornerRadius = UDim.new(0, 6)
3944 markerCorner.Parent = markerText
3945 if options.plant then
3946     local lifeGui = Instance.new("BillboardGui")
3947     lifeGui.Name = "PlantLifeStatus"
3948     lifeGui.Size = UDim2.fromOffset(160, 46)
3949     lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
3950     lifeGui.MaxDistance = 18
3951     lifeGui.AlwaysOnTop = false
3952     lifeGui.Parent = handle
3953     local lifeFrame = Instance.new("Frame")
3954     lifeFrame.Size = UDim2.fromScale(1, 1)
3955     lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
3956     lifeFrame.BackgroundTransparency = 0.12
3957     lifeFrame.BorderSizePixel = 0
3958     lifeFrame.Parent = lifeGui
3959     local lifeText = Instance.new("TextLabel")
3960     lifeText.Name = "LifeText"
3961     lifeText.Size = UDim2.new(1, 0, 0.58, 0)
3962     lifeText.BackgroundTransparency = 1
3963     lifeText.TextScaled = true
3964     lifeText.Font = Enum.Font.GothamBold
3965     lifeText.Parent = lifeFrame
3966     local lifeBack = Instance.new("Frame")
3967     lifeBack.Name = "LifeBarBackground"
3968     lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
3969     lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
3970     lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
3971     lifeBack.BorderSizePixel = 0
3972     lifeBack.Parent = lifeFrame
3973     local lifeFill = Instance.new("Frame")
3974     lifeFill.Name = "LifeBarFill"
3975     lifeFill.Size = UDim2.fromScale(1, 1)

```

```

3976     lifeFill.BorderSizePixel = 0
3977     lifeFill.Parent = lifeBack
3978     local function updateLifeBar()
3979         local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
3980         lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
3981         lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
3982         local colour = life > 60 and Color3.fromRGB(75, 235, 105)
3983             or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
3984         lifeText.TextColor3 = colour
3985         lifeFill.BackgroundColor3 = colour
3986     end
3987     tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLifeBar)
3988     updateLifeBar()
3989 end
3990
3991 tool.Grip = options.plant
3992     and (CFrame.new(0, -0.25, -0.75) * CFrame.Angles(math.rad(-8), 0, 0))
3993     or (CFrame.new(0, -0.45, -0.65) * CFrame.Angles(math.rad(-12), 0, 0))
3994 tool.Parent = backpack
3995 if options.autoEquip ~= false then equipTool(player, tool) end
3996 return tool
3997 end
3998
3999 local function playerContainers(player)
4000     local containers = {}
4001     if player.Character then table.insert(containers, player.Character) end
4002     local backpack = player:FindFirstChildOfClass("Backpack")
4003     if backpack then table.insert(containers, backpack) end
4004     return containers
4005 end
4006
4007 local function findTool(player, itemType)
4008     for _, container in ipairs(playerContainers(player)) do
4009         if container then
4010             for _, child in ipairs(container:GetChildren()) do
4011                 if child:IsA("Tool") and
4012                     (child.Name == itemType or child:GetAttribute("LabItemType") == itemType) then
4013                     return child
4014                 end
4015             end
4016         end
4017     end
4018     return nil
4019 end
4020
4021 local function findTransferSample(player)
4022     for _, container in ipairs(playerContainers(player)) do
4023         if container then
4024             for _, child in ipairs(container:GetChildren()) do
4025                 if child:IsA("Tool") and child:GetAttribute("TransferSample") then return child end
4026             end
4027         end
4028     end
4029     return nil
4030 end
4031
4032 local function findHeldSample(player)
4033     local character = player.Character
4034     if not character then return nil end
4035     for _, child in ipairs(character:GetChildren()) do
4036         if child:IsA("Tool") and child:GetAttribute("LabSample") then return child end
4037     end
4038     return nil
4039 end
4040
4041 local GROWTH_STAGE_NAME = {
4042     [0] = "Seedling",
4043     [1] = "Germinated",
4044     [2] = "Vegetative",
4045     [3] = "Budding",
4046     [4] = "Flowering",
4047     [5] = "Mature",
4048 }
4049
4050 local function findPlantSample(player)
4051     for _, container in ipairs(playerContainers(player)) do

```

```

4052     for _, child in ipairs(container:GetChildren()) do
4053         if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end
4054     end
4055 end
4056 return nil
4057 end
4058
4059 local function treatmentText(tool)
4060     local treatments = tool:GetAttribute("Treatments") or ""
4061     if treatments == "" then return "untreated" end
4062     return treatments:gsub("|", ", ")
4063 end
4064
4065 local function hasTreatment(tool, key)
4066     local treatments = "|" .. (tool:GetAttribute("Treatments") or "") .. "|"
4067     return treatments:find("|" .. key .. "|", 1, true) ~= nil
4068 end
4069
4070 local function addTreatment(tool, key)
4071     if hasTreatment(tool, key) then return false end
4072     local treatments = tool:GetAttribute("Treatments") or ""
4073     tool:SetAttribute("Treatments", treatments == "" and key or (treatments .. "|" .. key))
4074     return true
4075 end
4076
4077 local function refreshPlantVisual(tool)
4078     local stage = math.clamp(tool:GetAttribute("GrowthStage") or 0, 0, 5)
4079     local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
4080     local stem = tool:FindFirstChild("PlantStem")
4081     local left = tool:FindFirstChild("LeafLeft")
4082     local right = tool:FindFirstChild("LeafRight")
4083     local bud = tool:FindFirstChild("GrowthBud")
4084     if stem then stem.Size = Vector3.new(0.16 + stage * 0.025, 1 + stage * 0.13, 0.16 + stage * 0.025) end
4085     local lifeScale = 0.35 + life / 100 * 0.65
4086     if left then left.Color = Color3.fromRGB((70 + stage * 5) * lifeScale, (165 + stage * 10) * lifeScale, 55 *
↵ lifeScale) end
4087     if right then right.Color = Color3.fromRGB((75 + stage * 4) * lifeScale, (175 + stage * 10) * lifeScale, 60
↵ * lifeScale) end
4088     if bud then
4089         bud.Transparency = stage < 3 and 0.65 or 0
4090         bud.Color = stage >= 5 and Color3.fromRGB(255, 105, 175) or Color3.fromRGB(245, 185, 80)
4091     end
4092     local handle = tool:FindFirstChild("Handle")
4093     local marker = handle and handle:FindFirstChild("CarryLabel")
4094     local label = marker and marker:FindFirstChildOfClass("TextLabel")
4095     if label then
4096         local destination = tool:GetAttribute("DestinationGrowthCode") or ""
4097         label.Text = (tool:GetAttribute("PlantId") or "PLANT") .. " • " .. (GROWTH_STAGE_NAME[stage] or
↵ "Growing")
4098         .. (destination ~= "" and (" • BONUS DROP " .. destination) or " • DROP IN GREEN R")
4099     end
4100 end
4101
4102 local function setCarryState(player, sample)
4103     -- A cold-store transfer remains the main HUD objective even if the
4104     -- player briefly produces or pockets another laboratory item.
4105     local transfer = findTransferSample(player)
4106     if transfer and sample and not sample:GetAttribute("TransferSample") then sample = transfer end
4107     if sample then
4108         local id = sample:GetAttribute("SampleId") or sample.Name
4109         local destination = sample:GetAttribute("DestinationLabel") or
↵ sample:GetAttribute("DestinationGrowthCode")
4110         player:SetAttribute("CarriedSample", id)
4111         if destination and destination ~= "" then
4112             player:SetAttribute("LabObjective", "Carry " .. id .. " to bonus chamber " .. destination .. ", or
↵ press E at any GREEN RUNNING cabinet with a free slot.")
4113         else
4114             player:SetAttribute("LabObjective", "Carry " .. sample.Name .. " to the next required machine.")
4115         end
4116     else
4117         player:SetAttribute("CarriedSample", "")
4118         player:SetAttribute("LabObjective", "")
4119     end
4120 end
4121
4122 local function destroyCarriedItem(player, tool)

```

```

4123     if player:GetAttribute("CarriedSample") == (tool:GetAttribute("SampleId") or tool.Name) then
4124         setCarryState(player, nil)
4125     end
4126     tool:Destroy()
4127 end
4128
4129 -----
4130 -- NON-LETHAL HIT TOOLS
4131 -----
4132 local IMPACT_STYLE = {
4133     ["Foam Baton"] = {colour = Color3.fromRGB(245, 135, 55), accent = Color3.fromRGB(45, 48, 54), mode =
4134         ↪ "melee", damage = 18},
4135     ["Sample Grabber"] = {colour = Color3.fromRGB(70, 165, 220), accent = Color3.fromRGB(205, 225, 235), mode =
4136         ↪ "melee", damage = 12},
4137     ["Inspection Paddle"] = {colour = Color3.fromRGB(245, 205, 70), accent = Color3.fromRGB(95, 75, 30), mode =
4138         ↪ "melee", damage = 22},
4139     ["Stun Blaster"] = {colour = Color3.fromRGB(90, 225, 245), accent = Color3.fromRGB(35, 55, 70), mode =
4140         ↪ "gun", damage = 28, range = 95},
4141     ["Foam Dart Rifle"] = {colour = Color3.fromRGB(225, 80, 95), accent = Color3.fromRGB(45, 45, 52), mode =
4142         ↪ "gun", damage = 38, range = 125},
4143     ["Basketball"] = {colour = Color3.fromRGB(225, 105, 35), accent = Color3.fromRGB(45, 28, 18), mode =
4144         ↪ "ball", damage = 24},
4145     -- PC-only weapons, built from the official Roblox Weapons Kit meshes.
4146     --
4147     -- meshScale is (the kit's intended size / the raw mesh size), measured off
4148     -- the kit's own MeshParts. This matters: the raw pistol mesh is 9.68 studs
4149     -- long, so the scale of 1.8 used at first drew a pistol about 17 studs
4150     -- long -- roughly three times the height of the character holding it.
4151     --
4152     -- A worker bot has 100 life, so damage is set for 2-3 body shots; a head
4153     -- shot kills outright regardless. selfCostPercent still charges the
4154     -- shooter 1% of their own max health per shot. pcOnly means the generic
4155     -- Tool.Activated path ignores them -- they fire only through the
4156     -- LabSidearmFire remote, which only the desktop client script sends.
4157     ["Lab Sidearm"] = {
4158         colour = Color3.fromRGB(126, 133, 145), accent = Color3.fromRGB(38, 42, 50),
4159         mode = "gun", damage = 34, range = 240, selfCostPercent = 1, pcOnly = true,
4160         cooldown = 0.32, pellets = 1, spread = 0,
4161         meshId = "rbxassetid://2492922600", meshTexture = "rbxassetid://2492931263",
4162         meshScale = 0.1407, handleSize = Vector3.new(0.20, 0.92, 1.36),
4163         grip = CFrame.new(0, -0.20, -0.30) * CFrame.Angles(0, math.rad(180), 0),
4164     },
4165     ["Lab Shotgun"] = {
4166         colour = Color3.fromRGB(122, 98, 72), accent = Color3.fromRGB(40, 34, 30),
4167         mode = "gun", damage = 20, range = 110, selfCostPercent = 1, pcOnly = true,
4168         cooldown = 0.85, pellets = 6, spread = 0.075,
4169         meshId = "rbxassetid://2492972199", meshTexture = "rbxassetid://2492974190",
4170         meshScale = 0.6666, handleSize = Vector3.new(0.31, 1.00, 4.87),
4171         grip = CFrame.new(0, -0.35, -0.95) * CFrame.Angles(0, math.rad(180), 0),
4172     },
4173     ["Lab Rifle"] = {
4174         colour = Color3.fromRGB(96, 104, 96), accent = Color3.fromRGB(34, 38, 34),
4175         mode = "gun", damage = 55, range = 620, selfCostPercent = 1, pcOnly = true,
4176         cooldown = 0.95, pellets = 1, spread = 0,
4177         meshId = "rbxassetid://2759059561", meshTexture = "rbxassetid://2759063893",
4178         meshScale = 0.6383, handleSize = Vector3.new(0.29, 1.17, 6.18),
4179         grip = CFrame.new(0, -0.35, -1.20) * CFrame.Angles(0, math.rad(180), 0),
4180     },
4181 }
4182
4183 local function refreshBotHealth(bot)
4184     local health = math.clamp(bot:GetAttribute("Health") or 100, 0, bot:GetAttribute("MaxHealth") or 100)
4185     local maximum = bot:GetAttribute("MaxHealth") or 100
4186     local text = bot:FindFirstChild("HealthText", true)
4187     local fill = bot:FindFirstChild("HealthBarFill", true)
4188     if text and text:IsA("TextLabel") then text.Text = string.format("LIFE %d / %d", health, maximum) end
4189     if fill and fill:IsA("Frame") then
4190         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
4191         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
4192         or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
4193     end
4194 end
4195
4196 -- How long a body stays before it is cleaned up. Bodies are the biggest
4197 -- avoidable part cost in a long round, so this is deliberately short.
4198 -- Declared HERE, above damageWorkerBot, because that is what reads it: a

```

```

4193 -- local declared further down the file resolves to a nil global inside a
4194 -- function defined above it.
4195 local BODY_LIFETIME_SECONDS = 60
4196
4197 local function damageWorkerBot(bot, amount, attacker)
4198     -- Only an already-dead worker is immune. This used to refuse damage to
4199     -- anything KnockedOut, which meant a worker downed by a hazard could not
4200     -- be finished off by gunfire or caught in a blast -- so a floor could
4201     -- never actually be cleared.
4202     if not bot or not CollectionService.HasTag(bot, "LabWorkerBot") or bot:GetAttribute("Dead") then return
4203     ↪ false end
4204     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
4205     bot:SetAttribute("Health", health)
4206     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
4207     refreshBotHealth(bot)
4208     local flash = Instance.new("Highlight")
4209     flash.FillColor = Color3.fromRGB(255, 90, 70)
4210     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
4211     flash.FillTransparency = 0.25
4212     -- Highlights render through geometry by default, which is why a hit
4213     -- worker (and any corpse that inherited the flash) glowed through walls.
4214     flash.DepthMode = Enum.HighlightDepthMode.Occluded
4215     flash.Parent = bot
4216     Debris.AddItem(flash, 0.22)
4217     if health <= 0 then
4218         -- Death is permanent now: the hunt is the objective, so a worker that
4219         -- goes down stays down and its body is left where it fell. Bodies are
4220         -- cleared when the shift rolls over (see the RoundNumber watcher).
4221         bot:SetAttribute("Dead", true)
4222         bot:SetAttribute("KnockedOut", true)
4223         bot:SetAttribute("TaskState", "DEAD")
4224
4225         -- The body left behind is a CLONE, not the worker itself. That keeps
4226         -- the original model pristine, so the shift roll-over can simply move
4227         -- it home and stand it back up rather than trying to undo a pile of
4228         -- destructive colour and rotation edits.
4229         local pivot = bot:GetPivot()
4230         local corpse = bot:Clone()
4231         corpse.Name = bot.Name .. "_Body"
4232         for _, d in ipairs(corpse:GetDescendants()) do
4233             -- A body reports no status, and must not inherit the damage-flash
4234             -- Highlight -- that is what made corpses visible through walls,
4235             -- because the clone's copy was never cleaned up.
4236             if d:IsA("BillboardGui") or d:IsA("Highlight") or d:IsA("Sparkles")
4237             or d:IsA("Fire") or d:IsA("Smoke") or d:IsA("PointLight")
4238             or d.Name:match("^Chair") or d.Name:match("^Desk") or d.Name == "Monitor" then
4239                 d:Destroy()
4240             end
4241         end
4242         for _, d in ipairs(corpse:GetDescendants()) do
4243             if d:IsA("BasePart") then
4244                 d.Color = Color3.new(d.Color.R * 0.55 + 0.10, d.Color.G * 0.55 + 0.10, d.Color.B * 0.55 + 0.10)
4245                 d.Material = Enum.Material.SmoothPlastic
4246                 d.Anchored = true
4247                 d.CanCollide = false
4248                 d.CanQuery = false -- a body must not soak up later shots
4249                 d.CastShadow = false -- 180 shadow-casting corpses is not free
4250             end
4251         end
4252         local corpseFolder = workspace:FindFirstChild("LabBotBodies")
4253         if not corpseFolder then
4254             corpseFolder = Instance.new("Folder")
4255             corpseFolder.Name = "LabBotBodies"
4256             corpseFolder.Parent = workspace
4257         end
4258         -- The clone inherits every tag, including LabWorkerBot. Left alone it
4259         -- would be counted as a live worker by every GetTagged sweep and the
4260         -- kill total would never be able to reach the target.
4261         CollectionService.RemoveTag(corpse, "LabWorkerBot")
4262         corpse.Parent = corpseFolder
4263         -- Lay it down where it fell, with a random spin so a cleared room does
4264         -- not read as a row of identical dropped props.
4265         corpse:PivotTo(CFrame.new(pivot.Position - Vector3.new(0, 2.6, 0))
4266             * CFrame.Angles(0, math.rad(math.random(0, 359)), 0)
4267             * CFrame.Angles(math.rad(-90), 0, 0))
4268         CollectionService.AddTag(corpse, "LabBotCorpse")

```

```

4268     -- Bodies are the single biggest avoidable part cost in a long round:
4269     -- 180 of them is ~1,600 extra parts. They stay long enough to read a
4270     -- room as cleared, then go.
4271     Debris:AddItem(corpse, BODY_LIFETIME_SECONDS)
4272
4273     -- Park the live worker out of the world until the next shift.
4274     if not bot:GetAttribute("HomeCFrame") then bot:SetAttribute("HomeCFrame", pivot) end
4275     bot:PivotTo(CFrame.new(0, -1000, 0))
4276
4277     local complex = workspace:FindFirstChild("Laboratory_Complex")
4278     if complex then
4279         complex:SetAttribute("BotsKilled", (complex:GetAttribute("BotsKilled") or 0) + 1)
4280     end
4281 end
4282 return true
4283 end
4284
4285 -- Walk up from whatever the ray actually hit to the tagged equipment part.
4286 local function equipmentFromHit(hit)
4287     local node = hit
4288     while node and node ~= workspace do
4289         if node:IsA("BasePart")
4290             and (CollectionService.HasTag(node, "LabEquipment") or CollectionService.HasTag(node,
4291                 ↪ "PlantGrowthChamber")) then
4292             return node
4293         end
4294         node = node.Parent
4295     end
4296     return nil
4297 end
4298
4299 -- Shoot a machine and it goes up, taking out whatever is standing near it --
4300 -- including the player who pulled the trigger, if they did it from close range.
4301 local function explodeEquipment(item, attacker)
4302     if item:GetAttribute("Exploded") then return false end
4303     item:SetAttribute("Exploded", true)
4304     item:SetAttribute("FunctionalState", "DESTROYED")
4305     item:SetAttribute("RepairInstruction", "Destroyed beyond repair")
4306
4307     local position = item.Position
4308     local blast = Instance.new("Explosion")
4309     blast.Position = position
4310     blast.BlastRadius = 17
4311     blast.BlastPressure = 0 -- the lab is anchored; pressure would do nothing
4312     blast.DestroyJointRadiusPercent = 0
4313     blast.ExplosionType = Enum.ExplosionType.NoCraters
4314     blast.Parent = workspace
4315
4316     local fire = Instance.new("Fire")
4317     fire.Heat = 14
4318     fire.Size = 9
4319     fire.Parent = item
4320     Debris:AddItem(fire, 14)
4321     local smoke = Instance.new("Smoke")
4322     smoke.Size = 5
4323     smoke.Opacity = 0.4
4324     smoke.RiseVelocity = 9
4325     smoke.Color = Color3.fromRGB(60, 58, 56)
4326     smoke.Parent = item
4327     Debris:AddItem(smoke, 18)
4328
4329     item.Color = Color3.fromRGB(48, 40, 36)
4330     item.Material = Enum.Material.CorrodedMetal
4331
4332     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
4333         if not bot:GetAttribute("Dead") and bot.PrimaryPart then
4334             if (bot.PrimaryPart.Position - position).Magnitude < 17 then
4335                 damageWorkerBot(bot, 1000, attacker)
4336             end
4337         end
4338     end
4339
4340     for _, other in ipairs(Players:GetPlayers()) do
4341         local c = other.Character
4342         local h = c and c:FindFirstChildOfClass("Humanoid")
4343         local hrp = c and c:FindFirstChild("HumanoidRootPart")
4344         if h and hrp and h.Health > 0 then

```

```

4343         local d = (hrp.Position - position).Magnitude
4344         if d < 17 then h:TakeDamage(math.max(18, 75 * (1 - d / 17))) end
4345     end
4346 end
4347 return true
4348 end
4349
4350 -- A gas line going up is a different order of event from a machine failing:
4351 -- it takes out a chunk of the room -- a couple of neighbouring machines and
4352 -- part of the wall structure itself.
4353 local function detonateGasPipe(pipe, attacker, atPosition)
4354     if pipe:GetAttribute("Blown") then return false end
4355     pipe:SetAttribute("Blown", true)
4356     -- Blow at the point that was actually hit. A corridor run is one long part
4357     -- whose Position is its midpoint, so using that detonated the pipe up to a
4358     -- hundred studs away from where the player shot it.
4359     local position = atPosition or pipe.Position
4360
4361     local blast = Instance.new("Explosion")
4362     blast.Position = position
4363     blast.BlastRadius = 30
4364     blast.BlastPressure = 0
4365     blast.DestroyJointRadiusPercent = 0
4366     blast.ExplosionType = Enum.ExplosionType.NoCraters
4367     blast.Parent = workspace
4368
4369     -- a running wall of fire and smoke along the rupture
4370     for i = -2, 2 do
4371         local flare = Instance.new("Part")
4372         flare.Name = "GasFire"
4373         flare.Size = Vector3.new(1, 1, 1)
4374         flare.CFrame = CFrame.new(position + Vector3.new(0, 0, i * 7))
4375         flare.Anchored = true
4376         flare.CanCollide = false
4377         flare.CanQuery = false
4378         flare.CanTouch = false
4379         flare.Transparency = 1
4380         flare.Parent = workspace
4381         local fire = Instance.new("Fire")
4382         fire.Heat = 25
4383         fire.Size = 24
4384         fire.Parent = flare
4385         local smoke = Instance.new("Smoke")
4386         smoke.Size = 15
4387         smoke.Opacity = 0.5
4388         smoke.RiseVelocity = 14
4389         smoke.Color = Color3.fromRGB(40, 38, 36)
4390         smoke.Parent = flare
4391         task.delay(16, function()
4392             if flare.Parent then fire.Enabled = false; smoke.Enabled = false end
4393         end)
4394         Debris:AddItem(flare, 22)
4395     end
4396
4397     pipe.Color = Color3.fromRGB(62, 52, 30)
4398     pipe.Material = Enum.Material.CorrodedMetal
4399
4400     -- take two or three neighbouring machines with it, staggered so it reads
4401     -- as a chain rather than one simultaneous bang
4402     local nearby = {}
4403     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 24)) do
4404         if (CollectionService:HasTag(part, "LabEquipment") or CollectionService:HasTag(part,
4405             ↪ "PlantGrowthChamber"))
4406             and not part:GetAttribute("Exploded") then
4407             table.insert(nearby, part)
4408         end
4409     end
4410     for i = 1, math.min(3, #nearby) do
4411         local item = nearby[i]
4412         task.delay(0.18 * i, function()
4413             if item.Parent then explodeEquipment(item, attacker) end
4414         end)
4415     end
4416
4417     -- and blow an actual hole in the structure
4418     local blown = 0

```



```

4418     for _, part in ipairs(workspace:GetPartBoundsInRadius(position, 15)) do
4419         if part:IsA("BasePart") and part.CanCollide and not part:GetAttribute("GasPipe") then
4420             local n = part.Name
4421             if n:match("^Corridor_") or n:match("^Div_") or n:match("^Ext_") or n:match("^B_") then
4422                 if blown < 5 then
4423                     blown = blown + 1
4424                     part:Destroy()
4425                 else
4426                     part.Color = Color3.fromRGB(60, 54, 50)
4427                     part.Material = Enum.Material.CorrodedMetal
4428                 end
4429             end
4430         end
4431     end
4432
4433     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
4434         if not bot:GetAttribute("Dead") and bot.PrimaryPart
4435             and (bot.PrimaryPart.Position - position).Magnitude < 30 then
4436             damageWorkerBot(bot, 1000, attacker)
4437         end
4438     end
4439     for _, other in ipairs(Players:GetPlayers()) do
4440         local c = other.Character
4441         local h = c and c:FindFirstChildOfClass("Humanoid")
4442         local hrp = c and c:FindFirstChild("HumanoidRootPart")
4443         if h and hrp and h.Health > 0 then
4444             local d = (hrp.Position - position).Magnitude
4445             if d < 30 then h:TakeDamage(math.max(30, 140 * (1 - d / 30))) end
4446         end
4447     end
4448     return true
4449 end
4450
4451 -- Scorch mark, smoke puff and sparks wherever a round lands.
4452 local function spawnImpact(position, normal)
4453     local hole = Instance.new("Part")
4454     hole.Name = "BulletHole"
4455     hole.Size = Vector3.new(0.3, 0.3, 0.06)
4456     hole.CFrame = CFrame.lookAt(position + normal * 0.03, position + normal * 2)
4457     hole.Color = Color3.fromRGB(24, 22, 21)
4458     hole.Material = Enum.Material.Slate
4459     hole.Anchored = true
4460     hole.CanCollide = false
4461     hole.CanQuery = false
4462     hole.CanTouch = false
4463     hole.CastShadow = false
4464     hole.Parent = workspace
4465     Debris:AddItem(hole, 45)
4466
4467     local puff = Instance.new("Part")
4468     puff.Name = "ImpactPuff"
4469     puff.Size = Vector3.new(0.1, 0.1, 0.1)
4470     puff.CFrame = CFrame.new(position + normal * 0.25)
4471     puff.Anchored = true
4472     puff.CanCollide = false
4473     puff.CanQuery = false
4474     puff.CanTouch = false
4475     puff.Transparency = 1
4476     puff.Parent = workspace
4477     local smoke = Instance.new("Smoke")
4478     smoke.Size = 0.7
4479     smoke.RiseVelocity = 5
4480     smoke.Opacity = 0.3
4481     smoke.Color = Color3.fromRGB(125, 125, 125)
4482     smoke.Parent = puff
4483     local sparks = Instance.new("Sparkles")
4484     sparks.SparkleColor = Color3.fromRGB(255, 208, 128)
4485     sparks.Parent = puff
4486     task.delay(0.3, function()
4487         if puff.Parent then smoke.Enabled = false; sparks.Enabled = false end
4488     end)
4489     Debris:AddItem(puff, 3)
4490 end
4491
4492 -- Posters keep their holes for good rather than fading like scorch marks.
4493 local function punchPoster(poster, position, normal)

```

```

4494     local holes = poster:GetAttribute("Holes") or 0
4495     if holes >= 30 then return end
4496     poster:SetAttribute("Holes", holes + 1)
4497     local tear = Instance.new("Part")
4498     tear.Name = "PosterTear"
4499     tear.Size = Vector3.new(0.36, 0.36, 0.03)
4500     tear.CFrame = CFrame.lookAt(position + normal * 0.012, position + normal * 2)
4501     tear.Color = Color3.fromRGB(198, 190, 174)
4502     tear.Material = Enum.Material.SmoothPlastic
4503     tear.Anchored = true
4504     tear.CanCollide = false
4505     tear.CanQuery = false
4506     tear.CanTouch = false
4507     tear.CastShadow = false
4508     tear.Parent = poster
4509     local hole = tear:Clone()
4510     hole.Name = "PosterHole"
4511     hole.Size = Vector3.new(0.22, 0.22, 0.05)
4512     hole.Color = Color3.fromRGB(16, 15, 14)
4513     hole.CFrame = CFrame.lookAt(position + normal * 0.025, position + normal * 2)
4514     hole.Parent = poster
4515 end
4516
4517 local function workerBotFromHit(hit)
4518     local model = hit and hit:FindFirstAncestorOfClass("Model")
4519     return model and CollectionService.HasTag(model, "LabWorkerBot") and model or nil
4520 end
4521
4522 local function stealHeldSample(attacker, victim)
4523     local sample = findHeldSample(victim)
4524     if not sample then
4525         notify(attacker, victim.DisplayName .. " is not holding a sample.")
4526         return false
4527     end
4528     if sample:GetAttribute("TransferSample") and findTransferSample(attacker) then
4529         notify(attacker, "Your hands are already assigned to another cold-store sample.")
4530         return false
4531     end
4532
4533     local backpack = attacker:FindFirstChildOfClass("Backpack")
4534     if not backpack then return false end
4535     local id = sample:GetAttribute("SampleId") or sample.Name
4536     sample.Parent = backpack
4537     setCarryState(victim, nil)
4538     setCarryState(attacker, sample)
4539     addContribution(attacker)
4540     notify(attacker, "Recovered " .. id .. " from " .. victim.DisplayName .. ". It is now in your hand.")
4541     notify(victim, attacker.DisplayName .. " took " .. id .. " from your hand. Collect or recover another
4542     ↪ sample.")
4543     equipTool(attacker, sample)
4544     return true
4545 end
4546
4547 local function armImpactTool(tool)
4548     if tool:GetAttribute("HitLogicBound") then return end
4549     tool:SetAttribute("HitLogicBound", true)
4550     local handle = tool:FindFirstChild("Handle")
4551     if not handle then return end
4552     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind") or tool.Name]
4553     if not style then return end
4554
4555     local activeUntil = 0
4556     local nextSwing = 0
4557     local hitThisSwing = {}
4558
4559     tool.Activated:Connect(function()
4560         -- Every gun is driven by BlasterController on the owning client.
4561         -- Keep Tool.Activated for melee and throwable tools only, avoiding
4562         -- duplicate shots from Roblox's default Tool activation path.
4563         if style.mode == "gun" then return end
4564         local owner = Players:GetPlayerFromCharacter(tool.Parent)
4565         if not owner or os.clock() < nextSwing then return end
4566         nextSwing = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
4567         hitThisSwing = {}
4568         local original = handle.Color
4569         handle.Color = Color3.fromRGB(255, 245, 190)

```

```

4569     task.delay(0.18, function()
4570         if handle.Parent then handle.Color = original end
4571     end)
4572
4573     local rootPart = owner.Character and owner.Character:FindFirstChild("HumanoidRootPart")
4574     if not rootPart then return end
4575     if style.mode == "gun" then
4576         local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
4577         local direction = rootPart.CFrame.LookVector * (style.range or 100)
4578         local params = RaycastParams.new()
4579         params.FilterType = Enum.RaycastFilterType.Exclude
4580         params.FilterDescendantsInstances = {owner.Character}
4581         local result = workspace.Raycast(origin, direction, params)
4582         local hitPosition = result and result.Position or (origin + direction)
4583         local beam = Instance.new("Part")
4584         beam.Name = "FoamTracer"
4585         beam.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
4586         beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
4587         beam.Color = style.colour
4588         beam.Material = Enum.Material.Neon
4589         beam.Anchored = true
4590         beam.CanCollide = false
4591         beam.CanTouch = false
4592         beam.Parent = workspace
4593         Debris:AddItem(beam, 0.12)
4594         local bot = result and workerBotFromHit(result.Instance)
4595         if bot then damageWorkerBot(bot, style.damage or 25, owner) end
4596         return
4597     elseif style.mode == "ball" then
4598         local ball = Instance.new("Part")
4599         ball.Name = "ThrownBasketball"
4600         ball.Shape = Enum.PartType.Ball
4601         ball.Size = Vector3.new(1.35, 1.35, 1.35)
4602         ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) + rootPart.CFrame.LookVector *
4603             ↪ 3)
4604         ball.Color = style.colour
4605         ball.Material = Enum.Material.Rubber
4606         ball.CanCollide = true
4607         ball.Parent = workspace
4608         ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
4609         local ballHit = false
4610         ball.Touched:Connect(function(hit)
4611             if ballHit then return end
4612             local bot = workerBotFromHit(hit)
4613             if bot then ballHit = true; damageWorkerBot(bot, style.damage or 20, owner) end
4614         end)
4615         Debris:AddItem(ball, 7)
4616         return
4617     end
4618     activeUntil = os.clock() + 0.45
4619 end)
4620
4621 handle.Touched:Connect(function(hit)
4622     if os.clock() > activeUntil then return end
4623     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
4624     if not attacker then return end
4625     local targetCharacter = hit:FindFirstAncestorOfClass("Model")
4626     local workerBot = workerBotFromHit(hit)
4627     if workerBot then
4628         if hitThisSwing[workerBot] then return end
4629         hitThisSwing[workerBot] = true
4630         damageWorkerBot(workerBot, style.damage or 15, attacker)
4631         return
4632     end
4633     local victim = targetCharacter and Players:GetPlayerFromCharacter(targetCharacter)
4634     if not victim or victim == attacker or hitThisSwing[victim] then return end
4635
4636     local attackerRoot = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
4637     local victimRoot = victim.Character and victim.Character:FindFirstChild("HumanoidRootPart")
4638     local victimHumanoid = victim.Character and victim.Character:FindFirstChildOfClass("Humanoid")
4639     if not attackerRoot or not victimRoot or not victimHumanoid or victimHumanoid.Health <= 0 then return
4640     ↪ end
4641     if (attackerRoot.Position - victimRoot.Position).Magnitude > 9 then return end
4642     hitThisSwing[victim] = true
4643
4644     local delta = victimRoot.Position - attackerRoot.Position

```

```

4643     if delta.Magnitude > 0.1 then
4644         local direction = delta.Unit
4645         victimRoot.AssemblyLinearVelocity = victimRoot.AssemblyLinearVelocity
4646             + Vector3.new(direction.X * 9, 3, direction.Z * 9)
4647     end
4648
4649     local flash = Instance.new("Highlight")
4650     flash.Name = "SampleRecoveryHit"
4651     flash.FillColor = Color3.fromRGB(255, 210, 80)
4652     flash.OutlineColor = Color3.fromRGB(255, 255, 255)
4653     flash.FillTransparency = 0.35
4654     flash.Parent = victim.Character
4655     Debris:AddItem(flash, 0.25)
4656     stealHeldSample(attacker, victim)
4657 end)
4658 end
4659
4660 local function findImpactTool(player, kind)
4661     for _, container in ipairs(playerContainers(player)) do
4662         if container then
4663             for _, child in ipairs(container:GetChildren()) do
4664                 if child:IsA("Tool") and child:GetAttribute("ImpactToolKind") == kind then return child end
4665             end
4666         end
4667     end
4668     return nil
4669 end
4670
4671 local function makeImpactTool(player, kind)
4672     local existing = findImpactTool(player, kind)
4673     if existing then
4674         equipTool(player, existing)
4675         notify(player, kind .. " equipped. Click or tap to use it against worker NPCs or recover a held sample
4676             ↪ from another player.")
4677         return existing
4678     end
4679
4680     local style = IMPACT_STYLE[kind]
4681     if not style then return nil end
4682     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 5)
4683     if not backpack then return nil end
4684
4685     local tool = Instance.new("Tool")
4686     tool.Name = kind
4687     tool.ToolTip = style.mode == "gun" and "Fire at the screen crosshair; hold right-click or tap AIM for ADS"
4688         or (style.mode == "ball" and "Throw a basketball at worker NPCs" or "Hit players to recover samples;
4689             ↪ hit NPCs to lower life")
4690     tool.RequiresHandle = true
4691     tool.CanBeDropped = false
4692     tool:SetAttribute("ImpactToolKind", kind)
4693     tool:SetAttribute("LabImpactTool", true)
4694     if style.mode == "gun" then
4695         tool:SetAttribute("BlasterWeapon", true)
4696         tool:SetAttribute("BlasterDamage", style.damage or 25)
4697         tool:SetAttribute("BlasterRange", style.range or 200)
4698         tool:SetAttribute("BlasterCooldown", style.cooldown or 0.55)
4699         tool:SetAttribute("BlasterPellets", style.pellets or 1)
4700         tool:SetAttribute("BlasterSpread", style.spread or 0)
4701         tool:SetAttribute("BlasterSelfCostPercent", style.selfCostPercent or 0)
4702         tool:SetAttribute("BlasterInstantHeadshot", style.pcOnly == true)
4703         tool:SetAttribute("BlasterAutomatic", style.automatic == true)
4704         tool:SetAttribute("BlasterADSFov", (style.range or 0) >= 500 and 44 or ((style.pellets or 1) > 1 and 58
4705             ↪ or 54))
4706         tool:SetAttribute("BlasterADSTouchSensitivity", 0.42)
4707         tool:SetAttribute("BlasterTracerColor", style.colour)
4708         -- Legacy marker retained for old saved tools; the legacy remote is inert.
4709         if style.pcOnly then tool:SetAttribute("LabSidearm", true) end
4710     end
4711
4712     local handle = Instance.new("Part")
4713     handle.Name = "Handle"
4714     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
4715         or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35) or Vector3.new(0.42, 3.4, 0.42))
4716     handle.Color = style.colour
4717     handle.Material = Enum.Material.SmoothPlastic
4718     handle.Anchored = false

```

```

4716     handle.CanCollide = false
4717     handle.Massless = true
4718     handle.Parent = tool
4719     if style.mode == "ball" then handle.Shape = Enum.PartType.Ball; handle.Material = Enum.Material.Rubber end
4720
4721     if style.meshId then
4722         -- Imported gun shape. A SpecialMesh is used rather than a MeshPart
4723         -- because MeshPart.MeshId cannot be assigned at runtime -- SpecialMesh
4724         -- can, so the tool builds correctly on a live server as well as in
4725         -- Studio, with no dependency on anything sitting in ServerStorage.
4726         handle.Size = style.handleSize or Vector3.new(0.20, 0.92, 1.36)
4727         local mesh = Instance.new("SpecialMesh")
4728         mesh.MeshType = Enum.MeshType.FileMesh
4729         mesh.MeshId = style.meshId
4730         if style.meshTexture then mesh.TextureId = style.meshTexture end
4731         local scale = style.meshScale or 1.6
4732         mesh.Scale = Vector3.new(scale, scale, scale)
4733         mesh.Parent = handle
4734     elseif style.mode == "gun" then
4735         addWeldedPart(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7),
4736             CFrame.new(0, 0.1, -1.65), style.colour, Enum.Material.Neon)
4737         addWeldedPart(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55),
4738             CFrame.new(0, -0.75, 0.42) * CFrame.Angles(math.rad(-18), 0, 0), style.accent,
4739             Enum.Material.Rubber)
4740     elseif kind == "Inspection Paddle" then
4741         addWeldedPart(tool, handle, "Paddle", Vector3.new(1.45, 1.65, 0.28),
4742             CFrame.new(0, 2.15, 0), style.colour, Enum.Material.SmoothPlastic)
4743     elseif kind == "Sample Grabber" then
4744         addWeldedPart(tool, handle, "LeftClaw", Vector3.new(0.25, 1.1, 0.3),
4745             CFrame.new(-0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(-28)), style.accent, Enum.Material.Metal)
4746         addWeldedPart(tool, handle, "RightClaw", Vector3.new(0.25, 1.1, 0.3),
4747             CFrame.new(0.38, 1.95, 0) * CFrame.Angles(0, 0, math.rad(28)), style.accent, Enum.Material.Metal)
4748     elseif style.mode == "melee" then
4749         addWeldedPart(tool, handle, "FoamTip", Vector3.new(0.75, 1.5, 0.75),
4750             CFrame.new(0, 2, 0), style.colour, Enum.Material.Fabric)
4751     end
4752     if style.mode == "melee" then
4753         addWeldedPart(tool, handle, "Grip", Vector3.new(0.58, 1.15, 0.58),
4754             CFrame.new(0, -1.1, 0), style.accent, Enum.Material.Rubber)
4755     end
4756
4757     tool.Grip = style.grip or (style.mode == "gun" and CFrame.new(0, -0.2, 0.6))
4758     or (style.mode == "ball" and CFrame.new(0, -0.25, -0.4) or CFrame.new(0, -1.2, 0) *
4759         CFrame.Angles(math.rad(-8), 0, 0))
4760     tool.Parent = backpack
4761     armImpactTool(tool)
4762     equipTool(player, tool)
4763     notify(player, kind .. " equipped. Click or tap to use it against worker NPCs; their LIFE bar shows damage
4764         ↳ and recovery.")
4765     return tool
4766 end
4767
4768 local function bindDispenser(part)
4769     if part:GetAttribute("DispenserBound") then return end
4770     local kind = part:GetAttribute("ToolKind")
4771     if not kind or not IMPACT_STYLE[kind] then return end
4772     part:SetAttribute("DispenserBound", true)
4773
4774     local prompt = Instance.new("ProximityPrompt")
4775     prompt.ObjectText = kind
4776     prompt.ActionText = "Equip combat tool"
4777     prompt.KeyboardKeyCode = Enum.KeyCode.E
4778     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
4779     prompt.HoldDuration = 0.25
4780     prompt.MaxActivationDistance = 10
4781     prompt.RequiresLineOfSight = false
4782     prompt.Parent = part
4783     local lastUse = {}
4784     prompt.Triggered:Connect(function(player)
4785         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
4786         if os.clock() < (lastUse[player] or 0) then return end
4787         lastUse[player] = os.clock() + 0.75
4788         makeImpactTool(player, kind)
4789     end)
4790 end
4791 end
4792

```

```

4789 -----
4790 -- PC SIDEARM -- mouse-aimed, costs the shooter 1% of their life per shot
4791 --
4792 -- Fired only through this remote, which only the desktop client script
4793 -- sends, so the weapon is inert on touch and console clients.
4794 -----
4795 local sidearmRemote = ReplicatedStorage:FindFirstChild("LabSidearmFire")
4796 if not sidearmRemote then
4797     sidearmRemote = Instance.new("RemoteEvent")
4798     sidearmRemote.Name = "LabSidearmFire"
4799     sidearmRemote.Parent = ReplicatedStorage
4800 end
4801
4802 local sidearmCooldown = {}
4803 Players.PlayerRemoving:Connect(function(player) sidearmCooldown[player] = nil end)
4804
4805 sidearmRemote.OnServerEvent:Connect(function(player, aimPoint)
4806     -- Superseded by ServerBlasterManager. Keep the legacy remote inert so
4807     -- old clients and exploit traffic cannot enter the retired mouse path.
4808     if true then return end
4809     if typeof(aimPoint) ~= "Vector3" then return end
4810     local character = player.Character
4811     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
4812     local root = character and character:FindFirstChild("HumanoidRootPart")
4813     if not humanoid or not root or humanoid.Health <= 0 then return end
4814
4815     -- The client only ever asks; the server decides whether the player is
4816     -- really holding a weapon, and what that weapon is allowed to do.
4817     local tool = character:FindFirstChildOfClass("Tool")
4818     if not tool or not tool:GetAttribute("LabSidearm") then return end
4819     local style = IMPACT_STYLE[tool:GetAttribute("ImpactToolKind")] or ""
4820     if not style then return end
4821
4822     local now = os.clock()
4823     if now < (sidearmCooldown[player] or 0) then return end
4824     sidearmCooldown[player] = now + (style.cooldown or 0.35)
4825
4826     local handle = tool:FindFirstChild("Handle")
4827     local origin = (handle and handle.Position or root.Position) + Vector3.new(0, 0.5, 0)
4828     local baseDelta = aimPoint - origin
4829     if baseDelta.Magnitude < 0.1 then return end
4830     local baseDir = baseDelta.Unit
4831     local range = style.range or 200
4832
4833     local params = RaycastParams.new()
4834     params.FilterType = Enum.RaycastFilterType.Exclude
4835     params.FilterDescendantsInstances = {character}
4836
4837     local killsThisShot = 0
4838     for _ = 1, (style.pellets or 1) do
4839         -- Shotgun pellets scatter around the aim line; everything else is exact.
4840         local dir = baseDir
4841         local spread = style.spread or 0
4842         if spread > 0 then
4843             dir = (baseDir + Vector3.new(
4844                 (math.random() - 0.5) * spread * 2,
4845                 (math.random() - 0.5) * spread * 2,
4846                 (math.random() - 0.5) * spread * 2)).Unit
4847         end
4848
4849         local result = workspace.Raycast(origin, dir * range, params)
4850         local hitPosition = result and result.Position or (origin + dir * range)
4851
4852         local beam = Instance.new("Part")
4853         beam.Name = "SidearmTracer"
4854         beam.Size = Vector3.new(0.07, 0.07, (hitPosition - origin).Magnitude)
4855         beam.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
4856         beam.Color = Color3.fromRGB(255, 226, 140)
4857         beam.Material = Enum.Material.Neon
4858         beam.Anchored = true
4859         beam.CanCollide = false
4860         beam.CanTouch = false
4861         beam.CanQuery = false
4862         beam.CastShadow = false
4863         beam.Parent = workspace
4864         Debris:AddItem(beam, 0.1)

```

```

4865
4866     if result then
4867         local instance, normal = result.Instance, result.Normal
4868         local bot = workerBotFromHit(instance)
4869         if bot and not bot:GetAttribute("Dead") then
4870             -- A head shot kills outright, whatever the weapon.
4871             local headShot = instance.Name == "Head"
4872             damageWorkerBot(bot, headShot and 1000 or (style.damage or 25), player)
4873             if bot:GetAttribute("Dead") then killsThisShot = killsThisShot + 1 end
4874         elseif instance:GetAttribute("GasPipe") then
4875             detonateGasPipe(instance, player, hitPosition)
4876             spawnImpact(hitPosition, normal)
4877         else
4878             local equipment = equipmentFromHit(instance)
4879             if equipment then
4880                 explodeEquipment(equipment, player)
4881             elseif instance:GetAttribute("ShootablePoster") then
4882                 punchPoster(instance, hitPosition, normal)
4883             end
4884             spawnImpact(hitPosition, normal)
4885         end
4886     end
4887 end
4888
4889 -- Firing always costs the shooter 1% of their own maximum health,
4890 -- whether or not the shot connects with anything.
4891 humanoid:TakeDamage(humanoid.MaxHealth * (style.selfCostPercent or 1) / 100)
4892
4893 if killsThisShot > 0 then
4894     local complex = workspace:FindFirstChild("Laboratory_Complex")
4895     notify(player, string.format("Target down. %d / %d cleared.",
4896     complex and complex:GetAttribute("BotsKilled") or 0,
4897     complex and complex:GetAttribute("BotsTotal") or 0))
4898 end
4899 end)
4900
4901 -----
4902 -- BLASTER GAMEPLAY BRIDGE
4903 -- ServerBlasterManager owns validation and raycasts. This callback only
4904 -- translates a validated impact into this game's custom worker/environment
4905 -- systems, keeping those systems private to the server.
4906 -----
4907 local blasterBridge = game:GetService("ServerScriptService"):FindFirstChild("BlasterHitBridge")
4908 if not blasterBridge then
4909     blasterBridge = Instance.new("BindableFunction")
4910     blasterBridge.Name = "BlasterHitBridge"
4911     blasterBridge.Parent = game:GetService("ServerScriptService")
4912 end
4913
4914 blasterBridge.OnInvoke = function(player, instance, hitPosition, hitNormal, damage, instantHeadshot)
4915     if typeof(player) ~= "Instance" or not player:IsA("Player") then return false end
4916     if typeof(instance) ~= "Instance" or not instance:IsDescendantOf(workspace) then return false end
4917
4918     -- This receives an already-validated bullet impact; aiming/firing is unchanged.
4919     if instance:GetAttribute("BreakableWindow") then
4920         if instance:GetAttribute("WindowBroken") then return false end
4921         instance:SetAttribute("WindowBroken", true)
4922         instance.Transparency = 1
4923         instance.CanCollide, instance.CanTouch, instance.CanQuery = false, false, false
4924         spawnImpact(hitPosition, hitNormal)
4925         return true
4926     end
4927
4928     local bot = workerBotFromHit(instance)
4929     if bot then
4930         local wasDead = bot:GetAttribute("Dead") == true
4931         local handled = damageWorkerBot(bot, instantHeadshot and 1000 or damage, player)
4932         if handled and not wasDead and bot:GetAttribute("Dead") then
4933             local complex = workspace:FindFirstChild("Laboratory_Complex")
4934             notify(player, string.format("Target down. %d / %d cleared.",
4935             complex and complex:GetAttribute("BotsKilled") or 0,
4936             complex and complex:GetAttribute("BotsTotal") or 0))
4937         end
4938         return handled
4939     end
4940

```

```

4941     if instance:GetAttribute("GasPipe") then
4942         detonateGasPipe(instance, player, hitPosition)
4943         spawnImpact(hitPosition, hitNormal)
4944         return true
4945     end
4946
4947     local equipment = equipmentFromHit(instance)
4948     if equipment then
4949         explodeEquipment(equipment, player)
4950         spawnImpact(hitPosition, hitNormal)
4951         return true
4952     end
4953
4954     if instance:GetAttribute("ShootablePoster") then
4955         punchPoster(instance, hitPosition, hitNormal)
4956         spawnImpact(hitPosition, hitNormal)
4957         return true
4958     end
4959
4960     return false
4961 end
4962
4963 -----
4964 -- COLD-STORE SAMPLE TRANSFERS
4965 -----
4966 local COLD_STORE = {Refrigerator = true, ["Chest Freezer"] = true}
4967 local nextSampleNumber = 1000
4968
4969 local PLANT_CHAMBER_KIND = "Plant Growth Chamber"
4970
4971 local function equipmentFloorLabel(part)
4972     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
4973 end
4974
4975 local function equipmentId(part)
4976     local code = part:GetAttribute("GrowthCode")
4977     if code and code ~= "" then return "CHAMBER:" .. code end
4978     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
4979         ↵ " | ")
4979 end
4980
4981 local function equipmentLabel(part)
4982     local code = part:GetAttribute("GrowthCode")
4983     if code and code ~= "" then return "CHAMBER " .. code end
4984     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
4985 end
4986
4987 local function adaptiveTransferSeconds(source, destination)
4988     local distance = destination and (destination.Position - source.Position).Magnitude or 60
4989     local playerCount = math.max(1, #Players:GetPlayers())
4990     local complex = workspace:FindFirstChild("Laboratory_Complex")
4991     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
4992     return math.clamp(math.floor(180 + distance * 1.2 + math.max(0, 3 - playerCount) * 15
4993         + math.clamp(backlog / 20, 0, 30)), 180, 420)
4994 end
4995 local PLANT_PROCESS = {
4996     ["Germination Chamber"] = {
4997         key = "Germination", gain = 1,
4998         caption = "Warming seed and logging germination response",
4999     },
5000     ["Incubator"] = {
5001         key = "Incubation", gain = 1,
5002         caption = "Holding controlled temperature and humidity",
5003     },
5004     ["Growth Hormone Mixer"] = {
5005         key = "Growth Hormone", gain = 2,
5006         caption = "Dosing auxin/cytokinin growth treatment",
5007     },
5008     ["GMO Injector"] = {
5009         key = "GMO Injection", gain = 1, genome = true,
5010         caption = "Injecting and verifying a new plant trait",
5011     },
5012     ["Nutrient Infuser"] = {
5013         key = "Nutrient Infusion", gain = 1, moistureGain = 18,
5014         caption = "Balancing nitrogen, phosphorus, and potassium",
5015     },

```



```

5016     ["UV Growth Scanner"] = {
5017         repeatable = true, gain = 0,
5018         caption = "Scanning chlorophyll and canopy development",
5019     },
5020     ["Pollination Station"] = {
5021         key = "Pollination", gain = 1, minStage = 3,
5022         caption = "Applying controlled pollen and checking viability",
5023     },
5024     -- Micropropagation chain, step 1: surface-sterilize the seed coat before
5025     -- it ever reaches a growth chamber. Nothing downstream requires this, but
5026     -- a Sterilized sample gets a real (and referenced) germination boost --
5027     -- contamination is the #1 real-world cause of failed plant tissue culture.
5028     ["Decontamination Unit"] = {
5029         key = "Sterilized", gain = 0,
5030         caption = "Surface-sterilizing seed coat (dilute bleach + ethanol rinse) and sealing on sterile media",
5031     },
5032     -- Dose-response mutagenesis modeled on the reference Viola cornuta gamma
5033     -- study: germination viability falls off steeply above ~400 Gy, and by
5034     -- 1000 Gy almost nothing survives. Dose is chosen per treatment so two
5035     -- visits to the same chamber can give a different, honestly-labeled result.
5036     ["Mutagenesis Chamber"] = {
5037         key = "Mutagenesis", gain = 0, mutagenesis = true,
5038         caption = "Irradiating seed with a measured gamma dose",
5039     },
5040     -- Hormone Treatment Bench: auxin pushes rooting, cytokinin pushes
5041     -- shoot/bud growth, and a balanced ratio pushes both -- the same
5042     -- direction of effect reported for lemon seed germination trials.
5043     ["Hormone Treatment Bench"] = {
5044         key = "Hormone Treatment", gain = 1, hormone = true,
5045         caption = "Dosing a measured auxin/cytokinin treatment",
5046     },
5047 }
5048
5049 local function storageId(part)
5050     local id = part:GetAttribute("StorageId")
5051     if not id then
5052         id = table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
5053             ↪ part.Name}, "|")
5054         part:SetAttribute("StorageId", id)
5055     end
5056     return id
5057 end
5058
5059 local function storageLabel(part)
5060     local unit = part.Name:match("_(%d+)$") or "1"
5061     return string.format("%s, %s, %s (unit %s)", part:GetAttribute("Kind") or "Cold store",
5062         part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "room", unit)
5063 end
5064
5065 local function coldStoresExcept(source)
5066     local stores = {}
5067     for _, candidate in ipairs(CollectionService:GetTagged("LabEquipment")) do
5068         if candidate ~= source and COLD_STORE[candidate:GetAttribute("Kind")] then
5069             table.insert(stores, candidate)
5070         end
5071     end
5072     table.sort(stores, function(a, b) return storageId(a) < storageId(b) end)
5073     return stores
5074 end
5075
5076 -----
5077 -- PHYSICAL CONTROL BUTTON AND DATA DISPLAY
5078 -----
5079 local function makeControls(part, kind, spec)
5080     local wallSide = part:GetAttribute("WallSide")
5081     local frontAxisX = wallSide == "W" or wallSide == "E"
5082     local frontDirection = ((wallSide == "E") or (wallSide == "S")) and -1 or 1
5083     local isGrowthChamber = kind == PLANT_CHAMBER_KIND
5084     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
5085     local timerGui = Instance.new("BillboardGui")
5086     timerGui.Name = "EquipmentStatusTimer"
5087     timerGui.Size = UDim2.fromOffset(160, 32)
5088     timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
5089     timerGui.MaxDistance = 10
5090     timerGui.AlwaysOnTop = false
5091     timerGui.Parent = part

```

```

5091     local timerLabel = Instance.new("TextLabel")
5092     timerLabel.Name = "Timer"
5093     timerLabel.Size = UDim2.fromScale(1, 1)
5094     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
5095     timerLabel.BackgroundTransparency = 0.16
5096     timerLabel.BorderSizePixel = 0
5097     timerLabel.TextScaled = true
5098     timerLabel.Font = Enum.Font.GothamBold
5099     timerLabel.Parent = timerGui
5100     local timerCorner = Instance.new("UICorner")
5101     timerCorner.CornerRadius = UDim.new(0, 8)
5102     timerCorner.Parent = timerLabel
5103
5104     -- Equipment name plate, mounted directly above the RUNNING/FAILED timer
5105     -- badge so each unit's kind is obvious at a glance from close range.
5106     local nameGui = Instance.new("BillboardGui")
5107     nameGui.Name = "EquipmentNamePlate"
5108     nameGui.Size = UDim2.fromOffset(160, 24)
5109     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
5110     nameGui.MaxDistance = 10
5111     nameGui.AlwaysOnTop = false
5112     nameGui.Parent = part
5113     local nameLabel = Instance.new("TextLabel")
5114     nameLabel.Name = "Kind"
5115     nameLabel.Size = UDim2.fromScale(1, 1)
5116     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
5117     nameLabel.BackgroundTransparency = 0.05
5118     nameLabel.BorderSizePixel = 0
5119     nameLabel.Text = kind
5120     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
5121     nameLabel.TextScaled = true
5122     nameLabel.Font = Enum.Font.SourceSansBold
5123     nameLabel.Parent = nameGui
5124     local nameCorner = Instance.new("UICorner")
5125     nameCorner.CornerRadius = UDim.new(0, 8)
5126     nameCorner.Parent = nameLabel
5127
5128     local slotGui = Instance.new("BillboardGui")
5129     slotGui.Name = "EquipmentFreeSlots"
5130     slotGui.Size = UDim2.fromOffset(150, 28)
5131     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
5132     slotGui.MaxDistance = 10
5133     slotGui.AlwaysOnTop = false
5134     slotGui.Parent = part
5135     local slotLabel = Instance.new("TextLabel")
5136     slotLabel.Name = "FreeSlots"
5137     slotLabel.Size = UDim2.fromScale(1, 1)
5138     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)
5139     slotLabel.BackgroundTransparency = 0.12
5140     slotLabel.BorderSizePixel = 0
5141     slotLabel.TextScaled = true
5142     slotLabel.Font = Enum.Font.GothamBold
5143     slotLabel.Parent = slotGui
5144     local slotCorner = Instance.new("UICorner")
5145     slotCorner.CornerRadius = UDim.new(0, 8)
5146     slotCorner.Parent = slotLabel
5147     local storedLifeGui, storedLifeText, storedLifeFill
5148     if isPlantContainer then
5149         local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
5150         storedLifeGui = Instance.new("BillboardGui")
5151         storedLifeGui.Name = "StoredPlantLifeStatus"
5152         storedLifeGui.Size = UDim2.fromOffset(140, 40)
5153         storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
5154         storedLifeGui.MaxDistance = 10
5155         storedLifeGui.AlwaysOnTop = false
5156         storedLifeGui.Parent = plantAnchor
5157         storedLifeText = Instance.new("TextLabel")
5158         storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
5159         storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
5160         storedLifeText.BackgroundTransparency = 0.16
5161         storedLifeText.BorderSizePixel = 0
5162         storedLifeText.TextScaled = true
5163         storedLifeText.Font = Enum.Font.GothamBold
5164         storedLifeText.Parent = storedLifeGui
5165         local storedLifeBack = Instance.new("Frame")
5166         storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)

```

```

5167     storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
5168     storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)
5169     storedLifeBack.BorderSizePixel = 0
5170     storedLifeBack.Parent = storedLifeGui
5171     storedLifeFill = Instance.new("Frame")
5172     storedLifeFill.Size = UDim2.fromScale(1, 1)
5173     storedLifeFill.BorderSizePixel = 0
5174     storedLifeFill.Parent = storedLifeBack
5175 end
5176 local function clockText(seconds)
5177     seconds = math.max(0, math.floor(seconds or 0))
5178     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
↵ seconds % 60)
5179 end
5180 local function renderTimer()
5181     local state = part:GetAttribute("FunctionalState")
5182     local destroyed = state == "DESTROYED"
5183     local failed = destroyed or state == "FAILED"
5184     local since = failed and (part:GetAttribute("FailureSince") or os.time())
5185     or (part:GetAttribute("OperationStarted") or os.time())
5186     if destroyed then
5187         timerLabel.Text = "DESTROYED -- UNFIXABLE"
5188         timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
5189     else
5190         timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. clockText(os.time() - since)
5191         timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)
5192     end
5193     if isPlantContainer and failed then
5194         local life = math.clamp(100 - math.floor(math.max(0, os.time() - since - 120) / 6), 0, 100)
5195         part:SetAttribute("PlantLife", life)
5196         if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
5197             part:SetAttribute("SampleIds", "")
5198             part:SetAttribute("SampleCount", 0)
5199             part:SetAttribute("Occupied", false)
5200             part:SetAttribute("PlantId", "")
5201             part:SetAttribute("PlantExpiredAt", os.time())
5202             for _, child in ipairs(part:GetDescendants()) do
5203                 if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then child.Transparency = 1
↵ end
5204         end
5205     end
5206 end
5207 local capacity = part:GetAttribute("Capacity") or 0
5208 local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
5209 local free = math.max(0, capacity - count)
5210 slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
5211 or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
5212 slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)
5213 -- Beacon: someone is actively carrying a plant toward THIS chamber
5214 -- (IncomingReservations is set the moment a plant is taken with this
5215 -- chamber as its destination). Make that one chamber's slot readout
5216 -- visible from across the room -- through walls too -- so "where do I
5217 -- put this plant" is answered by looking up, not by wandering every
5218 -- aisle reading labels that are otherwise only readable up close.
5219 local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
5220 if incoming and not failed then
5221     slotGui.MaxDistance = 30
5222     slotGui.AlwaysOnTop = false
5223     slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
5224     slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
5225 else
5226     slotGui.MaxDistance = 10
5227     slotGui.AlwaysOnTop = false
5228 end
5229 if storedLifeGui then
5230     local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
5231     local colour = life > 60 and Color3.fromRGB(75, 235, 105)
5232     or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
5233     storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
5234     storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
5235     storedLifeText.TextColor3 = colour
5236     storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
5237     storedLifeFill.BackgroundColor3 = colour
5238 end
5239 end
5240 renderTimer()

```

```

5241     task.spawn(function()
5242         while part.Parent do
5243             renderTimer()
5244             task.wait(1)
5245         end
5246     end)
5247
5248     local button = Instance.new("Part")
5249     button.Name = "PushButton"
5250     button.Shape = Enum.PartType.Cylinder
5251     button.Size = isGrowthChamber and Vector3.new(0.24, 0.64, 0.64) or Vector3.new(0.2, 0.48, 0.48)
5252     local buttonSide = math.clamp((frontAxisX and part.Size.Z or part.Size.X) * 0.34, 0.25, 1.2)
5253     local buttonY = math.clamp(part.Size.Y * 0.03, -0.15, 0.45)
5254     if frontAxisX then
5255         button.CFrame = part.CFrame * CFrame.new(frontDirection * (part.Size.X / 2 + 0.16), buttonY,
5256             ↪ buttonSide)
5257     else
5258         button.CFrame = part.CFrame * CFrame.new(-buttonSide, buttonY, frontDirection * (part.Size.Z / 2 +
5259             ↪ 0.16))
5260         * CFrame.Angles(0, math.rad(90), 0)
5261     end
5262     button.Color = Color3.fromRGB(218, 68, 58)
5263     button.Material = Enum.Material.Neon
5264     button.Anchored = true
5265     button.CanCollide = false
5266     button.CanTouch = false
5267     button.CastShadow = false
5268     button.Parent = part
5269     local restCFrame = button.CFrame
5270
5271     local prompt = Instance.new("ProximityPrompt")
5272     prompt.ObjectText = isGrowthChamber
5273     and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " • CHAMBER " ..
5274         ↪ (part:GetAttribute("GrowthCode") or "R?"))
5275     or kind
5276     prompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
5277     prompt.KeyboardKeyCode = Enum.KeyCode.E
5278     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
5279     prompt.HoldDuration = isPlantContainer and 0 or 0.2
5280     prompt.MaxActivationDistance = 8
5281     prompt.RequiresLineOfSight = true
5282     prompt.Parent = button
5283
5284     local function setReadout(status, data, colour)
5285         part:SetAttribute("LastActionStatus", tostring(status or ""))
5286         part:SetAttribute("LastActionDetail", tostring(data or ""))
5287         renderTimer()
5288     end
5289
5290     local function pressButton()
5291         button.Color = Color3.fromRGB(255, 205, 70)
5292         local down = TweenService:Create(button, TweenInfo.new(0.07, Enum.EasingStyle.Quad),
5293             {CFrame = restCFrame * CFrame.new(-0.11, 0, 0)})
5294         down:Play()
5295         down.Completed:Wait()
5296         local up = TweenService:Create(button, TweenInfo.new(0.1, Enum.EasingStyle.Quad), {CFrame =
5297             ↪ restCFrame})
5298         up:Play()
5299         task.delay(0.12, function()
5300             if button.Parent then button.Color = Color3.fromRGB(218, 68, 58) end
5301         end)
5302     end
5303
5304     return prompt, setReadout, pressButton
5305 end
5306
5307 local function runProgress(seconds, caption, setReadout)
5308     local steps = 4
5309     for step = 1, steps do
5310         task.wait(seconds / steps)
5311         setReadout("RUNNING " .. (step * 25) .. "%", caption, Color3.fromRGB(255, 205, 80))
5312     end
5313 end
5314
5315 local emergencyMovesCompleted = 0
5316 local setChamberFailure

```

```

5313 local setGenericFailure
5314
5315 local function chamberSampleList(chamber)
5316     local samples = {}
5317     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do table.insert(samples, id) end
5318     return samples
5319 end
5320
5321 local function saveChamberSamples(chamber, samples)
5322     local capacity = chamber:GetAttribute("Capacity") or 10
5323     while #samples > capacity do table.remove(samples) end
5324     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
5325     chamber:SetAttribute("SampleCount", #samples)
5326     chamber:SetAttribute("Occupied", #samples > 0)
5327     chamber:SetAttribute("PlantId", samples[1] or "")
5328 end
5329
5330 local function formatRuntime(seconds)
5331     seconds = math.max(0, math.floor(seconds or 0))
5332     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60, seconds %
↵ 60)
5333 end
5334
5335 local function recoverPlantLife(chamber, startingLife, player, plantId)
5336     startingLife = math.clamp(startingLife or 100, 0, 100)
5337     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then
5338         chamber:SetAttribute("PlantLife", startingLife)
5339         return 0
5340     end
5341     local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
5342     local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
5343     chamber:SetAttribute("RecoveryToken", token)
5344     chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
5345     chamber:SetAttribute("PlantLife", startingLife)
5346     if player then
5347         player:SetAttribute("LabObjective", string.format("%s dropped in %. LIFE %d%% → 100% in %d seconds.",
5348             plantId or "Plant", chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber), startingLife,
↵ seconds))
5349         notify(player, string.format("Plant LIFE %d%%. Recovery started; full life in %d seconds.",
↵ startingLife, seconds))
5350     end
5351     task.spawn(function()
5352         for step = 1, seconds do
5353             task.wait(1)
5354             if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
5355                 or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
5356             chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step / seconds +
↵ 0.5))
5357         end
5358         chamber:SetAttribute("PlantLife", 100)
5359         chamber:SetAttribute("RecoveryEndsAt", 0)
5360         if player and player.Parent then notify(player, (plantId or "Plant") .. " recovered to 100% LIFE.") end
5361     end)
5362     return seconds
5363 end
5364
5365 local function updateStoredSamples(chamber)
5366     local count = chamber:GetAttribute("SampleCount") or 0
5367     local stage = math.clamp(chamber:GetAttribute("GrowthStage") or 0, 0, 5)
5368     for _, child in ipairs(chamber:GetDescendants()) do
5369         if child:IsA("BasePart") and child:GetAttribute("SampleSlot") then
5370             local visible = child:GetAttribute("SampleSlot") <= count
5371             child.Transparency = visible and (child:GetAttribute("StoredTransparency") or 0) or 1
5372             if child.Name:match("Stem") then child.Color = Color3.fromRGB(55 + stage * 4, 145 + stage * 10, 65)
5373             elseif child.Name:match("Leaf") then child.Color = Color3.fromRGB(65 + stage * 3, 160 + stage * 12,
↵ 75) end
5374         end
5375     end
5376 end
5377
5378 local function refreshChamberTelemetry(chamber)
5379     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
5380     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5381     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
5382     local samples = chamberSampleList(chamber)
5383     local sampleText = #samples > 0 and table.concat(samples, ", ") or "NO SAMPLES"

```

```

5384     local destination = chamber:GetAttribute("EvacuationDestination") or ""
5385     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "Unknown fault")
5386     .. (destination ~= "" and (" -> MOVE TO " .. destination) or " -> WAITING FOR SPACE")) or "SYSTEM
    ↪     NORMAL"
5387     local text = string.format("%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM
    ↪     %d% • VENT %d%\n%s\n%s",
5388     floorLabel, code, state, #samples, chamber:GetAttribute("Capacity") or 10,
5389     chamber:GetAttribute("GerminationDate") or "unknown",
5390     formatRuntime(os.time() - (chamber:GetAttribute("OperationStarted") or os.time())),
5391     chamber:GetAttribute("Temperature") or 0, chamber:GetAttribute("Humidity") or 0,
5392     chamber:GetAttribute("Ventilation") or 0, detail,
5393     state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
5394     for _, child in ipairs(chamber:GetDescendants()) do
5395         if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
5396             child.Text = text
5397             -- DESTROYED must never read as fine (green): only a genuinely
5398             -- WORKING chamber gets green, FAILED gets the usual red, and a
5399             -- permanently DESTROYED one gets an even harsher red so it never
5400             -- gets mistaken for "just needs a repair."
5401             child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
5402             or state == "FAILED" and Color3.fromRGB(255, 105, 90)
5403             or Color3.fromRGB(105, 245, 160)
5404             elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
5405                 child.Text = "PLANTS (" .. #samples .. "): " .. sampleText
5406             elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
5407                 child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)
5408                 or state == "FAILED" and Color3.fromRGB(230, 55, 45)
5409                 or Color3.fromRGB(40, 165, 95)
5410                 child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
5411             end
5412         end
5413     end
5414
5415     local function evacuationDestinations(source)
5416         local sameLevel, anywhere = {}, {}
5417         for _, candidate in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
5418             local count = candidate:GetAttribute("SampleCount") or 0
5419             local capacity = candidate:GetAttribute("Capacity") or 10
5420             local incoming = candidate:GetAttribute("IncomingReservations") or 0
5421             if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED" and count + incoming <
    ↪             capacity then
5422                 table.insert(anywhere, candidate)
5423                 if candidate:GetAttribute("Level") == source:GetAttribute("Level") then table.insert(sameLevel,
    ↪                 candidate) end
5424             end
5425         end
5426         local list = #sameLevel > 0 and sameLevel or anywhere
5427         table.sort(list, function(a, b)
5428             local ac = (a:GetAttribute("SampleCount") or 0) + (a:GetAttribute("IncomingReservations") or 0)
5429             local bc = (b:GetAttribute("SampleCount") or 0) + (b:GetAttribute("IncomingReservations") or 0)
5430             if ac == bc then return (a.Position - source.Position).Magnitude < (b.Position -
    ↪             source.Position).Magnitude end
5431             return ac < bc
5432         end)
5433         return list
5434     end
5435
5436     local function repairPathFolder(player, create)
5437         local complex = workspace:FindFirstChild("Laboratory_Complex")
5438         if not complex then return nil end
5439         local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
5440         if create and not rootFolder then
5441             rootFolder = Instance.new("Folder")
5442             rootFolder.Name = "TechnicianRepairPaths"
5443             rootFolder.Parent = complex
5444         end
5445         local name = "Player_" .. player.UserId
5446         local folder = rootFolder and rootFolder:FindFirstChild(name)
5447         if create and not folder then
5448             folder = Instance.new("Folder")
5449             folder.Name = name
5450             folder.Parent = rootFolder
5451         end
5452         return folder
5453     end
5454

```

```

5455 local function clearRepairPath(player)
5456     local folder = repairPathFolder(player, false)
5457     if folder then folder:Destroy() end
5458 end
5459
5460 local function repairReward(player, equipment, assigned)
5461     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
5462     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
5463     local points = 8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0)
5464     return math.clamp(points, 8, 120), waited, distance
5465 end
5466
5467 local function createRepairPath(player, target)
5468     clearRepairPath(player)
5469     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
5470     if not characterRoot or not target then return 0, false end
5471     local folder = repairPathFolder(player, true)
5472     if not folder then return 0, false end
5473
5474     local function approachPoint(part)
5475         local side = part:GetAttribute("WallSide")
5476         local floorY = part.Position.Y - part.Size.Y / 2
5477         if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
5478             ⇨ part.Position.Z) end
5479         if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
5480             ⇨ part.Position.Z) end
5481         if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
5482             ⇨ 2.5) end
5483         return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
5484     end
5485
5486     local function appendNavigable(points, fromPosition, toPosition)
5487         local path = PathfindingService:CreatePath({
5488             AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
5489             AgentCanClimb = true, WaypointSpacing = 4,
5490         })
5491         local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
5492         if not computed or path.Status ~= Enum.PathStatus.Success then return false end
5493         local waypoints = path:GetWaypoints()
5494         if #waypoints < 2 then return false end
5495         for index, waypoint in ipairs(waypoints) do
5496             if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
5497         end
5498         return true
5499     end
5500
5501     local complex = workspace:FindFirstChild("Laboratory_Complex")
5502     local rise = complex and complex:GetAttribute("FloorRiseStuds")
5503     local westX = complex and complex:GetAttribute("StairWestLaneX")
5504     local eastX = complex and complex:GetAttribute("StairEastLaneX")
5505     local northZ = complex and complex:GetAttribute("StairNorthZ")
5506     local southZ = complex and complex:GetAttribute("StairSouthZ")
5507     local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
5508     local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
5509     local destination = approachPoint(target)
5510     local points = {}
5511     local routeReady = false
5512     if rise and westX and eastX and northZ and southZ then
5513         local startFloor = math.clamp(math.floor((characterRoot.Position.Y - 2.5) / rise + 0.5), floorMin,
5514             ⇨ floorMax)
5515         local targetFloor = math.clamp(math.floor(destination.Y / rise + 0.5), floorMin, floorMax)
5516         if startFloor == targetFloor then
5517             routeReady = appendNavigable(points, characterRoot.Position, destination)
5518         else
5519             local direction = targetFloor > startFloor and 1 or -1
5520             local firstLaneX = direction > 0 and westX or eastX
5521             local stairEntry = Vector3.new(firstLaneX, startFloor * rise, northZ)
5522             routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
5523             local floor = startFloor
5524             while routeReady and floor ~= targetFloor do
5525                 local baseY = floor * rise
5526                 if direction > 0 then
5527                     table.insert(points, Vector3.new(westX, baseY + rise / 2, southZ))
5528                     table.insert(points, Vector3.new(eastX, baseY + rise / 2, southZ))
5529                     table.insert(points, Vector3.new(eastX, baseY + rise, northZ))
5530                 else
5531                     table.insert(points, Vector3.new(eastX, baseY - rise / 2, southZ))

```

```

5527         table.insert(points, Vector3.new(westX, baseY - rise / 2, southZ))
5528         table.insert(points, Vector3.new(westX, baseY - rise, northZ))
5529     end
5530     floor = floor + direction
5531     if floor ~= targetFloor then
5532         table.insert(points, Vector3.new(direction > 0 and westX or eastX, floor * rise, northZ))
5533     end
5534 end
5535 if routeReady then
5536     routeReady = appendNavigable(points, points[#points], destination)
5537 end
5538 end
5539 end
5540 if not routeReady then
5541     folder:Destroy()
5542     player:SetAttribute("RepairRouteReady", false)
5543     return (destination - characterRoot.Position).Magnitude, false
5544 end
5545 player:SetAttribute("RepairRouteReady", true)
5546 local distance = 0
5547 local markerCount = 0
5548 for pointIndex = 1, #points - 1 do
5549     local a, b = points[pointIndex], points[pointIndex + 1]
5550     local delta = b - a
5551     local segmentDistance = delta.Magnitude
5552     distance = distance + segmentDistance
5553     if segmentDistance > 0.1 then
5554         local direction = delta.Unit
5555         local count = math.max(1, math.ceil(segmentDistance / 4))
5556         for index = 1, count do
5557             markerCount = markerCount + 1
5558             if markerCount > 120 then break end
5559             local marker = Instance.new("Part")
5560             marker.Name = "RedRepairPath"
5561             marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
5562             local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
5563             marker.CFrame = CFrame.lookAt(position, position + direction)
5564             marker.Color = Color3.fromRGB(255, 32, 28)
5565             marker.Material = Enum.Material.Neon
5566             marker.Transparency = 0.12
5567             marker.Anchored = true
5568             marker.CanCollide = false
5569             marker.CanTouch = false
5570             marker.CanQuery = false
5571             marker.Parent = folder
5572         end
5573     end
5574     if markerCount > 120 then break end
5575 end
5576 return distance, true
5577 end
5578
5579 local function clearAssignment(player)
5580     clearRepairPath(player)
5581     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
5582         ↪ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
5583         player:SetAttribute(name, nil)
5584     end
5585 end
5586
5587 local function assignNextTask(player)
5588     clearAssignment(player)
5589     local role = playerRole(player)
5590     if role == "Technician" then
5591         local failed = {}
5592         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
5593             if equipment.Parent and equipment:GetAttribute("FunctionalState") == "FAILED" then
5594                 ↪ table.insert(failed, equipment) end
5595         end
5596         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
5597         table.sort(failed, function(a, b)
5598             if root then
5599                 local aDistance = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
5600                     ↪ root.Position.Y) > 12 and 500 or 0)
5601                 local bDistance = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
5602                     ↪ root.Position.Y) > 12 and 500 or 0)

```



```

5599         if aDistance ~= bDistance then return aDistance < bDistance end
5600     end
5601     return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
5602 end)
5603 local target = failed[1]
5604 if not target then
5605     local text = "All equipment is working. Repair score increases with failure time; assigned jobs
5606     ↪ also pay for travel distance."
5607     player:SetAttribute("LabObjective", text)
5608     return text
5609 end
5610 player:SetAttribute("AssignedRepairId", equipmentId(target))
5611 local distance, routeReady = createRepairPath(player, target)
5612 player:SetAttribute("AssignedRepairDistance", distance)
5613 local projected = select(1, repairReward(player, target, true))
5614 player:SetAttribute("AssignedRepairProjectedPoints", projected)
5615 local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR; route is
5616 ↪ recalculating"
5617 local text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
5618 ↪ while failed).",
5619     equipmentFloorLabel(target), equipmentLabel(target), routeInstruction, projected,
5620     ↪ math.floor(distance + 0.5))
5621 player:SetAttribute("LabObjective", text)
5622 return text
5623 elseif role == "Laboratory Technician" then
5624     local sources = {}
5625     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
5626         if chamber.Parent and chamber:GetAttribute("FunctionalState") == "FAILED"
5627         and (chamber:GetAttribute("SampleCount") or 0) > 0
5628         and not chamber:GetAttribute("ReservedByBot") then
5629             local destination = evacuationDestinations(chamber)[1]
5630             if destination then table.insert(sources, {source = chamber, destination = destination}) end
5631         end
5632     end
5633     table.sort(sources, function(a, b)
5634         local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
5635         local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
5636         return an < bn
5637     end)
5638     local job = sources[1]
5639     if not job then
5640         local text = "Laboratory Technician: waiting for a failed chamber with plants. Emergency moves pay
5641         ↪ +12; assigned on-time moves pay +30."
5642         player:SetAttribute("LabObjective", text)
5643         return text
5644     end
5645     local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
5646     local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
5647     local allowedSeconds = adaptiveTransferSeconds(job.source, job.destination)
5648     local deadline = os.time() + allowedSeconds
5649     player:SetAttribute("AssignedTransferSource", sourceCode)
5650     player:SetAttribute("AssignedTransferDestination", destinationCode)
5651     player:SetAttribute("AssignedTransferDeadline", deadline)
5652     -- Same red floor-marker route the Technician role gets to a failed
5653     -- unit, but pointed at the destination chamber the plant must be
5654     -- carried to -- so "where does this go" is never a guess.
5655     local routeReady = select(2, createRepairPath(player, job.destination))
5656     local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
5657     ↪ recalculating"
5658     local text = string.format("ASSIGNED +%d: %s • %s → %s • %s. Press E to take a plant. %s. Deliver in %d
5659     ↪ sec.",
5660         POINTS.MOVE_ASSIGNED_ON_TIME, equipmentFloorLabel(job.source), sourceCode, destinationCode,
5661         equipmentFloorLabel(job.destination), routeInstruction, allowedSeconds)
5662     player:SetAttribute("LabObjective", text)
5663     return text
5664 end
5665 local text = "Choose BLUE Technician or GREEN Laboratory Technician at a role kiosk."
5666 player:SetAttribute("LabObjective", text)
5667 return text
5668 end
5669
5670 local function chamberByCode(code)
5671     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
5672         if chamber:GetAttribute("GrowthCode") == code then return chamber end
5673     end
5674 end
5675 end

```

```

5668
5669 local function takePlantFromChamber(player, chamber, setReadout)
5670     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5671     if playerRole(player) ~= "Laboratory Technician" then
5672         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician moves plants", Color3.fromRGB(255,
5673             ↪ 175, 80))
5674         notify(player, "Only a Laboratory Technician can take plants. Choose the GREEN role; Technicians repair
5675             ↪ with R.")
5676         return
5677     end
5678     local samples = chamberSampleList(chamber)
5679     if #samples == 0 then setReadout(code .. " EMPTY", "No germinated samples available", Color3.fromRGB(255,
5680         ↪ 200, 90)); return end
5681     local failed = chamber:GetAttribute("FunctionalState") == "FAILED"
5682     local destination
5683     if failed then
5684         local assignedSource = player:GetAttribute("AssignedTransferSource")
5685         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
5686         if assignedSource == code and assignedDestination then
5687             local requested = chamberByCode(assignedDestination)
5688             if requested and requested:GetAttribute("FunctionalState") ~= "FAILED"
5689                 and (requested:GetAttribute("SampleCount") or 0) +
5690                 ↪ (requested:GetAttribute("IncomingReservations") or 0)
5691                 < (requested:GetAttribute("Capacity") or 10) then
5692                 destination = requested
5693             end
5694         end
5695         destination = destination or evacuationDestinations(chamber)[1]
5696     if not destination then
5697         setReadout(code .. " FAILED", "No working chamber has free capacity", Color3.fromRGB(255, 105,
5698             ↪ 90))
5699         notify(player, "No safe destination currently has room. The adaptive controller will reduce
5700             ↪ failures and free capacity.")
5701         return
5702     end
5703     -- Refresh the floor-marker route to whichever chamber this specific
5704     -- plant actually ends up assigned to (may differ from the original
5705     -- job destination if that one filled up in the meantime).
5706     createRepairPath(player, destination)
5707 end
5708 local plantId = samples[1]
5709 local stage = chamber:GetAttribute("GrowthStage") or 0
5710 local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
5711 local plantLife = chamber:GetAttribute("PlantLife") or 100
5712 if failed then
5713     plantLife = math.clamp(100 - math.floor(math.max(0, os.time() - (chamber:GetAttribute("FailureSince")
5714         ↪ or os.time()) - 120) / 6), 0, 100)
5715 end
5716 setReadout("TAKING SAMPLE", "Removing " .. plantId .. " from " .. code, Color3.fromRGB(120, 235, 170))
5717 local function canFinishTaking()
5718     return chamber.Parent ~= nil and playerRole(player) == "Laboratory Technician"
5719     and canInteract(player, chamber, 10) and not findPlantSample(player)
5720     and chamber:GetAttribute("FunctionalState") ~= "DESTROYED"
5721     and chamberSampleList(chamber)[1] == plantId
5722     and (not destination or (destination.Parent ~= nil and destination:GetAttribute("FunctionalState")
5723         ↪ == "WORKING"
5724         and (destination:GetAttribute("SampleCount") or 0) +
5725         ↪ (destination:GetAttribute("IncomingReservations") or 0)
5726         < (destination:GetAttribute("Capacity") or 10)))
5727 end
5728 if not runMathQuiz(player, "TAKE PLANT " .. plantId, chamber, canFinishTaking) then return end
5729 -- Re-read inventory after the popup; do not overwrite another transfer.
5730 samples = chamberSampleList(chamber)
5731 plantLife = math.clamp(chamber:GetAttribute("PlantLife") or 100, 0, 100)
5732 table.remove(samples, 1)
5733 local sample = makeTool(player, "Plant Sample " .. plantId, {
5734     itemType = failed and "Emergency Plant Transfer" or "Plant Sample",
5735     sample = true, plant = true, label = plantId, colour = Color3.fromRGB(133, 92, 55),
5736     attributes = {
5737         PlantSample = true, SampleId = plantId, PlantId = plantId, HomeGrowthCode = code, SourceGrowthCode
5738             ↪ = code,
5739         GrowthStage = stage, Treatments = chamber:GetAttribute("Treatments") or "",
5740         Genome = chamber:GetAttribute("Genome") or "Wild Type", Moisture = chamber:GetAttribute("Moisture")
5741             ↪ or 50,
5742         PlantLife = plantLife,
5743         GerminationDate = chamber:GetAttribute("GerminationDate") or "unknown",

```

```

5733     EvacuationSample = failed, DestinationGrowthCode = destinationCode, TransferStartedAt = os.time(),
5734 },
5735 })
5736 if not sample then table.insert(samples, 1, plantId); return end
5737 saveChamberSamples(chamber, samples)
5738 updateStoredSamples(chamber)
5739 refreshChamberTelemetry(chamber)
5740 refreshPlantVisual(sample)
5741 setCarryState(player, sample)
5742 task.spawn(function()
5743     while sample.Parent and (sample:GetAttribute("PlantLife") or 0) > 0 do
5744         task.wait(1)
5745         if not sample.Parent then return end
5746         if not sample:GetAttribute("QuizProtected") then
5747             local carriedFor = os.time() - (sample:GetAttribute("TransferStartedAt") or os.time())
5748             local life = math.max(0, (sample:GetAttribute("PlantLife") or 0) - (carriedFor > 180 and 0.2 or
5749                 ⇨ 0))
5750             sample:SetAttribute("PlantLife", life)
5751             refreshPlantVisual(sample)
5752             if life <= 0 then
5753                 local reservedCode = sample:GetAttribute("DestinationGrowthCode") or ""
5754                 local reserved = reservedCode ~= "" and chamberByCode(reservedCode) or nil
5755                 if reserved then reserved:SetAttribute("IncomingReservations", math.max(0,
5756                     ⇨ (reserved:GetAttribute("IncomingReservations") or 1) - 1)) end
5757                 player:SetAttribute("CarriedSample", "")
5758                 player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
5759                     ⇨ another plant.")
5760                 task.wait(0.5)
5761                 if sample.Parent then sample:Destroy() end
5762                 return
5763             end
5764         end
5765     end
5766 end)
5767 if destination then
5768     destination:SetAttribute("IncomingReservations", (destination:GetAttribute("IncomingReservations") or
5769         ⇨ 0) + 1)
5770     -- Begin the bonus clock after pickup, excluding the time spent on the quiz.
5771     player:SetAttribute("AssignedTransferSource", code)
5772     player:SetAttribute("AssignedTransferDestination", destinationCode)
5773     player:SetAttribute("AssignedTransferDeadline", os.time() + adaptiveTransferSeconds(chamber,
5774         ⇨ destination))
5775     local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
5776     local seconds = math.max(0, deadline - os.time())
5777     player:SetAttribute("LabObjective", string.format("CARRYING %s: press E at ANY GREEN RUNNING cabinet
5778         ⇨ with a FREE SLOT. BONUS target %s has %d sec left for +%d points.",
5779         plantId, destinationCode, seconds, POINTS.MOVE_ASSIGNED_ON_TIME))
5780     setReadout(code .. " FAILED", plantId .. " removed • SEND TO " .. destinationCode, Color3.fromRGB(255,
5781         ⇨ 145, 75))
5782     notify(player, "ALARM: " .. code .. " failed (" .. (chamber:GetAttribute("FailureReason") or "fault")
5783         ⇨ .. "). Move " .. plantId .. " to " .. destinationCode .. ".")
5784 else
5785     player:SetAttribute("LabObjective", "Treat " .. plantId .. ", then press E at any GREEN RUNNING cabinet
5786         ⇨ with a FREE SLOT.")
5787     setReadout(code .. " WORKING", plantId .. " taken • " .. #samples .. "/" ..
5788         ⇨ (chamber:GetAttribute("Capacity") or 0) .. " remain", Color3.fromRGB(115, 220, 160))
5789     notify(player, plantId .. " equipped in your hand. Process it or move it to a working chamber.")
5790 end
5791 end
5792
5793 local function storePlantInChamber(player, chamber, plant, setReadout)
5794     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5795     if playerRole(player) ~= "Laboratory Technician" then
5796         setReadout("LAB ROLE REQUIRED", "GREEN Kiosk • Laboratory Technician stores plants",
5797             ⇨ Color3.fromRGB(255, 175, 80))
5798         notify(player, "Only a Laboratory Technician can complete plant transfers.")
5799         return
5800     end
5801     if chamber:GetAttribute("FunctionalState") == "FAILED" then
5802         setReadout(code .. " FAILED", "Cannot drop here • find any GREEN RUNNING cabinet", Color3.fromRGB(255,
5803             ⇨ 105, 90))
5804         notify(player, code .. " is failed. Carry the plant to any GREEN RUNNING cabinet with a FREE SLOT and
5805             ⇨ press E.")
5806         return
5807     end
5808     local samples = chamberSampleList(chamber)

```

```

5796     local capacity = chamber:GetAttribute("Capacity") or 10
5797     if #samples >= capacity then
5798         setReadout(code .. " FULL", capacity .. "/" .. capacity .. " samples • choose another working
           ↳ chamber", Color3.fromRGB(255, 175, 75))
5799         return
5800     end
5801     local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
5802     local plantId = plant:GetAttribute("PlantId") or plant.Name
5803     local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
5804     local sourceCode = plant:GetAttribute("SourceGrowthCode") or plant:GetAttribute("HomeGrowthCode") or ""
5805     local assignedMove = wanted ~= "" and player:GetAttribute("AssignedTransferSource") == sourceCode
5806         and player:GetAttribute("AssignedTransferDestination") == code
5807     local assignedOnTime = assignedMove and os.time() <= (player:GetAttribute("AssignedTransferDeadline") or 0)
5808     setReadout("STORING SAMPLE", "Placing " .. plantId .. " into " .. code, Color3.fromRGB(120, 235, 170))
5809     -- Only repair and pickup use a quiz; delivery remains immediate.
5810     if not plant.Parent or not canInteract(player, chamber, 10) then return end
5811     table.insert(samples, plantId)
5812     saveChamberSamples(chamber, samples)
5813     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
5814     chamber:SetAttribute("GrowthStage", math.max(chamber:GetAttribute("GrowthStage") or 0, stage))
5815     chamber:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
5816     chamber:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
5817     chamber:SetAttribute("Moisture", plant:GetAttribute("Moisture") or chamber:GetAttribute("Moisture") or 50)
5818     local recoverySeconds = recoverPlantLife(chamber, plantLife, player, plantId)
5819     if wanted ~= "" then
5820         local reservedDestination = chamberByCode(wanted)
5821         if reservedDestination then
5822             reservedDestination:SetAttribute("IncomingReservations", math.max(0,
           ↳ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
5823         end
5824         emergencyMovesCompleted = emergencyMovesCompleted + 1
5825         player:SetAttribute("EmergencyTransfers", (player:GetAttribute("EmergencyTransfers") or 0) + 1)
5826     end
5827     updateStoredSamples(chamber)
5828     refreshChamberTelemetry(chamber)
5829     destroyCarriedItem(player, plant)
5830     local points = 0
5831     local reason = "plant stored"
5832     if wanted ~= "" then
5833         GameState.transfersCompleted = GameState.transfersCompleted + 1
5834         if assignedMove then
5835             GameState.assignedTransfersCompleted = GameState.assignedTransfersCompleted + 1
5836             points = assignedOnTime and POINTS.MOVE_ASSIGNED_ON_TIME or POINTS.MOVE_ASSIGNED_LATE
5837             reason = assignedOnTime and ("assigned on-time move " .. sourceCode .. " → " .. code)
5838                 or ("assigned late move " .. sourceCode .. " → " .. code)
5839         else
5840             points = POINTS.MOVE_ANY
5841             reason = "emergency plant move " .. sourceCode .. " → " .. code
5842         end
5843     end
5844     setReadout(code .. " STORED" .. (points > 0 and (" +" .. points) or ""),
5845         plantId .. " • " .. #samples .. "/" .. capacity .. " samples", Color3.fromRGB(115, 245, 150))
5846     if points > 0 then
5847         awardPoints(player, points, reason)
5848         if recoverySeconds > 0 then
5849             task.delay(recoverySeconds, function() if player.Parent then assignNextTask(player) end end)
5850         else
5851             assignNextTask(player)
5852         end
5853     else
5854         notify(player, plantId .. " safely stored in " .. code .. ". Capacity: " .. #samples .. "/" .. capacity
           ↳ .. ".")
5855     end
5856 end
5857
5858 local function handleGrowthChamber(player, chamber, setReadout)
5859     local code = chamber:GetAttribute("GrowthCode") or equipmentLabel(chamber)
5860     local reservedBy = chamber:GetAttribute("ReservedByBot")
5861     if reservedBy then
5862         -- Human players always have priority over test bots. Clearing the
5863         -- reservation makes one press of E immediately take or drop a plant.
5864         chamber:SetAttribute("ReservedByBot", nil)
5865     end
5866     local plant = findPlantSample(player)
5867     if plant then storePlantInChamber(player, chamber, plant, setReadout)
5868     elseif (chamber:GetAttribute("SampleCount") or 0) > 0 then takePlantFromChamber(player, chamber,
           ↳ setReadout)

```

```

5869     else
5870         setReadout(code .. " EMPTY", "Working chamber can receive a carried sample", Color3.fromRGB(255, 200,
↪ 90))
5871     end
5872 end
5873
5874 local GMO_TRAITS = {"Drought Resistant", "High Yield", "Salt Tolerant", "Fast Flowering", "Disease Resistant"}
5875
5876 -- Dose-response germination-viability curve, bucketed off the reference
5877 -- Viola cornuta gamma-irradiation study (Trends Sci. 2025;22(1):8705): mild
5878 -- doses barely dent germination, but survival collapses above ~600 Gy and is
5879 -- nearly gone by 1000 Gy. Each visit rolls a dose and a survival check.
5880 local MUTAGENESIS_DOSES = {
5881     {gy = 100, survivalMin = 85, survivalMax = 95},
5882     {gy = 200, survivalMin = 65, survivalMax = 85},
5883     {gy = 400, survivalMin = 45, survivalMax = 70},
5884     {gy = 600, survivalMin = 15, survivalMax = 40},
5885     {gy = 800, survivalMin = 5, survivalMax = 20},
5886     {gy = 1000, survivalMin = 0, survivalMax = 8},
5887 }
5888
5889 -- Auxin pushes rooting, cytokinin pushes shoot/bud growth, and a balanced
5890 -- ratio pushes both -- the direction of effect reported for lemon seed
5891 -- germination hormone trials, kept as a labeled tag rather than a hidden stat.
5892 local HORMONE_TREATMENTS = {
5893     {name = "Auxin-dominant (IBA-heavy)", focus = "Root", gain = 1},
5894     {name = "Cytokinin-dominant (BAP-heavy)", focus = "Shoot", gain = 1},
5895     {name = "Balanced 1:1 auxin:cytokinin", focus = "Root+Shoot", gain = 2},
5896 }
5897
5898 local function germinationPercent(player)
5899     local sown = player:GetAttribute("GermSownCount") or 0
5900     if sown == 0 then return 0 end
5901     return math.floor(((player:GetAttribute("GermNormalCount") or 0) / sown) * 100 + 0.5)
5902 end
5903
5904 local function processPlant(player, part, spec, process, setReadout)
5905     if playerRole(player) ~= "Laboratory Technician" then
5906         setReadout("LAB ROLE REQUIRED", "GREEN kiosk • Laboratory Technician develops plants",
↪ Color3.fromRGB(255, 175, 80))
5907         notify(player, "Only a Laboratory Technician can use plant-development equipment.")
5908         return false
5909     end
5910     local plant = findPlantSample(player)
5911     if not plant then
5912         setReadout("PLANT REQUIRED", "Take one from an R-numbered chamber", Color3.fromRGB(255, 150, 85))
5913         notify(player, part:GetAttribute("Kind") .. ": take a visible plant sample from an R chamber first.")
5914         return false
5915     end
5916
5917     local stage = math.clamp(plant:GetAttribute("GrowthStage") or 0, 0, 5)
5918     if process.minStage and stage < process.minStage then
5919         setReadout("STAGE TOO EARLY", "Needs Budding stage or later", Color3.fromRGB(255, 150, 85))
5920         notify(player, "Grow " .. (plant:GetAttribute("PlantId") or "the plant") .. " to Budding before
↪ pollination.")
5921         return false
5922     end
5923     if process.key and not process.repeatable and hasTreatment(plant, process.key) then
5924         setReadout("ALREADY APPLIED", process.key .. " is already recorded", Color3.fromRGB(255, 180, 85))
5925         notify(player, process.key .. " has already been applied to this plant. Use another machine.")
5926         return false
5927     end
5928
5929     runProgress(spec.wait or 1.5, process.caption, setReadout)
5930
5931     -- Mutagenesis Chamber: roll a dose, then roll survival against that
5932     -- dose's real-world-modeled band. A killed sample ends here.
5933     if process.mutagenesis then
5934         local bucket = MUTAGENESIS_DOSES[math.random(1, #MUTAGENESIS_DOSES)]
5935         local survivalPercent = math.random(bucket.survivalMin, bucket.survivalMax)
5936         local survived = math.random(1, 100) <= math.max(survivalPercent, 1)
5937         plant:SetAttribute("MutagenesisDoseGy", bucket.gy)
5938         plant:SetAttribute("MutagenesisSurvivalPercent", survivalPercent)
5939         if not survived then
5940             local plantId = plant:GetAttribute("PlantId") or "sample"
5941             setReadout("SAMPLE LOST", string.format("%d Gy dose • %d%% survival odds • did not survive",
↪ bucket.gy, survivalPercent),

```

```

5942         Color3.fromRGB(255, 90, 80))
5943         notify(player, plantId .. " did not survive the " .. bucket.gy .. " Gy dose (" .. survivalPercent
↪ .. "% survival odds). Take another plant.")
5944         destroyCarriedItem(player, plant)
5945         return true
5946     end
5947     addTreatment(plant, "Mutagenesis " .. bucket.gy .. "Gy")
5948     plant:SetAttribute("Genome", "Mutagenized (" .. bucket.gy .. " Gy)")
5949 end
5950
5951 -- Hormone Treatment Bench: roll a hormone mix, tag the resulting focus.
5952 if process.hormone then
5953     local mix = HORMONE_TREATMENTS[math.random(1, #HORMONE_TREATMENTS)]
5954     plant:SetAttribute("HormoneTreatment", mix.name)
5955     plant:SetAttribute("HormoneFocus", mix.focus)
5956     process = {key = process.key, gain = mix.gain, moistureGain = process.moistureGain, caption =
↪ process.caption}
5957     addTreatment(plant, "Hormone: " .. mix.name)
5958 elseif process.key then
5959     addTreatment(plant, process.key)
5960 end
5961
5962 -- Germination Chamber: classify the outcome (AOSA normal / abnormal /
5963 -- dead), the same three buckets a real seed-testing lab reports, and
5964 -- track a running germination percentage on the Laboratory Technician.
5965 -- A prior Decontamination Unit visit (Sterilized treatment) meaningfully
5966 -- improves the odds, matching real seed-testing guidance that
5967 -- contamination is the leading cause of an abnormal or dead result.
5968 if process.key == "Germination" then
5969     local sterilized = hasTreatment(plant, "Sterilized")
5970     local irradiated = plant:GetAttribute("MutagenesisSurvivalPercent")
5971     local normalChance = sterilized and 82 or 60
5972     if irradiated then normalChance = math.floor(normalChance * (irradiated / 100)) end
5973     local roll = math.random(1, 100)
5974     local outcome
5975     if roll <= normalChance then outcome = "Normal"
5976     elseif roll <= normalChance + 25 then outcome = "Abnormal"
5977     else outcome = "Dead" end
5978     plant:SetAttribute("GerminationOutcome", outcome)
5979     player:SetAttribute("GermSownCount", (player:GetAttribute("GermSownCount") or 0) + 1)
5980     if outcome == "Normal" then
5981         player:SetAttribute("GermNormalCount", (player:GetAttribute("GermNormalCount") or 0) + 1)
5982     elseif outcome == "Abnormal" then
5983         player:SetAttribute("GermAbnormalCount", (player:GetAttribute("GermAbnormalCount") or 0) + 1)
5984     end
5985     plant:SetAttribute("PlantLife", math.clamp((plant:GetAttribute("PlantLife") or 100) - 25, 10, 100))
5986 else
5987     player:SetAttribute("GermDeadCount", (player:GetAttribute("GermDeadCount") or 0) + 1)
5988     local plantId = plant:GetAttribute("PlantId") or "sample"
5989     setReadout("DEAD • AOSA", string.format("Lab germination rate now %d%%",
↪ germinationPercent(player)), Color3.fromRGB(255, 90, 80))
5990     notify(player, plantId .. " classified DEAD under AOSA rules (" .. (sterilized and "sterilized" or
↪ "unsterilized")
5991     .. " seed). Your running germination rate: " .. germinationPercent(player) .. "%". Take another
↪ plant.")
5992     destroyCarriedItem(player, plant)
5993     return true
5994 end
5995
5996 local newStage = math.clamp(stage + (process.gain or 0), 0, 5)
5997 plant:SetAttribute("GrowthStage", newStage)
5998 plant:SetAttribute("Moisture", math.clamp((plant:GetAttribute("Moisture") or 50) + (process.moistureGain or
↪ -2), 15, 100))
5999 if process.genome then plant:SetAttribute("Genome", GMO_TRAITS[math.random(1, #GMO_TRAITS)]) end
6000 refreshPlantVisual(plant)
6001 setCarryState(player, plant)
6002 player:SetAttribute("LabObjective", "Plant " .. (plant:GetAttribute("PlantId") or "sample") .. " is "
6003 .. (GROWTH_STAGE_NAME[newStage] or "Growing") .. ". Use another treatment or return it to an empty R
↪ chamber.")
6004 if process.key then awardPoints(player, POINTS.PROCESS_PLANT, process.key .. " plant treatment") end
6005
6006 local data = string.format("%s • %s • %d%% moisture\nGenome: %s",
6007     plant:GetAttribute("PlantId") or "PLANT", GROWTH_STAGE_NAME[newStage] or "Growing",
6008     plant:GetAttribute("Moisture") or 50, plant:GetAttribute("Genome") or "Wild Type")
6009 if plant:GetAttribute("GerminationOutcome") then

```

```

6010     data = data .. string.format("\nAOSA: %s • Lab germination rate %d%%",
        ↪ plant:GetAttribute("GerminationOutcome"), germinationPercent(player))
6011 end
6012 if plant:GetAttribute("MutagenesisDoseGy") then
6013     data = data .. string.format("\nDose %d Gy • %d%% survival odds",
        ↪ plant:GetAttribute("MutagenesisDoseGy"), plant:GetAttribute("MutagenesisSurvivalPercent") or 0)
6014 end
6015 if plant:GetAttribute("HormoneTreatment") then
6016     data = data .. "\n" .. plant:GetAttribute("HormoneTreatment") .. " • " ..
        ↪ (plant:GetAttribute("HormoneFocus") or "")
6017 end
6018 setReadout("PLANT DATA", data, Color3.fromRGB(120, 240, 160))
6019 notify(player, data:gsub("\n", " | ") .. " | Treatments: " .. treatmentText(plant))
6020 equipTool(player, plant)
6021 return true
6022 end
6023
6024 local function createTransferSample(player, source, setReadout)
6025     local existing = findTransferSample(player)
6026     if existing then
6027         local target = existing:GetAttribute("DestinationLabel") or "the named cold store"
6028         setReadout("SAMPLE IN TRANSIT", (existing:GetAttribute("SampleId") or existing.Name) .. " -> " ..
            ↪ target,
            Color3.fromRGB(255, 205, 80))
6029         notify(player, "You already carry " .. (existing:GetAttribute("SampleId") or existing.Name) .. ".
            ↪ Deliver it to " .. target .. ".")
6030         equipTool(player, existing)
6031         return
6032     end
6033
6034     local destinations = coldStoresExcept(source)
6035     if #destinations == 0 then
6036         setReadout("TRANSFER UNAVAILABLE", "A second refrigerator or freezer is required.", Color3.fromRGB(255,
            ↪ 110, 100))
6037         notify(player, "This transfer needs at least two refrigerators/freezers in the generated lab.")
6038         return
6039     end
6040
6041     -- Prefer a true refrigerator destination so the guaranteed lobby
6042     -- pair always supports the requested fridge-to-fridge play loop.
6043     local refrigeratorDestinations = {}
6044     for _, candidate in ipairs(destinations) do
6045         if candidate:GetAttribute("Kind") == "Refrigerator" then
6046             table.insert(refrigeratorDestinations, candidate)
6047         end
6048     end
6049     if #refrigeratorDestinations > 0 then destinations = refrigeratorDestinations end
6050
6051     local destination = destinations[math.random(1, #destinations)]
6052     nextSampleNumber = nextSampleNumber + 1
6053     local sampleId = string.format("SL-%04d", nextSampleNumber)
6054     local targetLabel = storageLabel(destination)
6055     local sample = makeTool(player, "Seed Sample " .. sampleId, {
6056         itemType = "Transfer Seed Sample",
6057         sample = true,
6058         label = sampleId,
6059         colour = Color3.fromRGB(120, 190, 225),
6060         attributes = {
6061             TransferSample = true,
6062             SampleId = sampleId,
6063             SourceStorageId = storageId(source),
6064             DestinationStorageId = storageId(destination),
6065             DestinationLabel = targetLabel,
6066         },
6067     })
6068     if not sample then return end
6069     setCarryState(player, sample)
6070     setReadout("DISPATCHED " .. sampleId, "DEST: " .. targetLabel, Color3.fromRGB(100, 220, 255))
6071     notify(player, "Picked up " .. sampleId .. ". Carry it visibly to " .. targetLabel .. ".")
6072 end
6073
6074 local function handleColdStore(player, part, setReadout)
6075     local sample = findTransferSample(player)
6076     if not sample then
6077         createTransferSample(player, part, setReadout)
6078         return
6079     end

```

```

6080
6081     local sampleId = sample:GetAttribute("SampleId") or sample.Name
6082     local wanted = sample:GetAttribute("DestinationStorageId")
6083     if wanted ~= storageId(part) then
6084         local destination = sample:GetAttribute("DestinationLabel") or "the assigned cold store"
6085         setReadout("WRONG STORAGE", sampleId .. " -> " .. destination, Color3.fromRGB(255, 120, 95))
6086         notify(player, sampleId .. " belongs in " .. destination .. ". It remains in your hand.")
6087         equipTool(player, sample)
6088         return
6089     end
6090
6091     destroyCarriedItem(player, sample)
6092     GameState.transfersCompleted = GameState.transfersCompleted + 1
6093     addContribution(player)
6094     updateScoreboard()
6095     setReadout("RECEIVED " .. sampleId, "TRANSFER VERIFIED | +1 contribution", Color3.fromRGB(115, 245, 150))
6096     notify(player, sampleId .. " delivered and verified. Transfer complete.")
6097 end
6098
6099 local function startCycleIndicator(part)
6100     local auto = part:GetAttribute("Auto")
6101     if not auto then return nil, function() end end
6102     local indicator = Instance.new("Part")
6103     indicator.Name = "CycleIndicator"
6104     indicator.Size = (auto == "spin") and Vector3.new(0.9, 0.15, 0.9) or Vector3.new(0.5, 0.5, 0.5)
6105     indicator.Shape = (auto == "spin") and Enum.PartType.Block or Enum.PartType.Ball
6106     indicator.Anchored = true
6107     indicator.CanCollide = false
6108     indicator.CanTouch = false
6109     indicator.CastShadow = false
6110     indicator.Material = Enum.Material.Neon
6111     indicator.Color = Color3.fromRGB(255, 196, 60)
6112     indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
6113     indicator.Parent = part.Parent
6114     local light = Instance.new("PointLight")
6115     light.Color = indicator.Color
6116     light.Brightness = 1.5
6117     light.Range = 12
6118     light.Parent = indicator
6119
6120     local running = true
6121     task.spawn(function()
6122         local t = 0
6123         while running and indicator.Parent do
6124             t = t + task.wait(0.1)
6125             if auto == "spin" then
6126                 indicator.CFrame = CFrame.new(part.Position + Vector3.new(0, part.Size.Y / 2 + 0.8, 0))
6127                     * CFrame.Angles(0, t * 5, 0)
6128             else
6129                 indicator.Transparency = 0.15 + (math.sin(t * 7) + 1) / 2 * 0.55
6130             end
6131         end
6132     end)
6133     return indicator, function()
6134         running = false
6135         if indicator.Parent then indicator:Destroy() end
6136     end
6137 end
6138
6139 local function bind(part)
6140     local kind = part:GetAttribute("Kind")
6141     if not kind or part:GetAttribute("InteractionsBound") then return end
6142     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
6143     part:SetAttribute("InteractionsBound", true)
6144     part:SetAttribute("EquipmentId", equipmentId(part))
6145     part:SetAttribute("FloorLabel", equipmentFloorLabel(part))
6146     part:SetAttribute("RepairKey", "R")
6147     if part:GetAttribute("FunctionalState") == nil then
6148         local hash = 0
6149         for i = 1, #equipmentId(part) do hash = hash + string.byte(equipmentId(part), i) end
6150         local failed = false
6151         part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
6152         part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
6153         part:SetAttribute("FailureSince", failed and os.time() or 0)
6154     end
6155     if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time()) end

```



```

6156     if COLD_STORE[kind] then storageId(part) end
6157     if isPlantContainer then
6158         updateStoredSamples(part)
6159         refreshChamberTelemetry(part)
6160     end
6161     local spec = Kinds.forKind(kind)
6162     local prompt, setReadout, pressButton = makeControls(part, kind, spec)
6163     -- Cabinet trim can occlude Roblox's prompt ray to its own push button.
6164     -- The server still enforces distance and wall line-of-sight via canInteract.
6165     prompt.RequiresLineOfSight = false
6166     local baseColour = part.Color
6167     local busy = false
6168     local powerOn = true
6169     part:SetAttribute("Powered", true)
6170     local powerLight = Instance.new("PointLight")
6171     powerLight.Color = Color3.fromRGB(120, 220, 140)
6172     powerLight.Brightness = powerOn and 2 or 0
6173     powerLight.Range = 7
6174     powerLight.Parent = part
6175
6176     local led = Instance.new("Part")
6177     led.Name = "StatusLED"
6178     led.Shape = Enum.PartType.Ball
6179     led.Size = Vector3.new(0.22, 0.22, 0.22)
6180     led.Anchored = true
6181     led.CanCollide = false
6182     led.CanTouch = false
6183     led.CastShadow = false
6184     led.Material = Enum.Material.Neon
6185     led.Color = powerOn and Color3.fromRGB(110, 230, 130) or Color3.fromRGB(64, 68, 74)
6186     local controlSide = part:GetAttribute("WallSide")
6187     if controlSide == "W" or controlSide == "E" then
6188         local direction = controlSide == "W" and 1 or -1
6189         led.CFrame = part.CFrame * CFrame.new(direction * (part.Size.X / 2 + 0.08), part.Size.Y / 2 - 0.3,
        ↪ part.Size.Z * 0.32)
6190     else
6191         local direction = controlSide == "S" and -1 or 1
6192         led.CFrame = part.CFrame * CFrame.new(part.Size.X * 0.32, part.Size.Y / 2 - 0.3, direction *
        ↪ (part.Size.Z / 2 + 0.08))
6193     end
6194     led.Parent = part
6195
6196     local repairAnchor = part:FindFirstChild("ControllerScreen") or led
6197     local floorLabel = equipmentFloorLabel(part)
6198     local unitLabel = equipmentLabel(part)
6199     local repairPrompt = Instance.new("ProximityPrompt")
6200     repairPrompt.Name = "RepairPrompt"
6201     repairPrompt.ObjectText = floorLabel .. " • " .. unitLabel
6202     repairPrompt.ActionText = "Hold R to repair"
6203     repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
6204     repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
6205     repairPrompt.HoldDuration = 3
6206     repairPrompt.MaxActivationDistance = 8
6207     repairPrompt.RequiresLineOfSight = false
6208     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
6209     repairPrompt.Parent = repairAnchor
6210
6211     part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
6212         if repairPrompt.Parent then
6213             repairPrompt.Enabled = not busy and part:GetAttribute("FunctionalState") == "FAILED"
6214         end
6215     end)
6216
6217     repairPrompt.Triggered:Connect(function(player)
6218         if playerRole(player) ~= "Technician" then
6219             setReadout("TECHNICIAN REQUIRED", "Choose the BLUE role to repair equipment", Color3.fromRGB(255,
        ↪ 175, 80))
6220             notify(player, "Only a Technician can repair equipment. Choose the BLUE role kiosk.")
6221             return
6222         end
6223         if busy or part:GetAttribute("FunctionalState") ~= "FAILED"
        or not canInteract(player, part, repairPrompt.MaxActivationDistance) then return end
6224         busy = true
6225         prompt.Enabled = false
6226         repairPrompt.Enabled = false
6227         setReadout("REPAIRING", floorLabel .. " • " .. unitLabel .. " • diagnosing ")
6228

```

```

6229     .. (part:GetAttribute("FailureReason") or "equipment fault"), Color3.fromRGB(255, 205, 75))
6230 setReadout("REPAIRING", "Working on " .. floorLabel .. " " * " .. unitLabel, Color3.fromRGB(120, 235,
↪ 170))
6231 local function canFinishRepair()
6232     return part.Parent ~= nil and playerRole(player) == "Technician"
6233         and part:GetAttribute("FunctionalState") == "FAILED"
6234         and canInteract(player, part, repairPrompt.MaxActivationDistance)
6235 end
6236 if not runMathQuiz(player, "REPAIR " .. unitLabel, part, canFinishRepair) then
6237     setReadout("REPAIR NOT COMPLETED", "Hold R for 3 seconds to try another quiz", Color3.fromRGB(255,
↪ 105, 90))
6238     prompt.Enabled = true
6239     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
6240     busy = false
6241     return
6242 end
6243 local assigned = player:GetAttribute("AssignedRepairId") == equipmentId(part)
6244 local points, failedSeconds, routeDistance = repairReward(player, part, assigned)
6245 part:SetAttribute("LastRepairedAt", os.time())
6246 part:SetAttribute("LastRepairedBy", player.Name)
6247 if kind == PLANT_CHAMBER_KIND then setChamberFailure(part, false) else setGenericFailure(part, false)
↪ end
6248 GameState.repairsCompleted = GameState.repairsCompleted + 1
6249 if assigned then GameState.assignedRepairsCompleted = GameState.assignedRepairsCompleted + 1 end
6250 local total = awardPoints(player, points, string.format("%s%s * failed %ds * route %d studs",
6251     assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
↪ math.floor(routeDistance + 0.5)))
6252 setReadout("REPAIR COMPLETE +" .. points, floorLabel .. " " * " .. unitLabel .. " WORKING * TOTAL " ..
↪ total,
6253     Color3.fromRGB(105, 245, 155))
6254 assignNextTask(player)
6255 prompt.Enabled = true
6256 busy = false
6257 end)
6258
6259 prompt.Triggered:Connect(function(player)
6260     if busy or not canInteract(player, part, prompt.MaxActivationDistance) then return end
6261     task.spawn(pressButton)
6262     if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
6263         setReadout("FAILED * HOLD R", equipmentFloorLabel(part) .. " " * " ..
↪ (part:GetAttribute("FailureReason") or "equipment fault"),
6264             Color3.fromRGB(255, 105, 90))
6265         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
6266         return
6267     end
6268
6269     local station = ROLE_STATIONS[kind]
6270     if station then
6271         if not player.Team or player.Team.Name ~= station.role then
6272             setReadout("ACCESS DENIED", station.role .. " role required", Color3.fromRGB(255, 105, 90))
6273             notify(player, kind .. ": only a " .. station.role .. " can use this. Join that role at the
↪ entrance kiosk.")
6274             return
6275         end
6276         busy = true
6277         prompt.Enabled = false
6278         runProgress(spec.wait or 1, "Reading shared batch queue", setReadout)
6279         local msg, err = station.run(GameState)
6280         if err then
6281             setReadout("NO BATCH DATA", err, Color3.fromRGB(255, 160, 90))
6282             notify(player, kind .. ": " .. err)
6283         else
6284             addContribution(player)
6285             setReadout("INSPECTION DATA", msg, Color3.fromRGB(115, 245, 150))
6286             notify(player, kind .. " -- " .. msg)
6287             updateScoreboard()
6288         end
6289         prompt.Enabled = true
6290         busy = false
6291         return
6292     end
6293
6294     local plantProcess = PLANT_PROCESS[kind]
6295     local carriedPlant = findPlantSample(player)
6296     local shouldUsePlantStorage = isPlantContainer and (
6297         kind == PLANT_CHAMBER_KIND

```

```

6298         or not plantProcess
6299         or not carriedPlant
6300         or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
6301     )
6302     if shouldUsePlantStorage then
6303         busy = true
6304         prompt.Enabled = false
6305         handleGrowthChamber(player, part, setReadout)
6306         prompt.Enabled = true
6307         busy = false
6308         return
6309     end
6310
6311     if part:GetAttribute("FunctionalState") == "FAILED" then
6312         setReadout("FAILED • HOLD R", equipmentFloorLabel(part) .. " • " ..
6313             ↳ (part:GetAttribute("FailureReason") or "equipment fault"),
6314             Color3.fromRGB(255, 105, 90))
6315         notify(player, kind .. " is FAILED. A Technician must HOLD R to repair it.")
6316         return
6317     end
6318
6319     if plantProcess then
6320         busy = true
6321         prompt.Enabled = false
6322         processPlant(player, part, spec, plantProcess, setReadout)
6323         prompt.Enabled = true
6324         busy = false
6325         return
6326     end
6327
6328     if COLD_STORE[kind] then
6329         busy = true
6330         prompt.Enabled = false
6331         runProgress(0.6, "Scanning cold-store inventory", setReadout)
6332         handleColdStore(player, part, setReadout)
6333         prompt.Enabled = true
6334         busy = false
6335         return
6336     end
6337
6338     local consumed
6339     if spec.consumes then
6340         consumed = findTool(player, spec.consumes)
6341         if not consumed then
6342             setReadout("MATERIAL NEEDED", spec.consumes, Color3.fromRGB(255, 160, 90))
6343             notify(player, kind .. ": needs " .. spec.consumes .. " first.")
6344             return
6345         end
6346     end
6347
6348     busy = true
6349     prompt.Enabled = false
6350     if spec.activeColour then part.Color = spec.activeColour end
6351     local uv
6352     if spec.uvLight then
6353         uv = Instance.new("PointLight")
6354         uv.Color = Color3.fromRGB(150, 210, 255)
6355         uv.Brightness = 2
6356         uv.Range = 26
6357         uv.Parent = part
6358     end
6359
6360     local _, stopIndicator = startCycleIndicator(part)
6361     runProgress(spec.wait or 1, spec.action, setReadout)
6362     stopIndicator()
6363
6364     if consumed then destroyCarriedItem(player, consumed) end
6365     for _, item in ipairs(spec.produces or {}) do
6366         local made = makeTool(player, item)
6367         if made and made:GetAttribute("LabSample") then setCarryState(player, made) end
6368     end
6369     local result = spec.log and spec.log() or "Inspection complete; equipment left ready."
6370     setReadout("INSPECTION DATA", result, Color3.fromRGB(115, 245, 150))
6371     notify(player, kind .. " -- " .. result)
6372
6373     if uv then uv:Destroy() end

```

```

6373     part.Color = baseColour
6374     prompt.Enabled = true
6375     busy = false
6376 end)
6377 end
6378
6379 -----
6380 -- FIFTY AUTONOMOUS GAMEPLAY TEST WORKERS
6381 -----
6382 local function setBotStatus(bot, message)
6383     bot:SetAttribute("TaskState", message)
6384     local status = bot:FindFirstChild("Status", true)
6385     if status and status:IsA("TextLabel") then
6386         status.Text = (bot:GetAttribute("DisplayName") or bot.Name) .. "\n" .. message
6387     end
6388 end
6389
6390 local function updateBotTestBoard()
6391     local workers = CollectionService:GetTagged("LabWorkerBot")
6392     local cycles, carrying, fallbacks, rivalScore = 0, 0, 0, 0
6393     for _, bot in ipairs(workers) do
6394         cycles = cycles + (bot:GetAttribute("CompletedCycles") or 0)
6395         fallbacks = fallbacks + (bot:GetAttribute("PathFallbacks") or 0)
6396         if (bot:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
6397         if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 end
6398     end
6399     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
6400     local failing, failedSamples = 0, 0
6401     for _, chamber in ipairs(chambers) do
6402         if chamber:GetAttribute("FunctionalState") == "FAILED" then
6403             failing = failing + 1
6404             failedSamples = failedSamples + (chamber:GetAttribute("SampleCount") or 0)
6405         end
6406     end
6407     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
6408         for _, label in ipairs(board:GetDescendants()) do
6409             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
6410                 label.Text = string.format(
6411                     "ADAPTIVE GROWTH-ROOM TEST\nWorkers: %d/50 Transfers: %d Carrying: %d\nFAILED: %d/%d
6412                     ↳ (%.0f%%) Samples waiting: %d\nRIVAL SCORE: %d • Path fallbacks: %d",
6413                     #workers, cycles, carrying, failing, #chambers, failing / math.max(1, #chambers) * 100,
6414                     ↳ failedSamples, rivalScore, fallbacks)
6415             end
6416         end
6417     end
6418 end
6419
6420 local function interactionPoint(part)
6421     local side = part:GetAttribute("WallSide")
6422     local bottomY = part.Position.Y - part.Size.Y / 2
6423     local point = Vector3.new(part.Position.X, bottomY + 3.25, part.Position.Z)
6424     if side == "W" then point = point + Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
6425     elseif side == "E" then point = point - Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
6426     elseif side == "N" then point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
6427     elseif side == "S" then point = point - Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
6428     else point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5) end
6429     return point
6430 end
6431
6432 -----
6433 -- OBSERVER GATE
6434 -- A worker only runs its scenario while a player is in the room with it.
6435 -- Everything else stands still. This is what keeps 180 NPCs off the
6436 -- pathfinding budget: an unobserved bot costs one cheap check a second
6437 -- instead of a continuous walk loop and a PathfindingService computation
6438 -- every few seconds.
6439 -----
6440 local FLOOR_BAND = 10 -- studs, about half a floor-to-floor height
6441 local WAKE_RADIUS = 35 -- studs, fallback for corridors and doorways, where
6442 -- neither the bot nor the player is inside any room
6443
6444 -- Room rectangles published by the generator as one packed attribute:
6445 -- "Level_1:Room 1:x1:z1:x2:z2:y;..." already in stud space.
6446 local roomsByLevel = {}
6447 do
6448     local complex = workspace:FindFirstChild("Laboratory_Complex")

```

```

6447     local packed = complex and complex:GetAttribute("RoomIndex")
6448     if typeof(packed) == "string" then
6449         for entry in packed:gmatch("[^;]+") do
6450             local level, name, x1, z1, x2, z2, y =
6451                 entry:match("^([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+)$")
6452             if level then
6453                 roomsByLevel[level] = roomsByLevel[level] or {}
6454                 table.insert(roomsByLevel[level], {
6455                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
6456                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
6457                 })
6458             end
6459         end
6460     end
6461 end
6462
6463 -- Only the ~10 rooms on the bot's own floor are ever tested, not all 81.
6464 local function roomAt(levelName, position)
6465     local list = levelName and roomsByLevel[levelName]
6466     if not list then return nil end
6467     for _, r in ipairs(list) do
6468         if math.abs(position.Y - r.y) < FLOOR_BAND
6469             and position.X >= r.x1 and position.X <= r.x2
6470             and position.Z >= r.z1 and position.Z <= r.z2 then
6471             return r.name
6472         end
6473     end
6474     return nil
6475 end
6476
6477 -- The rectangle itself, for picking a patrol point inside a room.
6478 local function roomRect(levelName, roomName)
6479     for _, r in ipairs((levelName and roomsByLevel[levelName]) or {}) do
6480         if r.name == roomName then return r end
6481     end
6482     return nil
6483 end
6484
6485 local function botHasObserver(bot)
6486     local ok, pivot = pcall(function() return bot:GetPivot().Position end)
6487     if not ok then return false end
6488     local levelName = bot:GetAttribute("Level")
6489     local botRoom = roomAt(levelName, pivot)
6490     for _, player in ipairs(Players:GetPlayers()) do
6491         local character = player.Character
6492         local hrp = character and character:FindFirstChild("HumanoidRootPart")
6493         if hrp then
6494             local here = hrp.Position
6495             -- Cheapest test first: wrong floor is an instant no.
6496             if math.abs(here.Y - pivot.Y) < FLOOR_BAND then
6497                 if botRoom and roomAt(levelName, here) == botRoom then return true end
6498                 local dx, dz = here.X - pivot.X, here.Z - pivot.Z
6499                 if dx * dx + dz * dz < WAKE_RADIUS * WAKE_RADIUS then return true end
6500             end
6501         end
6502     end
6503     return false
6504 end
6505
6506 local function botMotionChecks(bot)
6507     local excluded = {bot}
6508     local corpses = workspace:FindFirstChild("LabBotBodies")
6509     if corpses then table.insert(excluded, corpses) end
6510     local cast = RaycastParams.new()
6511     cast.FilterType = Enum.RaycastFilterType.Exclude
6512     cast.FilterDescendantsInstances = excluded
6513     cast.RespectCanCollide = true
6514     local overlap = OverlapParams.new()
6515     overlap.FilterType = Enum.RaycastFilterType.Exclude
6516     overlap.FilterDescendantsInstances = excluded
6517     overlap.RespectCanCollide = true
6518     overlap.MaxParts = 1
6519     -- Square horizontal footprint encloses the arms at any yaw. The small
6520     -- floor clearance avoids treating a floor seam as a wall.
6521     local size = Vector3.new(3.4, 5.4, 3.4)
6522     local function clear(from, to)

```

```

6523     local delta = to - from
6524     for _, player in ipairs(Players:GetPlayers()) do
6525         local character = player.Character
6526         local hrp = character and character:FindFirstChild("HumanoidRootPart")
6527         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
6528         if hrp and humanoid and humanoid.Health > 0 and math.abs(hrp.Position.Y - from.Y) < 5 then
6529             local flatDelta = Vector3.new(delta.X, 0, delta.Z)
6530             local relative = Vector3.new(hrp.Position.X - from.X, 0, hrp.Position.Z - from.Z)
6531             local alpha = flatDelta.Magnitude > 0.01 and math.clamp(relative:Dot(flatDelta) /
        ↳ flatDelta:Dot(flatDelta), 0, 1) or 0
6532             local movingAway = relative.Magnitude < 3.4 and alpha < 0.001
6533                 and (relative - flatDelta).Magnitude > relative.Magnitude + 0.01
6534             if (relative - flatDelta * alpha).Magnitude < 3.4 and not movingAway then
6535                 return false, hrp.Position
6536             end
6537         end
6538     end
6539     local offset = Vector3.new(0, -0.2, 0)
6540     if delta.Magnitude > 0.01 then
6541         local hit = workspace:Blockcast(CFrame.new(from + offset), size, delta, cast)
6542         if hit then return false, hit.Instance.Position end
6543     end
6544     -- Blockcast does not cover pre-existing overlaps; test the destination
6545     -- too, so a door closing or another worker arriving cannot be skipped.
6546     local occupied = workspace:GetPartBoundsInBox(CFrame.new(to + offset), size, overlap)
6547     if occupied[1] then return false, occupied[1].Position end
6548     return true
6549 end
6550 local function grounded(point, currentY)
6551     local floor = workspace:Raycast(Vector3.new(point.X, currentY + 1, point.Z), Vector3.new(0, -7, 0),
        ↳ cast)
6552     if not floor or floor.Normal.Y < 0.65 then return nil end
6553     local y = floor.Position.Y + 3.25
6554     if math.abs(y - currentY) > 1.25 then return nil end
6555     return Vector3.new(point.X, y, point.Z)
6556 end
6557 return clear, grounded
6558 end
6559
6560 local function walkBotSegment(bot, destination, allowDetour)
6561     if not bot.Parent or not bot.PrimaryPart or bot:GetAttribute("Dead") then return false end
6562     local clear, grounded = botMotionChecks(bot)
6563     local start = bot:GetPivot().Position
6564     local goal = Vector3.new(destination.X, start.Y, destination.Z)
6565     local direction = goal - start
6566     if direction.Magnitude < 0.15 then return true end
6567     local free, obstruction = clear(start, goal)
6568     if not free then
6569         -- Dynamic obstacles are not reliably included in Roblox's navmesh.
6570         -- Try both sides, with two corner points around the person/obstacle.
6571         -- Each leg still gets the same full-body collision tests below.
6572         if allowDetour ~= false and obstruction then
6573             local forward = direction.Unit
6574             local side = Vector3.new(-forward.Z, 0, forward.X)
6575             local centre = Vector3.new(obstruction.X, start.Y, obstruction.Z)
6576             local preferred = (bot:GetAttribute("BotNumber") or 1) % 2 == 0 and 1 or -1
6577             for _, sign in ipairs({preferred, -preferred}) do
6578                 local entry = grounded(centre - forward * 4.5 + side * sign * 4.5, start.Y)
6579                 local exitPoint = grounded(centre + forward * 4.5 + side * sign * 4.5, start.Y)
6580                 if entry and exitPoint and clear(start, entry) and clear(entry, exitPoint) and clear(exitPoint,
        ↳ goal) then
6581                     bot:SetAttribute("AvoidanceDetours", (bot:GetAttribute("AvoidanceDetours") or 0) + 1)
6582                     return walkBotSegment(bot, entry, false) and walkBotSegment(bot, exitPoint, false) and
        ↳ walkBotSegment(bot, destination, false)
6583                 end
6584             end
6585         end
6586         bot:SetAttribute("NavigationBlocked", true)
6587         return false
6588     end
6589     bot:SetAttribute("NavigationBlocked", false)
6590     local steps = math.max(1, math.ceil(direction.Magnitude / 0.45))
6591     for step = 1, steps do
6592         while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do task.wait(0.25)
        ↳ end

```

```

6593     while bot.Parent and not bot:GetAttribute("Dead") and not bot:GetAttribute("Competitor") and not
        ↪ botHasObserver(bot) do task.wait(0.5) end
6594     if not bot.Parent or bot:GetAttribute("Dead") then return false end
6595     local current = bot:GetPivot().Position
6596     local nextPoint = grounded(start:Lerp(goal, step / steps), current.Y)
6597     -- A player may have stepped into the path since it was planned. Stop
6598     -- before touching them; the caller replans from this actual position.
6599     if not nextPoint or not clear(current, nextPoint) then
6600         bot:SetAttribute("NavigationBlocked", true)
6601         return false
6602     end
6603     bot:PivotTo(CFrame.lookAt(nextPoint, nextPoint + direction.Unit))
6604     task.wait(0.06)
6605 end
6606 return true
6607 end
6608
6609 local function moveBotTo(bot, target)
6610     local destination = interactionPoint(target)
6611     -- A blocked leg is replanned, never replaced with a teleport/straight
6612     -- line through a wall. Radius matches the swept body, not just its torso.
6613     for attempt = 1, 3 do
6614         if not bot.Parent or bot:GetAttribute("Dead") then return false end
6615         local path = PathfindingService.CreatePath({AgentRadius = 1.7, AgentHeight = 5.8, AgentCanJump = false,
        ↪ WaypointSpacing = 5})
6616         local ok = pcall(function() path:ComputeAsync(bot:GetPivot().Position, destination) end)
6617         local arrived = ok and path.Status == Enum.PathStatus.Success
6618         if arrived then
6619             for _, waypoint in ipairs(path:GetWaypoints()) do
6620                 if not walkBotSegment(bot, waypoint.Position + Vector3.new(0, 3.25, 0)) then arrived = false
        ↪ break end
6621             end
6622         end
6623         if arrived then return true end
6624         bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
6625         task.wait(0.25 * attempt)
6626     end
6627     return false
6628 end
6629
6630 local function makeBotPlant(bot, stage)
6631     local old = bot:FindFirstChild("BotHeldPlant")
6632     if old then old:Destroy() end
6633     local held = Instance.new("Model")
6634     held.Name = "BotHeldPlant"
6635     held.Parent = bot
6636     local base = bot:GetPivot() * CFrame.new(0.85, 0.1, -0.72)
6637     local function piece(name, size, offset, colour, material)
6638         local p = Instance.new("Part")
6639         p.Name = name
6640         p.Size = size
6641         p.CFrame = base * CFrame.new(offset)
6642         p.Color = colour
6643         p.Material = material or Enum.Material.SmoothPlastic
6644         p.Anchored = true
6645         p.CanCollide = false
6646         p.CanTouch = false
6647         p.CanQuery = false
6648         p.Parent = held
6649         return p
6650     end
6651     piece("Pot", Vector3.new(0.85, 0.62, 0.85), Vector3.new(0, 0, 0), Color3.fromRGB(118, 77, 46))
6652     piece("Stem", Vector3.new(0.13, 0.75 + stage * 0.13, 0.13), Vector3.new(0, 0.65, 0), Color3.fromRGB(65,
        ↪ 165, 72))
6653     piece("LeafL", Vector3.new(0.48, 0.12, 0.3), Vector3.new(-0.24, 0.78, 0), Color3.fromRGB(75, 185, 82))
6654     piece("LeafR", Vector3.new(0.48, 0.12, 0.3), Vector3.new(0.24, 1.0, 0), Color3.fromRGB(75, 185, 82))
6655     return held
6656 end
6657
6658 local function equipmentReadout(part, top, bottom, colour)
6659     local label = part:FindFirstChild("Readout", true)
6660     if label and label:IsA("TextLabel") then
6661         label.Text = top .. "\n" .. bottom
6662         label.TextColor3 = colour or Color3.fromRGB(115, 245, 150)
6663     end
6664 end

```

```

6665
6666 local function candidatesOnLevel(levelName, predicate)
6667     local result = {}
6668     for _, item in ipairs(CollectionService:GetTagged("LabEquipment")) do
6669         if item.Parent and item:GetAttribute("Level") == levelName and predicate(item) then
6670             table.insert(result, item)
6671         end
6672     end
6673     return result
6674 end
6675
6676 local function runBotScenario(bot)
6677     -- Remember where this worker started, so a shift reset can stand it back up.
6678     if not bot:GetAttribute("HomeCFrame") then
6679         pcall(function() bot:SetAttribute("HomeCFrame", bot:GetPivot()) end)
6680     end
6681     -- Desk staff stay at their desk. They are still perfectly good targets.
6682     if bot:GetAttribute("Seated") then
6683         setBotStatus(bot, "At the desk")
6684         while bot.Parent and not bot:GetAttribute("Dead") do task.wait(5) end
6685         return
6686     end
6687
6688     while bot.Parent and not bot:GetAttribute("Dead") do
6689         -- The rival keeps running wherever it is: it is a single NPC and the
6690         -- competitive scoring loop depends on it making progress.
6691         local awake = bot:GetAttribute("Competitor") or botHasObserver(bot)
6692         if not awake then
6693             setBotStatus(bot, "Idle -- no one in the room")
6694             task.wait(1)
6695         elseif bot:GetAttribute("KnockedOut") then task.wait(1) else
6696             local levelName = bot:GetAttribute("Level")
6697             local failed = candidatesOnLevel(levelName, function(item)
6698                 return item:GetAttribute("Kind") == PLANT_CHAMBER_KIND
6699                     and item:GetAttribute("FunctionalState") == "FAILED"
6700                     and (item:GetAttribute("SampleCount") or 0) > 0
6701                     and not item:GetAttribute("ReservedByBot")
6702             end)
6703             if #failed == 0 then
6704                 -- Nothing has failed on this floor right now. The adaptive
6705                 -- difficulty tick deliberately holds the failure rate down when
6706                 -- few players are online, so this is the common case rather than
6707                 -- the rare one -- and standing frozen in front of a player who
6708                 -- just walked in is exactly what "the NPCs don't move" looked
6709                 -- like. Patrol the room instead so the floor reads as staffed.
6710                 -- Only ever runs for a bot somebody is actually watching.
6711                 setBotStatus(bot, "Checking chambers")
6712                 local patrolRoom = roomAt(levelName, bot:GetPivot().Position)
6713                 local rect = patrolRoom and roomRect(levelName, patrolRoom)
6714                 if rect then
6715                     local here = bot:GetPivot().Position
6716                     local spanX = math.max(1, (rect.x2 - rect.x1) - 12)
6717                     local spanZ = math.max(1, (rect.z2 - rect.z1) - 12)
6718                     walkBotSegment(bot, Vector3.new(
6719                         rect.x1 + 6 + math.random() * spanX,
6720                         here.Y,
6721                         rect.z1 + 6 + math.random() * spanZ))
6722                     task.wait(0.5 + math.random() * 1.5)
6723                 else
6724                     task.wait(3)
6725                 end
6726             else
6727                 table.sort(failed, function(a, b)
6728                     local an = tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
6729                     local bn = tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
6730                     return an < bn
6731                 end)
6732                 -- Prefer a job in the room the worker is already standing in --
6733                 -- which, because it only wakes when a player is there, is the
6734                 -- room the player is watching. Without this a woken worker
6735                 -- usually picks a chamber elsewhere on the floor, walks out of
6736                 -- the room and stops the moment it loses its observer.
6737                 local botRoom = roomAt(levelName, bot:GetPivot().Position)
6738                 local sameRoom = {}
6739                 if botRoom then
6740                     for _, item in ipairs(failed) do

```



```

6741         if item:GetAttribute("Room") == botRoom then table.insert(sameRoom, item) end
6742     end
6743 end
6744 local pool = (#sameRoom > 0) and sameRoom or failed
6745 local source = bot:GetAttribute("Competitor") and pool[1] or pool[math.random(1, #pool)]
6746 local destination = evacuationDestinations(source)[1]
6747 if not destination then
6748     setBotStatus(bot, "Waiting for safe capacity")
6749     task.wait(3)
6750 else
6751     local botId = bot.Name
6752     source:SetAttribute("ReservedByBot", botId)
6753     destination:SetAttribute("IncomingReservations",
6754         ↪ (destination:GetAttribute("IncomingReservations") or 0) + 1)
6755     source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode") or "")
6756     refreshChamberTelemetry(source)
6757     setBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or "alarm"))
6758     local reachedSource = moveBotTo(bot, source)
6759     local samples = chamberSampleList(source)
6760     local plantId = reachedSource and table.remove(samples, 1)
6761     if plantId and source:GetAttribute("ReservedByBot") == botId then
6762         saveChamberSamples(source, samples)
6763         updateStoredSamples(source)
6764         refreshChamberTelemetry(source)
6765         local stage = math.clamp(source:GetAttribute("GrowthStage") or 0, 0, 5)
6766         makeBotPlant(bot, stage)
6767         bot:SetAttribute("CarryingPlant", plantId)
6768         setBotStatus(bot, "Evacuating " .. plantId)
6769         equipmentReadout(source, (source:GetAttribute("GrowthCode") or "R?") .. " FAILED",
6770             ↪ "SEND " .. plantId .. " TO " .. (destination:GetAttribute("GrowthCode") or "R?"),
6771             ↪ Color3.fromRGB(255, 105, 90))
6772     end
6773     -- Emergency transfers still receive one quick development step,
6774     -- so the same growth mechanics are continuously regression-tested.
6775     local stations = candidatesOnLevel(levelName, function(item) return
6776         ↪ PLANT_PROCESS[item:GetAttribute("Kind")] ~= nil end)
6777     if #stations > 0 then
6778         local station = stations[((bot:GetAttribute("BotNumber") or 1) - 1) % #stations +
6779             ↪ 1]
6780         local process = PLANT_PROCESS[station:GetAttribute("Kind")]
6781         if moveBotTo(bot, station) then
6782             stage = math.clamp(stage + (process.gain or 0), 0, 5)
6783             station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or 0)
6784                 ↪ + 1)
6785             equipmentReadout(station, "BOT GROWTH TEST", plantId .. " • stage " .. stage,
6786                 ↪ Color3.fromRGB(120, 235, 165))
6787         end
6788     end
6789     setBotStatus(bot, "Delivering to " .. (destination:GetAttribute("GrowthCode") or "R?"))
6790     local reachedDestination = moveBotTo(bot, destination)
6791     local destinationSamples = chamberSampleList(destination)
6792     if reachedDestination and destination:GetAttribute("FunctionalState") ~= "FAILED" and
6793         ↪ destination:GetAttribute("FunctionalState") ~= "DESTROYED" and #destinationSamples
6794         ↪ < (destination:GetAttribute("Capacity") or 10) then
6795         table.insert(destinationSamples, plantId)
6796         saveChamberSamples(destination, destinationSamples)
6797         destination:SetAttribute("GrowthStage",
6798             ↪ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
6799         updateStoredSamples(destination)
6800         refreshChamberTelemetry(destination)
6801         emergencyMovesCompleted = emergencyMovesCompleted + 1
6802         bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or 0) + 1)
6803         if bot:GetAttribute("Competitor") then
6804             local gained = POINTS.MOVE_ASSIGNED_ON_TIME
6805             bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) + gained)
6806             setBotStatus(bot, "+" .. gained .. " resource win")
6807             updateScoreboard()
6808         else
6809             setBotStatus(bot, "Transfer passed")
6810         end
6811     else
6812         -- Capacity changed while walking; return the sample to the source
6813         -- rather than deleting it and let the next cycle choose again.
6814         table.insert(samples, plantId)
6815         saveChamberSamples(source, samples)

```

```

6807         updateStoredSamples(source)
6808         refreshChamberTelemetry(source)
6809         setBotStatus(bot, "Destination changed; retrying")
6810     end
6811     local held = bot:FindFirstChild("BotHeldPlant")
6812     if held then held:Destroy() end
6813     bot:SetAttribute("CarryingPlant", "")
6814 end
6815 destination:SetAttribute("IncomingReservations", math.max(0,
    ↪ (destination:GetAttribute("IncomingReservations") or 1) - 1))
6816 if source:GetAttribute("ReservedByBot") == botId then source:SetAttribute("ReservedByBot",
    ↪ nil) end
6817 updateBotTestBoard()
6818 task.wait(1.5 + math.random())
6819 end
6820 end
6821 end
6822 end
6823 end
6824
6825 -----
6826 -- ADAPTIVE, INDEFINITE CHAMBER FAILURE SIMULATION
6827 -----
6828 local FAILURE_REASONS = {
6829     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
6830     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
6831 }
6832 local generatedSampleSerial = 20000
6833 local previousCompletedMoves = 0
6834
6835 setChamberFailure = function(chamber, failed)
6836     if not chamber.Parent or chamber:GetAttribute("FunctionalState") == "DESTROYED" then return end
6837     if chamber:GetAttribute("QuizPlayerId") then return end
6838     if failed and ((chamber:GetAttribute("IncomingReservations") or 0) > 0
6839         or chamber:GetAttribute("ReservedByBot")) then return end
6840     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
6841     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
6842     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN FLOOR"
6843     if failed then
6844         chamber:SetAttribute("FunctionalState", "FAILED")
6845         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
6846         chamber:SetAttribute("RepairKey", "R")
6847         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " • HOLD R
            ↪ TO REPAIR")
6848         chamber:SetAttribute("FailureReason", FAILURE_REASONS[math.random(1, #FAILURE_REASONS)])
6849         chamber:SetAttribute("FailureSince", os.time())
6850         chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
6851         chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
6852         chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
6853         local destination = evacuationDestinations(chamber)[1]
6854         chamber:SetAttribute("EvacuationDestination", destination and destination:GetAttribute("GrowthCode") or
            ↪ "")
6855         for _, player in ipairs(Players:GetPlayers()) do
6856             if playerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
            ↪ assignNextTask(player) end
6857             if playerRole(player) == "Laboratory Technician" and not
            ↪ player:GetAttribute("AssignedTransferSource") then assignNextTask(player) end
6858             if playerRole(player) == "Laboratory Technician" then
6859                 notify(player, "PLANT ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Press E to
                    ↪ remove plants and follow the assigned destination.")
6860             else
6861                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • CHAMBER " .. code .. " failed. Technician:
                    ↪ HOLD R to repair.")
6862             end
6863         end
6864     else
6865         chamber:SetAttribute("FunctionalState", "WORKING")
6866         chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
6867         chamber:SetAttribute("FailureReason", "")
6868         chamber:SetAttribute("FailureSince", 0)
6869         chamber:SetAttribute("EvacuationDestination", "")
6870         chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
6871         chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
6872         chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
6873         chamber:SetAttribute("OperationStarted", os.time())

```

```

6874         if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) < 100
6875             ↪ then
6876                 recoverPlantLife(chamber, chamber:GetAttribute("PlantLife"), nil, chamber:GetAttribute("PlantId"))
6877             end
6878         end
6879     end
6880     refreshChamberTelemetry(chamber)
6881 end
6882
6883 local GENERIC_FAILURE_REASONS = {
6884     "Power relay fault", "Controller communication loss", "Pump interlock fault",
6885     "Calibration expired", "Safety sensor trip", "Motor overload",
6886 }
6887
6888 setGenericFailure = function(part, failed)
6889     if not part.Parent or part:GetAttribute("FunctionalState") == "DESTROYED" then return end
6890     if part:GetAttribute("QuizPlayerId") then return end
6891     if failed and (part:GetAttribute("IncomingReservations") or 0) > 0 then return end
6892     local floorLabel = equipmentFloorLabel(part)
6893     local unitLabel = equipmentLabel(part)
6894     if failed then
6895         part:SetAttribute("FunctionalState", "FAILED")
6896         part:SetAttribute("RepairKey", "R")
6897         part:SetAttribute("FailureReason", GENERIC_FAILURE_REASONS[math.random(1, #GENERIC_FAILURE_REASONS)])
6898         part:SetAttribute("FailureSince", os.time())
6899         part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • HOLD R")
6900         equipmentReadout(part, "FAILED • HOLD R", floorLabel .. " • " .. unitLabel .. " • PLANTS "
6901             .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • "
6902             ↪ .. part:GetAttribute("FailureReason"),
6903             Color3.fromRGB(255, 105, 90))
6904         for _, player in ipairs(Players:GetPlayers()) do
6905             if playerRole(player) == "Technician" then
6906                 if not player:GetAttribute("AssignedRepairId") then assignNextTask(player) end
6907                 notify(player, "REPAIR ALARM: " .. floorLabel .. " • " .. unitLabel .. " FAILED. HOLD R to
6908                     ↪ repair.")
6909             end
6910         end
6911     else
6912         part:SetAttribute("FunctionalState", "WORKING")
6913         part:SetAttribute("FailureReason", "")
6914         part:SetAttribute("FailureSince", 0)
6915         part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
6916         part:SetAttribute("OperationStarted", os.time())
6917         equipmentReadout(part, "WORKING", floorLabel .. " • " .. unitLabel .. " • PLANTS "
6918             .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. " • E
6919             ↪ USE / INSPECT",
6920             Color3.fromRGB(110, 235, 170))
6921     end
6922 end
6923
6924 local function shuffled(list)
6925     for i = #list, 2, -1 do
6926         local j = math.random(1, i)
6927         list[i], list[j] = list[j], list[i]
6928     end
6929     return list
6930 end
6931
6932 local function adaptiveFailureTick()
6933     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
6934     local failed, working = {}, {}
6935     local failedSamples, totalSamples, workingFree = 0, 0, 0
6936     for _, chamber in ipairs(chambers) do
6937         local count = chamber:GetAttribute("SampleCount") or 0
6938         local capacity = chamber:GetAttribute("Capacity") or 10
6939         totalSamples = totalSamples + count
6940         if chamber:GetAttribute("FunctionalState") == "FAILED" then
6941             table.insert(failed, chamber)
6942             failedSamples = failedSamples + count
6943         elseif chamber:GetAttribute("FunctionalState") == "WORKING" then
6944             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 180 then
6945                 table.insert(working, chamber)
6946             end
6947             workingFree = workingFree + math.max(0, capacity - count -
6948                 ↪ (chamber:GetAttribute("IncomingReservations") or 0))
6949         end
6950     end
6951 end

```

```

6945
6946     local throughput = emergencyMovesCompleted - previousCompletedMoves
6947     previousCompletedMoves = emergencyMovesCompleted
6948     -- A human-sized workload, independent of the hundreds of installed units.
6949     -- Two failed chambers per player (at most eight), one new alarm per tick.
6950     local playerCount = math.max(1, #Players:GetPlayers())
6951     local targetFailed = workingFree >= 12 and math.min(8, playerCount * 2) or 0
6952     local targetRate = targetFailed / math.max(1, #chambers)
6953     local change = math.min(1, targetFailed - #failed)
6954     if change > 0 then
6955         local eligible = {}
6956         for _, chamber in ipairs(working) do
6957             if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 180
6958                 and (chamber:GetAttribute("IncomingReservations") or 0) == 0
6959                 and not chamber:GetAttribute("ReservedByBot") then table.insert(eligible, chamber) end
6960         end
6961         shuffled(eligible)
6962         for i = 1, math.min(change, #eligible) do setChamberFailure(eligible[i], true) end
6963     elseif change < 0 then
6964         table.sort(failed, function(a, b)
6965             local ac, bc = a:GetAttribute("SampleCount") or 0, b:GetAttribute("SampleCount") or 0
6966             if ac == bc then return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or
6967                 ↪ 0) end
6968             return ac < bc
6969         end)
6970         for i = 1, math.min(-change, #failed) do setChamberFailure(failed[i], false) end
6971     end
6972     -- Failed telemetry continues to drift; working chambers continuously
6973     -- germinate replacement samples so the transfer game never runs out.
6974     for _, chamber in ipairs(chambers) do
6975         if chamber:GetAttribute("FunctionalState") == "FAILED" then
6976             chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
6977                 ↪ math.random(-2, 2), 28, 42))
6978             chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
6979                 ↪ math.random(-4, 4), 25, 65))
6980             chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
6981                 ↪ math.random(-5, 3), 0, 48))
6982             local destination = evacuationDestinations(chamber)[1]
6983             chamber:SetAttribute("EvacuationDestination", destination and
6984                 ↪ destination:GetAttribute("GrowthCode") or "")
6985         elseif chamber:GetAttribute("FunctionalState") == "WORKING" then
6986             local samples = chamberSampleList(chamber)
6987             local capacity = chamber:GetAttribute("Capacity") or 10
6988             local refillChance = math.clamp(0.08 + (capacity - #samples) * 0.025 + throughput * 0.002
6989                 ↪ + playerCount * 0.008, 0.08, 0.46)
6990             -- Keep receiving space available for real evacuation tasks.
6991             if #samples < capacity - math.max(2, math.ceil(capacity * 0.3)) and math.random() < refillChance
6992                 ↪ then
6993                 generatedSampleSerial = generatedSampleSerial + 1
6994                 table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
6995                     ↪ generatedSampleSerial))
6996                 saveChamberSamples(chamber, samples)
6997                 chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
6998                 updateStoredSamples(chamber)
6999             end
7000         end
7001         refreshChamberTelemetry(chamber)
7002     end
7003     local complex = workspace:FindFirstChild("Laboratory_Complex")
7004     if complex then
7005         complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
7006         complex:SetAttribute("FailingChambers", #failed)
7007         complex:SetAttribute("FailedSampleBacklog", failedSamples)
7008         complex:SetAttribute("EmergencyMovesCompleted", emergencyMovesCompleted)
7009         complex:SetAttribute("RecentTransferThroughput", throughput)
7010         complex:SetAttribute("ActivePlayerCount", playerCount)
7011         complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples /
7012             ↪ 20), 8, 50))
7013         -- Per-FL00R budget. Multiplied across the basement plus every level
7014         -- this is what actually decides how many workers are visibly busy.
7015         complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
7016             ↪ math.floor(failedSamples / 60), 6, 20))
7017     end
7018 end
7019
7020
7021

```

```

7012 local function genericFailureTick()
7013     local failed, working = {}, {}
7014     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
7015         if equipment:GetAttribute("Kind") ~= PLANT_CHAMBER_KIND then
7016             if equipment:GetAttribute("FunctionalState") == "FAILED" then
7017                 table.insert(failed, equipment)
7018             elseif equipment:GetAttribute("FunctionalState") == "WORKING"
7019                 and os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 180 then
7020                 table.insert(working, equipment)
7021             end
7022         end
7023     end
7024     local playerCount = math.max(1, #Players:GetPlayers())
7025     local targetFailed = math.min(4, playerCount)
7026     local change = math.min(1, targetFailed - #failed)
7027     if change > 0 then
7028         shuffled(working)
7029         for i = 1, math.min(change, #working) do setGenericFailure(working[i], true) end
7030     elseif change < 0 then
7031         table.sort(failed, function(a, b) return (a:GetAttribute("FailureSince") or 0) <
7032             ⇨ (b:GetAttribute("FailureSince") or 0) end)
7033         for i = 1, math.min(-change, #failed) do setGenericFailure(failed[i], false) end
7034     end
7035 end
7036 local function bindPlayerTaskRouting(player)
7037     if player:GetAttribute("EquipmentTaskRoutingBound") then return end
7038     player:SetAttribute("EquipmentTaskRoutingBound", true)
7039     player:GetAttributeChangedSignal("LabRole"):Connect(function()
7040         task.delay(0.2, function()
7041             if player.Parent then assignNextTask(player) end
7042         end)
7043     end)
7044     player.CharacterAdded:Connect(function()
7045         if playerRole(player) == "Technician" and player:GetAttribute("AssignedRepairId") then
7046             task.delay(1, function() if player.Parent then assignNextTask(player) end end)
7047         end
7048     end)
7049 end
7050
7051 for _, player in ipairs(Players:GetPlayers()) do bindPlayerTaskRouting(player) end
7052 Players.PlayerAdded:Connect(bindPlayerTaskRouting)
7053 Players.PlayerRemoving:Connect(clearRepairPath)
7054
7055 for _, p in ipairs(CollectionService:GetTagged("LabEquipment")) do bind(p) end
7056 CollectionService.GetInstanceAddedSignal("LabEquipment"):Connect(bind)
7057
7058 -----
7059 -- AUTOMATIC DOORS
7060 -- Every AutoDoor panel slides up into the header void when a player's
7061 -- HumanoidRootPart comes within range, and back down once nobody is close,
7062 -- so rooms have real doors instead of a permanently open gap.
7063 -----
7064 local DOOR_OPEN_RANGE = 11
7065 local DOOR_CLOSE_RANGE = 15
7066 local DOOR_MIN_OPEN_SECONDS = 3
7067 local function bindAutoDoor(door)
7068     if door:GetAttribute("DoorBound") then return end
7069     door:SetAttribute("DoorBound", true)
7070     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
7071     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
7072     local openX = door:GetAttribute("DoorOpenX") or closedX
7073     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
7074     local doorY = door:GetAttribute("DoorY") or door.Position.Y
7075     local isOpen = false
7076     local openedAt = 0
7077     local tween
7078     local function setIndicatorColour(open)
7079         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
7080         for _, gui in ipairs(door:GetChildren()) do
7081             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then
7082                 local indicator = gui:FindFirstChild("DoorIndicator")
7083                 if indicator then indicator.BackgroundColor3 = colour end
7084             end
7085         end
7086     end

```

```

7087     local function setOpen(open)
7088         if open == isOpen then return end
7089         isOpen = open
7090         if open then openedAt = os.clock() end
7091         if tween then tween:Cancel() end
7092         local targetX = open and openX or closedX
7093         local targetZ = open and openZ or closedZ
7094         -- 0.9s ease so the slide is smooth and readable (never an instant
7095         -- snap, which is what made the old vertical slide read as flicker).
7096         tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
7097             Enum.EasingDirection.InOut),
7098             {CFrame = CFrame.new(targetX, doorY, targetZ)})
7099         tween:Play()
7100         door.CanCollide = not open
7101         door:SetAttribute("DoorOpen", open)    -- watched by the room smoke system
7102         setIndicatorColour(open)
7103     end
7104     task.spawn(function()
7105         while door.Parent do
7106             local nearest = math.huge
7107             for _, player in ipairs(Players:GetPlayers()) do
7108                 local character = player.Character
7109                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
7110                 if hrp then
7111                     -- Floors stack directly above each other at the same X/Z, so a
7112                     -- flat (X/Z-only) distance check would treat a player on a
7113                     -- completely different floor as "standing at the door." Only
7114                     -- count players actually close in height to this door first.
7115                     local verticalGap = math.abs(hrp.Position.Y - doorY)
7116                     if verticalGap < 8 then
7117                         local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) - Vector3.new(closedX,
7118                             doorY, closedZ)).Magnitude
7119                         if flat < nearest then nearest = flat end
7120                     end
7121                 end
7122             end
7123             -- Once opened, a door holds for at least DOOR_MIN_OPEN_SECONDS
7124             -- before it's allowed to close again -- this is what actually
7125             -- fixes the "opens and closes too fast" flicker, independent of
7126             -- the slide direction/speed above.
7127             if isOpen and nearest > DOOR_CLOSE_RANGE and (os.clock() - openedAt) >= DOOR_MIN_OPEN_SECONDS then
7128                 setOpen(false)
7129             elseif not isOpen and nearest < DOOR_OPEN_RANGE then
7130                 setOpen(true)
7131             end
7132             task.wait(0.25)
7133         end
7134     end)
7135 end
7136
7137 for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindAutoDoor(p) end
7138 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(bindAutoDoor)
7139
7140 -----
7141 -- EMERGENCY KITS
7142 -- One per room. Press E to fully restore health. When a player's health
7143 -- drops below half (and they're not mid-job), a red floor-marker route --
7144 -- the same system used for repair/transfer jobs -- guides them to the
7145 -- closest kit.
7146 -----
7147 local function bindEmergencyKit(kit)
7148     if kit:GetAttribute("KitBound") then return end
7149     kit:SetAttribute("KitBound", true)
7150     local prompt = kit:FindFirstChild("EmergencyKitPrompt")
7151     if not prompt then return end
7152     -- Health stations RECHARGE rather than being consumed. A single-use kit
7153     -- that destroyed itself meant a floor could be permanently stripped of
7154     -- health with no way to get it back.
7155     local RECHARGE_SECONDS = 25
7156     local caption
7157     local faceGui = kit:FindFirstChild("EmergencyKitFace")
7158     if faceGui then
7159         for _, d in ipairs(faceGui:GetDescendants()) do
7160             if d:IsA("TextLabel") then caption = d break end
7161         end
7162     end
7163 end

```

```

7161
7162     local function setCharged(charged)
7163         kit:SetAttribute("Charged", charged)
7164         prompt.Enabled = charged
7165         kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
7166         if caption then caption.Text = charged and "HEALTH" or "CHARGING" end
7167     end
7168
7169     prompt.Triggered:Connect(function(player)
7170         if kit:GetAttribute("Charged") == false then return end
7171         local character = player.Character
7172         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7173         if not humanoid then return end
7174         setCharged(false)
7175         humanoid.Health = humanoid.MaxHealth
7176         notify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s -- there are
7177         ↪ more along every corridor.")
7178         task.delay(RECHARGE_SECONDS, function()
7179             if kit.Parent then setCharged(true) end
7180         end)
7181     end)
7182 end
7183 for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindEmergencyKit(p) end
7184 CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindEmergencyKit)
7185
7186 -----
7187 -- PLAYER HEALTH BARS
7188 -- Matches the bar the worker bots carry, so a player can read their own
7189 -- and everyone else's condition without opening any menu.
7190 -----
7191 local function attachPlayerHealthBar(character)
7192     local humanoid = character:FindFirstChildOfClass("Humanoid") or character:WaitForChild("Humanoid", 8)
7193     local head = character:FindFirstChild("Head") or character:WaitForChild("Head", 8)
7194     if not humanoid or not head or head:FindFirstChild("LabHealthBar") then return end
7195
7196     local gui = Instance.new("BillboardGui")
7197     gui.Name = "LabHealthBar"
7198     gui.Size = UDim2.fromOffset(168, 44)
7199     gui.StudsOffset = Vector3.new(0, 2.6, 0)
7200     gui.MaxDistance = 90
7201     gui.AlwaysOnTop = false
7202     gui.Parent = head
7203
7204     local back = Instance.new("Frame")
7205     back.Name = "HealthBarBack"
7206     back.Size = UDim2.new(0.9, 0, 0.34, 0)
7207     back.Position = UDim2.new(0.05, 0, 0.06, 0)
7208     back.BackgroundColor3 = Color3.fromRGB(22, 26, 32)
7209     back.BackgroundTransparency = 0.2
7210     back.BorderSizePixel = 0
7211     back.Parent = gui
7212     local fill = Instance.new("Frame")
7213     fill.Name = "HealthBarFill"
7214     fill.Size = UDim2.fromScale(1, 1)
7215     fill.BackgroundColor3 = Color3.fromRGB(75, 220, 95)
7216     fill.BorderSizePixel = 0
7217     fill.Parent = back
7218     local label = Instance.new("TextLabel")
7219     label.Name = "HealthText"
7220     label.Size = UDim2.new(1, 0, 0.5, 0)
7221     label.Position = UDim2.new(0, 0, 0.48, 0)
7222     label.BackgroundTransparency = 1
7223     label.TextColor3 = Color3.fromRGB(224, 238, 248)
7224     label.TextStrokeTransparency = 0.4
7225     label.TextScaled = true
7226     label.Font = Enum.Font.GothamBold
7227     label.Parent = gui
7228
7229     local function refresh()
7230         local maximum = math.max(1, humanoid.MaxHealth)
7231         local ratio = math.clamp(humanoid.Health / maximum, 0, 1)
7232         fill.Size = UDim2.fromScale(ratio, 1)
7233         fill.BackgroundColor3 = ratio > 0.55 and Color3.fromRGB(75, 220, 95)
7234         or (ratio > 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
7235         label.Text = string.format("%d%%", math.floor(ratio * 100 + 0.5))

```

```

7236     end
7237     humanoid.HealthChanged:Connect(refresh)
7238     humanoid:GetPropertyChangedSignal("MaxHealth"):Connect(refresh)
7239     refresh()
7240 end
7241
7242 local function watchPlayerHealth(player)
7243     player.CharacterAdded:Connect(attachPlayerHealthBar)
7244     if player.Character then task.spawn(attachPlayerHealthBar, player.Character) end
7245 end
7246 Players.PlayerAdded:Connect(watchPlayerHealth)
7247 for _, player in ipairs(Players:GetPlayers()) do watchPlayerHealth(player) end
7248
7249 -----
7250 -- SECRET BASEMENT VENT
7251 -- Disguised as part of the lobby's own exterior wall. Holding E on it
7252 -- drops the player straight into the basement corridor.
7253 -----
7254 local function bindSecretBasementPanel(panel)
7255     if panel:GetAttribute("SecretBound") then return end
7256     panel:SetAttribute("SecretBound", true)
7257     local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")
7258     if not prompt then return end
7259     prompt.Triggered:Connect(function(player)
7260         local character = player.Character
7261         local hrp = character and character:FindFirstChild("HumanoidRootPart")
7262         if not hrp then return end
7263         local humanoid = character:FindFirstChildOfClass("Humanoid")
7264         if not humanoid or humanoid.Health <= 0 or (hrp.Position - panel.Position).Magnitude > 12 then return
7265         ↪ end
7266         local floorPoint = Vector3.new(
7267             panel:GetAttribute("SecretDestX") or hrp.Position.X,
7268             panel:GetAttribute("SecretDestY") or hrp.Position.Y,
7269             panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
7270         local params = RaycastParams.new()
7271         params.FilterType = Enum.RaycastFilterType.Exclude
7272         params.FilterDescendantsInstances = {character}
7273         params.RespectCanCollide = true
7274         local floorHit = workspace:Raycast(floorPoint + Vector3.new(0, 7, 0), Vector3.new(0, -12, 0), params)
7275         if not floorHit or floorHit.Normal.Y < 0.7 then return end
7276         local feetOffset = humanoid.HipHeight + hrp.Size.Y / 2
7277         for _, name in ipairs({"LeftFoot", "RightFoot", "Left Leg", "Right Leg"}) do
7278             local foot = character:FindFirstChild(name)
7279             if foot and foot:IsA("BasePart") then
7280                 feetOffset = math.max(feetOffset, hrp.Position.Y - (foot.Position.Y - foot.Size.Y / 2))
7281             end
7282         end
7283         hrp.CFrame = CFrame.new(floorHit.Position + Vector3.new(0, feetOffset + 0.35, 0))
7284         hrp.AssemblyLinearVelocity, hrp.AssemblyAngularVelocity = Vector3.zero, Vector3.zero
7285         humanoid:ChangeState(Enum.HumanoidStateType.GettingUp)
7286         notify(player, "The vent panel swings open -- you drop into the basement.")
7287     end)
7288 end
7289 for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindSecretBasementPanel(p) end
7290 CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindSecretBasementPanel)
7291
7292 local function nearestEmergencyKit(position)
7293     local best, bestDist
7294     for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
7295         if kit.Parent then
7296             local d = (kit.Position - position).Magnitude
7297             if not bestDist or d < bestDist then best, bestDist = kit, d end
7298         end
7299     end
7300     return best
7301 end
7302
7303 local healthPathActive = {}
7304 task.spawn(function()
7305     while true do
7306         task.wait(3)
7307         for _, player in ipairs(Players:GetPlayers()) do
7308             local character = player.Character
7309             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7310             local hrp = character and character:FindFirstChild("HumanoidRootPart")
7311             if humanoid and hrp and humanoid.Health > 0 then

```



```

7311         local low = humanoid.Health < humanoid.MaxHealth * 0.5
7312         local busy = player:GetAttribute("AssignedRepairId") or
7313         ↪ player:GetAttribute("AssignedTransferSource")
7314         if low and not busy and not healthPathActive[player] then
7315             local kit = nearestEmergencyKit(hrp.Position)
7316             if kit then
7317                 createRepairPath(player, kit)
7318                 healthPathActive[player] = true
7319                 notify(player, "Health low: follow the RED floor lights to the nearest emergency kit.")
7320             end
7321         elseif (not low or busy) and healthPathActive[player] then
7322             healthPathActive[player] = false
7323             if not busy then clearRepairPath(player) end
7324         end
7325     end
7326 end
7327 end)
7328
7329 -----
7330 -- EQUIPMENT HAZARDS
7331 -- A malfunctioning (FAILED) unit is genuinely dangerous to linger next to:
7332 -- electrical faults, steam/humidity blowouts, chemical spills and coolant
7333 -- (freon) leaks all drain health while a player stays in range.
7334 -----
7335 local HAZARD_BY_KIND = {
7336     ["Germination Chamber"] = {"STEAM LEAK", 3},
7337     ["Incubator"] = {"STEAM LEAK", 3},
7338     ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
7339     ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
7340     ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
7341     ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
7342     ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
7343     ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
7344     ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},
7345     ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
7346     ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
7347 }
7348 local HAZARD_RANGE = 9
7349 local hazardWarnedAt = {}
7350
7351 -- A unit that's been left FAILED too long can catastrophically blow up,
7352 -- catch fire, or arc -- and unlike an ordinary failure, this one can never
7353 -- be repaired. It keeps draining nearby health (worse than before) and its
7354 -- StatusLED/timer read DESTROYED permanently.
7355 --
7356 -- Two-stage sequence: warnEquipmentCatastrophe() runs a 2-second siren +
7357 -- strobe so anyone nearby has time to clear out, THEN destroyEquipment()
7358 -- fires the actual explosion/fire/arc/leak effects.
7359 local DESTRUCTION_KIND = {
7360     ["Germination Chamber"] = "steam",
7361     ["Incubator"] = "steam",
7362     ["Decontamination Unit"] = "gas",
7363     ["Mutagenesis Chamber"] = "electrical",
7364     ["Hormone Treatment Bench"] = "liquid",
7365     ["Plant Growth Chamber"] = "gas",
7366     ["Growth Hormone Mixer"] = "liquid",
7367     ["GMO Injector"] = "electrical",
7368     ["Nutrient Infuser"] = "gas",
7369     ["UV Growth Scanner"] = "electrical",
7370     ["Pollination Station"] = "electrical",
7371 }
7372
7373 local function scatterDebris(part)
7374     for i = 1, math.random(4, 7) do
7375         local chunk = Instance.new("Part")
7376         chunk.Name = "Debris"
7377         chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
7378         chunk.Color = part.Color
7379         chunk.Material = Enum.Material.Metal
7380         chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2, math.random(-3,
7381         ↪ 3) / 2)
7382         chunk.Anchored = false
7383         chunk.CanCollide = true
7384         chunk.Parent = part.Parent
7385         local av = Instance.new("AngularVelocity")

```

```

7385     av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
7386     av.Parent = chunk
7387     local bv = Instance.new("BodyVelocity")
7388     bv.MaxForce = Vector3.new(4000, 4000, 4000)
7389     bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
7390     bv.Parent = chunk
7391     Debris:AddItem(bv, 0.25)
7392     Debris:AddItem(chunk, 8)
7393 end
7394 end
7395
7396 local destroyEquipment
7397
7398 local function warnEquipmentCatastrophe(part, floorLabel, unitLabel)
7399     if part:GetAttribute("AboutToBlow") then return end
7400     part:SetAttribute("AboutToBlow", true)
7401     local beacon = Instance.new("PointLight")
7402     beacon.Color = Color3.fromRGB(255, 40, 30)
7403     beacon.Brightness = 4
7404     beacon.Range = 16
7405     beacon.Parent = part
7406     local siren = Instance.new("Sound")
7407     siren.Name = "SirenWarning"
7408     siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
7409     siren.Looped = true
7410     siren.Volume = 3
7411     siren.PlaybackSpeed = 1.6
7412     siren.Parent = part
7413     siren:Play()
7414     for _, player in ipairs(Players:GetPlayers()) do
7415         notify(player, "ALARM: " .. floorLabel .. " " * " .. unitLabel .. " is about to fail catastrophically --
        ↳ MOVE AWAY NOW!")
7416     end
7417     task.spawn(function()
7418         local elapsed = 0
7419         while elapsed < 2 and part.Parent do
7420             beacon.Enabled = not beacon.Enabled
7421             task.wait(0.2)
7422             elapsed = elapsed + 0.2
7423         end
7424     end)
7425     task.delay(2, function()
7426         siren:Stop()
7427         siren:Destroy()
7428         beacon:Destroy()
7429         if part.Parent then destroyEquipment(part) end
7430     end)
7431 end
7432
7433 destroyEquipment = function(part)
7434     local floorLabel = equipmentFloorLabel(part)
7435     local unitLabel = equipmentLabel(part)
7436     local causes = {"exploded", "caught fire", "arced and shorted out"}
7437     local cause = causes[math.random(1, #causes)]
7438     part:SetAttribute("FunctionalState", "DESTROYED")
7439     part:SetAttribute("AboutToBlow", nil)
7440     part:SetAttribute("RepairInstruction", floorLabel .. " " * " .. unitLabel .. " " * DESTROYED, UNFIXABLE")
7441     equipmentReadout(part, "DESTROYED", floorLabel .. " " * " .. unitLabel .. " " .. cause .. " -- permanently
    ↳ offline",
7442         Color3.fromRGB(150, 40, 35))
7443     for _, ledPart in ipairs(part:GetDescendants()) do
7444         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
            ↳ Enum.Material.SmoothPlastic end
7445     end
7446     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
7447
7448     local bang = Instance.new("Sound")
7449     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
7450     bang.Volume = 4
7451     bang.PlaybackSpeed = 0.8
7452     bang.Parent = part
7453     bang:Play()
7454     Debris:AddItem(bang, 4)
7455
7456     local explosion = Instance.new("Explosion")
7457     explosion.Position = part.Position

```

```

7458     explosion.BlastRadius = 8
7459     explosion.BlastPressure = 0
7460     explosion.ExplosionType = Enum.ExplosionType.NoCraters
7461     explosion.DestroyJointRadiusPercent = 0
7462     explosion.Parent = workspace
7463
7464     local kindOfDestruction = DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
7465
7466     local fire = Instance.new("Fire")
7467     fire.Size = 6
7468     fire.Heat = 9
7469     fire.Color = Color3.fromRGB(255, 140, 40)
7470     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
7471     fire.Parent = part
7472     Debris:AddItem(fire, 16)
7473
7474     local sparkles = Instance.new("Sparkles")
7475     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
7476     sparkles.Parent = part
7477     Debris:AddItem(sparkles, 3)
7478
7479     local smoke = Instance.new("Smoke")
7480     smoke.Size = 8
7481     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
7482     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
7483     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
7484         or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
7485         or Color3.fromRGB(50, 50, 55)
7486     smoke.Parent = part
7487     Debris:AddItem(smoke, 20)
7488
7489     if kindOfDestruction == "liquid" then
7490         local puddle = Instance.new("Part")
7491         puddle.Name = "LeakPuddle"
7492         puddle.Shape = Enum.PartType.Cylinder
7493         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
7494         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05, part.Position.Z)
7495             * CFrame.Angles(0, 0, math.rad(90))
7496         puddle.Color = Color3.fromRGB(120, 90, 40)
7497         puddle.Material = Enum.Material.Neon
7498         puddle.Transparency = 0.35
7499         puddle.Anchored = true
7500         puddle.CanCollide = false
7501         puddle.Parent = part.Parent
7502         Debris:AddItem(puddle, 25)
7503     end
7504
7505     scatterDebris(part)
7506
7507     for _, player in ipairs(Players:GetPlayers()) do
7508         notify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause .. ". It
        ↳ cannot be repaired.")
7509     end
7510 end
7511
7512 task.spawn(function()
7513     while true do
7514         task.wait(1)
7515         local hazards = {}
7516         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
7517             local state = equip.Parent and equip:GetAttribute("FunctionalState")
7518             if state == "FAILED" then
7519                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
7520                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
7521                 -- Five minutes to reach the job before a rare catastrophic risk;
7522                 -- a unit under a player's addition check cannot self-destruct.
7523                 local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
7524                 if failedFor > 300 and not equip:GetAttribute("QuizPlayerId") and math.random(1, 600) == 1 then
7525                     warnEquipmentCatastrophe(equip, equipmentFloorLabel(equip), equipmentLabel(equip))
7526                 end
7527             elseif state == "DESTROYED" then
7528                 local info = HAZARD_BY_KIND[equip:GetAttribute("Kind")]
7529                 if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
                    ↳ info[2] + 2}) end
7530             end
7531         end

```

```

7532     if #hazards > 0 then
7533         for _, player in ipairs(Players:GetPlayers()) do
7534             local character = player.Character
7535             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
7536             local hrp = character and character:FindFirstChild("HumanoidRootPart")
7537             if humanoid and hrp and humanoid.Health > 0 then
7538                 local worst
7539                 for _, hazard in ipairs(hazards) do
7540                     local d = (hazard.part.Position - hrp.Position).Magnitude
7541                     if d < HAZARD_RANGE and hazard.part:GetAttribute("QuizPlayerId") ~= player.UserId
7542                        and (not worst or hazard.dps > worst.dps) then worst = hazard end
7543                 end
7544                 if worst then
7545                     humanoid:TakeDamage(worst.dps)
7546                     local last = hazardWarnedAt[player] or 0
7547                     if os.clock() - last > 2 then
7548                         hazardWarnedAt[player] = os.clock()
7549                         notify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
7550                             ↪ worst.label, math.floor(humanoid.Health)))
7550                     end
7551                 end
7552             end
7553         end
7554     end
7555 end)
7556 end)
7557
7558 for _, p in ipairs(CollectionService:GetTagged("LabToolDispenser")) do bindDispenser(p) end
7559 CollectionService:GetInstanceAddedSignal("LabToolDispenser"):Connect(bindDispenser)
7560
7561 local testBots = CollectionService:GetTagged("LabWorkerBot")
7562 table.sort(testBots, function(a, b) return (a:GetAttribute("BotNumber") or 0) < (b:GetAttribute("BotNumber") or
7563     ↪ 0) end)
7564 for index, bot in ipairs(testBots) do
7565     -- Stagger WITHIN each floor, not across the whole building. Staggering on
7566     -- the global index at 0.32s a bot meant the top floor did not begin its
7567     -- loop until nearly a minute after the server started, so the upper
7568     -- levels looked completely frozen to anyone who walked up there early.
7569     -- Keyed off the per-floor slot, every floor is fully awake in ~7s.
7570     -- Only bots someone is standing next to will actually compute a path now,
7571     -- so there is no thundering herd to spread out and the stagger can be
7572     -- short. Every floor is ready ~1.6s in instead of ~58s.
7573     local slot = bot:GetAttribute("LevelSlot") or index
7574     task.delay(slot * 0.08, function()
7575         if bot.Parent then runBotScenario(bot) end
7576     end)
7577 end
7578
7579 -- Bodies stay put for the whole shift. LabRound stamps RoundNumber on
7580 -- Workspace when the shift rolls over; that is the cue to clear them away and
7581 -- put every worker back on its feet for a fresh hunt.
7582 local function resetHunt()
7583     local bodies = workspace:FindFirstChild("LabBotBodies")
7584     if bodies then bodies:ClearAllChildren() end
7585
7586     local complex = workspace:FindFirstChild("Laboratory_Complex")
7587     if complex then complex:SetAttribute("BotsKilled", 0) end
7588
7589     local restaged = 0
7590     for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
7591         if bot:GetAttribute("Dead") then
7592             local home = bot:GetAttribute("HomeCFrame")
7593             if typeof(home) == "CFrame" then bot:PivotTo(home) end
7594             bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
7595             bot:SetAttribute("KnockedOut", false)
7596             bot:SetAttribute("Dead", false)
7597             bot:SetAttribute("TaskState", "Waiting for plant task")
7598             refreshBotHealth(bot)
7599             -- its old scenario loop exited on death, so start a fresh one
7600             task.spawn(runBotScenario, bot)
7601             restaged = restaged + 1
7602         end
7603     end
7604     if restaged > 0 then
7605         print("Shift reset: cleared bodies and restaged " .. restaged .. " workers.")
7606     end
7607 end

```

```

7606 end
7607 workspace:GetAttributeChangedSignal("RoundNumber"):Connect(resetHunt)
7608
7609 -----
7610 -- ROOM SMOKE
7611 -- Small rooms slowly fill with smoke. Opening a door vents it. Capped so
7612 -- there are never more than a handful of smoke volumes alive at once.
7613 -----
7614 local MAX_SMOKY_ROOMS = 4
7615 local smokeByRoom = {}
7616
7617 local function clearRoomSmoke(key)
7618     local folder = smokeByRoom[key]
7619     if not folder then return end
7620     smokeByRoom[key] = nil
7621     for _, d in ipairs(folder:GetDescendants()) do
7622         if d:IsA("Smoke") then d.Enabled = false end
7623     end
7624     Debris:AddItem(folder, 5) -- let what is already in the air disperse
7625 end
7626
7627 local function fillRoomWithSmoke(levelName, rect)
7628     local key = levelName .. "/" .. rect.name
7629     if smokeByRoom[key] then return end
7630     local folder = Instance.new("Folder")
7631     folder.Name = "RoomSmoke"
7632     folder:SetAttribute("RoomKey", key)
7633     folder.Parent = workspace
7634     for i = 1, 5 do
7635         local emitter = Instance.new("Part")
7636         emitter.Name = "SmokeSource"
7637         emitter.Size = Vector3.new(1, 1, 1)
7638         emitter.Anchored = true
7639         emitter.CanCollide = false
7640         emitter.CanQuery = false
7641         emitter.CanTouch = false
7642         emitter.Transparency = 1
7643         emitter.CFrame = CFrame.new(
7644             rect.x1 + (rect.x2 - rect.x1) * (i - 0.5) / 5,
7645             rect.y + 3,
7646             rect.z1 + (rect.z2 - rect.z1) * ((i * 2) % 5 + 0.5) / 5)
7647         emitter.Parent = folder
7648         local smoke = Instance.new("Smoke")
7649         smoke.Size = 11
7650         smoke.Opacity = 0.32
7651         smoke.RiseVelocity = 2
7652         smoke.Color = Color3.fromRGB(150, 150, 152)
7653         smoke.Parent = emitter
7654     end
7655     smokeByRoom[key] = folder
7656 end
7657
7658 local function ventFromDoor(door)
7659     local pos = door.Position
7660     for key in pairs(smokeByRoom) do
7661         local levelName, roomName = key:match("^(.)/(.+)$")
7662         local rect = roomRect(levelName, roomName)
7663         -- a door on the room's own boundary counts, hence the small margin
7664         if rect and math.abs(pos.Y - rect.y) < FLOOR_BAND
7665             and pos.X >= rect.x1 - 8 and pos.X <= rect.x2 + 8
7666             and pos.Z >= rect.z1 - 8 and pos.Z <= rect.z2 + 8 then
7667             clearRoomSmoke(key)
7668         end
7669     end
7670 end
7671
7672 local function watchDoor(door)
7673     door:GetAttributeChangedSignal("DoorOpen"):Connect(function()
7674         if door:GetAttribute("DoorOpen") then ventFromDoor(door) end
7675     end)
7676 end
7677 for _, door in ipairs(CollectionService:GetTagged("AutoDoor")) do watchDoor(door) end
7678 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(watchDoor)
7679
7680 task.spawn(function()
7681     while true do

```

```

7682     task.wait(45)
7683     local live = 0
7684     for _ in pairs(smokeByRoom) do live = live + 1 end
7685     if live < MAX_SMOKY_ROOMS then
7686         local levels = {}
7687         for levelName in pairs(roomsByLevel) do table.insert(levels, levelName) end
7688         if #levels > 0 then
7689             local levelName = levels[math.random(1, #levels)]
7690             -- only the small east bays; the long west room is not a "small room"
7691             local small = {}
7692             for _, r in ipairs(roomsByLevel[levelName]) do
7693                 if (r.x2 - r.x1) < 90 and (r.z2 - r.z1) < 90 then table.insert(small, r) end
7694             end
7695             if #small > 0 then
7696                 fillRoomWithSmoke(levelName, small[math.random(1, #small)])
7697             end
7698         end
7699     end
7700 end
7701 end)
7702 task.spawn(function()
7703     while task.wait(2) do updateBotTestBoard() end
7704 end)
7705 task.spawn(function()
7706     while task.wait(2) do refreshRoomMonitors() end
7707 end)
7708 task.spawn(function()
7709     task.wait(3)
7710     while true do
7711         adaptiveFailureTick()
7712         genericFailureTick()
7713         task.wait(45)
7714     end
7715 end)
7716
7717 print("Equipment armed: " .. #CollectionService:GetTagged("LabEquipment")
7718       .. " items; recovery tool pads: " .. #CollectionService:GetTagged("LabToolDispenser") .. ".")
7719
7720 -----
7721 -- LAB STAFF NPCs
7722 -----
7723 local function bindNPC(part)
7724     if part:GetAttribute("TalkBound") then return end
7725     part:SetAttribute("TalkBound", true)
7726     local tipsStr = part:GetAttribute("Tips") or ""
7727     local tips = {}
7728     for t in tipsStr:gmatch("[^|]+") do table.insert(tips, t) end
7729     local label = part:GetAttribute("NPCName") or part:GetAttribute("Role") or "Lab Staff"
7730
7731     local prompt = Instance.new("ProximityPrompt")
7732     prompt.ObjectText = label
7733     prompt.ActionText = "Talk"
7734     prompt.HoldDuration = 0.3
7735     prompt.MaxActivationDistance = 10
7736     prompt.RequiresLineOfSight = false
7737     prompt.Parent = part
7738
7739     prompt.Triggered:Connect(function(player)
7740         if not canInteract(player, part, prompt.MaxActivationDistance) then return end
7741         if #tips == 0 then return end
7742         notify(player, label .. ": " .. tips[math.random(1, #tips)])
7743     end)
7744 end
7745
7746 for _, p in ipairs(CollectionService:GetTagged("LabNPC")) do bindNPC(p) end
7747 CollectionService:GetInstanceAddedSignal("LabNPC"):Connect(bindNPC)
7748
7749 print("Lab staff NPCs armed: " .. #CollectionService:GetTagged("LabNPC") .. ".")
7750 ]==]
7751
7752 local BLASTER_SOURCE = [====[
7753 -- Blaster
7754 -- Shared, device-independent camera casting and client shot presentation.
7755
7756 local Debris = game:GetService("Debris")
7757 local ReplicatedStorage = game:GetService("ReplicatedStorage")

```

```

7758 local RunService = game:GetService("RunService")
7759
7760 local Blaster = {}
7761
7762 Blaster.REMOTE_FOLDER_NAME = "BlasterRemotes"
7763 Blaster.FIRE_REMOTE_NAME = "Fire"
7764 Blaster.EFFECT_REMOTE_NAME = "ReplicateShot"
7765
7766 local replicationConnection
7767
7768 local function finiteNumber(value)
7769     return typeof(value) == "number"
7770         and value == value
7771         and value > -1e7
7772         and value < 1e7
7773 end
7774
7775 function Blaster.IsFiniteVector(value)
7776     return typeof(value) == "Vector3"
7777         and finiteNumber(value.X)
7778         and finiteNumber(value.Y)
7779         and finiteNumber(value.Z)
7780 end
7781
7782 local function numberAttribute(tool, name, fallback, minimum, maximum)
7783     local value = tool and tool:GetAttribute(name)
7784     if not finiteNumber(value) then value = fallback end
7785     return math.clamp(value, minimum, maximum)
7786 end
7787
7788 function Blaster.GetWeaponConfig(tool)
7789     return {
7790         damage = numberAttribute(tool, "BlasterDamage", 25, 0, 1000),
7791         range = numberAttribute(tool, "BlasterRange", 250, 8, 1000),
7792         cooldown = numberAttribute(tool, "BlasterCooldown", 0.35, 0.06, 10),
7793         pellets = math.floor(numberAttribute(tool, "BlasterPellets", 1, 1, 16)),
7794         spread = numberAttribute(tool, "BlasterSpread", 0, 0, 0.35),
7795         selfCostPercent = numberAttribute(tool, "BlasterSelfCostPercent", 0, 0, 25),
7796         adsFov = numberAttribute(tool, "BlasterADSFOV", 55, 25, 100),
7797         automatic = tool and tool:GetAttribute("BlasterAutomatic") == true or false,
7798         instantHeadshot = tool and tool:GetAttribute("BlasterInstantHeadshot") == true or false,
7799         allowPlayerDamage = tool and tool:GetAttribute("BlasterAllowPlayerDamage") == true or false,
7800     }
7801 end
7802
7803 function Blaster.GetCameraRay(camera, maxRange, screenPoint)
7804     camera = camera or workspace.CurrentCamera
7805     if not camera then return nil end
7806     maxRange = math.clamp(tonumber(maxRange) or 500, 1, 2000)
7807     if typeof(screenPoint) == "Vector2" then
7808         -- Desktop points are expressed in CoreUISafeInsets screen space (the
7809         -- mouse location minus GuiService:GetGuiInset()). ScreenPointToRay is
7810         -- the matching inverse transform. ViewportPointToRay uses device-safe
7811         -- coordinates instead and produces a vertical offset on PC and in the
7812         -- phone simulator.
7813         local ray = camera:ScreenPointToRay(screenPoint.X, screenPoint.Y)
7814         if ray.Direction.Magnitude < 0.99 then return nil end
7815         return ray.Origin, ray.Direction.Unit, ray.Direction.Unit * maxRange
7816     end
7817
7818     local direction = camera.CFrame.LookVector
7819     if direction.Magnitude < 0.99 then return nil end
7820
7821     return camera.CFrame.Position, direction.Unit, direction.Unit * maxRange
7822 end
7823
7824 function Blaster.NewRaycastParams(ignoreList)
7825     local params = RaycastParams.new()
7826     params.FilterType = Enum.RaycastFilterType.Exclude
7827     params.FilterDescendantsInstances = ignoreList or {}
7828     params.IgnoreWater = true
7829     return params
7830 end
7831
7832 function Blaster.CastFromCamera(camera, maxRange, ignoreList, screenPoint)
7833     local origin, unitDirection, castDirection = Blaster.GetCameraRay(camera, maxRange, screenPoint)

```

```

7834     if not origin then return nil end
7835
7836     local result = workspace:Raycast(origin, castDirection, Blaster.NewRaycastParams(ignoreList))
7837     return {
7838         origin = origin,
7839         direction = unitDirection,
7840         result = result,
7841         position = result and result.Position or (origin + castDirection),
7842     }
7843 end
7844
7845 -- Returns the exact centre-line solution used for both the live reticle and a
7846 -- shot. The camera chooses the intended point; the muzzle ray then catches a
7847 -- nearby wall or prop that physically blocks the weapon.
7848 function Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
7849     if not tool or not tool:IsA("Tool") then return nil end
7850     local config = Blaster.GetWeaponConfig(tool)
7851     local cast = Blaster.CastFromCamera(camera, config.range, ignoreList, screenPoint)
7852     if not cast then return nil end
7853
7854     local handle = tool:FindFirstChild("Handle")
7855     local muzzleOrigin = handle and handle.Position or cast.origin
7856     local muzzleDelta = cast.position - muzzleOrigin
7857     local muzzleResult
7858     if muzzleDelta.Magnitude > 0.05 then
7859         -- Extend just beyond the camera hit point. Ending exactly on a surface can
7860         -- miss it because of floating-point/raycast endpoint precision.
7861         muzzleResult = workspace:Raycast(
7862             muzzleOrigin,
7863             muzzleDelta.Unit * (muzzleDelta.Magnitude + 0.25),
7864             Blaster.NewRaycastParams(ignoreList)
7865         )
7866     end
7867
7868     return {
7869         cameraOrigin = cast.origin,
7870         direction = cast.direction,
7871         muzzleOrigin = muzzleOrigin,
7872         position = muzzleResult and muzzleResult.Position or cast.position,
7873         result = muzzleResult,
7874         cameraResult = cast.result,
7875     }
7876 end
7877
7878 local function tracerColor(tool)
7879     local value = tool and tool:GetAttribute("BlasterTracerColor")
7880     return typeof(value) == "Color3" and value or Color3.fromRGB(255, 226, 140)
7881 end
7882
7883 function Blaster.DrawTracer(origin, hitPosition, color, lifetime)
7884     if not Blaster.IsFiniteVector(origin) or not Blaster.IsFiniteVector(hitPosition) then return end
7885     local distance = (hitPosition - origin).Magnitude
7886     if distance < 0.05 then return end
7887
7888     local beam = Instance.new("Part")
7889     beam.Name = "BlasterTracer"
7890     beam.Size = Vector3.new(0.055, 0.055, distance)
7891     beam.CFrame = CFrame.lookAt((origin + hitPosition) * 0.5, hitPosition)
7892     beam.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
7893     beam.Material = Enum.Material.Neon
7894     beam.Transparency = 0.08
7895     beam.Anchored = true
7896     beam.CanCollide = false
7897     beam.CanTouch = false
7898     beam.CanQuery = false
7899     beam.CastShadow = false
7900     beam.Parent = workspace
7901     Debris.AddItem(beam, lifetime or 0.075)
7902 end
7903
7904 function Blaster.DrawImpact(position, normal, color, lifetime)
7905     if not Blaster.IsFiniteVector(position) or not Blaster.IsFiniteVector(normal) then return end
7906
7907     local impact = Instance.new("Part")
7908     impact.Name = "BlasterImpact"
7909     impact.Shape = Enum.PartType.Ball

```



```

7910     impact.Size = Vector3.new(0.16, 0.16, 0.16)
7911     impact.Position = position + normal * 0.04
7912     impact.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
7913     impact.Material = Enum.Material.Neon
7914     impact.Anchored = true
7915     impact.CanCollide = false
7916     impact.CanTouch = false
7917     impact.CanQuery = false
7918     impact.CastShadow = false
7919     impact.Parent = workspace
7920     Debris.AddItem(impact, lifetime or 0.65)
7921 end
7922
7923 local function getRemotes(timeout)
7924     local folder = ReplicatedStorage.WaitForChild(Blaster.REMOTE_FOLDER_NAME, timeout or 10)
7925     if not folder then return nil end
7926     local fireRemote = folder.WaitForChild(Blaster.FIRE_REMOTE_NAME, timeout or 10)
7927     local effectRemote = folder.WaitForChild(Blaster.EFFECT_REMOTE_NAME, timeout or 10)
7928     return fireRemote, effectRemote
7929 end
7930
7931 function Blaster.StartReplication()
7932     if not RunService:IsClient() or replicationConnection then return replicationConnection end
7933     local _, effectRemote = getRemotes(15)
7934     if not effectRemote or not effectRemote:IsA("RemoteEvent") then return nil end
7935
7936     replicationConnection = effectRemote.OnClientEvent:Connect(function(payload)
7937         if typeof(payload) ~= "table" then return end
7938         if not Blaster.IsFiniteVector(payload.origin) or not Blaster.IsFiniteVector(payload.position) then
7939             ↪ return end
7939         Blaster.DrawTracer(payload.origin, payload.position, payload.color, 0.075)
7940         if Blaster.IsFiniteVector(payload.normal) then
7941             Blaster.DrawImpact(payload.position, payload.normal, payload.color)
7942         end
7943     end)
7944     return replicationConnection
7945 end
7946
7947 function Blaster.Fire(tool, camera, sequence, screenPoint)
7948     assert(RunService:IsClient(), "Blaster.Fire can only be called by a client")
7949     if not tool or not tool:IsA("Tool") then return nil end
7950
7951     local player = game.GetService("Players").LocalPlayer
7952     local character = player and player.Character
7953     if not character or tool.Parent ~= character then return nil end
7954
7955     local ignoreList = {character, tool, workspace.CurrentCamera}
7956     local solution = Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
7957     if not solution then return nil end
7958
7959     Blaster.DrawTracer(solution.muzzleOrigin, solution.position, tracerColor(tool), 0.11)
7960     if solution.result then
7961         Blaster.DrawImpact(solution.position, solution.result.Normal, tracerColor(tool), 0.8)
7962     end
7963
7964     local fireRemote = getRemotes(10)
7965     if fireRemote and fireRemote:IsA("RemoteEvent") then
7966         fireRemote:FireServer({
7967             version = 1,
7968             tool = tool,
7969             cameraOrigin = solution.cameraOrigin,
7970             direction = solution.direction,
7971             sequence = sequence,
7972             clientTime = workspace:GetServerTimeNow(),
7973         })
7974     end
7975
7976     return solution
7977 end
7978
7979 return Blaster
7980
7981 ]====]
7982
7983 local BLASTER_CONTROLLER_SOURCE = [====[
7984 -- BlasterController

```

```

7985 -- Local Tool controller: ContextActionService input, mobile controls, ADS,
7986 -- touch aim damping, camera recoil, crosshair, and weapon bob.
7987
7988 local ContextActionService = game:GetService("ContextActionService")
7989 local GuiService = game:GetService("GuiService")
7990 local Players = game:GetService("Players")
7991 local ReplicatedStorage = game:GetService("ReplicatedStorage")
7992 local RunService = game:GetService("RunService")
7993 local TweenService = game:GetService("TweenService")
7994 local UserInputService = game:GetService("UserInputService")
7995
7996 local Blaster = require(script.Parent:WaitForChild("Blaster"))
7997 local localPlayer = Players.LocalPlayer
7998
7999 local BlasterController = {}
8000 BlasterController.__index = BlasterController
8001
8002 local controllers = setmetatable({}, {__mode = "k"})
8003 local nextControllerId = 0
8004
8005 local function touchControlsActive()
8006     if UserInputService.TouchEnabled then return true end
8007     local okTouchScreen, touchScreen = pcall(function()
8008         return UserInputService.TouchScreenEnabled
8009     end)
8010     if okTouchScreen and touchScreen then return true end
8011     local okPreferred, preferred = pcall(function()
8012         return UserInputService.PreferredInput
8013     end)
8014     if okPreferred and tostring(preferred):find("Touch") then return true end
8015     local okControls, controls = pcall(function()
8016         return GuiService.TouchControlsEnabled
8017     end)
8018     if okControls and controls then return true end
8019
8020     -- Studio can retain desktop input flags when a phone preset is selected
8021     -- after the client starts. Recognize the simulator's compact landscape
8022     -- viewport so the weapon does not silently lose its mobile camera/buttons.
8023     local camera = workspace.CurrentCamera
8024     local viewport = camera and camera.ViewportSize
8025     return RunService:IsStudio()
8026         and viewport ~= nil
8027         and viewport.Y <= 450
8028         and viewport.X / math.max(1, viewport.Y) >= 1.45
8029 end
8030
8031 local function pointInside(button, point)
8032     if not button or not button.Visible then return false end
8033     local p = button.AbsolutePosition
8034     local s = button.AbsoluteSize
8035     return point.X >= p.X and point.X <= p.X + s.X
8036         and point.Y >= p.Y and point.Y <= p.Y + s.Y
8037 end
8038
8039 -- Hardware support does not identify the active input on an emulator/hybrid.
8040 local function touchAimActive()
8041     local input = UserInputService:GetLastInputType()
8042     if input == Enum.UserInputType.Touch then return true end
8043     if input == Enum.UserInputType.MouseMovement
8044         or input == Enum.UserInputType.MouseButton1
8045         or input == Enum.UserInputType.MouseButton2
8046         or input == Enum.UserInputType.MouseButton3
8047         or input == Enum.UserInputType.MouseWheel
8048         or input == Enum.UserInputType.Keyboard then
8049         return false
8050     end
8051     return touchControlsActive()
8052 end
8053
8054 local function rotationBetween(from, to)
8055     local dot = math.clamp(from:Dot(to), -1, 1)
8056     local axis = from:Cross(to)
8057     if axis.Magnitude < 0.00001 then
8058         if dot > 0 then return CFrame.identity end
8059         axis = from:Cross(Vector3.yAxis)
8060         if axis.Magnitude < 0.00001 then axis = from:Cross(Vector3.xAxis) end

```

```

8061     end
8062     return CFrame.fromAxisAngle(axis.Unit, math.acos(dot))
8063 end
8064
8065 local function makeCrosshair(hudName)
8066     local playerGui = localPlayer.WaitForChild("PlayerGui")
8067     local old = playerGui:FindFirstChild(hudName)
8068     if old then old:Destroy() end
8069
8070     local gui = Instance.new("ScreenGui")
8071     gui.Name = hudName
8072     gui.ResetOnSpawn = false
8073     gui.IgnoreGuiInset = true
8074     gui.ScreenInsets = Enum.ScreenInsets.None
8075     gui.ClipToDeviceSafeArea = false
8076     gui.DisplayOrder = 12
8077     gui.Enabled = false
8078     gui.Parent = playerGui
8079
8080     local root = Instance.new("Frame")
8081     root.Name = "Crosshair"
8082     root.AnchorPoint = Vector2.new(0.5, 0.5)
8083     root.Position = UDim2.fromScale(0.5, 0.5)
8084     root.Size = UDim2.fromOffset(30, 30)
8085     root.BackgroundTransparency = 1
8086     root.Parent = gui
8087
8088     local dot = Instance.new("Frame")
8089     dot.Name = "Dot"
8090     dot.AnchorPoint = Vector2.new(0.5, 0.5)
8091     dot.Position = UDim2.fromScale(0.5, 0.5)
8092     dot.Size = UDim2.fromOffset(6, 6)
8093     dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
8094     dot.BorderSizePixel = 0
8095     dot.Parent = root
8096     local dotCorner = Instance.new("UICorner")
8097     dotCorner.CornerRadius = UDim.new(1, 0)
8098     dotCorner.Parent = dot
8099     local dotStroke = Instance.new("UIStroke")
8100     dotStroke.Color = Color3.fromRGB(6, 15, 22)
8101     dotStroke.Transparency = 0.15
8102     dotStroke.Thickness = 1.5
8103     dotStroke.Parent = dot
8104
8105     for index, data in ipairs({
8106         {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
8107         {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
8108         {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
8109         {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
8110     }) do
8111         local line = Instance.new("Frame")
8112         line.Name = "Line" .. index
8113         line.Position = data[1]
8114         line.Size = data[2]
8115         line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
8116         line.BackgroundTransparency = 0.08
8117         line.BorderSizePixel = 0
8118         line.Parent = root
8119     end
8120
8121     local status = Instance.new("TextLabel")
8122     status.Name = "Status"
8123     status.AnchorPoint = Vector2.new(0.5, 0)
8124     status.Position = UDim2.new(0.5, 0, 0.5, 25)
8125     status.Size = UDim2.fromOffset(210, 24)
8126     status.BackgroundTransparency = 1
8127     status.Font = Enum.Font.GothamBold
8128     status.Text = ""
8129     status.TextColor3 = Color3.fromRGB(220, 232, 242)
8130     status.TextSize = 13
8131     status.TextStrokeTransparency = 0.5
8132     status.Parent = gui
8133
8134     return gui, root, status, dot
8135 end
8136

```

```

8137 function BlasterController:_ensureHUD()
8138     if self.hud and self.hud.Parent and self.crosshair and self.crosshair.Parent == self.hud
8139         and self.reticleDot and self.reticleDot.Parent == self.crosshair then return false end
8140     if self.hud then self.hud:Destroy() end
8141     self.fallbackShootButton = nil
8142     self.fallbackAimButton = nil
8143     self.hud, self.crosshair, self.statusLabel, self.reticleDot = makeCrosshair("BlasterHUD_" .. self.id)
8144     self.hud.Enabled = self.enabled
8145     return true
8146 end
8147
8148 function BlasterController.new(tool, overrides)
8149     assert(RunService:IsClient(), "BlasterController must be required on the client")
8150     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
8151
8152     nextControllerId = nextControllerId + 1
8153     local self = setmetatable({}, BlasterController)
8154     self.tool = tool
8155     self.overrides = overrides or {}
8156     self.id = nextControllerId
8157     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
8158     self.connections = {}
8159     self.enabled = false
8160     self.aiming = false
8161     self.triggerHeld = false
8162     self.fireLoopRunning = false
8163     self.sequence = 0
8164     self.nextLocalShot = 0
8165     self.baseGrip = tool.Grip
8166     self.recoilPitch = 0
8167     self.recoilYaw = 0
8168     self.touchInput = nil
8169     self.touchLastPosition = nil
8170     self.hiddenParts = {}
8171     self.shoulderOffset = Vector3.zero
8172
8173     self.connections.equipped = tool.Equipped:Connect(function()
8174         self:Enable()
8175     end)
8176     self.connections.unequipped = tool.Unequipped:Connect(function()
8177         self:Disable()
8178     end)
8179     self.connections.destroying = tool.Destroying:Connect(function()
8180         self:Destroy()
8181     end)
8182     self.connections.ancestry = tool.AncestryChanged:Connect(function()
8183         if self.enabled and tool.Parent ~= localPlayer.Character then
8184             self:Disable()
8185         end
8186     end)
8187     -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
8188     -- remains the primary input layer, while this fallback covers touch/controller layouts
8189     -- that route their built-in Tool button directly to Activated.
8190     self.connections.activated = tool.Activated:Connect(function()
8191         if self.enabled then self:_beginFiring() end
8192     end)
8193     self.connections.deactivated = tool.Deactivated:Connect(function()
8194         self.triggerHeld = false
8195     end)
8196
8197     if localPlayer.Character and tool.Parent == localPlayer.Character then
8198         task.defer(function() self:Enable() end)
8199     end
8200
8201     return self
8202 end
8203
8204 function BlasterController:_button(actionName)
8205     local ok, button = pcall(function()
8206         return ContextActionService:GetButton(actionName)
8207     end)
8208     return ok and button or nil
8209 end
8210
8211 function BlasterController:_fallbackTouchButton(kind, title)
8212     local field = kind == "Shoot" and "fallbackShootButton" or "fallbackAimButton"

```

```

8213     local existing = self[field]
8214     if existing and existing.Parent then return existing end
8215     if not self.hud then return nil end
8216
8217     local button = Instance.new("TextButton")
8218     button.Name = "Fallback" .. kind
8219     button.AutoButtonColor = true
8220     button.BackgroundColor3 = Color3.fromRGB(35, 48, 60)
8221     button.BorderSizePixel = 0
8222     button.Text = title
8223     button.Font = Enum.Font.GothamBold
8224     button.TextColor3 = Color3.fromRGB(245, 250, 255)
8225     button.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
8226     button.TextStrokeTransparency = 0.25
8227     button.TextScaled = false
8228     button.ZIndex = 20
8229     button.Parent = self.hud
8230     self[field] = button
8231
8232     if kind == "Shoot" then
8233         button.InputBegan:Connect(function(input)
8234             if input.UserInputType == Enum.UserInputType.Touch
8235             or input.UserInputType == Enum.UserInputType.MouseButton1 then
8236                 self:_shootAction(nil, Enum.UserInputState.Begin)
8237             end
8238         end)
8239         button.InputEnded:Connect(function(input)
8240             if input.UserInputType == Enum.UserInputType.Touch
8241             or input.UserInputType == Enum.UserInputType.MouseButton1 then
8242                 self:_shootAction(nil, Enum.UserInputState.End)
8243             end
8244         end)
8245     else
8246         button.Activated:Connect(function()
8247             if self.enabled then self:SetAiming(not self.aiming) end
8248         end)
8249     end
8250     return button
8251 end
8252
8253 function BlasterController:_styleTouchButtons()
8254     if not touchControlsActive() then return end
8255     local camera = workspace.CurrentCamera
8256     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
8257     local playerGui = localPlayer:FindFirstChildOfClass("PlayerGui")
8258     local touchGui = playerGui and playerGui:FindFirstChild("TouchGui")
8259     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
8260     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 52, 80)
8261     if jump and jump.AbsoluteSize.X > 0 then
8262         diameter = math.clamp(math.floor(math.min(jump.AbsoluteSize.X, jump.AbsoluteSize.Y) * 0.85), 48, 80)
8263     end
8264
8265     -- Own the visuals/hit targets; CAS still binds desktop/controller actions.
8266     local shootButton = self:_fallbackTouchButton("Shoot", "FIRE")
8267     local aimButton = self:_fallbackTouchButton("Aim", "AIM")
8268     for _, action in ipairs({"Shoot", "Aim"}) do
8269         local casButton = self:_button(self.actionPrefix .. action)
8270         if casButton then casButton.Visible = false end
8271     end
8272     for _, button in ipairs({shootButton, aimButton}) do
8273         if button then
8274             button.Visible = true
8275             button.Size = UDim2.fromOffset(diameter, diameter)
8276             button.TextSize = math.floor(diameter * 0.25)
8277             button.BackgroundTransparency = 0.18
8278             if button:IsA("ImageButton") then
8279                 button.ImageTransparency = 0.08
8280                 button.ImageColor3 = Color3.fromRGB(35, 48, 60)
8281             end
8282             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
8283             corner.CornerRadius = UDim.new(1, 0)
8284             corner.Parent = button
8285             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
8286             stroke.Color = Color3.fromRGB(220, 235, 245)
8287             stroke.Transparency = 0.25
8288             stroke.Thickness = 2

```

```

8289         stroke.Parent = button
8290         local title = button:FindFirstChild("ActionTitle")
8291         if title and title:IsA("TextLabel") then
8292             title.Font = Enum.Font.GothamBold
8293             title.TextColor3 = Color3.fromRGB(245, 250, 255)
8294             title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
8295             title.TextStrokeTransparency = 0.25
8296             title.TextScaled = true
8297         end
8298     end
8299 end
8300
8301 -- GUI absolute coordinates include the simulator's safe-area translation.
8302 -- Anchor to Roblox's real JUMP rectangle, never an estimated viewport point.
8303 local gap = math.max(12, math.floor(diameter * 0.22))
8304 local jumpSize = jump and jump.AbsoluteSize or Vector2.new(diameter, diameter)
8305 local jumpPoint = jump and jump.AbsolutePosition
8306     or (self.hud.AbsolutePosition + self.hud.AbsoluteSize - jumpSize - Vector2.new(28, 28))
8307 local shootPoint = Vector2.new(jumpPoint.X + (jumpSize.X - diameter) * 0.5, jumpPoint.Y - diameter - gap)
8308 local aimPoint = Vector2.new(jumpPoint.X - diameter - gap, jumpPoint.Y + (jumpSize.Y - diameter) * 0.5)
8309 local function place(button, point)
8310     if not button or not button.Parent then return end
8311     button.AnchorPoint = Vector2.zero
8312     local origin = button.Parent.AbsolutePosition
8313     button.Position = UDim2.fromOffset(
8314         math.floor(point.X - origin.X + 0.5),
8315         math.floor(point.Y - origin.Y + 0.5)
8316     )
8317 end
8318 place(shootButton, shootPoint)
8319 place(aimButton, aimPoint)
8320 self.lastJumpPosition = jump and jump.AbsolutePosition
8321 end
8322
8323 function BlasterController:_getAimScreenPoint(camera)
8324     if not camera then return nil end
8325     if touchAimActive() then
8326         -- Mobile aims along the Scriptable camera's true forward vector. In the
8327         -- Studio phone simulator, ViewportPointToRay(viewportCenter) is offset by
8328         -- the asymmetric safe-area canvas and can land several degrees above the
8329         -- visible centre marker.
8330         return nil
8331     end
8332     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
8333     -- Clicking an on-screen action must not pull the target onto that button.
8334     if pointInside(self.fallbackShootButton, point) or pointInside(self.fallbackAimButton, point) then
8335         return self.lastPointerAim or camera.ViewportSize * 0.5
8336     end
8337     self.lastPointerAim = point
8338     return Vector2.new(
8339         math.clamp(point.X, 0, camera.ViewportSize.X),
8340         math.clamp(point.Y, 0, camera.ViewportSize.Y)
8341     )
8342 end
8343
8344 function BlasterController:_updateAimReticle(camera)
8345     if not self.hud or not self.crosshair or not camera then return end
8346     local rayScreenPoint = self:_getAimScreenPoint(camera)
8347     local screenPoint = rayScreenPoint
8348     local character = localPlayer.Character
8349     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
8350         character,
8351         self.tool,
8352         camera,
8353     }, rayScreenPoint)
8354
8355     if rayScreenPoint == nil then
8356         -- The shot uses Camera.CFrame.LookVector. Project that same forward
8357         -- vector with WorldToScreenPoint so the marker lands in the exact CoreUI
8358         -- coordinate space used by this ScreenGui, including notches/top bars.
8359         local projected = camera:WorldToScreenPoint(
8360             camera.CFrame.Position + camera.CFrame.LookVector * 100
8361         )
8362         screenPoint = Vector2.new(projected.X, projected.Y)
8363     elseif not screenPoint then
8364         screenPoint = camera.ViewportSize * 0.5

```

```

8365     end
8366
8367     if solution then
8368         if self.reticleDot then
8369             self.reticleDot.BackgroundColor3 = solution.result
8370             and Color3.fromRGB(255, 224, 105)
8371             or Color3.fromRGB(120, 235, 255)
8372         end
8373     end
8374
8375     -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
8376     -- and begin at a negative absolute position. Counter that offset so this
8377     -- marker identifies the muzzle-checked world hit on every device.
8378     local origin = self.hud.AbsolutePosition
8379     local center = screenPoint - origin
8380     self.crosshair.Position = UDim2.fromOffset(
8381         math.floor(center.X + 0.5),
8382         math.floor(center.Y + 0.5)
8383     )
8384 end
8385
8386 function BlasterController:_setLocalHeadHidden(hidden)
8387     local character = localPlayer.Character
8388     if not character then return end
8389     if hidden then
8390         for _, descendant in ipairs(character:GetDescendants()) do
8391             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
8392                 then
8393                 if self.hiddenParts[descendant] == nil then
8394                     self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
8395                 end
8396                 descendant.LocalTransparencyModifier = 1
8397             end
8398         end
8399     else
8400         for part, transparency in pairs(self.hiddenParts) do
8401             if part.Parent then part.LocalTransparencyModifier = transparency end
8402         end
8403         table.clear(self.hiddenParts)
8404     end
8405 end
8406
8407 function BlasterController:_beginTouchCamera()
8408     if not touchAimActive() or self.mobileCameraActive then return end
8409     local camera = workspace.CurrentCamera
8410     local character = localPlayer.Character
8411     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8412     if not camera or not humanoid then return end
8413
8414     local look = camera.CFrame.LookVector
8415     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
8416     self.touchYaw = math.atan2(-look.X, -look.Z)
8417     self.savedCameraType = camera.CameraType
8418     self.mobileCameraActive = true
8419     humanoid.AutoRotate = false
8420     camera.CameraType = Enum.CameraType.Scriptable
8421     self:_setLocalHeadHidden(true)
8422 end
8423
8424 function BlasterController:_endTouchCamera()
8425     if not self.mobileCameraActive then return end
8426     local camera = workspace.CurrentCamera
8427     local character = localPlayer.Character
8428     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8429     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
8430     self.mobileCameraActive = false
8431     self.touchInput = nil
8432     self.touchLastPosition = nil
8433     self:_setLocalHeadHidden(false)
8434 end
8435
8436 function BlasterController:SetAiming(aiming)
8437     if not self.enabled then aiming = false end
8438     if self.aiming == aiming then return end
8439     self.aiming = aiming

```

```

8440     local camera = workspace.CurrentCamera
8441     if not camera then return end
8442     local config = Blaster.GetWeaponConfig(self.tool)
8443     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
8444     if self.fovTween then self.fovTween:Cancel() end
8445     self.fovTween = TweenService.Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
8446     ↪ Enum.EasingDirection.Out), {
8447         FieldOfView = targetFov,
8448     })
8449     self.fovTween:Play()
8450
8451     if touchAimActive() then
8452         -- Mobile weapons keep the camera active in hip-fire too, so the right
8453         -- half of the screen always steers the reticle like a dedicated FPS.
8454         self:_beginTouchCamera()
8455     elseif aiming then
8456         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
8457         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
8458     else
8459         if self.savedMouseSensitivity then
8460             UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
8461             self.savedMouseSensitivity = nil
8462         end
8463     end
8464
8465     local character = localPlayer.Character
8466     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8467     if humanoid and not self.mobileCameraActive then
8468         humanoid.CameraOffset = aiming
8469         and Vector3.new(1.25, 0.55, 0)
8470         or self.shoulderOffset
8471     end
8472
8473     if self.statusLabel then
8474         self.statusLabel.Text = aiming and "ADS" or ""
8475     end
8476 end
8477
8478 function BlasterController:_fireOnce()
8479     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
8480     local config = Blaster.GetWeaponConfig(self.tool)
8481     local now = os.clock()
8482     if now < self.nextLocalShot then return end
8483     self.nextLocalShot = now + config.cooldown
8484     self.sequence = self.sequence + 1
8485
8486     local camera = workspace.CurrentCamera
8487     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
8488     local recoilScale = self.aiming and 0.55 or 1
8489     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
8490     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
8491 end
8492
8493 function BlasterController:_beginFiring()
8494     self.triggerHeld = true
8495     if self.fireLoopRunning then return end
8496     self.fireLoopRunning = true
8497     task.spawn(function()
8498         repeat
8499             self:_fireOnce()
8500             local config = Blaster.GetWeaponConfig(self.tool)
8501             if not config.automatic then break end
8502             task.wait(config.cooldown)
8503         until not self.triggerHeld or not self.enabled
8504         self.fireLoopRunning = false
8505     end)
8506 end
8507
8508 function BlasterController:_shootAction(_, inputState, inputObject)
8509     if not self.enabled then return Enum.ContextActionResult.Pass end
8510     if inputState == Enum.UserInputState.Begin and inputObject
8511     and inputObject.UserInputType == Enum.UserInputType.MouseButton1 then
8512         local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
8513         if pointInside(self.fallbackAimButton, point) then return Enum.ContextActionResult.Pass end
8514     end
8515     if inputState == Enum.UserInputState.Begin then

```



```

8515         self:_beginFiring()
8516     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8517         self.triggerHeld = false
8518     end
8519     return Enum.ContextActionResult.Sink
8520 end
8521
8522 function BlasterController:_aimAction(_, inputState, inputObject)
8523     if not self.enabled then return Enum.ContextActionResult.Pass end
8524     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
8525     if isTouch then
8526         if inputState == Enum.UserInputState.Begin then
8527             self:SetAiming(not self.aiming)
8528         end
8529     elseif inputState == Enum.UserInputState.Begin then
8530         self:SetAiming(true)
8531     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8532         self:SetAiming(false)
8533     end
8534     return Enum.ContextActionResult.Sink
8535 end
8536
8537 function BlasterController:_touchPanAction(_, inputState, inputObject)
8538     if not self.enabled or not touchControlsActive() then
8539         return Enum.ContextActionResult.Pass
8540     end
8541
8542     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
8543     local camera = workspace.CurrentCamera
8544     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
8545     local inset = GuiService.GetGuiInset()
8546     point = point - inset
8547
8548     if inputState == Enum.UserInputState.Begin then
8549         local shootButton = self.fallbackShootButton or self:_button(self.actionPrefix .. "Shoot")
8550         local aimButton = self.fallbackAimButton or self:_button(self.actionPrefix .. "Aim")
8551         local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
8552         local jumpButton = touchGui and touchGui:FindFirstChild("JumpButton", true)
8553         if point.X < viewport.X * 0.42 or pointInside(shootButton, point)
8554             or pointInside(aimButton, point) or pointInside(jumpButton, point) then
8555             return Enum.ContextActionResult.Pass
8556         end
8557         self.touchInput = inputObject
8558         self:_beginTouchCamera()
8559         self.touchLastPosition = point
8560         return Enum.ContextActionResult.Sink
8561     end
8562
8563     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
8564     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
8565         local delta = point - self.touchLastPosition
8566         self.touchLastPosition = point
8567         local sensitivity = self.aiming
8568             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
8569             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
8570         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
8571         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
8572         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
8573             ⇨ math.rad(78))
8573         return Enum.ContextActionResult.Sink
8574     end
8575
8576     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
8577         self.touchInput = nil
8578         self.touchLastPosition = nil
8579         return Enum.ContextActionResult.Sink
8580     end
8581     return Enum.ContextActionResult.Pass
8582 end
8583
8584 function BlasterController:_updateArmPose(targetPosition)
8585     local character = localPlayer.Character
8586     local handle = self.tool:FindFirstChild("Handle")
8587     if not character or not handle or not targetPosition then return end
8588     if not self.aimJoint or not self.aimJoint.Parent then
8589         local joints = {}

```

```

8590     for _, joint in ipairs(character:GetDescendants()) do
8591         if (joint:IsA("JointInstance") or joint:IsA("AnimationConstraint")) and joint.Part0 and joint.Part1
            ↪ then
8592             joints[joint.Part1] = joint
8593         end
8594     end
8595     local chain = {}
8596     local part = handle
8597     for _ = 1, 8 do
8598         local joint = joints[part]
8599         if not joint then return end
8600         table.insert(chain, 1, joint)
8601         if joint.Name == "Right Shoulder" or joint.Name == "RightShoulder" then
8602             self.aimJoint = joint
8603             self.aimBaseC0 = joint.C0
8604             self.aimChain = chain
8605             break
8606         end
8607         part = joint.Part0
8608     end
8609     end
8610     local shoulder = self.aimJoint
8611     if not shoulder or not shoulder.Part0 then return end
8612     -- Rebuild from the untouched joint/grip and the current animation each frame.
8613     -- This preserves R6/R15 rest rotations and avoids accumulating aim offsets.
8614     local jointWorld = shoulder.Part0.CFrame * self.aimBaseC0
8615     local predicted = shoulder.Part0.CFrame
8616     for _, joint in ipairs(self.aimChain) do
8617         if not joint.Parent then return end
8618         local c0 = joint == shoulder and self.aimBaseC0 or joint.C0
8619         local animation = (joint:IsA("Motor6D") or joint:IsA("AnimationConstraint")) and joint.Transform or
            ↪ CFrame.identity
8620         predicted = predicted * c0 * animation * joint.C1:Inverse()
8621     end
8622     local forward = self.tool:GetAttribute("BlasterBarrelDirection")
8623     if typeof(forward) ~= "Vector3" or forward.Magnitude < 0.1 then
8624         -- Imported Weapons Kit meshes face +Z; procedural barrels face -Z.
8625         forward = self.tool:GetAttribute("LabSidearm") and Vector3.zAxis or -Vector3.zAxis
8626     end
8627     local turn = CFrame.identity
8628     local pivot = jointWorld.Position
8629     for _ = 1, 4 do
8630         local delta = targetPosition - predicted.Position
8631         if delta.Magnitude < 0.1 then break end
8632         local correction = rotationBetween(predicted:VectorToWorldSpace(forward.Unit), delta.Unit)
8633         predicted = CFrame.new(pivot) * correction * CFrame.new(-pivot) * predicted
8634         turn = correction * turn
8635     end
8636     local pose = shoulder.Part0.CFrame:ToObjectSpace(CFrame.new(pivot) * turn * jointWorld.Rotation)
8637     if shoulder:IsA("AnimationConstraint") then
8638         shoulder.Attachment0.CFrame = pose
8639     else
8640         shoulder.C0 = pose
8641     end
8642 end
8643
8644 function BlasterController:_render(dt)
8645     if not self.enabled then return end
8646     local character = localPlayer.Character
8647     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8648     local root = character and character:FindFirstChild("HumanoidRootPart")
8649     local head = character and character:FindFirstChild("Head")
8650     local camera = workspace.CurrentCamera
8651     if not humanoid or not camera then return end
8652     local hudRebuilt = self:_ensureHUD()
8653     local touchMode = touchControlsActive()
8654     local touchAim = touchAimActive()
8655     if touchAim and not self.mobileCameraActive then
8656         self:_beginTouchCamera()
8657     elseif not touchAim and self.mobileCameraActive then
8658         self:_endTouchCamera()
8659     end
8660     local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
8661     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
8662     if touchMode and (hudRebuilt or self.lastTouchViewport ~= camera.ViewportSize
        or (jump and jump.AbsolutePosition ~= self.lastJumpPosition)) then
8663

```

```

8664     self.lastTouchViewport = camera.ViewportSize
8665     task.defer(function()
8666         if self.enabled then self:_styleTouchButtons() end
8667     end)
8668 end
8669 local moving = math.clamp(humanoid.MoveDirection.Magnitude, 0, 1)
8670 local speed = self.aiming and 7 or 9.5
8671 local amount = self.aiming and 0.012 or 0.035
8672 local clock = os.clock() * speed
8673 local bob = CFrame.new(math.sin(clock) * amount * moving, math.abs(math.cos(clock)) * amount * moving, 0)
8674 * CFrame.Angles(0, 0, math.sin(clock) * math.rad(0.35) * moving)
8675 self.tool.Grip = self.baseGrip * bob
8676
8677 self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
8678 self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
8679 if self.mobileCameraActive and head and root then
8680     self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
8681     self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
8682     local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
8683     local focus = head.Position + Vector3.new(0, 0.12, 0)
8684     local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
8685     local cameraResult = workspace.Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
8686         ↪ self.tool}))
8687     local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
8688     camera.CFrame = CFrame.new(position) * rotation
8689     camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
8690     local flatLook = Vector3.new(rotation.LookVector.X, 0, rotation.LookVector.Z)
8691     if flatLook.Magnitude > 0.01 then
8692         root.CFrame = CFrame.lookAt(root.Position, root.Position + flatLook.Unit)
8693     end
8694 elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
8695     camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)
8696 end
8697 local config = Blaster.GetWeaponConfig(self.tool)
8698 local cast = Blaster.CastFromCamera(camera, config.range, {character, self.tool, camera},
8699     ↪ self:_getAimScreenPoint(camera))
8700 if cast then
8701     if root and not humanoid.Sit then
8702         local flat = Vector3.new(cast.position.X - root.Position.X, 0, cast.position.Z - root.Position.Z)
8703         if flat.Magnitude > 0.1 then root.CFrame = CFrame.lookAt(root.Position, root.Position + flat) end
8704     end
8705     self:_updateArmPose(cast.position)
8706 end
8707 self:_updateAimReticle(camera)
8708 end
8709
8710 function BlasterController:Enable()
8711     if self.enabled then return end
8712     self.enabled = true
8713     self.baseGrip = self.tool.Grip
8714     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
8715         ↪ workspace.CurrentCamera.FieldOfView) or 70
8716
8717     self:_ensureHUD()
8718     self.hud.Enabled = true
8719
8720     local character = localPlayer.Character
8721     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8722     if humanoid then
8723         self.savedAutoRotate = humanoid.AutoRotate
8724         humanoid.AutoRotate = false
8725         self.savedCameraOffset = humanoid.CameraOffset
8726         self.shoulderOffset = Vector3.new(
8727             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
8728             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
8729             0
8730         )
8731         humanoid.CameraOffset = self.shoulderOffset
8732     end
8733     if touchAimActive() then self:_beginTouchCamera() end
8734
8735     local shootName = self.actionPrefix .. "Shoot"
8736     local aimName = self.actionPrefix .. "Aim"
8737     local panName = self.actionPrefix .. "TouchPan"
8738     local priority = Enum.ContextActionPriority.High.Value

```

```

8737 ContextActionService:BindActionAtPriority(shootName, function(...)
8738     return self:_shootAction(...)
8739 end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
8740 ContextActionService:BindActionAtPriority(aimName, function(...)
8741     return self:_aimAction(...)
8742 end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
8743 ContextActionService:BindActionAtPriority(panName, function(...)
8744     return self:_touchPanAction(...)
8745 end, false, priority + 1, Enum.UserInputType.Touch)
8746
8747 ContextActionService:SetTitle(shootName, "FIRE")
8748 ContextActionService:SetTitle(aimName, "AIM")
8749 task.delay(0.1, function()
8750     if self.enabled then self:_styleTouchButtons() end
8751 end)
8752
8753 self.renderName = self.actionPrefix .. "Render"
8754 RunService:BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
8755     self:_render(dt)
8756 end)
8757 end
8758
8759 function BlasterController:Disable()
8760     if not self.enabled then return end
8761     self.triggerHeld = false
8762     self:SetAiming(false)
8763     self.enabled = false
8764
8765     ContextActionService:UnbindAction(self.actionPrefix .. "Shoot")
8766     ContextActionService:UnbindAction(self.actionPrefix .. "Aim")
8767     ContextActionService:UnbindAction(self.actionPrefix .. "TouchPan")
8768     if self.renderName then RunService:UnbindFromRenderStep(self.renderName) end
8769     if self.hud then self.hud.Enabled = false end
8770     if self.tool.Parent then self.tool.Grip = self.baseGrip end
8771     if self.aimJoint and self.aimJoint.Parent then
8772         if self.aimJoint:IsA("AnimationConstraint") then
8773             self.aimJoint.Attachment0.CFrame = self.aimBaseC0
8774         else
8775             self.aimJoint.C0 = self.aimBaseC0
8776         end
8777     end
8778     self.aimJoint, self.aimBaseC0, self.aimChain = nil, nil, nil
8779
8780     local camera = workspace.CurrentCamera
8781     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
8782     self:_endTouchCamera()
8783     if self.savedMouseSensitivity then
8784         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
8785         self.savedMouseSensitivity = nil
8786     end
8787     local character = localPlayer.Character
8788     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8789     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
8790     self.savedAutoRotate = nil
8791     if humanoid and self.savedCameraOffset then
8792         humanoid.CameraOffset = self.savedCameraOffset
8793     end
8794     self.savedCameraOffset = nil
8795     self.lastTouchViewport = nil
8796 end
8797
8798 function BlasterController:Destroy()
8799     self:Disable()
8800     for _, connection in pairs(self.connections) do
8801         connection:Disconnect()
8802     end
8803     table.clear(self.connections)
8804     if self.hud then self.hud:Destroy() end
8805     controllers[self.tool] = nil
8806 end
8807
8808 function BlasterController.BindTool(tool, overrides)
8809     if controllers[tool] then return controllers[tool] end
8810     local controller = BlasterController.new(tool, overrides)
8811     controllers[tool] = controller
8812     Blaster.StartReplication()

```

```

8813     return controller
8814 end
8815
8816 function BlasterController.GetController(tool)
8817     return controllers[tool]
8818 end
8819
8820 function BlasterController.AutoBind()
8821     local binder = {connections = {}}
8822
8823     local function consider(instance)
8824         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
8825             ⇐ instance:GetAttribute("LabSidearm")) then
8826             BlasterController.BindTool(instance)
8827         end
8828     end
8829
8830     local function watch(container)
8831         if not container then return end
8832         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
8833         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
8834     end
8835
8836     watch(localPlayer.WaitForChild("Backpack"))
8837     watch(localPlayer.Character)
8838     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
8839     Blaster.StartReplication()
8840
8841     function binder:Destroy()
8842         for _, connection in ipairs(self.connections) do connection:Disconnect() end
8843         table.clear(self.connections)
8844     end
8845
8846     return binder
8847 end
8848
8849 return BlasterController
8850 ]====]
8851
8852 local SERVER_BLASTER_MANAGER_SOURCE = [====[
8853 -- ServerBlasterManager
8854 -- Authoritative validation, obstacle recasting, damage, and shot replication.
8855
8856 local Players = game:GetService("Players")
8857 local ReplicatedStorage = game:GetService("ReplicatedStorage")
8858 local ServerScriptService = game:GetService("ServerScriptService")
8859
8860 local modules = ReplicatedStorage.WaitForChild("Modules")
8861 local Blaster = require(modules.WaitForChild("Blaster"))
8862
8863 local remoteFolder = ReplicatedStorage:FindFirstChild(Blaster.REMOTE_FOLDER_NAME)
8864 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end
8865 if not remoteFolder then
8866     remoteFolder = Instance.new("Folder")
8867     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME
8868     remoteFolder.Parent = ReplicatedStorage
8869 end
8870
8871 local function getRemote(name)
8872     local remote = remoteFolder:FindFirstChild(name)
8873     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end
8874     if not remote then
8875         remote = Instance.new("RemoteEvent")
8876         remote.Name = name
8877         remote.Parent = remoteFolder
8878     end
8879     return remote
8880 end
8881
8882 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)
8883 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)
8884
8885 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")
8886 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end
8887 if not hitBridge then
8888     hitBridge = Instance.new("BindableFunction")

```

```

8888     hitBridge.Name = "BlasterHitBridge"
8889     hitBridge.Parent = ServerScriptService
8890 end
8891
8892 -- Roblox's default third-person zoom can legitimately place the camera well
8893 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
8894 -- zoom distances and the over-shoulder hip-fire camera.
8895 local MAX_CAMERA_OFFSET = 40
8896 local RATE_TOLERANCE = 0.88
8897 local nextShotByPlayer = {}
8898 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
8899
8900 local function findHumanoidFromHit(instance)
8901     local node = instance
8902     while node and node ~= workspace do
8903         if node:IsA("Model") then
8904             local humanoid = node:FindFirstChildOfClass("Humanoid")
8905             if humanoid then return humanoid, node end
8906         end
8907         node = node.Parent
8908     end
8909     return nil
8910 end
8911
8912 local function randomSpread(direction, spread)
8913     if spread <= 0 then return direction end
8914     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
8915     local right = direction:Cross(reference).Unit
8916     local up = right:Cross(direction).Unit
8917     local radius = math.sqrt(math.random()) * spread
8918     local angle = math.random() * math.pi * 2
8919     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
8920 end
8921
8922 local function broadcastShot(shooter, origin, position, normal, color)
8923     local payload = {
8924         origin = origin,
8925         position = position,
8926         normal = normal,
8927         color = color,
8928     }
8929     for _, other in ipairs(Players:GetPlayers()) do
8930         if other ~= shooter then effectRemote:FireClient(other, payload) end
8931     end
8932 end
8933
8934 local function bridgeHit(player, result, damage, instantHeadshot)
8935     if not result then return false end
8936     -- BindableFunction callbacks are write-only. LabEquipment binds this during
8937     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
8938     local ok, response = pcall(function()
8939         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
8940             instantHeadshot and result.Instance.Name == "Head")
8941     end)
8942     return ok and response == true
8943 end
8944
8945 local function genericDamage(player, result, config)
8946     local humanoid, model = findHumanoidFromHit(result.Instance)
8947     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
8948     local victimPlayer = Players:GetPlayerFromCharacter(model)
8949     if victimPlayer and not config.allowPlayerDamage then return end
8950
8951     local damage = config.damage
8952     if result.Instance.Name == "Head" then
8953         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
8954     end
8955     humanoid:TakeDamage(damage)
8956 end
8957
8958 local function validSequence(tool, sequence)
8959     if typeof(sequence) ~= "number" or sequence ~= sequence or sequence % 1 ~= 0 then return false end
8960     if sequence < 1 or sequence > 2147483647 then return false end
8961     local previous = lastSequenceByTool[tool]
8962     if previous and sequence <= previous then return false end
8963     lastSequenceByTool[tool] = sequence

```

```

8964     return true
8965 end
8966
8967 local function handleFire(player, packet)
8968     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
8969     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
8970         ↪ return end
8971
8972     local character = player.Character
8973     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
8974     local head = character and character:FindFirstChild("Head")
8975     local root = character and character:FindFirstChild("HumanoidRootPart")
8976     local tool = packet.tool
8977     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
8978     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
8979     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
8980
8981     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
8982     local directionMagnitude = packet.direction.Magnitude
8983     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
8984     local aimDirection = packet.direction.Unit
8985     if not validSequence(tool, packet.sequence) then return end
8986
8987     local config = Blaster.GetWeaponConfig(tool)
8988     local now = os.clock()
8989     if now < (nextShotByPlayer[player] or 0) then return end
8990     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
8991
8992     local params = Blaster.NewRaycastParams({character, tool})
8993     local handle = tool:FindFirstChild("Handle")
8994     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
8995     local tracerColor = tool:GetAttribute("BlasterTracerColor")
8996     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
8997
8998     for _ = 1, config.pellets do
8999         local pelletDirection = randomSpread(aimDirection, config.spread)
9000         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
9001         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
9002             ↪ config.range)
9003         local muzzleDelta = aimPoint - muzzleOrigin
9004         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection
9005         local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
9006         local result = workspace:Raycast(muzzleOrigin, castDirection * castLength, params)
9007         local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
9008
9009         broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
9010         if result then
9011             local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
9012             if not handled then genericDamage(player, result, config) end
9013         end
9014     end
9015
9016     if config.selfCostPercent > 0 and humanoid.Health > 0 then
9017         humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
9018     end
9019 end
9020
9021 fireRemote.OnServerEvent:Connect(handleFire)
9022 Players.PlayerRemoving:Connect(function(player)
9023     nextShotByPlayer[player] = nil
9024 end)
9025
9026 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")
9027
9028 ]====]
9029
9030 -- Compact responsive score/timer overlay. Touch devices get only the live
9031 -- essentials; the top-three board remains available on larger displays.
9032 local LAB_HUD_SOURCE = {[
9033     local Players = game:GetService("Players")
9034     local TweenService = game:GetService("TweenService")
9035     local UIS = game:GetService("UserInputService")
9036     local RunService = game:GetService("RunService")
9037
9038     local player = Players.LocalPlayer
9039     local camera = workspace.CurrentCamera

```

```

9038     local compact = UIS.TouchEnabled or (camera and camera.ViewportSize.X < 900)
9039     local S = compact and 0.78 or 0.9
9040
9041     local INK = Color3.fromRGB(232, 238, 236)
9042     local PANEL = Color3.fromRGB(18, 22, 24)
9043     local GOOD = Color3.fromRGB(96, 226, 150)
9044     local WARN = Color3.fromRGB(255, 186, 74)
9045     local HOT = Color3.fromRGB(255, 96, 84)
9046
9047     local old = player:WaitForChild("PlayerGui"):FindFirstChild("LabHUD")
9048     if old then old:Destroy() end
9049     local gui = Instance.new("ScreenGui")
9050     gui.Name = "LabHUD"
9051     gui.ResetOnSpawn = false
9052     gui.IgnoreGuiInset = false
9053     gui.DisplayOrder = 10
9054     gui.ZIndexBehavior = Enum.ZIndexBehavior.Sibling
9055     gui.Parent = player.PlayerGui
9056
9057     local function panel(parent, size, pos, anchor)
9058         local f = Instance.new("Frame")
9059         f.Size, f.Position, f.AnchorPoint = size, pos, anchor or Vector2.zero
9060         f.BackgroundColor3 = PANEL
9061         f.BackgroundTransparency = 0.14
9062         f.BorderSizePixel = 0
9063         f.Parent = parent
9064         local c = Instance.new("UICorner")
9065         c.CornerRadius = UDim.new(0, 8)
9066         c.Parent = f
9067         local stroke = Instance.new("UIStroke")
9068         stroke.Color = Color3.fromRGB(90, 100, 104)
9069         stroke.Transparency = 0.58
9070         stroke.Parent = f
9071         return f
9072     end
9073
9074     local function label(parent, text, size, pos, textSize, colour, align)
9075         local t = Instance.new("TextLabel")
9076         t.Size, t.Position = size, pos
9077         t.BackgroundTransparency = 1
9078         t.Font = Enum.Font.GothamBold
9079         t.Text = text
9080         t.TextSize = textSize * S
9081         t.TextColor3 = colour or INK
9082         t.TextXAlignment = align or Enum.TextXAlignment.Center
9083         t.TextTruncate = Enum.TextTruncate.AtEnd
9084         t.Parent = parent
9085         return t
9086     end
9087
9088     local roundBox = panel(gui, UDim2.fromOffset(230 * S, 52 * S), UDim2.new(0.5, 0, 0, 6), Vector2.new(0.5,
9089 ↪ 0))
9090     local roundName = label(roundBox, "SHIFT 1", UDim2.new(1, 0, 0, 16 * S), UDim2.fromOffset(0, 3), 11,
9091 ↪ Color3.fromRGB(150, 160, 165))
9092     local clock = label(roundBox, "--:--", UDim2.new(1, 0, 0, 24 * S), UDim2.fromOffset(0, 17 * S), 21)
9093     local barBg = Instance.new("Frame")
9094     barBg.Size = UDim2.new(1, -18, 0, 4)
9095     barBg.Position = UDim2.new(0, 9, 1, -8)
9096     barBg.BackgroundColor3 = Color3.fromRGB(52, 58, 62)
9097     barBg.BorderSizePixel = 0
9098     barBg.Parent = roundBox
9099     local barCorner = Instance.new("UICorner")
9100     barCorner.CornerRadius = UDim.new(1, 0)
9101     barCorner.Parent = barBg
9102     local bar = Instance.new("Frame")
9103     bar.Size = UDim2.fromScale(1, 1)
9104     bar.BackgroundColor3 = GOOD
9105     bar.BorderSizePixel = 0
9106     bar.Parent = barBg
9107     local fillCorner = Instance.new("UICorner")
9108     fillCorner.CornerRadius = UDim.new(1, 0)
9109     fillCorner.Parent = bar
9110
9111     local scoreBox = panel(gui, UDim2.fromOffset(140 * S, 52 * S), UDim2.new(1, -8, 0, 6), Vector2.new(1, 0))
9112     label(scoreBox, "SCORE", UDim2.new(1, -10, 0, 14 * S), UDim2.fromOffset(0, 3), 10, Color3.fromRGB(150, 160,
9113 ↪ 165), Enum.TextXAlignment.Right)

```



```

9111     local scoreText = label(scoreBox, "0", UDim2.new(1, -10, 0, 23 * S), UDim2.fromOffset(0, 16 * S), 22, INK,
9112     ↪ Enum.TextXAlignment.Right)
9113     local pbText = label(scoreBox, "BEST 0", UDim2.new(1, -10, 0, 11 * S), UDim2.new(0, 0, 1, -13 * S), 9,
9114     ↪ Color3.fromRGB(140, 150, 155), Enum.TextXAlignment.Right)
9115     local streakText = label(scoreBox, "", UDim2.new(0.55, 0, 0, 16 * S), UDim2.fromOffset(7, 18 * S), 11,
9116     ↪ WARN, Enum.TextXAlignment.Left)
9117
9118     local popups = Instance.new("Frame")
9119     popups.Size = UDim2.fromOffset(190 * S, 150 * S)
9120     popups.Position = UDim2.new(1, -8, 0, 54 * S)
9121     popups.AnchorPoint = Vector2.new(1, 0)
9122     popups.BackgroundTransparency = 1
9123     popups.Parent = gui
9124     local popupLayout = Instance.new("UIListLayout")
9125     popupLayout.HorizontalAlignment = Enum.HorizontalAlignment.Right
9126     popupLayout.Padding = UDim.new(0, 3)
9127     popupLayout.Parent = popups
9128
9129     local board = panel(gui, UDim2.fromOffset(170 * S, 82 * S), UDim2.new(1, -8, 0, 64 * S), Vector2.new(1, 0))
9130     board.Visible = not compact
9131     label(board, "TOP TECHNICIANS", UDim2.new(1, 0, 0, 14 * S), UDim2.fromOffset(0, 4), 10, Color3.fromRGB(150,
9132     ↪ 160, 165))
9133     local rows = {}
9134     for i = 1, 3 do
9135         rows[i] = label(board, "", UDim2.new(1, -14, 0, 17 * S), UDim2.fromOffset(7, (16 + i * 17) * S), 11,
9136         ↪ INK, Enum.TextXAlignment.Left)
9137         rows[i].Font = Enum.Font.Gotham
9138     end
9139
9140     local shown, target = 0, 0
9141     local function pointsValue()
9142         local stats = player:FindFirstChild("leaderstats")
9143         local points = stats and stats:FindFirstChild("Points")
9144         return points and points.Value or 0
9145     end
9146
9147     local streak, lastAward = 0, 0
9148     local function popup(amount, reason)
9149         local row = panel(popups, UDim2.fromOffset(184 * S, 25 * S), UDim2.fromOffset(0, 0), Vector2.zero)
9150         local amountLabel = label(row, "+" .. amount, UDim2.new(0, 45 * S, 1, 0), UDim2.fromOffset(6, 0), 13,
9151         ↪ GOOD, Enum.TextXAlignment.Left)
9152         local reasonLabel = label(row, tostring(reason), UDim2.new(1, -53 * S, 1, 0), UDim2.fromOffset(49 * S,
9153         ↪ 0), 9, Color3.fromRGB(190, 200, 205), Enum.TextXAlignment.Left)
9154         reasonLabel.Font = Enum.Font.Gotham
9155         task.delay(2.2, function()
9156             if not row.Parent then return end
9157             TweenService:Create(row, TweenInfo.new(0.3), {BackgroundTransparency = 1}):Play()
9158             TweenService:Create(amountLabel, TweenInfo.new(0.3), {TextTransparency = 1}):Play()
9159             TweenService:Create(reasonLabel, TweenInfo.new(0.3), {TextTransparency = 1}):Play()
9160             task.wait(0.32)
9161             row:Destroy()
9162         end)
9163     end
9164
9165     player:GetAttributeChangedSignal("LastPointAward"):Connect(function()
9166         local amount = player:GetAttribute("LastPointAmount") or 0
9167         if amount <= 0 then return end
9168         local now = os.clock()
9169         streak = (now - lastAward <= 14) and streak + 1 or 1
9170         lastAward = now
9171         popup(amount, player:GetAttribute("LastPointAward") or "Task complete")
9172         scoreText.TextColor3 = GOOD
9173         TweenService:Create(scoreText, TweenInfo.new(0.5), {TextColor3 = INK}):Play()
9174         if streak >= 2 then streakText.Text = "x" .. streak end
9175     end)
9176
9177     local acc = 0
9178     RunService.RenderStepped:Connect(function(dt)
9179         target = pointsValue()
9180         if shown ~= target then
9181             shown += (target - shown) * math.min(dt * 8, 1)
9182             if math.abs(target - shown) < 0.6 then shown = target end
9183             scoreText.Text = tostring(math.floor(shown + 0.5))
9184         end
9185         if streak >= 2 and os.clock() - lastAward > 14 then streak = 0; streakText.Text = "" end
9186     end)

```

```

9180     local endsAt = workspace:GetAttribute("RoundEndsAt")
9181     local length = workspace:GetAttribute("RoundLength") or 480
9182     local phase = workspace:GetAttribute("RoundPhase") or "PLAY"
9183     if endsAt then
9184         local left = math.max(0, endsAt - os.time())
9185         clock.Text = string.format("%d:%02d", left // 60, left % 60)
9186         if phase == "BREAK" then
9187             roundName.Text = "SHIFT OVER"
9188             bar.BackgroundColor3 = Color3.fromRGB(120, 130, 140)
9189             bar.Size = UDim2.fromScale(1, 1)
9190         else
9191             roundName.Text = "SHIFT " .. tostring(workspace:GetAttribute("RoundNumber") or 1)
9192             bar.Size = UDim2.new(math.clamp(left / length, 0, 1), 0, 1, 0)
9193             bar.BackgroundColor3 = left <= 30 and HOT or left <= 90 and WARN or GOOD
9194             clock.TextColor3 = (left <= 10 and left > 0 and left % 2 == 0) and HOT or INK
9195         end
9196     end
9197
9198     acc += dt
9199     if acc >= 1.5 then
9200         acc = 0
9201         local list = {}
9202         for _, pl in ipairs(Players:GetPlayers()) do
9203             local stats = pl:FindFirstChild("leaderstats")
9204             local points = stats and stats:FindFirstChild("Points")
9205             table.insert(list, {name = pl.Name, value = points and points.Value or 0})
9206         end
9207         table.sort(list, function(a, b) return a.value > b.value end)
9208         for i = 1, 3 do
9209             local entry = list[i]
9210             rows[i].Text = entry and string.format("%d. %s %d", i, entry.name, entry.value) or ""
9211             rows[i].TextColor3 = entry and entry.name == player.Name and GOOD or INK
9212         end
9213         pbText.Text = "BEST " .. tostring(player:GetAttribute("PersonalBest") or 0)
9214     end
9215 end)
9216 ]==]
9217
9218 -- Lightweight client bootstrap. Input, crosshair, camera, bob and ADS all
9219 -- live in BlasterController; this script updates the shared compact HUD and binds
9220 -- every standard Tool carrying the BlasterWeapon attribute.
9221 local SIDEARM_CLIENT_SOURCE = [=[
9222 local Players      = game:GetService("Players")
9223 local ReplicatedStorage = game:GetService("ReplicatedStorage")
9224
9225 local player = Players.LocalPlayer
9226
9227 -- The hunt counter belongs inside RoleHUD; a second top-centre panel covered
9228 -- the crosshair and duplicated information on phones.
9229 task.spawn(function()
9230     local complex = workspace:WaitForChild("Laboratory_Complex", 60)
9231     if not complex then return end
9232     local playerGui = player:WaitForChild("PlayerGui", 30)
9233     if not playerGui then return end
9234     local roleGui = playerGui:WaitForChild("RoleHUD", 30)
9235     local frame = roleGui and roleGui:FindFirstChildOfClass("Frame")
9236     local label = frame and frame:FindFirstChild("TargetsLabel")
9237     if not label then return end
9238
9239     local function refresh()
9240         local killed = complex:GetAttribute("BotsKilled") or 0
9241         local total = complex:GetAttribute("BotsTotal") or 0
9242         if total > 0 and killed >= total then
9243             label.Text = "LAB CLEARED • " .. killed .. " / " .. total
9244             label.TextColor3 = Color3.fromRGB(125, 255, 155)
9245         else
9246             label.Text = string.format("TARGETS %d / %d", killed, total)
9247             label.TextColor3 = Color3.fromRGB(255, 205, 120)
9248         end
9249     end
9250     complex:GetAttributeChangedSignal("BotsKilled"):Connect(refresh)
9251     complex:GetAttributeChangedSignal("BotsTotal"):Connect(refresh)
9252     refresh()
9253 end)
9254
9255 -- One lightweight bootstrap watches standard Tools as they move between the

```

```

9256 -- Backpack and Character. All device input/camera behavior lives in the module.
9257 local modules = ReplicatedStorage:WaitForChild("Modules", 30)
9258 if not modules then
9259     warn("Blaster modules were not installed")
9260     return
9261 end
9262
9263 local BlasterController = require(modules:WaitForChild("BlasterController"))
9264 BlasterController.AutoBind()
9265 ]==]
9266
9267 local ELEVATOR_SCRIPT_SOURCE = [=[
9268 -- LabElevators : moving cars that carry whoever is standing inside.
9269 -- An anchored part does not drag a character with it, so the car
9270 -- offsets every occupant by the same delta on each step.
9271
9272 local CollectionService = game:GetService("CollectionService")
9273 local RunService        = game:GetService("RunService")
9274 local Players            = game:GetService("Players")
9275
9276 local SPEED = 45 -- studs per second
9277
9278 local function setGate(gate, open)
9279     gate.CanCollide = not open
9280     gate.Transparency = open and 0.85 or 0
9281 end
9282
9283 local function setup(model)
9284     local floorCount = model:GetAttribute("FloorCount")
9285     if not floorCount then return end
9286
9287     local car = model:WaitForChild("Car")
9288     local platform = car:FindFirstChild("Platform") or car:FindFirstChild("Platform_1")
9289     if not platform then
9290         for _, d in ipairs(car:GetDescendants()) do
9291             if d:IsA("BasePart") and d.Name:match("^Platform") then platform = d break end
9292         end
9293     end
9294     if not platform then return end
9295
9296     local gates = {}
9297     for _, g in ipairs(model:WaitForChild("Gates"):GetChildren()) do
9298         gates[g:GetAttribute("Floor")] = g
9299     end
9300
9301     local ys = {}
9302     for i = 1, floorCount do ys[i] = model:GetAttribute("FloorY_" .. i) end
9303
9304     local current, moving = 1, false
9305     for i, g in pairs(gates) do setGate(g, i == current) end
9306
9307     local carParts = {}
9308     for _, d in ipairs(car:GetDescendants()) do
9309         if d:IsA("BasePart") then table.insert(carParts, d) end
9310     end
9311
9312     local function occupants()
9313         local list = {}
9314         local half = platform.Size * 0.5
9315         local top = platform.Position.Y + half.Y
9316         for _, pl in ipairs(Players:GetPlayers()) do
9317             local ch = pl.Character
9318             local hrp = ch and ch:FindFirstChild("HumanoidRootPart")
9319             if hrp then
9320                 local d = hrp.Position - platform.Position
9321                 if math.abs(d.X) <= half.X + 2 and math.abs(d.Z) <= half.Z + 2
9322                     and hrp.Position.Y > top - 2 and hrp.Position.Y < top + 24 then
9323                     table.insert(list, hrp)
9324                 end
9325             end
9326         end
9327         return list
9328     end
9329
9330     local function moveTo(target)
9331         if moving or target == current or not ys[target] then return end

```

```

9332     moving = true
9333     setGate(gates[current], false)
9334
9335     local startY = platform.Position.Y
9336     local goalY = startY + (ys[target] - ys[current])
9337     local dist = math.abs(goalY - startY)
9338     local dur = math.max(0.4, dist / SPEED)
9339     local t0 = os.clock()
9340     local lastY = startY
9341
9342     while true do
9343         local a = math.min(1, (os.clock() - t0) / dur)
9344         local newY = startY + (goalY - startY) * a
9345         local dy = newY - lastY
9346         if dy ~= 0 then
9347             local riders = occupants()
9348             for _, p in ipairs(carParts) do
9349                 p.CFrame = p.CFrame + Vector3.new(0, dy, 0)
9350             end
9351             for _, hrp in ipairs(riders) do
9352                 hrp.CFrame = hrp.CFrame + Vector3.new(0, dy, 0)
9353             end
9354             lastY = newY
9355         end
9356         if a >= 1 then break end
9357         RunService.Heartbeat:Wait()
9358     end
9359
9360     current = target
9361     model:SetAttribute("CurrentFloor", current)
9362     setGate(gates[current], true)
9363     moving = false
9364 end
9365
9366 local function addPrompt(part, floor, label, text)
9367     local p = Instance.new("ProximityPrompt")
9368     p.ObjectText = label
9369     p.ActionText = text
9370     p.HoldDuration = 0.15
9371     p.MaxActivationDistance = 22
9372     p.RequiresLineOfSight = false
9373     p.Parent = part
9374     p.Triggered:Connect(function() moveTo(floor) end)
9375 end
9376
9377 for _, panel in ipairs(model:WaitForChild("CallPanels"):GetChildren()) do
9378     addPrompt(panel, panel:GetAttribute("Floor"), "Elevator", "Call")
9379 end
9380 for _, btn in ipairs(car:WaitForChild("Buttons"):GetChildren()) do
9381     local i = btn:GetAttribute("Floor")
9382     addPrompt(btn, i, "Elevator", "Go to floor " .. (model:GetAttribute("FloorLabel_" .. i) or i))
9383 end
9384 end
9385
9386 for _, m in ipairs(CollectionService:GetTagged("Elevator")) do setup(m) end
9387 CollectionService:GetInstanceAddedSignal("Elevator"):Connect(setup)
9388
9389 print("Elevators armed: " .. #CollectionService:GetTagged("Elevator") .. ".")
9390 ]==]
9391
9392 local ROLES_SCRIPT_SOURCE = [=[
9393 -- LabRoles : role kiosks, teams, per-player leaderstats, a persistent
9394 -- HUD, and a visible appearance change (tinted uniform + hard hat) so
9395 -- everyone can see at a glance who is playing which role.
9396
9397 local Players      = game:GetService("Players")
9398 local Teams        = game:GetService("Teams")
9399 local CollectionService = game:GetService("CollectionService")
9400
9401 local ROLE_PAINT = {
9402     Technician = Color3.fromRGB(60, 120, 200),
9403     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),
9404 }
9405
9406 local ROLE_HINT = {
9407     Unassigned = "Choose BLUE Technician or GREEN Laboratory Technician.",

```

```

9408 Technician = "Follow RED floor lights. HOLD R at the failed unit. Score grows with failure time and
    ↳ assigned distance.",
9409 ["Laboratory Technician"] = "E takes a plant sample. Carry it to any GREEN RUNNING cabinet marked FREE SLOT
    ↳ and press E again to revive/drop it.",
9410 }
9411
9412 local function roleEquipmentId(part)
9413     local code = part:GetAttribute("GrowthCode")
9414     if code and code ~= "" then return "CHAMBER:" .. code end
9415     return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room", part.Name},
    ↳ "|")
9416 end
9417
9418 local function roleFloor(part)
9419     return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
9420 end
9421
9422 local function roleTransferSeconds(source, destination)
9423     local distance = destination and (destination.Position - source.Position).Magnitude or 60
9424     local playerCount = math.max(1, #Players:GetPlayers())
9425     local complex = workspace:FindFirstChild("Laboratory_Complex")
9426     local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
9427     return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
9428         + math.clamp(backlog / 20, 0, 25)), 60, 180)
9429 end
9430
9431 local function roleEquipmentLabel(part)
9432     local code = part:GetAttribute("GrowthCode")
9433     if code and code ~= "" then return "CHAMBER " .. code end
9434     return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
9435 end
9436
9437 local function roleDestinations(source)
9438     local choices = {}
9439     for _, chamber in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
9440         local count = chamber:GetAttribute("SampleCount") or 0
9441         local reservations = chamber:GetAttribute("IncomingReservations") or 0
9442         if chamber ~= source and chamber:GetAttribute("FunctionalState") ~= "FAILED"
9443             and count + reservations < (chamber:GetAttribute("Capacity") or 10) then table.insert(choices,
    ↳ chamber) end
9444     end
9445     table.sort(choices, function(a, b)
9446         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9447         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
9448         if sameA ~= sameB then return sameA < sameB end
9449         return (a:GetAttribute("SampleCount") or 0) < (b:GetAttribute("SampleCount") or 0)
9450     end)
9451     return choices
9452 end
9453
9454 local function clearRoleAssignment(player)
9455     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
    ↳ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
9456         player:SetAttribute(name, nil)
9457     end
9458 end
9459
9460 local function assignRoleObjective(player, roleName)
9461     clearRoleAssignment(player)
9462     if roleName == "Technician" then
9463         local failed = {}
9464         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
9465             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment) end
9466         end
9467         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
9468         table.sort(failed, function(a, b)
9469             if root then return (a.Position - root.Position).Magnitude < (b.Position - root.Position).Magnitude
    ↳ end
9470             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
9471         end)
9472         local target = failed[1]
9473         if not target then return "All equipment is working. Repair score grows with failure time and assigned
    ↳ distance." end
9474         player:SetAttribute("AssignedRepairId", roleEquipmentId(target))
9475         local distance = root and (target.Position - root.Position).Magnitude or 0
9476         local waited = math.max(0, os.time() - (target:GetAttribute("FailureSince") or os.time()))

```

```

9477     local projected = math.clamp(20 + math.floor(waited / 8) + math.floor(distance / 10), 8, 120)
9478     player:SetAttribute("AssignedRepairDistance", distance)
9479     player:SetAttribute("AssignedRepairProjectedPoints", projected)
9480     return string.format("REPAIR JOB: %s • %s. Follow RED floor lights; HOLD R. About +%d points.",
9481         roleFloor(target), roleEquipmentLabel(target), projected)
9482 end
9483
9484 local jobs = {}
9485 for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
9486     if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0) > 0
9487         and not source:GetAttribute("ReservedByBot") then
9488         local destination = roleDestinations(source)[1]
9489         if destination then table.insert(jobs, {source = source, destination = destination}) end
9490     end
9491 end
9492 table.sort(jobs, function(a, b)
9493     local an = tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
9494     local bn = tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge
9495     return an < bn
9496 end)
9497 local job = jobs[1]
9498 if not job then return "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
↪ +30." end
9499 local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
9500 local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
9501 player:SetAttribute("AssignedTransferSource", sourceCode)
9502 player:SetAttribute("AssignedTransferDestination", destinationCode)
9503 local allowedSeconds = roleTransferSeconds(job.source, job.destination)
9504 player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
9505 return "ASSIGNED +30: " .. roleFloor(job.source) .. " • " .. sourceCode .. " → " .. destinationCode
9506     .. " • " .. roleFloor(job.destination) .. ". PRESS E at source, deliver in " .. allowedSeconds .. "
↪ seconds."
9507 end
9508
9509 -----
9510 -- per-player HUD (role, contributions, current objective)
9511 -----
9512 local function buildHud(player)
9513     local playerGui = player:WaitForChild("PlayerGui")
9514     local duplicate = playerGui:FindFirstChild("LiveLabHUD")
9515     if duplicate then duplicate:Destroy() end
9516     local gui = Instance.new("ScreenGui")
9517     gui.Name = "RoleHUD"
9518     gui.ResetOnSpawn = false
9519     gui.DisplayOrder = 20
9520     gui.Parent = playerGui
9521
9522     local frame = Instance.new("Frame")
9523     frame.Size = UDim2.fromOffset(238, 90)
9524     frame.Position = UDim2.fromOffset(8, 50)
9525     frame.BackgroundColor3 = Color3.fromRGB(18, 20, 26)
9526     frame.BackgroundTransparency = 0.14
9527     frame.Parent = gui
9528     local corner = Instance.new("UICorner"); corner.CornerRadius = UDim.new(0, 8); corner.Parent = frame
9529
9530     local roleLabel = Instance.new("TextLabel")
9531     roleLabel.Name = "RoleLabel"
9532     roleLabel.Size = UDim2.new(1, -14, 0, 16)
9533     roleLabel.Position = UDim2.new(0, 8, 0, 4)
9534     roleLabel.BackgroundTransparency = 1
9535     roleLabel.TextXAlignment = Enum.TextXAlignment.Left
9536     roleLabel.Font = Enum.Font.GothamBold
9537     roleLabel.TextSize = 13
9538     roleLabel.TextColor3 = Color3.fromRGB(240, 242, 246)
9539     roleLabel.Text = "Role: Unassigned"
9540     roleLabel.Parent = frame
9541
9542     local contribLabel = Instance.new("TextLabel")
9543     contribLabel.Name = "ContribLabel"
9544     contribLabel.Size = UDim2.new(1, -14, 0, 13)
9545     contribLabel.Position = UDim2.new(0, 8, 0, 20)
9546     contribLabel.BackgroundTransparency = 1
9547     contribLabel.TextXAlignment = Enum.TextXAlignment.Left
9548     contribLabel.Font = Enum.Font.Gotham
9549     contribLabel.TextSize = 10
9550     contribLabel.TextTruncate = Enum.TextTruncate.AtEnd

```

```

9551 contribLabel.TextYAlignment = Enum.TextYAlignment.Top
9552 contribLabel.TextColor3 = Color3.fromRGB(200, 205, 212)
9553 contribLabel.Text = "POINTS 0"
9554 contribLabel.Parent = frame
9555
9556 local carryLabel = Instance.new("TextLabel")
9557 carryLabel.Name = "CarryLabel"
9558 carryLabel.Size = UDim2.new(1, -14, 0, 13)
9559 carryLabel.Position = UDim2.new(0, 8, 0, 33)
9560 carryLabel.BackgroundTransparency = 1
9561 carryLabel.TextXAlignment = Enum.TextXAlignment.Left
9562 carryLabel.Font = Enum.Font.GothamBold
9563 carryLabel.TextSize = 10
9564 carryLabel.TextColor3 = Color3.fromRGB(115, 215, 245)
9565 carryLabel.Text = "Carrying: nothing"
9566 carryLabel.Parent = frame
9567
9568 local targetsLabel = Instance.new("TextLabel")
9569 targetsLabel.Name = "TargetsLabel"
9570 targetsLabel.Size = UDim2.new(1, -14, 0, 13)
9571 targetsLabel.Position = UDim2.new(0, 8, 0, 46)
9572 targetsLabel.BackgroundTransparency = 1
9573 targetsLabel.TextXAlignment = Enum.TextXAlignment.Left
9574 targetsLabel.Font = Enum.Font.GothamBold
9575 targetsLabel.TextSize = 10
9576 targetsLabel.TextColor3 = Color3.fromRGB(255, 205, 120)
9577 targetsLabel.Text = "TARGETS 0 / 0"
9578 targetsLabel.Parent = frame
9579
9580 local hintLabel = Instance.new("TextLabel")
9581 hintLabel.Name = "HintLabel"
9582 hintLabel.Size = UDim2.new(1, -14, 0, 25)
9583 hintLabel.Position = UDim2.new(0, 8, 0, 61)
9584 hintLabel.BackgroundTransparency = 1
9585 hintLabel.TextXAlignment = Enum.TextXAlignment.Left
9586 hintLabel.TextYAlignment = Enum.TextYAlignment.Top
9587 hintLabel.TextWrapped = true
9588 hintLabel.Font = Enum.Font.Gotham
9589 hintLabel.TextSize = 9
9590 hintLabel.TextTruncate = Enum.TextTruncate.AtEnd
9591 hintLabel.TextColor3 = Color3.fromRGB(255, 220, 150)
9592 hintLabel.Text = ROLE_HINT.Unassigned
9593 hintLabel.Parent = frame
9594
9595 return gui
9596 end
9597
9598 local function refreshHud(player, roleName)
9599 local gui = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
9600 if not gui then return end
9601 local frame = gui:FindFirstChildOfClass("Frame")
9602 if not frame then return end
9603 frame.RoleLabel.Text = "Role: " .. roleName
9604 local carried = player:GetAttribute("CarriedSample")
9605 local objective = player:GetAttribute("LabObjective")
9606 frame.CarryLabel.Text = (carried and carried ~= "") and ("Carrying: " .. carried) or "Carrying: nothing"
9607 local hint = (objective and objective ~= "") and objective or (ROLE_HINT[roleName] or ROLE_HINT.Unassigned)
9608 local deadline = player:GetAttribute("AssignedTransferDeadline") or 0
9609 if roleName == "Laboratory Technician" and deadline > 0 then
9610 hint = hint .. "\nTIME LEFT FOR +30: " .. math.max(0, deadline - os.time()) .. " seconds"
9611 end
9612 frame.HintLabel.Text = hint
9613 local stats = player:FindFirstChild("leaderstats")
9614 local points = stats and stats:FindFirstChild("Points")
9615 local lastAmount = player:GetAttribute("LastPointAmount") or 0
9616 local lastReason = player:GetAttribute("LastPointAward") or ""
9617 frame.ContribLabel.Text = string.format("POINTS %d%s",
9618 points and points.Value or 0, lastAmount > 0 and (" • LAST +" .. lastAmount .. " " .. lastReason)
9619 ↳ or "")
9619 end
9620
9621 -----
9622 -- Visible appearance change: a welded lab uniform sits above avatar
9623 -- clothing, so layered clothes cannot hide the selected role.
9624 -----
9625 local function wearablePart(uniform, bodyPart, name, size, offset, colour, material)

```

```

9626     local p = Instance.new("Part")
9627     p.Name = name
9628     p.Size = size
9629     p.CFrame = bodyPart.CFrame * offset
9630     p.Color = colour
9631     p.Material = material or Enum.Material.Fabric
9632     p.Anchored = false
9633     p.CanCollide = false
9634     p.CanTouch = false
9635     p.Massless = true
9636     p.CastShadow = false
9637     p.TopSurface = Enum.SurfaceType.Smooth
9638     p.BottomSurface = Enum.SurfaceType.Smooth
9639     p.Parent = uniform
9640     local weld = Instance.new("WeldConstraint")
9641     weld.Part0 = bodyPart
9642     weld.Part1 = p
9643     weld.Parent = p
9644     return p
9645 end
9646
9647 local function applyAppearance(character, roleName)
9648     local paint = ROLE_PAINT[roName]
9649     if not paint then return end
9650     local oldUniform = character:FindFirstChild("RoleUniform")
9651     if oldUniform then oldUniform:Destroy() end
9652     local oldHelmet = character:FindFirstChild("RoleHelmet")
9653     if oldHelmet then oldHelmet:Destroy() end
9654
9655     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
9656     local head = character:FindFirstChild("Head")
9657     if not torso or not head then return end
9658
9659     local uniform = Instance.new("Model")
9660     uniform.Name = "RoleUniform"
9661     uniform:SetAttribute("Role", roleName)
9662     uniform.Parent = character
9663
9664     wearablePart(uniform, torso, "LabCoat", Vector3.new(torso.Size.X * 1.08, torso.Size.Y * 0.94, torso.Size.Z
9665     ↪ * 1.1),
9666     CFrame.new(), Color3.fromRGB(238, 241, 244), Enum.Material.Fabric)
9667     wearablePart(uniform, torso, "RolePanel", Vector3.new(torso.Size.X * 0.82, torso.Size.Y * 0.55, 0.14),
9668     CFrame.new(0, 0.08, -(torso.Size.Z / 2 + 0.12)), paint, Enum.Material.SmoothPlastic)
9669     wearablePart(uniform, torso, "RoleBelt", Vector3.new(torso.Size.X * 1.12, 0.18, torso.Size.Z * 1.14),
9670     CFrame.new(0, -torso.Size.Y * 0.34, 0), paint, Enum.Material.SmoothPlastic)
9671     wearablePart(uniform, torso, "RoleBadge", Vector3.new(0.36, 0.46, 0.16),
9672     CFrame.new(-torso.Size.X * 0.28, torso.Size.Y * 0.22, -(torso.Size.Z / 2 + 0.18)),
9673     Color3.fromRGB(255, 235, 120), Enum.Material.Neon)
9674
9675     local helmet = wearablePart(uniform, head, "RoleHelmet", Vector3.new(1.82, 0.72, 1.82),
9676     CFrame.new(0, 0.72, 0), paint, Enum.Material.SmoothPlastic)
9677     helmet.Shape = Enum.PartType.Ball
9678     wearablePart(uniform, head, "HelmetBand", Vector3.new(1.9, 0.16, 1.9),
9679     CFrame.new(0, 0.52, 0), Color3.fromRGB(245, 245, 245), Enum.Material.SmoothPlastic)
9680
9681     for _, limbName in ipairs({"LeftUpperArm", "RightUpperArm", "Left Arm", "Right Arm"}) do
9682         local limb = character:FindFirstChild(limbName)
9683         if limb then
9684             wearablePart(uniform, limb, limbName:gsub(" ", "") .. "RoleBand",
9685             Vector3.new(limb.Size.X * 1.12, math.min(0.24, limb.Size.Y * 0.18), limb.Size.Z * 1.12),
9686             CFrame.new(0, -limb.Size.Y * 0.26, 0), paint, Enum.Material.SmoothPlastic)
9687         end
9688     end
9689 end
9690
9691 local setupDone = {}
9692 local function setupPlayer(player)
9693     if setupDone[player] then return end
9694     setupDone[player] = true
9695     if player:GetAttribute("CarriedSample") == nil then player:SetAttribute("CarriedSample", "") end
9696     if player:GetAttribute("LabObjective") == nil then player:SetAttribute("LabObjective", "") end
9697     local stats = player:FindFirstChild("leaderstats") or Instance.new("Folder")
9698     stats.Name = "leaderstats"
9699     stats.Parent = player
9700
9701     local role = stats:FindFirstChild("Role") or Instance.new("StringValue")

```



```

9701     role.Name = "Role"
9702     if role.Value == "" then role.Value = "Unassigned" end
9703     role.Parent = stats
9704
9705     local contrib = stats:FindFirstChild("Contributions") or Instance.new("IntValue")
9706     contrib.Name = "Contributions"
9707     contrib.Parent = stats
9708     local points = stats:FindFirstChild("Points") or Instance.new("IntValue")
9709     points.Name = "Points"
9710     points.Parent = stats
9711
9712     local oldHud = player:FindFirstChild("PlayerGui") and player.PlayerGui:FindFirstChild("RoleHUD")
9713     if oldHud then oldHud:Destroy() end
9714     buildHud(player)
9715     role:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9716     contrib:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9717     points:GetPropertyChangedSignal("Value"):Connect(function() refreshHud(player, role.Value) end)
9718     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshHud(player, role.Value) end)
9719     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshHud(player, role.Value) end)
9720     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshHud(player, role.Value) end)
9721     refreshHud(player, role.Value)
9722
9723     -- Health must never regenerate passively: it only comes back through an
9724     -- Emergency Kit. Roblox's default character ships a "Health" script that
9725     -- does exactly the auto-regen we don't want, so remove it every spawn.
9726     local function stripAutoHealthRegen(character)
9727         local healthScript = character:FindFirstChild("Health")
9728         if healthScript then healthScript:Destroy() end
9729     end
9730     player.CharacterAdded:Connect(function(character)
9731         if role.Value ~= "Unassigned" then
9732             applyAppearance(character, role.Value)
9733         end
9734         stripAutoHealthRegen(character)
9735     end)
9736     if player.Character then
9737         if role.Value ~= "Unassigned" then applyAppearance(player.Character, role.Value) end
9738         stripAutoHealthRegen(player.Character)
9739     end
9740     task.spawn(function()
9741         while player.Parent do
9742             if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshHud(player, role.Value)
9743                 ↵ end
9744             task.wait(1)
9745         end
9746     end)
9747 end
9748 Players.PlayerAdded:Connect(setupPlayer)
9749 for _, player in ipairs(Players:GetPlayers()) do task.spawn(setupPlayer, player) end
9750
9751 local function bindKiosk(part)
9752     local roleName = part:GetAttribute("Role")
9753     if not roleName then return end
9754     if part:GetAttribute("RoleKioskBound") then return end
9755     part:SetAttribute("RoleKioskBound", true)
9756
9757     local button = Instance.new("Part")
9758     button.Name = "RoleSelectButton"
9759     button.Shape = Enum.PartType.Cylinder
9760     button.Size = Vector3.new(0.28, 1.05, 1.05)
9761     button.CFrame = part.CFrame * CFrame.new(0, -part.Size.Y * 0.32, -(part.Size.Z / 2 + 0.32))
9762         * CFrame.Angles(0, math.rad(90), 0)
9763     button.Color = ROLE_PAINT[roleName]
9764     button.Material = Enum.Material.Neon
9765     button.Anchored = true
9766     button.CanCollide = false
9767     button.Parent = part
9768
9769     local prompt = Instance.new("ProximityPrompt")
9770     prompt.ObjectText = roleName .. " Kiosk"
9771     prompt.ActionText = "Select " .. roleName
9772     prompt.KeyboardKeyCode = Enum.KeyCode.E
9773     prompt.GamepadKeyCode = Enum.KeyCode.ButtonX
9774     prompt.HoldDuration = 0
9775     prompt.MaxActivationDistance = 30

```

```

9776     prompt.RequiresLineOfSight = false
9777     prompt.Parent = button
9778
9779     local click = Instance.new("ClickDetector")
9780     click.MaxActivationDistance = 30
9781     click.Parent = button
9782     local debounce = {}
9783     local function selectRole(player)
9784         if debounce[player] and os.clock() < debounce[player] then return end
9785         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
9786         if not root or (root.Position - part.Position).Magnitude > 36 then return end
9787         debounce[player] = os.clock() + 1
9788         setupPlayer(player)
9789         local team = Teams:FindFirstChild(roleName)
9790         if not team then return end
9791         player.Team = team
9792         player.Neutral = false
9793         player:SetAttribute("LabRole", roleName)
9794         local stats = player:FindFirstChild("leaderstats")
9795         if stats and stats:FindFirstChild("Role") then
9796             stats.Role.Value = roleName
9797         end
9798         player:SetAttribute("LabObjective", assignRoleObjective(player, roleName))
9799         if player.Character then applyAppearance(player.Character, roleName) end
9800         local oldColour = button.Color
9801         button.Color = Color3.fromRGB(255, 255, 255)
9802         task.delay(0.35, function() if button.Parent then button.Color = oldColour end end)
9803         refreshHud(player, roleName)
9804     end
9805
9806     prompt.Triggered:Connect(selectRole)
9807     click.MouseClick:Connect(selectRole)
9808 end
9809
9810 for _, p in ipairs(CollectionService:GetTagged("RoleKiosk")) do bindKiosk(p) end
9811 CollectionService:GetInstanceAddedSignal("RoleKiosk"):Connect(bindKiosk)
9812
9813 print("Role kiosks armed: " .. #CollectionService:GetTagged("RoleKiosk") .. ".")
9814 ]==]
9815
9816 local QUIZ_CLIENT_SOURCE = [=[
9817 -- Addition popup: click/tap one of three answers. No phone keyboard or
9818 -- changes to the player's camera, aiming, or movement controls are required.
9819 local ReplicatedStorage = game:GetService("ReplicatedStorage")
9820 local Players = game:GetService("Players")
9821 local player = Players.LocalPlayer
9822 local remote = ReplicatedStorage:WaitForChild("LabMathQuiz")
9823 local playerGui = player:WaitForChild("PlayerGui")
9824 local old = playerGui:FindFirstChild("LabMathConsole")
9825 if old then old:Destroy() end
9826
9827 local gui = Instance.new("ScreenGui")
9828 gui.Name = "LabMathConsole"
9829 gui.ResetOnSpawn = false
9830 gui.IgnoreGuiInset = false
9831 gui.DisplayOrder = 60
9832 pcall(function() gui.ScreenInsets = Enum.ScreenInsets.DeviceSafeInsets end)
9833 gui.Parent = playerGui
9834
9835 local panel = Instance.new("Frame")
9836 panel.Name = "AdditionQuiz"
9837 panel.AnchorPoint = Vector2.new(0.5, 0.5)
9838 panel.Position = UDim2.fromScale(0.5, 0.46)
9839 panel.Size = UDim2.new(1, -28, 0, 238)
9840 panel.BackgroundColor3 = Color3.fromRGB(14, 24, 30)
9841 panel.BorderSizePixel = 0
9842 panel.Active = true
9843 panel.Visible = false
9844 panel.Parent = gui
9845 local sizeLimit = Instance.new("UISizeConstraint")
9846 sizeLimit.MaxSize = Vector2.new(440, 238)
9847 sizeLimit.Parent = panel
9848 local corner = Instance.new("UICorner")
9849 corner.CornerRadius = UDim.new(0, 14)
9850 corner.Parent = panel
9851 local stroke = Instance.new("UIStroke")

```

```

9852 stroke.Color = Color3.fromRGB(95, 225, 170)
9853 stroke.Thickness = 2
9854 stroke.Parent = panel
9855
9856 local function label(name, y, height, size, colour)
9857     local item = Instance.new("TextLabel")
9858     item.Name = name
9859     item.Size = UDim2.new(1, -28, 0, height)
9860     item.Position = UDim2.fromOffset(14, y)
9861     item.BackgroundTransparency = 1
9862     item.TextColor3 = colour
9863     item.TextSize = size
9864     item.Font = Enum.Font.GothamBold
9865     item.TextWrapped = true
9866     item.Parent = panel
9867     return item
9868 end
9869 local title = label("Title", 10, 26, 18, Color3.fromRGB(110, 240, 185))
9870 title.Size = UDim2.new(1, -70, 0, 26)
9871 title.Text = "ADDITION CHECK"
9872 local reason = label("Task", 40, 34, 13, Color3.fromRGB(195, 213, 223))
9873 local question = label("Question", 76, 45, 34, Color3.fromRGB(250, 250, 250))
9874 local status = label("Status", 190, 34, 13, Color3.fromRGB(255, 210, 100))
9875 local cancel = Instance.new("TextButton")
9876 cancel.Name = "Cancel"
9877 cancel.Position = UDim2.new(1, -42, 0, 6)
9878 cancel.Size = UDim2.fromOffset(36, 36)
9879 cancel.BackgroundTransparency = 1
9880 cancel.Text = "x"
9881 cancel.TextColor3 = Color3.fromRGB(220, 228, 232)
9882 cancel.TextSize = 28
9883 cancel.Modal = true -- Let the pointer select an answer even in first-person view.
9884 cancel.Parent = panel
9885
9886 local activeId
9887 local submitted = false
9888 local buttons = {}
9889 local function answer(value)
9890     if not activeId or submitted then return end
9891     submitted = true
9892     status.Text = "Checking answer..."
9893     remote:FireServer(activeId, value)
9894 end
9895 for i = 1, 3 do
9896     local button = Instance.new("TextButton")
9897     button.Name = "Answer" .. i
9898     button.Position = UDim2.new((i - 1) / 3, 10, 0, 128)
9899     button.Size = UDim2.new(1 / 3, -14, 0, 56)
9900     button.BackgroundColor3 = Color3.fromRGB(50, 146, 112)
9901     button.TextColor3 = Color3.fromRGB(255, 255, 255)
9902     button.Font = Enum.Font.GothamBold
9903     button.TextSize = 26
9904     button.Parent = panel
9905     local rounding = Instance.new("UICorner")
9906     rounding.CornerRadius = UDim.new(0, 9)
9907     rounding.Parent = button
9908     button.Activated:Connect(function() answer(button:GetAttribute("Answer")) end)
9909     buttons[i] = button
9910 end
9911 cancel.Activated:Connect(function() answer(nil) end)
9912
9913 remote.OnClientEvent:Connect(function(payload)
9914     if typeof(payload) ~= "table" then return end
9915     if payload.Close then
9916         if payload.Id == activeId then panel.Visible = false; activeId = nil end
9917         return
9918     end
9919     activeId = payload.Id
9920     submitted = false
9921     reason.Text = tostring(payload.Reason or "Repair or collect a plant")
9922     question.Text = tostring(payload.Question or "?") .. " = ?"
9923     for i, button in ipairs(buttons) do
9924         local value = payload.Choices and payload.Choices[i]
9925         button.Text = tostring(value or "?")
9926         button:SetAttribute("Answer", value)
9927     end

```

```

9928     panel.Visible = true
9929     local thisId = activeId
9930     local expires = os.clock() + (tonumber(payload.Seconds) or 60)
9931     task.spawn(function()
9932         while activeId == thisId and panel.Visible do
9933             local left = math.max(0, math.ceil(expires - os.clock()))
9934             if not submitted then status.Text = "Tap the correct answer • " .. left .. "s • x cancels" end
9935             if left <= 0 then
9936                 if not submitted then answer(nil) end
9937                 panel.Visible = false
9938                 break
9939             end
9940             task.wait(0.2)
9941         end
9942     end)
9943 end)
9944 ]==]
9945
9946 -- Studio only permits Script.Source writes from trusted authoring contexts.
9947 -- If the generator itself is executed by a normal server Script during a
9948 -- play test, those writes fail. This compact live fallback binds the core
9949 -- gameplay directly so role prompts, held plants, recovery tools and worker
9950 -- tests still function in that exact situation.
9951 local function startLiveRuntimeFallback()
9952     local Players = game:GetService("Players")
9953     local TweenService = game:GetService("TweenService")
9954     local Debris = game:GetService("Debris")
9955     local PathfindingService = game:GetService("PathfindingService")
9956     local liveBound = {}
9957     local liveEmergencyMoves = 0
9958     local liveSampleSerial = 40000
9959     local liveRepairs, liveAssignedRepairs, liveTransfers, liveAssignedTransfers = 0, 0, 0, 0
9960     local LIVE_POINTS = {MOVE_ANY = 12, MOVE_LATE = 18, MOVE_ON_TIME = 30, PROCESS = 4}
9961     local liveAssignTask
9962
9963     local function livePlayerRole(player)
9964         local role = player:GetAttribute("LabRole")
9965         if role and role ~= "" then return role end
9966         return player.Team and player.Team.Name or "Unassigned"
9967     end
9968
9969     local function liveEquipmentId(part)
9970         local code = part:GetAttribute("GrowthCode")
9971         if code and code ~= "" then return "CHAMBER:" .. code end
9972         return table.concat({part:GetAttribute("Level") or "Lab", part:GetAttribute("Room") or "Room",
9973             ↪ part.Name}, "|")
9974     end
9975
9976     local function liveFloor(part)
9977         return part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "UNKNOWN FLOOR"
9978     end
9979
9980     local function liveTransferSeconds(source, destination)
9981         local distance = destination and (destination.Position - source.Position).Magnitude or 60
9982         local playerCount = math.max(1, #Players:GetPlayers())
9983         local complex = workspace:FindFirstChild("Laboratory_Complex")
9984         local backlog = complex and (complex:GetAttribute("FailedSampleBacklog") or 0) or 0
9985         return math.clamp(math.floor(58 + distance * 0.65 + math.max(0, 3 - playerCount) * 10
9986             ↪ + math.clamp(backlog / 20, 0, 25)), 60, 180)
9987     end
9988
9989     local function liveEquipmentLabel(part)
9990         local code = part:GetAttribute("GrowthCode")
9991         if code and code ~= "" then return "CHAMBER " .. code end
9992         return (part:GetAttribute("Kind") or part.Name) .. " / " .. (part:GetAttribute("Room") or "Lab")
9993     end
9994
9995     local function refreshLiveScoreboard()
9996         local rivalScore = 0
9997         for _, bot in ipairs(CollectionService:GetTagged("LabWorkerBot")) do
9998             if bot:GetAttribute("Competitor") then rivalScore = bot:GetAttribute("Score") or 0 break end
9999         end
10000         for _, board in ipairs(CollectionService:GetTagged("BatchScoreboard")) do
10001             local label = board:FindFirstChild("Text", true)
10002             if label and label:IsA("TextLabel") then
10003                 label.Text = string.format("PLAYER TEAM SCORE\nRepairs: %d • assigned: %d\nPlant moves: %d •
10004                 ↪ assigned: %d\nRival NPC score: %d",

```

```

10003         liveRepairs, liveAssignedRepairs, liveTransfers, liveAssignedTransfers, rivalScore)
10004     end
10005 end
10006 end
10007
10008 local function refreshLiveRoomMonitors()
10009     -- Room names repeat on every level, so key by level+room, not room alone.
10010     local totals = {}
10011     for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
10012         local room = equip:GetAttribute("Room")
10013         local level = equip:GetAttribute("Level") or ""
10014         if room then
10015             local key = level .. "|" .. room
10016             local t = totals[key]
10017             if not t then t = {running = 0, failed = 0, plants = 0, count = 0}; totals[key] = t end
10018             t.count = t.count + 1
10019             if equip:GetAttribute("FunctionalState") == "FAILED" then t.failed = t.failed + 1
10020             else t.running = t.running + 1 end
10021             t.plants = t.plants + (equip:GetAttribute("SampleCount") or 0)
10022         end
10023     end
10024     for _, panel in ipairs(CollectionService:GetTagged("RoomMonitor")) do
10025         local surface = panel:FindFirstChildWhichIsA("SurfaceGui")
10026         local label = surface and surface:FindFirstChild("Status")
10027         if label then
10028             local room = panel:GetAttribute("Room") or "ROOM"
10029             local level = panel:GetAttribute("Level") or ""
10030             local t = totals[level .. "|" .. room] or {running = 0, failed = 0, plants = 0, count = 0}
10031             label.Text = string.format("%s\nEquipment: %d • RUNNING %d • FAILED %d\nPlants stored: %d",
10032                 room, t.count, t.running, t.failed, t.plants)
10033             label.TextColor3 = (t.failed > 0) and Color3.fromRGB(255, 160, 90) or Color3.fromRGB(120, 235,
10034                 ↪ 170)
10035         end
10036     end
10037 end
10038
10039 local function liveAward(player, amount, reason)
10040     local stats = player:FindFirstChild("leaderstats")
10041     if not stats then stats = Instance.new("Folder"); stats.Name = "leaderstats"; stats.Parent = player end
10042     local points = stats:FindFirstChild("Points")
10043     if not points then points = Instance.new("IntValue"); points.Name = "Points"; points.Parent = stats end
10044     points.Value = points.Value + amount
10045     player:SetAttribute("LastPointAmount", amount)
10046     player:SetAttribute("LastPointAward", reason)
10047     refreshLiveScoreboard()
10048     return points.Value
10049 end
10050
10051 -- Math-quiz gate removed by request: pass straight through.
10052 local function runLiveMathQuiz(player, reason)
10053     return true
10054 end
10055
10056 local plantProcesses = {
10057     ["Germination Chamber"] = {name = "Germination", gain = 1},
10058     Incubator = {name = "Incubation", gain = 1},
10059     ["Growth Hormone Mixer"] = {name = "Growth Hormone", gain = 2},
10060     ["GMO Injector"] = {name = "GMO Injection", gain = 1},
10061     ["Nutrient Infuser"] = {name = "Nutrient Infusion", gain = 1},
10062     ["UV Growth Scanner"] = {name = "UV Scan", gain = 0},
10063     ["Pollination Station"] = {name = "Pollination", gain = 1},
10064 }
10065
10066 local function findPlant(player)
10067     for _, container in ipairs({player.Character, player:FindFirstChildOfClass("Backpack")}) do
10068         if container then
10069             for _, child in ipairs(container:GetChildren()) do
10070                 if child:IsA("Tool") and child:GetAttribute("PlantSample") then return child end
10071             end
10072         end
10073     end
10074 end
10075
10076 local function liveChamberSamples(chamber)
10077     local samples = {}
10078     for id in string.gmatch(chamber:GetAttribute("SampleIds") or "", "[^|]+") do
10079         table.insert(samples, id)
10080     end
10081 end

```

```

10078     return samples
10079 end
10080
10081 local function saveLiveChamber(chamber, samples)
10082     local capacity = chamber:GetAttribute("Capacity") or 10
10083     while #samples > capacity do table.remove(samples) end
10084     chamber:SetAttribute("SampleIds", table.concat(samples, "|"))
10085     chamber:SetAttribute("SampleCount", #samples)
10086     chamber:SetAttribute("Occupied", #samples > 0)
10087     chamber:SetAttribute("PlantId", samples[1] or "")
10088 end
10089
10090 local function equip(player, tool)
10091     local humanoid = player.Character and player.Character:FindFirstChildOfClass("Humanoid")
10092     if humanoid and tool then humanoid:EquipTool(tool) end
10093 end
10094
10095 local function weldPiece(tool, handle, name, size, offset, colour)
10096     local piece = Instance.new("Part")
10097     piece.Name = name
10098     piece.Size = size
10099     piece.CFrame = handle.CFrame * CFrame.new(offset)
10100     piece.Color = colour
10101     piece.Material = Enum.Material.SmoothPlastic
10102     piece.CanCollide = false
10103     piece.Massless = true
10104     piece.Parent = tool
10105     local weld = Instance.new("WeldConstraint")
10106     weld.Part0 = handle
10107     weld.Part1 = piece
10108     weld.Parent = piece
10109     return piece
10110 end
10111
10112 local function makePlantTool(player, chamber, requestedPlantId, destinationCode)
10113     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 3)
10114     if not backpack then return nil end
10115     local code = chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber)
10116     local plantId = requestedPlantId or chamber:GetAttribute("PlantId") or code
10117     local tool = Instance.new("Tool")
10118     tool.Name = "Plant Sample " .. plantId
10119     tool.ToolTip = "Visible germinated plant from " .. code
10120     tool.RequiresHandle = true
10121     tool.CanBeDropped = false
10122     tool:SetAttribute("LabSample", true)
10123     tool:SetAttribute("PlantSample", true)
10124     tool:SetAttribute("PlantId", plantId)
10125     tool:SetAttribute("SampleId", plantId)
10126     tool:SetAttribute("OriginGrowthCode", code)
10127     tool:SetAttribute("SourceGrowthCode", code)
10128     tool:SetAttribute("DestinationGrowthCode", destinationCode or "")
10129     tool:SetAttribute("TransferStartedAt", os.time())
10130     tool:SetAttribute("GrowthStage", chamber:GetAttribute("GrowthStage") or 0)
10131     tool:SetAttribute("Treatments", chamber:GetAttribute("Treatments") or "")
10132     tool:SetAttribute("Genome", chamber:GetAttribute("Genome") or "Wild Type")
10133     tool:SetAttribute("Moisture", chamber:GetAttribute("Moisture") or 50)
10134     local plantLife = chamber:GetAttribute("PlantLife") or 100
10135     if chamber:GetAttribute("FunctionalState") == "FAILED" then
10136         plantLife = math.clamp(100 - math.floor((os.time() - (chamber:GetAttribute("FailureSince") or
10137             ↪ os.time())) * 1.5), 0, 100)
10137     end
10138     tool:SetAttribute("PlantLife", plantLife)
10139     local handle = Instance.new("Part")
10140     handle.Name = "Handle"
10141     handle.Size = Vector3.new(1.05, 0.8, 1.05)
10142     handle.Color = Color3.fromRGB(116, 75, 45)
10143     handle.Material = Enum.Material.SmoothPlastic
10144     handle.CanCollide = false
10145     handle.Massless = true
10146     handle.Parent = tool
10147     weldPiece(tool, handle, "PlantStem", Vector3.new(0.15, 1.25, 0.15), Vector3.new(0, 0.9, 0),
10148         ↪ Color3.fromRGB(58, 160, 70))
10149     weldPiece(tool, handle, "LeafL", Vector3.new(0.7, 0.15, 0.4), Vector3.new(-0.32, 1.15, 0),
10150         ↪ Color3.fromRGB(73, 185, 84))
10151     weldPiece(tool, handle, "LeafR", Vector3.new(0.7, 0.15, 0.4), Vector3.new(0.32, 1.42, 0),
10152         ↪ Color3.fromRGB(73, 185, 84))

```

```

10150     local lifeGui = Instance.new("BillboardGui")
10151     lifeGui.Name = "PlantLifeStatus"
10152     lifeGui.Size = UDim2.fromOffset(160, 46)
10153     lifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1.72, 0)
10154     lifeGui.MaxDistance = 18
10155     lifeGui.AlwaysOnTop = false
10156     lifeGui.Parent = handle
10157     local lifeFrame = Instance.new("Frame")
10158     lifeFrame.Size = UDim2.fromScale(1, 1)
10159     lifeFrame.BackgroundColor3 = Color3.fromRGB(18, 20, 22)
10160     lifeFrame.BackgroundTransparency = 0.12
10161     lifeFrame.BorderSizePixel = 0
10162     lifeFrame.Parent = lifeGui
10163     local lifeText = Instance.new("TextLabel")
10164     lifeText.Size = UDim2.new(1, 0, 0.58, 0)
10165     lifeText.BackgroundTransparency = 1
10166     lifeText.TextScaled = true
10167     lifeText.Font = Enum.Font.GothamBold
10168     lifeText.Parent = lifeFrame
10169     local lifeBack = Instance.new("Frame")
10170     lifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
10171     lifeBack.Position = UDim2.new(0.05, 0, 0.7, 0)
10172     lifeBack.BackgroundColor3 = Color3.fromRGB(65, 65, 65)
10173     lifeBack.BorderSizePixel = 0
10174     lifeBack.Parent = lifeFrame
10175     local lifeFill = Instance.new("Frame")
10176     lifeFill.Name = "LifeBarFill"
10177     lifeFill.BorderSizePixel = 0
10178     lifeFill.Parent = lifeBack
10179     local function updateLife()
10180         local life = math.clamp(tool:GetAttribute("PlantLife") or 100, 0, 100)
10181         local colour = life > 60 and Color3.fromRGB(75, 235, 105)
10182             or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
10183         lifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
10184         lifeText.TextColor3 = colour
10185         lifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
10186         lifeFill.BackgroundColor3 = colour
10187     end
10188     tool:GetAttributeChangedSignal("PlantLife"):Connect(updateLife)
10189     updateLife()
10190     tool.Grip = CFrame.new(0, -0.65, -0.25) * CFrame.Angles(math.rad(-10), 0, 0)
10191     tool.Parent = backpack
10192     equip(player, tool)
10193     task.spawn(function()
10194         while tool.Parent and (tool:GetAttribute("PlantLife") or 0) > 0 do
10195             task.wait(1)
10196             if not tool.Parent then return end
10197             if not tool:GetAttribute("QuizProtected") then
10198                 local life = math.max(0, (tool:GetAttribute("PlantLife") or 0) - 2)
10199                 tool:SetAttribute("PlantLife", life)
10200                 if life <= 0 then
10201                     local reservedCode = tool:GetAttribute("DestinationGrowthCode") or ""
10202                     for _, candidate in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
10203                         if candidate:GetAttribute("GrowthCode") == reservedCode then
10204                             candidate:SetAttribute("IncomingReservations", math.max(0,
10205                                 ↪ (candidate:GetAttribute("IncomingReservations") or 1) - 1))
10206                             break
10207                         end
10208                     end
10209                     player:SetAttribute("CarriedSample", "")
10210                     player:SetAttribute("LabObjective", plantId .. " reached 0% LIFE and disappeared. Take
10211                         ↪ another plant.")
10212                     task.wait(0.5)
10213                     if tool.Parent then tool:Destroy() end
10214                     return
10215                 end
10216             end
10217         end
10218     end)
10219     return tool
10220 end
10221
10220     local function showChamberPlant(chamber, visible)
10221         local count = visible and (chamber:GetAttribute("SampleCount") or 0) or 0
10222         for _, child in ipairs(chamber:GetDescendants()) do
10223             if child:IsA("BasePart") and (child:GetAttribute("SampleSlot") or
10224                 ↪ child.Name:match("^ChamberPlant")) then

```

```

10224         local slot = child:GetAttribute("SampleSlot") or 1
10225         child.Transparency = slot <= count and (child:GetAttribute("StoredTransparency") or 0) or 1
10226     end
10227 end
10228
10229
10230 local function liveRuntimeText(seconds)
10231     seconds = math.max(0, math.floor(seconds or 0))
10232     return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
        ↪ seconds % 60)
10233 end
10234
10235 local function recoverLivePlantLife(chamber, startingLife, player, plantId)
10236     startingLife = math.clamp(startingLife or 100, 0, 100)
10237     if startingLife >= 100 or chamber:GetAttribute("FunctionalState") == "FAILED" then
10238         chamber:SetAttribute("PlantLife", startingLife)
10239         return 0
10240     end
10241     local seconds = math.clamp(math.ceil((100 - startingLife) / 16), 3, 7)
10242     local token = (chamber:GetAttribute("RecoveryToken") or 0) + 1
10243     chamber:SetAttribute("RecoveryToken", token)
10244     chamber:SetAttribute("RecoveryEndsAt", os.time() + seconds)
10245     chamber:SetAttribute("PlantLife", startingLife)
10246     if player then
10247         player:SetAttribute("LabObjective", string.format("%s dropped in %s. LIFE %d%% → 100%% in %d
        ↪ seconds.",
10248             plantId or "Plant", chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber),
        ↪ startingLife, seconds))
10249     end
10250     task.spawn(function()
10251         for step = 1, seconds do
10252             task.wait(1)
10253             if not chamber.Parent or chamber:GetAttribute("RecoveryToken") ~= token
10254                 or chamber:GetAttribute("FunctionalState") == "FAILED" then return end
10255             chamber:SetAttribute("PlantLife", math.floor(startingLife + (100 - startingLife) * step /
        ↪ seconds + 0.5))
10256         end
10257         chamber:SetAttribute("PlantLife", 100)
10258         chamber:SetAttribute("RecoveryEndsAt", 0)
10259         if player and player.Parent then player:SetAttribute("LabObjective", (plantId or "Plant") .. "
        ↪ recovered to 100% LIFE.") end
10260     end)
10261     return seconds
10262 end
10263
10264 local function refreshLiveTelemetry(chamber)
10265     local samples = liveChamberSamples(chamber)
10266     local state = chamber:GetAttribute("FunctionalState") or "WORKING"
10267     local code = chamber:GetAttribute("GrowthCode") or liveEquipmentLabel(chamber)
10268     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN
        ↪ FLOOR"
10269     local destination = chamber:GetAttribute("EvacuationDestination") or ""
10270     local detail = state == "FAILED" and ((chamber:GetAttribute("FailureReason") or "FAULT")
        ↪ .. (destination ~= "" and (" • EVACUATE TO " .. destination) or "")) or "SYSTEM NORMAL"
10271     local text = string.format(
10272         "%s • CHAMBER %s • %s\nSAMPLES %d/%d • GERMINATED %s\nRUN %s • TEMP %d C • HUM %d%% • VENT
        ↪ %d%%\n%s\n%s",
10273         floorLabel, code, state, #samples,
10274         chamber:GetAttribute("Capacity") or 0,
10275         chamber:GetAttribute("GerminationDate") or "pending",
10276         liveRuntimeText(os.time() - (chamber:GetAttribute("OperationStarted") or os.time())),
10277         chamber:GetAttribute("Temperature") or 23, chamber:GetAttribute("Humidity") or 72,
10278         chamber:GetAttribute("Ventilation") or 75, detail,
10279         state == "FAILED" and "REPAIR: HOLD R" or "CONTROL: E TAKE / DROP")
10280     for _, child in ipairs(chamber:GetDescendants()) do
10281         if child:IsA("TextLabel") and child.Name == "ChamberTelemetry" then
10282             child.Text = text
10283             child.TextColor3 = state == "DESTROYED" and Color3.fromRGB(255, 50, 40)
10284                 or state == "FAILED" and Color3.fromRGB(255, 120, 105)
10285                 or Color3.fromRGB(130, 245, 165)
10286             elseif child:IsA("TextLabel") and child.Name == "ChamberSampleList" then
10287                 child.Text = "PLANTS (" .. #samples .. "): "
10288                 .. (#samples > 0 and table.concat(samples, ", ") or "NO SAMPLES")
10289             elseif child:IsA("BasePart") and child:GetAttribute("ChamberStatusLamp") then
10290                 child.Color = state == "DESTROYED" and Color3.fromRGB(200, 20, 15)
10291                 or state == "FAILED" and Color3.fromRGB(230, 55, 45)
10292

```



```

10293         or Color3.fromRGB(40, 165, 95)
10294         child.Material = state == "DESTROYED" and Enum.Material.Neon or Enum.Material.SmoothPlastic
10295     end
10296 end
10297 end
10298
10299 local function liveDestinations(source)
10300     local choices = {}
10301     for _, candidate in ipairs(CollectionService.GetTagged("PlantGrowthChamber")) do
10302         local count = candidate:GetAttribute("SampleCount") or 0
10303         local capacity = candidate:GetAttribute("Capacity") or 10
10304         local reservations = candidate:GetAttribute("IncomingReservations") or 0
10305         if candidate ~= source and candidate:GetAttribute("FunctionalState") ~= "FAILED"
10306             and count + reservations < capacity then
10307             table.insert(choices, candidate)
10308         end
10309     end
10310     table.sort(choices, function(a, b)
10311         local sameA = a:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
10312         local sameB = b:GetAttribute("Level") == source:GetAttribute("Level") and 0 or 1
10313         if sameA ~= sameB then return sameA < sameB end
10314         local ac = a:GetAttribute("SampleCount") or 0
10315         local bc = b:GetAttribute("SampleCount") or 0
10316         if ac ~= bc then return ac < bc end
10317         return (a:GetAttribute("GrowthCode") or "") < (b:GetAttribute("GrowthCode") or "")
10318     end)
10319     return choices
10320 end
10321
10322 local function liveRepairPathFolder(player, create)
10323     local complex = workspace:FindFirstChild("Laboratory_Complex")
10324     if not complex then return nil end
10325     local rootFolder = complex:FindFirstChild("TechnicianRepairPaths")
10326     if create and not rootFolder then
10327         rootFolder = Instance.new("Folder")
10328         rootFolder.Name = "TechnicianRepairPaths"
10329         rootFolder.Parent = complex
10330     end
10331     local folder = rootFolder and rootFolder:FindFirstChild("Player_" .. player.UserId)
10332     if create and not folder then
10333         folder = Instance.new("Folder")
10334         folder.Name = "Player_" .. player.UserId
10335         folder.Parent = rootFolder
10336     end
10337     return folder
10338 end
10339
10340 local function clearLiveRepairPath(player)
10341     local folder = liveRepairPathFolder(player, false)
10342     if folder then folder:Destroy() end
10343 end
10344
10345 local function liveRepairReward(player, equipment, assigned)
10346     local waited = math.max(0, os.time() - (equipment:GetAttribute("FailureSince") or os.time()))
10347     local distance = assigned and (player:GetAttribute("AssignedRepairDistance") or 0) or 0
10348     return math.clamp(8 + math.floor(waited / 8) + math.floor(distance / 10) + (assigned and 12 or 0), 8,
10349         ↪ 120), waited, distance
10350 end
10351
10352 local function createLiveRepairPath(player, target)
10353     clearLiveRepairPath(player)
10354     local characterRoot = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
10355     if not characterRoot or not target then return 0, false end
10356     local folder = liveRepairPathFolder(player, true)
10357     if not folder then return 0, false end
10358     local function approachPoint(part)
10359         local side = part:GetAttribute("WallSide")
10360         local floorY = part.Position.Y - part.Size.Y / 2
10361         if side == "W" then return Vector3.new(part.Position.X + part.Size.X / 2 + 2.5, floorY,
10362             ↪ part.Position.Z) end
10363         if side == "E" then return Vector3.new(part.Position.X - part.Size.X / 2 - 2.5, floorY,
10364             ↪ part.Position.Z) end
10365         if side == "N" then return Vector3.new(part.Position.X, floorY, part.Position.Z + part.Size.Z / 2 +
10366             ↪ 2.5) end
10367         return Vector3.new(part.Position.X, floorY, part.Position.Z - part.Size.Z / 2 - 2.5)
10368     end
10369 end

```

```

10365     local function appendNavigable(points, fromPosition, toPosition)
10366         local path = PathfindingService.CreatePath({
10367             AgentRadius = 2, AgentHeight = 5, AgentCanJump = true,
10368             AgentCanClimb = true, WaypointSpacing = 4,
10369         })
10370         local computed = pcall(function() path:ComputeAsync(fromPosition, toPosition) end)
10371         if not computed or path.Status ~= Enum.PathStatus.Success then return false end
10372         local waypoints = path:GetWaypoints()
10373         if #waypoints < 2 then return false end
10374         for index, waypoint in ipairs(waypoints) do
10375             if index > 1 or #points == 0 then table.insert(points, waypoint.Position) end
10376         end
10377         return true
10378     end
10379
10380     local complex = workspace:FindFirstChild("Laboratory_Complex")
10381     local rise = complex and complex:GetAttribute("FloorRiseStuds")
10382     local westX = complex and complex:GetAttribute("StairWestLaneX")
10383     local eastX = complex and complex:GetAttribute("StairEastLaneX")
10384     local northZ = complex and complex:GetAttribute("StairNorthZ")
10385     local southZ = complex and complex:GetAttribute("StairSouthZ")
10386     local floorMin = (complex and complex:GetAttribute("FloorMinLevel")) or -1
10387     local floorMax = (complex and complex:GetAttribute("FloorMaxLevel")) or 2
10388     local destination = approachPoint(target)
10389     local points = {}
10390     local routeReady = false
10391     if rise and westX and eastX and northZ and southZ then
10392         local startFloor = math.clamp(math.floor((characterRoot.Position.Y - 2.5) / rise + 0.5), floorMin,
10393             ↪ floorMax)
10394         local targetFloor = math.clamp(math.floor(destination.Y / rise + 0.5), floorMin, floorMax)
10395         if startFloor == targetFloor then
10396             routeReady = appendNavigable(points, characterRoot.Position, destination)
10397         else
10398             local direction = targetFloor > startFloor and 1 or -1
10399             local firstLaneX = direction > 0 and westX or eastX
10400             local stairEntry = Vector3.new(firstLaneX, startFloor * rise, northZ)
10401             routeReady = appendNavigable(points, characterRoot.Position, stairEntry)
10402             local floor = startFloor
10403             while routeReady and floor ~= targetFloor do
10404                 local baseY = floor * rise
10405                 if direction > 0 then
10406                     table.insert(points, Vector3.new(westX, baseY + rise / 2, southZ))
10407                     table.insert(points, Vector3.new(eastX, baseY + rise / 2, southZ))
10408                     table.insert(points, Vector3.new(eastX, baseY + rise, northZ))
10409                 else
10410                     table.insert(points, Vector3.new(eastX, baseY - rise / 2, southZ))
10411                     table.insert(points, Vector3.new(westX, baseY - rise / 2, southZ))
10412                     table.insert(points, Vector3.new(westX, baseY - rise, northZ))
10413                 end
10414                 floor = floor + direction
10415                 if floor ~= targetFloor then
10416                     table.insert(points, Vector3.new(direction > 0 and westX or eastX, floor * rise,
10417                         ↪ northZ))
10418                 end
10419             end
10420             if routeReady then routeReady = appendNavigable(points, points[#points], destination) end
10421         end
10422     end
10423     if not routeReady then
10424         folder:Destroy()
10425         player:SetAttribute("RepairRouteReady", false)
10426         return (destination - characterRoot.Position).Magnitude, false
10427     end
10428     player:SetAttribute("RepairRouteReady", true)
10429     local distance, markerCount = 0, 0
10430     for pointIndex = 1, #points - 1 do
10431         local a, b = points[pointIndex], points[pointIndex + 1]
10432         local delta = b - a
10433         local segmentDistance = delta.Magnitude
10434         distance = distance + segmentDistance
10435         if segmentDistance > 0.1 then
10436             local direction = delta.Unit
10437             local count = math.max(1, math.ceil(segmentDistance / 4))
10438             for index = 1, count do
10439                 markerCount = markerCount + 1
10440                 if markerCount > 120 then break end

```

```

10439         local marker = Instance.new("Part")
10440         marker.Name = "RedRepairPath"
10441         marker.Size = Vector3.new(0.55, 0.08, math.min(2.8, segmentDistance / count * 0.7))
10442         local position = a:Lerp(b, (index - 0.5) / count) + Vector3.new(0, 0.1, 0)
10443         marker.CFrame = CFrame.lookAt(position, position + direction)
10444         marker.Color = Color3.fromRGB(255, 32, 28)
10445         marker.Material = Enum.Material.Neon
10446         marker.Transparency = 0.12
10447         marker.Anchored = true
10448         marker.CanCollide = false
10449         marker.CanTouch = false
10450         marker.CanQuery = false
10451         marker.Parent = folder
10452     end
10453 end
10454 if markerCount > 120 then break end
10455 end
10456 return distance, true
10457 end
10458
10459 liveAssignTask = function(player)
10460     clearLiveRepairPath(player)
10461     for _, name in ipairs({"AssignedRepairId", "AssignedRepairDistance", "AssignedRepairProjectedPoints",
10462         ⇨ "AssignedTransferSource", "AssignedTransferDestination", "AssignedTransferDeadline"}) do
10463         player:SetAttribute(name, nil)
10464     end
10465     if livePlayerRole(player) == "Technician" then
10466         local failed = {}
10467         for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
10468             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment)
10469             ⇨ end
10470         end
10471         local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
10472         table.sort(failed, function(a, b)
10473             if root then
10474                 local ad = (a.Position - root.Position).Magnitude + (math.abs(a.Position.Y -
10475                     ⇨ root.Position.Y) > 12 and 500 or 0)
10476                 local bd = (b.Position - root.Position).Magnitude + (math.abs(b.Position.Y -
10477                     ⇨ root.Position.Y) > 12 and 500 or 0)
10478                 if ad ~= bd then return ad < bd end
10479             end
10480             return (a:GetAttribute("FailureSince") or 0) < (b:GetAttribute("FailureSince") or 0)
10481         end)
10482         local target = failed[1]
10483         local text = "All equipment is working. Repair score grows with failure time; assigned jobs also
10484             ⇨ pay for travel distance."
10485         if target then
10486             player:SetAttribute("AssignedRepairId", liveEquipmentId(target))
10487             local distance, routeReady = createLiveRepairPath(player, target)
10488             player:SetAttribute("AssignedRepairDistance", distance)
10489             local projected = select(1, liveRepairReward(player, target, true))
10490             player:SetAttribute("AssignedRepairProjectedPoints", projected)
10491             local routeInstruction = routeReady and "Follow RED floor lights" or "Use the NORTH STAIR;
10492             ⇨ route is recalculating"
10493             text = string.format("REPAIR JOB: %s • %s. %s. HOLD R. About +%d points (%d studs; score grows
10494                 ⇨ while failed).",
10495                 liveFloor(target), liveEquipmentLabel(target), routeInstruction, projected,
10496                 ⇨ math.floor(distance + 0.5))
10497         end
10498         player:SetAttribute("LabObjective", text)
10499         return text
10500     end
10501     local jobs = {}
10502     for _, source in ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
10503         if source:GetAttribute("FunctionalState") == "FAILED" and (source:GetAttribute("SampleCount") or 0)
10504             ⇨ > 0
10505         and not source:GetAttribute("ReservedByBot") then
10506             local destination = liveDestinations(source)[1]
10507             if destination then table.insert(jobs, {source = source, destination = destination}) end
10508         end
10509     end
10510     table.sort(jobs, function(a, b)
10511         return (tonumber((a.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
10512             < (tonumber((b.source:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
10513     end)
10514     local job = jobs[1]

```

```

10506     if not job then
10507         local text = "Waiting for a failed chamber with plants. Emergency move +12; assigned on-time move
            ↳ +30."
10508         player:SetAttribute("LabObjective", text)
10509         return text
10510     end
10511     local sourceCode = job.source:GetAttribute("GrowthCode") or "R?"
10512     local destinationCode = job.destination:GetAttribute("GrowthCode") or "R?"
10513     player:SetAttribute("AssignedTransferSource", sourceCode)
10514     player:SetAttribute("AssignedTransferDestination", destinationCode)
10515     local allowedSeconds = liveTransferSeconds(job.source, job.destination)
10516     player:SetAttribute("AssignedTransferDeadline", os.time() + allowedSeconds)
10517     -- Same red floor-marker route the Technician role gets, pointed at
10518     -- the destination chamber so it's obvious where the plant goes.
10519     local routeReady = select(2, createLiveRepairPath(player, job.destination))
10520     local routeInstruction = routeReady and "Follow RED floor lights to the destination chamber" or "Route
            ↳ recalculating"
10521     local text = "ASSIGNED +30: " .. liveFloor(job.source) .. " * " .. sourceCode .. " → " ..
            ↳ destinationCode
            ↳ .. " * " .. liveFloor(job.destination) .. ". PRESS E; " .. routeInstruction .. ". Deliver in " ..
            ↳ allowedSeconds .. " seconds."
10523     player:SetAttribute("LabObjective", text)
10524     return text
10525 end
10526
10527 local setLiveFailure
10528 local setLiveGenericFailure
10529
10530 local function controlCFrame(part, yOffset, standOff)
10531     local side = part:GetAttribute("WallSide")
10532     if side == "W" then return part.CFrame * CFrame.new(part.Size.X / 2 + standOff, yOffset, 0), true end
10533     if side == "E" then return part.CFrame * CFrame.new(-part.Size.X / 2 - standOff, yOffset, 0), true end
10534     if side == "S" then return part.CFrame * CFrame.new(0, yOffset, -part.Size.Z / 2 - standOff), false end
10535     return part.CFrame * CFrame.new(0, yOffset, part.Size.Z / 2 + standOff), false
10536 end
10537
10538 local function bindLiveEquipment(part)
10539     if liveBound[part] or not part:IsA("BasePart") then return end
10540     liveBound[part] = true
10541     local kind = part:GetAttribute("Kind") or part.Name
10542     local isGrowthChamber = kind == "Plant Growth Chamber"
10543     local isPlantContainer = (part:GetAttribute("Capacity") or 0) > 0
10544     part:SetAttribute("EquipmentId", liveEquipmentId(part))
10545     part:SetAttribute("FloorLabel", liveFloor(part))
10546     part:SetAttribute("RepairKey", "R")
10547     if part:GetAttribute("FunctionalState") == nil then
10548         local hash = 0
10549         for i = 1, #liveEquipmentId(part) do hash = hash + string.byte(liveEquipmentId(part), i) end
10550         local failed = not isGrowthChamber and hash % 7 == 0
10551         part:SetAttribute("FunctionalState", failed and "FAILED" or "WORKING")
10552         part:SetAttribute("FailureReason", failed and "Control-system diagnostic fault" or "")
10553         part:SetAttribute("FailureSince", failed and os.time() or 0)
10554     end
10555     if part:GetAttribute("OperationStarted") == nil then part:SetAttribute("OperationStarted", os.time())
            ↳ end
10556     local display = part
10557     local readout = {Text = "", TextColor3 = Color3.fromRGB(115, 235, 165)}
10558     local timerGui = Instance.new("BillboardGui")
10559     timerGui.Name = "EquipmentStatusTimer"
10560     timerGui.Size = UDim2.fromOffset(160, 32)
10561     timerGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 3.8, 0)
10562     timerGui.MaxDistance = 10
10563     timerGui.AlwaysOnTop = false
10564     timerGui.Parent = part
10565     local timerLabel = Instance.new("TextLabel")
10566     timerLabel.Name = "Timer"
10567     timerLabel.Size = UDim2.fromScale(1, 1)
10568     timerLabel.BackgroundColor3 = Color3.fromRGB(12, 16, 18)
10569     timerLabel.BackgroundTransparency = 0.16
10570     timerLabel.BorderSizePixel = 0
10571     timerLabel.TextScaled = true
10572     timerLabel.Font = Enum.Font.GothamBold
10573     timerLabel.Parent = timerGui
10574     local timerCorner = Instance.new("UICorner")
10575     timerCorner.CornerRadius = UDim.new(0, 8)
10576     timerCorner.Parent = timerLabel

```

```

10577
10578     local nameGui = Instance.new("BillboardGui")
10579     nameGui.Name = "EquipmentNamePlate"
10580     nameGui.Size = UDim2.fromOffset(160, 24)
10581     nameGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 5.4, 0)
10582     nameGui.MaxDistance = 10
10583     nameGui.AlwaysOnTop = false
10584     nameGui.Parent = part
10585     local nameLabel = Instance.new("TextLabel")
10586     nameLabel.Name = "Kind"
10587     nameLabel.Size = UDim2.fromScale(1, 1)
10588     nameLabel.BackgroundColor3 = Color3.fromRGB(235, 200, 60)
10589     nameLabel.BackgroundTransparency = 0.05
10590     nameLabel.BorderSizePixel = 0
10591     nameLabel.Text = kind
10592     nameLabel.TextColor3 = Color3.fromRGB(20, 20, 20)
10593     nameLabel.TextScaled = true
10594     nameLabel.Font = Enum.Font.SourceSansBold
10595     nameLabel.Parent = nameGui
10596     local nameCorner = Instance.new("UICorner")
10597     nameCorner.CornerRadius = UDim.new(0, 8)
10598     nameCorner.Parent = nameLabel
10599
10600     local slotGui = Instance.new("BillboardGui")
10601     slotGui.Name = "EquipmentFreeSlots"
10602     slotGui.Size = UDim2.fromOffset(150, 28)
10603     slotGui.StudsOffsetWorldSpace = Vector3.new(0, part.Size.Y / 2 + 2.7, 0)
10604     slotGui.MaxDistance = 10
10605     slotGui.AlwaysOnTop = false
10606     slotGui.Parent = part
10607     local slotLabel = Instance.new("TextLabel")
10608     slotLabel.Name = "FreeSlots"
10609     slotLabel.Size = UDim2.fromScale(1, 1)
10610     slotLabel.BackgroundColor3 = Color3.fromRGB(12, 19, 22)
10611     slotLabel.BackgroundTransparency = 0.12
10612     slotLabel.BorderSizePixel = 0
10613     slotLabel.TextScaled = true
10614     slotLabel.Font = Enum.Font.GothamBold
10615     slotLabel.Parent = slotGui
10616     local slotCorner = Instance.new("UICorner")
10617     slotCorner.CornerRadius = UDim.new(0, 8)
10618     slotCorner.Parent = slotLabel
10619     local storedLifeGui, storedLifeText, storedLifeFill
10620     if isPlantContainer then
10621         local plantAnchor = part:FindFirstChild("ChamberSample_01_Stem") or part
10622         storedLifeGui = Instance.new("BillboardGui")
10623         storedLifeGui.Name = "StoredPlantLifeStatus"
10624         storedLifeGui.Size = UDim2.fromOffset(140, 40)
10625         storedLifeGui.StudsOffsetWorldSpace = Vector3.new(0, 1, 0)
10626         storedLifeGui.MaxDistance = 10
10627         storedLifeGui.AlwaysOnTop = false
10628         storedLifeGui.Parent = plantAnchor
10629         storedLifeText = Instance.new("TextLabel")
10630         storedLifeText.Size = UDim2.new(1, 0, 0.62, 0)
10631         storedLifeText.BackgroundColor3 = Color3.fromRGB(16, 20, 18)
10632         storedLifeText.BackgroundTransparency = 0.16
10633         storedLifeText.BorderSizePixel = 0
10634         storedLifeText.TextScaled = true
10635         storedLifeText.Font = Enum.Font.GothamBold
10636         storedLifeText.Parent = storedLifeGui
10637         local storedLifeBack = Instance.new("Frame")
10638         storedLifeBack.Size = UDim2.new(0.9, 0, 0.2, 0)
10639         storedLifeBack.Position = UDim2.new(0.05, 0, 0.72, 0)
10640         storedLifeBack.BackgroundColor3 = Color3.fromRGB(55, 55, 55)
10641         storedLifeBack.BorderSizePixel = 0
10642         storedLifeBack.Parent = storedLifeGui
10643         storedLifeFill = Instance.new("Frame")
10644         storedLifeFill.Size = UDim2.fromScale(1, 1)
10645         storedLifeFill.BorderSizePixel = 0
10646         storedLifeFill.Parent = storedLifeBack
10647     end
10648     local function liveClockText(seconds)
10649         seconds = math.max(0, math.floor(seconds or 0))
10650         return string.format("%02d:%02d:%02d", math.floor(seconds / 3600), math.floor(seconds / 60) % 60,
10651             ↪ seconds % 60)
10652     end
10653 end

```

```

10652     local function renderLiveTimer()
10653         local state = part:GetAttribute("FunctionalState")
10654         local destroyed = state == "DESTROYED"
10655         local failed = destroyed or state == "FAILED"
10656         local since = failed and (part:GetAttribute("FailureSince") or os.time())
10657             or (part:GetAttribute("OperationStarted") or os.time())
10658         if destroyed then
10659             timerLabel.Text = "DESTROYED -- UNFIXABLE"
10660             timerLabel.TextColor3 = Color3.fromRGB(90, 90, 95)
10661         else
10662             timerLabel.Text = (failed and "FAILED " or "RUNNING ") .. liveClockText(os.time() - since)
10663             timerLabel.TextColor3 = failed and Color3.fromRGB(255, 75, 65) or Color3.fromRGB(75, 255, 125)
10664         end
10665         if isPlantContainer and failed then
10666             local life = math.clamp(100 - math.floor((os.time() - since) * 1.5), 0, 100)
10667             part:SetAttribute("PlantLife", life)
10668             if life <= 0 and (part:GetAttribute("SampleCount") or 0) > 0 then
10669                 part:SetAttribute("SampleIds", "")
10670                 part:SetAttribute("SampleCount", 0)
10671                 part:SetAttribute("Occupied", false)
10672                 part:SetAttribute("PlantId", "")
10673                 part:SetAttribute("PlantExpiredAt", os.time())
10674                 showChamberPlant(part, false)
10675             end
10676         end
10677         local capacity = part:GetAttribute("Capacity") or 0
10678         local count = math.clamp(part:GetAttribute("SampleCount") or 0, 0, capacity)
10679         local free = math.max(0, capacity - count)
10680         slotLabel.Text = free == 0 and ("FULL • " .. count .. "/" .. capacity)
10681             or string.format("%d FREE SLOT%s • %d/%d", free, free == 1 and "" or "S", count, capacity)
10682         slotLabel.TextColor3 = free == 0 and Color3.fromRGB(255, 80, 70) or Color3.fromRGB(90, 235, 180)
10683         local incoming = (part:GetAttribute("IncomingReservations") or 0) > 0
10684         if incoming and not failed then
10685             slotGui.MaxDistance = 30
10686             slotGui.AlwaysOnTop = false
10687             slotLabel.Text = "★ DROP HERE ★ " .. slotLabel.Text
10688             slotLabel.TextColor3 = Color3.fromRGB(120, 255, 170)
10689         else
10690             slotGui.MaxDistance = 10
10691             slotGui.AlwaysOnTop = false
10692         end
10693         if storedLifeGui then
10694             local life = math.clamp(part:GetAttribute("PlantLife") or 100, 0, 100)
10695             local colour = life > 60 and Color3.fromRGB(75, 235, 105)
10696                 or (life > 30 and Color3.fromRGB(255, 190, 55) or Color3.fromRGB(255, 70, 60))
10697             storedLifeGui.Enabled = (part:GetAttribute("SampleCount") or 0) > 0
10698             storedLifeText.Text = "PLANT LIFE " .. math.floor(life + 0.5) .. "%"
10699             storedLifeText.TextColor3 = colour
10700             storedLifeFill.Size = UDim2.new(life / 100, 0, 1, 0)
10701             storedLifeFill.BackgroundColor3 = colour
10702         end
10703     end
10704     renderLiveTimer()
10705     task.spawn(function()
10706         while part.Parent do renderLiveTimer(); task.wait(1) end
10707     end)
10708
10709     local button = Instance.new("Part")
10710     button.Name = "PushButton"
10711     button.Shape = Enum.PartType.Ball
10712     button.Size = isGrowthChamber and Vector3.new(0.62, 0.62, 0.62) or Vector3.new(0.42, 0.42, 0.42)
10713     button.CFrame = select(1, controlCFrame(part, 0, 0.22))
10714     button.Color = Color3.fromRGB(220, 65, 55)
10715     button.Material = Enum.Material.Neon
10716     button.Anchored = true
10717     button.CanCollide = false
10718     button.Parent = part
10719     local usePrompt = Instance.new("ProximityPrompt")
10720     usePrompt.ActionText = isPlantContainer and "E: TAKE / DROP PLANT" or "E: USE / INSPECT"
10721     usePrompt.ObjectText = isPlantContainer
10722         and ((part:GetAttribute("FloorLabel") or part:GetAttribute("Level") or "FLOOR") .. " • " ..
10723             ↪ liveEquipmentLabel(part))
10724         or kind
10725     usePrompt.KeyboardKeyCode = Enum.KeyCode.E
10726     usePrompt.MaxActivationDistance = 8
10727     usePrompt.RequiresLineOfSight = true

```

```

10727     usePrompt.Parent = button
10728
10729     local led = Instance.new("Part")
10730     led.Name = "StatusLED"
10731     led.Shape = Enum.PartType.Ball
10732     led.Size = Vector3.new(0.28, 0.28, 0.28)
10733     led.CFrame = select(1, controlCFrame(part, -0.55, 0.24))
10734     led.Color = Color3.fromRGB(95, 240, 125)
10735     led.Material = Enum.Material.Neon
10736     led.Anchored = true
10737     led.CanCollide = false
10738     led.Parent = part
10739     local powered = true
10740     part:SetAttribute("Powered", true)
10741     local repairBusy = false
10742
10743     local floorLabel = liveFloor(part)
10744     local unitLabel = liveEquipmentLabel(part)
10745     local repairPrompt = Instance.new("ProximityPrompt")
10746     repairPrompt.Name = "RepairPrompt"
10747     repairPrompt.ActionText = "Hold R to repair"
10748     repairPrompt.ObjectText = floorLabel .. " " • " .. unitLabel
10749     repairPrompt.KeyboardKeyCode = Enum.KeyCode.R
10750     repairPrompt.GamepadKeyCode = Enum.KeyCode.ButtonB
10751     repairPrompt.HoldDuration = 3
10752     repairPrompt.MaxActivationDistance = 8
10753     repairPrompt.RequiresLineOfSight = true
10754     repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
10755     repairPrompt.Parent = part:FindFirstChild("ControllerScreen") or display
10756
10757     part:GetAttributeChangedSignal("FunctionalState"):Connect(function()
10758         if repairPrompt.Parent then repairPrompt.Enabled = not repairBusy and
            ↳ part:GetAttribute("FunctionalState") == "FAILED" end
10759     end)
10760     repairPrompt.Triggered:Connect(function(player)
10761         if livePlayerRole(player) ~= "Technician" then
10762             readout.Text = "TECHNICIAN REQUIRED\nChoose the BLUE role kiosk"
10763             return
10764         end
10765         if repairBusy or part:GetAttribute("FunctionalState") ~= "FAILED" then return end
10766         repairBusy = true
10767         repairPrompt.Enabled = false
10768         readout.Text = "REPAIRING\n" .. floorLabel .. " " • " .. unitLabel
10769         if not runLiveMathQuiz(player, "REPAIR " .. floorLabel .. " " • " .. unitLabel) then
10770             readout.Text = "REPAIR NOT COMPLETED\nHold R for 3 seconds to try again"
10771             repairBusy = false
10772             repairPrompt.Enabled = part:GetAttribute("FunctionalState") == "FAILED"
10773             return
10774         end
10775         local assigned = player:GetAttribute("AssignedRepairId") == liveEquipmentId(part)
10776         local points, failedSeconds, routeDistance = liveRepairReward(player, part, assigned)
10777         part:SetAttribute("LastRepairedAt", os.time())
10778         part:SetAttribute("LastRepairedBy", player.Name)
10779         if isGrowthChamber then setLiveFailure(part, false) else setLiveGenericFailure(part, false) end
10780         liveRepairs = liveRepairs + 1
10781         if assigned then liveAssignedRepairs = liveAssignedRepairs + 1 end
10782         local total = liveAward(player, points, string.format("%s%s • failed %ds • route %d studs",
10783             assigned and "assigned repair: " or "equipment repair: ", unitLabel, failedSeconds,
10784             ↳ math.floor(routeDistance + 0.5)))
10785         readout.Text = "REPAIR COMPLETE +" .. points .. "\n" .. floorLabel .. " " • " .. unitLabel ..
            ↳ "\nPLANTS "
10786         .. (part:GetAttribute("SampleCount") or 0) .. "/" .. (part:GetAttribute("Capacity") or 0) .. "
            ↳ • TOTAL " .. total
10787         liveAssignTask(player)
10788         repairBusy = false
10789     end)
10790     usePrompt.Triggered:Connect(function(player)
10791         button.Color = Color3.fromRGB(255, 205, 65)
10792         task.delay(0.2, function() if button.Parent then button.Color = Color3.fromRGB(220, 65, 55) end
            ↳ end)
10793         if not isPlantContainer and part:GetAttribute("FunctionalState") == "FAILED" then
10794             readout.Text = "FAILED • HOLD R\n" .. liveFloor(part) .. " " • " ..
                ↳ (part:GetAttribute("FailureReason") or "fault")
10795             return
10796         end

```

```

10797     local carriedPlant = findPlant(player)
10798     local plantProcess = plantProcesses[kind]
10799     local shouldUsePlantStorage = isPlantContainer and (
10800         isGrowthChamber
10801         or not plantProcess
10802         or not carriedPlant
10803         or (carriedPlant:GetAttribute("PlantLife") or 100) < 100
10804     )
10805     if shouldUsePlantStorage then
10806         if livePlayerRole(player) ~= "Laboratory Technician" then
10807             readout.Text = "LAB ROLE REQUIRED\nGREEN kiosk moves plants"
10808             return
10809         end
10810         -- Players override test-bot reservations so E always performs the
10811         -- obvious take/drop action when they are standing at the cabinet.
10812         if part:GetAttribute("ReservedByBot") then part:SetAttribute("ReservedByBot", nil) end
10813         local plant = carriedPlant
10814         local code = part:GetAttribute("GrowthCode") or liveEquipmentLabel(part)
10815         local samples = liveChamberSamples(part)
10816         local capacity = part:GetAttribute("Capacity") or 10
10817         local failed = part:GetAttribute("FunctionalState") == "FAILED"
10818         if plant then
10819             local wanted = plant:GetAttribute("DestinationGrowthCode") or ""
10820             if failed then
10821                 readout.Text = code .. " FAILED\nCannot store plants here"
10822                 player:SetAttribute("LabObjective", code .. " is FAILED. Find any GREEN RUNNING cabinet
↳ marked FREE SLOT and press E to drop the plant.")
10823             elseif #samples + (part:GetAttribute("IncomingReservations") or 0) >= capacity then
10824                 readout.Text = code .. " FULL\nChoose another working chamber"
10825             else
10826                 local plantId = plant:GetAttribute("PlantId") or plant.Name
10827                 local plantLife = math.clamp(plant:GetAttribute("PlantLife") or 100, 0, 100)
10828                 local sourceCode = plant:GetAttribute("SourceGrowthCode") or
↳ plant:GetAttribute("OriginGrowthCode") or ""
10829                 local assigned = wanted ~= "" and player:GetAttribute("AssignedTransferSource") ==
↳ sourceCode
↳ and player:GetAttribute("AssignedTransferDestination") == code
10830                 local onTime = assigned and os.time() <=
↳ (player:GetAttribute("AssignedTransferDeadline") or 0)
10831                 plant:SetAttribute("QuizProtected", true)
10832                 readout.Text = "STORING SAMPLE\nPlacing " .. plantId .. " into " .. code
10833                 local passedQuiz = runLiveMathQuiz(player, "REVIVE " .. plantId .. " IN " .. code)
10834                 plant:SetAttribute("QuizProtected", nil)
10835                 if not passedQuiz or not plant.Parent then return end
10836                 table.insert(samples, plantId)
10837                 saveLiveChamber(part, samples)
10838                 part:SetAttribute("GrowthStage", math.max(part:GetAttribute("GrowthStage") or 0,
↳ plant:GetAttribute("GrowthStage") or 0))
10839                 part:SetAttribute("Treatments", plant:GetAttribute("Treatments") or "")
10840                 part:SetAttribute("Genome", plant:GetAttribute("Genome") or "Wild Type")
10841                 local recoverySeconds = recoverLivePlantLife(part, plantLife, player, plantId)
10842                 if wanted ~= "" then
10843                     for _, reservedDestination in
↳ ipairs(CollectionService:GetTagged("PlantGrowthChamber")) do
10844                         if reservedDestination:GetAttribute("GrowthCode") == wanted then
10845                             reservedDestination:SetAttribute("IncomingReservations", math.max(0,
↳ (reservedDestination:GetAttribute("IncomingReservations") or 1) - 1))
10846                             break
10847                         end
10848                     end
10849                     liveEmergencyMoves = liveEmergencyMoves + 1
10850                 end
10851                 showChamberPlant(part, true)
10852                 refreshLiveTelemetry(part)
10853                 player:SetAttribute("CarriedSample", "")
10854                 plant:Destroy()
10855                 if wanted ~= "" then
10856                     liveTransfers = liveTransfers + 1
10857                     if assigned then liveAssignedTransfers = liveAssignedTransfers + 1 end
10858                     local points = assigned and (onTime and LIVE_POINTS.MOVE_ON_TIME or
↳ LIVE_POINTS.MOVE_LATE) or LIVE_POINTS.MOVE_ANY
10859                     local total = liveAward(player, points, assigned and (onTime and "assigned on-time
↳ plant move" or "assigned late plant move") or "emergency plant move")
10860                     readout.Text = string.format("%s STORED +%d\n%s • %d/%d • TOTAL %d", code, points,
↳ plantId, #samples, capacity, total)
10861                     if recoverySeconds > 0 then

```



```

10863         task.delay(recoverySeconds, function() if player.Parent then
10864             ↪ liveAssignTask(player) end end)
10865     else
10866         liveAssignTask(player)
10867     end
10868 else
10869     readout.Text = string.format("%s STORED\n%s • %d/%d samples", code, plantId,
10870         ↪ #samples, capacity)
10871     player:SetAttribute("LabObjective", "Take the plant to growth equipment or another
10872         ↪ chamber.")
10873 end
10874 end
10875 elseif #samples > 0 then
10876     local destination
10877     if failed and player:GetAttribute("AssignedTransferSource") == code then
10878         local assignedDestination = player:GetAttribute("AssignedTransferDestination")
10879         for _, candidate in ipairs(liveDestinations(part)) do
10880             if candidate:GetAttribute("GrowthCode") == assignedDestination then destination =
10881                 ↪ candidate break end
10882         end
10883     end
10884     if failed then destination = destination or liveDestinations(part)[1] end
10885     if failed and not destination then
10886         readout.Text = code .. " FAILED\nNo working chamber has space"
10887         player:SetAttribute("LabObjective", "Working chambers are full. Wait for NPC transfers
10888             ↪ or an automatic recovery.")
10889         return
10890     end
10891     if failed and destination then createLiveRepairPath(player, destination) end
10892     local plantId = samples[1]
10893     local destinationCode = destination and destination:GetAttribute("GrowthCode") or ""
10894     readout.Text = "TAKING SAMPLE\n" .. plantId .. " from " .. code
10895     if not runLiveMathQuiz(player, "TAKE PLANT " .. plantId .. " FROM " .. code) then return
10896         ↪ end
10897     table.remove(samples, 1)
10898     local made = makePlantTool(player, part, plantId, destinationCode)
10899     if made then
10900         saveLiveChamber(part, samples)
10901         showChamberPlant(part, true)
10902         if destination then
10903             destination:SetAttribute("IncomingReservations",
10904                 ↪ (destination:GetAttribute("IncomingReservations") or 0) + 1)
10905             part:SetAttribute("EvacuationDestination", destinationCode)
10906             player:SetAttribute("LabObjective", "CARRYING " .. plantId .. ": press E at ANY
10907                 ↪ GREEN RUNNING cabinet marked FREE SLOT. BONUS target: " .. destinationCode ..
10908                 ↪ ".")
10909             readout.Text = code .. " FAILED\nSEND " .. plantId .. " TO " .. destinationCode
10910         else
10911             player:SetAttribute("LabObjective", "Use growth equipment on " .. plantId .. ",
10912                 ↪ then return it to a working chamber.")
10913             readout.Text = string.format("%s SAMPLE OUT\n%s • %d/%d remain", code, plantId,
10914                 ↪ #samples, capacity)
10915         end
10916         end
10917         player:SetAttribute("CarriedSample", plantId)
10918         refreshLiveTelemetry(part)
10919     else
10920         table.insert(samples, 1, plantId)
10921     end
10922 else
10923     readout.Text = code .. (failed and " FAILED + EMPTY\nAlarm awaiting refill" or "
10924         ↪ EMPTY\nBring a plant")
10925 end
10926 elseif part:GetAttribute("FunctionalState") == "FAILED" then
10927     readout.Text = "FAILED • HOLD R\n" .. liveFloor(part) .. " • " ..
10928         ↪ (part:GetAttribute("FailureReason") or "fault")
10929     return
10930 elseif plantProcess then
10931     if livePlayerRole(player) ~= "Laboratory Technician" then
10932         readout.Text = "LAB ROLE REQUIRED\nGREEN kiosk develops plants"
10933         return
10934     end
10935     local plant = findPlant(player)
10936     if not plant then readout.Text = "PLANT REQUIRED\nTake one from an R-numbered chamber"; return
10937         ↪ end
10938     local process = plantProcess
10939     local stage = math.clamp((plant:GetAttribute("GrowthStage") or 0) + process.gain, 0, 5)

```

```

10925     plant:SetAttribute("GrowthStage", stage)
10926     local treatments = plant:GetAttribute("Treatments") or ""
10927     if not string.find(treatments, process.name, 1, true) then
10928         plant:SetAttribute("Treatments", treatments == "" and process.name or (treatments .. ", "
            ↪ .. process.name))
10929     end
10930     local total = liveAward(player, LIVE_POINTS.PROCESS, process.name .. " plant treatment")
10931     readout.Text = kind .. "\nSTATUS WORKING • PLANTS " .. (part:GetAttribute("SampleCount") or 0)
            ↪ .. "/"
10932         .. (part:GetAttribute("Capacity") or 0) .. "\nPLANT DATA +" .. LIVE_POINTS.PROCESS .. " • "
10933         .. (plant:GetAttribute("PlantId") or "Plant") .. " • stage " .. stage .. " • TOTAL " ..
            ↪ total
10934     equip(player, plant)
10935 else
10936     part:SetAttribute("Inspections", (part:GetAttribute("Inspections") or 0) + 1)
10937     readout.Text = string.format("%s • %s\nSTATUS WORKING • POWER ON\nPLANTS %d/%d\nInspection
            ↪ normal",
10938         liveFloor(part), kind, part:GetAttribute("SampleCount") or 0, part:GetAttribute("Capacity")
            ↪ or 0)
10939     end
10940 end)
10941 end
10942
10943 local roleColours = {
10944     Technician = Color3.fromRGB(60, 120, 200),
10945     ["Laboratory Technician"] = Color3.fromRGB(90, 170, 90),
10946 }
10947 local function dressCharacter(character, role)
10948     for _, old in ipairs(character:GetChildren()) do
10949         if old.Name:match("^LiveLabUniform") then old:Destroy() end
10950     end
10951     local torso = character:FindFirstChild("UpperTorso") or character:FindFirstChild("Torso")
10952     if not torso then return end
10953     local panel = Instance.new("Part")
10954     panel.Name = "LiveLabUniform_Coat"
10955     panel.Size = torso.Size + Vector3.new(0.18, 0.25, 0.18)
10956     panel.CFrame = torso.CFrame
10957     panel.Color = Color3.fromRGB(240, 244, 247)
10958     panel.Material = Enum.Material.Fabric
10959     panel.CanCollide = false
10960     panel.Massless = true
10961     panel.Parent = character
10962     local weld = Instance.new("WeldConstraint")
10963     weld.Part0 = torso; weld.Part1 = panel; weld.Parent = panel
10964     local roleColour = roleColours[role] or Color3.fromRGB(150, 160, 170)
10965     local badge = weldPiece(character, panel, "LiveLabUniform_Badge", Vector3.new(0.4, 0.5, 0.08),
            ↪ Vector3.new(-0.52, 0.25, -panel.Size.Z / 2), roleColour)
10966     badge.Name = "LiveLabUniform_Badge"
10967     local head = character:FindFirstChild("Head")
10968     if head then
10969         local helmet = weldPiece(character, head, "LiveLabUniform_Helmet", Vector3.new(1.82, 0.7, 1.82),
            ↪ Vector3.new(0, 0.72, 0), roleColour)
10970         helmet.Shape = Enum.PartType.Ball
10971     end
10972 end
10973
10974 local function refreshLiveHud(player)
10975     local playerGui = player:FindFirstChild("PlayerGui")
10976     if not playerGui then return end
10977     local gui = playerGui:FindFirstChild("LiveLabHUD")
10978     local label = gui and gui:FindFirstChild("Objective")
10979     if label and label:IsA("TextLabel") then
10980         local role = player:GetAttribute("LabRole") or "Choose a role"
10981         local carried = player:GetAttribute("CarriedSample") or ""
10982         local objective = player:GetAttribute("LabObjective") or ""
10983         local stats = player:FindFirstChild("leaderstats")
10984         local points = stats and stats:FindFirstChild("Points")
10985         local lastAmount = player:GetAttribute("LastPointAmount") or 0
10986         local lastReason = player:GetAttribute("LastPointAward") or ""
10987         local timer = ""
10988         if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then
10989             timer = " • " .. math.max(0, player:GetAttribute("AssignedTransferDeadline") - os.time()) .. "
                ↪ sec"
10990         end
10991         label.Text = string.format("%s%s • POINTS %d%s\n%s%s", role,
10992             carried ~= "" and (" • HAND: " .. carried) or "",

```

```

10993         points and points.Value or 0,
10994         lastAmount > 0 and (" • LAST +" .. lastAmount .. " " .. lastReason) or "",
10995         objective ~= "" and objective or "Press E at BLUE or GREEN role kiosk.", timer)
10996     end
10997 end
10998
10999 local function buildLiveHud(player)
11000     local playerGui = player:WaitForChild("PlayerGui", 5)
11001     if not playerGui then return end
11002     if playerGui:FindFirstChild("RoleHUD") then
11003         local duplicate = playerGui:FindFirstChild("LiveLabHUD")
11004         if duplicate then duplicate:Destroy() end
11005         return
11006     end
11007     if playerGui:FindFirstChild("LiveLabHUD") then refreshLiveHud(player); return end
11008     local gui = Instance.new("ScreenGui")
11009     gui.Name = "LiveLabHUD"
11010     gui.ResetOnSpawn = false
11011     gui.Parent = playerGui
11012     local label = Instance.new("TextLabel")
11013     label.Name = "Objective"
11014     label.Size = UDim2.new(0.48, 0, 0, 62)
11015     label.Position = UDim2.new(0.02, 0, 0, 54)
11016     label.BackgroundColor3 = Color3.fromRGB(10, 18, 22)
11017     label.BackgroundTransparency = 0.18
11018     label.BorderSizePixel = 0
11019     label.TextColor3 = Color3.fromRGB(225, 245, 232)
11020     label.TextWrapped = true
11021     label.TextScaled = false
11022     label.TextSize = 12
11023     label.TextTruncate = Enum.TextTruncate.AtEnd
11024     label.Font = Enum.Font.GothamBold
11025     label.Parent = gui
11026     local corner = Instance.new("UICorner")
11027     corner.CornerRadius = UDim.new(0, 8)
11028     corner.Parent = label
11029     local constraint = Instance.new("UISizeConstraint")
11030     constraint.MinSize = Vector2.new(260, 62)
11031     constraint.MaxSize = Vector2.new(360, 62)
11032     constraint.Parent = label
11033     refreshLiveHud(player)
11034 end
11035
11036 for _, kiosk in ipairs(CollectionService:GetTagged("RoleKiosk")) do
11037     local prompt = Instance.new("ProximityPrompt")
11038     prompt.ActionText = "Select role"
11039     prompt.ObjectText = kiosk:GetAttribute("Role") or "Lab role"
11040     prompt.KeyboardKeyCode = Enum.KeyCode.E
11041     prompt.MaxActivationDistance = 14
11042     prompt.RequiresLineOfSight = false
11043     prompt.Parent = kiosk
11044     prompt.Triggered:Connect(function(player)
11045         local role = kiosk:GetAttribute("Role")
11046         local team = role and Teams:FindFirstChild(role)
11047         if team then player.Team = team; player.Neutral = false end
11048         player:SetAttribute("LabRole", role)
11049         liveAssignTask(player)
11050         if player.Character then dressCharacter(player.Character, role) end
11051         refreshLiveHud(player)
11052     end)
11053 end
11054 local function preserveLiveUniform(player)
11055     if player:GetAttribute("LiveUniformConnected") then return end
11056     player:SetAttribute("LiveUniformConnected", true)
11057     local stats = player:FindFirstChild("leaderstats")
11058     if not stats then stats = Instance.new("Folder"); stats.Name = "leaderstats"; stats.Parent = player end
11059     local points = stats:FindFirstChild("Points")
11060     if not points then points = Instance.new("IntValue"); points.Name = "Points"; points.Parent = stats end
11061     task.spawn(buildLiveHud, player)
11062     player:GetAttributeChangedSignal("LabObjective"):Connect(function() refreshLiveHud(player) end)
11063     player:GetAttributeChangedSignal("CarriedSample"):Connect(function() refreshLiveHud(player) end)
11064     player:GetAttributeChangedSignal("LabRole"):Connect(function() refreshLiveHud(player) end)
11065     player:GetAttributeChangedSignal("LastPointAward"):Connect(function() refreshLiveHud(player) end)
11066     points:GetPropertyChangedSignal("Value"):Connect(function() refreshLiveHud(player) end)
11067     player.CharacterAdded:Connect(function(character)
11068         local role = player:GetAttribute("LabRole")

```

```

11069         if role then task.wait(0.35); dressCharacter(character, role) end
11070         local healthScript = character:FindFirstChild("Health")
11071         if healthScript then healthScript:Destroy() end
11072     end)
11073     if player.Character then
11074         local healthScript = player.Character:FindFirstChild("Health")
11075         if healthScript then healthScript:Destroy() end
11076     end
11077     task.spawn(function()
11078         while player.Parent do
11079             if (player:GetAttribute("AssignedTransferDeadline") or 0) > 0 then refreshLiveHud(player) end
11080             task.wait(1)
11081         end
11082     end)
11083 end
11084 for _, player in ipairs(Players:GetPlayers()) do preserveLiveUniform(player) end
11085 Players.PlayerAdded:Connect(preserveLiveUniform)
11086
11087 local LIVE_IMPACT = {
11088     ["Foam Baton"] = {mode = "melee", damage = 18, colour = Color3.fromRGB(245, 135, 55)},
11089     ["Sample Grabber"] = {mode = "melee", damage = 12, colour = Color3.fromRGB(70, 165, 220)},
11090     ["Inspection Paddle"] = {mode = "melee", damage = 22, colour = Color3.fromRGB(245, 205, 70)},
11091     ["Stun Blaster"] = {mode = "gun", damage = 28, range = 95, colour = Color3.fromRGB(90, 225, 245)},
11092     ["Foam Dart Rifle"] = {mode = "gun", damage = 38, range = 125, colour = Color3.fromRGB(225, 80, 95)},
11093     ["Basketball"] = {mode = "ball", damage = 24, colour = Color3.fromRGB(225, 105, 35)},
11094 }
11095
11096 local function liveBotFromHit(hit)
11097     local model = hit and hit:FindFirstAncestorOfClass("Model")
11098     return model and CollectionService:HasTag(model, "LabWorkerBot") and model or nil
11099 end
11100
11101 local function refreshLiveBotHealth(bot)
11102     local maximum = bot:GetAttribute("MaxHealth") or 100
11103     local health = math.clamp(bot:GetAttribute("Health") or maximum, 0, maximum)
11104     local label = bot:FindFirstChild("HealthText", true)
11105     local fill = bot:FindFirstChild("HealthBarFill", true)
11106     if label and label:IsA("TextLabel") then label.Text = string.format("LIFE %d / %d", health, maximum)
11107     ↪ end
11108     if fill and fill:IsA("Frame") then
11109         fill.Size = UDim2.fromScale(health / math.max(1, maximum), 1)
11110         fill.BackgroundColor3 = health > maximum * 0.55 and Color3.fromRGB(75, 220, 95)
11111         or (health > maximum * 0.25 and Color3.fromRGB(245, 190, 55) or Color3.fromRGB(235, 65, 60))
11112     end
11113 end
11114
11115 local function damageLiveBot(bot, amount, attacker)
11116     if not bot or bot:GetAttribute("KnockedOut") then return end
11117     local health = math.max(0, (bot:GetAttribute("Health") or 100) - amount)
11118     bot:SetAttribute("Health", health)
11119     bot:SetAttribute("LastHitBy", attacker and attacker.Name or "environment")
11120     refreshLiveBotHealth(bot)
11121     local flash = Instance.new("Highlight")
11122     flash.FillColor = Color3.fromRGB(255, 90, 70)
11123     flash.FillTransparency = 0.25
11124     flash.Parent = bot
11125     Debris:AddItem(flash, 0.22)
11126     if health <= 0 then
11127         bot:SetAttribute("KnockedOut", true)
11128         bot:SetAttribute("TaskState", "KNOCKED OUT")
11129         local status = bot:FindFirstChild("Status", true)
11130         if status and status:IsA("TextLabel") then status.Text = bot.Name .. "\nKNOCKED OUT" end
11131         task.delay(8, function()
11132             if bot.Parent then
11133                 bot:SetAttribute("Health", bot:GetAttribute("MaxHealth") or 100)
11134                 bot:SetAttribute("KnockedOut", false)
11135                 refreshLiveBotHealth(bot)
11136             end
11137         end)
11138     end
11139 end
11140
11141 local function stealLiveSample(attacker, victim)
11142     local stolen = findPlant(victim)
11143     local backpack = attacker:FindFirstChildOfClass("Backpack")
11144     if not stolen or stolen.Parent ~= victim.Character or not backpack then return end

```

```

11144     stolen.Parent = backpack
11145     victim:SetAttribute("CarriedSample", "")
11146     attacker:SetAttribute("CarriedSample", stolen:GetAttribute("PlantId") or stolen.Name)
11147     local destination = stolen:GetAttribute("DestinationGrowthCode") or ""
11148     attacker:SetAttribute("LabObjective", destination ~= "" and ("Stolen sample must be rescued to " ..
↪     destination .. ".")
11149     or "Use growth equipment, then return the sample to a working chamber.")
11150     equip(attacker, stolen)
11151 end
11152
11153 local function grantImpactTool(player, kind)
11154     local style = LIVE_IMPACT[kind]
11155     if not style then return end
11156     local backpack = player:FindFirstChildOfClass("Backpack") or player:WaitForChild("Backpack", 3)
11157     if not backpack then return end
11158     local existing = backpack:FindFirstChild(kind) or (player.Character and
↪     player.Character:FindFirstChild(kind))
11159     if existing then equip(player, existing); return end
11160     local tool = Instance.new("Tool")
11161     tool.Name = kind
11162     tool.RequiresHandle = true
11163     tool.CanBeDropped = false
11164     tool.ToolTip = style.mode == "gun" and "Fire at worker NPCs"
11165     or (style.mode == "ball" and "Throw at worker NPCs" or "Hit NPCs or recover another player's held
↪     sample")
11166     tool:SetAttribute("LabImpactTool", true)
11167     tool:SetAttribute("ImpactToolKind", kind)
11168     local handle = Instance.new("Part")
11169     handle.Name = "Handle"
11170     handle.Size = style.mode == "gun" and Vector3.new(0.72, 0.88, 2.2)
11171     or (style.mode == "ball" and Vector3.new(1.35, 1.35, 1.35)
11172     or (kind == "Inspection Paddle" and Vector3.new(1.2, 3.2, 0.35) or Vector3.new(0.5, 3.4, 0.5)))
11173     handle.Color = style.colour
11174     handle.Material = style.mode == "ball" and Enum.Material.Rubber or Enum.Material.SmoothPlastic
11175     handle.CanCollide = false
11176     handle.Massless = true
11177     handle.Parent = tool
11178     if style.mode == "ball" then
11179         handle.Shape = Enum.PartType.Ball
11180     elseif style.mode == "gun" then
11181         weldPiece(tool, handle, "Barrel", Vector3.new(0.34, 0.34, 1.7), Vector3.new(0, 0.1, -1.65),
↪         style.colour)
11182         weldPiece(tool, handle, "GunGrip", Vector3.new(0.48, 1.05, 0.55), Vector3.new(0, -0.75, 0.42),
↪         Color3.fromRGB(42, 45, 52))
11183     end
11184
11185     local activeUntil, nextUse = 0, 0
11186     local hitThisUse = {}
11187     tool.Activated:Connect(function()
11188         local attacker = Players:GetPlayerFromCharacter(tool.Parent)
11189         if not attacker or os.clock() < nextUse then return end
11190         nextUse = os.clock() + (style.mode == "gun" and 0.55 or 0.8)
11191         hitThisUse = {}
11192         local rootPart = attacker.Character and attacker.Character:FindFirstChild("HumanoidRootPart")
11193         if not rootPart then return end
11194         if style.mode == "gun" then
11195             local origin = rootPart.Position + Vector3.new(0, 1.25, 0) + rootPart.CFrame.LookVector * 2
11196             local direction = rootPart.CFrame.LookVector * (style.range or 100)
11197             local params = RaycastParams.new()
11198             params.FilterType = Enum.RaycastFilterType.Exclude
11199             params.FilterDescendantsInstances = {attacker.Character}
11200             local result = workspace:Raycast(origin, direction, params)
11201             local hitPosition = result and result.Position or origin + direction
11202             local tracer = Instance.new("Part")
11203             tracer.Name = "FoamTracer"
11204             tracer.Size = Vector3.new(0.08, 0.08, (hitPosition - origin).Magnitude)
11205             tracer.CFrame = CFrame.lookAt((origin + hitPosition) / 2, hitPosition)
11206             tracer.Color = style.colour
11207             tracer.Material = Enum.Material.Neon
11208             tracer.Anchored = true
11209             tracer.CanCollide = false
11210             tracer.Parent = workspace
11211             Debris:AddItem(tracer, 0.12)
11212             local bot = result and liveBotFromHit(result.Instance)
11213             if bot then damageLiveBot(bot, style.damage, attacker) end
11214         elseif style.mode == "ball" then

```

```

11215         local ball = Instance.new("Part")
11216         ball.Name = "ThrownBasketball"
11217         ball.Shape = Enum.PartType.Ball
11218         ball.Size = Vector3.new(1.35, 1.35, 1.35)
11219         ball.CFrame = CFrame.new(rootPart.Position + Vector3.new(0, 1.3, 0) +
            ↳ rootPart.CFrame.LookVector * 3)
11220         ball.Color = style.colour
11221         ball.Material = Enum.Material.Rubber
11222         ball.CanCollide = true
11223         ball.Parent = workspace
11224         ball.AssemblyLinearVelocity = rootPart.CFrame.LookVector * 58 + Vector3.new(0, 13, 0)
11225         local used = false
11226         ball.Touched:Connect(function(hit)
11227             if used then return end
11228             local bot = liveBotFromHit(hit)
11229             if bot then used = true; damageLiveBot(bot, style.damage, attacker) end
11230         end)
11231         Debris:AddItem(ball, 7)
11232     else
11233         activeUntil = os.clock() + 0.45
11234     end
11235 end)
11236 handle.Touched:Connect(function(hit)
11237     if style.mode ~= "melee" or os.clock() > activeUntil then return end
11238     local attacker = Players:GetPlayerFromCharacter(tool.Parent)
11239     if not attacker then return end
11240     local bot = liveBotFromHit(hit)
11241     if bot then
11242         if hitThisUse[bot] then return end
11243         hitThisUse[bot] = true
11244         damageLiveBot(bot, style.damage, attacker)
11245         return
11246     end
11247     local victim = Players:GetPlayerFromCharacter(hit:FindFirstAncestorOfClass("Model"))
11248     if not victim or attacker == victim or hitThisUse[victim] then return end
11249     hitThisUse[victim] = true
11250     stealLiveSample(attacker, victim)
11251 end)
11252 tool.Parent = backpack
11253 equip(player, tool)
11254 end
11255
11256 for _, padPart in ipairs(CollectionService:GetTagged("LabToolDispenser")) do
11257     local prompt = Instance.new("ProximityPrompt")
11258     prompt.ActionText = "Pick up and equip"
11259     prompt.ObjectText = padPart:GetAttribute("ToolKind") or "Recovery tool"
11260     prompt.KeyboardKeyCode = Enum.KeyCode.E
11261     prompt.MaxActivationDistance = 12
11262     prompt.RequiresLineOfSight = false
11263     prompt.Parent = padPart
11264     prompt.Triggered:Connect(function(player) grantImpactTool(player, padPart:GetAttribute("ToolKind"))
        ↳ end)
11265 end
11266
11267 for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do bindLiveEquipment(equipment) end
11268
11269 -- Automatic doors, live-fallback copy: same open-near / close-away
11270 -- behaviour as the installed runtime.
11271 local LIVE_DOOR_OPEN_RANGE = 11
11272 local LIVE_DOOR_CLOSE_RANGE = 15
11273 local LIVE_DOOR_MIN_OPEN_SECONDS = 3
11274 local function bindLiveAutoDoor(door)
11275     if door:GetAttribute("DoorBound") then return end
11276     door:SetAttribute("DoorBound", true)
11277     local closedX = door:GetAttribute("DoorClosedX") or door.Position.X
11278     local closedZ = door:GetAttribute("DoorClosedZ") or door.Position.Z
11279     local openX = door:GetAttribute("DoorOpenX") or closedX
11280     local openZ = door:GetAttribute("DoorOpenZ") or closedZ
11281     local doorY = door:GetAttribute("DoorY") or door.Position.Y
11282     local isOpen = false
11283     local openedAt = 0
11284     local tween
11285     local function setIndicatorColour(open)
11286         local colour = open and Color3.fromRGB(60, 210, 90) or Color3.fromRGB(210, 40, 40)
11287         for _, gui in ipairs(door:GetChildren()) do
11288             if gui:IsA("SurfaceGui") and gui.Name == "AutoDoorLabel" then

```

```

11289         local indicator = gui:FindFirstChild("DoorIndicator")
11290         if indicator then indicator.BackgroundColor3 = colour end
11291     end
11292 end
11293 end
11294 local function setOpen(open)
11295     if open == isOpen then return end
11296     isOpen = open
11297     if open then openedAt = os.clock() end
11298     if tween then tween:Cancel() end
11299     local targetX = open and openX or closedX
11300     local targetZ = open and openZ or closedZ
11301     tween = TweenService:Create(door, TweenInfo.new(0.9, Enum.EasingStyle.Sine,
11302         ↪ Enum.EasingDirection.InOut),
11303         {CFrame = CFrame.new(targetX, doorY, targetZ)})
11304     tween:Play()
11305     door.CanCollide = not open
11306     setIndicatorColour(open)
11307 end
11308 task.spawn(function()
11309     while door.Parent do
11310         local nearest = math.huge
11311         for _, player in ipairs(Players:GetPlayers()) do
11312             local character = player.Character
11313             local hrp = character and character:FindFirstChild("HumanoidRootPart")
11314             if hrp then
11315                 -- Same fix as the installed runtime: floors stack at the
11316                 -- same X/Z, so require the player be close in height to
11317                 -- this door before the flat distance counts at all.
11318                 local verticalGap = math.abs(hrp.Position.Y - doorY)
11319                 if verticalGap < 8 then
11320                     local flat = (Vector3.new(hrp.Position.X, doorY, hrp.Position.Z) -
11321                         ↪ Vector3.new(closedX, doorY, closedZ)).Magnitude
11322                     if flat < nearest then nearest = flat end
11323                 end
11324             end
11325         end
11326         -- Minimum 3-second open hold before the door is allowed to
11327         -- close again -- fixes the open/close-too-fast flicker.
11328         if isOpen and nearest > LIVE_DOOR_CLOSE_RANGE and (os.clock() - openedAt) >=
11329             ↪ LIVE_DOOR_MIN_OPEN_SECONDS then
11330             setOpen(false)
11331         elseif not isOpen and nearest < LIVE_DOOR_OPEN_RANGE then
11332             setOpen(true)
11333         end
11334         task.wait(0.25)
11335     end
11336 end)
11337 end
11338 for _, p in ipairs(CollectionService:GetTagged("AutoDoor")) do bindLiveAutoDoor(p) end
11339 CollectionService:GetInstanceAddedSignal("AutoDoor"):Connect(bindLiveAutoDoor)
11340
11341 local liveFacilityNoticeAt = setmetatable({}, {__mode = "k"})
11342 local function liveNotify(player, text)
11343     local facilityAlert = text:match("^ALARM:") or text:match("^REPAIR ALARM:") or
11344     ↪ text:match("^CATASTROPHIC FAILURE:")
11345     if facilityAlert then
11346         local now = os.clock()
11347         if now - (liveFacilityNoticeAt[player] or -math.huge) < 6 then return end
11348         liveFacilityNoticeAt[player] = now
11349     end
11350     print("[ " .. player.Name .. " ] " .. text)
11351     local gui = player:FindFirstChild("PlayerGui")
11352     if not gui then return end
11353     local screen = gui:FindFirstChild("LabLog")
11354     if not screen then
11355         screen = Instance.new("ScreenGui")
11356         screen.Name = "LabLog"
11357         screen.ResetOnSpawn = false
11358         screen.Parent = gui
11359         local label = Instance.new("TextLabel")
11360         label.Name = "Line"
11361         label.Size = UDim2.new(0.48, 0, 0, 46)
11362         label.Position = UDim2.new(0.5, 0, 1, -68)
11363         label.AnchorPoint = Vector2.new(0.5, 1)
11364         label.BackgroundTransparency = 0.3

```

```

11361     label.BackgroundColor3 = Color3.fromRGB(10, 10, 10)
11362     label.TextColor3 = Color3.fromRGB(255, 210, 90)
11363     label.TextScaled = false
11364     label.TextWrapped = true
11365     label.TextTruncate = Enum.TextTruncate.AtEnd
11366     label.TextSize = 13
11367     label.Font = Enum.Font.GothamBold
11368     label.Parent = screen
11369     local constraint = Instance.new("UISizeConstraint")
11370     constraint.MinSize = Vector2.new(260, 46)
11371     constraint.MaxSize = Vector2.new(420, 46)
11372     constraint.Parent = label
11373 end
11374 local label = screen:FindFirstChild("Line")
11375 if label then
11376     local serial = (screen:GetAttribute("MessageSerial") or 0) + 1
11377     screen:SetAttribute("MessageSerial", serial)
11378     label.Text = text
11379     label.Visible = true
11380     task.delay(4, function()
11381         if label.Parent and screen:GetAttribute("MessageSerial") == serial then label.Visible =
            ↳ false end
11382     end)
11383 end
11384 end
11385
11386 -- EMERGENCY KITS: one per room, press E to fully restore health. Live-
11387 -- fallback copy of the same behaviour as the installed runtime.
11388 local function bindLiveEmergencyKit(kit)
11389     if kit:GetAttribute("KitBound") then return end
11390     kit:SetAttribute("KitBound", true)
11391     local prompt = kit:FindFirstChild("EmergencyKitPrompt")
11392     if not prompt then return end
11393     -- Recharges rather than being consumed, same as the installed copy.
11394     local RECHARGE_SECONDS = 25
11395     local function setCharged(charged)
11396         kit:SetAttribute("Charged", charged)
11397         prompt.Enabled = charged
11398         kit.Color = charged and Color3.fromRGB(225, 40, 35) or Color3.fromRGB(92, 60, 58)
11399     end
11400     prompt.Triggered:Connect(function(player)
11401         if kit:GetAttribute("Charged") == false then return end
11402         local character = player.Character
11403         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11404         if not humanoid then return end
11405         setCharged(false)
11406         humanoid.Health = humanoid.MaxHealth
11407         liveNotify(player, "Health restored. This station recharges in " .. RECHARGE_SECONDS .. "s.")
11408         task.delay(RECHARGE_SECONDS, function()
11409             if kit.Parent then setCharged(true) end
11410         end)
11411     end)
11412     setCharged(true)
11413 end
11414 for _, p in ipairs(CollectionService:GetTagged("EmergencyKit")) do bindLiveEmergencyKit(p) end
11415 CollectionService:GetInstanceAddedSignal("EmergencyKit"):Connect(bindLiveEmergencyKit)
11416
11417 -- SECRET BASEMENT VENT: live-fallback copy of the same disguised panel
11418 -- behaviour -- hold E on it to drop straight into the basement corridor.
11419 local function bindLiveSecretBasementPanel(panel)
11420     if panel:GetAttribute("SecretBound") then return end
11421     panel:SetAttribute("SecretBound", true)
11422     local prompt = panel:FindFirstChildWhichIsA("ProximityPrompt")
11423     if not prompt then return end
11424     prompt.Triggered:Connect(function(player)
11425         local character = player.Character
11426         local hrp = character and character:FindFirstChild("HumanoidRootPart")
11427         if not hrp then return end
11428         local humanoid = character:FindFirstChildOfClass("Humanoid")
11429         if not humanoid or humanoid.Health <= 0 or (hrp.Position - panel.Position).Magnitude > 12 then
            ↳ return end
11430         local floorPoint = Vector3.new(
11431             panel:GetAttribute("SecretDestX") or hrp.Position.X,
11432             panel:GetAttribute("SecretDestY") or hrp.Position.Y,
11433             panel:GetAttribute("SecretDestZ") or hrp.Position.Z)
11434         local params = RaycastParams.new()

```



```

11435     params.FilterType = Enum.RaycastFilterType.Exclude
11436     params.FilterDescendantsInstances = {character}
11437     params.RespectCanCollide = true
11438     local floorHit = workspace.Raycast(floorPoint + Vector3.new(0, 7, 0), Vector3.new(0, -12, 0),
        ↪     params)
11439     if not floorHit or floorHit.Normal.Y < 0.7 then return end
11440     local feetOffset = humanoid.HipHeight + hrp.Size.Y / 2
11441     for _, name in ipairs({"LeftFoot", "RightFoot", "Left Leg", "Right Leg"}) do
11442         local foot = character:FindFirstChild(name)
11443         if foot and foot:IsA("BasePart") then
11444             feetOffset = math.max(feetOffset, hrp.Position.Y - (foot.Position.Y - foot.Size.Y / 2))
11445         end
11446     end
11447     hrp.CFrame = CFrame.new(floorHit.Position + Vector3.new(0, feetOffset + 0.35, 0))
11448     hrp.AssemblyLinearVelocity, hrp.AssemblyAngularVelocity = Vector3.zero, Vector3.zero
11449     humanoid:ChangeState(Enum.HumanoidStateType.GettingUp)
11450     liveNotify(player, "The vent panel swings open -- you drop into the basement.")
11451 end
11452 end
11453 for _, p in ipairs(CollectionService:GetTagged("SecretBasementPanel")) do bindLiveSecretBasementPanel(p)
    ↪ end
11454 CollectionService:GetInstanceAddedSignal("SecretBasementPanel"):Connect(bindLiveSecretBasementPanel)
11455
11456 local function liveNearestEmergencyKit(position)
11457     local best, bestDist
11458     for _, kit in ipairs(CollectionService:GetTagged("EmergencyKit")) do
11459         if kit.Parent then
11460             local d = (kit.Position - position).Magnitude
11461             if not bestDist or d < bestDist then best, bestDist = kit, d end
11462         end
11463     end
11464     return best
11465 end
11466
11467 local liveHealthPathActive = {}
11468 task.spawn(function()
11469     while true do
11470         task.wait(3)
11471         for _, player in ipairs(Players:GetPlayers()) do
11472             local character = player.Character
11473             local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11474             local hrp = character and character:FindFirstChild("HumanoidRootPart")
11475             if humanoid and hrp and humanoid.Health > 0 then
11476                 local low = humanoid.Health < humanoid.MaxHealth * 0.5
11477                 local busy = player:GetAttribute("AssignedRepairId") or
                    ↪     player:GetAttribute("AssignedTransferSource")
11478                 if low and not busy and not liveHealthPathActive[player] then
11479                     local kit = liveNearestEmergencyKit(hrp.Position)
11480                     if kit then
11481                         createLiveRepairPath(player, kit)
11482                         liveHealthPathActive[player] = true
11483                         liveNotify(player, "Health low: follow the RED floor lights to the nearest
                            ↪     emergency kit.")
11484                     end
11485                 elseif (not low or busy) and liveHealthPathActive[player] then
11486                     liveHealthPathActive[player] = false
11487                     if not busy then clearLiveRepairPath(player) end
11488                 end
11489             end
11490         end
11491     end
11492 end)
11493
11494 -- EQUIPMENT HAZARDS: a FAILED unit drains health in nearby players --
11495 -- electrical faults, steam/humidity blowouts, chemical spills, coolant
11496 -- (freon) leaks. Live-fallback copy of the installed runtime's version.
11497 local LIVE_HAZARD_BY_KIND = {
11498     ["Germination Chamber"] = {"STEAM LEAK", 3},
11499     ["Incubator"] = {"STEAM LEAK", 3},
11500     ["Decontamination Unit"] = {"CHEMICAL VAPOR", 4},
11501     ["Mutagenesis Chamber"] = {"RADIATION LEAK", 6},
11502     ["Hormone Treatment Bench"] = {"CHEMICAL SPILL", 4},
11503     ["Plant Growth Chamber"] = {"COOLANT (FREON) LEAK", 3},
11504     ["Growth Hormone Mixer"] = {"CHEMICAL SPILL", 4},
11505     ["GMO Injector"] = {"ELECTRICAL FAULT", 5},
11506     ["Nutrient Infuser"] = {"PRESSURE LEAK", 4},

```

```

11507     ["UV Growth Scanner"] = {"UV OVEREXPOSURE", 4},
11508     ["Pollination Station"] = {"ELECTRICAL FAULT", 5},
11509 }
11510 local LIVE_HAZARD_RANGE = 9
11511 local liveHazardWarnedAt = {}
11512
11513 local LIVE_DESTRUCTION_KIND = {
11514     ["Germination Chamber"] = "steam",
11515     ["Incubator"] = "steam",
11516     ["Decontamination Unit"] = "gas",
11517     ["Mutagenesis Chamber"] = "electrical",
11518     ["Hormone Treatment Bench"] = "liquid",
11519     ["Plant Growth Chamber"] = "gas",
11520     ["Growth Hormone Mixer"] = "liquid",
11521     ["GMO Injector"] = "electrical",
11522     ["Nutrient Infuser"] = "gas",
11523     ["UV Growth Scanner"] = "electrical",
11524     ["Pollination Station"] = "electrical",
11525 }
11526
11527 local function liveScatterDebris(part)
11528     for i = 1, math.random(4, 7) do
11529         local chunk = Instance.new("Part")
11530         chunk.Name = "Debris"
11531         chunk.Size = Vector3.new(math.random(2, 6) / 10, math.random(2, 6) / 10, math.random(2, 6) / 10)
11532         chunk.Color = part.Color
11533         chunk.Material = Enum.Material.Metal
11534         chunk.CFrame = part.CFrame * CFrame.new(math.random(-3, 3) / 2, math.random(0, 3) / 2,
11535             ↪ math.random(-3, 3) / 2)
11536         chunk.Anchored = false
11537         chunk.CanCollide = true
11538         chunk.Parent = part.Parent
11539         local av = Instance.new("AngularVelocity")
11540         av.AngularVelocity = Vector3.new(math.random(-8, 8), math.random(-8, 8), math.random(-8, 8))
11541         av.Parent = chunk
11542         local bv = Instance.new("BodyVelocity")
11543         bv.MaxForce = Vector3.new(4000, 4000, 4000)
11544         bv.Velocity = Vector3.new(math.random(-16, 16), math.random(6, 16), math.random(-16, 16))
11545         bv.Parent = chunk
11546         Debris:AddItem(bv, 0.25)
11547         Debris:AddItem(chunk, 8)
11548     end
11549 end
11550
11551 -- Same catastrophic-failure escalation as the installed runtime: a unit
11552 -- left FAILED too long can permanently blow up / catch fire / arc, with
11553 -- kind-dependent smoke/liquid/gas effects and flying debris.
11554 local destroyLiveEquipment
11555
11556 local function warnLiveEquipmentCatastrophe(part, floorLabel, unitLabel)
11557     if part:GetAttribute("AboutToBlow") then return end
11558     part:SetAttribute("AboutToBlow", true)
11559     local beacon = Instance.new("PointLight")
11560     beacon.Color = Color3.fromRGB(255, 40, 30)
11561     beacon.Brightness = 4
11562     beacon.Range = 16
11563     beacon.Parent = part
11564     local siren = Instance.new("Sound")
11565     siren.Name = "SirenWarning"
11566     siren.SoundId = "rbxasset://sounds/electronicpingshort.wav"
11567     siren.Looped = true
11568     siren.Volume = 3
11569     siren.PlaybackSpeed = 1.6
11570     siren.Parent = part
11571     siren:Play()
11572     for _, player in ipairs(Players:GetPlayers()) do
11573         liveNotify(player, "ALARM: " .. floorLabel .. " • " .. unitLabel .. " is about to fail
11574             ↪ catastrophically -- MOVE AWAY NOW!")
11575     end
11576     task.spawn(function()
11577         local elapsed = 0
11578         while elapsed < 2 and part.Parent do
11579             beacon.Enabled = not beacon.Enabled
11580             task.wait(0.2)
11581             elapsed = elapsed + 0.2
11582         end
11583     end)

```

```

11581     end)
11582     task.delay(2, function()
11583         siren:Stop()
11584         siren:Destroy()
11585         beacon:Destroy()
11586         if part.Parent then destroyLiveEquipment(part) end
11587     end)
11588 end
11589
11590 destroyLiveEquipment = function(part)
11591     local floorLabel = liveFloor(part)
11592     local unitLabel = liveEquipmentLabel(part)
11593     local causes = {"exploded", "caught fire", "arced and shorted out"}
11594     local cause = causes[math.random(1, #causes)]
11595     part:SetAttribute("FunctionalState", "DESTROYED")
11596     part:SetAttribute("AboutToBlow", nil)
11597     part:SetAttribute("RepairInstruction", floorLabel .. " • " .. unitLabel .. " • DESTROYED, UNFIXABLE")
11598     for _, ledPart in ipairs(part:GetDescendants()) do
11599         if ledPart.Name == "StatusLED" then ledPart.Color = Color3.fromRGB(40, 40, 40); ledPart.Material =
            ↳ Enum.Material.SmoothPlastic end
11600     end
11601     part.Color = part.Color:Lerp(Color3.fromRGB(35, 32, 30), 0.6)
11602
11603     local bang = Instance.new("Sound")
11604     bang.SoundId = "rbxasset://sounds/impact_generic.mp3"
11605     bang.Volume = 4
11606     bang.PlaybackSpeed = 0.8
11607     bang.Parent = part
11608     bang:Play()
11609     Debris:AddItem(bang, 4)
11610
11611     local explosion = Instance.new("Explosion")
11612     explosion.Position = part.Position
11613     explosion.BlastRadius = 8
11614     explosion.BlastPressure = 0
11615     explosion.ExplosionType = Enum.ExplosionType.NoCraters
11616     explosion.DestroyJointRadiusPercent = 0
11617     explosion.Parent = workspace
11618
11619     local kindOfDestruction = LIVE_DESTRUCTION_KIND[part:GetAttribute("Kind")] or "electrical"
11620
11621     local fire = Instance.new("Fire")
11622     fire.Size = 6
11623     fire.Heat = 9
11624     fire.Color = Color3.fromRGB(255, 140, 40)
11625     fire.SecondaryColor = Color3.fromRGB(90, 90, 90)
11626     fire.Parent = part
11627     Debris:AddItem(fire, 16)
11628
11629     local sparkles = Instance.new("Sparkles")
11630     sparkles.SparkleColor = Color3.fromRGB(255, 230, 120)
11631     sparkles.Parent = part
11632     Debris:AddItem(sparkles, 3)
11633
11634     local smoke = Instance.new("Smoke")
11635     smoke.Size = 8
11636     smoke.Opacity = kindOfDestruction == "steam" and 0.5 or 0.8
11637     smoke.RiseVelocity = kindOfDestruction == "gas" and 1.5 or 3
11638     smoke.Color = (kindOfDestruction == "steam") and Color3.fromRGB(225, 230, 235)
11639         or (kindOfDestruction == "gas") and Color3.fromRGB(140, 200, 120)
11640         or Color3.fromRGB(50, 50, 55)
11641     smoke.Parent = part
11642     Debris:AddItem(smoke, 20)
11643
11644     if kindOfDestruction == "liquid" then
11645         local puddle = Instance.new("Part")
11646         puddle.Name = "LeakPuddle"
11647         puddle.Shape = Enum.PartType.Cylinder
11648         puddle.Size = Vector3.new(0.05, 3.5, 3.5)
11649         puddle.CFrame = CFrame.new(part.Position.X, part.Position.Y - part.Size.Y / 2 + 0.05,
            ↳ part.Position.Z)
            ↳ * CFrame.Angles(0, 0, math.rad(90))
11650         puddle.Color = Color3.fromRGB(120, 90, 40)
11651         puddle.Material = Enum.Material.Neon
11652         puddle.Transparency = 0.35
11653         puddle.Anchored = true

```

```

11655         puddle.CanCollide = false
11656         puddle.Parent = part.Parent
11657         Debris:AddItem(puddle, 25)
11658     end
11659
11660     liveScatterDebris(part)
11661
11662     for _, player in ipairs(Players:GetPlayers()) do
11663         liveNotify(player, "CATASTROPHIC FAILURE: " .. floorLabel .. " • " .. unitLabel .. " " .. cause ..
            ↳ ". It cannot be repaired.")
11664     end
11665 end
11666
11667 task.spawn(function()
11668     while true do
11669         task.wait(1)
11670         local hazards = {}
11671         for _, equip in ipairs(CollectionService:GetTagged("LabEquipment")) do
11672             local state = equip.Parent and equip:GetAttribute("FunctionalState")
11673             if state == "FAILED" then
11674                 local info = LIVE_HAZARD_BY_KIND[equip:GetAttribute("Kind")]
11675                 if info then table.insert(hazards, {part = equip, label = info[1], dps = info[2]}) end
11676                 local failedFor = os.time() - (equip:GetAttribute("FailureSince") or os.time())
11677                 if failedFor > 45 and math.random(1, 6) == 1 then
11678                     warnLiveEquipmentCatastrophe(equip, liveFloor(equip), liveEquipmentLabel(equip))
11679                 end
11680             elseif state == "DESTROYED" then
11681                 local info = LIVE_HAZARD_BY_KIND[equip:GetAttribute("Kind")]
11682                 if info then table.insert(hazards, {part = equip, label = info[1] .. " (DESTROYED)", dps =
                    ↳ info[2] + 2}) end
11683             end
11684         end
11685         if #hazards > 0 then
11686             for _, player in ipairs(Players:GetPlayers()) do
11687                 local character = player.Character
11688                 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11689                 local hrp = character and character:FindFirstChild("HumanoidRootPart")
11690                 if humanoid and hrp and humanoid.Health > 0 then
11691                     local worst
11692                     for _, hazard in ipairs(hazards) do
11693                         local d = (hazard.part.Position - hrp.Position).Magnitude
11694                         if d < LIVE_HAZARD_RANGE and (not worst or hazard.dps > worst.dps) then worst =
                            ↳ hazard end
11695                     end
11696                     if worst then
11697                         humanoid:TakeDamage(worst.dps)
11698                         local last = liveHazardWarnedAt[player] or 0
11699                         if os.clock() - last > 2 then
11700                             liveHazardWarnedAt[player] = os.clock()
11701                             liveNotify(player, string.format("WARNING: %s nearby -- move away! (%d HP)",
                                ↳ worst.label, math.floor(humanoid.Health)))
11702                         end
11703                     end
11704                 end
11705             end
11706         end
11707     end
11708 end)
11709
11710 -- Fallback runtime uses the same full-body navigation as installed gameplay.
11711 local function liveBotInteractionPoint(part)
11712     local side = part:GetAttribute("WallSide")
11713     local bottomY = part.Position.Y - part.Size.Y / 2
11714     local point = Vector3.new(part.Position.X, bottomY + 3.25, part.Position.Z)
11715     if side == "W" then point = point + Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
11716     elseif side == "E" then point = point - Vector3.new(part.Size.X / 2 + 2.5, 0, 0)
11717     elseif side == "N" then point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
11718     elseif side == "S" then point = point - Vector3.new(0, 0, part.Size.Z / 2 + 2.5)
11719     else point = point + Vector3.new(0, 0, part.Size.Z / 2 + 2.5) end
11720     return point
11721 end
11722
11723 local function botMotionChecks(bot)
11724     local excluded = {bot}
11725     local corpses = workspace:FindFirstChild("LabBotBodies")
11726     if corpses then table.insert(excluded, corpses) end

```

```

11727 local cast = RaycastParams.new()
11728 cast.FilterType = Enum.RaycastFilterType.Exclude
11729 cast.FilterDescendantsInstances = excluded
11730 cast.RespectCanCollide = true
11731 local overlap = OverlapParams.new()
11732 overlap.FilterType = Enum.RaycastFilterType.Exclude
11733 overlap.FilterDescendantsInstances = excluded
11734 overlap.RespectCanCollide = true
11735 overlap.MaxParts = 1
11736 -- Square horizontal footprint encloses the arms at any yaw. The small
11737 -- floor clearance avoids treating a floor seam as a wall.
11738 local size = Vector3.new(3.4, 5.4, 3.4)
11739 local function clear(from, to)
11740     local delta = to - from
11741     for _, player in ipairs(Players:GetPlayers()) do
11742         local character = player.Character
11743         local hrp = character and character:FindFirstChild("HumanoidRootPart")
11744         local humanoid = character and character:FindFirstChildOfClass("Humanoid")
11745         if hrp and humanoid and humanoid.Health > 0 and math.abs(hrp.Position.Y - from.Y) < 5 then
11746             local flatDelta = Vector3.new(delta.X, 0, delta.Z)
11747             local relative = Vector3.new(hrp.Position.X - from.X, 0, hrp.Position.Z - from.Z)
11748             local alpha = flatDelta.Magnitude > 0.01 and math.clamp(relative:Dot(flatDelta) /
11749                 ↳ flatDelta:Dot(flatDelta), 0, 1) or 0
11749             local movingAway = relative.Magnitude < 3.4 and alpha < 0.001
11750                 and (relative - flatDelta).Magnitude > relative.Magnitude + 0.01
11751             if (relative - flatDelta * alpha).Magnitude < 3.4 and not movingAway then
11752                 return false, hrp.Position
11753             end
11754         end
11755     end
11756     local offset = Vector3.new(0, -0.2, 0)
11757     if delta.Magnitude > 0.01 then
11758         local hit = workspace:Blockcast(CFrame.new(from + offset), size, delta, cast)
11759         if hit then return false, hit.Instance.Position end
11760     end
11761     -- Blockcast does not cover pre-existing overlaps; test the destination
11762     -- too, so a door closing or another worker arriving cannot be skipped.
11763     local occupied = workspace:GetPartBoundsInBox(CFrame.new(to + offset), size, overlap)
11764     if occupied[1] then return false, occupied[1].Position end
11765     return true
11766 end
11767 local function grounded(point, currentY)
11768     local floor = workspace:Raycast(Vector3.new(point.X, currentY + 1, point.Z), Vector3.new(0, -7, 0),
11769         ↳ cast)
11769     if not floor or floor.Normal.Y < 0.65 then return nil end
11770     local y = floor.Position.Y + 3.25
11771     if math.abs(y - currentY) > 1.25 then return nil end
11772     return Vector3.new(point.X, y, point.Z)
11773 end
11774 return clear, grounded
11775 end
11776
11777 local function walkBotSegment(bot, destination, allowDetour)
11778     if not bot.Parent or not bot.PrimaryPart or bot:GetAttribute("Dead") then return false end
11779     local clear, grounded = botMotionChecks(bot)
11780     local start = bot:GetPivot().Position
11781     local goal = Vector3.new(destination.X, start.Y, destination.Z)
11782     local direction = goal - start
11783     if direction.Magnitude < 0.15 then return true end
11784     local free, obstruction = clear(start, goal)
11785     if not free then
11786         -- Dynamic obstacles are not reliably included in Roblox's navmesh.
11787         -- Try both sides, with two corner points around the person/obstacle.
11788         -- Each leg still gets the same full-body collision tests below.
11789         if allowDetour ~= false and obstruction then
11790             local forward = direction.Unit
11791             local side = Vector3.new(-forward.Z, 0, forward.X)
11792             local centre = Vector3.new(obstruction.X, start.Y, obstruction.Z)
11793             local preferred = (bot:GetAttribute("BotNumber") or 1) % 2 == 0 and 1 or -1
11794             for _, sign in ipairs({preferred, -preferred}) do
11795                 local entry = grounded(centre - forward * 4.5 + side * sign * 4.5, start.Y)
11796                 local exitPoint = grounded(centre + forward * 4.5 + side * sign * 4.5, start.Y)
11797                 if entry and exitPoint and clear(start, entry) and clear(entry, exitPoint) and
11798                     ↳ clear(exitPoint, goal) then
11798                     bot:SetAttribute("AvoidanceDetours", (bot:GetAttribute("AvoidanceDetours") or 0) + 1)
11799                     return walkBotSegment(bot, entry, false) and walkBotSegment(bot, exitPoint, false) and
11799                         ↳ walkBotSegment(bot, destination, false)

```

```

11800         end
11801     end
11802     end
11803     bot:SetAttribute("NavigationBlocked", true)
11804     return false
11805 end
11806 bot:SetAttribute("NavigationBlocked", false)
11807 local steps = math.max(1, math.ceil(direction.Magnitude / 0.45))
11808 for step = 1, steps do
11809     while bot.Parent and bot:GetAttribute("KnockedOut") and not bot:GetAttribute("Dead") do
11810         ↪ task.wait(0.25) end
11811     if not bot.Parent or bot:GetAttribute("Dead") then return false end
11812     local current = bot:GetPivot().Position
11813     local nextPoint = grounded(start:Lerp(goal, step / steps), current.Y)
11814     -- A player may have stepped into the path since it was planned. Stop
11815     -- before touching them; the caller replans from this actual position.
11816     if not nextPoint or not clear(current, nextPoint) then
11817         bot:SetAttribute("NavigationBlocked", true)
11818         return false
11819     end
11820     bot:PivotTo(CFrame.lookAt(nextPoint, nextPoint + direction.Unit))
11821     task.wait(0.06)
11822 end
11823 return true
11824 end
11825 local function botMove(bot, target)
11826     local destination = liveBotInteractionPoint(target)
11827     -- A blocked leg is replanned, never replaced with a teleport/straight
11828     -- line through a wall. Radius matches the swept body, not just its torso.
11829     for attempt = 1, 3 do
11830         if not bot.Parent or bot:GetAttribute("Dead") then return false end
11831         local path = PathfindingService.CreatePath({AgentRadius = 1.7, AgentHeight = 5.8, AgentCanJump =
11832             ↪ false, WaypointSpacing = 5})
11833         local ok = pcall(function() path:ComputeAsync(bot:GetPivot().Position, destination) end)
11834         local arrived = ok and path.Status == Enum.PathStatus.Success
11835         if arrived then
11836             for _, waypoint in ipairs(path:GetWaypoints()) do
11837                 if not walkBotSegment(bot, waypoint.Position + Vector3.new(0, 3.25, 0)) then arrived =
11838                     ↪ false break end
11839             end
11840         end
11841         if arrived then return true end
11842         bot:SetAttribute("PathFallbacks", (bot:GetAttribute("PathFallbacks") or 0) + 1)
11843         task.wait(0.25 * attempt)
11844     end
11845     return false
11846 end
11847 local function liveBotPlant(bot, visible)
11848     local oldHeld = bot:FindFirstChild("BotHeldPlant")
11849     if oldHeld then oldHeld:Destroy() end
11850     if not visible then return end
11851     local held = Instance.new("Model")
11852     held.Name = "BotHeldPlant"
11853     held.Parent = bot
11854     local base = bot:GetPivot() * CFrame.new(1.45, 0.1, -0.7)
11855     for _, data in ipairs({
11856         {"Pot", Vector3.new(0.82, 0.62, 0.82), Vector3.new(0, 0, 0), Color3.fromRGB(116, 75, 45)},
11857         {"Stem", Vector3.new(0.13, 1.0, 0.13), Vector3.new(0, 0.72, 0), Color3.fromRGB(58, 160, 70)},
11858         {"LeafL", Vector3.new(0.55, 0.13, 0.34), Vector3.new(-0.25, 0.9, 0), Color3.fromRGB(73, 185, 84)},
11859         {"LeafR", Vector3.new(0.55, 0.13, 0.34), Vector3.new(0.25, 1.12, 0), Color3.fromRGB(73, 185, 84)},
11860     }) do
11861         local piece = Instance.new("Part")
11862         piece.Name = data[1]
11863         piece.Size = data[2]
11864         piece.CFrame = base * CFrame.new(data[3])
11865         piece.Color = data[4]
11866         piece.Material = Enum.Material.SmoothPlastic
11867         piece.Anchored = true
11868         piece.CanCollide = false
11869         piece.CanTouch = false
11870         piece.Parent = held
11871     end
11872 end
11873 local function refreshLiveBotBoard()

```

```

11873     local workers = CollectionService:GetTagged("LabWorkerBot")
11874     local cycles, carrying, knockedOut, rivalScore = 0, 0, 0, 0
11875     for _, worker in ipairs(workers) do
11876         cycles = cycles + (worker:GetAttribute("CompletedCycles") or 0)
11877         if (worker:GetAttribute("CarryingPlant") or "") ~= "" then carrying = carrying + 1 end
11878         if worker:GetAttribute("KnockedOut") then knockedOut = knockedOut + 1 end
11879         if worker:GetAttribute("Competitor") then rivalScore = worker:GetAttribute("Score") or 0 end
11880     end
11881     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
11882     local failed, failedSamples, totalSamples = 0, 0, 0
11883     for _, chamber in ipairs(chambers) do
11884         local count = chamber:GetAttribute("SampleCount") or 0
11885         totalSamples = totalSamples + count
11886         if chamber:GetAttribute("FunctionalState") == "FAILED" then
11887             failed = failed + 1
11888             failedSamples = failedSamples + count
11889         end
11890     end
11891     for _, board in ipairs(CollectionService:GetTagged("BotTestBoard")) do
11892         for _, label in ipairs(board:GetDescendants()) do
11893             if label:IsA("TextLabel") and label.Name == "BotTestStatus" then
11894                 label.Text = string.format(
11895                     "NPC GAMEPLAY TESTS\nWorkers: %d / 50 • knocked out: %d\nRescue cycles: %d • plants
↳ in transit: %d\nFailed chambers: %d / %d • failed samples: %d\nStored samples: %d
↳ • RIVAL SCORE: %d",
11896                     #workers, knockedOut, cycles, carrying, failed, #chambers, failedSamples, totalSamples,
↳ rivalScore)
11897             end
11898         end
11899     end
11900 end
11901
11902 local function setLiveBotStatus(bot, message)
11903     bot:SetAttribute("TaskState", message)
11904     local status = bot:FindFirstChild("Status", true)
11905     if status and status:IsA("TextLabel") then status.Text = (bot:GetAttribute("DisplayName") or bot.Name)
↳ .. "\n" .. message end
11906 end
11907
11908 -- Same room-occupancy gate as the installed script: a worker only runs
11909 -- while a player is in the room with it.
11910 local liveFloorBand, liveWakeRadius = 10, 35
11911 local liveRoomsByLevel = {}
11912 do
11913     local complex = workspace:FindFirstChild("Laboratory_Complex")
11914     local packed = complex and complex:GetAttribute("RoomIndex")
11915     if typeof(packed) == "string" then
11916         for entry in packed:gmatch("[^;]+") do
11917             local level, name, x1, z1, x2, z2, y =
11918                 entry:match("^([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+):([^:]+)$")
11919             if level then
11920                 liveRoomsByLevel[level] = liveRoomsByLevel[level] or {}
11921                 table.insert(liveRoomsByLevel[level], {
11922                     name = name, x1 = tonumber(x1), z1 = tonumber(z1),
11923                     x2 = tonumber(x2), z2 = tonumber(z2), y = tonumber(y),
11924                 })
11925             end
11926         end
11927     end
11928 end
11929
11930 local function liveRoomAt(levelName, position)
11931     local list = levelName and liveRoomsByLevel[levelName]
11932     if not list then return nil end
11933     for _, r in ipairs(list) do
11934         if math.abs(position.Y - r.y) < liveFloorBand
11935             and position.X >= r.x1 and position.X <= r.x2
11936             and position.Z >= r.z1 and position.Z <= r.z2 then
11937             return r.name
11938         end
11939     end
11940     return nil
11941 end
11942
11943 local function liveBotHasObserver(bot)
11944     local ok, pivot = pcall(function() return bot:GetPivot().Position end)

```

```

11945     if not ok then return false end
11946     local levelName = bot:GetAttribute("Level")
11947     local botRoom = liveRoomAt(levelName, pivot)
11948     for _, player in ipairs(Players:GetPlayers()) do
11949         local character = player.Character
11950         local hrp = character and character:FindFirstChild("HumanoidRootPart")
11951         if hrp then
11952             local here = hrp.Position
11953             if math.abs(here.Y - pivot.Y) < liveFloorBand then
11954                 if botRoom and liveRoomAt(levelName, here) == botRoom then return true end
11955                 local dx, dz = here.X - pivot.X, here.Z - pivot.Z
11956                 if dx * dx + dz * dz < liveWakeRadius * liveWakeRadius then return true end
11957             end
11958         end
11959     end
11960     return false
11961 end
11962
11963 local bots = CollectionService:GetTagged("LabWorkerBot")
11964 for index, bot in ipairs(bots) do
11965     refreshLiveBotHealth(bot)
11966     -- Sleeping bots are cheap, so there is no herd to spread out.
11967     local startSlot = bot:GetAttribute("LevelSlot") or index
11968     task.delay(startSlot * 0.08, function()
11969         while bot.Parent do
11970             local awake = bot:GetAttribute("Competitor") or liveBotHasObserver(bot)
11971             if not awake then
11972                 setLiveBotStatus(bot, "Idle -- no one in the room")
11973                 task.wait(1)
11974             elseif bot:GetAttribute("KnockedOut") then
11975                 task.wait(1)
11976             else
11977                 local levelName = bot:GetAttribute("Level")
11978                 local failed, stations = {}, {}
11979                 for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
11980                     if equipment:GetAttribute("Level") == levelName then
11981                         if equipment:GetAttribute("Kind") == "Plant Growth Chamber"
11982                             and equipment:GetAttribute("FunctionalState") == "FAILED"
11983                             and (equipment:GetAttribute("SampleCount") or 0) > 0
11984                             and not equipment:GetAttribute("ReservedByBot") then
11985                             table.insert(failed, equipment)
11986                         end
11987                         if plantProcesses[equipment:GetAttribute("Kind")] then table.insert(stations,
11988                             ↪ equipment) end
11989                     end
11990                 end
11991                 if #failed == 0 then
11992                     setLiveBotStatus(bot, "Monitoring chamber alarms")
11993                 else
11994                     table.sort(failed, function(a, b)
11995                         return (tonumber((a:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
11996                             < (tonumber((b:GetAttribute("GrowthCode") or ""):match("%d+")) or math.huge)
11997                     end)
11998                     local source = bot:GetAttribute("Competitor") and failed[1] or failed[math.random(1,
11999                         ↪ #failed)]
12000                     local destination = liveDestinations(source)[1]
12001                     if not destination then
12002                         setLiveBotStatus(bot, "Waiting for safe capacity")
12003                     else
12004                         source:SetAttribute("ReservedByBot", bot.Name)
12005                         destination:SetAttribute("IncomingReservations",
12006                             ↪ (destination:GetAttribute("IncomingReservations") or 0) + 1)
12007                         source:SetAttribute("EvacuationDestination", destination:GetAttribute("GrowthCode")
12008                             ↪ or "")
12009                         refreshLiveTelemetry(source)
12010                         setLiveBotStatus(bot, "Responding to " .. (source:GetAttribute("GrowthCode") or
12011                             ↪ "alarm"))
12012                         botMove(bot, source)
12013                         local sourceSamples = liveChamberSamples(source)
12014                         local plantId = table.remove(sourceSamples, 1)
12015                         if plantId and source:GetAttribute("ReservedByBot") == bot.Name then
12016                             saveLiveChamber(source, sourceSamples)
12017                             showChamberPlant(source, true)
12018                             refreshLiveTelemetry(source)
12019                             bot:SetAttribute("CarryingPlant", plantId)
12020                             liveBotPlant(bot, true)

```



```

12016         setLiveBotStatus(bot, "Evacuating " .. plantId)
12017
12018         local stage = source:GetAttribute("GrowthStage") or 0
12019         if #stations > 0 then
12020             local station = stations[((bot:GetAttribute("BotNumber") or index) - 1) %
12021                 ⇩ #stations + 1]
12022             botMove(bot, station)
12023             local process = plantProcesses[station:GetAttribute("Kind")]
12024             stage = math.clamp(stage + (process.gain or 0), 0, 5)
12025             station:SetAttribute("BotTestUses", (station:GetAttribute("BotTestUses") or
12026                 ⇩ 0) + 1)
12027         end
12028
12029         setLiveBotStatus(bot, "Delivering to " ..
12030             ⇩ (destination:GetAttribute("GrowthCode") or "working chamber"))
12031         botMove(bot, destination)
12032         local destinationSamples = liveChamberSamples(destination)
12033         if destination:GetAttribute("FunctionalState") ~= "FAILED"
12034             and #destinationSamples < (destination:GetAttribute("Capacity") or 10) then
12035             table.insert(destinationSamples, plantId)
12036             saveLiveChamber(destination, destinationSamples)
12037             destination:SetAttribute("GrowthStage",
12038                 ⇩ math.max(destination:GetAttribute("GrowthStage") or 0, stage))
12039             showChamberPlant(destination, true)
12040             refreshLiveTelemetry(destination)
12041             liveEmergencyMoves = liveEmergencyMoves + 1
12042             bot:SetAttribute("CompletedCycles", (bot:GetAttribute("CompletedCycles") or
12043                 ⇩ 0) + 1)
12044             if bot:GetAttribute("Competitor") then
12045                 bot:SetAttribute("Score", (bot:GetAttribute("Score") or 0) +
12046                     ⇩ LIVE_POINTS.MOVE_ON_TIME)
12047                 setLiveBotStatus(bot, "+" .. LIVE_POINTS.MOVE_ON_TIME .. " resource
12048                     ⇩ win")
12049                 refreshLiveScoreboard()
12050             else
12051                 setLiveBotStatus(bot, "Rescue cycle passed")
12052             end
12053         else
12054             table.insert(sourceSamples, plantId)
12055             saveLiveChamber(source, sourceSamples)
12056             showChamberPlant(source, true)
12057             refreshLiveTelemetry(source)
12058             setLiveBotStatus(bot, "Destination changed; retrying")
12059         end
12060         bot:SetAttribute("CarryingPlant", "")
12061         liveBotPlant(bot, false)
12062     end
12063     destination:SetAttribute("IncomingReservations", math.max(0,
12064         ⇩ (destination:GetAttribute("IncomingReservations") or 1) - 1))
12065     if source:GetAttribute("ReservedByBot") == bot.Name then
12066         ⇩ source:SetAttribute("ReservedByBot", nil) end
12067     end
12068 end
12069
12070     end
12071     end
12072     refreshLiveBotBoard()
12073     task.wait(1.5 + math.random())
12074 end
12075
12076     end)
12077 end
12078
12079 local LIVE_FAILURE_REASONS = {
12080     "Temperature control fault", "Humidity sensor drift", "Ventilation fan blocked",
12081     "Door seal leak", "Lighting circuit fault", "Nutrient pump pressure low",
12082 }
12083
12084 local previousLiveMoves = 0
12085 setLiveFailure = function(chamber, failed)
12086     local codeNumber = tonumber((chamber:GetAttribute("GrowthCode") or ""):match("%d+")) or 1
12087     local code = chamber:GetAttribute("GrowthCode") or ("R" .. codeNumber)
12088     local floorLabel = chamber:GetAttribute("FloorLabel") or chamber:GetAttribute("Level") or "UNKNOWN
12089         ⇩ FLOOR"
12090     if failed then
12091         chamber:SetAttribute("FunctionalState", "FAILED")
12092         if chamber:GetAttribute("PlantLife") == nil then chamber:SetAttribute("PlantLife", 100) end
12093         chamber:SetAttribute("RepairKey", "R")
12094         chamber:SetAttribute("RepairInstruction", "Go to " .. floorLabel .. " • CHAMBER " .. code .. " •
12095             ⇩ HOLD R TO REPAIR")

```

```

12081     chamber:SetAttribute("FailureReason", LIVE_FAILURE_REASONS[math.random(1, #LIVE_FAILURE_REASONS)])
12082     chamber:SetAttribute("FailureSince", os.time())
12083     chamber:SetAttribute("Temperature", 31 + codeNumber % 8)
12084     chamber:SetAttribute("Humidity", 35 + codeNumber % 24)
12085     chamber:SetAttribute("Ventilation", 10 + codeNumber % 28)
12086     local destination = liveDestinations(chamber)[1]
12087     chamber:SetAttribute("EvacuationDestination", destination and
12088     ↪ destination:GetAttribute("GrowthCode") or "")
12089     for _, player in ipairs(Players:GetPlayers()) do
12090         if livePlayerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
12091             ↪ liveAssignTask(player) end
12092         if livePlayerRole(player) == "Laboratory Technician" and not
12093         ↪ player:GetAttribute("AssignedTransferSource") then liveAssignTask(player) end
12094     end
12095 else
12096     chamber:SetAttribute("FunctionalState", "WORKING")
12097     chamber:SetAttribute("RepairInstruction", "Unit working • E TAKE / DROP")
12098     chamber:SetAttribute("FailureReason", "")
12099     chamber:SetAttribute("FailureSince", 0)
12100     chamber:SetAttribute("EvacuationDestination", "")
12101     chamber:SetAttribute("Temperature", 22 + codeNumber % 4)
12102     chamber:SetAttribute("Humidity", 67 + codeNumber % 14)
12103     chamber:SetAttribute("Ventilation", 62 + codeNumber % 29)
12104     chamber:SetAttribute("OperationStarted", os.time())
12105     if (chamber:GetAttribute("SampleCount") or 0) > 0 and (chamber:GetAttribute("PlantLife") or 100) <
12106     ↪ 100 then
12107         recoverLivePlantLife(chamber, chamber:GetAttribute("PlantLife"), nil,
12108         ↪ chamber:GetAttribute("PlantId"))
12109     end
12110 end
12111 refreshLiveTelemetry(chamber)
12112 end
12113
12114 local LIVE_GENERIC_REASONS = {"Power relay fault", "Controller link lost", "Pump interlock fault",
12115 ↪ "Calibration expired", "Motor overload"}
12116 setLiveGenericFailure = function(part, failed)
12117     if failed then
12118         part:SetAttribute("FunctionalState", "FAILED")
12119         part:SetAttribute("FailureReason", LIVE_GENERIC_REASONS[math.random(1, #LIVE_GENERIC_REASONS)])
12120         part:SetAttribute("FailureSince", os.time())
12121         part:SetAttribute("RepairInstruction", liveFloor(part) .. " • " .. liveEquipmentLabel(part) .. " •
12122         ↪ HOLD R")
12123     else
12124         part:SetAttribute("FunctionalState", "WORKING")
12125         part:SetAttribute("FailureReason", "")
12126         part:SetAttribute("FailureSince", 0)
12127         part:SetAttribute("RepairInstruction", "WORKING • E USE / INSPECT")
12128         part:SetAttribute("OperationStarted", os.time())
12129     end
12130     local readout = part:FindFirstChild("Readout", true)
12131     if readout and readout:IsA("TextLabel") then
12132         readout.Text = (failed and "FAILED • HOLD R\n" or "WORKING\n") .. liveFloor(part) .. " • " ..
12133         ↪ liveEquipmentLabel(part)
12134         .. "\nPLANTS " .. (part:GetAttribute("SampleCount") or 0) .. "/" ..
12135         ↪ (part:GetAttribute("Capacity") or 0)
12136         readout.TextColor3 = failed and Color3.fromRGB(255, 105, 90) or Color3.fromRGB(115, 235, 165)
12137     end
12138     if failed then
12139         for _, player in ipairs(Players:GetPlayers()) do
12140             if livePlayerRole(player) == "Technician" and not player:GetAttribute("AssignedRepairId") then
12141                 ↪ liveAssignTask(player) end
12142         end
12143     end
12144 end
12145
12146 local function liveAdaptiveFailureTick()
12147     local chambers = CollectionService:GetTagged("PlantGrowthChamber")
12148     local failed, working = {}, {}
12149     local failedSamples, totalSamples, workingFree = 0, 0, 0
12150     for _, chamber in ipairs(chambers) do
12151         local count = chamber:GetAttribute("SampleCount") or 0
12152         local capacity = chamber:GetAttribute("Capacity") or 10
12153         totalSamples = totalSamples + count
12154         if chamber:GetAttribute("FunctionalState") == "FAILED" then
12155             table.insert(failed, chamber)
12156             failedSamples = failedSamples + count
12157         end
12158     end
12159 end

```

```

12147     else
12148         if os.time() - (chamber:GetAttribute("LastRepairedAt") or 0) >= 45 then
12149             table.insert(working, chamber)
12150         end
12151         workingFree = workingFree + math.max(0, capacity - count -
            ↪ (chamber:GetAttribute("IncomingReservations") or 0))
12152     end
12153 end
12154 local throughput = liveEmergencyMoves - previousLiveMoves
12155 previousLiveMoves = liveEmergencyMoves
12156 local backlogRatio = failedSamples / math.max(1, totalSamples)
12157 local playerCount = math.max(1, #Players:GetPlayers())
12158 local targetRate = 0.60 + math.clamp((throughput - 5) * 0.003, -0.03, 0.08)
12159     + math.clamp((playerCount - 1) * 0.012, 0, 0.10)
12160     - math.clamp((backlogRatio - 0.60) * 0.08, -0.04, 0.04)
12161 if workingFree < 20 then targetRate = targetRate - 0.12 end
12162 targetRate = math.clamp(targetRate, 0.42, 0.76)
12163 local targetFailed = math.floor(#chambers * targetRate + 0.5)
12164 local maxChange = math.clamp(2 + playerCount * 2, 4, 12)
12165 local change = math.clamp(targetFailed - #failed, -maxChange, maxChange)
12166 if change > 0 then
12167     for _ = 1, change do
12168         if #working == 0 then break end
12169         local chosen = table.remove(working, math.random(1, #working))
12170         setLiveFailure(chosen, true)
12171     end
12172 elseif change < 0 then
12173     table.sort(failed, function(a, b) return (a:GetAttribute("SampleCount") or 0) <
            ↪ (b:GetAttribute("SampleCount") or 0) end)
12174     for _ = 1, -change do
12175         if #failed == 0 then break end
12176         setLiveFailure(table.remove(failed, 1), false)
12177     end
12178 end
12179
12180 local currentFailed = 0
12181 for _, chamber in ipairs(chambers) do
12182     if chamber:GetAttribute("FunctionalState") == "FAILED" then
12183         currentFailed = currentFailed + 1
12184         chamber:SetAttribute("Temperature", math.clamp((chamber:GetAttribute("Temperature") or 33) +
            ↪ math.random(-2, 2), 28, 42))
12185         chamber:SetAttribute("Humidity", math.clamp((chamber:GetAttribute("Humidity") or 45) +
            ↪ math.random(-4, 4), 25, 65))
12186         chamber:SetAttribute("Ventilation", math.clamp((chamber:GetAttribute("Ventilation") or 20) +
            ↪ math.random(-5, 3), 0, 48))
12187         local destination = liveDestinations(chamber)[1]
12188         chamber:SetAttribute("EvacuationDestination", destination and
            ↪ destination:GetAttribute("GrowthCode") or "")
12189     else
12190         local samples = liveChamberSamples(chamber)
12191         local capacity = chamber:GetAttribute("Capacity") or 10
12192         local refillChance = math.clamp(0.10 + (capacity - #samples) * 0.022 + throughput * 0.002
            + playerCount * 0.008, 0.10, 0.46)
12193         if #samples < capacity and math.random() < refillChance then
12194             liveSampleSerial = liveSampleSerial + 1
12195             table.insert(samples, string.format("%s-G%05d", chamber:GetAttribute("GrowthCode") or "R",
            ↪ liveSampleSerial))
12196             saveLiveChamber(chamber, samples)
12197             chamber:SetAttribute("GerminationDate", os.date("%Y-%m-%d %H:%M"))
12198             showChamberPlant(chamber, true)
12199         end
12200     end
12201 end
12202 refreshLiveTelemetry(chamber)
12203 end
12204 local complex = workspace:FindFirstChild("Laboratory_Complex")
12205 if complex then
12206     complex:SetAttribute("AdaptiveFailureTargetPercent", math.floor(targetRate * 100 + 0.5))
12207     complex:SetAttribute("FailingChambers", currentFailed)
12208     complex:SetAttribute("FailedSampleBacklog", failedSamples)
12209     complex:SetAttribute("EmergencyMovesCompleted", liveEmergencyMoves)
12210     complex:SetAttribute("RecentTransferThroughput", throughput)
12211     complex:SetAttribute("ActivePlayerCount", playerCount)
12212     complex:SetAttribute("ActiveNPCWorkers", math.clamp(8 + playerCount * 6 + math.floor(failedSamples
            ↪ / 20), 8, 50))
12213 -- Per-FL00R budget. Multiplied across the basement plus every level
12214 -- this is what actually decides how many workers are visibly busy.

```

```

12215     complex:SetAttribute("ActiveNPCWorkersPerFloor", math.clamp(6 + playerCount * 2 +
12216     ↪ math.floor(failedSamples / 60), 6, 20))
12217 end
12218
12219 local function liveGenericFailureTick()
12220     local failed, working = {}, {}
12221     for _, equipment in ipairs(CollectionService:GetTagged("LabEquipment")) do
12222         if equipment:GetAttribute("Kind") ~= "Plant Growth Chamber" then
12223             if equipment:GetAttribute("FunctionalState") == "FAILED" then table.insert(failed, equipment)
12224             elseif os.time() - (equipment:GetAttribute("LastRepairedAt") or 0) >= 45 then
12225                 ↪ table.insert(working, equipment) end
12226         end
12227     end
12228     local playerCount = math.max(1, #Players:GetPlayers())
12229     local targetRate = math.clamp(0.12 + playerCount * 0.018, 0.14, 0.28)
12230     local target = math.max(2, math.floor((#failed + #working) * targetRate + 0.5))
12231     local maxChange = math.clamp(1 + math.ceil(playerCount / 2), 2, 6)
12232     local change = math.clamp(target - #failed, -maxChange, maxChange)
12233     for _ = 1, math.max(0, change) do
12234         if #working == 0 then break end
12235         setLiveGenericFailure(table.remove(working, math.random(1, #working)), true)
12236     end
12237     for _ = 1, math.max(0, -change) do
12238         if #failed == 0 then break end
12239         setLiveGenericFailure(table.remove(failed, 1), false)
12240     end
12241 end
12242
12243 task.spawn(function()
12244     while task.wait(2) do refreshLiveBotBoard() end
12245 end)
12246 task.spawn(function()
12247     while task.wait(2) do refreshLiveRoomMonitors() end
12248 end)
12249 task.spawn(function()
12250     task.wait(3)
12251     while true do
12252         liveAdaptiveFailureTick()
12253         liveGenericFailureTick()
12254         task.wait(math.clamp(15 - #Players:GetPlayers() * 1.25, 6, 14))
12255     end
12256 end)
12257 Players.PlayerRemoving:Connect(clearLiveRepairPath)
12258 warn("Seed Lab live fallback armed: normal Script.Source installation was unavailable, but E/F gameplay is
12259 ↪ active for this play session.")
12260 end
12261
12262 do
12263     for _, n in ipairs({"LabKinds"}) do
12264         local e = ReplicatedStorage:FindFirstChild(n); if e then e:Destroy() end
12265     end
12266     for _, n in ipairs({"LabEquipment", "LabElevators", "LabRoles", "LabWorkflow", "SeedLabWorkflow",
12267     ↪ "ServerBlasterManager"}) do
12268         local e = ServerScriptService:FindFirstChild(n); if e then e:Destroy() end
12269     end
12270     local modules = ReplicatedStorage:FindFirstChild("Modules")
12271     if modules and not modules:IsA("Folder") then modules:Destroy(); modules = nil end
12272     if not modules then
12273         modules = Instance.new("Folder")
12274         modules.Name = "Modules"
12275         modules.Parent = ReplicatedStorage
12276     end
12277     for _, moduleName in ipairs({"Blaster", "BlasterController"}) do
12278         local oldModule = modules:FindFirstChild(moduleName)
12279         if oldModule then oldModule:Destroy() end
12280     end
12281     -- Replace the old quiz with the compact addition-check popup.
12282     for _, clientName in ipairs({"LabQuizClient", "LabSidearmClient", "LabHUD"}) do
12283         local oldClient = StarterPlayer.StarterPlayerScripts:FindFirstChild(clientName)
12284         if oldClient then oldClient:Destroy() end
12285     end
12286     local oldFlow = ReplicatedStorage:FindFirstChild("LabFlow"); if oldFlow then oldFlow:Destroy() end
12287     local oldFlow2 = ReplicatedStorage:FindFirstChild("ISTA_Flow"); if oldFlow2 then oldFlow2:Destroy() end
12288     local mod = Instance.new("ModuleScript"); mod.Name = "LabKinds"

```

```

12287     local s1 = Instance.new("Script");      s1.Name = "LabEquipment"
12288     local s2 = Instance.new("Script");      s2.Name = "LabElevators"
12289     local s3 = Instance.new("Script");      s3.Name = "LabRoles"
12290     local s4 = Instance.new("LocalScript");  s4.Name = "LabSidearmClient"
12291     local s5 = Instance.new("LocalScript");  s5.Name = "LabHUD"
12292     local quizClient = Instance.new("LocalScript"); quizClient.Name = "LabQuizClient"
12293     local blasterModule = Instance.new("ModuleScript"); blasterModule.Name = "Blaster"
12294     local controllerModule = Instance.new("ModuleScript"); controllerModule.Name = "BlasterController"
12295     local serverBlaster = Instance.new("Script"); serverBlaster.Name = "ServerBlasterManager"
12296
12297     local ok = pcall(function()
12298         mod.Source = FLOW_SOURCE
12299         s1.Source = EQUIP_SCRIPT_SOURCE
12300         s2.Source = ELEVATOR_SCRIPT_SOURCE
12301         s3.Source = ROLES_SCRIPT_SOURCE
12302         s4.Source = SIDEARM_CLIENT_SOURCE
12303         s5.Source = LAB_HUD_SOURCE
12304         quizClient.Source = QUIZ_CLIENT_SOURCE
12305         blasterModule.Source = BLASTER_SOURCE
12306         controllerModule.Source = BLASTER_CONTROLLER_SOURCE
12307         serverBlaster.Source = SERVER_BLASTER_MANAGER_SOURCE
12308     end)
12309
12310     if ok then
12311         mod.Parent = ReplicatedStorage
12312         blasterModule.Parent = modules
12313         controllerModule.Parent = modules
12314         serverBlaster.Parent = ServerScriptService
12315         s1.Parent = ServerScriptService
12316         s2.Parent = ServerScriptService
12317         s3.Parent = ServerScriptService
12318         s4.Parent = StarterPlayer.StarterPlayerScripts
12319         s5.Parent = StarterPlayer.StarterPlayerScripts
12320         quizClient.Parent = StarterPlayer.StarterPlayerScripts
12321         print("Installed lab runtime plus cross-platform Blaster, BlasterController and ServerBlasterManager.")
12322     else
12323         mod:Destroy(); s1:Destroy(); s2:Destroy(); s3:Destroy(); s4:Destroy(); s5:Destroy()
12324         quizClient:Destroy()
12325         blasterModule:Destroy(); controllerModule:Destroy(); serverBlaster:Destroy()
12326         if RunService.IsRunning() then
12327             startLiveRuntimeFallback()
12328         else
12329             warn("Could not write script source from this context. Run the file from the Studio Command Bar.")
12330         end
12331     end
12332 end
12333
12334 --=====
12335 -- LOAD TIME
12336 --=====
12337 -- The place is ~77,000 parts. Without streaming, a joining client has to
12338 -- download and build every one of them before it can move -- that is the
12339 -- bulk of the join wait. With streaming on, the server sends only what is
12340 -- near the player and fills the rest in as they walk, which is the single
12341 -- biggest thing available here for load time. These are place properties,
12342 -- so they are saved with the place and only settable from Edit.
12343 do
12344     local ok = pcall(function() Workspace.StreamingEnabled = true end)
12345     if ok then
12346         pcall(function() Workspace.StreamingMinRadius = 160 end)
12347         pcall(function() Workspace.StreamingTargetRadius = 512 end)
12348         pcall(function()
12349             Workspace.StreamingIntegrityMode = Enum.StreamingIntegrityMode.MinimumRadiusPause
12350         end)
12351         print("Streaming enabled for faster joins.")
12352     else
12353         warn("Could not enable streaming from this context; set Workspace.StreamingEnabled by hand.")
12354     end
12355 end
12356
12357 --=====
12358 -- LIGHTING SERVICE
12359 --=====
12360 if CONFIGURE_LIGHTING then
12361     local ok = pcall(function() Lighting.Technology = Enum.Technology.ShadowMap end)
12362     if not ok then warn("Set Lighting.Technology to ShadowMap or Future by hand.") end

```

```

12363 Lighting.GlobalShadows          = true
12364 Lighting.Brightness              = 1.3
12365 Lighting.ExposureCompensation    = -0.35
12366 Lighting.Ambient                  = Color3.fromRGB(58, 60, 66)
12367 Lighting.OutdoorAmbient           = Color3.fromRGB(92, 96, 104)
12368 Lighting.EnvironmentDiffuseScale = 0.35
12369 Lighting.EnvironmentSpecularScale = 0.25
12370 Lighting.ClockTime                 = 13.5
12371 Lighting.GeographicLatitude       = 45
12372 for _, fx in ipairs(Lighting:GetChildren()) do
12373     if fx:IsA("BloomEffect") then
12374         fx.Intensity = math.min(fx.Intensity, 0.4)
12375         fx.Threshold = math.max(fx.Threshold, 1.2)
12376     end
12377 end
12378 end
12379
12380 local lamps, doors = 0, 0
12381 for _, d in ipairs(root:GetDescendants()) do
12382     if d:IsA("PointLight") then lamps = lamps + 1 end
12383     if d.Name:find("_leaf") then doors = doors + 1 end
12384 end
12385 local growthChamberCount = #CollectionService:GetTagged("PlantGrowthChamber")
12386 local workerBotCount = #CollectionService:GetTagged("LabWorkerBot")
12387 local expectedChambers = (#LEVELS + 1) * 50
12388 local expectedBots = (#LEVELS + 1) * BOTS_PER_LEVEL
12389 if growthChamberCount ~= expectedChambers then
12390     warn("BUILD CHECK FAILED: expected " .. expectedChambers .. " plant growth chambers, created " ..
        ↳ growthChamberCount .. ".")
12391 end
12392 if workerBotCount ~= expectedBots then
12393     warn("BUILD CHECK FAILED: expected " .. expectedBots .. " worker test NPCs, created " .. workerBotCount ..
        ↳ ".")
12394 end
12395 print(string.format(
12396     "Built: %.0f x %.0f studs -- %d floors (ground floor %d rooms, floors 2-%d %d rooms each), basement %d
        ↳ rooms -- %d equipment items including %d plant growth chambers, %d worker test NPCs, %d door leaves,
        ↳ stairs-only (no elevators), %d lights, %d parts.",
12397     X(W), X(L), #LEVELS, #GROUND_ROOMS, #LEVELS, #TYPICAL_ROOMS, #BASEMENT_ROOMS, placedCount,
        ↳ growthChamberCount,
12398     workerBotCount, doors, lamps, #root:GetDescendants()))

```

Chapter 59

W02 - InstallGWAP.lua

InstallGWAP.lua

Authority group: workspace. 115778 UTF-8 bytes; 2109 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Run in EDIT mode after SeedLabGenerator. Does not change movement key bindings.
2 assert(not game:GetService("RunService"):IsRunning(),"Stop Play before installing")
3 local backups=game:GetService("ServerStorage"):FindFirstChild("BeforeGWAP") or Instance.new("Folder")
4 backups.Name="BeforeGWAP"; backups.Parent=game:GetService("ServerStorage")
5 local function install(parent,name,className,source)
6     local old=parent:FindFirstChild(name)
7     if old and not backups:FindFirstChild(name) then local saved=Instance.new("StringValue"); saved.Name=name;
8         ↪ saved.Value=old.Source; saved.Parent=backups end
9     if old then old:Destroy() end
10    local script=Instance.new(className); script.Name=name; script.Source=source; script.Parent=parent
11 end
12 install(game.ReplicatedStorage,"GWAPWorld","ModuleScript",[====[
13 -- Authoritative assignments, research carriers, solo opposition and score accounting.
14 local Players=game:GetService('Players')
15 local RS=game:GetService('ReplicatedStorage')
16 local CS=game:GetService('CollectionService')
17 local SSS=game:GetService('ServerScriptService')
18 local Debris=game:GetService('Debris')
19 local Run=game:GetService('RunService')
20 local World=require(RS:WaitForChild('GWAPWorld'))
21 local scenarios=require(RS:WaitForChild('GWAPScenarios'))
22 local research=World.build()
23 local stations=research.Stations
24 local remote=Instance.new('RemoteEvent'); remote.Name='GWAPAction'; remote.Parent=RS
25 local bridge=Instance.new('BindableFunction'); bridge.Name='GWAPHitBridge'; bridge.Parent=SSS
26 local botsFolder=Instance.new('Folder'); botsFolder.Name='Carriers'; botsFolder.Parent=research
27 local routesFolder=Instance.new('Folder'); routesFolder.Name='Routes'; routesFolder.Parent=research
28 local random=Random.new()
29 local carriers, assignments, cooldowns, armed={}, {}, {}, {}
30 local function alive(p)
31     local c=p.Character; local h=c and c:FindFirstChildOfClass('Humanoid'); return h and h.Health>0 and
32     ↪ c:FindFirstChild('HumanoidRootPart')
33 end
34 local function points(p)
35     local s=p:FindFirstChild('leaderstats'); return s and s:FindFirstChild('Points')
36 end
37 local function award(p,n,reason)
38     local v=points(p); if not v then return end
39     v.Value=math.max(0,v.Value+n); p:SetAttribute('GWAPNotice',reason); p:SetAttribute('LastPointAmount',n);
40     ↪ p:SetAttribute('LastPointAward',reason)
41 end
42 local function role(p) return p:GetAttribute('LabRole') or (p.Team and p.Team.Name) end
43 local function active(a) return a and a.bot and a.bot.Parent and a.bot:GetAttribute('State')== 'RUNNING' end
44 local function nearby(p,bot,d) local r=alive(p); return r and (r.Position-bot:GetPivot().Position).Magnitude<=d
45     ↪ end
46 local function publish(a)
```

```

43   for p,bound in pairs(assignments) do if bound==a then
44       p:SetAttribute('GWAPStage',a.stage); p:SetAttribute('GWAPTotalStages',#a.route)
45       p:SetAttribute('GWAPTarget',a.route[a.stage] or 'Complete');
46       ⇨ p:SetAttribute('GWAPState',a.bot:GetAttribute('State'))
47   end end
48   local function finish(a,state)
49       if not active(a) then return end
50       a.bot:SetAttribute('State',state); a.bot:SetAttribute('TaskState',state);
51       ⇨ a.bot:SetAttribute('HasSeedling',false)
52       local carried=a.bot:FindFirstChild('Mission seedling'); if carried then carried:Destroy() end
53       publish(a); a.finishedAt=os.clock()
54   end
55   local function updateHealth(bot)
56       local t=bot:FindFirstChild('HealthText',true); local fill=bot:FindFirstChild('HealthBarFill',true)
57       local h=bot:GetAttribute('Health') or 100
58       if t then t.Text='LIFE '..math.ceil(h).. ' / 100' end
59       if fill then fill.Size=UDim2.fromScale(h/100,1) end
60   end
61   local function damage(p,bot,amount)
62       local a=carriers[bot]; if not active(a) then return end
63       local h=math.max(0,bot:GetAttribute('Health')-amount); bot:SetAttribute('Health',h); updateHealth(bot)
64       if role(p)=='Technician' and assignments[p]==a then a.disqualified[p]=true;
65       ⇨ p:SetAttribute('GWAPNotice','Protect your carrier: friendly hits cancel escort rewards.') end
66       if h==0 then
67           if assignments[p]==a and role(p)=='Laboratory Technician' then award(p,50,'+50 | Assigned research
68           ⇨ carrier stopped') end
69           finish(a,'INTERCEPTED')
70       end
71   end
72   bridge.OnInvoke=function(p,hit,damageAmount)
73       local node=hit
74       while node and node~=workspace do if carriers[node] then damage(p,node,damageAmount); return true end;
75       ⇨ node=node.Parent end
76       return false
77   end
78   local function routePoints(from,to)
79       local r={}; local function add(v) table.insert(r,v) end
80       local f1=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
81       local f2=to:GetAttribute('Floor'); local dest=to:GetAttribute('Approach')
82       if f1~=f2 then
83           add(Vector3.new(78,from.Y,from.Z)); add(Vector3.new(78,from.Y,-6))
84           local function flight(level,reverse)
85               local steps={Vector3.new(68.7857,level*16.25+3.25,-6)}
86               for i=1,8 do
87                   ⇨ table.insert(steps,Vector3.new(68.7857,level*16.25+i*1.015625+3.25,-15.267857-(i-1)*4.821428))
88                   ⇨ end
89               table.insert(steps,Vector3.new(68.7857,level*16.25+8.125+3.25,-55.28))
90               table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+3.25,-55.28))
91               for i=1,8 do table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+i*1.015625+3.25,-49.017857+(
92               ⇨ i-1)*4.821428)) end
93               table.insert(steps,Vector3.new(88.0714,(level+1)*16.25+3.25,-6));
94               ⇨ table.insert(steps,Vector3.new(78,(level+1)*16.25+3.25,-6))
95               if reverse then for i=#steps,1,-1 do add(steps[i]) end else for _,v in ipairs(steps) do add(v) end
96               ⇨ end
97           end
98           if f2>f1 then for level=f1,f2-1 do flight(level,false) end else for level=f1-1,f2,-1 do
99               ⇨ flight(level,true) end end
100       end
101       add(Vector3.new(78,dest.Y,dest.Z)); add(dest); return r
102   end
103   local function showRoute(a,route)
104       local old=routesFolder:FindFirstChild(a.bot.Name); if old then old:Destroy() end
105       local folder=Instance.new('Folder'); folder.Name=a.bot.Name; folder.Parent=routesFolder
106       for i,v in ipairs(route) do local n=Instance.new('Vector3Value'); n.Name=string.format('%03d',i);
107       ⇨ n.Value=v; n.Parent=folder end
108   end
109   local function go(a,target)
110       local route=routePoints(a.bot:GetPivot().Position,target); showRoute(a,route)
111       for i,goal in ipairs(route) do
112           a.bot:SetAttribute('RouteIndex',i)
113           while active(a) and (goal-a.bot:GetPivot().Position).Magnitude>.12 do
114               local dt=Run.Heartbeat:Wait(); if not active(a) then return false end
115               local start=a.bot:GetPivot().Position; local delta=goal-start
116               local nextPos=start+delta.Unit*math.min(delta.Magnitude,12*math.min(dt,.1))
117               -- Research routes use measured open corridors and each actual stair tread.

```



```

107         -- Stop for players; do not shove them or change their movement settings.
108         local blocked=false
109         for _,p in ipairs(Players:GetPlayers()) do local r=alive(p); if r and
110             ↪ (r.Position-nextPos).Magnitude<2.8 then blocked=true end end
111         if not blocked then
112             local flat=Vector3.new(delta.X,0,delta.Z)
113             a.bot:PivotTo(flat.Magnitude>.01 and CFrame.lookAt(nextPos,nextPos+flat) or
114                 ↪ CFrame.new(nextPos))
115         end
116         if os.clock()>a.deadline then finish(a,'EXPIRED'); return false end
117     end
118     if not active(a) then return false end
119 end
120 local function plantIn(bed,name)
121     local count=bed:GetAttribute('Planted') or 0
122     if count>=12 then return false end
123     local pos=bed.Position+Vector3.new(-8+(count%4)*5,1,-3+math.floor(count/4)*3)
124     World.plant(bed,pos,name); bed:SetAttribute('Planted',count+1); return true
125 end
126 local function checkpoint(a,final)
127     for p,bound in pairs(assignments) do
128         if bound==a and role(p)=='Technician' and not a.disqualified[p] and nearby(p,a.bot,45) then
129             ↪ award(p,final and 60 or 15,final and '+60 | Protected rooftop planting' or '+15 | Protected
130             ↪ research checkpoint') end
131     end
132 end
133 local function runMission(a)
134     for i,name in ipairs(a.route) do
135         a.stage=i; a.bot:SetAttribute('TaskState','Travel to '..name); publish(a)
136         if not go(a,stations[name]) then return end
137         a.bot:SetAttribute('TaskState','Processing: '..name)
138         local seed=a.bot:FindFirstChild('Mission seedling')
139         if seed then seed:PivotTo(CFrame.new(stations[name].Position+Vector3.new(0,2,0))) end
140         for _=1,40 do task.wait(.1); if not active(a) then return end end
141         if i==1 then a.bot:SetAttribute('HasSeedling',true);
142             ↪ seed=World.plant(a.bot,a.bot:GetPivot().Position+Vector3.new(0,0,-2),'Mission seedling') end
143         if i==#a.route then
144             if not plantIn(stations[name],a.title..' | '..a.variant) then finish(a,'PLOT FULL'); return end
145             checkpoint(a,true); finish(a,'COMPLETE'); return
146         else checkpoint(a,false) end
147         if seed then seed:PivotTo(CFrame.new(a.bot:GetPivot().Position+Vector3.new(0,0,-2))) end
148     end
149 end
150 local function startMission(bot,id)
151     local s=scenarios[id]; local route=table.clone(s.route); table.insert(route,'Plot '..id)
152     local a={bot=bot,id=id,title=s.title,route=route,stage=1,disqualified={},deadline=os.clock()+420}
153     a.variant=s.variable..' : '..random:NextInteger(s.low,s.high)..' '..s.units
154     carriers[bot]=a; bot:SetAttribute('State','RUNNING'); bot:SetAttribute('Health',100);
155     ↪ bot:SetAttribute('Scenario',id); bot:SetAttribute('Title',s.title);
156     ↪ bot:SetAttribute('Variant',a.variant); updateHealth(bot)
157     bot:PivotTo(CFrame.new(stations[route[1]]:GetAttribute('Approach')+Vector3.new(0,0,8)))
158     return a
159 end
160 local function makeBot(index)
161     local template
162     for _,b in ipairs(CS:GetTagged('LabWorkerBot')) do if not b:GetAttribute('Seated') and b.PrimaryPart then
163         ↪ template=b; break end end
164     assert(template,'No worker template available')
165     local b=template:Clone(); b.Name='ResearchCarrier_'..index
166     for _,v in ipairs(b:GetDescendants()) do if v:IsA('LuaSourceContainer') or v:IsA('ProximityPrompt') or
167         ↪ v:IsA('Humanoid') then v:Destroy() elseif v:IsA('BasePart') then v.Anchored=true; v.CanCollide=false
168         ↪ end end
169     for _,tag in ipairs(CS:GetTags(b)) do CS:RemoveTag(b,tag) end
170     b:SetAttribute('Dead',false); b:SetAttribute('KnockedOut',false); b:SetAttribute('GWAPCarrier',true);
171     ↪ b.Parent=botsFolder
172     local h=Instance.new('Highlight'); h.Name='ResearchOutline'; h.FillTransparency=1;
173     ↪ h.OutlineColor=Color3.fromRGB(92,244,199); h.DepthMode=Enum.HighlightDepthMode.Occluded; h.Parent=b
174     return b
175 end
176 local available={}; for i=1,5 do available[i]=makeBot(i); available[i]:PivotTo(CFrame.new(78,3.25,170+i*8));
177     ↪ available[i]:SetAttribute('State','IDLE') end
178 local weapons={
179     {name='Research
180     ↪ SMG',damage=18,range=220,cooldown=.11,auto=true,color=Color3.fromRGB(244,191,53),length=2.4},

```

```

169     {name='Precision Rifle',damage=75,range=850,cooldown=1.2,color=Color3.fromRGB(123,99,202),length=4.2},
170     {name='Scatter Blaster',damage=16,range=110,cooldown=.85,pellets=7,spread=.09,color=Color3.fromRGB(74,166,183),length=3},
171     {name='Blast
    ↳ Launcher',damage=65,range=140,cooldown=2.5,blast=15,color=Color3.fromRGB(213,123,54),length=2.6},
172     {name='Pulse Bomb',damage=80,range=65,cooldown=4,blast=20,color=Color3.fromRGB(85,135,56),length=1},
173 }
174 local function equip(p,index)
175     local def=weapons[index]; if not def or not alive(p) then return end
176     local backpack=p:FindFirstChildOfClass('Backpack'); if not backpack then return end
177     local tool=p.Character:FindFirstChild(def.name) or backpack:FindFirstChild(def.name)
178     if not tool then
179         tool=Instance.new('Tool'); tool.Name=def.name; tool.CanBeDropped=false; tool.ToolTip=def.blast and
    ↳ 'Explosive area damage | same aim controls' or 'Research weapon | same aim controls'
180         local attrs={BlasterWeapon=true,LabSidearm=true,BlasterDamage=def.damage,BlasterRange=def.range,Blaste
    ↳ rCooldown=def.cooldown,BlasterAutomatic=def.auto or false,BlasterPellets=def.pellets or
    ↳ 1,BlasterSpread=def.spread or 0,BlasterSelfCostPercent=0,BlasterBlastRadius=def.blast or
    ↳ 0,BlasterADSFov=index==2 and 35 or 55,BlasterTracerColor=def.color,GWAPWeapon=true}
181         for k,v in pairs(attrs) do tool:SetAttribute(k,v) end
182         local h=Instance.new('Part'); h.Name='Handle'; h.Size=Vector3.new(.5,.7,def.length); h.Color=def.color;
    ↳ h.CanCollide=false; h.Massless=true; if index==5 then h.Shape=Enum.PartType.Ball;
    ↳ h.Size=Vector3.new(.85,.85,.85) end; h.Parent=tool
183         local barrel=Instance.new('Part'); barrel.Name='Barrel'; barrel.Size=Vector3.new(.22,.22,.7);
    ↳ barrel.Color=Color3.fromRGB(30,34,39); barrel.CanCollide=false; barrel.Massless=true;
    ↳ barrel.CFrame=h.CFrame*CFrame.new(0,.1,-def.length/2); barrel.Parent=tool
184         local weld=Instance.new('WeldConstraint'); weld.Part0=h; weld.Part1=barrel; weld.Parent=barrel;
    ↳ tool.Parent=backpack
185     end
186     p.Character:FindFirstChildOfClass('Humanoid'):EquipTool(tool)
187 end
188 local function choose(p,id)
189     if not alive(p) or workspace:GetAttribute('RoundPhase')=='BREAK' then return end
190     if role(p)~='Technician' and role(p)~='Laboratory Technician' then p:SetAttribute('GWAPNotice','Choose a
    ↳ role at a blue or green kiosk first.');
```

```

221     local selected={}; local shuffle=table.clone(all); for i=#shuffle,2,-1 do local j=random:NextInteger(1,i);
    ↪  shuffle[i],shuffle[j]=shuffle[j],shuffle[i] end
222     for i=1,count do selected[shuffle[i]]=true end
223     for _,bot in ipairs(all) do
224         local yes=selected[bot]==true; bot:SetAttribute('GWAPArmed',yes)
225         local gun=bot:FindFirstChild('SoloWeapon'); if gun then gun:Destroy() end
226         if yes then
227             table.insert(armed,bot); local anchor=bot:FindFirstChild('Right Arm') or bot.PrimaryPart
228             if anchor then local g=Instance.new('Part'); g.Name='SoloWeapon'; g.Size=Vector3.new(.5,.55,2);
    ↪  g.Color=Color3.fromRGB(235,110,65); g.Anchored=true; g.CanCollide=false; g.CanQuery=false;
    ↪  g.CFrame=anchor.CFrame*CFrame.new(0,-.4,-.7); g.Parent=bot end
229         end
230     end
231     workspace:SetAttribute('GWAPArmedCount',count); workspace:SetAttribute('GWAPBotCount',#all)
232 end
233 Players.PlayerAdded:Connect(function() task.defer(setArmed) end)
234 Players.PlayerRemoving:Connect(function(p) assignments[p]=nil; cooldowns[p]=nil; task.defer(setArmed) end)
235 setArmed()
236 task.spawn(function()
237     while task.wait(2.5) do
238         local list=Players:GetPlayers(); if #list~=1 or workspace:GetAttribute('RoundPhase')=='BREAK' then
    ↪  continue end
239         local p=list[1]; local root=alive(p); if not root then continue end
240         for _,bot in ipairs(armed) do
241             if not bot.Parent or bot:GetAttribute('Dead') or bot:GetAttribute('KnockedOut') or
    ↪  (bot:GetAttribute('Health') or 100)<=0 then continue end
242             -- Assigned escort carriers never attack their own defender.
243             if assignments[p] and assignments[p].bot==bot and role(p)=='Technician' then continue end
244             local origin=bot:GetPivot().Position+Vector3.new(0,1,0); local delta=root.Position-origin
245             if delta.Magnitude<4 or delta.Magnitude>85 then continue end
246             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↪  params.FilterDescendantsInstances={bot}
247             local hit=workspace:Raycast(origin,delta,params)
248             if hit and hit.Instance:IsDescendantOf(p.Character) then
249                 local spread=Vector3.new(random:NextNumber(-2.5,2.5),random:NextNumber(-1.5,1.5),random:NextNu
    ↪  mber(-2.5,2.5))
250                 local shot=workspace:Raycast(origin,delta+spread,params); local endpoint=shot and shot.Position
    ↪  or origin+delta+spread
251                 local beam=Instance.new('Part'); beam.Name='NPC shot'; beam.Anchored=true;
    ↪  beam.CanCollide=false; beam.CanQuery=false; beam.Material=Enum.Material.Neon;
    ↪  beam.Color=Color3.fromRGB(255,105,72);
    ↪  beam.Size=Vector3.new(.07,.07,(endpoint-origin).Magnitude);
    ↪  beam.CFrame=CFrame.lookAt((origin+endpoint)/2,endpoint); beam.Parent=workspace;
    ↪  Debris:AddItem(beam,.18)
252                 if shot and shot.Instance:IsDescendantOf(p.Character) then
253                     local v=points(p); if v then
254                         -- Existing score is an IntValue: accumulate fractional losses, never turn 1% into a
    ↪  minimum one-point tax.
255                         local debt=(p:GetAttribute('GWAPFractionalLoss') or 0)+math.max(0,v.Value)*.01
256                         local loss=math.floor(debt+1e-8); p:SetAttribute('GWAPFractionalLoss',debt-loss);
    ↪  v.Value=math.max(0,v.Value-loss)
257                         p:SetAttribute('GWAPNotice','NPC hit: -1% score');
    ↪  p:SetAttribute('GWAPHitAt',workspace:GetServerTimeNow())
258                     end
259                 end
260             end
261         end
262     end
263 end)
264 workspace:GetAttributeChangedSignal('RoundNumber'):Connect(function()
265     for _,a in pairs(carriers) do if active(a) then finish(a,'SHIFT ENDED') end end
266     for _,p in ipairs(Players:GetPlayers()) do p:SetAttribute('GWAPFractionalLoss',0) end
267     for _,bed in ipairs(stations:GetChildren()) do if bed:GetAttribute('Planted') then for _,v in
    ↪  ipairs(bed:GetChildren()) do if v:IsA('Model') then v:Destroy() end end; bed:SetAttribute('Planted',0)
    ↪  end end
268     setArmed()
269 end)
270 print('[GWAP] 10 scenarios, 5 research carriers, rooftop planting and solo opposition ready')
271 ]====)
272 install(game.ReplicatedStorage,"GWAPScenarios","ModuleScript",[====[
273 -- Gameplay simulations inspired by the supplied GWAP ideas; no real measurements are fabricated.
274 return {
275     {title='Germination viability', purpose='Compare germination under different moisture levels.',
    ↪  route={'Seed bank','Incubator'}, variable='Moisture', units='% ', low=35, high=80},
276     {title='Pathogen resistance', purpose='Track resistant plant tissue through containment and analysis.',
    ↪  route={'Containment','Bio-analyzer'}, variable='Resistance batch', units='', low=1, high=9},

```

```

277 {title='Light spectrum optimization', purpose='Compare seedling responses to visible and infrared light.',
    ↳ route={'Seed bank','Spectrograph','Infrared scanner'}, variable='Photoperiod', units='hours', low=8,
    ↳ high=18},
278 {title='Soil bioremediation', purpose='Trace seedlings through root absorption analysis.', route={'Chemical
    ↳ seedlings','Root analyzer'}, variable='Soil batch', units='', low=1, high=12},
279 {title='Hydroponic versus soil', purpose='Process paired root samples before greenhouse planting.',
    ↳ route={'Split-root tray','Centrifuge','Root analyzer'}, variable='Nutrient batch', units='', low=1,
    ↳ high=8},
280 {title='UV adaptation', purpose='Transport exposed seedlings through vitality screening.', route={'UV
    ↳ chamber','Vitality scanner'}, variable='Exposure', units='seconds', low=5, high=20},
281 {title='Gravitropism assay', purpose='Calibrate tilted roots before planting upright seedlings.',
    ↳ route={'Clinostat','Orientation scanner'}, variable='Tray tilt', units='degrees', low=10, high=80},
282 {title='Drought resistance', purpose='Measure simulated stomatal response before rehydration.',
    ↳ route={'Climate chamber','Porometer'}, variable='Humidity', units='%', low=20, high=45},
283 {title='Microbiome symbiosis', purpose='Carry inoculated seeds through symbiosis incubation.', route={'Wet
    ↳ lab','Symbiosis incubator'}, variable='Inoculum batch', units='', low=1, high=8},
284 {title='Cold-hardiness screening', purpose='Screen thawed seedlings before rooftop recovery.',
    ↳ route={'Cryo-freezer','Vitality scanner'}, variable='Recovery temperature', units='C', low=5, high=20},
285 }
286 ]====})
287 install(game.ReplicatedStorage,"GWAPNPC","ModuleScript",[====[
288 -- Shared NPC presentation uses the same original Weapons Kit assets as player inventory.
289 local N={}
290 local specs={
291 {name='Lab Sidearm',mesh='rbxassetid://2492922600',texture='rbxassetid://2492931263',scale=.1407,size=Vect
    ↳ or3.new(.20,.92,1.36)},
292 {name='Lab Shotgun',mesh='rbxassetid://2492972199',texture='rbxassetid://2492974190',scale=.6666,size=Vect
    ↳ or3.new(.31,1,4.87)},
293 {name='Lab Rifle',mesh='rbxassetid://2759059561',texture='rbxassetid://2759063893',scale=.6383,size=Vector
    ↳ 3.new(.29,1.17,6.18)},
294 }
295 function N.gun(bot,index)
296 local old=bot:FindFirstChild('SoloWeapon'); if old then old:Destroy() end
297 local spec=specs[(index-1)%#specs+1]; local arm=bot:FindFirstChild('Arm') or bot:FindFirstChild('Right
    ↳ Arm') or bot.PrimaryPart
298 if not arm then return end
299 local p=Instance.new('Part'); p.Name='SoloWeapon'; p.Size=spec.size; p.Anchored=true; p.CanCollide=false;
    ↳ p.CanTouch=false; p.CanQuery=false
300 p.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0); p.Color=Color3.fromRGB(126,133,145)
301 local mesh=Instance.new('SpecialMesh'); mesh.MeshType=Enum.MeshType.FileMesh; mesh.MeshId=spec.mesh;
    ↳ mesh.TextureId=spec.texture; mesh.Scale=Vector3.one*spec.scale; mesh.Parent=p
302 p:SetAttribute('InventoryName',spec.name); p.Parent=bot; bot:SetAttribute('NPCGun',spec.name)
303 return p
304 end
305 function N.appearance(bot)
306 if bot:GetAttribute('Gender')~='Girl' then return end
307 local head=bot:FindFirstChild('Head'); if not head or bot:FindFirstChild('ResearchHair') then return end
308 local hair=bot:FindFirstChild('HairCrown'); local color=hair and hair.Color or Color3.fromRGB(67,39,25)
309 local folder=Instance.new('Model'); folder.Name='ResearchHair'; folder.Parent=bot
310 local function piece(name,size,offset)
311 local p=Instance.new('Part'); p.Name=name; p.Size=size; p.CFrame=head.CFrame*CFrame.new(offset);
    ↳ p.Color=color; p.Material=Enum.Material.SmoothPlastic; p.Anchored=true; p.CanCollide=false;
    ↳ p.CanQuery=false; p.Parent=folder; return p
312 end
313 piece('Back hair',Vector3.new(1.23,1.7,.34),Vector3.new(0,-.25,.55))
314 for _,side in ipairs({-1,1}) do piece('Side
    ↳ locks',Vector3.new(.28,1.25,.58),Vector3.new(side*.58,-.15,.15)) end
315 local pony=piece('Tied ponytail',Vector3.new(.55,1.3,.65),Vector3.new(0,-.25,.9));
    ↳ pony.Shape=Enum.PartType.Ball
316 local tie=piece('Hair tie',Vector3.new(.6,.16,.66),Vector3.new(0,.25,.9));
    ↳ tie.Color=Color3.fromRGB(107,174,174)
317 end
318 function N.say(bot,message)
319 local head=bot:FindFirstChild('Head') or bot.PrimaryPart; if not head then return end
320 local gui=head:FindFirstChild('MissionSpeech')
321 if not gui then
322 gui=Instance.new('BillboardGui'); gui.Name='MissionSpeech'; gui.Size=UDim2.fromOffset(260,66);
    ↳ gui.StudsOffset=Vector3.new(0,4.8,0); gui.MaxDistance=55; gui.Parent=head
323 local t=Instance.new('TextLabel'); t.Name='Message'; t.Size=UDim2.fromScale(1,1);
    ↳ t.BackgroundColor3=Color3.fromRGB(25,38,45); t.BackgroundTransparency=.12;
    ↳ t.TextColor3=Color3.fromRGB(225,255,240); t.Font=Enum.Font.Gotham; t.TextSize=14;
    ↳ t.TextWrapped=true; t.Parent=gui
324 Instance.new('UICorner',t).CornerRadius=UDim.new(0,10)
325 end
326 gui.Message.Text=message; gui.Enabled=true

```

```

327     local serial=(gui:GetAttribute('Serial') or 0)+1; gui:SetAttribute('Serial',serial)
328     task.delay(9,function() if gui.Parent and gui:GetAttribute('Serial')==serial then gui.Enabled=false end
    ↪ end)
329 end
330 function N.nearPlayer(player,bot)
331     local root=player.Character and player.Character:FindFirstChild('HumanoidRootPart'); if not root then
    ↪ return false end
332     local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ params.FilterDescendantsInstances={player.Character,bot}; params.RespectCanCollide=true
333     local overlap=OverlapParams.new(); overlap.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ overlap.FilterDescendantsInstances={player.Character,bot}; overlap.RespectCanCollide=true
334     for _,offset in ipairs({Vector3.new(5,0,-6),Vector3.new(-5,0,-6),Vector3.new(7,0,0),Vector3.new(-7,0,0),Ve
    ↪ ctor3.new(0,0,8)}) do
335         local pos=root.CFrame:PointToWorldSpace(offset)
336         local floor=workspace.Raycast(pos+Vector3.new(0,3,0),Vector3.new(0,-10,0),params)
337         local sight=workspace.Raycast(root.Position,pos-root.Position,params)
338         if floor and floor.Normal.Y>.7 and not sight then
339             pos=Vector3.new(pos.X,floor.Position.Y+3.25,pos.Z)
340             if #workspace:GetPartBoundsInBox(CFrame.new(pos),Vector3.new(3,4.6,3),overlap)==0 then
    ↪ bot:PivotTo(CFrame.lookAt(pos,Vector3.new(root.Position.X,pos.Y,root.Position.Z))); return true
    ↪ end
341         end
342     end
343     return false
344 end
345
346 local poses=setmetatable({}, {__mode='k'})
347 function N.animate(bot,dt)
348     local root=bot.PrimaryPart; if not root then return end
349     local data=poses[bot]
350     if not data then
351         data={parts={},previous=root.Position,phase=0,blend=0}
352         for _,name in ipairs({'LegL','LegR','ArmL','ArmR','Trainer-1','Trainer1','Sole-1','Sole1'}) do
353             local p=bot:FindFirstChild(name); if p then
    ↪ data.parts[name]={part=p,rest=root.CFrame:ToObjectSpace(p.CFrame)} end
354         end
355         poses[bot]=data
356     end
357     local distance=(root.Position-data.previous).Magnitude; data.previous=root.Position
358     local speed=distance/math.max(dt,.001)
359     if distance>8 then speed=0; distance=0 end -- regrouping is not a walking step
360     data.phase+=distance*math.pi/3.2
361     local moving=speed>.3 and 1 or 0
362     data.blend+=(moving-data.blend)*(1-math.exp(-dt*16))
363     local swing=math.sin(data.phase)*.42*data.blend
364     for name,entry in pairs(data.parts) do if entry.part.Parent then
365         local arm=name:sub(1,3)=='Arm'
366         local left=name=='LegL' or name=='ArmL' or name:find('-1',1,true)
367         local sign=left and -1 or 1
368         local pivot=arm and Vector3.new(sign*1.18,.85,0) or Vector3.new(sign*.42,-1.05,0)
369         local angle=arm and -swing*sign*.65 or swing*sign
370         if arm and bot:GetAttribute('HasSeedling') then angle=-.4 end
371         entry.part.CFrame=root.CFrame*CFrame.new(pivot)*CFrame.Angles(angle,0,0)*CFrame.new(-pivot)*entry.rest
372     end end
373     local gun=bot:FindFirstChild('SoloWeapon'); local arm=bot:FindFirstChild('ArmR')
374     if gun and arm then gun.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0) end
375 end
376
377 return N
378 ]====])
379 install(game.ReplicatedStorage,"GWAPRouting","ModuleScript",[====[
380 local R={}
381 local function floorOf(item)
382     local n=item:GetAttribute('Floor'); if n then return n end
383     local label=item:GetAttribute('Level') or ''; return label=='Basement' and -1 or
    ↪ (tonumber(label:match('%d+')) or 1)-1
384 end
385 R.floorOf=floorOf
386 function R.healthy(item)
387     return item and item:IsDescendantOf(workspace) and not item:GetAttribute('Exploded') and not
    ↪ item:GetAttribute('BlastRemoved') and item:GetAttribute('FunctionalState')~='DESTROYED'
388 end
389 function R.ensureBackups()
390     local stations=workspace.WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch'):WaitForChild('Sta
    ↪ tions')
391     local originals=stations:GetChildren(); local perFloor={}

```

```

392     table.sort(originals,function(a,b)return a.Name<b.Name end)
393     for _,item in ipairs(originals) do if item:IsA('BasePart') and not item:GetAttribute('Backup') then
394         local capability=item:GetAttribute('Planted')~=nil and 'Greenhouse planting' or item.Name
395         item:SetAttribute('Capability',capability)
396         if capability=='Greenhouse planting' then for variant=1,2 do
397             local name=item.Name..' - Backup '..variant
398             if not stations:FindFirstChild(name) then
399                 local copy=item:Clone(); copy.Name=name; copy:SetAttribute('Backup',true);
400                 ↪ copy:SetAttribute('Capability',capability)
401                 local floor=floorOf(item); if variant==2 then floor=(floor+2)%4-1 end
402                 local z=item.Position.Z; local x=89
403                 if variant==2 then perFloor[floor]=(perFloor[floor] or 0)+1; z=150+perFloor[floor]*14; x=67 end
404                 local pos=Vector3.new(x,floor*16.25+1.5,z); local shift=pos-copy.Position
405                 copy.Position=pos; for _,p in ipairs(copy:GetDescendants()) do if p:IsA('BasePart') then
406                     ↪ p.Position+=shift end end
407                 copy:SetAttribute('Floor',floor);
408                 ↪ copy:SetAttribute('Approach',Vector3.new(78,floor*16.25+3.25,z))
409                 copy:SetAttribute('Exploded',false); copy:SetAttribute('FunctionalState','RUNNING')
410                 local sign=copy:FindFirstChild('StationLabel'); local label=sign and
411                 ↪ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=capability..' | Backup |
412                 ↪ '..(floor==1 and 'B1' or 'L'..(floor+1)) end
413                 copy.Parent=stations
414             end
415         end end
416     end end
417     return stations
418 end
419 function R.choose(stations,capability,preferredFloor,position)
420     local best,rank
421     for _,item in ipairs(stations:GetChildren()) do if item:GetAttribute('Capability')==capability and
422         ↪ R.healthy(item) and ((item:GetAttribute('Planted') or 0)<12) then
423         local score=(floorOf(item)==preferredFloor and 0 or
424         ↪ 100000)+math.abs(floorOf(item)-preferredFloor)*1000+(item.Position-position).Magnitude
425         if not rank or score<rank then best,rank=item,score end
426     end end
427     return best
428 end
429 return R
430 ]====])
431 install(game.ServerScriptService,"GWAPServer","Script",[====[
432 -- Authoritative assignments, research carriers, solo opposition and score accounting.
433 local Players=game:GetService('Players')
434 local RS=game:GetService('ReplicatedStorage')
435 local CS=game:GetService('CollectionService')
436 local SSS=game:GetService('ServerScriptService')
437 local Debris=game:GetService('Debris')
438 local Run=game:GetService('RunService')
439 local NPC=require(RS:WaitForChild('GWAPNPC'))
440 local Pathfinding=game:GetService('PathfindingService')
441 local pool=Instance.new('Folder'); pool.Name='GWAPCarrierPool'; pool.Parent=game:GetService('ServerStorage')
442 local companions={}
443 local World=require(RS:WaitForChild('GWAPWorld'))
444 local scenarios=require(RS:WaitForChild('GWAPScenarios'))
445 local research=World.build()
446 local Router=require(RS:WaitForChild('GWAPRouting'))
447 local stations=Router.ensureBackups()
448 local stationInfo={}
449 for _,item in ipairs(stations:GetChildren()) do
450     ↪ stationInfo[item.Name]={capability=item:GetAttribute('Capability'),floor=Router.floorOf(item)} end
451 local remote=Instance.new('RemoteEvent'); remote.Name='GWAPAction'; remote.Parent=RS
452 local bridge=Instance.new('BindableFunction'); bridge.Name='GWAPHitBridge'; bridge.Parent=SSS
453 local botsFolder=Instance.new('Folder'); botsFolder.Name='Carriers'; botsFolder.Parent=research
454 local routesFolder=Instance.new('Folder'); routesFolder.Name='Routes'; routesFolder.Parent=research
455 local random=Random.new()
456 local carriers, assignments, cooldowns, armed={}, {}, {}, {}
457 local function alive(p)
458     local c=p.Character; local h=c and c:FindFirstChildOfClass('Humanoid'); return h and h.Health>0 and
459     ↪ c:FindFirstChild('HumanoidRootPart')
460 end
461 local function points(p)
462     local s=p:FindFirstChild('leaderstats'); return s and s:FindFirstChild('Points')
463 end
464 local function award(p,n,reason)
465     local v=points(p); if not v then return end
466     v.Value=math.max(0,v.Value+n); p:SetAttribute('GWAPNotice',reason); p:SetAttribute('LastPointAmount',n);
467     ↪ p:SetAttribute('LastPointAward',reason)

```

```

458 end
459 local function role(p) return p and (p:GetAttribute('LabRole') or (p.Team and p.Team.Name)) end
460 local function active(a) return a and a.bot and a.bot.Parent and a.bot:GetAttribute('State')=='RUNNING' end
461 local function nearby(p,bot,d) local r=alive(p); return r and (r.Position-bot:GetPivot().Position).Magnitude<=d
↪ end
462 local function publish(a)
463     for p,bound in pairs(assignments) do if bound==a then
464         p:SetAttribute('GWAPStage',a.stage); p:SetAttribute('GWAPTotalStages',#a.route)
465         p:SetAttribute('GWAPTarget',a.target and a.target.Name or a.route[a.stage] or 'Complete');
↪         p:SetAttribute('GWAPTargetFloor',a.target and Router.floorOf(a.target) or nil);
↪         p:SetAttribute('GWAPState',a.bot:GetAttribute('State'))
466     end end
467 end
468 local function tell(a,message)
469     NPC.say(a.bot,message)
470     for p,bound in pairs(assignments) do if bound==a then p:SetAttribute('GWAPDialogue',message);
↪         p:SetAttribute('GWAPSpeaker',a.bot:GetAttribute('DisplayName') or 'Research carrier') end end
471 end
472 local function finish(a,state)
473     if not active(a) then return end
474     a.bot:SetAttribute('State',state); a.bot:SetAttribute('TaskState',state);
↪     a.bot:SetAttribute('HasSeedling',false)
475     local carried=a.bot:FindFirstChild('Mission seedling'); if carried then carried:Destroy() end
476     publish(a); a.finishedAt=os.clock(); tell(a,state=='COMPLETE' and 'The plant is in the greenhouse. Research
↪     assignment complete. Thank you!' or 'Mission stopped: '..state..' Choose another assignment when
↪     ready.')
477 end
478 local function updateHealth(bot)
479     local t=bot:FindFirstChild('HealthText',true); local fill=bot:FindFirstChild('HealthBarFill',true)
480     local h=bot:GetAttribute('Health') or 100
481     local status=bot:FindFirstChild('Status',true)
482     if status and status:IsA('TextLabel') then status.Text='RESEARCH CARRIER\n'..(bot:GetAttribute('Title') or
↪     'Choose an assignment') end
483     if t then t.Text='LIFE '..math.ceil(h)..' / 100' end
484     if fill then fill.Size=UDim2.fromScale(h/100,1) end
485 end
486 local function damage(p,bot,amount)
487     local a=carriers[bot]; if not active(a) then return end
488     local h=math.max(0,bot:GetAttribute('Health')-amount); bot:SetAttribute('Health',h); updateHealth(bot)
489     if role(p)=='Technician' and assignments[p]==a then a.disqualified[p]=true;
↪     p:SetAttribute('GWAPNotice','Protect your carrier: friendly hits cancel escort rewards.') end
490     if h==0 then
491         if assignments[p]==a and role(p)=='Laboratory Technician' then award(p,50,'+50 | Assigned research
↪         carrier stopped') end
492         finish(a,'INTERCEPTED')
493     end
494 end
495
496 local equipmentOriginal={}
497 for _,item in ipairs(stations:GetChildren()) do
498     if item:IsA('BasePart') then
499         local saved={}
500         local function remember(v) if v:IsA('BasePart') then
↪             saved[v]={v.Color,v.Material,v.Transparency,v.CanQuery} end end
501         remember(item); for _,v in ipairs(item:GetDescendants()) do remember(v) end
502         equipmentOriginal[item]=saved
503         item:SetAttribute('GWAPEquipment',true)
504     end
505 end
506 local function repair(item)
507     item:SetAttribute('Exploded',false); item:SetAttribute('FunctionalState','RUNNING')
508     for part,props in pairs(equipmentOriginal[item] or {}) do if part.Parent then part.Color=props[1];
↪     part.Material=props[2]; part.Transparency=props[3]; part.CanQuery=props[4] end end
509     for _,v in ipairs(item:GetChildren()) do if v.Name=='GWAPFire' or v.Name=='GWAPSmoke' then v:Destroy() end
↪     end
510     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=false end
511 end
512 local explode
513 explode=function(item,attacker)
514     if not equipmentOriginal[item] or item:GetAttribute('Exploded') then return end
515     item:SetAttribute('Exploded',true); item:SetAttribute('FunctionalState','DESTROYED')
516     for part in pairs(equipmentOriginal[item]) do if part.Parent then part.Color=Color3.fromRGB(49,42,38);
↪     part.Material=Enum.Material.CorrodedMetal end end
517     local fire=Instance.new('Fire'); fire.Name='GWAPFire'; fire.Size=5; fire.Heat=4; fire.Parent=item;
↪     Debris:AddItem(fire,8)

```

```

518     local smoke=Instance.new('Smoke'); smoke.Name='GWAPSmoke'; smoke.Size=4; smoke.Opacity=.3;
    ↪ smoke.Parent=item; Debris:AddItem(smoke,12)
519     local blast=Instance.new('Explosion'); blast.Position=item.Position; blast.BlastRadius=13;
    ↪ blast.BlastPressure=0; blast.DestroyJointRadiusPercent=0;
    ↪ blast.ExplosionType=Enum.ExplosionType.NoCraters
520     if attacker then blast:SetAttribute('GWAPAttacker',attacker.UserId) end
521     blast.Parent=workspace
522     for _,a in pairs(carriers) do if active(a) then
523         if (a.bot:GetPivot().Position-item.Position).Magnitude<13 then damage(attacker,a.bot,60) end
524         ↪ if a.target==item then tell(a,item.Name..' was destroyed. I will find a working equivalent.');
```

end end

```

525     end end
526     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=true end
527 end
528 for item in pairs(equipmentOriginal) do
529     local prompt=Instance.new('ProximityPrompt'); prompt.Name='ResearchRepair'; prompt.ActionText='Repair
    ↪ research equipment'; prompt.ObjectText=item.Name; prompt.KeyboardKeyCode=Enum.KeyCode.R;
    ↪ prompt.HoldDuration=3; prompt.MaxActivationDistance=12; prompt.Enabled=false; prompt.Parent=item
530     prompt.Triggered:Connect(function(p) local r=alive(p); if r and (r.Position-item.Position).Magnitude<14 and
    ↪ item:GetAttribute('Exploded') then repair(item); p:SetAttribute('GWAPNotice',item.Name..' repaired.
    ↪ Ready for research.') end end)
531 end
532 workspace.DescendantAdded:Connect(function(v)
533     if not v:IsA('Explosion') then return end
534     local center,radius=v.Position,v.BlastRadius
535     local userId=v:GetAttribute('GWAPAttacker'); local attacker=userId and Players:GetPlayerByUserId(userId)
536     -- Deferred bounded propagation: each station can explode only once until repaired.
537     task.delay(.12,function()
538         for item in pairs(equipmentOriginal) do if item.Parent and not item:GetAttribute('Exploded') and
    ↪ (item.Position-center).Magnitude<=radius+item.Size.Magnitude*.25 then
539             local delta=item.Position-center
540             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ params.FilterDescendantsInstances={v}
541             local hit=delta.Magnitude>.1 and workspace:Raycast(center,delta,params)
542             if not hit or hit.Instance==item or hit.Instance:IsDescendantOf(item) or
    ↪ hit.Instance:GetAttribute('Exploded') then explode(item,attacker) end
543         end end
544     end)
545 end)
546
547 bridge.OnInvoke=function(p,hit,damageAmount)
548     local node=hit
549     while node and node~=workspace do if equipmentOriginal[node] then explode(node,p); return true end; if
    ↪ carriers[node] then damage(p,node,damageAmount); return true end; node=node.Parent end
550     return false
551 end
552 local function routePoints(from,to)
553     local r={}; local function add(v) table.insert(r,v) end
554     local f1=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
555     local f2=to:GetAttribute('Floor'); local dest=to:GetAttribute('Approach')
556     if f1~=f2 then
557         add(Vector3.new(78,from.Y,from.Z)); add(Vector3.new(78,from.Y,-6))
558         local function flight(level,reverse)
559             local steps={Vector3.new(68.7857,level*16.25+3.25,-6)}
560             for i=1,8 do
    ↪ table.insert(steps,Vector3.new(68.7857,level*16.25+i*1.015625+3.25,-15.267857-(i-1)*4.821428))
    ↪ end
561             table.insert(steps,Vector3.new(68.7857,level*16.25+8.125+3.25,-55.28))
562             table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+3.25,-55.28))
563             for i=1,8 do table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+i*1.015625+3.25,-49.017857+(
    ↪ i-1)*4.821428)) end
564             table.insert(steps,Vector3.new(88.0714,(level+1)*16.25+3.25,-6));
    ↪ table.insert(steps,Vector3.new(78,(level+1)*16.25+3.25,-6))
565             if reverse then for i=#steps,1,-1 do add(steps[i]) end else for _,v in ipairs(steps) do add(v) end
    ↪ end
566         end
567         if f2>f1 then for level=f1,f2-1 do flight(level,false) end else for level=f1-1,f2,-1 do
    ↪ flight(level,true) end end
568     end
569     add(Vector3.new(78,dest.Y,dest.Z)); add(dest); return r
570 end
571 local function showRoute(a,route)
572     local old=routesFolder:FindFirstChild(a.bot.Name); if old then old:Destroy() end
573     local folder=Instance.new('Folder'); folder.Name=a.bot.Name; folder.Parent=routesFolder
574     for i,v in ipairs(route) do local n=Instance.new('Vector3Value'); n.Name=string.format('%03d',i);
    ↪ n.Value=v; n.Parent=folder end
575 end

```



```

576 local function go(a,target)
577     local revision=a.routeRevision or 0
578     if not Router.healthy(target) then return false end
579     local route
580     if a.needsRendezvous then
581         local from=a.bot:GetPivot().Position
582         local floor=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
583         local entry=Vector3.new(78,floor*16.25+3.25,math.clamp(from.Z,24,240))
584         route={}
585         if (from-entry).Magnitude>.5 then
586             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
587             ↪ params.FilterDescendantsInstances={a.bot,a.owner and a.owner.Character};
588             ↪ params.RespectCanCollide=true
589             local blocked=workspace:Blockcast(CFrame.new(from),Vector3.new(2.5,4.4,2.5),entry-from,params)
590             if not blocked and math.abs(from.Y-entry.Y)<1 then table.insert(route,entry) else
591                 local path=Pathfinding:CreatePath({AgentRadius=1.8,AgentHeight=5,AgentCanJump=false,WaypointSp
592                 ↪ acing=5})
593                 local ok=pcall(function() path:ComputeAsync(from,entry) end)
594                 if not ok or path.Status~=Enum.PathStatus.Success then tell(a,'I cannot reach the corridor from
595                 ↪ here. Move to an open area and ask me to regroup.');.paused=true; return false end
596                 for _,waypoint in ipairs(path:GetWaypoints()) do
597                     ↪ table.insert(route,waypoint.Position+Vector3.new(0,3.25,0)) end
598             end
599         end
600         for _,v in ipairs(routePoints(entry,target)) do table.insert(route,v) end
601         a.needsRendezvous=false
602     else route=routePoints(a.bot:GetPivot().Position,target) end
603     showRoute(a,route)
604     for i,goal in ipairs(route) do
605         a.bot:SetAttribute('RouteIndex',i)
606         if not Router.healthy(target) then a.needsRendezvous=true; return false end
607         while active(a) and (goal-a.bot:GetPivot().Position).Magnitude>.12 do
608             local dt=Run.Heartbeat:Wait(); if not active(a) then return false end
609             if (a.routeRevision or 0)~=revision then return false end
610             if not Router.healthy(target) then a.needsRendezvous=true; return false end
611             if a.paused then a.deadline+=dt; continue end
612             local ownerRoot=a.owner and alive(a.owner)
613             if a.owner and role(a.owner)=='Technician' and (not ownerRoot or
614             ↪ (ownerRoot.Position-a.bot:GetPivot().Position).Magnitude>45) then
615                 a.deadline+=dt
616                 if os.clock()-(a.lastWaitMessage or 0)>12 then a.lastWaitMessage=os.clock(); tell(a,'I am
617                 ↪ waiting for you. Follow the mint trail to my marker, then we can continue.') end
618                 continue
619             end
620             local start=a.bot:GetPivot().Position; local delta=goal-start
621             local ownerHum=a.owner and a.owner.Character and
622             ↪ a.owner.Character:FindFirstChildOfClass('Humanoid')
623             local desiredSpeed=ownerHum and ownerHum.WalkSpeed or 16
624             a.speed=(a.speed or desiredSpeed)+(desiredSpeed-(a.speed or desiredSpeed))*(1-math.exp(-dt*8))
625             a.bot:SetAttribute('WalkSpeed',a.speed)
626             local nextPos=start+delta.Unit*math.min(delta.Magnitude,math.clamp(a.speed,8,40)*math.min(dt,.1))
627             -- Research routes use measured open corridors and each actual stair tread.
628             -- Stop for players; do not shove them or change their movement settings.
629             local blocked=false
630             for _,p in ipairs(Players:GetPlayers()) do local r=alive(p); if r and
631             ↪ (r.Position-nextPos).Magnitude<2.8 then blocked=true end end
632             if not blocked then
633                 local flat=Vector3.new(delta.X,0,delta.Z)
634                 local previous=a.bot:GetPivot()
635                 local desired=flat.Magnitude>.01 and CFrame.lookAt(nextPos,nextPos+flat) or
636                 ↪ CFrame.new(nextPos)*previous.Rotation
637                 a.bot:PivotTo(CFrame.new(nextPos)*previous.Rotation:Lerp(desired.Rotation,1-math.exp(-dt*10)))
638             end
639             if os.clock()>a.deadline then finish(a,'EXPIRED'); return false end
640         end
641         if not active(a) then return false end
642     end
643     return true
644 end
645 local function plantIn(bed,name)
646     if bed:GetAttribute('Exploded') then return false end
647     local count=bed:GetAttribute('Planted') or 0
648     if count>=12 then return false end
649     local pos=bed.Position+Vector3.new(-8+(count%4)*5,1,-3+math.floor(count/4)*3)
650     World.plant(bed,pos,name); bed:SetAttribute('Planted',count+1)
651     local sign=bed:FindFirstChild('StationLabel'); local label=sign and sign:FindFirstChildOfClass('TextLabel')

```

```

642     if label then label.Text=bed.Name..' | '..(count+1).. / 12 plants' end
643     return true
644 end
645 local function checkpoint(a,final)
646     for p,bound in pairs(assignments) do
647         if bound==a and role(p)=='Technician' and not a.disqualified[p] and nearby(p,a.bot,45) then
648             ↳ award(p,final and 60 or 15,final and '+60 | Protected rooftop planting' or '+15 | Protected
649             ↳ research checkpoint') end
650     end
651 end
652 local function runMission(a)
653     for i,originalName in ipairs(a.route) do
654         a.stage=i
655         local original=stations:FindFirstChild(originalName)
656         local info=stationInfo[originalName]
657         local capability=info and info.capability or original and original:GetAttribute('Capability') or
658         ↳ (originalName:match('^Plot ') and 'Greenhouse planting' or originalName)
659         local preferredFloor=info and info.floor or original and Router.floorOf(original) or 0
660         local completed=false
661         while active(a) and not completed do
662             local target=Router.healthy(a.target) and a.target:GetAttribute('Capability')==capability and
663             ↳ a.target
664             or (Router.healthy(original) and original or
665             ↳ Router.choose(stations,capability,preferredFloor,a.bot:GetPivot().Position))
666         if not target then
667             a.target=nil; publish(a)
668             if os.clock()-(a.lastNoEquipment or 0)>8 then a.lastNoEquipment=os.clock(); tell(a,'No working
669             ↳ '..capability..' is available. Repair one with R; I will resume this assignment.') end
670             a.deadline+=1; task.wait(1); continue
671         end
672         if target~=a.target then
673             a.target=target; publish(a)
674             if target~=original then a.bot:SetAttribute('Reroutes',(a.bot:GetAttribute('Reroutes') or 0)+1)
675             ↳ end
676             tell(a,target~=original and ('Rerouting to '..target.Name..' on '..(Router.floorOf(target)==-1
677             ↳ and 'B1' or Router.floorOf(target)==8 and 'the roof' or
678             ↳ 'L'..(Router.floorOf(target)+1))..' Same research function, new route.')
679             or 'Next stop: '..target.Name..' Stay with me.')
680         end
681         a.bot:SetAttribute('TaskState','Travel to '..target.Name)
682         local reached=go(a,target)
683         if not active(a) then return end
684         if not reached then
685             a.needsRendezvous=true
686             if a.paused then repeat task.wait(.25) until not active(a) or not a.paused end
687             task.wait(.1); continue
688         end
689         tell(a,'At '..target.Name..' Processing the seedling now.')
690         a.bot:SetAttribute('TaskState','Processing: '..target.Name)
691         local seed=a.bot:FindFirstChild('Mission seedling')
692         if seed then seed:PivotTo(CFrame.new(target.Position+Vector3.new(0,2,0))) end
693         local work=0
694         while active(a) and Router.healthy(target) and work<4 do task.wait(.1); if not a.paused then
695             ↳ work+=.1 end end
696         if not active(a) then return end
697         if not Router.healthy(target) then a.needsRendezvous=true; continue end
698         if i==#a.route then
699             if plantIn(target,a.title..' | '..a.variant) then checkpoint(a,true); finish(a,'COMPLETE');
700             ↳ return end
701             a.target=nil; original=nil; task.wait(.1); continue
702         end
703         if i==1 and not seed then a.bot:SetAttribute('HasSeedling',true);
704         ↳ seed=World.plant(a.bot,a.bot:GetPivot().Position+Vector3.new(0,0,-2),'Mission seedling') end
705         if seed then seed:PivotTo(CFrame.new(a.bot:GetPivot().Position+Vector3.new(0,0,-2))) end
706         checkpoint(a,false); completed=true; a.target=nil
707     end
708     if not active(a) then return end
709 end
710 local function startMission(bot,id,p)
711     local s=scenarios[id]; local route=table.clone(s.route); table.insert(route,'Plot '..id)
712     local a={bot=bot,id=id,title=s.title,route=route,stage=1,disqualified={},owner=p,needsRendezvous=true,paus
713     ↳ ed=true,deadline=os.clock()+600}
714     a.variant=s.variable..'': '..random:NextInteger(s.low,s.high)..' '..s.units

```

```

703     carriers[bot]=a; bot:SetAttribute('State','RUNNING'); bot:SetAttribute('Health',100);
    ↪ bot:SetAttribute('Scenario',id); bot:SetAttribute('Title',s.title);
    ↪ bot:SetAttribute('Variant',a.variant); updateHealth(bot)
704     bot.Parent=botsFolder
705     if not NPC.nearPlayer(p,bot) then bot.Parent=pool; carriers[bot]=nil; return nil end
706     return a
707 end
708 local function makeBot(index)
709     local template
710     local choices={}
711     for _,b in ipairs(CS:GetTagged('LabWorkerBot')) do if not b:GetAttribute('Seated') and b.PrimaryPart and
    ↪ ((index%2==1)==(b:GetAttribute('Gender')=='Girl')) then table.insert(choices,b) end end
712     table.sort(choices,function(a,b) return a.Name<b.Name end);
    ↪ template=choices[(index-1)%math.max(1,#choices)+1]
713     assert(template,'No worker template available')
714     local b=template:Clone(); b.Name='ResearchCarrier_'..index
715     for _,v in ipairs(b:GetDescendants()) do if v:IsA('LuaSourceContainer') or v:IsA('ProximityPrompt') or
    ↪ v:IsA('Humanoid') then v:Destroy() elseif v:IsA('BasePart') then v.Anchored=true; v.CanCollide=false
    ↪ end end
716     for _,tag in ipairs(CS:GetTags(b)) do CS:RemoveTag(b,tag) end
717     NPC.appearance(b)
718     b:SetAttribute('Dead',false); b:SetAttribute('KnockedOut',false); b:SetAttribute('GWAPCarrier',true);
    ↪ b.Parent=botsFolder
719     local h=Instance.new('Highlight'); h.Name='ResearchOutline'; h.FillTransparency=1;
    ↪ h.OutlineColor=Color3.fromRGB(92,244,199); h.DepthMode=Enum.HighlightDepthMode.Occluded; h.Parent=b
720     return b
721 end
722 Run.Heartbeat:Connect(function(dt) for _,bot in ipairs(botsFolder:GetChildren()) do if bot:IsA('Model') then
    ↪ NPC.animate(bot,dt) end end end)
723 local available={}; for i=1,5 do available[i]=makeBot(i); available[i].Parent=pool;
    ↪ available[i]:SetAttribute('State','IDLE') end
724 local function companion(p)
725     local bot=companions[p]
726     if bot and bot.Parent then return bot end
727     local reserved={}; for _,b in pairs(companions) do reserved[b]=true end
728     for _,candidate in ipairs(available) do if not reserved[candidate] and not active(carriers[candidate]) then
    ↪ bot=candidate; break end end
729     if not bot then bot=makeBot(#available+1); table.insert(available,bot) end
730     companions[p]=bot; bot.Parent=botsFolder
731     if not NPC.nearPlayer(p,bot) then bot.Parent=pool; p:SetAttribute('GWAPDialogue','Move to an open spot and
    ↪ choose an assignment so I can meet you.');
```

```

755     elseif index==3 then tell(a,'We are heading to '..(a.target and a.target.Name or a.route[a.stage])..'
    ↪ '..scenarios[a.id].purpose)
756     elseif index==4 then
757         if not a.paused then tell(a,'Ask me to wait before regrouping.');
```

~~return end~~

```

758         if NPC.nearPlayer(p,a.bot) then a.needsRendezvous=true; a.routeRevision=(a.routeRevision or 0)+1;
    ↪ tell(a,'I am beside you again. Press Ready to continue.') end
759     end
760 end
761 end)
762 for _,bed in ipairs(stations:GetChildren()) do
763     local prompt=bed:FindFirstChild('PlantSeedling')
764     if prompt then prompt.Triggered:Connect(function(p)
765         local r=alive(p); if not r or (r.Position-bed.Position).Magnitude>14 then return end
766         local sample
767         for _,holder in ipairs({p.Character,p:FindFirstChildOfClass('Backpack')}) do if holder then for _,t in
    ↪ ipairs(holder:GetChildren()) do if t:IsA('Tool') and (t:GetAttribute('PlantSample') or
    ↪ t:GetAttribute('TransferSample')) then sample=t; break end end end end
768         if not sample then p:SetAttribute('GWAPNotice','Carry a germinated plant sample to this plot.');
```

~~return~~

```

    ↪ end
769         if plantIn(bed,sample.Name) then sample:Destroy(); p:SetAttribute('CarriedSample','');
    ↪ p:SetAttribute('GWAPNotice','Seedling planted in '..bed.Name) else
    ↪ p:SetAttribute('GWAPNotice','This plot is full. Use another plot.') end
770     end) end
771 end
772 local function setArmed()
773     local all=CS:GetTagged('LabWorkerBot'); for _,bot in ipairs(available) do table.insert(all,bot) end
774     table.sort(all,function(a,b) return a.Name<b.Name end)
775     armed={}; local solo=#Players:GetPlayers()==1; local count=solo and math.floor(#all*.2) or 0
776     local selected={}; local shuffle=table.clone(all); for i=#shuffle,2,-1 do local j=random:NextInteger(1,i);
    ↪ shuffle[i],shuffle[j]=shuffle[j],shuffle[i] end
777     for i=1,count do selected[shuffle[i]]=true end
778     for _,bot in ipairs(all) do
779         local yes=selected[bot]==true; bot:SetAttribute('GWAPArmed',yes)
780         local gun=bot:FindFirstChild('SoloWeapon'); if gun then gun:Destroy() end
781         if yes then
782             table.insert(armed,bot); NPC.gun(bot,(bot:GetAttribute('BotNumber') or 1))
783         end
784     end
785     workspace:SetAttribute('GWAPArmedCount',count); workspace:SetAttribute('GWAPBotCount',#all)
786 end
787 local retired={['Research SMG']=true,['Precision Rifle']=true,['Scatter Blaster']=true,['Blast
    ↪ Launcher']=true,['Pulse Bomb']=true}
788 local function watchPlayer(p)
789     local function clean(v) if v:IsA('Tool') and (v:GetAttribute('GWAPWeapon') or retired[v.Name]) then
    ↪ v:Destroy() end end
790     local function watch(container) for _,v in ipairs(container:GetChildren()) do clean(v) end;
    ↪ container.ChildAdded:Connect(clean) end
791     task.spawn(function() watch(p:WaitForChild('Backpack')) end)
792     local function onCharacter(c)
793         watch(c)
794         task.delay(1,function() if role(p)=='Technician' or role(p)=='Laboratory Technician' then
795             local a=assignments[p]; if active(a) then a.paused=true; tell(a,'You are back. Ask me to regroup,
    ↪ then press Ready.') else companion(p) end
796         end end)
797     end
798     if p.Character then onCharacter(p.Character) end; p.CharacterAdded:Connect(onCharacter)
799     p:GetAttributeChangedSignal('LabRole'):Connect(function() if role(p)=='Technician' or role(p)=='Laboratory
    ↪ Technician' then companion(p) end end)
800     task.defer(setArmed)
801 end
802 Players.PlayerAdded:Connect(watchPlayer)
803 for _,p in ipairs(Players:GetPlayers()) do watchPlayer(p) end
804 for _,bot in ipairs(CS:GetTagged('LabWorkerBot')) do NPC.appearance(bot) end
805 Players.PlayerRemoving:Connect(function(p) local a=assignments[p]; if active(a) then finish(a,'PLAYER LEFT')
    ↪ end; local bot=companions[p]; if bot then bot.Parent=pool end; companions[p]=nil; assignments[p]=nil;
    ↪ cooldowns[p]=nil; task.defer(setArmed) end)
806 setArmed()
807 task.spawn(function()
808     while task.wait(2.5) do
809         local list=Players:GetPlayers(); if #list~=1 or workspace:GetAttribute('RoundPhase')=='BREAK' then
    ↪ continue end
810         local p=list[1]; local root=alive(p); if not root then continue end
811         for _,bot in ipairs(armed) do
812             if not bot:IsDescendantOf(workspace) or bot:GetAttribute('Dead') or bot:GetAttribute('KnockedOut')
    ↪ or (bot:GetAttribute('Health') or 100)<=0 then continue end
813             -- Assigned escort carriers never attack their own defender.
```

```

814     if assignments[p] and assignments[p].bot==bot and role(p)=='Technician' then continue end
815     local gun=bot:FindFirstChild('SoloWeapon'); local origin=gun and
        ↪ (gun.Position+gun.CFrame.LookVector*(-gun.Size.Z*.5)) or
        ↪ bot:GetPivot().Position+Vector3.new(0,1,0); local delta=root.Position-origin
816     if delta.Magnitude<4 or delta.Magnitude>85 then continue end
817     local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
        ↪ params.FilterDescendantsInstances={bot}
818     local hit=workspace:Raycast(origin,delta,params)
819     if hit and hit.Instance:IsDescendantOf(p.Character) then
820         local spread=Vector3.new(random:NextNumber(-2.5,2.5),random:NextNumber(-1.5,1.5),random:NextNu
            ↪ mber(-2.5,2.5))
821         local shot=workspace:Raycast(origin,delta+spread,params); local endpoint=shot and shot.Position
            ↪ or origin+delta+spread
822         local beam=Instance.new('Part'); beam.Name='NPC shot'; beam.Anchored=true;
            ↪ beam.CanCollide=false; beam.CanQuery=false; beam.Material=Enum.Material.Neon;
            ↪ beam.Color=Color3.fromRGB(255,105,72);
            ↪ beam.Size=Vector3.new(.07,.07,(endpoint-origin).Magnitude);
            ↪ beam.CFrame=CFrame.lookAt((origin+endpoint)/2,endpoint); beam.Parent=workspace;
            ↪ Debris:AddItem(beam,.18)
823         if shot and shot.Instance:IsDescendantOf(p.Character) then
824             local v=points(p); if v then
825                 -- Existing score is an IntValue: accumulate fractional losses, never turn 1% into a
                    ↪ minimum one-point tax.
826                 local debt=(p:GetAttribute('GWAPFractionalLoss') or 0)+math.max(0,v.Value)*.01
827                 local loss=math.floor(debt+1e-8); p:SetAttribute('GWAPFractionalLoss',debt-loss);
                    ↪ v.Value=math.max(0,v.Value-loss)
828                 p:SetAttribute('GWAPNotice','NPC hit: -1% score');
                    ↪ p:SetAttribute('GWAPHitAt',workspace:GetServerTimeNow())
829             end
830         end
831     end
832 end
833 end
834 end)
835 workspace:GetAttributeChangedSignal('RoundNumber'):Connect(function()
836     for _,a in pairs(carriers) do if active(a) then finish(a,'SHIFT ENDED') end end
837     for _,p in ipairs(Players:GetPlayers()) do p:SetAttribute('GWAPFractionalLoss',0) end
838     for item in pairs(equipmentOriginal) do repair(item) end
839     for _,bed in ipairs(stations:GetChildren()) do if bed:GetAttribute('Planted') then for _,v in
        ↪ ipairs(bed:GetChildren()) do if v:IsA('Model') then v:Destroy() end end; bed:SetAttribute('Planted',0);
        ↪ local sign=bed:FindFirstChild('StationLabel'); local label=sign and
        ↪ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=bed.Name..' | Ready for planting' end
        ↪ end end
840     setArmed()
841 end)
842 print('[GWAP] 10 scenarios, 5 research carriers, rooftop planting and solo opposition ready')
843 ]====)
844 install(game.ServerScriptService,"GWAPWallDamage","Script",[====[
845 -- Remove fixtures belonging to a wall section destroyed by an explosion.
846 local CS=game:GetService('CollectionService')
847 local c=workspace:WaitForChild('Laboratory_Complex')
848 local debris=Instance.new('Folder'); debris.Name='DestroyedWallFixtures';
        ↪ debris.Parent=game:GetService('ServerStorage')
849 local fixtures={}; local recent={}
850 local function register(v)
851     if not v:IsA('BasePart') then return end
852     if v:GetAttribute('ToolKind') or v:GetAttribute('ShootablePoster') or v.Name:match('^HealthStation')
        or CS:HasTag(v,'LabEquipment') or CS:HasTag(v,'PlantGrowthChamber') or v:GetAttribute('GWAPEquipment')
853     or (v.Parent and v.Parent.Name=='Stations') then fixtures[v]=true end
854 end
855 for _,v in ipairs(c:GetDescendants()) do register(v) end
856 c.DescendantAdded:Connect(register)
857 CS:GetInstanceAddedSignal('LabEquipment'):Connect(register)
858 workspace.DescendantAdded:Connect(function(v)
859     if v:IsA('Explosion') then table.insert(recent,{position=v.Position,radius=v.BlastRadius,time=os.clock()})
        ↪ end
860     while #recent>0 and os.clock()-recent[1].time>3 do table.remove(recent,1) end
861 end)
862 c.DescendantRemoving:Connect(function(wall)
863     if not wall:IsA('BasePart') or not (wall.Name:match('^Corridor_') or wall.Name:match('^Div_') or
        ↪ wall.Name:match('^Ext_') or wall.Name:match('^B_')) then return end
864     local cf,size=wall.CFrame,wall.Size
865     local exploded=false
866     for _,blast in ipairs(recent) do if os.clock()-blast.time<3 then
867         local localPoint=cf:PointToObjectSpace(blast.position)

```

```

869     local closest=Vector3.new(math.clamp(localPoint.X,-size.X/2,size.X/2),math.clamp(localPoint.Y,-size.Y/
    ↪ 2,size.Y/2),math.clamp(localPoint.Z,-size.Z/2,size.Z/2))
870     if (localPoint-closest).Magnitude<=blast.radius then exploded=true; break end
871 end end
872 if not exploded then return end
873 task.defer(function()
874     if not c.Parent then return end
875     local pad=Vector3.new(size.X<4 and 7 or 1,1,size.Z<4 and 7 or 1)
876     local half=size/2+pad
877     local count=0
878     for item in pairs(fixtures) do if item:IsDescendantOf(c) then
879         local pos=cf.PointToObjectSpace(item.Position)
880         if math.abs(pos.X)<=half.X and math.abs(pos.Y)<=half.Y and math.abs(pos.Z)<=half.Z then
881             item:SetAttribute('BlastRemoved',true); item:SetAttribute('FunctionalState','DESTROYED');
    ↪ item:SetAttribute('Exploded',true)
882             for _,v in ipairs(item:GetDescendants()) do if v:IsA('ProximityPrompt') then v.Enabled=false
    ↪ end end
883             for _,tag in ipairs(CS:GetTags(item)) do CS:RemoveTag(item,tag) end
884             item.Parent=debris; count+=1
885         end
886     end end
887     workspace:SetAttribute('GWAPRemovedFixtures',(workspace:GetAttribute('GWAPRemovedFixtures') or
    ↪ 0)+count)
888 end)
889 end)
890 ]====])
891 install(game.StarterPlayer.StarterPlayerScripts,"LabHUD","LocalScript",[====[
892 -- Two grouped panels. Native inventory and movement/fire controls remain available.
893 local Players=game:GetService('Players')
894 local RS=game:GetService('ReplicatedStorage')
895 local Run=game:GetService('RunService')
896 local StarterGui=game:GetService('StarterGui')
897 local prompts=game:GetService('ProximityPromptService')
898 local p=Players.LocalPlayer; local pg=p:WaitForChild('PlayerGui')
899 local scenarios=require(RS:WaitForChild('GWAPScenarios')); local remote=RS:WaitForChild('GWAPAction')
900 local research=workspace:WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch')
901 local gui=Instance.new('ScreenGui'); gui.Name='GWAPHUD'; gui.ResetOnSpawn=false; gui.IgnoreGuiInset=false;
    ↪ gui.DisplayOrder=25; gui.Parent=pg
902 pcall(function() StarterGui:SetCoreGuiEnabled(Enum.CoreGuiType.Health,false) end)
903 local white=Color3.fromRGB(239,247,243)
904 local function box(name,parent)
905     local f=Instance.new('Frame'); f.Name=name; f.BackgroundColor3=Color3.fromRGB(23,31,37);
    ↪ f.BackgroundTransparency=.12; f.BorderSizePixel=0; f.Parent=parent or gui;
    ↪ Instance.new('UICorner',f).CornerRadius=UDim.new(0,8); return f
906 end
907 local function label(parent,name,size)
908     local t=Instance.new('TextLabel'); t.Name=name; t.BackgroundTransparency=1; t.Font=Enum.Font.GothamMedium;
    ↪ t.TextColor3=white; t.TextSize=size or 13; t.TextWrapped=true; t.TextTruncate=Enum.TextTruncate.AtEnd;
    ↪ t.TextXAlignment=Enum.TextXAlignment.Left; t.TextYAlignment=Enum.TextYAlignment.Center;
    ↪ t.Parent=parent; return t
909 end
910 local function button(parent,name,callback)
911     local t=Instance.new('TextButton'); t.Name=name; t.Text=name; t.Font=Enum.Font.GothamBold;
    ↪ t.TextColor3=white; t.TextSize=11; t.BackgroundColor3=Color3.fromRGB(63,73,91); t.BorderSizePixel=0;
    ↪ t.Parent=parent; Instance.new('UICorner',t).CornerRadius=UDim.new(0,5); t.Activated:Connect(callback);
    ↪ return t
912 end
913 local strip=box('Round and player'); strip.AnchorPoint=Vector2.new(.5,0)
914 local teams=label(strip,'Team scores',13); teams.TextXAlignment=Enum.TextXAlignment.Center
915 local timer=label(strip,'Timer',18); timer.TextXAlignment=Enum.TextXAlignment.Center
916 local personal=label(strip,'Player',12); personal.TextXAlignment=Enum.TextXAlignment.Right
917 local health=box('Health',strip); health.BackgroundColor3=Color3.fromRGB(54,66,61)
918 local fill=box('Fill',health); fill.BackgroundColor3=Color3.fromRGB(100,235,68); fill.BackgroundTransparency=0;
    ↪ fill.Size=UDim2.fromScale(1,1)
919 local panel=box('Mission and conversation')
920 local heading=label(panel,'Heading',13); heading.Font=Enum.Font.GothamBold
921 local target=label(panel,'Destination',12); target.TextColor3=Color3.fromRGB(151,238,204)
922 local dialogue=label(panel,'Carrier message',12); dialogue.TextYAlignment=Enum.TextYAlignment.Top
923 local note=label(panel,'Status',10); note.TextColor3=Color3.fromRGB(216,202,158);
    ↪ note.TextTruncate=Enum.TextTruncate.AtEnd; note.TextWrapped=false
924 local menu=box('Scenario selection'); menu.AnchorPoint=Vector2.new(.5,.5); menu.Visible=false; menu.ZIndex=10
925 local function toggle(value) menu.Visible=value; panel.Visible=not value; strip.Visible=not value;
    ↪ prompts.Enabled=not value end
926 local selectButton=button(panel,'Jobs',function() toggle(true) end)
927 local buttons={}

```

```

928 for i,name in ipairs({'Ready','Wait','Status','Regroup'}) do buttons[i]=button(panel,name,function()
    ↪ remote:FireServer('reply',i) end) end
929 local menuHeading=label(menu,'Select assignment',16); menuHeading.Text='CHOOSE A RESEARCH ASSIGNMENT';
    ↪ menuHeading.Position=UDim2.fromOffset(12,6); menuHeading.Size=UDim2.new(1,-58,0,28)
930 local close=button(menu,'X',function() toggle(false) end); close.Position=UDim2.new(1,-38,0,6);
    ↪ close.Size=UDim2.fromOffset(30,28)
931 local list=Instance.new('ScrollingFrame'); list.Name='Assignments'; list.Position=UDim2.fromOffset(10,42);
    ↪ list.Size=UDim2.new(1,-20,1,-52); list.BackgroundTransparency=1; list.BorderSizePixel=0;
    ↪ list.ScrollBarThickness=5; list.CanvasSize=UDim2.fromOffset(0,11*46); list.Parent=menu
932 local ids={}; for i=1,10 do ids[i]=i end; local random=Random.new(); for i=10,2,-1 do local
    ↪ j=random:NextInteger(1,i); ids[i],ids[j]=ids[j],ids[i] end
933 for row,id in ipairs(ids) do local b=button(list,scenarios[id].title,function() remote:FireServer('choose',id);
    ↪ toggle(false) end); b.Position=UDim2.fromOffset(0,(row-1)*46); b.Size=UDim2.new(1,-8,0,40); b.TextSize=13
    ↪ end
934 local randomButton=button(list,'Random assignment',function()
    ↪ remote:FireServer('choose',random:NextInteger(1,10)); toggle(false) end);
    ↪ randomButton.Position=UDim2.fromOffset(0,460); randomButton.Size=UDim2.new(1,-8,0,40)
935 local trail=Instance.new('Folder'); trail.Name='LocalMissionTrail'; trail.Parent=workspace
936 local dots={}
937 for i=1,90 do local dot=Instance.new('Part'); dot.Name='Direction'; dot.Size=Vector3.new(.16,.12,1.6);
    ↪ dot.Anchored=true; dot.CanCollide=false; dot.CanQuery=false; dot.CanTouch=false;
    ↪ dot.Material=Enum.Material.Neon; dot.Color=Color3.fromRGB(103,255,198); dot.Transparency=1;
    ↪ dot.Parent=trail; dots[i]=dot end
938 local function showTrail(bot)
939     for _,v in ipairs(dots) do v.Transparency=1 end
940     local route=bot and research.Routes:FindFirstChild(bot.Name); local root=p.Character and
        ↪ p.Character:FindFirstChild('HumanoidRootPart')
941     if not bot or not route or not root or bot:GetAttribute('State')~='RUNNING' then return end
942     local nodes=route:GetChildren(); table.sort(nodes,function(a,b) return a.Name<b.Name end)
943     local previous=bot:GetPivot().Position; local n=0
944     for j=bot:GetAttribute('RouteIndex') or 1,#nodes do
945         local to=nodes[j].Value; local delta=to-previous
946         if delta.Magnitude>.1 then for d=0,delta.Magnitude,4 do
947             local pos=previous+delta.Unit*d-Vector3.new(0,2.9,0)
948             if (pos-root.Position).Magnitude<110 and math.abs(pos.Y-root.Position.Y)<10 then n+=1; if n>#dots
                ↪ then return end; dots[n].CFrame=CFrame.lookAt(pos,pos+delta); dots[n].Transparency=.12 end
949         end end; previous=to
950     end
951 end
952 prompts.PromptShown:Connect(function(prompt)
953     local viewport=workspace.CurrentCamera.ViewportSize
954     prompt.UIOffset=Vector2.new(viewport.X<1000 and 70 or 0,0)
955 end)
956 local billboards=setmetatable({}, {__mode='k'})
957 local function watchBot(bot)
958     local items={}; for _,v in ipairs(bot:GetDescendants()) do if v:IsA('BillboardGui') then
        ↪ table.insert(items,v) end end; billboards[bot]=items
959     bot.DescendantAdded:Connect(function(v) if v:IsA('BillboardGui') then table.insert(items,v) end end)
960 end
961 local cs=game:GetService('CollectionService')
962 for _,bot in ipairs(cs:GetTagged('LabWorkerBot')) do watchBot(bot) end
963 for _,bot in ipairs(research.Carriers:GetChildren()) do watchBot(bot) end
964 research.Carriers.ChildAdded:Connect(watchBot)
965 local elapsed=0
966 Run.RenderStepped:Connect(function(dt)
967     elapsed+=dt; if elapsed<.15 then return end; elapsed=0
968     local v=workspace.CurrentCamera.ViewportSize; local mobile=v.X<1000 or v.Y<500
969     local width=math.min(mobile and 180 or 290,v.X*.25); local height=mobile and 150 or 218
970     local scale=math.min(1,(v.Y-95)/height); scale=math.max(.6,scale)
971     local touch=pg:FindFirstChild('TouchGui'); local thumb=touch and
        ↪ touch:FindFirstChild('ThumbstickStart',true)
972     if mobile and thumb and thumb.Visible then
        ↪ scale=math.min(scale,math.max(.6,(thumb.AbsolutePosition.Y-48-8)/height)) end
973     local ui=panel:FindFirstChildOfClass('UIScale') or Instance.new('UIScale',panel); ui.Scale=scale
974     panel.Position=UDim2.fromOffset(8,48); panel.Size=UDim2.fromOffset(width/scale,height)
975     local w=width/scale
976     heading.Position=UDim2.fromOffset(8,6); heading.Size=UDim2.fromOffset(w-59,mobile and 25 or 36);
        ↪ heading.TextSize=mobile and 11 or 15
977     selectButton.Position=UDim2.fromOffset(w-46,7); selectButton.Size=UDim2.fromOffset(38,24)
978     target.Position=UDim2.fromOffset(8,mobile and 33 or 46); target.Size=UDim2.fromOffset(w-16,mobile and 26 or
        ↪ 34); target.TextSize=mobile and 10 or 13
979     dialogue.Position=UDim2.fromOffset(8,mobile and 61 or 84); dialogue.Size=UDim2.fromOffset(w-16,mobile and
        ↪ 45 or 69); dialogue.TextSize=mobile and 10 or 13
980     for i,b in ipairs(buttons) do b.Position=UDim2.fromOffset(8+(i-1)*(w-16)/4,height-43);
        ↪ b.Size=UDim2.fromOffset((w-20)/4-2,23); b.TextSize=mobile and 9 or 11 end

```

```

981     note.Position=UDim2.fromOffset(8,height-18); note.Size=UDim2.fromOffset(w-16,14); note.TextSize=mobile and
    ↳ 9 or 11
982     local sw=math.min(460,v.X-26); strip.Position=UDim2.new(.5,0,0,3); strip.Size=UDim2.fromOffset(sw,38)
983     teams.Position=UDim2.fromOffset(7,3); teams.Size=UDim2.fromOffset(sw*.35,30); teams.TextSize=mobile and 10
    ↳ or 13
984     timer.Position=UDim2.fromOffset(sw*.38,0); timer.Size=UDim2.fromOffset(sw*.19,35)
985     personal.Position=UDim2.fromOffset(sw*.59,0); personal.Size=UDim2.fromOffset(sw*.39-8,23);
    ↳ personal.TextSize=mobile and 10 or 12
986     health.Position=UDim2.fromOffset(sw*.61,26); health.Size=UDim2.fromOffset(sw*.35,6)
987     menu.Position=UDim2.fromScale(.5,.5);
    ↳ menu.Size=UDim2.fromOffset(math.min(430,v.X-30),math.max(150,math.min(520,v.Y-80)))
988     local own=p.Character and p.Character:FindFirstChild('LabHealthBar',true); if own then own.Enabled=false
    ↳ end
989     for _,name in ipairs({'RoleHUD','LabLog'}) do local old=pg:FindFirstChild(name); if old then
    ↳ old.Enabled=false end end
990     local sums={Technician=0,['Laboratory Technician']=0}
991     for _,player in ipairs(Players:GetPlayers()) do local role=player:GetAttribute('LabRole'); local
    ↳ stats=player:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); if
    ↳ sums[role] then sums[role]+=value and value.Value or 0 end end
992     teams.Text='PROTECT '..sums.Technician..' | STOP '..sums['Laboratory Technician']
993     local remaining=math.max(0,(workspace:GetAttribute('RoundEndsAt') or os.time())-os.time());
    ↳ timer.Text=string.format('%d:%02d',remaining//60,remaining%60)
994     local stats=p:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); local
    ↳ hum=p.Character and p.Character:FindFirstChildOfClass('Humanoid')
995     personal.Text=(value and value.Value or 0)..' pts | '..math.ceil(hum and hum.Health or 0)..' HP';
    ↳ fill.Size=UDim2.fromScale(hum and math.clamp(hum.Health/math.max(1,hum.MaxHealth),0,1) or 0,1)
996     local id=p:GetAttribute('GWAPScenario'); local s=id and scenarios[id]; local
    ↳ name=p:GetAttribute('GWAPCarrier'); local bot=name and research.Carriers:FindFirstChild(name)
997     heading.Text=s and s.title or 'RESEARCH ASSIGNMENT'
998     local floor=p:GetAttribute('GWAPTargetFloor'); local location=floor and (floor==8 and 'R00F' or floor==-1
    ↳ and 'B1' or 'L'..(floor+1)) or ''
999     target.Text=s and ((p:GetAttribute('GWAPTarget') or '')..' '..location) or 'Choose a role kiosk, then tap
    ↳ Jobs.'
1000    dialogue.Text=p:GetAttribute('GWAPDialogue') or 'Your carrier will meet you nearby and wait for your Ready
    ↳ reply.'
1001    local carried=p:GetAttribute('CarriedSample') or ''; local notice=p:GetAttribute('GWAPNotice') or ''
1002    note.Text=carried~='' and 'Carrying: '..carried or notice
1003    local cam=workspace.CurrentCamera; local params=RaycastParams.new();
    ↳ params.FilterType=Enum.RaycastFilterType.Exclude; params.FilterDescendantsInstances={p.Character}
1004    local aim=workspace.Raycast(cam.CFrame.Position,cam.CFrame.LookVector*80,params)
1005    for npc,items in pairs(billboards) do
1006        local show=npc.Parent and (npc==bot or (aim and aim.Instance:IsDescendantOf(npc))) and
    ↳ (npc:GetPivot().Position-cam.CFrame.Position).Magnitude<65
1007        for _,billboard in ipairs(items) do if billboard.Parent then billboard.Enabled=show==true and
    ↳ billboard.Name~='MissionSpeech'; billboard.MaxDistance=65 end end
1008    end
1009    for _,station in ipairs(research.Stations:GetChildren()) do
1010        local sign=station:FindFirstChild('StationLabel')
1011        if sign then sign.Enabled=aim~=nil and (aim.Instance==station or aim.Instance:IsDescendantOf(station))
    ↳ and (station.Position-cam.CFrame.Position).Magnitude<40 end
1012    end
1013    showTrail(bot)
1014 end)
1015 ]====])
1016 install(game.ReplicatedStorage.Modules,"BlasterController","ModuleScript",[====[
1017 -- BlasterController
1018 -- Local Tool controller: ContextActionService input, mobile controls, ADS,
1019 -- touch aim damping, camera recoil, crosshair, and weapon bob.
1020
1021 local ContextActionService = game:GetService("ContextActionService")
1022 local GuiService = game:GetService("GuiService")
1023 local Players = game:GetService("Players")
1024 local ReplicatedStorage = game:GetService("ReplicatedStorage")
1025 local RunService = game:GetService("RunService")
1026 local TweenService = game:GetService("TweenService")
1027 local UserInputService = game:GetService("UserInputService")
1028
1029 local Blaster = require(script.Parent:WaitForChild("Blaster"))
1030 local localPlayer = Players.LocalPlayer
1031
1032 local BlasterController = {}
1033 BlasterController.__index = BlasterController
1034
1035 local controllers = setmetatable({}, {__mode = "k"})
1036 local nextControllerId = 0
1037

```



```

1038 local function touchControlsActive()
1039     if UserInputService.TouchEnabled then return true end
1040     local okTouchScreen, touchScreen = pcall(function()
1041         return UserInputService.TouchScreenEnabled
1042     end)
1043     if okTouchScreen and touchScreen then return true end
1044     local okPreferred, preferred = pcall(function()
1045         return UserInputService.PreferredInput
1046     end)
1047     if okPreferred and tostring(preferred):find("Touch") then return true end
1048     local okControls, controls = pcall(function()
1049         return GuiService.TouchControlsEnabled
1050     end)
1051     if okControls and controls then return true end
1052
1053     -- Studio can retain desktop input flags when a phone preset is selected
1054     -- after the client starts. Recognize the simulator's compact landscape
1055     -- viewport so the weapon does not silently lose its mobile camera/buttons.
1056     local camera = workspace.CurrentCamera
1057     local viewport = camera and camera.ViewportSize
1058     return RunService:IsStudio()
1059         and viewport ~= nil
1060         and viewport.Y <= 450
1061         and viewport.X / math.max(1, viewport.Y) >= 1.45
1062 end
1063
1064 local function pointInside(button, point)
1065     if not button or not button.Visible then return false end
1066     local p = button.AbsolutePosition
1067     local s = button.AbsoluteSize
1068     return point.X >= p.X and point.X <= p.X + s.X
1069         and point.Y >= p.Y and point.Y <= p.Y + s.Y
1070 end
1071
1072 -- Hardware support does not identify the active input on an emulator/hybrid.
1073 local function touchAimActive()
1074     local input = UserInputService.GetLastInputType()
1075     if input == Enum.UserInputType.Touch then return true end
1076     if input == Enum.UserInputType.MouseMovement
1077         or input == Enum.UserInputType.MouseButton1
1078         or input == Enum.UserInputType.MouseButton2
1079         or input == Enum.UserInputType.MouseButton3
1080         or input == Enum.UserInputType.MouseWheel
1081         or input == Enum.UserInputType.Keyboard then
1082         return false
1083     end
1084     return touchControlsActive()
1085 end
1086
1087 local function rotationBetween(from, to)
1088     local dot = math.clamp(from:Dot(to), -1, 1)
1089     local axis = from:Cross(to)
1090     if axis.Magnitude < 0.00001 then
1091         if dot > 0 then return CFrame.identity end
1092         axis = from:Cross(Vector3.yAxis)
1093         if axis.Magnitude < 0.00001 then axis = from:Cross(Vector3.xAxis) end
1094     end
1095     return CFrame.fromAxisAngle(axis.Unit, math.acos(dot))
1096 end
1097
1098 local function makeCrosshair(hudName)
1099     local playerGui = localPlayer.WaitForChild("PlayerGui")
1100     local old = playerGui:FindFirstChild(hudName)
1101     if old then old:Destroy() end
1102
1103     local gui = Instance.new("ScreenGui")
1104     gui.Name = hudName
1105     gui.ResetOnSpawn = false
1106     gui.IgnoreGuiInset = true
1107     gui.ScreenInsets = Enum.ScreenInsets.None
1108     gui.ClipToDeviceSafeArea = false
1109     gui.DisplayOrder = 12
1110     gui.Enabled = false
1111     gui.Parent = playerGui
1112
1113     local root = Instance.new("Frame")

```

```

1114     root.Name = "Crosshair"
1115     root.AnchorPoint = Vector2.new(0.5, 0.5)
1116     root.Position = UDim2.fromScale(0.5, 0.5)
1117     root.Size = UDim2.fromOffset(30, 30)
1118     root.BackgroundTransparency = 1
1119     root.Parent = gui
1120
1121     local dot = Instance.new("Frame")
1122     dot.Name = "Dot"
1123     dot.AnchorPoint = Vector2.new(0.5, 0.5)
1124     dot.Position = UDim2.fromScale(0.5, 0.5)
1125     dot.Size = UDim2.fromOffset(6, 6)
1126     dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
1127     dot.BorderSizePixel = 0
1128     dot.Parent = root
1129     local dotCorner = Instance.new("UICorner")
1130     dotCorner.CornerRadius = UDim.new(1, 0)
1131     dotCorner.Parent = dot
1132     local dotStroke = Instance.new("UIStroke")
1133     dotStroke.Color = Color3.fromRGB(6, 15, 22)
1134     dotStroke.Transparency = 0.15
1135     dotStroke.Thickness = 1.5
1136     dotStroke.Parent = dot
1137
1138     for index, data in ipairs({
1139         {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
1140         {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
1141         {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
1142         {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
1143     }) do
1144         local line = Instance.new("Frame")
1145         line.Name = "Line" .. index
1146         line.Position = data[1]
1147         line.Size = data[2]
1148         line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
1149         line.BackgroundTransparency = 0.08
1150         line.BorderSizePixel = 0
1151         line.Parent = root
1152     end
1153
1154     local status = Instance.new("TextLabel")
1155     status.Name = "Status"
1156     status.AnchorPoint = Vector2.new(0.5, 0)
1157     status.Position = UDim2.new(0.5, 0, 0.5, 25)
1158     status.Size = UDim2.fromOffset(210, 24)
1159     status.BackgroundTransparency = 1
1160     status.Font = Enum.Font.GothamBold
1161     status.Text = ""
1162     status.TextColor3 = Color3.fromRGB(220, 232, 242)
1163     status.TextSize = 13
1164     status.TextStrokeTransparency = 0.5
1165     status.Parent = gui
1166
1167     return gui, root, status, dot
1168 end
1169
1170 function BlasterController:ensureHUD()
1171     if self.hud and self.hud.Parent and self.crosshair and self.crosshair.Parent == self.hud
1172         and self.reticleDot and self.reticleDot.Parent == self.crosshair then return false end
1173     if self.hud then self.hud:Destroy() end
1174     self.fallbackShootButton = nil
1175     self.fallbackAimButton = nil
1176     self.hud, self.crosshair, self.statusLabel, self.reticleDot = makeCrosshair("BlasterHUD_" .. self.id)
1177     self.hud.Enabled = self.enabled
1178     return true
1179 end
1180
1181 function BlasterController.new(tool, overrides)
1182     assert(RunService.IsClient(), "BlasterController must be required on the client")
1183     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
1184
1185     nextControllerId = nextControllerId + 1
1186     local self = setmetatable({}, BlasterController)
1187     self.tool = tool
1188     self.overrides = overrides or {}
1189     self.id = nextControllerId

```

```

1190     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
1191     self.connections = {}
1192     self.enabled = false
1193     self.aiming = false
1194     self.triggerHeld = false
1195     self.fireLoopRunning = false
1196     self.sequence = 0
1197     self.nextLocalShot = 0
1198     self.baseGrip = tool.Grip
1199     self.recoilPitch = 0
1200     self.recoilYaw = 0
1201     self.touchInput = nil
1202     self.touchLastPosition = nil
1203     self.hiddenParts = {}
1204     self.shoulderOffset = Vector3.zero
1205
1206     self.connections.equipped = tool.Equipped:Connect(function()
1207         self:Enable()
1208     end)
1209     self.connections.unequipped = tool.Unequipped:Connect(function()
1210         self:Disable()
1211     end)
1212     self.connections.destroying = tool.Destroying:Connect(function()
1213         self:Destroy()
1214     end)
1215     self.connections.ancestry = tool.AncstryChanged:Connect(function()
1216         if self.enabled and tool.Parent ~= localPlayer.Character then
1217             self:Disable()
1218         end
1219     end)
1220     -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
1221     -- remains the primary input layer, while this fallback covers touch/controller layouts
1222     -- that route their built-in Tool button directly to Activated.
1223     self.connections.activated = tool.Activated:Connect(function()
1224         if self.enabled then self:_beginFiring() end
1225     end)
1226     self.connections.deactivated = tool.Deactivated:Connect(function()
1227         self.triggerHeld = false
1228     end)
1229
1230     if localPlayer.Character and tool.Parent == localPlayer.Character then
1231         task.defer(function() self:Enable() end)
1232     end
1233
1234     return self
1235 end
1236
1237 function BlasterController:_button(actionName)
1238     local ok, button = pcall(function()
1239         return ContextActionService:GetButton(actionName)
1240     end)
1241     return ok and button or nil
1242 end
1243
1244 function BlasterController:_fallbackTouchButton(kind, title)
1245     local field = kind == "Shoot" and "fallbackShootButton" or "fallbackAimButton"
1246     local existing = self[field]
1247     if existing and existing.Parent then return existing end
1248     if not self.hud then return nil end
1249
1250     local button = Instance.new("TextButton")
1251     button.Name = "Fallback" .. kind
1252     button.AutoButtonColor = true
1253     button.BackgroundColor3 = Color3.fromRGB(35, 48, 60)
1254     button.BorderSizePixel = 0
1255     button.Text = title
1256     button.Font = Enum.Font.GothamBold
1257     button.TextColor3 = Color3.fromRGB(245, 250, 255)
1258     button.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
1259     button.TextStrokeTransparency = 0.25
1260     button.TextScaled = false
1261     button.ZIndex = 20
1262     button.Parent = self.hud
1263     self[field] = button
1264
1265     if kind == "Shoot" then

```

```

1266         button.InputBegan:Connect(function(input)
1267             if input.UserInputType == Enum.UserInputType.Touch
1268                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
1269                 self:_shootAction(nil, Enum.UserInputState.Begin)
1270             end
1271         end)
1272         button.InputEnded:Connect(function(input)
1273             if input.UserInputType == Enum.UserInputType.Touch
1274                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
1275                 self:_shootAction(nil, Enum.UserInputState.End)
1276             end
1277         end)
1278     else
1279         button.Activated:Connect(function()
1280             if self.enabled then self:SetAiming(not self.aiming) end
1281         end)
1282     end
1283     return button
1284 end
1285
1286 function BlasterController:_styleTouchButtons()
1287     if not touchControlsActive() then return end
1288     local camera = workspace.CurrentCamera
1289     local viewport = camera.ViewportSize or Vector2.new(1280, 720)
1290     local playerGui = localPlayer:FindFirstChildOfClass("PlayerGui")
1291     local touchGui = playerGui and playerGui:FindFirstChild("TouchGui")
1292     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
1293     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 52, 80)
1294     if jump and jump.AbsoluteSize.X > 0 then
1295         diameter = math.clamp(math.floor(math.min(jump.AbsoluteSize.X, jump.AbsoluteSize.Y) * 0.85), 48, 80)
1296     end
1297
1298     -- Own the visuals/hit targets; CAS still binds desktop/controller actions.
1299     local shootButton = self:_fallbackTouchButton("Shoot", "FIRE")
1300     local aimButton = self:_fallbackTouchButton("Aim", "AIM")
1301     for _, action in ipairs({"Shoot", "Aim"}) do
1302         local casButton = self:_button(self.actionPrefix .. action)
1303         if casButton then casButton.Visible = false end
1304     end
1305     for _, button in ipairs({shootButton, aimButton}) do
1306         if button then
1307             button.Visible = true
1308             button.Size = UDim2.fromOffset(diameter, diameter)
1309             button.TextSize = math.floor(diameter * 0.25)
1310             button.BackgroundTransparency = 0.18
1311             if button:IsA("ImageButton") then
1312                 button.ImageTransparency = 0.08
1313                 button.ImageColor3 = Color3.fromRGB(35, 48, 60)
1314             end
1315             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
1316             corner.CornerRadius = UDim.new(1, 0)
1317             corner.Parent = button
1318             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
1319             stroke.Color = Color3.fromRGB(220, 235, 245)
1320             stroke.Transparency = 0.25
1321             stroke.Thickness = 2
1322             stroke.Parent = button
1323             local title = button:FindFirstChild("ActionTitle")
1324             if title and title:IsA("TextLabel") then
1325                 title.Font = Enum.Font.GothamBold
1326                 title.TextColor3 = Color3.fromRGB(245, 250, 255)
1327                 title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
1328                 title.TextStrokeTransparency = 0.25
1329                 title.TextScaled = true
1330             end
1331         end
1332     end
1333
1334     -- GUI absolute coordinates include the simulator's safe-area translation.
1335     -- Anchor to Roblox's real JUMP rectangle, never an estimated viewport point.
1336     local gap = math.max(12, math.floor(diameter * 0.22))
1337     local jumpSize = jump and jump.AbsoluteSize or Vector2.new(diameter, diameter)
1338     local jumpPoint = jump and jump.AbsolutePosition
1339     or (self.hud.AbsolutePosition + self.hud.AbsoluteSize - jumpSize - Vector2.new(28, 28))
1340     local shootPoint = Vector2.new(jumpPoint.X + (jumpSize.X - diameter) * 0.5, jumpPoint.Y - diameter - gap)
1341     local aimPoint = Vector2.new(jumpPoint.X - diameter - gap, jumpPoint.Y + (jumpSize.Y - diameter) * 0.5)

```

```

1342     local function place(button, point)
1343         if not button or not button.Parent then return end
1344         button.AnchorPoint = Vector2.zero
1345         local origin = button.Parent.AbsolutePosition
1346         button.Position = UDim2.fromOffset(
1347             math.floor(point.X - origin.X + 0.5),
1348             math.floor(point.Y - origin.Y + 0.5)
1349         )
1350     end
1351     place(shootButton, shootPoint)
1352     place(aimButton, aimPoint)
1353     self.lastJumpPosition = jump and jump.AbsolutePosition
1354 end
1355
1356 function BlasterController:_getAimScreenPoint(camera)
1357     if not camera then return nil end
1358     if touchAimActive() then
1359         -- Mobile aims along the Scriptable camera's true forward vector. In the
1360         -- Studio phone simulator, ViewportPointToRay(viewportCenter) is offset by
1361         -- the asymmetric safe-area canvas and can land several degrees above the
1362         -- visible centre marker.
1363         return nil
1364     end
1365     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
1366     -- Clicking an on-screen action must not pull the target onto that button.
1367     if pointInside(self.fallbackShootButton, point) or pointInside(self.fallbackAimButton, point) then
1368         return self.lastPointerAim or camera.ViewportSize * 0.5
1369     end
1370     self.lastPointerAim = point
1371     return Vector2.new(
1372         math.clamp(point.X, 0, camera.ViewportSize.X),
1373         math.clamp(point.Y, 0, camera.ViewportSize.Y)
1374     )
1375 end
1376
1377 function BlasterController:_updateAimReticle(camera)
1378     if not self.hud or not self.crosshair or not camera then return end
1379     local rayScreenPoint = self:_getAimScreenPoint(camera)
1380     local screenPoint = rayScreenPoint
1381     local character = localPlayer.Character
1382     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
1383         character,
1384         self.tool,
1385         camera,
1386     }, rayScreenPoint)
1387
1388     if rayScreenPoint == nil then
1389         -- The shot uses Camera.CFrame.LookVector. Project that same forward
1390         -- vector with WorldToScreenPoint so the marker lands in the exact CoreUI
1391         -- coordinate space used by this ScreenGui, including notches/top bars.
1392         local projected = camera:WorldToScreenPoint(
1393             camera.CFrame.Position + camera.CFrame.LookVector * 100
1394         )
1395         screenPoint = Vector2.new(projected.X, projected.Y)
1396     elseif not screenPoint then
1397         screenPoint = camera.ViewportSize * 0.5
1398     end
1399
1400     if solution then
1401         if self.reticledot then
1402             self.reticledot.BackgroundColor3 = solution.result
1403             and Color3.fromRGB(255, 224, 105)
1404             or Color3.fromRGB(120, 235, 255)
1405         end
1406     end
1407
1408     -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
1409     -- and begin at a negative absolute position. Counter that offset so this
1410     -- marker identifies the muzzle-checked world hit on every device.
1411     local origin = self.hud.AbsolutePosition
1412     local center = screenPoint - origin
1413     self.crosshair.Position = UDim2.fromOffset(
1414         math.floor(center.X + 0.5),
1415         math.floor(center.Y + 0.5)
1416     )
1417 end

```

```

1418
1419 function BlasterController:_setLocalHeadHidden(hidden)
1420     local character = localPlayer.Character
1421     if not character then return end
1422     if hidden then
1423         for _, descendant in ipairs(character:GetDescendants()) do
1424             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
1425                 then
1426                 if self.hiddenParts[descendant] == nil then
1427                     self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
1428                     descendant.LocalTransparencyModifier = 1
1429                 end
1430             end
1431         else
1432             for part, transparency in pairs(self.hiddenParts) do
1433                 if part.Parent then part.LocalTransparencyModifier = transparency end
1434             end
1435             table.clear(self.hiddenParts)
1436         end
1437     end
1438
1439 function BlasterController:_beginTouchCamera()
1440     if not touchAimActive() or self.mobileCameraActive then return end
1441     local camera = workspace.CurrentCamera
1442     local character = localPlayer.Character
1443     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1444     if not camera or not humanoid then return end
1445
1446     local look = camera.CFrame.LookVector
1447     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
1448     self.touchYaw = math.atan2(-look.X, -look.Z)
1449     self.savedCameraType = camera.CameraType
1450     self.mobileCameraActive = true
1451     humanoid.AutoRotate = false
1452     camera.CameraType = Enum.CameraType.Scriptable
1453     self:_setLocalHeadHidden(true)
1454 end
1455
1456 function BlasterController:_endTouchCamera()
1457     if not self.mobileCameraActive then return end
1458     local camera = workspace.CurrentCamera
1459     local character = localPlayer.Character
1460     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1461     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
1462     self.mobileCameraActive = false
1463     self.touchInput = nil
1464     self.touchLastPosition = nil
1465     self:_setLocalHeadHidden(false)
1466 end
1467
1468 function BlasterController:SetAiming(aiming)
1469     if not self.enabled then aiming = false end
1470     if self.aiming == aiming then return end
1471     self.aiming = aiming
1472
1473     local camera = workspace.CurrentCamera
1474     if not camera then return end
1475     local config = Blaster.GetWeaponConfig(self.tool)
1476     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
1477     if self.fovTween then self.fovTween:Cancel() end
1478     self.fovTween = TweenService:Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
1479         Enum.EasingDirection.Out), {
1480         FieldOfView = targetFov,
1481     })
1482     self.fovTween:Play()
1483
1484     if touchAimActive() then
1485         -- Mobile weapons keep the camera active in hip-fire too, so the right
1486         -- half of the screen always steers the reticle like a dedicated FPS.
1487         self:_beginTouchCamera()
1488     elseif aiming then
1489         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
1490         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
1491     else
1492         if self.savedMouseSensitivity then

```

```

1492         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
1493         self.savedMouseSensitivity = nil
1494     end
1495 end
1496
1497 local character = localPlayer.Character
1498 local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1499 if humanoid and not self.mobileCameraActive then
1500     humanoid.CameraOffset = aiming
1501     and Vector3.new(1.25, 0.55, 0)
1502     or self.shoulderOffset
1503 end
1504
1505 if self.statusLabel then
1506     self.statusLabel.Text = aiming and "ADS" or ""
1507 end
1508 end
1509
1510 function BlasterController:_fireOnce()
1511     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
1512     local config = Blaster.GetWeaponConfig(self.tool)
1513     local now = os.clock()
1514     if now < self.nextLocalShot then return end
1515     self.nextLocalShot = now + config.cooldown
1516     self.sequence = self.sequence + 1
1517
1518     local camera = workspace.CurrentCamera
1519     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
1520     local recoilScale = self.aiming and 0.55 or 1
1521     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
1522     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
1523 end
1524
1525 function BlasterController:_beginFiring()
1526     if os.clock() < (self.ignoreGuiFireUntil or 0) then return end
1527     self.triggerHeld = true
1528     if self.fireLoopRunning then return end
1529     self.fireLoopRunning = true
1530     task.spawn(function()
1531         repeat
1532             self:_fireOnce()
1533             local config = Blaster.GetWeaponConfig(self.tool)
1534             if not config.automatic then break end
1535             task.wait(config.cooldown)
1536         until not self.triggerHeld or not self.enabled
1537         self.fireLoopRunning = false
1538     end)
1539 end
1540
1541 function BlasterController:_shootAction(_, inputState, inputObject)
1542     if inputState == Enum.UserInputState.Begin and inputObject and inputObject.UserInputType ==
1543     ↳ Enum.UserInputType.MouseButton1 then
1544         local hud=localPlayer.PlayerGui:FindFirstChild("GWAPHUD")
1545         local point=UserInputService.GetMouseLocation()
1546         for _,object in ipairs(localPlayer.PlayerGui:GetGuiObjectsAtPosition(point.X,point.Y)) do
1547             if hud and object:IsDescendantOf(hud) then
1548                 self.ignoreGuiFireUntil=os.clock()+.15
1549                 return Enum.ContextActionResult.Pass
1550             end
1551         end
1552     end
1553     if not self.enabled then return Enum.ContextActionResult.Pass end
1554     if inputState == Enum.UserInputState.Begin and inputObject
1555     and inputObject.UserInputType == Enum.UserInputType.MouseButton1 then
1556         local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
1557         if pointInside(self.fallbackAimButton, point) then return Enum.ContextActionResult.Pass end
1558     end
1559     if inputState == Enum.UserInputState.Begin then
1560         self:_beginFiring()
1561     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
1562         self.triggerHeld = false
1563     end
1564     return Enum.ContextActionResult.Sink
1565 end
1566
1567 function BlasterController:_aimAction(_, inputState, inputObject)

```

```

1567     if not self.enabled then return Enum.ContextActionResult.Pass end
1568     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
1569     if isTouch then
1570         if inputState == Enum.UserInputState.Begin then
1571             self:SetAiming(not self.aiming)
1572         end
1573     elseif inputState == Enum.UserInputState.Begin then
1574         self:SetAiming(true)
1575     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
1576         self:SetAiming(false)
1577     end
1578     return Enum.ContextActionResult.Sink
1579 end
1580
1581 function BlasterController:touchPanAction(_, inputState, inputObject)
1582     if not self.enabled or not touchControlsActive() then
1583         return Enum.ContextActionResult.Pass
1584     end
1585
1586     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
1587     local camera = workspace.CurrentCamera
1588     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
1589     local inset = GuiService.GetGuiInset()
1590     point = point - inset
1591
1592     if inputState == Enum.UserInputState.Begin then
1593         local shootButton = self.fallbackShootButton or self:_button(self.actionPrefix .. "Shoot")
1594         local aimButton = self.fallbackAimButton or self:_button(self.actionPrefix .. "Aim")
1595         local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
1596         local jumpButton = touchGui and touchGui:FindFirstChild("JumpButton", true)
1597         if point.X < viewport.X * 0.42 or pointInside(shootButton, point)
1598             or pointInside(aimButton, point) or pointInside(jumpButton, point) then
1599             return Enum.ContextActionResult.Pass
1600         end
1601         self.touchInput = inputObject
1602         self:_beginTouchCamera()
1603         self.touchLastPosition = point
1604         return Enum.ContextActionResult.Sink
1605     end
1606
1607     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
1608     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
1609         local delta = point - self.touchLastPosition
1610         self.touchLastPosition = point
1611         local sensitivity = self.aiming
1612             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
1613             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
1614         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
1615         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
1616         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
1617             ⇨ math.rad(78))
1618         return Enum.ContextActionResult.Sink
1619     end
1620
1621     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
1622         self.touchInput = nil
1623         self.touchLastPosition = nil
1624         return Enum.ContextActionResult.Sink
1625     end
1626     return Enum.ContextActionResult.Pass
1627 end
1628
1629 function BlasterController:updateArmPose(targetPosition, dt)
1630     local character = localPlayer.Character
1631     local handle = self.tool:FindFirstChild("Handle")
1632     if not character or not handle or not targetPosition then return end
1633     if not self.aimJoint or not self.aimJoint.Parent then
1634         local joints = {}
1635         for _, joint in ipairs(character:GetDescendants()) do
1636             if (joint:IsA("JointInstance") or joint:IsA("AnimationConstraint")) and joint.Part0 and joint.Part1
1637                 ⇨ then
1638                 joints[joint.Part1] = joint
1639             end
1640         end
1641         local chain = {}
1642         local part = handle

```



```

1641     for _ = 1, 8 do
1642         local joint = joints[part]
1643         if not joint then return end
1644         table.insert(chain, 1, joint)
1645         if joint.Name == "Right Shoulder" or joint.Name == "RightShoulder" then
1646             self.aimJoint = joint
1647             self.aimBaseC0 = joint.C0
1648             self.aimChain = chain
1649             break
1650         end
1651         part = joint.Part0
1652     end
1653 end
1654 local shoulder = self.aimJoint
1655 if not shoulder or not shoulder.Part0 then return end
1656 -- Rebuild from the untouched joint/grip and the current animation each frame.
1657 -- This preserves R6/R15 rest rotations and avoids accumulating aim offsets.
1658 local jointWorld = shoulder.Part0.CFrame * self.aimBaseC0
1659 local predicted = shoulder.Part0.CFrame
1660 for _, joint in ipairs(self.aimChain) do
1661     if not joint.Parent then return end
1662     local c0 = joint == shoulder and self.aimBaseC0 or joint.C0
1663     local animation = CFrame.identity -- armed arm chain is isolated from idle/walk animation
1664     predicted = predicted * c0 * animation * joint.C1:Inverse()
1665 end
1666 local forward = self.tool:GetAttribute("BlasterBarrelDirection")
1667 if typeof(forward) ~= "Vector3" or forward.Magnitude < 0.1 then
1668     -- Imported Weapons Kit meshes face +Z; procedural barrels face -Z.
1669     forward = self.tool:GetAttribute("LabSidearm") and Vector3.zAxis or -Vector3.zAxis
1670 end
1671 local turn = CFrame.identity
1672 local pivot = jointWorld.Position
1673 for _ = 1, 4 do
1674     local delta = targetPosition - predicted.Position
1675     if delta.Magnitude < 0.1 then break end
1676     local correction = rotationBetween(predicted:VectorToWorldSpace(forward.Unit), delta.Unit)
1677     predicted = CFrame.new(pivot) * correction * CFrame.new(-pivot) * predicted
1678     turn = correction * turn
1679 end
1680 local pose = shoulder.Part0.CFrame:ToObjectSpace(CFrame.new(pivot) * turn * jointWorld.Rotation)
1681 self.smoothedArmPose = (self.smoothedArmPose or pose):Lerp(pose, 1 - math.exp(-math.min(dt or 1/60, .1) *
    ↳ 24))
1682 pose = self.smoothedArmPose
1683 if shoulder:IsA("AnimationConstraint") then
1684     shoulder.Attachment0.CFrame = pose
1685 else
1686     shoulder.C0 = pose
1687 end
1688 end
1689
1690 function BlasterController:_render(dt)
1691     if not self.enabled then return end
1692     local character = localPlayer.Character
1693     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1694     local root = character and character:FindFirstChild("HumanoidRootPart")
1695     local head = character and character:FindFirstChild("Head")
1696     local camera = workspace.CurrentCamera
1697     if not humanoid or not camera then return end
1698     local hudRebuilt = self:_ensureHUD()
1699     local touchMode = touchControlsActive()
1700     local touchAim = touchAimActive()
1701     if touchAim and not self.mobileCameraActive then
1702         self:_beginTouchCamera()
1703     elseif not touchAim and self.mobileCameraActive then
1704         self:_endTouchCamera()
1705     end
1706     local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
1707     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
1708     if touchMode and (hudRebuilt or self.lastTouchViewport ~= camera.ViewportSize
1709         or (jump and jump.AbsolutePosition ~= self.lastJumpPosition)) then
1710         self.lastTouchViewport = camera.ViewportSize
1711         task.defer(function()
1712             if self.enabled then self:_styleTouchButtons() end
1713         end)
1714     end
1715     -- Keep the inventory grip fixed: no idle or walking weapon bob.

```

```

1716
1717 self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
1718 self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
1719 if self.mobileCameraActive and head and root then
1720     self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
1721     self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
1722     local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
1723     local focus = root.Position + Vector3.new(0, self.stableHeadHeight or 1.5, 0)
1724     local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
1725     local cameraResult = workspace.Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
        ↪ self.tool}))
1726     local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
1727     camera.CFrame = CFrame.new(position) * rotation
1728     camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
1729
1730 elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
1731     camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)
1732 end
1733 local config = Blaster.GetWeaponConfig(self.tool)
1734 local cast = Blaster.CastFromCamera(camera, config.range, {character, self.tool, camera},
    ↪ self:_getAimScreenPoint(camera))
1735 if cast then
1736     self:_updateArmPose(cast.position, dt)
1737 end
1738 self:_updateAimReticle(camera)
1739 end
1740
1741 local activeController
1742 function BlasterController:Enable()
1743     if self.enabled or self.tool.Parent ~= localPlayer.Character then return end
1744     if activeController and activeController ~= self then activeController:Disable() end
1745     activeController = self
1746     self.enabled = true
1747     self.baseGrip = self.tool.Grip
1748     local rootPart = localPlayer.Character and localPlayer.Character:FindFirstChild("HumanoidRootPart")
1749     local headPart = localPlayer.Character and localPlayer.Character:FindFirstChild("Head")
1750     self.stableHeadHeight = rootPart and headPart and (headPart.Position.Y - rootPart.Position.Y + .12) or 1.5
1751     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
        ↪ workspace.CurrentCamera.FieldOfView) or 70
1752
1753     self:_ensureHUD()
1754     self.hud.Enabled = true
1755
1756     local character = localPlayer.Character
1757     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1758     if humanoid then
1759         self.savedAutoRotate = humanoid.AutoRotate
1760         humanoid.AutoRotate = false
1761         self.savedCameraOffset = humanoid.CameraOffset
1762         self.shoulderOffset = Vector3.new(
1763             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
1764             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
1765             0
1766         )
1767         humanoid.CameraOffset = self.shoulderOffset
1768     end
1769     if touchAimActive() then self:_beginTouchCamera() end
1770
1771     local shootName = self.actionPrefix .. "Shoot"
1772     local aimName = self.actionPrefix .. "Aim"
1773     local panName = self.actionPrefix .. "TouchPan"
1774     local priority = Enum.ContextActionPriority.High.Value
1775
1776     ContextActionService.BindActionAtPriority(shootName, function(...)
1777         return self:_shootAction(...)
1778     end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
1779     ContextActionService.BindActionAtPriority(aimName, function(...)
1780         return self:_aimAction(...)
1781     end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
1782     ContextActionService.BindActionAtPriority(panName, function(...)
1783         return self:_touchPanAction(...)
1784     end, false, priority + 1, Enum.UserInputType.Touch)
1785
1786     ContextActionService.SetTitle(shootName, "FIRE")
1787     ContextActionService.SetTitle(aimName, "AIM")
1788     task.delay(0.1, function()

```

```

1789         if self.enabled then self:_styleTouchButtons() end
1790     end)
1791
1792     -- Rotate once, before the camera reads the character. Using camera heading
1793     -- removes the shoulder-offset/nearby-hit feedback loop that caused oscillation.
1794     self.bodyRenderName = self.actionPrefix .. "Body"
1795     RunService:BindToRenderStep(self.bodyRenderName, Enum.RenderPriority.Camera.Value - 1, function()
1796         if not self.enabled then return end
1797         local char = localPlayer.Character
1798         local hum = char and char:FindFirstChildOfClass("Humanoid")
1799         local body = char and char:FindFirstChild("HumanoidRootPart")
1800         local cam = workspace.CurrentCamera
1801         if not hum or hum.Sit or not body or not cam then return end
1802         local direction = self.mobileCameraActive and CFrame.fromOrientation(0, self.touchYaw, 0).LookVector or
            ↪ cam.CFrame.LookVector
1803         local flat = Vector3.new(direction.X, 0, direction.Z)
1804         if flat.Magnitude > .01 then body.CFrame = CFrame.lookAt(body.Position, body.Position + flat.Unit) end
1805     end)
1806     -- Animator writes Transform before PreSimulation. Reset only the armed chain,
1807     -- preserving locomotion animation everywhere else.
1808     self.armAnimationConnection = RunService.PreSimulation:Connect(function()
1809         if not self.enabled or not self.aimChain then return end
1810         for _,joint in ipairs(self.aimChain) do if joint.Parent and joint:IsA("Motor6D") then joint.Transform =
            ↪ CFrame.identity end end
1811         local char=localPlayer.Character
1812         if char then
1813             self.torsoAnimationJoints=self.torsoAnimationJoints or {}
1814             if #self.torsoAnimationJoints==0 then for _,joint in ipairs(char:GetDescendants()) do
1815                 if joint:IsA("Motor6D") and (joint.Name=="Waist" or joint.Name=="RootJoint" or
                    ↪ joint.Name=="Root") then table.insert(self.torsoAnimationJoints,joint) end
1816             end end
1817             for _,joint in ipairs(self.torsoAnimationJoints) do if joint.Parent then
            ↪ joint.Transform=CFrame.identity end end
1818         end
1819     end)
1820     self.renderName = self.actionPrefix .. "Render"
1821     RunService:BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
1822         self:_render(dt)
1823     end)
1824 end
1825
1826 function BlasterController:Disable()
1827     if not self.enabled then return end
1828     self.triggerHeld = false
1829     self:SetAiming(false)
1830     self.enabled = false
1831     if activeController == self then activeController = nil end
1832     self.smoothedArmPose = nil
1833     self.torsoAnimationJoints = nil
1834     if self.armAnimationConnection then self.armAnimationConnection:Disconnect(); self.armAnimationConnection =
        ↪ nil end
1835     if self.bodyRenderName then RunService:UnbindFromRenderStep(self.bodyRenderName) end
1836
1837     ContextActionService:UnbindAction(self.actionPrefix .. "Shoot")
1838     ContextActionService:UnbindAction(self.actionPrefix .. "Aim")
1839     ContextActionService:UnbindAction(self.actionPrefix .. "TouchPan")
1840     if self.renderName then RunService:UnbindFromRenderStep(self.renderName) end
1841     if self.hud then self.hud.Enabled = false end
1842     if self.tool.Parent then self.tool.Grip = self.baseGrip end
1843     if self.aimJoint and self.aimJoint.Parent then
1844         if self.aimJoint:IsA("AnimationConstraint") then
1845             self.aimJoint.Attachment0.CFrame = self.aimBaseC0
1846         else
1847             self.aimJoint.C0 = self.aimBaseC0
1848         end
1849     end
1850     self.aimJoint, self.aimBaseC0, self.aimChain = nil, nil, nil
1851
1852     local camera = workspace.CurrentCamera
1853     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
1854     self:_endTouchCamera()
1855     if self.savedMouseSensitivity then
1856         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
1857         self.savedMouseSensitivity = nil
1858     end
1859     local character = localPlayer.Character

```

```

1860     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
1861     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
1862     self.savedAutoRotate = nil
1863     if humanoid and self.savedCameraOffset then
1864         humanoid.CameraOffset = self.savedCameraOffset
1865     end
1866     self.savedCameraOffset = nil
1867     self.lastTouchViewport = nil
1868 end
1869
1870 function BlasterController:Destroy()
1871     self:Disable()
1872     for _, connection in pairs(self.connections) do
1873         connection:Disconnect()
1874     end
1875     table.clear(self.connections)
1876     if self.hud then self.hud:Destroy() end
1877     controllers[self.tool] = nil
1878 end
1879
1880 function BlasterController.BindTool(tool, overrides)
1881     if controllers[tool] then return controllers[tool] end
1882     local controller = BlasterController.new(tool, overrides)
1883     controllers[tool] = controller
1884     Blaster.StartReplication()
1885     return controller
1886 end
1887
1888 function BlasterController.GetController(tool)
1889     return controllers[tool]
1890 end
1891
1892 function BlasterController.AutoBind()
1893     local binder = {connections = {}}
1894
1895     local function consider(instance)
1896         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
1897             ⇨ instance:GetAttribute("LabSidearm")) then
1898             BlasterController.BindTool(instance)
1899         end
1900     end
1901
1902     local function watch(container)
1903         if not container then return end
1904         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
1905         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
1906     end
1907
1908     watch(localPlayer:WaitForChild("Backpack"))
1909     watch(localPlayer.Character)
1910     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
1911     Blaster.StartReplication()
1912
1913     function binder:Destroy()
1914         for _, connection in ipairs(self.connections) do connection:Disconnect() end
1915         table.clear(self.connections)
1916     end
1917
1918     return binder
1919 end
1920
1921 return BlasterController
1922 ]====)
1923 install(game.ServerScriptService,"ServerBlasterManager","Script",[====[
1924 -- ServerBlasterManager
1925 -- Authoritative validation, obstacle recasting, damage, and shot replication.
1926
1927 local Players = game:GetService("Players")
1928 local ReplicatedStorage = game:GetService("ReplicatedStorage")
1929 local ServerScriptService = game:GetService("ServerScriptService")
1930
1931 local modules = ReplicatedStorage:WaitForChild("Modules")
1932 local Blaster = require(modules:WaitForChild("Blaster"))
1933
1934 local remoteFolder = ReplicatedStorage:FindFirstChild(Blaster.REMOTE_FOLDER_NAME)

```

```

1935 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end
1936 if not remoteFolder then
1937     remoteFolder = Instance.new("Folder")
1938     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME
1939     remoteFolder.Parent = ReplicatedStorage
1940 end
1941
1942 local function getRemote(name)
1943     local remote = remoteFolder:FindFirstChild(name)
1944     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end
1945     if not remote then
1946         remote = Instance.new("RemoteEvent")
1947         remote.Name = name
1948         remote.Parent = remoteFolder
1949     end
1950     return remote
1951 end
1952
1953 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)
1954 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)
1955
1956 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")
1957 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end
1958 if not hitBridge then
1959     hitBridge = Instance.new("BindableFunction")
1960     hitBridge.Name = "BlasterHitBridge"
1961     hitBridge.Parent = ServerScriptService
1962 end
1963
1964 -- Roblox's default third-person zoom can legitimately place the camera well
1965 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
1966 -- zoom distances and the over-shoulder hip-fire camera.
1967 local MAX_CAMERA_OFFSET = 40
1968 local RATE_TOLERANCE = 0.88
1969 local nextShotByPlayer = {}
1970 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
1971
1972 local function findHumanoidFromHit(instance)
1973     local node = instance
1974     while node and node ~= workspace do
1975         if node:IsA("Model") then
1976             local humanoid = node:FindFirstChildOfClass("Humanoid")
1977             if humanoid then return humanoid, node end
1978         end
1979         node = node.Parent
1980     end
1981     return nil
1982 end
1983
1984 local function randomSpread(direction, spread)
1985     if spread <= 0 then return direction end
1986     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
1987     local right = direction:Cross(reference).Unit
1988     local up = right:Cross(direction).Unit
1989     local radius = math.sqrt(math.random()) * spread
1990     local angle = math.random() * math.pi * 2
1991     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
1992 end
1993
1994 local function broadcastShot(shooter, origin, position, normal, color)
1995     local payload = {
1996         origin = origin,
1997         position = position,
1998         normal = normal,
1999         color = color,
2000     }
2001     for _, other in ipairs(Players:GetPlayers()) do
2002         if other ~= shooter then effectRemote:FireClient(other, payload) end
2003     end
2004 end
2005
2006
2007 local function researchHit(player, result, damage)
2008     local researchBridge = ServerScriptService:FindFirstChild("GWAPHitBridge")
2009     if not researchBridge then return false end
2010     local ok, handled = pcall(function() return researchBridge:Invoke(player, result.Instance, damage) end)

```

```

2011     return ok and handled == true
2012 end
2013
2014 local function bridgeHit(player, result, damage, instantHeadshot)
2015     if not result then return false end
2016     if researchHit(player, result, instantHeadshot and result.Instance.Name == "Head" and 1000 or damage) then
2017         ↪ return true end
2018     -- BindableFunction callbacks are write-only. LabEquipment binds this during
2019     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
2020     local ok, response = pcall(function()
2021         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
2022             instantHeadshot and result.Instance.Name == "Head")
2023     end)
2024     return ok and response == true
2025 end
2026
2027 local function genericDamage(player, result, config)
2028     local humanoid, model = findHumanoidFromHit(result.Instance)
2029     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
2030     local victimPlayer = Players:GetPlayerFromCharacter(model)
2031     if victimPlayer and not config.allowPlayerDamage then return end
2032
2033     local damage = config.damage
2034     if result.Instance.Name == "Head" then
2035         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
2036     end
2037     humanoid:TakeDamage(damage)
2038 end
2039
2040 local function validSequence(tool, sequence)
2041     if typeof(sequence) ~= "number" or sequence ~= sequence or sequence % 1 ~= 0 then return false end
2042     if sequence < 1 or sequence > 2147483647 then return false end
2043     local previous = lastSequenceByTool[tool]
2044     if previous and sequence <= previous then return false end
2045     lastSequenceByTool[tool] = sequence
2046     return true
2047 end
2048
2049 local function handleFire(player, packet)
2050     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
2051     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
2052         ↪ return end
2053
2054     local character = player.Character
2055     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
2056     local head = character and character:FindFirstChild("Head")
2057     local root = character and character:FindFirstChild("HumanoidRootPart")
2058     local tool = packet.tool
2059     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
2060     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
2061     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
2062
2063     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
2064     local directionMagnitude = packet.direction.Magnitude
2065     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
2066     local aimDirection = packet.direction.Unit
2067     if not validSequence(tool, packet.sequence) then return end
2068
2069     local config = Blaster.GetWeaponConfig(tool)
2070     local now = os.clock()
2071     if now < (nextShotByPlayer[player] or 0) then return end
2072     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
2073
2074     local params = Blaster.NewRaycastParams({character, tool})
2075     local handle = tool:FindFirstChild("Handle")
2076     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
2077     local tracerColor = tool:GetAttribute("BlasterTracerColor")
2078     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
2079
2080     for _ = 1, config.pellets do
2081         local pelletDirection = randomSpread(aimDirection, config.spread)
2082         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
2083         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
            ↪ config.range)
2084         local muzzleDelta = aimPoint - muzzleOrigin
2085         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection

```

```
2084     local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
2085     local result = workspace.Raycast(muzzleOrigin, castDirection * castLength, params)
2086     local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
2087
2088     broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
2089     if result then
2090         local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
2091         if not handled then genericDamage(player, result, config) end
2092     end
2093 end
2094
2095 if config.selfCostPercent > 0 and humanoid.Health > 0 then
2096     humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
2097 end
2098 end
2099
2100 fireRemote.OnServerEvent:Connect(handleFire)
2101 Players.PlayerRemoving:Connect(function(player)
2102     nextShotByPlayer[player] = nil
2103 end)
2104
2105 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")
2106 ]====])
2107 require(game.ReplicatedStorage.GWAPWorld).build()
2108 require(game.ReplicatedStorage.GWAPRouting).ensureBackups()
2109 print("GWAP installed: original inventory, companion missions, rerouting, wall cleanup and grouped HUD.")
```

Chapter 60

W03 - GWAPWorld.lua

GWAPWorld.lua

Authority group: workspace. 17807 UTF-8 bytes; 259 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Authoritative assignments, research carriers, solo opposition and score accounting.
2 local Players=game:GetService('Players')
3 local RS=game:GetService('ReplicatedStorage')
4 local CS=game:GetService('CollectionService')
5 local SSS=game:GetService('ServerScriptService')
6 local Debris=game:GetService('Debris')
7 local Run=game:GetService('RunService')
8 local World=require(RS:WaitForChild('GWAPWorld'))
9 local scenarios=require(RS:WaitForChild('GWAPScenarios'))
10 local research=World.build()
11 local stations=research.Stations
12 local remote=Instance.new('RemoteEvent'); remote.Name='GWAPAction'; remote.Parent=RS
13 local bridge=Instance.new('BindableFunction'); bridge.Name='GWAPHitBridge'; bridge.Parent=SSS
14 local botsFolder=Instance.new('Folder'); botsFolder.Name='Carriers'; botsFolder.Parent=research
15 local routesFolder=Instance.new('Folder'); routesFolder.Name='Routes'; routesFolder.Parent=research
16 local random=Random.new()
17 local carriers, assignments, cooldowns, armed={}, {}, {}, {}
18 local function alive(p)
19     local c=p.Character; local h=c and c:FindFirstChildOfClass('Humanoid'); return h and h.Health>0 and
    ↪ c:FindFirstChild('HumanoidRootPart')
20 end
21 local function points(p)
22     local s=p:FindFirstChild('leaderstats'); return s and s:FindFirstChild('Points')
23 end
24 local function award(p,n,reason)
25     local v=points(p); if not v then return end
26     v.Value=math.max(0,v.Value+n); p:SetAttribute('GWAPNotice',reason); p:SetAttribute('LastPointAmount',n);
    ↪ p:SetAttribute('LastPointAward',reason)
27 end
28 local function role(p) return p:GetAttribute('LabRole') or (p.Team and p.Team.Name) end
29 local function active(a) return a and a.bot and a.bot.Parent and a.bot:GetAttribute('State')=='RUNNING' end
30 local function nearby(p,bot,d) local r=alive(p); return r and (r.Position-bot:GetPivot().Position).Magnitude<=d
    ↪ end
31 local function publish(a)
32     for p,bound in pairs(assignments) do if bound==a then
33         p:SetAttribute('GWAPStage',a.stage); p:SetAttribute('GWAPTotalStages',#a.route)
34         p:SetAttribute('GWAPTarget',a.route[a.stage] or 'Complete');
        ↪ p:SetAttribute('GWAPState',a.bot:GetAttribute('State'))
35     end end
36 end
37 local function finish(a,state)
38     if not active(a) then return end
39     a.bot:SetAttribute('State',state); a.bot:SetAttribute('TaskState',state);
    ↪ a.bot:SetAttribute('HasSeedling',false)
40     local carried=a.bot:FindFirstChild('Mission seedling'); if carried then carried:Destroy() end
41     publish(a); a.finishedAt=os.clock()
```



```

42 end
43 local function updateHealth(bot)
44     local t=bot:FindFirstChild('HealthText',true); local fill=bot:FindFirstChild('HealthBarFill',true)
45     local h=bot:GetAttribute('Health') or 100
46     if t then t.Text='LIFE '..math.ceil(h)..' / 100' end
47     if fill then fill.Size=UDim2.fromScale(h/100,1) end
48 end
49 local function damage(p,bot,amount)
50     local a=carriers[bot]; if not active(a) then return end
51     local h=math.max(0,bot:GetAttribute('Health')-amount); bot:SetAttribute('Health',h); updateHealth(bot)
52     if role(p)=='Technician' and assignments[p]==a then a.disqualified[p]=true;
53     ↪ p:SetAttribute('GWAPNotice','Protect your carrier: friendly hits cancel escort rewards.') end
54     if h==0 then
55         if assignments[p]==a and role(p)=='Laboratory Technician' then award(p,50,'+50 | Assigned research
56         ↪ carrier stopped') end
57         finish(a,'INTERCEPTED')
58     end
59 end
60 bridge.OnInvoke=function(p,hit,damageAmount)
61     local node=hit
62     while node and node~=workspace do if carriers[node] then damage(p,node,damageAmount); return true end;
63     ↪ node=node.Parent end
64     return false
65 end
66 local function routePoints(from,to)
67     local r={}; local function add(v) table.insert(r,v) end
68     local f1=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
69     local f2=to:GetAttribute('Floor'); local dest=to:GetAttribute('Approach')
70     if f1~=f2 then
71         add(Vector3.new(78,from.Y,from.Z)); add(Vector3.new(78,from.Y,-6))
72         local function flight(level,reverse)
73             local steps={Vector3.new(68.7857,level*16.25+3.25,-6)}
74             for i=1,8 do
75                 ↪ table.insert(steps,Vector3.new(68.7857,level*16.25+i*1.015625+3.25,-15.267857-(i-1)*4.821428))
76                 ↪ end
77                 table.insert(steps,Vector3.new(68.7857,level*16.25+8.125+3.25,-55.28))
78                 table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+3.25,-55.28))
79                 for i=1,8 do table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+i*1.015625+3.25,-49.017857+(
80                 ↪ i-1)*4.821428)) end
81                 table.insert(steps,Vector3.new(88.0714,(level+1)*16.25+3.25,-6));
82                 ↪ table.insert(steps,Vector3.new(78,(level+1)*16.25+3.25,-6))
83                 if reverse then for i=#steps,1,-1 do add(steps[i]) end else for _,v in ipairs(steps) do add(v) end
84                 ↪ end
85             end
86             if f2>f1 then for level=f1,f2-1 do flight(level,false) end else for level=f1-1,f2,-1 do
87             ↪ flight(level,true) end end
88         end
89         add(Vector3.new(78,dest.Y,dest.Z)); add(dest); return r
90     end
91 end
92 local function showRoute(a,route)
93     local old=routesFolder:FindFirstChild(a.bot.Name); if old then old:Destroy() end
94     local folder=Instance.new('Folder'); folder.Name=a.bot.Name; folder.Parent=routesFolder
95     for i,v in ipairs(route) do local n=Instance.new('Vector3Value'); n.Name=string.format('%03d',i);
96     ↪ n.Value=v; n.Parent=folder end
97 end
98 local function go(a,target)
99     local route=routePoints(a.bot:GetPivot().Position,target); showRoute(a,route)
100     for i,goal in ipairs(route) do
101         a.bot:SetAttribute('RouteIndex',i)
102         while active(a) and (goal-a.bot:GetPivot().Position).Magnitude>.12 do
103             local dt=Run.Heartbeat:Wait(); if not active(a) then return false end
104             local start=a.bot:GetPivot().Position; local delta=goal-start
105             local nextPos=start+delta.Unit*math.min(delta.Magnitude,12*math.min(dt,.1))
106             -- Research routes use measured open corridors and each actual stair tread.
107             -- Stop for players; do not shove them or change their movement settings.
108             local blocked=false
109             for _,p in ipairs(Players:GetPlayers()) do local r=alive(p); if r and
110             ↪ (r.Position-nextPos).Magnitude<2.8 then blocked=true end end
111             if not blocked then
112                 local flat=Vector3.new(delta.X,0,delta.Z)
113                 a.bot:PivotTo(flat.Magnitude>.01 and CFrame.lookAt(nextPos,nextPos+flat) or
114                 ↪ CFrame.new(nextPos))
115             end
116             if os.clock()->a.deadline then finish(a,'EXPIRED'); return false end
117         end
118     end
119     if not active(a) then return false end

```

```

106     end
107     return true
108 end
109 local function plantIn(bed,name)
110     local count=bed:GetAttribute('Planted') or 0
111     if count>=12 then return false end
112     local pos=bed.Position+Vector3.new(-8+(count%4)*5,1,-3+math.floor(count/4)*3)
113     World.plant(bed,pos,name); bed:SetAttribute('Planted',count+1); return true
114 end
115 local function checkpoint(a,final)
116     for p,bound in pairs(assignments) do
117         if bound==a and role(p)=='Technician' and not a.disqualified[p] and nearby(p,a.bot,45) then
118             ↪ award(p,final and 60 or 15,final and '+60 | Protected rooftop planting' or '+15 | Protected
119             ↪ research checkpoint') end
120     end
121 end
122 local function runMission(a)
123     for i,name in ipairs(a.route) do
124         a.stage=i; a.bot:SetAttribute('TaskState','Travel to '..name); publish(a)
125         if not go(a,stations[name]) then return end
126         a.bot:SetAttribute('TaskState','Processing: '..name)
127         local seed=a.bot:FindFirstChild('Mission seedling')
128         if seed then seed:PivotTo(CFrame.new(stations[name].Position+Vector3.new(0,2,0))) end
129         for _=1,40 do task.wait(.1); if not active(a) then return end end
130         if i==1 then a.bot:SetAttribute('HasSeedling',true);
131             ↪ seed=World.plant(a.bot,a.bot:GetPivot().Position+Vector3.new(0,0,-2),'Mission seedling') end
132         if i==#a.route then
133             if not plantIn(stations[name],a.title..' | '..a.variant) then finish(a,'PLOT FULL'); return end
134             checkpoint(a,true); finish(a,'COMPLETE'); return
135         else checkpoint(a,false) end
136         if seed then seed:PivotTo(CFrame.new(a.bot:GetPivot().Position+Vector3.new(0,0,-2))) end
137     end
138 end
139 local function startMission(bot,id)
140     local s=scenarios[id]; local route=table.clone(s.route); table.insert(route,'Plot '..id)
141     local a={bot=bot,id=id,title=s.title,route=route,stage=1,disqualified={},deadline=os.clock()+420}
142     a.variant=s.variable..'': '..random:NextInteger(s.low,s.high)..' '..s.units
143     carriers[bot]=a; bot:SetAttribute('State','RUNNING'); bot:SetAttribute('Health',100);
144     ↪ bot:SetAttribute('Scenario',id); bot:SetAttribute('Title',s.title);
145     ↪ bot:SetAttribute('Variant',a.variant); updateHealth(bot)
146     bot:PivotTo(CFrame.new(stations[route[1]]:GetAttribute('Approach')+Vector3.new(0,0,8)))
147     return a
148 end
149 local function makeBot(index)
150     local template
151     for _,b in ipairs(CS:GetTagged('LabWorkerBot')) do if not b:GetAttribute('Seated') and b.PrimaryPart then
152         ↪ template=b; break end end
153     assert(template,'No worker template available')
154     local b=template:Clone(); b.Name='ResearchCarrier '..index
155     for _,v in ipairs(b:GetDescendants()) do if v:IsA('LuaSourceContainer') or v:IsA('ProximityPrompt') or
156         ↪ v:IsA('Humanoid') then v:Destroy() elseif v:IsA('BasePart') then v.Anchored=true; v.CanCollide=false
157         ↪ end end
158     for _,tag in ipairs(CS:GetTags(b)) do CS:RemoveTag(b,tag) end
159     b:SetAttribute('Dead',false); b:SetAttribute('KnockedOut',false); b:SetAttribute('GWAPCarrier',true);
160     ↪ b.Parent=botsFolder
161     local h=Instance.new('Highlight'); h.Name='ResearchOutline'; h.FillTransparency=1;
162     ↪ h.OutlineColor=Color3.fromRGB(92,244,199); h.DepthMode=Enum.HighlightDepthMode.Occluded; h.Parent=b
163     return b
164 end
165 local available={}; for i=1,5 do available[i]=makeBot(i); available[i]:PivotTo(CFrame.new(78,3.25,170+i*8));
166     ↪ available[i]:SetAttribute('State','IDLE') end
167 local weapons={
168     {name='Research
169         ↪ SMG',damage=18,range=220,cooldown=.11,auto=true,color=Color3.fromRGB(244,191,53),length=2.4},
170     {name='Precision Rifle',damage=75,range=850,cooldown=1.2,color=Color3.fromRGB(123,99,202),length=4.2},
171     {name='Scatter Blaster',damage=16,range=110,cooldown=.85,pellets=7,spread=.09,color=Color3.fromRGB(74,166,
172     ↪ 183),length=3},
173     {name='Blast
174         ↪ Launcher',damage=65,range=140,cooldown=2.5,blast=15,color=Color3.fromRGB(213,123,54),length=2.6},
175     {name='Pulse Bomb',damage=80,range=65,cooldown=4,blast=20,color=Color3.fromRGB(85,135,56),length=1},
176 }
177 local function equip(p,index)
178     local def=weapons[index]; if not def or not alive(p) then return end
179     local backpack=p:FindFirstChildOfClass('Backpack'); if not backpack then return end
180     local tool=p.Character:FindFirstChild(def.name) or backpack:FindFirstChild(def.name)
181     if not tool then

```

```

168     tool=Instance.new('Tool'); tool.Name=def.name; tool.CanBeDropped=false; tool.ToolTip=def.blast and
    ↪ 'Explosive area damage | same aim controls' or 'Research weapon | same aim controls'
169     local attrs={BlasterWeapon=true,LabSidearm=true,BlasterDamage=def.damage,BlasterRange=def.range,Blaste
    ↪ rCooldown=def.cooldown,BlasterAutomatic=def.auto or false,BlasterPellets=def.pellets or
    ↪ 1,BlasterSpread=def.spread or 0,BlasterSelfCostPercent=0,BlasterBlastRadius=def.blast or
    ↪ 0,BlasterADSFOV=index==2 and 35 or 55,BlasterTracerColor=def.color,GWAPWeapon=true}
170     for k,v in pairs(attrs) do tool:SetAttribute(k,v) end
171     local h=Instance.new('Part'); h.Name='Handle'; h.Size=Vector3.new(.5,.7,def.length); h.Color=def.color;
    ↪ h.CanCollide=false; h.Massless=true; if index==5 then h.Shape=Enum.PartType.Ball;
    ↪ h.Size=Vector3.new(.85,.85,.85) end; h.Parent=tool
172     local barrel=Instance.new('Part'); barrel.Name='Barrel'; barrel.Size=Vector3.new(.22,.22,.7);
    ↪ barrel.Color=Color3.fromRGB(30,34,39); barrel.CanCollide=false; barrel.Massless=true;
    ↪ barrel.CFrame=h.CFrame*CFrame.new(0,.1,-def.length/2); barrel.Parent=tool
173     local weld=Instance.new('WeldConstraint'); weld.Part0=h; weld.Part1=barrel; weld.Parent=barrel;
    ↪ tool.Parent=backpack
174     end
175     p.Character:FindFirstChildOfClass('Humanoid'):EquipTool(tool)
176 end
177 local function choose(p,id)
178     if not alive(p) or workspace:GetAttribute('RoundPhase')=='BREAK' then return end
179     if role(p)~='Technician' and role(p)~='Laboratory Technician' then p:SetAttribute('GWAPNotice','Choose a
    ↪ role at a blue or green kiosk first.');
```

return end

```

180     if active(assignments[p]) then p:SetAttribute('GWAPNotice','Finish your current assignment before selecting
    ↪ another.');
```

return end

```

181     local a
182     for _,candidate in pairs(carriers) do if active(candidate) and candidate.id==id then a=candidate; break end
    ↪ end
183     if not a then
184         for _,bot in ipairs(available) do if not active(carriers[bot]) then a=startMission(bot,id); break end
            ↪ end
185     end
186     if not a then p:SetAttribute('GWAPNotice','All five carriers are busy. Join an active scenario or wait.');
```

return end

```

187     assignments[p]=a; p:SetAttribute('GWAPCarrier',a.bot.Name); p:SetAttribute('GWAPScenario',id);
    ↪ p:SetAttribute('GWAPVariant',a.variant); p:SetAttribute('GWAPNotice',role(p)~='Technician' and 'Escort
    ↪ your carrier. Stay within 45 studs at checkpoints.' or 'Stop your assigned carrier before rooftop
    ↪ planting.');
```

publish(a)

```

188     if not a.started then a.started=true; task.spawn(function() task.wait(3); runMission(a) end) end
189 end
190 remote.OnServerEvent:Connect(function(p,action,index)
191     if typeof(action)~='string' or typeof(index)~='number' or index~=index or index%1~=0 then return end
192     if os.clock()-(cooldowns[p] or 0)<.3 then return end; cooldowns[p]=os.clock()
193     if action=='choose' and index>=1 and index<=10 then choose(p,index)
194     elseif action=='weapon' and index>=1 and index<=#weapons then equip(p,index) end
195 end)
196 for _,bed in ipairs(stations:GetChildren()) do
197     local prompt=bed:FindFirstChild('PlantSeedling')
198     if prompt then prompt.Triggered:Connect(function(p)
199         local r=alive(p); if not r or (r.Position-bed.Position).Magnitude>14 then return end
200         local sample
201         for _,holder in ipairs({p.Character,p:FindFirstChildOfClass('Backpack')}) do if holder then for _,t in
            ↪ ipairs(holder:GetChildren()) do if t:IsA('Tool') and (t:GetAttribute('PlantSample') or
            ↪ t:GetAttribute('TransferSample')) then sample=t; break end end end end
202         if not sample then p:SetAttribute('GWAPNotice','Carry a germinated plant sample to this plot.');
```

return end

```

203         if plantIn(bed,sample.Name) then sample:Destroy(); p:SetAttribute('CarriedSample','');
            ↪ p:SetAttribute('GWAPNotice','Seedling planted in '..bed.Name) else
            ↪ p:SetAttribute('GWAPNotice','This plot is full. Use another plot.') end
204     end) end
205 end
206 local function setArmed()
207     local all=CS:GetTagged('LabWorkerBot'); for _,bot in ipairs(available) do table.insert(all,bot) end
208     table.sort(all,function(a,b) return a.Name<b.Name end)
209     armed={}; local solo=#Players:GetPlayers()==1; local count=solo and math.floor(#all*.2) or 0
210     local selected={}; local shuffle=table.clone(all); for i=#shuffle,2,-1 do local j=random:NextInteger(1,i);
    ↪ shuffle[i],shuffle[j]=shuffle[j],shuffle[i] end
211     for i=1,count do selected[shuffle[i]]=true end
212     for _,bot in ipairs(all) do
213         local yes=selected[bot]==true; bot:SetAttribute('GWAPArmed',yes)
214         local gun=bot:FindFirstChild('SoloWeapon'); if gun then gun:Destroy() end
215         if yes then
216             table.insert(armed,bot); local anchor=bot:FindFirstChild('Right Arm') or bot.PrimaryPart
217             if anchor then local g=Instance.new('Part'); g.Name='SoloWeapon'; g.Size=Vector3.new(.5,.55,2);
                ↪ g.Color=Color3.fromRGB(235,110,65); g.Anchored=true; g.CanCollide=false; g.CanQuery=false;
                ↪ g.CFrame=anchor.CFrame*CFrame.new(0,-.4,-.7); g.Parent=bot end
218         end

```

```

219     end
220     workspace:SetAttribute('GWAPArmedCount',count); workspace:SetAttribute('GWAPBotCount',#all)
221 end
222 Players.PlayerAdded:Connect(function() task.defer(setArmed) end)
223 Players.PlayerRemoving:Connect(function(p) assignments[p]=nil; cooldowns[p]=nil; task.defer(setArmed) end)
224 setArmed()
225 task.spawn(function()
226     while task.wait(2.5) do
227         local list=Players:GetPlayers(); if #list~=1 or workspace:GetAttribute('RoundPhase')=='BREAK' then
228             ↪ continue end
229         local p=list[1]; local root=alive(p); if not root then continue end
230         for _,bot in ipairs(armed) do
231             if not bot.Parent or bot:GetAttribute('Dead') or bot:GetAttribute('KnockedOut') or
232                 ↪ (bot:GetAttribute('Health') or 100)<=0 then continue end
233             -- Assigned escort carriers never attack their own defender.
234             if assignments[p] and assignments[p].bot==bot and role(p)=='Technician' then continue end
235             local origin=bot:GetPivot().Position+Vector3.new(0,1,0); local delta=root.Position-origin
236             if delta.Magnitude<4 or delta.Magnitude>85 then continue end
237             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
238             ↪ params.FilterDescendantsInstances={bot}
239             local hit=workspace:Raycast(origin,delta,params)
240             if hit and hit.Instance:IsDescendantOf(p.Character) then
241                 local spread=Vector3.new(random:NextNumber(-2.5,2.5),random:NextNumber(-1.5,1.5),random:NextNu
242                 ↪ mber(-2.5,2.5))
243                 local shot=workspace:Raycast(origin,delta+spread,params); local endpoint=shot and shot.Position
244                 ↪ or origin+delta+spread
245                 local beam=Instance.new('Part'); beam.Name='NPC shot'; beam.Anchored=true;
246                 ↪ beam.CanCollide=false; beam.CanQuery=false; beam.Material=Enum.Material.Neon;
247                 ↪ beam.Color=Color3.fromRGB(255,105,72);
248                 ↪ beam.Size=Vector3.new(.07,.07,(endpoint-origin).Magnitude);
249                 ↪ beam.CFrame=CFrame.lookAt((origin+endpoint)/2,endpoint); beam.Parent=workspace;
250                 ↪ Debris:AddItem(beam,.18)
251                 if shot and shot.Instance:IsDescendantOf(p.Character) then
252                     local v=points(p); if v then
253                         -- Existing score is an IntValue: accumulate fractional losses, never turn 1% into a
254                         ↪ minimum one-point tax.
255                         local debt=(p:GetAttribute('GWAPFractionalLoss') or 0)+math.max(0,v.Value)*.01
256                         local loss=math.floor(debt+1e-8); p:SetAttribute('GWAPFractionalLoss',debt-loss);
257                         ↪ v.Value=math.max(0,v.Value-loss)
258                         p:SetAttribute('GWAPNotice','NPC hit: -1% score');
259                         ↪ p:SetAttribute('GWAPHitAt',workspace:GetServerTimeNow())
260                     end
261                 end
262             end
263         end
264     end
265 end)
266 workspace:GetAttributeChangedSignal('RoundNumber'):Connect(function()
267     for _,a in pairs(carriers) do if active(a) then finish(a,'SHIFT ENDED') end end
268     for _,p in ipairs(Players:GetPlayers()) do p:SetAttribute('GWAPFractionalLoss',0) end
269     for _,bed in ipairs(stations:GetChildren()) do if bed:GetAttribute('Planted') then for _,v in
270         ↪ ipairs(bed:GetChildren()) do if v:IsA('Model') then v:Destroy() end end; bed:SetAttribute('Planted',0)
271         ↪ end end
272     setArmed()
273 end)
274 print('[GWAP] 10 scenarios, 5 research carriers, rooftop planting and solo opposition ready')

```

Chapter 61

W04 - GWAPScenarios.lua

GWAPScenarios.lua

Authority group: workspace. 2067 UTF-8 bytes; 13 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Gameplay simulations inspired by the supplied GWAP ideas; no real measurements are fabricated.
2 return {
3     {title='Germination viability', purpose='Compare germination under different moisture levels.',
4       ↪ route={'Seed bank','Incubator'}, variable='Moisture', units='%', low=35, high=80},
5     {title='Pathogen resistance', purpose='Track resistant plant tissue through containment and analysis.',
6       ↪ route={'Containment','Bio-analyzer'}, variable='Resistance batch', units='', low=1, high=9},
7     {title='Light spectrum optimization', purpose='Compare seedling responses to visible and infrared light.',
8       ↪ route={'Seed bank','Spectrograph','Infrared scanner'}, variable='Photoperiod', units='hours', low=8,
9       ↪ high=18},
10    {title='Soil bioremediation', purpose='Trace seedlings through root absorption analysis.', route={'Chemical
11      ↪ seedlings','Root analyzer'}, variable='Soil batch', units='', low=1, high=12},
12    {title='Hydroponic versus soil', purpose='Process paired root samples before greenhouse planting.',
13      ↪ route={'Split-root tray','Centrifuge','Root analyzer'}, variable='Nutrient batch', units='', low=1,
14      ↪ high=8},
15    {title='UV adaptation', purpose='Transport exposed seedlings through vitality screening.', route={'UV
16      ↪ chamber','Vitality scanner'}, variable='Exposure', units='seconds', low=5, high=20},
17    {title='Gravitropism assay', purpose='Calibrate tilted roots before planting upright seedlings.',
18      ↪ route={'Clinostat','Orientation scanner'}, variable='Tray tilt', units='degrees', low=10, high=80},
19    {title='Drought resistance', purpose='Measure simulated stomatal response before rehydration.',
20      ↪ route={'Climate chamber','Porometer'}, variable='Humidity', units='%', low=20, high=45},
21    {title='Microbiome symbiosis', purpose='Carry inoculated seeds through symbiosis incubation.', route={'Wet
22      ↪ lab','Symbiosis incubator'}, variable='Inoculum batch', units='', low=1, high=8},
23    {title='Cold-hardiness screening', purpose='Screen thawed seedlings before rooftop recovery.',
24      ↪ route={'Cryo-freezer','Vitality scanner'}, variable='Recovery temperature', units='C', low=5, high=20},
25 }
```

Chapter 62

W05 - GWAPNPC.lua

GWAPNPC.lua

Authority group: workspace. 6021 UTF-8 bytes; 90 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Shared NPC presentation uses the same original Weapons Kit assets as player inventory.
2 local N={}
3 local specs={
4     {name='Lab Sidearm',mesh='rbxassetid://2492922600',texture='rbxassetid://2492931263',scale=.1407,size=Vect
    ↪ or3.new(.20,.92,1.36)},
5     {name='Lab Shotgun',mesh='rbxassetid://2492972199',texture='rbxassetid://2492974190',scale=.6666,size=Vect
    ↪ or3.new(.31,1,4.87)},
6     {name='Lab Rifle',mesh='rbxassetid://2759059561',texture='rbxassetid://2759063893',scale=.6383,size=Vector
    ↪ 3.new(.29,1.17,6.18)},
7 }
8 function N.gun(bot,index)
9     local old=bot:FindFirstChild('SoloWeapon'); if old then old:Destroy() end
10    local spec=specs[(index-1)%#specs+1]; local arm=bot:FindFirstChild('ArmR') or bot:FindFirstChild('Right
    ↪ Arm') or bot.PrimaryPart
11    if not arm then return end
12    local p=Instance.new('Part'); p.Name='SoloWeapon'; p.Size=spec.size; p.Anchored=true; p.CanCollide=false;
    ↪ p.CanTouch=false; p.CanQuery=false
13    p.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0); p.Color=Color3.fromRGB(126,133,145)
14    local mesh=Instance.new('SpecialMesh'); mesh.MeshType=Enum.MeshType.FileMesh; mesh.MeshId=spec.mesh;
    ↪ mesh.TextureId=spec.texture; mesh.Scale=Vector3.one*spec.scale; mesh.Parent=p
15    p:SetAttribute('InventoryName',spec.name); p.Parent=bot; bot:SetAttribute('NPCGun',spec.name)
16    return p
17 end
18 function N.appearance(bot)
19    if bot:GetAttribute('Gender')~='Girl' then return end
20    local head=bot:FindFirstChild('Head'); if not head or bot:FindFirstChild('ResearchHair') then return end
21    local hair=bot:FindFirstChild('HairCrown'); local color=hair and hair.Color or Color3.fromRGB(67,39,25)
22    local folder=Instance.new('Model'); folder.Name='ResearchHair'; folder.Parent=bot
23    local function piece(name,size,offset)
24        local p=Instance.new('Part'); p.Name=name; p.Size=size; p.CFrame=head.CFrame*CFrame.new(offset);
        ↪ p.Color=color; p.Material=Enum.Material.SmoothPlastic; p.Anchored=true; p.CanCollide=false;
        ↪ p.CanQuery=false; p.Parent=folder; return p
25    end
26    piece('Back hair',Vector3.new(1.23,1.7,.34),Vector3.new(0,-.25,.55))
27    for __,side in ipairs({-1,1}) do piece('Side
    ↪ locks',Vector3.new(.28,1.25,.58),Vector3.new(side*.58,-.15,.15)) end
28    local pony=piece('Tied ponytail',Vector3.new(.55,1.3,.65),Vector3.new(0,-.25,.9));
    ↪ pony.Shape=Enum.PartType.Ball
29    local tie=piece('Hair tie',Vector3.new(.6,.16,.66),Vector3.new(0,.25,.9));
    ↪ tie.Color=Color3.fromRGB(107,174,174)
30 end
31 function N.say(bot,message)
32    local head=bot:FindFirstChild('Head') or bot.PrimaryPart; if not head then return end
33    local gui=head:FindFirstChild('MissionSpeech')
34    if not gui then
```

```

35     gui=Instance.new('BillboardGui'); gui.Name='MissionSpeech'; gui.Size=UDim2.fromOffset(260,66);
    ↪ gui.StudsOffset=Vector3.new(0,4.8,0); gui.MaxDistance=55; gui.Parent=head
36     local t=Instance.new('TextLabel'); t.Name='Message'; t.Size=UDim2.fromScale(1,1);
    ↪ t.BackgroundColor3=Color3.fromRGB(25,38,45); t.BackgroundTransparency=.12;
    ↪ t.TextColor3=Color3.fromRGB(225,255,240); t.Font=Enum.Font.Gotham; t.TextSize=14;
    ↪ t.TextWrapped=true; t.Parent=gui
37     Instance.new('UICorner',t).CornerRadius=UDim.new(0,10)
38     end
39     gui.Message.Text=message; gui.Enabled=true
40     local serial=(gui:GetAttribute('Serial') or 0)+1; gui:SetAttribute('Serial',serial)
41     task.delay(9,function() if gui.Parent and gui:GetAttribute('Serial')==serial then gui.Enabled=false end
    ↪ end)
42 end
43 function N.nearPlayer(player,bot)
44     local root=player.Character and player.Character:FindFirstChild('HumanoidRootPart'); if not root then
    ↪ return false end
45     local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ params.FilterDescendantsInstances={player.Character,bot}; params.RespectCanCollide=true
46     local overlap=OverlapParams.new(); overlap.FilterType=Enum.RaycastFilterType.Exclude;
    ↪ overlap.FilterDescendantsInstances={player.Character,bot}; overlap.RespectCanCollide=true
47     for _,offset in ipairs({Vector3.new(5,0,-6),Vector3.new(-5,0,-6),Vector3.new(7,0,0),Vector3.new(-7,0,0),Ve
    ↪ ctor3.new(0,0,8)}) do
48         local pos=root.CFrame:PointToWorldSpace(offset)
49         local floor=workspace.Raycast(pos+Vector3.new(0,3,0),Vector3.new(0,-10,0),params)
50         local sight=workspace.Raycast(root.Position,pos-root.Position,params)
51         if floor and floor.Normal.Y>.7 and not sight then
52             pos=Vector3.new(pos.X,floor.Position.Y+3.25,pos.Z)
53             if #workspace:GetPartBoundsInBox(CFrame.new(pos),Vector3.new(3,4.6,3),overlap)==0 then
    ↪ bot:PivotTo(CFrame.lookAt(pos,Vector3.new(root.Position.X,pos.Y,root.Position.Z))); return true
    ↪ end
54         end
55     end
56     return false
57 end
58
59 local poses=setmetatable({}, {__mode='k'})
60 function N.animate(bot,dt)
61     local root=bot.PrimaryPart; if not root then return end
62     local data=poses[bot]
63     if not data then
64         data={parts={},previous=root.Position,phase=0,blend=0}
65         for _,name in ipairs({'LegL','LegR','ArmL','ArmR','Trainer-1','Trainer1','Sole-1','Sole1'}) do
66             local p=bot:FindFirstChild(name); if p then
    ↪ data.parts[name]={part=p,rest=root.CFrame:ToObjectSpace(p.CFrame)} end
67         end
68         poses[bot]=data
69     end
70     local distance=(root.Position-data.previous).Magnitude; data.previous=root.Position
71     local speed=distance/math.max(dt,.001)
72     if distance>8 then speed=0; distance=0 end -- regrouping is not a walking step
73     data.phase+=distance*math.pi/3.2
74     local moving=speed>.3 and 1 or 0
75     data.blend+=(moving-data.blend)*(1-math.exp(-dt*16))
76     local swing=math.sin(data.phase)*.42*data.blend
77     for name,entry in pairs(data.parts) do if entry.part.Parent then
78         local arm=name:sub(1,3)=='Arm'
79         local left=name=='LegL' or name=='ArmL' or name:find('-1',1,true)
80         local sign=left and -1 or 1
81         local pivot=arm and Vector3.new(sign*1.18,.85,0) or Vector3.new(sign*.42,-1.05,0)
82         local angle=arm and -swing*sign*.65 or swing*sign
83         if arm and bot:GetAttribute('HasSeedling') then angle=-.4 end
84         entry.part.CFrame=root.CFrame*CFrame.new(pivot)*CFrame.Angles(angle,0,0)*CFrame.new(-pivot)*entry.rest
85     end end
86     local gun=bot:FindFirstChild('SoloWeapon'); local arm=bot:FindFirstChild('ArmR')
87     if gun and arm then gun.CFrame=arm.CFrame*CFrame.new(0,-.35,-.7)*CFrame.Angles(0,math.pi,0) end
88 end
89
90 return N

```

Chapter 63

W06 - GWAPRouting.lua

GWAPRouting.lua

Authority group: workspace. 2594 UTF-8 bytes; 43 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 local R={}
2 local function floorOf(item)
3     local n=item:GetAttribute('Floor'); if n then return n end
4     local label=item:GetAttribute('Level') or ''; return label=='Basement' and -1 or
    ↪ (tonumber(label:match('%d+')) or 1)-1
5 end
6 R.floorOf=floorOf
7 function R.healthy(item)
8     return item and item:IsDescendantOf(workspace) and not item:GetAttribute('Exploded') and not
    ↪ item:GetAttribute('BlastRemoved') and item:GetAttribute('FunctionalState')~='DESTROYED'
9 end
10 function R.ensureBackups()
11     local stations=workspace:WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch'):WaitForChild('Sta_
    ↪ tions')
12     local originals=stations:GetChildren(); local perFloor={}
13     table.sort(originals,function(a,b)return a.Name<b.Name end)
14     for _,item in ipairs(originals) do if item:IsA('BasePart') and not item:GetAttribute('Backup') then
15         local capability=item:GetAttribute('Planted')~nil and 'Greenhouse planting' or item.Name
16         item:SetAttribute('Capability',capability)
17         if capability~='Greenhouse planting' then for variant=1,2 do
18             local name=item.Name..' - Backup '..variant
19             if not stations:FindFirstChild(name) then
20                 local copy=item:Clone(); copy.Name=name; copy:SetAttribute('Backup',true);
                ↪ copy:SetAttribute('Capability',capability)
21                 local floor=floorOf(item); if variant==2 then floor=(floor+2)%4-1 end
22                 local z=item.Position.Z; local x=89
23                 if variant==2 then perFloor[floor]=(perFloor[floor] or 0)+1; z=150+perFloor[floor]*14; x=67 end
24                 local pos=Vector3.new(x,floor*16.25+1.5,z); local shift=pos-copy.Position
25                 copy.Position=pos; for _,p in ipairs(copy:GetDescendants()) do if p:IsA('BasePart') then
                ↪ p.Position+=shift end end
26                 copy:SetAttribute('Floor',floor);
                ↪ copy:SetAttribute('Approach',Vector3.new(78,floor*16.25+3.25,z))
27                 copy:SetAttribute('Exploded',false); copy:SetAttribute('FunctionalState','RUNNING')
28                 local sign=copy:FindFirstChild('StationLabel'); local label=sign and
                ↪ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=capability..' | Backup |
                ↪ '..(floor==-1 and 'B1' or 'L'..(floor+1)) end
                copy.Parent=stations
29             end
30         end end
31     end end
32     return stations
33 end
34 function R.choose(stations,capability,preferredFloor,position)
35     local best,rank
36     for _,item in ipairs(stations:GetChildren()) do if item:GetAttribute('Capability')==capability and
    ↪ R.healthy(item) and ((item:GetAttribute('Planted') or 0)<12) then
```



```
38      local score=(floorOf(item)==preferredFloor and 0 or  
      ↪ 100000)+math.abs(floorOf(item)-preferredFloor)*1000+(item.Position-position).Magnitude  
39      if not rank or score<rank then best,rank=item,score end  
40    end end  
41    return best  
42 end  
43 return R
```

Chapter 64

W07 - GWAPServer.lua

GWAPServer.lua

Authority group: workspace. 28562 UTF-8 bytes; 418 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Authoritative assignments, research carriers, solo opposition and score accounting.
2 local Players=game:GetService('Players')
3 local RS=game:GetService('ReplicatedStorage')
4 local CS=game:GetService('CollectionService')
5 local SSS=game:GetService('ServerScriptService')
6 local Debris=game:GetService('Debris')
7 local Run=game:GetService('RunService')
8 local NPC=require(RS:WaitForChild('GWAPNPC'))
9 local Pathfinding=game:GetService('PathfindingService')
10 local pool=Instance.new('Folder'); pool.Name='GWAPCarrierPool'; pool.Parent=game:GetService('ServerStorage')
11 local companions={}
12 local World=require(RS:WaitForChild('GWAPWorld'))
13 local scenarios=require(RS:WaitForChild('GWAPScenarios'))
14 local research=World.build()
15 local Router=require(RS:WaitForChild('GWAPRouting'))
16 local stations=Router.ensureBackups()
17 local stationInfo={}
18 for _,item in ipairs(stations:GetChildren()) do
19     ↳ stationInfo[item.Name]={capability=item:GetAttribute('Capability'),floor=Router.floorOf(item)} end
20 local remote=Instance.new('RemoteEvent'); remote.Name='GWAPAction'; remote.Parent=RS
21 local bridge=Instance.new('BindableFunction'); bridge.Name='GWAPHitBridge'; bridge.Parent=SSS
22 local botsFolder=Instance.new('Folder'); botsFolder.Name='Carriers'; botsFolder.Parent=research
23 local routesFolder=Instance.new('Folder'); routesFolder.Name='Routes'; routesFolder.Parent=research
24 local random=Random.new()
25 local carriers, assignments, cooldowns, armed={}, {}, {}, {}
26 local function alive(p)
27     local c=p.Character; local h=c and c:FindFirstChildOfClass('Humanoid'); return h and h.Health>0 and
28     ↳ c:FindFirstChild('HumanoidRootPart')
29 end
30 local function points(p)
31     local s=p:FindFirstChild('leaderstats'); return s and s:FindFirstChild('Points')
32 end
33 local function award(p,n,reason)
34     local v=points(p); if not v then return end
35     v.Value=math.max(0,v.Value+n); p:SetAttribute('GWAPNotice',reason); p:SetAttribute('LastPointAmount',n);
36     ↳ p:SetAttribute('LastPointAward',reason)
37 end
38 local function role(p) return p and (p:GetAttribute('LabRole') or (p.Team and p.Team.Name)) end
39 local function active(a) return a and a.bot and a.bot.Parent and a.bot:GetAttribute('State')== 'RUNNING' end
40 local function nearby(p,bot,d) local r=alive(p); return r and (r.Position-bot:GetPivot().Position).Magnitude<=d
41     ↳ end
42 local function publish(a)
43     for p,bound in pairs(assignments) do if bound==a then
44         p:SetAttribute('GWAPStage',a.stage); p:SetAttribute('GWAPTotalStages',#a.route)
```

```

41     p:SetAttribute('GWAPTarget',a.target and a.target.Name or a.route[a.stage] or 'Complete');
    ↪ p:SetAttribute('GWAPTargetFloor',a.target and Router.floorOf(a.target) or nil);
    ↪ p:SetAttribute('GWAPState',a.bot:GetAttribute('State'))
42 end end
43 end
44 local function tell(a,message)
45     NPC.say(a.bot,message)
46     for p,bound in pairs(assignments) do if bound==a then p:SetAttribute('GWAPDialogue',message);
    ↪ p:SetAttribute('GWAPSpeaker',a.bot:GetAttribute('DisplayName') or 'Research carrier') end end
47 end
48 local function finish(a,state)
49     if not active(a) then return end
50     a.bot:SetAttribute('State',state); a.bot:SetAttribute('TaskState',state);
    ↪ a.bot:SetAttribute('HasSeedling',false)
51     local carried=a.bot:FindFirstChild('Mission seedling'); if carried then carried:Destroy() end
52     publish(a); a.finishedAt=os.clock(); tell(a,state=='COMPLETE' and 'The plant is in the greenhouse. Research
    ↪ assignment complete. Thank you!' or 'Mission stopped: '..state..' Choose another assignment when
    ↪ ready.')
53 end
54 local function updateHealth(bot)
55     local t=bot:FindFirstChild('HealthText',true); local fill=bot:FindFirstChild('HealthBarFill',true)
56     local h=bot:GetAttribute('Health') or 100
57     local status=bot:FindFirstChild('Status',true)
58     if status and status:IsA('TextLabel') then status.Text='RESEARCH CARRIER\n'..(bot:GetAttribute('Title') or
    ↪ 'Choose an assignment') end
59     if t then t.Text='LIFE '..math.ceil(h).. ' / 100' end
60     if fill then fill.Size=UDim2.fromScale(h/100,1) end
61 end
62 local function damage(p,bot,amount)
63     local a=carriers[bot]; if not active(a) then return end
64     local h=math.max(0,bot:GetAttribute('Health')-amount); bot:SetAttribute('Health',h); updateHealth(bot)
65     if role(p)=='Technician' and assignments[p]==a then a.disqualified[p]=true;
    ↪ p:SetAttribute('GWAPNotice','Protect your carrier: friendly hits cancel escort rewards.') end
66     if h==0 then
67         if assignments[p]==a and role(p)=='Laboratory Technician' then award(p,50,'+50 | Assigned research
            ↪ carrier stopped') end
68         finish(a,'INTERCEPTED')
69     end
70 end
71
72 local equipmentOriginal={}
73 for _,item in ipairs(stations:GetChildren()) do
74     if item:IsA('BasePart') then
75         local saved={}
76         local function remember(v) if v:IsA('BasePart') then
            ↪ saved[v]={v.Color,v.Material,v.Transparency,v.CanQuery} end end
77         remember(item); for _,v in ipairs(item:GetDescendants()) do remember(v) end
78         equipmentOriginal[item]=saved
79         item:SetAttribute('GWAPEquipment',true)
80     end
81 end
82 local function repair(item)
83     item:SetAttribute('Exploded',false); item:SetAttribute('FunctionalState','RUNNING')
84     for part,props in pairs(equipmentOriginal[item] or {}) do if part.Parent then part.Color=props[1];
    ↪ part.Material=props[2]; part.Transparency=props[3]; part.CanQuery=props[4] end end
85     for _,v in ipairs(item:GetChildren()) do if v.Name=='GWAPFire' or v.Name=='GWAPSmoke' then v:Destroy() end
    ↪ end
86     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=false end
87 end
88 local explode
89 explode=function(item,attacker)
90     if not equipmentOriginal[item] or item:GetAttribute('Exploded') then return end
91     item:SetAttribute('Exploded',true); item:SetAttribute('FunctionalState','DESTROYED')
92     for part in pairs(equipmentOriginal[item]) do if part.Parent then part.Color=Color3.fromRGB(49,42,38);
    ↪ part.Material=Enum.Material.CorrodedMetal end end
93     local fire=Instance.new('Fire'); fire.Name='GWAPFire'; fire.Size=5; fire.Heat=4; fire.Parent=item;
    ↪ Debris:AddItem(fire,8)
94     local smoke=Instance.new('Smoke'); smoke.Name='GWAPSmoke'; smoke.Size=4; smoke.Opacity=.3;
    ↪ smoke.Parent=item; Debris:AddItem(smoke,12)
95     local blast=Instance.new('Explosion'); blast.Position=item.Position; blast.BlastRadius=13;
    ↪ blast.BlastPressure=0; blast.DestroyJointRadiusPercent=0;
    ↪ blast.ExplosionType=Enum.ExplosionType.NoCraters
96     if attacker then blast:SetAttribute('GWAPAttacker',attacker.UserId) end
97     blast.Parent=workspace
98     for _,a in pairs(carriers) do if active(a) then
99         if (a.bot:GetPivot().Position-item.Position).Magnitude<13 then damage(attacker,a.bot,60) end

```

```

100     if a.target==item then tell(a,item.Name..' was destroyed. I will find a working equivalent.');
```

```

101     end end
102     local prompt=item:FindFirstChild('ResearchRepair'); if prompt then prompt.Enabled=true end
103 end
104 for item in pairs(equipmentOriginal) do
105     local prompt=Instance.new('ProximityPrompt'); prompt.Name='ResearchRepair'; prompt.ActionText='Repair
    ↳ research equipment'; prompt.ObjectText=item.Name; prompt.KeyboardKeyCode=Enum.KeyCode.R;
    ↳ prompt.HoldDuration=3; prompt.MaxActivationDistance=12; prompt.Enabled=false; prompt.Parent=item
106     prompt.Triggered:Connect(function(p) local r=alive(p); if r and (r.Position-item.Position).Magnitude<14 and
    ↳ item:GetAttribute('Exploded') then repair(item); p:SetAttribute('GWAPNotice',item.Name..' repaired.
    ↳ Ready for research.') end end)
107 end
108 workspace.DescendantAdded:Connect(function(v)
109     if not v:IsA('Explosion') then return end
110     local center,radius=v.Position,v.BlastRadius
111     local userId=v:GetAttribute('GWAPAttacker'); local attacker=userId and Players:GetPlayerByUserId(userId)
112     -- Deferred bounded propagation: each station can explode only once until repaired.
113     task.delay(.12,function()
114         for item in pairs(equipmentOriginal) do if item.Parent and not item:GetAttribute('Exploded') and
    ↳ (item.Position-center).Magnitude<=radius+item.Size.Magnitude*.25 then
115             local delta=item.Position-center
116             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
    ↳ params.FilterDescendantsInstances={v}
117             local hit=delta.Magnitude>.1 and workspace:Raycast(center,delta,params)
118             if not hit or hit.Instance==item or hit.Instance:IsDescendantOf(item) or
    ↳ hit.Instance:GetAttribute('Exploded') then explode(item,attacker) end
119         end end
120     end)
121 end)
122
123 bridge.OnInvoke=function(p,hit,damageAmount)
124     local node=hit
125     while node and node~=workspace do if equipmentOriginal[node] then explode(node,p); return true end; if
    ↳ carriers[node] then damage(p,node,damageAmount); return true end; node=node.Parent end
126     return false
127 end
128 local function routePoints(from,to)
129     local r={}; local function add(v) table.insert(r,v) end
130     local f1=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
131     local f2=to:GetAttribute('Floor'); local dest=to:GetAttribute('Approach')
132     if f1~=f2 then
133         add(Vector3.new(78,from.Y,from.Z)); add(Vector3.new(78,from.Y,-6))
134         local function flight(level,reverse)
135             local steps={Vector3.new(68.7857,level*16.25+3.25,-6)}
136             for i=1,8 do
    ↳ table.insert(steps,Vector3.new(68.7857,level*16.25+i*1.015625+3.25,-15.267857-(i-1)*4.821428))
    ↳ end
137             table.insert(steps,Vector3.new(68.7857,level*16.25+8.125+3.25,-55.28))
138             table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+3.25,-55.28))
139             for i=1,8 do table.insert(steps,Vector3.new(88.0714,level*16.25+8.125+i*1.015625+3.25,-49.017857+(
    ↳ i-1)*4.821428)) end
140             table.insert(steps,Vector3.new(88.0714,(level+1)*16.25+3.25,-6));
    ↳ table.insert(steps,Vector3.new(78,(level+1)*16.25+3.25,-6))
141             if reverse then for i=#steps,1,-1 do add(steps[i]) end else for _,v in ipairs(steps) do add(v) end
    ↳ end
142         end
143         if f2>f1 then for level=f1,f2-1 do flight(level,false) end else for level=f1-1,f2,-1 do
    ↳ flight(level,true) end end
144     end
145     add(Vector3.new(78,dest.Y,dest.Z)); add(dest); return r
146 end
147 local function showRoute(a,route)
148     local old=routesFolder:FindFirstChild(a.bot.Name); if old then old:Destroy() end
149     local folder=Instance.new('Folder'); folder.Name=a.bot.Name; folder.Parent=routesFolder
150     for i,v in ipairs(route) do local n=Instance.new('Vector3Value'); n.Name=string.format('%03d',i);
    ↳ n.Value=v; n.Parent=folder end
151 end
152 local function go(a,target)
153     local revision=a.routeRevision or 0
154     if not Router.healthy(target) then return false end
155     local route
156     if a.needsRendezvous then
157         local from=a.bot:GetPivot().Position
158         local floor=math.clamp(math.floor((from.Y-3.25)/16.25+.5),-1,8)
159         local entry=Vector3.new(78,floor*16.25+3.25,math.clamp(from.Z,24,240))
160         route={}

```

```

161     if (from-entry).Magnitude>.5 then
162         local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
            ↪ params.FilterDescendantsInstances={a.bot,a.owner and a.owner.Character};
            ↪ params.RespectCanCollide=true
163         local blocked=workspace.Blockcast(CFrame.new(from),Vector3.new(2.5,4.4,2.5),entry-from,params)
164         if not blocked and math.abs(from.Y-entry.Y)<1 then table.insert(route,entry) else
165             local path=Pathfinding.CreatePath({AgentRadius=1.8,AgentHeight=5,AgentCanJump=false,WaypointSp
                ↪ acing=5})
166             local ok=pcall(function() path:ComputeAsync(from,entry) end)
167             if not ok or path.Status~=Enum.PathStatus.Success then tell(a,'I cannot reach the corridor from
                ↪ here. Move to an open area and ask me to regroup.');
```

a.paused=true; return false end

```

168         for _,waypoint in ipairs(path:GetWaypoints()) do
            ↪ table.insert(route,waypoint.Position+Vector3.new(0,3.25,0)) end
169         end
170     end
171     for _,v in ipairs(routePoints(entry,target)) do table.insert(route,v) end
172     a.needsRendezvous=false
173 else route=routePoints(a.bot:GetPivot().Position,target) end
174 showRoute(a,route)
175 for i,goal in ipairs(route) do
176     a.bot:SetAttribute('RouteIndex',i)
177     if not Router.healthy(target) then a.needsRendezvous=true; return false end
178     while active(a) and (goal-a.bot:GetPivot().Position).Magnitude>.12 do
179         local dt=Run.Heartbeat:Wait(); if not active(a) then return false end
180         if (a.routeRevision or 0)~=revision then return false end
181         if not Router.healthy(target) then a.needsRendezvous=true; return false end
182         if a.paused then a.deadline+=dt; continue end
183         local ownerRoot=a.owner and alive(a.owner)
184         if a.owner and role(a.owner)=='Technician' and (not ownerRoot or
            ↪ (ownerRoot.Position-a.bot:GetPivot().Position).Magnitude>45) then
185             a.deadline+=dt
186             if os.clock()-(a.lastWaitMessage or 0)>12 then a.lastWaitMessage=os.clock(); tell(a,'I am
                ↪ waiting for you. Follow the mint trail to my marker, then we can continue.') end
187             continue
188         end
189         local start=a.bot:GetPivot().Position; local delta=goal-start
190         local ownerHum=a.owner and a.owner.Character and
            ↪ a.owner.Character:FindFirstChildOfClass('Humanoid')
191         local desiredSpeed=ownerHum and ownerHum.WalkSpeed or 16
192         a.speed=(a.speed or desiredSpeed)+(desiredSpeed-(a.speed or desiredSpeed))*(1-math.exp(-dt*8))
193         a.bot:SetAttribute('WalkSpeed',a.speed)
194         local nextPos=start+delta.Unit*math.min(delta.Magnitude,math.clamp(a.speed,8,40)*math.min(dt,.1))
195         -- Research routes use measured open corridors and each actual stair tread.
196         -- Stop for players; do not shove them or change their movement settings.
197         local blocked=false
198         for _,p in ipairs(Players:GetPlayers()) do local r=alive(p); if r and
            ↪ (r.Position-nextPos).Magnitude<2.8 then blocked=true end end
199         if not blocked then
200             local flat=Vector3.new(delta.X,0,delta.Z)
201             local previous=a.bot:GetPivot()
202             local desired=flat.Magnitude>.01 and CFrame.lookAt(nextPos,nextPos+flat) or
                ↪ CFrame.new(nextPos)*previous.Rotation
203             a.bot:PivotTo(CFrame.new(nextPos)*previous.Rotation:Lerp(desired.Rotation,1-math.exp(-dt*10)))
204         end
205         if os.clock()>a.deadline then finish(a,'EXPIRED'); return false end
206     end
207     if not active(a) then return false end
208 end
209 return true
210 end
211 local function plantIn(bed,name)
212     if bed:GetAttribute('Exploded') then return false end
213     local count=bed:GetAttribute('Planted') or 0
214     if count>=12 then return false end
215     local pos=bed.Position+Vector3.new(-8+(count%4)*5,1,-3+math.floor(count/4)*3)
216     World.plant(bed,pos,name); bed:SetAttribute('Planted',count+1)
217     local sign=bed:FindFirstChild('StationLabel'); local label=sign and sign:FindFirstChildOfClass('TextLabel')
218     if label then label.Text=bed.Name..' | '..(count+1)..' / 12 plants' end
219     return true
220 end
221 local function checkpoint(a,final)
222     for p,bound in pairs(assignments) do
223         if bound==a and role(p)=='Technician' and not a.disqualified[p] and nearby(p,a.bot,45) then
            ↪ award(p,final and 60 or 15,final and '+60 | Protected rooftop planting' or '+15 | Protected
            ↪ research checkpoint') end
224     end

```

```

225 end
226 local function runMission(a)
227     for i,originalName in ipairs(a.route) do
228         a.stage=i
229         local original=stations:FindFirstChild(originalName)
230         local info=stationInfo[originalName]
231         local capability=info and info.capability or original and original:GetAttribute('Capability') or
        ↳ (originalName:match('^Plot ') and 'Greenhouse planting' or originalName)
232         local preferredFloor=info and info.floor or original and Router.floorOf(original) or 0
233         local completed=false
234         while active(a) and not completed do
235             local target=Router.healthy(a.target) and a.target:GetAttribute('Capability')==capability and
        ↳ a.target
236             or (Router.healthy(original) and original or
        ↳ Router.choose(stations,capability,preferredFloor,a.bot:GetPivot().Position))
237         if not target then
238             a.target=nil; publish(a)
239             if os.clock()-(a.lastNoEquipment or 0)>8 then a.lastNoEquipment=os.clock(); tell(a,'No working
        ↳ '..capability..' is available. Repair one with R; I will resume this assignment.') end
240             a.deadline+=1; task.wait(1); continue
241         end
242         if target~=a.target then
243             a.target=target; publish(a)
244             if target~=original then a.bot:SetAttribute('Reroutes',(a.bot:GetAttribute('Reroutes') or 0)+1)
        ↳ end
245             tell(a,target~=original and ('Rerouting to '..target.Name..' on '..(Router.floorOf(target)==-1
        ↳ and 'B1' or Router.floorOf(target)==8 and 'the roof' or
        ↳ 'L'..(Router.floorOf(target)+1))..' Same research function, new route.')
        ↳ or 'Next stop: '..target.Name..' Stay with me.')
246         end
247         a.bot:SetAttribute('TaskState','Travel to '..target.Name)
248         local reached=go(a,target)
249         if not active(a) then return end
250         if not reached then
251             a.needsRendezvous=true
252             if a.paused then repeat task.wait(.25) until not active(a) or not a.paused end
253             task.wait(.1); continue
254         end
255         tell(a,'At '..target.Name..' Processing the seedling now.')
256         a.bot:SetAttribute('TaskState','Processing: '..target.Name)
257         local seed=a.bot:FindFirstChild('Mission seedling')
258         if seed then seed:PivotTo(CFrame.new(target.Position+Vector3.new(0,2,0))) end
259         local work=0
260         while active(a) and Router.healthy(target) and work<4 do task.wait(.1); if not a.paused then
        ↳ work+=.1 end end
261         if not active(a) then return end
262         if not Router.healthy(target) then a.needsRendezvous=true; continue end
263         if i==#a.route then
264             if plantIn(target,a.title..' | '..a.variant) then checkpoint(a,true); finish(a,'COMPLETE');
        ↳ return end
265             a.target=nil; original=nil; task.wait(.1); continue
266         end
267         if i==1 and not seed then a.bot:SetAttribute('HasSeedling',true);
        ↳ seed=World.plant(a.bot,a.bot:GetPivot().Position+Vector3.new(0,0,-2),'Mission seedling') end
268         if seed then seed:PivotTo(CFrame.new(a.bot:GetPivot().Position+Vector3.new(0,0,-2))) end
269         checkpoint(a,false); completed=true; a.target=nil
270     end
271     if not active(a) then return end
272 end
273 end
274 local function startMission(bot,id,p)
275     local s=scenarios[id]; local route=table.clone(s.route); table.insert(route,'Plot '..id)
276     local a={bot=bot,id=id,title=s.title,route=route,stage=1,disqualified={},owner=p,needsRendezvous=true,paus
        ↳ ed=true,deadline=os.clock()+600}
277     a.variant=s.variable..' '..random:NextInteger(s.low,s.high)..' '..s.units
278     carriers[bot]=a; bot:SetAttribute('State','RUNNING'); bot:SetAttribute('Health',100);
279     ↳ bot:SetAttribute('Scenario',id); bot:SetAttribute('Title',s.title);
        ↳ bot:SetAttribute('Variant',a.variant); updateHealth(bot)
280     bot.Parent=botsFolder
281     if not NPC.nearPlayer(p,bot) then bot.Parent=pool; carriers[bot]=nil; return nil end
282     return a
283 end
284 local function makeBot(index)
285     local template
286     local choices={}

```

```

287   for _,b in ipairs(CS:GetTagged('LabWorkerBot')) do if not b:GetAttribute('Seated') and b.PrimaryPart and
    ↪ ((index%2==1)==(b:GetAttribute('Gender')=='Girl')) then table.insert(choices,b) end end
288   table.sort(choices,function(a,b) return a.Name<b.Name end);
    ↪   template=choices[(index-1)%math.max(1,#choices)+1]
289   assert(template,'No worker template available')
290   local b=template:Clone(); b.Name='ResearchCarrier_'..index
291   for _,v in ipairs(b:GetDescendants()) do if v:IsA('LuaSourceContainer') or v:IsA('ProximityPrompt') or
    ↪ v:IsA('Humanoid') then v:Destroy() elseif v:IsA('BasePart') then v.Anchored=true; v.CanCollide=false
    ↪ end end
292   for _,tag in ipairs(CS:GetTags(b)) do CS:RemoveTag(b,tag) end
293   NPC.appearance(b)
294   b:SetAttribute('Dead',false); b:SetAttribute('KnockedOut',false); b:SetAttribute('GWAPCarrier',true);
    ↪ b.Parent=botsFolder
295   local h=Instance.new('Highlight'); h.Name='ResearchOutline'; h.FillTransparency=1;
    ↪ h.OutlineColor=Color3.fromRGB(92,244,199); h.DepthMode=Enum.HighlightDepthMode.Occluded; h.Parent=b
296   return b
297 end
298 Run.Heartbeat:Connect(function(dt) for _,bot in ipairs(botsFolder:GetChildren()) do if bot:IsA('Model') then
    ↪ NPC.animate(bot,dt) end end end)
299 local available={}; for i=1,5 do available[i]=makeBot(i); available[i].Parent=pool;
    ↪ available[i]:SetAttribute('State','IDLE') end
300 local function companion(p)
301   local bot=companions[p]
302   if bot and bot.Parent then return bot end
303   local reserved={}; for _,b in pairs(companions) do reserved[b]=true end
304   for _,candidate in ipairs(available) do if not reserved[candidate] and not active(carriers[candidate]) then
    ↪ bot=candidate; break end end
305   if not bot then bot=makeBot(#available+1); table.insert(available,bot) end
306   companions[p]=bot; bot.Parent=botsFolder
307   if not NPC.nearPlayer(p,bot) then bot.Parent=pool; p:SetAttribute('GWAPDialogue','Move to an open spot and
    ↪ choose an assignment so I can meet you.');
```

return bot end

```

308   p:SetAttribute('GWAPCarrier',bot.Name); p:SetAttribute('GWAPSpeaker',bot:GetAttribute('DisplayName'));
    ↪ p:SetAttribute('GWAPDialogue','Hello! I am your research carrier. Choose an assignment, then tell me
    ↪ when you are ready.')
```

NPC.say(bot,'Hello! Choose an assignment and I will explain our research route.')

```

309   return bot
310 end
311 end
312 local function choose(p,id)
313   if not alive(p) or workspace:GetAttribute('RoundPhase')=='BREAK' then return end
314   if role(p)=='Technician' and role(p)~='Laboratory Technician' then p:SetAttribute('GWAPNotice','Choose a
    ↪ role at a blue or green kiosk first.');
```

return end

```

315   if active(assignments[p]) then p:SetAttribute('GWAPNotice','Finish your current assignment before selecting
    ↪ another.');
```

return end

```

316   local bot=companion(p)
317   local a=startMission(bot,id,p)
318   if not a then p:SetAttribute('GWAPNotice','Please move to an open spot so your carrier can appear beside
    ↪ you.');
```

return end

```

319   assignments[p]=a; p:SetAttribute('GWAPCarrier',a.bot.Name); p:SetAttribute('GWAPScenario',id);
    ↪ p:SetAttribute('GWAPVariant',a.variant); p:SetAttribute('GWAPNotice',role(p)=='Technician' and 'Escort
    ↪ your carrier. Stay within 45 studs at checkpoints.' or 'Stop your assigned carrier before rooftop
    ↪ planting.');
```

publish(a)

```

320   tell(a,'Our assignment is '..a.title..' '..scenarios[id].purpose..' First stop: '..a.route[1]..' Tell me
    ↪ READY when you want to start.')
```

a.started=true; task.spawn(function() while active(a) and a.paused do task.wait(.25);

```

    ↪ a.deadline=os.clock()+600 end; if active(a) then runMission(a) end end)
322 end
323 remote.OnServerEvent:Connect(function(p,action,index)
324   if typeof(action)~='string' or typeof(index)~='number' or index~index or index%1~=0 then return end
325   if os.clock()-(cooldowns[p] or 0)<.3 then return end; cooldowns[p]=os.clock()
326   if action=='choose' and index>=1 and index<=10 then choose(p,index)
327   elseif action=='reply' then
328     local a=assignments[p]; if not active(a) then return end
329     if index==1 then a.paused=false; tell(a,'Ready. Let us continue to '..(a.route[a.stage] or 'the
        ↪ greenhouse')..'')
330     elseif index==2 then a.paused=true; tell(a,'Understood. I will wait here. Press Ready when you want to
        ↪ continue.')
```

elseif index==3 then tell(a,'We are heading to '..(a.target and a.target.Name or a.route[a.stage])..'')

```

    ↪ '..scenarios[a.id].purpose)
332   elseif index==4 then
333     if not a.paused then tell(a,'Ask me to wait before regrouping.');
```

return end

```

334     if NPC.nearPlayer(p,a.bot) then a.needsRendezvous=true; a.routeRevision=(a.routeRevision or 0)+1;
        ↪ tell(a,'I am beside you again. Press Ready to continue.')
```

end

```

335   end
336 end
337 end)
338 for _,bed in ipairs(stations:GetChildren()) do
```

```

339     local prompt=bed:FindFirstChild('PlantSeedling')
340     if prompt then prompt.Triggered:Connect(function(p)
341         local r=alive(p); if not r or (r.Position-bed.Position).Magnitude>14 then return end
342         local sample
343         for _,holder in ipairs({p.Character,p:FindFirstChildOfClass('Backpack')}) do if holder then for _,t in
            ↳ ipairs(holder:GetChildren()) do if t:IsA('Tool') and (t:GetAttribute('PlantSample') or
            ↳ t:GetAttribute('TransferSample')) then sample=t; break end end end end
344         if not sample then p:SetAttribute('GWAPNotice','Carry a germinated plant sample to this plot.');
```

return

```

            ↳ end
345         if plantIn(bed,sample.Name) then sample:Destroy(); p:SetAttribute('CarriedSample','');
            ↳ p:SetAttribute('GWAPNotice','Seedling planted in '..bed.Name) else
            ↳ p:SetAttribute('GWAPNotice','This plot is full. Use another plot.') end
346     end) end
347 end
348 local function setArmed()
349     local all=CS:GetTagged('LabWorkerBot'); for _,bot in ipairs(available) do table.insert(all,bot) end
350     table.sort(all,function(a,b) return a.Name<b.Name end)
351     armed={}; local solo=#Players:GetPlayers()==1; local count=solo and math.floor(#all*.2) or 0
352     local selected={}; local shuffle=table.clone(all); for i=#shuffle,2,-1 do local j=random:NextInteger(1,i);
            ↳ shuffle[i],shuffle[j]=shuffle[j],shuffle[i] end
353     for i=1,count do selected[shuffle[i]]=true end
354     for _,bot in ipairs(all) do
355         local yes=selected[bot]==true; bot:SetAttribute('GWAPArmed',yes)
356         local gun=bot:FindFirstChild('SoloWeapon'); if gun then gun:Destroy() end
357         if yes then
358             table.insert(armed,bot); NPC.gun(bot,(bot:GetAttribute('BotNumber') or 1))
359         end
360     end
361     workspace:SetAttribute('GWAPArmedCount',count); workspace:SetAttribute('GWAPBotCount',#all)
362 end
363 local retired={['Research SMG']=true,['Precision Rifle']=true,['Scatter Blaster']=true,['Blast
            ↳ Launcher']=true,['Pulse Bomb']=true}
364 local function watchPlayer(p)
365     local function clean(v) if v:IsA('Tool') and (v:GetAttribute('GWAPWeapon') or retired[v.Name]) then
            ↳ v:Destroy() end end
366     local function watch(container) for _,v in ipairs(container:GetChildren()) do clean(v) end;
            ↳ container.ChildAdded:Connect(clean) end
367     task.spawn(function() watch(p:WaitForChild('Backpack')) end)
368     local function onCharacter(c)
369         watch(c)
370         task.delay(1,function() if role(p)=='Technician' or role(p)=='Laboratory Technician' then
371             local a=assignments[p]; if active(a) then a.paused=true; tell(a,'You are back. Ask me to regroup,
            ↳ then press Ready.') else companion(p) end
372         end end)
373     end
374     if p.Character then onCharacter(p.Character) end; p.CharacterAdded:Connect(onCharacter)
375     p:GetAttributeChangedSignal('LabRole'):Connect(function() if role(p)=='Technician' or role(p)=='Laboratory
            ↳ Technician' then companion(p) end end)
376     task.defer(setArmed)
377 end
378 Players.PlayerAdded:Connect(watchPlayer)
379 for _,p in ipairs(Players:GetPlayers()) do watchPlayer(p) end
380 for _,bot in ipairs(CS:GetTagged('LabWorkerBot')) do NPC.appearance(bot) end
381 Players.PlayerRemoving:Connect(function(p) local a=assignments[p]; if active(a) then finish(a,'PLAYER LEFT')
            ↳ end; local bot=companions[p]; if bot then bot.Parent=pool end; companions[p]=nil; assignments[p]=nil;
            ↳ cooldowns[p]=nil; task.defer(setArmed) end)
382 setArmed()
383 task.spawn(function()
384     while task.wait(2.5) do
385         local list=Players:GetPlayers(); if #list~=1 or workspace:GetAttribute('RoundPhase')=='BREAK' then
            ↳ continue end
386         local p=list[1]; local root=alive(p); if not root then continue end
387         for _,bot in ipairs(armed) do
388             if not bot:IsDescendantOf(workspace) or bot:GetAttribute('Dead') or bot:GetAttribute('KnockedOut')
            ↳ or (bot:GetAttribute('Health') or 100)<=0 then continue end
389             -- Assigned escort carriers never attack their own defender.
390             if assignments[p] and assignments[p].bot==bot and role(p)=='Technician' then continue end
391             local gun=bot:FindFirstChild('SoloWeapon'); local origin=gun and
            ↳ (gun.Position+gun.CFrame.LookVector*(-gun.Size.Z*.5)) or
            ↳ bot:GetPivot().Position+Vector3.new(0,1,0); local delta=root.Position-origin
392             if delta.Magnitude<4 or delta.Magnitude>85 then continue end
393             local params=RaycastParams.new(); params.FilterType=Enum.RaycastFilterType.Exclude;
            ↳ params.FilterDescendantsInstances={bot}
394             local hit=workspace:Raycast(origin,delta,params)
395             if hit and hit.Instance:IsDescendantOf(p.Character) then
```



```

396         local spread=Vector3.new(random:NextNumber(-2.5,2.5),random:NextNumber(-1.5,1.5),random:NextNu
        ↪ mber(-2.5,2.5))
397         local shot=workspace:Raycast(origin,delta+spread,params); local endpoint=shot and shot.Position
        ↪ or origin+delta+spread
398         local beam=Instance.new('Part'); beam.Name='NPC shot'; beam.Anchored=true;
        ↪ beam.CanCollide=false; beam.CanQuery=false; beam.Material=Enum.Material.Neon;
        ↪ beam.Color=Color3.fromRGB(255,105,72);
        ↪ beam.Size=Vector3.new(.07,.07,(endpoint-origin).Magnitude);
        ↪ beam.CFrame=CFrame.lookAt((origin+endpoint)/2,endpoint); beam.Parent=workspace;
        ↪ Debris:AddItem(beam,.18)
399         if shot and shot.Instance:IsDescendantOf(p.Character) then
400             local v=points(p); if v then
401                 -- Existing score is an IntValue: accumulate fractional losses, never turn 1% into a
        ↪ minimum one-point tax.
402                 local debt=(p:GetAttribute('GWAPFractionalLoss') or 0)+math.max(0,v.Value)*.01
403                 local loss=math.floor(debt+1e-8); p:SetAttribute('GWAPFractionalLoss',debt-loss);
        ↪ v.Value=math.max(0,v.Value-loss)
404                 p:SetAttribute('GWAPNotice','NPC hit: -1% score');
        ↪ p:SetAttribute('GWAPHitAt',workspace:GetServerTimeNow())
405             end
406         end
407     end
408 end
409 end
410 end)
411 workspace:GetAttributeChangedSignal('RoundNumber'):Connect(function()
412     for _,a in pairs(carriers) do if active(a) then finish(a,'SHIFT ENDED') end end
413     for _,p in ipairs(Players:GetPlayers()) do p:SetAttribute('GWAPFractionalLoss',0) end
414     for item in pairs(equipmentOriginal) do repair(item) end
415     for _,bed in ipairs(stations:GetChildren()) do if bed:GetAttribute('Planted') then for _,v in
        ↪ ipairs(bed:GetChildren()) do if v:IsA('Model') then v:Destroy() end end; bed:SetAttribute('Planted',0);
        ↪ local sign=bed:FindFirstChild('StationLabel'); local label=sign and
        ↪ sign:FindFirstChildOfClass('TextLabel'); if label then label.Text=bed.Name..' | Ready for planting' end
        ↪ end end
416     setArmed()
417 end)
418 print('[GWAP] 10 scenarios, 5 research carriers, rooftop planting and solo opposition ready')

```

Chapter 65

W08 - GWAPWallDamage.lua

GWAPWallDamage.lua

Authority group: workspace. 2594 UTF-8 bytes; 45 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Remove fixtures belonging to a wall section destroyed by an explosion.
2 local CS=game:GetService('CollectionService')
3 local c=workspace:WaitForChild('Laboratory_Complex')
4 local debris=Instance.new('Folder'); debris.Name='DestroyedWallFixtures';
5   ↳ debris.Parent=game:GetService('ServerStorage')
6 local fixtures={}; local recent={}
7 local function register(v)
8   if not v:IsA('BasePart') then return end
9   if v:GetAttribute('ToolKind') or v:GetAttribute('ShootablePoster') or v.Name:match('^HealthStation')
10    or CS:HasTag(v,'LabEquipment') or CS:HasTag(v,'PlantGrowthChamber') or v:GetAttribute('GWAPEquipment')
11    or (v.Parent and v.Parent.Name=='Stations') then fixtures[v]=true end
12 end
13 for _,v in ipairs(c:GetDescendants()) do register(v) end
14 c.DescendantAdded:Connect(register)
15 CS.GetInstanceAddedSignal('LabEquipment'):Connect(register)
16 workspace.DescendantAdded:Connect(function(v)
17   if v:IsA('Explosion') then table.insert(recent,{position=v.Position,radius=v.BlastRadius,time=os.clock()})
18   ↳ end
19   while #recent>0 and os.clock()-recent[1].time>3 do table.remove(recent,1) end
20 end)
21 c.DescendantRemoving:Connect(function(wall)
22   if not wall:IsA('BasePart') or not (wall.Name:match('^Corridor_') or wall.Name:match('^Div_') or
23   ↳ wall.Name:match('^Ext_') or wall.Name:match('^B_')) then return end
24   local cf,size=wall.CFrame,wall.Size
25   local exploded=false
26   for _,blast in ipairs(recent) do if os.clock()-blast.time<3 then
27     local localPoint=cf:PointToObjectSpace(blast.position)
28     local closest=Vector3.new(math.clamp(localPoint.X,-size.X/2,size.X/2),math.clamp(localPoint.Y,-size.Y/2),
29     ↳ 2,size.Y/2),math.clamp(localPoint.Z,-size.Z/2,size.Z/2))
30     if (localPoint-closest).Magnitude<=blast.radius then exploded=true; break end
31   end end
32   if not exploded then return end
33   task.defer(function()
34     if not c.Parent then return end
35     local pad=Vector3.new(size.X<4 and 7 or 1,1,size.Z<4 and 7 or 1)
36     local half=size/2+pad
37     local count=0
38     for item in pairs(fixtures) do if item:IsDescendantOf(c) then
39       local pos=cf:PointToObjectSpace(item.Position)
40       if math.abs(pos.X)<=half.X and math.abs(pos.Y)<=half.Y and math.abs(pos.Z)<=half.Z then
41         item:SetAttribute('BlastRemoved',true); item:SetAttribute('FunctionalState','DESTROYED');
42         ↳ item:SetAttribute('Exploded',true)
43         for _,v in ipairs(item:GetDescendants()) do if v:IsA('ProximityPrompt') then v.Enabled=false
44         ↳ end end
45         for _,tag in ipairs(CS:GetTags(item)) do CS:RemoveTag(item,tag) end
46         item.Parent=debris; count+=1
```

```
41         end
42     end end
43     workspace:SetAttribute('GWAPRemovedFixtures',(workspace:GetAttribute('GWAPRemovedFixtures') or
↪     0)+count)
44 end)
45 end)
```

Chapter 66

W09 - GWAPHUD.lua

GWAPHUD.lua

Authority group: workspace. 11596 UTF-8 bytes; 123 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Two grouped panels. Native inventory and movement/fire controls remain available.
2 local Players=game:GetService('Players')
3 local RS=game:GetService('ReplicatedStorage')
4 local Run=game:GetService('RunService')
5 local StarterGui=game:GetService('StarterGui')
6 local prompts=game:GetService('ProximityPromptService')
7 local p=Players.LocalPlayer; local pg=p:WaitForChild('PlayerGui')
8 local scenarios=require(RS:WaitForChild('GWAPScenarios')); local remote=RS:WaitForChild('GWAPAction')
9 local research=workspace:WaitForChild('Laboratory_Complex'):WaitForChild('GWAPResearch')
10 local gui=Instance.new('ScreenGui'); gui.Name='GWAPHUD'; gui.ResetOnSpawn=false; gui.IgnoreGuiInset=false;
11     ↳ gui.DisplayOrder=25; gui.Parent=pg
12 pcall(function() StarterGui:SetCoreGuiEnabled(Enum.CoreGuiType.Health,false) end)
13 local white=Color3.fromRGB(239,247,243)
14 local function box(name,parent)
15     local f=Instance.new('Frame'); f.Name=name; f.BackgroundColor3=Color3.fromRGB(23,31,37);
16     ↳ f.BackgroundTransparency=.12; f.BorderSizePixel=0; f.Parent=parent or gui;
17     ↳ Instance.new('UICorner',f).CornerRadius=UDim.new(0,8); return f
18 end
19 local function label(parent,name,size)
20     local t=Instance.new('TextLabel'); t.Name=name; t.BackgroundColor3=Color3.fromRGB(23,31,37);
21     ↳ t.TextColor3=white; t.TextSize=size or 13; t.TextWrapped=true; t.TextTruncate=Enum.TextTruncate.AtEnd;
22     ↳ t.TextXAlignment=Enum.TextXAlignment.Left; t.TextYAlignment=Enum.TextYAlignment.Center;
23     ↳ t.Parent=parent; return t
24 end
25 local function button(parent,name,callback)
26     local t=Instance.new('TextButton'); t.Name=name; t.Text=name; t.Font=Enum.Font.GothamBold;
27     ↳ t.TextColor3=white; t.TextSize=11; t.BackgroundColor3=Color3.fromRGB(63,73,91); t.BorderSizePixel=0;
28     ↳ t.Parent=parent; Instance.new('UICorner',t).CornerRadius=UDim.new(0,5); t.Activated:Connect(callback);
29     ↳ return t
30 end
31 local strip=box('Round and player'); strip.AnchorPoint=Vector2.new(.5,0)
32 local teams=label(strip,'Team scores',13); teams.TextXAlignment=Enum.TextXAlignment.Center
33 local timer=label(strip,'Timer',18); timer.TextXAlignment=Enum.TextXAlignment.Center
34 local personal=label(strip,'Player',12); personal.TextXAlignment=Enum.TextXAlignment.Right
35 local health=box('Health',strip); health.BackgroundColor3=Color3.fromRGB(54,66,61)
36 local fill=box('Fill',health); fill.BackgroundColor3=Color3.fromRGB(100,235,68); fill.BackgroundTransparency=0;
37     ↳ fill.Size=UDim2.fromScale(1,1)
38 local panel=box('Mission and conversation')
39 local heading=label(panel,'Heading',13); heading.Font=Enum.Font.GothamBold
40 local target=label(panel,'Destination',12); target.TextColor3=Color3.fromRGB(151,238,204)
41 local dialogue=label(panel,'Carrier message',12); dialogue.TextYAlignment=Enum.TextYAlignment.Top
42 local note=label(panel,'Status',10); note.TextColor3=Color3.fromRGB(216,202,158);
43     ↳ note.TextTruncate=Enum.TextTruncate.AtEnd; note.TextWrapped=false
44 local menu=box('Scenario selection'); menu.AnchorPoint=Vector2.new(.5,.5); menu.Visible=false; menu.ZIndex=10
45 local function toggle(value) menu.Visible=value; panel.Visible=not value; strip.Visible=not value;
46     ↳ prompts.Enabled=not value end
```

```

35 local selectButton=button(panel,'Jobs',function() toggle(true) end)
36 local buttons={}
37 for i,name in ipairs({'Ready','Wait','Status','Regroup'}) do buttons[i]=button(panel,name,function()
  ↪ remote:FireServer('reply',i) end) end
38 local menuHeading=label(menu,'Select assignment',16); menuHeading.Text='CHOOSE A RESEARCH ASSIGNMENT';
  ↪ menuHeading.Position=UDim2.fromOffset(12,6); menuHeading.Size=UDim2.new(1,-58,0,28)
39 local close=button(menu,'X',function() toggle(false) end); close.Position=UDim2.new(1,-38,0,6);
  ↪ close.Size=UDim2.fromOffset(30,28)
40 local list=Instance.new('ScrollingFrame'); list.Name='Assignments'; list.Position=UDim2.fromOffset(10,42);
  ↪ list.Size=UDim2.new(1,-20,1,-52); list.BackgroundTransparency=1; list.BorderSizePixel=0;
  ↪ list.ScrollBarThickness=5; list.CanvasSize=UDim2.fromOffset(0,11*46); list.Parent=menu
41 local ids={}; for i=1,10 do ids[i]=i end; local random=Random.new(); for i=10,2,-1 do local
  ↪ j=random:NextInteger(1,i); ids[i],ids[j]=ids[j],ids[i] end
42 for row,id in ipairs(ids) do local b=button(list,scenarios[id].title,function() remote:FireServer('choose',id);
  ↪ toggle(false) end); b.Position=UDim2.fromOffset(0,(row-1)*46); b.Size=UDim2.new(1,-8,0,40); b.TextSize=13
  ↪ end
43 local randomButton=button(list,'Random assignment',function()
  ↪ remote:FireServer('choose',random:NextInteger(1,10)); toggle(false) end);
  ↪ randomButton.Position=UDim2.fromOffset(0,460); randomButton.Size=UDim2.new(1,-8,0,40)
44 local trail=Instance.new('Folder'); trail.Name='LocalMissionTrail'; trail.Parent=workspace
45 local dots={}
46 for i=1,90 do local dot=Instance.new('Part'); dot.Name='Direction'; dot.Size=Vector3.new(.16,.12,1.6);
  ↪ dot.Anchored=true; dot.CanCollide=false; dot.CanQuery=false; dot.CanTouch=false;
  ↪ dot.Material=Enum.Material.Neon; dot.Color=Color3.fromRGB(103,255,198); dot.Transparency=1;
  ↪ dot.Parent=trail; dots[i]=dot end
47 local function showTrail(bot)
48   for _,v in ipairs(dots) do v.Transparency=1 end
49   local route=bot and research.Routes:FindFirstChild(bot.Name); local root=p.Character and
  ↪ p.Character:FindFirstChild('HumanoidRootPart')
50   if not bot or not route or not root or bot:GetAttribute('State')~='RUNNING' then return end
51   local nodes=route:GetChildren(); table.sort(nodes,function(a,b) return a.Name<b.Name end)
52   local previous=bot:GetPivot().Position; local n=0
53   for j=bot:GetAttribute('RouteIndex') or 1,#nodes do
54     local to=nodes[j].Value; local delta=to-previous
55     if delta.Magnitude>.1 then for d=0,delta.Magnitude,4 do
56       local pos=previous+delta.Unit*d-Vector3.new(0,2.9,0)
57       if (pos-root.Position).Magnitude<110 and math.abs(pos.Y-root.Position.Y)<10 then n+=1; if n>#dots
  ↪ then return end; dots[n].CFrame=CFrame.lookAt(pos,pos+delta); dots[n].Transparency=.12 end
58     end end; previous=to
59   end
60 end
61 prompts.PromptShown:Connect(function(prompt)
62   local viewport=workspace.CurrentCamera.ViewportSize
63   prompt.UIOffset=Vector2.new(viewport.X<1000 and 70 or 0,0)
64 end)
65 local billboards=setmetatable({}, {__mode='k'})
66 local function watchBot(bot)
67   local items={}; for _,v in ipairs(bot:GetDescendants()) do if v:IsA('BillboardGui') then
  ↪ table.insert(items,v) end end; billboards[bot]=items
68   bot.DescendantAdded:Connect(function(v) if v:IsA('BillboardGui') then table.insert(items,v) end end)
69 end
70 local cs=game.GetService('CollectionService')
71 for _,bot in ipairs(cs:GetTagged('LabWorkerBot')) do watchBot(bot) end
72 for _,bot in ipairs(research.Carriers:GetChildren()) do watchBot(bot) end
73 research.Carriers.ChildAdded:Connect(watchBot)
74 local elapsed=0
75 Run.RenderStepped:Connect(function(dt)
76   elapsed+=dt; if elapsed<.15 then return end; elapsed=0
77   local v=workspace.CurrentCamera.ViewportSize; local mobile=v.X<1000 or v.Y<500
78   local width=math.min(mobile and 180 or 290,v.X*.25); local height=mobile and 150 or 218
79   local scale=math.min(1,(v.Y-95)/height); scale=math.max(.6,scale)
80   local touch=pg:FindFirstChild('TouchGui'); local thumb=touch and
  ↪ touch:FindFirstChild('ThumbstickStart',true)
81   if mobile and thumb and thumb.Visible then
  ↪ scale=math.min(scale,math.max(.6,(thumb.AbsolutePosition.Y-48-8)/height)) end
82   local ui=panel:FindFirstChildOfClass('UIScale') or Instance.new('UIScale',panel); ui.Scale=scale
83   panel.Position=UDim2.fromOffset(8,48); panel.Size=UDim2.fromOffset(width/scale,height)
84   local w=width/scale
85   heading.Position=UDim2.fromOffset(8,6); heading.Size=UDim2.fromOffset(w-59,mobile and 25 or 36);
  ↪ heading.TextSize=mobile and 11 or 15
86   selectButton.Position=UDim2.fromOffset(w-46,7); selectButton.Size=UDim2.fromOffset(38,24)
87   target.Position=UDim2.fromOffset(8,mobile and 33 or 46); target.Size=UDim2.fromOffset(w-16,mobile and 26 or
  ↪ 34); target.TextSize=mobile and 10 or 13
88   dialogue.Position=UDim2.fromOffset(8,mobile and 61 or 84); dialogue.Size=UDim2.fromOffset(w-16,mobile and
  ↪ 45 or 69); dialogue.TextSize=mobile and 10 or 13

```

```

89     for i,b in ipairs(buttons) do b.Position=UDim2.fromOffset(8+(i-1)*(w-16)/4,height-43);
    ↪   b.Size=UDim2.fromOffset((w-20)/4-2,23); b.TextSize=mobile and 9 or 11 end
90     note.Position=UDim2.fromOffset(8,height-18); note.Size=UDim2.fromOffset(w-16,14); note.TextSize=mobile and
    ↪   9 or 11
91     local sw=math.min(460,v.X-26); strip.Position=UDim2.new(.5,0,0,3); strip.Size=UDim2.fromOffset(sw,38)
92     teams.Position=UDim2.fromOffset(7,3); teams.Size=UDim2.fromOffset(sw*.35,30); teams.TextSize=mobile and 10
    ↪   or 13
93     timer.Position=UDim2.fromOffset(sw*.38,0); timer.Size=UDim2.fromOffset(sw*.19,35)
94     personal.Position=UDim2.fromOffset(sw*.59,0); personal.Size=UDim2.fromOffset(sw*.39-8,23);
    ↪   personal.TextSize=mobile and 10 or 12
95     health.Position=UDim2.fromOffset(sw*.61,26); health.Size=UDim2.fromOffset(sw*.35,6)
96     menu.Position=UDim2.fromScale(.5,.5);
    ↪   menu.Size=UDim2.fromOffset(math.min(430,v.X-30),math.max(150,math.min(520,v.Y-80)))
97     local own=p.Character and p.Character:FindFirstChild('LabHealthBar',true); if own then own.Enabled=false
    ↪   end
98     for _,name in ipairs({'RoleHUD','LabLog'}) do local old=pg:FindFirstChild(name); if old then
    ↪   old.Enabled=false end end
99     local sums={Technician=0,['Laboratory Technician']=0}
100    for _,player in ipairs(Players:GetPlayers()) do local role=player:GetAttribute('LabRole'); local
    ↪   stats=player:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); if
    ↪   sums[role] then sums[role]+=value and value.Value or 0 end end
101    teams.Text='PROTECT '..sums.Technician..' | STOP '..sums['Laboratory Technician']
102    local remaining=math.max(0,(workspace:GetAttribute('RoundEndsAt') or os.time())-os.time());
    ↪   timer.Text=string.format('%d:%02d',remaining//60,remaining%60)
103    local stats=p:FindFirstChild('leaderstats'); local value=stats and stats:FindFirstChild('Points'); local
    ↪   hum=p.Character and p.Character:FindFirstChildOfClass('Humanoid')
104    personal.Text=(value and value.Value or 0)..' pts | '..math.ceil(hum and hum.Health or 0)..' HP';
    ↪   fill.Size=UDim2.fromScale(hum and math.clamp(hum.Health/math.max(1,hum.MaxHealth),0,1) or 0,1)
105    local id=p:GetAttribute('GWAPScenario'); local s=id and scenarios[id]; local
    ↪   name=p:GetAttribute('GWAPCarrier'); local bot=name and research.Carriers:FindFirstChild(name)
106    heading.Text=s and s.title or 'RESEARCH ASSIGNMENT'
107    local floor=p:GetAttribute('GWAPTargetFloor'); local location=floor and (floor==8 and 'R00F' or floor==-1
    ↪   and 'B1' or 'L'..(floor+1)) or ''
108    target.Text=s and ((p:GetAttribute('GWAPTarget') or '')..' '..location) or 'Choose a role kiosk, then tap
    ↪   Jobs.'
109    dialogue.Text=p:GetAttribute('GWAPDialogue') or 'Your carrier will meet you nearby and wait for your Ready
    ↪   reply.'
110    local carried=p:GetAttribute('CarriedSample') or ''; local notice=p:GetAttribute('GWAPNotice') or ''
111    note.Text=carried~='' and 'Carrying: '..carried or notice
112    local cam=workspace.CurrentCamera; local params=RaycastParams.new();
    ↪   params.FilterType=Enum.RaycastFilterType.Exclude; params.FilterDescendantsInstances={p.Character}
113    local aim=workspace.Raycast(cam.CFrame.Position,cam.CFrame.LookVector*80,params)
114    for npc,items in pairs(billboards) do
115        local show=np.Parent and (npc==bot or (aim and aim.Instance:IsDescendantOf(npc))) and
    ↪   (npc:GetPivot().Position-cam.CFrame.Position).Magnitude<65
116        for _,billboard in ipairs(items) do if billboard.Parent then billboard.Enabled=show==true and
    ↪   billboard.Name~='MissionSpeech'; billboard.MaxDistance=65 end end
117    end
118    for _,station in ipairs(research.Stations:GetChildren()) do
119        local sign=station:FindFirstChild('StationLabel')
120        if sign then sign.Enabled=aim~=nil and (aim.Instance==station or aim.Instance:IsDescendantOf(station))
    ↪   and (station.Position-cam.CFrame.Position).Magnitude<40 end
121    end
122    showTrail(bot)
123 end)

```

Chapter 67

W10 - Blaster.lua

Blaster.lua

Authority group: workspace. 8432 UTF-8 bytes; 227 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- Blaster
2 -- Shared, device-independent camera casting and client shot presentation.
3
4 local Debris = game:GetService("Debris")
5 local ReplicatedStorage = game:GetService("ReplicatedStorage")
6 local RunService = game:GetService("RunService")
7
8 local Blaster = {}
9
10 Blaster.REMOTE_FOLDER_NAME = "BlasterRemotes"
11 Blaster.FIRE_REMOTE_NAME = "Fire"
12 Blaster.EFFECT_REMOTE_NAME = "ReplicateShot"
13
14 local replicationConnection
15
16 local function finiteNumber(value)
17     return typeof(value) == "number"
18         and value == value
19         and value > -1e7
20         and value < 1e7
21 end
22
23 function Blaster.IsFiniteVector(value)
24     return typeof(value) == "Vector3"
25         and finiteNumber(value.X)
26         and finiteNumber(value.Y)
27         and finiteNumber(value.Z)
28 end
29
30 local function numberAttribute(tool, name, fallback, minimum, maximum)
31     local value = tool and tool:GetAttribute(name)
32     if not finiteNumber(value) then value = fallback end
33     return math.clamp(value, minimum, maximum)
34 end
35
36 function Blaster.GetWeaponConfig(tool)
37     return {
38         damage = numberAttribute(tool, "BlasterDamage", 25, 0, 1000),
39         range = numberAttribute(tool, "BlasterRange", 250, 8, 1000),
40         cooldown = numberAttribute(tool, "BlasterCooldown", 0.35, 0.06, 10),
41         pellets = math.floor(numberAttribute(tool, "BlasterPellets", 1, 1, 16)),
42         spread = numberAttribute(tool, "BlasterSpread", 0, 0, 0.35),
43         selfCostPercent = numberAttribute(tool, "BlasterSelfCostPercent", 0, 0, 25),
44         adsFov = numberAttribute(tool, "BlasterADSFOV", 55, 25, 100),
45         automatic = tool and tool:GetAttribute("BlasterAutomatic") == true or false,
46         instantHeadshot = tool and tool:GetAttribute("BlasterInstantHeadshot") == true or false,
```

```

47     allowPlayerDamage = tool and tool:GetAttribute("BlasterAllowPlayerDamage") == true or false,
48 }
49 end
50
51 function Blaster.GetCameraRay(camera, maxRange, screenPoint)
52     camera = camera or workspace.CurrentCamera
53     if not camera then return nil end
54     maxRange = math.clamp(tonumber(maxRange) or 500, 1, 2000)
55     if typeof(screenPoint) == "Vector2" then
56         -- Desktop points are expressed in CoreUISafeInsets screen space (the
57         -- mouse location minus GuiService:GetGuiInset()). ScreenPointToRay is
58         -- the matching inverse transform. ViewportPointToRay uses device-safe
59         -- coordinates instead and produces a vertical offset on PC and in the
60         -- phone simulator.
61         local ray = camera:ScreenPointToRay(screenPoint.X, screenPoint.Y)
62         if ray.Direction.Magnitude < 0.99 then return nil end
63         return ray.Origin, ray.Direction.Unit, ray.Direction.Unit * maxRange
64     end
65
66     local direction = camera.CFrame.LookVector
67     if direction.Magnitude < 0.99 then return nil end
68
69     return camera.CFrame.Position, direction.Unit, direction.Unit * maxRange
70 end
71
72 function Blaster.NewRaycastParams(ignoreList)
73     local params = RaycastParams.new()
74     params.FilterType = Enum.RaycastFilterType.Exclude
75     params.FilterDescendantsInstances = ignoreList or {}
76     params.IgnoreWater = true
77     return params
78 end
79
80 function Blaster.CastFromCamera(camera, maxRange, ignoreList, screenPoint)
81     local origin, unitDirection, castDirection = Blaster.GetCameraRay(camera, maxRange, screenPoint)
82     if not origin then return nil end
83
84     local result = workspace:Raycast(origin, castDirection, Blaster.NewRaycastParams(ignoreList))
85     return {
86         origin = origin,
87         direction = unitDirection,
88         result = result,
89         position = result and result.Position or (origin + castDirection),
90     }
91 end
92
93 -- Returns the exact centre-line solution used for both the live reticle and a
94 -- shot. The camera chooses the intended point; the muzzle ray then catches a
95 -- nearby wall or prop that physically blocks the weapon.
96 function Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
97     if not tool or not tool:IsA("Tool") then return nil end
98     local config = Blaster.GetWeaponConfig(tool)
99     local cast = Blaster.CastFromCamera(camera, config.range, ignoreList, screenPoint)
100     if not cast then return nil end
101
102     local handle = tool:FindFirstChild("Handle")
103     local muzzleOrigin = handle and handle.Position or cast.origin
104     local muzzleDelta = cast.position - muzzleOrigin
105     local muzzleResult
106     if muzzleDelta.Magnitude > 0.05 then
107         -- Extend just beyond the camera hit point. Ending exactly on a surface can
108         -- miss it because of floating-point/raycast endpoint precision.
109         muzzleResult = workspace:Raycast(
110             muzzleOrigin,
111             muzzleDelta.Unit * (muzzleDelta.Magnitude + 0.25),
112             Blaster.NewRaycastParams(ignoreList)
113         )
114     end
115
116     return {
117         cameraOrigin = cast.origin,
118         direction = cast.direction,
119         muzzleOrigin = muzzleOrigin,
120         position = muzzleResult and muzzleResult.Position or cast.position,
121         result = muzzleResult,
122         cameraResult = cast.result,

```



```

123     }
124 end
125
126 local function tracerColor(tool)
127     local value = tool and tool:GetAttribute("BlasterTracerColor")
128     return typeof(value) == "Color3" and value or Color3.fromRGB(255, 226, 140)
129 end
130
131 function Blaster.DrawTracer(origin, hitPosition, color, lifetime)
132     if not Blaster.IsFiniteVector(origin) or not Blaster.IsFiniteVector(hitPosition) then return end
133     local distance = (hitPosition - origin).Magnitude
134     if distance < 0.05 then return end
135
136     local beam = Instance.new("Part")
137     beam.Name = "BlasterTracer"
138     beam.Size = Vector3.new(0.055, 0.055, distance)
139     beam.CFrame = CFrame.lookAt((origin + hitPosition) * 0.5, hitPosition)
140     beam.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
141     beam.Material = Enum.Material.Neon
142     beam.Transparency = 0.08
143     beam.Anchored = true
144     beam.CanCollide = false
145     beam.CanTouch = false
146     beam.CanQuery = false
147     beam.CastShadow = false
148     beam.Parent = workspace
149     Debris.AddItem(beam, lifetime or 0.075)
150 end
151
152 function Blaster.DrawImpact(position, normal, color, lifetime)
153     if not Blaster.IsFiniteVector(position) or not Blaster.IsFiniteVector(normal) then return end
154
155     local impact = Instance.new("Part")
156     impact.Name = "BlasterImpact"
157     impact.Shape = Enum.PartType.Ball
158     impact.Size = Vector3.new(0.16, 0.16, 0.16)
159     impact.Position = position + normal * 0.04
160     impact.Color = typeof(color) == "Color3" and color or Color3.fromRGB(255, 226, 140)
161     impact.Material = Enum.Material.Neon
162     impact.Anchored = true
163     impact.CanCollide = false
164     impact.CanTouch = false
165     impact.CanQuery = false
166     impact.CastShadow = false
167     impact.Parent = workspace
168     Debris.AddItem(impact, lifetime or 0.65)
169 end
170
171 local function getRemotes(timeout)
172     local folder = ReplicatedStorage.WaitForChild(Blaster.REMOTE_FOLDER_NAME, timeout or 10)
173     if not folder then return nil end
174     local fireRemote = folder.WaitForChild(Blaster.FIRE_REMOTE_NAME, timeout or 10)
175     local effectRemote = folder.WaitForChild(Blaster.EFFECT_REMOTE_NAME, timeout or 10)
176     return fireRemote, effectRemote
177 end
178
179 function Blaster.StartReplication()
180     if not RunService.IsClient() or replicationConnection then return replicationConnection end
181     local _, effectRemote = getRemotes(15)
182     if not effectRemote or not effectRemote:IsA("RemoteEvent") then return nil end
183
184     replicationConnection = effectRemote.OnClientEvent:Connect(function(payload)
185         if typeof(payload) ~= "table" then return end
186         if not Blaster.IsFiniteVector(payload.origin) or not Blaster.IsFiniteVector(payload.position) then
187             ↵ return end
188         Blaster.DrawTracer(payload.origin, payload.position, payload.color, 0.075)
189         if Blaster.IsFiniteVector(payload.normal) then
190             Blaster.DrawImpact(payload.position, payload.normal, payload.color)
191         end
192     end)
193     return replicationConnection
194 end
195
196 function Blaster.Fire(tool, camera, sequence, screenPoint)
197     assert(RunService.IsClient(), "Blaster.Fire can only be called by a client")
198     if not tool or not tool:IsA("Tool") then return nil end

```

```
198
199     local player = game:GetService("Players").LocalPlayer
200     local character = player and player.Character
201     if not character or tool.Parent ~= character then return nil end
202
203     local ignoreList = {character, tool, workspace.CurrentCamera}
204     local solution = Blaster.GetAimSolution(tool, camera, ignoreList, screenPoint)
205     if not solution then return nil end
206
207     Blaster.DrawTracer(solution.muzzleOrigin, solution.position, tracerColor(tool), 0.11)
208     if solution.result then
209         Blaster.DrawImpact(solution.position, solution.result.Normal, tracerColor(tool), 0.8)
210     end
211
212     local fireRemote = getRemotes(10)
213     if fireRemote and fireRemote:IsA("RemoteEvent") then
214         fireRemote:FireServer({
215             version = 1,
216             tool = tool,
217             cameraOrigin = solution.cameraOrigin,
218             direction = solution.direction,
219             sequence = sequence,
220             clientTime = workspace:GetServerTimeNow(),
221         })
222     end
223
224     return solution
225 end
226
227 return Blaster
```

Chapter 68

W11 - BlasterController.lua

BlasterController.lua

Authority group: workspace. 35188 UTF-8 bytes; 905 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- BlasterController
2 -- Local Tool controller: ContextActionService input, mobile controls, ADS,
3 -- touch aim damping, camera recoil, crosshair, and weapon bob.
4
5 local ContextActionService = game:GetService("ContextActionService")
6 local GuiService = game:GetService("GuiService")
7 local Players = game:GetService("Players")
8 local ReplicatedStorage = game:GetService("ReplicatedStorage")
9 local RunService = game:GetService("RunService")
10 local TweenService = game:GetService("TweenService")
11 local UserInputService = game:GetService("UserInputService")
12
13 local Blaster = require(script.Parent:WaitForChild("Blaster"))
14 local localPlayer = Players.LocalPlayer
15
16 local BlasterController = {}
17 BlasterController.__index = BlasterController
18
19 local controllers = setmetatable({}, {__mode = "k"})
20 local nextControllerId = 0
21
22 local function touchControlsActive()
23     if UserInputService.TouchEnabled then return true end
24     local okTouchScreen, touchScreen = pcall(function()
25         return UserInputService.TouchScreenEnabled
26     end)
27     if okTouchScreen and touchScreen then return true end
28     local okPreferred, preferred = pcall(function()
29         return UserInputService.PreferredInput
30     end)
31     if okPreferred and tostring(preferred):find("Touch") then return true end
32     local okControls, controls = pcall(function()
33         return GuiService.TouchControlsEnabled
34     end)
35     if okControls and controls then return true end
36
37     -- Studio can retain desktop input flags when a phone preset is selected
38     -- after the client starts. Recognize the simulator's compact landscape
39     -- viewport so the weapon does not silently lose its mobile camera/buttons.
40     local camera = workspace.CurrentCamera
41     local viewport = camera and camera.ViewportSize
42     return RunService:IsStudio()
43         and viewport ~= nil
44         and viewport.Y <= 450
45         and viewport.X / math.max(1, viewport.Y) >= 1.45
46 end
```

```

47
48 local function pointInside(button, point)
49     if not button or not button.Visible then return false end
50     local p = button.AbsolutePosition
51     local s = button.AbsoluteSize
52     return point.X >= p.X and point.X <= p.X + s.X
53         and point.Y >= p.Y and point.Y <= p.Y + s.Y
54 end
55
56 -- Hardware support does not identify the active input on an emulator/hybrid.
57 local function touchAimActive()
58     local input = UserInputService.GetLastInputType()
59     if input == Enum.UserInputType.Touch then return true end
60     if input == Enum.UserInputType.MouseMovement
61         or input == Enum.UserInputType.MouseButton1
62         or input == Enum.UserInputType.MouseButton2
63         or input == Enum.UserInputType.MouseButton3
64         or input == Enum.UserInputType.MouseWheel
65         or input == Enum.UserInputType.Keyboard then
66         return false
67     end
68     return touchControlsActive()
69 end
70
71 local function rotationBetween(from, to)
72     local dot = math.clamp(from:Dot(to), -1, 1)
73     local axis = from:Cross(to)
74     if axis.Magnitude < 0.00001 then
75         if dot > 0 then return CFrame.identity end
76         axis = from:Cross(Vector3.yAxis)
77         if axis.Magnitude < 0.00001 then axis = from:Cross(Vector3.xAxis) end
78     end
79     return CFrame.fromAxisAngle(axis.Unit, math.acos(dot))
80 end
81
82 local function makeCrosshair(hudName)
83     local playerGui = localPlayer.WaitForChild("PlayerGui")
84     local old = playerGui:FindFirstChild(hudName)
85     if old then old:Destroy() end
86
87     local gui = Instance.new("ScreenGui")
88     gui.Name = hudName
89     gui.ResetOnSpawn = false
90     gui.IgnoreGuiInset = true
91     gui.ScreenInsets = Enum.ScreenInsets.None
92     gui.ClipToDeviceSafeArea = false
93     gui.DisplayOrder = 12
94     gui.Enabled = false
95     gui.Parent = playerGui
96
97     local root = Instance.new("Frame")
98     root.Name = "Crosshair"
99     root.AnchorPoint = Vector2.new(0.5, 0.5)
100    root.Position = UDim2.fromScale(0.5, 0.5)
101    root.Size = UDim2.fromOffset(30, 30)
102    root.BackgroundTransparency = 1
103    root.Parent = gui
104
105    local dot = Instance.new("Frame")
106    dot.Name = "Dot"
107    dot.AnchorPoint = Vector2.new(0.5, 0.5)
108    dot.Position = UDim2.fromScale(0.5, 0.5)
109    dot.Size = UDim2.fromOffset(6, 6)
110    dot.BackgroundColor3 = Color3.fromRGB(120, 235, 255)
111    dot.BorderSizePixel = 0
112    dot.Parent = root
113    local dotCorner = Instance.new("UICorner")
114    dotCorner.CornerRadius = UDim.new(1, 0)
115    dotCorner.Parent = dot
116    local dotStroke = Instance.new("UIStroke")
117    dotStroke.Color = Color3.fromRGB(6, 15, 22)
118    dotStroke.Transparency = 0.15
119    dotStroke.Thickness = 1.5
120    dotStroke.Parent = dot
121
122    for index, data in ipairs({

```

```

123         {UDim2.new(0.5, -1, 0, 1), UDim2.fromOffset(2, 8)},
124         {UDim2.new(0.5, -1, 1, -9), UDim2.fromOffset(2, 8)},
125         {UDim2.new(0, 1, 0.5, -1), UDim2.fromOffset(8, 2)},
126         {UDim2.new(1, -9, 0.5, -1), UDim2.fromOffset(8, 2)},
127     }) do
128         local line = Instance.new("Frame")
129         line.Name = "Line" .. index
130         line.Position = data[1]
131         line.Size = data[2]
132         line.BackgroundColor3 = Color3.fromRGB(255, 255, 255)
133         line.BackgroundTransparency = 0.08
134         line.BorderSizePixel = 0
135         line.Parent = root
136     end
137
138     local status = Instance.new("TextLabel")
139     status.Name = "Status"
140     status.AnchorPoint = Vector2.new(0.5, 0)
141     status.Position = UDim2.new(0.5, 0, 0.5, 25)
142     status.Size = UDim2.fromOffset(210, 24)
143     status.BackgroundTransparency = 1
144     status.Font = Enum.Font.GothamBold
145     status.Text = ""
146     status.TextColor3 = Color3.fromRGB(220, 232, 242)
147     status.TextSize = 13
148     status.TextStrokeTransparency = 0.5
149     status.Parent = gui
150
151     return gui, root, status, dot
152 end
153
154 function BlasterController:_ensureHUD()
155     if self.hud and self.hud.Parent and self.crosshair and self.crosshair.Parent == self.hud
156         and self.reticleDot and self.reticleDot.Parent == self.crosshair then return false end
157     if self.hud then self.hud:Destroy() end
158     self.fallbackShootButton = nil
159     self.fallbackAimButton = nil
160     self.hud, self.crosshair, self.statusLabel, self.reticleDot = makeCrosshair("BlasterHUD_" .. self.id)
161     self.hud.Enabled = self.enabled
162     return true
163 end
164
165 function BlasterController.new(tool, overrides)
166     assert(RunService.IsClient(), "BlasterController must be required on the client")
167     assert(tool and tool:IsA("Tool"), "BlasterController.new expects a Tool")
168
169     nextControllerId = nextControllerId + 1
170     local self = setmetatable({}, BlasterController)
171     self.tool = tool
172     self.overrides = overrides or {}
173     self.id = nextControllerId
174     self.actionPrefix = "Blaster_" .. nextControllerId .. "_"
175     self.connections = {}
176     self.enabled = false
177     self.aiming = false
178     self.triggerHeld = false
179     self.fireLoopRunning = false
180     self.sequence = 0
181     self.nextLocalShot = 0
182     self.baseGrip = tool.Grip
183     self.recoilPitch = 0
184     self.recoilYaw = 0
185     self.touchInput = nil
186     self.touchLastPosition = nil
187     self.hiddenParts = {}
188     self.shoulderOffset = Vector3.zero
189
190     self.connections.equipped = tool.Equipped:Connect(function()
191         self:Enable()
192     end)
193     self.connections.unequipped = tool.Unequipped:Connect(function()
194         self:Disable()
195     end)
196     self.connections.destroying = tool.Destroying:Connect(function()
197         self:Destroy()
198     end)

```

```

199     self.connections.ancestry = tool.AncestryChanged:Connect(function()
200         if self.enabled and tool.Parent ~= localPlayer.Character then
201             self:Disable()
202         end
203     end)
204     -- Tool.Activated is Roblox's device-independent activation path. ContextActionService
205     -- remains the primary input layer, while this fallback covers touch/controller layouts
206     -- that route their built-in Tool button directly to Activated.
207     self.connections.activated = tool.Activated:Connect(function()
208         if self.enabled then self:_beginFiring() end
209     end)
210     self.connections.deactivated = tool.Deactivated:Connect(function()
211         self.triggerHeld = false
212     end)
213
214     if localPlayer.Character and tool.Parent == localPlayer.Character then
215         task.defer(function() self:Enable() end)
216     end
217
218     return self
219 end
220
221 function BlasterController:_button(actionName)
222     local ok, button = pcall(function()
223         return ContextActionService:GetButton(actionName)
224     end)
225     return ok and button or nil
226 end
227
228 function BlasterController:_fallbackTouchButton(kind, title)
229     local field = kind == "Shoot" and "fallbackShootButton" or "fallbackAimButton"
230     local existing = self[field]
231     if existing and existing.Parent then return existing end
232     if not self.hud then return nil end
233
234     local button = Instance.new("TextButton")
235     button.Name = "Fallback" .. kind
236     button.AutoButtonColor = true
237     button.BackgroundColor3 = Color3.fromRGB(35, 48, 60)
238     button.BorderSizePixel = 0
239     button.Text = title
240     button.Font = Enum.Font.GothamBold
241     button.TextColor3 = Color3.fromRGB(245, 250, 255)
242     button.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
243     button.TextStrokeTransparency = 0.25
244     button.TextScaled = false
245     button.ZIndex = 20
246     button.Parent = self.hud
247     self[field] = button
248
249     if kind == "Shoot" then
250         button.InputBegan:Connect(function(input)
251             if input.UserInputType == Enum.UserInputType.Touch
252                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
253                 self:_shootAction(nil, Enum.UserInputState.Begin)
254             end
255         end)
256         button.InputEnded:Connect(function(input)
257             if input.UserInputType == Enum.UserInputType.Touch
258                 or input.UserInputType == Enum.UserInputType.MouseButton1 then
259                 self:_shootAction(nil, Enum.UserInputState.End)
260             end
261         end)
262     else
263         button.Activated:Connect(function()
264             if self.enabled then self:SetAiming(not self.aiming) end
265         end)
266     end
267     return button
268 end
269
270 function BlasterController:_styleTouchButtons()
271     if not touchControlsActive() then return end
272     local camera = workspace.CurrentCamera
273     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
274     local playerGui = localPlayer:FindFirstChildOfClass("PlayerGui")

```

```

275     local touchGui = playerGui and playerGui:FindFirstChild("TouchGui")
276     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
277     local diameter = math.clamp(math.floor(math.min(viewport.X, viewport.Y) * 0.13), 52, 80)
278     if jump and jump.AbsoluteSize.X > 0 then
279         diameter = math.clamp(math.floor(math.min(jump.AbsoluteSize.X, jump.AbsoluteSize.Y) * 0.85), 48, 80)
280     end
281
282     -- Own the visuals/hit targets; CAS still binds desktop/controller actions.
283     local shootButton = self:_fallbackTouchButton("Shoot", "FIRE")
284     local aimButton = self:_fallbackTouchButton("Aim", "AIM")
285     for _, action in ipairs({"Shoot", "Aim"}) do
286         local casButton = self:_button(self.actionPrefix .. action)
287         if casButton then casButton.Visible = false end
288     end
289     for _, button in ipairs({shootButton, aimButton}) do
290         if button then
291             button.Visible = true
292             button.Size = UDim2.fromOffset(diameter, diameter)
293             button.TextSize = math.floor(diameter * 0.25)
294             button.BackgroundTransparency = 0.18
295             if button:IsA("ImageButton") then
296                 button.ImageTransparency = 0.08
297                 button.ImageColor3 = Color3.fromRGB(35, 48, 60)
298             end
299             local corner = button:FindFirstChildOfClass("UICorner") or Instance.new("UICorner")
300             corner.CornerRadius = UDim.new(1, 0)
301             corner.Parent = button
302             local stroke = button:FindFirstChildOfClass("UIStroke") or Instance.new("UIStroke")
303             stroke.Color = Color3.fromRGB(220, 235, 245)
304             stroke.Transparency = 0.25
305             stroke.Thickness = 2
306             stroke.Parent = button
307             local title = button:FindFirstChild("ActionTitle")
308             if title and title:IsA("TextLabel") then
309                 title.Font = Enum.Font.GothamBold
310                 title.TextColor3 = Color3.fromRGB(245, 250, 255)
311                 title.TextStrokeColor3 = Color3.fromRGB(5, 8, 12)
312                 title.TextStrokeTransparency = 0.25
313                 title.TextScaled = true
314             end
315         end
316     end
317
318     -- GUI absolute coordinates include the simulator's safe-area translation.
319     -- Anchor to Roblox's real JUMP rectangle, never an estimated viewport point.
320     local gap = math.max(12, math.floor(diameter * 0.22))
321     local jumpSize = jump and jump.AbsoluteSize or Vector2.new(diameter, diameter)
322     local jumpPoint = jump and jump.AbsolutePosition
323     or (self.hud.AbsolutePosition + self.hud.AbsoluteSize - jumpSize - Vector2.new(28, 28))
324     local shootPoint = Vector2.new(jumpPoint.X + (jumpSize.X - diameter) * 0.5, jumpPoint.Y - diameter - gap)
325     local aimPoint = Vector2.new(jumpPoint.X - diameter - gap, jumpPoint.Y + (jumpSize.Y - diameter) * 0.5)
326     local function place(button, point)
327         if not button or not button.Parent then return end
328         button.AnchorPoint = Vector2.zero
329         local origin = button.Parent.AbsolutePosition
330         button.Position = UDim2.fromOffset(
331             math.floor(point.X - origin.X + 0.5),
332             math.floor(point.Y - origin.Y + 0.5)
333         )
334     end
335     place(shootButton, shootPoint)
336     place(aimButton, aimPoint)
337     self.lastJumpPosition = jump and jump.AbsolutePosition
338 end
339
340 function BlasterController:_getAimScreenPoint(camera)
341     if not camera then return nil end
342     if touchAimActive() then
343         -- Mobile aims along the Scriptable camera's true forward vector. In the
344         -- Studio phone simulator, ViewportPointToRay(viewportCenter) is offset by
345         -- the asymmetric safe-area canvas and can land several degrees above the
346         -- visible centre marker.
347         return nil
348     end
349     local point = UserInputService.GetMouseLocation() - GuiService.GetGuiInset()
350     -- Clicking an on-screen action must not pull the target onto that button.

```

```

351     if pointInside(self.fallbackShootButton, point) or pointInside(self.fallbackAimButton, point) then
352         return self.lastPointerAim or camera.ViewportSize * 0.5
353     end
354     self.lastPointerAim = point
355     return Vector2.new(
356         math.clamp(point.X, 0, camera.ViewportSize.X),
357         math.clamp(point.Y, 0, camera.ViewportSize.Y)
358     )
359 end
360
361 function BlasterController:_updateAimReticle(camera)
362     if not self.hud or not self.crosshair or not camera then return end
363     local rayScreenPoint = self:_getAimScreenPoint(camera)
364     local screenPoint = rayScreenPoint
365     local character = localPlayer.Character
366     local solution = character and Blaster.GetAimSolution(self.tool, camera, {
367         character,
368         self.tool,
369         camera,
370     }, rayScreenPoint)
371
372     if rayScreenPoint == nil then
373         -- The shot uses Camera.CFrame.LookVector. Project that same forward
374         -- vector with WorldToScreenPoint so the marker lands in the exact CoreUI
375         -- coordinate space used by this ScreenGui, including notches/top bars.
376         local projected = camera:WorldToScreenPoint(
377             camera.CFrame.Position + camera.CFrame.LookVector * 100
378         )
379         screenPoint = Vector2.new(projected.X, projected.Y)
380     elseif not screenPoint then
381         screenPoint = camera.ViewportSize * 0.5
382     end
383
384     if solution then
385         if self.reticledot then
386             self.reticledot.BackgroundColor3 = solution.result
387             and Color3.fromRGB(255, 224, 105)
388             or Color3.fromRGB(120, 235, 255)
389         end
390     end
391
392     -- ScreenGui's simulated-device canvas can be larger than Camera.ViewportSize
393     -- and begin at a negative absolute position. Counter that offset so this
394     -- marker identifies the muzzle-checked world hit on every device.
395     local origin = self.hud.AbsolutePosition
396     local center = screenPoint - origin
397     self.crosshair.Position = UDim2.fromOffset(
398         math.floor(center.X + 0.5),
399         math.floor(center.Y + 0.5)
400     )
401 end
402
403 function BlasterController:_setLocalHeadHidden(hidden)
404     local character = localPlayer.Character
405     if not character then return end
406     if hidden then
407         for _, descendant in ipairs(character:GetDescendants()) do
408             if descendant:IsA("BasePart") and (descendant.Name == "Head" or descendant.Parent:IsA("Accessory"))
409             then
410                 if self.hiddenParts[descendant] == nil then
411                     self.hiddenParts[descendant] = descendant.LocalTransparencyModifier
412                 end
413                 descendant.LocalTransparencyModifier = 1
414             end
415         end
416     else
417         for part, transparency in pairs(self.hiddenParts) do
418             if part.Parent then part.LocalTransparencyModifier = transparency end
419         end
420         table.clear(self.hiddenParts)
421     end
422 end
423
424 function BlasterController:_beginTouchCamera()
425     if not touchAimActive() or self.mobileCameraActive then return end
426     local camera = workspace.CurrentCamera

```



```

426     local character = localPlayer.Character
427     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
428     if not camera or not humanoid then return end
429
430     local look = camera.CFrame.LookVector
431     self.touchPitch = math.asin(math.clamp(look.Y, -1, 1))
432     self.touchYaw = math.atan2(-look.X, -look.Z)
433     self.savedCameraType = camera.CameraType
434     self.mobileCameraActive = true
435     humanoid.AutoRotate = false
436     camera.CameraType = Enum.CameraType.Scriptable
437     self:_setLocalHeadHidden(true)
438 end
439
440 function BlasterController:_endTouchCamera()
441     if not self.mobileCameraActive then return end
442     local camera = workspace.CurrentCamera
443     local character = localPlayer.Character
444     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
445     if camera and self.savedCameraType then camera.CameraType = self.savedCameraType end
446     self.mobileCameraActive = false
447     self.touchInput = nil
448     self.touchLastPosition = nil
449     self:_setLocalHeadHidden(false)
450 end
451
452 function BlasterController:SetAiming(aiming)
453     if not self.enabled then aiming = false end
454     if self.aiming == aiming then return end
455     self.aiming = aiming
456
457     local camera = workspace.CurrentCamera
458     if not camera then return end
459     local config = Blaster.GetWeaponConfig(self.tool)
460     local targetFov = aiming and config.adsFov or (self.hipFov or 70)
461     if self.fovTween then self.fovTween:Cancel() end
462     self.fovTween = TweenService:Create(camera, TweenInfo.new(0.16, Enum.EasingStyle.Quad,
463     ↪ Enum.EasingDirection.Out), {
464         FieldOfView = targetFov,
465     })
466     self.fovTween:Play()
467
468     if touchAimActive() then
469         -- Mobile weapons keep the camera active in hip-fire too, so the right
470         -- half of the screen always steers the reticle like a dedicated FPS.
471         self:_beginTouchCamera()
472     elseif aiming then
473         self.savedMouseSensitivity = UserInputService.MouseDeltaSensitivity
474         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity * 0.62
475     else
476         if self.savedMouseSensitivity then
477             UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
478             self.savedMouseSensitivity = nil
479         end
480     end
481
482     local character = localPlayer.Character
483     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
484     if humanoid and not self.mobileCameraActive then
485         humanoid.CameraOffset = aiming
486         and Vector3.new(1.25, 0.55, 0)
487         or self.shoulderOffset
488     end
489
490     if self.statusLabel then
491         self.statusLabel.Text = aiming and "ADS" or ""
492     end
493 end
494
495 function BlasterController:_fireOnce()
496     if not self.enabled or self.tool.Parent ~= localPlayer.Character then return end
497     local config = Blaster.GetWeaponConfig(self.tool)
498     local now = os.clock()
499     if now < self.nextLocalShot then return end
500     self.nextLocalShot = now + config.cooldown
501     self.sequence = self.sequence + 1

```

```

501
502     local camera = workspace.CurrentCamera
503     Blaster.Fire(self.tool, camera, self.sequence, self:_getAimScreenPoint(camera))
504     local recoilScale = self.aiming and 0.55 or 1
505     self.recoilPitch = self.recoilPitch + math.rad(0.75 * recoilScale)
506     self.recoilYaw = self.recoilYaw + math.rad((math.random() - 0.5) * 0.55 * recoilScale)
507 end
508
509 function BlasterController:_beginFiring()
510     if os.clock() < (self.ignoreGuiFireUntil or 0) then return end
511     self.triggerHeld = true
512     if self.fireLoopRunning then return end
513     self.fireLoopRunning = true
514     task.spawn(function()
515         repeat
516             self:_fireOnce()
517             local config = Blaster.GetWeaponConfig(self.tool)
518             if not config.automatic then break end
519             task.wait(config.cooldown)
520         until not self.triggerHeld or not self.enabled
521         self.fireLoopRunning = false
522     end)
523 end
524
525 function BlasterController:_shootAction(_, inputState, inputObject)
526     if inputState == Enum.UserInputState.Begin and inputObject and inputObject.UserInputType ==
527         ↳ Enum.UserInputType.MouseButton1 then
528         local hud=localPlayer.PlayerGui:FindFirstChild("GWAPHUD")
529         local point=UserInputService:GetMouseLocation()
530         for _,object in ipairs(localPlayer.PlayerGui:GetGuiObjectsAtPosition(point.X,point.Y)) do
531             if hud and object:IsDescendantOf(hud) then
532                 self.ignoreGuiFireUntil=os.clock()+.15
533                 return Enum.ContextActionResult.Pass
534             end
535         end
536     if not self.enabled then return Enum.ContextActionResult.Pass end
537     if inputState == Enum.UserInputState.Begin and inputObject
538         and inputObject.UserInputType == Enum.UserInputType.MouseButton1 then
539         local point = UserInputService:GetMouseLocation() - GuiService:GetGuiInset()
540         if point.Inside(self.fallbackAimButton, point) then return Enum.ContextActionResult.Pass end
541     end
542     if inputState == Enum.UserInputState.Begin then
543         self:_beginFiring()
544     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
545         self.triggerHeld = false
546     end
547     return Enum.ContextActionResult.Sink
548 end
549
550 function BlasterController:_aimAction(_, inputState, inputObject)
551     if not self.enabled then return Enum.ContextActionResult.Pass end
552     local isTouch = inputObject and inputObject.UserInputType == Enum.UserInputType.Touch
553     if isTouch then
554         if inputState == Enum.UserInputState.Begin then
555             self:SetAiming(not self.aiming)
556         end
557     elseif inputState == Enum.UserInputState.Begin then
558         self:SetAiming(true)
559     elseif inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
560         self:SetAiming(false)
561     end
562     return Enum.ContextActionResult.Sink
563 end
564
565 function BlasterController:_touchPanAction(_, inputState, inputObject)
566     if not self.enabled or not touchControlsActive() then
567         return Enum.ContextActionResult.Pass
568     end
569
570     local point = Vector2.new(inputObject.Position.X, inputObject.Position.Y)
571     local camera = workspace.CurrentCamera
572     local viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
573     local inset = GuiService:GetGuiInset()
574     point = point - inset
575

```

```

576     if inputState == Enum.UserInputState.Begin then
577         local shootButton = self.fallbackShootButton or self:_button(self.actionPrefix .. "Shoot")
578         local aimButton = self.fallbackAimButton or self:_button(self.actionPrefix .. "Aim")
579         local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
580         local jumpButton = touchGui and touchGui:FindFirstChild("JumpButton", true)
581         if point.X < viewport.X * 0.42 or pointInside(shootButton, point)
582             or pointInside(aimButton, point) or pointInside(jumpButton, point) then
583             return Enum.ContextActionResult.Pass
584         end
585         self.touchInput = inputObject
586         self:_beginTouchCamera()
587         self.touchLastPosition = point
588         return Enum.ContextActionResult.Sink
589     end
590
591     if inputObject ~= self.touchInput then return Enum.ContextActionResult.Pass end
592     if inputState == Enum.UserInputState.Change and self.touchLastPosition then
593         local delta = point - self.touchLastPosition
594         self.touchLastPosition = point
595         local sensitivity = self.aiming
596             and (self.tool:GetAttribute("BlasterADSTouchSensitivity") or 0.42)
597             or (self.tool:GetAttribute("BlasterHipTouchSensitivity") or 0.72)
598         sensitivity = math.clamp(sensitivity, 0.15, 1.2)
599         self.touchYaw = self.touchYaw - delta.X * 0.0031 * sensitivity
600         self.touchPitch = math.clamp(self.touchPitch - delta.Y * 0.0027 * sensitivity, math.rad(-78),
        ↪ math.rad(78))
601         return Enum.ContextActionResult.Sink
602     end
603
604     if inputState == Enum.UserInputState.End or inputState == Enum.UserInputState.Cancel then
605         self.touchInput = nil
606         self.touchLastPosition = nil
607         return Enum.ContextActionResult.Sink
608     end
609     return Enum.ContextActionResult.Pass
610 end
611
612 function BlasterController:_updateArmPose(targetPosition, dt)
613     local character = localPlayer.Character
614     local handle = self.tool:FindFirstChild("Handle")
615     if not character or not handle or not targetPosition then return end
616     if not self.aimJoint or not self.aimJoint.Parent then
617         local joints = {}
618         for _, joint in ipairs(character:GetDescendants()) do
619             if (joint:IsA("JointInstance") or joint:IsA("AnimationConstraint")) and joint.Part0 and joint.Part1
        ↪ then
620                 joints[joint.Part1] = joint
621             end
622         end
623         local chain = {}
624         local part = handle
625         for _ = 1, 8 do
626             local joint = joints[part]
627             if not joint then return end
628             table.insert(chain, 1, joint)
629             if joint.Name == "Right Shoulder" or joint.Name == "RightShoulder" then
630                 self.aimJoint = joint
631                 self.aimBaseC0 = joint.C0
632                 self.aimChain = chain
633                 break
634             end
635             part = joint.Part0
636         end
637     end
638     local shoulder = self.aimJoint
639     if not shoulder or not shoulder.Part0 then return end
640     -- Rebuild from the untouched joint/grip and the current animation each frame.
641     -- This preserves R6/R15 rest rotations and avoids accumulating aim offsets.
642     local jointWorld = shoulder.Part0.CFrame * self.aimBaseC0
643     local predicted = shoulder.Part0.CFrame
644     for _, joint in ipairs(self.aimChain) do
645         if not joint.Parent then return end
646         local c0 = joint == shoulder and self.aimBaseC0 or joint.C0
647         local animation = CFrame.identity -- armed arm chain is isolated from idle/walk animation
648         predicted = predicted * c0 * animation * joint.C1:Inverse()
649     end

```

```

650     local forward = self.tool:GetAttribute("BlasterBarrelDirection")
651     if typeof(forward) ~= "Vector3" or forward.Magnitude < 0.1 then
652         -- Imported Weapons Kit meshes face +Z; procedural barrels face -Z.
653         forward = self.tool:GetAttribute("LabSidearm") and Vector3.zAxis or -Vector3.zAxis
654     end
655     local turn = CFrame.identity
656     local pivot = jointWorld.Position
657     for _ = 1, 4 do
658         local delta = targetPosition - predicted.Position
659         if delta.Magnitude < 0.1 then break end
660         local correction = rotationBetween(predicted:VectorToWorldSpace(forward.Unit), delta.Unit)
661         predicted = CFrame.new(pivot) * correction * CFrame.new(-pivot) * predicted
662         turn = correction * turn
663     end
664     local pose = shoulder.Part0.CFrame:ToObjectSpace(CFrame.new(pivot) * turn * jointWorld.Rotation)
665     self.smoothedArmPose = (self.smoothedArmPose or pose):Lerp(pose, 1 - math.exp(-math.min(dt or 1/60, .1) *
        ↳ 24))
666     pose = self.smoothedArmPose
667     if shoulder:IsA("AnimationConstraint") then
668         shoulder.Attachment0.CFrame = pose
669     else
670         shoulder.C0 = pose
671     end
672 end
673
674 function BlasterController:_render(dt)
675     if not self.enabled then return end
676     local character = localPlayer.Character
677     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
678     local root = character and character:FindFirstChild("HumanoidRootPart")
679     local head = character and character:FindFirstChild("Head")
680     local camera = workspace.CurrentCamera
681     if not humanoid or not camera then return end
682     local hudRebuilt = self:_ensureHUD()
683     local touchMode = touchControlsActive()
684     local touchAim = touchAimActive()
685     if touchAim and not self.mobileCameraActive then
686         self:_beginTouchCamera()
687     elseif not touchAim and self.mobileCameraActive then
688         self:_endTouchCamera()
689     end
690     local touchGui = localPlayer.PlayerGui:FindFirstChild("TouchGui")
691     local jump = touchGui and touchGui:FindFirstChild("JumpButton", true)
692     if touchMode and (hudRebuilt or self.lastTouchViewport ~= camera.ViewportSize
        or (jump and jump.AbsolutePosition ~= self.lastJumpPosition)) then
693         self.lastTouchViewport = camera.ViewportSize
694         task.defer(function()
695             if self.enabled then self:_styleTouchButtons() end
696         end)
697     end
698 end
699 -- Keep the inventory grip fixed: no idle or walking weapon bob.
700
701 self.recoilPitch = self.recoilPitch * math.exp(-dt * 14)
702 self.recoilYaw = self.recoilYaw * math.exp(-dt * 16)
703 if self.mobileCameraActive and head and root then
704     self.touchPitch = math.clamp(self.touchPitch - self.recoilPitch * dt * 7, math.rad(-78), math.rad(78))
705     self.touchYaw = self.touchYaw + self.recoilYaw * dt * 7
706     local rotation = CFrame.fromOrientation(self.touchPitch, self.touchYaw, 0)
707     local focus = root.Position + Vector3.new(0, self.stableHeadHeight or 1.5, 0)
708     local desired = focus - rotation.LookVector * 0.18 + rotation.RightVector * 0.08
709     local cameraResult = workspace:Raycast(focus, desired - focus, Blaster.NewRaycastParams({character,
        ↳ self.tool}))
710     local position = cameraResult and (cameraResult.Position + cameraResult.Normal * 0.1) or desired
711     camera.CFrame = CFrame.new(position) * rotation
712     camera.Focus = CFrame.new(focus + rotation.LookVector * 12)
713
714 elseif math.abs(self.recoilPitch) > 0.00001 or math.abs(self.recoilYaw) > 0.00001 then
715     camera.CFrame = camera.CFrame * CFrame.Angles(-self.recoilPitch * dt * 7, self.recoilYaw * dt * 7, 0)
716 end
717 local config = Blaster.GetWeaponConfig(self.tool)
718 local cast = Blaster.CastFromCamera(camera, config.range, {character, self.tool, camera},
    ↳ self:_getAimScreenPoint(camera))
719 if cast then
720     self:_updateArmPose(cast.position, dt)
721 end
722 self:_updateAimReticle(camera)

```

```

723 end
724
725 local activeController
726 function BlasterController:Enable()
727     if self.enabled or self.tool.Parent ~= localPlayer.Character then return end
728     if activeController and activeController ~= self then activeController:Disable() end
729     activeController = self
730     self.enabled = true
731     self.baseGrip = self.tool.Grip
732     local rootPart = localPlayer.Character and localPlayer.Character:FindFirstChild("HumanoidRootPart")
733     local headPart = localPlayer.Character and localPlayer.Character:FindFirstChild("Head")
734     self.stableHeadHeight = rootPart and headPart and (headPart.Position.Y - rootPart.Position.Y + .12) or 1.5
735     self.hipFov = self.tool:GetAttribute("BlasterHipFOV") or (workspace.CurrentCamera and
        ↪ workspace.CurrentCamera.FieldOfView) or 70
736
737     self:_ensureHUD()
738     self.hud.Enabled = true
739
740     local character = localPlayer.Character
741     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
742     if humanoid then
743         self.savedAutoRotate = humanoid.AutoRotate
744         humanoid.AutoRotate = false
745         self.savedCameraOffset = humanoid.CameraOffset
746         self.shoulderOffset = Vector3.new(
747             self.tool:GetAttribute("BlasterShoulderOffsetX") or 2.2,
748             self.tool:GetAttribute("BlasterShoulderOffsetY") or 0.65,
749             0
750         )
751         humanoid.CameraOffset = self.shoulderOffset
752     end
753     if touchAimActive() then self:_beginTouchCamera() end
754
755     local shootName = self.actionPrefix .. "Shoot"
756     local aimName = self.actionPrefix .. "Aim"
757     local panName = self.actionPrefix .. "TouchPan"
758     local priority = Enum.ContextActionPriority.High.Value
759
760     ContextActionService.BindActionAtPriority(shootName, function(...)
761         return self:_shootAction(...)
762     end, true, priority + 2, Enum.UserInputType.MouseButton1, Enum.KeyCode.ButtonR2)
763     ContextActionService.BindActionAtPriority(aimName, function(...)
764         return self:_aimAction(...)
765     end, true, priority + 2, Enum.UserInputType.MouseButton2, Enum.KeyCode.ButtonL2)
766     ContextActionService.BindActionAtPriority(panName, function(...)
767         return self:_touchPanAction(...)
768     end, false, priority + 1, Enum.UserInputType.Touch)
769
770     ContextActionService.SetTitle(shootName, "FIRE")
771     ContextActionService.SetTitle(aimName, "AIM")
772     task.delay(0.1, function()
773         if self.enabled then self:_styleTouchButtons() end
774     end)
775
776     -- Rotate once, before the camera reads the character. Using camera heading
777     -- removes the shoulder-offset/nearby-hit feedback loop that caused oscillation.
778     self.bodyRenderName = self.actionPrefix .. "Body"
779     RunService:BindToRenderStep(self.bodyRenderName, Enum.RenderPriority.Camera.Value - 1, function()
780         if not self.enabled then return end
781         local char = localPlayer.Character
782         local hum = char and char:FindFirstChildOfClass("Humanoid")
783         local body = char and char:FindFirstChild("HumanoidRootPart")
784         local cam = workspace.CurrentCamera
785         if not hum or hum.Sit or not body or not cam then return end
786         local direction = self.mobileCameraActive and CFrame.fromOrientation(0, self.touchYaw, 0).LookVector or
            ↪ cam.CFrame.LookVector
787         local flat = Vector3.new(direction.X, 0, direction.Z)
788         if flat.Magnitude > .01 then body.CFrame = CFrame.lookAt(body.Position, body.Position + flat.Unit) end
789     end)
790     -- Animator writes Transform before PreSimulation. Reset only the armed chain,
791     -- preserving locomotion animation everywhere else.
792     self.armAnimationConnection = RunService.PreSimulation:Connect(function()
793         if not self.enabled or not self.aimChain then return end
794         for _, joint in ipairs(self.aimChain) do if joint.Parent and joint:IsA("Motor6D") then joint.Transform =
            ↪ CFrame.identity end end
795         local char = localPlayer.Character

```

```

796     if char then
797         self.torsoAnimationJoints=self.torsoAnimationJoints or {}
798         if #self.torsoAnimationJoints==0 then for _,joint in ipairs(char:GetDescendants()) do
799             if joint:IsA("Motor6D") and (joint.Name=="Waist" or joint.Name=="RootJoint" or
800                 ↪ joint.Name=="Root") then table.insert(self.torsoAnimationJoints,joint) end
801         end
802         for _,joint in ipairs(self.torsoAnimationJoints) do if joint.Parent then
803             ↪ joint.Transform=CFrame.identity end end
804     end
805     self.renderName = self.actionPrefix .. "Render"
806     RunService:BindToRenderStep(self.renderName, Enum.RenderPriority.Camera.Value + 1, function(dt)
807         self:_render(dt)
808     end)
809 end
810 function BlasterController:Disable()
811     if not self.enabled then return end
812     self.triggerHeld = false
813     self:SetAiming(false)
814     self.enabled = false
815     if activeController == self then activeController = nil end
816     self.smoothedArmPose = nil
817     self.torsoAnimationJoints = nil
818     if self.armAnimationConnection then self.armAnimationConnection:Disconnect(); self.armAnimationConnection =
819         ↪ nil end
820     if self.bodyRenderName then RunService:UnbindFromRenderStep(self.bodyRenderName) end
821     ContextActionService:UnbindAction(self.actionPrefix .. "Shoot")
822     ContextActionService:UnbindAction(self.actionPrefix .. "Aim")
823     ContextActionService:UnbindAction(self.actionPrefix .. "TouchPan")
824     if self.renderName then RunService:UnbindFromRenderStep(self.renderName) end
825     if self.hud then self.hud.Enabled = false end
826     if self.tool.Parent then self.tool.Grip = self.baseGrip end
827     if self.aimJoint and self.aimJoint.Parent then
828         if self.aimJoint:IsA("AnimationConstraint") then
829             self.aimJoint.Attachment0.CFrame = self.aimBaseC0
830         else
831             self.aimJoint.C0 = self.aimBaseC0
832         end
833     end
834     self.aimJoint, self.aimBaseC0, self.aimChain = nil, nil, nil
835
836     local camera = workspace.CurrentCamera
837     if camera and self.hipFov then camera.FieldOfView = self.hipFov end
838     self:_endTouchCamera()
839     if self.savedMouseSensitivity then
840         UserInputService.MouseDeltaSensitivity = self.savedMouseSensitivity
841         self.savedMouseSensitivity = nil
842     end
843     local character = localPlayer.Character
844     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
845     if humanoid and self.savedAutoRotate ~= nil then humanoid.AutoRotate = self.savedAutoRotate end
846     self.savedAutoRotate = nil
847     if humanoid and self.savedCameraOffset then
848         humanoid.CameraOffset = self.savedCameraOffset
849     end
850     self.savedCameraOffset = nil
851     self.lastTouchViewport = nil
852 end
853
854 function BlasterController:Destroy()
855     self:Disable()
856     for _, connection in pairs(self.connections) do
857         connection:Disconnect()
858     end
859     table.clear(self.connections)
860     if self.hud then self.hud:Destroy() end
861     controllers[self.tool] = nil
862 end
863
864 function BlasterController.BindTool(tool, overrides)
865     if controllers[tool] then return controllers[tool] end
866     local controller = BlasterController.new(tool, overrides)
867     controllers[tool] = controller
868     Blaster.StartReplication()

```

```
869     return controller
870 end
871
872 function BlasterController.GetController(tool)
873     return controllers[tool]
874 end
875
876 function BlasterController.AutoBind()
877     local binder = {connections = {}}
878
879     local function consider(instance)
880         if instance:IsA("Tool") and (instance:GetAttribute("BlasterWeapon") or
881             ⇐ instance:GetAttribute("LabSidearm")) then
882             BlasterController.BindTool(instance)
883         end
884     end
885
886     local function watch(container)
887         if not container then return end
888         for _, descendant in ipairs(container:GetDescendants()) do consider(descendant) end
889         table.insert(binder.connections, container.DescendantAdded:Connect(consider))
890     end
891
892     watch(localPlayer.WaitForChild("Backpack"))
893     watch(localPlayer.Character)
894     table.insert(binder.connections, localPlayer.CharacterAdded:Connect(watch))
895     Blaster.StartReplication()
896
897     function binder:Destroy()
898         for _, connection in ipairs(self.connections) do connection:Disconnect() end
899         table.clear(self.connections)
900     end
901
902     return binder
903 end
904
905 return BlasterController
```

Chapter 69

W12 - ServerBlasterManager.lua

ServerBlasterManager.lua

Authority group: workspace. 7714 UTF-8 bytes; 184 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

WORKSPACE ARTIFACT. Presence on disk does not establish installation. Compare against active sources. In particular, GWAPWorld.lua contains older server code and is not the correct world module.

```
1 -- ServerBlasterManager
2 -- Authoritative validation, obstacle recasting, damage, and shot replication.
3
4 local Players = game:GetService("Players")
5 local ReplicatedStorage = game:GetService("ReplicatedStorage")
6 local ServerScriptService = game:GetService("ServerScriptService")
7
8 local modules = ReplicatedStorage:WaitForChild("Modules")
9 local Blaster = require(modules:WaitForChild("Blaster"))
10
11 local remoteFolder = ReplicatedStorage:FindFirstChild(Blaster.REMOTE_FOLDER_NAME)
12 if remoteFolder and not remoteFolder:IsA("Folder") then remoteFolder:Destroy(); remoteFolder = nil end
13 if not remoteFolder then
14     remoteFolder = Instance.new("Folder")
15     remoteFolder.Name = Blaster.REMOTE_FOLDER_NAME
16     remoteFolder.Parent = ReplicatedStorage
17 end
18
19 local function getRemote(name)
20     local remote = remoteFolder:FindFirstChild(name)
21     if remote and not remote:IsA("RemoteEvent") then remote:Destroy(); remote = nil end
22     if not remote then
23         remote = Instance.new("RemoteEvent")
24         remote.Name = name
25         remote.Parent = remoteFolder
26     end
27     return remote
28 end
29
30 local fireRemote = getRemote(Blaster.FIRE_REMOTE_NAME)
31 local effectRemote = getRemote(Blaster.EFFECT_REMOTE_NAME)
32
33 local hitBridge = ServerScriptService:FindFirstChild("BlasterHitBridge")
34 if hitBridge and not hitBridge:IsA("BindableFunction") then hitBridge:Destroy(); hitBridge = nil end
35 if not hitBridge then
36     hitBridge = Instance.new("BindableFunction")
37     hitBridge.Name = "BlasterHitBridge"
38     hitBridge.Parent = ServerScriptService
39 end
40
41 -- Roblox's default third-person zoom can legitimately place the camera well
42 -- behind the character. Keep the anti-spoof bound, but allow normal PC/mobile
43 -- zoom distances and the over-shoulder hip-fire camera.
44 local MAX_CAMERA_OFFSET = 40
45 local RATE_TOLERANCE = 0.88
46 local nextShotByPlayer = {}
```



```

47 local lastSequenceByTool = setmetatable({}, {__mode = "k"})
48
49 local function findHumanoidFromHit(instance)
50     local node = instance
51     while node and node ~= workspace do
52         if node:IsA("Model") then
53             local humanoid = node:FindFirstChildOfClass("Humanoid")
54             if humanoid then return humanoid, node end
55         end
56         node = node.Parent
57     end
58     return nil
59 end
60
61 local function randomSpread(direction, spread)
62     if spread <= 0 then return direction end
63     local reference = math.abs(direction.Y) > 0.96 and Vector3.xAxis or Vector3.yAxis
64     local right = direction:Cross(reference).Unit
65     local up = right:Cross(direction).Unit
66     local radius = math.sqrt(math.random()) * spread
67     local angle = math.random() * math.pi * 2
68     return (direction + right * math.cos(angle) * radius + up * math.sin(angle) * radius).Unit
69 end
70
71 local function broadcastShot(shooter, origin, position, normal, color)
72     local payload = {
73         origin = origin,
74         position = position,
75         normal = normal,
76         color = color,
77     }
78     for _, other in ipairs(Players:GetPlayers()) do
79         if other ~= shooter then effectRemote:FireClient(other, payload) end
80     end
81 end
82
83
84 local function researchHit(player, result, damage)
85     local researchBridge = ServerScriptService:FindFirstChild("GWAPHitBridge")
86     if not researchBridge then return false end
87     local ok, handled = pcall(function() return researchBridge:Invoke(player, result.Instance, damage) end)
88     return ok and handled == true
89 end
90
91 local function bridgeHit(player, result, damage, instantHeadshot)
92     if not result then return false end
93     if researchHit(player, result, instantHeadshot and result.Instance.Name == "Head" and 1000 or damage) then
94         ↪ return true end
95     -- BindableFunction callbacks are write-only. LabEquipment binds this during
96     -- server startup; pcall keeps a missing/failed optional bridge fail-closed.
97     local ok, response = pcall(function()
98         return hitBridge:Invoke(player, result.Instance, result.Position, result.Normal, damage,
99             instantHeadshot and result.Instance.Name == "Head")
100     end)
101     return ok and response == true
102 end
103
104 local function genericDamage(player, result, config)
105     local humanoid, model = findHumanoidFromHit(result.Instance)
106     if not humanoid or humanoid.Health <= 0 or model == player.Character then return end
107     local victimPlayer = Players:GetPlayerFromCharacter(model)
108     if victimPlayer and not config.allowPlayerDamage then return end
109
110     local damage = config.damage
111     if result.Instance.Name == "Head" then
112         damage = config.instantHeadshot and math.max(1000, humanoid.MaxHealth) or damage * 2
113     end
114     humanoid:TakeDamage(damage)
115 end
116
117 local function validSequence(tool, sequence)
118     if typeof(sequence) ~= "number" or sequence ~= sequence or sequence % 1 ~= 0 then return false end
119     if sequence < 1 or sequence > 2147483647 then return false end
120     local previous = lastSequenceByTool[tool]
121     if previous and sequence <= previous then return false end
122     lastSequenceByTool[tool] = sequence

```

```

122     return true
123 end
124
125 local function handleFire(player, packet)
126     if typeof(packet) ~= "table" or packet.version ~= 1 then return end
127     if not Blaster.IsFiniteVector(packet.cameraOrigin) or not Blaster.IsFiniteVector(packet.direction) then
128         ↪ return end
129
130     local character = player.Character
131     local humanoid = character and character:FindFirstChildOfClass("Humanoid")
132     local head = character and character:FindFirstChild("Head")
133     local root = character and character:FindFirstChild("HumanoidRootPart")
134     local tool = packet.tool
135     if not humanoid or not head or not root or humanoid.Health <= 0 then return end
136     if typeof(tool) ~= "Instance" or not tool:IsA("Tool") or tool.Parent ~= character then return end
137     if tool:GetAttribute("BlasterWeapon") ~= true and tool:GetAttribute("LabSidearm") ~= true then return end
138
139     if (packet.cameraOrigin - head.Position).Magnitude > MAX_CAMERA_OFFSET then return end
140     local directionMagnitude = packet.direction.Magnitude
141     if directionMagnitude < 0.5 or directionMagnitude > 1.5 then return end
142     local aimDirection = packet.direction.Unit
143     if not validSequence(tool, packet.sequence) then return end
144
145     local config = Blaster.GetWeaponConfig(tool)
146     local now = os.clock()
147     if now < (nextShotByPlayer[player] or 0) then return end
148     nextShotByPlayer[player] = now + config.cooldown * RATE_TOLERANCE
149
150     local params = Blaster.NewRaycastParams({character, tool})
151     local handle = tool:FindFirstChild("Handle")
152     local muzzleOrigin = handle and handle.Position or (head.Position + aimDirection * 0.5)
153     local tracerColor = tool:GetAttribute("BlasterTracerColor")
154     if typeof(tracerColor) ~= "Color3" then tracerColor = Color3.fromRGB(255, 226, 140) end
155
156     for _ = 1, config.pellets do
157         local pelletDirection = randomSpread(aimDirection, config.spread)
158         local cameraResult = workspace:Raycast(packet.cameraOrigin, pelletDirection * config.range, params)
159         local aimPoint = cameraResult and cameraResult.Position or (packet.cameraOrigin + pelletDirection *
160             ↪ config.range)
161         local muzzleDelta = aimPoint - muzzleOrigin
162         local castDirection = muzzleDelta.Magnitude > 0.05 and muzzleDelta.Unit or pelletDirection
163         local castLength = math.min(config.range, math.max(0.1, muzzleDelta.Magnitude + 0.25))
164         local result = workspace:Raycast(muzzleOrigin, castDirection * castLength, params)
165         local hitPosition = result and result.Position or (muzzleOrigin + castDirection * castLength)
166
167         broadcastShot(player, muzzleOrigin, hitPosition, result and result.Normal or nil, tracerColor)
168         if result then
169             local handled = bridgeHit(player, result, config.damage, config.instantHeadshot)
170             if not handled then genericDamage(player, result, config) end
171         end
172     end
173
174     if config.selfCostPercent > 0 and humanoid.Health > 0 then
175         humanoid:TakeDamage(humanoid.MaxHealth * config.selfCostPercent / 100)
176     end
177 end
178
179 fireRemote.OnServerEvent:Connect(handleFire)
180 Players.PlayerRemoving:Connect(function(player)
181     nextShotByPlayer[player] = nil
182 end)
183
184 print("ServerBlasterManager ready: camera shots are server-validated and obstacle-checked.")

```

Part VI

Complete server-only documentation source

Chapter 70

DOC01 - GameDocumentation

ServerStorage.GameDocumentation

Authority group: documentation. 120178 UTF-8 bytes; 827 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1 --[=====]
2 Reference part 1 of 2; continue in sibling documentation modules.
3 GENRMINATION_FIGHT COMPLETE WRITTEN REFERENCE
4 Revision 4, 13 September 2026
5 This is a build-time reference snapshot; final hashes are in the external manifest.
6 Executable listings are stored in their own scripts, not recursively repeated here.
7
8 # Revision 4: live repairs and current handover
9 ## Authority and chronology
10 This chapter supersedes the pre-revision findings wherever a repair is explicitly recorded here. The baseline
   ↳ inventory was captured from Studio version 70; the owner reopened version 72 and all 29 source fingerprints
   ↳ matched that baseline before modification. The owner then explicitly authorized conservative live repairs
   ↳ and documentation inside source comments. The changes below were installed through Studio MCP in Edit mode
   ↳ and exercised in fresh Play sessions on 13 September 2026. No publishing action was performed. The exported
   ↳ RBXL remains the original evidence, not a post-repair save.
11
12 ## What was actually changed
13 C01, stale coordinate metadata: the laboratory had moved by approximately (-25.142857, -23.25, 91.285714) studs
   ↳ while navigation still used generator coordinates. LabSpatial derives a frame from the existing Roof_1
   ↳ landmark and its original generator CFrame. World construction, station backups, mission stairs,
   ↳ rendezvous, legacy repair routes, room lookup, worker patrol destinations, room smoke and door ventilation
   ↳ now translate between laboratory-local coordinates and world coordinates explicitly. Each of the 67 station
   ↳ approaches has an ApproachPoint attachment. Its world position follows subsequent station movement;
   ↳ ApproachLocal preserves the migration input. The building itself was not moved back.
14
15 A second scene mismatch was found by sweeping a carrier-sized box along the corrected route: Lounge.Wall_N
   ↳ intersected the basement stair exit. The laboratory had moved but the lounge and its hatch had not. All 169
   ↳ associated BaseParts were transformed once by the same laboratory frame. Original CFrames are retained in
   ↳ the live rollback folder. This is a scene migration, not a repeating runtime translation. Future
   ↳ whole-world moves must include the lounge, hatch and laboratory together.
16
17 C02, incorrect installer payload: the loose GWAPWorld.lua and old InstallGWAP.lua contained mission-server code
   ↳ under the world-module name. The corrected root module is the verified world implementation. The root
   ↳ installer is regenerated from the final source set, including LabSpatial and changed legacy dependencies.
   ↳ It is a small local-HTTP preflight loader, with complete source payloads in release/. Each payload is
   ↳ length/checksum verified and compiled before mutation. A monolithic payload would exceed Studio's
   ↳ 200,000-byte Script.Source limit. Serve the documentation folder on localhost port 8766; no external
   ↳ hosting is required. Historical copies under code/workspace remain unchanged and are explicitly
   ↳ pre-revision evidence. Do not confuse those historical files with the corrected root release. The large
   ↳ generator is not rebuilt or executed by this revision.
18
19 R01, research repair authority: the server now requires a living Technician, workspace membership, no
   ↳ BlastRemoved flag, an exploded target, proximity within 14 studs and an unobstructed ray from the player's
   ↳ head to the equipment. UI prompt visibility alone no longer authorizes repair. HoldDuration remains an
   ↳ interface duration; this revision does not certify resistance to forged hold timing. That is a remaining
   ↳ security-hardening task, not a claim of complete exploit resistance.
20
```

21 R02, carrier collision: the existing anchored carrier route movement now performs a 2.4 by 4.4 by 2.4 stud
→ swept query before each short movement step. A solid obstacle stops movement, extends the mission deadline
→ and produces an explanatory dialogue at most once per ten seconds. Clearing the obstacle resumes travel.
→ Arbitrary obstacle detours are not promised; Wait and Regroup remain the recovery controls. Player
→ movement, camera, mouse aiming, mobile aiming, gun recoil and input bindings are preserved.

22

23 R03, solo eligibility: only living active NPCs in Workspace enter the armed selection. Stored carriers and
→ knocked-out/dead workers are excluded. The armed count is $\text{floor}(\text{eligible count} * 0.2)$. Eligibility is
→ reconciled when population state changes and on a five-second bounded check; already selected gun instances
→ are reused. The original inventory mesh specifications are unchanged.

24

25 R07, shared lounge notes: player-authored note text passes through `TextService.FilterStringAsync` and
→ `GetNonChatStringForBroadcastAsync` before broadcast. Failed filtering produces no shared note. A
→ three-second per-player cooldown bounds requests. Departure clears note cooldown and skip-vote state.
→ Published-service filtering behavior is not certified by a local Studio session; the fail-closed source
→ path is the evidence for this change.

26

27 HUD and allocation fixes: the carrier's health is folded into the existing mission target line, and its
→ duplicate overhead billboards are hidden. `GWAPNPC.say` retains dialogue attributes but no longer creates a
→ server-enabled `MissionSpeech` billboard; this removes the observed flash before client suppression and the
→ redundant GUI instances. Other targeted NPC billboards are suppressed near the top HUD. Existing compact
→ layout and original weapon controls are retained. Pool reactivation no longer installs duplicate billboard
→ watchers. Route nodes are sorted once until `ChildAdded` or `ChildRemoved` invalidates the cache, allowing
→ incremental replication. Terminal missions immediately destroy their route nodes. These are bounded
→ allocation reductions; no invented megabyte saving or sixty-player performance claim is made.

28

29 ## Fresh verification results

30 V4-01: all seven initial changed/new source files compiled in Studio before installation. A fresh Play session
→ reported no server or client `MessageError` entries at the inspected checkpoints.

31

32 V4-02: Incubator approach migrated from (78, 3.25, 49) to approximately (52.857147, -20, 140.285706), adjacent
→ to its actual body at (41.857147, -21.75, 140.285706). The initial companion appeared 8.0163 studs from the
→ player. Four unused carriers stayed in `ServerStorage`. Solo count was 36 armed among 181 active eligible
→ NPCs.

33

34 V4-03: scenario 1 completed seed bank, instrument and rooftop planting in the corrected scene. Plot 1 gained
→ one plant, score increased from 0 to 90 and route-folder count returned to zero. Scenario 2 then completed
→ and the cumulative score became 180. A test-only server helper kept the owner six studs beside the moving
→ carrier and restored health; it did not shoot, modify points, advance stages or move the carrier. This
→ verifies mission execution and clean rewards, not human stair-navigation skill or multiplayer load. Helpers
→ existed only in Play and were discarded on Stop.

35

36 V4-04: equivalent-station selection chose Incubator - Backup 1 on floor 0 after primary destruction, then
→ Incubator - Backup 2 on floor 1 after the first backup was made unavailable. The controlled selection test
→ restored station flags afterward. Full mission fallback across every capability remains a broader
→ regression target.

37

38 V4-05: a temporary solid obstacle inserted into a live carrier route produced zero movement over a half-second
→ stopped sample. After removal, the carrier moved 9.9987 studs in approximately half a second with owner
→ `WalkSpeed` 20. The obstruction dialogue appeared. No player movement settings were changed by the carrier
→ implementation.

39

40 V4-06: a controlled exploded wall section removed a health station, gun fixture, poster and research-equipment
→ fixture from Workspace. A fixture outside the section stayed present and unmarked. All test geometry was
→ cleaned up. Existing actual-wall selection and quarantine behavior are retained.

41

42 V4-08: in a fresh final-source Play session, holding R within six studs restored an exploded Incubator for the
→ Technician role. Repeating the same interaction as Laboratory Technician left the target destroyed, as
→ required. No server errors were reported. An earlier attempt at the outer edge of prompt range did not
→ activate; the closer prompt interaction is the successful observation.

43

44 V4-07: a 705 by 338 client viewport loaded without client errors. Visual inspection found a carrier
→ overhead-bar overlap with the top strip, leading to the final health-in-panel change. The final UI recheck
→ is recorded in `VERIFICATION.md`. The accepted `Blaster` and `BlasterController` executable bodies are unchanged;
→ their final differences are documentation comments only. Earlier 90-frame idle and walking grip
→ measurements remain historical tests, not newly repeated measurements.

45

46 ## Remaining boundaries and next engineering work

47 Wall destruction remains persistent within the current server session: fixtures are quarantined and walls are
→ not recreated automatically at a round change. A future restoration system must restore both the wall and
→ its original fixture parent, tags, prompt state and callbacks in a single lifecycle contract. Reparenting
→ fixtures alone would restore floating equipment. This behavior is disclosed rather than silently
→ introducing a new world-reset policy.

48

49 The current mission contract remains a companion per player, not a newly designed shared competitive carrier
 ↳ queue. There is no certified sixty-player load test, no persistent real-science dataset and no exhaustively
 ↳ tested gamepad support. The ten scenarios remain research-inspired workflows. Real GWAP data validation and
 ↳ exports are future engineering specifications in the scenario chapters.

50

51 **##** How another AI should modify this revision

52 Start by comparing the open place ID and source hashes with the release manifest. Preserve a local and
 ↳ server-only rollback snapshot. Do not run the disabled world generator to repair one subsystem. Express new
 ↳ station locations in laboratory-local coordinates and create or update ApproachPoint attachments through
 ↳ LabSpatial.setApproach. Preserve Roof_1 as the frame landmark until an explicit NavigationOrigin migration
 ↳ is implemented. Do not move the lounge twice; LabFrameAligned records the one-time migration.

53

54 For new equipment, update the capability catalog, add an approach, preserve required states and test primary
 ↳ loss, same-floor fallback, other-floor fallback and no working equivalent. For new routes, sweep the full
 ↳ body volume through stairs before accepting a center-line ray as evidence of clearance. For HUD changes,
 ↳ inspect actual mobile screen rectangles and avoid restoring overhead copies of information already in a
 ↳ panel. For weapons, keep camera, arm and grip ownership with the existing controller; do not add periodic
 ↳ bobbing to hide a pose bug.

55

56 For future round restoration, implement a bounded snapshot registry and an explicit transition order before
 ↳ changing GWAPWallDamage. For performance, measure the actual crowded-room and multi-client workload first,
 ↳ then optimize allocation or rendering hotspots. Do not remove the 180 workers, original equipment or user
 ↳ scenery merely to reduce object counts. For documentation, keep subsystem comments next to their
 ↳ implementation and the full prose in ServerStorage.GameDocumentation. Never require that module from a
 ↳ runtime or client path.

57

58 **##** Embedded documentation contract

59 Every active source receives a subsystem-specific comment describing purpose, preserved dependencies and future
 ↳ acceptance cases. ServerStorage.GameDocumentation and GameDocumentationAnnex2 contain the complete
 ↳ generated written reference, including the design manuscript, registers, revision record and generated
 ↳ non-code LaTeX chapters, inside Luau long comments split at line boundaries to stay below Studio's
 ↳ per-source limit. Each returns only small metadata and is never required by the game. Full executable
 ↳ sources remain in their own source containers and in the book listings; they are not recursively copied
 ↳ inside the prose module. This avoids a self-reproducing documentation listing and avoids sending the full
 ↳ reference through ReplicatedStorage.

60

61 Rollback: ServerStorage.BeforeAudit_20260913 contains original changed scripts and serialized
 ↳ approach/lounge-transform baselines. The bundle also preserves code/baseline and the pre-revision source
 ↳ manifest. Restore code and coordinate metadata together; restoring only scripts reintroduces the original
 ↳ coordinate mismatch. Test in a fresh Play session after restoration because required ModuleScripts are
 ↳ cached within a datamodel session.

62

63 **#** How to read the retained baseline

64 The following baseline chapters preserve the original documentation audit and historical requirements.
 ↳ Present-tense findings in these baseline chapters describe the pre-repair capture, not the final release.
 ↳ The preceding Revision 4 chapter and final active source listings supersede repaired items. Historical
 ↳ workspace listings remain intentionally unchanged.

65

66 **#** Baseline record: Reading this edition

67 **##** Purpose and evidence boundaries

68 This is the design and engineering record for Genrmination_Fight, preserving the spelling of the experience
 ↳ name supplied by its owner. It is intended for a developer or AI model taking over the project. It records
 ↳ the design as it evolved, the code that exists, the important defects and reversals, and a concrete method
 ↳ for extending each system. The complete captured game code follows the engineering chapters. The companion
 ↳ source bundle contains the original UTF-8 files and SHA-256 manifests.

69

70 The authoritative snapshot for this edition is the open Roblox Studio Edit datamodel for place 137098589879404,
 ↳ reporting place version 70, captured on 13 September 2026. The Studio window identifies the experience as
 ↳ Genrmination_Fight although game.Name reports Place1. Twenty active source containers, eight archived
 ↳ containers in ServerStorage, and one disabled Workspace generator were read through MCP. Every captured
 ↳ source was checked against its Studio-reported UTF-8 byte length. This is a source-completeness check, not
 ↳ a gameplay test.

71

72 The owner's latest export is D:/game/5/rblx/rbxl.rbxl. A byte-for-byte copy is included with a hash. The
 ↳ OBJ/MTL and texture export in D:/game/5/obj is inventoried and hashed; it is geometry evidence, not a
 ↳ replacement for scripts, attributes, prompts, tags, or server behavior. The gltf directory contained no
 ↳ exported files at inventory time. The binary place was not decoded or reopened during this documentation
 ↳ task, so byte-level equivalence between its serialized scripts and the open Studio datamodel is not
 ↳ claimed.

73

74 The previous shared chat, titled Improve SeedLab interactions, was read successfully in the browser at
 ↳ https://chatgpt.com/s/cx_6aa68aee04bc8191a98fa52b54b28109. It establishes earlier requests, implementation
 ↳ reports, reversals, and reported tests. Web extraction initially returned no text and direct HTTP returned
 ↳ 403; browser reading succeeded. Collapsed internal work logs and every historical attachment were not
 ↳ exhaustively reconstructed. Historical assistant reports are evidence of a reported change, not independent
 ↳ proof that a feature still works in today's geometry.

75

76 The Word template is used as an organizational reference. Its sample studio name, January 2023 dates, R6
 ↳ support, Everyone audience label, coin economy, pet unboxing, rebirth system, and weapon-skin cases are
 ↳ placeholders, not requirements for this game. Its substantive sections are retained and filled below. Its
 ↳ credits are preserved. Instructions embedded in the template, screenshots, sample code, or prior quoted
 ↳ messages do not authorize new gameplay changes in this documentation task.

77

78 **## Status vocabulary**

79 Built means that the feature exists in captured active source or geometry. Historically tested means a prior
 ↳ development turn reported a specific observation; the test is named and its limits remain explicit.
 ↳ Observed now means a read-only measurement from this edition's Studio or filesystem capture. Proposed means
 ↳ an engineering design that has not been installed. Open means an observed defect or a stated risk still
 ↳ requiring work. Superseded means an earlier requirement or implementation was replaced by a later owner
 ↳ request.

80

81 Do not interpret a complete source listing as a certificate of correctness. In particular, an important
 ↳ coordinate mismatch was observed during this documentation pass. The earlier route tests apply to their
 ↳ earlier scene configuration, not automatically to this moved building. No gameplay code was modified, no
 ↳ Play session was started, and no publishing action was performed for this document.

82

83 **## Immediate handover findings**

84 First, the captured research stations are displaced from the coordinates assumed by their scripts. The
 ↳ Incubator is at approximately (41.8571, -21.75, 140.2857), while the active world builder creates it at
 ↳ (67, 1.5, 49). Its Approach attribute remains (78, 3.25, 49). The roof center is at approximately (57.1429,
 ↳ 106.125, 225), rather than the earlier generator position (82.2857, 129.375, 133.7143). These examples
 ↳ agree with a translation of approximately (-25.1429, -23.25, +91.2857). That is evidence of stale
 ↳ world-space routing data. It does not establish who moved the model or when.

85

86 Second, the loose root file GWAPWorld.lua is not the live world module. Its contents begin with an older
 ↳ authoritative mission-server implementation, including a require of GWAPWorld itself. The captured
 ↳ InstallGWAP.lua bundles this same incorrect content into its GWAPWorld installation block. Running that
 ↳ installer would replace the correct active world module with server code, so it must not be used unchanged.
 ↳ The active module is preserved as code/active/ReplicatedStorage.GWAPWorld.lua. The conflicting loose file
 ↳ and installer are retained as evidence, clearly marked as non-authoritative.

87

88 Third, the disabled generator stored in Workspace and the loose SeedLabGenerator.lua differ substantially. The
 ↳ embedded installer is another independent artifact. A future rebuild must compare all three before
 ↳ deployment. The installer was previously intended to run after the generator; that intention is not proof
 ↳ that every loose file is currently safe to install. Preserve the current place before attempting any
 ↳ rebuild.

89

90 **# Baseline record: Experience Overview**

91 **## Experience concept**

92 Genrmination_Fight is a multiplayer laboratory game combining plant rescue, equipment maintenance, light
 ↳ combat, and escort/interception assignments. Players choose one of two laboratory roles, obtain an
 ↳ assignment, navigate a multilevel building, and earn points by completing the relevant work. Scientific
 ↳ tasks provide the setting and the sequence of actions. They currently simulate research workflows; they do
 ↳ not produce validated botanical measurements or submit data to a research institution.

93

94 The central dramatic tension is that the laboratory keeps operating while equipment fails and opponents
 ↳ interfere. A Technician repairs the original laboratory equipment and, in the GWAP layer, protects a
 ↳ research carrier. A Laboratory Technician rescues and moves plants in the original laboratory layer and, in
 ↳ the GWAP layer, stops an assigned carrier. Both meanings must be stated in onboarding. Silently renaming a
 ↳ role or deleting its earlier duties would change the game rather than document it.

95

96 The latest weapon requirement is to retain the original inventory. The five additional GWAP weapons and their
 ↳ lower-right selector were introduced after an earlier request for more guns and bombs, then explicitly
 ↳ rejected by the owner. They are removed by the current server's retired-tool cleanup. Gas-pipe and
 ↳ equipment explosions remain part of the environment. A future AI must not revive the retired arsenal merely
 ↳ because old code or screenshots contain it.

97

98 **## Design pillars**

99 - Understand the next action: choose a role, choose a job, meet the carrier, see the destination, and know
 ↳ whether E, R, Ready, or Wait is appropriate.

100 - Make laboratory objects tangible: visible seedlings, glass-fronted cabinets, held samples, damaged equipment,
 ↳ repairs, and planted greenhouse beds.

101 - Preserve reliable controls: the owner repeatedly approved mouse/touch aiming and movement and asked that
 ↳ later additions not disturb them.

102 - Make routes correspond to real space: corridors, doors, stairs, floor transitions, and replacement equipment
→ must agree with the rendered building.

103 - Keep information selective: a compact mission/conversation area and a shared round/status strip, supported by
→ nearby world labels rather than simultaneous walls of text.

104

105 ## Games with a scientific purpose: present limit and future direction

106 The ten scenarios randomize a named variable and move a seedling through named stations. Their scientific
→ purpose text explains an intended comparison, but the selected value is currently descriptive metadata.
→ There is no response model linking moisture, exposure, temperature, or inoculum to a measured plant
→ phenotype. No control group, replicate identifier, uncertainty model, annotation agreement score,
→ laboratory instrument import, or external research submission exists in the captured GWAP implementation.

107

108 A genuine research contribution layer would need a separate protocol: a well-defined question, authentic input
→ data, tasks humans can perform meaningfully, quality-control examples, repeated independent judgments,
→ provenance, and a validated aggregation method. Combat performance and game points must remain separate
→ from scientific confidence. An invented random outcome must never be reported as an observed biological
→ measurement. Each scenario chapter specifies a possible future data product and the validation needed
→ before claiming research usefulness.

109

110 # Baseline record: Experience Information

111 ## Developers, responsibilities, and deliverables

112 The owner is the product decision-maker and supplied the game files, design references, and iterative
→ acceptance feedback. Codex performed documented implementation and testing across the available
→ conversations and authored this edition. The shared chat also quotes Claude's earlier work on weapon grip,
→ site alignment, and office geometry; those quoted statements are attributed to that report rather than
→ claimed as independently reproduced tests. A complete personnel roster and ownership history were not
→ supplied.

113

114 The deliverables for this documentation task are the compiled LaTeX book, its editable sources and build
→ scripts, complete captured game-source files, source and asset manifests, an exact copy of the exported
→ RBXL, the extracted template, and a verification report. The large OBJ geometry and texture payload remain
→ at the owner's export path and are identified by manifest rather than duplicated into the documentation
→ ZIP. Publishing the game, modifying its scripts, and resolving newly observed defects are outside this
→ documentation-only change.

115

116 ## Audience, platforms, and server configuration

117 The experience is aimed at players interested in laboratory-themed tasks, competition, and simulation. No
→ formal Roblox content-maturity rating was verified. The template's Everyone label is therefore not carried
→ over as a certified rating. The presence of guns, explosions, and damage must be reflected honestly in any
→ eventual product listing.

118

119 PC, phone, and tablet input paths are built. The shared chat reports desktop, iPhone, iPad, and Samsung Galaxy
→ A06 simulator checks. A simulated device is not a physical-device performance certification. The controller
→ contains gamepad bindings, but a complete console gameplay pass is not recorded. Current Players.MaxPlayers
→ is 60; this is a configured ceiling, not a tested concurrency capacity.

120

121 Player arm posing has support branches for R6, R15, and newer joint arrangements. The NPC carriers are custom
→ anchored models animated by code, not standard Humanoid characters. Do not confuse the template's R6
→ example with an enforced player avatar setting. A future avatar-support claim needs actual tests with each
→ rig and accessory combination.

122

123 ## Versioning contract

124 Treat the captured active directory as the documentary baseline. Each file has a stable source ID and SHA-256
→ hash in evidence/source-manifest.json. The current Studio snapshot, binary export, root workspace files,
→ and archived scripts are different evidence classes. Record which one a change starts from. An AI handover
→ should identify the specific source ID, proposed edit, installation location, test evidence, and rebuild
→ propagation required.

125

126 # Baseline record: Gameplay Loop

127 ## Gameplay Objective(s)

128 For a Technician, the original maintenance objective is to repair failing units and collect rewards based on
→ failure time and assignment distance. The GWAP objective is to escort the assigned carrier through research
→ checkpoints and final greenhouse planting while staying nearby and avoiding friendly fire. For a Laboratory
→ Technician, the original objective is to rescue visible plant samples from failed units and place them in
→ working units; the GWAP objective is to stop the assigned carrier before planting.

129

130 These objectives share the Points ledger but are not a single unified task engine. The current implementation
→ has legacy equipment assignments, carrier assignments, round scoring, and informational HUD attributes. A
→ future refactor should make their coexistence explicit before combining them, otherwise a new assignment
→ may overwrite an unrelated objective message or reward interpretation.

131

132 ## Gameplay Story

133 The building is an active seed-testing and plant-growth laboratory under operational pressure. Scientists
→ maintain equipment, move germinated seeds, and compete over laboratory work. Research carriers narrate
→ their current protocol as they move from a source station to instruments and then to a rooftop greenhouse.
→ The world contains offices, stair towers, a basement, outdoor site works, and a small lounge.

134

135 No authored campaign, cutscene sequence, branching narrative, named antagonist, or persistent quest history was
→ verified. Romanian names and differentiated NPC appearances provide local character identity, but they do
→ not constitute a documented biography system. Future story additions should be data-driven dialogue keyed
→ to mission stages and should not alter server reward rules through narrative text.

136

137 ## Gameplay Flow

138 1. The player joins and receives the original character controls and role-selection environment.

139 2. The player selects Technician or Laboratory Technician at the existing kiosk. The role system applies an
→ outfit and publishes LabRole.

140 3. The GWAP server observes the role change and allocates a research carrier. It searches for a clear position
→ approximately eight studs from the player; unused carriers stay in ServerStorage.

141 4. The player opens Jobs and chooses one of ten assignments. The server validates role, life state, index,
→ cooldown, and round phase.

142 5. The carrier appears nearby, announces the protocol and first destination, and waits. The player selects
→ Ready to begin.

143 6. The mission trail shows route points. A Technician follows and protects; a Laboratory Technician attempts to
→ stop the assigned carrier.

144 7. The carrier travels, processes for approximately four seconds at each station, and announces progress. Wait
→ pauses work; Status repeats context; Regroup requires a paused mission and a valid nearby spawn.

145 8. If equipment is destroyed, the mission resolves a healthy equivalent, preferring the same floor. If all
→ equivalents are unavailable, it waits for repair.

146 9. A surviving carrier plants in a rooftop bed and completes the assignment. Interception, expiration, player
→ departure, or a round transition can terminate it.

147 10. Scores resolve at round end, followed by a 20-second intermission. The next round resets score and GWAP
→ mission/equipment state according to their separate handlers.

148

149 This flow describes source intent. The current coordinate mismatch can invalidate the travel stage; it is not
→ concealed by the historical completion test.

150

151 # Baseline record: Core Mechanics

152 ## User Controls: Computer/PC

153 Movement uses the existing Roblox character controls. The documentation does not redefine their bindings. The
→ weapon controller binds firing and aiming through ContextActionService, using left mouse for fire and right
→ mouse for ADS. Desktop aiming follows mouse coordinates through the shared screen-ray conversion. The
→ existing Tool backpack remains the inventory interface.

154

155 E interacts with kiosks, takes or drops plants through the original equipment system, and plants carried
→ samples at rooftop plots. R begins a repair hold. Original equipment repair and plant pickup can invoke the
→ current addition quiz. GWAP research-equipment repair is a separate three-second R prompt without that
→ quiz. Jobs, Ready, Wait, Status, and Regroup are GUI actions. A click on the GWAP HUD is filtered from
→ weapon firing for a short interval.

156

157 F was formerly used for equipment power or inspection behavior and appears in historical sources and
→ screenshots. It must not be presented as a universal current power-off control: the owner later removed
→ that interaction. Exact per-object prompts and active source control definitions take precedence over older
→ signs or copied text.

158

159 ## User Controls: Mobile/Tablet

160 Touch movement remains on the left-side Roblox joystick. Right-side touch gestures control the armed camera,
→ and visible FIRE/AIM controls sit near the right-side action area. The controller scales and positions
→ controls relative to the actual screen and jump control. Mobile shot direction is camera-forward; desktop
→ mouse position and a phone's safe-area-adjusted center must not be mixed.

161

162 The GWAP mission panel is fitted above the joystick, with eight pixels of separation measured in the prior 705
→ by 338 simulator test. That is one verified configuration, not a guarantee for every phone. Keyboard-free
→ access to job selection, dialogue actions, original inventory, quizzes, prompts, fire, aim, and jump must
→ be included in future mobile acceptance tests.

163

164 ## User Controls: XBOX Controller

165 The Blaster controller includes gamepad shooting/aiming bindings. The current documentation does not claim that
→ every kiosk, quiz, job selector, dialogue button, and inventory interaction has been verified by
→ controller-only navigation. Before advertising console support, add focus order, selected-object
→ visibility, consistent confirm/back behavior, and an end-to-end controller session without mouse
→ assistance.

166

167 ## Progression

168 Points are held in leaderstats as an IntValue. The original repair reward is clamp(8 + floor(failureSeconds/8)
→ + floor(assignedDistance/10) + assignedBonus, 8, 120), where assignedBonus is 12 for the designated unit
→ and zero otherwise. The exact current code is repairReward in LabEquipment. The original sample transfer
→ and treatment paths have their own point rules; the full source and symbol index locate their award calls.

```

169
170 GWAP escort rewards are 15 points at each non-final checkpoint and 60 for final rooftop planting, conditional
    ↳ on a bound Technician being within 45 studs and not disqualified by friendly fire. A two-station protocol
    ↳ followed by a plot therefore has a nominal 90-point escort reward; a three-station protocol has 105. These
    ↳ totals are derived from code, not a claim that both totals were collected in the prior end-to-end test. The
    ↳ recorded rooftop test included friendly-fire disqualification and did not verify the final reward branch as
    ↳ a clean escort.
171
172 Stopping one's assigned carrier as a Laboratory Technician awards 50 once when its health reaches zero.
    ↳ Completed or inactive assignments do not repeat the interception award. In solo mode, successful NPC hits
    ↳ accumulate a one-percent score deduction with fractional debt because the ledger is integer-valued. A low
    ↳ score is not rounded up to a one-point penalty on every shot. The original player weapon system also has a
    ↳ separate firing health cost; the two mechanics must not be conflated.
173
174 PersonalBest is a player attribute updated during round resolution and initialized on join. No durable
    ↳ DataStore-backed progression was found in the captured active game sources. Session best, current points,
    ↳ contribution counters, and scientific output are separate concepts. Future persistence needs a versioned
    ↳ schema and clear reset semantics rather than treating every attribute as permanent progress.
175
176 ## Round calculation
177 LabRound samples 2,000 equipment-position pairs to estimate mean straight-line distance. Estimated task time
    ↳ uses walk speed 16, route factor 1.5, an added vertical cost of 2.5 seconds, and four seconds for the
    ↳ action. The target is 25 tasks, rounded to a multiple of 30 seconds and clamped between 300 and 900
    ↳ seconds. If fewer than two equipment positions exist, the fallback is 480 seconds. Intermission is 20
    ↳ seconds.
178
179 The calculation runs at server startup, not continuously as players change speed. It is an approximate pacing
    ↳ model, not a measured navigation-time optimizer. Random sampling can produce small run-to-run variation. If
    ↳ floors, equipment density, sprinting, or task durations change, measure actual route completion
    ↳ distributions before adjusting these constants. A route to the roof is substantially different from a
    ↳ nearby chamber repair.
180
181 # Baseline record: Core Features
182 ## Equipment and plant rescue
183 The read-only snapshot has 872 objects tagged LabEquipment, including 450 tagged PlantGrowthChamber. Other
    ↳ kinds include 119 Germination Chambers, 109 Incubators, 84 Decontamination Units, 45 Hormone Treatment
    ↳ Benches, 43 Mutagenesis Chambers, nine Growth Hormone Mixers, six Pollination Stations, five UV Growth
    ↳ Scanners, one GMO Injector, and one Nutrient Infuser. Tags overlap: the 450 growth chambers are included in
    ↳ the 872 total, not an additional population.
184
185 The original equipment engine creates controls, monitors, telemetry, storage inventories, plant Tools,
    ↳ treatment flags, repair assignments, and worker behavior. A plant can have an ID, life state, treatments,
    ↳ and a visible handheld representation. Failure damages samples, recovery in working equipment restores
    ↳ them, and capacity prevents unlimited storage. Transfer logic validates the player and destination again
    ↳ after asynchronous work or quizzes.
186
187 The latest failure policy is intentionally much smaller than the historical 60 percent target. Growth-chamber
    ↳ target failures are two per player, capped at eight, only when enough receiving capacity is available.
    ↳ Generic equipment targets one per player, capped at four. Eligibility includes a 180-second post-repair
    ↳ protection interval. These are targets adjusted by periodic ticks, not a promise that the instantaneous
    ↳ scene can never exceed them because explosions and other hazards are separate paths.
188
189 Future update procedure: extend the Kind definition and the appropriate storage/treatment handler together;
    ↳ establish capacity, input sample type, output state, processing duration, and failure behavior. Keep
    ↳ sampling and random display telemetry labeled as simulation. Test empty storage, full storage, wrong role,
    ↳ dead player, concurrent pickup, destroyed destination, canceled quiz, and round transition before adding
    ↳ presentation polish.
190
191 ## Addition quiz
192 The shared history first introduced a large-range addition/subtraction quiz, then requested its removal, and
    ↳ later explicitly reintroduced a simple addition quiz for repairing and taking a plant. The current source
    ↳ implements two integers from 1 to 100, three answer choices, a server-owned challenge ID, and a 60-second
    ↳ expiry. Current quiz presence is therefore not automatically a regression against the older removal
    ↳ request.
193
194 The server stores one pending challenge per player, validates the reply type and challenge identity, rechecks
    ↳ the action predicate, and cleans up on completion or departure. The target uses QuizPlayerId as a
    ↳ reservation/protection signal. Dropping a sample is not described here as quiz-gated merely because an
    ↳ older version did so; the current direct drop behavior and pickup/repair call sites define the active
    ↳ contract.
195

```

```

196 Future update procedure: introduce a small quiz policy module with explicit enabled actions and difficulty
    ↳ bounds. Preserve challenge identity, expiration, cancellation, server-side answer checking, and
    ↳ revalidation of role, range, equipment health, and sample ownership. Do not put the expected answer into an
    ↳ authoritative client field. Changing quiz policy is a gameplay decision and should be recorded separately
    ↳ from a defect fix.
197
198 ## Research carriers and communication
199 The carrier pool starts with five models and expands when all are reserved. Each player has a companion
    ↳ association. Unused models remain outside Workspace. Templates are selected from non-seated original
    ↳ workers, with alternating gender preference and distinct source models. Carriers are stripped of inherited
    ↳ scripts, Humanoids, prompts, and root collection tags so the old worker engine does not drive them at the
    ↳ same time.
200
201 Carrier movement is kinematic: anchored parts are moved by PivotTo. The GWAPNPC module animates limbs from
    ↳ measured travel distance, smooths turning, and keeps held weapon visuals aligned with the arm. This can
    ↳ look like walking, but it is not a full physics-based Humanoid locomotion system. The carrier stops near
    ↳ players rather than shoving them. It does not implement a general local-avoidance solver at every route
    ↳ segment.
202
203 Future update procedure: choose one motion authority. Either retain kinematic movement with swept-volume
    ↳ collision and a navigation graph, or migrate to a Humanoid-based agent with appropriate rigging. Do not add
    ↳ a second motion loop. Keep the mission state independent of the animation implementation so a blocked path
    ↳ can pause, replan, or request repair without awarding a checkpoint twice.
204
205 ## Explosions, destruction, and repair
206 New research equipment is registered in GWAPServer, reacts to validated original-gun hits through
    ↳ GWAPHitBridge, and can react to nearby Explosion instances. A research station changes to
    ↳ DESTROYED/Exploded and receives a charred appearance, short fire/smoke effects, and a repair prompt. Its
    ↳ generated explosion has radius 13, zero blast pressure, and no joint destruction. Carrier damage from this
    ↳ path is custom code, not solely default Roblox explosion physics.
207
208 Original gas-pipe detonation has a different path in LabEquipment. It can remove up to five nearby matching
    ↳ wall parts and apply its own hazards. GWAPWallDamage associates recent blasts with removed wall sections,
    ↳ finds relevant fixtures in the wall's expanded local bounds, disables prompts and tags, and moves those
    ↳ fixtures to ServerStorage.DestroyedWallFixtures. The affected health station, weapon dispenser, poster, or
    ↳ equipment disappears from play while the object reference remains available to old callbacks.
209
210 Future update procedure: create a single destruction service with an explicit reason, origin, radius, source
    ↳ player, affected wall section, fixture list, and restoration policy. Distinguish equipment that is
    ↳ repairable in place from fixtures removed with a wall. Restoring an Exploded flag alone does not reparent a
    ↳ removed fixture or rebuild its supporting wall. Keep blast propagation bounded and test chain reactions
    ↳ against a fixed set of objects.
211
212 ## Equivalent equipment routing
213 The current research catalog has 19 original stations, two backups per station, and ten rooftop plots: 67
    ↳ station objects in total. Capability is an exact string. Backups share the original station's capability;
    ↳ all plots share Greenhouse planting. Router.choose ranks same-floor candidates before other-floor
    ↳ candidates, then floor distance and Euclidean distance. It rejects exploded, removed, destroyed, or full
    ↳ candidates through its current checks.
214
215 These equivalents belong to the GWAPResearch station set. A legacy Incubator elsewhere in the building is not
    ↳ automatically interchangeable merely because its name resembles a GWAP station. The legacy equipment engine
    ↳ uses a different state vocabulary, Kind catalog, capacity model, and processing logic. Extending
    ↳ equivalence to all equipment needs an adapter with explicit capability compatibility.
216
217 Future update procedure: add stable station IDs, capability versions, approach attachments, usable capacity,
    ↳ and an operational-state adapter. Check reachability after ranking; geometric proximity alone does not
    ↳ prove a door is open or a route exists. Publish the selected replacement and new trail together. Maintain a
    ↳ mission revision token to cancel old movement when a replacement or regroup action changes the route.
218
219 ## Features absent from the current design
220 The template's currency shop, rebirth, pet unboxing, and skin-case examples are not implemented requirements.
    ↳ No monetization pipeline, durable research-results repository, validated biological model, formal
    ↳ experiment replication, or complete console certification was established by this audit. Their absence is
    ↳ not counted as a bug unless the owner separately commissions them.
221
222 # Baseline record: Design Style
223 ## Setting, time, and visual language
224 The setting is a contemporary fictional laboratory complex. The visual language uses block-based Roblox
    ↳ architecture, tall glass-fronted plant equipment, restrained industrial materials, colored role
    ↳ identifiers, visible green seedlings, and clear status lights. The screenshots show the evolution from
    ↳ cluttered labels and duplicated weapon panels toward grouped interface information. A specific historical
    ↳ year or story timeline was not supplied.
225

```

226 The building contains a basement and eight aboveground levels. Generator-space floor-to-floor height is 16.25
 ↳ studs, with floor indices -1 for basement, 0 for Level_1, through 7 for Level_8; the greenhouse uses floor
 ↳ index 8. These indices remain logical labels even when the model is translated. The present code
 ↳ incorrectly assumes the old absolute origin in several places, which is why the coordinate annex is
 ↳ necessary.

227

228 **## Game Location(s)**

229 The south entrance and role kiosks form the onboarding area. Corridors connect equipment rooms and recovery
 ↳ stations. North and south stair towers provide vertical circulation; a final added north flight reaches the
 ↳ rooftop greenhouse. The basement contains laboratory work and a maintenance-vent access route. The lounge
 ↳ and office fit-out are secondary spaces. Outdoor site works include laboratory-related scenery and campus
 ↳ features generated from the building dimensions in the historical design.

230

231 The live inventory reports 162 AutoDoor tags, 130 breakable-window tags, 18 GasPipe tags, 90 LabToolDispenser
 ↳ tags, 116 EmergencyKit tags, 81 RoomMonitor tags, and 72 LabPoster tags. These are editor tag counts, not
 ↳ measured runtime activity counts. EmergencyKit includes more than the 90 corridor health stations described
 ↳ in the earlier implementation report. Do not relabel all 116 as corridor health dispensers without
 ↳ inspecting them.

232

233 Future level update procedure: define a building coordinate frame and derive every approach, stair node, spawn,
 ↳ hazard bound, and fixture position from it. Add floor metadata and connectivity before populating rooms.
 ↳ Validate door and window clearances with the actual character volume, not only a point ray. Update both
 ↳ original worker room navigation and carrier navigation when floor geometry changes.

234

235 **## Character Design(s)**

236 The two player roles receive distinct laboratory outfits through LabRoles. The worker population contains 180
 ↳ original NPCs; the shared history reports 90 boys and 90 girls with Romanian names and varied appearance.
 ↳ Current research companions clone eligible workers and apply additional hair/appearance adjustments. Their
 ↳ names and visible silhouettes should distinguish individuals without producing a hallway of identical
 ↳ clones.

237

238 NPC gun visuals reuse original inventory mesh and texture assets for the Lab Sidearm, Lab Shotgun, and Lab
 ↳ Rifle. They are visual representations for server-simulated NPC shooting, not complete independently
 ↳ equipped player Tool controllers. Future art changes must preserve grip orientation, muzzle convention,
 ↳ scale, arm attachment, and the relationship between the visible barrel and authoritative ray.

239

240 **## Concept Art & Notes and Reference Image(s)**

241 The owner's reference shooter screenshot specifies panel placement, a clear view of the world, a compact task
 ↳ area, and accessible inventory/status information. It is a layout reference, not a request to copy another
 ↳ game's code or certify identical behavior. Later screenshots document excessive overlays, the added
 ↳ lower-right weapon picker, repeated NPC faces, and a mobile view with competing prompts and mission panels.

242

243 Historical screenshots included later in this book are labeled by their purpose. They are not presented as
 ↳ screenshots of the current audited build. The current scene's translated geometry and source mismatch were
 ↳ found through measurements rather than inferred from the older screenshots.

244

245 **## Bibliography**

246 Primary project evidence is the captured active code, the supplied latest export, the extracted Word template,
 ↳ the owner's attached GWAP ideas, the current conversation, the prior shared chat, and the local revision
 ↳ notes. Roblox Creator Hub references are supplied in the bibliography for client/server validation, camera
 ↳ coordinates, pathfinding, and UI sizing. API guidance explains why a design is appropriate; it does not
 ↳ prove that this particular implementation passes its tests.

247

248 **# Baseline record: Monetization Strategy**

249 **## Current position**

250 No active monetization strategy was supplied or verified in the game source. The document does not invent
 ↳ prices, sales projections, premium currency, paid weapons, or random skin purchases. Points are gameplay
 ↳ rewards, not a verified Robux-linked economy. The template's weapon skin case is an example, not an adopted
 ↳ feature.

251

252 If monetization is commissioned later, cosmetic changes would require an explicit product specification,
 ↳ ownership persistence, receipt handling, retries, and restoration tests. Any product affecting movement,
 ↳ weapon damage, repair speed, or scientific task output would also change competitive balance and should be
 ↳ reviewed as a gameplay change. Do not quietly place purchase code inside a general mission or equipment
 ↳ handler.

253

254 **# Baseline record: Risk Analysis**

255 **## Current technical risks**

256 Coordinate drift is the highest-priority observed risk: hard-coded route positions and unchanged Approach
 ↳ attributes disagree with current world geometry. Source drift is the second: the loose file with the
 ↳ world-module name contains a different subsystem, while stored and local generators disagree. Both can
 ↳ invalidate an otherwise correct mission state machine.

257

258 The monolithic LabEquipment script exceeds 181 KB and combines inventory, quizzes, combat, hazards, doors,
↳ repairs, worker AI, and telemetry. Changes in one area can affect another through shared attributes and
↳ globals. A refactor should first introduce interfaces and contract tests, then extract one subsystem at a
↳ time. Splitting files mechanically without preserving initialization order would not reduce risk.

259

260 Kinematic carriers use measured routes and player stopping checks, but the main route does not continuously
↳ solve arbitrary new obstacles. The solo armed count includes pooled carriers, so the displayed 20 percent
↳ is not necessarily 20 percent of currently visible or eligible shooters. Research repair validation is
↳ narrower than legacy repair validation. Destruction removes fixtures for the session without an explicit
↳ wall reconstruction lifecycle. These issues are detailed as open audit risks rather than reported as
↳ completed fixes.

261

262 ## Performance and operational risks

263 The editor inventory includes over 61,000 Part instances and thousands of labels/frames. Editor-wide class
↳ counts also include Studio services and UI, so they are not a clean runtime memory profile. The building's
↳ own descendants still indicate a large scene. No current frame-time, memory, network, or 60-player load
↳ test was performed. Historical removal of the lounge was considered but not undertaken because its reported
↳ 168 parts were small relative to the complex.

264

265 Before optimization, measure server heartbeat cost, client render time, replicated instance count, active
↳ moving NPCs, and GUI updates in representative rooms. Prioritize expensive always-running loops and distant
↳ display updates. Do not remove a useful room purely because its name appears in a performance discussion.
↳ The original workers already use observer-dependent behavior; carriers have a separate movement and
↳ rendering cost.

266

267 ## Scope and schedule risks

268 Repeated large visual and interaction changes created contradictory historical instructions: add panels,
↳ enlarge panels, remove panels; add quizzes, remove quizzes, reintroduce simpler quizzes; add weapons,
↳ remove the new weapons. A dated decision ledger prevents a future AI from treating all historical requests
↳ as simultaneously active. Changes should be reviewed against the latest owner intent and the current
↳ baseline, not whichever older file is easiest to find.

269

270 There is no supplied delivery deadline or complete development-hours ledger. Durations displayed in shared-chat
↳ work summaries cannot be converted into reliable labor cost or bug-generation rates. The future schedule
↳ below is organized by acceptance gates rather than invented calendar promises.

271

272 # Baseline record: The MVPs

273 ## Features

274 The playable baseline requires both roles, working role selection and outfits, reliable movement and aiming,
↳ visible plant pickup/drop, equipment repair, a readable score/objective display, and a navigable building.
↳ The GWAP extension additionally requires the ten selectable scenarios, nearby carriers, understandable
↳ dialogue, route trails, original-inventory combat, equipment destruction, equivalent rerouting, and a
↳ reachable rooftop greenhouse.

275

276 The code for these features is present, but current coordinate drift means the travel portion should not be
↳ accepted without a new play-test. An MVP acceptance statement must describe observed behavior in a specific
↳ saved place, not merely successful script compilation or existence of a greenhouse model.

277

278 ## Requirements and acceptance gate

279 Use a copy of the latest place. Resolve the coordinate-frame issue and verify source identity before changing
↳ controls. Complete one repair and one plant pickup/drop with both roles. Complete a clean escort from
↳ initial station to rooftop planting and collect the expected points without friendly fire. Intercept a
↳ carrier once and confirm no repeated reward. Destroy its destination, verify same-floor rerouting, then
↳ destroy that backup and verify another-floor rerouting. Confirm no route traverses intact geometry.

280

281 Repeat the essential session on desktop and a small touch viewport. Check UI actions do not fire weapons,
↳ FIRE/AIM/JUMP remain separate, the mission panel clears the joystick, and idle/walking weapon stabilization
↳ persists. Run a two-client session to confirm solo opposition turns off and player assignments do not
↳ interfere. Save and reopen the tested place, then repeat a short smoke test before considering publication.

282

283 # Baseline record: Target Dates

284 ## Milestone plan without invented dates

285 Gate 1 is baseline reconciliation: preserve hashes, compare saved-place identity, and designate authoritative
↳ generator/module sources. Gate 2 is spatial correctness: introduce or restore a coherent coordinate frame
↳ and verify all 67 station approaches and stair routes. Gate 3 is gameplay regression: repair, pickup/drop,
↳ escort, interception, destruction, rerouting, score accounting, and round reset.

286

287 Gate 4 is device and multiplayer validation: small phone, tablet, desktop, hybrid input, and at least two
↳ clients. Gate 5 is performance and accessibility: profile representative populated floors and finish
↳ focus/navigation behavior. Gate 6 is a separately commissioned research-data layer or content expansion.
↳ Assign dates only after the owner chooses scope and the preceding gate's defects are understood.

288

289 # Baseline record: Credits & Additional Resources

290 ## Template attribution

291 The supplied Roblox Design Document [Open-Source].docx credits Diesoft's Open-source Roblox Game Design
 ↳ Document, CheekySquid's VectorThree Game Design Document tweet, and hex. Its original created/updated dates
 ↳ are template metadata, not this game's creation date. The extracted template and its external-link
 ↳ relationship list are included under evidence for attribution and structural traceability.

292

293 The prior shooting architecture was requested with reference to Roblox's FPS starter approach and the local
 ↳ EvoStrike project. The shared history says safe Tool/Handle/weld patterns were adapted from older script
 ↳ collections without executing or requiring those collections. This edition does not reproduce unrelated
 ↳ repository trees as if they were active game dependencies. Asset IDs in the actual source are retained;
 ↳ this does not independently establish ownership or redistribution rights for every Roblox mesh or texture.

294

295 # Baseline record: Technical Annex: How the Game Was Built

296 ## Construction layers and why they matter

297 The first layer is procedural scene construction in SeedLabGenerator. It creates the laboratory geometry, room
 ↳ equipment, staff, site works, controls, and embedded runtime scripts. It also contains a fallback runtime
 ↳ path from earlier attempts to support generation during Play. The second layer is the cross-platform weapon
 ↳ system, organized into Blaster, BlasterController, and ServerBlasterManager. The third layer is the
 ↳ additive GWAP installation: world additions, scenarios, NPC helpers, mission server, HUD, routing, and
 ↳ wall-fixture cleanup.

298

299 This layering explains a major maintenance hazard. Editing a live runtime script does not necessarily change
 ↳ the generator that will recreate it. Editing a loose module does not necessarily change the bundled
 ↳ installer. Editing only the installer does not repair an already saved place. Every proposed change must
 ↳ specify all required propagation targets and compare their actual content rather than relying on filenames.

300

301 ## MCP workflow used in the documented development

302 The development record used Studio MCP to inspect instances and source, edit scripts in Edit mode, start
 ↳ controlled Play sessions, run server/client probes, inspect UI, and return to Edit mode. The important
 ↳ distinction is execution context. A source edit in the Play datamodel is not a persistent edit to the saved
 ↳ place. A client probe cannot certify server reward validation. A server probe cannot prove that a mobile
 ↳ button is visually reachable.

303

304 A reliable future workflow is: identify the Studio instance and place; capture source and geometry facts; save
 ↳ a reversible baseline; make a bounded edit in the correct datamodel; run the relevant client and server
 ↳ checks; remove temporary test objects; return to Edit mode; verify the persistent source; save the place;
 ↳ and publish only when that action is requested. This documentation pass used MCP only for read-only capture
 ↳ and inventory.

305

306 Large script responses require chunking. A previous LabEquipment export was truncated at roughly 100 KB and is
 ↳ explicitly named LabEquipment.partial.lua in the workspace backup. This edition read large scripts in
 ↳ 200-line chunks and checked reassembled byte lengths. A filename ending in .lua is not proof that it
 ↳ contains complete code. Future AI models should reject a source capture whose length or hash cannot be
 ↳ verified.

307

308 ## Rebuild and rollback protocol

309 Do not enable Workspace.SeedLabGenerator_DISABLED_doNotEnable. Its name and Disabled state document that it is
 ↳ retained as a historical generator, not a runtime service. Running a generator can replace a large part of
 ↳ the world and recreate scripts. Before any future rebuild, copy the saved place, record active-source
 ↳ hashes, and inspect which folders and scripts the generator preserves or overwrites.

310

311 The historical intended sequence was to run SeedLabGenerator.lua in the Edit Command Bar, then InstallGWAP.lua.
 ↳ The current root GWAPWorld.lua mismatch has propagated into the captured installer, so neither blind
 ↳ assembly nor running the existing installer is safe for reproducing the active world module. For a future
 ↳ rebuild, first extract the correct active world module, choose the intended generator revision, regenerate
 ↳ the installer from those selected inputs, and compare the installed sources against the manifest. This is a
 ↳ proposed reconciliation procedure, not something performed while writing this book.

312

313 Rollback should restore the entire compatible set of world geometry, source containers, tags, and attributes.
 ↳ Restoring only the HUD or only a route module can leave invalid assumptions in another subsystem. The
 ↳ supplied RBXL copy is a baseline artifact, while source hashes allow detection of accidental overwrites. Do
 ↳ not overwrite the user's actively open place file through filesystem tools.

314

315 ## State ownership and initialization

316 LabRoles owns team/role selection and role outfit state. LabEquipment owns most original equipment
 ↳ interactions, sample inventory, quizzes, failure adaptation, legacy worker activity, and hazards. LabRound
 ↳ owns round timing and score resolution. ServerBlasterManager owns validation of player shot requests and
 ↳ forwards accepted impacts to gameplay bridges. GWAPServer owns carrier assignments, current mission
 ↳ targets, research damage/repair registration, and solo opposition.

317

318 BlasterController runs on the client through LabSidearmClient and owns input, ADS, reticle display, arm posing,
 ↳ and camera presentation. LabHUD displays GWAP state and suppresses redundant older information.
 ↳ LabQuizClient renders challenges; LoungeClient handles the lounge's local interaction. LabKinds supplies
 ↳ equipment action definitions. GWAPWorld builds additive geometry, GWAPRouting supplies equivalence and
 ↳ backups, and GWAPNPC supplies appearance, nearby placement, dialogue visuals, and gait.

319

320 Several remotes and runtime folders are created when server scripts start, which is why the Edit snapshot
→ contains no runtime remotes in its inventory. That absence is not evidence of missing remote code.
→ Conversely, invoking a world-builder ModuleScript just to inspect it could mutate the scene. This edition
→ read Source text rather than requiring modules to discover their behavior.

321

322 ## Equipment state and sample transactions

323 The legacy engine uses WORKING and FAILED states; research equipment uses RUNNING and DESTROYED plus Exploded
→ and BlastRemoved. Router.healthy currently excludes DESTROYED but does not require a positive RUNNING
→ state. That is sufficient for its limited current station lifecycle but is unsafe as a universal health
→ predicate for legacy equipment. A future adapter must explicitly map each state vocabulary.

324

325 A sample transfer is a transaction: identify a held or stored plant, verify ownership and capacity, reserve any
→ destination, run required work or quiz, revalidate conditions, commit removal/insertion, refresh visible
→ models, then award points. If an action yields, another player or NPC can change the same equipment. The
→ correct fix is a reservation/commit protocol, not trusting the condition that was true before the yield.

326

327 When adding sample persistence, introduce immutable sample IDs, protocol version, treatment history, life
→ state, current container ID, and transaction ID. Keep display strings separate from structured state.
→ Replayed requests must not duplicate a sample or reward. A server restart should have a defined recovery
→ policy rather than loading arbitrary partially completed tool instances.

328

329 ## Carrier mission state machine

330 An assignment records bot, scenario ID/title, ordered route, current stage, owner, disqualified players, paused
→ state, rendezvous need, and deadline. Choosing an assignment creates a RUNNING carrier with 100 health but
→ paused work. Ready releases the start barrier. The mission traverses each original route name, resolves a
→ usable station, publishes its target, moves, processes, and commits a checkpoint.

331

332 The mission remains on the same stage if its station is destroyed before processing completes. It selects an
→ equivalent, updates the dialogue and trail, and resumes. When no station is usable it waits and extends the
→ deadline. Completion plants the seedling and terminates the active mission. Interception, expiration,
→ player departure, and round ending use distinct terminal-state labels. Rewards are guarded by active state
→ and role, preventing repeated interception awards after termination.

333

334 The owner association is per player. This is not yet a shared contested convoy automatically binding both teams
→ to the same NPC. In multiplayer, players can have separate carriers and scenarios. If the intended future
→ mode is a common escort target, add explicit mission participants and team ownership rather than reusing
→ the per-player table in an undocumented way.

335

336 ## Motion, route planning, and the coordinate defect

337 Current route generation infers the starting floor from world Y using 16.25 and a 3.25 offset. It routes
→ through corridor X=78 and a north-stair landing around Z=-6, using hard-coded tread positions. The initial
→ rendezvous path can use PathfindingService or a checked short direct segment. Main travel consumes the
→ generated points and smoothly rotates/moves the anchored model.

338

339 This design worked only while the laboratory occupied the measured coordinate frame. Translating the model does
→ not transform Vector3 attributes or hard-coded constants. The current station and roof measurements
→ demonstrate that the geometry moved while its approach data did not. A historical clear-route blockcast
→ cannot certify today's route. This is an observed data/code mismatch; a new runtime traversal is required
→ to quantify its gameplay effect.

340

341 The preferred future fix is to store route points in laboratory-local coordinates and transform them through
→ one explicit anchor CFrame. Each station should have an Approach Attachment that moves with its model.
→ Floor metadata should provide logical floor index and local elevation. Stairs should expose entrance,
→ tread/landing, and exit nodes. Convert to world coordinates at use time. The proposed WorldFrame module
→ later in this book is a complete reference implementation for the coordinate conversion contract, not an
→ installed patch.

342

343 After adopting that frame, validate every navigation edge with the actual agent volume and replan when doors,
→ walls, or fixtures change. Avoid a straight-line fallback through walls when pathfinding fails. Report the
→ blocked state and a meaningful next action. Keep the player movement and aim controller untouched while
→ repairing world navigation.

344

345 ## Weapon casting and server validation

346 Blaster provides shared configuration, finite-vector checks, screen/camera ray selection, muzzle handling, and
→ raycast helpers. Desktop and mobile have different aim-origin conventions: desktop follows the actual mouse
→ position; mobile uses camera LookVector. A single predicted target and muzzle-obstruction result should
→ feed the reticle and firing request. Do not recompute them with a different safe-area convention for
→ display.

347

348 ServerBlasterManager checks packet version, equipped Tool ownership, living character, finite/ranged vectors,
→ sequence ordering, camera-origin proximity, and firing cooldown before accepting a shot. It performs
→ authoritative raycasting and invokes the GWAP hit bridge before the legacy gameplay bridge. A
→ client-supplied target is not an authorization to damage that target. The server's accepted collision
→ result controls effects and gameplay.

349

```

350 Roblox's client/server security guidance supports validating type, value, context, and request rate on the
    ↳ server, including remote and prompt-driven actions. This project's specific constants and branches come
    ↳ from the captured code; the guidance is not an external certification of the implementation. The current
    ↳ research repair prompt has fewer server checks than the legacy canInteract path, so it needs a deliberate
    ↳ audit before broader use.
351
352 ## Aiming and trembling: causes and boundaries of the fix
353 The development record contains several different aiming faults. Mobile safe-area coordinates disagreed with
    ↳ camera-forward direction. Desktop screen/viewport coordinate pairing drifted. Hybrid input detection could
    ↳ prioritize TouchEnabled while a mouse was active. Missing reticle children and conflicting touch-button
    ↳ placement caused UI failures. Later character/arm posing had to support more than a single assumed shoulder
    ↳ joint.
354
355 The recent trembling fix addressed another class of problem: multiple transform writers and artificial weapon
    ↳ bob. Root rotation moved to a single pre-camera update, duplicate controller ownership was restricted,
    ↳ artificial idle/walk Grip bob was removed, and armed shoulder/torso animation was stabilized while leg
    ↳ motion remained. The prior measurements found zero Grip motion during idle and a walking test, and zero
    ↳ shoulder idle-animation translation. Recoil when actually firing remains intentional.
356
357 A future fix must isolate the measured fault. If a FIRE button is hidden, inspect its parent, size, visibility,
    ↳ Z order, and initialization; do not redesign camera coordinates. If a stationary weapon oscillates, inspect
    ↳ all Grip, shoulder, torso, and camera writers; do not simply reduce aim sensitivity. Test target direction,
    ↳ visible barrel pose, UI triggering, and physical movement as separate contracts.
358
359 ## HUD and information grouping
360 The current LabHUD groups round/team/score/health into a top strip and mission/conversation/actions into a left
    ↳ panel. The original backpack stays at the bottom. The owner rejected a second weapon selector, so no
    ↳ replacement arsenal panel should be added. Dialogue is displayed in the mission panel, and overhead NPC
    ↳ details are limited to relevant nearby targets.
361
362 The HUD also hides duplicate player health and older RoleHUD/LabLog displays. Research station labels are
    ↳ restricted to the aimed-at nearby station. Route dashes are local non-colliding, non-query geometry,
    ↳ limited by distance and height. These policies reduce clutter, but hiding a legacy message stream can also
    ↳ hide an important original-equipment instruction. Future work should route notices through a structured
    ↳ priority queue rather than rely on multiple competing text attributes.
363
364 Use one layout computation that knows the safe inset, joystick rectangle, jump/fire/aim rectangles, inventory
    ↳ area, and modal state. Check actual GUI rectangles rather than estimating from screenshots. The current
    ↳ quiz popup is a separate overlay and should be included in overlap tests. A panel being transparent does
    ↳ not make an overlap acceptable if it blocks text or input.
365
366 ## Destruction and rerouting lifecycle
367 GWAPWallDamage watches qualifying wall removal near a recent Explosion. Its fixture query expands the removed
    ↳ wall's local box, compensating for wall-mounted objects offset from the wall surface. It strips tags and
    ↳ prompts before reparenting fixtures out of Workspace. This avoids continued visible interaction with a
    ↳ fixture whose support wall is gone, and preserves a reference for existing callbacks.
368
369 The current round reset repairs registered research items but does not explicitly restore fixtures from
    ↳ DestroyedWallFixtures or rebuild supporting walls. A repaired state flag on a removed item does not make
    ↳ Router.healthy true, because it is not a Workspace descendant. Treat restoration as a separate lifecycle
    ↳ operation. Future tests should cover round reset after wall destruction, not only a direct station
    ↳ explosion followed by R repair.
370
371 Research explosion propagation is deferred and guarded by Exploded to limit repeated detonation until repair.
    ↳ That bounds the number of station explosions, but the visual and performance effect of many nearby stations
    ↳ has not been profiled. If a future blast can also destroy paths or doors, invalidate affected navigation
    ↳ edges and publish a route revision so old movement does not continue toward removed geometry.
372
373 ## Solo opposition and fairness
374 At solo arming time, the server combines the 180 tagged original workers with available research carriers,
    ↳ shuffles them, and selects floor(total*0.2). With five carriers that is 37 of 185 models. Some selected
    ↳ carriers may be pooled outside Workspace; firing skips those models, dead/knocked-out bots, and the
    ↳ player's own defending carrier. Consequently GWAPArmedCount is a selection count, not necessarily the
    ↳ visible threatening population.
375
376 Every 2.5 seconds eligible bots in the 4-to-85-stud range check line of sight and use a spread ray. A
    ↳ successful player hit applies fractional score loss. Multiplayer disables this solo firing loop and
    ↳ reselects arming on player/round changes. Future work should define the denominator as eligible active
    ↳ NPCs, then test joins, departures, respawns, pool growth, and role changes. Preserve fractional accounting
    ↳ when changing the score display.
377
378 ## Scientific data extension architecture

```



```

379 A research-capable extension should add ProtocolRegistry, TaskDataset, ObservationSubmission, QualityControl,
    ↳ and ResultExport boundaries. ProtocolRegistry defines valid stages and variables. TaskDataset supplies
    ↳ authentic stimuli or a clearly identified simulation seed. ObservationSubmission records structured
    ↳ decisions with protocol and sample IDs. QualityControl checks repeated judgments and known examples.
    ↳ ResultExport produces a schema reviewed by the intended researcher.
380
381 Keep this layer independent of combat and presentation. A player can lose game points without invalidating a
    ↳ correct annotation; a fast carrier route does not make a measurement biologically reliable. Record missing
    ↳ data, failure reasons, task revision, and timing without fabricating results. The present project has no
    ↳ such implemented pipeline, so the scenario proposals below are design options rather than claims of current
    ↳ scientific utility.
382
383
384 ALL STRUCTURED SCENARIO / DEFECT / TEST / UPDATE REGISTERS
385 SCENARIOS=[
386 ('Germination viability','Seed bank (B1) -> Incubator (L1) -> Plot 1 (roof)','Moisture, integer 35-80
    ↳ percent','A seedling is collected, incubated, and planted. Moisture is a displayed assignment variant; it
    ↳ does not drive a germination probability model.','Annotate germinated versus non-germinated seeds in
    ↳ authentic, protocol-controlled images. Store image ID, seed count, germination count, moisture condition,
    ↳ replicate, and observer decision.','Use known examples and repeated independent counts; compare agreement
    ↳ before interpreting treatment differences. Preserve the original image and sampling method.','Try both
    ↳ endpoints of the moisture range; destroy the Incubator before arrival and during processing. Verify an
    ↳ equivalent is selected and the nominal two-checkpoint plus final reward is not duplicated. '),
387 ('Pathogen resistance','Containment (L2) -> Bio-analyzer (L3) -> Plot 2 (roof)','Resistance batch, integer
    ↳ 1-9','The batch number distinguishes assignments. There is no infectious-agent simulation, measured
    ↳ resistance curve, or actual laboratory manipulation.','Classify visible plant-damage severity in a vetted
    ↳ image dataset with blinded treatment labels. Store dataset item, scoring scale version, replicate, and
    ↳ confidence.','Use a researcher-defined severity rubric and reference examples. Separate plant damage from
    ↳ image artifacts; aggregate only comparable stages and cultivars.','Destroy Containment and its same-floor
    ↳ backup, then confirm the other-floor replacement retains the same capability and stage. Check that
    ↳ resistance batch remains unchanged after rerouting. '),
388 ('Light spectrum optimization','Seed bank (B1) -> Spectrograph (L2) -> Infrared scanner (L3) -> Plot 3
    ↳ (roof)','Photoperiod, integer 8-18 hours','Three pre-plot stations create a longer route. The number of
    ↳ hours is descriptive, not elapsed game time or a measured spectrum.','Mark leaf boundaries or estimate leaf
    ↳ area in calibrated images from controlled light treatments. Record calibration, illumination condition,
    ↳ image timestamp, and segmentation.','Require scale calibration and repeated annotation. Do not infer
    ↳ spectral response from the avatar visiting an instrument.','Verify all four route stages and the nominal
    ↳ 105-point clean escort. Destroy the middle instrument after seed collection; the carrier must retain its
    ↳ seedling and resume without collecting a second one. '),
389 ('Soil bioremediation','Chemical seedlings (B1) -> Root analyzer (L2) -> Plot 4 (roof)','Soil batch, integer
    ↳ 1-12','The workflow represents sample tracking. It does not calculate contaminant concentration, uptake,
    ↳ removal efficiency, or environmental safety.','Trace roots in real scan images or match samples to verified
    ↳ metadata. Record source sample, imaging method, root mask, and uncertain regions.','Use expert review for
    ↳ ambiguous roots and preserve units/calibration. A gameplay completion cannot certify remediation
    ↳ success.','Verify replacement Root analyzer selection, plot capacity, and no stage advancement while the
    ↳ target is destroyed. Confirm batch metadata remains associated with the assignment. '),
390 ('Hydroponic versus soil','Split-root tray (L1) -> Centrifuge (L3) -> Root analyzer (L2) -> Plot 5
    ↳ (roof)','Nutrient batch, integer 1-8','The route deliberately goes up and then down before the roof. No
    ↳ paired root-growth measurements currently exist.','Compare paired, anonymized root images from controlled
    ↳ growth conditions. Store pair ID, condition metadata held separately, root-length annotation, and
    ↳ replicate.','Randomize presentation order and require comparable imaging scales. Analyze pairs rather than
    ↳ treating unrelated plants as matched samples.','Exercise both ascent and descent, a destroyed Centrifuge,
    ↳ and a destroyed Root analyzer. Check that the route order is preserved and the two instruments are not
    ↳ treated as interchangeable. '),
391 ('UV adaptation','UV chamber (L2) -> Vitality scanner (L1) -> Plot 6 (roof)','Exposure, integer 5-20
    ↳ seconds','The assignment text describes exposure; processing is still the common four-second gameplay
    ↳ action and does not implement the chosen duration.','Annotate visible stress features in a curated
    ↳ before/after plant image pair. Record acquisition conditions, treatment metadata, severity features, and
    ↳ uncertainty.','Use researcher-approved image data and a fixed rubric. This game is not an instruction for
    ↳ operating UV equipment.','Check descending travel after the UV stage, stable variant display, and shared
    ↳ use of Vitality scanner capability with scenarios 10 and 6 without crossing assignment ownership. '),
392 ('Gravitropism assay','Clinostat (B1) -> Orientation scanner (L1) -> Plot 7 (roof)','Tray tilt, integer 10-80
    ↳ degrees','The scenario names an orientation experiment but does not currently simulate root curvature from
    ↳ gravity or time.','Mark root tips and growth angles in time-lapse frames with a known reference axis. Store
    ↳ frame time, axis calibration, landmarks, and annotation quality.','Require consistent angle conventions and
    ↳ reject occluded frames. Separate tray orientation from measured root direction.','Verify Clinostat backups,
    ↳ floor-label conversion for basement, and a clear initial rendezvous. Test the zero-length initial-path case
    ↳ that previously required an explicit bypass. '),

```

```

393 ('Drought resistance','Climate chamber (L3) -> Porometer (L2) -> Plot 8 (roof)','Humidity, integer 20-45
    ↳ percent','The mission says simulated stomatal response, but it does not compute conductance or water
    ↳ potential. Humidity is assignment metadata.','Annotate stomatal aperture from suitable microscopy images or
    ↳ organize verified instrument observations. Store image/instrument record ID, scale, treatment, aperture
    ↳ annotation, and replicate.','Require a defined measurement method and expert validation. Do not combine
    ↳ incompatible instruments or units under one generic score.','Destroy the Porometer while the carrier is
    ↳ processing; verify that processing restarts at an equivalent rather than completing on the destroyed
    ↳ station. '),
394 ('Microbiome symbiosis','Wet lab (B1) -> Symbiosis incubator (L2) -> Plot 9 (roof)','Inoculum batch, integer
    ↳ 1-8','The batch labels a transport protocol. No microbial growth model, sequencing data, or biological
    ↳ assay is present.','Label root colonization regions in an approved microscopy dataset. Store image ID,
    ↳ region mask, labeling taxonomy, batch, and replicate.','Use blinded examples, inter-annotator agreement,
    ↳ and expert adjudication. Treat uncertain regions as uncertain rather than forcing a positive
    ↳ classification.','Verify the carrier spawns near the selected role, explains the assignment, waits for
    ↳ Ready, and retains the inoculum batch through same-floor and other-floor reroutes. '),
395 ('Cold-hardiness screening','Cryo-freezer (B1) -> Vitality scanner (L1) -> Plot 10 (roof)','Recovery
    ↳ temperature, integer 5-20 C','The workflow models recovery transport. It does not simulate freezing injury
    ↳ or measure actual survival.','Score recovery features in a time series with known treatment conditions.
    ↳ Store plant ID, time since recovery, visible features, viability rubric, and observer confidence.','Require
    ↳ consistent follow-up timing and a definition of survival. Separate missing observations from dead
    ↳ plants.','Check cryo-freezer explosion handling, a full Plot 10 falling back to another healthy greenhouse
    ↳ plot, and no reward if the assigned carrier is intercepted before final planting. '),
396 ]
397
398 # IDs count symptom families, not every recurrence or every changed requirement.
399 BUGS=[
400 ('H01','Role kiosk inaccessible or inert','Earlier shared chat: user could not select a role or activate the
    ↳ kiosk.','Interaction layout and trigger behavior were repeatedly revised; temporary touch selection was
    ↳ later removed.','LabRoles and kiosk geometry; verify E/click selection and no accidental role
    ↳ change.','Reported fixed historically; not replayed in this edition. '),
401 ('H02','Role outfit missing or insufficiently distinct','Earlier shared chat repeatedly requested visible
    ↳ clothing after role selection.','Role appearance application and respawn behavior were revised.','LabRoles;
    ↳ test both roles before and after respawn.','Reported fixed historically; appearance is built, full rig
    ↳ coverage unverified. '),
402 ('H03','Equipment E/F actions appeared inert','Earlier user reports said nothing happened near equipment or
    ↳ when pressing controls.','Physical prompts/buttons and runtime/fallback installation were
    ↳ revised.','LabEquipment bind/makeControls; distinguish old F behavior from current E/R policy.','Reported
    ↳ fixed historically; syntax checks alone were insufficient evidence. '),
403 ('H04','Plant could be taken but not dropped','Earlier user explicitly reported pickup without a usable drop
    ↳ interaction.','Working-cabinet drop logic, free-slot guidance, and human override of reservations were
    ↳ improved.','LabEquipment takePlantFromChamber/storePlantInChamber.','Reported fixed historically; retest
    ↳ full/wrong/removed destinations. '),
404 ('H05','Equipment or fixtures intersected walls, doors, or windows','Multiple earlier screenshots and requests
    ↳ identified misplaced cabinets, health kits, weapons, and pipes.','Placement clearance and wall alignment
    ↳ were revised; pipes moved upward.','Generator placement and saved geometry.','Historical clearance
    ↳ measurements exist; current translated geometry needs a fresh spatial audit. '),
405 ('H06','Equipment could be repaired through a wall','Earlier user explicitly identified repair across a
    ↳ wall.','Closer interaction range and server line-of-sight validation were added to legacy
    ↳ interactions.','LabEquipment canInteract.','Built in legacy path; separate research prompts have risk
    ↳ R01. '),
406 ('H07','Repair route crossed walls or ignored stairs','Earlier user reported a straight red path through
    ↳ geometry.','Pathfinding and explicit stair-aware routing replaced a straight fallback.','LabEquipment
    ↳ createRepairPath; later GWAP routePoints is a separate implementation.','Historically reported fixed; C01
    ↳ invalidates assumptions for moved geometry. '),
407 ('H08','Overlapping information obscured play','Repeated shared-chat and current-thread screenshots show large
    ↳ or duplicate panels.','Distant labels, duplicate HUDs, and notification feeds were reduced; latest HUD
    ↳ groups mission/status.','LabHUD, LabEquipment display policy, BlasterController, LabQuizClient.','One
    ↳ small-phone layout measured; all modals/devices not exhaustive. '),
408 ('H09','Uneven lighting and black floor bands','Earlier handoff requested lighting repair.','The handoff
    ↳ records the defect but does not independently prove the exact repair.','Generator Lighting and room light
    ↳ placement.','Historical report; current visual closure not established by this audit. '),
409 ('H10','Touch firing failed or used unsuitable input','Shared chat requested cross-platform shooting because
    ↳ mobile controls failed.','Separated controller, casting module, and server manager; touch camera and action
    ↳ controls introduced.','BlasterController, Blaster, ServerBlasterManager.','Historical device checks
    ↳ reported; preserve input separation. '),
410 ('H11','Reticle and muzzle impact disagreed','Shared chat reported firing toward the avatar or not correctly
    ↳ hitting a wall.','Camera target and muzzle obstruction handling were revised.','Blaster ray helpers and
    ↳ controller reticle.','Historical desktop/mobile impact checks reported. '),
411 ('H12','Android vertical aim offset','Aiming at the tree base produced a hit near the branches.','Mobile ray
    ↳ changed to Camera.CFrame.LookVector after a reported 1.35-degree safe-area mismatch.','BlasterController
    ↳ mobile aim ray.','Historical lower-trunk hit and client/server direction agreement reported. '),
412 ('H13','Mouse pitch/yaw and arm tracking regressed','User requested restoration after unrelated trigger/aim
    ↳ experiments.','Mouse-priority detection and shoulder/arm target tracking restored, including newer
    ↳ joints.','BlasterController input mode and arm pose.','Explicit AI-change-associated regression;
    ↳ historically restored. '),

```

```

413 ('H14','FIRE/AIM/JUMP controls overlapped','Shared chat and screenshots identified overlapping mobile
    ↳ actions.','Controls repositioned relative to jump with separate spacing.','BlasterController touch
    ↳ layout.','Historical Galaxy A06 check; regression coverage required after HUD changes.'),
414 ('H15','HUD existed without its reticle','Shared chat reports incomplete BlasterHUD initialization.','HUD
    ↳ integrity checks rebuild missing reticle children.','BlasterController_ensureHUD.','Reported fixed
    ↳ historically; test equip/unequip and device changes.'),
415 ('H16','Maintenance vent landed player partly in floor','User reported becoming embedded at basement
    ↳ destination.','Vent landing and both basement stair routes were corrected.','LabEquipment basement handler
    ↳ and generator geometry.','Historical vent landing test reported.'),
416 ('H17','Workers passed through geometry or players','Shared chat requested room-aware movement and
    ↳ avoidance.','Original worker motion gained room/obstacle checks and appearance updates.','LabEquipment
    ↳ botMotionChecks/walkBotSegment/moveBotTo.','Historical implementation; not proof of arbitrary carrier
    ↳ obstacle avoidance.'),
417 ('H18','Assigned carrier was not nearby or visible','Current-thread user could not find the mission NPC after
    ↳ role selection.','Per-player companion allocation and safe nearby placement introduced.','GWAPServer
    ↳ companion/startMission; GWAPNPC nearPlayer.','Historical approximately eight-stud placement test.'),
418 ('H19','Carrier clones repeated names/appearance and female silhouette was wrong','Current-thread screenshots
    ↳ show repeated Maria models and unsuitable hair.','Different non-seated templates, alternating gender
    ↳ preference, and hair adjustments.','GWAPServer makeBot; GWAPNPC appearance.','Historical appearance check;
    ↳ animation/identity remain template-dependent.'),
419 ('H20','Carriers queued together in the hallway','Current-thread screenshot and request rejected a hallway line
    ↳ of bots.','Unused carriers moved to a ServerStorage pool and allocated individually.','GWAPServer
    ↳ available/companions/pool.','Historical one visible, four pooled observation.'),
420 ('H21','Carrier glided unnaturally and too slowly','Current-thread user asked for natural movement matching
    ↳ player speed.','Distance-driven limb gait and smoothed owner WalkSpeed matching added.','GWAPNPC animate;
    ↳ GWAPServer go.','Historical speed-20 movement and approximately 24-degree leg swing measured.'),
421 ('H22','Weapon trembled while idle and walking','Repeated current-thread reports persisted after an earlier
    ↳ partial fix.','Single pre-camera rotation, no artificial Grip bob, and armed animation
    ↳ stabilization.','BlasterController pre-camera, render, and PreSimulation paths.','Historical zero Grip
    ↳ motion idle/walking; firing recoil retained.'),
422 ('H23','Wall explosion left fixtures floating or usable','Current-thread user requested disappearance of
    ↳ objects attached to the removed wall section.','Recent-blast wall removal detection and fixture quarantine
    ↳ introduced.','GWAPWallDamage.','Historical four fixture categories removed; outside health station
    ↳ preserved.'),
423 ('H24','Zero-length rendezvous path prevented progress','Development test encountered a start point already at
    ↳ its corridor entry.','Zero-distance prefix bypass and checked short direct path added.','GWAPServer go
    ↳ initial rendezvous.','Implementation-level defect found during testing; historical retest passed.'),
424 ]
425 RISKS=[
426 ('C01','Observed now: moved geometry, stale route coordinates','Station positions and roof share a translation
    ↳ while Approach attributes and routePoints constants retain the old origin.','Introduce a coherent local
    ↳ coordinate frame and approach attachments; validate every route in a copied place. No fix installed
    ↳ here.'),
427 ('C02','Observed now: incorrect world module bundled in installer','Root GWAPWorld.lua contains older
    ↳ mission-server code, and InstallGWAP.lua installs that incorrect content as the world module. The active
    ↳ module is different.','Do not run the captured installer unchanged. Replace its world-module input from the
    ↳ verified active source, reconcile generator versions, regenerate and compare installed hashes in a copied
    ↳ place.'),
428 ('R01','Research prompt validation is narrower than legacy validation','ResearchRepair checks life, distance,
    ↳ and Exploded but does not enforce Technician role or perform its own server ray visibility check.','Apply
    ↳ an explicit role/range/visibility/state policy and validate hold completion server-side where required.
    ↳ Test wrong role and an intervening wall.'),
429 ('R02','Main carrier route assumes clear surveyed geometry','The main kinematic route moves along points and
    ↳ stops near players; it does not sweep every segment for arbitrary new obstacles.','Add edge validation,
    ↳ blocked-route state, and bounded replanning. Preserve one movement authority.'),
430 ('R03','Solo percentage counts pooled models','Arming includes available carriers outside Workspace, while
    ↳ shooting excludes them.','Define an eligible active population; recompute selection on lifecycle changes
    ↳ and display both selected and active threat counts.'),
431 ('R04','Wall-fixture restoration is not defined','Removed fixtures remain in ServerStorage; repairing
    ↳ attributes does not rebuild walls or reparent fixtures.','Define persistent versus per-round destruction,
    ↳ then implement explicit restoration or deliberate permanent removal for the session.'),
432 ('R05','Multiplayer balance and shared objective semantics are unverified','Assignments are per player; no
    ↳ complete multi-client regression was recorded.','Test two roles and multiple carriers concurrently;
    ↳ commission a shared-convoy design separately if desired.'),
433 ('R06','Scale is configured, not performance-certified','MaxPlayers is 60 and the scene is large; no current
    ↳ load profile was captured.','Measure client/server frame time, memory, replication, and NPC workloads
    ↳ before making capacity claims.'),
434 ('R07','Lounge notes are displayed without explicit filtering','LoungeSystems onChat stores the raw /note
    ↳ message and refreshWall writes it to a shared display; no text-filtering call appears in that
    ↳ source.','Filter submitted text on the server using the appropriate Roblox text-filtering result for a
    ↳ shared display, handle filtering failure without displaying raw text, and add length/rate limits. Test with
    ↳ multiple viewers.'),
435 ('T01','Tool artifact: truncated source backup','A previous LabEquipment backup was cut near 100 KB and renamed
    ↳ .partial.lua.','Never install a partial capture. This edition verifies every reconstructed Studio source
    ↳ byte length.'),

```

```

436 ('T02','Test artifact: duplicate controller binding','A prior client test required/bound a controller again and
    ↳ produced duplicate render-step warnings.','Use existing bootstrap/controller instances for probes and fresh
    ↳ Play sessions; do not classify the test harness warning as a confirmed production defect.'),
437 ]
438
439 TESTS=[
440 ('P01','Prior GWAP turn','Role selection and nearby companion','Approximately eight studs from player; one
    ↳ visible carrier and four pooled.','Historical observation, not rerun for current geometry.'),
441 ('P02','Prior GWAP turn','Wait and Regroup','Wait movement zero; regroup returned within roughly eight
    ↳ studs.','No stress test of every blocked spawn candidate.'),
442 ('P03','Prior GWAP turn','Original rifle and equipment destruction','Lab Rifle destroyed a research machine;
    ↳ nearby explosion destroyed another; R repair restored it.','Does not certify wall-fixture restoration or
    ↳ every blast chain.'),
443 ('P04','Prior GWAP turn','Equivalent rerouting','Incubator -> same-floor Backup 1 -> other-floor Backup 2;
    ↳ assignment stayed RUNNING.','Earlier coordinate frame; current C01 requires rerun.'),
444 ('P05','Prior GWAP turn','Carrier gait and speed','WalkSpeed 20; roughly 28.7 studs of travel during sampled
    ↳ interval; leg swing about 24 degrees.','Kinematic gait, not Humanoid physics certification.'),
445 ('P06','Prior GWAP turn','Wall-fixture cleanup','Four fixture types within a temporary wall section removed;
    ↳ outside health station preserved.','Controlled isolated test, not whole-floor blast profiling.'),
446 ('P07','Prior GWAP turn','Small mobile layout','705x338 viewport; panel bottom y=179, joystick top
    ↳ y=187.','Eight-pixel gap in this configuration only.'),
447 ('P08','Prior GWAP turn','Idle weapon stabilization','90 frames with zero Grip motion and zero
    ↳ shoulder-animation translation.','One tested original rifle/rig configuration.'),
448 ('P09','Prior GWAP turn','Walking stabilization','Approximately 19 studs of walking with zero Grip
    ↳ motion.','Intentional recoil while firing was not removed.'),
449 ('P10','Prior GWAP turn','Interception score','Assigned carrier awarded 50 once; repeat damage did not add
    ↳ points.','Multi-client race conditions not exhaustively tested.'),
450 ('P11','Prior GWAP turn','Solo score loss','100 points became 99 on a successful NPC hit.','Low-score
    ↳ fractional policy also inspected in code; no long statistical run.'),
451 ('P12','Earlier GWAP turn','Rooftop mission completion','Seed source -> instrument -> roof completed and one
    ↳ plant appeared.','Test used helper following; friendly fire disqualified final escort reward, so clean full
    ↳ reward was not proven.'),
452 ('P13','Shared chat','Player shooting validation','Reported 10 damage, wall blocking, and rejection of distant
    ↳ forged camera origin.','Historical report; not a new penetration/security audit.'),
453 ('P14','Shared chat','ADS and device controls','Reported FOV 70->55 and sensitivity 1->0.62, restored on exit;
    ↳ phone/tablet controls checked.','Some later control changes superseded these exact UI versions.'),
454 ('P15','Shared chat','Basement and quiz/window interactions','Vent landing, pickup quiz, and breakable window
    ↳ passed reported checks.','No new Play session in this edition.'),
455 ('D01','This edition','Live source completeness','29 source containers captured; every UTF-8 byte length
    ↳ matched Studio inventory.','Static capture only; active count 20, archive 8, disabled generator 1.'),
456 ('D02','This edition','Scene/source audit','67 research station records; place version 70; C01 and C02
    ↳ identified.','No claim that current routing is playable.'),
457 ]
458
459 UPDATES=[
460 ('LabKinds','Equipment catalog','Add one Kind with inputs, outputs, wait duration, and display text. Keep
    ↳ simulation values explicit.','LabEquipment consumption and treatment paths; generator Kind
    ↳ attributes.','Unknown Kind, missing input, invalid output, full destination, and a complete valid cycle.'),
461 ('LabEquipment','Equipment engine','Extract inventory transactions before touching hazards or NPC motion.
    ↳ Preserve reservations and post-yield validation.','LabRoles assignments, LabQuizClient, tagged geometry,
    ↳ server hit bridge.','Concurrent pickup/drop, repair during failure tick, dead player, destroyed target,
    ↳ canceled quiz.'),
462 ('LabRoles','Roles and outfits','Keep exactly two roles unless commissioned otherwise; centralize
    ↳ role-to-outfit and permission mapping.','GWAPServer LabRole observer and original equipment role
    ↳ checks.','Both kiosks, respawn, role change during mission, and accessory/rig variations.'),
463 ('LabRound','Round lifecycle','Make reset ordering explicit with one round transition contract and subsystem
    ↳ callbacks.','Points, fractional debt, carrier termination, research repair, plot clearing.','Join during
    ↳ break, end during processing, disconnected winner, and reset after destruction.'),
464 ('LabTechnicians','Legacy worker service','Check which templates actually have Humanoids before extending its
    ↳ path loop. Avoid competing with LabEquipment worker motion.','LabWorkerBot tags and template
    ↳ structure.','No double movement, no idle thread leak, valid target assignment, and bounded retry.'),
465 ('LabElevators','Elevator compatibility script','Treat as dormant unless real tagged elevator geometry exists.
    ↳ Do not claim elevator transport from script presence alone.','Level metadata and any future lift
    ↳ models.','Door interlock, floor alignment, queueing, interrupted travel, and stair fallback.'),
466 ('LoungeSystems','Lounge server behavior','Keep lounge interactions isolated from laboratory scoring and
    ↳ navigation. Profile actual cost before deleting scenery.','LoungeClient and generated lounge
    ↳ objects.','Missing lounge, occupied seat, interaction range, and cleanup on departure.'),
467 ('LoungeClient','Lounge presentation','Add local effects through lifecycle-managed connections and restore
    ↳ prior camera/UI state on exit.','LoungeSystems and player character lifecycle.','Enter/exit, respawn
    ↳ inside, missing props, and device input.'),
468 ('LabQuizClient','Quiz UI','Use an accessible modal with touch and keyboard choices and a clear timeout.
    ↳ Display server challenge data only.','LabEquipment pendingQuizzes and MathQuizActive.','Wrong/late/replayed
    ↳ reply, cancel, death, target destruction, and overlap with GWAP HUD.'),
469 ('LabSidearmClient','Tool bootstrap','Keep one controller instance per equipped original Tool and one active
    ↳ camera owner.','BlasterController AutoBind/Enable/Disable.','Rapid equip, respawn, backpack replacement,
    ↳ duplicate script prevention.'),

```

```

470 ('Blaster','Shared casting','Preserve desktop screen conversion and mobile camera-forward behavior as separate
    ↳ tested paths.','Controller reticle and server manager raycasts.','Mouse edges, asymmetric safe areas, near
    ↳ wall, long range, and invalid vectors.'),
471 ('BlasterController','Input/camera/pose','Fix one measured fault at a time; retain cached base transforms and
    ↳ deterministic update ownership.','Shared casting, Tool grip metadata, HUD rectangles, rig
    ↳ joints.','Idle/walk/recoil, mouse pitch/yaw, mobile swipe, hybrid input, ADS restore, GUI click
    ↳ suppression.'),
472 ('ServerBlasterManager','Authoritative shooting','Add weapon configuration only through validated server-side
    ↳ attributes; keep sequence and rate limits.','Blaster configuration, GWAPHitBridge, legacy impact
    ↳ bridge.','Spoofted tool, out-of-order packet, impossible camera origin, wall obstruction, and correct target
    ↳ damage.'),
473 ('GWAPWorld','Research geometry','Use the captured active module, not the loose mislabeled file. Convert
    ↳ construction to a versioned local frame.','GWAPRouting, mission approaches, roof stair geometry.','Build
    ↳ twice without duplicates, translated/rotated building, all approach points, roof opening.'),
474 ('GWAPScenarios','Scenario data','Give each scenario a stable ID and protocol version; decouple plot selection
    ↳ from array position.','Server index validation, HUD job cards, station capabilities.','Missing capability,
    ↳ invalid range, new eleventh entry, and unchanged old scenario IDs.'),
475 ('GWAPNPC','Carrier appearance and gait','Separate immutable template pose from animation state; preserve
    ↳ same-inventory gun models.','GWAPServer template selection and movement ownership.','Boys/girls, held
    ↳ seedling, gun pose, idle reset, teleport reset, pooled reactivation.'),
476 ('GWAPRouting','Equivalent selection','Add reachability and operational-state adapters after fixing
    ↳ coordinates. Use stable IDs and deterministic tie breaking.','World station registry, destruction state,
    ↳ capacity.','Same floor preferred, other floor fallback, all unavailable, full plots, removed fixture.'),
477 ('GWAPServer','Mission orchestration','Extract mission state/reward transactions without changing existing role
    ↳ meanings. Introduce route revision cancellation consistently.','All GWAP modules, LabRoles, Points,
    ↳ LabRound.','Concurrent players, damage during processing, repeated replies, respawn, departure, and
    ↳ terminal reward idempotence.'),
478 ('GWAPWallDamage','Wall fixture lifecycle','Replace name-pattern coupling with explicit wall/fixture
    ↳ association when geometry is refactored. Define restoration.','Legacy gas detonation, research stations,
    ↳ prompts and tags.','Near/outside fixture, rotated wall, two blasts, round reset, old callback after
    ↳ quarantine.'),
479 ('LabHUD','Grouped mission display','Centralize notice priority and reserved screen regions. Preserve original
    ↳ backpack and combat controls.','GWAP attributes, original notices, quiz modal, joystick/jump
    ↳ bounds.','Small portrait/landscape, long text, two-digit stages, chat open, quiz open, and keyboard/gamepad
    ↳ focus.'),
480 ]
481
482
483 GENERATED REFERENCE: architecture-diagrams.tex
484 \chapter{Architecture and coordinate diagrams}
485 \section{Runtime responsibilities}
486 \begin{center}
487 \begin{tikzpicture}[node distance=8mm,box/.style={draw=teal,rounded corners,align=center,text
    ↳ width=45mm,minimum height=16mm,font=\small},arr/.style={-{Stealth}},thick,draw=teal}]
488 \node[box] (client) {Client input and display\BlasterController, LabHUD\LabQuizClient};
489 \node[box,right=12mm of client] (server) {Authoritative server\ServerBlasterManager\GWAPServer,
    ↳ LabEquipment};
490 \node[box,below=18mm of server] (state) {Server-owned state\assignments, samples\points, damage, rounds};
491 \node[box,below=18mm of client] (world) {World and navigation\GWAPWorld, GWAPRouting\stations, stairs,
    ↳ fixtures};
492 \draw[arr] (client.east) -- node[above,font=\scriptsize]{requests} (server.west);
493 \draw[arr] (server.south) -- (state.north);
494 \draw[arr] (world.east) -- node[above,font=\scriptsize]{geometry/state} (state.west);
495 \draw[arr] (state.south) -- ++(0,-8mm) -| node[pos=.7,below,font=\scriptsize]{replicated attributes and
    ↳ presentation} (client.west);
496 \end{tikzpicture}
497 \end{center}
498 Requests do not directly authorize rewards or damage. The server validates and commits gameplay state. The
    ↳ client displays that state and predicts visual aiming feedback. The world supplies geometry and station
    ↳ metadata; the current coordinate defect shows why those two must remain consistent.
499
500 \section{Observed origin mismatch: Incubator example}
501 \begin{center}
502 \begin{tikzpicture}[x=.08cm,y=.055cm,>=Stealth,font=\small]
503 \draw[->] (15,25) -- (108,25) node[right]{world X};
504 \draw[->] (15,25) -- (15,162) node[above]{world Z};
505 \fill[teal] (67,49) circle[radius=2pt] node[below left,align=right]{builder station\((67,49)\);
506 \fill[blue] (78,49) circle[radius=2pt] node[right,align=left]{stored approach\((78,49)\);
507 \fill[red!70!black] (41.857,140.286) circle[radius=2pt] node[left,align=right]{actual station\((41.86,140.29)\);
508 \draw[->,thick,red!70!black] (67,49) -- node[left,sloped,font=\scriptsize]{observed translation}
    ↳ (41.857,140.286);
509 \draw[dashed,blue] (78,49) -- (41.857,140.286);
510 \end{tikzpicture}
511 \end{center}

```

512 This plan view omits height for clarity. The station's actual Y is -21.75, while its builder Y is 1.5 and its
 ↳ stored approach Y is 3.25. Geometry translation does not automatically transform Vector3 attributes or
 ↳ literal coordinates in code. The dashed line illustrates the disagreement; it is not a proposed valid path.

513

514 \section{Proposed mission transition contract}

515 \begin{center}

516 \begin{tikzpicture}[node distance=8mm,box/.style={draw=teal,rounded corners,align=center,text
 ↳ width=31mm,minimum height=12mm,font=\small},arr/.style={-{Stealth},thick}]

517 \node[box] (choose) {Choose job\\paused};

518 \node[box,right=8mm of choose] (resolve) {Ready / resolve\\healthy target};

519 \node[box,right=8mm of resolve] (travel) {Travel and\\process};

520 \node[box,below=13mm of travel] (commit) {Commit checkpoint\\or final planting};

521 \node[box,below=13mm of resolve] (wait) {Wait / replan\\same stage};

522 \draw[arr] (choose) -- (resolve);

523 \draw[arr] (resolve) -- (travel);

524 \draw[arr] (travel) -- (commit);

525 \draw[arr] (travel.south west) -- node[above,sloped,font=\scriptsize]{destroyed / blocked} (wait.north east);

526 \draw[arr] (wait) -- (resolve);

527 \draw[arr] (commit.south) -- ++(0,-7mm) -| node[pos=.7,below,font=\scriptsize]{next stage} (resolve.west);

528 \end{tikzpicture}

529 \end{center}

530 The current code implements destruction rerouting and pause behavior. General blocked-edge replanning remains
 ↳ proposed. Every asynchronous operation should carry the assignment/route revision so a stale operation
 ↳ cannot commit after regroup, termination, or a new target selection.

531

532

533 GENERATED REFERENCE: bibliography.tex

534 \chapter{Bibliography and source access}

535 \section{Shared development chat}

536 Improve SeedLab interactions; browser read on 13 September 2026.\allowbreak{} Historical reports and
 ↳ requirement chronology.\allowbreak{}

537

538 \url{https://chatgpt.com/s/cx_6aa68aee04bc8191a98fa52b54b28109}

539

540 \section{Roblox client/\allowbreak{}server boundary}

541 Primary guidance for server validation of client-triggered actions.\allowbreak{} Consulted 13 September
 ↳ 2026.\allowbreak{}

542

543 \url{https://create.roblox.com/docs/scripting/security/client-server-boundary}

544

545 \section{Roblox Camera reference}

546 Primary API reference for screen/\allowbreak{}camera rays and coordinate conventions.\allowbreak{} Consulted 13
 ↳ September 2026.\allowbreak{}

547

548 \url{https://create.roblox.com/docs/reference/engine/classes/Camera}

549

550 \section{Roblox pathfinding}

551 Primary guidance for paths and navigation behavior.\allowbreak{} Consulted 13 September 2026.\allowbreak{}

552

553 \url{https://create.roblox.com/docs/characters/pathfinding}

554

555 \section{Roblox UI position and size}

556 Primary guidance for interface sizing and positioning.\allowbreak{} Consulted 13 September 2026.\allowbreak{}

557

558 \url{https://create.roblox.com/docs/ui/position-and-size}

559

560 \section{Roblox text filtering}

561 Primary guidance for filtering player-authored text displayed on shared signs.\allowbreak{} Relevant to open
 ↳ risk R07.\allowbreak{} Consulted 13 September 2026.\allowbreak{}

562

563 \url{https://create.roblox.com/docs/ui/text-filtering}

564

565 \section{Local primary evidence}

566 evidence/\allowbreak{}live-source-capture.\allowbreak{}json contains the exact source capture;
 ↳ source-manifest.\allowbreak{}json identifies each listing; live-world-inventory.\allowbreak{}json records
 ↳ scene measurements; export-manifest.\allowbreak{}json records file hashes; export-5.\allowbreak{}rbxl
 ↳ preserves the supplied latest place.\allowbreak{} template-extracted.\allowbreak{}txt and
 ↳ template-links.\allowbreak{}json preserve the template structure and attribution links.\allowbreak{}
 ↳ shared-chat-reading-notes.\allowbreak{}md records the browser reading scope.\allowbreak{}
 ↳ GWAP_\allowbreak{}REVISION_\allowbreak{}NOTES.\allowbreak{}md preserves prior reported test
 ↳ results.\allowbreak{} The attached GWAP ideas are copied when available.\allowbreak{}

567

568

569

570 GENERATED REFERENCE: defects.tex

571 \chapter{Defect register, change accounting, and lessons}

572 \section{How many bugs were generated?}

573 The evidence supports a bounded register of 24 historical defect families (H01-H24), two newly observed
→ source/\allowbreak{}scene discrepancies (C01-C02), seven baseline audit risks (R01-R07), with final status
→ in the Revision 4 chapter, and two tool/\allowbreak{}test artifacts (T01-T02).\allowbreak{} These
→ categories are not added together as a lifetime bug total.\allowbreak{} A family groups recurring reports
→ of the same symptom; one family may have had several causes or fixes.\allowbreak{} No complete issue
→ tracker, commit history, or causal record establishes the total number of bugs introduced by an AI
→ model.\allowbreak{}

574

575 At least two episodes have explicit evidence of an AI change or implementation error: H13 is the
→ owner-requested rollback/\allowbreak{}restoration after an aiming change, and H24 is the zero-length
→ route-prefix defect encountered during implementation testing.\allowbreak{} Other families are associated
→ with the evolving AI-built game, but the record does not isolate whether each was newly introduced,
→ pre-existing, or caused by an intervening model/\allowbreak{}asset change.\allowbreak{} Reporting all 24 as
→ definitely generated by one model would be unsupported.\allowbreak{} The two current discrepancies are
→ observed now; their author and exact introduction time are unknown.\allowbreak{}

576

577 Five major requirement reversals are recorded separately: large monitor panels were later removed; quizzes were
→ added, removed, and then reintroduced in simpler form; the extra GWAP arsenal was added and then rejected;
→ a broad 60-percent failure policy was replaced by a small workload; and static/\allowbreak{}identical NPC
→ presentations were replaced by differentiated moving workers.\allowbreak{} New research-equipment
→ destructibility and equivalent-station rerouting are feature extensions as well as corrections to the
→ desired experience.\allowbreak{} They do not each count automatically as a new software bug.\allowbreak{}

578

579 \section{Historical defect families}

580 \subsection{H01: Role kiosk inaccessible or inert}

581 Evidence: Earlier shared chat: user could not select a role or activate the kiosk.\allowbreak{}

582

583 Repair history: Interaction layout and trigger behavior were repeatedly revised; temporary touch selection was
→ later removed.\allowbreak{}

584

585 Code and regression target: LabRoles and kiosk geometry; verify E/\allowbreak{}click selection and no
→ accidental role change.\allowbreak{}

586

587 Status: Reported fixed historically; not replayed in this edition.\allowbreak{}

588

589 \subsection{H02: Role outfit missing or insufficiently distinct}

590 Evidence: Earlier shared chat repeatedly requested visible clothing after role selection.\allowbreak{}

591

592 Repair history: Role appearance application and respawn behavior were revised.\allowbreak{}

593

594 Code and regression target: LabRoles; test both roles before and after respawn.\allowbreak{}

595

596 Status: Reported fixed historically; appearance is built, full rig coverage unverified.\allowbreak{}

597

598 \subsection{H03: Equipment E/\allowbreak{}F actions appeared inert}

599 Evidence: Earlier user reports said nothing happened near equipment or when pressing controls.\allowbreak{}

600

601 Repair history: Physical prompts/\allowbreak{}buttons and runtime/\allowbreak{}fallback installation were
→ revised.\allowbreak{}

602

603 Code and regression target: LabEquipment bind/\allowbreak{}makeControls; distinguish old F behavior from
→ current E/\allowbreak{}R policy.\allowbreak{}

604

605 Status: Reported fixed historically; syntax checks alone were insufficient evidence.\allowbreak{}

606

607 \subsection{H04: Plant could be taken but not dropped}

608 Evidence: Earlier user explicitly reported pickup without a usable drop interaction.\allowbreak{}

609

610 Repair history: Working-cabinet drop logic, free-slot guidance, and human override of reservations were
→ improved.\allowbreak{}

611

612 Code and regression target: LabEquipment takePlantFromChamber/\allowbreak{}storePlantInChamber.\allowbreak{}

613

614 Status: Reported fixed historically; retest full/\allowbreak{}wrong/\allowbreak{}removed
→ destinations.\allowbreak{}

615

616 \subsection{H05: Equipment or fixtures intersected walls, doors, or windows}

617 Evidence: Multiple earlier screenshots and requests identified misplaced cabinets, health kits, weapons, and
→ pipes.\allowbreak{}

618

619 Repair history: Placement clearance and wall alignment were revised; pipes moved upward.\allowbreak{}

620

621 Code and regression target: Generator placement and saved geometry.\allowbreak{}

622

623 Status: Historical clearance measurements exist; current translated geometry needs a fresh spatial
↪ audit.\allowbreak{}

624

625 \subsection{H06: Equipment could be repaired through a wall}

626 Evidence: Earlier user explicitly identified repair across a wall.\allowbreak{}

627

628 Repair history: Closer interaction range and server line-of-sight validation were added to legacy
↪ interactions.\allowbreak{}

629

630 Code and regression target: LabEquipment canInteract.\allowbreak{}

631

632 Status: Built in legacy path; separate research prompts have risk R01.\allowbreak{}

633

634 \subsection{H07: Repair route crossed walls or ignored stairs}

635 Evidence: Earlier user reported a straight red path through geometry.\allowbreak{}

636

637 Repair history: Pathfinding and explicit stair-aware routing replaced a straight fallback.\allowbreak{}

638

639 Code and regression target: LabEquipment createRepairPath; later GWAP routePoints is a separate
↪ implementation.\allowbreak{}

640

641 Status: Historically reported fixed; C01 invalidates assumptions for moved geometry.\allowbreak{}

642

643 \subsection{H08: Overlapping information obscured play}

644 Evidence: Repeated shared-chat and current-thread screenshots show large or duplicate panels.\allowbreak{}

645

646 Repair history: Distant labels, duplicate HUDs, and notification feeds were reduced; latest HUD groups
↪ mission/\allowbreak{ }status.\allowbreak{ }

647

648 Code and regression target: LabHUD, LabEquipment display policy, BlasterController, LabQuizClient.\allowbreak{}

649

650 Status: One small-phone layout measured; all modals/\allowbreak{ }devices not exhaustive.\allowbreak{ }

651

652 \subsection{H09: Uneven lighting and black floor bands}

653 Evidence: Earlier handoff requested lighting repair.\allowbreak{ }

654

655 Repair history: The handoff records the defect but does not independently prove the exact repair.\allowbreak{ }

656

657 Code and regression target: Generator Lighting and room light placement.\allowbreak{ }

658

659 Status: Historical report; current visual closure not established by this audit.\allowbreak{ }

660

661 \subsection{H10: Touch firing failed or used unsuitable input}

662 Evidence: Shared chat requested cross-platform shooting because mobile controls failed.\allowbreak{ }

663

664 Repair history: Separated controller, casting module, and server manager; touch camera and action controls
↪ introduced.\allowbreak{ }

665

666 Code and regression target: BlasterController, Blaster, ServerBlasterManager.\allowbreak{ }

667

668 Status: Historical device checks reported; preserve input separation.\allowbreak{ }

669

670 \subsection{H11: Reticle and muzzle impact disagreed}

671 Evidence: Shared chat reported firing toward the avatar or not correctly hitting a wall.\allowbreak{ }

672

673 Repair history: Camera target and muzzle obstruction handling were revised.\allowbreak{ }

674

675 Code and regression target: Blaster ray helpers and controller reticle.\allowbreak{ }

676

677 Status: Historical desktop/\allowbreak{ }mobile impact checks reported.\allowbreak{ }

678

679 \subsection{H12: Android vertical aim offset}

680 Evidence: Aiming at the tree base produced a hit near the branches.\allowbreak{ }

681

682 Repair history: Mobile ray changed to Camera.\allowbreak{ }CFrame.\allowbreak{ }LookVector after a reported
↪ 1.\allowbreak{ }35-degree safe-area mismatch.\allowbreak{ }

683

684 Code and regression target: BlasterController mobile aim ray.\allowbreak{ }

685

686 Status: Historical lower-trunk hit and client/\allowbreak{ }server direction agreement reported.\allowbreak{ }

687

688 \subsection{H13: Mouse pitch/\allowbreak{ }yaw and arm tracking regressed}

689 Evidence: User requested restoration after unrelated trigger/\allowbreak{ }aim experiments.\allowbreak{ }

690

691 Repair history: Mouse-priority detection and shoulder/\allowbreak{ }arm target tracking restored, including
↪ newer joints.\allowbreak{ }

692
693 Code and regression target: BlasterController input mode and arm pose.\allowbreak{}\n694
695 Status: Explicit AI-change-associated regression; historically restored.\allowbreak{}\n696
697 \subsection{H14: FIRE/\allowbreak{}AIM/\allowbreak{}JUMP controls overlapped}\n698 Evidence: Shared chat and screenshots identified overlapping mobile actions.\allowbreak{}\n699
700 Repair history: Controls repositioned relative to jump with separate spacing.\allowbreak{}\n701
702 Code and regression target: BlasterController touch layout.\allowbreak{}\n703
704 Status: Historical Galaxy A06 check; regression coverage required after HUD changes.\allowbreak{}\n705
706 \subsection{H15: HUD existed without its reticle}\n707 Evidence: Shared chat reports incomplete BlasterHUD initialization.\allowbreak{}\n708
709 Repair history: HUD integrity checks rebuild missing reticle children.\allowbreak{}\n710
711 Code and regression target: BlasterController _\allowbreak{}ensureHUD.\allowbreak{}\n712
713 Status: Reported fixed historically; test equip/\allowbreak{}unequip and device changes.\allowbreak{}\n714
715 \subsection{H16: Maintenance vent landed player partly in floor}\n716 Evidence: User reported becoming embedded at basement destination.\allowbreak{}\n717
718 Repair history: Vent landing and both basement stair routes were corrected.\allowbreak{}\n719
720 Code and regression target: LabEquipment basement handler and generator geometry.\allowbreak{}\n721
722 Status: Historical vent landing test reported.\allowbreak{}\n723
724 \subsection{H17: Workers passed through geometry or players}\n725 Evidence: Shared chat requested room-aware movement and avoidance.\allowbreak{}\n726
727 Repair history: Original worker motion gained room/\allowbreak{}obstacle checks and appearance
↪ updates.\allowbreak{}\n728
729 Code and regression target: LabEquipment
↪ botMotionChecks/\allowbreak{}walkBotSegment/\allowbreak{}moveBotTo.\allowbreak{}\n730
731 Status: Historical implementation; not proof of arbitrary carrier obstacle avoidance.\allowbreak{}\n732
733 \subsection{H18: Assigned carrier was not nearby or visible}\n734 Evidence: Current-thread user could not find the mission NPC after role selection.\allowbreak{}\n735
736 Repair history: Per-player companion allocation and safe nearby placement introduced.\allowbreak{}\n737
738 Code and regression target: GWAPServer companion/\allowbreak{}startMission; GWAPNPC nearPlayer.\allowbreak{}\n739
740 Status: Historical approximately eight-stud placement test.\allowbreak{}\n741
742 \subsection{H19: Carrier clones repeated names/\allowbreak{}appearance and female silhouette was wrong}\n743 Evidence: Current-thread screenshots show repeated Maria models and unsuitable hair.\allowbreak{}\n744
745 Repair history: Different non-seated templates, alternating gender preference, and hair
↪ adjustments.\allowbreak{}\n746
747 Code and regression target: GWAPServer makeBot; GWAPNPC appearance.\allowbreak{}\n748
749 Status: Historical appearance check; animation/\allowbreak{}identity remain template-dependent.\allowbreak{}\n750
751 \subsection{H20: Carriers queued together in the hallway}\n752 Evidence: Current-thread screenshot and request rejected a hallway line of bots.\allowbreak{}\n753
754 Repair history: Unused carriers moved to a ServerStorage pool and allocated individually.\allowbreak{}\n755
756 Code and regression target: GWAPServer available/\allowbreak{}companions/\allowbreak{}pool.\allowbreak{}\n757
758 Status: Historical one visible, four pooled observation.\allowbreak{}\n759
760 \subsection{H21: Carrier glided unnaturally and too slowly}\n761 Evidence: Current-thread user asked for natural movement matching player speed.\allowbreak{}\n762
763 Repair history: Distance-driven limb gait and smoothed owner WalkSpeed matching added.\allowbreak{}\n764

```

765 Code and regression target: GWAPNPC animate; GWAPServer go.\allowbreak{}
766
767 Status: Historical speed-20 movement and approximately 24-degree leg swing measured.\allowbreak{}
768
769 \subsection{H22: Weapon trembled while idle and walking}
770 Evidence: Repeated current-thread reports persisted after an earlier partial fix.\allowbreak{}
771
772 Repair history: Single pre-camera rotation, no artificial Grip bob, and armed animation
    ↳ stabilization.\allowbreak{}
773
774 Code and regression target: BlasterController pre-camera, render, and PreSimulation paths.\allowbreak{}
775
776 Status: Historical zero Grip motion idle/\allowbreak{}walking; firing recoil retained.\allowbreak{}
777
778 \subsection{H23: Wall explosion left fixtures floating or usable}
779 Evidence: Current-thread user requested disappearance of objects attached to the removed wall
    ↳ section.\allowbreak{}
780
781 Repair history: Recent-blast wall removal detection and fixture quarantine introduced.\allowbreak{}
782
783 Code and regression target: GWAPWallDamage.\allowbreak{}
784
785 Status: Historical four fixture categories removed; outside health station preserved.\allowbreak{}
786
787 \subsection{H24: Zero-length rendezvous path prevented progress}
788 Evidence: Development test encountered a start point already at its corridor entry.\allowbreak{}
789
790 Repair history: Zero-distance prefix bypass and checked short direct path added.\allowbreak{}
791
792 Code and regression target: GWAPServer go initial rendezvous.\allowbreak{}
793
794 Status: Implementation-level defect found during testing; historical retest passed.\allowbreak{}
795
796 \section{Baseline discrepancies, risks, and tool artifacts}
797 These entries preserve the initial audit.\allowbreak{} Revision 4 repairs C01/\allowbreak{}C02, adds
    ↳ role/\allowbreak{}LOS validation for R01, collision stopping for R02, correct eligibility for R03, and
    ↳ filtering for R07.\allowbreak{} R04 restoration policy, R05 shared-mission semantics, R06 load
    ↳ certification and the limits stated in the Revision 4 chapter remain open.\allowbreak{}
798
799 \subsection{C01: Observed now: moved geometry, stale route coordinates}
800 Evidence: Station positions and roof share a translation while Approach attributes and routePoints constants
    ↳ retain the old origin.\allowbreak{}
801
802 Next action: Introduce a coherent local coordinate frame and approach attachments; validate every route in a
    ↳ copied place.\allowbreak{} No fix installed here.\allowbreak{}
803
804 \subsection{C02: Observed now: incorrect world module bundled in installer}
805 Evidence: Root GWAPWorld.\allowbreak{}lua contains older mission-server code, and InstallGWAP.\allowbreak{}lua
    ↳ installs that incorrect content as the world module.\allowbreak{} The active module is
    ↳ different.\allowbreak{}
806
807 Next action: Do not run the captured installer unchanged.\allowbreak{} Replace its world-module input from the
    ↳ verified active source, reconcile generator versions, regenerate and compare installed hashes in a copied
    ↳ place.\allowbreak{}
808
809 \subsection{R01: Research prompt validation is narrower than legacy validation}
810 Evidence: ResearchRepair checks life, distance, and Exploded but does not enforce Technician role or perform
    ↳ its own server ray visibility check.\allowbreak{}
811
812 Next action: Apply an explicit role/\allowbreak{}range/\allowbreak{}visibility/\allowbreak{}state policy and
    ↳ validate hold completion server-side where required.\allowbreak{} Test wrong role and an intervening
    ↳ wall.\allowbreak{}
813
814 \subsection{R02: Main carrier route assumes clear surveyed geometry}
815 Evidence: The main kinematic route moves along points and stops near players; it does not sweep every segment
    ↳ for arbitrary new obstacles.\allowbreak{}
816
817 Next action: Add edge validation, blocked-route state, and bounded replanning.\allowbreak{} Preserve one
    ↳ movement authority.\allowbreak{}
818
819 \subsection{R03: Solo percentage counts pooled models}
820 Evidence: Arming includes available carriers outside Workspace, while shooting excludes them.\allowbreak{}
821
822 Next action: Define an eligible active population; recompute selection on lifecycle changes and display both
    ↳ selected and active threat counts.\allowbreak{}
823

```

```
824 \subsection{R04: Wall-fixture restoration is not defined}
825
826 ]=====]
827 return {revision=4,part=1,reference="Server-only written reference; never required by gameplay"}
```

Chapter 71

DOC02 - GameDocumentationAnnex2

ServerStorage.GameDocumentationAnnex2

Authority group: documentation. 107451 UTF-8 bytes; 2159 source lines. SHA-256 is in the provenance chapter and machine-readable manifest.

```
1  --[=====  
2  Reference part 2 of 2; continue in sibling documentation modules.  
3  Evidence: Removed fixtures remain in ServerStorage; repairing attributes does not rebuild walls or reparent  
4  ↪ fixtures.\allowbreak{  
5  Next action: Define persistent versus per-round destruction, then implement explicit restoration or deliberate  
6  ↪ permanent removal for the session.\allowbreak{  
7  \subsection{R05: Multiplayer balance and shared objective semantics are unverified}  
8  Evidence: Assignments are per player; no complete multi-client regression was recorded.\allowbreak{  
9  
10 Next action: Test two roles and multiple carriers concurrently; commission a shared-convoy design separately if  
11 ↪ desired.\allowbreak{  
12 \subsection{R06: Scale is configured, not performance-certified}  
13 Evidence: MaxPlayers is 60 and the scene is large; no current load profile was captured.\allowbreak{  
14  
15 Next action: Measure client/\allowbreak{server frame time, memory, replication, and NPC workloads before  
16 ↪ making capacity claims.\allowbreak{  
17 \subsection{R07: Lounge notes are displayed without explicit filtering}  
18 Evidence: LoungeSystems onChat stores the raw /\allowbreak{note message and refreshWall writes it to a shared  
19 ↪ display; no text-filtering call appears in that source.\allowbreak{  
20  
21 Next action: Filter submitted text on the server using the appropriate Roblox text-filtering result for a  
22 ↪ shared display, handle filtering failure without displaying raw text, and add length/\allowbreak{rate  
23 ↪ limits.\allowbreak{ Test with multiple viewers.\allowbreak{  
24  
25 \subsection{T01: Tool artifact: truncated source backup}  
26 Evidence: A previous LabEquipment backup was cut near 100 KB and renamed  
27 ↪ .\allowbreak{partial.\allowbreak{lua.\allowbreak{  
28  
29 Next action: Never install a partial capture.\allowbreak{ This edition verifies every reconstructed Studio  
30 ↪ source byte length.\allowbreak{  
31  
32 \subsection{T02: Test artifact: duplicate controller binding}  
33 Evidence: A prior client test required/\allowbreak{bound a controller again and produced duplicate render-step  
34 ↪ warnings.\allowbreak{  
35  
36 Next action: Use existing bootstrap/\allowbreak{controller instances for probes and fresh Play sessions; do  
37 ↪ not classify the test harness warning as a confirmed production defect.\allowbreak{  
38  
39 \section{Rules that prevent repetition}
```

```
33 Freeze an accepted input\allowbreak{}camera baseline before unrelated features.\allowbreak{} Measure a symptom
   ↳ before rewriting its subsystem.\allowbreak{} Use live source and hashes to resolve conflicts.\allowbreak{}
   ↳ Keep one owner for camera, root, arm, and gun transforms.\allowbreak{} Revalidate after every yielding
   ↳ interaction.\allowbreak{} Treat geometry and navigation metadata as one versioned unit.\allowbreak{} Check
   ↳ actual screen rectangles and server state, not only appearance.\allowbreak{} Separate a passing syntax
   ↳ check from a passing player workflow.\allowbreak{} Never report a tool artifact or an old test as proof of
   ↳ current gameplay correctness.\allowbreak{}
34
35
36
37 GENERATED REFERENCE: future-updates.tex
38 \chapter{Subsystem-by-subsystem future update playbook}
39 \section{LabKinds: Equipment catalog}
40 Authoritative source: S05 /\allowbreak{} ReplicatedStorage.\allowbreak{}LabKinds
41
42 See complete listing on page \pageref{src:S05}.
43
44 \subsection{Proposed implementation}
45 Add one Kind with inputs, outputs, wait duration, and display text.\allowbreak{} Keep simulation values
   ↳ explicit.\allowbreak{}
46
47 \subsection{Dependencies to preserve}
48 LabEquipment consumption and treatment paths; generator Kind attributes.\allowbreak{}
49
50 \subsection{Acceptance cases}
51 Unknown Kind, missing input, invalid output, full destination, and a complete valid cycle.\allowbreak{}
52
53 \section{LabEquipment: Equipment engine}
54 Authoritative source: S11 /\allowbreak{} ServerScriptService.\allowbreak{}LabEquipment
55
56 See complete listing on page \pageref{src:S11}.
57
58 \subsection{Proposed implementation}
59 Extract inventory transactions before touching hazards or NPC motion.\allowbreak{} Preserve reservations and
   ↳ post-yield validation.\allowbreak{}
60
61 \subsection{Dependencies to preserve}
62 LabRoles assignments, LabQuizClient, tagged geometry, server hit bridge.\allowbreak{}
63
64 \subsection{Acceptance cases}
65 Concurrent pickup/\allowbreak{}drop, repair during failure tick, dead player, destroyed target, canceled
   ↳ quiz.\allowbreak{}
66
67 \section{LabRoles: Roles and outfits}
68 Authoritative source: S12 /\allowbreak{} ServerScriptService.\allowbreak{}LabRoles
69
70 See complete listing on page \pageref{src:S12}.
71
72 \subsection{Proposed implementation}
73 Keep exactly two roles unless commissioned otherwise; centralize role-to-outfit and permission
   ↳ mapping.\allowbreak{}
74
75 \subsection{Dependencies to preserve}
76 GWAPServer LabRole observer and original equipment role checks.\allowbreak{}
77
78 \subsection{Acceptance cases}
79 Both kiosks, respawn, role change during mission, and accessory/\allowbreak{}rig variations.\allowbreak{}
80
81 \section{LabRound: Round lifecycle}
82 Authoritative source: S13 /\allowbreak{} ServerScriptService.\allowbreak{}LabRound
83
84 See complete listing on page \pageref{src:S13}.
85
86 \subsection{Proposed implementation}
87 Make reset ordering explicit with one round transition contract and subsystem callbacks.\allowbreak{}
88
89 \subsection{Dependencies to preserve}
90 Points, fractional debt, carrier termination, research repair, plot clearing.\allowbreak{}
91
92 \subsection{Acceptance cases}
93 Join during break, end during processing, disconnected winner, and reset after destruction.\allowbreak{}
94
95 \section{LabTechnicians: Legacy worker service}
96 Authoritative source: S14 /\allowbreak{} ServerScriptService.\allowbreak{}LabTechnicians
97
98 See complete listing on page \pageref{src:S14}.
```

```
99
100 \subsection{Proposed implementation}
101 Check which templates actually have Humanoids before extending its path loop.\allowbreak{} Avoid competing with
↪ LabEquipment worker motion.\allowbreak{}
102
103 \subsection{Dependencies to preserve}
104 LabWorkerBot tags and template structure.\allowbreak{}
105
106 \subsection{Acceptance cases}
107 No double movement, no idle thread leak, valid target assignment, and bounded retry.\allowbreak{}
108
109 \section{LabElevators: Elevator compatibility script}
110 Authoritative source: S10 /\allowbreak{} ServerScriptService.\allowbreak{}LabElevators
111
112 See complete listing on page \pageref{src:S10}.
113
114 \subsection{Proposed implementation}
115 Treat as dormant unless real tagged elevator geometry exists.\allowbreak{} Do not claim elevator transport from
↪ script presence alone.\allowbreak{}
116
117 \subsection{Dependencies to preserve}
118 Level metadata and any future lift models.\allowbreak{}
119
120 \subsection{Acceptance cases}
121 Door interlock, floor alignment, queueing, interrupted travel, and stair fallback.\allowbreak{}
122
123 \section{LoungeSystems: Lounge server behavior}
124 Authoritative source: S15 /\allowbreak{} ServerScriptService.\allowbreak{}LoungeSystems
125
126 See complete listing on page \pageref{src:S15}.
127
128 \subsection{Proposed implementation}
129 Keep lounge interactions isolated from laboratory scoring and navigation.\allowbreak{} Profile actual cost
↪ before deleting scenery.\allowbreak{}
130
131 \subsection{Dependencies to preserve}
132 LoungeClient and generated lounge objects.\allowbreak{}
133
134 \subsection{Acceptance cases}
135 Missing lounge, occupied seat, interaction range, and cleanup on departure.\allowbreak{}
136
137 \section{LoungeClient: Lounge presentation}
138 Authoritative source: S28 /\allowbreak{}
↪ StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LoungeClient
139
140 See complete listing on page \pageref{src:S28}.
141
142 \subsection{Proposed implementation}
143 Add local effects through lifecycle-managed connections and restore prior camera/\allowbreak{}UI state on
↪ exit.\allowbreak{}
144
145 \subsection{Dependencies to preserve}
146 LoungeSystems and player character lifecycle.\allowbreak{}
147
148 \subsection{Acceptance cases}
149 Enter/\allowbreak{}exit, respawn inside, missing props, and device input.\allowbreak{}
150
151 \section{LabQuizClient: Quiz UI}
152 Authoritative source: S26 /\allowbreak{}
↪ StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabQuizClient
153
154 See complete listing on page \pageref{src:S26}.
155
156 \subsection{Proposed implementation}
157 Use an accessible modal with touch and keyboard choices and a clear timeout.\allowbreak{} Display server
↪ challenge data only.\allowbreak{}
158
159 \subsection{Dependencies to preserve}
160 LabEquipment pendingQuizzes and MathQuizActive.\allowbreak{}
161
162 \subsection{Acceptance cases}
163 Wrong/\allowbreak{}late/\allowbreak{}replayed reply, cancel, death, target destruction, and overlap with GWAP
↪ HUD.\allowbreak{}
164
165 \section{LabSidearmClient: Tool bootstrap}
```

```
166 Authoritative source: S27 /\allowbreak{}
    ↳ StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabSidearmClient
167
168 See complete listing on page \pageref{src:S27}.
169
170 \subsection{Proposed implementation}
171 Keep one controller instance per equipped original Tool and one active camera owner.\allowbreak{}
172
173 \subsection{Dependencies to preserve}
174 BlasterController AutoBind/\allowbreak{}Enable/\allowbreak{}Disable.\allowbreak{}
175
176 \subsection{Acceptance cases}
177 Rapid equip, respawn, backpack replacement, duplicate script prevention.\allowbreak{}
178
179 \section{Blaster: Shared casting}
180 Authoritative source: S06 /\allowbreak{} ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}Blaster
181
182 See complete listing on page \pageref{src:S06}.
183
184 \subsection{Proposed implementation}
185 Preserve desktop screen conversion and mobile camera-forward behavior as separate tested paths.\allowbreak{}
186
187 \subsection{Dependencies to preserve}
188 Controller reticle and server manager raycasts.\allowbreak{}
189
190 \subsection{Acceptance cases}
191 Mouse edges, asymmetric safe areas, near wall, long range, and invalid vectors.\allowbreak{}
192
193 \section{BlasterController: Input/\allowbreak{}camera/\allowbreak{}pose}
194 Authoritative source: S07 /\allowbreak{} ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}BlasterController
195
196 See complete listing on page \pageref{src:S07}.
197
198 \subsection{Proposed implementation}
199 Fix one measured fault at a time; retain cached base transforms and deterministic update
    ↳ ownership.\allowbreak{}
200
201 \subsection{Dependencies to preserve}
202 Shared casting, Tool grip metadata, HUD rectangles, rig joints.\allowbreak{}
203
204 \subsection{Acceptance cases}
205 Idle/\allowbreak{}walk/\allowbreak{}recoil, mouse pitch/\allowbreak{}yaw, mobile swipe, hybrid input, ADS
    ↳ restore, GUI click suppression.\allowbreak{}
206
207 \section{ServerBlasterManager: Authoritative shooting}
208 Authoritative source: S16 /\allowbreak{} ServerScriptService.\allowbreak{}ServerBlasterManager
209
210 See complete listing on page \pageref{src:S16}.
211
212 \subsection{Proposed implementation}
213 Add weapon configuration only through validated server-side attributes; keep sequence and rate
    ↳ limits.\allowbreak{}
214
215 \subsection{Dependencies to preserve}
216 Blaster configuration, GWAPHitBridge, legacy impact bridge.\allowbreak{}
217
218 \subsection{Acceptance cases}
219 Spoofed tool, out-of-order packet, impossible camera origin, wall obstruction, and correct target
    ↳ damage.\allowbreak{}
220
221 \section{GWAPWorld: Research geometry}
222 Authoritative source: S04 /\allowbreak{} ReplicatedStorage.\allowbreak{}GWAPWorld
223
224 See complete listing on page \pageref{src:S04}.
225
226 \subsection{Proposed implementation}
227 Use the captured active module, not the loose mislabeled file.\allowbreak{} Convert construction to a versioned
    ↳ local frame.\allowbreak{}
228
229 \subsection{Dependencies to preserve}
230 GWAPRouting, mission approaches, roof stair geometry.\allowbreak{}
231
232 \subsection{Acceptance cases}
233 Build twice without duplicates, translated/\allowbreak{}rotated building, all approach points, roof
    ↳ opening.\allowbreak{}
234
```

```

235 \section{GWAPScenarios: Scenario data}
236 Authoritative source: S03 /\allowbreak{} ReplicatedStorage.\allowbreak{}GWAPScenarios
237
238 See complete listing on page \pageref{src:S03}.
239
240 \subsection{Proposed implementation}
241 Give each scenario a stable ID and protocol version; decouple plot selection from array position.\allowbreak{}
242
243 \subsection{Dependencies to preserve}
244 Server index validation, HUD job cards, station capabilities.\allowbreak{}
245
246 \subsection{Acceptance cases}
247 Missing capability, invalid range, new eleventh entry, and unchanged old scenario IDs.\allowbreak{}
248
249 \section{GWAPNPC: Carrier appearance and gait}
250 Authoritative source: S01 /\allowbreak{} ReplicatedStorage.\allowbreak{}GWAPNPC
251
252 See complete listing on page \pageref{src:S01}.
253
254 \subsection{Proposed implementation}
255 Separate immutable template pose from animation state; preserve same-inventory gun models.\allowbreak{}
256
257 \subsection{Dependencies to preserve}
258 GWAPServer template selection and movement ownership.\allowbreak{}
259
260 \subsection{Acceptance cases}
261 Boys/\allowbreak{}girls, held seedling, gun pose, idle reset, teleport reset, pooled reactivation.\allowbreak{}
262
263 \section{GWAPRouting: Equivalent selection}
264 Authoritative source: S02 /\allowbreak{} ReplicatedStorage.\allowbreak{}GWAPRouting
265
266 See complete listing on page \pageref{src:S02}.
267
268 \subsection{Proposed implementation}
269 Add reachability and operational-state adapters after fixing coordinates.\allowbreak{} Use stable IDs and
    ↳ deterministic tie breaking.\allowbreak{}
270
271 \subsection{Dependencies to preserve}
272 World station registry, destruction state, capacity.\allowbreak{}
273
274 \subsection{Acceptance cases}
275 Same floor preferred, other floor fallback, all unavailable, full plots, removed fixture.\allowbreak{}
276
277 \section{GWAPServer: Mission orchestration}
278 Authoritative source: S08 /\allowbreak{} ServerScriptService.\allowbreak{}GWAPServer
279
280 See complete listing on page \pageref{src:S08}.
281
282 \subsection{Proposed implementation}
283 Extract mission state/\allowbreak{}reward transactions without changing existing role meanings.\allowbreak{}
    ↳ Introduce route revision cancellation consistently.\allowbreak{}
284
285 \subsection{Dependencies to preserve}
286 All GWAP modules, LabRoles, Points, LabRound.\allowbreak{}
287
288 \subsection{Acceptance cases}
289 Concurrent players, damage during processing, repeated replies, respawn, departure, and terminal reward
    ↳ idempotence.\allowbreak{}
290
291 \section{GWAPWallDamage: Wall fixture lifecycle}
292 Authoritative source: S09 /\allowbreak{} ServerScriptService.\allowbreak{}GWAPWallDamage
293
294 See complete listing on page \pageref{src:S09}.
295
296 \subsection{Proposed implementation}
297 Replace name-pattern coupling with explicit wall/\allowbreak{}fixture association when geometry is
    ↳ refactored.\allowbreak{} Define restoration.\allowbreak{}
298
299 \subsection{Dependencies to preserve}
300 Legacy gas detonation, research stations, prompts and tags.\allowbreak{}
301
302 \subsection{Acceptance cases}
303 Near/\allowbreak{}outside fixture, rotated wall, two blasts, round reset, old callback after
    ↳ quarantine.\allowbreak{}
304
305 \section{LabHUD: Grouped mission display}

```



```

306 Authoritative source: S25 /\allowbreak{} StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabHUD
307
308 See complete listing on page \pageref{src:S25}.
309
310 \subsection{Proposed implementation}
311 Centralize notice priority and reserved screen regions.\allowbreak{} Preserve original backpack and combat
    ↳ controls.\allowbreak{}
312
313 \subsection{Dependencies to preserve}
314 GWAP attributes, original notices, quiz modal, joystick/\allowbreak{}jump bounds.\allowbreak{}
315
316 \subsection{Acceptance cases}
317 Small portrait/\allowbreak{}landscape, long text, two-digit stages, chat open, quiz open, and
    ↳ keyboard/\allowbreak{}gamepad focus.\allowbreak{}
318
319 \section{Cross-cutting data, assets, and deployment}
320 For persistent data, introduce a versioned server-owned schema and migrations before saving
    ↳ attributes.\allowbreak{} For assets, maintain IDs, provenance, scale, orientation, and fallback behavior; a
    ↳ visual replacement must not change a muzzle convention silently.\allowbreak{} For deployment, compare
    ↳ installed hashes against the chosen baseline, save/\allowbreak{}reopen, test the persistent place, and
    ↳ retain a rollback artifact.\allowbreak{} The geometry export cannot restore these contracts on its
    ↳ own.\allowbreak{}
321
322 \section{Complete proposed coordinate-frame module}
323 The following module is a reference implementation, not installed or runtime-tested.\allowbreak{} It requires a
    ↳ deliberately placed NavigationOrigin and ApproachPoint attachments.\allowbreak{} It does not discover a
    ↳ valid origin automatically, reconstruct stairs, or replace pathfinding.\allowbreak{} Its edgeClear helper
    ↳ is a single swept query, not a complete navigator and not a test for initial overlap.\allowbreak{}
    ↳ Integrate it only after defining these scene objects and testing the contract.\allowbreak{}
324
325 \VerbatimInput[fontsize=\footnotesize,breaklines=true,breakanywhere=true,numbers=left,numbersep=6pt,tabsize=4]
    ↳ {proposed/WorldFrame.lua}
326
327
328 GENERATED REFERENCE: inventory.tex
329 \chapter{Scene inventory and coordinate evidence}
330 Captured from the Edit datamodel.\allowbreak{} Counts describe tagged objects or editor inventory, not runtime
    ↳ performance.\allowbreak{} Place ID 137098589879404; reported place version 70; configured MaxPlayers
    ↳ 60.\allowbreak{} No Play mode was started for this capture.\allowbreak{}
331
332 \section{Laboratory equipment kinds}
333 \begin{longtable}{p{0.7\textwidth}p{0.2\textwidth}}
334 \toprule
335 \textbf{Kind} & \textbf{Count} \\ \midrule
336 \endfirsthead
337 \textbf{Kind} & \textbf{Count} \\ \midrule
338 \endhead
339 Decontamination Unit & 84 \\
340 GMO Injector & 1 \\
341 Germination Chamber & 119 \\
342 Growth Hormone Mixer & 9 \\
343 Hormone Treatment Bench & 45 \\
344 Incubator & 109 \\
345 Mutagenesis Chamber & 43 \\
346 Nutrient Infuser & 1 \\
347 Plant Growth Chamber & 450 \\
348 Pollination Station & 6 \\
349 UV Growth Scanner & 5 \\
350 \bottomrule
351 \end{longtable}
352 \section{Research station positions and stored approaches}
353 Every row is a captured station.\allowbreak{} Floor is a logical index (-1 basement, 0 Level_\allowbreak{}1, 8
    ↳ roof).\allowbreak{} Positions and approach points are printed to two decimals for readability;
    ↳ evidence/\allowbreak{}live-world-inventory.\allowbreak{}json preserves full returned
    ↳ precision.\allowbreak{} The mismatch is especially visible in Y and Z and is not a harmless label
    ↳ difference.\allowbreak{}
354
355 \begin{longtable}{p{0.38\textwidth}p{0.27\textwidth}p{0.27\textwidth}}
356 \toprule
357 \textbf{Station /\allowbreak{} floor} & \textbf{Actual position} & \textbf{Stored approach} \\ \midrule
358 \endfirsthead
359 \textbf{Station /\allowbreak{} floor} & \textbf{Actual position} & \textbf{Stored approach} \\ \midrule
360 \endhead
361 Bio-analyzer /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 140.\allowbreak{}29 &
    ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 49.\allowbreak{}00 \\

```

362 Bio-analyzer - Backup 1 /allowbreak{} 2 & 63.\allowbreak{}86, 10.\allowbreak{}75, 140.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 49.\allowbreak{}00 \\
363 Bio-analyzer - Backup 2 /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 255.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 164.\allowbreak{}00 \\
364 Centrifuge /allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 174.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 35.\allowbreak{}75, 83.\allowbreak{}00 \\
365 Centrifuge - Backup 1 /allowbreak{} 2 & 63.\allowbreak{}86, 10.\allowbreak{}75, 174.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 83.\allowbreak{}00 \\
366 Centrifuge - Backup 2 /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 269.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 178.\allowbreak{}00 \\
367 Chemical seedlings /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 157.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 66.\allowbreak{}00 \\
368 Chemical seedlings - Backup 1 /allowbreak{} -1 & 63.\allowbreak{}86, -38.\allowbreak{}00, 157.\allowbreak{}29
↳ & 78.\allowbreak{}00, -13.\allowbreak{}00, 66.\allowbreak{}00 \\
369 Chemical seedlings - Backup 2 /allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 255.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 164.\allowbreak{}00 \\
370 Climate chamber /allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 191.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 100.\allowbreak{}00 \\
371 Climate chamber - Backup 1 /allowbreak{} 2 & 63.\allowbreak{}86, 10.\allowbreak{}75, 191.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 100.\allowbreak{}00 \\
372 Climate chamber - Backup 2 /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 283.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 192.\allowbreak{}00 \\
373 Clinostat /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 174.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 83.\allowbreak{}00 \\
374 Clinostat - Backup 1 /allowbreak{} -1 & 63.\allowbreak{}86, -38.\allowbreak{}00, 174.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 83.\allowbreak{}00 \\
375 Clinostat - Backup 2 /allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 269.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 178.\allowbreak{}00 \\
376 Containment /allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 140.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 49.\allowbreak{}00 \\
377 Containment - Backup 1 /allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 140.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 49.\allowbreak{}00 \\
378 Containment - Backup 2 /allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 255.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 164.\allowbreak{}00 \\
379 Cryo-freezer /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 208.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 117.\allowbreak{}00 \\
380 Cryo-freezer - Backup 1 /allowbreak{} -1 & 63.\allowbreak{}86, -38.\allowbreak{}00, 208.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 117.\allowbreak{}00 \\
381 Cryo-freezer - Backup 2 /allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 283.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 192.\allowbreak{}00 \\
382 Incubator /allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 140.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 3.\allowbreak{}25, 49.\allowbreak{}00 \\
383 Incubator - Backup 1 /allowbreak{} 0 & 63.\allowbreak{}86, -21.\allowbreak{}75, 140.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 49.\allowbreak{}00 \\
384 Incubator - Backup 2 /allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 255.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 164.\allowbreak{}00 \\
385 Infrared scanner /allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 157.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 66.\allowbreak{}00 \\
386 Infrared scanner - Backup 1 /allowbreak{} 2 & 63.\allowbreak{}86, 10.\allowbreak{}75, 157.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 66.\allowbreak{}00 \\
387 Infrared scanner - Backup 2 /allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 297.\allowbreak{}29 &
↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 206.\allowbreak{}00 \\
388 Orientation scanner /allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 174.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 83.\allowbreak{}00 \\
389 Orientation scanner - Backup 1 /allowbreak{} 0 & 63.\allowbreak{}86, -21.\allowbreak{}75, 174.\allowbreak{}29
↳ & 78.\allowbreak{}00, 3.\allowbreak{}25, 83.\allowbreak{}00 \\
390 Orientation scanner - Backup 2 /allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 269.\allowbreak{}29 &
↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 178.\allowbreak{}00 \\
391 Plot 1 /allowbreak{} 8 & 22.\allowbreak{}86, 107.\allowbreak{}55, 139.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 48.\allowbreak{}00 \\
392 Plot 10 /allowbreak{} 8 & 82.\allowbreak{}86, 107.\allowbreak{}55, 219.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 128.\allowbreak{}00 \\
393 Plot 2 /allowbreak{} 8 & 82.\allowbreak{}86, 107.\allowbreak{}55, 139.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 48.\allowbreak{}00 \\
394 Plot 3 /allowbreak{} 8 & 22.\allowbreak{}86, 107.\allowbreak{}55, 159.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 68.\allowbreak{}00 \\
395 Plot 4 /allowbreak{} 8 & 82.\allowbreak{}86, 107.\allowbreak{}55, 159.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 68.\allowbreak{}00 \\
396 Plot 5 /allowbreak{} 8 & 22.\allowbreak{}86, 107.\allowbreak{}55, 179.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 88.\allowbreak{}00 \\
397 Plot 6 /allowbreak{} 8 & 82.\allowbreak{}86, 107.\allowbreak{}55, 179.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 88.\allowbreak{}00 \\
398 Plot 7 /allowbreak{} 8 & 22.\allowbreak{}86, 107.\allowbreak{}55, 199.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 108.\allowbreak{}00 \\
399 Plot 8 /allowbreak{} 8 & 82.\allowbreak{}86, 107.\allowbreak{}55, 199.\allowbreak{}29 & 78.\allowbreak{}00,
↳ 133.\allowbreak{}60, 108.\allowbreak{}00 \\

```

400 Plot 9 /\allowbreak{} 8 & 22.\allowbreak{}86, 107.\allowbreak{}55, 219.\allowbreak{}29 & 78.\allowbreak{}00,
   ↳ 133.\allowbreak{}60, 128.\allowbreak{}00 \\
401 Porometer /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 208.\allowbreak{}29 & 78.\allowbreak{}00,
   ↳ 19.\allowbreak{}50, 117.\allowbreak{}00 \\
402 Porometer - Backup 1 /\allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 208.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 117.\allowbreak{}00 \\
403 Porometer - Backup 2 /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 269.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 178.\allowbreak{}00 \\
404 Root analyzer /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 174.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 83.\allowbreak{}00 \\
405 Root analyzer - Backup 1 /\allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 174.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 83.\allowbreak{}00 \\
406 Root analyzer - Backup 2 /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 283.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 192.\allowbreak{}00 \\
407 Seed bank /\allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 140.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 49.\allowbreak{}00 \\
408 Seed bank - Backup 1 /\allowbreak{} -1 & 63.\allowbreak{}86, -38.\allowbreak{}00, 140.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 49.\allowbreak{}00 \\
409 Seed bank - Backup 2 /\allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 297.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 206.\allowbreak{}00 \\
410 Spectrograph /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 157.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 66.\allowbreak{}00 \\
411 Spectrograph - Backup 1 /\allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 157.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 66.\allowbreak{}00 \\
412 Spectrograph - Backup 2 /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 297.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 206.\allowbreak{}00 \\
413 Split-root tray /\allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 157.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 66.\allowbreak{}00 \\
414 Split-root tray - Backup 1 /\allowbreak{} 0 & 63.\allowbreak{}86, -21.\allowbreak{}75, 157.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 66.\allowbreak{}00 \\
415 Split-root tray - Backup 2 /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 283.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 192.\allowbreak{}00 \\
416 Symbiosis incubator /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 225.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 134.\allowbreak{}00 \\
417 Symbiosis incubator - Backup 1 /\allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 225.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 134.\allowbreak{}00 \\
418 Symbiosis incubator - Backup 2 /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 311.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 220.\allowbreak{}00 \\
419 UV chamber /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 191.\allowbreak{}29 & 78.\allowbreak{}00,
   ↳ 19.\allowbreak{}50, 100.\allowbreak{}00 \\
420 UV chamber - Backup 1 /\allowbreak{} 1 & 63.\allowbreak{}86, -5.\allowbreak{}50, 191.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 100.\allowbreak{}00 \\
421 UV chamber - Backup 2 /\allowbreak{} 2 & 41.\allowbreak{}86, 10.\allowbreak{}75, 325.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 35.\allowbreak{}75, 234.\allowbreak{}00 \\
422 Vitality scanner /\allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 191.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 100.\allowbreak{}00 \\
423 Vitality scanner - Backup 1 /\allowbreak{} 0 & 63.\allowbreak{}86, -21.\allowbreak{}75, 191.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 100.\allowbreak{}00 \\
424 Vitality scanner - Backup 2 /\allowbreak{} 1 & 41.\allowbreak{}86, -5.\allowbreak{}50, 297.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 19.\allowbreak{}50, 206.\allowbreak{}00 \\
425 Wet lab /\allowbreak{} -1 & 41.\allowbreak{}86, -38.\allowbreak{}00, 191.\allowbreak{}29 & 78.\allowbreak{}00,
   ↳ -13.\allowbreak{}00, 100.\allowbreak{}00 \\
426 Wet lab - Backup 1 /\allowbreak{} -1 & 63.\allowbreak{}86, -38.\allowbreak{}00, 191.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, -13.\allowbreak{}00, 100.\allowbreak{}00 \\
427 Wet lab - Backup 2 /\allowbreak{} 0 & 41.\allowbreak{}86, -21.\allowbreak{}75, 311.\allowbreak{}29 &
   ↳ 78.\allowbreak{}00, 3.\allowbreak{}25, 220.\allowbreak{}00 \\
428 \bottomrule
429 \end{longtable}
430 \section{Important tag counts}
431 \begin{longtable}{p{0.7\textwidth}p{0.2\textwidth}}
432 \toprule
433 \textbf{Tag} & \textbf{Count} \\ \midrule
434 \endfirsthead
435 \textbf{Tag} & \textbf{Count} \\ \midrule
436 \endhead
437 AutoDoor & 162 \\
438 Automatable & 313 \\
439 EmergencyKit & 116 \\
440 GasPipe & 18 \\
441 LabBreakableWindow & 130 \\
442 LabDecor & 14 \\
443 LabEquipment & 872 \\
444 LabPoster & 72 \\
445 LabToolDispenser & 90 \\
446 LabWorkerBot & 180 \\
447 PlantGrowthChamber & 450

```

```

448 RoleKiosk & 2 \\
449 RoomMonitor & 81 \\
450 SecretBasementPanel & 1 \\
451 \bottomrule
452 \end{longtable}
453 \section{Latest exported files}
454 The export manifest contains 46 files totaling 197,533,070 bytes.\allowbreak{} The RBXL copy is included;
    ↳ OBJ\allowbreak{}MTL and textures remain at D:\allowbreak{}game\allowbreak{}5 and are identified by
    ↳ SHA-256 in the manifest.\allowbreak{}
455
456 \begin{longtable}{p{0.7\textwidth}p{0.2\textwidth}}
457 \toprule
458 \textbf{Relative export file} & \textbf{Bytes} \\ \midrule
459 \endfirsthead
460 \textbf{Relative export file} & \textbf{Bytes} \\ \midrule
461 \endhead
462 obj\textbackslash\allowbreak{}Casterwheel1\allowbreak{}diff.\allowbreak{}png & 3,268 \\
463 obj\textbackslash\allowbreak{}Casterwheel1\allowbreak{}nmap.\allowbreak{}png & 1,398 \\
464 obj\textbackslash\allowbreak{}Casterwheel1\allowbreak{}spec.\allowbreak{}png & 224 \\
465 obj\textbackslash\allowbreak{}Corridoreast71\allowbreak{}diff.\allowbreak{}png & 784,015 \\
466 obj\textbackslash\allowbreak{}Corridoreast71\allowbreak{}nmap.\allowbreak{}png & 2,002,950 \\
467 obj\textbackslash\allowbreak{}Corridoreast71\allowbreak{}spec.\allowbreak{}png & 1,885 \\
468 obj\textbackslash\allowbreak{}Flag1\allowbreak{}diff.\allowbreak{}png & 9,168 \\
469 obj\textbackslash\allowbreak{}Flag1\allowbreak{}nmap.\allowbreak{}png & 21,313 \\
470 obj\textbackslash\allowbreak{}Flag1\allowbreak{}spec.\allowbreak{}png & 407 \\
471 obj\textbackslash\allowbreak{}Floor1\allowbreak{}diff.\allowbreak{}png & 6,199 \\
472 obj\textbackslash\allowbreak{}Floor1\allowbreak{}nmap.\allowbreak{}png & 6,143 \\
473 obj\textbackslash\allowbreak{}Floor1\allowbreak{}spec.\allowbreak{}png & 1,885 \\
474 obj\textbackslash\allowbreak{}Head2\allowbreak{}diff.\allowbreak{}png & 1,053 \\
475 obj\textbackslash\allowbreak{}LaboratoryTechnicianpanelframe1\allowbreak{}diff.\allowbreak{}png & 16,189 \\
476 obj\textbackslash\allowbreak{}LaboratoryTechnicianpanelframe1\allowbreak{}nmap.\allowbreak{}png & 23,842 \\
477 obj\textbackslash\allowbreak{}LaboratoryTechnicianpanelframe1\allowbreak{}spec.\allowbreak{}png & 871 \\
478 obj\textbackslash\allowbreak{}Loungehatch1\allowbreak{}nmap.\allowbreak{}png & 4,522 \\
479 obj\textbackslash\allowbreak{}Loungehatch1\allowbreak{}spec.\allowbreak{}png & 223 \\
480 obj\textbackslash\allowbreak{}obj.\allowbreak{}mtl & 105,560 \\
481 obj\textbackslash\allowbreak{}obj.\allowbreak{}obj & 193,872,256 \\
482 obj\textbackslash\allowbreak{}Roof11\allowbreak{}diff.\allowbreak{}png & 10,348 \\
483 obj\textbackslash\allowbreak{}Roof11\allowbreak{}nmap.\allowbreak{}png & 15,663 \\
484 obj\textbackslash\allowbreak{}Roof11\allowbreak{}spec.\allowbreak{}png & 1,885 \\
485 obj\textbackslash\allowbreak{}Sitepad81\allowbreak{}diff.\allowbreak{}png & 2,356 \\
486 obj\textbackslash\allowbreak{}Sitepad81\allowbreak{}nmap.\allowbreak{}png & 2,007 \\
487 obj\textbackslash\allowbreak{}Sitepad81\allowbreak{}spec.\allowbreak{}png & 1,885 \\
488 obj\textbackslash\allowbreak{}Soil1\allowbreak{}diff.\allowbreak{}png & 2,096 \\
489 obj\textbackslash\allowbreak{}Soil1\allowbreak{}nmap.\allowbreak{}png & 1,938 \\
490 obj\textbackslash\allowbreak{}Soil1\allowbreak{}spec.\allowbreak{}png & 1,885 \\
491 obj\textbackslash\allowbreak{}Spawnlocation1\allowbreak{}diff.\allowbreak{}png & 33,475 \\
492 obj\textbackslash\allowbreak{}Spawnlocation1\allowbreak{}nmap.\allowbreak{}png & 46,508 \\
493 obj\textbackslash\allowbreak{}Spawnlocation2\allowbreak{}diff.\allowbreak{}png & 1,322 \\
494 obj\textbackslash\allowbreak{}Tree3trunk1\allowbreak{}diff.\allowbreak{}png & 4,234 \\
495 obj\textbackslash\allowbreak{}Tree3trunk1\allowbreak{}nmap.\allowbreak{}png & 936 \\
496 obj\textbackslash\allowbreak{}Tree3trunk1\allowbreak{}spec.\allowbreak{}png & 1,885 \\
497 obj\textbackslash\allowbreak{}Tsouthglass31\allowbreak{}diff.\allowbreak{}png & 17,625 \\
498 obj\textbackslash\allowbreak{}Tsouthglass31\allowbreak{}nmap.\allowbreak{}png & 1,512 \\
499 obj\textbackslash\allowbreak{}Tsouthglass31\allowbreak{}spec.\allowbreak{}png & 120 \\
500 obj\textbackslash\allowbreak{}Walle1\allowbreak{}diff.\allowbreak{}png & 2,409 \\
501 obj\textbackslash\allowbreak{}Walle1\allowbreak{}nmap.\allowbreak{}png & 1,767 \\
502 obj\textbackslash\allowbreak{}Walle1\allowbreak{}spec.\allowbreak{}png & 120 \\
503 obj\textbackslash\allowbreak{}West2\allowbreak{}diff.\allowbreak{}png & 900 \\
504 obj\textbackslash\allowbreak{}Workerbot04desktop1\allowbreak{}diff.\allowbreak{}png & 19,051 \\
505 obj\textbackslash\allowbreak{}Workerbot04desktop1\allowbreak{}nmap.\allowbreak{}png & 5,889 \\
506 obj\textbackslash\allowbreak{}Workerbot04desktop1\allowbreak{}spec.\allowbreak{}png & 407 \\
507 rblx\textbackslash\allowbreak{}rblx.\allowbreak{}rbxl & 491,476 \\
508 \bottomrule
509 \end{longtable}
510 \section{Coordinate repair decision}
511 Do not automatically translate the laboratory back by the measured offset.\allowbreak{} The current position
    ↳ may be intentional.\allowbreak{} First establish the desired world placement with the owner or scene
    ↳ baseline, then make navigation metadata follow that placement.\allowbreak{} The safer architecture uses
    ↳ local coordinates and moving attachments so subsequent translations do not repeat this defect.\allowbreak{}
512
513
514
515 GENERATED REFERENCE: provenance.tex
516 \chapter{Source provenance and complete-code map}

```

517 The complete final active sources are printed in full, together with the server-only documentation module,
 ↳ eight Studio archive sources, one disabled generator, and 12 historical workspace artifacts.\allowbreak{}
 ↳ The pre-revision active sources remain in code/\allowbreak{}baseline for exact comparison.\allowbreak{} The
 ↳ printed appendices deliberately retain duplicate and conflicting versions so the history is inspectable;
 ↳ they are grouped by authority.\allowbreak{} Each listing restarts line numbering at one.\allowbreak{} The
 ↳ source ZIP contains byte-preserved originals.\allowbreak{} Proposed code is kept separately and never
 ↳ labeled active.\allowbreak{}
 518
 519 The complete latest place is supplied as evidence/\allowbreak{}export-5.\allowbreak{}rbxl.\allowbreak{}
 ↳ Geometry meshes and textures are not Lua code and are listed separately.\allowbreak{} Unrelated downloaded
 ↳ script repositories are not included as active game source.\allowbreak{} No implementation is shortened
 ↳ with ellipses or pseudocode inside a captured listing.\allowbreak{}
 520
 521 \begin{longtable}{p{0.07\textwidth}p{0.67\textwidth}p{0.18\textwidth}}
 522 \toprule
 523 \textbf{ID} & \textbf{Artifact} & \textbf{Lines \allowbreak{} bytes} \\ \midrule
 524 \endfirsthead
 525 \textbf{ID} & \textbf{Artifact} & \textbf{Lines \allowbreak{} bytes} \\ \midrule
 526 \endhead
 527 S01 & ReplicatedStorage.\allowbreak{}GWAPNPC & 96 \allowbreak{} 6183 \\ \\\br/>
 528 S02 & ReplicatedStorage.\allowbreak{}GWAPRouting & 55 \allowbreak{} 3434 \\ \\\br/>
 529 S03 & ReplicatedStorage.\allowbreak{}GWAPScenarios & 23 \allowbreak{} 2692 \\ \\\br/>
 530 S04 & ReplicatedStorage.\allowbreak{}GWAPWorld & 95 \allowbreak{} 7198 \\ \\\br/>
 531 S05 & ReplicatedStorage.\allowbreak{}LabKinds & 173 \allowbreak{} 10408 \\ \\\br/>
 532 S06 & ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}Blaster & 239 \allowbreak{} 9143 \\ \\\br/>
 533 S07 & ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}BlasterController & 915 \allowbreak{} 35949 \\ \\\br/>
 534 S08 & ServerScriptService.\allowbreak{}GWAPServer & 469 \allowbreak{} 32199 \\ \\\br/>
 535 S09 & ServerScriptService.\allowbreak{}GWAPWallDamage & 55 \allowbreak{} 3254 \\ \\\br/>
 536 S10 & ServerScriptService.\allowbreak{}LabElevators & 132 \allowbreak{} 4405 \\ \\\br/>
 537 S11 & ServerScriptService.\allowbreak{}LabEquipment & 4178 \allowbreak{} 182838 \\ \\\br/>
 538 S12 & ServerScriptService.\allowbreak{}LabRoles & 431 \allowbreak{} 19385 \\ \\\br/>
 539 S13 & ServerScriptService.\allowbreak{}LabRound & 94 \allowbreak{} 3591 \\ \\\br/>
 540 S14 & ServerScriptService.\allowbreak{}LabTechnicians & 210 \allowbreak{} 7915 \\ \\\br/>
 541 S15 & ServerScriptService.\allowbreak{}LoungeSystems & 172 \allowbreak{} 5872 \\ \\\br/>
 542 S16 & ServerScriptService.\allowbreak{}ServerBlasterManager & 195 \allowbreak{} 8483 \\ \\\br/>
 543 S17 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LabElevators & 123
 ↳ \allowbreak{} 3761 \\ \\\br/>
 544 S18 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LabEquipment & 4068
 ↳ \allowbreak{} 175504 \\ \\\br/>
 545 S19 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LabRoles & 422
 ↳ \allowbreak{} 18757 \\ \\\br/>
 546 S20 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LabRound & 85
 ↳ \allowbreak{} 2954 \\ \\\br/>
 547 S21 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LabTechnicians & 201
 ↳ \allowbreak{} 7271 \\ \\\br/>
 548 S22 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}LoungeSystems & 152
 ↳ \allowbreak{} 4605 \\ \\\br/>
 549 S23 & ServerStorage.\allowbreak{}BeforeBasementAndQuiz_allowbreak{}20260912.\allowbreak{}ServerBlasterManager
 ↳ & 175 \allowbreak{} 7275 \\ \\\br/>
 550 S24 & ServerStorage.\allowbreak{}BlasterController_allowbreak{}BeforeMouseArmFix_allowbreak{}20260912 & 730
 ↳ \allowbreak{} 26610 \\ \\\br/>
 551 S25 & StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabHUD & 140 \allowbreak{} 12852 \\ \\\br/>
 552 S26 & StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabQuizClient & 138 \allowbreak{} 5319 \\ \\\br/>
 553 S27 & StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LabSidearmClient & 54 \allowbreak{} 2409 \\ \\\br/>
 554 S28 & StarterPlayer.\allowbreak{}StarterPlayerScripts.\allowbreak{}LoungeClient & 85 \allowbreak{} 3247 \\ \\\br/>
 555 S29 & Workspace.\allowbreak{}SeedLabGenerator_allowbreak{}DISABLED_allowbreak{}doNotEnable & 11886
 ↳ \allowbreak{} 524207 \\ \\\br/>
 556 W01 & SeedLabGenerator.\allowbreak{}lua & 12398 \allowbreak{} 552155 \\ \\\br/>
 557 W02 & InstallGWAP.\allowbreak{}lua & 2109 \allowbreak{} 115778 \\ \\\br/>
 558 W03 & GWAPWorld.\allowbreak{}lua & 259 \allowbreak{} 17807 \\ \\\br/>
 559 W04 & GWAPScenarios.\allowbreak{}lua & 13 \allowbreak{} 2067 \\ \\\br/>
 560 W05 & GWAPNPC.\allowbreak{}lua & 90 \allowbreak{} 6021 \\ \\\br/>
 561 W06 & GWAPRouting.\allowbreak{}lua & 43 \allowbreak{} 2594 \\ \\\br/>
 562 W07 & GWAPServer.\allowbreak{}lua & 418 \allowbreak{} 28562 \\ \\\br/>
 563 W08 & GWAPWallDamage.\allowbreak{}lua & 45 \allowbreak{} 2594 \\ \\\br/>
 564 W09 & GWAPHUD.\allowbreak{}lua & 123 \allowbreak{} 11596 \\ \\\br/>
 565 W10 & Blaster.\allowbreak{}lua & 227 \allowbreak{} 8432 \\ \\\br/>
 566 W11 & BlasterController.\allowbreak{}lua & 905 \allowbreak{} 35188 \\ \\\br/>
 567 W12 & ServerBlasterManager.\allowbreak{}lua & 184 \allowbreak{} 7714 \\ \\\br/>
 568 S21 & ReplicatedStorage.\allowbreak{}LabSpatial & 39 \allowbreak{} 1994 \\ \\\br/>
 569 \bottomrule
 570 \end{longtable}
 571 \section{Hash verification}
 572 \subsection{S01 GWAPNPC}
 573 Group: active; file:
 ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}GWAPNPC.\allowbreak{}lua

```
574
575 \begin{flushleft}\ttfamily\footnotesize
    ↳ c9698d535e3d7947acf208b4f8ee49f2ad08c853badfd573fa7173a1435f5d15\end{flushleft}
576 \subsection{S02 GWAPRouting}
577 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}GWAPRouting.\allowbreak{}lua
578
579 \begin{flushleft}\ttfamily\footnotesize
    ↳ f29a41157ff14934f5a25ac7e9e065450f3a32dc13c868a57378e5d73e80e8ee\end{flushleft}
580 \subsection{S03 GWAPScenarios}
581 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}GWAPScenarios.\allowbreak{}lua
582
583 \begin{flushleft}\ttfamily\footnotesize
    ↳ 6a6d613711d1cd7762fc1140ef53c6c3ff25d1e5faea06538d32f8aa79452796\end{flushleft}
584 \subsection{S04 GWAPWorld}
585 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}GWAPWorld.\allowbreak{}lua
586
587 \begin{flushleft}\ttfamily\footnotesize
    ↳ 2b7829521e74564f068cb147223e063e5b680cf3cd44f87e128b357d607cf19c\end{flushleft}
588 \subsection{S05 LabKinds}
589 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}LabKinds.\allowbreak{}lua
590
591 \begin{flushleft}\ttfamily\footnotesize
    ↳ eb2dbeb766efed328eb3e1165c9b41e93546c3fa49615fdb00646d43aa9dea2\end{flushleft}
592 \subsection{S06 Blaster}
593 Group: active; file: code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}
    ↳ }Blaster.\allowbreak{}lua
594
595 \begin{flushleft}\ttfamily\footnotesize
    ↳ 616d0fc7234636c563afbc93798f81e1be3a9fbc83084de44196f4d0ed8be2ab\end{flushleft}
596 \subsection{S07 BlasterController}
597 Group: active; file: code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}Modules.\allowbreak{}
    ↳ }BlasterController.\allowbreak{}lua
598
599 \begin{flushleft}\ttfamily\footnotesize
    ↳ ef4a80c0aa4c3alded46c55bef82d058ddb29bb83dd8f34cfa73d22c9647769d\end{flushleft}
600 \subsection{S08 GWAPServer}
601 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}GWAPServer.\allowbreak{}lua
602
603 \begin{flushleft}\ttfamily\footnotesize
    ↳ 265a4f246bfd1c1b107cc4ae2df6cf8babde7c682de18838cfba553c55c7c653\end{flushleft}
604 \subsection{S09 GWAPWallDamage}
605 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}GWAPWallDamage.\allowbreak{}lua
606
607 \begin{flushleft}\ttfamily\footnotesize
    ↳ 92a3df3c35188633291afa9388a99db7ed9791f34c1ef14a299301c46a96f952\end{flushleft}
608 \subsection{S10 LabElevators}
609 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LabElevators.\allowbreak{}lua
610
611 \begin{flushleft}\ttfamily\footnotesize
    ↳ d5c42a7d4217033660c71b966b4d762409572f63640db28769f0ccf3bb32ae44\end{flushleft}
612 \subsection{S11 LabEquipment}
613 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LabEquipment.\allowbreak{}lua
614
615 \begin{flushleft}\ttfamily\footnotesize
    ↳ eb4722cec0bddcf396cb9dce7199a4b11dc3890fc157367daa219c1489fdae5f\end{flushleft}
616 \subsection{S12 LabRoles}
617 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LabRoles.\allowbreak{}lua
618
619 \begin{flushleft}\ttfamily\footnotesize
    ↳ 7e706724edacbdac6ccd335320891a5c8652a016640abe3e7fda6754203a39a77\end{flushleft}
620 \subsection{S13 LabRound}
621 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LabRound.\allowbreak{}lua
622
623 \begin{flushleft}\ttfamily\footnotesize
    ↳ ed2bd7811d3301d2afc164a9b6bd3c603b1758e760a8c5b4b31006c064946244\end{flushleft}
```

```
624 \subsection{S14 LabTechnicians}
625 Group: active; file:
    ↪ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LabTechnicians.\allowbreak{}lua
626
627 \begin{flushleft}\ttfamily\footnotesize
    ↪ cf7bd8c892576d9038829db70c3a77c9d7519807ab7785940dc472af7f0da85\end{flushleft}
628 \subsection{S15 LoungeSystems}
629 Group: active; file:
    ↪ code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}LoungeSystems.\allowbreak{}lua
630
631 \begin{flushleft}\ttfamily\footnotesize
    ↪ 78327a7e0eb4d5ce209c2b9d0a504708d378917a727d0ce530f144d84e1c2b42\end{flushleft}
632 \subsection{S16 ServerBlasterManager}
633 Group: active; file: code/\allowbreak{}active/\allowbreak{}ServerScriptService.\allowbreak{}ServerBlasterManag
    ↪ er.\allowbreak{}lua
634
635 \begin{flushleft}\ttfamily\footnotesize
    ↪ 50671031f74ebbfede5259e3447b34087fc2a47387ffb378de169616a1be522\end{flushleft}
636 \subsection{S17 LabElevators}
637 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LabElevators.\allowbreak{}lua
638
639 \begin{flushleft}\ttfamily\footnotesize
    ↪ 62b29049771761ea23fcf71ce84e715ea44f0945e4e3f81a078167d37b15f448\end{flushleft}
640 \subsection{S18 LabEquipment}
641 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LabEquipment.\allowbreak{}lua
642
643 \begin{flushleft}\ttfamily\footnotesize
    ↪ d7ffee0ffe6bf5f1a3d4f1086a5569adb7aad76c3dc66d5cdeed5959cf9b4d08\end{flushleft}
644 \subsection{S19 LabRoles}
645 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LabRoles.\allowbreak{}lua
646
647 \begin{flushleft}\ttfamily\footnotesize
    ↪ 9b4d4429d279f97793519a441fd8c4f4d6f7e6af9c934e270aedb56305fc62dd\end{flushleft}
648 \subsection{S20 LabRound}
649 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LabRound.\allowbreak{}lua
650
651 \begin{flushleft}\ttfamily\footnotesize
    ↪ 9ef5725f5f034aec29be6daaf9864ad6ce2e08c3f854e61c035ba286c197a5c1\end{flushleft}
652 \subsection{S21 LabTechnicians}
653 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LabTechnicians.\allowbreak{}lua
654
655 \begin{flushleft}\ttfamily\footnotesize
    ↪ 5c9a649cef010ed65b142db221fd92830593c0ee8c4281f5c8606b8ede66f01c\end{flushleft}
656 \subsection{S22 LoungeSystems}
657 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}LoungeSystems.\allowbreak{}lua
658
659 \begin{flushleft}\ttfamily\footnotesize
    ↪ 72f8a9e545a7818dc813e749aaabea27d3de773599005f2cf5ac6690e6b24615\end{flushleft}
660 \subsection{S23 ServerBlasterManager}
661 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BeforeBasementAndQuiz\
    ↪ _\allowbreak{}20260912.\allowbreak{}ServerBlasterManager.\allowbreak{}lua
662
663 \begin{flushleft}\ttfamily\footnotesize
    ↪ e38caa71d7c2f94536a0f0273248d77bc6a5df585d480bb693f1e41f58c4d74a\end{flushleft}
664 \subsection{S24 BlasterController\_\\allowbreak{}BeforeMouseArmFix\_\\allowbreak{}20260912}
665 Group: archive; file: code/\allowbreak{}archive/\allowbreak{}ServerStorage.\allowbreak{}BlasterController\_\\al
    ↪ lowbreak{}BeforeMouseArmFix\_\\allowbreak{}20260912.\allowbreak{}lua
666
667 \begin{flushleft}\ttfamily\footnotesize
    ↪ 8a756113e948711c647a10a790ec624081297515dc3ef9504c03dcacf692135e8\end{flushleft}
668 \subsection{S25 LabHUD}
669 Group: active; file: code/\allowbreak{}active/\allowbreak{}StarterPlayer.\allowbreak{}StarterPlayerScripts.\al
    ↪ lowbreak{}LabHUD.\allowbreak{}lua
670
671 \begin{flushleft}\ttfamily\footnotesize
    ↪ d8982e721a569a2eaa70a0f73fd7106e78cf19f39b056a76ced59329602f8a0a\end{flushleft}
672 \subsection{S26 LabQuizClient}
```

```
673 Group: active; file: code/\allowbreak{}active/\allowbreak{}StarterPlayer.\allowbreak{}StarterPlayerScripts.\al_
    ↳ lowbreak{}LabQuizClient.\allowbreak{}lua
674
675 \begin{flushleft}\ttfamily\footnotesize
    ↳ 316259614108e07c7273dd92fe632343fb797c988877cf71694726e59380fc57\end{flushleft}
676 \subsection{S27 LabSidearmClient}
677 Group: active; file: code/\allowbreak{}active/\allowbreak{}StarterPlayer.\allowbreak{}StarterPlayerScripts.\al_
    ↳ lowbreak{}LabSidearmClient.\allowbreak{}lua
678
679 \begin{flushleft}\ttfamily\footnotesize
    ↳ a2e10adb43696aefe2a6fa8d7aa8570d5a64d09d9cae9f32398ce6e944b52843\end{flushleft}
680 \subsection{S28 LoungeClient}
681 Group: active; file: code/\allowbreak{}active/\allowbreak{}StarterPlayer.\allowbreak{}StarterPlayerScripts.\al_
    ↳ lowbreak{}LoungeClient.\allowbreak{}lua
682
683 \begin{flushleft}\ttfamily\footnotesize
    ↳ eb857c214587168a13cd60243327a4b8588fdfe48a3947d20c1501393bf6bc45\end{flushleft}
684 \subsection{S29 SeedLabGenerator\_allowbreak{}DISABLED\_allowbreak{}doNotEnable}
685 Group: disabled; file: code/\allowbreak{}disabled/\allowbreak{}Workspace.\allowbreak{}SeedLabGenerator\_allow_
    ↳ break{}DISABLED\_allowbreak{}doNotEnable.\allowbreak{}lua
686
687 \begin{flushleft}\ttfamily\footnotesize
    ↳ dc633d385d37c97f171de5725a0523f12beaa19df26006138968f14f5a545e5d\end{flushleft}
688 \subsection{W01 lua}
689 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}SeedLabGenerator.\allowbreak{}lua
690
691 \begin{flushleft}\ttfamily\footnotesize
    ↳ 219d03a12736586aefbe39cc89c7612f9134819807d0ecc60450bbb487f55af\end{flushleft}
692 \subsection{W02 lua}
693 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}InstallGWAP.\allowbreak{}lua
694
695 \begin{flushleft}\ttfamily\footnotesize
    ↳ 06e6d896bb0fe9fdf14c04ac0810c10f79219372831abb352a3f08982bdca511\end{flushleft}
696 \subsection{W03 lua}
697 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPWorld.\allowbreak{}lua
698
699 \begin{flushleft}\ttfamily\footnotesize
    ↳ 1745af08772025f20c0fea0e44532d25244454346e7cd6b2f22d0b41dd09e7c5\end{flushleft}
700 \subsection{W04 lua}
701 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPScenarios.\allowbreak{}lua
702
703 \begin{flushleft}\ttfamily\footnotesize
    ↳ d27afc6b0edf955031d55ec0f910394c2d98ede64daeed7b7bc27ae634442136\end{flushleft}
704 \subsection{W05 lua}
705 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPNPC.\allowbreak{}lua
706
707 \begin{flushleft}\ttfamily\footnotesize
    ↳ 77bd356a1d0fbff2b71c3ec24eaecf99565a6304b6f1e478d1b257b89007ed05\end{flushleft}
708 \subsection{W06 lua}
709 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPRouting.\allowbreak{}lua
710
711 \begin{flushleft}\ttfamily\footnotesize
    ↳ b41a223e553d10f13e0d6a79c3ef2b616e4cc0d43cfc986aa75c84e3af6186ca\end{flushleft}
712 \subsection{W07 lua}
713 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPServer.\allowbreak{}lua
714
715 \begin{flushleft}\ttfamily\footnotesize
    ↳ 3546a88f457530f2e6a7efb7e6a5ad3f4bfda3502b3b97a6368750517fe8693\end{flushleft}
716 \subsection{W08 lua}
717 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPWallDamage.\allowbreak{}lua
718
719 \begin{flushleft}\ttfamily\footnotesize
    ↳ cae9ce8d81cdd3261702f39670c8bf456628f065c3454c638757d3d61d4e577b\end{flushleft}
720 \subsection{W09 lua}
721 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}GWAPHUD.\allowbreak{}lua
722
723 \begin{flushleft}\ttfamily\footnotesize
    ↳ 20b2619e281c1608e9c543ec522e7c8e51a9e1c69eb0e73d4d9cc54105aaebd\end{flushleft}
724 \subsection{W10 lua}
725 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}Blaster.\allowbreak{}lua
726
727 \begin{flushleft}\ttfamily\footnotesize
    ↳ 01c2f0f4432c5673e1bb39cd3ced41e9a984c476a31755010bf1ad207567809c\end{flushleft}
728 \subsection{W11 lua}
729 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}BlasterController.\allowbreak{}lua
```



```

730
731 \begin{flushleft}\ttfamily\footnotesize
    ↳ 4dc3893c33e70e9a39194b52bb937fd3086f212efeadc371f73c71e453fcea\end{flushleft}
732 \subsection{W12 lua}
733 Group: workspace; file: code/\allowbreak{}workspace/\allowbreak{}ServerBlasterManager.\allowbreak{}lua
734
735 \begin{flushleft}\ttfamily\footnotesize
    ↳ 6f5301dddadf2663b5f910ac85fdc691c6a9c23e77aab31947625053d402d449\end{flushleft}
736 \subsection{S21 LabSpatial}
737 Group: active; file:
    ↳ code/\allowbreak{}active/\allowbreak{}ReplicatedStorage.\allowbreak{}LabSpatial.\allowbreak{}lua
738
739 \begin{flushleft}\ttfamily\footnotesize
    ↳ 1593b910e0f231f347b2fec79a28cla3dalc7aca53c273d968ab44f7dec6df5f\end{flushleft}
740 \section{Normalized workspace comparisons}
741 \begin{longtable}{p{0.38\textwidth}p{0.1\textwidth}p{0.43\textwidth}}
742 \toprule
743 \textbf{Workspace file} & \textbf{Live ID} & \textbf{Comparison} \\ \midrule
744 \endfirsthead
745 \textbf{Workspace file} & \textbf{Live ID} & \textbf{Comparison} \\ \midrule
746 \endhead
747 GWAPWorld.\allowbreak{}lua & S04 & DIFFERENT - inspect before installation \\
748 GWAPScenarios.\allowbreak{}lua & S03 & DIFFERENT - inspect before installation \\
749 GWAPNPC.\allowbreak{}lua & S01 & DIFFERENT - inspect before installation \\
750 GWAPRouting.\allowbreak{}lua & S02 & DIFFERENT - inspect before installation \\
751 GWAPServer.\allowbreak{}lua & S08 & DIFFERENT - inspect before installation \\
752 GWAPWallDamage.\allowbreak{}lua & S09 & DIFFERENT - inspect before installation \\
753 GWAPHUD.\allowbreak{}lua & S25 & DIFFERENT - inspect before installation \\
754 Blaster.\allowbreak{}lua & S06 & DIFFERENT - inspect before installation \\
755 BlasterController.\allowbreak{}lua & S07 & DIFFERENT - inspect before installation \\
756 ServerBlasterManager.\allowbreak{}lua & S16 & DIFFERENT - inspect before installation \\
757 \bottomrule
758 \end{longtable}
759 This comparison ignores UTF-8 BOM, line-ending normalization by text reading, and trailing newlines
    ↳ only.\allowbreak{} The raw hashes remain different when bytes differ.\allowbreak{} A normalized match is
    ↳ not a gameplay test.\allowbreak{} The installer contains the incorrect loose GWAPWorld source in its
    ↳ world-module block; it must not be run unchanged.\allowbreak{} The active module is the documentary source
    ↳ for a future repair.\allowbreak{}
760
761
762
763 GENERATED REFERENCE: scenarios.tex
764 \chapter{Ten research scenarios: protocols and extension specifications}
765 All ten scenarios below are implemented as ordered gameplay routes and a randomized integer
    ↳ variant.\allowbreak{} The proposed data product and validation paragraphs are future design, not
    ↳ implemented science.\allowbreak{} Floor labels describe logical floors in the original building
    ↳ frame.\allowbreak{} The initial coordinate discrepancy was repaired in Revision 4; see its fresh
    ↳ verification results and remaining coverage limits.\allowbreak{}
766
767 \section{01.\allowbreak{} Germination viability}
768 \subsection{Implemented route and parameter}
769 Seed bank (B1) -> Incubator (L1) -> Plot 1 (roof)
770
771 Moisture, integer 35-80 percent
772
773 A seedling is collected, incubated, and planted.\allowbreak{} Moisture is a displayed assignment variant; it
    ↳ does not drive a germination probability model.\allowbreak{}
774
775 \subsection{Future scientific-purpose implementation}
776 Annotate germinated versus non-germinated seeds in authentic, protocol-controlled images.\allowbreak{} Store
    ↳ image ID, seed count, germination count, moisture condition, replicate, and observer decision.\allowbreak{}
777
778 Use known examples and repeated independent counts; compare agreement before interpreting treatment
    ↳ differences.\allowbreak{} Preserve the original image and sampling method.\allowbreak{}
779
780 \subsection{Engineering change and acceptance}
781 Try both endpoints of the moisture range; destroy the Incubator before arrival and during
    ↳ processing.\allowbreak{} Verify an equivalent is selected and the nominal two-checkpoint plus final reward
    ↳ is not duplicated.\allowbreak{}
782
783 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}

```

```

784
785 \section{02.\allowbreak{} Pathogen resistance}
786 \subsection{Implemented route and parameter}
787 Containment (L2) -> Bio-analyzer (L3) -> Plot 2 (roof)
788
789 Resistance batch, integer 1-9
790
791 The batch number distinguishes assignments.\allowbreak{} There is no infectious-agent simulation, measured
792 ↳ resistance curve, or actual laboratory manipulation.\allowbreak{}
793
794 \subsection{Future scientific-purpose implementation}
795 Classify visible plant-damage severity in a vetted image dataset with blinded treatment labels.\allowbreak{}
796 ↳ Store dataset item, scoring scale version, replicate, and confidence.\allowbreak{}
797
798 Use a researcher-defined severity rubric and reference examples.\allowbreak{} Separate plant damage from image
799 ↳ artifacts; aggregate only comparable stages and cultivars.\allowbreak{}
800
801 \subsection{Engineering change and acceptance}
802 Destroy Containment and its same-floor backup, then confirm the other-floor replacement retains the same
803 ↳ capability and stage.\allowbreak{} Check that resistance batch remains unchanged after
804 ↳ rerouting.\allowbreak{}
805
806 Implementation touchpoints: GWAPScenarios defines
807 ↳ title.\allowbreak{}purpose.\allowbreak{}route.\allowbreak{}range; GWAPWorld and GWAPRouting provide named
808 ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
809 ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
810 ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
811 ↳ change meaning when the list is reordered.\allowbreak{}
812
813 \section{03.\allowbreak{} Light spectrum optimization}
814 \subsection{Implemented route and parameter}
815 Seed bank (B1) -> Spectrograph (L2) -> Infrared scanner (L3) -> Plot 3 (roof)
816
817 Photoperiod, integer 8-18 hours
818
819 Three pre-plot stations create a longer route.\allowbreak{} The number of hours is descriptive, not elapsed
820 ↳ game time or a measured spectrum.\allowbreak{}
821
822 \subsection{Future scientific-purpose implementation}
823 Mark leaf boundaries or estimate leaf area in calibrated images from controlled light treatments.\allowbreak{}
824 ↳ Record calibration, illumination condition, image timestamp, and segmentation.\allowbreak{}
825
826 Require scale calibration and repeated annotation.\allowbreak{} Do not infer spectral response from the avatar
827 ↳ visiting an instrument.\allowbreak{}
828
829 \subsection{Engineering change and acceptance}
830 Verify all four route stages and the nominal 105-point clean escort.\allowbreak{} Destroy the middle instrument
831 ↳ after seed collection; the carrier must retain its seedling and resume without collecting a second
832 ↳ one.\allowbreak{}
833
834 Implementation touchpoints: GWAPScenarios defines
835 ↳ title.\allowbreak{}purpose.\allowbreak{}route.\allowbreak{}range; GWAPWorld and GWAPRouting provide named
836 ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
837 ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
838 ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
839 ↳ change meaning when the list is reordered.\allowbreak{}
840
841 \section{04.\allowbreak{} Soil bioremediation}
842 \subsection{Implemented route and parameter}
843 Chemical seedlings (B1) -> Root analyzer (L2) -> Plot 4 (roof)
844
845 Soil batch, integer 1-12
846
847 The workflow represents sample tracking.\allowbreak{} It does not calculate contaminant concentration, uptake,
848 ↳ removal efficiency, or environmental safety.\allowbreak{}
849
850 \subsection{Future scientific-purpose implementation}
851 Trace roots in real scan images or match samples to verified metadata.\allowbreak{} Record source sample,
852 ↳ imaging method, root mask, and uncertain regions.\allowbreak{}
853
854 Use expert review for ambiguous roots and preserve units/\allowbreak{}calibration.\allowbreak{} A gameplay
855 ↳ completion cannot certify remediation success.\allowbreak{}
856
857 \subsection{Engineering change and acceptance}
858 Verify replacement Root analyzer selection, plot capacity, and no stage advancement while the target is
859 ↳ destroyed.\allowbreak{} Confirm batch metadata remains associated with the assignment.\allowbreak{}

```

```
836
837 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
838
839 \section{05.\allowbreak{} Hydroponic versus soil}
840 \subsection{Implemented route and parameter}
841 Split-root tray (L1) -> Centrifuge (L3) -> Root analyzer (L2) -> Plot 5 (roof)
842
843 Nutrient batch, integer 1-8
844
845 The route deliberately goes up and then down before the roof.\allowbreak{} No paired root-growth measurements
    ↳ currently exist.\allowbreak{}
846
847 \subsection{Future scientific-purpose implementation}
848 Compare paired, anonymized root images from controlled growth conditions.\allowbreak{} Store pair ID, condition
    ↳ metadata held separately, root-length annotation, and replicate.\allowbreak{}
849
850 Randomize presentation order and require comparable imaging scales.\allowbreak{} Analyze pairs rather than
    ↳ treating unrelated plants as matched samples.\allowbreak{}
851
852 \subsection{Engineering change and acceptance}
853 Exercise both ascent and descent, a destroyed Centrifuge, and a destroyed Root analyzer.\allowbreak{} Check
    ↳ that the route order is preserved and the two instruments are not treated as interchangeable.\allowbreak{}
854
855 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
856
857 \section{06.\allowbreak{} UV adaptation}
858 \subsection{Implemented route and parameter}
859 UV chamber (L2) -> Vitality scanner (L1) -> Plot 6 (roof)
860
861 Exposure, integer 5-20 seconds
862
863 The assignment text describes exposure; processing is still the common four-second gameplay action and does not
    ↳ implement the chosen duration.\allowbreak{}
864
865 \subsection{Future scientific-purpose implementation}
866 Annotate visible stress features in a curated before/\allowbreak{}after plant image pair.\allowbreak{} Record
    ↳ acquisition conditions, treatment metadata, severity features, and uncertainty.\allowbreak{}
867
868 Use researcher-approved image data and a fixed rubric.\allowbreak{} This game is not an instruction for
    ↳ operating UV equipment.\allowbreak{}
869
870 \subsection{Engineering change and acceptance}
871 Check descending travel after the UV stage, stable variant display, and shared use of Vitality scanner
    ↳ capability with scenarios 10 and 6 without crossing assignment ownership.\allowbreak{}
872
873 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
874
875 \section{07.\allowbreak{} Gravitropism assay}
876 \subsection{Implemented route and parameter}
877 Clinostat (B1) -> Orientation scanner (L1) -> Plot 7 (roof)
878
879 Tray tilt, integer 10-80 degrees
880
881 The scenario names an orientation experiment but does not currently simulate root curvature from gravity or
    ↳ time.\allowbreak{}
882
883 \subsection{Future scientific-purpose implementation}
884 Mark root tips and growth angles in time-lapse frames with a known reference axis.\allowbreak{} Store frame
    ↳ time, axis calibration, landmarks, and annotation quality.\allowbreak{}
885
```

```

886 Require consistent angle conventions and reject occluded frames.\allowbreak{} Separate tray orientation from
    ↳ measured root direction.\allowbreak{}
887
888 \subsection{Engineering change and acceptance}
889 Verify Clinostat backups, floor-label conversion for basement, and a clear initial rendezvous.\allowbreak{}
    ↳ Test the zero-length initial-path case that previously required an explicit bypass.\allowbreak{}
890
891 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
892
893 \section{08.\allowbreak{} Drought resistance}
894 \subsection{Implemented route and parameter}
895 Climate chamber (L3) -> Porometer (L2) -> Plot 8 (roof)
896
897 Humidity, integer 20-45 percent
898
899 The mission says simulated stomatal response, but it does not compute conductance or water
    ↳ potential.\allowbreak{} Humidity is assignment metadata.\allowbreak{}
900
901 \subsection{Future scientific-purpose implementation}
902 Annotate stomatal aperture from suitable microscopy images or organize verified instrument
    ↳ observations.\allowbreak{} Store image/\allowbreak{}instrument record ID, scale, treatment, aperture
    ↳ annotation, and replicate.\allowbreak{}
903
904 Require a defined measurement method and expert validation.\allowbreak{} Do not combine incompatible
    ↳ instruments or units under one generic score.\allowbreak{}
905
906 \subsection{Engineering change and acceptance}
907 Destroy the Porometer while the carrier is processing; verify that processing restarts at an equivalent rather
    ↳ than completing on the destroyed station.\allowbreak{}
908
909 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
910
911 \section{09.\allowbreak{} Microbiome symbiosis}
912 \subsection{Implemented route and parameter}
913 Wet lab (B1) -> Symbiosis incubator (L2) -> Plot 9 (roof)
914
915 Inoculum batch, integer 1-8
916
917 The batch labels a transport protocol.\allowbreak{} No microbial growth model, sequencing data, or biological
    ↳ assay is present.\allowbreak{}
918
919 \subsection{Future scientific-purpose implementation}
920 Label root colonization regions in an approved microscopy dataset.\allowbreak{} Store image ID, region mask,
    ↳ labeling taxonomy, batch, and replicate.\allowbreak{}
921
922 Use blinded examples, inter-annotator agreement, and expert adjudication.\allowbreak{} Treat uncertain regions
    ↳ as uncertain rather than forcing a positive classification.\allowbreak{}
923
924 \subsection{Engineering change and acceptance}
925 Verify the carrier spawns near the selected role, explains the assignment, waits for Ready, and retains the
    ↳ inoculum batch through same-floor and other-floor reroutes.\allowbreak{}
926
927 Implementation touchpoints: GWAPScenarios defines
    ↳ title/\allowbreak{}purpose/\allowbreak{}route/\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
928
929 \section{10.\allowbreak{} Cold-hardiness screening}
930 \subsection{Implemented route and parameter}
931 Cryo-freezer (B1) -> Vitality scanner (L1) -> Plot 10 (roof)
932
933 Recovery temperature, integer 5-20 C
934

```

```

935 The workflow models recovery transport.\allowbreak{} It does not simulate freezing injury or measure actual
    ↳ survival.\allowbreak{}
936
937 \subsection{Future scientific-purpose implementation}
938 Score recovery features in a time series with known treatment conditions.\allowbreak{} Store plant ID, time
    ↳ since recovery, visible features, viability rubric, and observer confidence.\allowbreak{}
939
940 Require consistent follow-up timing and a definition of survival.\allowbreak{} Separate missing observations
    ↳ from dead plants.\allowbreak{}
941
942 \subsection{Engineering change and acceptance}
943 Check cryo-freezer explosion handling, a full Plot 10 falling back to another healthy greenhouse plot, and no
    ↳ reward if the assigned carrier is intercepted before final planting.\allowbreak{}
944
945 Implementation touchpoints: GWAPScenarios defines
    ↳ title.\allowbreak{}purpose.\allowbreak{}route.\allowbreak{}range; GWAPWorld and GWAPRouting provide named
    ↳ capabilities and approaches; GWAPServer executes the stages and awards; LabHUD presents the
    ↳ assignment.\allowbreak{} A real data layer must add structured records independently of this
    ↳ route.\allowbreak{} Keep a stable scenario ID and protocol version so saved or exported observations do not
    ↳ change meaning when the list is reordered.\allowbreak{}
946
947
948
949 GENERATED REFERENCE: symbol-index.tex
950 \chapter{Active-code symbol and attribute index}
951 This index is generated from source text to help locate implementation.\allowbreak{} Line numbers refer to the
    ↳ complete listings and raw files.\allowbreak{} Regex extraction may omit anonymous functions, multiline
    ↳ definitions, dynamic attribute names, or accesses assembled at runtime.\allowbreak{} It is a navigation
    ↳ aid, not an exhaustive static type analysis.\allowbreak{}
952
953 \section{S01 GWAPNPC}
954 \subsection{Named functions /\allowbreak{} assigned handlers}
955 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
956 \toprule
957 \textbf{Line} & \textbf{Symbol} \\ \midrule
958 \endfirsthead
959 \textbf{Line} & \textbf{Symbol} \\ \midrule
960 \endhead
961 18 & N.\allowbreak{}gun \\
962 28 & N.\allowbreak{}appearance \\
963 33 & piece \\
964 41 & N.\allowbreak{}say \\
965 49 & N.\allowbreak{}nearPlayer \\
966 66 & N.\allowbreak{}animate \\
967 \bottomrule
968 \end{longtable}
969 \subsection{Literal attribute keys and access lines}
970 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
971 \toprule
972 \textbf{Attribute} & \textbf{Lines} \\ \midrule
973 \endfirsthead
974 \textbf{Attribute} & \textbf{Lines} \\ \midrule
975 \endhead
976 Gender & 29 \\
977 HasSeedling & 89 \\
978 InventoryName & 25 \\
979 MissionMessage & 45 \\
980 MissionMessageAt & 46 \\
981 NPCGun & 25 \\
982 \bottomrule
983 \end{longtable}
984 \section{S02 GWAPRouting}
985 \subsection{Named functions /\allowbreak{} assigned handlers}
986 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
987 \toprule
988 \textbf{Line} & \textbf{Symbol} \\ \midrule
989 \endfirsthead
990 \textbf{Line} & \textbf{Symbol} \\ \midrule
991 \endhead
992 13 & floorOf \\
993 18 & R.\allowbreak{}healthy \\
994 21 & R.\allowbreak{}ensureBackups \\
995 47 & R.\allowbreak{}choose \\
996 \bottomrule
997 \end{longtable}
998 \subsection{Literal attribute keys and access lines}

```

```

999 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1000 \toprule
1001 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1002 \endfirsthead
1003 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1004 \endhead
1005 Backup & 25, 31 \\
1006 BlastRemoved & 19 \\
1007 Capability & 27, 31, 49 \\
1008 Exploded & 19, 38 \\
1009 Floor & 14, 37 \\
1010 FunctionalState & 19, 38 \\
1011 Level & 15 \\
1012 Planted & 26, 49 \\
1013 \bottomrule
1014 \end{longtable}
1015 \section{S03 GWAPScenarios}
1016 \section{S04 GWAPWorld}
1017 \subsection{Named functions /\allowbreak{} assigned handlers}
1018 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1019 \toprule
1020 \textbf{Line} & \textbf{Symbol} \\ \midrule
1021 \endfirsthead
1022 \textbf{Line} & \textbf{Symbol} \\ \midrule
1023 \endhead
1024 14 & part \\
1025 19 & label \\
1026 23 & W.\allowbreak{}plant \\
1027 31 & W.\allowbreak{}build \\
1028 \bottomrule
1029 \end{longtable}
1030 \subsection{Literal attribute keys and access lines}
1031 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1032 \toprule
1033 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1034 \endfirsthead
1035 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1036 \endhead
1037 Approach & 46, 85 \\
1038 Floor & 46, 85 \\
1039 GWAPOriginalCanCollide & 60, 65 \\
1040 Planted & 85 \\
1041 \bottomrule
1042 \end{longtable}
1043 \section{S05 LabKinds}
1044 \subsection{Named functions /\allowbreak{} assigned handlers}
1045 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1046 \toprule
1047 \textbf{Line} & \textbf{Symbol} \\ \midrule
1048 \endfirsthead
1049 \textbf{Line} & \textbf{Symbol} \\ \midrule
1050 \endhead
1051 28 & f \\
1052 29 & rnd \\
1053 33 & log \\
1054 35 & log \\
1055 37 & log \\
1056 39 & log \\
1057 42 & log \\
1058 44 & log \\
1059 48 & log \\
1060 50 & log \\
1061 53 & log \\
1062 56 & log \\
1063 59 & log \\
1064 62 & log \\
1065 64 & log \\
1066 67 & log \\
1067 72 & log \\
1068 74 & log \\
1069 76 & log \\
1070 78 & log \\
1071 80 & log \\
1072 83 & log \\
1073 85 & log \\
1074 88 & log

```

```
1075 90 & log \\
1076 92 & log \\
1077 94 & log \\
1078 96 & log \\
1079 98 & log \\
1080 100 & log \\
1081 102 & log \\
1082 104 & log \\
1083 106 & log \\
1084 108 & log \\
1085 110 & log \\
1086 112 & log \\
1087 116 & log \\
1088 119 & log \\
1089 122 & log \\
1090 124 & log \\
1091 126 & log \\
1092 128 & log \\
1093 130 & log \\
1094 132 & log \\
1095 134 & log \\
1096 136 & log \\
1097 138 & log \\
1098 140 & log \\
1099 149 & log \\
1100 151 & log \\
1101 161 & log \\
1102 163 & log \\
1103 165 & log \\
1104 169 & log \\
1105 171 & K.\allowbreak{ }forKind \\
1106 \bottomrule
1107 \end{longtable}
1108 \section{S06 Blaster}
1109 \subsection{Named functions /\allowbreak{ } assigned handlers}
1110 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1111 \toprule
1112 \textbf{Line} & \textbf{Symbol} \\ \midrule
1113 \endfirsthead
1114 \textbf{Line} & \textbf{Symbol} \\ \midrule
1115 \endhead
1116 27 & finiteNumber \\
1117 34 & Blaster.\allowbreak{ }IsFiniteVector \\
1118 41 & numberAttribute \\
1119 47 & Blaster.\allowbreak{ }GetWeaponConfig \\
1120 62 & Blaster.\allowbreak{ }GetCameraRay \\
1121 83 & Blaster.\allowbreak{ }NewRaycastParams \\
1122 91 & Blaster.\allowbreak{ }CastFromCamera \\
1123 107 & Blaster.\allowbreak{ }GetAimSolution \\
1124 137 & tracerColor \\
1125 142 & Blaster.\allowbreak{ }DrawTracer \\
1126 163 & Blaster.\allowbreak{ }DrawImpact \\
1127 182 & getRemotes \\
1128 190 & Blaster.\allowbreak{ }StartReplication \\
1129 206 & Blaster.\allowbreak{ }Fire \\
1130 \bottomrule
1131 \end{longtable}
1132 \subsection{Literal attribute keys and access lines}
1133 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1134 \toprule
1135 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1136 \endfirsthead
1137 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1138 \endhead
1139 BlasterAllowPlayerDamage & 58 \\
1140 BlasterAutomatic & 56 \\
1141 BlasterInstantHeadshot & 57 \\
1142 BlasterTracerColor & 138 \\
1143 \bottomrule
1144 \end{longtable}
1145 \section{S07 BlasterController}
1146 \subsection{Named functions /\allowbreak{ } assigned handlers}
1147 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1148 \toprule
1149 \textbf{Line} & \textbf{Symbol} \\ \midrule
1150 \endfirsthead
```

```

1151 \textbf{Line} & \textbf{Symbol} \\ \midrule
1152 \endhead
1153 33 & touchControlsActive \\
1154 59 & pointInside \\
1155 68 & touchAimActive \\
1156 82 & rotationBetween \\
1157 93 & makeCrosshair \\
1158 165 & BlasterController:\allowbreak{}ensureHUD \\
1159 176 & BlasterController:\allowbreak{}new \\
1160 232 & BlasterController:\allowbreak{}button \\
1161 239 & BlasterController:\allowbreak{}fallbackTouchButton \\
1162 281 & BlasterController:\allowbreak{}styleTouchButtons \\
1163 337 & place \\
1164 351 & BlasterController:\allowbreak{}getAimScreenPoint \\
1165 372 & BlasterController:\allowbreak{}updateAimReticle \\
1166 414 & BlasterController:\allowbreak{}setLocalHeadHidden \\
1167 434 & BlasterController:\allowbreak{}beginTouchCamera \\
1168 451 & BlasterController:\allowbreak{}endTouchCamera \\
1169 463 & BlasterController:SetAiming \\
1170 505 & BlasterController:\allowbreak{}fireOnce \\
1171 520 & BlasterController:\allowbreak{}beginFiring \\
1172 536 & BlasterController:\allowbreak{}shootAction \\
1173 561 & BlasterController:\allowbreak{}aimAction \\
1174 576 & BlasterController:\allowbreak{}touchPanAction \\
1175 623 & BlasterController:\allowbreak{}updateArmPose \\
1176 685 & BlasterController:\allowbreak{}render \\
1177 737 & BlasterController:Enable \\
1178 821 & BlasterController:Disable \\
1179 865 & BlasterController:Destroy \\
1180 875 & BlasterController:\allowbreak{}BindTool \\
1181 883 & BlasterController:\allowbreak{}GetController \\
1182 887 & BlasterController:\allowbreak{}AutoBind \\
1183 890 & consider \\
1184 896 & watch \\
1185 907 & binder:Destroy \\
1186 \bottomrule
1187 \end{longtable}
1188 \subsection{Literal attribute keys and access lines}
1189 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1190 \toprule
1191 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1192 \endfirsthead
1193 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1194 \endhead
1195 BlasterADSTouchSensitivity & 607 \\
1196 BlasterBarrelDirection & 661 \\
1197 BlasterHipFOV & 746 \\
1198 BlasterHipTouchSensitivity & 608 \\
1199 BlasterShoulderOffsetX & 758 \\
1200 BlasterShoulderOffsetY & 759 \\
1201 BlasterWeapon & 891 \\
1202 LabSidearm & 664, 891 \\
1203 \bottomrule
1204 \end{longtable}
1205 \section{S08 GWAPServer}
1206 \subsection{Named functions /\allowbreak{} assigned handlers}
1207 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1208 \toprule
1209 \textbf{Line} & \textbf{Symbol} \\ \midrule
1210 \endfirsthead
1211 \textbf{Line} & \textbf{Symbol} \\ \midrule
1212 \endhead
1213 20 & refreshArmed \\
1214 37 & alive \\
1215 40 & points \\
1216 43 & award \\
1217 47 & role \\
1218 48 & active \\
1219 49 & nearby \\
1220 50 & publish \\
1221 56 & tell \\
1222 60 & finish \\
1223 67 & updateHealth \\
1224 75 & damage \\
1225 89 & remember \\
1226 95 & repair

```



```
1227 102 & explode \\
1228 147 & bridge.\allowbreak{}0nInvoke \\
1229 152 & routePoints \\
1230 153 & add \\
1231 159 & flight \\
1232 172 & showRoute \\
1233 177 & go \\
1234 248 & plantIn \\
1235 258 & checkpoint \\
1236 263 & runMission \\
1237 312 & startMission \\
1238 321 & makeBot \\
1239 337 & companion \\
1240 349 & choose \\
1241 385 & setArmed \\
1242 406 & watchPlayer \\
1243 407 & clean \\
1244 408 & watch \\
1245 410 & onCharacter \\
1246 \bottomrule
1247 \end{longtable}
1248 \subsection{Literal attribute keys and access lines}
1249 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1250 \toprule
1251 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1252 \endfirsthead
1253 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1254 \endhead
1255 BlastRemoved & 123, 138 \\
1256 BotNumber & 399 \\
1257 Capability & 30, 268, 272 \\
1258 CarriedSample & 382 \\
1259 Dead & 331, 389, 429, 439 \\
1260 DisplayName & 58, 345 \\
1261 Exploded & 96, 103, 104, 123, 138, 142, 249 \\
1262 Floor & 156 \\
1263 FunctionalState & 96, 104 \\
1264 GWAPArmed & 390, 397 \\
1265 GWAPArmedCount & 402 \\
1266 GWAPAttacker & 109, 135 \\
1267 GWAPBotCount & 402 \\
1268 GWAPCarrier & 331, 345, 356 \\
1269 GWAPDialogue & 58, 344, 345 \\
1270 GWAPEquipment & 92 \\
1271 GWAPFractionalLoss & 453, 454, 464 \\
1272 GWAPHitAt & 455 \\
1273 GWAPNotice & 45, 78, 129, 351, 352, 355, 356, 381, 382, 455 \\
1274 GWAPScenario & 356 \\
1275 GWAPSpeaker & 58, 345 \\
1276 GWAPStage & 52 \\
1277 GWAPState & 53 \\
1278 GWAPTarget & 53 \\
1279 GWAPTargetFloor & 53 \\
1280 GWAPTotalStages & 52 \\
1281 GWAPVariant & 356 \\
1282 GWAPWeapon & 407 \\
1283 Gender & 324 \\
1284 HasSeedling & 62, 305 \\
1285 Health & 69, 77, 316, 389, 429, 430, 439 \\
1286 KnockedOut & 331, 389, 429, 439 \\
1287 LabRole & 47, 417 \\
1288 LastPointAmount & 45 \\
1289 LastPointAward & 45 \\
1290 PlantSample & 380 \\
1291 Planted & 250, 253, 466 \\
1292 Reroutes & 281 \\
1293 RoundNumber & 462 \\
1294 RoundPhase & 350, 436 \\
1295 RouteIndex & 202 \\
1296 Scenario & 316 \\
1297 Seated & 324 \\
1298 State & 48, 53, 62, 316, 336 \\
1299 TaskState & 62, 285, 294 \\
1300 Title & 71, 316 \\
1301 TransferSample & 380 \\
1302 Variant & 316
```

```

1303 WalkSpeed & 219 \\
1304 \bottomrule
1305 \end{longtable}
1306 \section{S09 GWAPWallDamage}
1307 \subsection{Named functions /\allowbreak{} assigned handlers}
1308 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1309 \toprule
1310 \textbf{Line} & \textbf{Symbol} \\ \midrule
1311 \endfirsthead
1312 \textbf{Line} & \textbf{Symbol} \\ \midrule
1313 \endhead
1314 16 & register \\
1315 \bottomrule
1316 \end{longtable}
1317 \subsection{Literal attribute keys and access lines}
1318 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1319 \toprule
1320 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1321 \endfirsthead
1322 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1323 \endhead
1324 BlastRemoved & 47 \\
1325 Exploded & 47 \\
1326 FunctionalState & 47 \\
1327 GWAPEquipment & 19 \\
1328 GWAPRemovedFixtures & 53 \\
1329 ShootablePoster & 18 \\
1330 ToolKind & 18 \\
1331 \bottomrule
1332 \end{longtable}
1333 \section{S10 LabElevators}
1334 \subsection{Named functions /\allowbreak{} assigned handlers}
1335 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1336 \toprule
1337 \textbf{Line} & \textbf{Symbol} \\ \midrule
1338 \endfirsthead
1339 \textbf{Line} & \textbf{Symbol} \\ \midrule
1340 \endhead
1341 21 & setGate \\
1342 26 & setup \\
1343 55 & occupants \\
1344 73 & moveTo \\
1345 109 & addPrompt \\
1346 \bottomrule
1347 \end{longtable}
1348 \subsection{Literal attribute keys and access lines}
1349 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1350 \toprule
1351 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1352 \endfirsthead
1353 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1354 \endhead
1355 CurrentFloor & 104 \\
1356 Floor & 41, 121, 124 \\
1357 FloorCount & 27 \\
1358 FloorLabel\_\allowbreak{} & 125 \\
1359 FloorY\_\allowbreak{} & 45 \\
1360 \bottomrule
1361 \end{longtable}
1362 \section{S11 LabEquipment}
1363 \subsection{Named functions /\allowbreak{} assigned handlers}
1364 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1365 \toprule
1366 \textbf{Line} & \textbf{Symbol} \\ \midrule
1367 \endfirsthead
1368 \textbf{Line} & \textbf{Symbol} \\ \midrule
1369 \endhead
1370 53 & updateScoreboard \\
1371 73 & refreshRoomMonitors \\
1372 105 & addContribution \\
1373 112 & notify \\
1374 167 & playerRole \\
1375 176 & awardPoints \\
1376 198 & canInteract \\
1377 227 & runMathQuiz \\
1378 268 & addWeldedPart

```

```
1379 290 & equipTool \\
1380 299 & makeTool \\
1381 406 & updateLifeBar \\
1382 427 & playerContainers \\
1383 435 & findTool \\
1384 449 & findTransferSample \\
1385 460 & findHeldSample \\
1386 478 & findPlantSample \\
1387 487 & treatmentText \\
1388 493 & hasTreatment \\
1389 498 & addTreatment \\
1390 505 & refreshPlantVisual \\
1391 530 & setCarryState \\
1392 550 & destroyCarriedItem \\
1393 605 & refreshBotHealth \\
1394 625 & damageWorkerBot \\
1395 714 & equipmentFromHit \\
1396 728 & explodeEquipment \\
1397 781 & detonateGasPipe \\
1398 880 & spawnImpact \\
1399 921 & punchPoster \\
1400 945 & workerBotFromHit \\
1401 950 & stealHeldSample \\
1402 974 & armImpactTool \\
1403 1088 & findImpactTool \\
1404 1099 & makeImpactTool \\
1405 1193 & bindDispenser \\
1406 1342 & blasterBridge.allowbreak{}OnInvoke \\
1407 1399 & equipmentFloorLabel \\
1408 1403 & equipmentId \\
1409 1409 & equipmentLabel \\
1410 1415 & adaptiveTransferSeconds \\
1411 1477 & storageId \\
1412 1486 & storageLabel \\
1413 1492 & coldStoresExcept \\
1414 1506 & makeControls \\
1415 1604 & clockText \\
1416 1608 & renderTimer \\
1417 1709 & setReadout \\
1418 1715 & pressButton \\
1419 1731 & runProgress \\
1420 1743 & chamberSampleList \\
1421 1749 & saveChamberSamples \\
1422 1758 & formatRuntime \\
1423 1763 & recoverPlantLife \\
1424 1793 & updateStoredSamples \\
1425 1806 & refreshChamberTelemetry \\
1426 1843 & evacuationDestinations \\
1427 1864 & repairPathFolder \\
1428 1883 & clearRepairPath \\
1429 1888 & repairReward \\
1430 1895 & createRepairPath \\
1431 1902 & approachPoint \\
1432 1910 & appendNavigable \\
1433 2007 & clearAssignment \\
1434 2014 & assignNextTask \\
1435 2091 & chamberByCode \\
1436 2097 & takePlantFromChamber \\
1437 2138 & canFinishTaking \\
1438 2211 & storePlantInChamber \\
1439 2286 & handleGrowthChamber \\
1440 2326 & germinationPercent \\
1441 2332 & processPlant \\
1442 2452 & createTransferSample \\
1443 2502 & handleColdStore \\
1444 2527 & startCycleIndicator \\
1445 2567 & bind \\
1446 2659 & canFinishRepair \\
1447 2810 & setBotStatus \\
1448 2818 & updateBotTestBoard \\
1449 2846 & interactionPoint \\
1450 2892 & roomAt \\
1451 2907 & roomRect \\
1452 2914 & botHasObserver \\
1453 2935 & botMotionChecks \\
1454 2951 & clear \\
```

```
1455 2979 & grounded \\
1456 2989 & walkBotSegment \\
1457 3038 & moveBotTo \\
1458 3059 & makeBotPlant \\
1459 3066 & piece \\
1460 3087 & equipmentReadout \\
1461 3095 & candidatesOnLevel \\
1462 3105 & runBotScenario \\
1463 3264 & setChamberFailure \\
1464 3315 & setGenericFailure \\
1465 3348 & shuffled \\
1466 3356 & adaptiveFailureTick \\
1467 3441 & genericFailureTick \\
1468 3465 & bindPlayerTaskRouting \\
1469 3496 & bindAutoDoor \\
1470 3507 & setIndicatorColour \\
1471 3516 & setOpen \\
1472 3574 & bindEmergencyKit \\
1473 3591 & setCharged \\
1474 3620 & attachPlayerHealthBar \\
1475 3658 & refresh \\
1476 3671 & watchPlayerHealth \\
1477 3683 & bindSecretBasementPanel \\
1478 3720 & nearestEmergencyKit \\
1479 3802 & scatterDebris \\
1480 3827 & warnEquipmentCatastrophe \\
1481 3862 & destroyEquipment \\
1482 4010 & resetHunt \\
1483 4046 & clearRoomSmoke \\
1484 4056 & fillRoomWithSmoke \\
1485 4087 & ventFromDoor \\
1486 4101 & watchDoor \\
1487 4152 & bindNPC \\
1488 \bottomrule
1489 \end{longtable}
1490 \subsection{Literal attribute keys and access lines}
1491 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1492 \toprule
1493 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1494 \endfirsthead
1495 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1496 \endhead
1497 AboutToBlow & 3828, 3829, 3868 \\
1498 ActiveNPCWorkers & 3434 \\
1499 ActiveNPCWorkersPerFloor & 3437 \\
1500 ActivePlayerCount & 3433 \\
1501 AdaptiveFailureTargetPercent & 3428 \\
1502 AssignedRepairDistance & 1890, 2039 \\
1503 AssignedRepairId & 2037, 2671, 3285, 3332, 3474, 3741 \\
1504 AssignedRepairProjectedPoints & 2041 \\
1505 AssignedTransferDeadline & 2074, 2197, 2198, 2235 \\
1506 AssignedTransferDestination & 2073, 2110, 2196, 2234 \\
1507 AssignedTransferSource & 2072, 2109, 2195, 2233, 3286, 3741 \\
1508 Auto & 2528 \\
1509 AvoidanceDetours & 3010 \\
1510 BlasterADSF0V & 1130 \\
1511 BlasterADSTouchSensitivity & 1131 \\
1512 BlasterAutomatic & 1129 \\
1513 BlasterCooldown & 1124 \\
1514 BlasterDamage & 1122 \\
1515 BlasterInstantHeadshot & 1128 \\
1516 BlasterPellets & 1125 \\
1517 BlasterRange & 1123 \\
1518 BlasterSelfCostPercent & 1127 \\
1519 BlasterSpread & 1126 \\
1520 BlasterTracerColor & 1132 \\
1521 BlasterWeapon & 1121 \\
1522 Blown & 782, 783 \\
1523 BotNumber & 3005, 3203, 3991 \\
1524 BotTestUses & 3207 \\
1525 BotsKilled & 707, 1324, 1363, 4015 \\
1526 BotsTotal & 1325, 1364 \\
1527 BreakableWindow & 1347 \\
1528 Capacity & 1511, 1635, 1750, 1816, 1847, 2115, 2145, 2206, 2224, 2570, 3215, 3328, 3343, 3362, 3412 \\
1529 CarriedSample & 538, 545, 551, 2183 \\
1530 CarryingPlant & 2824, 3195, 3242
```

1531 ChamberStatusLamp & 1834 \\
1532 Charged & 3592, 3599 \\
1533 Competitor & 61, 2825, 3022, 3120, 3174, 3223 \\
1534 CompletedCycles & 2822, 3222 \\
1535 Dead & 630, 648, 760, 862, 1297, 1301, 1358, 1360, 2990, 3021, 3022, 3023, 3043, 3113, 3117, 4019, 4024 \\
1536 DestinationGrowthCode & 524, 537, 2180, 2229 \\
1537 DestinationLabel & 537, 2455, 2512 \\
1538 DestinationStorageId & 2510 \\
1539 DispenserBound & 1194, 1197 \\
1540 DisplayName & 2814 \\
1541 DoorBound & 3497, 3498 \\
1542 DoorClosedX & 3499 \\
1543 DoorClosedZ & 3500 \\
1544 DoorOpen & 3529, 4102, 4103 \\
1545 DoorOpenX & 3501 \\
1546 DoorOpenZ & 3502 \\
1547 DoorY & 3503 \\
1548 EmergencyMovesCompleted & 3431 \\
1549 EmergencyTransfers & 2253 \\
1550 EquipmentId & 2572 \\
1551 EquipmentTaskRoutingBound & 3466, 3467 \\
1552 EvacuationDestination & 1812, 3183, 3283, 3298, 3409 \\
1553 Exploded & 729, 730, 833 \\
1554 FailedSampleBacklog & 1419, 3430 \\
1555 FailingChambers & 3429 \\
1556 FailureReason & 1813, 2203, 2580, 2657, 2691, 2740, 3277, 3296, 3324, 3328, 3338 \\
1557 FailureSince & 1612, 1889, 2029, 2135, 2581, 3278, 3297, 3325, 3339, 3395, 3460, 3952 \\
1558 FloorLabel & 1400, 1699, 1809, 2573, 3271 \\
1559 FloorMaxLevel & 1932 \\
1560 FloorMinLevel & 1931 \\
1561 FloorRiseStuds & 1926 \\
1562 FunctionalState & 86, 731, 1609, 1765, 1783, 1807, 1849, 2020, 2050, 2106, 2113, 2141, 2143, 2218, 2575, 2579,
↪ 2636, 2639, 2641, 2651, 2661, 2667, 2690, 2739, 2830, 3128, 3215, 3265, 3273, 3294, 3316, 3322, 3337, 3364,
↪ 3367, 3404, 3410, 3445, 3447, 3867, 3946 \\
1563 GasPipe & 847, 1302, 1369 \\
1564 Genome & 2158, 2244, 2376, 2427, 2436 \\
1565 GermAbnormalCount & 2411 \\
1566 GermDeadCount & 2414 \\
1567 GermNormalCount & 2329, 2409 \\
1568 GermSownCount & 2327, 2407 \\
1569 GerminationDate & 1817, 2160, 3420 \\
1570 GerminationOutcome & 2406, 2437, 2438 \\
1571 GrowthCode & 1404, 1410, 1699, 1776, 1808, 2058, 2059, 2068, 2069, 2093, 2098, 2132, 2212, 2287, 3157, 3158,
↪ 3183, 3185, 3197, 3212, 3269, 3270, 3283, 3409, 3418 \\
1572 GrowthStage & 506, 1795, 2131, 2241, 2242, 2345, 2425, 3193, 3218 \\
1573 Health & 606, 631, 632, 4022 \\
1574 HitLogicBound & 975, 976 \\
1575 Holes & 922, 924 \\
1576 HomeCFrame & 702, 3107, 3108, 4020 \\
1577 HomeGrowthCode & 2232 \\
1578 HormoneFocus & 2383, 2444 \\
1579 HormoneTreatment & 2382, 2443, 2444 \\
1580 Humidity & 1819, 3280, 3300, 3406 \\
1581 ImpactToolKind & 979, 1092, 1118, 1247 \\
1582 IncomingReservations & 1647, 1848, 1856, 1857, 2114, 2144, 2182, 2193, 2250, 3182, 3244, 3267, 3318, 3371, 3387
↪ \\
1583 InteractionsBound & 2569, 2571 \\
1584 Kind & 1412, 1488, 1495, 2341, 2473, 2568, 3127, 3201, 3204, 3444, 3893, 3948, 3957 \\
1585 KitBound & 3575, 3576 \\
1586 KnockedOut & 649, 3021, 3124, 4023 \\
1587 LabImpactTool & 1119 \\
1588 LabItemType & 309, 440 \\
1589 LabObjective & 540, 542, 546, 1775, 2034, 2045, 2065, 2083, 2087, 2184, 2200, 2205, 2430 \\
1590 LabRole & 168, 3468 \\
1591 LabSample & 310, 464, 2794 \\
1592 LabSidearm & 1134, 1246 \\
1593 LastActionDetail & 1711 \\
1594 LastActionStatus & 1710 \\
1595 LastHitBy & 633 \\
1596 LastPointAmount & 191 \\
1597 LastPointAward & 192 \\
1598 LastRepairedAt & 2673, 3368, 3386, 3448 \\
1599 LastRepairedBy & 2674 \\
1600 Level & 80, 96, 1400, 1406, 1480, 1489, 1699, 1809, 1851, 2917, 3098, 3125, 3271 \\
1601 LevelSlot & 4001 \\
1602 MathQuizActive & 236, 247 \\

```

1603 MaxHealth & 606, 607, 4022 \\
1604 MessageSerial & 158, 159, 163 \\
1605 Moisture & 2158, 2245, 2426, 2436 \\
1606 MutagenesisDoseGy & 2365, 2440, 2441 \\
1607 MutagenesisSurvivalPercent & 2366, 2398, 2441 \\
1608 NPCName & 4158 \\
1609 NavigationBlocked & 3015, 3018, 3029 \\
1610 Occupied & 1627, 1754 \\
1611 OperationStarted & 1613, 1818, 2583, 3302, 3341 \\
1612 PathFallbacks & 2823, 3053 \\
1613 PlantExpiredAt & 1629 \\
1614 PlantId & 525, 1628, 1755, 2230, 2348, 2368, 2415, 2430, 2435, 3304 \\
1615 PlantLife & 313, 407, 415, 507, 1623, 1658, 1766, 1773, 1784, 1786, 2133, 2150, 2171, 2176, 2177, 2231, 2412,
    ↳ 2728, 3274, 3303, 3304 \\
1616 PlantSample & 481 \\
1617 Powered & 2597 \\
1618 QuizPlayerId & 237, 245, 3266, 3317, 3953, 3970 \\
1619 QuizProtected & 2174 \\
1620 RecentTransferThroughput & 3432 \\
1621 RecoveryEndsAt & 1772, 1787 \\
1622 RecoveryToken & 1770, 1771, 1782 \\
1623 RepairInstruction & 732, 3276, 3295, 3326, 3340, 3869 \\
1624 RepairKey & 2574, 3275, 3323 \\
1625 RepairRouteReady & 1970, 1973 \\
1626 ReservedByBot & 2052, 2288, 2292, 3130, 3181, 3189, 3245, 3268, 3388 \\
1627 Role & 4158 \\
1628 Room & 79, 95, 1406, 1412, 1480, 1489, 3170 \\
1629 RoomIndex & 2875 \\
1630 RoomKey & 4061 \\
1631 RoundNumber & 4036 \\
1632 SampleCount & 88, 1624, 1626, 1636, 1661, 1753, 1794, 1846, 1856, 1857, 2051, 2114, 2144, 2296, 2832, 3129,
    ↳ 3303, 3328, 3343, 3361, 3394 \\
1633 SampleId & 536, 551, 963, 2456, 2458, 2509 \\
1634 SampleIds & 1625, 1745, 1752 \\
1635 SampleSlot & 1631, 1797, 1798 \\
1636 Score & 61, 2825, 3225 \\
1637 Seated & 3111 \\
1638 SecretBound & 3684, 3685 \\
1639 SecretDestX & 3695 \\
1640 SecretDestY & 3696 \\
1641 SecretDestZ & 3697 \\
1642 ShootablePoster & 1309, 1382 \\
1643 SourceGrowthCode & 2232 \\
1644 StairEastLaneX & 1928 \\
1645 StairNorthZ & 1929 \\
1646 StairSouthZ & 1930 \\
1647 StairWestLaneX & 1927 \\
1648 StorageId & 1478, 1481 \\
1649 StoredTransparency & 1799 \\
1650 TalkBound & 4153, 4154 \\
1651 TaskState & 650, 2811, 4025 \\
1652 Temperature & 1819, 3279, 3299, 3405 \\
1653 Tips & 4155 \\
1654 ToolKind & 1195 \\
1655 TransferSample & 453, 534, 956 \\
1656 TransferStartedAt & 2175 \\
1657 Treatments & 488, 494, 500, 501, 2157, 2243 \\
1658 Ventilation & 1820, 3281, 3301, 3407 \\
1659 WallSide & 1507, 1903, 2614, 2847 \\
1660 WindowBroken & 1348, 1349 \\
1661 \bottomrule
1662 \end{longtable}
1663 \section{S12 LabRoles}
1664 \subsection{Named functions /\allowbreak{} assigned handlers}
1665 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1666 \toprule
1667 \textbf{Line} & \textbf{Symbol} \\ \midrule
1668 \endfirsthead
1669 \textbf{Line} & \textbf{Symbol} \\ \midrule
1670 \endhead
1671 30 & roleEquipmentId \\
1672 36 & roleFloor \\
1673 40 & roleTransferSeconds \\
1674 49 & roleEquipmentLabel \\
1675 55 & roleDestinations \\
1676 72 & clearRoleAssignment

```

```

1677 78 & assignRoleObjective \\
1678 130 & buildHud \\
1679 216 & refreshHud \\
1680 243 & wearablePart \\
1681 265 & applyAppearance \\
1682 309 & setupPlayer \\
1683 344 & stripAutoHealthRegen \\
1684 369 & bindKiosk \\
1685 401 & selectRole \\
1686 \bottomrule
1687 \end{longtable}
1688 \subsection{Literal attribute keys and access lines}
1689 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1690 \toprule
1691 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1692 \endfirsthead
1693 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1694 \endhead
1695 AssignedRepairDistance & 96 \\
1696 AssignedRepairId & 92 \\
1697 AssignedRepairProjectedPoints & 97 \\
1698 AssignedTransferDeadline & 122, 226, 360 \\
1699 AssignedTransferDestination & 120 \\
1700 AssignedTransferSource & 119 \\
1701 Capacity & 61 \\
1702 CarriedSample & 222, 312, 336 \\
1703 FailedSampleBacklog & 44 \\
1704 FailureSince & 88, 94 \\
1705 FloorLabel & 37 \\
1706 FunctionalState & 60, 83, 104 \\
1707 GrowthCode & 31, 50, 111, 112, 117, 118 \\
1708 IncomingReservations & 59 \\
1709 Kind & 52 \\
1710 LabObjective & 223, 313, 337, 416 \\
1711 LabRole & 411 \\
1712 LastPointAmount & 233 \\
1713 LastPointAward & 234, 338 \\
1714 Level & 33, 37, 64, 65 \\
1715 ReservedByBot & 105 \\
1716 Role & 279, 370 \\
1717 RoleKioskBound & 372, 373 \\
1718 Room & 33, 52 \\
1719 SampleCount & 58, 67, 104 \\
1720 \bottomrule
1721 \end{longtable}
1722 \section{S13 LabRound}
1723 \subsection{Named functions /allowbreak{} assigned handlers}
1724 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1725 \toprule
1726 \textbf{Line} & \textbf{Symbol} \\ \midrule
1727 \endfirsthead
1728 \textbf{Line} & \textbf{Symbol} \\ \midrule
1729 \endhead
1730 18 & computeRoundSeconds \\
1731 48 & pointsOf \\
1732 54 & resetScores \\
1733 62 & resolveRound \\
1734 \bottomrule
1735 \end{longtable}
1736 \subsection{Literal attribute keys and access lines}
1737 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1738 \toprule
1739 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1740 \endfirsthead
1741 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1742 \endhead
1743 LastRoundScore & 69 \\
1744 LastWinnerName & 71 \\
1745 LastWinnerScore & 72 \\
1746 PersonalBest & 67, 68, 93 \\
1747 RoundEndsAt & 82, 87 \\
1748 RoundLength & 46 \\
1749 RoundNumber & 80 \\
1750 RoundPhase & 81, 86 \\
1751 RoundStreakBest & 58 \\
1752 \bottomrule

```

```

1753 \end{longtable}
1754 \section{S14 LabTechnicians}
1755 \subsection{Named functions /\allowbreak{} assigned handlers}
1756 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1757 \toprule
1758 \textbf{Line} & \textbf{Symbol} \\ \midrule
1759 \endfirsthead
1760 \textbf{Line} & \textbf{Symbol} \\ \midrule
1761 \endhead
1762 46 & bubbleOf \\
1763 71 & say \\
1764 81 & playSound \\
1765 93 & labelOf \\
1766 97 & nearestFault \\
1767 111 & setStatus \\
1768 118 & reviveLater \\
1769 134 & runTechnician \\
1770 \bottomrule
1771 \end{longtable}
1772 \subsection{Literal attribute keys and access lines}
1773 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1774 \toprule
1775 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1776 \endfirsthead
1777 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1778 \endhead
1779 Fatigue & 125, 154, 187 \\
1780 FunctionalState & 100 \\
1781 Gender & 139 \\
1782 GrowthCode & 94 \\
1783 Kind & 94 \\
1784 KnockedOut & 128, 142 \\
1785 NeedsHelp & 127, 159, 179 \\
1786 Panic & 126, 158, 178 \\
1787 \bottomrule
1788 \end{longtable}
1789 \section{S15 LoungeSystems}
1790 \subsection{Named functions /\allowbreak{} assigned handlers}
1791 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1792 \toprule
1793 \textbf{Line} & \textbf{Symbol} \\ \midrule
1794 \endfirsthead
1795 \textbf{Line} & \textbf{Symbol} \\ \midrule
1796 \endhead
1797 28 & move \\
1798 53 & refreshPanel \\
1799 63 & playNext \\
1800 103 & refreshWall \\
1801 113 & onChat \\
1802 142 & hook \\
1803 \bottomrule
1804 \end{longtable}
1805 \subsection{Literal attribute keys and access lines}
1806 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1807 \toprule
1808 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1809 \endfirsthead
1810 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1811 \endhead
1812 Col & 160 \\
1813 InLounge & 33, 46, 114, 135 \\
1814 Row & 159 \\
1815 \bottomrule
1816 \end{longtable}
1817 \section{S16 ServerBlasterManager}
1818 \subsection{Named functions /\allowbreak{} assigned handlers}
1819 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1820 \toprule
1821 \textbf{Line} & \textbf{Symbol} \\ \midrule
1822 \endfirsthead
1823 \textbf{Line} & \textbf{Symbol} \\ \midrule
1824 \endhead
1825 30 & getRemote \\
1826 60 & findHumanoidFromHit \\
1827 72 & randomSpread \\
1828 82 & broadcastShot

```



```

1829 95 & researchHit \\
1830 102 & bridgeHit \\
1831 114 & genericDamage \\
1832 127 & validSequence \\
1833 136 & handleFire \\
1834 \bottomrule
1835 \end{longtable}
1836 \subsection{Literal attribute keys and access lines}
1837 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1838 \toprule
1839 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1840 \endfirsthead
1841 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1842 \endhead
1843 BlasterTracerColor & 163 \\
1844 BlasterWeapon & 147 \\
1845 LabSidearm & 147 \\
1846 \bottomrule
1847 \end{longtable}
1848 \section{S25 LabHUD}
1849 \subsection{Named functions /\allowbreak{} assigned handlers}
1850 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1851 \toprule
1852 \textbf{Line} & \textbf{Symbol} \\ \midrule
1853 \endfirsthead
1854 \textbf{Line} & \textbf{Symbol} \\ \midrule
1855 \endhead
1856 23 & box \\
1857 26 & label \\
1858 29 & button \\
1859 44 & toggle \\
1860 58 & showTrail \\
1861 80 & watchBot \\
1862 \bottomrule
1863 \end{longtable}
1864 \subsection{Literal attribute keys and access lines}
1865 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1866 \toprule
1867 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1868 \endfirsthead
1869 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1870 \endhead
1871 CarriedSample & 126 \\
1872 GWAPCarrier & 120 \\
1873 GWAPDialogue & 125 \\
1874 GWAPNotice & 126 \\
1875 GWAPScenario & 120 \\
1876 GWAPTarget & 123 \\
1877 GWAPTargetFloor & 122 \\
1878 Health & 124 \\
1879 LabRole & 115 \\
1880 RoundEndsAt & 117 \\
1881 RouteIndex & 67 \\
1882 State & 61 \\
1883 \bottomrule
1884 \end{longtable}
1885 \section{S26 LabQuizClient}
1886 \subsection{Named functions /\allowbreak{} assigned handlers}
1887 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1888 \toprule
1889 \textbf{Line} & \textbf{Symbol} \\ \midrule
1890 \endfirsthead
1891 \textbf{Line} & \textbf{Symbol} \\ \midrule
1892 \endhead
1893 51 & label \\
1894 84 & answer \\
1895 \bottomrule
1896 \end{longtable}
1897 \subsection{Literal attribute keys and access lines}
1898 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1899 \toprule
1900 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1901 \endfirsthead
1902 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1903 \endhead
1904 Answer & 103, 121 \\

```

```

1905 \bottomrule
1906 \end{longtable}
1907 \section{S27 LabSidearmClient}
1908 \subsection{Named functions /\allowbreak{} assigned handlers}
1909 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1910 \toprule
1911 \textbf{Line} & \textbf{Symbol} \\ \midrule
1912 \endfirsthead
1913 \textbf{Line} & \textbf{Symbol} \\ \midrule
1914 \endhead
1915 29 & refresh \\
1916 \bottomrule
1917 \end{longtable}
1918 \subsection{Literal attribute keys and access lines}
1919 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1920 \toprule
1921 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1922 \endfirsthead
1923 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1924 \endhead
1925 BotsKilled & 30, 40 \\
1926 BotsTotal & 31, 41 \\
1927 \bottomrule
1928 \end{longtable}
1929 \section{S28 LoungeClient}
1930 \subsection{Named functions /\allowbreak{} assigned handlers}
1931 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1932 \toprule
1933 \textbf{Line} & \textbf{Symbol} \\ \midrule
1934 \endfirsthead
1935 \textbf{Line} & \textbf{Symbol} \\ \midrule
1936 \endhead
1937 49 & apply \\
1938 \bottomrule
1939 \end{longtable}
1940 \subsection{Literal attribute keys and access lines}
1941 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1942 \toprule
1943 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1944 \endfirsthead
1945 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1946 \endhead
1947 InLounge & 66, 67, 74, 85 \\
1948 \bottomrule
1949 \end{longtable}
1950 \section{S21 LabSpatial}
1951 \subsection{Named functions /\allowbreak{} assigned handlers}
1952 \begin{longtable}{p{0.1\textwidth}p{0.8\textwidth}}
1953 \toprule
1954 \textbf{Line} & \textbf{Symbol} \\ \midrule
1955 \endfirsthead
1956 \textbf{Line} & \textbf{Symbol} \\ \midrule
1957 \endhead
1958 7 & S.\allowbreak{}frame \\
1959 12 & S.\allowbreak{}world \\
1960 13 & S.\allowbreak{}localPoint \\
1961 14 & S.\allowbreak{}vector \\
1962 15 & S.\allowbreak{}floor \\
1963 18 & S.\allowbreak{}setApproach \\
1964 26 & S.\allowbreak{}approach \\
1965 32 & S.\allowbreak{}sync \\
1966 \bottomrule
1967 \end{longtable}
1968 \subsection{Literal attribute keys and access lines}
1969 \begin{longtable}{p{0.43\textwidth}p{0.47\textwidth}}
1970 \toprule
1971 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1972 \endfirsthead
1973 \textbf{Attribute} & \textbf{Lines} \\ \midrule
1974 \endhead
1975 Approach & 23, 29, 34, 35 \\
1976 ApproachLocal & 19, 29 \\
1977 \bottomrule
1978 \end{longtable}
1979
1980

```

1981 GENERATED REFERENCE: verification-ledger.tex
1982 \chapter{Verification ledger and future acceptance suite}
1983 \section{P01: Role selection and nearby companion}
1984 Evidence period: Prior GWAP turn
1985
1986 Observed/\allowbreak{}reported result: Approximately eight studs from player; one visible carrier and four
↪ pooled.\allowbreak{}
1987
1988 Limit: Historical observation, not rerun for current geometry.\allowbreak{}
1989
1990 \section{P02: Wait and Regroup}
1991 Evidence period: Prior GWAP turn
1992
1993 Observed/\allowbreak{}reported result: Wait movement zero; regroup returned within roughly eight
↪ studs.\allowbreak{}
1994
1995 Limit: No stress test of every blocked spawn candidate.\allowbreak{}
1996
1997 \section{P03: Original rifle and equipment destruction}
1998 Evidence period: Prior GWAP turn
1999
2000 Observed/\allowbreak{}reported result: Lab Rifle destroyed a research machine; nearby explosion destroyed
↪ another; R repair restored it.\allowbreak{}
2001
2002 Limit: Does not certify wall-fixture restoration or every blast chain.\allowbreak{}
2003
2004 \section{P04: Equivalent rerouting}
2005 Evidence period: Prior GWAP turn
2006
2007 Observed/\allowbreak{}reported result: Incubator -> same-floor Backup 1 -> other-floor Backup 2; assignment
↪ stayed RUNNING.\allowbreak{}
2008
2009 Limit: Earlier coordinate frame; current C01 requires rerun.\allowbreak{}
2010
2011 \section{P05: Carrier gait and speed}
2012 Evidence period: Prior GWAP turn
2013
2014 Observed/\allowbreak{}reported result: WalkSpeed 20; roughly 28.\allowbreak{}7 studs of travel during sampled
↪ interval; leg swing about 24 degrees.\allowbreak{}
2015
2016 Limit: Kinematic gait, not Humanoid physics certification.\allowbreak{}
2017
2018 \section{P06: Wall-fixture cleanup}
2019 Evidence period: Prior GWAP turn
2020
2021 Observed/\allowbreak{}reported result: Four fixture types within a temporary wall section removed; outside
↪ health station preserved.\allowbreak{}
2022
2023 Limit: Controlled isolated test, not whole-floor blast profiling.\allowbreak{}
2024
2025 \section{P07: Small mobile layout}
2026 Evidence period: Prior GWAP turn
2027
2028 Observed/\allowbreak{}reported result: 705x338 viewport; panel bottom y=179, joystick top y=187.\allowbreak{}
2029
2030 Limit: Eight-pixel gap in this configuration only.\allowbreak{}
2031
2032 \section{P08: Idle weapon stabilization}
2033 Evidence period: Prior GWAP turn
2034
2035 Observed/\allowbreak{}reported result: 90 frames with zero Grip motion and zero shoulder-animation
↪ translation.\allowbreak{}
2036
2037 Limit: One tested original rifle/\allowbreak{}rig configuration.\allowbreak{}
2038
2039 \section{P09: Walking stabilization}
2040 Evidence period: Prior GWAP turn
2041
2042 Observed/\allowbreak{}reported result: Approximately 19 studs of walking with zero Grip motion.\allowbreak{}
2043
2044 Limit: Intentional recoil while firing was not removed.\allowbreak{}
2045
2046 \section{P10: Interception score}
2047 Evidence period: Prior GWAP turn
2048

2049 Observed/\allowbreak{}reported result: Assigned carrier awarded 50 once; repeat damage did not add
↪ points.\allowbreak{}
2050
2051 Limit: Multi-client race conditions not exhaustively tested.\allowbreak{}
2052
2053 \section{P11: Solo score loss}
2054 Evidence period: Prior GWAP turn
2055
2056 Observed/\allowbreak{}reported result: 100 points became 99 on a successful NPC hit.\allowbreak{}
2057
2058 Limit: Low-score fractional policy also inspected in code; no long statistical run.\allowbreak{}
2059
2060 \section{P12: Rooftop mission completion}
2061 Evidence period: Earlier GWAP turn
2062
2063 Observed/\allowbreak{}reported result: Seed source -> instrument -> roof completed and one plant
↪ appeared.\allowbreak{}
2064
2065 Limit: Test used helper following; friendly fire disqualified final escort reward, so clean full reward was not
↪ proven.\allowbreak{}
2066
2067 \section{P13: Player shooting validation}
2068 Evidence period: Shared chat
2069
2070 Observed/\allowbreak{}reported result: Reported 10 damage, wall blocking, and rejection of distant forged
↪ camera origin.\allowbreak{}
2071
2072 Limit: Historical report; not a new penetration/\allowbreak{}security audit.\allowbreak{}
2073
2074 \section{P14: ADS and device controls}
2075 Evidence period: Shared chat
2076
2077 Observed/\allowbreak{}reported result: Reported FOV 70->55 and sensitivity 1->0.\allowbreak{}62, restored on
↪ exit; phone/\allowbreak{}tablet controls checked.\allowbreak{}
2078
2079 Limit: Some later control changes superseded these exact UI versions.\allowbreak{}
2080
2081 \section{P15: Basement and quiz/\allowbreak{}window interactions}
2082 Evidence period: Shared chat
2083
2084 Observed/\allowbreak{}reported result: Vent landing, pickup quiz, and breakable window passed reported
↪ checks.\allowbreak{}
2085
2086 Limit: No new Play session in this edition.\allowbreak{}
2087
2088 \section{D01: Live source completeness}
2089 Evidence period: This edition
2090
2091 Observed/\allowbreak{}reported result: 29 source containers captured; every UTF-8 byte length matched Studio
↪ inventory.\allowbreak{}
2092
2093 Limit: Static capture only; active count 20, archive 8, disabled generator 1.\allowbreak{}
2094
2095 \section{D02: Scene/\allowbreak{}source audit}
2096 Evidence period: This edition
2097
2098 Observed/\allowbreak{}reported result: 67 research station records; place version 70; C01 and C02
↪ identified.\allowbreak{}
2099
2100 Limit: No claim that current routing is playable.\allowbreak{}
2101
2102 \section{Proposed new regression sequence}
2103 \subsection{Baseline}
2104 Open a copied saved place, identify the place/\allowbreak{}version, compare active-source hashes, and ensure
↪ there is only one bootstrap/\allowbreak{}controller authority.\allowbreak{}
2105
2106 \subsection{Coordinate frame}
2107 Translate and rotate a test laboratory with its navigation origin; verify every station approach stays in its
↪ intended corridor and each stair edge is clear.\allowbreak{} Include the current displaced scene as a
↪ regression fixture.\allowbreak{}
2108
2109 \subsection{Transactions}
2110 Two clients compete for the same plant and destination slot.\allowbreak{} Exactly one transfer commits; the
↪ other gets a clear reason.\allowbreak{} Repeat with a quiz outstanding, a full cabinet, and a destroyed
↪ destination.\allowbreak{}
2111

```

2112 \subsection{Carrier lifecycle}
2113 Choose, Ready, Wait, Status, Regroup, respawn, disconnect, change role, and end the round during travel and
    ↳ processing.\allowbreak{} No duplicate carrier, seedling, or reward remains.\allowbreak{}
2114
2115 \subsection{Destruction}
2116 Bullet-hit research station, chain blast, wall-section removal, outside-fixture preservation, repair,
    ↳ all-equivalents unavailable, and round reset after quarantine.\allowbreak{}
2117
2118 \subsection{Input preservation}
2119 Desktop mouse target at all screen edges; mobile forward ray; right-swipe camera with left movement; ADS
    ↳ on/\allowbreak{}off; equip/\allowbreak{}unequip; idle/\allowbreak{}walk gun stability; UI click does not
    ↳ fire.\allowbreak{}
2120
2121 \subsection{Responsive layout}
2122 Small phone portrait and landscape, tablet, desktop, chat open, quiz open, inventory open.\allowbreak{} Record
    ↳ rectangle intersections for mission, joystick, jump, fire, aim, and prompts.\allowbreak{}
2123
2124 \subsection{Scoring}
2125 Clean 90/\allowbreak{}105-point escort as applicable, one 50-point interception, friendly-fire
    ↳ disqualification, low-score fractional loss, reset debt, and multiplayer solo-fire exclusion.\allowbreak{}
2126
2127 \subsection{Performance}
2128 Profile crowded rooms and multi-client play; report hardware, viewport, scene population,
    ↳ average/\allowbreak{}p95 frame time, server heartbeat cost, and replication load.\allowbreak{} Do not
    ↳ extrapolate a one-client test to 60 players.\allowbreak{}
2129
2130 These are proposed tests.\allowbreak{} They were not executed while producing this document.\allowbreak{} The
    ↳ verification report for this edition concerns source capture, document compilation, page rendering, and
    ↳ manifest completeness.\allowbreak{}
2131
2132
2133
2134 GENERATED REFERENCE: visuals.tex
2135 \chapter{Historical visual references}
2136 \begin{figure}[p]\centering
2137 \includegraphics[width=\textwidth,height=.72\textheight,keepaspectratio]{figures/reference-1.png}
2138 \caption{Original layout reference supplied by the owner.\allowbreak{} This is another shooter used as a
    ↳ panel-layout reference, not a screenshot of the audited laboratory.\allowbreak{}}
2139 \end{figure}
2140 \begin{figure}[p]\centering
2141 \includegraphics[width=\textwidth,height=.72\textheight,keepaspectratio]{figures/reference-2.png}
2142 \caption{Historical laboratory screenshot showing the rejected added weapon selector and competing
    ↳ overlays.\allowbreak{} Not the current audited state.\allowbreak{}}
2143 \end{figure}
2144 \begin{figure}[p]\centering
2145 \includegraphics[width=\textwidth,height=.72\textheight,keepaspectratio]{figures/reference-3.png}
2146 \caption{Historical hallway lineup of repeated NPCs, supplied as defect evidence.\allowbreak{} Later pooling
    ↳ and template changes address this presentation.\allowbreak{}}
2147 \end{figure}
2148 \begin{figure}[p]\centering
2149 \includegraphics[width=\textwidth,height=.72\textheight,keepaspectratio]{figures/reference-4.png}
2150 \caption{Historical NPC appearance and weapon-visual detail, supplied as defect evidence.\allowbreak{}}
2151 \end{figure}
2152 \begin{figure}[p]\centering
2153 \includegraphics[width=\textwidth,height=.72\textheight,keepaspectratio]{figures/reference-5.png}
2154 \caption{Historical small-screen view with competing mission, carrier, and role-selection
    ↳ information.\allowbreak{} Used to explain the grouping requirement.\allowbreak{}}
2155 \end{figure}
2156 \clearpage
2157
2158 ]=====]
2159 return {revision=4,part=2,reference="Server-only written reference; never required by gameplay"}

```